DocsLive

VERSIÓN OFICIAL v16.3.7

Next.js 16

Libro Guía Definitivo — Documentación y Referencia Técnica Completa

🚀 456 Secciones Técnicas | 📁 13 Capítulos Estructurados | ⚡ Directivas 'use cache' | 🧩 Cache Components & PPR | 📦 Turbopack por Defecto

Compilación oficial generada para DocsLive

30 de septiembre de 2026

📘 Next.js v16.3.7 — Libro Guía Definitivo

> **Autoría original:** Equipo de Next.js & Vercel
> **Compilación y Adaptación:** DocsLive / Antigravity AI
> **Versión oficial documentada:** Next.js v16.3.7
> **Fecha de compilación:** 30 de septiembre de 2026

Este libro unificado reúne los 13 capítulos fundamentales y las 456 especificaciones técnicas de la versión 16.3.7 de Next.js.

Tabla de Contenidos

- Capítulo 1: Primeros Pasos y Configuración Inicial
- Capítulo 2: Enrutamiento, Layouts y Navegación (App Router)
- Capítulo 3: Server Components, Client Components y Renderizado
- Capítulo 4: Data Fetching, Caching Moderno y Revalidación
- Capítulo 5: Server Actions, Formularios y Mutación de Datos
- Capítulo 6: Directivas de Nueva Generación en Next.js 16
- Capítulo 7: Optimización de Assets: Imágenes, Fuentes, Scripts y SEO
- Capítulo 8: Seguridad, Autenticación y Protección de Datos
- Capítulo 9: Patrones Avanzados, Agentes de IA y Arquitectura BFF
- Capítulo 10: Despliegue, Rendimiento en Producción y Turbopack
- Capítulo 11: Referencia de API Completa (App Router)
- Capítulo 12: Pages Router (Soporte Legacy) y Estrategias de Migración
- Capítulo 13: Arquitectura Interna, Compilador Next.js y Comunidad

---

Capítulo 1: Primeros Pasos y Configuración Inicial

> *Instalación, estructura de proyectos, archivos de configuración, TypeScript, ESLint y primeros conceptos.*

Installation

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/installation
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to create a new Next.js application with the* `create-next-app` *CLI, and set up TypeScript, ESLint, and Module Path Aliases.*

Create a new Next.js app and run it locally.

Quick start

1. Create a new Next.js app named `my-app`
2. `cd my-app` and start the dev server.

3. Visit `http://localhost:3000`.

```
# Manejador de paquetes: pnpm
pnpm create next-app@latest my-app --yes
cd my-app
pnpm dev
```

```
# Manejador de paquetes: npm
npx create-next-app@latest my-app --yes
cd my-app
npm run dev
```

```
# Manejador de paquetes: yarn
yarn create next-app@latest my-app --yes
cd my-app
yarn dev
```

```
# Manejador de paquetes: bun
bun create next-app@latest my-app --yes
cd my-app
bun dev
```

- `--yes` skips prompts using saved preferences or defaults. The default setup enables TypeScript, Tailwind CSS, ESLint, App Router, and Turbopack, with import alias `@/*`, and includes `AGENTS.md` (with a `CLAUDE.md` that references it) to guide coding agents to write up-to-date Next.js code.

System requirements

Before you begin, make sure your development environment meets the following requirements:

- Minimum Node.js version: 20.9
- Operating systems: macOS, Windows (including WSL), and Linux.

Supported browsers

Next.js supports modern browsers with zero configuration.

- Chrome 111+
- Edge 111+
- Firefox 111+
- Safari 16.4+

Learn more about browser support, including how to configure polyfills and target specific browsers.

Create with the CLI

The quickest way to create a new Next.js app is using `create-next-app`, which sets up everything automatically for you. To create a project, run:

```
# Manejador de paquetes: pnpm
pnpm create next-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest
```

```
# Manejador de paquetes: yarn
yarn create next-app
```

```
# Manejador de paquetes: bun
bun create next-app
```

On installation, you'll see the following prompts:

```
// Archivo: Terminal
What is your project named? my-app
Would you like to use the recommended Next.js defaults?
    Yes, use recommended defaults - TypeScript, ESLint, Tailwind CSS, App Router, AGENTS.md
    No, reuse previous settings
    No, customize settings - Choose your own preferences
```

If you choose to `customize settings`, you'll see the following prompts:

```
// Archivo: Terminal
Would you like to use TypeScript? No / Yes
Which linter would you like to use? ESLint / Biome / None
Would you like to use React Compiler? No / Yes
Would you like to use Tailwind CSS? No / Yes
Would you like your code inside a `src/` directory? No / Yes
Would you like to use App Router? (recommended) No / Yes
Would you like to customize the import alias (`@/*` by default)? No / Yes
What import alias would you like configured? @/*
Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? No / Yes
```

After the prompts, `create-next-app` will create a folder with your project name and install the required dependencies.

Manual installation

To manually create a new Next.js app, install the required packages:

```
# Manejador de paquetes: pnpm
pnpm i next@latest react@latest react-dom@latest
```

```
# Manejador de paquetes: npm
npm i next@latest react@latest react-dom@latest
```

```
# Manejador de paquetes: yarn
yarn add next@latest react@latest react-dom@latest
```

```
# Manejador de paquetes: bun
bun add next@latest react@latest react-dom@latest
```

> **Good to know**:
>
> - The `App Router` uses React canary releases built-in, which include all the stable React 19 changes, as well as newer features being validated in frameworks, but you should still declare react and react-dom in package.json for tooling and ecosystem compatibility.
> - The `Pages Router` uses the React version from your `package.json`.

Then, add the following scripts to your `package.json` file:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "lint:fix": "eslint --fix"
  }
}
```

These scripts refer to the different stages of developing an application:

- `next dev` : Starts the development server using Turbopack (default bundler).
- `next build` : Builds the application for production.
- `next start` : Starts the production server.
- `eslint` : Runs ESLint.

Turbopack is now the default bundler. To use Webpack run `next dev --webpack` or `next build --webpack` . See the Turbopack docs for configuration details.

Create the `app` directory

Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.

Create an `app` folder. Then, inside `app` , create a `layout.tsx` file. This file is the root layout. It's required and must contain the `<html>` and `<body>` tags.

```
// Archivo: app/layout.tsx switcher
export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

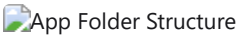
```
// Archivo: app/layout.js switcher
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

Create a home page `app/page.tsx` with some initial content:

```
// Archivo: app/page.tsx switcher
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}
```

Both `layout.tsx` and `page.tsx` will be rendered when the user visits the root of your application (`/`).



Good to know:

- If you forget to create the root layout, Next.js will automatically create this file when running the development server with `next dev`.
- You can optionally use a `src` folder in the root of your project to separate your application's code from configuration files.

📘 **Nota para Pages Router:**

Create the `pages` directory

Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.

Create a `pages` directory at the root of your project. Then, add an `index.tsx` file inside your `pages` folder. This will be your home page (`/`):

```
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}
```

```
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}
```

Next, add an `_app.tsx` file inside `pages/` to define the global layout. Learn more about the custom App file.

```
import type { AppProps } from 'next/app'

export default function App({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />
}
```

```
export default function App({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

Finally, add a `_document.tsx` file inside `pages/` to control the initial response from the server. Learn more about the custom Document file.

```
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html>
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

```
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html>
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

Create the `public` folder (optional)

Create a `public` folder at the root of your project to store static assets such as images, fonts, etc. Files inside `public` can then be referenced by your code starting from the base URL (`/`).

You can then reference these assets using the root path (`/`). For example, `public/profile.png` can be referenced as `/profile.png` :

```
// Archivo: app/page.tsx highlight={4} switcher
import Image from 'next/image'

export default function Page() {
  return

![Profile](https://nextjs.org/profile.png)


}
```

```
// Archivo: app/page.js highlight={4} switcher
import Image from 'next/image'

export default function Page() {
  return

![Profile](https://nextjs.org/profile.png)


}
```

Run the development server

1. Run `npm run dev` to start the development server.
2. Visit `http://localhost:3000` to view your application.
3. Edit the `app/page.tsx`

> 📘 **Nota para Pages Router:** `pages/index.tsx`

file and save it to see the updated result in your browser.

Set up TypeScript

> Minimum TypeScript version: `v5.1.0`

Next.js comes with built-in TypeScript support. To add TypeScript to your project, rename a file to `.ts` / `.tsx` and run `next dev` . Next.js will automatically install the necessary dependencies and add a `tsconfig.json` file with the recommended config options.

IDE Plugin

Next.js includes a custom TypeScript plugin and type checker, which VSCode and other code editors can use for advanced type-checking and auto-completion.

You can enable the plugin in VS Code by:

1. Opening the command palette (`Ctrl/⌘` + `Shift` + `P`)
2. Searching for "TypeScript: Select TypeScript Version"
3. Selecting "Use Workspace Version"

TypeScript Command Palette

See the TypeScript reference page for more information.

Set up your editor

The App Router names files by convention, like `page.tsx`, `layout.tsx`, and `route.ts`, so your editor quickly fills with same-named tabs. Label each tab with its enclosing folders, like `blog/[id]`, so you can tell them apart.

In VS Code 1.88+ or Cursor, add custom editor labels to `.vscode/settings.json`. Labeling two folders deep keeps dynamic routes like `blog/[id]/page.tsx` from all collapsing to the same `[id]` label:

```
// Archivo: .vscode/settings.json
{
  "workbench.editor.customLabels.patterns": {
    "**/app/**/page.tsx": "${dirname(1)}/${dirname} - page.tsx",
    "**/app/**/layout.tsx": "${dirname(1)}/${dirname} - layout.tsx",
    "**/app/**/loading.tsx": "${dirname(1)}/${dirname} - loading.tsx",
    "**/app/**/error.tsx": "${dirname(1)}/${dirname} - error.tsx",
    "**/app/**/not-found.tsx": "${dirname(1)}/${dirname} - not-found.tsx",
    "**/app/**/template.tsx": "${dirname(1)}/${dirname} - template.tsx",
    "**/app/**/default.tsx": "${dirname(1)}/${dirname} - default.tsx",
    "**/app/**/route.ts": "${dirname(1)}/${dirname} - route.ts"
  }
}
```

Or copy this prompt to have your coding agent set it up:

```
Set up custom editor labels so my Next.js App Router files are easy to tell apart. Read https://nextjs.org/docs/app/getting-started/installation#set-up-your-editor and add the workbench.editor.customLabels.patterns config shown there to my .vscode/settings.json, creating the file if it doesn't exist. Adjust the labels to taste. If I use a different editor, apply the equivalent setting or tell me it's automatic, and leave my other settings untouched.
```

> **Good to know:** JetBrains IDEs (WebStorm, IntelliJ) show the folder for same-named files automatically, so no setup is needed.

Set up linting

Next.js supports linting with either ESLint or Biome. Choose a linter and run it directly via `package.json` scripts.

- Use **ESLint** (comprehensive rules):

```
// Archivo: package.json
{
  "scripts": {
    "lint": "eslint",
    "lint:fix": "eslint --fix"
  }
}
```

- Or use **Biome** (fast linter + formatter):

```
// Archivo: package.json
{
  "scripts": {
    "lint": "biome check",
    "format": "biome format --write"
  }
}
```

If your project previously used `next lint`, migrate your scripts to the ESLint CLI with the codemod:

```
// Archivo: Terminal
npx @next/codemod@canary next-lint-to-eslint-cli .
```

If you use ESLint, create an explicit config (recommended `eslint.config.mjs`). ESLint supports both the legacy `.eslintrc.*` and the newer `eslint.config.mjs` formats. See the ESLint API reference for a recommended setup.

> **Good to know**: Starting with Next.js 16, `next build` no longer runs the linter automatically. Instead, you can run your linter through NPM scripts.

See the ESLint Plugin page for more information.

Set up Absolute Imports and Module Path Aliases

Next.js has in-built support for the `"paths"` and `"baseUrl"` options of `tsconfig.json` and `jsconfig.json` files.

These options allow you to alias project directories to absolute paths, making it easier and cleaner to import modules. For example:

```
// Before
import { Button } from '../../../components/button'

// After
import { Button } from '@/components/button'
```

To configure absolute imports, add the `baseUrl` configuration option to your `tsconfig.json` or `jsconfig.json` file. For example:

```
// Archivo: tsconfig.json or jsconfig.json
{
  "compilerOptions": {
    "baseUrl": "src/"
  }
}
```

In addition to configuring the `baseUrl` path, you can use the `"paths"` option to `"alias"` module paths.

For example, the following configuration maps `@/components/*` to `components/*`:

```
// Archivo: tsconfig.json or jsconfig.json
{
  "compilerOptions": {
    "baseUrl": "src/",
    "paths": {
      "@/styles/*": ["styles/*"],
      "@/components/*": ["components/*"]
    }
  }
}
```

Each of the `"paths"` are relative to the `baseUrl` location.

Upgrade your Next.js app

Keep your Next.js version up to date. Each release ships security patches, bug fixes, and performance optimizations alongside new features, and staying current keeps every individual upgrade small. Run the `upgrade` command:

```
# Manejador de paquetes: pnpm
pnpm next upgrade
```

```
# Manejador de paquetes: npm
npx next upgrade
```

```
# Manejador de paquetes: yarn
yarn next upgrade
```

```
# Manejador de paquetes: bun
bunx next upgrade
```

Upgrading also updates the documentation bundled inside the `next` package at `node_modules/next/dist/docs/`. New features arrive with their docs, and existing pages pick up new guidance and pitfalls discovered along the way. AI coding agents in your project then work from the version you have installed rather than their training data. After an upgrade, you can prompt your agent to catch up:

```
Let's get our Next.js knowledge up to speed, and give me a summary of what's new for you
```

See Upgrading for version guides and manual upgrade steps, or the preview docs to explore features before they ship in a stable version.

Project structure and organization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/project-structure
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn the folder and file conventions in Next.js, and how to organize your project.*

This page provides an overview of **all** the folder and file conventions in Next.js, and recommendations for organizing your project.

Folder and file conventions

Top-level folders

Top-level folders are used to organize your application's code and static assets.

Route segments to path segments

`app`	App Router
`pages`	Pages Router
`public`	Static assets to be served
`src`	Optional application source folder

Top-level files

Top-level files are used to configure your application, manage dependencies, run proxy, integrate monitoring tools, and define environment variables.

Next.js	
`next.config.js`	Configuration file for Next.js
`package.json`	Project dependencies and scripts
`instrumentation.ts`	OpenTelemetry and Instrumentation file
`proxy.ts`	Next.js request proxy
`.env`	Environment variables (should not be tracked by version control)
`.env.local`	Local environment variables (should not be tracked by version control)
`.env.production`	Production environment variables (should not be tracked by version control)
`.env.development`	Development environment variables (should not be tracked by version control)
`eslint.config.mjs`	Configuration file for ESLint
`.gitignore`	Git files and folders to ignore
`next-env.d.ts`	TypeScript declaration file for Next.js (should not be tracked by version control)
`tsconfig.json`	Configuration file for TypeScript
`jsconfig.json`	Configuration file for JavaScript

Routing Files

Add `page` to expose a route, `layout` for shared UI such as header, nav, or footer, `loading` for skeletons, `error` for error boundaries, and `route` for APIs.

`layout`	`.js` `.jsx` `.tsx`	Layout
`page`	`.js` `.jsx` `.tsx`	Page
`loading`	`.js` `.jsx` `.tsx`	Loading UI
`not-found`	`.js` `.jsx` `.tsx`	Not found UI
`error`	`.js` `.jsx` `.tsx`	Error UI
`global-error`	`.js` `.jsx` `.tsx`	Global error UI
`route`	`.js` `.ts`	API endpoint
`template`	`.js` `.jsx` `.tsx`	Re-rendered layout
`default`	`.js` `.jsx` `.tsx`	Parallel route fallback page

Nested routes

Folders define URL segments. Nesting folders nests segments. Layouts at any level wrap their child segments. A route becomes public when a `page` or `route` file exists.

Path	URL pattern	Notes
`app/layout.tsx`	—	Root layout wraps all routes
`app/blog/layout.tsx`	—	Wraps `/blog` and descendants
`app/page.tsx`	`/`	Public route
`app/blog/page.tsx`	`/blog`	Public route
`app/blog/authors/page.tsx`	`/blog/authors`	Public route

Dynamic routes

Parameterize segments with square brackets. Use `[segment]` for a single param, `[...segment]` for catch-all, and `[[...segment]]` for optional catch-all. Access values via the `params` prop.

Path	URL pattern
`app/blog/[slug]/page.tsx`	`/blog/my-first-post`
`app/shop/[...slug]/page.tsx`	`/shop/clothing`, `/shop/clothing/shirts`
`app/docs/[[...slug]]/page.tsx`	`/docs`, `/docs/layouts-and-pages`, `/docs/api-reference/use-router`

Route groups and private folders

Organize code without changing URLs with route groups `(group)`, and colocate non-routable files with private folders `_folder`.

Path	URL pattern	Notes
`app/(marketing)/page.tsx`	`/`	Group omitted from URL
`app/(shop)/cart/page.tsx`	`/cart`	Share layouts within `(shop)`
`app/blog/_components/Post.tsx`	—	Not routable; safe place for UI utilities
`app/blog/_lib/data.ts`	—	Not routable; safe place for utils

Parallel and Intercepted Routes

These features fit specific UI patterns, such as slot-based layouts or modal routing.

Use `@slot` for named slots rendered by a parent layout. Use intercept patterns to render another route inside the current layout without changing the URL, for example, to show a details view as a modal over a list.

Pattern (docs)	Meaning	Typical use case
`@folder`	Named slot	Sidebar + main content
`(.)folder`	Intercept same level	Preview sibling route in a modal
`(..)folder`	Intercept parent	Open a child of the parent as an overlay
`(..)(..)folder`	Intercept two levels	Deeply nested overlay
`(...)folder`	Intercept from root	Show arbitrary route in current view

Metadata file conventions

App icons

`favicon`	`.ico`	Favicon file
`icon`	`.ico` `.jpg` `.jpeg` `.png` `.svg`	App Icon file
`icon`	`.js` `.ts` `.tsx`	Generated App Icon
`apple-icon`	`.jpg` `.jpeg` `.png`	Apple App Icon file
`apple-icon`	`.js` `.ts` `.tsx`	Generated Apple App Icon

Open Graph and Twitter images

`opengraph-image`	`.jpg` `.jpeg` `.png` `.gif`	Open Graph image file
`opengraph-image`	`.js` `.ts` `.tsx`	Generated Open Graph image
`twitter-image`	`.jpg` `.jpeg` `.png` `.gif`	Twitter image file
`twitter-image`	`.js` `.ts` `.tsx`	Generated Twitter image

SEO

`sitemap`	`.xml`	Sitemap file
`sitemap`	`.js` `.ts`	Generated Sitemap
`robots`	`.txt`	Robots file
`robots`	`.js` `.ts`	Generated Robots file

📘 **Nota para Pages Router:**

File conventions

`_app`	`.js` `.jsx` `.tsx`	Custom App
`_document`	`.js` `.jsx` `.tsx`	Custom Document
`_error`	`.js` `.jsx` `.tsx`	Custom Error Page
`404`	`.js` `.jsx` `.tsx`	404 Error Page
`500`	`.js` `.jsx` `.tsx`	500 Error Page

Routes

Folder convention		
`index`	`.js` `.jsx` `.tsx`	Home page
`folder/index`	`.js` `.jsx` `.tsx`	Nested page
File convention		
`index`	`.js` `.jsx` `.tsx`	Home page
`file`	`.js` `.jsx` `.tsx`	Nested page

Dynamic routes

Folder convention		
`[folder]/index`	`.js` `.jsx` `.tsx`	Dynamic route segment
`[...folder]/index`	`.js` `.jsx` `.tsx`	Catch-all route segment
`[[...folder]]/index`	`.js` `.jsx` `.tsx`	Optional catch-all route segment
File convention		
`[file]`	`.js` `.jsx` `.tsx`	Dynamic route segment
`[...file]`	`.js` `.jsx` `.tsx`	Catch-all route segment
`[[...file]]`	`.js` `.jsx` `.tsx`	Optional catch-all route segment

Organizing your project

Next.js is **unopinionated** about how you organize and colocate your project files. But it does provide several features to help you organize your project.

Component hierarchy

The components defined in special files are rendered in a specific hierarchy:

- `layout.js`
- `template.js`
- `error.js` (React error boundary)
- `loading.js` (React suspense boundary)

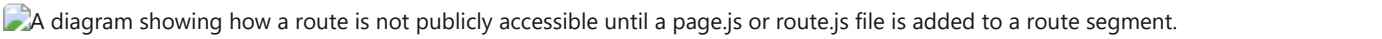
- `not-found.js` (React error boundary for "not found" UI)
- `page.js` or nested `layout.js`

The components are rendered recursively in nested routes, meaning the components of a route segment will be nested **inside** the components of its parent segment.

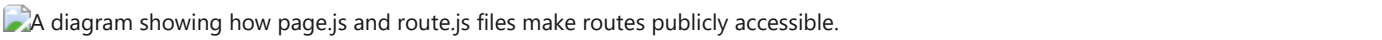
Colocation

In the `app` directory, nested folders define route structure. Each folder represents a route segment that is mapped to a corresponding segment in a URL path.

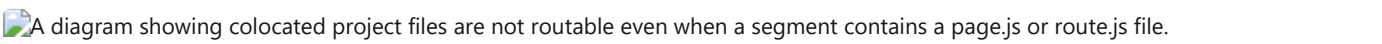
However, even though route structure is defined through folders, a route is **not publicly accessible** until a `page.js` or `route.js` file is added to a route segment.



And, even when a route is made publicly accessible, only the **content returned** by `page.js` or `route.js` is sent to the client.



This means that **project files** can be **safely colocated** inside route segments in the `app` directory without accidentally being routable.



> **Good to know**: While you **can** colocate your project files in `app` you don't **have** to. If you prefer, you can keep them outside the `app` directory.

Private folders

Private folders can be created by prefixing a folder with an underscore: `_folderName`

This indicates the folder is a private implementation detail and should not be considered by the routing system, thereby **opting the folder and all its subfolders** out of routing.

Since files in the `app` directory can be safely colocated by default, private folders are not required for colocation. However, they can be useful for:

- Separating UI logic from routing logic.
- Consistently organizing internal files across a project and the Next.js ecosystem.
- Sorting and grouping files in code editors.
- Avoiding potential naming conflicts with future Next.js file conventions.

> **Good to know**:
>
> - While not a framework convention, you might also consider marking files outside private folders as "private" using the same underscore pattern.
> - You can create URL segments that start with an underscore by prefixing the folder name with `%5F` (the URL-encoded form of an underscore): `%5FfolderName`.
> - If you don't use private folders, it would be helpful to know Next.js special file conventions to prevent unexpected naming conflicts.

Route groups

Route groups can be created by wrapping a folder in parenthesis: `(folderName)`

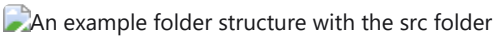
This indicates the folder is for organizational purposes and should **not be included** in the route's URL path.

Route groups are useful for:

- Organizing routes by site section, intent, or team. e.g. marketing pages, admin pages, etc.
- Enabling nested layouts in the same route segment level:
  - Creating multiple nested layouts in the same segment, including multiple root layouts
  - Adding a layout to a subset of routes in a common segment

`src` folder

Next.js supports storing application code (including `app`) inside an optional `src` folder. This separates application code from project configuration files which mostly live in the root of a project.


An example folder structure with the src folder

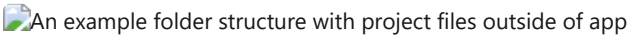
Examples

The following section lists a very high-level overview of common strategies. The simplest takeaway is to choose a strategy that works for you and your team and be consistent across the project.

> **Good to know**: In our examples below, we're using `components` and `lib` folders as generalized placeholders, their naming has no special framework significance and your projects might use other folders like `ui`, `utils`, `hooks`, `styles`, etc.

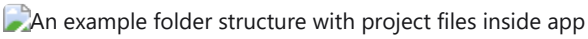
Store project files outside of `app`

This strategy stores all application code in shared folders in the **root of your project** and keeps the `app` directory purely for routing purposes.


An example folder structure with project files outside of app

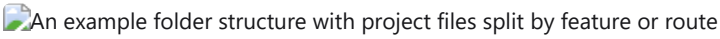
Store project files in top-level folders inside of `app`

This strategy stores all application code in shared folders in the **root of the `app` directory**.


An example folder structure with project files inside app

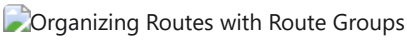
Split project files by feature or route

This strategy stores globally shared application code in the root `app` directory and **splits** more specific application code into the route segments that use them.

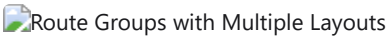

An example folder structure with project files split by feature or route

Organize routes without affecting the URL path

To organize routes without affecting the URL, create a group to keep related routes together. The folders in parenthesis will be omitted from the URL (e.g. `(marketing)` or `(shop)`).

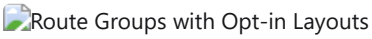

Organizing Routes with Route Groups

Even though routes inside `(marketing)` and `(shop)` share the same URL hierarchy, you can create a different layout for each group by adding a `layout.js` file inside their folders. These layouts nest within the existing app layout.


Route Groups with Multiple Layouts

Opting specific segments into a layout

To opt specific routes into a layout, create a new route group (e.g. `(shop)`) and move the routes that share the same layout into the group (e.g. `account` and `cart`). The routes outside of the group will not share the layout (e.g. `checkout`).


Route Groups with Opt-in Layouts

Opting for loading skeletons on a specific route

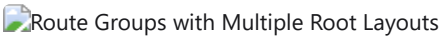
To apply a loading skeleton via a `loading.js` file to a specific route, create a new route group (e.g., `/(overview)`) and then move your `loading.tsx` inside that route group.

Folder structure showing a loading.tsx and a page.tsx inside the route group

Now, the `loading.tsx` file will only apply to your dashboard → overview page instead of all your dashboard pages without affecting the URL path structure.

Creating multiple root layouts

To create multiple root layouts, remove the top-level `layout.js` file, and add a `layout.js` file inside each route group. This is useful for partitioning an application into sections that have a completely different UI or experience. The `<html>` and `<body>` tags need to be added to each root layout.


Route Groups with Multiple Root Layouts

In the example above, both `(marketing)` and `(shop)` have their own root layout.

CSS

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/css
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn about the different ways to add CSS to your application, including Tailwind CSS, CSS Modules, Global CSS, and more.*

Next.js provides several ways to style your application using CSS, including:

- Tailwind CSS
- CSS Modules
- Global CSS
- External Stylesheets
- Sass
- CSS-in-JS

Tailwind CSS

Tailwind CSS is a utility-first CSS framework that provides low-level utility classes to build custom designs.

Install Tailwind CSS:

```
# Manejador de paquetes: pnpm
pnpm add -D tailwindcss @tailwindcss/postcss
```

```
# Manejador de paquetes: npm
npm install -D tailwindcss @tailwindcss/postcss
```

```
# Manejador de paquetes: yarn
yarn add -D tailwindcss @tailwindcss/postcss
```

```
# Manejador de paquetes: bun
bun add -D tailwindcss @tailwindcss/postcss
```

Add the PostCSS plugin to your `postcss.config.mjs` file:

```
// Archivo: postcss.config.mjs
export default {
  plugins: {
    '@tailwindcss/postcss': {},
  },
}
```

Import Tailwind in your global CSS file:

```
// Archivo: app/globals.css
@import 'tailwindcss';
```

Import the CSS file in your root layout:

```
// Archivo: app/layout.tsx switcher
import './globals.css'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import './globals.css'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

Now you can start using Tailwind's utility classes in your application:

```
// Archivo: app/page.tsx switcher
export default function Page() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-between p-24">
      <h1 className="text-4xl font-bold">Welcome to Next.js!</h1>
    </main>
  )
}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-between p-24">
      <h1 className="text-4xl font-bold">Welcome to Next.js!</h1>
    </main>
  )
}
```

📘 **Nota para Pages Router:** Install Tailwind CSS:

```
pnpm add -D tailwindcss @tailwindcss/postcss
```

```
npm install -D tailwindcss @tailwindcss/postcss
```

```
yarn add -D tailwindcss @tailwindcss/postcss
```

```
bun add -D tailwindcss @tailwindcss/postcss
```

Add the PostCSS plugin to your `postcss.config.mjs` file:

```
export default {
  plugins: {
    '@tailwindcss/postcss': {},
  },
}
```

Import Tailwind in your global CSS file:

```
@import 'tailwindcss';
```

Import the CSS file in your `pages/_app.js` file:

```
import '@/styles/globals.css'

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

Now you can start using Tailwind's utility classes in your application:

```
export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-between p-24">
      <h1 className="text-4xl font-bold">Welcome to Next.js!</h1>
    </main>
  )
}
```

```
export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-between p-24">
      <h1 className="text-4xl font-bold">Welcome to Next.js!</h1>
    </main>
  )
}
```

Good to know: If you need broader browser support for very old browsers, see the Tailwind CSS v3 setup instructions.

CSS Modules

CSS Modules locally scope CSS by generating unique class names. This allows you to use the same class in different files without worrying about naming collisions.

To start using CSS Modules, create a new file with the extension `.module.css` and import it into any component inside the `app` directory:

```
// Archivo: app/blog/blog.module.css
.blog {
  padding: 24px;
}
```

```
// Archivo: app/blog/page.tsx switcher
import styles from './blog.module.css'

export default function Page() {
  return <main className={styles.blog}></main>
}
```

```
// Archivo: app/blog/page.js switcher
import styles from './blog.module.css'

export default function Page() {
  return <main className={styles.blog}></main>
}
```

📘 **Nota para Pages Router:** To start using CSS Modules, create a new file with the extension `.module.css` and import it into any component inside the `pages` directory:

```
.blog {
  padding: 24px;
}
```

```
import styles from '@/styles/blog.module.css'

export default function Page() {
  return <main className={styles.blog}></main>
}
```

```
import styles from '@/styles/blog.module.css'

export default function Page() {
  return <main className={styles.blog}></main>
}
```

Global CSS

You can use global CSS to apply styles across your application.

Create a `app/global.css` file and import it in the root layout to apply the styles to **every route** in your application:

```
// Archivo: app/global.css
body {
  padding: 20px 20px 60px;
  max-width: 680px;
  margin: 0 auto;
}
```

```tsx
// Archivo: app/layout.tsx switcher
// These styles apply to every route in the application
import './global.css'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

```jsx
// Archivo: app/layout.js switcher
// These styles apply to every route in the application
import './global.css'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

Good to know: Global styles can be imported into any layout, page, or component inside the `app` directory. However, since Next.js uses React's built-in support for stylesheets to integrate with Suspense, this currently does not remove stylesheets as you navigate between routes which can lead to conflicts. We recommend using global styles for *truly* global CSS (like Tailwind's base styles), Tailwind CSS for component styling, and CSS Modules for custom scoped CSS when needed.

📘 **Nota para Pages Router:** Import the stylesheet in the `pages/_app.js` file to apply the styles to **every route** in your application:

```jsx
import '@/styles/global.css'

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

Due to the global nature of stylesheets, and to avoid conflicts, you should import them inside `pages/_app.js`.

External stylesheets

Stylesheets published by external packages can be imported anywhere in the `app` directory, including colocated components:

```tsx
// Archivo: app/layout.tsx switcher
import 'bootstrap/dist/css/bootstrap.css'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body className="container">{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import 'bootstrap/dist/css/bootstrap.css'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body className="container">{children}</body>
    </html>
  )
}
```

Good to know: In React 19, `<link rel="stylesheet" href="..." />` can also be used. See the React `link` documentation for more information.

📘 **Nota para Pages Router:** Next.js allows you to import CSS files from a JavaScript file. This is possible because Next.js extends the concept of `import` beyond JavaScript.

Import styles from `node_modules`

Since Next.js **9.5.4**, importing a CSS file from `node_modules` is permitted anywhere in your application.

For global stylesheets, like `bootstrap` or `nprogress`, you should import the file inside `pages/_app.js`. For example:

```
import 'bootstrap/dist/css/bootstrap.css'

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

To import CSS required by a third-party component, you can do so in your component. For example:

```
import { useState } from 'react'
import { Dialog } from '@reach/dialog'
import VisuallyHidden from '@reach/visually-hidden'
import '@reach/dialog/styles.css'

function ExampleDialog(props) {
  const [showDialog, setShowDialog] = useState(false)
  const open = () => setShowDialog(true)
  const close = () => setShowDialog(false)

  return (
    <div>
      <button onClick={open}>Open Dialog</button>
      <Dialog isOpen={showDialog} onDismiss={close}>
        <button className="close-button" onClick={close}>
          <VisuallyHidden>Close</VisuallyHidden>
          <span aria-hidden>×</span>
        </button>
        <p>Hello there. I am a dialog</p>
      </Dialog>
    </div>
  )
}
```

Ordering and Merging

Next.js optimizes CSS during production builds by automatically chunking (merging) stylesheets. The **order of your CSS** depends on the **order you import styles in your code**.

For example, `base-button.module.css` will be ordered before `page.module.css` since `<BaseButton>` is imported before `page.module.css`:

```
// Archivo: page.tsx switcher
import { BaseButton } from './base-button'
import styles from './page.module.css'

export default function Page() {
  return <BaseButton className={styles.primary} />
}
```

```
// Archivo: page.js switcher
import { BaseButton } from './base-button'
import styles from './page.module.css'

export default function Page() {
  return <BaseButton className={styles.primary} />
}
```

```
// Archivo: base-button.tsx switcher
import styles from './base-button.module.css'

export function BaseButton() {
  return <button className={styles.primary} />
}
```

```
// Archivo: base-button.js switcher
import styles from './base-button.module.css'

export function BaseButton() {
  return <button className={styles.primary} />
}
```

Recommendations

To keep CSS ordering predictable:

- Try to contain CSS imports to a single JavaScript or TypeScript entry file
- Import global styles and Tailwind stylesheets in the root of your application.
- **Use Tailwind CSS** for most styling needs as it covers common design patterns with utility classes.
- Use CSS Modules for component-specific styles when Tailwind utilities aren't sufficient.
- Use a consistent naming convention for your CSS modules. For example, using `<name>.module.css` over `<name>.tsx`.
- Extract shared styles into shared components to avoid duplicate imports.
- Turn off linters or formatters that auto-sort imports like ESLint's `sort-imports`.
- You can use the `cssChunking` option in `next.config.js` to control how CSS is chunked.

Development vs Production

- In development (`next dev`), CSS updates apply instantly with Fast Refresh.
- In production (`next build`), all CSS files are automatically concatenated into **many minified and code-split** `.css` files, ensuring the minimal amount of CSS is loaded for a route.
- CSS still loads with JavaScript disabled in production, but JavaScript is required in development for Fast Refresh.
- CSS ordering can behave differently in development, always ensure to check the build (`next build`) to verify the final CSS order.

Proxy

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/proxy
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Use Next.js Proxy to rewrite, redirect, modify headers, or respond directly before a request completes.*

Proxy

> **Good to know**: Starting with Next.js 16, Middleware is now called Proxy to better reflect its purpose. The functionality remains the same.

Proxy allows you to run code before a request is completed. Then, based on the incoming request, you can modify the response by rewriting, redirecting, modifying the request or response headers, or responding directly.

Use cases

Some common scenarios where Proxy is effective include:

- Modifying headers for all pages or a subset of pages
- Rewriting to different pages based on A/B tests or experiments
- Programmatic redirects based on incoming request properties

For simple redirects, consider using the `redirects` configuration in `next.config.ts` first. Proxy should be used when you need access to request data or more complex logic.

Proxy is *not* intended for slow data fetching. While Proxy can be helpful for optimistic checks such as permission-based redirects, it should not be used as a full session management or authorization solution.

Using fetch with `options.cache`, `options.next.revalidate`, or `options.next.tags`, has no effect in Proxy.

Convention

Create a `proxy.ts` (or `.js`) file in the project root, or inside `src` if applicable, so that it is located at the same level as `pages` or `app`.

> **Note**: While only one `proxy.ts` file is supported per project, you can still organize your proxy logic into modules. Break out proxy functionalities into separate `.ts` or `.js` files and import them into your main `proxy.ts` file. This allows for cleaner management of route-specific proxy, aggregated in the `proxy.ts` for centralized control. By enforcing a single proxy file, it simplifies configuration, prevents potential conflicts, and optimizes performance by avoiding multiple proxy layers.

Example

You can export your proxy function as either a default export or a named `proxy` export:

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

// This function can be marked `async` if using `await` inside
export function proxy(request: NextRequest) {
  return NextResponse.redirect(new URL('/home', request.url))
}

// Alternatively, you can use a default export:
// export default function proxy(request: NextRequest) { ... }

export const config = {
  matcher: '/about/:path*',
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

// This function can be marked `async` if using `await` inside
export function proxy(request) {
  return NextResponse.redirect(new URL('/home', request.url))
}

// Alternatively, you can use a default export:
// export default function proxy(request) { ... }

export const config = {
  matcher: '/about/:path*',
}
```

The `matcher` config allows you to filter Proxy to run on specific paths. See the Matcher documentation for more details on path matching.

Read more about using `proxy`, or refer to the `proxy` API reference.

Upgrading

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/upgrading
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to upgrade your Next.js application to the latest version or canary.*

Latest version

To update to the latest version of Next.js, you can use the `upgrade` command:

```
# Manejador de paquetes: pnpm
pnpm next upgrade
```

```
# Manejador de paquetes: npm
npx next upgrade
```

```
# Manejador de paquetes: yarn
yarn next upgrade
```

```
# Manejador de paquetes: bun
bunx next upgrade
```

Versions before Next.js 16.1.0 do not support the `upgrade` command and need to use a separate package instead:

```
// Archivo: Terminal
npx @next/codemod@canary upgrade latest
```

If you prefer to upgrade manually, install the latest Next.js and React versions:

```
# Manejador de paquetes: pnpm
pnpm i next@latest react@latest react-dom@latest eslint-config-next@latest
```

```
# Manejador de paquetes: npm
npm i next@latest react@latest react-dom@latest eslint-config-next@latest
```

```
# Manejador de paquetes: yarn
yarn add next@latest react@latest react-dom@latest eslint-config-next@latest
```

```
# Manejador de paquetes: bun
bun add next@latest react@latest react-dom@latest eslint-config-next@latest
```

Canary version

To update to the latest canary, make sure you're on the latest version of Next.js and everything is working as expected. Then, run the following command:

```
# Manejador de paquetes: pnpm
pnpm add next@canary
```

```
# Manejador de paquetes: npm
npm i next@canary
```

```
# Manejador de paquetes: yarn
yarn add next@canary
```

```
# Manejador de paquetes: bun
bun add next@canary
```

Features available in canary

The following features are currently available in canary:

Authentication:

- `forbidden`
- `unauthorized`
- `forbidden.js`
- `unauthorized.js`
- `authInterrupts`

Getting Started

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to create full-stack web applications with the Next.js App Router.*

Welcome to the Next.js documentation!

This **Getting Started** section will help you create your first Next.js app and learn the core features you'll use in every project.

Pre-requisite knowledge

Our documentation assumes some familiarity with web development. Before getting started, it'll help if you're comfortable with:

- HTML
- CSS
- JavaScript
- React

If you're new to React or need a refresher, we recommend starting with our React Foundations course, and the Next.js Foundations course that has you building an application as you learn.

Next Steps

Capítulo 2: Enrutamiento, Layouts y Navegación (App Router)

> *Layouts anidados, páginas, navegación dinámica, Link component, Route Handlers y organización de rutas.*

Layouts and Pages

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/layouts-and-pages
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to create your first pages and layouts, and link between them with the Link component.*

Next.js uses **file-system based routing**, meaning you can use folders and files to define routes. This page will guide you through how to create layouts and pages, and link between them.

Creating a page

A **page** is UI that is rendered on a specific route. To create a page, add a `page` file inside the `app` directory and default export a React component. For example, to create an index page (`/`):

page.js special file

```
// Archivo: app/page.tsx switcher
export default function Page() {
  return <h1>Hello Next.js!</h1>
}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  return <h1>Hello Next.js!</h1>
}
```

Creating a layout

A layout is UI that is **shared** between multiple pages. On navigation, layouts preserve state, remain interactive, and do not rerender.

You can define a layout by default exporting a React component from a `layout` file. The component should accept a `children` prop which can be a page or another layout.

For example, to create a layout that accepts your index page as child, add a `layout` file inside the `app` directory:

layout.js special file

```
// Archivo: app/layout.tsx switcher
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>


        <main>{children}</main>
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
export default function DashboardLayout({ children }) {
  return (
    <html lang="en">
      <body>


        <main>{children}</main>
      </body>
    </html>
  )
}
```

The layout above is called a root layout because it's defined at the root of the `app` directory. The root layout is **required** and must contain `html` and `body` tags.

Creating a nested route

A nested route is a route composed of multiple URL segments. For example, the `/blog/[slug]` route is composed of three segments:

- `/` (Root Segment)
- `blog` (Segment)
- `[slug]` (Leaf Segment)

In Next.js:

- **Folders** are used to define the route segments that map to URL segments.
- **Files** (like `page` and `layout`) are used to create UI that is shown for a segment.

To create nested routes, you can nest folders inside each other. For example, to add a route for `/blog`, create a folder called `blog` in the `app` directory. Then, to make `/blog` publicly accessible, add a `page.tsx` file:

File hierarchy showing blog folder and a page.js file

```tsx
// Archivo: app/blog/page.tsx switcher
// Dummy imports
import { getPosts } from '@/lib/posts'
import { Post } from '@/ui/post'

export default async function Page() {
  const posts = await getPosts()

  return (
    <ul>
      {posts.map((post) => (
        <Post key={post.id} post={post} />
      ))}
    </ul>
  )
}
```

```jsx
// Archivo: app/blog/page.js switcher
// Dummy imports
import { getPosts } from '@/lib/posts'
import { Post } from '@/ui/post'

export default async function Page() {
  const posts = await getPosts()

  return (
    <ul>
      {posts.map((post) => (
        <Post key={post.id} post={post} />
      ))}
    </ul>
  )
}
```

You can continue nesting folders to create nested routes. For example, to create a route for a specific blog post, create a new `[slug]` folder inside `blog` and add a `page` file:

File hierarchy showing blog folder with a nested slug folder and a page.js file

```tsx
// Archivo: app/blog/[slug]/page.tsx switcher
export function generateStaticParams() {}

export default function Page() {
  return <h1>Hello, Blog Post Page!</h1>
}
```

```jsx
// Archivo: app/blog/[slug]/page.js switcher
export function generateStaticParams() {}

export default function Page() {
  return <h1>Hello, Blog Post Page!</h1>
}
```

Wrapping a folder name in square brackets (e.g. `[slug]`) creates a dynamic route segment which is used to generate multiple pages from data. e.g. blog posts, product pages, etc.

Nesting layouts

By default, layouts in the folder hierarchy are also nested, which means they wrap child layouts via their `children` prop. You can nest layouts by adding `layout` inside specific route segments (folders).

For example, to create a layout for the `/blog` route, add a new `layout` file inside the `blog` folder.

File hierarchy showing root layout wrapping the blog layout

```tsx
// Archivo: app/blog/layout.tsx switcher
export default function BlogLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return <section>{children}</section>
}
```

```jsx
// Archivo: app/blog/layout.js switcher
export default function BlogLayout({ children }) {
  return <section>{children}</section>
}
```

If you were to combine the two layouts above, the root layout (`app/layout.js`) would wrap the blog layout (`app/blog/layout.js`), which would wrap the blog (`app/blog/page.js`) and blog post page (`app/blog/[slug]/page.js`).

Creating a dynamic segment

Dynamic segments allow you to create routes that are generated from data. For example, instead of manually creating a route for each individual blog post, you can create a dynamic segment to generate the routes based on blog post data.

To create a dynamic segment, wrap the segment (folder) name in square brackets: `[segmentName]`. For example, in the `app/blog/[slug]/page.tsx` route, the `[slug]` is the dynamic segment.

```tsx
// Archivo: app/blog/[slug]/page.tsx switcher
export default async function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await getPost(slug)

  return (
    <div>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </div>
  )
}
```

```jsx
// Archivo: app/blog/[slug]/page.js switcher
export default async function BlogPostPage({ params }) {
  const { slug } = await params
  const post = await getPost(slug)

  return (
    <div>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </div>
  )
}
```

Learn more about Dynamic Segments and the `params` props.

Nested layouts within Dynamic Segments can also access the `params` props.

Rendering with search params

In a Server Component **page**, you can access search parameters using the `searchParams` prop:

```tsx
// Archivo: app/page.tsx switcher
export default async function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const filters = (await searchParams).filters
}
```

```jsx
// Archivo: app/page.jsx switcher
export default async function Page({ searchParams }) {
  const filters = (await searchParams).filters
}
```

Using `searchParams` opts your page into **dynamic rendering** because it requires an incoming request to read the search parameters from.

Client Components can read search params using the `useSearchParams` hook.

Learn more about `useSearchParams` in prerendered and dynamically rendered routes.

What to use and when

- Use the `searchParams` prop when you need search parameters to **load data for the page** (e.g. pagination, filtering from a database).
- Use `useSearchParams` when search parameters are used **only on the client** (e.g. filtering a list already loaded via props).
- As a small optimization, you can use `new URLSearchParams(window.location.search)` in **callbacks or event handlers** to read search params without triggering re-renders.

Linking between pages

You can use the `<Link>` component to navigate between routes. `<Link>` is a built-in Next.js component that extends the HTML `<a>` tag to provide prefetching and client-side navigation.

For example, to generate a list of blog posts, import `<Link>` from `next/link` and pass a `href` prop to the component:

```tsx
// Archivo: app/ui/post.tsx highlight={1,2,11} switcher
import Link from 'next/link'
import { getPosts } from '@/lib/posts'

export default async function Posts() {
  const posts = await getPosts()

  return (
    <ul>
      {posts.map((post) => (
        <li key={post.slug}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/ui/post.js highlight={1,2,11} switcher
import Link from 'next/link'
import { getPosts } from '@/lib/posts'

export default async function Posts() {
  const posts = await getPosts()

  return (
    <ul>
      {posts.map((post) => (
        <li key={post.slug}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}
```

> **Good to know**: `<Link>` is the primary way to navigate between routes in Next.js. You can also use the `useRouter` hook for more advanced navigation.

Route Props Helpers

Next.js exposes utility types that infer `params` and named slots from your route structure:

- **PageProps**: Props for `page` components, including `params` and `searchParams`.
- **LayoutProps**: Props for `layout` components, including `children` and any named slots (e.g. folders like `@analytics`).

These are globally available helpers, generated when running either `next dev`, `next build` or `next typegen`.

```
// Archivo: app/blog/[slug]/page.tsx
export default async function Page(props: PageProps<'/blog/[slug]'>) {
  const { slug } = await props.params
  return <h1>Blog post: {slug}</h1>
}
```

```
// Archivo: app/dashboard/layout.tsx
export default function Layout(props: LayoutProps<'/dashboard'>) {
  return (
    <section>
      {props.children}


    </section>
  )
}
```

> **Good to know**
>
> - Static routes resolve `params` to `{}`.
> - `PageProps`, `LayoutProps` are global helpers — no imports required.
> - Types are generated during `next dev`, `next build` or `next typegen`.

Linking and Navigating

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/linking-and-navigating
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how the built-in navigation optimizations work, including prefetching, prerendering, and client-side navigation, and how to optimize navigation for dynamic routes and slow networks.*

In Next.js, routes are rendered on the server by default. This often means the client has to wait for a server response before a new route can be shown. Next.js comes with built-in prefetching, streaming, and client-side transitions ensuring navigation stays fast and responsive.

This guide explains how navigation works in Next.js and how you can optimize it for dynamic routes and slow networks.

How navigation works

To understand how navigation works in Next.js, it helps to be familiar with the following concepts:

- Server Rendering
- Prefetching
- Streaming
- Client-side transitions

Server Rendering

In Next.js, Layouts and Pages are React Server Components by default. On initial and subsequent navigations, the Server Component Payload is generated on the server before being sent to the client.

There are two types of server rendering, based on *when* it happens:

- **Prerendering** happens at build time or during revalidation and the result is cached.
- **Dynamic Rendering** happens at request time in response to a client request.

The trade-off of server rendering is that the client must wait for the server to respond before the new route can be shown. Next.js addresses this delay by prefetching routes the user is likely to visit and performing client-side transitions.

> **Good to know**: HTML is also generated for the initial visit.

Prefetching

Prefetching is the process of loading a route in the background before the user navigates to it. This makes navigation between routes in your application feel instant, because by the time a user clicks on a link, the data to render the next route is already available client side.

Next.js automatically prefetches routes linked with the `<Link>` component when they enter the user's viewport.

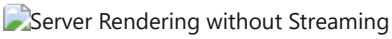
```
// Archivo: app/layout.tsx switcher
import Link from 'next/link'

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <html>
      <body>
        <nav>

          [Blog](/blog)

          <a href="/contact">Contact</a>
        </nav>
        {children}
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import Link from 'next/link'

export default function Layout({ children }) {
  return (
    <html>
      <body>
        <nav>

          [Blog](/blog)

          <a href="/contact">Contact</a>
        </nav>
        {children}
      </body>
    </html>
  )
}
```

How much of the route is prefetched depends on whether it's static or dynamic:

- **Static Route**: the full route is prefetched.
- **Dynamic Route**: prefetching is skipped, or the route is partially prefetched if `loading.tsx` is present.

By skipping or partially prefetching dynamic routes, Next.js avoids unnecessary work on the server for routes the users may never visit. However, waiting for a server response before navigation can give the users the impression that the app is not responding.


Server Rendering without Streaming

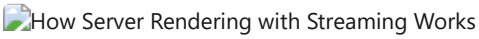
> **Good to know:** See the Prefetching guide for the full behavior, including how to control prefetching per link and how it changes when you adopt Partial Prefetching.

To improve the navigation experience to dynamic routes, you can use streaming.

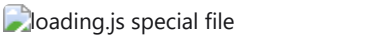
Streaming

Streaming allows the server to send parts of a dynamic route to the client as soon as they're ready, rather than waiting for the entire route to be rendered. This means users see something sooner, even if parts of the page are still loading. See the Streaming guide for a deep dive into how streaming works in Next.js.

For dynamic routes, it means they can be **partially prefetched**. That is, shared layouts and loading skeletons can be requested ahead of time.


How Server Rendering with Streaming Works

To use streaming, create a `loading.tsx` in your route folder:


loading.js special file

```
// Archivo: app/dashboard/loading.tsx switcher
export default function Loading() {
  // Add fallback UI that will be shown while the route is loading.
  return <LoadingSkeleton />
}
```

```
// Archivo: app/dashboard/loading.js switcher
export default function Loading() {
  // Add fallback UI that will be shown while the route is loading.
  return <LoadingSkeleton />
}
```

Behind the scenes, Next.js will automatically wrap the `page.tsx` contents in a `<Suspense>` boundary. The prefetched fallback UI will be shown while the route is loading, and swapped for the actual content once ready.

> **Good to know**: You can also use `<Suspense>` to create loading UI for nested components.

Benefits of `loading.tsx`:

- Immediate navigation and visual feedback for the user.
- Shared layouts remain interactive and navigation is interruptible.
- Improved Core Web Vitals: TTFB, FCP, and TTI.

To further improve the navigation experience, Next.js performs a client-side transition with the `<Link>` component.

Client-side transitions

Traditionally, navigation to a server-rendered page triggers a full page load. This clears state, resets scroll position, and blocks interactivity.

Next.js avoids this with client-side transitions using the `<Link>` component. Instead of reloading the page, it updates the content dynamically by:

- Keeping any shared layouts and UI.
- Replacing the current page with the prefetched loading state or a new page if available.

Client-side transitions make server-rendered apps *feel* like client-rendered apps. And when paired with prefetching and streaming, they enable fast transitions, even for dynamic routes.

Next.js also handles scrolling to the top of the page during client-side transitions. If content scrolls behind a sticky or fixed header after navigation, you can fix this with CSS `scroll-padding-top`.

What can make transitions slow?

These Next.js optimizations make navigation fast and responsive. However, under certain conditions, transitions can still *feel* slow. Here are some common causes and how to improve the user experience:

Dynamic routes without `loading.tsx`

When navigating to a dynamic route, the client must wait for the server response before showing the result. This can give the users the impression that the app is not responding.

We recommend adding `loading.tsx` to dynamic routes to enable partial prefetching, trigger immediate navigation, and display a loading UI while the route renders.

```
// Archivo: app/blog/[slug]/loading.tsx switcher
export default function Loading() {
  return <LoadingSkeleton />
}
```

```
// Archivo: app/blog/[slug]/loading.js switcher
export default function Loading() {
  return <LoadingSkeleton />
}
```

> **Good to know**: In development mode, you can use the Next.js Devtools to identify if the route is static or dynamic. See `devIndicators` for more information.

Dynamic segments without `generateStaticParams`

If a dynamic segment could be prerendered but isn't because it's missing `generateStaticParams`, the route will fallback to dynamic rendering at request time.

Ensure the route is statically generated at build time by adding `generateStaticParams`:

```
// Archivo: app/blog/[slug]/page.tsx switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}

export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}

export default async function Page({ params }) {
  const { slug } = await params
  // ...
}
```

Slow networks

On slow or unstable networks, prefetching may not finish before the user clicks a link. This can affect both static and dynamic routes. In these cases, the `loading.js` fallback may not appear immediately because it hasn't been prefetched yet.

To improve perceived performance, you can use the `useLinkStatus` hook to show immediate feedback while the transition is in progress.

```tsx
// Archivo: app/ui/loading-indicator.tsx switcher
'use client'

import { useLinkStatus } from 'next/link'

export default function LoadingIndicator() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}
```

```jsx
// Archivo: app/ui/loading-indicator.js switcher
'use client'

import { useLinkStatus } from 'next/link'

export default function LoadingIndicator() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}
```

You can "debounce" the hint by adding an initial animation delay (e.g. 100ms) and starting as invisible (e.g. `opacity: 0`). This means the loading indicator will only be shown if the navigation takes longer than the specified delay. See the `useLinkStatus` reference for a CSS example.

> **Good to know**: An **experimental** `useOffline` hook can keep prefetched routes navigable during connectivity drops. See the offline support guide.

> **Good to know**: You can use other visual feedback patterns like a progress bar. View an example here.

Disabling prefetching

You can opt out of prefetching by setting the `prefetch` prop to `false` on the `<Link>` component. This is useful to avoid unnecessary usage of resources when rendering large lists of links (e.g. an infinite scroll table).

```tsx
<Link prefetch={false} href="/blog">
  Blog
</Link>
```

However, disabling prefetching comes with trade-offs:

- **Static routes** will only be fetched when the user clicks the link.
- **Dynamic routes** will need to be rendered on the server first before the client can navigate to it.

To reduce resource usage without fully disabling prefetch, you can prefetch only on hover. This limits prefetching to routes the user is more *likely* to visit, rather than all links in the viewport.

```tsx
// Archivo: app/ui/hover-prefetch-link.tsx switcher
'use client'

import Link from 'next/link'
import { useState } from 'react'

function HoverPrefetchLink({
  href,
  children,
}: {
  href: string
  children: React.ReactNode
}) {
  const [active, setActive] = useState(false)

  return (
    <Link
      href={href}
      prefetch={active ? null : false}
      onMouseEnter={() => setActive(true)}
    >
      {children}
    </Link>
  )
}
```

```jsx
// Archivo: app/ui/hover-prefetch-link.js switcher
'use client'

import Link from 'next/link'
import { useState } from 'react'

function HoverPrefetchLink({ href, children }) {
  const [active, setActive] = useState(false)

  return (
    <Link
      href={href}
      prefetch={active ? null : false}
      onMouseEnter={() => setActive(true)}
    >
      {children}
    </Link>
  )
}
```

Hydration not completed

`<Link>` is a Client Component and must be hydrated before it can prefetch routes. On the initial visit, large JavaScript bundles can delay hydration, preventing prefetching from starting right away.

React mitigates this with Selective Hydration and you can further improve this by:

- Using the `@next/bundle-analyzer` plugin to identify and reduce bundle size by removing large dependencies.
- Moving logic from the client to the server where possible. See the Server and Client Components docs for guidance.

Examples

Native History API

Next.js allows you to use the native `window.history.pushState` and `window.history.replaceState` methods to update the browser's history stack without reloading the page.

`pushState` and `replaceState` calls integrate into the Next.js Router, allowing you to sync with `usePathname` and `useSearchParams`.

`window.history.pushState`

Use it to add a new entry to the browser's history stack. The user can navigate back to the previous state. For example, to sort a list of products:

```
'use client'

import { useSearchParams } from 'next/navigation'

export default function SortProducts() {
  const searchParams = useSearchParams()

  function updateSorting(sortOrder: string) {
    const params = new URLSearchParams(searchParams.toString())
    params.set('sort', sortOrder)
    window.history.pushState(null, '', `?${params.toString()}`)
  }

  return (
    <>
      <button onClick={() => updateSorting('asc')}>Sort Ascending</button>
      <button onClick={() => updateSorting('desc')}>Sort Descending</button>
    </>
  )
}
```

```
'use client'

import { useSearchParams } from 'next/navigation'

export default function SortProducts() {
  const searchParams = useSearchParams()

  function updateSorting(sortOrder) {
    const params = new URLSearchParams(searchParams.toString())
    params.set('sort', sortOrder)
    window.history.pushState(null, '', `?${params.toString()}`)
  }

  return (
    <>
      <button onClick={() => updateSorting('asc')}>Sort Ascending</button>
      <button onClick={() => updateSorting('desc')}>Sort Descending</button>
    </>
  )
}
```

`window.history.replaceState`

Use it to replace the current entry on the browser's history stack. The user is not able to navigate back to the previous state. For example, to switch the application's locale:

```
'use client'

import { usePathname } from 'next/navigation'

export function LocaleSwitcher() {
  const pathname = usePathname()

  function switchLocale(locale: string) {
    // e.g. '/en/about' or '/fr/contact'
    const newPath = `/${locale}${pathname}`
    window.history.replaceState(null, '', newPath)
  }

  return (
    <>
      <button onClick={() => switchLocale('en')}>English</button>
      <button onClick={() => switchLocale('fr')}>French</button>
    </>
  )
}
```

```
'use client'

import { usePathname } from 'next/navigation'

export function LocaleSwitcher() {
  const pathname = usePathname()

  function switchLocale(locale) {
    // e.g. '/en/about' or '/fr/contact'
    const newPath = `/${locale}${pathname}`
    window.history.replaceState(null, '', newPath)
  }

  return (
    <>
      <button onClick={() => switchLocale('en')}>English</button>
      <button onClick={() => switchLocale('fr')}>French</button>
    </>
  )
}
```

Route Handlers

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/route-handlers
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Create custom request handlers with Next.js Route Handlers using the Web Request and Response APIs.*

Route Handlers

Route Handlers allow you to create custom request handlers for a given route using the Web Request and Response APIs.

> **Good to know**: Route Handlers are only available inside the `app` directory. They are the equivalent of API Routes inside the `pages` directory meaning you **do not** need to use API Routes and Route Handlers together.

Convention

Route Handlers are defined in a `route.js|ts` file inside the `app` directory:

```
// Archivo: app/api/route.ts switcher
export async function GET(request: Request) {}
```

```
// Archivo: app/api/route.js switcher
export async function GET(request) {}
```

Route Handlers can be nested anywhere inside the `app` directory, similar to `page.js` and `layout.js`. But there **cannot** be a `route.js` file at the same route segment level as `page.js`.

Supported HTTP Methods

The following HTTP methods are supported: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, and `OPTIONS`. If an unsupported method is called, Next.js will return a `405 Method Not Allowed` response.

Extended `NextRequest` and `NextResponse` APIs

In addition to supporting the native Request and Response APIs, Next.js extends them with `NextRequest` and `NextResponse` to provide convenient helpers for advanced use cases.

Caching

Route Handlers are not cached by default. You can, however, opt into caching for `GET` methods. Other supported HTTP methods are **not** cached. To cache a `GET` method, use a route config option such as `export const dynamic = 'force-static'` in your Route Handler file.

```
// Archivo: app/items/route.ts switcher
export const dynamic = 'force-static'

export async function GET() {
  const res = await fetch('https://data.mongodb-api.com/...', {
    headers: {
      'Content-Type': 'application/json',
      'API-Key': process.env.DATA_API_KEY,
    },
  })
  const data = await res.json()

  return Response.json({ data })
}
```

```
// Archivo: app/items/route.js switcher
export const dynamic = 'force-static'

export async function GET() {
  const res = await fetch('https://data.mongodb-api.com/...', {
    headers: {
      'Content-Type': 'application/json',
      'API-Key': process.env.DATA_API_KEY,
    },
  })
  const data = await res.json()

  return Response.json({ data })
}
```

> **Good to know**: Other supported HTTP methods are **not** cached, even if they are placed alongside a `GET` method that is cached, in the same file.

With Cache Components

When Cache Components is enabled, `GET` Route Handlers follow the same model as normal UI routes in your application. They run at request time by default, can be prerendered when they don't access uncached or runtime data, and you can use `use cache` to include uncached data in the static response.

Static example - doesn't access uncached or runtime data, so it will be prerendered at build time:

```
// Archivo: app/api/project-info/route.ts
export async function GET() {
  return Response.json({
    projectName: 'Next.js',
  })
}
```

Dynamic example - accesses non-deterministic operations. During the build, prerendering stops when `Math.random()` is called, deferring to request-time rendering:

```
// Archivo: app/api/random-number/route.ts
export async function GET() {
  return Response.json({
    randomNumber: Math.random(),
  })
}
```

Runtime data example - accesses request-specific data. Prerendering terminates when runtime APIs like `headers()` are called:

```
// Archivo: app/api/user-agent/route.ts
import { headers } from 'next/headers'

export async function GET() {
  const headersList = await headers()
  const userAgent = headersList.get('user-agent')

  return Response.json({ userAgent })
}
```

> **Good to know**: Prerendering stops if the `GET` handler accesses network requests, database queries, async file system operations, request object properties (like `req.url`, `request.headers`, `request.cookies`, `request.body`), runtime APIs like `cookies()`, `headers()`, `connection()`, or non-deterministic operations.

Cached example - accesses uncached data (database query) but caches it with `use cache`, allowing it to be included in the prerendered response:

```
// Archivo: app/api/products/route.ts
import { cacheLife } from 'next/cache'

export async function GET() {
  const products = await getProducts()
  return Response.json(products)
}

async function getProducts() {
  'use cache'
  cacheLife('hours')

  return await db.query('SELECT * FROM products')
}
```

> **Good to know**: `use cache` cannot be used directly inside a Route Handler body; extract it to a helper function. Cached responses revalidate according to `cacheLife` when a new request arrives.

Special Route Handlers

Special Route Handlers like `sitemap.ts`, `opengraph-image.tsx`, and `icon.tsx`, and other metadata files remain static by default unless they use Request-time APIs or dynamic config options.

Route Resolution

You can consider a `route` the lowest level routing primitive.

- They **do not** participate in layouts or client-side navigations like `page`.
- There **cannot** be a `route.js` file at the same route as `page.js`.

Page	Route	Result
`app/page.js`	`app/route.js`	Conflict
`app/page.js`	`app/api/route.js`	Valid
`app/[user]/page.js`	`app/api/route.js`	Valid

Each `route.js` or `page.js` file takes over all HTTP verbs for that route.

```
// Archivo: app/page.ts switcher
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}

// Conflict
// `app/route.ts`
export async function POST(request: Request) {}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}

// Conflict
// `app/route.js`
export async function POST(request) {}
```

Read more about how Route Handlers complement your frontend application, or explore the Route Handlers API Reference.

Route Context Helper

In TypeScript, you can type the `context` parameter for Route Handlers with the globally available `RouteContext` helper:

```
// Archivo: app/users/[id]/route.ts switcher
import type { NextRequest } from 'next/server'

export async function GET(_req: NextRequest, ctx: RouteContext<'/users/[id]'>) {
  const { id } = await ctx.params
  return Response.json({ id })
}
```

> **Good to know**
>
> - Types are generated during `next dev`, `next build` or `next typegen`.

Adopting Partial Prefetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/adopting-partial-prefetching
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to enable Partial Prefetching and what changes for* `<Link>`.

Partial Prefetching changes what a `<Link>` downloads for a Cache Components route. With Partial Prefetching enabled, a `<Link>` prefetches the route's App Shell: its static content and the cached content that doesn't depend on the URL. Next.js builds one App Shell per route and reuses it for every link to that route, rather than prefetching each link separately as it did before.

To prefetch more than the App Shell, a link can opt into per-link prefetching with `<Link prefetch={true}>`. The prefetch can then resolve cached URL-specific content that depends on `params`, `searchParams`, or the full URL.

Along the way, Next.js surfaces instant navigation insights in development, naming the link or route to change.

> **Good to know**: Partial Prefetching only works when `cacheComponents` is enabled.

Use the adoption skill (recommended)

The `next-partial-prefetching-adoption` skill drives this adoption with a coding agent. It reviews your `<Link prefetch={true}>` calls, captures the prefetched UI to preserve with `instant()` tests when a production test rig is available, then enables the flag and checks each route for the insights this guide covers.

Install the skill:

```
// Archivo: Terminal
npx skills add vercel/next.js --skill next-partial-prefetching-adoption
```

Then give the agent this prompt:

```
Adopt Partial Prefetching in this project using the next-partial-prefetching-adoption skill.
```

Or adopt by hand

To adopt an existing app by hand:

1. Review existing full prefetches and choose which part of each destination should continue to be prefetched. You can use `instant()` tests to add regression coverage.
2. Enable `partialPrefetching`. For larger migrations, you can adopt incrementally with the global flag off. The dynamic data during prefetching insight identifies each destination that still uses the legacy full prefetch.
3. Move URL data behind Suspense, resolving the URL data outside of Suspense insight.
4. Optionally, prefetch URL data on the routes where streaming in after navigation isn't enough.

Both insights are development-only and never block the build. They appear in the dev overlay with fix cards that link the docs page for each fix. If a route isn't ready to adopt, export `instant = false` from its page or layout to opt the route out of instant-navigation validation, and return to it later.

Enable Partial Prefetching

> **Good to know**: In an existing app, review the UI delivered by `<Link prefetch={true}>` before enabling Partial Prefetching. These links use the legacy full prefetch, and enabling Partial Prefetching changes their behavior.

Enable `partialPrefetching` in `next.config.ts`:

```
// Archivo: next.config.ts highlight={5}
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
  partialPrefetching: true,
}

export default nextConfig
```

After enabling the flag, links using the default behavior, `prefetch="auto"`, or `prefetch={true}` prefetch the destination's App Shell. A link with `prefetch={true}` can also resolve cached URL-specific content, but it no longer includes uncached dynamic content from the legacy full prefetch. New projects can enable the flag without this audit because they have no legacy full prefetches to migrate.

What changes for `<Link>`

`<Link>` prop	Before (Cache Components default)	After Partial Prefetching
`[`	Prefetched the cached page render.	Prefetches the shared App Shell for `/x`.
`<Link href="/x" prefetch>`	Prefetched the cached page render **and** any dynamic content.	Prefetches the App Shell, plus cached URL-specific content through per-link prefetching when `/x` reads it.
`<Link href="/x" prefetch={false}>`	Disabled prefetching for this link.	Unchanged. Still disabled.

The App Shell is shared across every link to a given route, regardless of dynamic params, so rendering many `<Link>`s to the same destination doesn't multiply the work.

Migrate existing full prefetches

Before Partial Prefetching, a `<Link>` whose effective `prefetch` value is `true` prefetches the complete destination, including uncached dynamic content. This includes `prefetch={true}`, the bare `prefetch` prop, and wrapper or conditional props that resolve to `true`. The default value, `prefetch="auto"`, and `prefetch={false}` don't use the legacy full prefetch.

A legacy full prefetch can include more UI than the navigation needs. In most cases, the primary content at the top of the destination should be available before navigation, while secondary or frequently changing content can stream afterward.

> **Good to know**: `cookies()` and `headers()` don't tie a prefetch to a URL. They vary per session, not per link, so the App Shell still carries session content. Only `params` and `searchParams` are URL data, which varies per link and can't be included in the shared App Shell.

Verify prefetched UI with tests

To verify the migration programmatically, an `instant()` test can capture the UI that should stay available from an existing full prefetch. Run this baseline against a production build with `partialPrefetching` disabled because automatic prefetching doesn't run in `next dev`. Then rerun the same assertions after adopting the destination. A failure identifies UI the App Shell no longer carries, so you can cache the data or subtree that restores it. The passing test provides regression coverage.

Static or cached content

The output is already in the App Shell. Remove the now-redundant `prefetch={true}`:

```
// Archivo: app/nav.tsx
// Before
<Link href="/about" prefetch={true}>About](/x)
// After
[About](/about)
```

Uncached content

Cache it with `use cache` so it gets included in the App Shell, then remove `prefetch={true}` from the links:

```
// Archivo: app/products/page.tsx switcher
// Before
export default async function Page() {
  const res = await fetch('https://api.example.com/products')
  return <ProductList products={await res.json()} />
}
```

```
// Archivo: app/products/page.js switcher
// Before
export default async function Page() {
  const res = await fetch('https://api.example.com/products')
  return <ProductList products={await res.json()} />
}
```

```
// Archivo: app/products/page.tsx switcher
// After - cached, so the App Shell carries it
async function getProducts() {
  'use cache'
  const res = await fetch('https://api.example.com/products')
  return res.json()
}

export default async function Page() {
  return <ProductList products={await getProducts()} />
}
```

```
// Archivo: app/products/page.js switcher
// After - cached, so the App Shell carries it
async function getProducts() {
  'use cache'
  const res = await fetch('https://api.example.com/products')
  return res.json()
}

export default async function Page() {
  return <ProductList products={await getProducts()} />
}
```

> **Good to know**: The App Shell carries cached content whose `stale` time is at least 5 minutes, which holds for the `default` profile used above and every preset except `seconds`. Shorter-lived content streams in after the navigation instead. See Prerendering behavior.

Session content

Content behind `cookies()` or `headers()` varies per session, not per link, and session data resolves in the App Shell, so cached session content still gets included. Read the session value outside the cached function and pass it in, then remove `prefetch={true}` from the links:

```
// Archivo: app/dashboard/page.tsx switcher
// Before
import { Suspense } from 'react'
import { cookies } from 'next/headers'

async function TeamTopics() {
  const team = (await cookies()).get('team')?.value
  const topics = await db.topics.forTeam(team)
  return <TopicList topics={topics} />
}

export default function Page() {
  return (
    ``<Suspense>``}>
      <TeamTopics />
    `<Suspense>`
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
// Before
import { Suspense } from 'react'
import { cookies } from 'next/headers'

async function TeamTopics() {
  const team = (await cookies()).get('team')?.value
  const topics = await db.topics.forTeam(team)
  return <TopicList topics={topics} />
}

export default function Page() {
  return (
    ``<Suspense>``}>
      <TeamTopics />
    `<Suspense>`
  )
}
```

```
// Archivo: app/dashboard/page.tsx switcher
// After - the lookup is cached behind the session value
import { Suspense } from 'react'
import { cookies } from 'next/headers'

async function getTopics(team: string | undefined) {
  'use cache'
  return db.topics.forTeam(team)
}

async function TeamTopics() {
  const team = (await cookies()).get('team')?.value
  return <TopicList topics={await getTopics(team)} />
}

export default function Page() {
  return (
    ``<Suspense>``}>
      <TeamTopics />
    `<Suspense>`
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
// After - the lookup is cached behind the session value
import { Suspense } from 'react'
import { cookies } from 'next/headers'

async function getTopics(team) {
  'use cache'
  return db.topics.forTeam(team)
}

async function TeamTopics() {
  const team = (await cookies()).get('team')?.value
  return <TopicList topics={await getTopics(team)} />
}

export default function Page() {
  return (
    ``<Suspense>``}>
      <TeamTopics />
    `<Suspense>`
  )
}
```

URL data

URL data (`params` , `searchParams`) varies per link, so content that depends on it can't be included in the shared App Shell and streams in after navigation from behind its `````', highlight: true },]} />

> 💡 **Solución recomendada (block):** Allow blocking route — Ver guía de solución

The insight never blocks the build, so this pass can happen any time after the flag is on. Load every route in `next dev` to check for it.

The fix is to keep the URL-independent parts of the route outside the boundary and move the `params` or `searchParams` read into a child wrapped in `````) }

```
```jsx
// Archivo: app/products/[slug]/page.js switcher
// After - pass the promise down without awaiting it
import { Suspense } from 'react'
import { ProductDetails } from './product-details'

export default function Page({ params }) {
  return (
    <ProductLayout>
      ``<Suspense>``}>
        <ProductDetails params={params} />
      `<Suspense>`
    </ProductLayout>
  )
}
```

The child awaits the promise inside the boundary:

```
// Archivo: app/products/[slug]/product-details.tsx switcher
export async function ProductDetails({
  params,
}: Pick<PageProps<'/products/[slug]'>, 'params'>) {
  const { slug } = await params
  const product = await getProduct(slug)
  return ``<Details>``
}
```
```

```
// Archivo: app/products/[slug]/product-details.js switcher
export async function ProductDetails({ params }) {
  const { slug } = await params
  const product = await getProduct(slug)
  return ``<Details>``
}
```

Everything outside the boundary stays in the shared App Shell, and only the URL-specific region renders per navigation. Load or navigate to the route again to confirm the insight is gone and the page still paints meaningful UI.

> **Good to know**: A `params` or `searchParams` read inside `generateMetadata` surfaces as URL data in `generateMetadata()` instead.

Prefetching URL data

Content that depends on URL data (`params` , `searchParams`) can't be included in the shared App Shell, so it streams in after navigation. Per-link prefetching resolves it ahead of the click for links with `prefetch={true}` , at the cost of a server invocation per prefetchable link. To make the content resolvable at prefetch time, cache it behind the read with `use cache` . For a search page that reads `searchParams` :

```
// Archivo: app/search/page.tsx switcher
// Before - the results stream in after navigation
import { Suspense } from 'react'

async function getResults(query: string) {
  const res = await fetch(`https://api.example.com/search?q=${query}`)
  return res.json()
}

async function Results({
  searchParams,
}: Pick<PageProps<'/search'>, 'searchParams'>) {
  const { q } = await searchParams
  return <ResultList results={await getResults(q)} />
}

export default function Page({ searchParams }: PageProps<'/search'>) {
  return (
    ``<Suspense>``}>
      <Results searchParams={searchParams} />
    `<Suspense>`
  )
}
```

```
// Archivo: app/search/page.js switcher
// Before - the results stream in after navigation
import { Suspense } from 'react'

async function getResults(query) {
  const res = await fetch(`https://api.example.com/search?q=${query}`)
  return res.json()
}

async function Results({ searchParams }) {
  const { q } = await searchParams
  return <ResultList results={await getResults(q)} />
}

export default function Page({ searchParams }) {
  return (
    ``<Suspense>``}>
      <Results searchParams={searchParams} />
    `<Suspense>`
  )
}
```

```tsx
// Archivo: app/search/page.tsx switcher
// After - the results are cached and prefetched behind the resolved searchParams
import { Suspense } from 'react'

async function getResults(query: string) {
  'use cache'
  const res = await fetch(`https://api.example.com/search?q=${query}`)
  return res.json()
}

async function Results({
  searchParams,
}: Pick<PageProps<'/search'>, 'searchParams'>) {
  const { q } = await searchParams
  return <ResultList results={await getResults(q)} />
}

export default function Page({ searchParams }: PageProps<'/search'>) {
  return (
    ``<Suspense>``}>
      <Results searchParams={searchParams} />
    `<Suspense>`
  )
}
```

```jsx
// Archivo: app/search/page.js switcher
// After - the results are cached and prefetched behind the resolved searchParams
import { Suspense } from 'react'

async function getResults(query) {
  'use cache'
  const res = await fetch(`https://api.example.com/search?q=${query}`)
  return res.json()
}

async function Results({ searchParams }) {
  const { q } = await searchParams
  return <ResultList results={await getResults(q)} />
}

export default function Page({ searchParams }) {
  return (
    ``<Suspense>``}>
      <Results searchParams={searchParams} />
    `<Suspense>`
  )
}
```

The Optimizing prefetching guide covers when the cost pays off and the caching patterns behind runtime reads.

How to preview content with Draft Mode in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/draft-mode
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Bypass Next.js caching for a request so editors can preview unpublished content from a headless CMS.*

Draft Mode lets editors see how draft or in-progress content will render on your site, without waiting for revalidation. While an editor is in Draft Mode, cached or pre-rendered content is bypassed, and fetched from upstream sources directly. Other visitors continue to see the cached or pre-rendered version of the page.

Your data-fetching code does not need to change if your CMS serves draft and published content from the same URL. Otherwise, see When your CMS uses a separate draft endpoint.

What Draft Mode does

When Draft Mode is enabled for a request:

- `fetch()` calls skip the Next.js fetch cache and hit the network directly.
- Components and functions inside `'use cache'` re-execute on every request, and their results are not saved to the cache.
- `unstable_cache` reads and writes are bypassed in the same way.
- The page is excluded from the ISR response cache and is served with `Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate`.

The effect applies whether the page is statically generated, served from cache, or revalidated through ISR.

What this guide covers

This guide assumes:

- Your headless CMS supports configurable preview URLs (most do).
- The CMS opens a URL like `/api/draft?secret=XXX&slug=/posts/foo` in a new tab when an editor clicks "Preview". The secret is a shared token; the slug is the path to preview.
- Your Next.js app validates the secret, enables Draft Mode, and redirects to the slug.

With that contract in mind, the rest of this guide walks through:

1. Creating a Route Handler that enables Draft Mode by setting a cookie.
2. Securing that handler with the shared secret and slug from the CMS.
3. Rendering pages that read the latest draft.
4. Showing a preview banner with an exit form.

Then, depending on your setup:

- Draft Mode with Cache Components for surfacing the preview state from a `'use cache'` boundary.
- When your CMS uses a separate draft endpoint for branching the fetch URL on `isEnabled`.

> **Good to know:** `GET` is meant to be a safe, read-only method. Operations that affect future requests, like enabling Draft Mode via a cookie, should use `POST`. The entry handler uses `GET` because we're assuming a CMS preview integration: the CMS opens the URL in a new browser tab, which is a `GET` request. The exit flow in Step 4 uses `POST` (via Server Action or `POST` Route Handler).

Step 1: Create a Route Handler

Create a Route Handler that sets the Draft Mode cookie. It can have any name, for example, `app/api/draft/route.ts`.

```
// Archivo: app/api/draft/route.ts switcher
import { draftMode } from 'next/headers'

export async function GET(request: Request) {
  const draft = await draftMode()
  draft.enable()
  return new Response('Draft mode is enabled')
}
```

```
// Archivo: app/api/draft/route.js switcher
import { draftMode } from 'next/headers'

export async function GET(request) {
  const draft = await draftMode()
  draft.enable()
  return new Response('Draft mode is enabled')
}
```

`draft.enable()` sets a cookie named `__prerender_bypass`. Subsequent requests that carry this cookie skip every cache layer listed above.

You can test this manually by visiting `/api/draft` and looking at your browser's developer tools. Notice the `Set-Cookie` response header.

As written, the handler is public: anyone who hits `/api/draft` enables Draft Mode for themselves. Step 2 closes that with a shared secret so only your CMS can call it.

Step 2: Access the Route Handler from your headless CMS

> These steps assume that the headless CMS you're using supports setting **custom draft URLs**. If it doesn't, you can still use this method to secure your draft URLs, but you'll need to construct and access the draft URL manually. The specific steps will vary depending on which headless CMS you're using.

To securely access the Route Handler from your headless CMS:

1. Create a **secret token string** using a token generator of your choice. This secret is only known to your Next.js app and your headless CMS.
2. If your headless CMS supports setting custom draft URLs, specify a draft URL (this assumes that your Route Handler is located at `app/api/draft/route.ts`). For example:

```
// Archivo: Terminal
https://<your-site>/api/draft?secret=<token>&slug=<path>
```

> - `<your-site>` should be your deployment domain.
> - `<token>` should be replaced with the secret token you generated.
> - `<path>` should be the path for the page that you want to view. If you want to view `/posts/one`, then you should use `&slug=/posts/one`.
>
> Your headless CMS might allow you to include a variable in the draft URL so that `<path>` can be set dynamically based on the CMS's data like so: `&slug=/posts/{entry.fields.slug}`

3. In your Route Handler, check that the secret matches and that the `slug` parameter exists (if not, the request should fail), call `draft.enable()` to set the cookie, then redirect the browser to the path specified by `slug`:

```ts
// Archivo: app/api/draft/route.ts switcher
import { draftMode } from 'next/headers'
import { redirect } from 'next/navigation'

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url)
  const secret = searchParams.get('secret')
  const slug = searchParams.get('slug')

  // This secret should only be known to this Route Handler and the CMS
  if (secret !== 'MY_SECRET_TOKEN' || !slug) {
    return new Response('Invalid token', { status: 401 })
  }

  // Verify the slug exists in the CMS before enabling Draft Mode
  const post = await getPostBySlug(slug)
  if (!post) {
    return new Response('Invalid slug', { status: 401 })
  }

  const draft = await draftMode()
  draft.enable()

  // Redirect to the path from the fetched post, not from searchParams,
  // to avoid open redirect vulnerabilities
  redirect(post.slug)
}
```

```js
// Archivo: app/api/draft/route.js switcher
import { draftMode } from 'next/headers'
import { redirect } from 'next/navigation'

export async function GET(request) {
  const { searchParams } = new URL(request.url)
  const secret = searchParams.get('secret')
  const slug = searchParams.get('slug')

  if (secret !== 'MY_SECRET_TOKEN' || !slug) {
    return new Response('Invalid token', { status: 401 })
  }

  const post = await getPostBySlug(slug)
  if (!post) {
    return new Response('Invalid slug', { status: 401 })
  }

  const draft = await draftMode()
  draft.enable()

  redirect(post.slug)
}
```

If it succeeds, the browser is redirected to the target path with the Draft Mode cookie set.

Step 3: Preview the draft content

Because Draft Mode bypasses the cache automatically, your page does not need to know whether Draft Mode is on to receive fresh content. Fetch as you normally would:

```tsx
// Archivo: app/posts/[slug]/page.tsx switcher
async function getPost(slug: string) {
  const res = await fetch(`https://cms.example.com/posts/${slug}`)
  return res.json()
}

export default async function Page({ params }: PageProps<'/posts/[slug]'>) {
  const { slug } = await params
  const post = await getPost(slug)

  return (
    <main>
      <h1>{post.title}</h1>
      <article>{post.content}</article>
    </main>
  )
}
```

```jsx
// Archivo: app/posts/[slug]/page.js switcher
async function getPost(slug) {
  const res = await fetch(`https://cms.example.com/posts/${slug}`)
  return res.json()
}

export default async function Page({ params }) {
  const { slug } = await params
  const post = await getPost(slug)

  return (
    <main>
      <h1>{post.title}</h1>
      <article>{post.content}</article>
    </main>
  )
}
```

When the Draft Mode cookie is present, the `fetch` above skips the Next.js fetch cache and hits your CMS for the current draft. When it is not, the same request can be served from cache as usual.

If your CMS uses a different URL for drafts rather than serving them from the same endpoint, see When your CMS uses a separate draft endpoint.

Step 4: Show a preview indicator

`isEnabled` is most useful as a signal to the editor: a banner that confirms they are looking at draft content, plus a way to exit. Render an indicator from your root layout so it appears on every preview page.

```tsx
// Archivo: app/preview-banner.tsx switcher
import { draftMode } from 'next/headers'
import { redirect } from 'next/navigation'

async function exitPreview() {
  'use server'
  const draft = await draftMode()
  draft.disable()
  redirect('/')
}

export async function PreviewBanner() {
  const { isEnabled } = await draftMode()
  if (!isEnabled) return null

  return (
    <aside role="status">
      Preview mode is on.{' '}
      <form action={exitPreview}>
        <button type="submit">Exit preview</button>
      </form>
    </aside>
  )
}
```

```jsx
// Archivo: app/preview-banner.js switcher
import { draftMode } from 'next/headers'
import { redirect } from 'next/navigation'

async function exitPreview() {
  'use server'
  const draft = await draftMode()
  draft.disable()
  redirect('/')
}

export async function PreviewBanner() {
  const { isEnabled } = await draftMode()
  if (!isEnabled) return null

  return (
    <aside role="status">
      Preview mode is on.{' '}
      <form action={exitPreview}>
        <button type="submit">Exit preview</button>
      </form>
    </aside>
  )
}
```

Exiting Draft Mode also works with a `GET` Route Handler, but a `POST` is semantically more correct, for example via a form submitted through a Server Action or to a `POST` Route Handler.

If you do use a `GET` Route Handler, trigger it from a `<form method="GET">` rather than a `<Link>`. Next.js prefetches `<Link>` components by default, which would clear the cookie before the editor clicks. Forms are not prefetched, regardless of method.

Draft Mode with Cache Components

You can read `isEnabled` inside a `'use cache'` scope to render a preview indicator from a cached component. The cache bypass still applies, so the component re-executes with fresh data on every draft request.

```tsx
// Archivo: app/posts/[slug]/page.tsx switcher
import { draftMode } from 'next/headers'

async function Post({ slug }: { slug: string }) {
  'use cache'

  const post = await fetch(`https://cms.example.com/posts/${slug}`).then((r) =>
    r.json()
  )
  const { isEnabled } = await draftMode()

  return (
    <article>
      {isEnabled && <p role="status">Draft preview</p>}
      <h1>{post.title}</h1>
      <div>{post.content}</div>
    </article>
  )
}
```

```jsx
// Archivo: app/posts/[slug]/page.js switcher
import { draftMode } from 'next/headers'

async function Post({ slug }) {
  'use cache'

  const post = await fetch(`https://cms.example.com/posts/${slug}`).then((r) =>
    r.json()
  )
  const { isEnabled } = await draftMode()

  return (
    <article>
      {isEnabled && <p role="status">Draft preview</p>}
      <h1>{post.title}</h1>
      <div>{post.content}</div>
    </article>
  )
}
```

> **Good to know:** `draftMode().enable()` and `draftMode().disable()` cannot be called inside a caching directive scope; toggle Draft Mode from a Route Handler or Server Action instead.

When your CMS uses a separate draft endpoint

If your CMS exposes draft content at a different URL or requires different credentials, branch your fetch on `isEnabled`:

```tsx
// Archivo: app/posts/[slug]/page.tsx switcher
import { draftMode } from 'next/headers'

async function getPost(slug: string) {
  const { isEnabled } = await draftMode()
  const baseUrl = isEnabled
    ? 'https://cms.example.com/preview'
    : 'https://cms.example.com/published'

  const res = await fetch(`${baseUrl}/posts/${slug}`)
  return res.json()
}
```

```
// Archivo: app/posts/[slug]/page.js switcher
import { draftMode } from 'next/headers'

async function getPost(slug) {
  const { isEnabled } = await draftMode()
  const baseUrl = isEnabled
    ? 'https://cms.example.com/preview'
    : 'https://cms.example.com/published'

  const res = await fetch(`${baseUrl}/posts/${slug}`)
  return res.json()
}
```

The cache bypass still applies to both branches; the fork only chooses where to read from.

Ensuring instant navigations

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/instant-navigation
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to structure your app to prefetch and prerender more content, providing instant page loads and client navigations.*

This guide walks through understanding instant navigations, writing a route that navigates instantly, visualizing what's in the initial UI, and locking the behavior in with end-to-end tests.

What "instant" means

A navigation is **instant** when the browser can start rendering the new page the moment the user clicks, with static, cached, and fallback content showing up right away, while the server streams the remaining content into its fallbacks.

> **Good to know:** This definition assumes caches are warm. Cold caches still require the server to compute the cached result once, so the first navigation to a route may still wait.

A direct visit and a client navigation to the same route can produce different initial UI. **Direct visits** get the **static shell** as HTML, typically from a CDN. **Client navigations** only re-render below the layout the current and destination routes share, so the fallback UI defined by a

```
```

) }

type Params = PageProps<'/store/[slug]'>['params']

async function ProductInfo({ params }: { params: Params }) { const { slug } = await params const product = await getProduct(slug) return (<>

{product.name}

${product.price}

</>) }

async function getProduct(slug: string) { 'use cache' return db.products.findBySlug(slug) }

async function Inventory({ params }: { params: Params }) { const { slug } = await params const item = await db.inventory.findBySlug(slug) return

{item.count} in stock

}

```
Cache Components validates this route automatically in development. If something would block a navigation, the dev overlay
surfaces a **blocking-route** insight that names the offending component and points at these fixes:



> 💡 **Solución recomendada (cache):** Cache the component or data — [Ver guía de solución]
(https://nextjs.org/docs/messages/blocking-prerender-dynamic#cache-the-component-or-data)
' },
    ]}
  />

> 💡 **Solución recomendada (stream):** Wrap in or move into Suspense — [Ver guía de solución]
(https://nextjs.org/docs/messages/blocking-prerender-dynamic#wrap-in-or-move-into-suspense)
' },
      { text: '`<Suspense>`', highlight: true },
    ]}
  />


> **Good to know:** Each fix card links to a detailed walkthrough with patterns, code samples, and trade-offs. Click a card
to dive in, or use **Copy prompt** to hand the fix to your agent. See [AI workflow](#ai-workflow) for the loop.

Validation runs on every page load using the real request from your browser, so dynamic params like `[slug]` are checked
against actual values as you navigate.

### Client Component Pages

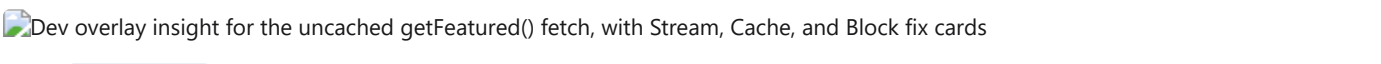
A soft navigation into a page with `"use client"` at the top behaves like a single-page app transition, with no server render
at navigation time, which makes it instant. The dev overlay doesn't include this in its fix cards because it has bigger
implications than the recommended approaches, which keep the page in the server-component model. Reach for `"use client"`
when the page is fully interactive and must be a client component.

> **Good to know**: `"use client"` doesn't skip validation for the static shell. Hooks like `useSearchParams()` still need a
```<Suspense>``
    </div>
)
}
```

Now `await props.params` and the product fetch suspend together. The product-fetch error clears, and validation moves on to the next blocker.

### Step 2: Cache the featured fetch

Validation now fires on `getFeatured()`.


Dev overlay insight for the uncached getFeatured() fetch, with Stream, Cache, and Block fix cards

Add a `"use cache"` directive to the fetching function:

```
// Archivo: app/products/[slug]/page.tsx
async function getFeatured() {
  'use cache'
  const res = await fetch('https://next-recipe-api.vercel.dev/products?limit=3')
  return res.json()
}
```

The result is cached at the fetch level. The featured list ships with the App Shell.

> **Good to know:** In serverless deployments, in-memory caching with `"use cache"` will not persist across instances. Consider using `"use cache: remote"` for persistent caching.

Validation passes. Open the DevTools and try a client navigation. The featured section appears immediately, and **"Loading product..."** shows where the product details will stream in.
```

Iterate on loading states

Validation passing means the navigation is instant. It does not mean the loading states are good. A `<Suspense>` boundary placed high in the tree (say, wrapping the whole page) might satisfy validation, but it replaces most of the page with a single fallback on every navigation.

The best loading states keep as much real, cached content visible as possible and only show fallbacks where data is actually in flight. A product page that keeps the header, image, and description visible with only the price and availability behind a fallback feels faster than a full-page skeleton, even at the same total load time.

Use the DevTools to see what your users see, or see the AI workflow for automating the loop with an agent.

AI workflow

At a high level, an agent optimizes a single navigation like this:

- **Observe**: read validation insights and choose the exact navigation and UI that should appear immediately.
- **Test**: confirm the target UI renders normally, then write an `instant()` test and verify that it fails before changing the route.
- **Fix**: validation errors name a specific component and suggest a fix (`use cache` or `<Suspense>`). The agent applies the fix and re-runs validation.
- **Verify**: run the test against a production-like build and keep the passing test as a regression guard.

The agent doesn't need to understand the full caching model, only to follow the insights and errors until they're gone. It does need your app-specific intent, though. Tell it what should appear immediately, what can stream in, and what must stay fresh. When a caching decision is unclear, keep the data fresh behind `<Suspense>` rather than guessing a cache lifetime.

For iterating on loading states, a prompt like "maximize my content, and reduce the amount that needs to be behind a spinner" works well for pushing boundaries down.

Agents working on a Cache Components route typically reach for three levers:

- **Push down**: extract I/O into a Suspense-wrapped child so the parent stays static and static siblings lift into the shell.
- **Cache**: pair `'use cache'` with `cacheLife` to assign a freshness profile.
- **Per-link prefetching** (nav-only): when a route reads URL data (`searchParams` or `params`), opt it into per-link prefetching so the framework resolves that data at link-prefetch time. Session data from `cookies()` or `headers()` already lands in the App Shell without it.

Each refactor should pair with a before/after capture to verify the change actually landed. Identical-looking captures mean the refactor didn't take effect.

For agents to see what their changes actually render, pair this with agent-browser. With React DevTools enabled it reports the component tree and which `<Suspense>` boundaries are still pending, so an agent can make a change, snapshot the shell, check what landed, and adjust. See Runtime visibility in the AI agents guide for the setup.

The `next-cache-components-optimizer` Skill packages this loop. It confirms the target UI renders normally, writes an `instant()` test that fails before the fix, then works it to green against a production-like build and ships it as a regression guard. Use it for the initial load (hard navigation), client-side navigation (soft navigation), or both.

Opting out

Not every layout or page can or should be instant. When the structural fix isn't worth the work, or when a route isn't a priority for instant navigation, refine validation at one of two scopes.

The dev overlay surfaces this as the **Block** fix alongside every insight:

> 💡 **Solución recomendada (block):** Allow blocking route — Ver guía de solución

Set `instant = false` on the page or layout file. This opts the segment out of validation feedback. The segment may still navigate instantly if its structure supports it; the framework just won't surface insights for it. Navigations between sibling segments below are still validated.

```
// Archivo: app/dashboard/layout.tsx
export const instant = false
```

With `false` on `/dashboard/layout.tsx`, validation no longer flags navigations into `/dashboard` from outside; navigations between `/dashboard/a` and `/dashboard/b` are still checked.

For opted-out segments, the navigation blocks on the server. If the content depends on cookies or headers but has a known cache lifetime, caching it with `use cache: private` lets the App Shell carry it ahead of the click instead of opting out, as long as its `stale` time is at least 5 minutes.

Next steps

- Adopting Partial Prefetching for the recommended `<Link>` defaults and the migration path off `unstable_eager`
- `instant` API reference for the full configuration
- Optimizing prefetching when parts of your route depend on URL data (`searchParams` or `params`) and should resolve before navigation
- Caching for background on `use cache`, Suspense, and Partial Prerendering
- Revalidating for how to expire cached data with `cacheLife` and `updateTag`

How to handle redirects in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/redirecting
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn the different ways to handle redirects in Next.js.*

There are a few ways you can handle redirects in Next.js. This page will go through each available option, use cases, and how to manage large numbers of redirects.

API	Purpose	Where	Status Code
`redirect`	Redirect user after a mutation or event	Server Components, Server Functions, Route Handlers	307 (Temporary) or 303 (Server Action)
`permanentRedirect`	Redirect user after a mutation or event	Server Components, Server Functions, Route Handlers	308 (Permanent)
`useRouter`	Perform a client-side navigation	Event Handlers in Client Components	N/A
`redirects` in `next.config.js`	Redirect an incoming request based on a path	`next.config.js` file	307 (Temporary) or 308 (Permanent)
`NextResponse.redirect`	Redirect an incoming request based on a condition	Proxy	Any

📘 **Nota para Pages Router:**

API	Purpose	Where	Status Code
`useRouter`	Perform a client-side navigation	Components	N/A
`redirects` in `next.config.js`	Redirect an incoming request based on a path	`next.config.js` file	307 (Temporary) or 308 (Permanent)
`NextResponse.redirect`	Redirect an incoming request based on a condition	Proxy	Any

`redirect` function

The `redirect` function allows you to redirect the user to another URL. You can call `redirect` in Server Components, Route Handlers, and Server Functions.

`redirect` is often used after a mutation or event. For example, creating a post:

```
// Archivo: app/actions.ts switcher
'use server'

import { redirect } from 'next/navigation'
import { revalidatePath } from 'next/cache'

export async function createPost(id: string) {
  try {
    // Call database
  } catch (error) {
    // Handle errors
  }

  revalidatePath('/posts') // Update cached posts
  redirect(`/post/${id}`) // Navigate to the new post page
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { redirect } from 'next/navigation'
import { revalidatePath } from 'next/cache'

export async function createPost(id) {
  try {
    // Call database
  } catch (error) {
    // Handle errors
  }

  revalidatePath('/posts') // Update cached posts
  redirect(`/post/${id}`) // Navigate to the new post page
}
```

Good to know:

- In a Server Action, `redirect` performs a client-side navigation when JavaScript is available. Without JavaScript, the form submission uses a 303 (See Other). In other contexts, `redirect` uses a 307 (Temporary Redirect).
- `redirect` throws an error so it should be called **outside** the `try` block when using `try/catch` statements.
- `redirect` can be called in Client Components during the rendering process but not in event handlers. You can use the `useRouter` hook instead.
- `redirect` also accepts absolute URLs and can be used to redirect to external links.
- If you'd like to redirect before the render process, use `next.config.js` or Proxy.

See the `redirect` API reference for more information.

`permanentRedirect` function

The `permanentRedirect` function allows you to **permanently** redirect the user to another URL. You can call `permanentRedirect` in Server Components, Route Handlers, and Server Functions.

`permanentRedirect` is often used after a mutation or event that changes an entity's canonical URL, such as updating a user's profile URL after they change their username:

```
// Archivo: app/actions.ts switcher
'use server'

import { permanentRedirect } from 'next/navigation'
import { revalidateTag } from 'next/cache'

export async function updateUsername(username: string, formData: FormData) {
  try {
    // Call database
  } catch (error) {
    // Handle errors
  }

  revalidateTag('username', 'max') // Update all references to the username
  permanentRedirect(`/profile/${username}`) // Navigate to the new user profile
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { permanentRedirect } from 'next/navigation'
import { revalidateTag } from 'next/cache'

export async function updateUsername(username, formData) {
  try {
    // Call database
  } catch (error) {
    // Handle errors
  }

  revalidateTag('username', 'max') // Update all references to the username
  permanentRedirect(`/profile/${username}`) // Navigate to the new user profile
}
```

Good to know:

- In a Server Action, `permanentRedirect` performs a client-side navigation when JavaScript is available. Without JavaScript, the form submission uses a 303 (See Other). In other contexts, `permanentRedirect` uses a 308 (Permanent Redirect).
- `permanentRedirect` also accepts absolute URLs and can be used to redirect to external links.
- If you'd like to redirect before the render process, use `next.config.js` or Proxy.

See the `permanentRedirect` API reference for more information.

`useRouter()` hook

If you need to redirect inside an event handler in a Client Component, you can use the `push` method from the `useRouter` hook. For example:

```
// Archivo: app/page.tsx switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

```
// Archivo: app/page.js switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

📘 **Nota para Pages Router:** If you need to redirect inside a component, you can use the `push` method from the `useRouter` hook. For example:

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

> **Good to know**:
>
> - If you don't need to programmatically navigate a user, you should use a `<Link>` component.

See the `useRouter` API reference for more information.

> 📘 **Nota para Pages Router:** See the `useRouter` API reference for more information.

`redirects` in `next.config.js`

The `redirects` option in the `next.config.js` file allows you to redirect an incoming request path to a different destination path. This is useful when you change the URL structure of pages or have a list of redirects that are known ahead of time.

`redirects` supports path, header, cookie, and query matching, giving you the flexibility to redirect users based on an incoming request.

To use `redirects`, add the option to your `next.config.js` file:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  async redirects() {
    return [
      // Basic redirect
      {
        source: '/about',
        destination: '/',
        permanent: true,
      },
      // Wildcard path matching
      {
        source: '/blog/:slug',
        destination: '/news/:slug',
        permanent: true,
      },
    ]
  },
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
module.exports = {
  async redirects() {
    return [
      // Basic redirect
      {
        source: '/about',
        destination: '/',
        permanent: true,
      },
      // Wildcard path matching
      {
        source: '/blog/:slug',
        destination: '/news/:slug',
        permanent: true,
      },
    ]
  },
}
```

See the `redirects` API reference for more information.

> **Good to know**:
>
> - `redirects` can return a 307 (Temporary Redirect) or 308 (Permanent Redirect) status code with the `permanent` option.
> - `redirects` may have a limit on platforms. For example, on Vercel, there's a limit of 1,024 redirects. To manage a large number of redirects (1000+), consider creating a custom solution using Proxy. See managing redirects at scale for more.
> - `redirects` runs **before** Proxy.

`NextResponse.redirect` in Proxy

Proxy allows you to run code before a request is completed. Then, based on the incoming request, redirect to a different URL using `NextResponse.redirect`. This is useful if you want to redirect users based on a condition (e.g. authentication, session management, etc) or have a large number of redirects.

For example, to redirect the user to a `/login` page if they are not authenticated:

```
// Archivo: proxy.ts switcher
import { NextResponse, NextRequest } from 'next/server'
import { authenticate } from 'auth-provider'

export function proxy(request: NextRequest) {
  const isAuthenticated = authenticate(request)

  // If the user is authenticated, continue as normal
  if (isAuthenticated) {
    return NextResponse.next()
  }

  // Redirect to login page if not authenticated
  return NextResponse.redirect(new URL('/login', request.url))
}

export const config = {
  matcher: '/dashboard/:path*',
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'
import { authenticate } from 'auth-provider'

export function proxy(request) {
  const isAuthenticated = authenticate(request)

  // If the user is authenticated, continue as normal
  if (isAuthenticated) {
    return NextResponse.next()
  }

  // Redirect to login page if not authenticated
  return NextResponse.redirect(new URL('/login', request.url))
}

export const config = {
  matcher: '/dashboard/:path*',
}
```

> **Good to know**:
>
> - Proxy runs **after** `redirects` in `next.config.js` and **before** rendering.

See the Proxy documentation for more information.

Managing redirects at scale (advanced)

To manage a large number of redirects (1000+), you may consider creating a custom solution using Proxy. This allows you to handle redirects programmatically without having to redeploy your application.

To do this, you'll need to consider:

1. Creating and storing a redirect map.
2. Optimizing data lookup performance.

> **Next.js Example**: See our Proxy with Bloom filter example for an implementation of the recommendations below.

1. Creating and storing a redirect map

A redirect map is a list of redirects that you can store in a database (usually a key-value store) or JSON file.

Consider the following data structure:

```
{
  "/old": {
    "destination": "/new",
    "permanent": true
  },
  "/blog/post-old": {
    "destination": "/blog/post-new",
    "permanent": true
  }
}
```

In Proxy, you can read from a database such as Vercel's Global Config or Redis, and redirect the user based on the incoming request:

```
// Archivo: proxy.ts switcher
import { NextResponse, NextRequest } from 'next/server'
import { get } from '@vercel/global-config'

type RedirectEntry = {
  destination: string
  permanent: boolean
}

export async function proxy(request: NextRequest) {
  const pathname = request.nextUrl.pathname
  const redirectData = await get(pathname)

  if (redirectData && typeof redirectData === 'string') {
    const redirectEntry: RedirectEntry = JSON.parse(redirectData)
    const statusCode = redirectEntry.permanent ? 308 : 307
    return NextResponse.redirect(redirectEntry.destination, statusCode)
  }

  // No redirect found, continue without redirecting
  return NextResponse.next()
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'
import { get } from '@vercel/global-config'

export async function proxy(request) {
  const pathname = request.nextUrl.pathname
  const redirectData = await get(pathname)

  if (redirectData) {
    const redirectEntry = JSON.parse(redirectData)
    const statusCode = redirectEntry.permanent ? 308 : 307
    return NextResponse.redirect(redirectEntry.destination, statusCode)
  }

  // No redirect found, continue without redirecting
  return NextResponse.next()
}
```

2. Optimizing data lookup performance

Reading a large dataset for every incoming request can be slow and expensive. There are two ways you can optimize data lookup performance:

- Use a database that is optimized for fast reads
- Use a data lookup strategy such as a Bloom filter to efficiently check if a redirect exists **before** reading the larger redirects file or database.

Considering the previous example, you can import a generated bloom filter file into Proxy, then, check if the incoming request pathname exists in the bloom filter.

If it does, forward the request to a Route Handler

> 📘 **Nota para Pages Router:** API Routes

which will check the actual file and redirect the user to the appropriate URL. This avoids importing a large redirects file into Proxy, which can slow down every incoming request.

```ts
// Archivo: proxy.ts switcher
import { NextResponse, NextRequest } from 'next/server'
import { ScalableBloomFilter } from 'bloom-filters'
import GeneratedBloomFilter from './redirects/bloom-filter.json'

type RedirectEntry = {
  destination: string
  permanent: boolean
}

// Initialize bloom filter from a generated JSON file
const bloomFilter = ScalableBloomFilter.fromJSON(GeneratedBloomFilter as any)

export async function proxy(request: NextRequest) {
  // Get the path for the incoming request
  const pathname = request.nextUrl.pathname

  // Check if the path is in the bloom filter
  if (bloomFilter.has(pathname)) {
    // Forward the pathname to the Route Handler
    const api = new URL(
      `/api/redirects?pathname=${encodeURIComponent(request.nextUrl.pathname)}`,
      request.nextUrl.origin
    )

    try {
      // Fetch redirect data from the Route Handler
      const redirectData = await fetch(api)

      if (redirectData.ok) {
        const redirectEntry: RedirectEntry | undefined =
          await redirectData.json()

        if (redirectEntry) {
          // Determine the status code
          const statusCode = redirectEntry.permanent ? 308 : 307

          // Redirect to the destination
          return NextResponse.redirect(redirectEntry.destination, statusCode)
        }
      }
    } catch (error) {
      console.error(error)
    }
  }

  // No redirect found, continue the request without redirecting
  return NextResponse.next()
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'
import { ScalableBloomFilter } from 'bloom-filters'
import GeneratedBloomFilter from './redirects/bloom-filter.json'

// Initialize bloom filter from a generated JSON file
const bloomFilter = ScalableBloomFilter.fromJSON(GeneratedBloomFilter)

export async function proxy(request) {
  // Get the path for the incoming request
  const pathname = request.nextUrl.pathname

  // Check if the path is in the bloom filter
  if (bloomFilter.has(pathname)) {
    // Forward the pathname to the Route Handler
    const api = new URL(
      `/api/redirects?pathname=${encodeURIComponent(request.nextUrl.pathname)}`,
      request.nextUrl.origin
    )

    try {
      // Fetch redirect data from the Route Handler
      const redirectData = await fetch(api)

      if (redirectData.ok) {
        const redirectEntry = await redirectData.json()

        if (redirectEntry) {
          // Determine the status code
          const statusCode = redirectEntry.permanent ? 308 : 307

          // Redirect to the destination
          return NextResponse.redirect(redirectEntry.destination, statusCode)
        }
      }
    } catch (error) {
      console.error(error)
    }
  }

  // No redirect found, continue the request without redirecting
  return NextResponse.next()
}
```

Then, in the Route Handler:

```ts
// Archivo: app/api/redirects/route.ts switcher
import { NextRequest, NextResponse } from 'next/server'
import redirects from '@/app/redirects/redirects.json'

type RedirectEntry = {
  destination: string
  permanent: boolean
}

export function GET(request: NextRequest) {
  const pathname = request.nextUrl.searchParams.get('pathname')
  if (!pathname) {
    return new Response('Bad Request', { status: 400 })
  }

  // Get the redirect entry from the redirects.json file
  const redirect = (redirects as Record<string, RedirectEntry>)[pathname]

  // Account for bloom filter false positives
  if (!redirect) {
    return new Response('No redirect', { status: 400 })
  }

  // Return the redirect entry
  return NextResponse.json(redirect)
}
```

```js
// Archivo: app/api/redirects/route.js switcher
import { NextResponse } from 'next/server'
import redirects from '@/app/redirects/redirects.json'

export function GET(request) {
  const pathname = request.nextUrl.searchParams.get('pathname')
  if (!pathname) {
    return new Response('Bad Request', { status: 400 })
  }

  // Get the redirect entry from the redirects.json file
  const redirect = redirects[pathname]

  // Account for bloom filter false positives
  if (!redirect) {
    return new Response('No redirect', { status: 400 })
  }

  // Return the redirect entry
  return NextResponse.json(redirect)
}
```

📘 **Nota para Pages Router:** Then, in the API Route:

```
import type { NextApiRequest, NextApiResponse } from 'next'
import redirects from '@/app/redirects/redirects.json'

type RedirectEntry = {
  destination: string
  permanent: boolean
}

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const pathname = req.query.pathname
  if (!pathname) {
    return res.status(400).json({ message: 'Bad Request' })
  }

  // Get the redirect entry from the redirects.json file
  const redirect = (redirects as Record<string, RedirectEntry>)[pathname]

  // Account for bloom filter false positives
  if (!redirect) {
    return res.status(400).json({ message: 'No redirect' })
  }

  // Return the redirect entry
  return res.json(redirect)
}
```

```
import redirects from '@/app/redirects/redirects.json'

export default function handler(req, res) {
  const pathname = req.query.pathname
  if (!pathname) {
    return res.status(400).json({ message: 'Bad Request' })
  }

  // Get the redirect entry from the redirects.json file
  const redirect = redirects[pathname]

  // Account for bloom filter false positives
  if (!redirect) {
    return res.status(400).json({ message: 'No redirect' })
  }

  // Return the redirect entry
  return res.json(redirect)
}
```

Good to know:

- To generate a bloom filter, you can use a library like `bloom-filters`.
- You should validate requests made to your Route Handler to prevent malicious requests.

Capítulo 3: Server Components, Client Components y Renderizado

React Server Components (RSC), Client Components, patrones de composición, Streaming y Partial Prerendering (PPR).

Server and Client Components

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/server-and-client-components
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how you can use React Server and Client Components to render parts of your application on the server or the client.*

By default, layouts and pages are Server Components, which lets you fetch data and render parts of your UI on the server, optionally cache the result, and stream it to the client. When you need interactivity or browser APIs, you can use Client Components to layer in functionality.

This page explains how Server and Client Components work in Next.js and when to use them, with examples of how to compose them together in your application.

> **Good to know:** For an explanation of where each component type runs and how the boundary works, see The Server and Client Boundary.

When to use Server and Client Components?

The client and server environments have different capabilities. Server and Client Components allow you to run logic in each environment depending on your use case.

Use **Client Components** when you need:

- State and event handlers. E.g. `onClick`, `onChange`.
- Lifecycle logic. E.g. `useEffect`.
- Browser-only APIs. E.g. `localStorage`, `window`, `navigator.geolocation`, etc.
- Custom hooks.

Use **Server Components** when you need:

- Fetch data from databases or APIs close to the source.
- Use API keys, tokens, and other secrets without exposing them to the client.
- Reduce the amount of JavaScript sent to the browser.
- Improve the First Contentful Paint (FCP), and stream content progressively to the client.

For example, the `<Page>` component is a Server Component that fetches data about a post, and passes it as props to the `<LikeButton>` which handles client-side interactivity.

```tsx
// Archivo: app/[id]/page.tsx highlight={1,17} switcher
import LikeButton from '@/app/ui/like-button'
import { getPost } from '@/lib/data'

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const post = await getPost(id)

  return (
    <div>
      <main>
        <h1>{post.title}</h1>

        <LikeButton likes={post.likes} />
      </main>
    </div>
  )
}
```

```jsx
// Archivo: app/[id]/page.js highlight={1,13} switcher
import LikeButton from '@/app/ui/like-button'
import { getPost } from '@/lib/data'

export default async function Page({ params }) {
  const { id } = await params
  const post = await getPost(id)

  return (
    <div>
      <main>
        <h1>{post.title}</h1>

        <LikeButton likes={post.likes} />
      </main>
    </div>
  )
}
```

```tsx
// Archivo: app/ui/like-button.tsx highlight={1} switcher
'use client'

import { useState } from 'react'

export default function LikeButton({ likes }: { likes: number }) {
  // ...
}
```

```jsx
// Archivo: app/ui/like-button.js highlight={1} switcher
'use client'

import { useState } from 'react'

export default function LikeButton({ likes }) {
  // ...
}
```

How do Server and Client Components work in Next.js?

On the server

On the server, Next.js uses React's APIs to orchestrate rendering. The rendering work is split into chunks, by individual route segments (layouts and pages), including parallel route slots whether or not they are displayed:

- **Server Components** are rendered into a special data format called the React Server Component Payload (RSC Payload).
- **Client Components** and the RSC Payload are used to prerender HTML.

> **What is the React Server Component Payload (RSC)?**
>
> The RSC Payload is a compact, serialized representation of the rendered React Server Components tree. It's used by React on the client to update the browser's DOM. The RSC Payload contains:
>
> - The rendered result of Server Components
> - Placeholders for where Client Components should be rendered and references to their JavaScript files
> - Any props passed from a Server Component to a Client Component

On the client (first load)

Then, on the client:

1. **HTML** is used to immediately show a fast non-interactive preview of the route to the user.
2. **RSC Payload** is used to reconcile the Client and Server Component trees.
3. **JavaScript** is used to hydrate Client Components and make the application interactive.

> **What is hydration?**
>
> Hydration is React's process for attaching event handlers to the DOM, to make the static HTML interactive.

Subsequent Navigations

On subsequent navigations:

- The **RSC Payload** is prefetched and cached for instant navigation.
- **Client Components** are rendered entirely on the client, without the server-rendered HTML.

Examples

Using Client Components

You can create a Client Component by adding the `"use client"` directive at the top of the file, above your imports.

```
// Archivo: app/ui/counter.tsx highlight={1} switcher
'use client'

import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>{count} likes</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  )
}
```

```
// Archivo: app/ui/counter.js highlight={1} switcher
'use client'

import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>{count} likes</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  )
}
```

`"use client"` is used to declare a **boundary** between the Server and Client module graphs (trees).

Once a file is marked with `"use client"`, **all of its imports and the components it directly renders are included in the client bundle**. This means you don't need to add the directive to every component that is intended for the client.

This behavior applies to components that are part of the Client Component's module graph, which includes the modules it imports and the components it renders directly. It does not apply to Server Components passed as children or other props. Those components are not imported into the Client Component's module graph. They are rendered on the server and passed to the Client Component as rendered output.

See Interleaving Server and Client Components for how Server and Client Components can be combined.

Reducing JS bundle size

To reduce the size of your client JavaScript bundles, add `'use client'` to specific interactive components instead of marking large parts of your UI as Client Components.

For example, the `<Layout>` component contains mostly static elements like a logo and navigation links, but includes an interactive search bar. `<Search />` needs to be a Client Component, while the rest of the layout can stay a Server Component.

```
// Archivo: app/layout.tsx highlight={12} switcher
// Client Component
import Search from './search'
// Server Component
import Logo from './logo'

// Layout is a Server Component by default
export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <>
      <nav>
        <Logo />
        <Search />
      </nav>
      <main>{children}</main>
    </>
  )
}
```

```
// Archivo: app/layout.js highlight={12} switcher
// Client Component
import Search from './search'
// Server Component
import Logo from './logo'

// Layout is a Server Component by default
export default function Layout({ children }) {
  return (
    <>
      <nav>
        <Logo />
        <Search />
      </nav>
      <main>{children}</main>
    </>
  )
}
```

```
// Archivo: app/ui/search.tsx highlight={1} switcher
'use client'

export default function Search() {
  // ...
}
```

```
// Archivo: app/ui/search.js highlight={1} switcher
'use client'

export default function Search() {
  // ...
}
```

Passing data from Server to Client Components

You can pass data from Server Components to Client Components using props.

```
// Archivo: app/[id]/page.tsx highlight={1,12} switcher
import LikeButton from '@/app/ui/like-button'
import { getPost } from '@/lib/data'

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const post = await getPost(id)

  return <LikeButton likes={post.likes} />
}
```

```
// Archivo: app/[id]/page.js highlight={1,8} switcher
import LikeButton from '@/app/ui/like-button'
import { getPost } from '@/lib/data'

export default async function Page({ params }) {
  const { id } = await params
  const post = await getPost(id)

  return <LikeButton likes={post.likes} />
}
```

```tsx
// Archivo: app/ui/like-button.tsx highlight={1} switcher
'use client'

export default function LikeButton({ likes }: { likes: number }) {
  // ...
}
```

```jsx
// Archivo: app/ui/like-button.js highlight={1} switcher
'use client'

export default function LikeButton({ likes }) {
  // ...
}
```

Alternatively, you can stream data from a Server Component to a Client Component with the `use` API. See an example.

> **Good to know**: Props passed to Client Components need to be serializable by React.

Interleaving Server and Client Components

You can pass Server Components as a prop to a Client Component. This allows you to visually nest server-rendered UI within Client components.

A common pattern is to use `children` to create a *slot* in a `<ClientComponent>`. For example, a `<Cart>` component that fetches data on the server, inside a `<Modal>` component that uses client state to toggle visibility.

```tsx
// Archivo: app/ui/modal.tsx switcher
'use client'

export default function Modal({ children }: { children: React.ReactNode }) {
  return <div>{children}</div>
}
```

```jsx
// Archivo: app/ui/modal.js switcher
'use client'

export default function Modal({ children }) {
  return <div>{children}</div>
}
```

Then, in a parent Server Component (e.g. `<Page>`), you can pass a `<Cart>` as the child of the `<Modal>`:

```tsx
// Archivo: app/page.tsx  highlight={7} switcher
import Modal from './ui/modal'
import Cart from './ui/cart'

export default function Page() {
  return (
    <Modal>
      <Cart />
    </Modal>
  )
}
```

```
// Archivo: app/page.js highlight={7} switcher
import Modal from './ui/modal'
import Cart from './ui/cart'

export default function Page() {
  return (
    <Modal>
      <Cart />
    </Modal>
  )
}
```

In this pattern, Server Components are rendered on the server ahead of time, even when passed as props to Client Components. The React Server Component Payload contains the rendered result of those Server Components, plus placeholders for where Client Components should be rendered and references to their JavaScript files.

Context providers

React context is commonly used to share global state like the current theme. However, React context is not supported in Server Components.

To use context, create a Client Component that accepts `children`:

```
// Archivo: app/theme-provider.tsx switcher
'use client'

import { createContext } from 'react'

export const ThemeContext = createContext({})

export default function ThemeProvider({
  children,
}: {
  children: React.ReactNode
}) {
  return <ThemeContext.Provider value="dark">{children}</ThemeContext.Provider>
}
```

```
// Archivo: app/theme-provider.js switcher
'use client'

import { createContext } from 'react'

export const ThemeContext = createContext({})

export default function ThemeProvider({ children }) {
  return <ThemeContext.Provider value="dark">{children}</ThemeContext.Provider>
}
```

Then, import it into a Server Component (e.g. `layout`):

```tsx
// Archivo: app/layout.tsx switcher
import ThemeProvider from './theme-provider'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html>
      <body>
        <ThemeProvider>{children}</ThemeProvider>
      </body>
    </html>
  )
}
```

```jsx
// Archivo: app/layout.js switcher
import ThemeProvider from './theme-provider'

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <ThemeProvider>{children}</ThemeProvider>
      </body>
    </html>
  )
}
```

Your Server Component will now be able to directly render your provider, and all other Client Components throughout your app will be able to consume this context.

> **Good to know**: You should render providers as deep as possible in the tree – notice how `ThemeProvider` only wraps `{children}` instead of the entire `<html>` document. This makes it easier for Next.js to optimize the static parts of your Server Components.

To pass server-fetched data through context and read it in Client Components with `use()`, see Using React's `use` within a Context Provider.

Third-party components

When using a third-party component that relies on client-only features, you can wrap it in a Client Component to ensure it works as expected.

For example, the `<Carousel />` can be imported from the `acme-carousel` package. This component uses `useState`, but it doesn't yet have the `"use client"` directive.

If you use `<Carousel />` within a Client Component, it will work as expected:

```tsx
// Archivo: app/gallery.tsx switcher
'use client'

import { useState } from 'react'
import { Carousel } from 'acme-carousel'

export default function Gallery() {
  const [isOpen, setIsOpen] = useState(false)

  return (
    <div>
      <button onClick={() => setIsOpen(true)}>View pictures</button>

      {isOpen && <Carousel />}
    </div>
  )
}
```

```
// Archivo: app/gallery.js switcher
'use client'

import { useState } from 'react'
import { Carousel } from 'acme-carousel'

export default function Gallery() {
  const [isOpen, setIsOpen] = useState(false)

  return (
    <div>
      <button onClick={() => setIsOpen(true)}>View pictures</button>

      {isOpen && <Carousel />}
    </div>
  )
}
```

However, if you try to use it directly within a Server Component, you'll see an error. This is because Next.js doesn't know `<Carousel />` is using client-only features.

To fix this, you can wrap third-party components that rely on client-only features in your own Client Components:

```
// Archivo: app/carousel.tsx switcher
'use client'

import { Carousel } from 'acme-carousel'

export default Carousel
```

```
// Archivo: app/carousel.js switcher
'use client'

import { Carousel } from 'acme-carousel'

export default Carousel
```

Now, you can use `<Carousel />` directly within a Server Component:

```
// Archivo: app/page.tsx switcher
import Carousel from './carousel'

export default function Page() {
  return (
    <div>
      <p>View pictures</p>

      <Carousel />
    </div>
  )
}
```

```
// Archivo: app/page.js switcher
import Carousel from './carousel'

export default function Page() {
  return (
    <div>
      <p>View pictures</p>

      <Carousel />
    </div>
  )
}
```

> **Advice for Library Authors**
>
> If you're building a component library, add the `"use client"` directive to entry points that rely on client-only features. This lets your users import components into Server Components without needing to create wrappers.
>
> It's worth noting some bundlers might strip out `"use client"` directives. You can find an example of how to configure esbuild to include the `"use client"` directive in the React Wrap Balancer and Vercel Analytics repositories.

Preventing environment poisoning

JavaScript modules can be shared between both Server and Client Components modules. This means it's possible to accidentally import server-only code into the client. For example, consider the following function:

```
// Archivo: lib/data.ts switcher
export async function getData() {
  const res = await fetch('https://external-service.com/data', {
    headers: {
      authorization: process.env.API_KEY,
    },
  })

  return res.json()
}
```

```
// Archivo: lib/data.js switcher
export async function getData() {
  const res = await fetch('https://external-service.com/data', {
    headers: {
      authorization: process.env.API_KEY,
    },
  })

  return res.json()
}
```

This function contains an `API_KEY` that should never be exposed to the client.

In Next.js, only environment variables prefixed with `NEXT_PUBLIC_` are included in the client bundle. If variables are not prefixed, Next.js replaces them with an empty string.

As a result, even though `getData()` can be imported and executed on the client, it won't work as expected.

To prevent accidental usage in Client Components, you can use the `server-only` package.

Then, import the package into a file that contains server-only code:

```
// Archivo: lib/data.js
import 'server-only'

export async function getData() {
  const res = await fetch('https://external-service.com/data', {
    headers: {
      authorization: process.env.API_KEY,
    },
  })

  return res.json()
}
```

Now, if you try to import the module into a Client Component, there will be a build-time error.

The corresponding `client-only` package can be used to mark modules that contain client-only logic like code that accesses the `window` object.

In Next.js, installing `server-only` or `client-only` is **optional**. However, if your linting rules flag extraneous dependencies, you may install them to avoid issues.

```
# Manejador de paquetes: npm
npm install server-only
```

```
# Manejador de paquetes: yarn
yarn add server-only
```

```
# Manejador de paquetes: pnpm
pnpm add server-only
```

```
# Manejador de paquetes: bun
bun add server-only
```

Next.js handles `server-only` and `client-only` imports internally to provide clearer error messages when a module is used in the wrong environment. The contents of these packages from NPM are not used by Next.js.

Next.js also provides its own type declarations for `server-only` and `client-only`, for TypeScript configurations where `noUncheckedSideEffectImports` is active.

How to lazy load Client Components and libraries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/lazy-loading
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Lazy load imported libraries and React Components to improve your application's loading performance.*

Lazy loading in Next.js helps improve the initial loading performance of an application by decreasing the amount of JavaScript needed to render a route.

It allows you to defer loading of **Client Components** and imported libraries, and only include them in the client bundle when they're needed. For example, you might want to defer loading a modal until a user clicks to open it.

There are two ways you can implement lazy loading in Next.js:

1. Using Dynamic Imports with `next/dynamic`
2. Using `React.lazy()` with Suspense

By default, Server Components are automatically code split, and you can use streaming to progressively send pieces of UI from the server to the client. Lazy loading applies to Client Components.

`next/dynamic`

`next/dynamic` is a composite of `React.lazy()` and Suspense. It behaves the same way in the `app` and `pages` directories to allow for incremental migration.

Examples

Importing Client Components

```
// Archivo: app/page.js
'use client'

import { useState } from 'react'
import dynamic from 'next/dynamic'

// Client Components:
const ComponentA = dynamic(() => import('../components/A'))
const ComponentB = dynamic(() => import('../components/B'))
const ComponentC = dynamic(() => import('../components/C'), { ssr: false })

export default function ClientComponentExample() {
  const [showMore, setShowMore] = useState(false)

  return (
    <div>

      <ComponentA />


      {showMore && <ComponentB />}
      <button onClick={() => setShowMore(!showMore)}>Toggle</button>


      <ComponentC />
    </div>
  )
}
```

> **Note:** When a Server Component dynamically imports a Client Component, automatic code splitting is currently **not** supported.

Skipping SSR

When using `React.lazy()` and Suspense, Client Components will be prerendered (SSR) by default.

> **Note:** `ssr: false` option will only work for Client Components, move it into Client Components ensure the client code-splitting working properly.

If you want to disable prerendering for a Client Component, you can use the `ssr` option set to `false`:

```
const ComponentC = dynamic(() => import('../components/C'), { ssr: false })
```

Importing Server Components

If you dynamically import a Server Component, only the Client Components that are children of the Server Component will be lazy-loaded - not the Server Component itself. It will also help preload the static assets such as CSS when you're using it in Server Components.

```
// Archivo: app/page.js
import dynamic from 'next/dynamic'

// Server Component:
const ServerComponent = dynamic(() => import('../components/ServerComponent'))

export default function ServerComponentExample() {
  return (
    <div>
      <ServerComponent />
    </div>
  )
}
```

> **Note:** `ssr: false` option is not supported in Server Components. You will see an error if you try to use it in Server Components. `ssr: false` is not allowed with `next/dynamic` in Server Components. Please move it into a Client Component.

Loading External Libraries

External libraries can be loaded on demand using the `import()` function. This example uses the external library `fuse.js` for fuzzy search. The module is only loaded on the client after the user types in the search input.

```
// Archivo: app/page.js
'use client'

import { useState } from 'react'

const names = ['Tim', 'Joe', 'Bel', 'Lee']

export default function Page() {
  const [results, setResults] = useState()

  return (
    <div>
      <input
        type="text"
        placeholder="Search"
        onChange={async (e) => {
          const { value } = e.currentTarget
          // Dynamically load fuse.js
          const Fuse = (await import('fuse.js')).default
          const fuse = new Fuse(names)

          setResults(fuse.search(value))
        }}
      />
      <pre>Results: {JSON.stringify(results, null, 2)}</pre>
    </div>
  )
}
```

Adding a custom loading component

```
// Archivo: app/page.js
'use client'

import dynamic from 'next/dynamic'

const WithCustomLoading = dynamic(
  () => import('../components/WithCustomLoading'),
  {
    loading: () => <p>Loading ... </p>,
  }
)

export default function Page() {
  return (
    <div>

      <WithCustomLoading />
    </div>
  )
}
```

Importing Named Exports

To dynamically import a named export, you can return it from the Promise returned by `import()` function:

```
// Archivo: components/hello.js
'use client'

export function Hello() {
  return <p>Hello!</p>
}
```

```
// Archivo: app/page.js
import dynamic from 'next/dynamic'

const ClientComponent = dynamic(() =>
  import('../components/hello').then((mod) => mod.Hello)
)
```

Magic Comments

Next.js supports magic comments to control how dynamic imports are handled by the bundler. These comments work with dynamic `import()`, `require()`, `require.resolve()`, and `new Worker()` expressions.

> **Good to know:** Magic comments do not work with static `import` statements (`import x from 'y'`). They only work with dynamic expressions.

`webpackIgnore` / `turbopackIgnore`

Use these comments to skip bundling a dynamic import. The import expression will be left as-is in the output, useful for runtime-only modules:

```
// Skip bundling - import happens at runtime
const runtime = await import(/* webpackIgnore: true */ 'runtime-module')

// Turbopack-specific variant
const plugin = await import(/* turbopackIgnore: true */ pluginPath)

// Also works with require
const mod = require(/* webpackIgnore: true */ 'runtime-module')
```

`turbopackOptional` (Turbopack only)

Use this comment to suppress build errors when a module might not exist. The import will still throw at runtime if the module is missing:

```
// No build error if './optional-feature' doesn't exist
// Runtime will throw MODULE_NOT_FOUND if executed
const feature = await import(/* turbopackOptional: true */ './optional-feature')

// Also works with require
const mod = require(/* turbopackOptional: true */ './optional-module')
```

This is useful for:

- Conditional features that may not be installed
- Plugin systems where modules are optional
- Gradual migrations where some files may not exist yet

> **Good to know:** `webpackOptional` is not supported. Use `turbopackOptional` instead when using Turbopack.

📘 **Nota para Pages Router:**

`next/dynamic`

`next/dynamic` is a composite of `React.lazy()` and Suspense. It behaves the same way in the `app` and `pages` directories to allow for incremental migration.

In the example below, by using `next/dynamic`, the header component will not be included in the page's initial JavaScript bundle. The page will render the Suspense `fallback` first, followed by the `Header` component when the `Suspense` boundary is resolved.

```
import dynamic from 'next/dynamic'

const DynamicHeader = dynamic(() => import('../components/header'), {
  loading: () => <p>Loading ...</p>,
})

export default function Home() {
  return <DynamicHeader />
}
```

> **Good to know**: In `import('path/to/component')`, the path must be explicitly written. It can't be a template string nor a variable. Furthermore the `import()` has to be inside the `dynamic()` call for Next.js to be able to match webpack bundles / module ids to the specific `dynamic()` call and preload them before rendering. `dynamic()` can't be used inside of React rendering as it needs to be marked in the top level of the module for preloading to work, similar to `React.lazy`.

Examples

With named exports

To dynamically import a named export, you can return it from the Promise returned by `import()`:

```
export function Hello() {
  return <p>Hello!</p>
}

// pages/index.js
import dynamic from 'next/dynamic'

const DynamicComponent = dynamic(() =>
  import('../components/hello').then((mod) => mod.Hello)
)
```

With no SSR

To dynamically load a component on the client side, you can use the `ssr` option to disable server-rendering. This is useful if an external dependency or component relies on browser APIs like `window`.

```
'use client'

import dynamic from 'next/dynamic'

const DynamicHeader = dynamic(() => import('../components/header'), {
  ssr: false,
})
```

With external libraries

This example uses the external library `fuse.js` for fuzzy search. The module is only loaded in the browser after the user types in the search input.

```
import { useState } from 'react'

const names = ['Tim', 'Joe', 'Bel', 'Lee']

export default function Page() {
  const [results, setResults] = useState()

  return (
    <div>
      <input
        type="text"
        placeholder="Search"
        onChange={async (e) => {
          const { value } = e.currentTarget
          // Dynamically load fuse.js
          const Fuse = (await import('fuse.js')).default
          const fuse = new Fuse(names)

          setResults(fuse.search(value))
        }}
      />
      <pre>Results: {JSON.stringify(results, null, 2)}</pre>
    </div>
  )
}
```

Streaming

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/streaming
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how streaming works in Next.js and how to use it to progressively render UI as data becomes available.*

What is streaming?

In traditional server-side rendering, the server produces the full HTML document before sending anything. A single slow database query or API call can block the entire page. Streaming changes this by using chunked transfer encoding to send parts of the response as they become ready. The browser starts rendering HTML while the server is still generating the rest.

This is especially impactful for pages that combine fast static content (headers, navigation, layout) with slower dynamic content (personalized data, analytics, recommendations). The static parts can be prerendered and served from a CDN, painting instantly, while the dynamic parts stream in from the server as they become ready.

React's server renderer produces HTML in chunks aligned with ``` `</div>` ```) }

```jsx
// Archivo: app/dashboard/page.js switcher
import { Suspense } from 'react'
import { Revenue } from './revenue'
import { RecentOrders } from './recent-orders'
import { Recommendations } from './recommendations'

export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <div className="grid grid-cols-2 gap-4">
        ``<Suspense>``
        ``<Suspense>``
      </div>
      ``<Suspense>``
    </div>
  )
}
```

In this example, if `Revenue` resolves in 200ms, `RecentOrders` in 1s, and `Recommendations` in 3s, the user sees each section appear as soon as its data is ready.

Nested boundaries for progressive detail

You can nest ````` `<Suspense>`) }

```jsx
// Archivo: app/product/[id]/page.js switcher
import { Suspense } from 'react'
import { ProductDetails } from './product-details'
import { Reviews } from './reviews'

export async function generateStaticParams() {
  const products = await getTopProducts()
  return products.map((product) => ({ id: product.id }))
}

export default async function ProductPage({ params }) {
  const { id } = await params

  return (
    <div>
      <h1>Product</h1>
      ``<Suspense>``
      `<Suspense>`
    </div>
  )
}
```

The outer boundary shows "Loading product details..." until `ProductDetails` resolves. Once it does, the inner boundary becomes visible, showing "Loading reviews..." until `Reviews` resolves. This creates a progressive reveal.

Push dynamic access down

The key to maximizing what streams instantly is to defer dynamic data access to the component that actually needs it. This applies to `params`, `searchParams`, `cookies()`, `headers()`, and data fetches. If you `await` any of these at the top of a layout or page, everything below that point becomes dynamic and cannot be prerendered as part of the static shell.

Instead, pass the promise down and let the consuming component resolve it inside a ````` {children}) }

```jsx
// Archivo: app/dashboard/layout.js switcher
import { Suspense } from 'react'
import { Nav } from './nav'
import { UserMenu } from './user-menu'
import { cookies } from 'next/headers'

export default function DashboardLayout({ children }) {
  const cookieStore = cookies() // Start the work, but don't await

  return (
    <div>
      <Nav>
        ``<Suspense>``
      </Nav>
      {children}
    </div>
  )
}
```

In this example, `<Nav>` and `{children}` render as part of the static shell because nothing in the layout awaits. Only `<UserMenu>` suspends when it resolves the cookie promise. If the layout had called `await cookies()` at the top instead, the entire layout and all its children would be blocked from prerendering.

The same principle applies to `params` and `searchParams`. Rather than destructuring them at the page level, pass the promise to the component that needs the value:

```tsx
// Archivo: app/shop/[category]/page.tsx switcher
import { Suspense } from 'react'
import { Hero } from './hero'
import { ProductGrid } from './product-grid'

export async function generateStaticParams() {
  const categories = await getCategories()
  return categories.map((c) => ({ category: c.slug }))
}

export default function ShopPage({
  params,
}: {
  params: Promise<{ category: string }>
}) {
  return (
    <div>
      <Hero />
      ``<Suspense>``
    </div>
  )
}
```

```jsx
// Archivo: app/shop/[category]/page.js switcher
import { Suspense } from 'react'
import { Hero } from './hero'
import { ProductGrid } from './product-grid'

export async function generateStaticParams() {
  const categories = await getCategories()
  return categories.map((c) => ({ category: c.slug }))
}

export default function ShopPage({ params }) {
  return (
    <div>
      <Hero />
      ``<Suspense>``
    </div>
  )
}
```

`<Hero />` paints as part of the static shell. `<ProductGrid>` resolves `params` when it needs the category, suspending only within its boundary.

You can also unwrap the promise inline with `.then()`, so the child component receives a plain value instead of a promise:

```
// Archivo: app/shop/[category]/page.tsx switcher
``<Suspense>``
```

```
// Archivo: app/shop/[category]/page.js switcher
``<Suspense>``
```

This keeps `ProductGrid` simple (it takes a `string`, not a `Promise`) while still deferring the `params` access to inside the Suspense boundary.

When to use `loading.js` vs `````

) }

````
```jsx
// Archivo: app/dashboard/page.js switcher
import { Suspense } from 'react'
import { StatsChart } from './stats-chart'

async function getStats() {
  const res = await fetch('https://api.example.com/stats')
  return res.json()
}

export default function Dashboard() {
  // Start the fetch during server render, don't await it
  const statsPromise = getStats()

  return (
    ``<Suspense>``
  )
}
````

```
// Archivo: app/dashboard/stats-chart.tsx switcher
'use client'

import { use } from 'react'

type Stats = { revenue: number; orders: number }

export function StatsChart({ dataPromise }: { dataPromise: Promise<Stats> }) {
  const stats = use(dataPromise)

  return <div></div>
}
```

```
// Archivo: app/dashboard/stats-chart.js switcher
'use client'

import { use } from 'react'

export function StatsChart({ dataPromise }) {
  const stats = use(dataPromise)

  return <div></div>
}
```

The fallback is sent immediately with the static shell. When the promise resolves, React streams the completed HTML into the page.
```

Sharing a promise across the tree

When multiple components need the same data, start the fetch once and pass the promise through a context provider so any component in the subtree can resolve it with `use()`:

```
// Archivo: app/layout.tsx
import { getUser } from '@/lib/data'
// Stores the promise in React context for the subtree
import { UserProvider } from './user-provider'

export default function Layout({ children }: { children: React.ReactNode }) {
  const userPromise = getUser()

  return <UserProvider userPromise={userPromise}>{children}</UserProvider>
}
```

See Using React's `use` within a Context Provider for the full pattern.

Streaming in Route Handlers

The patterns above rely on React and Suspense to stream UI. Outside of React rendering, Route Handlers can stream raw responses using the Web Streams API. This is useful for Server-Sent Events, large file generation, or any response where you want data to arrive progressively:

```
// Archivo: app/api/stream/route.ts switcher
export async function GET() {
  const encoder = new TextEncoder()

  const stream = new ReadableStream({
    async start(controller) {
      for (let i = 0; i < 10; i++) {
        controller.enqueue(encoder.encode(`Chunk ${i + 1}\n`))
        await new Promise((resolve) => setTimeout(resolve, 200))
      }
      controller.close()
    },
  })

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/plain; charset=utf-8',
      'X-Content-Type-Options': 'nosniff',
    },
  })
}
```

```
// Archivo: app/api/stream/route.js switcher
export async function GET() {
  const encoder = new TextEncoder()

  const stream = new ReadableStream({
    async start(controller) {
      for (let i = 0; i < 10; i++) {
        controller.enqueue(encoder.encode(`Chunk ${i + 1}\n`))
        await new Promise((resolve) => setTimeout(resolve, 200))
      }
      controller.close()
    },
  })

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/plain; charset=utf-8',
      'X-Content-Type-Options': 'nosniff',
    },
  })
}
```

Visit this route directly in the browser or with `curl` to see chunks arrive one at a time:

```
curl http://localhost:3000/api/stream
```

You can also stream files without loading them entirely into memory. Use `FileHandle.readableWebStream()` to get a Web `ReadableStream` directly from a file:

```
// Archivo: app/api/download/route.ts switcher
import { open } from 'node:fs/promises'

export async function GET() {
  const file = await open('/path/to/large-file.csv')

  return new Response(file.readableWebStream(), {
    headers: {
      'Content-Type': 'text/csv',
      'Content-Disposition': 'attachment; filename="data.csv"',
    },
  })
}
```

```
// Archivo: app/api/download/route.js switcher
import { open } from 'node:fs/promises'

export async function GET() {
  const file = await open('/path/to/large-file.csv')

  return new Response(file.readableWebStream(), {
    headers: {
      'Content-Type': 'text/csv',
      'Content-Disposition': 'attachment; filename="data.csv"',
    },
  })
}
```

See the Route Handler API reference for more details on building streaming endpoints.

Streaming and Web Vitals

Web Vitals are the metrics Google uses to measure user experience. Streaming directly affects several of them.

TTFB and FCP

Without streaming, the server waits for all data before sending any HTML, so TTFB equals the slowest query. With streaming, the server sends the static shell as soon as it's ready. TTFB drops to the time it takes to render your layouts and fallbacks. The browser paints the static shell immediately, so FCP is decoupled from your data fetching time.

LCP (Largest Contentful Paint)

If your LCP element (a hero image, a main heading, a product photo) is inside a Suspense boundary, it can't paint until that boundary's content is swapped in. The element then depends on the work the server does to render it, not on your initial server response time. Revealing it costs something on the client too, because React streams a small inline script alongside the boundary's HTML and the content only appears once that script runs.

Data fetching is not the only reason a boundary delays your LCP element. React also holds back a large boundary, because sending its HTML takes time. See what activates a Suspense boundary.

> **Good to know:** As a rule of thumb, if there's a Suspense boundary, React might use it. Under a slow network or a busy CPU, concurrent rendering can fall back to it even when you didn't expect it. Adding a boundary means accepting that, so don't add one you don't need.

To keep LCP fast:

- Keep LCP elements **outside** or **above** Suspense boundaries so they render as part of the static shell.

- Use the `preload` prop on `next/image` for LCP images. This injects a `<link rel="preload">` into the `<head>`, so the browser starts fetching the image from the very first chunk, before the `<img>` tag even appears in the HTML. It controls when the image is fetched, not when it paints. An image inside a boundary still waits for the swap.
- For non-image LCP elements (text, headings), render them outside Suspense boundaries.

CLS (Cumulative Layout Shift)

When a Suspense fallback is replaced by the resolved content, the browser reflows the page. If the fallback and the resolved content are different sizes, the surrounding layout shifts. To minimize CLS:

- Design skeleton fallbacks that **match the dimensions** of the content they represent. A skeleton with the same height and width as the final card grid prevents shifts.
- Use fixed or min-height containers around Suspense boundaries so the space is reserved before content arrives.

INP (Interaction to Next Paint)

Streaming enables selective hydration: React hydrates components independently as they stream in, and prioritizes hydrating whatever the user is interacting with. Each `````) }

```
```jsx
// Archivo: app/post/[slug]/page.js switcher
import { Suspense } from 'react'
import { notFound } from 'next/navigation'
import { PostContent } from './post-content'

export async function generateStaticParams() {
  const posts = await getPublishedPosts()
  return posts.map((post) => ({ slug: post.slug }))
}

export default async function PostPage({ params }) {
  const { slug } = await params
  const exists = await checkSlugExists(slug) // Fast existence check
  if (!exists) notFound() // Real 404, before any Suspense boundary

  return (
    ``<Suspense>``
  )
}
```

> **Good to know:** You can also reject requests early using `proxy` (for redirects, rewrites, or returning a response) or `next.config.js` redirects. Both run before the page renders, so HTTP status codes are still available.

Bots and crawlers

HTML-limited bots and crawlers need metadata to be available in the `<head>` of the initial HTML. Next.js detects them by their user agent and waits for `generateMetadata` to resolve before streaming the page content. Full browsers and DOM-capable crawlers can instead receive streaming metadata alongside the page content.

You can customize which bots receive blocking metadata with the `htmlLimitedBots` configuration option. See the `loading.js` SEO section for more details.

Cache Components

With Cache Components, visitors and DOM-capable crawlers receive the prerendered shell immediately, and dynamic content streams in as it resolves. HTML-limited bots skip the prerendered shell and render the page dynamically so metadata can be placed in the `<head>`. Once the metadata resolves, the remaining page content can still stream.

Keep this in mind when your prerendered shell depends on inputs that only exist while prerendering, such as build-time data or values that are not reachable in the request-time environment. Visitors and DOM-capable crawlers receive the shell without re-running that code, but an HTML-limited bot re-renders it dynamically, so a page that loads for a person can fail to render for a crawler. Make sure any data the shell relies on is also available at request time.
```

What can affect streaming

Any layer between your server and the client that buffers the response can diminish the benefits of streaming. The HTML may be fully generated progressively on the server, but if a proxy, CDN, or even the client itself collects all the chunks before rendering them, the user sees a single delayed response instead of progressive rendering.

Reverse proxies

Nginx and similar reverse proxies buffer responses by default. Disable buffering by setting the `X-Accel-Buffering` header to `no`:

```
// Archivo: next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/:path*{/}?',
        headers: [
          {
            key: 'X-Accel-Buffering',
            value: 'no',
          },
        ],
      },
    ]
  },
}
```

CDNs

Content Delivery Networks may buffer entire responses before forwarding them to the client. Check your CDN provider's documentation for streaming support. Some require specific configuration or plan tiers to pass through chunked responses.

Serverless platforms

Not all serverless environments support streaming. AWS Lambda, for example, requires response streaming mode to be explicitly enabled (it is not the default). Vercel supports streaming natively.

Compression

Gzip and Brotli compression can buffer chunks internally before flushing, as the compression algorithm needs enough data to compress efficiently. This can add latency to the first visible chunk. If you notice streaming delays, check whether your compression layer is flushing aggressively enough.

Clients

Buffering also happens at the client. Safari/WebKit buffers streaming responses until 1024 bytes have been received, so very small responses paint all at once instead of progressively. Real applications easily exceed this threshold (layouts, styles, scripts), so it only affects minimal demos or tiny Route Handler responses.

Command-line tools like `curl` also buffer by default. The `-N` flag disables output buffering, but `curl` still relies on newline characters to flush lines to the terminal. A stream that sends chunks without newlines may appear to stall even with `-N`.

Verifying that streaming works

This section is about confirming the HTTP response is actually arriving in chunks through your infrastructure. For guidance on designing meaningful loading states and placing Suspense boundaries effectively, see Granular streaming with `<Suspense>` and the Cache Components guide.

Check the Network tab. In Chrome DevTools, select the document request and look at the "Timing" breakdown. A long "Content Download" phase with an early "Time to First Byte" confirms the response is streaming rather than arriving all at once.

Observe raw chunks. To see exactly what the server sends and when, use a small script that reads the response as a stream. This is more reliable than `curl` for observing timed chunks, since `curl` has its own buffering behavior:

```
// Archivo: stream-observer.mjs
const res = await fetch(
  'https://streaming-demo.labs.vercel.dev/suspense-demo',
  {
    headers: { 'Accept-Encoding': 'identity' },
  }
)

const reader = res.body.getReader()
const decoder = new TextDecoder()
let i = 0
const start = Date.now()

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  console.log(`\nchunk ${i++} (+${Date.now() - start}ms)\n`)
  console.log(decoder.decode(value))
}
```

Run with `node stream-observer.mjs`. For a page with two sibling Suspense boundaries (like the companion demo's Suspense page), you will see output similar to:

```
// Archivo: Terminal
chunk 0 (+0ms)    # Static shell: <head>, CSS, nav, fallback skeletons,
                  # <template id="B:0"> and <template id="B:1"> placeholders,
                  # bootstrap scripts
chunk 1 (+170ms)  # Component payload (self.__next_f.push) for hydration
chunk 2 (+1000ms) # Weather widget: payload + <div hidden id="S:0"> (swaps B:0)
chunk 3 (+3000ms) # Analytics dashboard: payload + <div hidden id="S:1"> (swaps B:1)
```

The `<template id="B:0">` markers are the Suspense fallback placeholders. When a boundary resolves, React streams a `<div hidden id="S:0">` containing the completed HTML and a script that swaps it into the page. The timestamps show each boundary resolving independently.

> **Good to know:** The `Accept-Encoding: identity` header disables compression so chunks are not buffered by the compression layer.

Compare a bot request. Add a bot user agent to the same script with `headers: { 'User-Agent': 'Twitterbot/1.0', 'Accept-Encoding': 'identity' }`. Now `await fetch()` itself blocks until the full render completes (around 3 seconds for this page), because the server holds the response until it has the finished document. The body then arrives all at once, with none of the staggered `+1000ms` / `+3000ms` timestamps:

```
// Archivo: Terminal
chunk 0 (+0ms) # Entire document in a single burst, after fetch() already waited
```

This is the bots and crawlers behavior: the server waits for the full render and sends one fully formed HTML document instead of streaming.

Platform support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

See the Self-Hosting guide for detailed configuration instructions.

Summary

The trigger is **your code**: async work, non-deterministic output, or runtime data. When the framework encounters these, it walks up the tree looking for a `<Suspense>` boundary to use as a fallback. Everything above those boundaries forms the static shell, which is sent immediately. As each boundary resolves, React streams the result into the page.

The key decisions are **what to cache** and **where to place Suspense boundaries**. Cache what you can with `"use cache"` to grow the static shell. Push dynamic access down to the components that need it, and wrap those in `<Suspense>`. Everything else becomes part of the shell.

Further reading

- RSC Explorer - interactive tool to explore the component payload format and see how React reconstructs the tree from streamed chunks
- Streams API on web.dev - introduction to the Web Streams API that underpins streaming in Route Handlers
- Chunked transfer encoding (MDN) - the HTTP/1.1 mechanism that enables streaming responses
- browser.engineering - deep dive into how browsers handle network responses, rendering, and progressive display
- Preventing flash before hydration - how to update server-rendered HTML with client-specific values (locale, theme, persisted state) before the browser paints

---

How to lazy load Client Components and libraries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/lazy-loading
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Lazy load imported libraries and React Components to improve your application's overall loading performance.*

Capítulo 4: Data Fetching, Caching Moderno y Revalidación

> *El nuevo modelo de caché de Next.js 16: Cache Components, fetch caching, revalidación bajo demanda y CDN caching.*

Fetching Data

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/fetching-data
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to fetch data and stream content that depends on data.*

This page will walk you through how you can fetch data in Server and Client Components, and how to stream components that depend on uncached data.

Fetching data

Server Components

You can fetch data in Server Components using any asynchronous I/O, such as:

1. The `fetch` API
2. An ORM or database

With the `fetch` API

To fetch data with the `fetch` API, turn your component into an asynchronous function, and await the `fetch` call. For example:

```
// Archivo: app/blog/page.tsx switcher
export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/blog/page.js switcher
export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

Good to know:

- Identical `fetch` requests in a React component tree are memoized by default, so you can fetch data in the component that needs it instead of drilling props.
- `fetch` requests are not cached by default and will block the page from rendering until the request is complete. Use the `use cache` directive to cache results, or wrap the fetching component in [`````) }

```jsx
// Archivo: app/blog/page.js switcher
import { Suspense } from 'react'
import BlogList from '@/components/BlogList'
import BlogListSkeleton from '@/components/BlogListSkeleton'

export default function BlogPage() {
  return (
    <div>

      <header>
        <h1>Welcome to the Blog</h1>
        <p>Read the latest posts below.</p>
      </header>
      <main>

        ``<Suspense>``}>
          <BlogList />
        `<Suspense>`
      </main>
    </div>
  )
}
```

Creating meaningful loading states

An instant loading state is fallback UI that is shown immediately to the user after navigation. For the best user experience, we recommend designing loading states that are meaningful and help users understand the app is responding. For example, you can use skeletons and spinners, or a small but meaningful part of future screens such as a cover photo, title, etc.

In development, you can preview and inspect the loading state of your components using the React Devtools.

Client Components

There are two ways to fetch data in Client Components, using:

1. React's `use` API
2. A community library like SWR or React Query

Streaming data with the `use` API

You can use React's `use` API to stream data from the server to client. Start by fetching data in your Server component, and pass the promise to your Client Component as prop:

```
// Archivo: app/blog/page.tsx switcher
import Posts from '@/app/ui/posts'
import { Suspense } from 'react'

export default function Page() {
  // Don't await the data fetching function
  const posts = getPosts()

  return (
    ``<Suspense>``
  )
}
```

```
// Archivo: app/blog/page.js switcher
import Posts from '@/app/ui/posts'
import { Suspense } from 'react'

export default function Page() {
  // Don't await the data fetching function
  const posts = getPosts()

  return (
    ``<Suspense>``
  )
}
```

Then, in your Client Component, use the `use` API to read the promise:

```
// Archivo: app/ui/posts.tsx switcher
'use client'
import { use } from 'react'

export default function Posts({
  posts,
}: {
  posts: Promise<{ id: string; title: string }[]>
}) {
  const allPosts = use(posts)

  return (
    <ul>
      {allPosts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/ui/posts.js switcher
'use client'
import { use } from 'react'

export default function Posts({ posts }) {
  const allPosts = use(posts)

  return (
    <ul>
      {allPosts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

In the example above, the `<Posts>` component is wrapped in a [````` </>) }

async function Playlists({ artistID }: { artistID: string }) { // Use the artist ID to fetch playlists const playlists = await getArtistPlaylists(artistID)

return (

{playlists.map((playlist) => (

- {playlist.name}

))}

) }

```jsx
// Archivo: app/artist/[username]/page.js switcher
export default async function Page({ params }) {
  const { username } = await params
  // Get artist information
  const artist = await getArtist(username)

  return (
    <>
      <h1>{artist.name}</h1>

      ``<Suspense>``
    </>
  )
}

async function Playlists({ artistID }) {
  // Use the artist ID to fetch playlists
  const playlists = await getArtistPlaylists(artistID)

  return (
    <ul>
      {playlists.map((playlist) => (
        <li key={playlist.id}>{playlist.name}</li>
      ))}
    </ul>
  )
}
```

In this example, `<Suspense>` allows the playlists to stream in after the artist data loads. However, the page still waits for the artist data before displaying anything. To prevent this, you can wrap the entire page component in a `<Suspense>` boundary (for example, using a `loading.js` file) to show a loading state immediately.

Ensure your data source can resolve the first request quickly, as it blocks everything else. If you can't optimize the request further, consider caching the result if the data changes infrequently.

Parallel data fetching

Parallel data fetching happens when data requests in a route are eagerly initiated and start at the same time.

By default, layouts and pages are rendered in parallel. So each segment starts fetching data as soon as possible.

However, within *any* component, multiple `async` / `await` requests can still be sequential if placed after the other. For example, `getAlbums` will be blocked until `getArtist` is resolved:

```tsx
// Archivo: app/artist/[username]/page.tsx switcher
import { getArtist, getAlbums } from '@/app/lib/data'

export default async function Page({ params }) {
  // These requests will be sequential
  const { username } = await params
  const artist = await getArtist(username)
  const albums = await getAlbums(username)
  return <div>{artist.name}</div>
}
```

Start multiple requests by calling `fetch`, then await them with `Promise.all`. Requests begin as soon as `fetch` is called.

```tsx
// Archivo: app/artist/[username]/page.tsx highlight={3,8,24} switcher
import Albums from './albums'

async function getArtist(username: string) {
  const res = await fetch(`https://api.example.com/artist/${username}`)
  return res.json()
}

async function getAlbums(username: string) {
  const res = await fetch(`https://api.example.com/artist/${username}/albums`)
  return res.json()
}

export default async function Page({
  params,
}: {
  params: Promise<{ username: string }>
}) {
  const { username } = await params

  // Initiate requests
  const artistData = getArtist(username)
  const albumsData = getAlbums(username)

  const [artist, albums] = await Promise.all([artistData, albumsData])

  return (
    <>
      <h1>{artist.name}</h1>
      <Albums list={albums} />
    </>
  )
}
```

```jsx
// Archivo: app/artist/[username]/page.js highlight={3,8,20} switcher
import Albums from './albums'

async function getArtist(username) {
  const res = await fetch(`https://api.example.com/artist/${username}`)
  return res.json()
}

async function getAlbums(username) {
  const res = await fetch(`https://api.example.com/artist/${username}/albums`)
  return res.json()
}

export default async function Page({ params }) {
  const { username } = await params

  // Initiate requests
  const artistData = getArtist(username)
  const albumsData = getAlbums(username)

  const [artist, albums] = await Promise.all([artistData, albumsData])

  return (
    <>
      <h1>{artist.name}</h1>
      <Albums list={albums} />
    </>
  )
}
```

> **Good to know:** If one request fails when using `Promise.all`, the entire operation will fail. To handle this, you can use the `Promise.allSettled` method instead.

Reusing data with `React.cache`

For data access that does not use `fetch`, such as an ORM or database query, wrap the function in `React.cache`. Multiple components can then call the function within the same request while sharing one result:

```
// Archivo: app/lib/user.ts switcher
import { cache } from 'react'
import { db, eq, users } from '@/lib/db'

export const getUser = cache(async (id: string) => {
  return db.query.users.findFirst({
    where: eq(users.id, id),
  })
})
```

```
// Archivo: app/lib/user.js switcher
import { cache } from 'react'
import { db, eq, users } from '@/lib/db'

export const getUser = cache(async (id) => {
  return db.query.users.findFirst({
    where: eq(users.id, id),
  })
})
```

Server Components can call `getUser()` directly:

```
// Archivo: app/dashboard/page.tsx switcher
import { getUser } from '../lib/user'

export default async function DashboardPage() {
  const user = await getUser('1')

  if (!user) {
    return null
  }

  return <h1>Dashboard for {user.name}</h1>
}
```

```
// Archivo: app/dashboard/page.js switcher
import { getUser } from '../lib/user'

export default async function DashboardPage() {
  const user = await getUser('1')

  if (!user) {
    return null
  }

  return <h1>Dashboard for {user.name}</h1>
}
```

Since `getUser` is wrapped with `React.cache`, calls with the same `id` within one request return the same memoized result.

> **Good to know:** `React.cache` is scoped to the current request only. Each request gets its own memoization scope with no sharing between requests.

Preloading data

When a component renders after other blocking work, its data request starts late even if the request inputs are already available. Preloading starts the request earlier so it can run in parallel with that work and avoid a request waterfall.

To preload data, call the data-fetching function without `await` before blocking work, then call the same function in the component that consumes the result.

The data-fetching function must deduplicate matching calls so the component can reuse the request started during preloading. Use one of the following approaches:

- For `fetch`, identical requests are memoized automatically.
- For an ORM or database, wrap the data-fetching function in `React.cache`.
- With Cache Components, add `'use cache'` to the data-fetching function. If the function reads request APIs such as `cookies()` or `headers()`, use `'use cache: private'`.

In production, matching calls to a private Cache Function can reuse the same result within one request. This lets a component reuse a request started during preloading without storing the result in a server cache across requests.

Keep the preload function next to the component that consumes the data. This makes the dependency easier to find if you move or remove the component:

```
// Archivo: app/item/[id]/item.tsx switcher
async function getItem(id: string) {
  const res = await fetch(`https://api.example.com/items/${id}`)
  return res.json()
}

export const preload = (id: string) => {
  void getItem(id)
}

export default async function Item({ id }: { id: string }) {
  const item = await getItem(id)
  return <div>{item.name}</div>
}
```

```
// Archivo: app/item/[id]/item.js switcher
async function getItem(id) {
  const res = await fetch(`https://api.example.com/items/${id}`)
  return res.json()
}

export const preload = (id) => {
  void getItem(id)
}

export default async function Item({ id }) {
  const item = await getItem(id)
  return <div>{item.name}</div>
}
```

Call `preload()` before another blocking request to start loading the item earlier:

```
// Archivo: app/item/[id]/page.tsx switcher
import Item, { preload } from './item'
import { checkIsAvailable } from '@/app/lib/data'

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params

  preload(id)
  const isAvailable = await checkIsAvailable(id)

  return isAvailable ? ``<Item>`` : null
}
```

```
// Archivo: app/item/[id]/page.js switcher
import Item, { preload } from './item'
import { checkIsAvailable } from '@/app/lib/data'

export default async function Page({ params }) {
  const { id } = await params

  preload(id)
  const isAvailable = await checkIsAvailable(id)

  return isAvailable ? ``<Item>`` : null
}
```

The item request continues while `checkIsAvailable()` runs. If the page renders `<Item>`, the identical `fetch` call reuses the request that `preload()` started.

Caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/caching
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to cache data and UI in Next.js*

> This page covers caching with Cache Components, enabled by setting `cacheComponents: true` in your `next.config.ts` file. If you're not using Cache Components, see the Caching and Revalidating (Previous Model) guide.

Caching is a technique for storing the result of data fetching and other computations so that future requests for the same data can be served faster, without doing the work again.

Enabling Cache Components

You can enable Cache Components by adding the `cacheComponents` option to your Next config file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

> **Good to know:** When Cache Components is enabled, `GET` Route Handlers follow the same prerendering model as pages. See Route Handlers with Cache Components for details.

Usage

The `use cache` directive caches the return value of async functions and components. You can apply it at two levels:

- **Data-level**: Cache a function that fetches or computes data (e.g., `getProducts()`, `getUser(id)`)
- **UI-level**: Cache an entire component or page (e.g., `async function BlogPosts()`)

A cache directive gives a result a lifetime, information Next.js uses to apply rendering optimizations. See Prerendering for how cached results become part of the static shell and may be included in a prefetch.

> **Good to know:** We recommend pairing every cache directive with a `cacheLife`. Without one, the implicit `default` profile applies.

Arguments and any values captured from parent scopes automatically become part of the cache key, which means different inputs will produce separate cache entries. See cache output for what an entry holds, and serialization requirements and constraints for details on what can be cached and how arguments work.

Data-level caching

To cache an asynchronous function that fetches data, add the `use cache` directive at the top of the function body:

```
// Archivo: app/lib/data.ts highlight={1,4,5}
import { cacheLife } from 'next/cache'

export async function getUsers() {
  'use cache'
  cacheLife('hours')
  return db.query('SELECT * FROM users')
}
```

Data-level caching is useful when the same data is used across multiple components, or when you want to cache the data independently from the UI.

UI-level caching

To cache an entire component, page, or layout, add the `use cache` directive at the top of the component or page body:

```
// Archivo: app/page.tsx highlight={1,4,5}
import { cacheLife } from 'next/cache'

export default async function Page() {
  'use cache'
  cacheLife('hours')

  const users = await db.query('SELECT * FROM users')

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  )
}
```

> If you add "`use cache`" at the top of a file, all exported functions in the file will be cached.

Streaming uncached data

For components that fetch data from an asynchronous source such as an API, a database, or any other async operation, and require fresh data on every request, do not use `"use cache"`.

Instead, wrap the component in [````` </>) }

```
For example, `<p>Loading posts ... </p>` is included in the static shell, and the posts stream in at request time.

Without a ```<Suspense>```', highlight: true },
    ]}
  />


> **Good to know:** Each fix card links to a detailed walkthrough with patterns, code samples, and trade-offs. Click a card
to dive in.


```<Suspense>``
    </>
)
}
```

A runtime API access without `````', highlight: true }, ]} />

Runtime-dependent data can still be given a cache lifetime with `"use cache: private"`, another variant that ships with Cache Components. It gives a lifetime to a function that reads cookies, headers, or `searchParams` directly, so it can be included in a prefetch.

The following section shows an alternative to `use cache: private`: extracting a runtime value and passing it to a shared cached function.

### Passing runtime values to cached functions

You can extract values from runtime APIs and pass them as arguments to cached functions:

```
// Archivo: app/profile/page.tsx
import { cookies } from 'next/headers'
import { Suspense } from 'react'

export default function Page() {
  return (
    ``<Suspense>``
  )
}

// Component (not cached) reads runtime data
async function ProfileContent() {
  const session = (await cookies()).get('session')?.value
  return <CachedContent sessionId={session} />
}

// Cached component receives extracted value as a prop
async function CachedContent({ sessionId }: { sessionId: string }) {
  'use cache'
  // sessionId becomes part of the cache key
  const data = await fetchUserData(sessionId)
  return <div>{data}</div>
}
```

At request time, `<CachedContent />` executes if no matching cache entry is found, and stores the result for future requests with the same `sessionId`.

> **Good to know:** Because `<CachedContent />` is gated behind request data, it isn't added to the prerendered static shell. At runtime it's cached in-memory by default, which doesn't persist across serverless requests, so it may re-evaluate on each request. Reach for `'use cache: remote'` for durable, shared caching.

With this pattern, prefetching can prerender `<CachedContent />` with the user's actual session during a client transition and have the result ready before the click. This works even when server-side entries rarely survive between requests, because the lifetime you assign is what lets the result join the prefetch, where the client treats it as fresh for its `cacheLife` `stale` window.
```

Static, cached, and streaming

Here's a complete example showing static content, cached dynamic content, and streaming dynamic content working together on a single page:

```
// Archivo: app/blog/page.tsx
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { cacheLife, cacheTag } from 'next/cache'
import Link from 'next/link'

export default function BlogPage() {
  return (
    <>

      <header>
        <h1>Our Blog</h1>
        <nav>
          [Home](/) | [About](/about)
        </nav>
      </header>


      <BlogPosts />


      ``<Suspense>``
    </>
  )
}

type Post = { id: string; title: string; author: string; date: string }

// Everyone sees the same blog posts (revalidated every hour)
async function BlogPosts() {
  'use cache'
  cacheLife('hours')
  cacheTag('posts')

  const res = await fetch('https://api.vercel.app/blog')
  const posts: Post[] = await res.json()

  return (
    <section>
      <h2>Latest Posts</h2>
      <ul>
        {posts.map((post) => (
          <li key={post.id}>
            <h3>{post.title}</h3>
            <p>
              By {post.author} on {post.date}
            </p>
          </li>
        ))}
      </ul>
    </section>
  )
}

// UI that depends on a value stored in cookies
async function UserPreferences() {
  const theme = (await cookies()).get('theme')?.value || 'light'
  const favoriteCategory = (await cookies()).get('category')?.value

  return (
    <aside>
      <p>Your theme: {theme}</p>
      {favoriteCategory && <p>Favorite category: {favoriteCategory}</p>}
    </aside>
  )
}
```

During prerendering, the header (static) and blog posts (cached with `use cache`) become part of the static shell, along with the fallback UI for user preferences. The UI preferences stored in cookies stream in at request time.

Reading `cookies()` here doesn't opt-in the whole route into dynamic rendering, the way the previous rendering model did. The Suspense boundary provides fallback UI where the runtime access streams, while static and cached content still ship in the initial HTML.

Just as `````) }

```
Alternatively, you can **cache the result** so all users see the same value until revalidation:

```tsx
// Archivo: page.tsx
export default async function Page() {
  'use cache'
  const buildId = crypto.randomUUID()
  return <p>Build ID: {buildId}</p>
}
```

You don't need to memorize which operations behave this way. The dev overlay surfaces a **blocking-prerender-random**, **blocking-prerender-current-time**, or **blocking-prerender-crypto** insight (depending on the call) with these fixes:

> 💡 **Solución recomendada (dynamic):** Generate on every request — Ver guía de solución ' },]} />

> 💡 **Solución recomendada (cache):** Cache the value — Ver guía de solución

Predictable values

Unlike random values and timestamps, which can vary between renders, module imports, synchronous I/O, and pure computations produce the same result every time they run. Components using only these operations are prerendered automatically, and their output becomes part of the static HTML at build time.

```tsx
// Archivo: page.tsx
import fs from 'node:fs'

export default async function Page() {
  const constants = await import('./constants.json')
  const content = fs.readFileSync('./config.json', 'utf-8')
  const items = JSON.parse(content).items ?? []

  return (
    <div>
      <h1>{constants.appName}</h1>
      <ul>
        {items.map((item) => (
          <li key={item.id}>{item.value}</li>
        ))}
      </ul>
    </div>
  )
}
```

> **Good to know:** This includes queries to embedded databases with synchronous APIs, such as `better-sqlite3` or Node.js's built-in `node:sqlite`. If you need per-request data from a synchronous source, call `connection()` before the query.

Some asynchronous APIs read local resources that don't depend on the incoming request, such as fonts or configuration files. When those resources are expected to be the same for every request, read them once at module scope instead of during rendering

If the data should instead be computed during rendering and reused across requests, wrap the read in `use cache`. If the data depends on the incoming request or is expected to change over time, read it during request-time rendering.
```

```
// Archivo: page.tsx
import { readFile } from 'node:fs/promises'

const content = await readFile('./config.json', 'utf-8')
const items = JSON.parse(content).items ?? []

export default function Page() {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.value}</li>
      ))}
    </ul>
  )
}
```

In this example, the configuration file is expected to be the same for every request, so it is read once at module scope. Calling `await readFile()` inside the component would be treated as uncached data that must be either accessed within `use cache` or behind a ````` {children}) }

function SlugHeading({ slug }: { slug: string }) { return

{slug}

}

```
Now `<Sidebar />`, `{children}`, and the Suspense fallback are all part of the static shell. Only `SlugHeading` streams in at
request time. You can also pass the entire `params` promise and await it in the child component.

The same principle applies to `cookies()`, `headers()`, `searchParams`, and data fetches. See [Reusing data with
`React.cache`](/docs/app/getting-started/fetching-data#reusing-data-with-reactcache) for a related pattern.

### Instant navigation

Cache Components shipped in 16.0.0 with verification that direct visits to a route produce a static shell. Client navigations
are different: a ```<Suspense>``
  )
}

async function Results({
  searchParams,
}: Pick<PageProps<'/search'>, 'searchParams'>) {
  const { q } = await searchParams
  const results = await search(q)
  return (
    <ul>
      {results.map((result) => (
        <li key={result.id}>{result.title}</li>
      ))}
    </ul>
  )
}

async function search(query: string | string[] | undefined) {
  'use cache'
  return db.search(query)
}
```

On a direct visit, `<Results>` streams in behind the fallback.

When a `<Link>` to `/search?q=shoes` is prefetched, the framework resolves `searchParams` from the link's URL, so the cached `search` result is included in the runtime prerender before the click. The browser then reuses it until its `stale` time passes or the `searchParams` change.

See Adopting Partial Prefetching to understand how `<Link>` prefetching behaves and how to adopt it.

See the Optimizing prefetching guide for full patterns and the `prefetch` reference for all modes.

Where cached content is stored

A cached function's output is serialized into an **RSC payload**, at build time or at runtime. This payload is what everything else works from. Next.js renders it into HTML, keeps it in a server or remote store, or sends it to the browser, and `cacheLife` sets how long each copy stays fresh:

- **Prerendered HTML.** The payload is rendered to HTML and stored on disk when self-hosting, or in your platform's durable storage behind a CDN. That HTML is the static shell at build time and the concrete page after an ISR upgrade, with `revalidate` and `expire` controlling when it's rebuilt.
- **Shared store.** By default the result stays in a per-instance, in-memory store that is ephemeral on serverless. `use cache: remote` moves it to a durable cache handler shared across instances, a network roundtrip that pays off only at a **high hit rate**.
- **Browser.** The payload is included in the RSC sent for a client navigation or prefetch, where the browser keeps it fresh for its `stale` window. `use cache: private` results live only here.

> **Good to know:** An App Shell that reads `cookies()` or `headers()` is session-specific, cached per session on the client rather than in the shared server cache.

All of these stores are scoped to a single deployment. A new deploy starts fresh, new prerenders are built, and `use cache` entries don't carry over, even durable `remote` ones, because the cache key includes the build id. See Runtime caching considerations for per-environment behavior and Self-hosting for configuring the server cache.

Incremental Static Regeneration

In a route with dynamic param segments, `generateStaticParams` prerenders the URLs you list at build time. Any other URL is served the App Shell instantly, then upgraded in the background with its now-known params and cached for the next visitor.

See ISR with Cache Components for the full walkthrough.

Bots and crawlers

Browsers receive the static shell instantly. Bots and crawlers are detected by their user agent and handled differently: because they need a complete document, Next.js skips the shell and renders the entire page dynamically at request time, then sends the finished HTML once the render completes.

Because the shell is re-rendered instead of reused, work that completed during prerendering now runs at request time for a bot. If part of your shell depends on inputs that only exist while prerendering, such as build-time data or values that are not reachable in the request-time environment, a page that loads for a person can fail to render for a crawler. Make sure the data your shell relies on is also available at request time. See Bots and crawlers in the Streaming guide for more details.

Revalidating

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/revalidating
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to revalidate cached data using time-based and on-demand strategies.*

> This page covers revalidation with Cache Components, enabled by setting `cacheComponents: true` in your `next.config.ts` file. If you're not using Cache Components, see the Caching and Revalidating (Previous Model) guide.

Revalidation is the process of updating cached data. It lets you keep serving fast, cached responses while ensuring content stays fresh. There are two strategies:

- **Time-based revalidation**: Automatically refresh cached data after a set duration using `cacheLife`.
- **On-demand revalidation**: Manually invalidate cached data after a mutation using `revalidateTag`, `updateTag`, or `revalidatePath`.

`cacheLife`

`cacheLife` controls how long cached data remains valid. Use it inside a `use cache` scope to set the cache lifetime.

```
// Archivo: app/lib/data.ts highlight={1,4,5}
import { cacheLife } from 'next/cache'

export async function getProducts() {
  'use cache'
  cacheLife('hours')
  return db.query('SELECT * FROM products')
}
```

`cacheLife` accepts a profile name or a custom configuration object:

Profile	`stale`	`revalidate`	`expire`
`default`	5m	15m	never
`seconds`	30s	1s	60s
`minutes`	5m	1m	1h
`hours`	5m	1h	1d
`days`	5m	1d	1w
`weeks`	5m	1w	30d
`max`	5m	30d	1y

For fine-grained control, pass an object:

```
'use cache'
cacheLife({
  stale: 3600, // 1 hour until considered stale
  revalidate: 7200, // 2 hours until revalidated
  expire: 86400, // 1 day until expired
})
```

> **Good to know:** A cache is considered "short-lived" when it uses the `seconds` profile, `revalidate: 0`, or `expire` under 5 minutes. Short-lived caches are automatically excluded from prerenders and become dynamic holes instead. See Prerendering behavior for details.

See the `cacheLife` API reference for all profiles and custom configuration options.

`cacheTag`

`cacheTag` lets you tag cached data so it can be invalidated on-demand. Use it inside a `use cache` scope:

```
// Archivo: app/lib/data.ts switcher
import { cacheTag } from 'next/cache'

export async function getProducts() {
  'use cache'
  cacheTag('products')
  return db.query('SELECT * FROM products')
}
```

```
// Archivo: app/lib/data.js switcher
import { cacheTag } from 'next/cache'

export async function getProducts() {
  'use cache'
  cacheTag('products')
  return db.query('SELECT * FROM products')
}
```

Once tagged, invalidate the cache using `revalidateTag` or `updateTag`.

See the `cacheTag` API reference to learn more.

`revalidateTag`

`revalidateTag` invalidates cache entries by tag using stale-while-revalidate semantics — stale content is served immediately while fresh content loads in the background. This is ideal for content where a slight delay in updates is acceptable, like blog posts or product catalogs.

```
// Archivo: app/lib/actions.ts highlight={1,5} switcher
import { revalidateTag } from 'next/cache'

export async function updateUser(id: string) {
  // Mutate data
  revalidateTag('user', 'max') // Recommended: stale-while-revalidate
}
```

```
// Archivo: app/lib/actions.js highlight={1,5} switcher
import { revalidateTag } from 'next/cache'

export async function updateUser(id) {
  // Mutate data
  revalidateTag('user', 'max') // Recommended: stale-while-revalidate
}
```

You can reuse the same tag in multiple functions to revalidate them all at once. Call `revalidateTag` in a Server Action or Route Handler.

> **Good to know:** The second argument sets how long stale content can be served while fresh content generates in the background. Once it expires, subsequent requests block until fresh content is ready. Using `'max'` gives the longest stale window.

See the `revalidateTag` API reference to learn more.

`updateTag`

`updateTag` immediately expires cached data for read-your-own-writes scenarios — the user sees their change right away instead of stale content. Unlike `revalidateTag`, it can only be used in Server Actions.

```
// Archivo: app/lib/actions.ts highlight={1,12} switcher
import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData: FormData) {
  const post = await db.post.create({
    data: {
      title: formData.get('title'),
      content: formData.get('content'),
    },
  })

  updateTag('posts')
  redirect(`/posts/${post.id}`)
}
```

```
// Archivo: app/lib/actions.js highlight={1,12} switcher
import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData) {
  const post = await db.post.create({
    data: {
      title: formData.get('title'),
      content: formData.get('content'),
    },
  })

  updateTag('posts')
  redirect(`/posts/${post.id}`)
}
```

	`updateTag`	`revalidateTag`
Where	Server Actions only	Server Actions and Route Handlers
Behavior	Immediately expires cache	Stale-while-revalidate
Use case	Read-your-own-writes (user sees their change)	Background refresh (slight delay OK)

See the `updateTag` API reference to learn more.

`revalidatePath`

`revalidatePath` invalidates all cached data for a specific route path. Use it when you want to revalidate a route without knowing which tags are associated with it.

```
// Archivo: app/lib/actions.ts highlight={1,5} switcher
import { revalidatePath } from 'next/cache'

export async function updateUser(id: string) {
  // Mutate data
  revalidatePath('/profile')
}
```

```
// Archivo: app/lib/actions.js highlight={1,5} switcher
import { revalidatePath } from 'next/cache'

export async function updateUser(id) {
  // Mutate data
  revalidatePath('/profile')
}
```

> **Good to know**: Prefer tag-based revalidation (`revalidateTag` / `updateTag`) over path-based when possible — it's more precise and avoids over-invalidating.

See the `revalidatePath` API reference to learn more.

What should I cache?

Cache data that doesn't depend on runtime data and that you're OK serving from cache for a period of time. Use `use cache` with `cacheLife` to describe that behavior.

When content doesn't need time-based revalidation, for example data from a CMS, use `cacheTag` and a long `cacheLife` like `max` to keep it in the static shell. Configure the content source to trigger a webhook, or other notification, that calls `revalidateTag` when the content changes. This reduces unnecessary time-based revalidation for content that hasn't changed.

> **Good to know:** In serverless environments, in-memory cache entries may not persist across revalidations. See runtime caching considerations for details.

Caching and Revalidating (Previous Model)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/caching-without-cache-components
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to cache and revalidate data using fetch options, unstable_cache, and route segment configs for projects not using Cache Components.*

> This guide assumes you are **not** using Cache Components which was introduced in version 16 under the `cacheComponents` flag.

Caching `fetch` requests

By default, `fetch` requests are not cached. You can cache individual requests by setting the `cache` option to `'force-cache'`.

```
// Archivo: app/page.tsx switcher
export default async function Page() {
  const data = await fetch('https:// ... ', { cache: 'force-cache' })
}
```

```
// Archivo: app/page.jsx switcher
export default async function Page() {
  const data = await fetch('https:// ... ', { cache: 'force-cache' })
}
```

See the `fetch` API reference to learn more.

`unstable_cache` for non-`fetch` functions

`unstable_cache` allows you to cache the result of database queries and other async functions that don't use `fetch`. Wrap `unstable_cache` around the function:

```ts
// Archivo: app/lib/data.ts switcher
import { unstable_cache } from 'next/cache'
import { db } from '@/lib/db'

export const getCachedUser = unstable_cache(
  async (id: string) => {
    return db
      .select()
      .from(users)
      .where(eq(users.id, id))
      .then((res) => res[0])
  },
  ['user'], // cache key prefix
  {
    tags: ['user'],
    revalidate: 3600,
  }
)
```

```js
// Archivo: app/lib/data.js switcher
import { unstable_cache } from 'next/cache'
import { db } from '@/lib/db'

export const getCachedUser = unstable_cache(
  async (id) => {
    return db
      .select()
      .from(users)
      .where(eq(users.id, id))
      .then((res) => res[0])
  },
  ['user'], // cache key prefix
  {
    tags: ['user'],
    revalidate: 3600,
  }
)
```

The third argument accepts:

- `tags`: an array of tags for on-demand revalidation with `revalidateTag`.
- `revalidate`: the number of seconds before the cache is revalidated.

See the `unstable_cache` API reference to learn more.

Route segment config

You can configure caching behavior at the route level by exporting config options from a Page, Layout, or Route Handler.

`dynamic`

Change the dynamic behavior of a layout or page to fully static or fully dynamic.

```ts
// Archivo: layout.tsx | page.tsx | route.ts switcher
export const dynamic = 'auto'
// 'auto' | 'force-dynamic' | 'error' | 'force-static'
```

```js
// Archivo: layout.js | page.js | route.js switcher
export const dynamic = 'auto'
// 'auto' | 'force-dynamic' | 'error' | 'force-static'
```

- `'auto'` (default): The default option to cache as much as possible without preventing any components from opting into dynamic behavior.
- `'force-dynamic'`: Force dynamic rendering, which will result in routes being rendered for each user at request time. This option is equivalent to:

- Setting the option of every `fetch()` request in a layout or page to `{ cache: 'no-store', next: { revalidate: 0 } }`.
- Setting the segment config to `export const fetchCache = 'force-no-store'`

- `'error'`: Force prerendering and cache the data of a layout or page by causing an error if any components use Request-time APIs or uncached data. This option is equivalent to:
  - `getStaticProps()` in the `pages` directory.
  - Setting the option of every `fetch()` request in a layout or page to `{ cache: 'force-cache' }`.
  - Setting the segment config to `fetchCache = 'only-cache'`.
- `'force-static'`: Force prerendering and cache the data of a layout or page by forcing `cookies`, `headers()` and `useSearchParams()` to return empty values. It is possible to `revalidate`, `revalidatePath`, or `revalidateTag`, in pages or layouts rendered with `force-static`.

`fetchCache`

This is an advanced option that should only be used if you specifically need to override the default behavior.

Time-based revalidation

Use the `next.revalidate` option on `fetch` to revalidate data after a specified number of seconds:

```
// Archivo: app/page.tsx switcher
export default async function Page() {
  const data = await fetch('https:// ... ', { next: { revalidate: 3600 } })
}
```

```
// Archivo: app/page.jsx switcher
export default async function Page() {
  const data = await fetch('https:// ... ', { next: { revalidate: 3600 } })
}
```

For non-`fetch` functions, `unstable_cache` accepts a `revalidate` option in its configuration (see example above).

Route segment config `revalidate`

Set the default revalidation time for a layout or page. This option does not override the `revalidate` value set by individual `fetch` requests.

```
// Archivo: layout.tsx | page.tsx | route.ts switcher
export const revalidate = false
// false | 0 | number
```

```
// Archivo: layout.js | page.js | route.js switcher
export const revalidate = false
// false | 0 | number
```

- `false` (default): The default heuristic to cache any `fetch` requests that set their `cache` option to `'force-cache'` or are discovered before a Request-time API is used. Semantically equivalent to `revalidate: Infinity` which effectively means the resource should be cached indefinitely. It is still possible for individual `fetch` requests to use `cache: 'no-store'` or `revalidate: 0` to avoid being cached and make the route dynamically rendered. Or set `revalidate` to a positive number lower than the route default to increase the revalidation frequency of a route.
- `0`: Ensure a layout or page is always dynamically rendered even if no Request-time APIs or uncached data fetches are discovered. This option changes the default of `fetch` requests that do not set a `cache` option to `'no-store'` but leaves `fetch` requests that opt into `'force-cache'` or use a positive `revalidate` as is.
- `number`: (in seconds) Set the default revalidation frequency of a layout or page to `n` seconds.

Good to know:

- The revalidate value needs to be statically analyzable. For example `revalidate = 600` is valid, but `revalidate = 60 * 10` is not.
- The revalidate value is not available when using the deprecated `runtime = 'edge'`.
- In Development, Pages are *always* rendered on-demand and are never cached. This allows you to see changes immediately without waiting for a revalidation period to pass.

Revalidation frequency

- The lowest `revalidate` across each layout and page of a single route will determine the revalidation frequency of the *entire* route. This ensures that child pages are revalidated as frequently as their parent layouts.
- Individual `fetch` requests can set a lower `revalidate` than the route's default `revalidate` to increase the revalidation frequency of the entire route. This allows you to dynamically opt-in to more frequent revalidation for certain routes based on some criteria.

On-demand revalidation

To revalidate cached data after an event, use `revalidateTag` or `revalidatePath` in a Server Action or Route Handler.

Tagging cached data

Tag `fetch` requests with `next.tags` to enable on-demand cache invalidation:

```
// Archivo: app/lib/data.ts switcher
export async function getUserById(id: string) {
  const data = await fetch(`https:// ... `, {
    next: { tags: ['user'] },
  })
}
```

```
// Archivo: app/lib/data.js switcher
export async function getUserById(id) {
  const data = await fetch(`https:// ... `, {
    next: { tags: ['user'] },
  })
}
```

For non-`fetch` functions, `unstable_cache` also accepts a `tags` option (see example above).

`revalidateTag`

Invalidate cached data by tag using `revalidateTag`:

```
// Archivo: app/lib/actions.ts switcher
import { revalidateTag } from 'next/cache'

export async function updateUser(id: string) {
  // Mutate data
  revalidateTag('user', 'max')
}
```

```
// Archivo: app/lib/actions.js switcher
import { revalidateTag } from 'next/cache'

export async function updateUser(id) {
  // Mutate data
  revalidateTag('user', 'max')
}
```

`revalidatePath`

Invalidate all cached data for a specific route path using `revalidatePath`:

```
// Archivo: app/lib/actions.ts switcher
import { revalidatePath } from 'next/cache'

export async function updateUser(id: string) {
  // Mutate data
  revalidatePath('/profile')
}
```

```
// Archivo: app/lib/actions.js switcher
import { revalidatePath } from 'next/cache'

export async function updateUser(id) {
  // Mutate data
  revalidatePath('/profile')
}
```

Deduplicating requests

If you are not using `fetch` (which is automatically memoized), and instead using an ORM or database directly, you can wrap your data access with the React `cache` function to deduplicate requests within a single render pass:

```
// Archivo: app/lib/data.ts switcher
import { cache } from 'react'
import { db, posts, eq } from '@/lib/db'

export const getPost = cache(async (id: string) => {
  const post = await db.query.posts.findFirst({
    where: eq(posts.id, parseInt(id)),
  })
})
```

```
// Archivo: app/lib/data.js switcher
import { cache } from 'react'
import { db, posts, eq } from '@/lib/db'

export const getPost = cache(async (id) => {
  const post = await db.query.posts.findFirst({
    where: eq(posts.id, parseInt(id)),
  })
})
```

Preloading data

You can preload data by creating a utility function that you eagerly call above blocking requests. This lets you initiate data fetching early, so the data is already available by the time the component renders.

Combine the `server-only` package with React's `cache` to create a reusable preload utility:

```tsx
// Archivo: utils/get-item.ts switcher
import { cache } from 'react'
import 'server-only'

export const getItem = cache(async (id: string) => {
  // ...
})

export const preload = (id: string) => {
  void getItem(id)
}
```

```jsx
// Archivo: utils/get-item.js switcher
import { cache } from 'react'
import 'server-only'

export const getItem = cache(async (id) => {
  // ...
})

export const preload = (id) => {
  void getItem(id)
}
```

Then call `preload()` before any blocking work so the data starts loading immediately:

```tsx
// Archivo: app/item/[id]/page.tsx switcher
import { getItem, preload, checkIsAvailable } from '@/lib/data'

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  // Start loading item data
  preload(id)
  // Perform another asynchronous task
  const isAvailable = await checkIsAvailable()

  return isAvailable ? ``<Item>`` : null
}

async function Item({ id }: { id: string }) {
  const result = await getItem(id)
  // ...
}
```

```jsx
// Archivo: app/item/[id]/page.js switcher
import { getItem, preload, checkIsAvailable } from '@/lib/data'

export default async function Page({ params }) {
  const { id } = await params
  // Start loading item data
  preload(id)
  // Perform another asynchronous task
  const isAvailable = await checkIsAvailable()

  return isAvailable ? ``<Item>`` : null
}

async function Item({ id }) {
  const result = await getItem(id)
  // ...
}
```

Statically generating dynamic routes

To prerender dynamic routes with `generateStaticParams` and revalidate them over time, see the Incremental Static Regeneration guide.

Using a CDN with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/cdn-caching
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how CDN caching works with Next.js, including what works today, cache variability, and the direction toward pathname-based cache keying.*

Next.js sets standard `Cache-Control` headers that CDNs can use to cache responses at the edge. This page covers what works today, where CDN caching is challenging, and the direction toward eliminating custom-header dependencies.

What Works Today

Cache-Control headers

Next.js sets `Cache-Control` headers based on the rendering strategy of each route:

- **Static pages** (no revalidation): `s-maxage=31536000` (one year)
- **ISR pages** (time-based revalidation): `s-maxage={revalidate}, stale-while-revalidate={expire - revalidate}`. The default `expire` is one year, so `stale-while-revalidate` is included in the response header by default. You can customize this with `cacheLife`.
- **Dynamic pages** (no caching): `private, no-cache, no-store, max-age=0, must-revalidate`

CDNs that respect `s-maxage` and `stale-while-revalidate` can cache static and ISR pages at the edge. However, CDN-level caching alone does not support on-demand revalidation (`revalidateTag()` / `revalidatePath()`): those calls invalidate the Next.js server cache, but the CDN will continue serving its cached copy until the `s-maxage` TTL expires. To propagate on-demand revalidation to the CDN, trigger CDN purges alongside your revalidation call. A common pattern is: call `revalidateTag()` / `revalidatePath()` to invalidate the Next.js server cache, then call your CDN purge API for the affected keys (including both HTML and RSC variants).

Static assets

Static assets (JavaScript, CSS, images, fonts) served from `/_next/static/` include content hashes in their filenames and have a 1 year `max-age` and `immutable` directive: `public, max-age=31536000, immutable`

You can use `assetPrefix` to serve static assets from a different domain or CDN origin.

Static prefetches (PPR-enabled routes)

When a route has Partial Prerendering enabled and the `next-router-prefetch` header is set (indicating a static prefetch), the response is deterministic: it returns the same prerendered content regardless of the client's router state. The `next-router-state-tree` header is not parsed for these requests, so it does not affect the response.

For PPR-enabled routes, a CDN can cache static prefetch responses if it:

1. Includes the `_rsc` search parameter in the cache key (to distinguish prefetch variants from HTML responses).
2. Respects the `Cache-Control` headers Next.js sets on the response.

> **Good to know:** For routes without PPR, the `next-router-state-tree` header is read during prefetch requests to determine which segments to include, which increases cache `vary` as it passes the current router state. When Cache Components is enabled, segment-level prefetches already use pathname-based routes (for example, `/page.segments/_tree.segment.rsc`), and CDNs can cache these with standard pathname-based cache keys.

Where CDN Caching Is Challenging

App Router responses can vary based on several custom request headers. Next.js sets a `Vary` header on responses to signal this to CDNs:

- `rsc` — whether the request should return a React Server Components (RSC) payload instead of HTML
- `next-router-state-tree` — the client's current router state, used for targeted segment updates during dynamic navigations
- `next-router-prefetch` — whether this is a prefetch request
- `next-router-segment-prefetch` — the specific segment being prefetched
- `next-url` — added only for routes that use interception routes, carries the URL being intercepted

> **Good to know:** `proxy.js` (previously Middleware) should run before the CDN cache so it remains the source of truth for auth, redirects, and rewrites. If your deployment places `proxy.js` behind the CDN, configure the cache layer to bypass caching for routes that depend on `proxy.js` decisions.

Many CDNs don't support `Vary` without additional configuration. Next.js addresses this with the `_rsc` search parameter: a hash of the relevant request header values that acts as a cache-key, ensuring different response variants get different cache keys. This ensures correct responses even on CDNs that ignore `Vary`.

Handling Headers at the CDN

What you can safely ignore

These headers can be omitted in specific cases without causing protocol errors. The server still returns a parseable response, but it may be larger or less targeted to the specific navigation:

`next-router-state-tree`: when omitted on non-prefetch RSC requests, the server returns a full payload instead of a targeted segment update.

`next-router-segment-prefetch`: when omitted on prefetch requests, the server falls back to a broader prefetch payload instead of a segment-specific one.

`next-url`: used for interception routes to vary the response based on the referring page. If omitted, interception routes are not supported as the server doesn't know what original path to match against. The response returned is for regular navigation when `next-url` is omitted: the user sees the target page instead of the intercepted target page.

What you must preserve

The `rsc` header must be forwarded from the client to the server. This header tells the server to return an RSC payload instead of HTML. If a CDN strips it, the server returns HTML when the client-side router expects RSC data, which breaks client-side navigation, causing browser navigations instead. The `Vary` header and `_rsc` parameter exist specifically to prevent CDNs from serving a cached HTML response to an RSC request (or vice versa).

When `next-router-prefetch` is present, preserve both the prefetch header and the `_rsc` search parameter. For prefetch flows, `_rsc` is a required cache-busting discriminator and should be treated as mandatory.

The `_rsc` search parameter must be included in the cache key. It distinguishes response variants (HTML vs. RSC, different prefetch types). Ensure your CDN does not strip query parameters from cache keys, as some CDNs do this by default. By default, when an RSC request arrives without the correct `_rsc` value, the server responds with a **307 redirect** to the URL with the correct hash. This behavior can be disabled by setting `experimental.validateRSCRequestHeaders` to `false`. CDNs should follow this redirect. Platforms that compute the hash upstream can rewrite requests to include the correct `_rsc` before forwarding to avoid an extra round trip.

> **Good to know:** Today, `next-url` is included in the `_rsc` hash even during static prefetches. This means you cannot safely ignore it under the current scheme without potentially getting cache misses. The pathname-based direction described below resolves this gap.

Direction: Pathname-Based Cache Keying

The Next.js team is working on moving all cache-affecting inputs into the URL pathname, eliminating the need for `Vary` on custom headers and removing the `_rsc` search parameter. This resolves the CDN caching challenges described above.

How it works

The approach extends the routing scheme that `output: 'export'` and segment prefetches already use today. File extensions in the pathname identify the response type:

- **Full page RSC**: `/my/page.rsc` returns the RSC payload for the entire page
- **Segment RSC**: `/my/page.segments/path/to/segment.segment.rsc` returns the RSC payload for a specific segment

Under this model:

- **The pathname determines the cache key.** Anything in the pathname affects which response variant is returned.
- **Search parameters can be safely dropped** without affecting returned responses.
- **Standard HTTP cache headers** (`Cache-Control` , `max-age` , etc.) are respected as usual.
- **No** `Vary` **support needed** from the CDN.

A CDN would cache Next.js responses by using the pathname as the cache key, ignoring search parameters, and respecting standard `Cache-Control` headers. No need to understand `Vary` , inspect custom headers, or program edge logic.

What changes for interception routes

Under the current scheme, `next-url` contributes to the `_rsc` hash, so dropping it causes cache misses. Under the pathname-based scheme, interception variability would be encoded in a search parameter (not the pathname):

- If a CDN preserves search params, interception works correctly.
- If a CDN drops search params, interception is not supported. It would gracefully degrade to the non-intercepted page, client-side navigations won't break.

This makes interception route support an opt-in CDN capability rather than a requirement.

Current status

This direction extends patterns that are already operational in the codebase (segment prefetch paths, `output: 'export'` mode). It is in active design.

CDN Feature Compatibility

For a full table showing the infrastructure primitives available on every major CDN (edge compute, key-value storage, blob storage, PPR resuming), see Deploying to Platforms.

How to configure Continuous Integration (CI) build caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/ci-build-caching
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to configure CI to cache Next.js builds*

To improve build performance, Next.js saves a cache to `.next/cache` that is shared between builds.

To take advantage of this cache in Continuous Integration (CI) environments, your CI workflow will need to be configured to correctly persist the cache between builds.

> If your CI is not configured to persist `.next/cache` between builds, you may see a No Cache Detected error.

Here are some example cache configurations for common CI providers:

Vercel

Next.js caching is automatically configured for you. There's no action required on your part. If you are using Turborepo on Vercel, learn more here.

CircleCI

Edit your `save_cache` step in `.circleci/config.yml` to include `.next/cache`:

```
steps:
  - save_cache:
      key: dependency-cache-{{ checksum "yarn.lock" }}
      paths:
        - ./node_modules
        - ./.next/cache
```

If you do not have a `save_cache` key, please follow CircleCI's documentation on setting up build caching.

Travis CI

Add or merge the following into your `.travis.yml`:

```
cache:
  directories:
    - $HOME/.cache/yarn
    - node_modules
    - .next/cache
```

GitLab CI

Add or merge the following into your `.gitlab-ci.yml`:

```
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .next/cache/
```

Netlify CI

Use Netlify Plugins with `@netlify/plugin-nextjs`.

AWS CodeBuild

Add (or merge in) the following to your `buildspec.yml`:

```
cache:
  paths:
    - 'node_modules/**/*' # Cache `node_modules` for faster `yarn` or `npm i`
    - '.next/cache/**/*' # Cache Next.js for faster application rebuilds
```

GitHub Actions

Using GitHub's actions/cache, add the following step in your workflow file:

```
uses: actions/cache@v4
with:
  # See here for caching with `yarn`, `bun` or other package managers https://github.com/actions/cache/blob/main/examples.md
or you can leverage caching with actions/setup-node https://github.com/actions/setup-node
  path: |
    ~/.npm
    ${{ github.workspace }}/.next/cache
  # Generate a new cache whenever packages or source files change.
  key: ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-${{ hashFiles('**/*.js', '**/*.jsx', '**/*.ts',
'**/*.tsx') }}
  # If source files changed but packages didn't, rebuild from a prior cache.
  restore-keys: |
    ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-
```

Bitbucket Pipelines

Add or merge the following into your `bitbucket-pipelines.yml` at the top level (same level as `pipelines`):

```
definitions:
  caches:
    nextcache: .next/cache
```

Then reference it in the `caches` section of your pipeline's `step`:

```
- step:
    name: your_step_name
    caches:
      - node
      - nextcache
```

Heroku

Using Heroku's custom cache, add a `cacheDirectories` array in your top-level package.json:

```
"cacheDirectories": [".next/cache"]
```

Azure Pipelines

Using Azure Pipelines' Cache task, add the following task to your pipeline yaml file somewhere prior to the task that executes `next build`:

```
- task: Cache@2
  displayName: 'Cache .next/cache'
  inputs:
    key: next | $(Agent.OS) | yarn.lock
    path: '$(System.DefaultWorkingDirectory)/.next/cache'
```

Jenkins (Pipeline)

Using Jenkins' Job Cacher plugin, add the following build step to your `Jenkinsfile` where you would normally run `next build` or `npm install`:

```
stage("Restore npm packages") {
    steps {
        // Writes lock-file to cache based on the GIT_COMMIT hash
        writeFile file: "next-lock.cache", text: "$GIT_COMMIT"

        cache(caches: [
            arbitraryFileCache(
                path: "node_modules",
                includes: "**/*",
                cacheValidityDecidingFile: "package-lock.json"
            )
        ]) {
            sh "npm install"
        }
    }
}
stage("Build") {
    steps {
        // Writes lock-file to cache based on the GIT_COMMIT hash
        writeFile file: "next-lock.cache", text: "$GIT_COMMIT"

        cache(caches: [
            arbitraryFileCache(
                path: ".next/cache",
                includes: "**/*",
                cacheValidityDecidingFile: "next-lock.cache"
            )
        ]) {
            // aka `next build`
            sh "npm run build"
        }
    }
}
```

Client-side data fetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/client-side-data-fetching
> 📁 **Sección:** App > Guides > Client Side Data Fetching
> 📝 **Descripción:** *Fetch data in Client Components with a data-fetching library, optionally provide initial data from a Server Component, and coordinate server and client caches.*

Many apps can provide responsive interactions without a client data-fetching library. If a Client Component only needs to read server data once, pass it a Promise and unwrap it with React's `use()`.

This avoids adding a library for data that never revalidates on the client. See Building interactive apps for patterns using Server Functions, transitions, optimistic UI, and pending feedback.

Use a client data-fetching library such as SWR, TanStack Query, or Apollo Client when Client Components need a shared browser cache. These libraries can add focus revalidation, interval polling, request deduplication, or optimistic updates across components.

Choose a client fetching pattern

First decide whether the initial view needs data from the server or can wait for a browser request after hydration. Client data-fetching libraries support three common patterns:

Pattern	SWR	TanStack Query	When data becomes available
Inline loading states	`useSWR`	`useQuery`	Browser request after hydration
Suspense loading states	`useSWR` with `suspense: true`	`useSuspenseQuery`	Browser request after hydration
Provided by the server	`<SWRConfig fallback>`	`<HydrationBoundary>`	Initial render or streamed from server

Use inline loading states when each component should render its own loading UI. Use Suspense to define loading UI at a boundary and coordinate which parts of the interface reveal together or progressively. For client-only fetching, choose the pattern that matches the loading experience you want. Suspense coordinates rendering, while the data library and component structure determine when requests start.

For browser-driven interactions such as autocomplete, you can use either client-only pattern. The initial result waits for hydration and a browser request, which is often the right tradeoff for data that is not needed until an interaction.

Provide initial data from a Server Component when the server knows what the initial render needs. The value can be included in the initial render or streamed through Suspense. The library receives it in the React Server Component payload and can continue managing it in the browser.

Cache server data with Cache Components (optional)

Providing initial data and caching it on the server are independent choices. Add Cache Components when the server read or rendered view should be reused. When both are enabled, three cache layers can hold related data:

Layer	What it stores	Freshness control
Next.js server cache	Cached data and Server Component output	`cacheLife` `revalidate` and `expire`
Next.js client cache	React Server Component payloads for visited and prefetched routes	`cacheLife` `stale`
Client data-fetching library	Browser data stored under an SWR key or TanStack query key	The library's revalidation options and mutations

Next.js prefetching can place a route's React Server Component payload in the client cache before navigation.

The cache layers keep independent freshness policies and do not need matching durations. Cache identities and mutation invalidation must stay coordinated across layers.

Coordinate mutations

Server Components, data-fetching libraries, and mutations manage different parts of the data flow:

- **Server Components** provide the initial data, scoped to the segment that owns it.
- **The data-fetching library** stores the browser value under a shared cache identity.
- **Mutations** can update the browser cache immediately and invalidate cached server data so the next render can read a fresh value.

An optimistic update should restore the previous browser value if the write fails. If the server read is not cached, there is no server tag to invalidate.

After a mutation, invalidate any cached server read that provided the initial data:

Method	Use when	Next server read
`updateTag(tag)`	A Server Action must make its update visible immediately	Waits for fresh data
`revalidateTag(tag, 'max')`	The update is passive or stale data is acceptable	Serves stale data while revalidating
`revalidateTag(tag, { expire: 0 })`	A webhook or external system requires immediate expiration	Waits for fresh data

Apply these patterns with SWR or TanStack Query

- Client-side data fetching with SWR
- Client-side data fetching with TanStack Query

See both patterns in the live `next-spa-patterns` demo and its source code.

How to fetch client-side data with SWR

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/client-side-data-fetching/swr
> 📂 **Sección:** App > Guides > Client Side Data Fetching
> 📝 **Descripción:** *Fetch client-side data with SWR, optionally provide initial data from a Server Component, and coordinate server and client caches.*

Use SWR to fetch data in Client Components, provide initial data from Server Components, and coordinate browser mutations with cached server data. See Client-side data fetching to choose a pattern.

Fetch data on the client

SWR can fetch entirely in the browser when the initial view can wait for a browser request after hydration. Choose an inline or Suspense loading state based on where the loading UI should appear. In these examples, `query` starts empty and updates from client state after hydration.

Use `useSWR` when the component should render its own loading and error states. A conditional key delays the request until the interaction provides an input:

```tsx
// Archivo: app/product-autocomplete.tsx
'use client'

import useSWR from 'swr'

type Product = { id: string; name: string }

async function fetcher(url: string): Promise<Product[]> {
  const response = await fetch(url)
  if (!response.ok) throw new Error('Failed to fetch products')
  return response.json()
}

export function ProductAutocomplete({ query }: { query: string }) {
  const {
    data = [],
    error,
    isLoading,
  } = useSWR(
    query ? `/api/products?query=${encodeURIComponent(query)}` : null,
    fetcher
  )

  if (!query) return null
  if (error) return <p>Failed to load products.</p>
  if (isLoading) return <p>Loading products ...</p>

  return (
    <ul>
      {data.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}
```

Use Suspense for client data

Use `suspense: true` when the nearest Suspense boundary should define the loading UI. Keep the interactive shell outside the boundary so it remains available while the results load:

```
// Archivo: app/product-autocomplete.tsx
'use client'

import { Suspense } from 'react'
import useSWR from 'swr'

type Product = { id: string; name: string }

async function fetcher(url: string): Promise<Product[]> {
  const response = await fetch(url)
  if (!response.ok) throw new Error('Failed to fetch products')
  return response.json()
}

export function ProductAutocomplete({ query }: { query: string }) {
  if (!query) return null

  return (
    ``<Suspense>``
  )
}

function ProductResults({ query }: { query: string }) {
  const { data } = useSWR(
    `/api/products?query=${encodeURIComponent(query)}`,
    fetcher,
    { suspense: true }
  )

  return (
    <ul>
      {data.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}
```

With an unconditional key, SWR defines `data` after Suspense resolves. Handle request errors with the nearest error boundary.

The `isLoading` value is `true` when a request is running and there is no loaded data to display. The `isValidating` value is `true` whenever a request is running, including background revalidation.

With `suspense: true`, Suspense handles the initial no-data state. Later revalidation for the same key keeps the current data rendered instead of showing the Suspense fallback again. Use `isValidating` to provide background refresh feedback. Learn more about SWR loading states.

Learn more: SWR data fetching.

> **Good to know:** Independent Suspense reads can start in parallel when they render in sibling components. Multiple Suspense reads in one component run sequentially. Learn more about network waterfalls and SWR Suspense.

Provide initial data from a Server Component

Use the server-provided data pattern when the initial render needs the data and SWR should continue managing it in the browser. With SWR 2.3.0 and React 19, a Server Component can provide fallback data before the client takes over.

Scope `<SWRConfig>` to the route segment that owns the data. The provider keeps the `fallback` close to its consumer and avoids adding feature data to a shared layout:

```
// Archivo: app/products/[id]/page.tsx switcher
import { Suspense } from 'react'
import { SWRConfig } from 'swr'
import { getProduct } from './data' // some server-side function
import { productCache } from './product-cache'
import { ProductView } from './product-view'

export default function Page({ params }: PageProps<'/products/[id]'>) {
  return (
    ``<Suspense>``
  )
}

function ProductData({ id }: { id: string }) {
  return (
    <SWRConfig
      value={{
        fallback: {
          // Not awaited: only components that read this key suspend
          [productCache.key(id)]: getProduct(id),
        },
      }}
    >
      <ProductView id={id} />
    </SWRConfig>
  )
}
```

```
// Archivo: app/products/[id]/page.js switcher
import { Suspense } from 'react'
import { SWRConfig } from 'swr'
import { getProduct } from './data' // some server-side function
import { productCache } from './product-cache'
import { ProductView } from './product-view'

export default function Page({ params }) {
  return (
    ``<Suspense>``
  )
}

function ProductData({ id }) {
  return (
    <SWRConfig
      value={{
        fallback: {
          // Not awaited: only components that read this key suspend
          [productCache.key(id)]: getProduct(id),
        },
      }}
    >
      <ProductView id={id} />
    </SWRConfig>
  )
}
```

The fallback and Client Component must use the same SWR key. Define the key once so both call sites share the same identity:

```
// Archivo: app/products/[id]/product-cache.ts switcher
export const productCache = {
  key: (id: string) => `/api/products/${id}`,
}
```

```
// Archivo: app/products/[id]/product-cache.js switcher
export const productCache = {
  key: (id) => `/api/products/${id}`,
}
```

In the page example, the Promise returned by `params.then()` keeps the fallback visible until the route parameters resolve. `ProductData` then creates a separate, unawaited `getProduct(id)` Promise for the SWR `fallback`. React passes that Promise through the React Server Component payload, and the component reading the matching key suspends until the data resolves.

The Client Component reads the data with `useSWR` using the same key:

```
// Archivo: app/products/[id]/product-view.tsx switcher
'use client'

import useSWR from 'swr'
import { productCache } from './product-cache'

type Product = { id: string; name: string }

async function fetcher(url: string): Promise<Product> {
  const response = await fetch(url)
  if (!response.ok) throw new Error('Failed to fetch product')
  return response.json()
}

export function ProductView({ id }: { id: string }) {
  const { data } = useSWR(productCache.key(id), fetcher, { suspense: true })

  return <h1>{data.name}</h1>
}
```

```
// Archivo: app/products/[id]/product-view.js switcher
'use client'

import useSWR from 'swr'
import { productCache } from './product-cache'

async function fetcher(url) {
  const response = await fetch(url)
  if (!response.ok) throw new Error('Failed to fetch product')
  return response.json()
}

export function ProductView({ id }) {
  const { data } = useSWR(productCache.key(id), fetcher, { suspense: true })

  return <h1>{data.name}</h1>
}
```

> **Good to know:** The `fallback` key and the `useSWR` key must match exactly. If they drift, SWR ignores the fallback value and fetches on the client.

The `fallback` provides the hook's initial value. By default, SWR treats fallback data as stale and starts a browser revalidation after hydration.

SWR does not provide a time-based freshness window for fallback data. Setting `revalidateIfStale: false` skips revalidation when the hook mounts with cached data. This setting applies to every mount, unlike TanStack Query's `staleTime`.

Focus, reconnect, polling, and `mutate` can still revalidate the key. To refresh on a schedule, set `refreshInterval`.

The SWR key points to a Route Handler with a `GET` method. The Route Handler can call the same `getProduct` function that provides the fallback, while the browser uses the URL for revalidation and polling.

Learn more: SWR arguments and keys and SWR with Next.js App Router.

Cache server-provided data with Cache Components

Enable `cacheComponents` in `next.config.ts` before using this pattern. You can then cache the server data used as the SWR fallback. Add `use cache`, choose a `cacheLife` profile, and apply `cacheTag` so mutations can invalidate it:

```
// Archivo: app/products/[id]/data.ts switcher
import { cacheLife, cacheTag } from 'next/cache'

export async function getProduct(id: string) {
  'use cache'
  cacheLife('max')
  cacheTag(`product:${id}`)

  const product = await db.product.findUnique({ where: { id } })
  if (!product) throw new Error('Product not found')
  return product
}
```

```
// Archivo: app/products/[id]/data.js switcher
import { cacheLife, cacheTag } from 'next/cache'

export async function getProduct(id) {
  'use cache'
  cacheLife('max')
  cacheTag(`product:${id}`)

  const product = await db.product.findUnique({ where: { id } })
  if (!product) throw new Error('Product not found')
  return product
}
```

This example uses `cacheLife('max')` because writes invalidate the product tag. Within the cache profile, `stale` controls how long the Next.js client cache can reuse a prefetched payload, while `revalidate` and `expire` control the server cache. Choose a shorter profile when the server value should refresh with time.

SWR owns a separate browser cache, so its revalidation options do not need to match `cacheLife`.

When the same mutation updates the browser cache and invalidates cached server data, you may define both identities in one shared contract:

```
// Archivo: app/products/[id]/product-cache.ts switcher
 export const productCache = {
   key: (id: string) => `/api/products/${id}`,
+  tag: (id: string) => `product:${id}`,
 }
```

```
// Archivo: app/products/[id]/product-cache.js switcher
 export const productCache = {
   key: (id) => `/api/products/${id}`,
+  tag: (id) => `product:${id}`,
 }
```

The server function can then call `cacheTag(productCache.tag(id))`. Keep this contract free of server-only and client-only imports so both cache layers can reuse it.

Coordinate server and client caches after mutations

The fallback provides the initial value. After hydration, SWR manages the browser cache and revalidation. For reused data, keep the SWR key and server tag in the same cache contract:

```
// Archivo: app/activity/activity-cache.ts switcher
export const activityCache = {
  key: '/api/activity/unread',
  tag: (userId: string) => `activity:${userId}`,
}
```

```
// Archivo: app/activity/activity-cache.js switcher
export const activityCache = {
  key: '/api/activity/unread',
  tag: (userId) => `activity:${userId}`,
}
```

Use the same key for the fallback, the Client Component read, and the mutation. With SWR, pass the write to `mutate` and provide `optimisticData`. SWR shows the optimistic value immediately and rolls it back if the write fails. This example keeps the optimistic value after the action succeeds because the final value is known:

```
// Archivo: app/activity/mark-read-button.tsx switcher
'use client'

import { useSWRConfig } from 'swr'
import { markActivityReadAction } from './actions'
import { activityCache } from './activity-cache'

export function MarkReadButton() {
  const { mutate } = useSWRConfig()

  function markRead() {
    return mutate(
      activityCache.key,
      async () => {
        await markActivityReadAction()
        return { count: 0 }
      },
      {
        optimisticData: { count: 0 },
        revalidate: false,
        rollbackOnError: true,
        throwOnError: false,
      }
    )
  }

  return <button onClick={markRead}>Mark read</button>
}
```

```
// Archivo: app/activity/mark-read-button.js switcher
'use client'

import { useSWRConfig } from 'swr'
import { markActivityReadAction } from './actions'
import { activityCache } from './activity-cache'

export function MarkReadButton() {
  const { mutate } = useSWRConfig()

  function markRead() {
    return mutate(
      activityCache.key,
      async () => {
        await markActivityReadAction()
        return { count: 0 }
      },
      {
        optimisticData: { count: 0 },
        revalidate: false,
        rollbackOnError: true,
        throwOnError: false,
      }
    )
  }

  return <button onClick={markRead}>Mark read</button>
}
```

The Server Action writes to the database and expires the tagged server data with `updateTag`. Tag the cached server query with `cacheTag` so the tag matches:

```
// Archivo: app/activity/actions.ts switcher
'use server'

import { updateTag } from 'next/cache'
import {
  getCurrentUserId,
  markActivityRead as markActivityReadInDatabase,
} from './data'
import { activityCache } from './activity-cache'

export async function markActivityReadAction() {
  const userId = await getCurrentUserId()
  await markActivityReadInDatabase(userId)
  updateTag(activityCache.tag(userId))
}
```

```
// Archivo: app/activity/actions.js switcher
'use server'

import { updateTag } from 'next/cache'
import {
  getCurrentUserId,
  markActivityRead as markActivityReadInDatabase,
} from './data'
import { activityCache } from './activity-cache'

export async function markActivityReadAction() {
  const userId = await getCurrentUserId()
  await markActivityReadInDatabase(userId)
  updateTag(activityCache.tag(userId))
}
```

The optimistic SWR value updates the current screen. `updateTag` ensures the next cached server read returns fresh activity.

> **Good to know:** Call `updateTag` when a Server Action changes a cached read that must reflect the write immediately. An uncached read does not have a server tag to update. See Client-side data fetching for other invalidation behaviors.

See the live `next-spa-patterns` demo and its source code. Learn more about SWR mutation and optimistic updates and the SWR documentation.

How to fetch client-side data with TanStack Query

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/client-side-data-fetching/tanstack-query
> 📂 **Sección:** App > Guides > Client Side Data Fetching
> 📝 **Descripción:** *Fetch client-side data with TanStack Query, optionally provide initial data from a Server Component, and coordinate server and client caches.*

Use TanStack Query to fetch data in Client Components, provide initial data from Server Components, and coordinate browser mutations with cached server data. See Client-side data fetching to choose a pattern.

Set up the provider

Wrap the routes that use TanStack Query in a `QueryClientProvider`. Create a new query client for each server render and reuse one query client in the browser:

```
// Archivo: app/products/providers.tsx
'use client'

import type { ReactNode } from 'react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

let browserQueryClient: QueryClient | undefined

function getQueryClient() {
  // Keep server requests isolated and preserve the browser cache across renders.
  if (typeof window === 'undefined') return new QueryClient()
  browserQueryClient ??= new QueryClient()
  return browserQueryClient
}

export function Providers({ children }: { children: ReactNode }) {
  return (
    <QueryClientProvider client={getQueryClient()}>
      {children}
    </QueryClientProvider>
  )
}
```

Render the provider from the nearest shared layout:

```
// Archivo: app/products/layout.tsx
import { Providers } from './providers'

export default function Layout({ children }: LayoutProps<'/products'>) {
  return <Providers>{children}</Providers>
}
```

See the complete provider setup in the demo source or learn about provider options in the TanStack Query Advanced SSR guide.

Fetch data on the client

TanStack Query can fetch entirely in the browser when the initial view can wait for a browser request after hydration. Choose an inline or Suspense loading state based on where the loading UI should appear. In these examples, `query` starts empty and updates from client state after hydration.

Use `useQuery` when the component should render its own loading and error states. The `enabled` option delays the request until the interaction provides an input:

```tsx
// Archivo: app/product-autocomplete.tsx
'use client'

import { useQuery } from '@tanstack/react-query'

type Product = { id: string; name: string }

async function searchProducts(query: string): Promise<Product[]> {
  const response = await fetch(
    `/api/products?query=${encodeURIComponent(query)}`
  )
  if (!response.ok) throw new Error('Failed to fetch products')
  return response.json()
}

export function ProductAutocomplete({ query }: { query: string }) {
  const {
    data = [],
    error,
    isPending,
  } = useQuery({
    queryKey: ['product-search', query],
    queryFn: () => searchProducts(query),
    enabled: query.length > 0,
  })

  if (!query) return null
  if (error) return <p>Failed to load products.</p>
  if (isPending) return <p>Loading products ...</p>

  return (
    <ul>
      {data.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}
```

Learn more: TanStack Query queries.

Use Suspense for client data

Use `useSuspenseQuery` when the nearest Suspense boundary should define the loading UI. Keep the interactive shell outside the boundary so it remains available while the results load:

```
// Archivo: app/product-autocomplete.tsx
'use client'

import { useSuspenseQuery } from '@tanstack/react-query'
import { Suspense } from 'react'

type Product = { id: string; name: string }

async function searchProducts(query: string): Promise<Product[]> {
  const response = await fetch(
    `/api/products?query=${encodeURIComponent(query)}`
  )
  if (!response.ok) throw new Error('Failed to fetch products')
  return response.json()
}

export function ProductAutocomplete({ query }: { query: string }) {
  if (!query) return null

  return (
    ``<Suspense>``
  )
}

function ProductResults({ query }: { query: string }) {
  const { data } = useSuspenseQuery({
    queryKey: ['product-search', query],
    queryFn: () => searchProducts(query),
  })

  return (
    <ul>
      {data.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}
```

If the initial request fails, `useSuspenseQuery` propagates the error to the nearest error boundary.

After a query has data, later refetches for the same query keep the cached data rendered instead of showing the Suspense fallback again. Use `isFetching` to provide background refresh feedback.

If the initial view needs the data, provide it from a Server Component as shown below.

> **Good to know:** Multiple `useSuspenseQuery` calls in one component run sequentially. Put independent queries in sibling components, or use `useSuspenseQueries`. Learn more about request waterfalls.

Provide initial data from a Server Component

Use the server-provided data pattern when the initial render needs the data and TanStack Query should continue managing it in the browser. A Server Component can provide initial query data before the client takes over.

TanStack Query 5.40.0 or later can dehydrate pending queries. Start `prefetchQuery` without awaiting it, then pass the dehydrated state to `<HydrationBoundary>`. Override `queryFn` on the server because the Route Handler's relative URL only resolves in the browser:

```tsx
// Archivo: app/products/[id]/page.tsx switcher
import { Suspense } from 'react'
import {
  defaultShouldDehydrateQuery,
  dehydrate,
  HydrationBoundary,
  QueryClient,
} from '@tanstack/react-query'
import { getProduct } from './data'
import { productCache } from './product-cache'
import { ProductView } from './product-view'

export default function Page({ params }: PageProps<'/products/[id]'>) {
  return (
    ``<Suspense>``
  )
}

function ProductData({ id }: { id: string }) {
  const queryClient = new QueryClient()

  // Not awaited, so rendering is not blocked.
  void queryClient.prefetchQuery({
    ...productCache.options(id),
    queryFn: () => getProduct(id),
  })

  return (
    <HydrationBoundary
      state={dehydrate(queryClient, {
        shouldDehydrateQuery: (query) =>
          defaultShouldDehydrateQuery(query) ||
          query.state.status === 'pending',
      })}
    >
      <ProductView id={id} />
    </HydrationBoundary>
  )
}
```

```
// Archivo: app/products/[id]/page.js switcher
import { Suspense } from 'react'
import {
  defaultShouldDehydrateQuery,
  dehydrate,
  HydrationBoundary,
  QueryClient,
} from '@tanstack/react-query'
import { getProduct } from './data'
import { productCache } from './product-cache'
import { ProductView } from './product-view'

export default function Page({ params }) {
  return (
    ``<Suspense>``
  )
}

function ProductData({ id }) {
  const queryClient = new QueryClient()

  // Not awaited, so rendering is not blocked.
  void queryClient.prefetchQuery({
    ...productCache.options(id),
    queryFn: () => getProduct(id),
  })

  return (
    <HydrationBoundary
      state={dehydrate(queryClient, {
        shouldDehydrateQuery: (query) =>
          defaultShouldDehydrateQuery(query) ||
          query.state.status === 'pending',
      })}
    >
      <ProductView id={id} />
    </HydrationBoundary>
  )
}
```

The server and Client Component must use the same query key. Keep the key and query options together so both call sites share the same identity:

```
// Archivo: app/products/[id]/product-cache.ts switcher
import { queryOptions } from '@tanstack/react-query'

export type Product = { id: string; name: string }

export const productCache = {
  key: (id: string) => ['product', id] as const,
  options: (id: string) =>
    queryOptions({
      queryKey: productCache.key(id),
      queryFn: async (): Promise<Product> => {
        const res = await fetch(`/api/products/${id}`)
        if (!res.ok) throw new Error('Failed to fetch product')
        return res.json()
      },
      staleTime: 30_000,
    }),
}
```

```
// Archivo: app/products/[id]/product-cache.js switcher
import { queryOptions } from '@tanstack/react-query'

export const productCache = {
  key: (id) => ['product', id],
  options: (id) =>
    queryOptions({
      queryKey: productCache.key(id),
      queryFn: async () => {
        const res = await fetch(`/api/products/${id}`)
        if (!res.ok) throw new Error('Failed to fetch product')
        return res.json()
      },
      staleTime: 30_000,
    }),
}
```

The query function fetches a Route Handler so it can run on the client. The `staleTime` prevents an immediate client refetch by keeping the hydrated data fresh for 30 seconds. Choose a duration based on how quickly the data can change.

Larger features can place query options in a separate client-facing module as long as every caller imports the key from the same cache contract. Learn more about TanStack Query defaults.

As with SWR, `params.then()` resolves the `id` inside `<Suspense>`, and `ProductData` prefetches below the boundary.

The Client Component reads the same query key with `useSuspenseQuery` (or `useQuery` to render `isPending` and `error` states inline):

```
// Archivo: app/products/[id]/product-view.tsx switcher
'use client'

import { useSuspenseQuery } from '@tanstack/react-query'
import { productCache } from './product-cache'

export function ProductView({ id }: { id: string }) {
  const { data } = useSuspenseQuery(productCache.options(id))

  return <h1>{data.name}</h1>
}
```

```
// Archivo: app/products/[id]/product-view.js switcher
'use client'

import { useSuspenseQuery } from '@tanstack/react-query'
import { productCache } from './product-cache'

export function ProductView({ id }) {
  const { data } = useSuspenseQuery(productCache.options(id))

  return <h1>{data.name}</h1>
}
```

Learn more: TanStack Query Advanced SSR.

Cache server-provided data with Cache Components

Enable `cacheComponents` in `next.config.ts` before using this pattern. You can then cache the server data provided to TanStack Query. Add `use cache`, choose a `cacheLife` profile, and apply `cacheTag` so mutations can invalidate it:

```
// Archivo: app/products/[id]/data.ts switcher
import { cacheLife, cacheTag } from 'next/cache'
import type { Product } from './product-cache'

export async function getProduct(id: string): Promise<Product> {
  'use cache'
  cacheLife('max')
  cacheTag(`product:${id}`)

  const product = await db.product.findUnique({ where: { id } })
  if (!product) throw new Error('Product not found')
  return product
}
```

```
// Archivo: app/products/[id]/data.js switcher
import { cacheLife, cacheTag } from 'next/cache'

export async function getProduct(id) {
  'use cache'
  cacheLife('max')
  cacheTag(`product:${id}`)

  const product = await db.product.findUnique({ where: { id } })
  if (!product) throw new Error('Product not found')
  return product
}
```

This example uses `cacheLife('max')` because writes invalidate the product tag. Within the cache profile, `stale` controls how long the Next.js client cache can reuse a prefetched payload, while `revalidate` and `expire` control the server cache. Choose a shorter profile when the server value should refresh with time.

TanStack Query owns a separate browser cache, so its `staleTime` does not need to match `cacheLife`.

With Cache Components enabled, Next.js also prerenders Client Components. Keep a query needed during the initial render behind Suspense. TanStack Query can read the current time while creating active query state, and the boundary lets Next.js defer that work instead of raising a current-time prerender error.

When the same mutation updates the browser cache and invalidates cached server data, you may define both identities in one shared contract:

```
// Archivo: app/products/[id]/product-cache.ts switcher
 export const productCache = {
   key: (id: string) => ['product', id] as const,
+  tag: (id: string) => `product:${id}`,
   // ...
 }
```

```
// Archivo: app/products/[id]/product-cache.js switcher
 export const productCache = {
   key: (id) => ['product', id],
+  tag: (id) => `product:${id}`,
   // ...
 }
```

The server function can then call `cacheTag(productCache.tag(id))`. Keep this contract free of server-only and client-only imports so both cache layers can reuse it.

> **Good to know:** TanStack Query's `dehydrate()` reads the current time during Cache Components prerendering. Use the prerenderable hydration helper for cached initial data.

Coordinate server and client caches after mutations

Hydration provides the initial client cache value. After hydration, TanStack Query owns the browser copy and its revalidation. For reused data, keep the query key and server tag in the same cache contract:

```
// Archivo: app/activity/activity-cache.ts switcher
export const activityCache = {
  key: ['activity', 'unread'] as const,
  tag: (userId: string) => `activity:${userId}`,
}
```

```
// Archivo: app/activity/activity-cache.js switcher
export const activityCache = {
  key: ['activity', 'unread'],
  tag: (userId) => `activity:${userId}`,
}
```

On mutation, use `useMutation`'s `onMutate` callback to update the cache immediately and restore the previous value in `onError` if the write fails. This example keeps the optimistic value after the action succeeds because the final value is known:

```
// Archivo: app/activity/mark-read-button.tsx switcher
'use client'

import { useMutation, useQueryClient } from '@tanstack/react-query'
import { markActivityReadAction } from './actions'
import { activityCache } from './activity-cache'

export function MarkReadButton() {
  const queryClient = useQueryClient()
  const queryKey = activityCache.key

  const markRead = useMutation({
    mutationFn: markActivityReadAction,
    onMutate: async () => {
      await queryClient.cancelQueries({ queryKey })
      const previous = queryClient.getQueryData(queryKey)
      queryClient.setQueryData(queryKey, { count: 0 })
      return { previous }
    },
    onError: (_error, _variables, context) => {
      queryClient.setQueryData(queryKey, context?.previous)
    },
  })

  return <button onClick={() => markRead.mutate()}>Mark read</button>
}
```

```
// Archivo: app/activity/mark-read-button.js switcher
'use client'

import { useMutation, useQueryClient } from '@tanstack/react-query'
import { markActivityReadAction } from './actions'
import { activityCache } from './activity-cache'

export function MarkReadButton() {
  const queryClient = useQueryClient()
  const queryKey = activityCache.key

  const markRead = useMutation({
    mutationFn: markActivityReadAction,
    onMutate: async () => {
      await queryClient.cancelQueries({ queryKey })
      const previous = queryClient.getQueryData(queryKey)
      queryClient.setQueryData(queryKey, { count: 0 })
      return { previous }
    },
    onError: (_error, _variables, context) => {
      queryClient.setQueryData(queryKey, context?.previous)
    },
  })

  return <button onClick={() => markRead.mutate()}>Mark read</button>
}
```

The Server Action writes to the database and expires the tagged server data with `updateTag`. Tag the cached server query with `cacheTag` so the tag matches:

```
// Archivo: app/activity/actions.ts switcher
'use server'

import { updateTag } from 'next/cache'
import {
  getCurrentUserId,
  markActivityRead as markActivityReadInDatabase,
} from './data'
import { activityCache } from './activity-cache'

export async function markActivityReadAction() {
  const userId = await getCurrentUserId()
  await markActivityReadInDatabase(userId)
  updateTag(activityCache.tag(userId))
}
```

```
// Archivo: app/activity/actions.js switcher
'use server'

import { updateTag } from 'next/cache'
import {
  getCurrentUserId,
  markActivityRead as markActivityReadInDatabase,
} from './data'
import { activityCache } from './activity-cache'

export async function markActivityReadAction() {
  const userId = await getCurrentUserId()
  await markActivityReadInDatabase(userId)
  updateTag(activityCache.tag(userId))
}
```

The optimistic query value updates the current screen. `updateTag` ensures the next cached server read returns fresh activity.

> **Good to know:** Call `updateTag` when a Server Action changes a cached read that must reflect the write immediately. An uncached read does not have a server tag to update. See Client-side data fetching for other invalidation behaviors.

Build a prerenderable hydration state

The TanStack Query pattern above uses `dehydrate()` to create the hydration state. With Cache Components, `dehydrate()` reads the current time (`Date.now()`) while prerendering and causes a current-time prerender error.

Instead, cache only that timestamp and build the dehydrated state by hand. Wrap the time read in `use cache` with the same tags as the data reads. When a mutation invalidates those tags, the data and its timestamp advance together, so `<HydrationBoundary>` overwrites the client query on the next navigation:

```
// Archivo: app/lib/hydrate.ts switcher
import 'server-only'

import { cacheLife, cacheTag } from 'next/cache'
import {
  defaultShouldDehydrateQuery,
  QueryClient,
  type DehydratedState,
  type QueryKey,
} from '@tanstack/react-query'

type HydratedQuery = {
  queryKey: QueryKey
  data: unknown
}

type HydrationOptions = {
  tags: string[]
}

async function getHydrationUpdatedAt(tags: string[]) {
  'use cache'
  cacheTag( ... tags)
  cacheLife('max')
  return Date.now()
}

export async function dehydrate(
  queries: HydratedQuery[],
  options: HydrationOptions
): Promise<DehydratedState> {
  const updatedAt = await getHydrationUpdatedAt(options.tags)

  const queryClient = new QueryClient()

  for (const query of queries) {
    queryClient.setQueryData(query.queryKey, query.data, { updatedAt })
  }

  return {
    mutations: [],
    queries: queryClient
      .getQueryCache()
      .getAll()
      .filter((query) => defaultShouldDehydrateQuery(query))
      .map((query) => ({
        dehydratedAt: updatedAt,
        queryHash: query.queryHash,
        queryKey: query.queryKey,
        state: query.state,
        ... (query.meta ? { meta: query.meta } : {}),
      })),
  }
}
```

```
// Archivo: app/lib/hydrate.js switcher
import 'server-only'

import { cacheLife, cacheTag } from 'next/cache'
import { defaultShouldDehydrateQuery, QueryClient } from '@tanstack/react-query'

async function getHydrationUpdatedAt(tags) {
  'use cache'
  cacheTag( ... tags)
  cacheLife('max')
  return Date.now()
}

export async function dehydrate(queries, options) {
  const updatedAt = await getHydrationUpdatedAt(options.tags)

  const queryClient = new QueryClient()

  for (const query of queries) {
    queryClient.setQueryData(query.queryKey, query.data, { updatedAt })
  }

  return {
    mutations: [],
    queries: queryClient
      .getQueryCache()
      .getAll()
      .filter((query) => defaultShouldDehydrateQuery(query))
      .map((query) => ({
        dehydratedAt: updatedAt,
        queryHash: query.queryHash,
        queryKey: query.queryKey,
        state: query.state,
        ... (query.meta ? { meta: query.meta } : {}),
      })),
  }
}
```

Await the helper in the segment that owns the data, passing the data and the same tags used on the underlying `getProduct` read. Hand the result to `<HydrationBoundary>` inside `ProductData`:

```
// Archivo: app/products/[id]/page.tsx switcher
import { HydrationBoundary } from '@tanstack/react-query'
import { dehydrate } from '@/app/lib/hydrate'
import { getProduct } from './data'
import { productCache } from './product-cache'
import { ProductView } from './product-view'

async function ProductData({ id }: { id: string }) {
  const product = await getProduct(id)
  const state = await dehydrate(
    [{ queryKey: productCache.key(id), data: product }],
    { tags: [productCache.tag(id)] }
  )

  return (
    <HydrationBoundary state={state}>
      <ProductView id={id} />
    </HydrationBoundary>
  )
}
```

```
// Archivo: app/products/[id]/page.js switcher
import { HydrationBoundary } from '@tanstack/react-query'
import { dehydrate } from '@/app/lib/hydrate'
import { getProduct } from './data'
import { productCache } from './product-cache'
import { ProductView } from './product-view'

async function ProductData({ id }) {
  const product = await getProduct(id)
  const state = await dehydrate(
    [{ queryKey: productCache.key(id), data: product }],
    { tags: [productCache.tag(id)] }
  )

  return (
    <HydrationBoundary state={state}>
      <ProductView id={id} />
    </HydrationBoundary>
  )
}
```

> **Good to know:** The timestamp must advance whenever the hydrated data changes. The helper suits tag-driven server data because both use the same tags. For time-driven server data, derive the data and hydration timestamp from the same cached snapshot instead of maintaining unrelated time windows.

See the live `next-spa-patterns` demo and its source code. Learn more about TanStack Query optimistic updates and the TanStack Query documentation.

Incremental Static Regeneration with Cache Components

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/incremental-static-regeneration-cache-components
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to prerender a subset of dynamic routes, serve App Shells for the rest, and upgrade them after the first visit.*

Incremental Static Regeneration (ISR) with `cacheComponents` and Partial Prefetching gives every route an instant first visit, even for URLs that weren't included in the build.

During build, Partial Prerendering splits each render into two parts:

- The **App Shell**: the generic, reusable part of the page that doesn't depend on URL data
- The rest of the statically renderable content: the param-specific prerenders for the URLs you list in `generateStaticParams`

For a visit to a URL whose params were included in `generateStaticParams`, Next.js serves the fully prerendered page from the cache. For a visit to a URL whose params weren't, Next.js serves the App Shell instantly, then upgrades it in the background with the now-known params. Subsequent visits to that URL get the upgraded result from the cache, skipping the App Shell entirely.

If you have used ISR or `fallback: true` in the Pages Router, this is the Cache Components equivalent.

Enable both `cacheComponents` and Partial Prefetching. Cache Components produces the App Shell, while Partial Prefetching upgrades it to a full route once the params are known.

```
// Archivo: next.config.ts highlight={4-5}
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
  partialPrefetching: true,
}

export default nextConfig
```

Example

We'll build a product catalog with category layouts and product detail pages using the Recipe API. You can find the resources used in this example here:

- Demo
- Code

Prepare your routes

Use `generateStaticParams` to define which param values to prerender. When rendering these routes, the param values are known at build time. The prerender process builds a static shell until it hits runtime APIs or uncached data. See Prerendering for more details.

The category layout prerenders two categories:

```
// Archivo: app/[category]/layout.tsx highlight={5-8}
import Link from 'next/link'
import { Suspense } from 'react'
import { getCategory, getTopCategories } from '../lib/data'

export async function generateStaticParams() {
  const categories = await getTopCategories()
  return categories.map((c) => ({ category: c.slug }))
}

async function CategoryHeader({
  params,
}: Pick<LayoutProps<'/[category]'>, 'params'>) {
  const { category } = await params
  const data = await getCategory(category)

  return (
    <div>
      [&larr; All categories](/)
      <h1>{data?.name ?? 'Category'}</h1>
      {data?.description && <p>{data.description}</p>}
    </div>
  )
}

export default function CategoryLayout(props: LayoutProps<'/[category]'>) {
  return (
    <div>
      ``<Suspense>``
      {props.children}
    </div>
  )
}
```

Notice that `CategoryLayout` does not `await props.params` itself. Instead, it passes the `params` promise to `CategoryHeader` inside `````) }

````
> **Good to know:** You can also use [`loading.tsx`](/docs/app/api-reference/file-conventions/loading) files instead of
``<Suspense>`` in JSX. `loading.tsx` puts the boundary at the segment edge, while inline ``<Suspense>`` lets you place it
anywhere in the tree.

The data helpers use ''use cache'' at the module level so all exported functions are cached. This lets their results be
included in the static shell:

```ts
// Archivo: app/lib/data.ts highlight={1}
'use cache'

const API = 'https://next-recipe-api.vercel.dev'

export async function getCategory(slug: string) {
  const res = await fetch(`${API}/categories/${slug}`)
  if (!res.ok) return null
  return res.json()
}

export async function getProduct(category: string, slug: string) {
  const res = await fetch(`${API}/products/${category}/${slug}`)
  if (!res.ok) return null
  return res.json()
}

export async function getTopCategories() {
  const res = await fetch(`${API}/categories`)
  const categories = await res.json()
  return categories.slice(0, 2)
}

export async function getPopularProducts(category: string) {
  const res = await fetch(`${API}/products?category=${category}`)
  const products = await res.json()
  return products.slice(0, 1)
}
```
````

If your components access runtime APIs like `cookies` or `headers`, wrap them in `<Suspense>`. Their fallback UI is included in the static shell instead.

At build time

When you run `next build`, Next.js prerenders the layout for each known category (`tops`, `shorts`), plus one render where `await params` suspends, producing the App Shell for `[category]`.

It also prerenders the page for each known product under each category (`tee` under `tops`, `joggers` under `shorts`), plus one render where `await params` suspends, producing the App Shell for `[product]`.

These are combined into:

- `/tops/tee`, `/shorts/joggers`: fully static pages, both params known
- `/tops/[product]`, `/shorts/[product]`: category header rendered, product shows fallback
- `/[category]/[product]`: both show fallback

At runtime

A visitor navigates to `/tops/tee`. Both params were prerendered. They get a fully static page.

The first visit to `/tops/overshirt`. The product `overshirt` is unknown, but the category `tops` was prerendered. Next.js serves the App Shell for `/tops/[product]` with the category header already rendered. The product streams in.

The first visit to `/shoes/basketball-shoes`. Neither param was prerendered. Next.js serves the generic App Shell for `/[category]/[product]`. Both the category and the product stream in.

After the first visit, Next.js renders these routes in the background with the now-known params. The next visitor to the same URLs gets the upgraded result.

A prefetch counts as that first visit. When a `<Link>` to an unlisted URL enters the viewport, or you call `router.prefetch`, Next.js starts the background upgrade before the click, so navigation lands on the upgraded result.

> **Good to know**: The App Shell for unlisted params is served from Next.js 16.3. Earlier versions wait for a full server render before sending the response.

What the upgrade produces

After the first visit, Next.js renders the page in the background with the known params and tries to push the static boundary as far down the component tree as possible:

- If every data access is cached and all params are resolved, the upgrade produces a **fully static page**.
- If all params are resolved but the render still hits uncached data or runtime APIs (`cookies`, `headers`) wrapped in `<Suspense>` boundaries, the upgrade produces a **cached page with those fallbacks**. The uncached or runtime parts stream in at request time.
- Params are resolved in route order. A param value not returned by `generateStaticParams` stays unresolved and prevents any deeper params from upgrading.

Choosing what to prerender

Not every route needs to be prerendered. Every page you prerender increases build work and produces output that has to be stored and deployed. Many routes may never be visited before your next deployment, making that work unnecessary.

Instead, use `generateStaticParams` to prerender the routes that benefit most from being ready ahead of time, such as popular pages or predictable content. Less frequently visited routes are generated on demand and upgraded after their first visit, so you don't spend build time and storage on pages that may never be requested.

Coming from the Pages Router

If you are migrating from the Pages Router:

- `fallback: true` in `getStaticPaths` is now the default behavior with `cacheComponents`. Visitors get a `<Suspense>` fallback instantly and content streams in.
- `router.isFallback` is not needed. The prerendering step generates a static shell, and you can grow it further with `'use cache'`.
- `getStaticProps` with `revalidate` maps to `'use cache'` with `cacheLife`.
- `getStaticPaths` maps to `generateStaticParams`.

Next steps

- Caching with Cache Components for the full caching model
- `generateStaticParams` for controlling which param combinations are prerendered
- `loading.tsx` for providing skeleton UI in App Shells
- Streaming to learn how to progressively render UI as data becomes available
- Self-hosting to keep an existing ISR `cacheHandler` alongside `cacheHandlers` for `'use cache'`

How to implement Incremental Static Regeneration (ISR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/incremental-static-regeneration
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to create or update static pages at runtime with Incremental Static Regeneration.*

Examples

> **Good to know**: This guide covers ISR without Cache Components. If you are using `cacheComponents`, see ISR with Cache Components instead.

Incremental Static Regeneration (ISR) enables you to:

- Update static content without rebuilding the entire site
- Reduce server load by serving prerendered, static pages for most requests
- Ensure proper `cache-control` headers are automatically added to pages
- Handle large amounts of content pages without long `next build` times

Here's a minimal example:

```
// Archivo: app/blog/[id]/page.tsx switcher
interface Post {
  id: string
  title: string
  content: string
}

// Next.js will invalidate the cache when a
// request comes in, at most once every 60 seconds.
export const revalidate = 60

export async function generateStaticParams() {
  const posts: Post[] = await fetch('https://api.vercel.app/blog').then((res) =>
    res.json()
  )
  return posts.map((post) => ({
    id: String(post.id),
  }))
}

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const post: Post = await fetch(`https://api.vercel.app/blog/${id}`).then(
    (res) => res.json()
  )
  return (
    <main>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </main>
  )
}
```

```
// Archivo: app/blog/[id]/page.jsx switcher
// Next.js will invalidate the cache when a
// request comes in, at most once every 60 seconds.
export const revalidate = 60

export async function generateStaticParams() {
  const posts = await fetch('https://api.vercel.app/blog').then((res) =>
    res.json()
  )
  return posts.map((post) => ({
    id: String(post.id),
  }))
}

export default async function Page({ params }) {
  const { id } = await params
  const post = await fetch(`https://api.vercel.app/blog/${id}`).then((res) =>
    res.json()
  )
  return (
    <main>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </main>
  )
}
```

Here's how this example works:

1. `generateStaticParams` enables ISR for the dynamic route by returning the list of posts to prerender
2. During `next build`, a page is prerendered for each post
3. All requests made to these pages (e.g. `/blog/1`) are cached and instantaneous
4. After 60 seconds has passed, the next request will still return the cached (now stale) page
5. The cache is invalidated and a new version of the page begins generating in the background
6. Once generated successfully, the next request will return the updated page and cache it for subsequent requests
7. If `/blog/26` is requested, and it exists, the page will be generated on-demand. This behavior can be changed by using a different dynamicParams value. However, if the post does not exist, then 404 is returned.

📘 **Nota para Pages Router:**

```
import type { GetStaticPaths, GetStaticProps } from 'next'

interface Post {
  id: string
  title: string
  content: string
}

interface Props {
  post: Post
}

export const getStaticPaths: GetStaticPaths = async () => {
  const posts = await fetch('https://api.vercel.app/blog').then((res) =>
    res.json()
  )
  const paths = posts.map((post: Post) => ({
    params: { id: String(post.id) },
  }))

  return { paths, fallback: 'blocking' }
}

export const getStaticProps: GetStaticProps<Props> = async ({
  params,
}: {
  params: { id: string }
}) => {
  const post = await fetch(`https://api.vercel.app/blog/${params.id}`).then(
    (res) => res.json()
  )

  return {
    props: { post },
    // Next.js will invalidate the cache when a
    // request comes in, at most once every 60 seconds.
    revalidate: 60,
  }
}

export default function Page({ post }: Props) {
  return (
    <main>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </main>
  )
}
```

```
export async function getStaticPaths() {
  const posts = await fetch('https://api.vercel.app/blog').then((res) =>
    res.json()
  )
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  return { paths, fallback: 'blocking' }
}

export async function getStaticProps({ params }) {
  const post = await fetch(`https://api.vercel.app/blog/${params.id}`).then(
    (res) => res.json()
  )

  return {
    props: { post },
    // Next.js will invalidate the cache when a
    // request comes in, at most once every 60 seconds.
    revalidate: 60,
  }
}

export default function Page({ post }) {
  return (
    <main>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </main>
  )
}
```

Here's how this example works:

1. During `next build`, all known blog posts are generated
2. All requests made to these pages (e.g. `/blog/1`) are cached and instantaneous
3. After 60 seconds has passed, the next request will still return the cached (now stale) page
4. The cache is invalidated and a new version of the page begins generating in the background
5. Once generated successfully, the next request will return the updated page and cache it for subsequent requests
6. If `/blog/26` is requested, and it exists, the page will be generated on-demand. This behavior can be changed by using a different fallback value. However, if the post does not exist, then 404 is returned.

Reference

Route segment config

- `revalidate`
- `dynamicParams`

Functions

- `generateStaticParams`
- `revalidatePath`
- `revalidateTag`

> 📘 **Nota para Pages Router:**
>
> ### Functions
>
> - `getStaticProps`
> - `res.revalidate`

Examples

Time-based revalidation

This fetches and displays a list of blog posts on /blog. After an hour has passed, the next visitor will still receive the cached (stale) version of the page immediately for a fast response. Simultaneously, Next.js triggers regeneration of a fresh version in the background. Once the new version is successfully generated, it replaces the cached version, and subsequent visitors will receive the updated content.

```
// Archivo: app/blog/page.tsx switcher
interface Post {
  id: string
  title: string
  content: string
}

export const revalidate = 3600 // invalidate every hour

export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts: Post[] = await data.json()
  return (
    <main>
      <h1>Blog Posts</h1>
      <ul>
        {posts.map((post) => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </main>
  )
}
```

```
// Archivo: app/blog/page.js switcher
export const revalidate = 3600 // invalidate every hour

export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()
  return (
    <main>
      <h1>Blog Posts</h1>
      <ul>
        {posts.map((post) => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </main>
  )
}
```

We recommend setting a high revalidation time. For instance, 1 hour instead of 1 second. If you need more precision, consider using on-demand revalidation. If you need real-time data, consider switching to dynamic rendering.

On-demand revalidation with `revalidatePath`

For a more precise method of revalidation, invalidate cached pages on-demand with the `revalidatePath` function.

For example, this Server Action would get called after adding a new post. Regardless of how you retrieve your data in your Server Component, either using `fetch` or connecting to a database, this will invalidate the cache for the entire route. The next request to that route will trigger regeneration and serve fresh data, which will then be cached for subsequent requests.

> **Note:** `revalidatePath` invalidates the cache entries but regeneration happens on the next request. If you want to eagerly regenerate the cache entry immediately instead of waiting for the next request, you can use the Pages router `res.revalidate` method. We're working on adding new methods to provide eager regeneration capabilities for the App Router.

```
// Archivo: app/actions.ts switcher
'use server'

import { revalidatePath } from 'next/cache'

export async function createPost() {
  // Invalidate the cache for the /posts route
  revalidatePath('/posts')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { revalidatePath } from 'next/cache'

export async function createPost() {
  // Invalidate the cache for the /posts route
  revalidatePath('/posts')
}
```

View a demo and explore the source code.

On-demand revalidation with `revalidateTag`

For most use cases, prefer revalidating entire paths. If you need more granular control, you can use the `revalidateTag` function. For example, you can tag individual `fetch` calls:

```
// Archivo: app/blog/page.tsx switcher
export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog', {
    next: { tags: ['posts'] },
  })
  const posts = await data.json()
  // ...
}
```

```
// Archivo: app/blog/page.js switcher
export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog', {
    next: { tags: ['posts'] },
  })
  const posts = await data.json()
  // ...
}
```

If you are using an ORM or connecting to a database, you can use `unstable_cache`:

```tsx
// Archivo: app/blog/page.tsx switcher
import { unstable_cache } from 'next/cache'
import { db, posts } from '@/lib/db'

const getCachedPosts = unstable_cache(
  async () => {
    return await db.select().from(posts)
  },
  ['posts'],
  { revalidate: 3600, tags: ['posts'] }
)

export default async function Page() {
  const posts = getCachedPosts()
  // ...
}
```

```jsx
// Archivo: app/blog/page.js switcher
import { unstable_cache } from 'next/cache'
import { db, posts } from '@/lib/db'

const getCachedPosts = unstable_cache(
  async () => {
    return await db.select().from(posts)
  },
  ['posts'],
  { revalidate: 3600, tags: ['posts'] }
)

export default async function Page() {
  const posts = getCachedPosts()
  // ...
}
```

You can then use `revalidateTag` in a Server Actions or Route Handler:

```tsx
// Archivo: app/actions.ts switcher
'use server'

import { revalidateTag } from 'next/cache'

export async function createPost() {
  // Invalidate all data tagged with 'posts'
  revalidateTag('posts', 'max')
}
```

```jsx
// Archivo: app/actions.js switcher
'use server'

import { revalidateTag } from 'next/cache'

export async function createPost() {
  // Invalidate all data tagged with 'posts'
  revalidateTag('posts', 'max')
}
```

📘 **Nota para Pages Router:**

On-demand validation with `res.revalidate()`

For a more precise method of revalidation, use `res.revalidate` to generate a new page on-demand from an API Router.

For example, this API Route can be called at `/api/revalidate?secret=<token>` to revalidate a given blog post. Create a secret token only known by your Next.js app. This secret will be used to prevent unauthorized access to the revalidation API Route.

```
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  // Check for secret to confirm this is a valid request
  if (req.query.secret !== process.env.MY_SECRET_TOKEN) {
    return res.status(401).json({ message: 'Invalid token' })
  }

  try {
    // This should be the actual path not a rewritten path
    // e.g. for "/posts/[id]" this should be "/posts/1"
    await res.revalidate('/posts/1')
    return res.json({ revalidated: true })
  } catch (err) {
    // If there was an error, Next.js will continue
    // to show the last successfully generated page
    return res.status(500).send('Error revalidating')
  }
}
```

```
export default async function handler(req, res) {
  // Check for secret to confirm this is a valid request
  if (req.query.secret !== process.env.MY_SECRET_TOKEN) {
    return res.status(401).json({ message: 'Invalid token' })
  }

  try {
    // This should be the actual path not a rewritten path
    // e.g. for "/posts/[id]" this should be "/posts/1"
    await res.revalidate('/posts/1')
    return res.json({ revalidated: true })
  } catch (err) {
    // If there was an error, Next.js will continue
    // to show the last successfully generated page
    return res.status(500).send('Error revalidating')
  }
}
```

If you are using on-demand revalidation, you do not need to specify a `revalidate` time inside of `getStaticProps`. Next.js will use the default value of `false` (no revalidation) and only revalidate the page on-demand when `res.revalidate()` is called.

Handling uncaught exceptions

If an error is thrown while attempting to revalidate data, the last successfully generated data will continue to be served from the cache. On the next subsequent request, Next.js will retry revalidating the data. Learn more about error handling.

📘 **Nota para Pages Router:** If there is an error inside `getStaticProps` when handling background regeneration, or you manually throw an error, the last successfully generated page will continue to show. On the next subsequent request, Next.js will retry calling `getStaticProps`.

```
import type { GetStaticProps } from 'next'

interface Post {
  id: string
  title: string
  content: string
}

interface Props {
  post: Post
}

export const getStaticProps: GetStaticProps<Props> = async ({
  params,
}: {
  params: { id: string }
}) => {
  // If this request throws an uncaught error, Next.js will
  // not invalidate the currently shown page and
  // retry getStaticProps on the next request.
  const res = await fetch(`https://api.vercel.app/blog/${params.id}`)
  const post: Post = await res.json()

  if (!res.ok) {
    // If there is a server error, you might want to
    // throw an error instead of returning so that the cache is not updated
    // until the next successful request.
    throw new Error(`Failed to fetch posts, received status ${res.status}`)
  }

  return {
    props: { post },
    // Next.js will invalidate the cache when a
    // request comes in, at most once every 60 seconds.
    revalidate: 60,
  }
}
```

```
export async function getStaticProps({ params }) {
  // If this request throws an uncaught error, Next.js will
  // not invalidate the currently shown page and
  // retry getStaticProps on the next request.
  const res = await fetch(`https://api.vercel.app/blog/${params.id}`)
  const post = await res.json()

  if (!res.ok) {
    // If there is a server error, you might want to
    // throw an error instead of returning so that the cache is not updated
    // until the next successful request.
    throw new Error(`Failed to fetch posts, received status ${res.status}`)
  }

  return {
    props: { post },
    // Next.js will invalidate the cache when a
    // request comes in, at most once every 60 seconds.
    revalidate: 60,
  }
}
```

Customizing the cache location

You can configure the Next.js cache location if you want to persist cached pages and data to durable storage, or share the cache across multiple containers or instances of your Next.js application. Learn more.

Troubleshooting

Debugging cached data in local development

If you are using the `fetch` API, you can add additional logging to understand which requests are cached or uncached. Learn more about the `logging` option.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    fetches: {
      fullUrl: true,
    },
  },
}
```

Verifying correct production behavior

To verify your pages are cached and revalidated correctly in production, you can test locally by running `next build` and then `next start` to run the production Next.js server.

This will allow you to test ISR behavior as it would work in a production environment. For further debugging, add the following environment variable to your `.env` file:

```
// Archivo: .env
NEXT_PRIVATE_DEBUG_CACHE=1
```

This will make the Next.js server console log ISR cache hits and misses. You can inspect the output to see which pages are generated during `next build`, as well as how pages are updated as paths are accessed on-demand.

Caveats

- ISR is only supported when using the Node.js runtime (default).
- ISR is not supported when creating a Static Export.
- If you have multiple `fetch` requests in a prerendered route, and each has a different `revalidate` frequency, the lowest time will be used for ISR. However, those revalidate frequencies will still be respected by the cache.
- If any of the `fetch` requests used on a route have a `revalidate` time of `0`, or an explicit `no-store`, the route will be dynamically rendered.
- Proxy won't be executed for on-demand ISR requests, meaning any path rewrites or logic in Proxy will not be applied. Ensure you are revalidating the exact path. For example, `/post/1` instead of a rewritten `/post-1`.
- When running multiple instances, the default file-system cache is per-instance. On-demand revalidation only invalidates the instance that receives the call. Use a shared custom cache handler to coordinate across instances. See How Revalidation Works for the full architecture.
- Background regeneration (stale-while-revalidate) runs on the instance that receives the triggering request. On platforms with per-request billing, this background work counts as additional compute.
- You can use the `x-nextjs-cache` response header to observe cache behavior. Values are `HIT` (served from cache), `STALE` (served from cache, revalidating in background), `MISS` (not in cache, rendered fresh), or `REVALIDATED` (regenerated via on-demand revalidation).

> 📘 **Nota para Pages Router:**
>
> - ISR is only supported when using the Node.js runtime (default).
> - ISR is not supported when creating a Static Export.
> - Proxy won't be executed for on-demand ISR requests, meaning any path rewrites or logic in Proxy will not be applied. Ensure you are revalidating the exact path. For example, `/post/1` instead of a rewritten `/post-1`.

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Learn how to configure ISR when self-hosting Next.js.

Version history

Version	Changes
`v14.1.0`	Custom `cacheHandler` is stable.
`v13.0.0`	App Router is introduced.
`v12.2.0`	Pages Router: On-Demand ISR is stable
`v12.0.0`	Pages Router: Bot-aware ISR fallback added.
`v9.5.0`	Pages Router: Stable ISR introduced.

How to configure Continuous Integration (CI) build caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/ci-build-caching
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to configure CI to cache Next.js builds*

How to implement Incremental Static Regeneration (ISR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/incremental-static-regeneration
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to create or update static pages at runtime with Incremental Static Regeneration.*

Capítulo 5: Server Actions, Formularios y Mutación de Datos

> *Manejo de mutaciones con Server Actions, estados pendientes, UI optimista, manejo de errores y validación.*

Mutating Data

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/mutating-data
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to mutate data using Server Functions and Server Actions in Next.js.*

You can mutate data in Next.js using React Server Functions. This page will go through how you can create and invoke Server Functions. For Next.js-specific behaviors (single-roundtrip response, sequential dispatch, security, deployment), see Server Actions and Mutations.

What are Server Functions?

A **Server Function** is an asynchronous function that runs on the server. You can call them from the client through a network request, which is why they must be asynchronous.

In an `action` or mutation context, they are also called **Server Actions**.

By convention, a Server Action is an async function used with `startTransition`. This happens automatically when the function is:

- Passed to a `<form>` using the `action` prop.
- Passed to a `<button>` using the `formAction` prop.

When an action is invoked, Next.js can return both the updated UI and new data in a single server roundtrip.

Behind the scenes, actions use the `POST` method, and only this HTTP method can invoke them.

> [!WARNING] Server Functions are reachable via direct POST requests, not just through your application's UI. Always verify authentication and authorization inside every Server Function. See the Data Security guide for recommended patterns.

> **Good to know:** A Server Action is a Server Function used in a specific way (for handling form submissions and mutations). Server Function is the broader term.

Creating Server Functions

A Server Function can be defined by using the `use server` directive. You can place the directive at the top of an **asynchronous** function to mark the function as a Server Function, or at the top of a separate file to mark all exports of that file.

```ts
// Archivo: app/lib/actions.ts switcher
import { auth } from '@/lib/auth'

export async function createPost(formData: FormData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const title = formData.get('title')
  const content = formData.get('content')

  // Mutate data
  // Revalidate cache
}

export async function deletePost(formData: FormData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const id = formData.get('id')

  // Verify the user owns this resource before deleting
  // Mutate data
  // Revalidate cache
}
```

```js
// Archivo: app/lib/actions.js switcher
import { auth } from '@/lib/auth'

export async function createPost(formData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const title = formData.get('title')
  const content = formData.get('content')

  // Mutate data
  // Revalidate cache
}

export async function deletePost(formData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const id = formData.get('id')

  // Verify the user owns this resource before deleting
  // Mutate data
  // Revalidate cache
}
```

Server Components

Server Functions can be inlined in Server Components by adding the `"use server"` directive to the top of the function body:

```
// Archivo: app/page.tsx switcher
export default function Page() {
  // Server Action
  async function createPost(formData: FormData) {
    'use server'
    // ...
  }

  return <></>
}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  // Server Action
  async function createPost(formData) {
    'use server'
    // ...
  }

  return <></>
}
```

> **Good to know:** Server Components support progressive enhancement by default, meaning forms that call Server Actions will be submitted even if JavaScript hasn't loaded yet or is disabled.

Client Components

It's not possible to define Server Functions in Client Components. However, you can invoke them in Client Components by importing them from a file that has the `"use server"` directive at the top of it:

```
// Archivo: app/actions.ts switcher
'use server'

export async function createPost() {}
```

```
// Archivo: app/actions.js switcher
'use server'

export async function createPost() {}
```

```
// Archivo: app/ui/button.tsx switcher
'use client'

import { createPost } from '@/app/actions'

export function Button() {
  return <button formAction={createPost}>Create</button>
}
```

```
// Archivo: app/ui/button.js switcher
'use client'

import { createPost } from '@/app/actions'

export function Button() {
  return <button formAction={createPost}>Create</button>
}
```

> **Good to know:** In Client Components, forms invoking Server Actions will queue submissions if JavaScript isn't loaded yet, and will be prioritized for hydration. After hydration, the browser does not refresh on form submission.

Passing actions as props

You can also pass an action to a Client Component as a prop:

```
<ClientComponent updateItemAction={updateItem} />
```

```
// Archivo: app/client-component.tsx switcher
'use client'

export default function ClientComponent({
  updateItemAction,
}: {
  updateItemAction: (formData: FormData) => void
}) {
  return <form action={updateItemAction}></form>
}
```

```
// Archivo: app/client-component.js switcher
'use client'

export default function ClientComponent({ updateItemAction }) {
  return <form action={updateItemAction}></form>
}
```

Invoking Server Functions

There are two main ways you can invoke a Server Function:

1. Forms in Server and Client Components
2. Event Handlers and useEffect in Client Components

> **Good to know:** Server Functions are designed for server-side mutations. The client currently dispatches and awaits them one at a time. This is an implementation detail and may change. If you need parallel data fetching, use data fetching in Server Components, or perform parallel work inside a single Server Function or Route Handler.

Forms

React extends the HTML `<form>` element to allow a Server Function to be invoked with the HTML `action` prop.

When invoked in a form, the function automatically receives the `FormData` object. You can extract the data using the native `FormData` methods:

```
// Archivo: app/ui/form.tsx switcher
import { createPost } from '@/app/actions'

export function Form() {
  return (
    <form action={createPost}>
      <input type="text" name="title" />
      <input type="text" name="content" />
      <button type="submit">Create</button>
    </form>
  )
}
```

```
// Archivo: app/ui/form.js switcher
import { createPost } from '@/app/actions'

export function Form() {
  return (
    <form action={createPost}>
      <input type="text" name="title" />
      <input type="text" name="content" />
      <button type="submit">Create</button>
    </form>
  )
}
```

```
// Archivo: app/actions.ts switcher
'use server'

import { auth } from '@/lib/auth'

export async function createPost(formData: FormData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const title = formData.get('title')
  const content = formData.get('content')

  // Mutate data
  // Revalidate cache
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { auth } from '@/lib/auth'

export async function createPost(formData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const title = formData.get('title')
  const content = formData.get('content')

  // Mutate data
  // Revalidate cache
}
```

Event Handlers

You can invoke a Server Function in a Client Component by using event handlers such as `onClick`.

```tsx
// Archivo: app/like-button.tsx switcher
'use client'

import { incrementLike } from './actions'
import { useState } from 'react'

export default function LikeButton({ initialLikes }: { initialLikes: number }) {
  const [likes, setLikes] = useState(initialLikes)

  return (
    <>
      <p>Total Likes: {likes}</p>
      <button
        onClick={async () => {
          const updatedLikes = await incrementLike()
          setLikes(updatedLikes)
        }}
      >
        Like
      </button>
    </>
  )
}
```

```jsx
// Archivo: app/like-button.js switcher
'use client'

import { incrementLike } from './actions'
import { useState } from 'react'

export default function LikeButton({ initialLikes }) {
  const [likes, setLikes] = useState(initialLikes)

  return (
    <>
      <p>Total Likes: {likes}</p>
      <button
        onClick={async () => {
          const updatedLikes = await incrementLike()
          setLikes(updatedLikes)
        }}
      >
        Like
      </button>
    </>
  )
}
```

Examples

Showing a pending state

While executing a Server Function, you can show a loading indicator with React's `useActionState` hook. This hook returns a `pending` boolean:

```tsx
// Archivo: app/ui/button.tsx switcher
'use client'

import { useActionState, startTransition } from 'react'
import { createPost } from '@/app/actions'
import { LoadingSpinner } from '@/app/ui/loading-spinner'

export function Button() {
  const [state, action, pending] = useActionState(createPost, false)

  return (
    <button onClick={() => startTransition(action)}>
      {pending ? <LoadingSpinner /> : 'Create Post'}
    </button>
  )
}
```

```jsx
// Archivo: app/ui/button.js switcher
'use client'

import { useActionState, startTransition } from 'react'
import { createPost } from '@/app/actions'
import { LoadingSpinner } from '@/app/ui/loading-spinner'

export function Button() {
  const [state, action, pending] = useActionState(createPost, false)

  return (
    <button onClick={() => startTransition(action)}>
      {pending ? <LoadingSpinner /> : 'Create Post'}
    </button>
  )
}
```

See the Building interactive apps guide for a deeper walkthrough of responsive interactions, including pending feedback, optimistic UI, transitions, and error handling.

> **Good to know**: With the **experimental** `useOffline` config enabled, a Server Action interrupted by a connectivity drop stays pending and completes when the network returns.

Refresh data

After a mutation, you may want to refresh the current page to show the latest data. You can do this by calling `refresh` from `next/cache` in a Server Action:

```tsx
// Archivo: app/lib/actions.ts switcher
'use server'

import { auth } from '@/lib/auth'
import { refresh } from 'next/cache'

export async function updatePost(formData: FormData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...

  refresh()
}
```

```
// Archivo: app/lib/actions.js switcher
'use server'

import { auth } from '@/lib/auth'
import { refresh } from 'next/cache'

export async function updatePost(formData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...

  refresh()
}
```

This refreshes the client router, ensuring the UI reflects the latest state. The `refresh()` function does not revalidate tagged data. To revalidate tagged data, use `updateTag` or `revalidateTag` instead.

Revalidate data

After performing a mutation, you can revalidate the Next.js cache and show the updated data by calling `revalidatePath` or `revalidateTag` within the Server Function:

```
// Archivo: app/lib/actions.ts switcher
import { auth } from '@/lib/auth'
import { revalidatePath } from 'next/cache'

export async function createPost(formData: FormData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...

  revalidatePath('/posts')
}
```

```
// Archivo: app/lib/actions.js switcher
import { auth } from '@/lib/auth'
import { revalidatePath } from 'next/cache'

export async function createPost(formData) {
  'use server'
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...
  revalidatePath('/posts')
}
```

Redirect after a mutation

You may want to redirect the user to a different page after a mutation. You can do this by calling `redirect` within the Server Function.

```
// Archivo: app/lib/actions.ts switcher
'use server'

import { auth } from '@/lib/auth'
import { revalidatePath } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData: FormData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...

  revalidatePath('/posts')
  redirect('/posts')
}
```

```
// Archivo: app/lib/actions.js switcher
'use server'

import { auth } from '@/lib/auth'
import { revalidatePath } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  // Mutate data
  // ...

  revalidatePath('/posts')
  redirect('/posts')
}
```

Calling `redirect` throws a framework handled control-flow exception. Any code after it won't execute. If you need fresh data, call `revalidatePath` or `revalidateTag` beforehand.

Cookies

You can `get`, `set`, and `delete` cookies inside a Server Action using the `cookies` API.

When you set or delete a cookie in a Server Action, Next.js re-renders the current page and its layouts on the server so the **UI reflects the new cookie value**.

> **Good to know**: The server update applies to the current React tree, re-rendering, mounting, or unmounting components, as needed. Client state is preserved for re-rendered components, and effects re-run if their dependencies changed.

```
// Archivo: app/actions.ts switcher
'use server'

import { cookies } from 'next/headers'

export async function exampleAction() {
  const cookieStore = await cookies()

  // Get cookie
  cookieStore.get('name')?.value

  // Set cookie
  cookieStore.set('name', 'Delba')

  // Delete cookie
  cookieStore.delete('name')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { cookies } from 'next/headers'

export async function exampleAction() {
  // Get cookie
  const cookieStore = await cookies()

  // Get cookie
  cookieStore.get('name')?.value

  // Set cookie
  cookieStore.set('name', 'Delba')

  // Delete cookie
  cookieStore.delete('name')
}
```

useEffect

You can use the React `useEffect` hook to invoke a Server Action when the component mounts or a dependency changes. This is useful for mutations that depend on global events or need to be triggered automatically. For example, `onKeyDown` for app shortcuts, an intersection observer hook for infinite scrolling, or when the component mounts to update a view count:

```
// Archivo: app/view-count.tsx switcher
'use client'

import { incrementViews } from './actions'
import { useState, useEffect, useTransition } from 'react'

export default function ViewCount({ initialViews }: { initialViews: number }) {
  const [views, setViews] = useState(initialViews)
  const [isPending, startTransition] = useTransition()

  useEffect(() => {
    startTransition(async () => {
      const updatedViews = await incrementViews()
      setViews(updatedViews)
    })
  }, [])

  // You can use `isPending` to give users feedback
  return <p>Total Views: {views}</p>
}
```

```
// Archivo: app/view-count.js switcher
'use client'

import { incrementViews } from './actions'
import { useState, useEffect, useTransition } from 'react'

export default function ViewCount({ initialViews }) {
  const [views, setViews] = useState(initialViews)
  const [isPending, startTransition] = useTransition()

  useEffect(() => {
    startTransition(async () => {
      const updatedViews = await incrementViews()
      setViews(updatedViews)
    })
  }, [])

  // You can use `isPending` to give users feedback
  return <p>Total Views: {views}</p>
}
```

Error Handling

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/error-handling
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to display expected errors and handle uncaught exceptions.*

Errors can be divided into two categories: expected errors and uncaught exceptions. This page will walk you through how you can handle these errors in your Next.js application.

Handling expected errors

Expected errors are those that can occur during the normal operation of the application, such as those from server-side form validation or failed requests. These errors should be handled explicitly and returned to the client.

Server Functions

You can use the `useActionState` hook to handle expected errors in Server Functions.

For these errors, avoid using `try` / `catch` blocks and throw errors. Instead, model expected errors as return values.

```
// Archivo: app/actions.ts switcher
'use server'

export async function createPost(prevState: any, formData: FormData) {
  const title = formData.get('title')
  const content = formData.get('content')

  const res = await fetch('https://api.vercel.app/posts', {
    method: 'POST',
    body: { title, content },
  })
  const json = await res.json()

  if (!res.ok) {
    return { message: 'Failed to create post' }
  }
}
```

```
// Archivo: app/actions.js switcher
'use server'

export async function createPost(prevState, formData) {
  const title = formData.get('title')
  const content = formData.get('content')

  const res = await fetch('https://api.vercel.app/posts', {
    method: 'POST',
    body: { title, content },
  })
  const json = await res.json()

  if (!res.ok) {
    return { message: 'Failed to create post' }
  }
}
```

You can pass your action to the `useActionState` hook and use the returned `state` to display an error message.

```
// Archivo: app/ui/form.tsx highlight={11,19} switcher
'use client'

import { useActionState } from 'react'
import { createPost } from '@/app/actions'

const initialState = {
  message: '',
}

export function Form() {
  const [state, formAction, pending] = useActionState(createPost, initialState)

  return (
    <form action={formAction}>
      <label htmlFor="title">Title</label>
      <input type="text" id="title" name="title" required />
      <label htmlFor="content">Content</label>
      <textarea id="content" name="content" required />
      {state?.message && <p aria-live="polite">{state.message}</p>}
      <button disabled={pending}>Create Post</button>
    </form>
  )
}
```

```
// Archivo: app/ui/form.js highlight={11,19} switcher
'use client'

import { useActionState } from 'react'
import { createPost } from '@/app/actions'

const initialState = {
  message: '',
}

export function Form() {
  const [state, formAction, pending] = useActionState(createPost, initialState)

  return (
    <form action={formAction}>
      <label htmlFor="title">Title</label>
      <input type="text" id="title" name="title" required />
      <label htmlFor="content">Content</label>
      <textarea id="content" name="content" required />
      {state?.message && <p aria-live="polite">{state.message}</p>}
      <button disabled={pending}>Create Post</button>
    </form>
  )
}
```

Server Components

When fetching data inside of a Server Component, you can use the response to conditionally render an error message or `redirect`.

```
// Archivo: app/page.tsx switcher
export default async function Page() {
  const res = await fetch(`https:// ... `)
  const data = await res.json()

  if (!res.ok) {
    return 'There was an error.'
  }

  return ' ... '
}
```

```
// Archivo: app/page.js switcher
export default async function Page() {
  const res = await fetch(`https:// ... `)
  const data = await res.json()

  if (!res.ok) {
    return 'There was an error.'
  }

  return ' ... '
}
```

Not found

You can call the `notFound` function within a route segment and use the `not-found.js` file to show a 404 UI.

```
// Archivo: app/blog/[slug]/page.tsx switcher
import { notFound } from 'next/navigation'
import { getPostBySlug } from '@/lib/posts'

export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = getPostBySlug(slug)

  if (!post) {
    notFound()
  }

  return <div>{post.title}</div>
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
import { notFound } from 'next/navigation'
import { getPostBySlug } from '@/lib/posts'

export default async function Page({ params }) {
  const { slug } = await params
  const post = getPostBySlug(slug)

  if (!post) {
    notFound()
  }

  return <div>{post.title}</div>
}
```

```
// Archivo: app/blog/[slug]/not-found.tsx switcher
export default function NotFound() {
  return <div>404 - Page Not Found</div>
}
```

```
// Archivo: app/blog/[slug]/not-found.js switcher
export default function NotFound() {
  return <div>404 - Page Not Found</div>
}
```

Handling uncaught exceptions

Uncaught exceptions are unexpected errors that indicate bugs or issues that should not occur during the normal flow of your application. These should be handled by throwing errors, which will then be caught by error boundaries.

Nested error boundaries

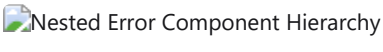
Next.js uses error boundaries to handle uncaught exceptions. Error boundaries catch errors in their child components and display a fallback UI instead of the component tree that crashed.

Create an error boundary by adding an `error.js` file inside a route segment and exporting a React component:

```
// Archivo: app/dashboard/error.tsx switcher
'use client' // Error boundaries must be Client Components

import { useEffect } from 'react'

export default function ErrorPage({
  error,
  retry,
}: {
  error: Error & { digest?: string }
  retry: () => void
}) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error(error)
  }, [error])

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button
        onClick={
          // Attempt to recover by re-fetching and re-rendering the segment
          () => retry()
        }
      >
        Try again
      </button>
    </div>
  )
}
```

```
// Archivo: app/dashboard/error.js switcher
'use client' // Error boundaries must be Client Components

import { useEffect } from 'react'

export default function ErrorPage({ error, retry }) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error(error)
  }, [error])

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button
        onClick={
          // Attempt to recover by re-fetching and re-rendering the segment
          () => retry()
        }
      >
        Try again
      </button>
    </div>
  )
}
```

Errors will bubble up to the nearest parent error boundary. This allows for granular error handling by placing `error.tsx` files at different levels in the route hierarchy.


Nested Error Component Hierarchy

For component-level error recovery, the `catchError` function lets you create error boundaries that can wrap any part of your component tree:

```
// Archivo: app/custom-error-boundary.tsx switcher
'use client'

import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(props: { title: string }, { error, retry }: ErrorInfo) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

```
// Archivo: app/custom-error-boundary.js switcher
'use client'

import { catchError } from 'next/error'

function ErrorFallback(props, { error, retry }) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

Then use the returned component as a wrapper in any layout or page:

```
// Archivo: app/some-component.tsx switcher
import ErrorBoundary from './custom-error-boundary'

export default function Component({ children }: { children: React.ReactNode }) {
  return <ErrorBoundary title="Dashboard Error">{children}</ErrorBoundary>
}
```

```
// Archivo: app/some-component.js switcher
import ErrorBoundary from './custom-error-boundary'

export default function Component({ children }) {
  return <ErrorBoundary title="Dashboard Error">{children}</ErrorBoundary>
}
```

Error boundaries don't catch errors inside event handlers. They're designed to catch errors during rendering to show a **fallback UI** instead of crashing the whole app.

In general, errors in event handlers or async code aren't handled by error boundaries because they run after rendering.

To handle these cases, catch the error manually and store it using `useState` or `useReducer`, then update the UI to inform the user.

```
'use client'

import { useState } from 'react'

export function Button() {
  const [error, setError] = useState(null)

  const handleClick = () => {
    try {
      // do some work that might fail
      throw new Error('Exception')
    } catch (reason) {
      setError(reason)
    }
  }

  if (error)

  return (
    <button type="button" onClick={handleClick}>
      Click me
    </button>
  )
}
```

Note that unhandled errors inside `startTransition` from `useTransition`, will bubble up to the nearest error boundary.

```
'use client'

import { useTransition } from 'react'

export function Button() {
  const [pending, startTransition] = useTransition()

  const handleClick = () =>
    startTransition(() => {
      throw new Error('Exception')
    })

  return (
    <button type="button" onClick={handleClick}>
      Click me
    </button>
  )
}
```

Global errors

While less common, you can handle errors in the root layout using the `global-error.js` file, located in the root app directory, even when leveraging internationalization. Global error UI must define its own `<html>` and `<body>` tags, since it is replacing the root layout or template when active.

```
// Archivo: app/global-error.tsx switcher
'use client' // Error boundaries must be Client Components

export default function GlobalError({
  error,
  retry,
}: {
  error: Error & { digest?: string }
  retry: () => void
}) {
  return (
    // global-error must include html and body tags
    <html>
      <body>
        <h2>Something went wrong!</h2>
        <button onClick={() => retry()}>Try again</button>
      </body>
    </html>
  )
}
```

```
// Archivo: app/global-error.js switcher
'use client' // Error boundaries must be Client Components

export default function GlobalError({ error, retry }) {
  return (
    // global-error must include html and body tags
    <html>
      <body>
        <h2>Something went wrong!</h2>
        <button onClick={() => retry()}>Try again</button>
      </body>
    </html>
  )
}
```

Deploying Next.js to different platforms

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/deploying-to-platforms
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Understand which Next.js features require specific platform capabilities and how to choose the right deployment target.*

Next.js treats static and dynamic content as a spectrum at the component level. Different features in this model require different platform capabilities. This page helps you understand what your platform needs to support and how to configure your deployment.

Minimum Requirements

To run Next.js, your platform needs **a Node.js server**. That's it.

A single `next start` process handles every Next.js feature correctly: Server Components, ISR, PPR, Cache Components, Server Actions, Proxy, and `after()`. Streaming support is needed for features like PPR and Server Components to deliver content progressively (without it, responses are buffered and sent as a whole, which still works but loses the streaming performance benefit). Additional infrastructure (CDN caching, edge compute, shared cache) primarily improves performance and multi-instance consistency. In multi-instance

deployments, shared cache and tag coordination reduce stale divergence between instances. The only additional dependency is the `sharp` package, which is required for Image Optimization.

Functional Fidelity vs. Performance Fidelity

When evaluating platform support for Next.js, it helps to distinguish between two levels:

Functional fidelity means every Next.js feature works correctly. The adapter test suite is the contract: if a platform's adapter passes the tests, it supports Next.js. This is binary: it passes or it doesn't.

Performance fidelity means features achieve their optimal performance characteristics. Examples include PPR's static shell served at CDN latency rather than origin latency, or ISR serving stale content instantly with sub-second revalidation propagation. Performance fidelity is a spectrum that every platform will achieve differently based on their architecture.

A platform that achieves functional fidelity is a fully supported deployment target for Next.js. Performance fidelity is how platforms differentiate, and it improves incrementally over time.

Use the feature matrix below through this lens: "Streaming Required" and "Shared Cache Recommended" describe what is needed for functional fidelity. "Edge Stitching" is a performance fidelity optimization.

Feature Support Matrix

Different features require different infrastructure capabilities. The "Edge Stitching" column is a **performance optimization**, not a correctness requirement. All features work correctly from a single origin server.

Feature	Streaming	Shared Cache	Edge Stitching	Notes
Server Components	Required	No	No	Basic streaming support
ISR (time-based)	No	Recommended	No	Works per-instance without shared cache
ISR (on-demand)	No	Recommended	No	Tag propagation needs shared cache for multi-instance
Partial Prerendering	Required	Recommended	Optional	See PPR Platform Guide
Cache Components (`use cache`)	Required	Recommended	No	Shared cache enables cross-instance consistency
Proxy / Middleware	No	No	No	Runs at edge or origin
Server Actions	Required	No	No	POST requests with streaming response
`after()`	No	No	No	Requires graceful shutdown support

Streaming Required means the platform must support chunked transfer encoding or HTTP/2 streaming and must not buffer the response before sending it to the client.

Shared Cache Recommended means multiple server instances benefit from shared cache backends to coordinate. For ISR and server response caching, use `cacheHandler`. For `'use cache'` entries, use `cacheHandlers`. Without shared cache, each instance maintains its own cache independently — features still work correctly on each instance, but revalidation events don't propagate across instances.

CDN Infrastructure Compatibility

The following table maps infrastructure primitives for each major CDN. These are available building blocks, not finished integrations:

CDN	Edge Compute	Key-Value / Tags	Blob Storage	PPR Resuming
Cloudflare	Workers	KV	R2	Yes (worker)
Akamai	EdgeWorkers	EdgeKV	Object Storage	Yes (worker)
Amazon CloudFront	Lambda@Edge	KeyValueStore	S3	Yes (Lambda)
Fastly	Compute	KV Store	Object Storage	Yes (WASM)
Azure	Functions	Managed Redis	Blob Storage	Yes (server)
Google Cloud	Cloud Run	Various KV	Cloud Storage	Yes (server)

These are available building blocks, not finished integrations. Most community adapters today deploy Next.js as a Docker container or Node.js server without leveraging CDN-specific primitives like edge KV or PPR resuming. See the Deploying page for the current list of adapters. For CDN-specific caching considerations (including known limitations with `Vary` on custom headers), see CDN Caching.

Adapters

Next.js provides a Deployment Adapter API that lets platforms customize how Next.js applications are built and deployed for their infrastructure. Adapters run at build time and produce platform-specific output from the standard Next.js build. Anyone can build an adapter using the public API with no special access required.

The adapter API plus Next.js caching interfaces form the complete platform integration surface. The adapter handles build-time output, while `cacheHandler` and `cacheHandlers` cover different runtime caching paths. `cacheHandler` (singular) covers server cache paths like ISR, route handlers, patched `fetch` / `unstable_cache`, and image optimization. `cacheHandlers` (plural) configures `'use cache'` directive backends.

Verified Adapters

A **verified adapter** is one that meets two requirements:

1. **Open source.** The adapter source code is publicly available so the community and the Next.js team can inspect, contribute to, and verify it.
2. **Runs the compatibility test suite.** The platform provides a way to run the full Next.js compatibility test suite against their adapter. This gives visibility into which features work, which are in progress, and where gaps remain.

Verified adapters are hosted under the Next.js GitHub organization, listed as supported deployment targets in the Next.js documentation, and maintained by their respective platform teams. There are no private framework hooks or integration paths: Vercel's adapter uses the same public API as every other adapter.

For verified adapters and platforms working toward verified status through the Ecosystem Working Group, the Next.js team commits to:

- **Coordinated testing.** Before major releases, we work with platform teams to run the compatibility test suite and surface issues early.
- **Early access.** Adapter authors receive early access to API changes during RFCs and release candidates.
- **Direct support.** When the adapter contract needs updating, we work directly with adapter teams.

> **Good to know:** Platforms can build closed-source adapters on the same public API and test suite. However, closed-source adapters will not be listed as verified, since the Next.js team cannot verify what it cannot inspect.

A Note on Infrastructure Requirements

Next.js's rendering model places the static/dynamic boundary at the component level rather than the route level. Finer-grained boundaries provide more flexibility for developers at the cost of broader requirements for hosting platforms. This is a deliberate trade-off: the infrastructure requirements on this page exist because of what the rendering model delivers.

How to create forms with Server Actions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/forms
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to create forms in Next.js with React Server Actions.*

React Server Actions are Server Functions that execute on the server. They can be called in Server and Client Components to handle form submissions. This guide will walk you through how to create forms in Next.js with Server Actions. For Server Action behaviors beyond forms (single-roundtrip response, sequential dispatch, security, deployment), see Server Actions and Mutations.

> [!WARNING] Always verify authentication and authorization inside each Server Action, even if the form is only rendered on an authenticated page. See the Data Security guide for more details.

How it works

React extends the HTML `<form>` element to allow Server Actions to be invoked with the `action` attribute.

When used in a form, the function automatically receives the `FormData` object. You can then extract the data using the native `FormData` methods:

```
// Archivo: app/invoices/page.tsx switcher
import { auth } from '@/lib/auth'

export default function Page() {
  async function createInvoice(formData: FormData) {
    'use server'

    const session = await auth()
    if (!session?.user) {
      throw new Error('Unauthorized')
    }

    const rawFormData = {
      customerId: formData.get('customerId'),
      amount: formData.get('amount'),
      status: formData.get('status'),
    }

    // mutate data
    // revalidate the cache
  }

  return <form action={createInvoice}> ... </form>
}
```

```
// Archivo: app/invoices/page.js switcher
import { auth } from '@/lib/auth'

export default function Page() {
  async function createInvoice(formData) {
    'use server'

    const session = await auth()
    if (!session?.user) {
      throw new Error('Unauthorized')
    }

    const rawFormData = {
      customerId: formData.get('customerId'),
      amount: formData.get('amount'),
      status: formData.get('status'),
    }

    // mutate data
    // revalidate the cache
  }

  return <form action={createInvoice}> ... </form>
}
```

> **Good to know:** When working with forms that have multiple fields, use JavaScript's `Object.fromEntries()`. For example: `const rawFormData = Object.fromEntries(formData)`. Note that this object will contain extra properties prefixed with `$ACTION_`.

Passing additional arguments

Outside of form fields, you can pass additional arguments to a Server Function using the JavaScript `bind` method. For example, to pass the `userId` argument to the `updateUser` Server Function:

```
// Archivo: app/client-component.tsx highlight={6} switcher
'use client'

import { updateUser } from './actions'

export function UserProfile({ userId }: { userId: string }) {
  const updateUserWithId = updateUser.bind(null, userId)

  return (
    <form action={updateUserWithId}>
      <input type="text" name="name" />
      <button type="submit">Update User Name</button>
    </form>
  )
}
```

```
// Archivo: app/client-component.js highlight={6} switcher
'use client'

import { updateUser } from './actions'

export function UserProfile({ userId }) {
  const updateUserWithId = updateUser.bind(null, userId)

  return (
    <form action={updateUserWithId}>
      <input type="text" name="name" />
      <button type="submit">Update User Name</button>
    </form>
  )
}
```

The Server Function will receive the `userId` as an additional argument:

```
// Archivo: app/actions.ts switcher
'use server'

export async function updateUser(userId: string, formData: FormData) {}
```

```
// Archivo: app/actions.js switcher
'use server'

export async function updateUser(userId, formData) {}
```

Good to know:

- An alternative is to pass arguments as hidden input fields in the form (e.g. `<input type="hidden" name="userId" value={userId} />`). However, the value will be part of the rendered HTML and will not be encoded.
- `bind` works in both Server and Client Components and supports progressive enhancement.

Form validation

Forms can be validated on the client or server.

- For **client-side validation**, you can use the HTML attributes like `required` and `type="email"` for basic validation.
- For **server-side validation**, you can use a schema validation library like Zod or Valibot to validate the form fields. For example:

```
// Archivo: app/actions.ts switcher
'use server'

import { z } from 'zod'

const schema = z.object({
  email: z.string({
    invalid_type_error: 'Invalid Email',
  }),
})

export default async function createUser(formData: FormData) {
  const validatedFields = schema.safeParse({
    email: formData.get('email'),
  })

  // Return early if the form data is invalid
  if (!validatedFields.success) {
    return {
      errors: validatedFields.error.flatten().fieldErrors,
    }
  }

  // Mutate data
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { z } from 'zod'

const schema = z.object({
  email: z.string({
    invalid_type_error: 'Invalid Email',
  }),
})

export default async function createUser(formData) {
  const validatedFields = schema.safeParse({
    email: formData.get('email'),
  })

  // Return early if the form data is invalid
  if (!validatedFields.success) {
    return {
      errors: validatedFields.error.flatten().fieldErrors,
    }
  }

  // Mutate data
}
```

Validation errors

To display validation errors or messages, turn the component that defines the `<form>` into a Client Component and use React `useActionState`.

When using `useActionState`, the Server function signature will change to receive a new `prevState` or `initialState` parameter as its first argument.

```
// Archivo: app/actions.ts highlight={5} switcher
'use server'

import { z } from 'zod'

export async function createUser(initialState: any, formData: FormData) {
  const validatedFields = schema.safeParse({
    email: formData.get('email'),
  })
  // ...
}
```

```
// Archivo: app/actions.ts highlight={7} switcher
'use server'

import { z } from 'zod'

// ...

export async function createUser(initialState, formData) {
  const validatedFields = schema.safeParse({
    email: formData.get('email'),
  })
  // ...
}
```

You can then conditionally render the error message based on the `state` object.

```tsx
// Archivo: app/ui/signup.tsx highlight={11,18-20} switcher
'use client'

import { useActionState } from 'react'
import { createUser } from '@/app/actions'

const initialState = {
  message: '',
}

export function Signup() {
  const [state, formAction, pending] = useActionState(createUser, initialState)

  return (
    <form action={formAction}>
      <label htmlFor="email">Email</label>
      <input type="text" id="email" name="email" required />

      <p aria-live="polite">{state?.message}</p>
      <button disabled={pending}>Sign up</button>
    </form>
  )
}
```

```jsx
// Archivo: app/ui/signup.js highlight={11,18-20} switcher
'use client'

import { useActionState } from 'react'
import { createUser } from '@/app/actions'

const initialState = {
  message: '',
}

export function Signup() {
  const [state, formAction, pending] = useActionState(createUser, initialState)

  return (
    <form action={formAction}>
      <label htmlFor="email">Email</label>
      <input type="text" id="email" name="email" required />

      <p aria-live="polite">{state?.message}</p>
      <button disabled={pending}>Sign up</button>
    </form>
  )
}
```

Pending states

The `useActionState` hook exposes a `pending` boolean that can be used to show a loading indicator or disable the submit button while the action is being executed.

```tsx
// Archivo: app/ui/signup.tsx highlight={7,12} switcher
'use client'

import { useActionState } from 'react'
import { createUser } from '@/app/actions'

export function Signup() {
  const [state, formAction, pending] = useActionState(createUser, initialState)

  return (
    <form action={formAction}>

      <button disabled={pending}>Sign up</button>
    </form>
  )
}
```

```jsx
// Archivo: app/ui/signup.js highlight={7,12} switcher
'use client'

import { useActionState } from 'react'
import { createUser } from '@/app/actions'

export function Signup() {
  const [state, formAction, pending] = useActionState(createUser, initialState)

  return (
    <form action={formAction}>

      <button disabled={pending}>Sign up</button>
    </form>
  )
}
```

Alternatively, you can use the `useFormStatus` hook to show a loading indicator while the action is being executed. When using this hook, you'll need to create a separate component to render the loading indicator. For example, to disable the button when the action is pending:

```tsx
// Archivo: app/ui/button.tsx highlight={6} switcher
'use client'

import { useFormStatus } from 'react-dom'

export function SubmitButton() {
  const { pending } = useFormStatus()

  return (
    <button disabled={pending} type="submit">
      Sign Up
    </button>
  )
}
```

```jsx
// Archivo: app/ui/button.js highlight={6} switcher
'use client'

import { useFormStatus } from 'react-dom'

export function SubmitButton() {
  const { pending } = useFormStatus()

  return (
    <button disabled={pending} type="submit">
      Sign Up
    </button>
  )
}
```

You can then nest the `SubmitButton` component inside the form:

```
// Archivo: app/ui/signup.tsx switcher
import { SubmitButton } from './button'
import { createUser } from '@/app/actions'

export function Signup() {
  return (
    <form action={createUser}>

      <SubmitButton />
    </form>
  )
}
```

```
// Archivo: app/ui/signup.js switcher
import { SubmitButton } from './button'
import { createUser } from '@/app/actions'

export function Signup() {
  return (
    <form action={createUser}>

      <SubmitButton />
    </form>
  )
}
```

> **Good to know:** In React 19, `useFormStatus` includes additional keys on the returned object, like data, method, and action. If you are not using React 19, only the `pending` key is available.

> **Good to know**: With the **experimental** `useOffline` config enabled, a Server Action interrupted by a connectivity drop stays pending and completes when the network returns, so a user does not lose their submission.

Optimistic updates

You can use the React `useOptimistic` hook to optimistically update the UI before the Server Function finishes executing, rather than waiting for the response:

```tsx
// Archivo: app/page.tsx switcher
'use client'

import { useOptimistic } from 'react'
import { send } from './actions'

type Message = {
  message: string
}

export function Thread({ messages }: { messages: Message[] }) {
  const [optimisticMessages, addOptimisticMessage] = useOptimistic<
    Message[],
    string
  >(messages, (state, newMessage) => [...state, { message: newMessage }])

  const formAction = async (formData: FormData) => {
    const message = formData.get('message') as string
    addOptimisticMessage(message)
    await send(message)
  }

  return (
    <div>
      {optimisticMessages.map((m, i) => (
        <div key={i}>{m.message}</div>
      ))}
      <form action={formAction}>
        <input type="text" name="message" />
        <button type="submit">Send</button>
      </form>
    </div>
  )
}
```

```jsx
// Archivo: app/page.js switcher
'use client'

import { useOptimistic } from 'react'
import { send } from './actions'

export function Thread({ messages }) {
  const [optimisticMessages, addOptimisticMessage] = useOptimistic(
    messages,
    (state, newMessage) => [...state, { message: newMessage }]
  )

  const formAction = async (formData) => {
    const message = formData.get('message')
    addOptimisticMessage(message)
    await send(message)
  }

  return (
    <div>
      {optimisticMessages.map((m) => (
        <div>{m.message}</div>
      ))}
      <form action={formAction}>
        <input type="text" name="message" />
        <button type="submit">Send</button>
      </form>
    </div>
  )
}
```

Nested form elements

You can call Server Actions in elements nested inside `<form>` such as `<button>`, `<input type="submit">`, and `<input type="image">`. These elements accept the `formAction` prop or event handlers.

This is useful in cases where you want to call multiple Server Actions within a form. For example, you can create a specific `<button>` element for saving a post draft in addition to publishing it. See the React `<form>` docs for more information.

Programmatic form submission

You can trigger a form submission programmatically using the `requestSubmit()` method. For example, when the user submits a form using the ⌘ + `Enter` keyboard shortcut, you can listen for the `onKeyDown` event:

```
// Archivo: app/entry.tsx switcher
'use client'

export function Entry() {
  const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
    if (
      (e.ctrlKey || e.metaKey) &&
      (e.key === 'Enter' || e.key === 'NumpadEnter')
    ) {
      e.preventDefault()
      e.currentTarget.form?.requestSubmit()
    }
  }

  return (
    <div>
      <textarea name="entry" rows={20} required onKeyDown={handleKeyDown} />
    </div>
  )
}
```

```
// Archivo: app/entry.js switcher
'use client'

export function Entry() {
  const handleKeyDown = (e) => {
    if (
      (e.ctrlKey || e.metaKey) &&
      (e.key === 'Enter' || e.key === 'NumpadEnter')
    ) {
      e.preventDefault()
      e.currentTarget.form?.requestSubmit()
    }
  }

  return (
    <div>
      <textarea name="entry" rows={20} required onKeyDown={handleKeyDown} />
    </div>
  )
}
```

This will trigger the submission of the nearest `<form>` ancestor, which will invoke the Server Function.

Server Actions and Mutations

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/server-actions
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *How Server Actions work in Next.js, including the single-roundtrip response model, sequential dispatch, security, and caching integration.*

A **Server Action** is a React Server Function invoked through React's action mechanisms, such as `<form action>`, `<button formAction>`, or a client-side transition.

You create one by adding the `'use server'` directive, then invoke it from a form, or from an event handler or `useEffect` wrapped in `startTransition`. For the basics of creating and invoking Server Functions, see Mutating data and the Forms guide.

This page covers the parts of Server Actions that are specific to Next.js: how they commonly map to mutations, how the response carries both returned data and re-rendered UI, how the client dispatches them, the security boundary the framework enforces, and the configuration available.

Sequential dispatch on the client

Next.js dispatches Server Actions one at a time per client. If a user triggers three actions in quick succession, the second waits for the first to finish, then the third waits for the second. This keeps the re-rendered server tree consistent with the action result that produced it.

A consequence: do not rely on `Promise.all` to parallelize Server Actions from the client. If you need parallel work, do it inside a single Server Action, fetch in parallel from a Server Component, or use a Route Handler for non-mutation requests.

> **Good to know:** This is a property of the client dispatcher, not of Server Functions in general. Server-side, an action runs in its own request and can do anything an async function can do.

A single response carries data and UI

When a Server Action triggers an immediate revalidation, Next.js does the work inside one HTTP request: it runs the action, then re-renders the current route server-side. The response that comes back contains both pieces in the same Flight stream:

- The action's return value, consumed by `useActionState` or the awaited promise on the client.
- A newly rendered RSC Payload for the current route, which the client commits as a seeded navigation.

Your application code does not need a follow-up fetch to see the updated UI for the current page.

A re-render is included in the same response when the action does any of these:

- Calls `updateTag` or `revalidatePath` to immediately invalidate cached data.
- Calls `refresh` to refetch the current route's RSC Payload.
- Mutates cookies through `cookies()`. Setting or deleting a cookie automatically re-renders the current page so the UI reflects the new value.
- Calls `redirect`. The response navigates the router and streams the destination's RSC Payload.

```
// Archivo: app/posts/actions.ts
'use server'

import { revalidatePath } from 'next/cache'
import { auth } from '@/lib/auth'
import { db } from '@/lib/db'

export async function createPost(formData: FormData) {
  const session = await auth()
  if (!session?.user) throw new Error('Unauthorized')

  await db.post.create({
    data: {
      title: String(formData.get('title')),
      authorId: session.user.id,
    },
  })

  revalidatePath('/posts')
}
```

The mutation, the cache invalidation, and the page re-render all complete in a single roundtrip. Because `redirect` throws a control-flow exception, any code after it does not run. Place revalidation calls before `redirect` if the destination needs the fresh data.

`revalidateTag` with a stale-while-revalidate profile is the exception: it marks the tag for background refresh and does **not** include a re-render in the action response. The page reflects the change on a later read. An action that does none of the above carries only its return value, and the current route is not re-rendered.

Security

A Server Action runs as a POST request against the page that invokes it. At build time, the `'use server'` directive tells the compiler to swap the function's implementation in client bundles for a reference (an action ID plus a dispatcher) that POSTs back to the server. The implementation stays on the server, but the route is reachable to anyone who can send the same POST. Treat every action as an untrusted entry point.

Next.js enforces a few framework-level protections:

- **CSRF check.** The request's `Origin` is compared to the `Host` (or `X-Forwarded-Host`). Mismatches are rejected. Configure `serverActions.allowedOrigins` for proxy or CDN domains.
- **Body size limit.** Action requests are capped at 1MB by default. Configure `serverActions.bodySizeLimit` when accepting larger payloads.
- **Encrypted action IDs and dead code elimination.** Action references are encrypted at build time, and unused Server Functions are stripped from client bundles so they have no public endpoint. See Built-in Server Actions security features.
- **Closure variable encryption.** Variables captured by an inline action are encrypted before being sent to the client. For multi-instance and self-hosted deployments, set `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` to a stable key shared across instances. See Closures and encryption.

Framework protections are not a substitute for application-level checks. Inside every action:

- **Authenticate and authorize.** Render-time gating (only rendering a form on an authenticated page) is not a security boundary, because requests can be sent without going through the UI.
- **Validate inputs.** Treat `FormData`, query parameters, and headers as untrusted.
- **Constrain return values.** Action returns are serialized to the client. Shape them to what the UI renders, not raw database records.

For end-to-end patterns including a Data Access Layer, return-value tainting, and rate limiting, see the Data Security guide.

Destructive operations like deletes may warrant stronger handling, such as elevated session checks or re-authentication, and a loud failure when those checks miss.

```
// Archivo: app/posts/actions.ts highlight={6,7,8}
'use server'

import { auth } from '@/lib/auth'

export async function deletePost(postId: string) {
  const session = await auth()
  if (!session?.user) throw new Error('Unauthorized')
  if (!(await canDelete(session.user, postId))) throw new Error('Forbidden')

  await db.post.delete({ where: { id: postId } })
}
```

If you've enabled the experimental `authInterrupts` flag, you can throw `unauthorized()` and `forbidden()` from `next/navigation` instead, so Next.js renders the corresponding `unauthorized.tsx` / `forbidden.tsx` UI segment automatically.

For example, a client legitimately tells the server *which* item to act on, but it should not supply the row's contents or ownership. Send a reference (typically an ID) plus the user's change, and re-read the rest from a trusted source using the session. Schema validation (zod or similar) only checks the *shape* of the input. A well-formed `Item` object can still refer to a row the caller does not own.

```
// Archivo: app/items/actions.ts
'use server'

import { auth } from '@/lib/auth'
import { db } from '@/lib/db'

// Unsafe: no auth, no ownership check. The whole item, including its id, comes
// from the client, so anyone who can POST here can mark any item complete.
export async function completeItemUnsafe(item: Item) {
  await db.item.update({ where: { id: item.id }, data: { completed: true } })
}

// Safe: take only the change, derive identity from the session, look up by ownership.
export async function completeItem(itemId: string) {
  const session = await auth()
  if (!session?.user) return

  const item = await db.item.findFirst({
    where: { id: itemId, ownerId: session.user.id },
  })
  if (!item) return

  await db.item.update({ where: { id: item.id }, data: { completed: true } })
}
```

Choosing a cache update

After mutating data, on-demand revalidation updates the server cache, the client router, or both. Choose based on what needs to change:

- `updateTag`: immediate expiration of a tag. The next read (including the route re-render that ships with the action's response) waits for fresh data. Use when the action needs **read-your-own-writes** so the user immediately sees their change. Server Actions only.
- `revalidateTag`: stale-while-revalidate refresh of a tag with a cache-life profile. Subsequent reads get the stale value while a fresh fetch happens in the background, so the action's own re-render does **not** wait for the new data.
- `revalidatePath`: invalidate by URL path. Use when one route is affected and tagging is overkill.
- `refresh`: refetch the current route's RSC Payload without invalidating cached data. Use when the view depends on state outside the cache that the action just changed.

When `updateTag`, `revalidatePath`, or `refresh` runs, Next.js re-renders the current route server-side and includes a newly rendered RSC Payload in the action's response, so the page reflects the change in the same roundtrip. `revalidateTag` with a stale-while-revalidate profile intentionally skips that immediate re-render.

Unlike `redirect`, none of these throw, so an action can call them and still return a value to the caller. See How revalidation works for the underlying model.

Configuration

The `serverActions` option in `next.config.js` controls framework-level behavior:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
module.exports = {
  experimental: {
    serverActions: {
      allowedOrigins: ['my-proxy.com', '*.my-proxy.com'],
      bodySizeLimit: '2mb',
    },
  },
}
```

For the closure encryption key, set `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` in the deployment environment. See Self-hosting: Server Functions encryption key for deployment-specific guidance.

Deployment considerations

Each Server Action is identified by the action ID that is part of its build artifacts. New deployments typically generate new IDs (Next.js rotates them at most every 14 days, even when the source is unchanged), so a client still running the previous build may invoke an action ID that no longer exists. The error surfaces as "Failed to find Server Action".

To minimize disruption:

- Prefer rolling deployments over abrupt cutovers when active users are likely to be mid-mutation.
- Keep `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` stable across instances so action references remain decryptable everywhere.
- Surface the error as a retry path in the UI rather than a hard failure, so a refresh recovers the user.

How to create forms with API Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/forms
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to handle form submissions and data mutations with Next.js.*

Forms enable you to create and update data in web applications. Next.js provides a powerful way to handle data mutations using **API Routes**. This guide will walk you through how to handle form submission on the server.

Server Forms

To handle form submissions on the server, create an API endpoint that securely mutates data.

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const data = req.body
  const id = await createItem(data)
  res.status(200).json({ id })
}
```

```
// Archivo: pages/api/submit.js switcher
export default function handler(req, res) {
  const data = req.body
  // call your database, etc.
  // const id = await createItem(data)
  // ...
  res.status(200).json({ data })
}
```

Then, call the API Route from the client with an event handler:

```
// Archivo: pages/index.tsx switcher
import { SubmitEvent } from 'react'

export default function Page() {
  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const response = await fetch('/api/submit', {
      method: 'POST',
      body: formData,
    })

    // Handle response if necessary
    const data = await response.json()
    // ...
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  )
}
```

```
// Archivo: pages/index.jsx switcher
export default function Page() {
  async function onSubmit(event) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const response = await fetch('/api/submit', {
      method: 'POST',
      body: formData,
    })

    // Handle response if necessary
    const data = await response.json()
    // ...
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  )
}
```

Good to know:

- API Routes do not specify CORS headers, meaning they are same-origin only by default.
- Since API Routes run on the server, we're able to use sensitive values (like API keys) through Environment Variables without exposing them to the client. This is critical for the security of your application.

Form validation

We recommend using HTML validation like `required` and `type="email"` for basic client-side form validation.

For more advanced server-side validation, you can use a schema validation library like Zod or Valibot to validate the form fields before mutating the data:

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'
import { z } from 'zod'

const schema = z.object({
  // ...
})

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const parsed = schema.parse(req.body)
  // ...
}
```

```
// Archivo: pages/api/submit.js switcher
import { z } from 'zod'

const schema = z.object({
  // ...
})

export default async function handler(req, res) {
  const parsed = schema.parse(req.body)
  // ...
}
```

Error handling

You can use React state to show an error message when a form submission fails:

```tsx
// Archivo: pages/index.tsx switcher
import React, { useState, SubmitEvent } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState<boolean>(false)
  const [error, setError] = useState<string | null>(null)

  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()
    setIsLoading(true)
    setError(null) // Clear previous errors when a new request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      if (!response.ok) {
        throw new Error('Failed to submit the data. Please try again.')
      }

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Capture the error message to display to the user
      setError(error.message)
      console.error(error)
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div>
      {error && <div style={{ color: 'red' }}>{error}</div>}
      <form onSubmit={onSubmit}>
        <input type="text" name="name" required />
        <button type="submit" disabled={isLoading}>
          {isLoading ? 'Loading ... ' : 'Submit'}
        </button>
      </form>
    </div>
  )
}
```

```
// Archivo: pages/index.jsx switcher
import React, { useState } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState(false)
  const [error, setError] = useState(null)

  async function onSubmit(event) {
    event.preventDefault()
    setIsLoading(true)
    setError(null) // Clear previous errors when a new request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      if (!response.ok) {
        throw new Error('Failed to submit the data. Please try again.')
      }

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Capture the error message to display to the user
      setError(error.message)
      console.error(error)
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div>
      {error && <div style={{ color: 'red' }}>{error}</div>}
      <form onSubmit={onSubmit}>
        <input type="text" name="name" required />
        <button type="submit" disabled={isLoading}>
          {isLoading ? 'Loading ... ' : 'Submit'}
        </button>
      </form>
    </div>
  )
}
```

Displaying loading state

You can use React state to show a loading state when a form is submitting on the server:

```tsx
// Archivo: pages/index.tsx switcher
import React, { useState, SubmitEvent } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState<boolean>(false)

  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()
    setIsLoading(true) // Set loading to true when the request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Handle error if necessary
      console.error(error)
    } finally {
      setIsLoading(false) // Set loading to false when the request completes
    }
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Submit'}
      </button>
    </form>
  )
}
```

```
// Archivo: pages/index.jsx switcher
import React, { useState } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState(false)

  async function onSubmit(event) {
    event.preventDefault()
    setIsLoading(true) // Set loading to true when the request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Handle error if necessary
      console.error(error)
    } finally {
      setIsLoading(false) // Set loading to false when the request completes
    }
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Submit'}
      </button>
    </form>
  )
}
```

Redirecting

If you would like to redirect the user to a different route after a mutation, you can `redirect` to any absolute or relative URL:

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const id = await addPost()
  res.redirect(307, `/post/${id}`)
}
```

```
// Archivo: pages/api/submit.js switcher
export default async function handler(req, res) {
  const id = await addPost()
  res.redirect(307, `/post/${id}`)
}
```

Error Handling

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/configuring/error-handling
> 📂 **Sección:** Pages > Building Your Application > Configuring
> 📝 **Descripción:** *Handle errors in your Next.js app.*

This documentation explains how you can handle development, server-side, and client-side errors.

Handling Errors in Development

When there is a runtime error during the development phase of your Next.js application, you will encounter an **overlay**. It is a modal that covers the webpage. It is **only** visible when the development server runs using `next dev` via `pnpm dev`, `npm run dev`, `yarn dev`, or `bun dev` and will not be shown in production. Fixing the error will automatically dismiss the overlay.

Handling Server Errors

Next.js provides a static 500 page by default to handle server-side errors that occur in your application. You can also customize this page by creating a `pages/500.js` file.

Having a 500 page in your application does not show specific errors to the app user.

You can also use 404 page to handle specific runtime error like `file not found`.

Handling Client Errors

React Error Boundaries is a graceful way to handle a JavaScript error on the client so that the other parts of the application continue working. In addition to preventing the page from crashing, it allows you to provide a custom fallback component and even log error information.

To use Error Boundaries for your Next.js application, you must create a class component `ErrorBoundary` and wrap the `Component` prop in the `pages/_app.js` file. This component will be responsible to:

- Render a fallback UI after an error is thrown
- Provide a way to reset the Application's state
- Log error information

You can create an `ErrorBoundary` class component by extending `React.Component`. For example:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props)

    // Define a state variable to track whether is an error or not
    this.state = { hasError: false }
  }
  static getDerivedStateFromError(error) {
    // Update state so the next render will show the fallback UI

    return { hasError: true }
  }
  componentDidCatch(error, errorInfo) {
    // You can use your own error logging service here
    console.log({ error, errorInfo })
  }
  render() {
    // Check if the error is thrown
    if (this.state.hasError) {
      // You can render any custom fallback UI
      return (
        <div>
          <h2>Oops, there is an error!</h2>
          <button
            type="button"
            onClick={() => this.setState({ hasError: false })}
          >
            Try again?
          </button>
        </div>
      )
    }

    // Return children components in case of no error

    return this.props.children
  }
}

export default ErrorBoundary
```

The `ErrorBoundary` component keeps track of an `hasError` state. The value of this state variable is a boolean. When the value of `hasError` is `true`, then the `ErrorBoundary` component will render a fallback UI. Otherwise, it will render the children components.

After creating an `ErrorBoundary` component, import it in the `pages/_app.js` file to wrap the `Component` prop in your Next.js application.

```
// Import the ErrorBoundary component
import ErrorBoundary from '../components/ErrorBoundary'

function MyApp({ Component, pageProps }) {
  return (
    // Wrap the Component prop with ErrorBoundary component
    <ErrorBoundary>
      <Component {...pageProps} />
    </ErrorBoundary>
  )
}

export default MyApp
```

You can learn more about Error Boundaries in React's documentation.

Reporting Errors

To monitor client errors, use a service like Sentry, Bugsnag or Datadog.

Capítulo 6: Directivas de Nueva Generación en Next.js 16

> *Guía exhaustiva de directivas: 'use cache', 'use cache: private', 'use cache: remote', 'use client' y 'use server'.*

Directives

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives
> 📂 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how React and Next.js directives define client entry points, Server Functions, and cached output.*

Directives are string literals that tell the compiler and bundler how to treat the code around them. On their own, directives look like plain strings, but Next.js and React read them as instructions that transform the code they cover. A directive can create a client entry point or a Server Function, or cache a function's or component's output.

Directive	Defined by	Effect
`'use client'`	React	Creates a client entry point from Server Components
`'use server'`	React	Exposes server-side functions as Server Functions
`'use cache'`	Next.js	Caches and reuses output based on the code's inputs

The `'use cache'` directive also has `'use cache: remote'` and `'use cache: private'` variants. Configure custom cache handlers to control where cached entries are stored.

> **Good to know:** Next.js applies directives during compilation in both development and production.
>
> The error overlay, indicators, and stack traces point to your source file and line rather than the generated output.

Where to place a directive

The `'use client'` directive must appear at the top of a file, before any imports. It applies to the entire module and defines a file-wide boundary because the module is bundled and shipped to the browser. You cannot use `'use client'` inline.

Place `'use server'` or `'use cache'` at the top of a file to apply the directive to every export. Place either directive at the top of a function to apply it only to that function.

```
// Archivo: app/data.ts switcher
// File-level: applies to every export
'use cache'

export async function getUser(id: string) {
  return { id }
}
```

```
// Archivo: app/data.js switcher
// File-level: applies to every export
'use cache'

export async function getUser(id) {
  return { id }
}
```

```
// Archivo: app/user.tsx switcher
// Function-level: applies to this function only
export async function User() {
  'use cache'
  return <p>User</p>
}
```

```
// Archivo: app/user.js switcher
// Function-level: applies to this function only
export async function User() {
  'use cache'
  return <p>User</p>
}
```

Declare `'use server'` and `'use cache'` in server modules, not in a `'use client'` module.

To import a `'use server'` or `'use cache'` function into a Client Component, place the directive at the top of the server module. The Client Component receives a reference that invokes the function on the server.

Place the directive inside a function when it should apply only to that function or component. When it should apply to several exported functions, place it at the top of the file instead of repeating it.

Every exported function covered by a file-level `'use server'` or `'use cache'` directive must be `async`.

What each directive requires

A directive imposes rules on the code it covers:

- `'use client'` marks its exports as the boundary between server and client. Client Component props that cross the boundary must be serializable. Ordinary functions, such as event handlers, cannot cross, but Server Functions can cross as references.
- `'use server'` marks the functions it covers as Server Functions. Server Functions must be `async`. When invoked from a Client Component, their arguments and return values are serialized across the network. The Client Component receives a reference that invokes the function on the server, not the function's code.
- `'use cache'` caches the output of the functions or components it covers based on their inputs. Cached functions and components must be `async`. Their arguments and return values must be serializable, except for non-serializable values that the cached code passes through without inspecting. Cached code cannot read request-time APIs like `cookies()`, `headers()`, or `searchParams` directly.

With `'use cache'` at the top of a page or layout file, exported functions such as `generateMetadata` and `generateStaticParams` must be `async`.

Placement decides what you cache

With `'use cache'`, placement determines the scope of a cache entry. A directive at the top of a page caches the page's output and the components it imports. The same directive inside a data function caches only that function's result. When one data request performs the expensive work, cache that function instead of the page.

With `'use cache: remote'`, a remote cache handler stores entries. Larger entries can increase storage and network costs.

---

use cache: private

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives/use-cache-private
> 📂 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how to use the "use cache: private" directive to cache functions that access runtime request APIs.*

The `'use cache: private'` directive allows functions to access runtime request APIs like `cookies()`, `headers()`, and `searchParams` within a cached scope. In production, matching calls within one request can reuse the same result, but Next.js does not store it in a server cache across requests.

The client router can keep the rendered output in browser memory for the `stale` time configured with `cacheLife`. This client-side cache does not persist across page reloads.

Reach for `'use cache: private'` when:

- You want to cache a function that already accesses runtime data, and refactoring to move the runtime access outside and pass values as arguments is not practical.
- You need request-specific data to be excluded from server caches that persist across production requests.

Private Cache Functions run at request time and are excluded from static shell generation. To start a private Cache Function before a component needs its result, see Preloading data.

It is **not** possible to configure custom cache handlers for `'use cache: private'`.

For a comparison of the different cache directives, see How `use cache: remote` differs from `use cache` and `use cache: private`.

Usage

To use `'use cache: private'`, enable the `cacheComponents` flag in your `next.config.ts` file:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

Then add `'use cache: private'` to your function along with a `cacheLife` configuration.

Basic example

In this example, we demonstrate that you can access cookies within a `'use cache: private'` scope:

```tsx
// Archivo: app/product/[id]/page.tsx switcher
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { cacheLife, cacheTag } from 'next/cache'

export async function generateStaticParams() {
  return [{ id: '1' }]
}

export default async function ProductPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params

  return (
    <div>
      <ProductDetails id={id} />
      ``<Suspense>``
    </div>
  )
}

async function Recommendations({ productId }: { productId: string }) {
  const recommendations = await getRecommendations(productId)

  return (
    <div>
      {recommendations.map((rec) => (
        <ProductCard key={rec.id} product={rec} />
      ))}
    </div>
  )
}

async function getRecommendations(productId: string) {
  'use cache: private'
  cacheTag(`recommendations-${productId}`)
  cacheLife({ stale: 60 })

  // Access cookies within private cache functions
  const sessionId = (await cookies()).get('session-id')?.value || 'guest'

  return getPersonalizedRecommendations(productId, sessionId)
}
```

```
// Archivo: app/product/[id]/page.js switcher
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { cacheLife, cacheTag } from 'next/cache'

export async function generateStaticParams() {
  return [{ id: '1' }]
}

export default async function ProductPage({ params }) {
  const { id } = await params

  return (
    <div>
      <ProductDetails id={id} />
      ``<Suspense>``
    </div>
  )
}

async function Recommendations({ productId }) {
  const recommendations = await getRecommendations(productId)

  return (
    <div>
      {recommendations.map((rec) => (
        <ProductCard key={rec.id} product={rec} />
      ))}
    </div>
  )
}

async function getRecommendations(productId) {
  'use cache: private'
  cacheTag(`recommendations-${productId}`)
  cacheLife({ stale: 60 })

  // Access cookies within private cache functions
  const sessionId = (await cookies()).get('session-id')?.value || 'guest'

  return getPersonalizedRecommendations(productId, sessionId)
}
```

> **Good to know:** The `stale` time must be at least 30 seconds for per-link prefetching to work, and at least 5 minutes for the content to be included in the route's App Shell. See `cacheLife` prerendering behavior for details.

Configuring the client stale time

Private Cache Functions contribute their `stale` time to the route's Client Cache. If a function only needs request-scoped deduplication, use `cacheLife({ stale: Infinity })` to keep it from lowering the route's stale time.

Next.js uses the shortest stale time from the route's cache entries, so another cache or route setting can still set a finite value. Setting `stale` to `Infinity` does not store the private result on the server across production requests. Use a finite value when the client router should revalidate personalized output after a known interval.

Request APIs allowed in private caches

The following request-specific APIs can be used inside `'use cache: private'` functions:

API	Allowed in `use cache`	Allowed in `'use cache: private'`
`cookies()`	No	Yes
`headers()`	No	Yes
`searchParams`	No	Yes
`connection()`	No	No

> **Note:** The `connection()` API is prohibited in both `use cache` and `'use cache: private'` as it provides connection-specific information that cannot be safely cached.

Version History

Version	Changes
`v16.0.0`	`"use cache: private"` is enabled with the Cache Components feature.

use cache: remote

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives/use-cache-remote
> 📁 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how to use the "use cache: remote" directive for persistent, shared caching using remote cache handlers.*

While the `use cache` directive is sufficient for most application needs, you might notice that cached operations are re-running more often than expected, or that your upstream services (CMS, databases, external APIs) are getting more hits than you'd expect. This can happen because `use cache` stores entries in-memory, which has inherent limitations:

- Cache entries being evicted to make room for new ones
- Memory constraints in your deployment environment
- Cache not persisting across requests or server restarts

Note that `use cache` still provides value beyond server-side caching: it informs Next.js what can be prefetched and defines stale times for client-side navigation.

The `'use cache: remote'` directive lets you declaratively specify that a cached output should be stored in a remote cache instead of in-memory, providing durable caching shared across all server instances. This comes with tradeoffs: infrastructure cost and network latency during cache lookups.

Usage

To use `'use cache: remote'`, enable the `cacheComponents` flag in your `next.config.ts` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

Then add `'use cache: remote'` to the functions or components where you've determined remote caching is justified. The handler implementation is configured via `cacheHandlers`, though hosting providers should typically provide this automatically. If you're self-hosting, see the `cacheHandlers` configuration reference to set up your cache storage.

When to avoid remote caching

- If you already have a server-side cache key-value store wrapping your data layer, `use cache` may be sufficient to include data in the static shell without adding another caching layer
- If operations are already fast (< 50ms) due to proximity or local access, the remote cache lookup might not improve performance
- If cache keys have mostly unique values per request (search filters, price ranges, user-specific parameters), cache utilization will be near-zero
- If data changes frequently (seconds to minutes), cache hits will quickly go stale, leading to frequent misses and waiting for upstream revalidation

When remote caching makes sense

Remote caching provides the most value when content is deferred to request time (outside the static shell). This typically happens when a component accesses request values like `cookies()`, `headers()`, or `searchParams`, placing it inside a Suspense boundary. In this context:

- Each request executes the component and looks up the cache
- In serverless environments, each instance has its own ephemeral memory with low cache hit rates
- Remote caching provides a shared cache across all instances, improving hit rates and reducing backend load

Compelling scenarios for `'use cache: remote'`:

- **Rate-limited APIs**: Your upstream service has rate limits or request quotas that you risk hitting
- **Protecting slow backends**: Your database or API becomes a bottleneck under high traffic
- **Expensive operations**: Database queries or computations that are costly to run repeatedly
- **Flaky or unreliable services**: External services that occasionally fail or have availability issues

In these cases, the cost and latency of remote caching is justified by avoiding worse outcomes (rate limit errors, backend overload, high compute bills, or degraded user experience).

For static shell content, `use cache` is usually sufficient. If your upstream source can't handle concurrent revalidation requests (like a rate-limited CMS), `use cache: remote` acts as a shared cache layer. This is the same pattern as putting a key-value store in front of a database, but declared in code.

How `use cache: remote` differs from `use cache` and `use cache: private`

Next.js provides three caching directives, each designed for different use cases:

Feature	`use cache`	`'use cache: remote'`	`'use cache: private'`
Server-side caching	In-memory or cache handler	Remote cache handler	None
Cache scope	Shared across all users	Shared across all users	Per-client (browser)
Can access cookies/headers directly	No (must pass as arguments)	No (must pass as arguments)	Yes
Server cache utilization	May be low outside static shell	High (shared across instances)	N/A
Additional costs	None	Infrastructure (storage, network)	None
Latency impact	None	Cache handler lookup	None
Persists across deploys	No	No	N/A

Persistence across deploys

Remote cache entries do not persist across deploys. The cache key includes the `deploymentId` (when configured) or the `buildId`, so a new build produces new keys and the previous build's entries are no longer reachable. See Cache keys for the full key composition.

This is intentional. Between builds, the function's identity hash or the shape of its return value can change. Upgrading a CMS client, refactoring a cached function, or changing a dependency could produce a value that doesn't match what older callers expect, so reusing entries across deploys risks serving stale or malformed data.

If you need entries that persist across deploys, use `unstable_cache` for non-`fetch` functions, or rely on the `fetch` cache.

Caching with runtime data

Both `use cache` and `'use cache: remote'` can't access runtime values like cookies or search params directly. You can extract these values and pass them as arguments to cached functions. See with runtime data for this pattern.

> **Good to know**: `use cache` stores entries in-memory. In serverless environments, memory is not shared between instances and is typically destroyed after serving a request, leading to frequent cache misses for runtime caching.

Cache key considerations

Be thoughtful about which values you include in cache keys. Each unique value creates a separate cache entry, reducing cache utilization. Consider this example with search filters:

```tsx
// Archivo: app/products/[category]/page.tsx
import { Suspense } from 'react'

export default async function ProductsPage({
  params,
  searchParams,
}: {
  params: Promise<{ category: string }>
  searchParams: Promise<{ minPrice?: string }>
}) {
  return (
    ``<Suspense>``
  )
}

async function ProductList({
  params,
  searchParams,
}: {
  params: Promise<{ category: string }>
  searchParams: Promise<{ minPrice?: string }>
}) {
  const { category } = await params

  const { minPrice } = await searchParams

  // Cache only on category (few unique values)
  // Don't include price filter (many unique values)
  const products = await getProductsByCategory(category)

  // Filter price in memory instead of creating cache entries
  // for every price value
  const filtered = minPrice
    ? products.filter((p) => p.price ≥ parseFloat(minPrice))
    : products

  return <div></div>
}

async function getProductsByCategory(category: string) {
  'use cache: remote'
  // Only category is part of the cache key
  // Much better utilization than caching every price filter value
  return db.products.findByCategory(category)
}
```

In this example, the remote handler stores more data per cache entry (all products in a category) to achieve better cache hit rates. This is worth it when the cost of cache misses (hitting your backend) outweighs the storage cost of larger entries.

The same principle applies to user-specific data. Rather than caching per-user data directly, use user preferences to determine what shared data to cache.

For example, if users have a language preference in their session, extract that preference and use it to cache shared content:

- Instead of remote caching `getUserProfile(sessionID)`, which creates one entry per user
- Remote cache `getCMSContent(language)` to create one entry per language

```
// Archivo: app/components/welcome-message.tsx
import { cookies } from 'next/headers'
import { cacheLife } from 'next/cache'

export async function WelcomeMessage() {
  // Extract the language preference (not unique per user)
  const language = (await cookies()).get('language')?.value || 'en'

  // Cache based on language (few unique values: en, es, fr, de, etc.)
  // All users who prefer 'en' share the same cache entry
  const content = await getCMSContent(language)

  return <div>{content.welcomeMessage}</div>
}

async function getCMSContent(language: string) {
  'use cache: remote'
  cacheLife({ expire: 3600 })
  // Creates ~10-50 cache entries (one per language)
  // instead of thousands (one per user)
  return cms.getHomeContent(language)
}
```

This way all users who prefer the same language share a cache entry, improving cache utilization and reducing load on your CMS.

The pattern is the same in both examples: find the dimension with fewer unique values (category vs. price, language vs. user ID), cache on that dimension, and filter or select the rest in memory.

If the service used by `getUserProfile` cannot scale with your frontend load, you may still be able to use the `use cache` directive with a short `cacheLife` for in-memory caching. However, for most user data, you likely want to fetch directly from the source (which might already be wrapped in a key/value store as mentioned in the guidelines above).

Only use `'use cache: private'` if you have compliance requirements or can't refactor to pass runtime data as arguments.

Nesting rules

Remote caches have specific nesting rules:

- Remote caches **can** be nested inside other remote caches (`'use cache: remote'`)
- Remote caches **can** be nested inside regular caches (`'use cache'`)
- Remote caches **cannot** be nested inside private caches (`'use cache: private'`)
- Private caches **cannot** be nested inside remote caches

```
// VALID: Remote inside remote
async function outerRemote() {
  'use cache: remote'
  const result = await innerRemote()
  return result
}

async function innerRemote() {
  'use cache: remote'
  return getData()
}

// VALID: Remote inside regular cache
async function outerCache() {
  'use cache'
  // The inner remote cache will work when deferred to request time
  const result = await innerRemote()
  return result
}

async function innerRemote() {
  'use cache: remote'
  return getData()
}

// INVALID: Remote inside private
async function outerPrivate() {
  'use cache: private'
  const result = await innerRemote() // Error!
  return result
}

async function innerRemote() {
  'use cache: remote'
  return getData()
}

// INVALID: Private inside remote
async function outerRemote() {
  'use cache: remote'
  const result = await innerPrivate() // Error!
  return result
}

async function innerPrivate() {
  'use cache: private'
  return getData()
}
```

Examples

The following examples demonstrate common patterns for using `'use cache: remote'`. For details about `cacheLife` parameters (`stale`, `revalidate`, `expire`), see the `cacheLife` API reference.

With user preferences

Cache product pricing based on the user's currency preference. Since the currency is stored in a cookie, this component renders at request time. Remote caching is valuable here because all users with the same currency share the cached price, and in serverless environments, all instances share the same remote cache.

```tsx
// Archivo: app/product/[id]/page.tsx switcher
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { cacheTag, cacheLife } from 'next/cache'

export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

export default async function ProductPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params

  return (
    <div>
      <ProductDetails id={id} />
      ``<Suspense>``
    </div>
  )
}

function ProductDetails({ id }: { id: string }) {
  return <div>Product: {id}</div>
}

async function ProductPrice({ productId }: { productId: string }) {
  // Reading cookies defers this component to request time
  const currency = (await cookies()).get('currency')?.value ?? 'USD'

  // Cache the price per product and currency combination
  // All users with the same currency share this cache entry
  const price = await getProductPrice(productId, currency)

  return (
    <div>
      Price: {price} {currency}
    </div>
  )
}

async function getProductPrice(productId: string, currency: string) {
  'use cache: remote'
  cacheTag(`product-price-${productId}`)
  cacheLife({ expire: 3600 }) // 1 hour

  // Cached per (productId, currency) - few currencies means high cache utilization
  return db.products.getPrice(productId, currency)
}
```

```
// Archivo: app/product/[id]/page.js switcher
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { cacheTag, cacheLife } from 'next/cache'

export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

export default async function ProductPage({ params }) {
  const { id } = await params

  return (
    <div>
      <ProductDetails id={id} />
      ``<Suspense>``
    </div>
  )
}

function ProductDetails({ id }) {
  return <div>Product: {id}</div>
}

async function ProductPrice({ productId }) {
  // Reading cookies defers this component to request time
  const currency = (await cookies()).get('currency')?.value ?? 'USD'

  // Cache the price per product and currency combination
  // All users with the same currency share this cache entry
  const price = await getProductPrice(productId, currency)

  return (
    <div>
      Price: {price} {currency}
    </div>
  )
}

async function getProductPrice(productId, currency) {
  'use cache: remote'
  cacheTag(`product-price-${productId}`)
  cacheLife({ expire: 3600 }) // 1 hour

  // Cached per (productId, currency) - few currencies means high cache utilization
  return db.products.getPrice(productId, currency)
}
```

Reducing database load

Cache expensive database queries, reducing load on your database. In this example, we don't access `cookies()`, `headers()`, or `searchParams`. If we had a requirement to not include these stats in the static shell, we could use `connection()` to explicitly defer to request time:

```
// Archivo: app/dashboard/page.tsx
import { Suspense } from 'react'
import { connection } from 'next/server'
import { cacheLife, cacheTag } from 'next/cache'

export default function DashboardPage() {
  return (
    ``<Suspense>``
  )
}

async function DashboardStats() {
  // Defer to request time
  await connection()

  const stats = await getGlobalStats()

  return <StatsDisplay stats={stats} />
}

async function getGlobalStats() {
  'use cache: remote'
  cacheTag('global-stats')
  cacheLife({ expire: 60 }) // 1 minute

  // This expensive database query is cached and shared across all users,
  // reducing load on your database
  const stats = await db.analytics.aggregate({
    total_users: 'count',
    active_sessions: 'count',
    revenue: 'sum',
  })

  return stats
}
```

With this setup, your upstream database sees at most one request per minute, regardless of how many users visit the dashboard.

API responses in streaming contexts

Cache API responses that are fetched during streaming or after dynamic operations:

```
// Archivo: app/feed/page.tsx
import { Suspense } from 'react'
import { connection } from 'next/server'
import { cacheLife, cacheTag } from 'next/cache'

export default async function FeedPage() {
  return (
    <div>
      ``<Suspense>``}>
        <FeedItems />
      `<Suspense>`
    </div>
  )
}

async function FeedItems() {
  // Defer to request time
  await connection()

  const items = await getFeedItems()

  return items.map((item) => <FeedItem key={item.id} item={item} />)
}

async function getFeedItems() {
  'use cache: remote'
  cacheTag('feed-items')
  cacheLife({ expire: 120 }) // 2 minutes

  // This API call is cached, reducing requests to your external service
  const response = await fetch('https://api.example.com/feed')
  return response.json()
}
```

Computed data after dynamic checks

Cache expensive computations that occur after dynamic security or feature checks:

```
// Archivo: app/reports/page.tsx
import { connection } from 'next/server'
import { cacheLife } from 'next/cache'

export default async function ReportsPage() {
  // Defer to request time (for security check)
  await connection()

  const report = await generateReport()

  return <ReportViewer report={report} />
}

async function generateReport() {
  'use cache: remote'
  cacheLife({ expire: 3600 }) // 1 hour

  // This expensive computation is cached and shared across all authorized users,
  // avoiding repeated calculations
  const data = await db.transactions.findMany()

  return {
    totalRevenue: calculateRevenue(data),
    topProducts: analyzeProducts(data),
    trends: calculateTrends(data),
  }
}
```

Mixed caching strategies

Combine static, remote, and private caching for optimal performance:

```tsx
// Archivo: app/product/[id]/page.tsx
import { Suspense } from 'react'
import { connection } from 'next/server'
import { cookies } from 'next/headers'
import { cacheLife, cacheTag } from 'next/cache'

// Static product data - prerendered at build time
async function getProduct(id: string) {
  'use cache'
  cacheTag(`product-${id}`)

  // This is cached at build time and shared across all users
  return db.products.find({ where: { id } })
}

// Shared pricing data - cached at runtime in remote handler
async function getProductPrice(id: string) {
  'use cache: remote'
  cacheTag(`product-price-${id}`)
  cacheLife({ expire: 300 }) // 5 minutes

  // This is cached at runtime and shared across all users
  return db.products.getPrice({ where: { id } })
}

// User-specific recommendations - private cache per user
async function getRecommendations(productId: string) {
  'use cache: private'
  cacheLife({ expire: 60 }) // 1 minute

  const sessionId = (await cookies()).get('session-id')?.value

  // This is cached per-user and never shared
  return db.recommendations.findMany({
    where: { productId, sessionId },
  })
}

export default async function ProductPage({ params }) {
  const { id } = await params

  // Static product data
  const product = await getProduct(id)

  return (
    <div>
      <ProductDetails product={product} />


      ``<Suspense>``}>
        <ProductPriceComponent productId={id} />
      `<Suspense>`


      ``<Suspense>``}>
        <ProductRecommendations productId={id} />
      `<Suspense>`
    </div>
  )
}

function ProductDetails({ product }) {
  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
    </div>
  )
}

async function ProductPriceComponent({ productId }) {
  // Defer to request time
  await connection()
```

```
  const price = await getProductPrice(productId)
  return <div>Price: ${price}</div>
}

async function ProductRecommendations({ productId }) {
  const recommendations = await getRecommendations(productId)
  return <RecommendationsList items={recommendations} />
}

function PriceSkeleton() {
  return <div>Loading price ... </div>
}

function RecommendationsSkeleton() {
  return <div>Loading recommendations ... </div>
}

function RecommendationsList({ items }) {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  )
}
```

Good to know:

- Remote caches are stored in server-side cache handlers and shared across all users
- `'use cache: remote'` works outside the static shell where `use cache` may not provide server-side cache hits
- Use `cacheTag()` and `revalidateTag()` to invalidate remote caches on-demand
- Use `cacheLife()` to configure cache expiration
- For user-specific data, use `'use cache: private'` instead of `'use cache: remote'`
- Remote caches reduce origin load by storing computed or fetched data server-side

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Yes

Version History

Version	Changes
`v16.0.0`	`"use cache: remote"` is enabled with the Cache Components feature.

use cache

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives/use-cache
> 📂 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how to use the "use cache" directive to cache data in your Next.js application.*

The `use cache` directive allows you to mark a route, React component, or a function as cacheable. It can be used at the top of a file to indicate that all exports in the file should be cached, or inline at the top of a function or component to cache the return value. Functions and components that use `use cache` must be async.

> **Good to know:**
>
> - To use cookies or headers, read them outside cached scopes and pass values as arguments. This is the preferred pattern.
> - If the in-memory cache isn't sufficient for runtime data, `'use cache: remote'` allows platforms to provide a dedicated cache handler, though it requires a network roundtrip to check the cache and typically incurs platform fees.
> - For compliance requirements or when you can't refactor to pass runtime data as arguments to a `use cache` scope, see `'use cache: private'`.

Usage

`use cache` is a Cache Components feature. To enable it, add the `cacheComponents` option to your `next.config.ts` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

Then, add `use cache` at the function, component, or file level. Cached functions and components must be async:

```
// Function level
export async function getData() {
  'use cache'
  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}

// Component level
export async function MyComponent() {
  'use cache'
  return <></>
}
```

When used at file level, every exported function becomes a cached function and must also be async:

```
// File level
'use cache'

export default async function Page() {
  // ...
}
```

How `use cache` works

Cache keys

A cache entry's key is generated using a serialized version of its inputs, which includes:

1. **Build ID** - Unique per build, changing this invalidates all cache entries. If `deploymentId` is configured, it overrides the build ID for cache key purposes.
2. **Function ID** - A secure hash of the function's location and signature in the codebase
3. **Serializable arguments** - Props (for components) or function arguments
4. **HMR refresh hash** (development only) - Invalidates cache on hot module replacement

When a cached function references variables from outer scopes, those variables are automatically captured and bound as arguments, making them part of the cache key.

```
// Archivo: lib/data.ts
async function Component({ userId }: { userId: string }) {
  const getData = async (filter: string) => {
    'use cache'
    // Cache key includes both userId (from closure) and filter (argument)
    const res = await fetch(
      `https://api.example.com/users/${userId}/data?filter=${filter}`
    )
    return res.json()
  }

  return getData('active')
}
```

In the snippet above, `userId` is captured from the outer scope and `filter` is passed as an argument, so both become part of the `getData` function's cache key. This means different user and filter combinations will have separate cache entries.

> **Good to know:** When a cached function reads root parameters, only the ones it actually reads become part of its cache key.

Cache output

A cached function produces the same output for the same inputs. The first call with a given set of inputs runs the function body and stores its output. Every later call with the same inputs reuses that output, within a render pass and across requests, for as long as the entry lasts.

Outputs are stored by a cache handler, in memory by default, and last until they revalidate.

```
// Archivo: lib/orders.ts
import { cacheLife } from 'next/cache'

export async function getOrderSummary(accountId: string) {
  'use cache'
  cacheLife('hours')

  const orders = await getOrders(accountId)
  const totals = await getOrderTotals(accountId)

  return { orders, totals }
}

export async function getOrders(accountId: string) {
  'use cache'
  cacheLife('hours')

  return db.orders.findMany({ where: { accountId } })
}

export async function getOrderTotals(accountId: string) {
  return db.orders.aggregate({ where: { accountId }, _sum: { amount: true } })
}
```

In the example above, each `accountId` has its own entry for `getOrderSummary`, holding the serialized `{ orders, totals }` object it returns. Calls to `getOrders` have their own entries, keyed by the same `accountId`. If an earlier call, through the summary or directly, already filled one, `db.orders.findMany` doesn't run and those orders become part of the summary's output. See nested caching behavior for how an inner lifetime affects the entry around it.

Note that `getOrderTotals` is not cached. It is exported so other parts of the application can read fresh totals in an uncached scope. Inside `getOrderSummary`, though, the totals query only runs when that function runs following the `'hours'` lifetime set there.

With Cache Components, prerendering fills the entry and keeps rendering, so this output contributes to the route's static shell and can contribute to its prefetch. A cache life too short to store safely leaves a hole that resolves at request time instead. See prerendering behavior for the thresholds.

Serialization

Arguments to cached functions and their return values must be serializable.

For a complete reference, see:

- Serializable arguments - Uses **React Server Components** serialization
- Serializable return types - Uses **React Client Components** serialization

> **Good to know:** Arguments and return values use different serialization systems. Server Component serialization (for arguments) is more restrictive than Client Component serialization (for return values). This means you can return JSX elements but cannot accept them as arguments unless using pass-through patterns.

Supported types

Arguments:

- Primitives: `string`, `number`, `boolean`, `null`, `undefined`
- Plain objects: `{ key: value }`
- Arrays: `[1, 2, 3]`
- Dates, Maps, Sets, TypedArrays, ArrayBuffers
- React elements (as pass-through only)

Return values:

- Same as arguments, plus JSX elements

Unsupported types

- Class instances
- Functions (except as pass-through)
- Symbols, WeakMaps, WeakSets
- URL instances

```
// Archivo: app/components/user-card.tsx
// Valid - primitives and plain objects
async function UserCard({
  id,
  config,
}: {
  id: string
  config: { theme: string }
}) {
  'use cache'
  return <div>{id}</div>
}

// Invalid - class instance
async function UserProfile({ user }: { user: UserClass }) {
  'use cache'
  // Error: Cannot serialize class instance
  return <div>{user.name}</div>
}
```

Pass-through (non-serializable arguments)

You can accept non-serializable values **as long as you don't introspect them**. This enables composition patterns with `children` and Server Actions:

```
// Archivo: app/components/cached-wrapper.tsx
async function CachedWrapper({ children }: { children: ReactNode }) {
  'use cache'
  // Don't read or modify children - just pass it through
  return (
    <div className="wrapper">
      <header>Cached Header</header>
      {children}
    </div>
  )
}

// Usage: children can be dynamic
export default function Page() {
  return (
    <CachedWrapper>
      <DynamicComponent />
    </CachedWrapper>
  )
}
```

You can also pass Server Actions through cached components:

```
// Archivo: app/components/cached-form.tsx
async function CachedForm({ action }: { action: () => Promise<void> }) {
  'use cache'
  // Don't call action here - just pass it through
  return <form action={action}></form>
}
```

Constraints

Cached functions execute in an isolated environment. The following constraints ensure cache behavior remains predictable and secure.

Request-time APIs

Cached functions and components **cannot** access runtime APIs like `cookies()`, `headers()`, or `searchParams`, and the restriction follows the call stack: a helper the cached function calls that reads one of these fails the same way, with the `next-request-in-use-cache` error. On a dynamically rendered route this surfaces when the route runs, so it can pass `next build` and fail under `next start`. Read these values outside the cached scope and pass them as arguments.

Runtime caching considerations

While `use cache` is designed primarily to include uncached data in the static shell, it can also cache data at runtime using in-memory LRU (Least Recently Used) storage.

With the default in-memory handler, runtime cache behavior depends on your hosting environment:

Environment	Runtime Caching Behavior
Serverless	Cache entries typically don't persist across requests (each request can be a different instance), or during revalidation. Build-time caching works normally.
Self-hosted	Cache entries persist across requests. Control cache size with `cacheMaxMemorySize`.

For example, in a serverless environment, a cached function shared by two pages executes on each static shell revalidation, whereas in self-hosted or environments with persistent memory, the cached output is reused if it's still fresh.

If the default in-memory cache isn't enough, consider `use cache: remote` which allows platforms to provide a dedicated cache handler (like Redis or KV database). This helps reduce hits against data sources not scaled to your total traffic, though it comes with costs (storage, network latency, platform fees).

> [!NOTE] With the default in-memory handler, serverless instances are ephemeral, so entries may not be reused between requests, unlike with `use cache: remote`. Neither caching directive carries over to a new deploy, because the cache key includes the build (or `deploymentId`) ID.
>
> For data that needs to persist across deploys, use `unstable_cache` for non-`fetch` functions or the `fetch` cache.

Very rarely, for compliance requirements or when you can't refactor your code to pass runtime data as arguments to a `use cache` scope, you might need `use cache: private`.

Draft Mode

When Draft Mode is enabled, all cached functions and components re-execute on every request, and results are not saved to the cache. This ensures draft content is always fresh without requiring any changes to your caching code.

You can read `isEnabled` from `draftMode()` inside a `use cache` scope, however, other runtime APIs like `cookies()` and `headers()` are not allowed, even when Draft Mode is active. See Passing runtime values to cached functions for the recommended pattern.

```
// Archivo: app/components/content.tsx
import { draftMode } from 'next/headers'

async function Content() {
  'use cache'

  const { isEnabled } = await draftMode()
  const url = isEnabled
    ? 'https://draft.example.com/content'
    : 'https://production.example.com/content'

  const data = await fetch(url)
  return <article></article>
}
```

Calling `enable()` or `disable()` inside a caching directive scope will also throw an error. Draft Mode can only be toggled in Route Handlers or Server Actions.

React.cache isolation

`React.cache` operates in an isolated scope inside `use cache` boundaries. Values stored via `React.cache` outside a `use cache` function are not visible inside it.

This means you cannot use `React.cache` to pass data into a `use cache` scope:

```
import { cache } from 'react'

const store = cache(() => ({ current: null as string | null }))

function Parent() {
  const shared = store()
  shared.current = 'value from parent'
  return <Child />
}

async function Child() {
  'use cache'
  const shared = store()
  // shared.current is null, not 'value from parent'
  // use cache has its own isolated React.cache scope
  return <div>{shared.current}</div>
}
```

This isolation ensures cached functions have predictable, self-contained behavior. To pass data into a `use cache` scope, use function arguments instead.

`use cache` at runtime

On the **server**, cache entries are stored in-memory and respect the `revalidate` and `expire` times from your `cacheLife` configuration. You can customize the cache storage by configuring `cacheHandlers` in your `next.config.js` file.

On the **client**, content from the server cache is stored in the browser's memory for the duration defined by the `stale` time. The client router enforces a **minimum 30-second stale time**, regardless of configuration.

The `x-nextjs-stale-time` response header communicates cache lifetime from server to client, ensuring coordinated behavior.

Revalidation

Cached functions revalidate based on the `revalidate` and `expire` times in their `cacheLife` profile, or on-demand through tags. These two approaches are not mutually exclusive and are often paired:

- **Time-based**: refresh automatically after a set duration with `cacheLife`.
- **On-demand**: invalidate after a mutation with `cacheTag` and `revalidateTag` or `updateTag`.

For example, a blog post that changes only when its author edits it can use a long `cacheLife` like `max` with a `cacheTag`, then invalidate on demand when the post is saved. A list of recent posts that updates throughout the day can use a shorter profile like `hours` to refresh on its own, without manual invalidation.

Time-based revalidation

Set an explicit cache lifetime with `cacheLife` in every `use cache` scope. It makes the cache behavior clear at the call site, instead of depending on the `default` profile or surrounding caches.

```
// Archivo: lib/data.ts
import { cacheLife } from 'next/cache'

async function getData() {
  'use cache'
  cacheLife('hours') // Use built-in 'hours' profile
  const res = await fetch('https://api.example.com/data')
  return res.json()
}
```

If you omit `cacheLife`, the `default` profile applies and the lifetime is no longer explicit at the call site:

- **stale**: 5 minutes (client-side)
- **revalidate**: 15 minutes (server-side)
- **expire**: never expires by time

```
// Archivo: lib/data.ts
async function getData() {
  'use cache'
  // Implicitly uses the 'default' profile
  const res = await fetch('https://api.example.com/data')
  return res.json()
}
```

Nesting a short-lived use cache inside one without an explicit cacheLife fails the build during prerendering. See Nested short-lived caches for the rule and fix.

On-demand revalidation

Use `cacheTag`, `updateTag`, or `revalidateTag` for on-demand cache invalidation:

```
// Archivo: lib/data.ts
import { cacheTag } from 'next/cache'

async function getProducts() {
  'use cache'
  cacheTag('products')
  const res = await fetch('https://api.example.com/products')
  return res.json()
}
```

```
// Archivo: app/actions.ts
'use server'

import { updateTag } from 'next/cache'

export async function updateProduct() {
  await db.products.update( ... )
  updateTag('products') // Invalidates all 'products' caches
}
```

Both `cacheLife` and `cacheTag` integrate across client and server caching layers, meaning you configure your caching semantics in one place and they apply everywhere.

Examples

Caching function output with `use cache`

You can add `use cache` to any asynchronous function, not only to components and routes. You might want to cache a network request, a database query, or a slow computation.

```ts
// Archivo: app/actions.ts highlight={2} switcher
export async function getData() {
  'use cache'

  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

```js
// Archivo: app/actions.js highlight={2} switcher
export async function getData() {
  'use cache'

  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

Caching a component's output with `use cache`

You can use `use cache` at the component level to cache any fetches or computations performed within that component. The cache entry will be reused as long as the serialized props produce the same value in each instance.

```ts
// Archivo: app/components/bookings.tsx highlight={2} switcher
export async function Bookings({ type = 'haircut' }: BookingsProps) {
  'use cache'
  async function getBookingsData() {
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    return data
  }
  return // ...
}

interface BookingsProps {
  type: string
}
```

```js
// Archivo: app/components/bookings.js highlight={2} switcher
export async function Bookings({ type = 'haircut' }) {
  'use cache'
  async function getBookingsData() {
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    return data
  }
  return // ...
}
```

Interleaving

In React, composition with `children` or slots is a well-known pattern for building flexible components. When using `use cache`, you can continue to compose your UI in this way. Anything included as `children`, or other compositional slots, in the returned JSX will be passed through the cached component without affecting its cache entry.

As long as you don't directly reference any of the JSX slots inside the body of the cacheable function itself, their presence in the returned output won't affect the cache entry.

```tsx
// Archivo: app/page.tsx switcher
export default async function Page() {
  const uncachedData = await getData()
  return (
    // Pass compositional slots as props, e.g. header and children
    <CacheComponent header={<h1>Home</h1>}>

      <DynamicComponent data={uncachedData} />
    </CacheComponent>
  )
}

async function CacheComponent({
  header, // header: a compositional slot, injected as a prop
  children, // children: another slot for nested composition
}: {
  header: ReactNode
  children: ReactNode
}) {
  'use cache'
  const res = await fetch('https://api.example.com/cached-data')
  const cachedData = await res.json()
  return (
    <div>
      {header}
      <PrerenderedComponent data={cachedData} />
      {children}
    </div>
  )
}
```

```jsx
// Archivo: app/page.js switcher
export default async function Page() {
  const uncachedData = await getData()
  return (
    // Pass compositional slots as props, e.g. header and children
    <CacheComponent header={<h1>Home</h1>}>

      <DynamicComponent data={uncachedData} />
    </CacheComponent>
  )
}

async function CacheComponent({
  header, // header: a compositional slot, injected as a prop
  children, // children: another slot for nested composition
}) {
  'use cache'
  const res = await fetch('https://api.example.com/cached-data')
  const cachedData = await res.json()
  return (
    <div>
      {header}
      <PrerenderedComponent data={cachedData} />
      {children}
    </div>
  )
}
```

You can also pass Server Actions through cached components to Client Components without invoking them inside the cacheable function.

```tsx
// Archivo: app/page.tsx switcher
import ClientComponent from './ClientComponent'

export default async function Page() {
  const performUpdate = async () => {
    'use server'
    // Perform some server-side update
    await db.update( ... )
  }

  return <CachedComponent performUpdate={performUpdate} />
}

async function CachedComponent({
  performUpdate,
}: {
  performUpdate: () => Promise<void>
}) {
  'use cache'
  // Do not call performUpdate here
  return <ClientComponent action={performUpdate} />
}
```

```jsx
// Archivo: app/page.js switcher
import ClientComponent from './ClientComponent'

export default async function Page() {
  const performUpdate = async () => {
    'use server'
    // Perform some server-side update
    await db.update( ... )
  }

  return <CachedComponent performUpdate={performUpdate} />
}

async function CachedComponent({ performUpdate }) {
  'use cache'
  // Do not call performUpdate here
  return <ClientComponent action={performUpdate} />
}
```

```tsx
// Archivo: app/ClientComponent.tsx switcher
'use client'

export default function ClientComponent({
  action,
}: {
  action: () => Promise<void>
}) {
  return <button onClick={action}>Update</button>
}
```

```jsx
// Archivo: app/ClientComponent.js switcher
'use client'

export default function ClientComponent({ action }) {
  return <button onClick={action}>Update</button>
}
```

Caching a module's exports with `use cache`

Placing the directive at the top of a file covers every export, instead of repeating it on each one. Every exported function it covers must be async.

```
// Archivo: app/lib/reports.ts
'use cache'

export async function getMonthlyTotals(accountId: string) {
  return db.orders.aggregate({ where: { accountId }, _sum: { amount: true } })
}

export async function getTopProducts() {
  return db.products.findMany({ orderBy: { sales: 'desc' }, take: 10 })
}
```

Framework function exports are covered like any other, so `generateMetadata` and `generateStaticParams` must be async in such a file.

> **Good to know**:
>
> - When a cached directive (`use cache` , `use cache: private` , or `use cache: remote`) is at the top of a file, you can import its exported functions into a Client Component and call them directly; they run on the server and return the result, similar to a Server Function. Prefer calling cached functions on the server and passing results down as props.

Caching a route segment's output with `use cache`

A `page` or `layout` file is a module too, so a file-level directive there follows the same rules. Each route segment is a separate entry point and is cached independently. To prerender a whole route, add `use cache` to the top of every segment file it renders: the `page` , the `layout` , and any parallel route slots.

```
// Archivo: app/layout.tsx switcher
'use cache'

export default async function Layout({ children }: { children: ReactNode }) {
  return <div>{children}</div>
}
```

```
// Archivo: app/layout.js switcher
'use cache'

export default async function Layout({ children }) {
  return <div>{children}</div>
}
```

A cached `layout` does not cache the `children` it renders, since slots pass through without affecting its entry. See Interleaving.

```
// Archivo: app/page.tsx switcher
'use cache'

async function Users() {
  const res = await fetch('https://api.example.com/users')
  const users = await res.json()
  // loop through users
}

export default async function Page() {
  return (
    <main>
      <Users />
    </main>
  )
}
```

```
// Archivo: app/page.js switcher
'use cache'

async function Users() {
  const res = await fetch('https://api.example.com/users')
  const users = await res.json()
  // loop through users
}

export default async function Page() {
  return (
    <main>
      <Users />
    </main>
  )
}
```

Troubleshooting

Debugging cache behavior

Verbose logging

Set `NEXT_PRIVATE_DEBUG_CACHE=1` for verbose cache logging:

```
NEXT_PRIVATE_DEBUG_CACHE=1 npm run dev
# or for production
NEXT_PRIVATE_DEBUG_CACHE=1 npm run start
```

> **Good to know:** This environment variable also logs ISR and other caching mechanisms. See Verifying correct production behavior for more details.

Console log replays

In development, console logs from cached functions appear with a `Cache` prefix.

Build Hangs (Cache Timeout)

If your build hangs, you're accessing Promises that resolve to uncached or runtime data, created outside a `use cache` boundary. The cached function waits for data that can't resolve during the build, causing a timeout after 50 seconds.

When the build timeouts you'll see this error message:

> Error: Filling a cache during prerender timed out, likely because request-specific arguments such as params, searchParams, cookies() or uncached data were used inside "use cache".

Common ways this happens: passing such Promises as props, accessing them via closure, or retrieving them from shared storage (Maps).

> **Good to know:** Directly calling `cookies()` or `headers()` inside `use cache` fails immediately with a different error, not a timeout.

Passing runtime data Promises as props:

```
// Archivo: app/page.tsx
import { cookies } from 'next/headers'
import { Suspense } from 'react'

export default function Page() {
  return (
    ``<Suspense>``
  )
}

async function Dynamic() {
  const cookieStore = cookies()
  return <Cached promise={cookieStore} /> // Build hangs
}

async function Cached({ promise }: { promise: Promise<unknown> }) {
  'use cache'
  const data = await promise // Waits for runtime data during build
  return <p>..</p>
}
```

Await the `cookies` store in the `Dynamic` component, and pass a cookie value to the `Cached` component.

Shared deduplication storage:

```
// Archivo: app/page.tsx
// Problem: Map stores dynamic Promises, accessed by cached code
import { Suspense } from 'react'

const cache = new Map<string, Promise<string>>()

export default function Page() {
  return (
    <>
      ``<Suspense>``
      <Cached id="data" />
    </>
  )
}

async function Dynamic({ id }: { id: string }) {
  // Stores dynamic Promise in shared Map
  cache.set(
    id,
    fetch(`https://api.example.com/${id}`).then((r) => r.text())
  )
  return <p>Dynamic</p>
}

async function Cached({ id }: { id: string }) {
  'use cache'
  return <p>{await cache.get(id)}</p> // Build hangs - retrieves dynamic Promise
}
```

Use Next.js's built-in `fetch()` deduplication or use separate Maps for cached and uncached contexts.

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Learn how to configure caching when self-hosting Next.js.

Version History

Version	Changes
`v16.0.0`	`"use cache"` is enabled with the Cache Components feature.
`v15.0.0`	`"use cache"` is introduced as an experimental feature.

use client

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives/use-client
> 📁 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how to use the use client directive to render a component on the client.*

The `'use client'` directive declares an entry point for the components to be rendered on the **client side** and should be used when creating interactive user interfaces (UI) that require client-side JavaScript capabilities, such as state management, event handling, and access to browser APIs. This is a React feature.

> **Good to know:**
>
> You do not need to add the `'use client'` directive to every file that contains Client Components. You only need to add it to the files whose components you want to render directly within Server Components. The `'use client'` directive defines the server and client boundary, and the components exported from such a file serve as entry points to the client.

Usage

To declare an entry point for the Client Components, add the `'use client'` directive **at the top of the file**, before any imports:

```
// Archivo: app/components/counter.tsx highlight={1} switcher
'use client'

import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  )
}
```

```
// Archivo: app/components/counter.js highlight={1} switcher
'use client'

import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  )
}
```

When using the `'use client'` directive, the props of the Client Components must be serializable. This means the props need to be in a format that React can serialize when sending data from the server to the client.

```
// Archivo: app/components/counter.tsx highlight={4} switcher
'use client'

export default function Counter({
  onClick /* ❌ Function is not serializable */,
}) {
  return (
    <div>
      <button onClick={onClick}>Increment</button>
    </div>
  )
}
```

```
// Archivo: app/components/counter.js highlight={4} switcher
'use client'

export default function Counter({
  onClick /* ❌ Function is not serializable */,
}) {
  return (
    <div>
      <button onClick={onClick}>Increment</button>
    </div>
  )
}
```

Nesting Client Components within Server Components

Combining Server and Client Components allows you to build applications that are both performant and interactive:

1. **Server Components**: Use for static content, data fetching, and SEO-friendly elements.
2. **Client Components**: Use for interactive elements that require state, effects, or browser APIs.
3. **Component composition**: Nest Client Components within Server Components as needed for a clear separation of server and client logic.

In the following example:

- `Header` is a Server Component handling static content.
- `Counter` is a Client Component enabling interactivity within the page.

```
// Archivo: app/page.tsx highlight={2,8} switcher
import Header from './header'
import Counter from './counter' // This is a Client Component

export default function Page() {
  return (
    <div>
      <Header />
      <Counter />
    </div>
  )
}
```

```
// Archivo: app/page.js highlight={2,8} switcher
import Header from './header'
import Counter from './counter' // This is a Client Component

export default function Page() {
  return (
    <div>
      <Header />
      <Counter />
    </div>
  )
}
```

Reference

See the React documentation for more information on `'use client'`.

use server

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/directives/use-server
> 📁 **Sección:** App > Api Reference > Directives
> 📝 **Descripción:** *Learn how to use the use server directive to execute code on the server.*

The `use server` directive designates a function or file to be executed on the **server side**. It can be used at the top of a file to indicate that all functions in the file are server-side, or inline at the top of a function to mark the function as a Server Function. This is a React feature.

For Next.js-specific Server Action behaviors (response model, security, configuration, deployment), see Server Actions and Mutations.

Using `use server` at the top of a file

The following example shows a file with a `use server` directive at the top. All functions in the file are executed on the server.

```tsx
// Archivo: app/actions.ts highlight={1} switcher
'use server'
import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth'

export async function createUser(data: { name: string; email: string }) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const user = await db.user.create({ data })
  return { id: user.id, name: user.name }
}
```

```jsx
// Archivo: app/actions.js highlight={1} switcher
'use server'
import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth'

export async function createUser(data) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const user = await db.user.create({ data })
  return { id: user.id, name: user.name }
}
```

Using Server Functions in a Client Component

To use Server Functions in Client Components you need to create your Server Functions in a dedicated file using the `use server` directive at the top of the file. These Server Functions can then be imported into Client and Server Components and executed.

Assuming you have a `fetchUsers` Server Function in `actions.ts`:

```tsx
// Archivo: app/actions.ts highlight={1} switcher
'use server'
import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth'

export async function fetchUsers() {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const users = await db.user.findMany({
    select: { id: true, name: true, email: true },
  })
  return users
}
```

```
// Archivo: app/actions.js highlight={1} switcher
'use server'
import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth'

export async function fetchUsers() {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const users = await db.user.findMany({
    select: { id: true, name: true, email: true },
  })
  return users
}
```

Then you can import the `fetchUsers` Server Function into a Client Component and execute it on the client-side.

```
// Archivo: app/components/my-button.tsx highlight={1,2,5} switcher
'use client'
import { fetchUsers } from '../actions'

export default function MyButton() {
  return <button onClick={() => fetchUsers()}>Fetch Users</button>
}
```

```
// Archivo: app/components/my-button.js highlight={1,2,5} switcher
'use client'
import { fetchUsers } from '../actions'

export default function MyButton() {
  return <button onClick={() => fetchUsers()}>Fetch Users</button>
}
```

Using `use server` inline

In the following example, `use server` is used inline at the top of a function to mark it as a Server Function:

```
// Archivo: app/posts/[id]/page.tsx switcher highlight={9}
import { EditPost } from './edit-post'
import { revalidatePath } from 'next/cache'

export default async function PostPage({ params }: PageProps<'/posts/[id]'>) {
  const { id } = await params
  const post = await getPost(id)

  async function updatePost(formData: FormData) {
    'use server'
    // Verify auth before saving (e.g. inside savePost)
    await savePost(id, formData)
    revalidatePath(`/posts/${id}`)
  }

  return <EditPost updatePostAction={updatePost} post={post} />
}
```

```
// Archivo: app/posts/[id]/page.js switcher highlight={9}
import { EditPost } from './edit-post'
import { revalidatePath } from 'next/cache'

export default async function PostPage({ params }) {
  const { id } = await params
  const post = await getPost(id)

  async function updatePost(formData) {
    'use server'
    // Verify auth before saving (e.g. inside savePost)
    await savePost(id, formData)
    revalidatePath(`/posts/${id}`)
  }

  return <EditPost updatePostAction={updatePost} post={post} />
}
```

Security considerations

Design your data access functions as secure primitives: validate inputs, check authentication and authorization, and constrain return types to only what the caller needs. When Server Functions delegate to a Data Access Layer, these guarantees live in one place and apply consistently.

Authentication and authorization

Always authenticate and authorize users before performing sensitive server-side operations. Read authentication from cookies or headers rather than accepting tokens as function parameters.

```
// Archivo: app/actions.ts highlight={1,7,8,9,10} switcher
'use server'

import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth' // Your authentication library

export async function createUser(data: { name: string; email: string }) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  const newUser = await db.user.create({ data })
  return { id: newUser.id, name: newUser.name }
}
```

```
// Archivo: app/actions.js highlight={1,7,8,9,10} switcher
'use server'

import { db } from '@/lib/db' // Your database client
import { auth } from '@/lib/auth' // Your authentication library

export async function createUser(data) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  const newUser = await db.user.create({ data })
  return { id: newUser.id, name: newUser.name }
}
```

Return values

Server Function return values are serialized and sent to the client. Only return data the UI needs, not raw database records. See the Data Security guide for more details.

Reference

See the React documentation for more information on `use server`.

Capítulo 7: Optimización de Assets: Imágenes, Fuentes, Scripts y SEO

> *next/image, next/font, Scripts de terceros, Metadata, Open Graph, Sitemap y Robots.txt.*

Image Optimization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/images
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to optimize images in Next.js*

The Next.js `<Image>` component extends the HTML `<img>` element to provide:

- **Size optimization:** Automatically serving correctly sized images for each device, using modern image formats like WebP.
- **Visual stability:** Preventing layout shift automatically when images are loading.
- **Faster page loads:** Only loading images when they enter the viewport using native browser lazy loading, with optional blur-up placeholders.
- **Asset flexibility:** Resizing images on-demand, even images stored on remote servers.

To start using `<Image>`, import it from `next/image` and render it within your component.

```
// Archivo: app/page.tsx switcher
import Image from 'next/image'

export default function Page() {
  return

![]()


}
```

```
// Archivo: app/page.js switcher
import Image from 'next/image'

export default function Page() {
  return

![]()


}
```

The `src` property can be a local or remote image.

> 🎥 **Watch:** Learn more about how to use `next/image` → YouTube (9 minutes).

Local images

You can store static files, like images and fonts, under a folder called `public` in the root directory. Files inside `public` can then be referenced by your code starting from the base URL (`/`).

Folder structure showing app and public folders

```
// Archivo: app/page.tsx switcher
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://nextjs.org/profile.png)


  )
}
```

```
// Archivo: app/page.js switcher
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://nextjs.org/profile.png)


  )
}
```

If the image is statically imported, Next.js will automatically determine the intrinsic `width` and `height`. These values are used to determine the image ratio and prevent Cumulative Layout Shift while your image is loading.

```
// Archivo: app/page.tsx switcher
import Image from 'next/image'
import ProfileImage from './profile.png'

export default function Page() {
  return (


![Picture of the author]()


  )
}
```

```
// Archivo: app/page.js switcher
import Image from 'next/image'
import ProfileImage from './profile.png'

export default function Page() {
  return (


![Picture of the author]()


  )
}
```

Images without static imports

If you can't use a static `import` for your images, you can use a dynamic `import()` in a Server Component to still get automatic `width`, `height`, and `blurDataURL`:

```
// Archivo: app/blog/[slug]/page.tsx switcher
import Image from 'next/image'

async function PostImage({
  imageFilename,
  alt,
}: {
  imageFilename: string
  alt: string
}) {
  const { default: image } = await import(
    `../content/blog/images/${imageFilename}`
  )
  // image contains width, height, and blurDataURL
  return

![Illustration]()


}
```

```
// Archivo: app/blog/[slug]/page.js switcher
import Image from 'next/image'

async function PostImage({ imageFilename, alt }) {
  const { default: image } = await import(
    `../content/blog/images/${imageFilename}`
  )
  // image contains width, height, and blurDataURL
  return

![Illustration]()


}
```

If you have a path alias configured (e.g. `@/`), you can use it instead of a relative path:

```
const { default: image } = await import(
  `@/content/blog/images/${imageFilename}`
)
```

The path must include a static prefix (like `../content/blog/images/`). Be as specific as possible, since **all** files matching that prefix are bundled. Only files in your specified directory are included, so external input cannot reach outside of it.

Remote images

To use a remote image, you can provide a URL string for the `src` property.

```
// Archivo: app/page.tsx switcher
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://s3.amazonaws.com/my-bucket/profile.png)


  )
}
```

```
// Archivo: app/page.js switcher
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://s3.amazonaws.com/my-bucket/profile.png)


  )
}
```

Since Next.js does not have access to remote files during the build process, you'll need to provide the `width`, `height` and optional `blurDataURL` props manually. The `width` and `height` are used to infer the correct aspect ratio of image and avoid layout shift from the image loading in. Alternatively, you can use the `fill` property to make the image fill the size of the parent element.

To safely allow images from remote servers, you need to define a list of supported URL patterns in `next.config.js`. Be as specific as possible to prevent malicious usage. For example, the following configuration will only allow images from a specific AWS S3 bucket:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const config: NextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 's3.amazonaws.com',
        port: '',
        pathname: '/my-bucket/**',
        search: '',
      },
    ],
  },
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 's3.amazonaws.com',
        port: '',
        pathname: '/my-bucket/**',
        search: '',
      },
    ],
  },
}
```

Font Optimization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/fonts
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to optimize fonts in Next.js*

The `next/font` module automatically optimizes your fonts and removes external network requests for improved privacy and performance.

It includes **built-in self-hosting** for any font file. This means you can optimally load web fonts with no layout shift.

To start using `next/font`, import it from `next/font/local` or `next/font/google`, call it as a function with the appropriate options, and set the `className` of the element you want to apply the font to. For example:

```
// Archivo: app/layout.tsx highlight={1,3-5,9} switcher
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={geist.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js highlight={1,3-5,9} switcher
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function Layout({ children }) {
  return (
    <html className={geist.className}>
      <body>{children}</body>
    </html>
  )
}
```

Fonts are scoped to the component they're used in. To apply a font to your entire application, add it to the Root Layout.

📘 **Nota para Pages Router:** To start using `next/font`, import it from `next/font/local` or `next/font/google`, call it as a function with the appropriate options, and set the `className` of the element you want to apply the font to. For example, you can apply fonts globally in your Custom App (`pages/_app`):

```
import { Geist } from 'next/font/google'
import type { AppProps } from 'next/app'

const geist = Geist({
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }: AppProps) {
  return (
    <main className={geist.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

```
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={geist.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

Google fonts

You can automatically self-host any Google Font. Fonts are included as static assets and served from the same domain as your deployment, meaning no requests are sent to Google by the browser when the user visits your site.

To start using a Google Font, import your chosen font from `next/font/google`:

```
// Archivo: app/layout.tsx switcher
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={geist.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={geist.className}>
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:**

```
import { Geist } from 'next/font/google'
import type { AppProps } from 'next/app'

const geist = Geist({
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }: AppProps) {
  return (
    <main className={geist.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

```
import { Geist } from 'next/font/google'

const geist = Geist({
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={geist.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

We recommend using variable fonts for the best performance and flexibility. But if you can't use a variable font, you will need to specify a weight:

```tsx
// Archivo: app/layout.tsx highlight={4} switcher
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={roboto.className}>
      <body>{children}</body>
    </html>
  )
}
```

```jsx
// Archivo: app/layout.js  highlight={4} switcher
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={roboto.className}>
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:**

```
import { Roboto } from 'next/font/google'
import type { AppProps } from 'next/app'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }: AppProps) {
  return (
    <main className={roboto.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

```
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={roboto.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

Local fonts

To use a local font, import the `localFont` function from `next/font/local` and specify the `src` of your local font file. The path is resolved relative to the file where `localFont` is called. Fonts can be stored anywhere in the project, including the `public` folder or co-located inside the `app` folder. For example, to use a font stored in `app/fonts/`:

```
// Archivo: app/layout.tsx switcher
import localFont from 'next/font/local'

const myFont = localFont({
  src: './my-font.woff2',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={myFont.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import localFont from 'next/font/local'

const myFont = localFont({
  src: './my-font.woff2',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={myFont.className}>
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:** To use a local font, import your font from `next/font/local` and specify the `src` of your local font file. Fonts can be stored in the `public` folder or inside the `pages` folder. For example:

```
import localFont from 'next/font/local'
import type { AppProps } from 'next/app'

const myFont = localFont({
  src: './my-font.woff2',
})

export default function MyApp({ Component, pageProps }: AppProps) {
  return (
    <main className={myFont.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

```
import localFont from 'next/font/local'

const myFont = localFont({
  src: './my-font.woff2',
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={myFont.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

If you want to use multiple files for a single font family, `src` can be an array:

```
const roboto = localFont({
  src: [
    {
      path: './Roboto-Regular.woff2',
      weight: '400',
      style: 'normal',
    },
    {
      path: './Roboto-Italic.woff2',
      weight: '400',
      style: 'italic',
    },
    {
      path: './Roboto-Bold.woff2',
      weight: '700',
      style: 'normal',
    },
    {
      path: './Roboto-BoldItalic.woff2',
      weight: '700',
      style: 'italic',
    },
  ],
})
```

Metadata and OG images

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/metadata-and-og-images
> 📂 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to add metadata to your pages and create dynamic OG images.*

The Metadata APIs can be used to define your application metadata for improved SEO and web shareability and include:

1. The static `metadata` object
2. The dynamic `generateMetadata` function
3. Special file conventions that can be used to add static or dynamically generated favicons and OG images.

With all the options above, Next.js will automatically generate the relevant `<head>` tags for your page, which can be inspected in the browser's developer tools.

The `metadata` object and `generateMetadata` function exports are only supported in Server Components.

Default fields

There are two default `meta` tags that are always added even if a route doesn't define metadata:

- The meta charset tag sets the character encoding for the website.
- The meta viewport tag sets the viewport width and scale for the website to adjust for different devices.

```
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

The other metadata fields can be defined with the `Metadata` object (for static metadata) or the `generateMetadata` function (for generated metadata).

Static metadata

To define static metadata, export a `Metadata` object from a static `layout.js` or `page.js` file. For example, to add a title and description to the blog route:

```
// Archivo: app/blog/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'My Blog',
  description: '...',
}

export default function Layout() {}
```

```
// Archivo: app/blog/layout.js switcher
export const metadata = {
  title: 'My Blog',
  description: '...',
}

export default function Layout() {}
```

You can view a full list of available options, in the `generateMetadata` documentation.

Generated metadata

You can use `generateMetadata` function to `fetch` metadata that depends on data. For example, to fetch the title and description for a specific blog post:

```
// Archivo: app/blog/[slug]/page.tsx switcher
import type { Metadata, ResolvingMetadata } from 'next'

type Props = {
  params: Promise<{ slug: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export async function generateMetadata(
  { params, searchParams }: Props,
  parent: ResolvingMetadata
): Promise<Metadata> {
  const slug = (await params).slug

  // fetch post information
  const post = await fetch(`https://api.vercel.app/blog/${slug}`).then((res) =>
    res.json()
  )

  return {
    title: post.title,
    description: post.description,
  }
}

export default function Page({ params, searchParams }: Props) {}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export async function generateMetadata({ params, searchParams }, parent) {
  const slug = (await params).slug

  // fetch post information
  const post = await fetch(`https://api.vercel.app/blog/${slug}`).then((res) =>
    res.json()
  )

  return {
    title: post.title,
    description: post.description,
  }
}

export default function Page({ params, searchParams }) {}
```

Streaming metadata

For dynamically rendered pages, Next.js streams metadata separately, injecting it into the HTML once `generateMetadata` resolves, without blocking UI rendering.

Streaming metadata improves perceived performance by allowing visual content to stream first.

Streaming metadata is **disabled for bots and crawlers** that expect metadata to be in the `<head>` tag (e.g. `Twitterbot`, `Slackbot`, `Bingbot`). These are detected by using the User Agent header from the incoming request.

You can customize or **disable** streaming metadata completely, with the `htmlLimitedBots` option in your Next.js config file.

Prerendered pages don't use streaming since metadata is resolved at build time.

Learn more about streaming metadata.

Memoizing data requests

There may be cases where you need to fetch the **same** data for metadata and the page itself. To avoid duplicate requests, you can use React's `cache` function to memoize the return value and only fetch the data once. For example, to fetch the blog post information for both the metadata and the page:

```
// Archivo: app/lib/data.ts highlight={5} switcher
import { cache } from 'react'
import { db } from '@/app/lib/db'

// getPost will be used twice, but execute only once
export const getPost = cache(async (slug: string) => {
  const res = await db.query.posts.findFirst({ where: eq(posts.slug, slug) })
  return res
})
```

```
// Archivo: app/lib/data.js highlight={5} switcher
import { cache } from 'react'
import { db } from '@/app/lib/db'

// getPost will be used twice, but execute only once
export const getPost = cache(async (slug) => {
  const res = await db.query.posts.findFirst({ where: eq(posts.slug, slug) })
  return res
})
```

```tsx
// Archivo: app/blog/[slug]/page.tsx switcher
import { getPost } from '@/app/lib/data'

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await getPost(slug)
  return {
    title: post.title,
    description: post.description,
  }
}

export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await getPost(slug)
  return <div>{post.title}</div>
}
```

```tsx
// Archivo: app/blog/[slug]/page.js switcher
import { getPost } from '@/app/lib/data'

export async function generateMetadata({ params }) {
  const { slug } = await params
  const post = await getPost(slug)
  return {
    title: post.title,
    description: post.description,
  }
}

export default async function Page({ params }) {
  const { slug } = await params
  const post = await getPost(slug)
  return <div>{post.title}</div>
}
```

File-based metadata

The following special files are available for metadata:

- favicon.ico, apple-icon.jpg, and icon.jpg
- opengraph-image.jpg and twitter-image.jpg
- robots.txt
- sitemap.xml

You can use these for static metadata, or you can programmatically generate these files with code.

Favicons

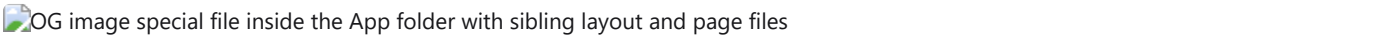
Favicons are small icons that represent your site in bookmarks and search results. To add a favicon to your application, create a `favicon.ico` and add to the root of the app folder.

Favicon Special File inside the App Folder with sibling layout and page files

> You can also programmatically generate favicons using code. See the favicon docs for more information.

Static Open Graph images

Open Graph (OG) images are images that represent your site in social media. To add a static OG image to your application, create a `opengraph-image.jpg` file in the root of the app folder.



You can also add OG images for specific routes by creating a `opengraph-image.jpg` deeper down the folder structure. For example, to create an OG image specific to the `/blog` route, add a `opengraph-image.jpg` file inside the `blog` folder.

The more specific image will take precedence over any OG images above it in the folder structure.

> Other image formats such as `jpeg`, `png`, and `gif` are also supported. See the Open Graph Image docs for more information.

Generated Open Graph images

The `ImageResponse` constructor allows you to generate dynamic images using JSX and CSS. This is useful for OG images that depend on data.

For example, to generate a unique OG image for each blog post, add an `opengraph-image.tsx` file inside the `blog/[slug]` folder, and import the `ImageResponse` constructor from `next/og`:

```
// Archivo: app/blog/[slug]/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { getPost } from '@/app/lib/data'

// Image metadata
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

// Image generation
export default async function Image({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await getPost(slug)

  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    )
  )
}
```

```
// Archivo: app/blog/[slug]/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { getPost } from '@/app/lib/data'

// Image metadata
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

// Image generation
export default async function Image({ params }) {
  const { slug } = await params
  const post = await getPost(slug)

  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    )
  )
}
```

`ImageResponse` supports common CSS properties including flexbox and absolute positioning, custom fonts, text wrapping, centering, and nested images. See the full list of supported CSS properties.

> **Good to know**:
>
> - Examples are available in the Vercel OG Playground.
> - `ImageResponse` uses `@vercel/og`, `satori`, and `resvg` to convert HTML and CSS into PNG.
> - Only flexbox and a subset of CSS properties are supported. Advanced layouts (e.g. `display: grid`) will not work.

How to load and optimize scripts

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/scripts
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Optimize 3rd party scripts with the built-in Script component.*

Layout Scripts

To load a third-party script for multiple routes, import `next/script` and include the script directly in your layout component:

```tsx
// Archivo: app/dashboard/layout.tsx switcher
import Script from 'next/script'

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <>
      <section>{children}</section>
      <Script src="https://example.com/script.js" />
    </>
  )
}
```

```jsx
// Archivo: app/dashboard/layout.js switcher
import Script from 'next/script'

export default function DashboardLayout({ children }) {
  return (
    <>
      <section>{children}</section>
      <Script src="https://example.com/script.js" />
    </>
  )
}
```

The third-party script is fetched when the folder route (e.g. `dashboard/page.js`) or any nested route (e.g. `dashboard/settings/page.js`) is accessed by the user. Next.js will ensure the script will **only load once**, even if a user navigates between multiple routes in the same layout.

Application Scripts

To load a third-party script for all routes, import `next/script` and include the script directly in your root layout:

```tsx
// Archivo: app/layout.tsx switcher
import Script from 'next/script'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
      <Script src="https://example.com/script.js" />
    </html>
  )
}
```

```jsx
// Archivo: app/layout.js switcher
import Script from 'next/script'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
      <Script src="https://example.com/script.js" />
    </html>
  )
}
```

> 📘 **Nota para Pages Router:** To load a third-party script for all routes, import `next/script` and include the script directly in your custom `_app`:

```
import Script from 'next/script'

export default function MyApp({ Component, pageProps }) {
  return (
    <>
      <Component {...pageProps} />
      <Script src="https://example.com/script.js" />
    </>
  )
}
```

This script will load and execute when *any* route in your application is accessed. Next.js will ensure the script will **only load once**, even if a user navigates between multiple pages.

> **Recommendation**: We recommend only including third-party scripts in specific pages or layouts in order to minimize any unnecessary impact to performance.

Strategy

Although the default behavior of `next/script` allows you to load third-party scripts in any page or layout, you can fine-tune its loading behavior by using the `strategy` property:

- `beforeInteractive`: Load the script before any Next.js code and before any page hydration occurs.
- `afterInteractive`: (**default**) Load the script early but after some hydration on the page occurs.
- `lazyOnload`: Load the script later during browser idle time.
- `worker`: (experimental) Load the script in a web worker.

Refer to the `next/script` API reference documentation to learn more about each strategy and their use cases.

Offloading Scripts To A Web Worker (experimental)

> **Warning:** The `worker` strategy is not yet stable and does not yet work with the App Router. Use with caution.

Scripts that use the `worker` strategy are offloaded and executed in a web worker with Partytown. This can improve the performance of your site by dedicating the main thread to the rest of your application code.

This strategy is still experimental and can only be used if the `nextScriptWorkers` flag is enabled in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    nextScriptWorkers: true,
  },
}
```

Then, run the development server and Next.js will guide you through the installation of the required packages to finish the setup:

```
# Manejador de paquetes: pnpm
pnpm dev
```

```
# Manejador de paquetes: npm
npm run dev
```

```
# Manejador de paquetes: yarn
yarn dev
```

```
# Manejador de paquetes: bun
bun dev
```

You'll see instructions like these: Please install Partytown by running `npm install @qwik.dev/partytown`

Once setup is complete, defining `strategy="worker"` will automatically instantiate Partytown in your application and offload the script to a web worker.

```
// Archivo: pages/home.tsx switcher
import Script from 'next/script'

export default function Home() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="worker" />
    </>
  )
}
```

```
// Archivo: pages/home.js switcher
import Script from 'next/script'

export default function Home() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="worker" />
    </>
  )
}
```

There are a number of trade-offs that need to be considered when loading a third-party script in a web worker. Please see Partytown's tradeoffs documentation for more information.

📘 **Nota para Pages Router:**

Using custom Partytown configuration

Although the `worker` strategy does not require any additional configuration to work, Partytown supports the use of a config object to modify some of its settings, including enabling `debug` mode and forwarding events and triggers.

If you would like to add additional configuration options, you can include it within the `<Head />` component used in a custom `_document.js`:

```
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html>
      <Head>
        <script
          data-partytown-config
          dangerouslySetInnerHTML={{
            __html: `
              partytown = {
                lib: "/_next/static/~partytown/",
                debug: true
              };
            `,
          }}
        />
      </Head>
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

In order to modify Partytown's configuration, the following conditions must be met:

1. The `data-partytown-config` attribute must be used in order to overwrite the default configuration used by Next.js
2. Unless you decide to save Partytown's library files in a separate directory, the `lib: "/_next/static/~partytown/"` property and value must be included in the configuration object in order to let Partytown know where Next.js stores the necessary static files.

> **Note**: If you are using an asset prefix and would like to modify Partytown's default configuration, you must include it as part of the `lib` path.

Take a look at Partytown's configuration options to see the full list of other properties that can be added.

Inline Scripts

Inline scripts, or scripts not loaded from an external file, are also supported by the Script component. They can be written by placing the JavaScript within curly braces:

```
<Script id="show-banner">
  {`document.getElementById('banner').classList.remove('hidden')`}
</Script>
```

Or by using the `dangerouslySetInnerHTML` property:

```
<Script
  id="show-banner"
  dangerouslySetInnerHTML={{
    __html: `document.getElementById('banner').classList.remove('hidden')`,
  }}
/>
```

> **Warning**: An `id` property must be assigned for inline scripts in order for Next.js to track and optimize the script.

Executing Additional Code

Event handlers can be used with the Script component to execute additional code after a certain event occurs:

- `onLoad`: Execute code after the script has finished loading.
- `onReady`: Execute code after the script has finished loading and every time the component is mounted.
- `onError`: Execute code if the script fails to load.

These handlers will only work when `next/script` is imported and used inside of a Client Component where `"use client"` is defined as the first line of code:

```
// Archivo: app/page.tsx switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onLoad={() => {
          console.log('Script has loaded')
        }}
      />
    </>
  )
}
```

```
// Archivo: app/page.js switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onLoad={() => {
          console.log('Script has loaded')
        }}
      />
    </>
  )
}
```

Refer to the `next/script` API reference to learn more about each event handler and view examples.

📘 **Nota para Pages Router:** These handlers will only work when `next/script` is imported and used inside of a Client Component where `"use client"` is defined as the first line of code:

```
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onLoad={() => {
          console.log('Script has loaded')
        }}
      />
    </>
  )
}
```

```
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onLoad={() => {
          console.log('Script has loaded')
        }}
      />
    </>
  )
}
```

Refer to the `next/script` API reference to learn more about each event handler and view examples.

Additional Attributes

There are many DOM attributes that can be assigned to a `<script>` element that are not used by the Script component, like `nonce` or custom data attributes. Including any additional attributes will automatically forward it to the final, optimized `<script>` element that is included in the HTML.

```
// Archivo: app/page.tsx switcher
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        id="example-script"
        nonce="XUENAJFW"
        data-test="script"
      />
    </>
  )
}
```

```
// Archivo: app/page.js switcher
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        id="example-script"
        nonce="XUENAJFW"
        data-test="script"
      />
    </>
  )
}
```

📘 **Nota para Pages Router:**

```
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        id="example-script"
        nonce="XUENAJFW"
        data-test="script"
      />
    </>
  )
}
```

```
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        id="example-script"
        nonce="XUENAJFW"
        data-test="script"
      />
    </>
  )
}
```

favicon, icon, and apple-icon

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/app-icons
📂 **Sección:** App > Api Reference > File Conventions > Metadata
📝 **Descripción:** *API Reference for the Favicon, Icon and Apple Icon file conventions.*

The `favicon`, `icon`, or `apple-icon` file conventions allow you to set icons for your application.

They are useful for adding app icons that appear in places like web browser tabs, phone home screens, and search engine results.

There are two ways to set app icons:

- Using image files (.ico, .jpg, .png)
- Using code to generate an icon (.js, .ts, .tsx)

Image files (.ico, .jpg, .png)

Use an image file to set an app icon by placing a `favicon`, `icon`, or `apple-icon` image file within your `/app` directory. The `favicon` image can only be located in the top level of `app/`.

Next.js will evaluate the file and automatically add the appropriate tags to your app's `<head>` element.

File convention	Supported file types	Valid locations
`favicon`	`.ico`	`app/`
`icon`	`.ico`, `.jpg`, `.jpeg`, `.png`, `.svg`	`app/**/*`
`apple-icon`	`.jpg`, `.jpeg`, `.png`	`app/**/*`

`favicon`

Add a `favicon.ico` image file to the root `/app` route segment.

```
// Archivo: <head> output
<link rel="icon" href="/favicon.ico" sizes="any" />
```

`icon`

Add an `icon.(ico|jpg|jpeg|png|svg)` image file.

```
// Archivo: <head> output
<link
  rel="icon"
  href="/icon?<generated>"
  type="image/<generated>"
  sizes="<generated>"
/>
```

`apple-icon`

Add an `apple-icon.(jpg|jpeg|png)` image file.

```
// Archivo: <head> output
<link
  rel="apple-touch-icon"
  href="/apple-icon?<generated>"
  type="image/<generated>"
  sizes="<generated>"
/>
```

> **Good to know**:
>
> - You can set multiple icons by adding a number suffix to the file name. For example, `icon1.png`, `icon2.png`, etc. Numbered files will sort lexically.
> - Favicons can only be set in the root `/app` segment. If you need more granularity, you can use `icon`.
> - The appropriate `<link>` tags and attributes such as `rel`, `href`, `type`, and `sizes` are determined by the icon type and metadata of the evaluated file.
> - For example, a 32 by 32px `.png` file will have `type="image/png"` and `sizes="32x32"` attributes.
> - `sizes="any"` is added to icons when the extension is `.svg` or the image size of the file is not determined. More details in this favicon handbook.

Generate icons using code (.js, .ts, .tsx)

In addition to using literal image files, you can programmatically **generate** icons using code.

Generate an app icon by creating an `icon` or `apple-icon` route that default exports a function.

File convention	Supported file types
`icon`	`.js` , `.ts` , `.tsx`
`apple-icon`	`.js` , `.ts` , `.tsx`

The easiest way to generate an icon is to use the `ImageResponse` API from `next/og` .

```
// Archivo: app/icon.tsx switcher
import { ImageResponse } from 'next/og'

// Image metadata
export const size = {
  width: 32,
  height: 32,
}
export const contentType = 'image/png'

// Image generation
export default function Icon() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 24,
          background: 'black',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          color: 'white',
        }}
      >
        A
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported icons size metadata
      // config to also set the ImageResponse's width and height.
      ...size,
    }
  )
}
```

```
// Archivo: app/icon.js switcher
import { ImageResponse } from 'next/og'

// Image metadata
export const size = {
  width: 32,
  height: 32,
}
export const contentType = 'image/png'

// Image generation
export default function Icon() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 24,
          background: 'black',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          color: 'white',
        }}
      >
        A
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported icons size metadata
      // config to also set the ImageResponse's width and height.
      ...size,
    }
  )
}
```

```
// Archivo: <head> output
<link rel="icon" href="/icon?<generated>" type="image/png" sizes="32x32" />
```

Good to know:

- By default, generated icons are **statically optimized** (generated at build time and cached) unless they use Request-time APIs or uncached data.
- You can generate multiple icons in the same file using `generateImageMetadata`.
- You cannot generate a `favicon` icon. Use `icon` or a favicon.ico file instead.
- App icons are special Route Handlers that are cached by default unless they use a Request-time API or dynamic config option.

Props

The default export function receives the following props:

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters object from the root segment down to the segment `icon` or `apple-icon` is colocated in.

Good to know: If you use `generateImageMetadata`, the function will also receive an `id` prop that is a promise resolving to the `id` value from one of the items returned by `generateImageMetadata`.

```
// Archivo: app/shop/[slug]/icon.tsx switcher
export default async function Icon({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/shop/[slug]/icon.js switcher
export default async function Icon({ params }) {
  const { slug } = await params
  // ...
}
```

Route	URL	`params`
`app/shop/icon.js`	`/shop`	`undefined`
`app/shop/[slug]/icon.js`	`/shop/1`	`Promise<{ slug: '1' }>`
`app/shop/[tag]/[item]/icon.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`

Returns

The default export function should return a `Blob` | `ArrayBuffer` | `TypedArray` | `DataView` | `ReadableStream` | `Response`.

> **Good to know**: `ImageResponse` satisfies this return type.

Config exports

You can optionally configure the icon's metadata by exporting `size` and `contentType` variables from the `icon` or `apple-icon` route.

Option	Type
`size`	`{ width: number; height: number }`
`contentType`	`string` - image MIME type

`size`

```
// Archivo: icon.tsx | apple-icon.tsx switcher
export const size = { width: 32, height: 32 }

export default function Icon() {}
```

```
// Archivo: icon.js | apple-icon.js switcher
export const size = { width: 32, height: 32 }

export default function Icon() {}
```

```
// Archivo: <head> output
<link rel="icon" sizes="32x32" />
```

`contentType`

```
// Archivo: icon.tsx | apple-icon.tsx switcher
export const contentType = 'image/png'

export default function Icon() {}
```

```
// Archivo: icon.js | apple-icon.js switcher
export const contentType = 'image/png'

export default function Icon() {}
```

```
// Archivo: <head> output
<link rel="icon" type="image/png" />
```

Route Segment Config

`icon` and `apple-icon` are specialized Route Handlers that can use the same route segment configuration options as Pages and Layouts.

Version History

Version	Changes
`v16.0.0`	`params` is now a promise that resolves to an object
`v13.3.0`	`favicon` `icon` and `apple-icon` introduced

Metadata Files API Reference

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API documentation for the metadata file conventions.*

This section of the docs covers **Metadata file conventions**. File-based metadata can be defined by adding special metadata files to route segments.

Each file convention can be defined using a static file (e.g. `opengraph-image.jpg`), or a dynamic variant that uses code to generate the file (e.g. `opengraph-image.js`).

Once a file is defined, Next.js will automatically serve the file (with hashes in production for caching) and update the relevant head elements with the correct metadata, such as the asset's URL, file type, and image size.

> **Good to know**:
>
> - Special Route Handlers like `sitemap.ts`, `opengraph-image.tsx`, and `icon.tsx`, and other metadata files are cached by default.
> - If using along with `proxy.ts`, configure the matcher to exclude the metadata files.

manifest.json

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/manifest
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for manifest.json file.*

Add or generate a `manifest.(json|webmanifest)` file that matches the Web Manifest Specification in the **root** of `app` directory to provide information about your web application for the browser.

Static Manifest file

```
// Archivo: app/manifest.json | app/manifest.webmanifest
{
  "name": "My Next.js Application",
  "short_name": "Next.js App",
  "description": "An application built with Next.js",
  "start_url": "/"
  // ...
}
```

Generate a Manifest file

Add a `manifest.js` or `manifest.ts` file that returns a `Manifest` object.

> Good to know: `manifest.js` is a special Route Handlers that is cached by default unless it uses a Request-time API or dynamic config option.

```
// Archivo: app/manifest.ts switcher
import type { MetadataRoute } from 'next'

export default function manifest(): MetadataRoute.Manifest {
  return {
    name: 'Next.js App',
    short_name: 'Next.js App',
    description: 'Next.js App',
    start_url: '/',
    display: 'standalone',
    background_color: '#fff',
    theme_color: '#fff',
    icons: [
      {
        src: '/favicon.ico',
        sizes: 'any',
        type: 'image/x-icon',
      },
    ],
  }
}
```

```
// Archivo: app/manifest.js switcher
export default function manifest() {
  return {
    name: 'Next.js App',
    short_name: 'Next.js App',
    description: 'Next.js App',
    start_url: '/',
    display: 'standalone',
    background_color: '#fff',
    theme_color: '#fff',
    icons: [
      {
        src: '/favicon.ico',
        sizes: 'any',
        type: 'image/x-icon',
      },
    ],
  }
}
```

Manifest Object

The manifest object contains an extensive list of options that may be updated due to new web standards. For information on all the current options, refer to the `MetadataRoute.Manifest` type in your code editor if using TypeScript or see the MDN docs.

---

opengraph-image and twitter-image

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/opengraph-image
> 📁 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for the Open Graph Image and Twitter Image file conventions.*

The `opengraph-image` and `twitter-image` file conventions allow you to set Open Graph and Twitter images for a route segment.

They are useful for setting the images that appear on social networks and messaging apps when a user shares a link to your site.

There are two ways to set Open Graph and Twitter images:

- Using image files (.jpg, .png, .gif)
- Using code to generate images (.js, .ts, .tsx)

Image files (.jpg, .png, .gif)

Use an image file to set a route segment's shared image by placing an `opengraph-image` or `twitter-image` image file in the segment.

Next.js will evaluate the file and automatically add the appropriate tags to your app's `<head>` element.

File convention	Supported file types
`opengraph-image`	`.jpg` , `.jpeg` , `.png` , `.gif`
`twitter-image`	`.jpg` , `.jpeg` , `.png` , `.gif`
`opengraph-image.alt`	`.txt`
`twitter-image.alt`	`.txt`

> **Good to know**:
>
> The `twitter-image` file size must not exceed 5MB, and the `opengraph-image` file size must not exceed 8MB. If the image file size exceeds these limits, the build will fail.

`opengraph-image`

Add an `opengraph-image.(jpg|jpeg|png|gif)` image file to any route segment.

```
// Archivo: <head> output
<meta property="og:image" content="<generated>" />
<meta property="og:image:type" content="<generated>" />
<meta property="og:image:width" content="<generated>" />
<meta property="og:image:height" content="<generated>" />
```

`twitter-image`

Add a `twitter-image.(jpg|jpeg|png|gif)` image file to any route segment.

```
// Archivo: <head> output
<meta name="twitter:image" content="<generated>" />
<meta name="twitter:image:type" content="<generated>" />
<meta name="twitter:image:width" content="<generated>" />
<meta name="twitter:image:height" content="<generated>" />
```

`opengraph-image.alt.txt`

Add an accompanying `opengraph-image.alt.txt` file in the same route segment as the `opengraph-image.(jpg|jpeg|png|gif)` image its alt text.

```
// Archivo: opengraph-image.alt.txt
About Acme
```

```
// Archivo: <head> output
<meta property="og:image:alt" content="About Acme" />
```

`twitter-image.alt.txt`

Add an accompanying `twitter-image.alt.txt` file in the same route segment as the `twitter-image.(jpg|jpeg|png|gif)` image its alt text.

```
// Archivo: twitter-image.alt.txt
About Acme
```

```
// Archivo: <head> output
<meta name="twitter:image:alt" content="About Acme" />
```

Generate images using code (.js, .ts, .tsx)

In addition to using literal image files, you can programmatically **generate** images using code.

Generate a route segment's shared image by creating an `opengraph-image` or `twitter-image` route that default exports a function.

File convention	Supported file types
`opengraph-image`	`.js`, `.ts`, `.tsx`
`twitter-image`	`.js`, `.ts`, `.tsx`

Good to know:

- By default, generated images are **statically optimized** (generated at build time and cached) unless they use Request-time APIs or uncached data.
- You can generate multiple Images in the same file using `generateImageMetadata`.
- `opengraph-image.js` and `twitter-image.js` are special Route Handlers that are cached by default unless it uses a Request-time API or dynamic config option.

The easiest way to generate an image is to use the ImageResponse API from `next/og`.

```
// Archivo: app/about/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { readFile } from 'node:fs/promises'
import { join } from 'node:path'

// Image metadata
export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

const interSemiBold = await readFile(
  join(process.cwd(), 'assets/Inter-SemiBold.ttf')
)

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        About Acme
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
      fonts: [
        {
          name: 'Inter',
          data: interSemiBold,
          style: 'normal',
          weight: 400,
        },
      ],
    }
  )
}
```

```
// Archivo: app/about/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { readFile } from 'node:fs/promises'
import { join } from 'node:path'

// Image metadata
export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

const interSemiBold = await readFile(
  join(process.cwd(), 'assets/Inter-SemiBold.ttf')
)

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        About Acme
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
      fonts: [
        {
          name: 'Inter',
          data: interSemiBold,
          style: 'normal',
          weight: 400,
        },
      ],
    }
  )
}
```

```
// Archivo: <head> output
<meta property="og:image" content="<generated>" />
<meta property="og:image:alt" content="About Acme" />
<meta property="og:image:type" content="image/png" />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
```

Props

The default export function receives the following props:

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters object from the root segment down to the segment `opengraph-image` or `twitter-image` is colocated in.

> **Good to know**: If you use `generateImageMetadata`, the function will also receive an `id` prop that is a promise resolving to the `id` value from one of the items returned by `generateImageMetadata`.

```
// Archivo: app/shop/[slug]/opengraph-image.tsx switcher
export default async function Image({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/shop/[slug]/opengraph-image.js switcher
export default async function Image({ params }) {
  const { slug } = await params
  // ...
}
```

Route	URL	`params`
`app/shop/opengraph-image.js`	`/shop`	`undefined`
`app/shop/[slug]/opengraph-image.js`	`/shop/1`	`Promise<{ slug: '1' }>`
`app/shop/[tag]/[item]/opengraph-image.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`

Returns

The default export function should return a `Response`.

> **Good to know**: `ImageResponse` satisfies this return type.

Config exports

You can optionally configure the image's metadata by exporting `alt`, `size`, and `contentType` variables from `opengraph-image` or `twitter-image` route.

Option	Type
`alt`	`string`
`size`	`{ width: number; height: number }`
`contentType`	`string` - image MIME type

`alt`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const alt = 'My images alt text'

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const alt = 'My images alt text'

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:alt" content="My images alt text" />
```

`size`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const size = { width: 1200, height: 630 }

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const size = { width: 1200, height: 630 }

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
```

`contentType`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const contentType = 'image/png'

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const contentType = 'image/png'

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:type" content="image/png" />
```

Route Segment Config

`opengraph-image` and `twitter-image` are specialized Route Handlers that can use the same route segment configuration options as Pages and Layouts.

Examples

Using external data

This example uses the `params` object and external data to generate the image.

> **Good to know**: By default, this generated image will be statically optimized. You can configure the individual `fetch` `options` or route segments options to change this behavior.

```tsx
// Archivo: app/posts/[slug]/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'

export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}
export const contentType = 'image/png'

export default async function Image({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await fetch(`https:// ... /posts/${slug}`).then((res) =>
    res.json()
  )

  return new ImageResponse(
    (
      <div
        style={{
          fontSize: 48,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    ),
    {
      ... size,
    }
  )
}
```

```
// Archivo: app/posts/[slug]/opengraph-image.js switcher
import { ImageResponse } from 'next/og'

export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}
export const contentType = 'image/png'

export default async function Image({ params }) {
  const { slug } = await params
  const post = await fetch(`https:// ... /posts/${slug}`).then((res) =>
    res.json()
  )

  return new ImageResponse(
    (
      <div
        style={{
          fontSize: 48,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    ),
    {
      ... size,
    }
  )
}
```

Using Node.js runtime with local assets

These examples use the Node.js runtime to fetch a local image from the file system and pass it to the `<img>` `src` attribute, either as a base64 string or an `ArrayBuffer`. Place the local asset relative to the project root, not the example source file.

The asset doesn't depend on request data, so read it once at module scope. See Predictable values.

```
// Archivo: app/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'), 'base64')
const logoSrc = `${logoData}`

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

```
// Archivo: app/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'), 'base64')
const logoSrc = `${logoData}`

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

Passing an `ArrayBuffer` to the `src` attribute of an `<img>` element is not part of the HTML spec. The rendering engine used by `next/og` supports it, but because TypeScript definitions follow the spec, you need a `@ts-expect-error` directive or similar to use this feature.

```
// Archivo: app/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'))
const logoSrc = Uint8Array.from(logoData).buffer

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >

        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

```
// Archivo: app/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'))
const logoSrc = Uint8Array.from(logoData).buffer

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

Version History

Version	Changes
`v16.0.0`	`params` is now a promise that resolves to an object
`v13.3.0`	`opengraph-image` and `twitter-image` introduced.

robots.txt

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/robots
> 📁 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for robots.txt file.*

Add or generate a `robots.txt` file that matches the Robots Exclusion Standard in the **root** of `app` directory to tell search engine crawlers which URLs they can access on your site.

Static `robots.txt`

```
// Archivo: app/robots.txt
User-Agent: *
Allow: /
Disallow: /private/

Sitemap: https://acme.com/sitemap.xml
```

Generate a Robots file

Add a `robots.js` or `robots.ts` file that returns a `Robots` object.

> **Good to know**: `robots.js` is a special Route Handler that is cached by default unless it uses a Request-time API or dynamic config option.

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: '/private/',
    },
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: '/private/',
    },
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

Output:

```
User-Agent: *
Allow: /
Disallow: /private/

Sitemap: https://acme.com/sitemap.xml
```

Customizing specific user agents

You can customize how individual search engine bots crawl your site by passing an array of user agents to the `rules` property. For example:

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      {
        userAgent: 'Googlebot',
        allow: ['/'],
        disallow: '/private/',
      },
      {
        userAgent: ['Applebot', 'Bingbot'],
        disallow: ['/'],
      },
    ],
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: [
      {
        userAgent: 'Googlebot',
        allow: ['/'],
        disallow: ['/private/'],
      },
      {
        userAgent: ['Applebot', 'Bingbot'],
        disallow: ['/'],
      },
    ],
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

Output:

```
User-Agent: Googlebot
Allow: /
Disallow: /private/

User-Agent: Applebot
Disallow: /

User-Agent: Bingbot
Disallow: /

Sitemap: https://acme.com/sitemap.xml
```

Non-standard directives

Some search engines support directives that aren't part of the Robots Exclusion Standard, such as `Request-Rate` (Seznam) or `Clean-param` (Yandex). Pass these through the `other` field on a rule. Keys preserve their casing and array values emit one line per entry, scoped to the rule's `User-Agent` block.

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      { userAgent: '*', allow: '/' },
      {
        userAgent: 'SeznamBot',
        allow: '/',
        other: {
          'Request-Rate': '10/1m',
        },
      },
    ],
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: [
      { userAgent: '*', allow: '/' },
      {
        userAgent: 'SeznamBot',
        allow: '/',
        other: {
          'Request-Rate': '10/1m',
        },
      },
    ],
  }
}
```

Output:

```
User-Agent: *
Allow: /

User-Agent: SeznamBot
Allow: /
Request-Rate: 10/1m
```

> **Good to know**: Values in `other` are passed through verbatim. Next.js does not validate directive names or values, so refer to the target search engine's documentation for the exact syntax.

Robots object

```
type Robots = {
  rules:
    | {
        userAgent?: string | string[]
        allow?: string | string[]
        disallow?: string | string[]
        crawlDelay?: number
        other?: Record<string, string | number | Array<string | number>>
      }
    | Array<{
        userAgent: string | string[]
        allow?: string | string[]
        disallow?: string | string[]
        crawlDelay?: number
        other?: Record<string, string | number | Array<string | number>>
      }>
  sitemap?: string | string[]
  host?: string
}
```

Version History

Version	Changes
`v16.3.0`	Added `other` field for non-standard per-agent directives.
`v13.3.0`	`robots` introduced.

sitemap.xml

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/sitemap
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for the sitemap.xml file.*

`sitemap.(xml|js|ts)` is a special file that matches the Sitemaps XML format to help search engine crawlers index your site more efficiently.

Sitemap files (.xml)

For smaller applications, you can create a `sitemap.xml` file and place it in the root of your `app` directory.

```
// Archivo: app/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>yearly</changefreq>
    <priority>1</priority>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Generating a sitemap using code (.js, .ts)

You can use the `sitemap.(js|ts)` file convention to programmatically **generate** a sitemap by exporting a default function that returns an array of URLs. If using TypeScript, a `Sitemap` type is available.

> **Good to know**: `sitemap.js` is a special Route Handler that is cached by default unless it uses a Request-time API or dynamic config option.

```ts
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      changeFrequency: 'yearly',
      priority: 1,
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.8,
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 0.5,
    },
  ]
}
```

```js
// Archivo: app/sitemap.js switcher
export default function sitemap() {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      changeFrequency: 'yearly',
      priority: 1,
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.8,
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 0.5,
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>yearly</changefreq>
    <priority>1</priority>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Image Sitemaps

You can use `images` property to create image sitemaps. Learn more details in the Google Developer Docs.

```
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://example.com',
      lastModified: '2021-01-01',
      changeFrequency: 'weekly',
      priority: 0.5,
      images: ['https://example.com/image.jpg'],
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<?xml version="1.0" encoding="UTF-8"?>
<urlset
  xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
  xmlns:image="http://www.google.com/schemas/sitemap-image/1.1"
>
  <url>
    <loc>https://example.com</loc>
    <image:image>
      <image:loc>https://example.com/image.jpg</image:loc>
    </image:image>
    <lastmod>2021-01-01</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Video Sitemaps

You can use `videos` property to create video sitemaps. Learn more details in the Google Developer Docs.

```ts
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://example.com',
      lastModified: '2021-01-01',
      changeFrequency: 'weekly',
      priority: 0.5,
      videos: [
        {
          title: 'example',
          thumbnail_loc: 'https://example.com/image.jpg',
          description: 'this is the description',
        },
      ],
    },
  ]
}
```

Output:

```xml
// Archivo: acme.com/sitemap.xml
<?xml version="1.0" encoding="UTF-8"?>
<urlset
  xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
  xmlns:video="http://www.google.com/schemas/sitemap-video/1.1"
>
  <url>
    <loc>https://example.com</loc>
    <video:video>
      <video:title>example</video:title>
      <video:thumbnail_loc>https://example.com/image.jpg</video:thumbnail_loc>
      <video:description>this is the description</video:description>
    </video:video>
    <lastmod>2021-01-01</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Generate a localized Sitemap

```
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es',
          de: 'https://acme.com/de',
        },
      },
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/about',
          de: 'https://acme.com/de/about',
        },
      },
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/blog',
          de: 'https://acme.com/de/blog',
        },
      },
    },
  ]
}
```

```
// Archivo: app/sitemap.js switcher
export default function sitemap() {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es',
          de: 'https://acme.com/de',
        },
      },
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/about',
          de: 'https://acme.com/de/about',
        },
      },
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/blog',
          de: 'https://acme.com/de/blog',
        },
      },
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://www.w3.org/1999/xhtml">
  <url>
    <loc>https://acme.com</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es/about" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de/about" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es/blog" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de/blog" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
</urlset>
```

Generating multiple sitemaps

While a single sitemap will work for most applications. For large web applications, you may need to split a sitemap into multiple files.

There are two ways you can create multiple sitemaps:

- By nesting `sitemap.(xml|js|ts)` inside multiple route segments e.g. `app/sitemap.xml` and `app/products/sitemap.xml`.
- By using the `generateSitemaps` function.

For example, to split a sitemap using `generateSitemaps`, return an array of objects with the sitemap `id`. Then, use the `id` to generate the unique sitemaps.

```tsx
// Archivo: app/product/sitemap.ts switcher
import type { MetadataRoute } from 'next'
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props: {
  id: Promise<string>
}): Promise<MetadataRoute.Sitemap> {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

```jsx
// Archivo: app/product/sitemap.js switcher
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props) {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

Your generated sitemaps will be available at `/ … /sitemap/[id].xml`. For example, `/product/sitemap/1.xml`.

See the `generateSitemaps` API reference for more information.

Returns

The default function exported from `sitemap.(xml|ts|js)` should return an array of objects with the following properties:

```
type Sitemap = Array<{
  url: string
  lastModified?: string | Date
  changeFrequency?:
    | 'always'
    | 'hourly'
    | 'daily'
    | 'weekly'
    | 'monthly'
    | 'yearly'
    | 'never'
  priority?: number
  alternates?: {
    languages?: Languages<string>
  }
  images?: string[]
  videos?: Videos[]
}>
```

Version History

Version	Changes
`v16.0.0`	`id` is now a promise that resolves to a `string`.
`v14.2.0`	Add localizations support.
`v13.4.14`	Add `changeFrequency` and `priority` attributes to sitemaps.
`v13.3.0`	`sitemap` introduced.

generateImageMetadata

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-image-metadata
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to generate multiple images in a single Metadata API special file.*

You can use `generateImageMetadata` to generate different versions of one image or return multiple images for one route segment. This is useful for when you want to avoid hard-coding metadata values, such as for icons.

Parameters

`generateImageMetadata` function accepts the following parameters:

`params` (optional)

An object containing the dynamic route parameters object from the root segment down to the segment `generateImageMetadata` is called from.

```
// Archivo: icon.tsx switcher
export function generateImageMetadata({
  params,
}: {
  params: { slug: string }
}) {
  // ...
}
```

```
// Archivo: icon.js switcher
export function generateImageMetadata({ params }) {
  // ...
}
```

Route	URL	`params`
`app/shop/icon.js`	`/shop`	`undefined`
`app/shop/[slug]/icon.js`	`/shop/1`	`{ slug: '1' }`
`app/shop/[tag]/[item]/icon.js`	`/shop/1/2`	`{ tag: '1', item: '2' }`

Returns

The `generateImageMetadata` function should return an `array` of objects containing the image's metadata such as `alt` and `size`. In addition, each item **must** include an `id` value which will be passed as a promise to the props of the image generating function.

Image Metadata Object	Type
`id`	`string` (required)
`alt`	`string`
`size`	`{ width: number; height: number }`
`contentType`	`string`

```
// Archivo: icon.tsx switcher
import { ImageResponse } from 'next/og'

export function generateImageMetadata() {
  return [
    {
      contentType: 'image/png',
      size: { width: 48, height: 48 },
      id: 'small',
    },
    {
      contentType: 'image/png',
      size: { width: 72, height: 72 },
      id: 'medium',
    },
  ]
}

export default async function Icon({ id }: { id: Promise<string | number> }) {
  const iconId = await id
  return new ImageResponse(
    (
      <div
        style={{
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          fontSize: 88,
          background: '#000',
          color: '#fafafa',
        }}
      >
        Icon {iconId}
      </div>
    )
  )
}
```

```
// Archivo: icon.js switcher
import { ImageResponse } from 'next/og'

export function generateImageMetadata() {
  return [
    {
      contentType: 'image/png',
      size: { width: 48, height: 48 },
      id: 'small',
    },
    {
      contentType: 'image/png',
      size: { width: 72, height: 72 },
      id: 'medium',
    },
  ]
}

export default async function Icon({ id }) {
  const iconId = await id
  return new ImageResponse(
    (
      <div
        style={{
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          fontSize: 88,
          background: '#000',
          color: '#fafafa',
        }}
      >
        Icon {iconId}
      </div>
    )
  )
}
```

Image generation function props

When using `generateImageMetadata`, the default export image generation function receives the following props:

`id`

A promise that resolves to the `id` value from one of the items returned by `generateImageMetadata`. The `id` will be a `string` or `number` depending on what was returned from `generateImageMetadata`.

```
// Archivo: icon.tsx switcher
export default async function Icon({ id }: { id: Promise<string | number> }) {
  const iconId = await id
  // Use iconId to generate the image
}
```

```
// Archivo: icon.js switcher
export default async function Icon({ id }) {
  const iconId = await id
  // Use iconId to generate the image
}
```

`params` **(optional)**

A promise that resolves to an object containing the dynamic route parameters from the root segment down to the segment the image is colocated in.

```tsx
// Archivo: icon.tsx switcher
export default async function Icon({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // Use slug to generate the image
}
```

```jsx
// Archivo: icon.js switcher
export default async function Icon({ params }) {
  const { slug } = await params
  // Use slug to generate the image
}
```

Examples

Using external data

This example uses the `params` object and external data to generate multiple Open Graph images for a route segment.

```tsx
// Archivo: app/products/[id]/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { getCaptionForImage, getOGImages } from '@/app/utils/images'

export async function generateImageMetadata({
  params,
}: {
  params: { id: string }
}) {
  const images = await getOGImages(params.id)

  return images.map((image, idx) => ({
    id: idx,
    size: { width: 1200, height: 600 },
    alt: image.text,
    contentType: 'image/png',
  }))
}

export default async function Image({
  params,
  id,
}: {
  params: Promise<{ id: string }>
  id: Promise<number>
}) {
  const productId = (await params).id
  const imageId = await id
  const text = await getCaptionForImage(productId, imageId)

  return new ImageResponse(
    (
      <div
        style={
          {
            // ...
          }
        }
      >
        {text}
      </div>
    )
  )
}
```

```
// Archivo: app/products/[id]/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { getCaptionForImage, getOGImages } from '@/app/utils/images'

export async function generateImageMetadata({ params }) {
  const images = await getOGImages(params.id)

  return images.map((image, idx) => ({
    id: idx,
    size: { width: 1200, height: 600 },
    alt: image.text,
    contentType: 'image/png',
  }))
}

export default async function Image({ params, id }) {
  const productId = (await params).id
  const imageId = await id
  const text = await getCaptionForImage(productId, imageId)

  return new ImageResponse(
    (
      <div
        style={
          {
            // ...
          }
        }
      >
        {text}
      </div>
    )
  )
}
```

Version History

Version	Changes
`v16.0.0`	`id` passed to the Image generation function is now a promise that resolves to `string` or `number`
`v16.0.0`	`params` passed to the Image generation function is now a promise that resolves to an object
`v13.3.0`	`generateImageMetadata` introduced.

generateMetadata

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-metadata
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to add Metadata to your Next.js application for improved search engine optimization (SEO) and web shareability.*

You can use the `metadata` object or the `generateMetadata` function to define metadata.

The `metadata` object

To define static metadata, export a `Metadata` object from a `layout.js` or `page.js` file.

```tsx
// Archivo: layout.tsx | page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: '...',
  description: '...',
}

export default function Page() {}
```

```jsx
// Archivo: layout.js | page.js switcher
export const metadata = {
  title: '...',
  description: '...',
}

export default function Page() {}
```

> See the Metadata Fields for a complete list of supported options.

`generateMetadata` function

Dynamic metadata depends on **dynamic information**, such as the current route parameters, external data, or `metadata` in parent segments, can be set by exporting a `generateMetadata` function that returns a `Metadata` object.

Resolving `generateMetadata` is part of rendering the page. If the page can be prerendered and `generateMetadata` doesn't introduce dynamic behavior, the resulting metadata is included in the page's initial HTML.

Otherwise the metadata resolved from `generateMetadata` can be streamed after sending the initial UI.

```tsx
// Archivo: app/products/[id]/page.tsx switcher
import type { Metadata, ResolvingMetadata } from 'next'

type Props = {
  params: Promise<{ id: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export async function generateMetadata(
  { params, searchParams }: Props,
  parent: ResolvingMetadata
): Promise<Metadata> {
  // read route params
  const { id } = await params

  // fetch data
  const product = await fetch(`https://.../${id}`).then((res) => res.json())

  // optionally access and extend (rather than replace) parent metadata
  const previousImages = (await parent).openGraph?.images || []

  return {
    title: product.title,
    openGraph: {
      images: ['/some-specific-page-image.jpg', ...previousImages],
    },
  }
}

export default function Page({ params, searchParams }: Props) {}
```

```
// Archivo: app/products/[id]/page.js switcher
export async function generateMetadata({ params, searchParams }, parent) {
  // read route params
  const { id } = await params

  // fetch data
  const product = await fetch(`https:// ... /${id}`).then((res) => res.json())

  // optionally access and extend (rather than replace) parent metadata
  const previousImages = (await parent).openGraph?.images || []

  return {
    title: product.title,
    openGraph: {
      images: ['/some-specific-page-image.jpg', ...previousImages],
    },
  }
}

export default function Page({ params, searchParams }) {}
```

For type completion of `params` and `searchParams`, you can type the first argument with `PageProps<'/route'>` or `LayoutProps<'/route'>` for pages and layouts respectively.

> **Good to know**:
>
> - Metadata can be added to `layout.js` and `page.js` files.
> - Next.js will automatically resolve the metadata, and create the relevant `<head>` tags for the page.
> - The `metadata` object and `generateMetadata` function exports are **only supported in Server Components**.
> - You cannot export both the `metadata` object and `generateMetadata` function from the same route segment.
> - `fetch` requests inside `generateMetadata` are automatically memoized for the same data across `generateMetadata`, `generateStaticParams`, Layouts, Pages, and Server Components.
> - React `cache` can be used if `fetch` is unavailable.
> - File-based metadata has the higher priority and will override the `metadata` object and `generateMetadata` function.

Why `generateMetadata` is Server Component only

`generateMetadata` and the `metadata` export are only supported in Server Components because metadata must be resolved on the server before the page component is rendered. This allows Next.js to include the metadata in the initial HTML response.

If you need to use Client Component features, keep your `page.tsx` as a Server Component and move the Client Component logic to a separate file:

```
// Archivo: app/page.tsx switcher
import type { Metadata } from 'next'
import { InteractiveComponent } from './interactive-component'

export const metadata: Metadata = {
  title: 'My Page',
}

export default function Page() {
  return <InteractiveComponent />
}
```

```
// Archivo: app/page.js switcher
import { InteractiveComponent } from './interactive-component'

export const metadata = {
  title: 'My Page',
}

export default function Page() {
  return <InteractiveComponent />
}
```

```
// Archivo: app/interactive-component.tsx switcher
'use client'

export function InteractiveComponent() {
  // Client-side interactivity (hooks, event handlers, etc.)
}
```

```
// Archivo: app/interactive-component.js switcher
'use client'

export function InteractiveComponent() {
  // Client-side interactivity (hooks, event handlers, etc.)
}
```

Reference

Parameters

`generateMetadata` function accepts the following parameters:

- `props` - An object containing the parameters of the current route:
  - `params` - An object containing the dynamic route parameters object from the root segment down to the segment `generateMetadata` is called from. Examples:

Route	URL	`params`
`app/shop/[slug]/page.js`	`/shop/1`	`{ slug: '1' }`
`app/shop/[tag]/[item]/page.js`	`/shop/1/2`	`{ tag: '1', item: '2' }`
`app/shop/[...slug]/page.js`	`/shop/1/2`	`{ slug: ['1', '2'] }`

  - `searchParams` - An object containing the current URL's search params. Examples:

URL	`searchParams`
`/shop?a=1`	`{ a: '1' }`
`/shop?a=1&b=2`	`{ a: '1', b: '2' }`
`/shop?a=1&a=2`	`{ a: ['1', '2'] }`

- `parent` - A promise of the resolved metadata from parent route segments.

Returns

`generateMetadata` should return a `Metadata` object containing one or more metadata fields.

Good to know:

- If metadata doesn't depend on request information, it should be defined using the static `metadata` object rather than `generateMetadata`.
- `fetch` requests are automatically memoized for the same data across `generateMetadata`, `generateStaticParams`, Layouts, Pages, and Server Components. React `cache` can be used if `fetch` is unavailable.
- `searchParams` are only available in `page.js` segments.
- The `redirect()` and `notFound()` Next.js methods can also be used inside `generateMetadata`.

Metadata Fields

The following fields are supported:

`title`

The `title` attribute is used to set the title of the document. It can be defined as a simple string or an optional template object.

String

```
// Archivo: layout.js | page.js
export const metadata = {
  title: 'Next.js',
}
```

```
// Archivo: <head> output hideLineNumbers
<title>Next.js</title>
```

`default`

`title.default` can be used to provide a **fallback title** to child route segments that don't define a `title`.

```
// Archivo: app/layout.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    default: 'Acme',
  },
}
```

```
// Archivo: app/about/page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {}

// Output: <title>Acme</title>
```

`template`

`title.template` can be used to add a prefix or a suffix to `titles` defined in **child** route segments.

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    template: '%s | Acme',
    default: 'Acme', // a default is required when creating a template
  },
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  title: {
    template: '%s | Acme',
    default: 'Acme', // a default is required when creating a template
  },
}
```

```
// Archivo: app/about/page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'About',
}

// Output: <title>About | Acme</title>
```

```
// Archivo: app/about/page.js switcher
export const metadata = {
  title: 'About',
}

// Output: <title>About | Acme</title>
```

Good to know:

- `title.template` applies to **child** route segments and **not** the segment it's defined in. This means:
  - `title.default` is **required** when you add a `title.template`.
  - `title.template` defined in `layout.js` will not apply to a `title` defined in a `page.js` of the same route segment.
  - `title.template` defined in `page.js` has no effect because a page is always the terminating segment (it doesn't have any children route segments).
- `title.template` has **no effect** if a route has not defined a `title` or `title.default`.

`absolute`

`title.absolute` can be used to provide a title that **ignores** `title.template` set in parent segments.

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    template: '%s | Acme',
  },
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  title: {
    template: '%s | Acme',
  },
}
```

```
// Archivo: app/about/page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    absolute: 'About',
  },
}

// Output: <title>About</title>
```

```
// Archivo: app/about/page.js switcher
export const metadata = {
  title: {
    absolute: 'About',
  },
}

// Output: <title>About</title>
```

Good to know:

- `layout.js`
  - `title` (string) and `title.default` define the default title for child segments (that do not define their own `title`). It will augment `title.template` from the closest parent segment if it exists.
  - `title.absolute` defines the default title for child segments. It ignores `title.template` from parent segments.
  - `title.template` defines a new title template for child segments.
- `page.js`
  - If a page does not define its own title the closest parents resolved title will be used.
  - `title` (string) defines the routes title. It will augment `title.template` from the closest parent segment if it exists.
  - `title.absolute` defines the route title. It ignores `title.template` from parent segments.
  - `title.template` has no effect in `page.js` because a page is always the terminating segment of a route.

`description`

```
// Archivo: layout.js | page.js
export const metadata = {
  description: 'The React Framework for the Web',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="description" content="The React Framework for the Web" />
```

Other fields

```
// Archivo: layout.js | page.js
export const metadata = {
  generator: 'Next.js',
  applicationName: 'Next.js',
  referrer: 'origin-when-cross-origin',
  keywords: ['Next.js', 'React', 'JavaScript'],
  authors: [{ name: 'Seb' }, { name: 'Josh', url: 'https://nextjs.org' }],
  creator: 'Jiachi Liu',
  publisher: 'Sebastian Markbåge',
  formatDetection: {
    email: false,
    address: false,
    telephone: false,
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="application-name" content="Next.js" />
<meta name="author" content="Seb" />
<link rel="author" href="https://nextjs.org" />
<meta name="author" content="Josh" />
<meta name="generator" content="Next.js" />
<meta name="keywords" content="Next.js,React,JavaScript" />
<meta name="referrer" content="origin-when-cross-origin" />
<meta name="creator" content="Jiachi Liu" />
<meta name="publisher" content="Sebastian Markbåge" />
<meta name="format-detection" content="telephone=no, address=no, email=no" />
```

`metadataBase`

`metadataBase` is a convenience option to set a base URL prefix for `metadata` fields that require a fully qualified URL.

- `metadataBase` allows URL-based `metadata` fields defined in the **current route segment and below** to use a **relative path** instead of an otherwise required absolute URL.
- The field's relative path will be composed with `metadataBase` to form a fully qualified URL.

```
// Archivo: layout.js | page.js
export const metadata = {
  metadataBase: new URL('https://acme.com'),
  alternates: {
    canonical: '/',
    languages: {
      'en-US': '/en-US',
      'de-DE': '/de-DE',
    },
  },
  openGraph: {
    images: '/og-image.png',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="canonical" href="https://acme.com" />
<link rel="alternate" hreflang="en-US" href="https://acme.com/en-US" />
<link rel="alternate" hreflang="de-DE" href="https://acme.com/de-DE" />
<meta property="og:image" content="https://acme.com/og-image.png" />
```

Good to know:

- `metadataBase` is typically set in root `app/layout.js` to apply to URL-based `metadata` fields across all routes.
- When `generateMetadata` uses `'use cache'`, its return value must be serializable. `URL` instances are not supported by Cache Functions, so values such as `metadataBase` should be returned as strings (for example, `url.toString()`). See `use cache` serialization requirements.
- All URL-based `metadata` fields that require absolute URLs can be configured with a `metadataBase` option.
- `metadataBase` can contain a subdomain e.g. `https://app.acme.com` or base path e.g. `https://acme.com/start/from/here`
- If a `metadata` field provides an absolute URL, `metadataBase` will be ignored.
- Using a relative path in a URL-based `metadata` field without configuring a `metadataBase` will cause a build error.
- Next.js will normalize duplicate slashes between `metadataBase` (e.g. `https://acme.com/`) and a relative field (e.g. `/path`) to a single slash (e.g. `https://acme.com/path`)

URL Composition

URL composition favors developer intent over default directory traversal semantics.

- Trailing slashes between `metadataBase` and `metadata` fields are normalized.
- An "absolute" path in a `metadata` field (that typically would replace the whole URL path) is treated as a "relative" path (starting from the end of `metadataBase`).

For example, given the following `metadataBase`:

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  metadataBase: new URL('https://acme.com'),
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  metadataBase: new URL('https://acme.com'),
}
```

Any `metadata` fields that inherit the above `metadataBase` and set their own value will be resolved as follows:

`metadata` field	Resolved URL
`/`	`https://acme.com`
`./`	`https://acme.com`
`payments`	`https://acme.com/payments`
`/payments`	`https://acme.com/payments`
`./payments`	`https://acme.com/payments`
`../payments`	`https://acme.com/payments`
`https://beta.acme.com/payments`	`https://beta.acme.com/payments`

`openGraph`

```
// Archivo: layout.js | page.js
export const metadata = {
  openGraph: {
    title: 'Next.js',
    description: 'The React Framework for the Web',
    url: 'https://nextjs.org',
    siteName: 'Next.js',
    images: [
      {
        url: 'https://nextjs.org/og.png', // Must be an absolute URL
        width: 800,
        height: 600,
      },
      {
        url: 'https://nextjs.org/og-alt.png', // Must be an absolute URL
        width: 1800,
        height: 1600,
        alt: 'My custom alt',
      },
    ],
    videos: [
      {
        url: 'https://nextjs.org/video.mp4', // Must be an absolute URL
        width: 800,
        height: 600,
      },
    ],
    audio: [
      {
        url: 'https://nextjs.org/audio.mp3', // Must be an absolute URL
      },
    ],
    locale: 'en_US',
    type: 'website',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="og:title" content="Next.js" />
<meta property="og:description" content="The React Framework for the Web" />
<meta property="og:url" content="https://nextjs.org/" />
<meta property="og:site_name" content="Next.js" />
<meta property="og:locale" content="en_US" />
<meta property="og:image" content="https://nextjs.org/og.png" />
<meta property="og:image:width" content="800" />
<meta property="og:image:height" content="600" />
<meta property="og:image" content="https://nextjs.org/og-alt.png" />
<meta property="og:image:width" content="1800" />
<meta property="og:image:height" content="1600" />
<meta property="og:image:alt" content="My custom alt" />
<meta property="og:video" content="https://nextjs.org/video.mp4" />
<meta property="og:video:width" content="800" />
<meta property="og:video:height" content="600" />
<meta property="og:audio" content="https://nextjs.org/audio.mp3" />
<meta property="og:type" content="website" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  openGraph: {
    title: 'Next.js',
    description: 'The React Framework for the Web',
    type: 'article',
    publishedTime: '2023-01-01T00:00:00.000Z',
    authors: ['Seb', 'Josh'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="og:title" content="Next.js" />
<meta property="og:description" content="The React Framework for the Web" />
<meta property="og:type" content="article" />
<meta property="article:published_time" content="2023-01-01T00:00:00.000Z" />
<meta property="article:author" content="Seb" />
<meta property="article:author" content="Josh" />
```

Good to know:

- It may be more convenient to use the file-based Metadata API for Open Graph images. Rather than having to sync the config export with actual files, the file-based API will automatically generate the correct metadata for you.

`robots`

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  robots: {
    index: true,
    follow: true,
    nocache: false,
    googleBot: {
      index: true,
      follow: true,
      noimageindex: false,
      'max-video-preview': -1,
      'max-image-preview': 'large',
      'max-snippet': -1,
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="robots" content="index, follow" />
<meta
  name="googlebot"
  content="index, follow, max-video-preview:-1, max-image-preview:large, max-snippet:-1"
/>
```

`icons`

Good to know: We recommend using the file-based Metadata API for icons where possible. Rather than having to sync the config export with actual files, the file-based API will automatically generate the correct metadata for you.

```
// Archivo: layout.js | page.js
export const metadata = {
  icons: {
    icon: '/icon.png',
    shortcut: '/shortcut-icon.png',
    apple: '/apple-icon.png',
    other: {
      rel: 'apple-touch-icon-precomposed',
      url: '/apple-touch-icon-precomposed.png',
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="shortcut icon" href="/shortcut-icon.png" />
<link rel="icon" href="/icon.png" />
<link rel="apple-touch-icon" href="/apple-icon.png" />
<link
  rel="apple-touch-icon-precomposed"
  href="/apple-touch-icon-precomposed.png"
/>
```

```
// Archivo: layout.js | page.js
export const metadata = {
  icons: {
    icon: [
      { url: '/icon.png' },
      new URL('/icon.png', 'https://example.com'),
      { url: '/icon-dark.png', media: '(prefers-color-scheme: dark)' },
    ],
    shortcut: ['/shortcut-icon.png'],
    apple: [
      { url: '/apple-icon.png' },
      { url: '/apple-icon-x3.png', sizes: '180x180', type: 'image/png' },
    ],
    other: [
      {
        rel: 'apple-touch-icon-precomposed',
        url: '/apple-touch-icon-precomposed.png',
      },
    ],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="shortcut icon" href="/shortcut-icon.png" />
<link rel="icon" href="/icon.png" />
<link rel="icon" href="https://example.com/icon.png" />
<link rel="icon" href="/icon-dark.png" media="(prefers-color-scheme: dark)" />
<link rel="apple-touch-icon" href="/apple-icon.png" />
<link
  rel="apple-touch-icon-precomposed"
  href="/apple-touch-icon-precomposed.png"
/>
<link
  rel="apple-touch-icon"
  href="/apple-icon-x3.png"
  sizes="180x180"
  type="image/png"
/>
```

> **Good to know**: The `msapplication-*` meta tags are no longer supported in Chromium builds of Microsoft Edge, and thus no longer needed.

`themeColor`

> **Deprecated**: The `themeColor` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`colorScheme`

> **Deprecated**: The `colorScheme` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`manifest`

A web application manifest, as defined in the Web Application Manifest specification.

```
// Archivo: layout.js | page.js
export const metadata = {
  manifest: 'https://nextjs.org/manifest.json',
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="manifest" href="https://nextjs.org/manifest.json" />
```

`twitter`

The Twitter specification is (surprisingly) used for more than just X (formerly known as Twitter).

Learn more about the Twitter Card markup reference.

```
// Archivo: layout.js | page.js
export const metadata = {
  twitter: {
    card: 'summary_large_image',
    title: 'Next.js',
    description: 'The React Framework for the Web',
    siteId: '1467726470533754880',
    creator: '@nextjs',
    creatorId: '1467726470533754880',
    images: ['https://nextjs.org/og.png'], // Must be an absolute URL
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:site:id" content="1467726470533754880" />
<meta name="twitter:creator" content="@nextjs" />
<meta name="twitter:creator:id" content="1467726470533754880" />
<meta name="twitter:title" content="Next.js" />
<meta name="twitter:description" content="The React Framework for the Web" />
<meta name="twitter:image" content="https://nextjs.org/og.png" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  twitter: {
    card: 'app',
    title: 'Next.js',
    description: 'The React Framework for the Web',
    siteId: '1467726470533754880',
    creator: '@nextjs',
    creatorId: '1467726470533754880',
    images: {
      url: 'https://nextjs.org/og.png',
      alt: 'Next.js Logo',
    },
    app: {
      name: 'twitter_app',
      id: {
        iphone: 'twitter_app://iphone',
        ipad: 'twitter_app://ipad',
        googleplay: 'twitter_app://googleplay',
      },
      url: {
        iphone: 'https://iphone_url',
        ipad: 'https://ipad_url',
      },
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="twitter:site:id" content="1467726470533754880" />
<meta name="twitter:creator" content="@nextjs" />
<meta name="twitter:creator:id" content="1467726470533754880" />
<meta name="twitter:title" content="Next.js" />
<meta name="twitter:description" content="The React Framework for the Web" />
<meta name="twitter:card" content="app" />
<meta name="twitter:image" content="https://nextjs.org/og.png" />
<meta name="twitter:image:alt" content="Next.js Logo" />
<meta name="twitter:app:name:iphone" content="twitter_app" />
<meta name="twitter:app:id:iphone" content="twitter_app://iphone" />
<meta name="twitter:app:id:ipad" content="twitter_app://ipad" />
<meta name="twitter:app:id:googleplay" content="twitter_app://googleplay" />
<meta name="twitter:app:url:iphone" content="https://iphone_url" />
<meta name="twitter:app:url:ipad" content="https://ipad_url" />
<meta name="twitter:app:name:ipad" content="twitter_app" />
<meta name="twitter:app:name:googleplay" content="twitter_app" />
```

`viewport`

> **Deprecated**: The `viewport` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`verification`

```
// Archivo: layout.js | page.js
export const metadata = {
  verification: {
    google: 'google',
    yandex: 'yandex',
    yahoo: 'yahoo',
    other: {
      me: ['my-email', 'my-link'],
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="google-site-verification" content="google" />
<meta name="y_key" content="yahoo" />
<meta name="yandex-verification" content="yandex" />
<meta name="me" content="my-email" />
<meta name="me" content="my-link" />
```

`appleWebApp`

```
// Archivo: layout.js | page.js
export const metadata = {
  itunes: {
    appId: 'myAppStoreID',
    appArgument: 'myAppArgument',
  },
  appleWebApp: {
    title: 'Apple Web App',
    statusBarStyle: 'black-translucent',
    startupImage: [
      '/assets/startup/apple-touch-startup-image-768x1004.png',
      {
        url: '/assets/startup/apple-touch-startup-image-1536x2008.png',
        media: '(device-width: 768px) and (device-height: 1024px)',
      },
    ],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta
  name="apple-itunes-app"
  content="app-id=myAppStoreID, app-argument=myAppArgument"
/>
<meta name="mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-title" content="Apple Web App" />
<link
  href="/assets/startup/apple-touch-startup-image-768x1004.png"
  rel="apple-touch-startup-image"
/>
<link
  href="/assets/startup/apple-touch-startup-image-1536x2008.png"
  media="(device-width: 768px) and (device-height: 1024px)"
  rel="apple-touch-startup-image"
/>
<meta
  name="apple-mobile-web-app-status-bar-style"
  content="black-translucent"
/>
```

`alternates`

```
// Archivo: layout.js | page.js
export const metadata = {
  alternates: {
    canonical: 'https://nextjs.org',
    languages: {
      'en-US': 'https://nextjs.org/en-US',
      'de-DE': 'https://nextjs.org/de-DE',
    },
    media: {
      'only screen and (max-width: 600px)': 'https://nextjs.org/mobile',
    },
    types: {
      'application/rss+xml': 'https://nextjs.org/rss',
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="canonical" href="https://nextjs.org" />
<link rel="alternate" hreflang="en-US" href="https://nextjs.org/en-US" />
<link rel="alternate" hreflang="de-DE" href="https://nextjs.org/de-DE" />
<link
  rel="alternate"
  media="only screen and (max-width: 600px)"
  href="https://nextjs.org/mobile"
/>
<link
  rel="alternate"
  type="application/rss+xml"
  href="https://nextjs.org/rss"
/>
```

`appLinks`

```
// Archivo: layout.js | page.js
export const metadata = {
  appLinks: {
    ios: {
      url: 'https://nextjs.org/ios',
      app_store_id: 'app_store_id',
    },
    android: {
      package: 'com.example.android/package',
      app_name: 'app_name_android',
    },
    web: {
      url: 'https://nextjs.org/web',
      should_fallback: true,
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="al:ios:url" content="https://nextjs.org/ios" />
<meta property="al:ios:app_store_id" content="app_store_id" />
<meta property="al:android:package" content="com.example.android/package" />
<meta property="al:android:app_name" content="app_name_android" />
<meta property="al:web:url" content="https://nextjs.org/web" />
<meta property="al:web:should_fallback" content="true" />
```

`archives`

Describes a collection of records, documents, or other materials of historical interest (source).

```
// Archivo: layout.js | page.js
export const metadata = {
  archives: ['https://nextjs.org/13'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="archives" href="https://nextjs.org/13" />
```

`assets`

```
// Archivo: layout.js | page.js
export const metadata = {
  assets: ['https://nextjs.org/assets'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="assets" href="https://nextjs.org/assets" />
```

`bookmarks`

```
// Archivo: layout.js | page.js
export const metadata = {
  bookmarks: ['https://nextjs.org/13'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="bookmarks" href="https://nextjs.org/13" />
```

`pagination`

Describes the previous and next pages in a paginated sequence.

```
// Archivo: layout.js | page.js
export const metadata = {
  pagination: {
    previous: 'https://nextjs.org/blog?page=1',
    next: 'https://nextjs.org/blog?page=3',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="prev" href="https://nextjs.org/blog?page=1" />
<link rel="next" href="https://nextjs.org/blog?page=3" />
```

`category`

```
// Archivo: layout.js | page.js
export const metadata = {
  category: 'technology',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="category" content="technology" />
```

`facebook`

You can connect a Facebook app or Facebook account to your webpage for certain Facebook Social Plugins Facebook Documentation

> **Good to know**: You can specify either appId or admins, but not both.

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    appId: '12345678',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:app_id" content="12345678" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    admins: '12345678',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:admins" content="12345678" />
```

If you want to generate multiple fb:admins meta tags you can use array value.

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    admins: ['12345678', '87654321'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:admins" content="12345678" />
<meta property="fb:admins" content="87654321" />
```

`pinterest`

You can enable or disable Pinterest Rich Pins on your webpage.

```
// Archivo: layout.js | page.js
export const metadata = {
  pinterest: {
    richPin: true,
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="pinterest-rich-pin" content="true" />
```

`other`

All metadata options should be covered using the built-in support. However, there may be custom metadata tags specific to your site, or brand new metadata tags just released. You can use the `other` option to render any custom metadata tag.

```
// Archivo: layout.js | page.js
export const metadata = {
  other: {
    custom: 'meta',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="custom" content="meta" />
```

If you want to generate multiple same key meta tags you can use array value.

```
// Archivo: layout.js | page.js
export const metadata = {
  other: {
    custom: ['meta1', 'meta2'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="custom" content="meta1" /> <meta name="custom" content="meta2" />
```

Types

You can add type safety to your metadata by using the `Metadata` type. If you are using the built-in TypeScript plugin in your IDE, you do not need to manually add the type, but you can still explicitly add it if you want.

`metadata` object

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Next.js',
}
```

`generateMetadata` function

Regular function

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export function generateMetadata(): Metadata {
  return {
    title: 'Next.js',
  }
}
```

Async function

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export async function generateMetadata(): Promise<Metadata> {
  return {
    title: 'Next.js',
  }
}
```

With segment props

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

type Props = {
  params: Promise<{ id: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export function generateMetadata({ params, searchParams }: Props): Metadata {
  return {
    title: 'Next.js',
  }
}

export default function Page({ params, searchParams }: Props) {}
```

With parent metadata

```tsx
// Archivo: layout.tsx | page.tsx
import type { Metadata, ResolvingMetadata } from 'next'

export async function generateMetadata(
  { params, searchParams }: Props,
  parent: ResolvingMetadata
): Promise<Metadata> {
  return {
    title: 'Next.js',
  }
}
```

JavaScript Projects

For JavaScript projects, you can use JSDoc to add type safety.

```jsx
// Archivo: layout.js | page.js
/** @type {import("next").Metadata} */
export const metadata = {
  title: 'Next.js',
}
```

Unsupported Metadata

The following metadata types do not currently have built-in support. However, they can still be rendered in the layout or page itself.

Metadata	Recommendation
`<meta http-equiv="...">`	Use appropriate HTTP Headers via `redirect()`, Proxy, Security Headers
`<base>`	Render the tag in the layout or page itself.
`<noscript>`	Render the tag in the layout or page itself.
`<style>`	Learn more about styling in Next.js.
`<script>`	Learn more about using scripts.
`<link rel="stylesheet" />`	`import` stylesheets directly in the layout or page itself.
`<link rel="preload" />`	Use ReactDOM preload method
`<link rel="preconnect" />`	Use ReactDOM preconnect method
`<link rel="dns-prefetch" />`	Use ReactDOM prefetchDNS method

Resource hints

The `<link>` element has a number of `rel` keywords that can be used to hint to the browser that an external resource is likely to be needed. The browser uses this information to apply preloading optimizations depending on the keyword.

While the Metadata API doesn't directly support these hints, you can use new `ReactDOM` methods to safely insert them into the `<head>` of the document.

```tsx
// Archivo: app/preload-resources.tsx switcher
'use client'

import ReactDOM from 'react-dom'

export function PreloadResources() {
  ReactDOM.preload('...', { as: '...' })
  ReactDOM.preconnect('...', { crossOrigin: '...' })
  ReactDOM.prefetchDNS('...')

  return '...'
}
```

```
// Archivo: app/preload-resources.js switcher
'use client'

import ReactDOM from 'react-dom'

export function PreloadResources() {
  ReactDOM.preload('...', { as: '...' })
  ReactDOM.preconnect('...', { crossOrigin: '...' })
  ReactDOM.prefetchDNS('...')

  return '...'
}
```

`<link rel="preload">`

Start loading a resource early in the page rendering (browser) lifecycle. MDN Docs.

```
ReactDOM.preload(href: string, options: { as: string })
```

```
// Archivo: <head> output hideLineNumbers
<link rel="preload" href="..." as="..." />
```

`<link rel="preconnect">`

Preemptively initiate a connection to an origin. MDN Docs.

```
ReactDOM.preconnect(href: string, options?: { crossOrigin?: string })
```

```
// Archivo: <head> output hideLineNumbers
<link rel="preconnect" href="..." crossorigin />
```

`<link rel="dns-prefetch">`

Attempt to resolve a domain name before resources get requested. MDN Docs.

```
ReactDOM.prefetchDNS(href: string)
```

```
// Archivo: <head> output hideLineNumbers
<link rel="dns-prefetch" href="..." />
```

> **Good to know**:
>
> - These methods are currently only supported in Client Components, which are still Server Side Rendered on initial page load.
> - Next.js in-built features such as `next/font`, `next/image` and `next/script` automatically handle relevant resource hints.

Behavior

Default Fields

There are two default `meta` tags that are always added even if a route doesn't define metadata:

- The meta charset tag sets the character encoding for the website.
- The meta viewport tag sets the viewport width and scale for the website to adjust for different devices.

```
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

> **Good to know**: You can overwrite the default `viewport` meta tag.

Streaming metadata

Streaming metadata allows Next.js to render and send the initial UI to the browser, without waiting for `generateMetadata` to complete.

When `generateMetadata` resolves, the resulting metadata tags are appended to the `<body>` tag. We have verified that metadata is interpreted correctly by bots that execute JavaScript and inspect the full DOM (e.g. `Googlebot`).

For **HTML-limited bots** that can't execute JavaScript (e.g. `facebookexternalhit`), metadata continues to block page rendering. The resulting metadata will be available in the `<head>` tag.

Next.js automatically detects **HTML-limited bots** by looking at the User Agent header. You can use the `htmlLimitedBots` option in your Next.js config file to override the default User Agent list.

To fully disable streaming metadata:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const config: NextConfig = {
  htmlLimitedBots: /.*/,
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  htmlLimitedBots: /.*/,
}
```

Streaming metadata improves perceived performance by reducing TTFB and can help lowering LCP time.

Overriding `htmlLimitedBots` could lead to longer response times. Streaming metadata is an advanced feature, and the default should be sufficient for most cases.

With Cache Components

When Cache Components is enabled, `generateMetadata` follows the same rules as other components. If metadata accesses runtime data (`cookies()`, `headers()`, `params`, `searchParams`) or performs uncached data fetching, it defers to request time.

How Next.js handles this depends on the rest of your page:

- **If other parts also defer to request time**: Prerendering generates a static shell, and metadata streams in with other deferred content.
- **If the page or layout is otherwise fully prerenderable**: Next.js requires an explicit choice: cache the data if possible, or signal that deferred rendering is intentional.

Streaming metadata at runtime while the rest of the page is fully prerenderable is not common. To ensure this behavior is intentional, an error is raised indicating which page or layout needs to be handled.

To resolve this, you have two options. If metadata depends on external data but not runtime data, use `use cache`:

```
// Archivo: app/page.tsx highlight={2}
export async function generateMetadata() {
  'use cache'
  const { title, description } = await db.query('site-metadata')
  return { title, description }
}
```

If metadata genuinely requires runtime data, add a dynamic marker component to your page:

```
// Archivo: app/page.tsx highlight={11-14,19,31}
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { connection } from 'next/server'

export async function generateMetadata() {
  const token = (await cookies()).get('token')?.value
  // ... use token to fetch personalized metadata
  return { title: 'Personalized Title' }
}

const Connection = async () => {
  await connection()
  return null
}

async function DynamicMarker() {
  return (
    ``<Suspense>``
  )
}

export default function Page() {
  // DO NOT place await connection() here
  // doing so prevents the article tag content from
  // being included in the static shell
  return (
    <>
      <article>Static content</article>
      <DynamicMarker />
    </>
  )
}
```

The `DynamicMarker` component renders nothing but tells Next.js the page has intentional dynamic content. By wrapping it in Suspense, the static content still prerenders normally.

See Next.js encountered runtime data in `generateMetadata()` and Next.js encountered uncached data in `generateMetadata()` for full fix options and trade-offs.

Ordering

Metadata is evaluated in order, starting from the root segment down to the segment closest to the final `page.js` segment. For example:

1. `app/layout.tsx` (Root Layout)
2. `app/blog/layout.tsx` (Nested Blog Layout)
3. `app/blog/[slug]/page.tsx` (Blog Page)

Merging

Following the evaluation order, Metadata objects exported from multiple segments in the same route are **shallowly** merged together to form the final metadata output of a route. Duplicate keys are **replaced** based on their ordering.

This means metadata with nested fields such as `openGraph` and `robots` that are defined in an earlier segment are **overwritten** by the last segment to define them.

Overwriting fields

```
// Archivo: app/layout.js
export const metadata = {
  title: 'Acme',
  openGraph: {
    title: 'Acme',
    description: 'Acme is a ...',
  },
}
```

```
// Archivo: app/blog/page.js
export const metadata = {
  title: 'Blog',
  openGraph: {
    title: 'Blog',
  },
}

// Output:
// <title>Blog</title>
// <meta property="og:title" content="Blog" />
```

In the example above:

- `title` from `app/layout.js` is **replaced** by `title` in `app/blog/page.js`.
- All `openGraph` fields from `app/layout.js` are **replaced** in `app/blog/page.js` because `app/blog/page.js` sets `openGraph` metadata. Note the absence of `openGraph.description`.

If you'd like to share some nested fields between segments while overwriting others, you can pull them out into a separate variable:

```
// Archivo: app/shared-metadata.js
export const openGraphImage = { images: ['http:// ... '] }
```

```
// Archivo: app/page.js
import { openGraphImage } from './shared-metadata'

export const metadata = {
  openGraph: {
    ... openGraphImage,
    title: 'Home',
  },
}
```

```
// Archivo: app/about/page.js
import { openGraphImage } from '../shared-metadata'

export const metadata = {
  openGraph: {
    ... openGraphImage,
    title: 'About',
  },
}
```

In the example above, the OG image is shared between `app/layout.js` and `app/about/page.js` while the titles are different.

Inheriting fields

```
// Archivo: app/layout.js
export const metadata = {
  title: 'Acme',
  openGraph: {
    title: 'Acme',
    description: 'Acme is a ... ',
  },
}
```

```
// Archivo: app/about/page.js
export const metadata = {
  title: 'About',
}

// Output:
// <title>About</title>
// <meta property="og:title" content="Acme" />
// <meta property="og:description" content="Acme is a ... " />
```

Notes

- `title` from `app/layout.js` is **replaced** by `title` in `app/about/page.js`.
- All `openGraph` fields from `app/layout.js` are **inherited** in `app/about/page.js` because `app/about/page.js` doesn't set `openGraph` metadata.

Version History

Version	Changes
`v15.2.0`	Introduced streaming support to `generateMetadata`.
`v14.0.0`	`viewport`, `themeColor`, and `colorScheme` deprecated in favor of the `viewport` configuration.
`v13.2.0`	`metadata` and `generateMetadata` introduced.

generateSitemaps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-sitemaps
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to use the generateSiteMaps function to create multiple sitemaps for your application.*

You can use the `generateSitemaps` function to generate multiple sitemaps for your application.

Returns

The `generateSitemaps` returns an array of objects with an `id` property.

URLs

Your generated sitemaps will be available at `/ ... /sitemap/[id].xml`. For example, `/product/sitemap/1.xml`.

Example

For example, to split a sitemap using `generateSitemaps`, return an array of objects with the sitemap `id`. Then, use the `id` to generate the unique sitemaps.

```ts
// Archivo: app/product/sitemap.ts switcher
import type { MetadataRoute } from 'next'
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props: {
  id: Promise<string>
}): Promise<MetadataRoute.Sitemap> {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

```js
// Archivo: app/product/sitemap.js switcher
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props) {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

Version History

Version	Changes
`v16.0.0`	The `id` values returned from `generateSitemaps` are now passed as a promise that resolves to a `string` to the sitemap function.
`v15.0.0`	`generateSitemaps` now generates consistent URLs between development and production
`v13.3.2`	`generateSitemaps` introduced. In development, you can view the generated sitemap on `/ ... /sitemap.xml/[id]`. For example, `/product/sitemap.xml/1`.

images

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/images
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Custom configuration for the next/image loader*

If you want to use a cloud provider to optimize images instead of using the Next.js built-in Image Optimization API, you can configure `next.config.js` with the following:

```
// Archivo: next.config.js
module.exports = {
  images: {
    loader: 'custom',
    loaderFile: './my/image/loader.js',
  },
}
```

This `loaderFile` must point to a file relative to the root of your Next.js application. The file must export a default function that returns a string, for example:

```
// Archivo: my/image/loader.js
'use client'

export default function myImageLoader({ src, width, quality }) {
  return `https://example.com/${src}?w=${width}&q=${quality || 75}`
}
```

Alternatively, you can use the `loader` prop to pass the function to each instance of `next/image`.

> **Good to know**: Customizing the image loader file, which accepts a function, requires using Client Components to serialize the provided function.

To learn more about configuring the behavior of the built-in Image Optimization API and the Image Component, see Image Configuration Options for available options.

> 📘 **Nota para Pages Router:**
>
> ```
> export default function myImageLoader({ src, width, quality }) {
>   return `https://example.com/${src}?w=${width}&q=${quality || 75}`
> }
> ```
>
> Alternatively, you can use the `loader` prop to pass the function to each instance of `next/image`.
>
> To learn more about configuring the behavior of the built-in Image Optimization API and the Image Component, see Image Configuration Options for available options.

Example Loader Configuration

- Akamai
- AWS CloudFront
- Cloudinary
- Cloudflare
- Contentful
- Fastly
- Gumlet

- ImageEngine
- Imgix
- PixelBin
- Sanity
- Sirv
- Supabase
- Thumbor
- Imagekit
- Nitrogen AIO

Akamai

```
// Docs: https://techdocs.akamai.com/ivm/reference/test-images-on-demand
export default function akamaiLoader({ src, width, quality }) {
  return `https://example.com/${src}?imwidth=${width}`
}
```

AWS CloudFront

```
// Docs: https://aws.amazon.com/developer/application-security-performance/articles/image-optimization
export default function cloudfrontLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('format', 'auto')
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Cloudinary

```
// Demo: https://res.cloudinary.com/demo/image/upload/w_300,c_limit,q_auto/turtles.jpg
export default function cloudinaryLoader({ src, width, quality }) {
  const params = ['f_auto', 'c_limit', `w_${width}`, `q_${quality || 'auto'}`]
  return `https://example.com/${params.join(',')}${src}`
}
```

Cloudflare

```
// Docs: https://developers.cloudflare.com/images/transform-images
export default function cloudflareLoader({ src, width, quality }) {
  const params = [`width=${width}`, `quality=${quality || 75}`, 'format=auto']
  return `https://example.com/cdn-cgi/image/${params.join(',')}/${src}`
}
```

Contentful

```
// Docs: https://www.contentful.com/developers/docs/references/images-api/
export default function contentfulLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('fm', 'webp')
  url.searchParams.set('w', width.toString())
  url.searchParams.set('q', (quality || 75).toString())
  return url.href
}
```

Fastly

```
// Docs: https://developer.fastly.com/reference/io/
export default function fastlyLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('auto', 'webp')
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Gumlet

```
// Docs: https://docs.gumlet.com/reference/image-transform-size
export default function gumletLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('format', 'auto')
  url.searchParams.set('w', width.toString())
  url.searchParams.set('q', (quality || 75).toString())
  return url.href
}
```

ImageEngine

```
// Docs: https://support.imageengine.io/hc/en-us/articles/360058880672-Directives
export default function imageengineLoader({ src, width, quality }) {
  const compression = 100 - (quality || 50)
  const params = [`w_${width}`, `cmpr_${compression}`]
  return `https://example.com${src}?imgeng=/${params.join('/')}`
}
```

Imgix

```
// Demo: https://static.imgix.net/daisy.png?format=auto&fit=max&w=300
export default function imgixLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  const params = url.searchParams
  params.set('auto', params.getAll('auto').join(',') || 'format')
  params.set('fit', params.get('fit') || 'max')
  params.set('w', params.get('w') || width.toString())
  params.set('q', (quality || 50).toString())
  return url.href
}
```

PixelBin

```
// Doc (Resize): https://www.pixelbin.io/docs/transformations/basic/resize/#width-w
// Doc (Optimise): https://www.pixelbin.io/docs/optimizations/quality/#image-quality-when-delivering
// Doc (Auto Format Delivery): https://www.pixelbin.io/docs/optimizations/format/#automatic-format-selection-with-f_auto-url-
parameter
export default function pixelBinLoader({ src, width, quality }) {
  const name = '<your-cloud-name>'
  const opt = `t.resize(w:${width})~t.compress(q:${quality || 75})`
  return `https://cdn.pixelbin.io/v2/${name}/${opt}/${src}?f_auto=true`
}
```

Sanity

```
// Docs: https://www.sanity.io/docs/image-urls
export default function sanityLoader({ src, width, quality }) {
  const prj = 'zp7mbokg'
  const dataset = 'production'
  const url = new URL(`https://cdn.sanity.io/images/${prj}/${dataset}${src}`)
  url.searchParams.set('auto', 'format')
  url.searchParams.set('fit', 'max')
  url.searchParams.set('w', width.toString())
  if (quality) {
    url.searchParams.set('q', quality.toString())
  }
  return url.href
}
```

Sirv

```
// Docs: https://sirv.com/help/articles/dynamic-imaging/
export default function sirvLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  const params = url.searchParams
  params.set('format', params.getAll('format').join(',') || 'optimal')
  params.set('w', params.get('w') || width.toString())
  params.set('q', (quality || 85).toString())
  return url.href
}
```

Supabase

```
// Docs: https://supabase.com/docs/guides/storage/image-transformations#nextjs-loader
export default function supabaseLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Thumbor

```
// Docs: https://thumbor.readthedocs.io/en/latest/
export default function thumborLoader({ src, width, quality }) {
  const params = [`${width}x0`, `filters:quality(${quality || 75})`]
  return `https://example.com${params.join('/')}${src}`
}
```

ImageKit.io

```
// Docs: https://imagekit.io/docs/image-transformation
export default function imageKitLoader({ src, width, quality }) {
  const params = [`w-${width}`, `q-${quality || 80}`]
  return `https://ik.imagekit.io/your_imagekit_id/${src}?tr=${params.join(',')}`
}
```

Nitrogen AIO

```
// Docs: https://docs.n7.io/aio/intergrations/
export default function aioLoader({ src, width, quality }) {
  const url = new URL(src, window.location.href)
  const params = url.searchParams
  const aioParams = params.getAll('aio')
  aioParams.push(`w-${width}`)
  if (quality) {
    aioParams.push(`q-${quality.toString()}`)
  }
  params.set('aio', aioParams.join(';'))
  return url.href
}
```

Image Optimization

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/images
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Optimize your images with the built-in* `next/image` *component.*

How to use fonts

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/fonts
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Learn how to use fonts in Next.js*

How to load and optimize scripts

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/scripts
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Optimize 3rd party scripts with the built-in Script component.*

images

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/images
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Custom configuration for the next/image loader*

Capítulo 8: Seguridad, Autenticación y Protección de Datos

> *Autenticación con Cache Components, Content Security Policy (CSP), protección contra ataques, cookies y middleware.*

How to implement authentication with Cache Components

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/authentication-with-cache-components
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to read the user session, show authenticated UI without slowing down the page, and cache data derived from the session when Cache Components is enabled.*

With Cache Components enabled, a session read happens at request time, so it can't be prerendered into the static shell. Authenticated UI streams in behind a `````) }

async function Announcements() { 'use cache' const announcements = await getAnnouncements() return (

{announcements.map((announcement) => (

- {announcement}

))}

) }

async function Dashboard() { const user = await getCurrentUser() return

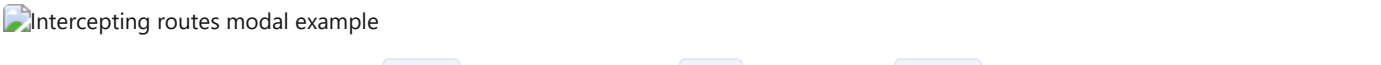
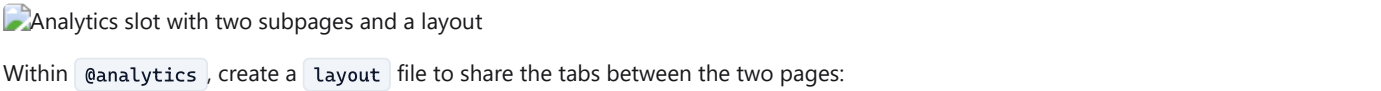
Welcome, {user.name}

}

````
Keep the session read out of a layout's top level, too. A top-level `await` on the session in a layout holds the whole segment, including `{children}`, behind that request, so push it into a component inside a boundary. See [Push dynamic access down](/docs/app/guides/streaming#push-dynamic-access-down).

> **Good to know:** `getCurrentUser` reads the session, checks it, and returns a narrow user. Centralizing those reads in one function is the [Data Access Layer](/docs/app/guides/authentication#creating-a-data-access-layer-dal) pattern.

## Step 3: Share the user across components

You don't have to read the session again in every component that needs the user. Read it once, then hand it to as many Server and Client Components as you like from inside the same boundary.

Server Components can call `getCurrentUser()` directly. To reach Client Components without prop drilling, create the promise once, pass it through context, and unwrap it with [`use()`](https://react.dev/reference/react/use). For the general pattern, see [Using React's `use` within a Context Provider](/docs/app/guides/single-page-applications#using-reacts-use-within-a-context-provider). Because `getCurrentUser` reads the request, create its promise inside the Suspense boundary, not at the top of a layout.

```tsx
// Archivo: app/user-provider.tsx
'use client'

import { createContext, use } from 'react'
import type { ReactNode } from 'react'
import type { User } from '@/lib/auth'

const UserContext = createContext<Promise<User> | null>(null)

export function UserProvider({
  userPromise,
  children,
}: {
  userPromise: Promise<User>
  children: ReactNode
}) {
  return <UserContext value={userPromise}>{children}</UserContext>
}

export function useUser() {
  const userPromise = use(UserContext)
  if (!userPromise) {
    throw new Error('useUser must be used within a UserProvider')
  }
  return use(userPromise)
}
```
````

The Server Component behind the boundary creates the promise and hands it to the provider without awaiting it. Each consumer then resolves it behind its own boundary, so the shared chrome renders without waiting on the session:

````
```tsx
// Archivo: app/page.tsx
function Dashboard() {
  const userPromise = getCurrentUser()

  return (
    <UserProvider userPromise={userPromise}>
      ```<Suspense>```
    </UserProvider>
  )
}
```
````

Client Components call `useUser()` to get the current user. Because `use()` suspends until the promise resolves, keep the component behind a `<Suspense>` boundary:
````

```
// Archivo: app/user-badge.tsx
'use client'

import { useUser } from './user-provider'

export function UserBadge() {
  const user = useUser()
  return <span>Signed in as {user.name}</span>
}
```

Good to know: Expose only what the client needs. The `getCurrentUser` helper returns a narrow `{ id, name }` rather than the raw session. To keep sensitive fields from reaching the client, see `taintUniqueValue`.

Step 4: Cache session-derived data

Now that you have the user, you can cache the data you fetch for them in one of two ways. Passing the user id into a plain `use cache` function keeps the result on the server, keyed by the id (it becomes part of the cache key), where a `cacheTag` can invalidate it later. Reading it inside a `use cache: private` scope keeps it in the browser only, never on the server, which matters when requirements forbid storing certain data server-side, even ephemerally. The example passes the id, since these notes are tagged and refreshed when they change.

A plain `use cache` scope can't read `cookies()`, so the exported function resolves the user and passes just the id to the cached function:

```
// Archivo: lib/data.ts
import 'server-only'
import { cacheLife, cacheTag } from 'next/cache'
import { getCurrentUser } from './auth'

export async function getNotes() {
  const user = await getCurrentUser()
  return getNotesByUserId(user.id)
}

async function getNotesByUserId(userId: string) {
  'use cache'
  cacheTag(`notes:${userId}`)
  cacheLife('minutes')

  return db.query.notes.findMany({
    where: (notes, { eq }) => eq(notes.userId, userId),
  })
}
```

Keep `getNotesByUserId` unexported so a caller can't request another user's notes by passing a different id. Resolving the user inside the exported getter is what makes that safe. See Data Security.

Call it from a Server Component that renders the notes. There's no user id to pass, and the `getCurrentUser()` call inside the getter hits the private cache, so the session isn't read again:

```
// Archivo: app/page.tsx
async function Notes() {
  const notes = await getNotes()
  // ...
}
```

Good to know: cache keys and tags are stored in plain text. A cached function's arguments and captured variables are serialized into its cache key, and `cacheTag` values are stored as written. Neither is hashed: the default cache holds them as plain-text map keys and tag lists, and a remote cache receives them the same way. Key and tag on a stable identifier like the user id, and keep secrets and sensitive personal data (tokens, passwords, raw emails) out of arguments and tags.

On the server, plain `use cache` keeps the entry in memory as a best effort: it's evicted under pressure and, in serverless, doesn't persist across instances. If that data must survive across instances and requests, opt into `use cache: remote` for durable, shared storage, where the cache key you choose drives your hit rate.

Step 5: Update session-derived data

When a Server Action changes a user's data, call `updateTag` with the same tag to refresh the cached entry. Re-read the session inside the action so it authorizes itself rather than trusting the client. See authentication with Server Actions for why this matters.

```
// Archivo: app/actions.ts
'use server'
import { redirect } from 'next/navigation'
import { updateTag } from 'next/cache'
import { getSession } from '@/lib/session'
import { saveNote } from '@/lib/data'

export async function addNote(formData: FormData) {
  const { userId } = await getSession()
  if (!userId) {
    redirect('/login')
  }

  const note = String(formData.get('note') ?? '').trim()
  if (note) {
    await saveNote(userId, note)
    updateTag(`notes:${userId}`)
  }
}
```

Step 6: Make authenticated navigations instant

Cached reads already have a lifetime, so this mostly happens on its own. A `use cache: private` scope uses the `default` profile (a five-minute `stale`) unless you set one, and a route that reads the session produces a per-session App Shell with the authenticated content, prefetched and cached per session. Navigations to it are already instant.

Two things keep it that way:

- If you tune the lifetime with `cacheLife`, keep `stale` at 30 seconds or more. Below that, the scope drops out of prefetching. See `cacheLife` client cache behavior.
- A route that *also* depends on the URL (a `params` or `searchParams` value) needs `<Link prefetch={true}>` on the links pointing at it. That opts into per-link prefetching, which resolves the per-link data ahead of the click.

```
// Archivo: app/page.tsx
<Link href={`/notes/${note.id}`} prefetch={true}>
  {note.text}
</Link>
```

The destination needs Partial Prefetching for this, so enable the `partialPrefetching` flag or set `prefetch = 'partial'` on the segment. Add the prop where the wait is worth it: that prefetch costs one server invocation per link, so a sidebar of `/chat/[id]` links pays that cost per item.

Common pitfalls

- **Reading `cookies()` or `headers()` inside a plain `use cache` function.** This throws. Read the request outside and pass the value in, or use `use cache: private`.
- **Putting secrets or personal data in cache keys or tags.** Arguments and `cacheTag` values are stored in plain text. Key and tag on a stable identifier, not on sensitive input.
- **Trusting the client for authorization.** UI checks hide elements, but they don't protect data. Re-verify the session in every Server Action and Route Handler, close to the data. See Authorization.

How to implement authentication in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/authentication
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to implement authentication in your Next.js application.*

Understanding authentication is crucial for protecting your application's data. This page will guide you through what React and Next.js features to use to implement auth.

Before starting, it helps to break down the process into three concepts:

1. **Authentication**: Verifies if the user is who they say they are. It requires the user to prove their identity with something they have, such as a username and password.
2. **Session Management**: Tracks the user's auth state across requests.
3. **Authorization**: Decides what routes and data the user can access.

This diagram shows the authentication flow using React and Next.js features:

Diagram showing the authentication flow with React and Next.js features

The examples on this page walk through basic username and password auth for educational purposes. While you can implement a custom auth solution, for increased security and simplicity, we recommend using an authentication library. These offer built-in solutions for authentication, session management, and authorization, as well as additional features such as social logins, multi-factor authentication, and role-based access control. You can find a list in the Auth Libraries section.

> **Good to know:** With Cache Components enabled, reading the session and caching per-user data follow specific rules. See Authentication with Cache Components.

Authentication

Sign-up and login functionality

You can use the `<form>` element with React's Server Actions and `useActionState` to capture user credentials, validate form fields, and call your Authentication Provider's API or database.

Since Server Actions always execute on the server, they provide a secure environment for handling authentication logic.

Here are the steps to implement signup/login functionality:

1. Capture user credentials

To capture user credentials, create a form that invokes a Server Action on submission. For example, a signup form that accepts the user's name, email, and password:

```tsx
// Archivo: app/ui/signup-form.tsx switcher
import { signup } from '@/app/actions/auth'

export function SignupForm() {
  return (
    <form action={signup}>
      <div>
        <label htmlFor="name">Name</label>
        <input id="name" name="name" placeholder="Name" />
      </div>
      <div>
        <label htmlFor="email">Email</label>
        <input id="email" name="email" type="email" placeholder="Email" />
      </div>
      <div>
        <label htmlFor="password">Password</label>
        <input id="password" name="password" type="password" />
      </div>
      <button type="submit">Sign Up</button>
    </form>
  )
}
```

```jsx
// Archivo: app/ui/signup-form.js switcher
import { signup } from '@/app/actions/auth'

export function SignupForm() {
  return (
    <form action={signup}>
      <div>
        <label htmlFor="name">Name</label>
        <input id="name" name="name" placeholder="Name" />
      </div>
      <div>
        <label htmlFor="email">Email</label>
        <input id="email" name="email" type="email" placeholder="Email" />
      </div>
      <div>
        <label htmlFor="password">Password</label>
        <input id="password" name="password" type="password" />
      </div>
      <button type="submit">Sign Up</button>
    </form>
  )
}
```

```tsx
// Archivo: app/actions/auth.ts switcher
export async function signup(formData: FormData) {}
```

```jsx
// Archivo: app/actions/auth.js switcher
export async function signup(formData) {}
```

2. Validate form fields on the server

Use the Server Action to validate the form fields on the server. If your authentication provider doesn't provide form validation, you can use a schema validation library like Zod, Valibot or Yup.

Using Zod as an example, you can define a form schema with appropriate error messages:

```
// Archivo: app/lib/definitions.ts switcher
import * as z from 'zod'

export const SignupFormSchema = z.object({
  name: z
    .string()
    .min(2, { error: 'Name must be at least 2 characters long.' })
    .trim(),
  email: z.email({ error: 'Please enter a valid email.' }).trim(),
  password: z
    .string()
    .min(8, { error: 'Be at least 8 characters long' })
    .regex(/[a-zA-Z]/, { error: 'Contain at least one letter.' })
    .regex(/[0-9]/, { error: 'Contain at least one number.' })
    .regex(/[^a-zA-Z0-9]/, {
      error: 'Contain at least one special character.',
    })
    .trim(),
})

export type FormState =
  | {
      errors?: {
        name?: string[]
        email?: string[]
        password?: string[]
      }
      message?: string
    }
  | undefined
```

```
// Archivo: app/lib/definitions.js switcher
import * as z from 'zod'

export const SignupFormSchema = z.object({
  name: z
    .string()
    .min(2, { error: 'Name must be at least 2 characters long.' })
    .trim(),
  email: z.email({ error: 'Please enter a valid email.' }).trim(),
  password: z
    .string()
    .min(8, { error: 'Be at least 8 characters long' })
    .regex(/[a-zA-Z]/, { error: 'Contain at least one letter.' })
    .regex(/[0-9]/, { error: 'Contain at least one number.' })
    .regex(/[^a-zA-Z0-9]/, {
      error: 'Contain at least one special character.',
    })
    .trim(),
})
```

To prevent unnecessary calls to your authentication provider's API or database, you can `return` early in the Server Action if any form fields do not match the defined schema.

```
// Archivo: app/actions/auth.ts switcher
import { SignupFormSchema, FormState } from '@/app/lib/definitions'

export async function signup(state: FormState, formData: FormData) {
  // Validate form fields
  const validatedFields = SignupFormSchema.safeParse({
    name: formData.get('name'),
    email: formData.get('email'),
    password: formData.get('password'),
  })

  // If any form fields are invalid, return early
  if (!validatedFields.success) {
    return {
      errors: validatedFields.error.flatten().fieldErrors,
    }
  }

  // Call the provider or db to create a user ...
}
```

```
// Archivo: app/actions/auth.js switcher
import { SignupFormSchema } from '@/app/lib/definitions'

export async function signup(state, formData) {
  // Validate form fields
  const validatedFields = SignupFormSchema.safeParse({
    name: formData.get('name'),
    email: formData.get('email'),
    password: formData.get('password'),
  })

  // If any form fields are invalid, return early
  if (!validatedFields.success) {
    return {
      errors: validatedFields.error.flatten().fieldErrors,
    }
  }

  // Call the provider or db to create a user ...
}
```

Back in your `<SignupForm />`, you can use React's `useActionState` hook to display validation errors while the form is submitting:

```tsx
// Archivo: app/ui/signup-form.tsx switcher highlight={7,15,21,27-36}
'use client'

import { signup } from '@/app/actions/auth'
import { useActionState } from 'react'

export default function SignupForm() {
  const [state, action, pending] = useActionState(signup, undefined)

  return (
    <form action={action}>
      <div>
        <label htmlFor="name">Name</label>
        <input id="name" name="name" placeholder="Name" />
      </div>
      {state?.errors?.name && <p>{state.errors.name}</p>}

      <div>
        <label htmlFor="email">Email</label>
        <input id="email" name="email" placeholder="Email" />
      </div>
      {state?.errors?.email && <p>{state.errors.email}</p>}

      <div>
        <label htmlFor="password">Password</label>
        <input id="password" name="password" type="password" />
      </div>
      {state?.errors?.password && (
        <div>
          <p>Password must:</p>
          <ul>
            {state.errors.password.map((error) => (
              <li key={error}>- {error}</li>
            ))}
          </ul>
        </div>
      )}
      <button disabled={pending} type="submit">
        Sign Up
      </button>
    </form>
  )
}
```

```
// Archivo: app/ui/signup-form.js switcher highlight={7,15,21,27-36}
'use client'

import { signup } from '@/app/actions/auth'
import { useActionState } from 'react'

export default function SignupForm() {
  const [state, action, pending] = useActionState(signup, undefined)

  return (
    <form action={action}>
      <div>
        <label htmlFor="name">Name</label>
        <input id="name" name="name" placeholder="Name" />
      </div>
      {state?.errors?.name && <p>{state.errors.name}</p>}

      <div>
        <label htmlFor="email">Email</label>
        <input id="email" name="email" placeholder="Email" />
      </div>
      {state?.errors?.email && <p>{state.errors.email}</p>}

      <div>
        <label htmlFor="password">Password</label>
        <input id="password" name="password" type="password" />
      </div>
      {state?.errors?.password && (
        <div>
          <p>Password must:</p>
          <ul>
            {state.errors.password.map((error) => (
              <li key={error}>- {error}</li>
            ))}
          </ul>
        </div>
      )}
      <button disabled={pending} type="submit">
        Sign Up
      </button>
    </form>
  )
}
```

Good to know:

- In React 19, `useFormStatus` includes additional keys on the returned object, like data, method, and action. If you are not using React 19, only the `pending` key is available.
- Before mutating data, you should always ensure a user is also authorized to perform the action. See Authentication and Authorization.

3. Create a user or check user credentials

After validating form fields, you can create a new user account or check if the user exists by calling your authentication provider's API or database.

Continuing from the previous example:

```tsx
// Archivo: app/actions/auth.tsx switcher
export async function signup(state: FormState, formData: FormData) {
  // 1. Validate form fields
  // ...

  // 2. Prepare data for insertion into database
  const { name, email, password } = validatedFields.data
  // e.g. Hash the user's password before storing it
  const hashedPassword = await bcrypt.hash(password, 10)

  // 3. Insert the user into the database or call an Auth Library's API
  const data = await db
    .insert(users)
    .values({
      name,
      email,
      password: hashedPassword,
    })
    .returning({ id: users.id })

  const user = data[0]

  if (!user) {
    return {
      message: 'An error occurred while creating your account.',
    }
  }

  // TODO:
  // 4. Create user session
  // 5. Redirect user
}
```

```jsx
// Archivo: app/actions/auth.js switcher
export async function signup(state, formData) {
  // 1. Validate form fields
  // ...

  // 2. Prepare data for insertion into database
  const { name, email, password } = validatedFields.data
  // e.g. Hash the user's password before storing it
  const hashedPassword = await bcrypt.hash(password, 10)

  // 3. Insert the user into the database or call an Library API
  const data = await db
    .insert(users)
    .values({
      name,
      email,
      password: hashedPassword,
    })
    .returning({ id: users.id })

  const user = data[0]

  if (!user) {
    return {
      message: 'An error occurred while creating your account.',
    }
  }

  // TODO:
  // 4. Create user session
  // 5. Redirect user
}
```

After successfully creating the user account or verifying the user credentials, you can create a session to manage the user's auth state. Depending on your session management strategy, the session can be stored in a cookie or database, or both. Continue to the Session Management section to learn more.

Tips:

- The example above is verbose since it breaks down the authentication steps for the purpose of education. This highlights that implementing your own secure solution can quickly become complex. Consider using an Auth Library to simplify the process.
- To improve the user experience, you may want to check for duplicate emails or usernames earlier in the registration flow. For example, as the user types in a username or the input field loses focus. This can help prevent unnecessary form submissions and provide immediate feedback to the user. You can debounce requests with libraries such as use-debounce to manage the frequency of these checks.

📘 **Nota para Pages Router:** Here are the steps to implement a sign-up and/or login form:

1. The user submits their credentials through a form.
2. The form sends a request that is handled by an API route.
3. Upon successful verification, the process is completed, indicating the user's successful authentication.
4. If verification is unsuccessful, an error message is shown.

Consider a login form where users can input their credentials:

```
import { FormEvent } from 'react'
import { useRouter } from 'next/router'

export default function LoginPage() {
  const router = useRouter()

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const email = formData.get('email')
    const password = formData.get('password')

    const response = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    })

    if (response.ok) {
      router.push('/profile')
    } else {
      // Handle errors
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" name="email" placeholder="Email" required />
      <input type="password" name="password" placeholder="Password" required />
      <button type="submit">Login</button>
    </form>
  )
}
```

```
import { FormEvent } from 'react'
import { useRouter } from 'next/router'

export default function LoginPage() {
  const router = useRouter()

  async function handleSubmit(event) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const email = formData.get('email')
    const password = formData.get('password')

    const response = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    })

    if (response.ok) {
      router.push('/profile')
    } else {
      // Handle errors
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" name="email" placeholder="Email" required />
      <input type="password" name="password" placeholder="Password" required />
      <button type="submit">Login</button>
    </form>
  )
}
```

The form above has two input fields for capturing the user's email and password. On submission, it triggers a function that sends a POST request to an API route (`/api/auth/login`).

You can then call your Authentication Provider's API in the API route to handle authentication:

```
import type { NextApiRequest, NextApiResponse } from 'next'
import { signIn } from '@/auth'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const { email, password } = req.body
    await signIn('credentials', { email, password })

    res.status(200).json({ success: true })
  } catch (error) {
    if (error.type === 'CredentialsSignin') {
      res.status(401).json({ error: 'Invalid credentials.' })
    } else {
      res.status(500).json({ error: 'Something went wrong.' })
    }
  }
}
```

```
import { signIn } from '@/auth'

export default async function handler(req, res) {
  try {
    const { email, password } = req.body
    await signIn('credentials', { email, password })

    res.status(200).json({ success: true })
  } catch (error) {
    if (error.type === 'CredentialsSignin') {
      res.status(401).json({ error: 'Invalid credentials.' })
    } else {
      res.status(500).json({ error: 'Something went wrong.' })
    }
  }
}
```

Session Management

Session management ensures that the user's authenticated state is preserved across requests. It involves creating, storing, refreshing, and deleting sessions or tokens.

There are two types of sessions:

1. **Stateless**: Session data (or a token) is stored in the browser's cookies. The cookie is sent with each request, allowing the session to be verified on the server. This method is simpler, but can be less secure if not implemented correctly.
2. **Database**: Session data is stored in a database, with the user's browser only receiving the encrypted session ID. This method is more secure, but can be complex and use more server resources.

> **Good to know:** While you can use either method, or both, we recommend using a session management library such as iron-session or Jose.

Stateless Sessions

To create and manage stateless sessions, there are a few steps you need to follow:

1. Generate a secret key, which will be used to sign your session, and store it as an environment variable.
2. Write logic to encrypt/decrypt session data using a session management library.
3. Manage cookies using the Next.js `cookies` API.

In addition to the above, consider adding functionality to update (or refresh) the session when the user returns to the application, and delete the session when the user logs out.

> **Good to know:** Check if your auth library includes session management.

1. Generating a secret key

There are a few ways you can generate secret key to sign your session. For example, you may choose to use the `openssl` command in your terminal:

```
// Archivo: terminal
openssl rand -base64 32
```

This command generates a 32-character random string that you can use as your secret key and store in your environment variables file:

```
// Archivo: .env
SESSION_SECRET=your_secret_key
```

You can then reference this key in your session management logic:

```
// Archivo: app/lib/session.js
const secretKey = process.env.SESSION_SECRET
```

2. Encrypting and decrypting sessions

Next, you can use your preferred session management library to encrypt and decrypt sessions. Continuing from the previous example, we'll use Jose and React's `server-only` package to ensure that your session management logic is only executed on the server.

```
// Archivo: app/lib/session.ts switcher
import 'server-only'
import { SignJWT, jwtVerify } from 'jose'
import { SessionPayload } from '@/app/lib/definitions'

const secretKey = process.env.SESSION_SECRET
const encodedKey = new TextEncoder().encode(secretKey)

export async function encrypt(payload: SessionPayload) {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(encodedKey)
}

export async function decrypt(session: string | undefined = '') {
  try {
    const { payload } = await jwtVerify(session, encodedKey, {
      algorithms: ['HS256'],
    })
    return payload
  } catch (error) {
    console.log('Failed to verify session')
  }
}
```

```
// Archivo: app/lib/session.js switcher
import 'server-only'
import { SignJWT, jwtVerify } from 'jose'

const secretKey = process.env.SESSION_SECRET
const encodedKey = new TextEncoder().encode(secretKey)

export async function encrypt(payload) {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(encodedKey)
}

export async function decrypt(session) {
  try {
    const { payload } = await jwtVerify(session, encodedKey, {
      algorithms: ['HS256'],
    })
    return payload
  } catch (error) {
    console.log('Failed to verify session')
  }
}
```

> **Tips**:
>
> - The payload should contain the **minimum**, unique user data that'll be used in subsequent requests, such as the user's ID, role, etc. It should not contain personally identifiable information like phone number, email address, credit card information, etc, or sensitive data like passwords.

3. Setting cookies (recommended options)

To store the session in a cookie, use the Next.js `cookies` API. The cookie should be set on the server, and include the recommended options:

- **HttpOnly**: Prevents client-side JavaScript from accessing the cookie.
- **Secure**: Use https to send the cookie.
- **SameSite**: Specify whether the cookie can be sent with cross-site requests.
- **Max-Age or Expires**: Delete the cookie after a certain period.
- **Path**: Define the URL path for the cookie.

Please refer to MDN for more information on each of these options.

```
// Archivo: app/lib/session.ts switcher
import 'server-only'
import { cookies } from 'next/headers'

export async function createSession(userId: string) {
  const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
  const session = await encrypt({ userId, expiresAt })
  const cookieStore = await cookies()

  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expiresAt,
    sameSite: 'lax',
    path: '/',
  })
}
```

```
// Archivo: app/lib/session.js switcher
import 'server-only'
import { cookies } from 'next/headers'

export async function createSession(userId) {
  const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
  const session = await encrypt({ userId, expiresAt })
  const cookieStore = await cookies()

  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expiresAt,
    sameSite: 'lax',
    path: '/',
  })
}
```

Back in your Server Action, you can invoke the `createSession()` function, and use the `redirect()` API to redirect the user to the appropriate page:

```
// Archivo: app/actions/auth.ts switcher
import { createSession } from '@/app/lib/session'

export async function signup(state: FormState, formData: FormData) {
  // Previous steps:
  // 1. Validate form fields
  // 2. Prepare data for insertion into database
  // 3. Insert the user into the database or call an Library API

  // Current steps:
  // 4. Create user session
  await createSession(user.id)
  // 5. Redirect user
  redirect('/profile')
}
```

```
// Archivo: app/actions/auth.js switcher
import { createSession } from '@/app/lib/session'

export async function signup(state, formData) {
  // Previous steps:
  // 1. Validate form fields
  // 2. Prepare data for insertion into database
  // 3. Insert the user into the database or call an Library API

  // Current steps:
  // 4. Create user session
  await createSession(user.id)
  // 5. Redirect user
  redirect('/profile')
}
```

> **Tips**:
>
> - **Cookies should be set on the server** to prevent client-side tampering.
> - 🎥 Watch: Learn more about stateless sessions and authentication with Next.js → YouTube (11 minutes).

Updating (or refreshing) sessions

You can also extend the session's expiration time. This is useful for keeping the user logged in after they access the application again. For example:

```
// Archivo: app/lib/session.ts switcher
import 'server-only'
import { cookies } from 'next/headers'
import { decrypt } from '@/app/lib/session'

export async function updateSession() {
  const session = (await cookies()).get('session')?.value
  const payload = await decrypt(session)

  if (!session || !payload) {
    return null
  }

  const expires = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)

  const cookieStore = await cookies()
  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expires,
    sameSite: 'lax',
    path: '/',
  })
}
```

```
// Archivo: app/lib/session.js switcher
import 'server-only'
import { cookies } from 'next/headers'
import { decrypt } from '@/app/lib/session'

export async function updateSession() {
  const cookieStore = await cookies()
  const session = cookieStore.get('session')?.value
  const payload = await decrypt(session)

  if (!session || !payload) {
    return null
  }

  const expires = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expires,
    sameSite: 'lax',
    path: '/',
  })
}
```

> **Tip:** Check if your auth library supports refresh tokens, which can be used to extend the user's session.

Deleting the session

To delete the session, you can delete the cookie:

```
// Archivo: app/lib/session.ts switcher
import 'server-only'
import { cookies } from 'next/headers'

export async function deleteSession() {
  const cookieStore = await cookies()
  cookieStore.delete('session')
}
```

```
// Archivo: app/lib/session.js switcher
import 'server-only'
import { cookies } from 'next/headers'

export async function deleteSession() {
  const cookieStore = await cookies()
  cookieStore.delete('session')
}
```

Then you can reuse the `deleteSession()` function in your application, for example, on logout:

```
// Archivo: app/actions/auth.ts switcher
import { cookies } from 'next/headers'
import { deleteSession } from '@/app/lib/session'

export async function logout() {
  await deleteSession()
  redirect('/login')
}
```

```
// Archivo: app/actions/auth.js switcher
import { cookies } from 'next/headers'
import { deleteSession } from '@/app/lib/session'

export async function logout() {
  await deleteSession()
  redirect('/login')
}
```

📘 **Nota para Pages Router:**

Setting and deleting cookies

You can use API Routes to set the session as a cookie on the server:

```
import { serialize } from 'cookie'
import type { NextApiRequest, NextApiResponse } from 'next'
import { encrypt } from '@/app/lib/session'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const sessionData = req.body
  const encryptedSessionData = encrypt(sessionData)

  const cookie = serialize('session', encryptedSessionData, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 60 * 60 * 24 * 7, // One week
    path: '/',
  })
  res.setHeader('Set-Cookie', cookie)
  res.status(200).json({ message: 'Successfully set cookie!' })
}
```

```
import { serialize } from 'cookie'
import { encrypt } from '@/app/lib/session'

export default function handler(req, res) {
  const sessionData = req.body
  const encryptedSessionData = encrypt(sessionData)

  const cookie = serialize('session', encryptedSessionData, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 60 * 60 * 24 * 7, // One week
    path: '/',
  })
  res.setHeader('Set-Cookie', cookie)
  res.status(200).json({ message: 'Successfully set cookie!' })
}
```

Database Sessions

To create and manage database sessions, you'll need to follow these steps:

1. Create a table in your database to store session and data (or check if your Auth Library handles this).
2. Implement functionality to insert, update, and delete sessions
3. Encrypt the session ID before storing it in the user's browser, and ensure the database and cookie stay in sync (this is optional, but recommended for optimistic auth checks in Proxy).

For example:

```ts
// Archivo: app/lib/session.ts switcher
import { cookies } from 'next/headers'
import { db } from '@/app/lib/db'
import { encrypt } from '@/app/lib/session'

export async function createSession(id: number) {
  const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)

  // 1. Create a session in the database
  const data = await db
    .insert(sessions)
    .values({
      userId: id,
      expiresAt,
    })
    // Return the session ID
    .returning({ id: sessions.id })

  const sessionId = data[0].id

  // 2. Encrypt the session ID
  const session = await encrypt({ sessionId, expiresAt })

  // 3. Store the session in cookies for optimistic auth checks
  const cookieStore = await cookies()
  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expiresAt,
    sameSite: 'lax',
    path: '/',
  })
}
```

```js
// Archivo: app/lib/session.js switcher
import { cookies } from 'next/headers'
import { db } from '@/app/lib/db'
import { encrypt } from '@/app/lib/session'

export async function createSession(id) {
  const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)

  // 1. Create a session in the database
  const data = await db
    .insert(sessions)
    .values({
      userId: id,
      expiresAt,
    })
    // Return the session ID
    .returning({ id: sessions.id })

  const sessionId = data[0].id

  // 2. Encrypt the session ID
  const session = await encrypt({ sessionId, expiresAt })

  // 3. Store the session in cookies for optimistic auth checks
  const cookieStore = await cookies()
  cookieStore.set('session', session, {
    httpOnly: true,
    secure: true,
    expires: expiresAt,
    sameSite: 'lax',
    path: '/',
  })
}
```

Tips:

- For faster access, you may consider adding server caching for the lifetime of the session. You can also keep the session data in your primary database, and combine data requests to reduce the number of queries.
- You may opt to use database sessions for more advanced use cases, such as keeping track of the last time a user logged in, or number of active devices, or give users the ability to log out of all devices.

After implementing session management, you'll need to add authorization logic to control what users can access and do within your application. Continue to the Authorization section to learn more.

📘 **Nota para Pages Router: Creating a Session on the Server**:

```
import db from '../../lib/db'
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const user = req.body
    const sessionId = generateSessionId()
    await db.insertSession({
      sessionId,
      userId: user.id,
      createdAt: new Date(),
    })

    res.status(200).json({ sessionId })
  } catch (error) {
    res.status(500).json({ error: 'Internal Server Error' })
  }
}
```

```
import db from '../../lib/db'

export default async function handler(req, res) {
  try {
    const user = req.body
    const sessionId = generateSessionId()
    await db.insertSession({
      sessionId,
      userId: user.id,
      createdAt: new Date(),
    })

    res.status(200).json({ sessionId })
  } catch (error) {
    res.status(500).json({ error: 'Internal Server Error' })
  }
}
```

Authorization

Once a user is authenticated and a session is created, you can implement authorization to control what the user can access and do within your application.

There are two main types of authorization checks:

1. **Optimistic**: Checks if the user is authorized to access a route or perform an action using the session data stored in the cookie. These checks are useful for quick operations, such as showing/hiding UI elements or redirecting users based on permissions or roles.
2. **Secure**: Checks if the user is authorized to access a route or perform an action using the session data stored in the database. These checks are more secure and are used for operations that require access to sensitive data or actions.

For both cases, we recommend:

- Creating a Data Access Layer to centralize your authorization logic
- Using Data Transfer Objects (DTO) to only return the necessary data
- Optionally use Proxy to perform optimistic checks.

Optimistic checks with Proxy (Optional)

There are some cases where you may want to use Proxy and redirect users based on permissions:

- To perform optimistic checks. Since Proxy runs on every route, it's a good way to centralize redirect logic and pre-filter unauthorized users.
- To protect static routes that share data between users (e.g. content behind a paywall).

However, since Proxy runs on every route, including prefetched routes, it's important to only read the session from the cookie (optimistic checks), and avoid database checks to prevent performance issues.

For example:

```
// Archivo: proxy.ts switcher
import { NextRequest, NextResponse } from 'next/server'
import { decrypt } from '@/app/lib/session'
import { cookies } from 'next/headers'

// 1. Specify protected and public routes
const protectedRoutes = ['/dashboard']
const publicRoutes = ['/login', '/signup', '/']

export default async function proxy(req: NextRequest) {
  // 2. Check if the current route is protected or public
  const path = req.nextUrl.pathname
  const isProtectedRoute = protectedRoutes.includes(path)
  const isPublicRoute = publicRoutes.includes(path)

  // 3. Decrypt the session from the cookie
  const cookie = (await cookies()).get('session')?.value
  const session = await decrypt(cookie)

  // 4. Redirect to /login if the user is not authenticated
  if (isProtectedRoute && !session?.userId) {
    return NextResponse.redirect(new URL('/login', req.nextUrl))
  }

  // 5. Redirect to /dashboard if the user is authenticated
  if (
    isPublicRoute &&
    session?.userId &&
    !req.nextUrl.pathname.startsWith('/dashboard')
  ) {
    return NextResponse.redirect(new URL('/dashboard', req.nextUrl))
  }

  return NextResponse.next()
}

// Routes Proxy should not run on
export const config = {
  matcher: ['/((?!api|_next/static|_next/image|.*\\.png$).*)'],
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'
import { decrypt } from '@/app/lib/session'
import { cookies } from 'next/headers'

// 1. Specify protected and public routes
const protectedRoutes = ['/dashboard']
const publicRoutes = ['/login', '/signup', '/']

export default async function proxy(req) {
  // 2. Check if the current route is protected or public
  const path = req.nextUrl.pathname
  const isProtectedRoute = protectedRoutes.includes(path)
  const isPublicRoute = publicRoutes.includes(path)

  // 3. Decrypt the session from the cookie
  const cookie = (await cookies()).get('session')?.value
  const session = await decrypt(cookie)

  // 5. Redirect to /login if the user is not authenticated
  if (isProtectedRoute && !session?.userId) {
    return NextResponse.redirect(new URL('/login', req.nextUrl))
  }

  // 6. Redirect to /dashboard if the user is authenticated
  if (
    isPublicRoute &&
    session?.userId &&
    !req.nextUrl.pathname.startsWith('/dashboard')
  ) {
    return NextResponse.redirect(new URL('/dashboard', req.nextUrl))
  }

  return NextResponse.next()
}

// Routes Proxy should not run on
export const config = {
  matcher: ['/((?!api|_next/static|_next/image|.*\\.png$).*)'],
}
```

While Proxy can be useful for initial checks, it should not be your only line of defense in protecting your data. The majority of security checks should be performed as close as possible to your data source, see Data Access Layer for more information.

> **Tips**:
>
> - In Proxy, you can also read cookies using `req.cookies.get('session')?.value`.
> - Proxy uses the Node.js runtime, check if your Auth library and session management library are compatible.
> - You can use the `matcher` property in the Proxy to specify which routes Proxy should run on. Although, for auth, it's recommended Proxy runs on all routes.

Creating a Data Access Layer (DAL)

We recommend creating a DAL to centralize your data requests and authorization logic.

The DAL should include a function that verifies the user's session as they interact with your application. At the very least, the function should check if the session is valid, then redirect or return the user information needed to make further requests.

For example, create a separate file for your DAL that includes a `verifySession()` function. Then use React's cache API to memoize the return value of the function during a React render pass:

```
// Archivo: app/lib/dal.ts switcher
import 'server-only'

import { cookies } from 'next/headers'
import { decrypt } from '@/app/lib/session'

export const verifySession = cache(async () => {
  const cookie = (await cookies()).get('session')?.value
  const session = await decrypt(cookie)

  if (!session?.userId) {
    redirect('/login')
  }

  return { isAuth: true, userId: session.userId }
})
```

```
// Archivo: app/lib/dal.js switcher
import 'server-only'

import { cookies } from 'next/headers'
import { decrypt } from '@/app/lib/session'

export const verifySession = cache(async () => {
  const cookie = (await cookies()).get('session')?.value
  const session = await decrypt(cookie)

  if (!session.userId) {
    redirect('/login')
  }

  return { isAuth: true, userId: session.userId }
})
```

You can then invoke the `verifySession()` function in your data requests, Server Actions, Route Handlers:

```
// Archivo: app/lib/dal.ts switcher
export const getUser = cache(async () => {
  const session = await verifySession()
  if (!session) return null

  try {
    const data = await db.query.users.findMany({
      where: eq(users.id, session.userId),
      // Explicitly return the columns you need rather than the whole user object
      columns: {
        id: true,
        name: true,
        email: true,
      },
    })

    const user = data[0]

    return user
  } catch (error) {
    console.log('Failed to fetch user')
    return null
  }
})
```

```
// Archivo: app/lib/dal.js switcher
export const getUser = cache(async () => {
  const session = await verifySession()
  if (!session) return null

  try {
    const data = await db.query.users.findMany({
      where: eq(users.id, session.userId),
      // Explicitly return the columns you need rather than the whole user object
      columns: {
        id: true,
        name: true,
        email: true,
      },
    })

    const user = data[0]

    return user
  } catch (error) {
    console.log('Failed to fetch user')
    return null
  }
})
```

Tip:

- A DAL can be used to protect data fetched at request time. However, for static routes that share data between users, data will be fetched at build time and not at request time. Use Proxy to protect static routes.
- For secure checks, you can check if the session is valid by comparing the session ID with your database. Use React's cache function to avoid unnecessary duplicate requests to the database during a render pass.
- You may wish to consolidate related data requests in a JavaScript class that runs `verifySession()` before any methods.

Using Data Transfer Objects (DTO)

When retrieving data, it's recommended you return only the necessary data that will be used in your application, and not entire objects. For example, if you're fetching user data, you might only return the user's ID and name, rather than the entire user object which could contain passwords, phone numbers, etc.

However, if you have no control over the returned data structure, or are working in a team where you want to avoid whole objects being passed to the client, you can use strategies such as specifying what fields are safe to be exposed to the client.

```ts
// Archivo: app/lib/dto.ts switcher
import 'server-only'
import { getUser } from '@/app/lib/dal'

function canSeeUsername(viewer: User) {
  return true
}

function canSeePhoneNumber(viewer: User, team: string) {
  return viewer.isAdmin || team === viewer.team
}

export async function getProfileDTO(slug: string) {
  const data = await db.query.users.findMany({
    where: eq(users.slug, slug),
    // Return specific columns here
  })
  const user = data[0]

  const currentUser = await getUser(user.id)

  // Or return only what's specific to the query here
  return {
    username: canSeeUsername(currentUser) ? user.username : null,
    phonenumber: canSeePhoneNumber(currentUser, user.team)
      ? user.phonenumber
      : null,
  }
}
```

```js
// Archivo: app/lib/dto.js switcher
import 'server-only'
import { getUser } from '@/app/lib/dal'

function canSeeUsername(viewer) {
  return true
}

function canSeePhoneNumber(viewer, team) {
  return viewer.isAdmin || team === viewer.team
}

export async function getProfileDTO(slug) {
  const data = await db.query.users.findMany({
    where: eq(users.slug, slug),
    // Return specific columns here
  })
  const user = data[0]

  const currentUser = await getUser(user.id)

  // Or return only what's specific to the query here
  return {
    username: canSeeUsername(currentUser) ? user.username : null,
    phonenumber: canSeePhoneNumber(currentUser, user.team)
      ? user.phonenumber
      : null,
  }
}
```

By centralizing your data requests and authorization logic in a DAL and using DTOs, you can ensure that all data requests are secure and consistent, making it easier to maintain, audit, and debug as your application scales.

Good to know:

- There are a couple of different ways you can define a DTO, from using `toJSON()`, to individual functions like the example above, or JS classes. Since these are JavaScript patterns and not a React or Next.js feature, we recommend doing some research to find the best pattern for your application.
- Learn more about security best practices in our Security in Next.js article.

Server Components

Auth check in Server Components are useful for role-based access. For example, to conditionally render components based on the user's role:

```
// Archivo: app/dashboard/page.tsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function Dashboard() {
  const session = await verifySession()
  const userRole = session?.user?.role // Assuming 'role' is part of the session object

  if (userRole === 'admin') {
    return <AdminDashboard />
  } else if (userRole === 'user') {
    return <UserDashboard />
  } else {
    redirect('/login')
  }
}
```

```
// Archivo: app/dashboard/page.jsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function Dashboard() {
  const session = await verifySession()
  const userRole = session?.user?.role // Assuming 'role' is part of the session object

  if (userRole === 'admin') {
    return <AdminDashboard />
  } else if (userRole === 'user') {
    return <UserDashboard />
  } else {
    redirect('/login')
  }
}
```

In the example, we use the `verifySession()` function from our DAL to check for 'admin', 'user', and unauthorized roles. This pattern ensures that each user interacts only with components appropriate to their role.

Layouts and auth checks

Due to Partial Rendering, be cautious when doing checks in Layouts as these don't re-render on navigation, meaning the user session won't be checked on every route change.

A layout also does not control whether the rest of the route renders. Route segments and parallel route slots are rendered by the router, so a layout that hides or swaps them does not stop them from running or from appearing in the RSC Payload.

Instead, you should do the checks close to your data source or the component that'll be conditionally rendered.

For example, consider a shared layout that fetches the user data and displays the user image in a nav. Instead of doing the auth check in the layout, you should fetch the user data (`getUser()`) in the layout and do the auth check in your DAL.

This guarantees that wherever `getUser()` is called within your application, the auth check is performed, and prevents developers from forgetting to check that the user is authorized to access the data.

Auth and streaming

Session and user data often appear in shell UI (header, nav) that repeats across routes. A top-level `await` on `cookies()`, `headers()`, or the DAL in a layout delays the first streamed chunk for that segment and holds `{children}` behind that work.

If only part of the shell needs session data (for example, a user menu), move the `await` into a nested Server Component and wrap it in `<Suspense>` so the rest of the page streams first. See Push dynamic access down for the pattern.

Client Components can't import the DAL. Run `verifySession()`, `getUser()`, or similar in a parent Server Component, then pass data to client children as props or through a context provider. For that pattern, see Using React's `use` within a Context Provider, and use React's `taintUniqueValue` API to keep sensitive session fields from reaching the client.

Auth checks in page components

For example, in a dashboard page, you can verify the user session and fetch the user data:

```
// Archivo: app/dashboard/page.tsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function DashboardPage() {
  const session = await verifySession()

  // Fetch user-specific data from your database or data source
  const user = await getUserData(session.userId)

  return (
    <div>
      <h1>Welcome, {user.name}</h1>

    </div>
  )
}
```

```
// Archivo: app/dashboard/page.jsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function DashboardPage() {
  const session = await verifySession()

  // Fetch user-specific data from your database or data source
  const user = await getUserData(session.userId)

  return (
    <div>
      <h1>Welcome, {user.name}</h1>

    </div>
  )
}
```

Auth checks in leaf components

You can also perform auth checks in leaf components that conditionally render UI elements based on user permissions. For example, a component that displays admin-only actions:

```tsx
// Archivo: app/ui/admin-actions.tsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function AdminActions() {
  const session = await verifySession()
  const userRole = session?.user?.role

  if (userRole !== 'admin') {
    return null
  }

  return (
    <div>
      <button>Delete User</button>
      <button>Edit Settings</button>
    </div>
  )
}
```

```jsx
// Archivo: app/ui/admin-actions.jsx switcher
import { verifySession } from '@/app/lib/dal'

export default async function AdminActions() {
  const session = await verifySession()
  const userRole = session?.user?.role

  if (userRole !== 'admin') {
    return null
  }

  return (
    <div>
      <button>Delete User</button>
      <button>Edit Settings</button>
    </div>
  )
}
```

This pattern allows you to show or hide UI elements based on user permissions while ensuring the auth check happens at render time in each component.

> **Good to know:**
>
> - A common pattern in SPAs is to `return null` in a layout or a top-level component if a user is not authorized. This pattern is **not recommended** since Next.js applications have multiple entry points, which will not prevent nested route segments and Server Actions from being accessed.
> - Ensure that any Server Actions called from these components also perform their own authorization checks, as client-side UI restrictions alone are not sufficient for security.

Server Actions

Treat Server Actions with the same security considerations as public-facing API endpoints, and verify if the user is allowed to perform a mutation.

In the example below, we check the user's role before allowing the action to proceed:

```
// Archivo: app/lib/actions.ts switcher
'use server'
import { verifySession } from '@/app/lib/dal'

export async function serverAction(formData: FormData) {
  const session = await verifySession()
  const userRole = session?.user?.role

  // Return early if user is not authorized to perform the action
  if (userRole !== 'admin') {
    return null
  }

  // Proceed with the action for authorized users
}
```

```
// Archivo: app/lib/actions.js switcher
'use server'
import { verifySession } from '@/app/lib/dal'

export async function serverAction() {
  const session = await verifySession()
  const userRole = session.user.role

  // Return early if user is not authorized to perform the action
  if (userRole !== 'admin') {
    return null
  }

  // Proceed with the action for authorized users
}
```

Route Handlers

Treat Route Handlers with the same security considerations as public-facing API endpoints, and verify if the user is allowed to access the Route Handler.

For example:

```
// Archivo: app/api/route.ts switcher
import { verifySession } from '@/app/lib/dal'

export async function GET() {
  // User authentication and role verification
  const session = await verifySession()

  // Check if the user is authenticated
  if (!session) {
    // User is not authenticated
    return new Response(null, { status: 401 })
  }

  // Check if the user has the 'admin' role
  if (session.user.role !== 'admin') {
    // User is authenticated but does not have the right permissions
    return new Response(null, { status: 403 })
  }

  // Continue for authorized users
}
```

```
// Archivo: app/api/route.js switcher
import { verifySession } from '@/app/lib/dal'

export async function GET() {
  // User authentication and role verification
  const session = await verifySession()

  // Check if the user is authenticated
  if (!session) {
    // User is not authenticated
    return new Response(null, { status: 401 })
  }

  // Check if the user has the 'admin' role
  if (session.user.role !== 'admin') {
    // User is authenticated but does not have the right permissions
    return new Response(null, { status: 403 })
  }

  // Continue for authorized users
}
```

The example above demonstrates a Route Handler with a two-tier security check. It first checks for an active session, and then verifies if the logged-in user is an 'admin'.

📘 **Nota para Pages Router:**

Creating a Data Access Layer (DAL)

Protecting API Routes

API Routes in Next.js are essential for handling server-side logic and data management. It's crucial to secure these routes to ensure that only authorized users can access specific functionalities. This typically involves verifying the user's authentication status and their role-based permissions.

Here's an example of securing an API Route:

```
import { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const session = await getSession(req)

  // Check if the user is authenticated
  if (!session) {
    res.status(401).json({
      error: 'User is not authenticated',
    })
    return
  }

  // Check if the user has the 'admin' role
  if (session.user.role !== 'admin') {
    res.status(401).json({
      error: 'Unauthorized access: User does not have admin privileges.',
    })
    return
  }

  // Proceed with the route for authorized users
  // ... implementation of the API Route
}
```

```
export default async function handler(req, res) {
  const session = await getSession(req)

  // Check if the user is authenticated
  if (!session) {
    res.status(401).json({
      error: 'User is not authenticated',
    })
    return
  }

  // Check if the user has the 'admin' role
  if (session.user.role !== 'admin') {
    res.status(401).json({
      error: 'Unauthorized access: User does not have admin privileges.',
    })
    return
  }

  // Proceed with the route for authorized users
  // ... implementation of the API Route
}
```

This example demonstrates an API Route with a two-tier security check for authentication and authorization. It first checks for an active session, and then verifies if the logged-in user is an 'admin'. This approach ensures secure access, limited to authenticated and authorized users, maintaining robust security for request processing.

Resources

Now that you've learned about authentication in Next.js, here are Next.js-compatible libraries and resources to help you implement secure authentication and session management:

Auth Libraries

- Auth0
- Better Auth
- Clerk
- Descope
- Kinde
- Logto
- NextAuth.js
- Ory
- Stack Auth
- Supabase
- Stytch
- WorkOS

Session Management Libraries

- Iron Session
- Jose

Further Reading

To continue learning about authentication and security, check out the following resources:

- How to think about security in Next.js
- Understanding XSS Attacks
- Understanding CSRF Attacks
- The Copenhagen Book

How to set a Content Security Policy (CSP) for your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/content-security-policy
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to set a Content Security Policy (CSP) for your Next.js application.*

Content Security Policy (CSP) is important to guard your Next.js application against various security threats such as cross-site scripting (XSS), clickjacking, and other code injection attacks.

By using CSP, developers can specify which origins are permissible for content sources, scripts, stylesheets, images, fonts, objects, media (audio, video), iframes, and more.

Examples

Nonces

A nonce is a unique, random string of characters created for a one-time use. It is used in conjunction with CSP to selectively allow certain inline scripts or styles to execute, bypassing strict CSP directives.

Why use a nonce?

CSP can block both inline and external scripts to prevent attacks. A nonce lets you safely allow specific scripts to run—only if they include the matching nonce value.

If an attacker wanted to load a script into your page, they'd need to guess the nonce value. That's why the nonce must be unpredictable and unique for every request.

Adding a nonce with Proxy

Proxy enables you to add headers and generate nonces before the page renders.

Every time a page is viewed, a fresh nonce should be generated. This means that you **must use dynamic rendering to add nonces**.

For example:

> **Good to know**: In development, `'unsafe-eval'` is required because React uses `eval` to provide enhanced debugging information, such as reconstructing server-side error stacks in the browser. `unsafe-eval` is not required for production. Neither React nor Next.js use `eval` in production by default.

```
// Archivo: proxy.ts switcher
import { NextRequest, NextResponse } from 'next/server'

export function proxy(request: NextRequest) {
  const nonce = Buffer.from(crypto.randomUUID()).toString('base64')
  const isDev = process.env.NODE_ENV === 'development'
  const cspHeader = `
    default-src 'self';
    script-src 'self' 'nonce-${nonce}' 'strict-dynamic'${isDev ? " 'unsafe-eval'" : ''};
    style-src 'self' 'nonce-${nonce}';
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`
  // Replace newline characters and spaces
  const contentSecurityPolicyHeaderValue = cspHeader
    .replace(/\s{2,}/g, ' ')
    .trim()

  const requestHeaders = new Headers(request.headers)
  requestHeaders.set('x-nonce', nonce)

  requestHeaders.set(
    'Content-Security-Policy',
    contentSecurityPolicyHeaderValue
  )

  const response = NextResponse.next({
    request: {
      headers: requestHeaders,
    },
  })
  response.headers.set(
    'Content-Security-Policy',
    contentSecurityPolicyHeaderValue
  )

  return response
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  const nonce = Buffer.from(crypto.randomUUID()).toString('base64')
  const isDev = process.env.NODE_ENV === 'development'
  const cspHeader = `
    default-src 'self';
    script-src 'self' 'nonce-${nonce}' 'strict-dynamic'${isDev ? " 'unsafe-eval'" : ''};
    style-src 'self' 'nonce-${nonce}';
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`
  // Replace newline characters and spaces
  const contentSecurityPolicyHeaderValue = cspHeader
    .replace(/\s{2,}/g, ' ')
    .trim()

  const requestHeaders = new Headers(request.headers)
  requestHeaders.set('x-nonce', nonce)
  requestHeaders.set(
    'Content-Security-Policy',
    contentSecurityPolicyHeaderValue
  )

  const response = NextResponse.next({
    request: {
      headers: requestHeaders,
    },
  })
  response.headers.set(
    'Content-Security-Policy',
    contentSecurityPolicyHeaderValue
  )

  return response
}
```

By default, Proxy runs on all requests. You can filter Proxy to run on specific paths using a `matcher`.

We recommend ignoring matching prefetches (from `next/link`) and static assets that don't need the CSP header.

```
// Archivo: proxy.ts switcher
export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - api (API routes)
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     */
    {
      source: '/((?!api|_next/static|_next/image|favicon.ico).*)',
      missing: [
        { type: 'header', key: 'next-router-prefetch' },
        { type: 'header', key: 'purpose', value: 'prefetch' },
      ],
    },
  ],
}
```

```
// Archivo: proxy.js switcher
export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - api (API routes)
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     */
    {
      source: '/((?!api|_next/static|_next/image|favicon.ico).*)',
      missing: [
        { type: 'header', key: 'next-router-prefetch' },
        { type: 'header', key: 'purpose', value: 'prefetch' },
      ],
    },
  ],
}
```

How nonces work in Next.js

To use a nonce, your page must be **dynamically rendered**. This is because Next.js applies nonces during **server-side rendering**, based on the CSP header present in the request. Static pages are generated at build time, when no request or response headers exist—so no nonce can be injected.

Here's how nonce support works in a dynamically rendered page:

1. **Proxy generates a nonce**: Your proxy creates a unique nonce for the request, adds it to your `Content-Security-Policy` header, and also sets it in a custom `x-nonce` header.
2. **Next.js extracts the nonce**: During rendering, Next.js parses the `Content-Security-Policy` header and extracts the nonce using the `'nonce-{value}'` pattern.
3. **Nonce is applied automatically**: Next.js attaches the nonce to:
   - Framework scripts (React, Next.js runtime)
   - Page-specific JavaScript bundles
   - Inline styles and scripts generated by Next.js
   - Any `<Script>` components using the `nonce` prop

Because of this automatic behavior, you don't need to manually add a nonce to each tag.

Forcing dynamic rendering

If you're using nonces, you may need to explicitly opt pages into dynamic rendering:

```
// Archivo: app/page.tsx switcher
import { connection } from 'next/server'

export default async function Page() {
  // wait for an incoming request to render this page
  await connection()
  // Your page content
}
```

```
// Archivo: app/page.jsx switcher
import { connection } from 'next/server'

export default async function Page() {
  // wait for an incoming request to render this page
  await connection()
  // Your page content
}
```

Reading the nonce

📘 **Nota para Pages Router:** You can provide the nonce to your page using `getServerSideProps`:

```
import Script from 'next/script'

import type { GetServerSideProps } from 'next'

export default function Page({ nonce }) {
  return (
    <Script
      src="https://www.googletagmanager.com/gtag/js"
      strategy="afterInteractive"
      nonce={nonce}
    />
  )
}

export const getServerSideProps: GetServerSideProps = async ({ req }) => {
  const nonce = req.headers['x-nonce']
  return { props: { nonce } }
}
```

```
import Script from 'next/script'
export default function Page({ nonce }) {
  return (
    <Script
      src="https://www.googletagmanager.com/gtag/js"
      strategy="afterInteractive"
      nonce={nonce}
    />
  )
}

export async function getServerSideProps({ req }) {
  const nonce = req.headers['x-nonce']
  return { props: { nonce } }
}
```

You can also access the nonce in `_document.tsx` for Pages Router applications:

```tsx
import Document, {
  Html,
  Head,
  Main,
  NextScript,
  DocumentContext,
  DocumentInitialProps,
} from 'next/document'

interface ExtendedDocumentProps extends DocumentInitialProps {
  nonce?: string
}

class MyDocument extends Document<ExtendedDocumentProps> {
  static async getInitialProps(
    ctx: DocumentContext
  ): Promise<ExtendedDocumentProps> {
    const initialProps = await Document.getInitialProps(ctx)
    const nonce = ctx.req?.headers?.['x-nonce'] as string | undefined

    return {
      ...initialProps,
      nonce,
    }
  }

  render() {
    const { nonce } = this.props

    return (
      <Html lang="en">
        <Head nonce={nonce} />
        <body>
          <Main />
          <NextScript nonce={nonce} />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

```
import Document, { Html, Head, Main, NextScript } from 'next/document'

class MyDocument extends Document {
  static async getInitialProps(ctx) {
    const initialProps = await Document.getInitialProps(ctx)
    const nonce = ctx.req?.headers?.['x-nonce']

    return {
      ...initialProps,
      nonce,
    }
  }

  render() {
    const { nonce } = this.props

    return (
      <Html lang="en">
        <Head nonce={nonce} />
        <body>
          <Main />
          <NextScript nonce={nonce} />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

You can read the nonce from a Server Component using `headers`:

```
// Archivo: app/page.tsx switcher
import { headers } from 'next/headers'
import Script from 'next/script'

export default async function Page() {
  const nonce = (await headers()).get('x-nonce')

  return (
    <Script
      src="https://www.googletagmanager.com/gtag/js"
      strategy="afterInteractive"
      nonce={nonce}
    />
  )
}
```

```
// Archivo: app/page.jsx switcher
import { headers } from 'next/headers'
import Script from 'next/script'

export default async function Page() {
  const nonce = (await headers()).get('x-nonce')

  return (
    <Script
      src="https://www.googletagmanager.com/gtag/js"
      strategy="afterInteractive"
      nonce={nonce}
    />
  )
}
```

Static vs Dynamic Rendering with CSP

Using nonces has important implications for how your Next.js application renders:

Dynamic Rendering Requirement

When you use nonces in your CSP, **all pages must be dynamically rendered**. This means:

- Pages will build successfully but may encounter runtime errors if not properly configured for dynamic rendering
- Each request generates a fresh page with a new nonce
- Static optimization and Incremental Static Regeneration (ISR) are disabled
- Pages cannot be cached by CDNs without additional configuration
- **Partial Prerendering (PPR) is incompatible** with nonce-based CSP since static shell scripts won't have access to the nonce

Performance Implications

The shift from static to dynamic rendering affects performance:

- **Slower initial page loads**: Pages must be generated on each request
- **Increased server load**: Every request requires server-side rendering
- **No CDN caching**: Dynamic pages cannot be cached at the edge by default
- **Higher hosting costs**: More server resources needed for dynamic rendering

When to use nonces

Consider nonces when:

- You have strict security requirements that prohibit `'unsafe-inline'`
- Your application handles sensitive data
- You need to allow specific inline scripts while blocking others
- Compliance requirements mandate strict CSP

Without Nonces

For applications that do not require nonces, you can set the CSP header directly in your `next.config.js` file:

```
// Archivo: next.config.js
const isDev = process.env.NODE_ENV === 'development'

const cspHeader = `
    default-src 'self';
    script-src 'self' 'unsafe-inline'${isDev ? " 'unsafe-eval'" : ''};
    style-src 'self' 'unsafe-inline';
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`

module.exports = {
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          {
            key: 'Content-Security-Policy',
            value: cspHeader.replace(/\n/g, ''),
          },
        ],
      },
    ]
  },
}
```

Subresource Integrity (Experimental)

As an alternative to nonces, Next.js offers experimental support for hash-based CSP using Subresource Integrity (SRI). This approach allows you to maintain static generation while still having a strict CSP.

> **Good to know**: This feature is experimental and available in App Router applications.

How SRI works

Instead of using nonces, SRI generates cryptographic hashes of your JavaScript files at build time. These hashes are added as `integrity` attributes to script tags, allowing browsers to verify that files haven't been modified during transit.

Enabling SRI

Add the experimental SRI configuration to your `next.config.js`:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    sri: {
      algorithm: 'sha256', // or 'sha384' or 'sha512'
    },
  },
}

module.exports = nextConfig
```

CSP configuration with SRI

When SRI is enabled, you can continue using your existing CSP policies. SRI works independently by adding `integrity` attributes to your assets:

> **Good to know**: For dynamic rendering scenarios, you can still generate nonces with proxy if needed, combining both SRI integrity attributes and nonce-based CSP approaches.

```
// Archivo: next.config.js
const isDev = process.env.NODE_ENV === 'development'

const cspHeader = `
    default-src 'self';
    script-src 'self'${isDev ? " 'unsafe-eval'" : ''};
    style-src 'self';
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`

module.exports = {
  experimental: {
    sri: {
      algorithm: 'sha256',
    },
  },
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          {
            key: 'Content-Security-Policy',
            value: cspHeader.replace(/\n/g, ''),
          },
        ],
      },
    ]
  },
}
```

Benefits of SRI over nonces

- **Static generation**: Pages can be statically generated and cached
- **CDN compatibility**: Static pages work with CDN caching
- **Better performance**: No server-side rendering required for each request
- **Build-time security**: Hashes are generated at build time, ensuring integrity

Limitations of SRI

- **Experimental**: Feature may change or be removed
- **App Router only**: Not supported in Pages Router
- **Build-time only**: Cannot handle dynamically generated scripts

Development vs Production Considerations

CSP implementation differs between development and production environments:

Development Environment

In development, you will need to enable `'unsafe-eval'` because React uses `eval` to provide enhanced debugging information, such as reconstructing server-side error stacks in the browser to show you where errors originated on the server:

```
// Archivo: proxy.ts switcher
export function proxy(request: NextRequest) {
  const nonce = Buffer.from(crypto.randomUUID()).toString('base64')
  const isDev = process.env.NODE_ENV === 'development'

  const cspHeader = `
    default-src 'self';
    script-src 'self' 'nonce-${nonce}' 'strict-dynamic' ${isDev ? "'unsafe-eval'" : ''};
    style-src 'self' ${isDev ? "'unsafe-inline'" : `'nonce-${nonce}'`};
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`

  // Rest of proxy implementation
}
```

```
// Archivo: proxy.js switcher
export function proxy(request) {
  const nonce = Buffer.from(crypto.randomUUID()).toString('base64')
  const isDev = process.env.NODE_ENV === 'development'

  const cspHeader = `
    default-src 'self';
    script-src 'self' 'nonce-${nonce}' 'strict-dynamic' ${isDev ? "'unsafe-eval'" : ''};
    style-src 'self' ${isDev ? "'unsafe-inline'" : `'nonce-${nonce}'`};
    img-src 'self' blob: data:;
    font-src 'self';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    frame-ancestors 'none';
    upgrade-insecure-requests;
`

  // Rest of proxy implementation
}
```

Production Deployment

Common issues in production:

- **Nonce not applied**: Ensure your proxy runs on all necessary routes
- **Static assets blocked**: Verify your CSP allows Next.js static assets
- **Third-party scripts**: Add necessary domains to your CSP policy

Troubleshooting

Third-party Scripts

When using third-party scripts with CSP:

```tsx
// Archivo: app/layout.tsx switcher
import { GoogleTagManager } from '@next/third-parties/google'
import { headers } from 'next/headers'

export default async function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  const nonce = (await headers()).get('x-nonce')

  return (
    <html lang="en">
      <body>
        {children}
        <GoogleTagManager gtmId="GTM-XYZ" nonce={nonce} />
      </body>
    </html>
  )
}
```

```jsx
// Archivo: app/layout.jsx switcher
import { GoogleTagManager } from '@next/third-parties/google'
import { headers } from 'next/headers'

export default async function RootLayout({ children }) {
  const nonce = (await headers()).get('x-nonce')

  return (
    <html lang="en">
      <body>
        {children}
        <GoogleTagManager gtmId="GTM-XYZ" nonce={nonce} />
      </body>
    </html>
  )
}
```

📘 **Nota para Pages Router:** When using third-party scripts with CSP, ensure you add the necessary domains and pass the nonce:

```
import type { AppProps } from 'next/app'
import Script from 'next/script'

export default function App({ Component, pageProps }: AppProps) {
  const nonce = pageProps.nonce

  return (
    <>
      <Component {...pageProps} />
      <Script
        src="https://www.googletagmanager.com/gtag/js"
        strategy="afterInteractive"
        nonce={nonce}
      />
    </>
  )
}
```

```
import Script from 'next/script'

export default function App({ Component, pageProps }) {
  const nonce = pageProps.nonce

  return (
    <>
      <Component {...pageProps} />
      <Script
        src="https://www.googletagmanager.com/gtag/js"
        strategy="afterInteractive"
        nonce={nonce}
      />
    </>
  )
}
```

Update your CSP to allow third-party domains:

```
// Archivo: proxy.ts switcher
const cspHeader = `
  default-src 'self';
  script-src 'self' 'nonce-${nonce}' 'strict-dynamic' https://www.googletagmanager.com;
  connect-src 'self' https://www.google-analytics.com;
  img-src 'self' data: https://www.google-analytics.com;
`
```

```
// Archivo: proxy.js switcher
const cspHeader = `
  default-src 'self';
  script-src 'self' 'nonce-${nonce}' 'strict-dynamic' https://www.googletagmanager.com;
  connect-src 'self' https://www.google-analytics.com;
  img-src 'self' data: https://www.google-analytics.com;
`
```

Common CSP Violations

1. **Inline styles**: Use CSS-in-JS libraries that support nonces or move styles to external files
2. **Dynamic imports**: Ensure dynamic imports are allowed in your script-src policy
3. **WebAssembly**: Add `'wasm-unsafe-eval'` if using WebAssembly
4. **Service workers**: Add appropriate policies for service worker scripts

Version History

Version	Changes
`v14.0.0`	Experimental SRI support added for hash-based CSP
`v13.4.20`	Recommended for proper nonce handling and CSP header parsing.

How to think about data security in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/data-security
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn the built-in data security features in Next.js and learn best practices for protecting your application's data.*

React Server Components improve performance and simplify data fetching, but also shift where and how data is accessed, changing some of the traditional security assumptions for handling data in frontend apps.

This guide will help you understand how to think about data security in Next.js and how to implement best practices.

Data fetching approaches

There are three main approaches we recommend for fetching data in Next.js, depending on the size and age of your project:

- HTTP APIs: for existing large applications and organizations.
- Data Access Layer: for new projects.
- Component-Level Data Access: for prototypes and learning.

We recommend choosing one data fetching approach and avoiding mixing them. This makes it clear for both developers working in your code base and security auditors what to expect.

External HTTP APIs

You should follow a **Zero Trust** model when adopting Server Components in an existing project. You can continue calling your existing API endpoints such as REST or GraphQL from Server Components using `fetch`, just as you would in Client Components.

```
// Archivo: app/page.tsx
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const token = cookieStore.get('AUTH_TOKEN')?.value

  const res = await fetch('https://api.example.com/profile', {
    headers: {
      Cookie: `AUTH_TOKEN=${token}`,
      // Other headers
    },
  })

  // ....
}
```

This approach works well when:

- You already have security practices in place.
- Separate backend teams use other languages or manage APIs independently.

Data Access Layer

For new projects, we recommend creating a dedicated **Data Access Layer (DAL)**. This is an internal library that controls how and when data is fetched, and what gets passed to your render context.

A Data Access Layer should:

- Only run on the server.
- Perform authorization checks.
- Return safe, minimal **Data Transfer Objects (DTOs)**.

This approach centralizes all data access logic, making it easier to enforce consistent data access and reduces the risk of authorization bugs. You also get the benefit of sharing an in-memory cache across different parts of a request.

```
// Archivo: data/auth.ts
import { cache } from 'react'
import { cookies } from 'next/headers'

// Cached helper methods makes it easy to get the same value in many places
// without manually passing it around. This discourages passing it from Server
// Component to Server Component which minimizes risk of passing it to a Client
// Component.
export const getCurrentUser = cache(async () => {
  const cookieStore = await cookies()
  const token = cookieStore.get('AUTH_TOKEN')
  const decodedToken = await decryptAndValidate(token)
  // Don't include secret tokens or private information as public fields.
  // Use classes to avoid accidentally passing the whole object to the client.
  return new User(decodedToken.id)
})
```

```
// Archivo: data/user-dto.tsx
import 'server-only'
import { getCurrentUser } from './auth'

function canSeeUsername(viewer: User) {
  // Public info for now, but can change
  return true
}

function canSeePhoneNumber(viewer: User, team: string) {
  // Privacy rules
  return viewer.isAdmin || team === viewer.team
}

export async function getProfileDTO(slug: string) {
  // Don't pass values, read back cached values, also solves context and easier to make it lazy

  // use a database API that supports safe templating of queries
  const [rows] = await sql`SELECT * FROM user WHERE slug = ${slug}`
  const userData = rows[0]

  const currentUser = await getCurrentUser()

  // only return the data relevant for this query and not everything
  // <https://www.w3.org/2001/tag/doc/APIMinimization>
  return {
    username: canSeeUsername(currentUser) ? userData.username : null,
    phonenumber: canSeePhoneNumber(currentUser, userData.team)
      ? userData.phonenumber
      : null,
  }
}
```

```
// Archivo: app/page.tsx
import { getProfileDTO } from '../../data/user-dto'

export default async function Page({ params }) {
  const { slug } = await params
  // This page can now safely pass around this profile knowing
  // that it shouldn't contain anything sensitive.
  const profile = await getProfileDTO(slug)
  ...
}
```

> **Good to know:** Secret keys should be stored in environment variables, but only the Data Access Layer should access `process.env`. This keeps secrets from being exposed to other parts of the application.

Component-level data access

For quick prototypes and iteration, database queries can be placed directly in Server Components.

This approach, however, makes it easier to accidentally expose private data to the client, for example:

```
// Archivo: app/page.tsx
import Profile from './components/profile.tsx'

export default async function Page({ params }) {
  const { slug } = await params
  const [rows] = await sql`SELECT * FROM user WHERE slug = ${slug}`
  const userData = rows[0]
  // EXPOSED: This exposes all the fields in userData to the client because
  // we are passing the data from the Server Component to the Client.
  return <Profile user={userData} />
}
```

```
// Archivo: app/ui/profile.tsx
'use client'

// BAD: This is a bad props interface because it accepts way more data than the
// Client Component needs and it encourages server components to pass all that
// data down. A better solution would be to accept a limited object with just
// the fields necessary for rendering the profile.
export default async function Profile({ user }: { user: User }) {
  return (
    <div>
      <h1>{user.name}</h1>
      ...
    </div>
  )
}
```

You should sanitize the data before passing it to the Client Component:

```
// Archivo: data/user.ts
import { sql } from './db'

export async function getUser(slug: string) {
  const [rows] = await sql`SELECT * FROM user WHERE slug = ${slug}`
  const user = rows[0]

  // Return only the public fields
  return {
    name: user.name,
  }
}
```

```
// Archivo: app/page.tsx
import { getUser } from '../data/user'
import Profile from './ui/profile'

export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const publicProfile = await getUser(slug)
  return <Profile user={publicProfile} />
}
```

Reading data

Passing data from server to client

On the initial load, both Server and Client Components run on the server to generate HTML. However, they execute in isolated module systems. This ensures that Server Components can access private data and APIs, while Client Components cannot.

Server Components:

- Run only on the server.
- Can safely access environment variables, secrets, databases, and internal APIs.

Client Components:

- Run on the server during prerendering, but must follow the same security assumptions as code running in the browser.
- Must not access privileged data or server-only modules.

This ensures the app is secure by default, but it's possible to accidentally expose private data through how data is fetched or passed to components.

Tainting

To prevent accidental exposure of private data to the client, you can use React Taint APIs:

- `experimental_taintObjectReference` for data objects.
- `experimental_taintUniqueValue` for specific values.

You can enable usage in your Next.js app with the `experimental.taint` option in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    taint: true,
  },
}
```

This prevents the tainted objects or values from being passed to the client. However, it's an additional layer of protection, you should still filter and sanitize the data in your DAL before passing it to React's render context.

> **Good to know:**
>
> - By default, environment variables are only available on the Server. Next.js exposes any environment variable prefixed with `NEXT_PUBLIC_` to the client. Learn more.
> - Functions and classes are already blocked from being passed to Client Components by default.

Preventing client-side execution of server-only code

To prevent server-only code from being executed on the client, you can mark a module with the `server-only` package:

```
// Archivo: lib/data.ts
import 'server-only'

// ...
```

This ensures that proprietary code or internal business logic stays on the server by causing a build error if the module is imported in the client environment.

Next.js handles `server-only` imports internally. The contents of these packages from NPM are not used. However, if your linting rules flag extraneous dependencies, you may install them to avoid issues.

```
# Manejador de paquetes: npm
npm install server-only
```

```
# Manejador de paquetes: yarn
yarn add server-only
```

```
# Manejador de paquetes: pnpm
pnpm add server-only
```

```
# Manejador de paquetes: bun
bun add server-only
```

Read more about `server-only` in the preventing environment poisoning section.

Mutating Data

Next.js handles mutations with Server Actions.

Built-in Server Actions Security features

By default, when a Server Action is created and exported, it is reachable via a direct POST request, not just through your application's UI. This means, even if a Server Action or utility function is not imported elsewhere in your code, it can still be called externally.

To improve security, Next.js has the following built-in features:

- **Secure action IDs:** Next.js creates encrypted, non-deterministic IDs to allow the client to reference and call the Server Action. These IDs are periodically recalculated between builds for enhanced security.
- **Dead code elimination:** Unused Server Actions (referenced by their IDs) are removed from client bundle to avoid public access.

> **Good to know**:
>
> The IDs are created during compilation and are cached for a maximum of 14 days. They will be regenerated when a new build is initiated or when the build cache is invalidated. This security improvement reduces the risk in cases where an authentication layer is missing. However, you should still treat Server Actions as reachable via direct POST requests and verify authentication and authorization inside each one.

```
// app/actions.js
'use server'

// If this action **is** used in our application, Next.js
// will create a secure ID to allow the client to reference
// and call the Server Action.
export async function updateUserAction(formData) {}

// If this action **is not** used in our application, Next.js
// will automatically remove this code during `next build`
// and will not create a public endpoint.
export async function deleteUserAction(formData) {}
```

Validating client input

You should always validate input from client, as they can be easily modified. For example, form data, URL parameters, headers, and searchParams:

```
// Archivo: app/page.tsx
// BAD: Trusting searchParams directly
export default async function Page({ searchParams }) {
  const isAdmin = (await searchParams).isAdmin
  if (isAdmin === 'true') {
    // Vulnerable: relies on untrusted client data
    return <AdminPanel />
  }
}

// GOOD: Re-verify every time
import { cookies } from 'next/headers'
import { verifyAdmin } from './auth'

export default async function Page() {
  const cookieStore = await cookies()
  const token = cookieStore.get('AUTH_TOKEN')
  const isAdmin = await verifyAdmin(token)

  if (isAdmin) {
    return <AdminPanel />
  }
}
```

Authentication and authorization

A page-level authentication check does not extend to the Server Actions defined within it. Always re-verify inside the action:

```
// Archivo: app/admin/page.tsx highlight={13,14,15,16}
import { auth } from '@/lib/auth'
import { redirect } from 'next/navigation'

export default async function AdminPage() {
  const session = await auth()
  if (!session?.user?.isAdmin) {
    redirect('/login')
  }

  return (
    <form
      action={async () => {
        'use server'
        const session = await auth()
        if (!session?.user?.isAdmin) {
          throw new Error('Unauthorized')
        }
        await db.record.deleteMany()
      }}
    >
      <button>Delete Records</button>
    </form>
  )
}
```

The highlighted `auth()` check inside the action is critical. The page-level redirect on line 6 controls which UI is rendered, but the Server Action is a separate entry point and must verify the caller on its own.

Beyond authentication (is the user logged in?), remember to check **authorization** (does this user have permission to act on this specific resource?). This prevents Insecure Direct Object Reference (IDOR) vulnerabilities:

```
// Archivo: app/actions.ts
'use server'

import { auth } from '@/lib/auth'
import { db } from '@/lib/db'

export async function deletePost(postId: string) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const post = await db.post.findUnique({ where: { id: postId } })

  // Check that the user owns this resource
  if (post.authorId !== session.user.id) {
    throw new Error('Forbidden')
  }

  await db.post.delete({ where: { id: postId } })
}
```

Learn more about Authentication in Next.js.

Using a Data Access Layer for mutations

Just as we recommend a Data Access Layer for reading data, you can apply the same pattern to mutations. This keeps authentication, authorization, and database logic in a dedicated `server-only` module, while `"use server"` actions stay thin.

```
// Archivo: data/posts.ts
import 'server-only'

import { auth } from '@/lib/auth'
import { db } from '@/lib/db'

export async function deletePost(postId: string) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }

  const post = await db.post.findUnique({ where: { id: postId } })

  if (post.authorId !== session.user.id) {
    throw new Error('Forbidden')
  }

  await db.post.delete({ where: { id: postId } })
}
```

The `"use server"` action then delegates to the DAL:

```
// Archivo: app/actions.ts
'use server'

import { deletePost } from '@/data/posts'
import { revalidatePath } from 'next/cache'

export async function deletePostAction(postId: string) {
  await deletePost(postId) // Auth + authz happen inside the DAL
  revalidatePath('/posts')
}
```

> **Good to know:** You can use `import 'server-only'` in both the Data Access Layer and the `"use server"` file itself. Both work when the action is imported into a Client Component (for example, to pass it to `useActionState`), because `"use server"` modules are resolved in a server-only webpack layer.

Controlling return values

Server Action return values are serialized and sent to the client. Only return what the UI needs, not raw database records.

```
// Archivo: app/actions.ts
'use server'

import { auth } from '@/lib/auth'
import { db } from '@/lib/db'

// BAD: Returns the full database record, which may include
// internal fields the client should not see.
export async function updateUser(data: FormData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  return db.user.update({
    where: { id: session.user.id },
    data: { name: data.get('name') as string },
  })
}

// GOOD: Returns only what the client needs.
export async function updateUserSafe(data: FormData) {
  const session = await auth()
  if (!session?.user) {
    throw new Error('Unauthorized')
  }
  await db.user.update({
    where: { id: session.user.id },
    data: { name: data.get('name') as string },
  })
  return { success: true }
}
```

Rate limiting

For expensive operations (sending emails, writing to a database), consider adding rate limiting to prevent abuse. See the Rate limiting example in the Backend for Frontend guide.

Closures and encryption

Defining a Server Action inside a component creates a closure where the action has access to the outer function's scope. For example, the `publish` action has access to the `publishVersion` variable:

```
// Archivo: app/page.tsx switcher
export default async function Page() {
  const publishVersion = await getLatestVersion();

  async function publish() {
    "use server";
    if (publishVersion !== await getLatestVersion()) {
      throw new Error('The version has changed since pressing publish');
    }
    ...
  }

  return (
    <form>
      <button formAction={publish}>Publish</button>
    </form>
  );
}
```

```
// Archivo: app/page.js switcher
export default async function Page() {
  const publishVersion = await getLatestVersion();

  async function publish() {
    "use server";
    if (publishVersion !== await getLatestVersion()) {
      throw new Error('The version has changed since pressing publish');
    }
    ...
  }

  return (
    <form>
      <button formAction={publish}>Publish</button>
    </form>
  );
}
```

Closures are useful when you need to capture a *snapshot* of data (e.g. `publishVersion`) at the time of rendering so that it can be used later when the action is invoked.

However, for this to happen, the captured variables are sent to the client and back to the server when the action is invoked. To prevent sensitive data from being exposed to the client, Next.js automatically encrypts the closed-over variables. A new private key is generated for each action every time a Next.js application is built. This means actions can only be invoked for a specific build.

> **Good to know:** We don't recommend relying on encryption alone to prevent sensitive values from being exposed on the client.

Overwriting encryption keys (advanced)

When **self-hosting** your Next.js application across multiple servers, each server instance may end up with a different encryption key, leading to potential inconsistencies.

To mitigate this, you can overwrite the encryption key using the `process.env.NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` environment variable. Specifying this variable ensures that your encryption keys are persistent across builds, and all server instances use the same key.

The key must be a base64-encoded value whose decoded length matches a valid AES key size (16, 24, or 32 bytes). Next.js generates 32-byte keys by default. You can generate a compatible key using your platform's cryptographic tools, for example:

```
openssl rand -base64 32
```

This is an advanced use case where consistent encryption behavior across multiple deployments is critical for your application. Follow standard security practices such as key rotation and signing. See the Self-Hosting guide for deployment-specific considerations.

Allowed origins (advanced)

Since Server Actions can be invoked in a `<form>` element, this opens them up to CSRF attacks.

Behind the scenes, Server Actions use the `POST` method, and only this HTTP method is allowed to invoke them. This prevents most CSRF vulnerabilities in modern browsers, particularly with SameSite cookies being the default.

As an additional protection, Server Actions in Next.js also compare the Origin header to the Host header (or `X-Forwarded-Host`). If these don't match, the request will be aborted. In other words, Server Actions can only be invoked on the same host as the page that hosts it.

For large applications that use reverse proxies or multi-layered backend architectures (where the server API differs from the production domain), it's recommended to use the configuration option `serverActions.allowedOrigins` option to specify a list of safe origins. The option accepts an array of strings.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
module.exports = {
  experimental: {
    serverActions: {
      allowedOrigins: ['my-proxy.com', '*.my-proxy.com'],
    },
  },
}
```

Learn more about Security and Server Actions.

Avoiding side-effects during rendering

Mutations (e.g. logging out users, updating databases, invalidating caches) should never be a side-effect, either in Server or Client Components. Next.js explicitly prevents setting cookies or triggering cache revalidation within render methods to avoid unintended side effects.

```
// Archivo: app/page.tsx
// BAD: Triggering a mutation during rendering
export default async function Page({ searchParams }) {
  if ((await searchParams).logout) {
    const cookieStore = await cookies()
    cookieStore.delete('AUTH_TOKEN')
  }

  return <UserProfile />
}
```

Instead, you should use Server Actions to handle mutations.

```
// Archivo: app/page.tsx
// GOOD: Using Server Actions to handle mutations
import { logout } from './actions'

export default function Page() {
  return (
    <>
      <UserProfile />
      <form action={logout}>
        <button type="submit">Logout</button>
      </form>
    </>
  )
}
```

> **Good to know:** Next.js uses `POST` requests to handle mutations. This prevents accidental side-effects from GET requests, reducing Cross-Site Request Forgery (CSRF) risks.

Auditing

If you're doing an audit of a Next.js project, here are a few things we recommend looking extra at:

- **Data Access Layer:** Is there an established practice for an isolated Data Access Layer? Verify that database packages and environment variables are not imported outside the Data Access Layer.
- `"use client"` **files:** Are the Component props expecting private data? Are the type signatures overly broad?
- `"use server"` **files:** Are the Action arguments validated in the action or inside the Data Access Layer? Is the user re-authorized inside the action? Does the action check ownership of the resource (authorization, not just authentication)? Are return values filtered to only what the client needs? Is database access delegated to a `server-only` Data Access Layer?
- `/[param]/.` Folders with brackets are user input. Are params validated?

- `proxy.ts` **and** `route.ts` **:** Have a lot of power. Spend extra time auditing these using traditional techniques. Perform Penetration Testing or Vulnerability Scanning regularly or in alignment with your team's software development lifecycle.

How to use environment variables in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/environment-variables
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn to add and access environment variables in your Next.js application.*

Next.js comes with built-in support for environment variables, which allows you to do the following:

- Use `.env` to load environment variables
- Bundle environment variables for the browser by prefixing with `NEXT_PUBLIC_`

> **Warning:** The default `create-next-app` template ensures all `.env` files are added to your `.gitignore`. You almost never want to commit these files to your repository.

Loading Environment Variables

Next.js has built-in support for loading environment variables from `.env*` files into `process.env`.

```
// Archivo: .env
DB_HOST=localhost
DB_USER=myuser
DB_PASS=mypassword
```

> 📘 **Nota para Pages Router:** This loads `process.env.DB_HOST`, `process.env.DB_USER`, and `process.env.DB_PASS` into the Node.js environment automatically allowing you to use them in Next.js data fetching methods and API routes.
>
> For example, using `getStaticProps`:
>
> ```
> export async function getStaticProps() {
>   const db = await myDB.connect({
>     host: process.env.DB_HOST,
>     username: process.env.DB_USER,
>     password: process.env.DB_PASS,
>   })
>   // ...
> }
> ```

> **Note**: Next.js also supports multiline variables inside of your `.env*` files:
>
> ```
> # .env
>
> # you can write with line breaks
> PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----
> ...
> Kh9NV...
> ...
> -----END DSA PRIVATE KEY-----"
>
> # or with `\n` inside double quotes
> PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----\nKh9NV...\n-----END DSA PRIVATE KEY-----\n"
> ```

> **Note**: If you are using a `/src` folder, please note that Next.js will load the .env files **only** from the parent folder and **not** from the `/src` folder. This loads `process.env.DB_HOST`, `process.env.DB_USER`, and `process.env.DB_PASS` into the Node.js environment automatically allowing you to use them in Route Handlers.

For example:

```
// Archivo: app/api/route.js
export async function GET() {
  const db = await myDB.connect({
    host: process.env.DB_HOST,
    username: process.env.DB_USER,
    password: process.env.DB_PASS,
  })
  // ...
}
```

Loading Environment Variables with `@next/env`

If you need to load environment variables outside of the Next.js runtime, such as in a root config file for an ORM or test runner, you can use the `@next/env` package.

This package is used internally by Next.js to load environment variables from `.env*` files.

To use it, install the package and use the `loadEnvConfig` function to load the environment variables:

```
# Manejador de paquetes: pnpm
pnpm add @next/env
```

```
# Manejador de paquetes: npm
npm install @next/env
```

```
# Manejador de paquetes: yarn
yarn add @next/env
```

```
# Manejador de paquetes: bun
bun add @next/env
```

```
// Archivo: envConfig.ts switcher
import { loadEnvConfig } from '@next/env'

const projectDir = process.cwd()
loadEnvConfig(projectDir)
```

```
// Archivo: envConfig.js switcher
import { loadEnvConfig } from '@next/env'

const projectDir = process.cwd()
loadEnvConfig(projectDir)
```

Then, you can import the configuration where needed. For example:

```
// Archivo: orm.config.ts switcher
import './envConfig.ts'

export default defineConfig({
  dbCredentials: {
    connectionString: process.env.DATABASE_URL!,
  },
})
```

```
// Archivo: orm.config.js switcher
import './envConfig.js'

export default defineConfig({
  dbCredentials: {
    connectionString: process.env.DATABASE_URL,
  },
})
```

Referencing Other Variables

Next.js will automatically expand variables that use `$` to reference other variables e.g. `$VARIABLE` inside of your `.env*` files. This allows you to reference other secrets. For example:

```
// Archivo: .env
TWITTER_USER=nextjs
TWITTER_URL=https://x.com/$TWITTER_USER
```

In the above example, `process.env.TWITTER_URL` would be set to `https://x.com/nextjs`.

> **Good to know**: If you need to use variable with a `$` in the actual value, it needs to be escaped e.g. `\$`.

Bundling Environment Variables for the Browser

Non-`NEXT_PUBLIC_` environment variables are only available in the Node.js environment, meaning they aren't accessible to the browser (the client runs in a different *environment*).

In order to make the value of an environment variable accessible in the browser, Next.js can "inline" a value, at build time, into the js bundle that is delivered to the client, replacing all references to `process.env.[variable]` with a hard-coded value. To tell it to do this, you just have to prefix the variable with `NEXT_PUBLIC_`. For example:

```
// Archivo: .env
NEXT_PUBLIC_ANALYTICS_ID=abcdefghijk
```

This will tell Next.js to replace all references to `process.env.NEXT_PUBLIC_ANALYTICS_ID` in the Node.js environment with the value from the environment in which you run `next build`, allowing you to use it anywhere in your code. It will be inlined into any JavaScript sent to the browser.

> **Note**: After being built, your app will no longer respond to changes to these environment variables. For instance, if you use a Heroku pipeline to promote slugs built in one environment to another environment, or if you build and deploy a single Docker image to multiple environments, all `NEXT_PUBLIC_` variables will be frozen with the value evaluated at build time, so these values need to be set appropriately when the project is built. If you need access to runtime environment values, you'll have to set up your own API to provide them to the client (either on demand or during initialization).

```
// Archivo: pages/index.js
import setupAnalyticsService from '../lib/my-analytics-service'

// 'NEXT_PUBLIC_ANALYTICS_ID' can be used here as it's prefixed by 'NEXT_PUBLIC_'.
// It will be transformed at build time to `setupAnalyticsService('abcdefghijk')`.
setupAnalyticsService(process.env.NEXT_PUBLIC_ANALYTICS_ID)

function HomePage() {
  return <h1>Hello World</h1>
}

export default HomePage
```

Note that dynamic lookups will *not* be inlined, such as:

```
// This will NOT be inlined, because it uses a variable
const varName = 'NEXT_PUBLIC_ANALYTICS_ID'
setupAnalyticsService(process.env[varName])

// This will NOT be inlined, because it uses a variable
const env = process.env
setupAnalyticsService(env.NEXT_PUBLIC_ANALYTICS_ID)
```

Runtime Environment Variables

Next.js can support both build time and runtime environment variables.

By default, environment variables are only available on the server. To expose an environment variable to the browser, it must be prefixed with `NEXT_PUBLIC_`. However, these public environment variables will be inlined into the JavaScript bundle during `next build`.

> 📘 **Nota para Pages Router:** To read runtime environment variables, we recommend using `getServerSideProps` or incrementally adopting the App Router.

You can safely read environment variables on the server during dynamic rendering:

```
// Archivo: app/page.ts switcher
import { connection } from 'next/server'

export default async function Component() {
  await connection()
  // cookies, headers, and other Request-time APIs
  // will also opt into dynamic rendering, meaning
  // this env variable is evaluated at runtime
  const value = process.env.MY_VALUE
  // ...
}
```

```
// Archivo: app/page.js switcher
import { connection } from 'next/server'

export default async function Component() {
  await connection()
  // cookies, headers, and other Request-time APIs
  // will also opt into dynamic rendering, meaning
  // this env variable is evaluated at runtime
  const value = process.env.MY_VALUE
  // ...
}
```

This allows you to use a singular Docker image that can be promoted through multiple environments with different values.

Good to know:

- You can run code on server startup using the `register` function.

Test Environment Variables

Apart from `development` and `production` environments, there is a 3rd option available: `test`. In the same way you can set defaults for development or production environments, you can do the same with a `.env.test` file for the `testing` environment (though this one is not as common as the previous two). Next.js will not load environment variables from `.env.development` or `.env.production` in the `testing` environment.

This one is useful when running tests with tools like `jest` or `cypress` where you need to set specific environment vars only for testing purposes. Test default values will be loaded if `NODE_ENV` is set to `test`, though you usually don't need to do this manually as testing tools will address it for you.

There is a small difference between `test` environment, and both `development` and `production` that you need to bear in mind: `.env.local` won't be loaded, as you expect tests to produce the same results for everyone. This way every test execution will use the

same env defaults across different executions by ignoring your `.env.local` (which is intended to override the default set).

> **Good to know**: similar to Default Environment Variables, `.env.test` file should be included in your repository, but `.env.test.local` shouldn't, as `.env*.local` are intended to be ignored through `.gitignore`.

While running unit tests you can make sure to load your environment variables the same way Next.js does by leveraging the `loadEnvConfig` function from the `@next/env` package.

```
// The below can be used in a Jest global setup file or similar for your testing set-up
import { loadEnvConfig } from '@next/env'

export default async () => {
  const projectDir = process.cwd()
  loadEnvConfig(projectDir)
}
```

Environment Variable Load Order

Environment variables are looked up in the following places, in order, stopping once the variable is found.

1. `process.env`
2. `.env.$(NODE_ENV).local`
3. `.env.local` (Not checked when `NODE_ENV` is `test`.)
4. `.env.$(NODE_ENV)`
5. `.env`

For example, if `NODE_ENV` is `development` and you define a variable in both `.env.development.local` and `.env`, the value in `.env.development.local` will be used.

> **Good to know**: The allowed values for `NODE_ENV` are `production`, `development` and `test`.

Good to know

- If you are using a `/src` directory, `.env.*` files should remain in the root of your project.
- If the environment variable `NODE_ENV` is unassigned, Next.js automatically assigns `development` when running the `next dev` command, or `production` for all other commands.

Version History

Version	Changes
`v9.4.0`	Support `.env` and `NEXT_PUBLIC_` introduced.

How to implement authentication in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/authentication
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to implement authentication in Next.js, covering best practices, securing routes, authorization techniques, and session management.*

How to set a Content Security Policy (CSP) for your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/content-security-policy
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to set a Content Security Policy (CSP) for your Next.js application.*

How to use environment variables in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/environment-variables
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn to add and access environment variables in your Next.js application.*

Capítulo 9: Patrones Avanzados, Agentes de IA y Arquitectura BFF

> *Desarrollo de Agentes de IA en Next.js, Backend-for-Frontend (BFF), internacionalización, testing y observabilidad.*

How to set up your Next.js project for AI coding agents

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/ai-agents
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to configure your Next.js project so AI coding agents use up-to-date documentation instead of outdated training data.*

Next.js ships version-matched documentation inside the `next` package, allowing AI coding agents to reference accurate, up-to-date APIs and patterns. An `AGENTS.md` file at the root of your project directs agents to these bundled docs instead of their training data.

Point agents at the bundled docs, give them runtime visibility into the dev server, let errors drive the fixes, and hand multi-step workflows to skills.

Step 1: Point agents at the bundled docs

Make sure `AGENTS.md` exists at your project root and directs agents to the bundled docs. When you install `next`, the Next.js documentation is bundled at `node_modules/next/dist/docs/`, mirroring the structure of the Next.js documentation site:

```
node_modules/next/dist/docs/
├── 01-app/
│   ├── 01-getting-started/
│   ├── 02-guides/
│   └── 03-api-reference/
├── 02-pages/
├── 03-architecture/
└── index.mdx
```

Agents always have access to docs that match your installed version, with no network request or external lookup required. Upgrading Next.js also upgrades the bundled docs, including new guidance for existing features. Most AI coding agents, including Claude Code, Codex, Cursor, and GitHub Copilot, automatically read `AGENTS.md` when they start a session.

New projects

`create-next-app` generates `AGENTS.md` and `CLAUDE.md` automatically. No additional setup is needed:

```
# Manejador de paquetes: pnpm
pnpm create next-app@canary
```

```
# Manejador de paquetes: npm
npx create-next-app@canary
```

```
# Manejador de paquetes: yarn
yarn create next-app@canary
```

```
# Manejador de paquetes: bun
bun create next-app@canary
```

If you don't want the agent files, pass `--no-agents-md`:

```
npx create-next-app@canary --no-agents-md
```

Existing projects

On Next.js 16.3 or later, run `next dev`. When an AI coding agent is detected in the environment and no managed block is present, Next.js auto-generates `AGENTS.md` and `CLAUDE.md` at the project root. Existing `AGENTS.md` or `CLAUDE.md` files are upserted, so content outside the managed block is preserved:

```
// Archivo: AGENTS.md
<!-- BEGIN:nextjs-agent-rules -->

# This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the
relevant guide in `node_modules/next/dist/docs/` (resolved from this file's directory; in monorepos the `next` package may
not be visible from the repo root) before writing any code. Heed deprecation notices.

This block is written and re-added by `next dev` — verify at `node_modules/next/dist/server/lib/generate-agent-files.js`.
Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

<!-- END:nextjs-agent-rules -->
```

```
// Archivo: CLAUDE.md
@AGENTS.md
```

Add your own project-specific instructions outside the `<!-- BEGIN:nextjs-agent-rules -->` and `<!-- END:nextjs-agent-rules -->` markers, and they're preserved when Next.js updates the managed block.

Opting out

We believe leaving auto-generation on is a good default. Benchmark results on nextjs.org/evals show agents do better when they read the bundled docs. If you really want to opt out, set `agentRules` to `false` in your config:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  agentRules: false,
}

export default nextConfig
```

For earlier versions

On version 16.2, the docs are bundled but `AGENTS.md` is not auto-generated. Add the file yourself with an instruction to read the bundled docs at `node_modules/next/dist/docs/` before writing code.

On version 16.1 and earlier, the docs are not bundled either. Use the legacy `agents-md` command, which downloads a version-matched copy to `.next-docs/` in the project root and indexes it in `AGENTS.md`:

```
npx @next/codemod@canary agents-md
```

Docs over the network

The Next.js docs are also available as Markdown over the network, for agents that fetch pages instead of reading `node_modules`. Append `.md` to any page URL on nextjs.org/docs for a plain Markdown version, and clients that send an `Accept: text/markdown` header get Markdown too. This includes the per-error pages under `/docs/messages`, which are not bundled.

The index at `/docs/llms.txt` and the single-file `/docs/llms-full.txt` follow the `llms.txt` convention, so agents that already read `llms.txt` for other tools can discover the Next.js docs the same way.

Step 2: Give agents runtime visibility

Run `next dev` and let the agent work against the running server. Runtime errors, client-side warnings, and rendered output live in the browser, where agents can't look. Next.js surfaces two complementary views that an agent can read from the terminal.

First, `next dev` forwards browser console errors and warnings to the terminal (the `logging.browserToTerminal` config), so the output agents already read carries the client-side failures they're asked to fix.

`next dev` also writes its PID, port, and URL to `.next/dev/lock`. A second `next dev` in the same project prints the running server's URL and the PID to kill, so an agent connects to the existing server instead of starting a duplicate.

The **framework's view** comes from the Next.js MCP server at `/_next/mcp`, which exposes the running dev server's routes, server logs, and compilation issues. Its `get_compilation_issues` and `compile_route` tools report whether the code compiles straight from the dev server, so an agent doesn't have to run a full `next build` to find out.

The **browser's view** comes from `agent-browser`, a CLI that exposes the DOM, console, network, and Web Vitals as structured text. With React DevTools enabled (pass `--enable react-devtools` to `agent-browser open`, which the `next-dev-loop` skill does for you), it also reports the component tree and which Suspense boundaries are still pending. An agent runs a command like `react tree`, reads the output, and decides what to inspect next, instead of looking at a DevTools panel it can't see.

The `next-dev-loop` skill builds on both views, combining the framework's and the browser's perspective into a single edit-and-verify loop.

> **Good to know:** For the story behind this tooling, see the Next.js 16.2 and Next.js 16.3 AI blog posts.

Step 3: Let errors drive the fixes

With Cache Components enabled, a blocking error presents labeled fixes, each making a different trade-off. The dev overlay adds a **Copy prompt** button that packages the chosen fix into a paste-ready prompt. The prompt walks the agent through reading the matching error page, applying the canonical pattern, and verifying the result at runtime.

The same menu prints in the `next dev` terminal and in `next build` output, so an agent reading CI logs sees it too:

```
// Archivo: Terminal
Route "/products/[slug]": Next.js encountered uncached data during prerendering.

`fetch( ... )` or `connection()` accessed outside of ``<Suspense>`` prevents the route
from being prerendered, blocking the page load and leading to a slower user experience.

Ways to fix this:
  - [stream] Provide a placeholder with ``<Suspense>`` around the data access
  - [cache] Cache the data access with `"use cache"` (does not apply to `connection()`)
  - [block] Set `export const instant = false` to allow a blocking route

Learn more: https://nextjs.org/docs/messages/blocking-prerender-dynamic
    at ProductPage (app/products/[slug]/page.tsx:52:32)
    ...
```

In `next dev` the stack frame resolves to your source, in both the dev overlay and the terminal. A production build minifies server code, so when the build error alone isn't enough, `next build --debug-prerender` turns on server source maps and continues past the first failure.

The `Learn more` link resolves to a per-error page under `/docs/messages` written for agents to read. Each page follows the same shape, with the canonical patterns for every fix, the trade-offs against the other fixes, and the gotchas an agent is likely to miss on a first attempt. The Instant navigation guide walks through the full loop an agent runs on these errors, from reading the insight to writing an `instant()` test that fails before the fix and passes once it's in place.

Step 4: Hand multi-step workflows to Skills

Framework knowledge comes from the bundled docs, not from Skills. Benchmark results show that always-available context outperforms on-demand retrieval. Skills cover the tasks that are workflows rather than lookups, such as adopting Cache Components or Partial Prefetching across an app. Next.js Skills package these as structured instructions an agent installs and follows, sequencing the work and pointing at the relevant docs and runtime tooling along the way.

You can browse the source for these Skills in the Next.js repository and find them on skills.sh.

The Skills serve three different workflow types:

- **Runtime foundations** such as `next-dev-loop` give any coding task a repeatable inspect, edit, and verify cycle.
- **Interactive workflows** make broader changes with user checkpoints, such as adopting Cache Components or Partial Prefetching across an app.
- **Unattended loops** work toward a verifiable goal and stop only for genuine decisions.

`next-dev-loop`

The `next-dev-loop` runtime foundation verifies changes against a running dev server using the MCP server and browser.

```
// Archivo: Terminal
npx skills add vercel/next.js --skill next-dev-loop
```

Then give the agent this prompt:

```
After every edit, verify the page still works at runtime using the next-dev-loop Skill.
```

`next-cache-components-adoption`

The `next-cache-components-adoption` Skill migrates an app to Cache Components:

1. Turns the flag on and finds the routes that can't prerender.
2. Fixes them one feature at a time, checking in with you before moving on.
3. Confirms a feature against `next dev` and `next build` before leaving it.

You choose whether the work lands as a series of PRs or stays on one branch.

```
// Archivo: Terminal
npx skills add vercel/next.js --skill next-cache-components-adoption
```

Then give the agent this prompt:

```
Adopt Cache Components in this project using the next-cache-components-adoption Skill.
```

`next-cache-components-optimizer`

The `next-cache-components-optimizer` Skill takes a target route or set of routes and works toward the UI you want on screen at click time:

1. Writes a failing `instant()` test for the UI you named.
2. Refactors the route until it passes, often by moving a data read below a `<Suspense>` boundary.
3. Commits the passing test with the refactoring, so it catches future regressions.

It needs a route that already builds with Cache Components.

```
// Archivo: Terminal
npx skills add vercel/next.js --skill next-cache-components-optimizer
```

Then give the agent a prompt like:

```
Make the navigation from /settings to /dashboard instant using the next-cache-components-optimizer Skill. The header and the project list should be part of the instant UI.
```

`next-partial-prefetching-adoption`

The `next-partial-prefetching-adoption` Skill moves an app onto Partial Prefetching, where links share one App Shell:

1. Audits existing `<Link prefetch={true}>` navigations and identifies the prefetched UI to preserve.
2. Captures that UI in passing `instant()` tests before enabling Partial Prefetching, then migrates each destination until the same tests pass unchanged.
3. Enables the flag, resolves the URL-data insights it surfaces, and marks optional per-link prefetching candidates for later.

It needs Cache Components already adopted and a production-like build it can run for prefetch verification.

```
// Archivo: Terminal
npx skills add vercel/next.js --skill next-partial-prefetching-adoption
```

Then give the agent this prompt:

```
Adopt Partial Prefetching in this project using the next-partial-prefetching-adoption Skill.
```

How to add analytics to your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/analytics
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Measure and track page performance using Next.js Speed Insights*

Next.js has built-in support for measuring and reporting performance metrics. You can either use the `useReportWebVitals` hook to manage reporting yourself, or alternatively, Vercel provides a managed service to automatically collect and visualize metrics for you.

Client Instrumentation

For more advanced analytics and monitoring needs, Next.js provides a `instrumentation-client.js|ts` file that runs before your application's frontend code starts executing. This is ideal for setting up global analytics, error tracking, or performance monitoring tools.

To use it, create an `instrumentation-client.js` or `instrumentation-client.ts` file in your application's root directory:

```
// Archivo: instrumentation-client.js
// Initialize analytics before the app starts
console.log('Analytics initialized')

// Set up global error tracking
window.addEventListener('error', (event) => {
  // Send to your error tracking service
  reportError(event.error)
})
```

Build Your Own

> 📘 **Nota para Pages Router:**
>
> ```
> import { useReportWebVitals } from 'next/web-vitals'
>
> function MyApp({ Component, pageProps }) {
>   useReportWebVitals((metric) => {
>     console.log(metric)
>   })
>
>   return <Component {...pageProps} />
> }
> ```
>
> View the API Reference for more information.

```
// Archivo: app/_components/web-vitals.js
'use client'

import { useReportWebVitals } from 'next/web-vitals'

export function WebVitals() {
  useReportWebVitals((metric) => {
    console.log(metric)
  })
}
```

```
// Archivo: app/layout.js
import { WebVitals } from './_components/web-vitals'

export default function Layout({ children }) {
  return (
    <html>
      <body>
        <WebVitals />
        {children}
      </body>
    </html>
  )
}
```

> Since the `useReportWebVitals` hook requires the `'use client'` directive, the most performant approach is to create a separate component that the root layout imports. This confines the client boundary exclusively to the `WebVitals` component.

View the API Reference for more information.

Web Vitals

Web Vitals are a set of useful metrics that aim to capture the user experience of a web page. The following web vitals are all included:

- Time to First Byte (TTFB)
- First Contentful Paint (FCP)
- Largest Contentful Paint (LCP)
- First Input Delay (FID)
- Cumulative Layout Shift (CLS)
- Interaction to Next Paint (INP)

You can handle all the results of these metrics using the `name` property.

> 📘 **Nota para Pages Router:**
>
> ```
> import { useReportWebVitals } from 'next/web-vitals'
>
> function MyApp({ Component, pageProps }) {
>   useReportWebVitals((metric) => {
>     switch (metric.name) {
>       case 'FCP': {
>         // handle FCP results
>       }
>       case 'LCP': {
>         // handle LCP results
>       }
>       // ...
>     }
>   })
>
>   return <Component {...pageProps} />
> }
> ```

```
// Archivo: app/_components/web-vitals.tsx switcher
'use client'

import { useReportWebVitals } from 'next/web-vitals'

export function WebVitals() {
  useReportWebVitals((metric) => {
    switch (metric.name) {
      case 'FCP': {
        // handle FCP results
      }
      case 'LCP': {
        // handle LCP results
      }
      // ...
    }
  })
}
```

```
// Archivo: app/_components/web-vitals.js switcher
'use client'

import { useReportWebVitals } from 'next/web-vitals'

export function WebVitals() {
  useReportWebVitals((metric) => {
    switch (metric.name) {
      case 'FCP': {
        // handle FCP results
      }
      case 'LCP': {
        // handle LCP results
      }
      // ...
    }
  })
}
```

📘 **Nota para Pages Router:**

Custom Metrics

In addition to the core metrics listed above, there are some additional custom metrics that measure the time it takes for the page to hydrate and render:

- `Next.js-hydration`: Length of time it takes for the page to start and finish hydrating (in ms)
- `Next.js-route-change-to-render`: Length of time it takes for a page to start rendering after a route change (in ms)
- `Next.js-render`: Length of time it takes for a page to finish render after a route change (in ms)

You can handle all the results of these metrics separately:

```
export function reportWebVitals(metric) {
  switch (metric.name) {
    case 'Next.js-hydration':
      // handle hydration results
      break
    case 'Next.js-route-change-to-render':
      // handle route-change to render results
      break
    case 'Next.js-render':
      // handle render results
      break
    default:
      break
  }
}
```

These metrics work in all browsers that support the User Timing API.

Sending results to external systems

You can send results to any endpoint to measure and track real user performance on your site. For example:

```
useReportWebVitals((metric) => {
  const body = JSON.stringify(metric)
  const url = 'https://example.com/analytics'

  // Use `navigator.sendBeacon()` if available, falling back to `fetch()`.
  if (navigator.sendBeacon) {
    navigator.sendBeacon(url, body)
  } else {
    fetch(url, { body, method: 'POST', keepalive: true })
  }
})
```

Good to know: If you use Google Analytics, using the `id` value can allow you to construct metric distributions manually (to calculate percentiles, etc.)

```
useReportWebVitals((metric) => {
  // Use `window.gtag` if you initialized Google Analytics as this example:
  // https://github.com/vercel/next.js/blob/canary/examples/with-google-analytics
  window.gtag('event', metric.name, {
    value: Math.round(
      metric.name === 'CLS' ? metric.value * 1000 : metric.value
    ), // values must be integers
    event_label: metric.id, // id unique to current page load
    non_interaction: true, // avoids affecting bounce rate.
  })
})
```

Read more about sending results to Google Analytics.

How to use Next.js as a backend for your frontend

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/backend-for-frontend
📂 **Sección:** App > Guides
📝 **Descripción:** *Learn how to use Next.js as a backend framework*

Next.js supports the "Backend for Frontend" pattern. This lets you create public endpoints to handle HTTP requests and return any content type—not just HTML. You can also access data sources and perform side effects like updating remote data.

If you are starting a new project, using `create-next-app` with the `--api` flag automatically includes an example `route.ts` in your new project's `app/` folder, demonstrating how to create an API endpoint.

```
# Manejador de paquetes: pnpm
pnpm create next-app --api
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --api
```

```
# Manejador de paquetes: yarn
yarn create next-app --api
```

```
# Manejador de paquetes: bun
bun create next-app --api
```

> **Good to know**: Next.js backend capabilities are not a full backend replacement. They serve as an API layer that:
>
> - is publicly reachable
> - handles any HTTP request
> - can return any content type

To implement this pattern, use:

- Route Handlers
- `proxy`
- In Pages Router, API Routes

Public Endpoints

Route Handlers are public HTTP endpoints. Any client can access them.

Create a Route Handler using the `route.ts` or `route.js` file convention:

```
// Archivo: /app/api/route.ts switcher
export function GET(request: Request) {}
```

```
// Archivo: /app/api/route.js switcher
export function GET(request) {}
```

This handles `GET` requests sent to `/api`.

Use `try/catch` blocks for operations that may throw an exception:

```
// Archivo: /app/api/route.ts switcher
import { submit } from '@/lib/submit'

export async function POST(request: Request) {
  try {
    await submit(request)
    return new Response(null, { status: 204 })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected error'

    return new Response(message, { status: 500 })
  }
}
```

```
// Archivo: /app/api/route.js switcher
import { submit } from '@/lib/submit'

export async function POST(request) {
  try {
    await submit(request)
    return new Response(null, { status: 204 })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected error'

    return new Response(message, { status: 500 })
  }
}
```

Avoid exposing sensitive information in error messages sent to the client.

To restrict access, implement authentication and authorization. See Authentication.

Content types

Route Handlers let you serve non-UI responses, including JSON, XML, images, files, and plain text.

Next.js uses file conventions for common endpoints:

- `sitemap.xml`
- `opengraph-image.jpg`, `twitter-image`
- favicon, app icon, and apple-icon
- `manifest.json`
- `robots.txt`

You can also define custom ones, such as:

- `llms.txt`
- `rss.xml`
- `.well-known`

For example, `app/rss.xml/route.ts` creates a Route Handler for `rss.xml`.

```
// Archivo: /app/rss.xml/route.ts switcher
export async function GET(request: Request) {
  const rssResponse = await fetch(/* rss endpoint */)
  const rssData = await rssResponse.json()

  const rssFeed = `<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">
<channel>
 <title>${rssData.title}</title>
 <description>${rssData.description}</description>
 <link>${rssData.link}</link>
 <copyright>${rssData.copyright}</copyright>
 ${rssData.items.map((item) => {
   return `<item>
    <title>${item.title}</title>
    <description>${item.description}</description>
    <link>${item.link}</link>
    <pubDate>${item.publishDate}</pubDate>
    <guid isPermaLink="false">${item.guid}</guid>
 </item>`
 })}
</channel>
</rss>`

  const headers = new Headers({ 'content-type': 'application/xml' })

  return new Response(rssFeed, { headers })
}
```

```
// Archivo: /app/rss.xml/route.js switcher
export async function GET(request) {
  const rssResponse = await fetch(/* rss endpoint */)
  const rssData = await rssResponse.json()

  const rssFeed = `<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">
<channel>
 <title>${rssData.title}</title>
 <description>${rssData.description}</description>
 <link>${rssData.link}</link>
 <copyright>${rssData.copyright}</copyright>
 ${rssData.items.map((item) => {
   return `<item>
    <title>${item.title}</title>
    <description>${item.description}</description>
    <link>${item.link}</link>
    <pubDate>${item.publishDate}</pubDate>
    <guid isPermaLink="false">${item.guid}</guid>
 </item>`
 })}
</channel>
</rss>`

  const headers = new Headers({ 'content-type': 'application/xml' })

  return new Response(rssFeed, { headers })
}
```

Sanitize any input used to generate markup.

Content negotiation

You can use rewrites with header matching to serve different content types from the same URL based on the request's `Accept` header. This is known as content negotiation.

For example, a documentation site might serve HTML pages to browsers and raw Markdown to AI agents from the same `/docs/…` URLs.

1. Configure a rewrite that matches the `Accept` header:

```
// Archivo: next.config.js
module.exports = {
  async rewrites() {
    return [
      {
        source: '/docs/:slug*',
        destination: '/docs/md/:slug*',
        has: [
          {
            type: 'header',
            key: 'accept',
            value: '(.*)text/markdown(.*)',
          },
        ],
      },
    ]
  },
}
```

When a request to `/docs/getting-started` includes `Accept: text/markdown`, the rewrite routes it to `/docs/md/getting-started`. A Route Handler at that path returns the Markdown response. Clients that do not send `text/markdown` in their `Accept` header continue to receive the normal HTML page.

2. Create a Route Handler for the Markdown response:

```
// Archivo: app/docs/md/[...slug]/route.ts switcher
import { getDocsMd, generateDocsStaticParams } from '@/lib/docs'

export async function generateStaticParams() {
  return generateDocsStaticParams()
}

export async function GET(_: Request, ctx: RouteContext<'/docs/md/[...slug]'>) {
  const { slug } = await ctx.params
  const mdDoc = await getDocsMd({ slug })

  if (mdDoc == null) {
    return new Response(null, { status: 404 })
  }

  return new Response(mdDoc, {
    headers: {
      'Content-Type': 'text/markdown; charset=utf-8',
      Vary: 'Accept',
    },
  })
}
```

```
// Archivo: app/docs/md/[...slug]/route.js switcher
import { getDocsMd, generateDocsStaticParams } from '@/lib/docs'

export async function generateStaticParams() {
  return generateDocsStaticParams()
}

export async function GET(_, { params }) {
  const { slug } = await params
  const mdDoc = await getDocsMd({ slug })

  if (mdDoc == null) {
    return new Response(null, { status: 404 })
  }

  return new Response(mdDoc, {
    headers: {
      'Content-Type': 'text/markdown; charset=utf-8',
      Vary: 'Accept',
    },
  })
}
```

The `Vary: Accept` response header tells caches that the response body depends on the `Accept` request header. Without it, a shared cache could serve a cached Markdown response to a browser (or vice versa). Most hosting providers already include the `Accept` header in their cache key, but setting `Vary` explicitly ensures correct behavior across all CDNs and proxy caches.

`generateStaticParams` lets you pre-render the Markdown variants at build time so they can be served from the edge without hitting the origin server on every request.

3. Test it with `curl`:

```
# Returns Markdown
curl -H "Accept: text/markdown" https://example.com/docs/getting-started

# Returns the normal HTML page
curl https://example.com/docs/getting-started
```

> **Good to know:**
>
> - The `/docs/md/...` route is still directly accessible without the rewrite. If you want to restrict it to only serve via the rewrite, use `proxy` to block direct requests that don't include the expected `Accept` header.
> - For more advanced negotiation logic, you can use `proxy` instead of rewrites for more flexibility.

Consuming request payloads

Use Request instance methods like `.json()`, `.formData()`, or `.text()` to access the request body.

`GET` and `HEAD` requests don't carry a body.

```
// Archivo: /app/api/echo-body/route.ts switcher
export async function POST(request: Request) {
  const res = await request.json()
  return Response.json({ res })
}
```

```
// Archivo: /app/api/echo-body/route.js switcher
export async function POST(request) {
  const res = await request.json()
  return Response.json({ res })
}
```

> **Good to know**: Validate data before passing it to other systems

```
// Archivo: /app/api/send-email/route.ts switcher
import { sendMail, validateInputs } from '@/lib/email-transporter'

export async function POST(request: Request) {
  const formData = await request.formData()
  const email = formData.get('email')
  const contents = formData.get('contents')

  try {
    await validateInputs({ email, contents })
    const info = await sendMail({ email, contents })

    return Response.json({ messageId: info.messageId })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

```
// Archivo: /app/api/send-email/route.js switcher
import { sendMail, validateInputs } from '@/lib/email-transporter'

export async function POST(request) {
  const formData = await request.formData()
  const email = formData.get('email')
  const contents = formData.get('contents')

  try {
    await validateInputs({ email, contents })
    const info = await sendMail({ email, contents })

    return Response.json({ messageId: info.messageId })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

You can only read the request body once. Clone the request if you need to read it again:

```ts
// Archivo: /app/api/clone/route.ts switcher
export async function POST(request: Request) {
  try {
    const clonedRequest = request.clone()

    await request.text()
    await clonedRequest.text()
    await request.text() // Throws error

    return new Response(null, { status: 204 })
  } catch {
    return new Response(null, { status: 500 })
  }
}
```

```js
// Archivo: /app/api/clone/route.js switcher
export async function POST(request) {
  try {
    const clonedRequest = request.clone()

    await request.text()
    await clonedRequest.text()
    await request.text() // Throws error

    return new Response(null, { status: 204 })
  } catch {
    return new Response(null, { status: 500 })
  }
}
```

Manipulating data

Route Handlers can transform, filter, and aggregate data from one or more sources. This keeps logic out of the frontend and avoids exposing internal systems.

You can also offload heavy computations to the server and reduce client battery and data usage.

```ts
import { parseWeatherData } from '@/lib/weather'

export async function POST(request: Request) {
  const body = await request.json()
  const searchParams = new URLSearchParams({ lat: body.lat, lng: body.lng })

  try {
    const weatherResponse = await fetch(`${weatherEndpoint}?${searchParams}`)

    if (!weatherResponse.ok)

    const weatherData = await weatherResponse.text()
    const payload = parseWeatherData.asJSON(weatherData)

    return new Response(payload, { status: 200 })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

```
import { parseWeatherData } from '@/lib/weather'

export async function POST(request) {
  const body = await request.json()
  const searchParams = new URLSearchParams({ lat: body.lat, lng: body.lng })

  try {
    const weatherResponse = await fetch(`${weatherEndpoint}?${searchParams}`)

    if (!weatherResponse.ok)

    const weatherData = await weatherResponse.text()
    const payload = parseWeatherData.asJSON(weatherData)

    return new Response(payload, { status: 200 })
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

> **Good to know**: This example uses `POST` to avoid putting geo-location data in the URL. `GET` requests may be cached or logged, which could expose sensitive info.

Proxying to a backend

You can use a Route Handler as a `proxy` to another backend. Add validation logic before forwarding the request.

```
// Archivo: /app/api/[...slug]/route.ts switcher
import { isValidRequest } from '@/lib/utils'

export async function POST(request: Request, { params }) {
  const clonedRequest = request.clone()
  const isValid = await isValidRequest(clonedRequest)

  if (!isValid) {
    return new Response(null, { status: 400, statusText: 'Bad Request' })
  }

  const { slug } = await params
  const pathname = slug.join('/')
  const proxyURL = new URL(pathname, 'https://nextjs.org')
  const proxyRequest = new Request(proxyURL, request)

  try {
    return fetch(proxyRequest)
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

```
// Archivo: /app/api/[...slug]/route.js switcher
import { isValidRequest } from '@/lib/utils'

export async function POST(request, { params }) {
  const clonedRequest = request.clone()
  const isValid = await isValidRequest(clonedRequest)

  if (!isValid) {
    return new Response(null, { status: 400, statusText: 'Bad Request' })
  }

  const { slug } = await params
  const pathname = slug.join('/')
  const proxyURL = new URL(pathname, 'https://nextjs.org')
  const proxyRequest = new Request(proxyURL, request)

  try {
    return fetch(proxyRequest)
  } catch (reason) {
    const message =
      reason instanceof Error ? reason.message : 'Unexpected exception'

    return new Response(message, { status: 500 })
  }
}
```

Or use:

- `proxy` rewrites
- `rewrites` in `next.config.js`.

NextRequest and NextResponse

Next.js extends the `Request` and `Response` Web APIs with methods that simplify common operations. These extensions are available in both Route Handlers and Proxy.

Both provide methods for reading and manipulating cookies.

`NextRequest` includes the `nextUrl` property, which exposes parsed values from the incoming request, for example, it makes it easier to access request pathname and search params.

`NextResponse` provides helpers like `next()`, `json()`, `redirect()`, and `rewrite()`.

You can pass `NextRequest` to any function expecting `Request`. Likewise, you can return `NextResponse` where a `Response` is expected.

```
// Archivo: /app/echo-pathname/route.ts switcher
import { type NextRequest, NextResponse } from 'next/server'

export async function GET(request: NextRequest) {
  const nextUrl = request.nextUrl

  if (nextUrl.searchParams.get('redirect')) {
    return NextResponse.redirect(new URL('/', request.url))
  }

  if (nextUrl.searchParams.get('rewrite')) {
    return NextResponse.rewrite(new URL('/', request.url))
  }

  return NextResponse.json({ pathname: nextUrl.pathname })
}
```

```
// Archivo: /app/echo-pathname/route.js switcher
import { NextResponse } from 'next/server'

export async function GET(request) {
  const nextUrl = request.nextUrl

  if (nextUrl.searchParams.get('redirect')) {
    return NextResponse.redirect(new URL('/', request.url))
  }

  if (nextUrl.searchParams.get('rewrite')) {
    return NextResponse.rewrite(new URL('/', request.url))
  }

  return NextResponse.json({ pathname: nextUrl.pathname })
}
```

Learn more about `NextRequest` and `NextResponse`.

Webhooks and callback URLs

Use Route Handlers to receive event notifications from third-party applications.

For example, revalidate a route when content changes in a CMS. Configure the CMS to call a specific endpoint on changes.

```
// Archivo: /app/webhook/route.ts switcher
import { type NextRequest, NextResponse } from 'next/server'

export async function GET(request: NextRequest) {
  const token = request.nextUrl.searchParams.get('token')

  if (token !== process.env.REVALIDATE_SECRET_TOKEN) {
    return NextResponse.json({ success: false }, { status: 401 })
  }

  const tag = request.nextUrl.searchParams.get('tag')

  if (!tag) {
    return NextResponse.json({ success: false }, { status: 400 })
  }

  revalidateTag(tag, 'max')

  return NextResponse.json({ success: true })
}
```

```
// Archivo: /app/webhook/route.js switcher
import { NextResponse } from 'next/server'

export async function GET(request) {
  const token = request.nextUrl.searchParams.get('token')

  if (token !== process.env.REVALIDATE_SECRET_TOKEN) {
    return NextResponse.json({ success: false }, { status: 401 })
  }

  const tag = request.nextUrl.searchParams.get('tag')

  if (!tag) {
    return NextResponse.json({ success: false }, { status: 400 })
  }

  revalidateTag(tag, 'max')

  return NextResponse.json({ success: true })
}
```

Callback URLs are another use case. When a user completes a third-party flow, the third party sends them to a callback URL. Use a Route Handler to verify the response and decide where to redirect the user.

```
// Archivo: /app/auth/callback/route.ts switcher
import { type NextRequest, NextResponse } from 'next/server'

export async function GET(request: NextRequest) {
  const token = request.nextUrl.searchParams.get('session_token')
  const redirectUrl = request.nextUrl.searchParams.get('redirect_url')

  const destination = new URL(redirectUrl ?? '/', request.url)
  // Prevent open redirects: only allow same-origin destinations
  if (destination.origin !== request.nextUrl.origin) {
    return new Response('Invalid redirect', { status: 400 })
  }

  const response = NextResponse.redirect(destination)

  response.cookies.set({
    value: token,
    name: '_token',
    path: '/',
    secure: true,
    httpOnly: true,
    expires: undefined, // session cookie
  })

  return response
}
```

```
// Archivo: /app/auth/callback/route.js switcher
import { NextResponse } from 'next/server'

export async function GET(request) {
  const token = request.nextUrl.searchParams.get('session_token')
  const redirectUrl = request.nextUrl.searchParams.get('redirect_url')

  const destination = new URL(redirectUrl ?? '/', request.url)
  // Prevent open redirects: only allow same-origin destinations
  if (destination.origin !== request.nextUrl.origin) {
    return new Response('Invalid redirect', { status: 400 })
  }

  const response = NextResponse.redirect(destination)

  response.cookies.set({
    value: token,
    name: '_token',
    path: '/',
    secure: true,
    httpOnly: true,
    expires: undefined, // session cookie
  })

  return response
}
```

Redirects

```
// Archivo: app/api/route.ts switcher
import { redirect } from 'next/navigation'

export async function GET(request: Request) {
  redirect('https://nextjs.org/')
}
```

```
// Archivo: app/api/route.js switcher
import { redirect } from 'next/navigation'

export async function GET(request) {
  redirect('https://nextjs.org/')
}
```

Learn more about redirects in `redirect` and `permanentRedirect`

Proxy

Only one `proxy` file is allowed per project. Use `config.matcher` to target specific paths. Learn more about `proxy`.

Use `proxy` to generate a response before the request reaches a route path.

```
// Archivo: proxy.ts switcher
import { isAuthenticated } from '@lib/auth'

export const config = {
  matcher: '/api/:function*',
}

export function proxy(request: Request) {
  if (!isAuthenticated(request)) {
    return Response.json(
      { success: false, message: 'authentication failed' },
      { status: 401 }
    )
  }
}
```

```
// Archivo: proxy.js switcher
import { isAuthenticated } from '@lib/auth'

export const config = {
  matcher: '/api/:function*',
}

export function proxy(request) {
  if (!isAuthenticated(request)) {
    return Response.json(
      { success: false, message: 'authentication failed' },
      { status: 401 }
    )
  }
}
```

You can also proxy requests using `proxy`:

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'

export function proxy(request: Request) {
  if (request.nextUrl.pathname === '/proxy-this-path') {
    const rewriteUrl = new URL('https://nextjs.org')
    return NextResponse.rewrite(rewriteUrl)
  }
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  if (request.nextUrl.pathname === '/proxy-this-path') {
    const rewriteUrl = new URL('https://nextjs.org')
    return NextResponse.rewrite(rewriteUrl)
  }
}
```

Another type of response `proxy` can produce are redirects:

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'

export function proxy(request: Request) {
  if (request.nextUrl.pathname === '/v1/docs') {
    request.nextUrl.pathname = '/v2/docs'
    return NextResponse.redirect(request.nextUrl)
  }
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  if (request.nextUrl.pathname === '/v1/docs') {
    request.nextUrl.pathname = '/v2/docs'
    return NextResponse.redirect(request.nextUrl)
  }
}
```

Security

Working with headers

Be deliberate about where headers go, and avoid directly passing incoming request headers to the outgoing response.

- **Upstream request headers**: In Proxy, `NextResponse.next({ request: { headers } })` modifies the headers your server receives and does not expose them to the client.
- **Response headers**: `new Response( ... , { headers })`, `NextResponse.json( ... , { headers })`, `NextResponse.next({ headers })`, or `response.headers.set( ... )` send headers back to the client. If sensitive values were appended to these headers, they will be visible to clients.

Learn more in NextResponse headers in Proxy.

Rate limiting

You can implement rate limiting in your Next.js backend. In addition to code-based checks, enable any rate limiting features provided by your host.

```
// Archivo: /app/resource/route.ts switcher
import { NextResponse } from 'next/server'
import { checkRateLimit } from '@/lib/rate-limit'

export async function POST(request: Request) {
  const { rateLimited } = await checkRateLimit(request)

  if (rateLimited) {
    return NextResponse.json({ error: 'Rate limit exceeded' }, { status: 429 })
  }

  return new Response(null, { status: 204 })
}
```

```
// Archivo: /app/resource/route.js switcher
import { NextResponse } from 'next/server'
import { checkRateLimit } from '@/lib/rate-limit'

export async function POST(request) {
  const { rateLimited } = await checkRateLimit(request)

  if (rateLimited) {
    return NextResponse.json({ error: 'Rate limit exceeded' }, { status: 429 })
  }

  return new Response(null, { status: 204 })
}
```

Verify payloads

Never trust incoming request data. Validate content type and size, and sanitize against XSS before use.

Use timeouts to prevent abuse and protect server resources.

Store user-generated static assets in dedicated services. When possible, upload them from the browser and store the returned URI in your database to reduce request size.

Access to protected resources

Always verify credentials before granting access. Do not rely on proxy alone for authentication and authorization.

Remove sensitive or unnecessary data from responses and backend logs.

Rotate credentials and API keys regularly.

Preflight Requests

Preflight requests use the `OPTIONS` method to ask the server if a request is allowed based on origin, method, and headers.

If `OPTIONS` is not defined, Next.js adds it automatically and sets the `Allow` header based on the other defined methods.

- CORS

Library patterns

Community libraries often use the factory pattern for Route Handlers.

```
// Archivo: /app/api/[...path]/route.ts
import { createHandler } from 'third-party-library'

const handler = createHandler()

export const GET = handler
// or
export { handler as POST }
```

This creates a shared handler for `GET` and `POST` requests. The library customizes behavior based on the `method` and `pathname` in the request.

Libraries can also provide a `proxy` factory.

```
// Archivo: proxy.ts
import { createMiddleware } from 'third-party-library'

export default createMiddleware()
```

> **Good to know**: Third-party libraries may still refer to `proxy` as `middleware`.

More examples

See more examples on using Router Handlers and the `proxy` API references.

These examples include, working with Cookies, Headers, Streaming, Proxy negative matching, and other useful code snippets.

Caveats

Server Components

Fetch data in Server Components directly from its source, not via Route Handlers.

For Server Components prerendered at build time, using Route Handlers will fail the build step. This is because, while building there is no server listening for these requests.

For Server Components rendered on demand, fetching from Route Handlers is slower due to the extra HTTP round trip between the handler and the render process.

> A server side `fetch` request uses absolute URLs. This implies an HTTP round trip, to an external server. During development, your own development server acts as the external server. At build time there is no server, and at runtime, the server is available through your public facing domain.

Server Components cover most data-fetching needs. However, fetching data client side might be necessary for:

- Data that depends on client-only Web APIs:
  - Geo-location API
  - Storage API
  - Audio API
  - File API
- Frequently polled data

For these, use community libraries like `swr` or `react-query`.

Server Actions

Server Actions let you run server-side code from the client. Their primary purpose is to mutate data from your frontend client.

Server Actions are queued. Using them for data fetching introduces sequential execution.

`export` mode

`export` mode outputs a static site without a runtime server. Features that require the Next.js runtime are not supported, because this mode produces a static site, and no runtime server.

In `export mode`, only `GET` Route Handlers are supported, in combination with the `dynamic` route segment config, set to `'force-static'`.

This can be used to generate static HTML, JSON, TXT, or other files.

```
// Archivo: app/hello-world/route.ts
export const dynamic = 'force-static'

export function GET() {
  return new Response('Hello World', { status: 200 })
}
```

Deployment environment

Some hosts deploy Route Handlers as lambda functions. This means:

- Route Handlers cannot share data between requests.
- The environment may not support writing to File System.
- Long-running handlers may be terminated due to timeouts.
- WebSockets won't work because the connection closes on timeout, or after the response is generated.

How to use debugging tools with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/debugging
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to debug your Next.js application with VS Code, Chrome DevTools, or Firefox DevTools.*

This documentation explains how you can debug your Next.js frontend and backend code with full source maps support using the VS Code debugger, Chrome DevTools, or Firefox DevTools.

Any debugger that can attach to Node.js can also be used to debug a Next.js application. You can find more details in the Node.js Debugging Guide.

Debugging with VS Code

Create a file named `.vscode/launch.json` at the root of your project with the following content:

```
// Archivo: launch.json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Next.js: debug server-side",
      "type": "node-terminal",
      "request": "launch",
      "command": "npm run dev -- --inspect"
    },
    {
      "name": "Next.js: debug client-side",
      "type": "chrome",
      "request": "launch",
      "url": "http://localhost:3000"
    },
    {
      "name": "Next.js: debug client-side (Firefox)",
      "type": "firefox",
      "request": "launch",
      "url": "http://localhost:3000",
      "reAttach": true,
      "pathMappings": [
        {
          "url": "webpack://_N_E",
          "path": "${workspaceFolder}"
        }
      ]
    },
    {
      "name": "Next.js: debug full stack",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/node_modules/next/dist/bin/next",
      "runtimeArgs": ["--inspect"],
      "skipFiles": ["<node_internals>/**"],
      "serverReadyAction": {
        "action": "debugWithEdge",
        "killOnServerStop": true,
        "pattern": "- Local:.+(https?://.+)",
        "uriFormat": "%s",
        "webRoot": "${workspaceFolder}"
      }
    }
  ]
}
```

> **Note**: To use Firefox debugging in VS Code, you'll need to install the Firefox Debugger extension.

`npm run dev` can be replaced with `yarn dev` if you're using Yarn or `pnpm dev` if you're using pnpm.

In the "Next.js: debug full stack" configuration, `serverReadyAction.action` specifies which browser to open when the server is ready. `debugWithEdge` means to launch the Edge browser. If you are using Chrome, change this value to `debugWithChrome`.

If you're changing the port number your application starts on, replace the `3000` in `http://localhost:3000` with the port you're using instead.

If you're running Next.js from a directory other than root (for example, if you're using Turborepo) then you need to add `cwd` to the server-side and full stack debugging tasks. For example, `"cwd": "${workspaceFolder}/apps/web"`.

Now go to the Debug panel (`Ctrl+Shift+D` on Windows/Linux, `⇧+⌘+D` on macOS), select a launch configuration, then press `F5` or select **Debug: Start Debugging** from the Command Palette to start your debugging session.

Using the Debugger in Jetbrains WebStorm

Click the drop down menu listing the runtime configuration, and click `Edit Configurations ...`. Create a `JavaScript Debug` debug configuration with `http://localhost:3000` as the URL. Customize to your liking (e.g. Browser for debugging, store as project file), and

click `OK`. Run this debug configuration, and the selected browser should automatically open. At this point, you should have 2 applications in debug mode: the Next.js node application, and the client/browser application.

Debugging with Browser DevTools

Client-side code

Start your development server as usual by running `next dev`, `npm run dev`, or `yarn dev`. Once the server starts, open `http://localhost:3000` (or your alternate URL) in your preferred browser.

For Chrome:

- Open Chrome's Developer Tools (`Ctrl+Shift+J` on Windows/Linux, `⌥+⌘+I` on macOS)
- Go to the **Sources** tab

For Firefox:

- Open Firefox's Developer Tools (`Ctrl+Shift+I` on Windows/Linux, `⌥+⌘+I` on macOS)
- Go to the **Debugger** tab

In either browser, any time your client-side code reaches a `debugger` statement, code execution will pause and that file will appear in the debug area. You can also search for files to set breakpoints manually:

- In Chrome: Press `Ctrl+P` on Windows/Linux or `⌘+P` on macOS
- In Firefox: Press `Ctrl+P` on Windows/Linux or `⌘+P` on macOS, or use the file tree in the left panel

Note that when searching, your source files will have paths starting with `webpack://_N_E/./`.

React Developer Tools

For React-specific debugging, install the React Developer Tools browser extension. This essential tool helps you:

- Inspect React components
- Edit props and state
- Identify performance problems

Server-side code

To debug server-side Next.js code with browser DevTools, you need to pass the `--inspect` flag:

```
# Manejador de paquetes: pnpm
pnpm dev --inspect
```

```
# Manejador de paquetes: npm
npm run dev -- --inspect
```

```
# Manejador de paquetes: yarn
yarn dev --inspect
```

```
# Manejador de paquetes: bun
bun run dev --inspect
```

The value of `--inspect` is passed to the underlying Node.js process. Check out the `--inspect` docs for advanced use cases.

> **Good to know**: Use `--inspect=0.0.0.0` to allow remote debugging access outside localhost, such as when running the app in a Docker container.

Launching the Next.js server with the `--inspect` flag will look something like this:

```
// Archivo: Terminal
Debugger listening on ws://127.0.0.1:9229/0cf90313-350d-4466-a748-cd60f4e47c95
For help, see: https://nodejs.org/learn/getting-started/debugging
ready - started server on 0.0.0.0:3000, url: http://localhost:3000
```

For Chrome:

1. Open a new tab and visit `chrome://inspect`
2. Look for your Next.js application in the **Remote Target** section
3. Click **inspect** to open a separate DevTools window
4. Go to the **Sources** tab

For Firefox:

1. Open a new tab and visit `about:debugging`
2. Click **This Firefox** in the left sidebar
3. Under **Remote Targets**, find your Next.js application
4. Click **Inspect** to open the debugger
5. Go to the **Debugger** tab

Debugging server-side code works similarly to client-side debugging. When searching for files (`Ctrl+P` / `⌘+P`), your source files will have paths starting with `webpack://{application-name}/./` (where `{application-name}` will be replaced with the name of your application according to your `package.json` file).

To use `--inspect-brk` or `--inspect-wait`, you have to specify `NODE_OPTIONS` instead. e.g. `NODE_OPTIONS=--inspect-brk next dev`.

Inspect Server Errors with Browser DevTools

When you encounter an error, inspecting the source code can help trace the root cause of errors.

Next.js will display a Node.js icon underneath the Next.js version indicator on the error overlay. By clicking that icon, the DevTools URL is copied to your clipboard. You can open a new browser tab with that URL to inspect the Next.js server process.

Debugging on Windows

Ensure Windows Defender is disabled on your machine. This external service will check *every file read*, which has been reported to greatly increase Fast Refresh time with `next dev`. This is a known issue, not related to Next.js, but it does affect Next.js development.

More information

To learn more about how to use a JavaScript debugger, take a look at the following documentation:

- Node.js debugging in VS Code: Breakpoints
- Chrome DevTools: Debug JavaScript
- Firefox DevTools: Debugger

Internationalization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/internationalization
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Add support for multiple languages with internationalized routing and localized content.*

Next.js enables you to configure the routing and rendering of content to support multiple languages. Making your site adaptive to different locales includes translated content (localization) and internationalized routes.

Terminology

- **Locale:** An identifier for a set of language and formatting preferences. This usually includes the preferred language of the user and possibly their geographic region.
  - `en-US`: English as spoken in the United States
  - `nl-NL`: Dutch as spoken in the Netherlands
  - `nl`: Dutch, no specific region

Routing Overview

It's recommended to use the user's language preferences in the browser to select which locale to use. Changing your preferred language will modify the incoming `Accept-Language` header to your application.

For example, using the following libraries, you can look at an incoming `Request` to determine which locale to select, based on the `Headers`, locales you plan to support, and the default locale.

```
// Archivo: proxy.js
import { match } from '@formatjs/intl-localematcher'
import Negotiator from 'negotiator'

let headers = { 'accept-language': 'en-US,en;q=0.5' }
let languages = new Negotiator({ headers }).languages()
let locales = ['en-US', 'nl-NL', 'nl']
let defaultLocale = 'en-US'

match(languages, locales, defaultLocale) // → 'en-US'
```

Routing can be internationalized by either the sub-path (`/fr/products`) or domain (`my-site.fr/products`). With this information, you can now redirect the user based on the locale inside Proxy.

```
// Archivo: proxy.js
import { NextResponse } from "next/server";

let locales = ['en-US', 'nl-NL', 'nl']

// Get the preferred locale, similar to the above or using a library
function getLocale(request) { ... }

export function proxy(request) {
  // Check if there is any supported locale in the pathname
  const { pathname } = request.nextUrl
  const pathnameHasLocale = locales.some(
    (locale) => pathname.startsWith(`/${locale}/`) || pathname === `/${locale}`
  )

  if (pathnameHasLocale) return

  // Redirect if there is no locale
  const locale = getLocale(request)
  request.nextUrl.pathname = `/${locale}${pathname}`
  // e.g. incoming request is /products
  // The new URL is now /en-US/products
  return NextResponse.redirect(request.nextUrl)
}

export const config = {
  matcher: [
    // Skip all internal paths (_next)
    '/((?!_next).*)',
    // Optional: only run on root (/) URL
    // '/'
  ],
}
```

Finally, ensure all special files inside `app/` are nested under `app/[lang]`. This enables the Next.js router to dynamically handle different locales in the route, and forward the `lang` parameter to every layout and page. For example:

```
// Archivo: app/[lang]/page.tsx switcher
// You now have access to the current locale
// e.g. /en-US/products → `lang` is "en-US"
export default async function Page({ params }: PageProps<'/[lang]'>) {
  const { lang } = await params
  return ...
}
```

```
// Archivo: app/[lang]/page.js switcher
// You now have access to the current locale
// e.g. /en-US/products → `lang` is "en-US"
export default async function Page({ params }) {
  const { lang } = await params
  return ...
}
```

> **Good to know:** `PageProps` and `LayoutProps` are globally available TypeScript helpers that provide strong typing for route parameters. See PageProps and LayoutProps for more details.

The root layout can also be nested in the new folder (e.g. `app/[lang]/layout.js`).

Localization

Changing displayed content based on the user's preferred locale, or localization, is not something specific to Next.js. The patterns described below would work the same with any web application.

Let's assume we want to support both English and Dutch content inside our application. We might maintain two different "dictionaries", which are objects that give us a mapping from some key to a localized string. For example:

```
// Archivo: dictionaries/en.json
{
  "products": {
    "cart": "Add to Cart"
  }
}
```

```
// Archivo: dictionaries/nl.json
{
  "products": {
    "cart": "Toevoegen aan Winkelwagen"
  }
}
```

We can then create a `getDictionary` function to load the translations for the requested locale:

```
// Archivo: app/[lang]/dictionaries.ts switcher
import 'server-only'

const dictionaries = {
  en: () => import('./dictionaries/en.json').then((module) => module.default),
  nl: () => import('./dictionaries/nl.json').then((module) => module.default),
}

export type Locale = keyof typeof dictionaries

export const hasLocale = (locale: string): locale is Locale =>
  locale in dictionaries

export const getDictionary = async (locale: Locale) => dictionaries[locale]()
```

```
// Archivo: app/[lang]/dictionaries.js switcher
import 'server-only'

const dictionaries = {
  en: () => import('./dictionaries/en.json').then((module) => module.default),
  nl: () => import('./dictionaries/nl.json').then((module) => module.default),
}

export const hasLocale = (locale) => locale in dictionaries

export const getDictionary = async (locale) => dictionaries[locale]()
```

Given the currently selected language, we can fetch the dictionary inside of a layout or page.

Since `lang` is typed as `string`, using `hasLocale` narrows the type to your supported locales. It also ensures a 404 is returned if a translation is missing, rather than a runtime error.

```
// Archivo: app/[lang]/page.tsx switcher
import { notFound } from 'next/navigation'
import { getDictionary, hasLocale } from './dictionaries'

export default async function Page({ params }: PageProps<'/[lang]'>) {
  const { lang } = await params

  if (!hasLocale(lang)) notFound()

  const dict = await getDictionary(lang)
  return <button>{dict.products.cart}</button> // Add to Cart
}
```

```
// Archivo: app/[lang]/page.js switcher
import { notFound } from 'next/navigation'
import { getDictionary, hasLocale } from './dictionaries'

export default async function Page({ params }) {
  const { lang } = await params

  if (!hasLocale(lang)) notFound()

  const dict = await getDictionary(lang)
  return <button>{dict.products.cart}</button> // Add to Cart
}
```

Because all layouts and pages in the `app/` directory default to Server Components, we do not need to worry about the size of the translation files affecting our client-side JavaScript bundle size. This code will **only run on the server**, and only the resulting HTML will be sent to the browser.

Sharing the locale across your app

The locale is often needed beyond the page that receives it, such as in shared data-fetching utilities or deeply nested components. Instead of prop drilling `lang` through each layer, we can read it directly with `next/root-params`.

`next/root-params` exports a getter for each dynamic segment above the root layout. Since every route is nested under `app/[lang]`, `lang` is a root parameter, and any Server Component or server-side utility can call its getter. We can move the locale lookup into `getDictionary`, so callers no longer pass `lang`:

```ts
// Archivo: app/[lang]/dictionaries.ts highlight={1,2,15,16,17} switcher
import { lang } from 'next/root-params'
import { notFound } from 'next/navigation'

const dictionaries = {
  en: () => import('./dictionaries/en.json').then((module) => module.default),
  nl: () => import('./dictionaries/nl.json').then((module) => module.default),
}

export type Locale = keyof typeof dictionaries

export const hasLocale = (locale: string): locale is Locale =>
  locale in dictionaries

export const getDictionary = async () => {
  const locale = await lang()
  if (!hasLocale(locale)) notFound()
  return dictionaries[locale]()
}
```

```js
// Archivo: app/[lang]/dictionaries.js highlight={1,2,12,13,14} switcher
import { lang } from 'next/root-params'
import { notFound } from 'next/navigation'

const dictionaries = {
  en: () => import('./dictionaries/en.json').then((module) => module.default),
  nl: () => import('./dictionaries/nl.json').then((module) => module.default),
}

export const hasLocale = (locale) => locale in dictionaries

export const getDictionary = async () => {
  const locale = await lang()
  if (!hasLocale(locale)) notFound()
  return dictionaries[locale]()
}
```

> **Good to know:** Files that import from `next/root-params` do not need `import 'server-only'`. The import already fails at build time if used in a Client Component.

Pages and components then call `getDictionary()` with no arguments, since the locale is resolved internally:

```tsx
// Archivo: app/[lang]/page.tsx highlight={4} switcher
import { getDictionary } from './dictionaries'

export default async function Page() {
  const dict = await getDictionary()
  return <button>{dict.products.cart}</button> // Add to Cart
}
```

```jsx
// Archivo: app/[lang]/page.js highlight={4} switcher
import { getDictionary } from './dictionaries'

export default async function Page() {
  const dict = await getDictionary()
  return <button>{dict.products.cart}</button> // Add to Cart
}
```

> **Good to know:** Root parameter getters run in Server Components and server-side utilities, but not in Client Components, Server Actions, or Route Handlers. See `next/root-params` for the full API and its behavior with caching.

Static Rendering

To generate static routes for a given set of locales, we can use `generateStaticParams` with any page or layout. This can be global, for example, in the root layout:

```
// Archivo: app/[lang]/layout.tsx switcher
export async function generateStaticParams() {
  return [{ lang: 'en-US' }, { lang: 'de' }]
}

export default async function RootLayout({
  children,
  params,
}: LayoutProps<'/[lang]'>) {
  return (
    <html lang={(await params).lang}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/[lang]/layout.js switcher
export async function generateStaticParams() {
  return [{ lang: 'en-US' }, { lang: 'de' }]
}

export default async function RootLayout({ children, params }) {
  return (
    <html lang={(await params).lang}>
      <body>{children}</body>
    </html>
  )
}
```

Resources

- Minimal i18n routing and translations
- `next-intl`
- `next-international`
- `next-i18n-router`
- `paraglide-next`
- `lingui`
- `tolgee`
- `next-intlayer`
- `gt-next`

How to set up Cypress with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/testing/cypress
> 📁 **Sección:** App > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Cypress with Next.js for End-to-End (E2E) and Component Testing.*

Cypress is a test runner used for **End-to-End (E2E)** and **Component Testing**. This page will show you how to set up Cypress with Next.js and write your first tests.

> **Warning:**
>
> - Cypress versions below 13.6.3 do not support TypeScript version 5 with `moduleResolution:"bundler"`. However, this issue has been resolved in Cypress version 13.6.3 and later. cypress v13.6.3

Quickstart

You can use `create-next-app` with the with-cypress example to quickly get started.

```
# Manejador de paquetes: pnpm
pnpm create next-app --example with-cypress with-cypress-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example with-cypress with-cypress-app
```

```
# Manejador de paquetes: yarn
yarn create next-app --example with-cypress with-cypress-app
```

```
# Manejador de paquetes: bun
bun create next-app --example with-cypress with-cypress-app
```

Manual setup

To manually set up Cypress, install `cypress` as a dev dependency:

```
# Manejador de paquetes: pnpm
pnpm add -D cypress
```

```
# Manejador de paquetes: npm
npm install -D cypress
```

```
# Manejador de paquetes: yarn
yarn add -D cypress
```

```
# Manejador de paquetes: bun
bun add -D cypress
```

Add the Cypress `open` command to the `package.json` scripts field:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "cypress:open": "cypress open"
  }
}
```

Run Cypress for the first time to open the Cypress testing suite:

```
# Manejador de paquetes: pnpm
pnpm cypress:open
```

```
# Manejador de paquetes: npm
npm run cypress:open
```

```
# Manejador de paquetes: yarn
yarn cypress:open
```

```
# Manejador de paquetes: bun
bun run cypress:open
```

You can choose to configure **E2E Testing** and/or **Component Testing**. Selecting any of these options will automatically create a `cypress.config.js` file and a `cypress` folder in your project.

Creating your first Cypress E2E test

Ensure your `cypress.config` file has the following configuration:

```
// Archivo: cypress.config.ts switcher
import { defineConfig } from 'cypress'

export default defineConfig({
  e2e: {
    setupNodeEvents(on, config) {},
  },
})
```

```
// Archivo: cypress.config.js switcher
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {},
  },
})
```

Then, create two new Next.js files:

```
// Archivo: app/page.js
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
// Archivo: app/about/page.js
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>About</h1>
      [Home](/)
    </div>
  )
}
```

📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Home() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
import Link from 'next/link'

export default function About() {
  return (
    <div>
      <h1>About</h1>
      [Home](/)
    </div>
  )
}
```

Add a test to check your navigation is working correctly:

```
// Archivo: cypress/e2e/app.cy.js
describe('Navigation', () => {
  it('should navigate to the about page', () => {
    // Start from the index page
    cy.visit('http://localhost:3000/')

    // Find a link with an href attribute containing "about" and click it
    cy.get('a[href*="about"]').click()

    // The new url should include "/about"
    cy.url().should('include', '/about')

    // The new page should contain an h1 with "About"
    cy.get('h1').contains('About')
  })
})
```

Running E2E Tests

Cypress will simulate a user navigating your application, this requires your Next.js server to be running. We recommend running your tests against your production code to more closely resemble how your application will behave.

Run `npm run build && npm run start` to build your Next.js application, then run `npm run cypress:open` in another terminal window to start Cypress and run your E2E Testing suite.

Good to know:

- You can use `cy.visit("/")` instead of `cy.visit("http://localhost:3000/")` by adding `baseUrl: 'http://localhost:3000'` to the `cypress.config.js` configuration file.
- Alternatively, you can install the `start-server-and-test` package to run the Next.js production server in conjunction with Cypress. After installation, add `"test": "start-server-and-test start http://localhost:3000 cypress"` to your `package.json` scripts field. Remember to rebuild your application after new changes.

Creating your first Cypress component test

Component tests build and mount a specific component without having to bundle your whole application or start a server.

Select **Component Testing** in the Cypress app, then select **Next.js** as your front-end framework. A `cypress/component` folder will be created in your project, and a `cypress.config.js` file will be updated to enable Component Testing.

Ensure your `cypress.config` file has the following configuration:

```
// Archivo: cypress.config.ts switcher
import { defineConfig } from 'cypress'

export default defineConfig({
  component: {
    devServer: {
      framework: 'next',
      bundler: 'webpack',
    },
  },
})
```

```
// Archivo: cypress.config.js switcher
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  component: {
    devServer: {
      framework: 'next',
      bundler: 'webpack',
    },
  },
})
```

Assuming the same components from the previous section, add a test to validate a component is rendering the expected output:

```
// Archivo: cypress/component/about.cy.tsx
import Page from '../../app/page'

describe('<Page />', () => {
  it('should render and display expected content', () => {
    // Mount the React component for the Home page
    cy.mount(<Page />)

    // The new page should contain an h1 with "Home"
    cy.get('h1').contains('Home')

    // Validate that a link with the expected URL is present
    // Following the link is better suited to an E2E test
    cy.get('a[href="/about"]').should('be.visible')
  })
})
```

📘 **Nota para Pages Router:**

```
import AboutPage from '../../pages/about'

describe('<AboutPage />', () => {
  it('should render and display expected content', () => {
    // Mount the React component for the About page
    cy.mount(<AboutPage />)

    // The new page should contain an h1 with "About page"
    cy.get('h1').contains('About')

    // Validate that a link with the expected URL is present
    // *Following* the link is better suited to an E2E test
    cy.get('a[href="/"]').should('be.visible')
  })
})
```

Good to know:

- Cypress currently doesn't support Component Testing for `async` Server Components. We recommend using E2E testing.
- Since component tests do not require a Next.js server, features like `

Illustration

` that rely on a server being available may not function out-of-the-box.

Running Component Tests

Run `npm run cypress:open` in your terminal to start Cypress and run your Component Testing suite.

Continuous Integration (CI)

In addition to interactive testing, you can also run Cypress headlessly using the `cypress run` command, which is better suited for CI environments:

```
// Archivo: package.json
{
  "scripts": {
    // ...
    "e2e": "start-server-and-test dev http://localhost:3000 \"cypress open --e2e\"",
    "e2e:headless": "start-server-and-test dev http://localhost:3000 \"cypress run --e2e\"",
    "component": "cypress open --component",
    "component:headless": "cypress run --component"
  }
}
```

You can learn more about Cypress and Continuous Integration from these resources:

- Next.js with Cypress example
- Cypress Continuous Integration Docs
- Cypress GitHub Actions Guide
- Official Cypress GitHub Action
- Cypress Discord

Testing

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/testing
📁 **Sección:** App > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with four commonly used testing tools — Cypress, Playwright, Vitest, and Jest.*

In React and Next.js, there are a few different types of tests you can write, each with its own purpose and use cases. This page provides an overview of types and commonly used tools you can use to test your application.

Types of tests

- **Unit Testing** involves testing individual units (or blocks of code) in isolation. In React, a unit can be a single function, hook, or component.
- **Component Testing** is a more focused version of unit testing where the primary subject of the tests is React components. This may involve testing how components are rendered, their interaction with props, and their behavior in response to user events.

- **Integration Testing** involves testing how multiple units work together. This can be a combination of components, hooks, and functions.
- **End-to-End (E2E) Testing** involves testing user flows in an environment that simulates real user scenarios, like the browser. This means testing specific tasks (e.g. signup flow) in a production-like environment.
- **Snapshot Testing** involves capturing the rendered output of a component and saving it to a snapshot file. When tests run, the current rendered output of the component is compared against the saved snapshot. Changes in the snapshot are used to indicate unexpected changes in behavior.

Async Server Components

Since `async` Server Components are new to the React ecosystem, some tools do not fully support them. In the meantime, we recommend using **End-to-End Testing** over **Unit Testing** for `async` components.

Guides

See the guides below to learn how to set up Next.js with these commonly used testing tools:

How to set up Jest with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/testing/jest
> 📂 **Sección:** App > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Jest with Next.js for Unit Testing and Snapshot Testing.*

Jest and React Testing Library are frequently used together for **Unit Testing** and **Snapshot Testing**. This guide will show you how to set up Jest with Next.js and write your first tests.

> **Good to know:** Since `async` Server Components are new to the React ecosystem, Jest currently does not support them. While you can still run **unit tests** for synchronous Server and Client Components, we recommend using an **E2E tests** for `async` components.

Quickstart

You can use `create-next-app` with the Next.js with-jest example to quickly get started:

```
# Manejador de paquetes: pnpm
pnpm create next-app --example with-jest with-jest-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example with-jest with-jest-app
```

```
# Manejador de paquetes: yarn
yarn create next-app --example with-jest with-jest-app
```

```
# Manejador de paquetes: bun
bun create next-app --example with-jest with-jest-app
```

Manual setup

Since the release of Next.js 12, Next.js now has built-in configuration for Jest.

To set up Jest, install `jest` and the following packages as dev dependencies:

```
# Manejador de paquetes: pnpm
pnpm add -D jest jest-environment-jsdom @testing-library/react @testing-library/dom @testing-library/jest-dom ts-node
@types/jest
```

```
# Manejador de paquetes: npm
npm install -D jest jest-environment-jsdom @testing-library/react @testing-library/dom @testing-library/jest-dom ts-node
@types/jest
```

```
# Manejador de paquetes: yarn
yarn add -D jest jest-environment-jsdom @testing-library/react @testing-library/dom @testing-library/jest-dom ts-node
@types/jest
```

```
# Manejador de paquetes: bun
bun add -D jest jest-environment-jsdom @testing-library/react @testing-library/dom @testing-library/jest-dom ts-node
@types/jest
```

Generate a basic Jest configuration file by running the following command:

```
# Manejador de paquetes: pnpm
pnpm create jest@latest
```

```
# Manejador de paquetes: npm
npm init jest@latest
```

```
# Manejador de paquetes: yarn
yarn create jest@latest
```

```
# Manejador de paquetes: bun
bun create jest@latest
```

This will take you through a series of prompts to set up Jest for your project, including automatically creating a `jest.config.ts|js` file.

Update your config file to use `next/jest`. This transformer has all the necessary configuration options for Jest to work with Next.js:

```
// Archivo: jest.config.ts switcher
import type { Config } from 'jest'
import nextJest from 'next/jest.js'

const createJestConfig = nextJest({
  // Provide the path to your Next.js app to load next.config.js and .env files in your test environment
  dir: './',
})

// Add any custom config to be passed to Jest
const config: Config = {
  coverageProvider: 'v8',
  testEnvironment: 'jsdom',
  // Add more setup options before each test is run
  // setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
}

// createJestConfig is exported this way to ensure that next/jest can load the Next.js config which is async
export default createJestConfig(config)
```

```
// Archivo: jest.config.js switcher
const nextJest = require('next/jest')

/** @type {import('jest').Config} */
const createJestConfig = nextJest({
  // Provide the path to your Next.js app to load next.config.js and .env files in your test environment
  dir: './',
})

// Add any custom config to be passed to Jest
const config = {
  coverageProvider: 'v8',
  testEnvironment: 'jsdom',
  // Add more setup options before each test is run
  // setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
}

// createJestConfig is exported this way to ensure that next/jest can load the Next.js config which is async
module.exports = createJestConfig(config)
```

Under the hood, `next/jest` is automatically configuring Jest for you, including:

- Setting up `transform` using the Next.js Compiler.
- Auto mocking stylesheets (`.css` , `.module.css` , and their scss variants), image imports and `next/font` .
- Loading `.env` (and all variants) into `process.env` .
- Ignoring `node_modules` from test resolving and transforms.
- Ignoring `.next` from test resolving.
- Loading `next.config.js` for flags that enable SWC transforms.

> **Good to know**: To test environment variables directly, load them manually in a separate setup script or in your `jest.config.ts` file. For more information, please see Test Environment Variables.

📘 **Nota para Pages Router:**

Setting up Jest (with Babel)

If you opt out of the Next.js Compiler and use Babel instead, you will need to manually configure Jest and install `babel-jest` and `identity-obj-proxy` in addition to the packages above.

Here are the recommended options to configure Jest for Next.js:

```
module.exports = {
  collectCoverage: true,
  // on node 14.x coverage provider v8 offers good speed and more or less good report
  coverageProvider: 'v8',
  collectCoverageFrom: [
    '**/*.{js,jsx,ts,tsx}',
    '!**/*.d.ts',
    '!**/node_modules/**',
    '!<rootDir>/out/**',
    '!<rootDir>/.next/**',
    '!<rootDir>/*.config.js',
    '!<rootDir>/coverage/**',
  ],
  moduleNameMapper: {
    // Handle CSS imports (with CSS modules)
    // https://jestjs.io/docs/webpack#mocking-css-modules
    '^.+\\.module\\.(css|sass|scss)$': 'identity-obj-proxy',

    // Handle CSS imports (without CSS modules)
    '^.+\\.(css|sass|scss)$': '<rootDir>/__mocks__/styleMock.js',

    // Handle image imports
    // https://jestjs.io/docs/webpack#handling-static-assets
    '^.+\\.(png|jpg|jpeg|gif|webp|avif|ico|bmp|svg)$': `<rootDir>/__mocks__/fileMock.js`,

    // Handle module aliases
    '^@/components/(.*)$': '<rootDir>/components/$1',

    // Handle @next/font
    '@next/font/(.*)': `<rootDir>/__mocks__/nextFontMock.js`,
    // Handle next/font
    'next/font/(.*)': `<rootDir>/__mocks__/nextFontMock.js`,
    // Disable server-only
    'server-only': `<rootDir>/__mocks__/empty.js`,
  },
  // Add more setup options before each test is run
  // setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
  testPathIgnorePatterns: ['<rootDir>/node_modules/', '<rootDir>/.next/'],
  testEnvironment: 'jsdom',
  transform: {
    // Use babel-jest to transpile tests with the next/babel preset
    // https://jestjs.io/docs/configuration#transform-objectstring-pathtotransformer--pathtotransformer-object
    '^.+\\.(js|jsx|ts|tsx)$': ['babel-jest', { presets: ['next/babel'] }],
  },
  transformIgnorePatterns: [
    '/node_modules/',
    '^.+\\.module\\.(css|sass|scss)$',
  ],
}
```

You can learn more about each configuration option in the Jest docs. We also recommend reviewing `next/jest` configuration to see how Next.js configures Jest.

Handling stylesheets and image imports

Stylesheets and images aren't used in the tests but importing them may cause errors, so they will need to be mocked.

Create the mock files referenced in the configuration above - `fileMock.js` and `styleMock.js` - inside a `__mocks__` directory:

```
module.exports = 'test-file-stub'
```

```
module.exports = {}
```

For more information on handling static assets, please refer to the Jest Docs.

Handling Fonts

To handle fonts, create the `nextFontMock.js` file inside the `__mocks__` directory, and add the following configuration:

```
module.exports = new Proxy(
  {},
  {
    get: function getter() {
      return () => ({
        className: 'className',
        variable: 'variable',
        style: { fontFamily: 'fontFamily' },
      })
    },
  }
)
```

Optional: Handling Absolute Imports and Module Path Aliases

If your project is using Module Path Aliases, you will need to configure Jest to resolve the imports by matching the paths option in the `jsconfig.json` file with the `moduleNameMapper` option in the `jest.config.js` file. For example:

```
// Archivo: tsconfig.json or jsconfig.json
{
  "compilerOptions": {
    "module": "esnext",
    "moduleResolution": "bundler",
    "baseUrl": "./",
    "paths": {
      "@/components/*": ["components/*"]
    }
  }
}
```

```
// Archivo: jest.config.js
moduleNameMapper: {
  // ...
  '^@/components/(.*)$': '<rootDir>/components/$1',
}
```

Optional: Extend Jest with custom matchers

`@testing-library/jest-dom` includes a set of convenient custom matchers such as `.toBeInTheDocument()` making it easier to write tests. You can import the custom matchers for every test by adding the following option to the Jest configuration file:

```
// Archivo: jest.config.ts switcher
setupFilesAfterEnv: ['<rootDir>/jest.setup.ts']
```

```
// Archivo: jest.config.js switcher
setupFilesAfterEnv: ['<rootDir>/jest.setup.js']
```

Then, inside `jest.setup`, add the following import:

```
// Archivo: jest.setup.ts switcher
import '@testing-library/jest-dom'
```

```
// Archivo: jest.setup.js switcher
import '@testing-library/jest-dom'
```

> **Good to know:** `extend-expect` was removed in `v6.0`, so if you are using `@testing-library/jest-dom` before version 6, you will need to import `@testing-library/jest-dom/extend-expect` instead.

If you need to add more setup options before each test, you can add them to the `jest.setup` file above.

Add a test script to `package.json`

Finally, add a Jest `test` script to your `package.json` file:

```
// Archivo: package.json highlight={6-7}
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "test": "jest",
    "test:watch": "jest --watch"
  }
}
```

`jest --watch` will re-run tests when a file is changed. For more Jest CLI options, please refer to the Jest Docs.

Creating your first test

Your project is now ready to run tests. Create a folder called `__tests__` in your project's root directory.

> 📘 **Nota para Pages Router:** For example, we can add a test to check if the `<Home />` component successfully renders a heading:
>
> ```
> export default function Home() {
>   return <h1>Home</h1>
> }
> ```
>
> ```
> import '@testing-library/jest-dom'
> import { render, screen } from '@testing-library/react'
> import Home from '../pages/index'
>
> describe('Home', () => {
>   it('renders a heading', () => {
>     render(<Home />)
>
>     const heading = screen.getByRole('heading', { level: 1 })
>
>     expect(heading).toBeInTheDocument()
>   })
> })
> ```

For example, we can add a test to check if the `<Page />` component successfully renders a heading:

```
// Archivo: app/page.js
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
// Archivo: __tests__/page.test.jsx
import '@testing-library/jest-dom'
import { render, screen } from '@testing-library/react'
import Page from '../app/page'

describe('Page', () => {
  it('renders a heading', () => {
    render(<Page />)

    const heading = screen.getByRole('heading', { level: 1 })

    expect(heading).toBeInTheDocument()
  })
})
```

Optionally, add a snapshot test to keep track of any unexpected changes in your component:

📘 **Nota para Pages Router:**

```
import { render } from '@testing-library/react'
import Home from '../pages/index'

it('renders homepage unchanged', () => {
  const { container } = render(<Home />)
  expect(container).toMatchSnapshot()
})
```

> **Good to know**: Test files should not be included inside the Pages Router because any files inside the Pages Router are considered routes.

```
// Archivo: __tests__/snapshot.js
import { render } from '@testing-library/react'
import Page from '../app/page'

it('renders homepage unchanged', () => {
  const { container } = render(<Page />)
  expect(container).toMatchSnapshot()
})
```

Running your tests

Then, run the following command to run your tests:

```
# Manejador de paquetes: pnpm
pnpm test
```

```
# Manejador de paquetes: npm
npm run test
```

```
# Manejador de paquetes: yarn
yarn test
```

```
# Manejador de paquetes: bun
bun run test
```

Additional Resources

For further reading, you may find these resources helpful:

- Next.js with Jest example
- Jest Docs
- React Testing Library Docs
- Testing Playground - use good testing practices to match elements.

How to set up Playwright with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/testing/playwright
> 📂 **Sección:** App > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Playwright with Next.js for End-to-End (E2E) Testing.*

Playwright is a testing framework that lets you automate Chromium, Firefox, and WebKit with a single API. You can use it to write **End-to-End (E2E)** testing. This guide will show you how to set up Playwright with Next.js and write your first tests.

Quickstart

The fastest way to get started is to use `create-next-app` with the with-playwright example. This will create a Next.js project complete with Playwright configured.

```
# Manejador de paquetes: pnpm
pnpm create next-app --example with-playwright with-playwright-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example with-playwright with-playwright-app
```

```
# Manejador de paquetes: yarn
yarn create next-app --example with-playwright with-playwright-app
```

```
# Manejador de paquetes: bun
bun create next-app --example with-playwright with-playwright-app
```

Manual setup

To install Playwright, run the following command:

```
# Manejador de paquetes: pnpm
pnpm create playwright
```

```
# Manejador de paquetes: npm
npm init playwright
```

```
# Manejador de paquetes: yarn
yarn create playwright
```

```
# Manejador de paquetes: bun
bun create playwright
```

This will take you through a series of prompts to set up and configure Playwright for your project, including adding a `playwright.config.ts` file. Please refer to the Playwright installation guide for the step-by-step guide.

Creating your first Playwright E2E test

Create two new Next.js pages:

```
// Archivo: app/page.tsx
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
// Archivo: app/about/page.tsx
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>About</h1>
      [Home](/)
    </div>
  )
}
```

📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Home() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
import Link from 'next/link'

export default function About() {
  return (
    <div>
      <h1>About</h1>
      [Home](/)
    </div>
  )
}
```

Then, add a test to verify that your navigation is working correctly:

```
// Archivo: tests/example.spec.ts
import { test, expect } from '@playwright/test'

test('should navigate to the about page', async ({ page }) => {
  // Start from the index page (the baseURL is set via the webServer in the playwright.config.ts)
  await page.goto('http://localhost:3000/')
  // Find an element with the text 'About' and click on it
  await page.click('text=About')
  // The new URL should be "/about" (baseURL is used there)
  await expect(page).toHaveURL('http://localhost:3000/about')
  // The new page should contain an h1 with "About"
  await expect(page.locator('h1')).toContainText('About')
})
```

Good to know: You can use `page.goto("/")` instead of `page.goto("http://localhost:3000/")`, if you add `"baseURL": "http://localhost:3000"` to the `playwright.config.ts` configuration file.

Running your Playwright tests

Playwright will simulate a user navigating your application using three browsers: Chromium, Firefox and Webkit, this requires your Next.js server to be running. We recommend running your tests against your production code to more closely resemble how your application will behave.

Run `npm run build` and `npm run start`, then run `npx playwright test` in another terminal window to run the Playwright tests.

Good to know: Alternatively, you can use the `webServer` feature to let Playwright start the development server and wait until it's fully available.

Running Playwright on Continuous Integration (CI)

Playwright will by default run your tests in the headless mode. To install all the Playwright dependencies, run `npx playwright install-deps`.

You can learn more about Playwright and Continuous Integration from these resources:

- Next.js with Playwright example

- Playwright on your CI provider
- Playwright Discord

How to set up Vitest with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/testing/vitest
> 📁 **Sección:** App > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Vitest with Next.js for Unit Testing.*

Vitest and React Testing Library are frequently used together for **Unit Testing**. This guide will show you how to set up Vitest with Next.js and write your first tests.

> **Good to know:** Since `async` Server Components are new to the React ecosystem, Vitest currently does not support them. While you can still run **unit tests** for synchronous Server and Client Components, we recommend using **E2E tests** for `async` components.

Quickstart

You can use `create-next-app` with the Next.js with-vitest example to quickly get started:

```
# Manejador de paquetes: pnpm
pnpm create next-app --example with-vitest with-vitest-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example with-vitest with-vitest-app
```

```
# Manejador de paquetes: yarn
yarn create next-app --example with-vitest with-vitest-app
```

```
# Manejador de paquetes: bun
bun create next-app --example with-vitest with-vitest-app
```

Manual Setup

To manually set up Vitest, install `vitest` and the following packages as dev dependencies:

```
# Manejador de paquetes: pnpm
# Using TypeScript
pnpm add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom vite-tsconfig-paths
# Using JavaScript
pnpm add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom
```

```
# Manejador de paquetes: npm
# Using TypeScript
npm install -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom vite-tsconfig-paths
# Using JavaScript
npm install -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom
```

```
# Manejador de paquetes: yarn
# Using TypeScript
yarn add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom vite-tsconfig-paths
# Using JavaScript
yarn add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom
```

```
# Manejador de paquetes: bun
# Using TypeScript
bun add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom vite-tsconfig-paths
# Using JavaScript
bun add -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom
```

Create a `vitest.config.mts|js` file in the root of your project, and add the following options:

```
// Archivo: vitest.config.mts switcher
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'
import tsconfigPaths from 'vite-tsconfig-paths'

export default defineConfig({
  plugins: [tsconfigPaths(), react()],
  test: {
    environment: 'jsdom',
  },
})
```

```
// Archivo: vitest.config.js switcher
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
  },
})
```

For more information on configuring Vitest, please refer to the Vitest Configuration docs.

Then, add a `test` script to your `package.json`:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "test": "vitest"
  }
}
```

When you run `npm run test`, Vitest will **watch** for changes in your project by default.

Creating your first Vitest Unit Test

Check that everything is working by creating a test to check if the `<Page />` component successfully renders a heading:

```tsx
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```jsx
// Archivo: app/page.jsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```tsx
// Archivo: __tests__/page.test.tsx switcher
import { expect, test } from 'vitest'
import { render, screen } from '@testing-library/react'
import Page from '../app/page'

test('Page', () => {
  render(<Page />)
  expect(screen.getByRole('heading', { level: 1, name: 'Home' })).toBeDefined()
})
```

```jsx
// Archivo: __tests__/page.test.jsx switcher
import { expect, test } from 'vitest'
import { render, screen } from '@testing-library/react'
import Page from '../app/page'

test('Page', () => {
  render(<Page />)
  expect(screen.getByRole('heading', { level: 1, name: 'Home' })).toBeDefined()
})
```

> **Good to know**: The example above uses the common `__tests__` convention, but test files can also be colocated inside the `app` router.

📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
import Link from 'next/link'

export default function Page() {
  return (
    <div>
      <h1>Home</h1>
      [About](/about)
    </div>
  )
}
```

```
import { expect, test } from 'vitest'
import { render, screen } from '@testing-library/react'
import Page from '../pages/index'

test('Page', () => {
  render(<Page />)
  expect(screen.getByRole('heading', { level: 1, name: 'Home' })).toBeDefined()
})
```

```
import { expect, test } from 'vitest'
import { render, screen } from '@testing-library/react'
import Page from '../pages/index'

test('Page', () => {
  render(<Page />)
  expect(screen.getByRole('heading', { level: 1, name: 'Home' })).toBeDefined()
})
```

Running your tests

Then, run the following command to run your tests:

```
# Manejador de paquetes: pnpm
pnpm test
```

```
# Manejador de paquetes: npm
npm run test
```

```
# Manejador de paquetes: yarn
yarn test
```

```
# Manejador de paquetes: bun
bun run test
```

Additional Resources

You may find these resources helpful:

- Next.js with Vitest example
- Vitest Docs
- React Testing Library Docs

Testing Adapters

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/testing-adapters
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Validate adapters with the Next.js compatibility test harness and custom lifecycle scripts.*

Next.js provides a test harness for validating adapters. Running the end-to-end tests for deployment.

Example GitHub Actions workflow:

```yaml
// Archivo: .github/workflows/test-e2e-deploy.yml
name: test-e2e-deploy

on:
  workflow_dispatch:
    inputs:
      nextjsRef:
        description: 'Next.js repo ref (branch/tag/SHA)'
        default: 'canary'
        type: string
  # schedule:
  #   - cron: '0 2 * * *'

jobs:
  build:
    name: Build Next.js + adapter
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
        with:
          path: adapter

      - uses: actions/checkout@v4
        with:
          repository: vercel/next.js
          ref: ${{ inputs.nextjsRef || 'canary' }}
          path: nextjs
          fetch-depth: 25

      - uses: actions/setup-node@v4
        with: { node-version: '20' }

      - name: Setup pnpm
        run: npm i -g corepack@0.31 && corepack enable

      - name: Install & build Next.js
        working-directory: nextjs
        run: pnpm install && pnpm build && pnpm install

      - name: Install Playwright
        working-directory: nextjs
        run: pnpm playwright install --with-deps chromium

      - name: Build adapter
        working-directory: adapter
        run: pnpm install && pnpm build

      - uses: actions/cache/save@v4
        with:
          path: |
            nextjs
            adapter
            ~/.cache/ms-playwright
          key: build-${{ github.sha }}-${{ github.run_id }}

  test:
    name: Tests (${{ matrix.group }})
    needs: build
    runs-on: ubuntu-latest
    timeout-minutes: 60
    strategy:
      fail-fast: false
      matrix:
        group:
          [
            1/16,
            2/16,
            3/16,
            4/16,
            5/16,
            6/16,
            7/16,
```

```
            8/16,
            9/16,
            10/16,
            11/16,
            12/16,
            13/16,
            14/16,
            15/16,
            16/16,
          ]
    steps:
      - uses: actions/cache/restore@v4
        with:
          path: |
            nextjs
            adapter
            ~/.cache/ms-playwright
          key: build-${{ github.sha }}-${{ github.run_id }}

      - uses: actions/setup-node@v4
        with: { node-version: '20' }

      - name: Setup pnpm
        run: npm i -g corepack@0.31 && corepack enable

      - name: Ensure Playwright browser
        working-directory: nextjs
        run: pnpm playwright install chromium

      - name: Make scripts executable
        run: chmod +x adapter/scripts/e2e-deploy.sh
          adapter/scripts/e2e-logs.sh
          adapter/scripts/e2e-cleanup.sh

      - name: Run deploy tests
        working-directory: nextjs
        env:
          NEXT_TEST_MODE: deploy
          NEXT_E2E_TEST_TIMEOUT: 240000
          NEXT_EXTERNAL_TESTS_FILTERS: test/deploy-tests-manifest.json
          ADAPTER_DIR: ${{ github.workspace }}/adapter
          IS_TURBOPACK_TEST: 1
          NEXT_TEST_JOB: 1
          NEXT_TELEMETRY_DISABLED: 1

          # Change these to your adapter's scripts
          # Keep as-is if the scripts are in the adapter repository `scripts` directory
          NEXT_TEST_DEPLOY_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-deploy.sh
          NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-logs.sh
          NEXT_TEST_CLEANUP_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-cleanup.sh
        run: node run-tests.js --timings -g ${{ matrix.group }} -c 2 --type e2e
```

The test harness looks for these environment variables:

- `NEXT_TEST_DEPLOY_SCRIPT_PATH`: Path to the executable that builds and deploys the isolated test app
- `NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH`: Path to the executable that returns build and runtime logs for that deployment
- `NEXT_TEST_CLEANUP_SCRIPT_PATH`: Path to the optional executable that tears the deployment down after the test run

Custom deploy script contract

The deploy script `NEXT_TEST_DEPLOY_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

The deploy script must follow this contract:

- Exit with a non-zero code on failure.
- Print the deployment URL to `stdout`. This will be used to verify the deployment. Avoid writing anything else to `stdout`.
- Write diagnostic output to `stderr` or to files inside the working directory.

Because the deploy script and logs script run as separate processes, any data you want to use later, such as build IDs or server logs, should be persisted to files inside the working directory.

Example deploy script:

```
// Archivo: scripts/e2e-deploy.sh
#!/usr/bin/env bash
set -euo pipefail

# Install the adapter, build the app, and deploy or start it.
node -e "
const pkg=JSON.parse(require('fs').readFileSync('package.json','utf8'));
pkg.dependencies=pkg.dependencies||{};
pkg.dependencies['adapter']='file:${ADAPTER_DIR}';
require('fs').writeFileSync('package.json',JSON.stringify(pkg,null,2));
" >&2

# Set the adapter path so that the app uses it.
export NEXT_ADAPTER_PATH="${ADAPTER_DIR}/dist/index.js"

# Build the app
pnpm build

# Write any metadata needed later to files in the working directory.
BUILD_ID="$(cat .next/BUILD_ID)"
DEPLOYMENT_ID="my-adapter-local"
# If your adapter enables immutable static assets, set this to "1".
NEXT_SUPPORTS_IMMUTABLE_ASSETS="0"

{
  echo "BUILD_ID: $BUILD_ID"
  echo "DEPLOYMENT_ID: $DEPLOYMENT_ID"
  echo "NEXT_SUPPORTS_IMMUTABLE_ASSETS: $NEXT_SUPPORTS_IMMUTABLE_ASSETS"
} >> .adapter-build.log

# Start or deploy the app. Capture the URL at this point or make the script output the URL to stdout.
provider-cli-to-deploy

# Example URL output:
# echo "http://127.0.0.1:3000"
```

Custom logs script contract

The logs script `NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

Additionally it receives `NEXT_TEST_DIR` and `NEXT_TEST_DEPLOY_URL` as environment variables.

Its output must include lines starting with:

- `BUILD_ID:`
- `DEPLOYMENT_ID:`
- `NEXT_SUPPORTS_IMMUTABLE_ASSETS:`

After those markers, the logs script can print any additional build or server logs that would help debug failures.

```
// Archivo: scripts/e2e-logs.sh
#!/usr/bin/env bash
set -euo pipefail

if [ -f ".adapter-build.log" ]; then
  cat ".adapter-build.log"
fi

if [ -f ".adapter-server.log" ]; then
  echo "=== .adapter-server.log ==="
  cat ".adapter-server.log"
fi
```

One pattern is to have the deploy script write `.adapter-build.log` and `.adapter-server.log`, then have the logs script replay those files so the harness can extract the required markers. This is one option, each platform has different ways to get the logs.

Custom cleanup script contract

The cleanup script `NEXT_TEST_CLEANUP_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

Additionally it receives `NEXT_TEST_DIR` and `NEXT_TEST_DEPLOY_URL` as environment variables.

The cleanup script can be used to clean up any resources created by the deploy script. It runs after the tests have completed.

How to set up analytics

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/analytics
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Measure and track page performance using Next.js*

How to use debugging tools with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/debugging
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to debug your Next.js application with VS Code or Chrome DevTools.*

How to implement internationalization in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/internationalization
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Next.js has built-in support for internationalized routing and language detection. Learn more here.*

Examples

Next.js has built-in support for internationalized (i18n) routing since `v10.0.0`. You can provide a list of locales, the default locale, and domain-specific locales and Next.js will automatically handle the routing.

The i18n routing support is currently meant to complement existing i18n library solutions like `react-intl`, `react-i18next`, `lingui`, `rosetta`, `next-intl`, `next-translate`, `next-multilingual`, `tolgee`, `paraglide-next`, `next-intlayer`, `gt-react` and others by streamlining the routes and locale parsing.

Getting started

To get started, add the `i18n` config to your `next.config.js` file.

Locales are UTS Locale Identifiers, a standardized format for defining locales.

Generally a Locale Identifier is made up of a language, region, and script separated by a dash: `language-region-script`. The region and script are optional. An example:

- `en-US` - English as spoken in the United States
- `nl-NL` - Dutch as spoken in the Netherlands
- `nl` - Dutch, no specific region

If user locale is `nl-BE` and it is not listed in your configuration, they will be redirected to `nl` if available, or to the default locale otherwise. If you don't plan to support all regions of a country, it is therefore a good practice to include country locales that will act as fallbacks.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    // These are all the locales you want to support in
    // your application
    locales: ['en-US', 'fr', 'nl-NL'],
    // This is the default locale you want to be used when visiting
    // a non-locale prefixed path e.g. `/hello`
    defaultLocale: 'en-US',
    // This is a list of locale domains and the default locale they
    // should handle (these are only required when setting up domain routing)
    // Note: subdomains must be included in the domain value to be matched e.g. "fr.example.com".
    domains: [
      {
        domain: 'example.com',
        defaultLocale: 'en-US',
      },
      {
        domain: 'example.nl',
        defaultLocale: 'nl-NL',
      },
      {
        domain: 'example.fr',
        defaultLocale: 'fr',
        // an optional http field can also be used to test
        // locale domains locally with http instead of https
        http: true,
      },
    ],
  },
}
```

Locale Strategies

There are two locale handling strategies: Sub-path Routing and Domain Routing.

Sub-path Routing

Sub-path Routing puts the locale in the url path.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['en-US', 'fr', 'nl-NL'],
    defaultLocale: 'en-US',
  },
}
```

With the above configuration `en-US`, `fr`, and `nl-NL` will be available to be routed to, and `en-US` is the default locale. If you have a `pages/blog.js` the following urls would be available:

- `/blog`
- `/fr/blog`
- `/nl-nl/blog`

The default locale does not have a prefix.

Domain Routing

By using domain routing you can configure locales to be served from different domains:

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['en-US', 'fr', 'nl-NL', 'nl-BE'],
    defaultLocale: 'en-US',

    domains: [
      {
        // Note: subdomains must be included in the domain value to be matched
        // e.g. www.example.com should be used if that is the expected hostname
        domain: 'example.com',
        defaultLocale: 'en-US',
      },
      {
        domain: 'example.fr',
        defaultLocale: 'fr',
      },
      {
        domain: 'example.nl',
        defaultLocale: 'nl-NL',
        // specify other locales that should be redirected
        // to this domain
        locales: ['nl-BE'],
      },
    ],
  },
}
```

For example if you have `pages/blog.js` the following urls will be available:

- `example.com/blog`
- `www.example.com/blog`
- `example.fr/blog`
- `example.nl/blog`
- `example.nl/nl-BE/blog`

Automatic Locale Detection

When a user visits the application root (generally `/`), Next.js will try to automatically detect which locale the user prefers based on the `Accept-Language` header and the current domain.

If a locale other than the default locale is detected, the user will be redirected to either:

- **When using Sub-path Routing:** The locale prefixed path
- **When using Domain Routing:** The domain with that locale specified as the default

When using Domain Routing, if a user with the `Accept-Language` header `fr;q=0.9` visits `example.com`, they will be redirected to `example.fr` since that domain handles the `fr` locale by default.

When using Sub-path Routing, the user would be redirected to `/fr`.

Prefixing the Default Locale

With Next.js 12 and Proxy, we can add a prefix to the default locale with a workaround.

For example, here's a `next.config.js` file with support for a few languages. Note the `"default"` locale has been added intentionally.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['default', 'en', 'de', 'fr'],
    defaultLocale: 'default',
    localeDetection: false,
  },
  trailingSlash: true,
}
```

Next, we can use Proxy to add custom routing rules:

```
// Archivo: proxy.ts
import { NextRequest, NextResponse } from 'next/server'

const PUBLIC_FILE = /\.(.*)$/

export async function proxy(req: NextRequest) {
  if (
    req.nextUrl.pathname.startsWith('/_next') ||
    req.nextUrl.pathname.includes('/api/') ||
    PUBLIC_FILE.test(req.nextUrl.pathname)
  ) {
    return
  }

  if (req.nextUrl.locale === 'default') {
    const locale = req.cookies.get('NEXT_LOCALE')?.value || 'en'

    return NextResponse.redirect(
      new URL(`/${locale}${req.nextUrl.pathname}${req.nextUrl.search}`, req.url)
    )
  }
}
```

This Proxy skips adding the default prefix to API Routes and public files like fonts or images. If a request is made to the default locale, we redirect to our prefix `/en`.

Disabling Automatic Locale Detection

The automatic locale detection can be disabled with:

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    localeDetection: false,
  },
}
```

When `localeDetection` is set to `false` Next.js will no longer automatically redirect based on the user's preferred locale and will only provide locale information detected from either the locale based domain or locale path as described above.

Accessing the locale information

You can access the locale information via the Next.js router. For example, using the `useRouter()` hook the following properties are available:

- `locale` contains the currently active locale.
- `locales` contains all configured locales.
- `defaultLocale` contains the configured default locale.

When prerendering pages with `getStaticProps` or `getServerSideProps`, the locale information is provided in the context provided to the function.

When leveraging `getStaticPaths`, the configured locales are provided in the context parameter of the function under `locales` and the configured defaultLocale under `defaultLocale`.

Transition between locales

You can use `next/link` or `next/router` to transition between locales.

For `next/link`, a `locale` prop can be provided to transition to a different locale from the currently active one. If no `locale` prop is provided, the currently active `locale` is used during client-transitions. For example:

```
import Link from 'next/link'

export default function IndexPage(props) {
  return (
    <Link href="/another" locale="fr">
      To /fr/another
    </Link>
  )
}
```

When using the `next/router` methods directly, you can specify the `locale` that should be used via the transition options. For example:

```
import { useRouter } from 'next/router'

export default function IndexPage(props) {
  const router = useRouter()

  return (
    <div
      onClick={() => {
        router.push('/another', '/another', { locale: 'fr' })
      }}
    >
      to /fr/another
    </div>
  )
}
```

Note that to handle switching only the `locale` while preserving all routing information such as dynamic route query values or hidden href query values, you can provide the `href` parameter as an object:

```
import { useRouter } from 'next/router'
const router = useRouter()
const { pathname, asPath, query } = router
// change just the locale and maintain all other route information including href's query
router.push({ pathname, query }, asPath, { locale: nextLocale })
```

See here for more information on the object structure for `router.push`.

If you have a `href` that already includes the locale you can opt-out of automatically handling the locale prefixing:

```
import Link from 'next/link'

export default function IndexPage(props) {
  return (
    <Link href="/fr/another" locale={false}>
      To /fr/another
    </Link>
  )
}
```

Leveraging the `NEXT_LOCALE` cookie

Next.js allows setting a `NEXT_LOCALE=the-locale` cookie, which takes priority over the accept-language header. This cookie can be set using a language switcher and then when a user comes back to the site it will leverage the locale specified in the cookie when redirecting from `/` to the correct locale location.

For example, if a user prefers the locale `fr` in their accept-language header but a `NEXT_LOCALE=en` cookie is set the `en` locale when visiting `/` the user will be redirected to the `en` locale location until the cookie is removed or expired.

Search Engine Optimization

Since Next.js knows what language the user is visiting it will automatically add the `lang` attribute to the `<html>` tag.

Next.js doesn't know about variants of a page so it's up to you to add the `hreflang` meta tags using `next/head`. You can learn more about `hreflang` in the Google Webmasters documentation.

How does this work with Static Generation?

> Note that Internationalized Routing does not integrate with `output: 'export'` as it does not leverage the Next.js routing layer. Hybrid Next.js applications that do not use `output: 'export'` are fully supported.

Dynamic Routes and `getStaticProps` Pages

For pages using `getStaticProps` with Dynamic Routes, all locale variants of the page desired to be prerendered need to be returned from `getStaticPaths`. Along with the `params` object returned for `paths`, you can also return a `locale` field specifying which locale you want to render. For example:

```
// Archivo: pages/blog/[slug].js
export const getStaticPaths = ({ locales }) => {
  return {
    paths: [
      // if no `locale` is provided only the defaultLocale will be generated
      { params: { slug: 'post-1' }, locale: 'en-US' },
      { params: { slug: 'post-1' }, locale: 'fr' },
    ],
    fallback: true,
  }
}
```

For Automatically Statically Optimized and non-dynamic `getStaticProps` pages, **a version of the page will be generated for each locale**. This is important to consider because it can increase build times depending on how many locales are configured inside `getStaticProps`.

For example, if you have 50 locales configured with 10 non-dynamic pages using `getStaticProps`, this means `getStaticProps` will be called 500 times. 50 versions of the 10 pages will be generated during each build.

To decrease the build time of dynamic pages with `getStaticProps`, use a `fallback` mode. This allows you to return only the most popular paths and locales from `getStaticPaths` for prerendering during the build. Then, Next.js will build the remaining pages at runtime as they are requested.

Automatically Statically Optimized Pages

For pages that are automatically statically optimized, a version of the page will be generated for each locale.

Non-dynamic getStaticProps Pages

For non-dynamic `getStaticProps` pages, a version is generated for each locale like above. `getStaticProps` is called with each `locale` that is being rendered. If you would like to opt-out of a certain locale from being prerendered, you can return `notFound: true` from `getStaticProps` and this variant of the page will not be generated.

```
export async function getStaticProps({ locale }) {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch(`https:// ... /posts?locale=${locale}`)
  const posts = await res.json()

  if (posts.length === 0) {
    return {
      notFound: true,
    }
  }

  // By returning { props: posts }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

Limits for the i18n config

- `locales`: 100 total locales
- `domains`: 100 total locale domain items

> **Good to know**: These limits have been added initially to prevent potential performance issues at build time. You can workaround these limits with custom routing using Proxy in Next.js 12.

How to set up Cypress with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/cypress
> 📁 **Sección:** Pages > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Next.js with Cypress for End-to-End (E2E) and Component Testing.*

Testing

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing
> 📁 **Sección:** Pages > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Next.js with three commonly used testing tools — Cypress, Playwright, Vitest, and Jest.*

How to set up Jest with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/jest
> 📁 **Sección:** Pages > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Next.js with Jest for Unit Testing.*

How to set up Playwright with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/playwright
> 📁 **Sección:** Pages > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Next.js with Playwright for End-to-End (E2E) and Integration testing.*

How to set up Vitest with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/vitest
> 📁 **Sección:** Pages > Guides > Testing
> 📝 **Descripción:** *Learn how to set up Next.js with Vitest and React Testing Library - two popular unit testing libraries.*

Testing Adapters

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/testing-adapters
> 📁 **Sección:** Pages > Api Reference > Adapters
> 📝 **Descripción:** *Validate adapters with the Next.js compatibility test harness and custom lifecycle scripts.*

Capítulo 10: Despliegue, Rendimiento en Producción y Turbopack

> *Compilación con Turbopack, despliegue en Vercel, Docker y servidores propios, CI build caching y servidor personalizado.*

Deploying

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/getting-started/deploying
> 📁 **Sección:** App > Getting Started
> 📝 **Descripción:** *Learn how to deploy your Next.js application.*

Next.js can be deployed as a Node.js server, Docker container, static export, or adapted to run on different platforms.

Deployment Option	Feature Support
Node.js server	All
Docker container	All
Static export	Limited
Adapters	Varies (verified adapters run the test suite)

Node.js server

Next.js can be deployed to any provider that supports Node.js. Ensure your `package.json` has the `"build"` and `"start"` scripts:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  }
}
```

Then, run `npm run build` to build your application and `npm run start` to start the Node.js server. This server supports all Next.js features. If needed, you can also eject to a custom server.

Node.js deployments support all Next.js features. Learn how to configure them for your infrastructure.

Templates

- Flightcontrol
- Railway
- Replit
- Hostinger

Docker

Next.js can be deployed to any provider that supports Docker containers. This includes container orchestrators like Kubernetes or a cloud provider that runs Docker. For best practices on containerizing your app, refer to Docker's official Next.js and React.js guides.

Docker deployments support all Next.js features. Learn how to configure them for your infrastructure.

> **Note for development:** While Docker is excellent for production deployments, consider using local development (`npm run dev`) instead of Docker during development on Mac and Windows for better performance. Learn more about optimizing local development.

Templates

The following examples demonstrate best practices for containerizing Next.js applications:

- Docker Standalone Output - Deploy a Next.js application using `output: "standalone"` to generate a minimal, production-ready Docker image with only the required runtime files and dependencies.
- Docker Export Output - Deploy a fully static Next.js application using `output: "export"` to generate optimized HTML files that can be served from a lightweight container or any static hosting environment.
- Docker Multi-Environment - Manage separate Docker configurations for development, staging, and production environments with different environment variables.

Additionally, hosting providers offer guidance on deploying Next.js:

- DigitalOcean

- Fly.io
- Google Cloud Run
- Render
- SST

Static export

Next.js enables starting as a static site or Single-Page Application (SPA), then later optionally upgrading to use features that require a server.

Since Next.js supports static exports, it can be deployed and hosted on any web server that can serve HTML/CSS/JS static assets. This includes tools like AWS S3, Nginx, or Apache.

Running as a static export **does not** support Next.js features that require a server. Learn more.

Templates

- GitHub Pages

Adapters

Next.js can be adapted to run on different platforms to support their infrastructure capabilities. The Deployment Adapter API lets platforms customize how Next.js applications are built and deployed.

Verified Adapters

Verified adapters are open source, run the full Next.js compatibility test suite, and are hosted under the Next.js GitHub organization. The Next.js team coordinates testing with these platforms before major releases. Publicly visible test results for each adapter are coming soon. Learn more about verified adapters.

- Vercel
- Bun

Cloudflare and Netlify are working on verified adapters built on the Adapter API. In the meantime, they offer their own Next.js integrations (see below).

Other Platforms

The following platforms offer their own Next.js integrations. These are not built on the public Adapter API and are not verified by the Next.js team, so feature support and compatibility may vary. Refer to each provider's documentation for details:

- Appwrite Sites
- AWS Amplify Hosting
- Cloudflare
- Deno Deploy
- Firebase App Hosting
- Netlify

For details on which Next.js features require specific platform capabilities, see Deploying to Platforms.

Building your application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/building
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Learn what happens when you run next build and how to read the build output.*

Running `next build` compiles your application for production. It bundles and optimizes your code, prerenders every route it can, and prints a route table that shows how each URL is served.

This guide will show you how to read the build output, including the route table and prerender-blocking errors.

> The output and errors on this page show Cache Components enabled, set with `cacheComponents: true` in your `next.config.ts` file. Much of the guide applies without it, but the route table symbols and the prerender-blocking errors differ. See Enabling Cache Components for setup.

What `next build` does

When you run `next build`, the build moves through these phases:

1. **Setup.** Loads environment variables (`.env` files), validates your `next.config`, and generates a build ID.
2. **Route discovery.** Scans the `app/` and `pages/` directories for routes, and detects root-level convention files like `proxy` and `instrumentation`. Generates TypeScript route definitions.
3. **Compilation.** Bundles client, server, and edge code with Turbopack (or webpack). Transpiles TypeScript and JSX, tree-shakes unused code, and optimizes CSS and fonts. Type checking runs in parallel.
4. **Static analysis.** Classifies each route for prerendering versus on-demand rendering. Collects `generateStaticParams` output. Checks for prerender-blocking errors.
5. **Prerendering.** Prerenders static pages and PPR shells to HTML. Generates RSC payloads for client-side navigation.
6. **Output.** Writes the build to `.next/`. For `output: 'standalone'`, bundles only the files needed at runtime. For `output: 'export'`, generates a full static site. Prints the route table.

Reading the build output

The build command depends on your package manager:

```
# Manejador de paquetes: pnpm
pnpm build
```

```
# Manejador de paquetes: npm
npm run build
```

```
# Manejador de paquetes: yarn
yarn build
```

```
# Manejador de paquetes: bun
bun run build
```

After a successful build, Next.js prints a route table with a symbol next to each route, showing how it is served:

Symbol	Name	Behavior
○	Static	Fully prerendered at build time. Served without server rendering.
◐	Partial Prerender	Static shell served immediately. Dynamic content streams in at request time.
●	SSG	Prerendered static HTML (from `generateStaticParams` or `getStaticProps`).
ƒ	Dynamic	Server-rendered on demand for each request.

`○` Static, `●` SSG, and `ƒ` Dynamic are the modes Next.js applications have always rendered in, including on the Pages Router: each route is either prerendered to static HTML or server-rendered on demand for each request.

Cache Components adds ◐ Partial Prerender and makes Partial Prerendering the default rendering model. Each route is split into a static shell, prerendered at build time, and dynamic parts that stream in at request time. Routes sit on a spectrum from fully static (○) to partially prerendered (◐) and are no longer fully server-rendered: a route shows ƒ only when it has nothing to prerender, such as Route Handlers that depend on the request, Proxy (Middleware), and dynamic metadata like `icon` or `opengraph-image`.

The symbol reflects what the route does at prerender time, not which validation configs (such as `instant`) it exports. Without Cache Components, or on the Pages Router, prerendered pages can also show ● (SSG).

Example

Prerender validation is one of the most common reasons production builds fail. This is a safeguard: Cache Components catches uncached and runtime data at build time, before it can slow down your app in production. The rest of this guide walks through one such error, from the failed build to the fix.

This example uses a store app with a dynamic product page that loads data by id:

```
app/
├── layout.tsx
├── page.tsx
└── products/
    └── [id]/
        └── page.tsx
```

We'll work in `app/products/[id]/page.tsx`:

```
// Archivo: app/products/[id]/page.tsx switcher
export default async function Page(props: PageProps<'/products/[id]'>) {
  const { id } = await props.params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

```
// Archivo: app/products/[id]/page.js switcher
export default async function Page({ params }) {
  const { id } = await params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

During the build, Next.js prerenders as much of each route as it can at build time. Runtime data (`params` , `searchParams` , `cookies()` , `headers()`) and uncached data (`fetch()` , database calls) are not available during that pass. Wrap runtime reads in `<Suspense>` , and cache data accesses with `use cache` or wrap them in `<Suspense>` too. Otherwise, the build fails with a `blocking-prerender-runtime` or `blocking-prerender-dynamic` error. Random values and timestamps such as `Math.random()` or `new Date()` can also fail the build.

Run `next build` . It fails with a prerender-blocking error because the `params` read and the uncached `fetch` run outside a `<Suspense>` boundary:

```
// Archivo: Terminal
Error: Route "/products/[id]": Next.js encountered uncached or runtime data during prerendering.

`fetch( ... )`, `cookies()`, `headers()`, `params`, `searchParams`, or `connection()` accessed outside of ``<Suspense>``
prevents the route from being prerendered, blocking the page load and leading to a slower user experience.

Ways to fix this:
  - [stream] Provide a placeholder with ``<Suspense>`` around the data access
  - [cache] For uncached data (`fetch`, database calls): cache the access with `"use cache"` (does not apply to
`connection()`)
  - [block] Set `export const instant = false` to allow a blocking route

Learn more: https://nextjs.org/docs/messages/blocking-prerender-dynamic
    at body (<anonymous>)
    at html (<anonymous>)
To get a more detailed stack trace and pinpoint the issue, try one of the following:
  - Start the app in development mode by running `next dev`, then open "/products/[id]" in your browser to investigate the
error.
  - Rerun the production build with `next build --debug-prerender` to generate better stack traces.
Error occurred prerendering page "/products/[id]".
```

Debugging build errors

Sometimes the error cannot provide enough information, because production builds minify server code and don't generate source maps for it. Rerun the build with `--debug-prerender`:

```
// Archivo: Terminal
next build --debug-prerender
```

That disables minification, turns on source maps for server bundles, and continues past the first failure, so remaining issues surface in one run. This helps with any error thrown during prerendering, not only blocking ones. The stack now points at the first blocked access in the page, the `params` read:

```
// Archivo: Terminal
Error: Route "/products/[id]": Next.js encountered uncached or runtime data during prerendering.
   ...
    at Page (app/products/[id]/page.tsx:2:30)
  1 | export default async function Page(props: PageProps<'/products/[id]'>) {
> 2 |   const { id } = await props.params
    |                              ^
  3 |   const res = await fetch(`https://api.example.com/products/${id}`)
  4 |   const product = await res.json()
```

> **Warning**: Do not deploy builds produced with `--debug-prerender`. They skip optimizations you need in production.

Running the route in `next dev` shows the full error immediately, with the component and line already resolved. See Ensuring instant navigations for working with these errors in depth.

With the cause located, let's apply each fix and see how the build output changes.

A streaming route

The error's first suggestion, `[stream]`, is to provide a placeholder around the data access. Add a `loading.js` file to wrap the whole segment in a `<Suspense>` boundary. Next.js prerenders the fallback as the route's static shell, while the params and data work runs at request time. For other boundary placements, see Streaming:

```
// Archivo: app/products/[id]/loading.tsx switcher
export default function Loading() {
  return <div>Loading ... </div>
}
```

```
// Archivo: app/products/[id]/loading.js switcher
export default function Loading() {
  return <div>Loading ... </div>
}
```

Run the build. It passes, and the route is partially prerendered:

```
// Archivo: Terminal
Route (app)
┌ ○ /                    # Prerendered at build time
├ ○ /_not-found
└   /products/[id]
  └ ◐ /products/[id]     # Shell prerendered, content streams on request

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

Product URLs now serve the prerendered shell instantly, then stream the page content.

Building specific routes

If we were iterating on this route alone in a larger app, we could also scope each build to it instead of rebuilding everything. Pass `--debug-build-paths` with the route files to include:

```
// Archivo: Terminal
next build --debug-build-paths="app/products/[id]/page.tsx"
```

Next.js compiles and prerenders only the matching routes, and skips the rest of the app. The option accepts comma-separated paths and glob patterns, with a `!` prefix to exclude, and combines with `--debug-prerender` while debugging. Our store only has a few routes, so we'll keep building the whole app.

Prerendered params

The dynamic segment shows a single fallback row because Next.js doesn't know which param values exist. Export `generateStaticParams` to list them:

```
// Archivo: app/products/[id]/page.tsx switcher
export async function generateStaticParams() {
  const res = await fetch('https://api.example.com/products')
  const products = await res.json()
  return products.map((product) => ({ id: product.id }))
}

export default async function Page(props: PageProps<'/products/[id]'>) {
  const { id } = await props.params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

```
// Archivo: app/products/[id]/page.js switcher
export async function generateStaticParams() {
  const res = await fetch('https://api.example.com/products')
  const products = await res.json()
  return products.map((product) => ({ id: product.id }))
}

export default async function Page({ params }) {
  const { id } = await params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

Running the build again adds a row for each listed param, indented under the `/products/[id]` route:

```
// Archivo: Terminal
Route (app)
┌ ○ /
├ ○ /_not-found
└   /products/[id]
  ├ ◐ /products/[id]
  ├ ◐ /products/1       # Params known, data still uncached
  ├ ◐ /products/2       # Params known, data still uncached
  └ ◐ /products/3       # Params known, data still uncached

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

> **Good to know**: `generateStaticParams` must return at least one param. An empty array causes a build error.

The new rows show `◐`, not `○`. Listing the params tells Next.js which pages exist, but their content still comes from the uncached lookup, which only runs at request time. Each page serves the static shell and streams its content.

> **Good to know**: Here the `fetch` runs during prerendering, so point it at a working endpoint or a fake async source that returns data. The `◐` rows appear only when the lookup is real uncached I/O; a synchronous in-memory value is treated as static and prerenders each param to `○` instead.

Cached data

The error's `[cache]` fix moves the lookup to build time. Add `use cache` so it can run during prerendering:

```
// Archivo: app/products/[id]/page.tsx switcher
export async function generateStaticParams() {
  const res = await fetch('https://api.example.com/products')
  const products = await res.json()
  return products.map((product) => ({ id: product.id }))
}

async function getProduct(id: string) {
  'use cache'
  const res = await fetch(`https://api.example.com/products/${id}`)
  return res.json()
}

export default async function Page(props: PageProps<'/products/[id]'>) {
  const { id } = await props.params
  const product = await getProduct(id)
  return <div>{product.name}</div>
}
```

```
// Archivo: app/products/[id]/page.js switcher
export async function generateStaticParams() {
  const res = await fetch('https://api.example.com/products')
  const products = await res.json()
  return products.map((product) => ({ id: product.id }))
}

async function getProduct(id) {
  'use cache'
  const res = await fetch(`https://api.example.com/products/${id}`)
  return res.json()
}

export default async function Page({ params }) {
  const { id } = await params
  const product = await getProduct(id)
  return <div>{product.name}</div>
}
```

Running the build again shows the listed params as `○`. Their params are known and their data is cached, so Next.js prerenders each page in full. The `◐` row remains for unlisted params, which serve the static shell while their content streams:

```
// Archivo: Terminal
Route (app)
┌ ○ /
├ ○ /_not-found
└   /products/[id]
  ├ ◐ /products/[id]    # Unlisted params stream on demand
  ├ ○ /products/1       # Prerendered in full at build time
  ├ ○ /products/2       # Prerendered in full at build time
  └ ○ /products/3       # Prerendered in full at build time

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

Pages that read `cookies()`, `headers()`, or `searchParams` stay `◐`, streaming those parts into the shell on each visit.

When a user visits an unlisted param, Next.js serves the static shell immediately while the content streams in, then upgrades the page in the background. See Incremental Static Regeneration with Cache Components.

When there are more prerendered paths than fit in the table, Next.js prints a short list ending with `[+N more paths]`.

Routes that contain cached functions or components also show `Revalidate` and `Expire` columns. A route reports the shortest `revalidate` and `expire` across all the caches it contains. This applies even without an explicit `cacheLife` call, since caches use the `default` profile. Let's add a `/products` listing page that renders the same cached product data:

```
// Archivo: Terminal
Route (app)           Revalidate  Expire
┌ ○ /
├ ○ /_not-found
├ ○ /products               15m      1y
└   /products/[id]
  ├ ◐ /products/[id]
  ├ ○ /products/1           15m      1y
  ├ ○ /products/2           15m      1y
  └ ○ /products/3           15m      1y
```

The routes revalidate after 15 minutes, the value of the `default` profile. The profile never expires, and the `Expire` column caps at one year, shown as `1y`. The table doesn't show which cache produced the values, so with multiple caches in a route, check their `cacheLife` calls.

A blocking route

The error's last suggestion, `[block]`, is the alternative to the fixes above. Say we had removed them, and only set `instant = false` on the original page. This doesn't change how the route renders. It only opts out of validation to purposely allow a blocking route:

```
// Archivo: app/products/[id]/page.tsx switcher
export const instant = false

export default async function Page(props: PageProps<'/products/[id]'>) {
  const { id } = await props.params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

```
// Archivo: app/products/[id]/page.js switcher
export const instant = false

export default async function Page({ params }) {
  const { id } = await params
  const res = await fetch(`https://api.example.com/products/${id}`)
  const product = await res.json()
  return <div>{product.name}</div>
}
```

The build passes, and the output looks the same as the streaming route. Dynamic segments show a `◐` fallback row even when nothing meaningful is prerendered:

```
// Archivo: Terminal
Route (app)
┌ ○ /
├ ○ /_not-found
└   /products/[id]
  └ ◐ /products/[id]    # Renders on demand, blocking on the lookup

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

Unlike the streaming route, there is no fallback UI: users see nothing until the lookup completes. See Opting out for when this is appropriate.

Next steps

We've learned how to run a production build, read the route table, and understand prerender-blocking errors. Next, learn how to:

- Deploy your application
- Cache data and UI
- Revalidate cached routes with ISR
- Configure CI build caching

How to configure Continuous Integration (CI) build caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/ci-build-caching
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Learn how to configure CI to cache Next.js builds*

To improve build performance, Next.js saves a cache to `.next/cache` that is shared between builds.

To take advantage of this cache in Continuous Integration (CI) environments, your CI workflow will need to be configured to correctly persist the cache between builds.

> If your CI is not configured to persist `.next/cache` between builds, you may see a No Cache Detected error.

Here are some example cache configurations for common CI providers:

Vercel

Next.js caching is automatically configured for you. There's no action required on your part. If you are using Turborepo on Vercel, learn more here.

CircleCI

Edit your `save_cache` step in `.circleci/config.yml` to include `.next/cache`:

```
steps:
  - save_cache:
      key: dependency-cache-{{ checksum "yarn.lock" }}
      paths:
        - ./node_modules
        - ./.next/cache
```

If you do not have a `save_cache` key, please follow CircleCI's documentation on setting up build caching.

Travis CI

Add or merge the following into your `.travis.yml`:

```
cache:
  directories:
    - $HOME/.cache/yarn
    - node_modules
    - .next/cache
```

GitLab CI

Add or merge the following into your `.gitlab-ci.yml`:

```
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .next/cache/
```

Netlify CI

Use Netlify Plugins with `@netlify/plugin-nextjs`.

AWS CodeBuild

Add (or merge in) the following to your `buildspec.yml`:

```
cache:
  paths:
    - 'node_modules/**/*' # Cache `node_modules` for faster `yarn` or `npm i`
    - '.next/cache/**/*' # Cache Next.js for faster application rebuilds
```

GitHub Actions

Using GitHub's actions/cache, add the following step in your workflow file:

```
uses: actions/cache@v4
with:
  # See here for caching with `yarn`, `bun` or other package managers https://github.com/actions/cache/blob/main/examples.md
or you can leverage caching with actions/setup-node https://github.com/actions/setup-node
  path: |
    ~/.npm
    ${{ github.workspace }}/.next/cache
  # Generate a new cache whenever packages or source files change.
  key: ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-${{ hashFiles('**/*.js', '**/*.jsx', '**/*.ts',
'**/*.tsx') }}
  # If source files changed but packages didn't, rebuild from a prior cache.
  restore-keys: |
    ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-
```

Bitbucket Pipelines

Add or merge the following into your `bitbucket-pipelines.yml` at the top level (same level as `pipelines`):

```
definitions:
  caches:
    nextcache: .next/cache
```

Then reference it in the `caches` section of your pipeline's `step`:

```
- step:
    name: your_step_name
    caches:
      - node
      - nextcache
```

Heroku

Using Heroku's custom cache, add a `cacheDirectories` array in your top-level package.json:

```
"cacheDirectories": [".next/cache"]
```

Azure Pipelines

Using Azure Pipelines' Cache task, add the following task to your pipeline yaml file somewhere prior to the task that executes `next build`:

```
- task: Cache@2
  displayName: 'Cache .next/cache'
  inputs:
    key: next | $(Agent.OS) | yarn.lock
    path: '$(System.DefaultWorkingDirectory)/.next/cache'
```

Jenkins (Pipeline)

Using Jenkins' Job Cacher plugin, add the following build step to your `Jenkinsfile` where you would normally run `next build` or `npm install`:

```
stage("Restore npm packages") {
    steps {
        // Writes lock-file to cache based on the GIT_COMMIT hash
        writeFile file: "next-lock.cache", text: "$GIT_COMMIT"

        cache(caches: [
            arbitraryFileCache(
                path: "node_modules",
                includes: "**/*",
                cacheValidityDecidingFile: "package-lock.json"
            )
        ]) {
            sh "npm install"
        }
    }
}
stage("Build") {
    steps {
        // Writes lock-file to cache based on the GIT_COMMIT hash
        writeFile file: "next-lock.cache", text: "$GIT_COMMIT"

        cache(caches: [
            arbitraryFileCache(
                path: ".next/cache",
                includes: "**/*",
                cacheValidityDecidingFile: "next-lock.cache"
            )
        ]) {
            // aka `next build`
            sh "npm run build"
        }
    }
}
```

How to set up a custom server in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/custom-server
> 📁 **Sección:** App > Guides
> 📝 **Descripción:** *Start a Next.js app programmatically using a custom server.*

Next.js includes its own server with `next start` by default. If you have an existing backend, you can still use it with Next.js (this is not a custom server). A custom Next.js server allows you to programmatically start a server for custom patterns. The majority of the time, you will not need this approach. However, it's available if you need to eject.

> **Good to know**:
>
> - Before deciding to use a custom server, keep in mind that it should only be used when the integrated router of Next.js can't meet your app requirements.
> - When using standalone output mode, it does not trace custom server files. This mode outputs a separate minimal `server.js` file, instead. These cannot be used together.

Take a look at the following example of a custom server:

```
// Archivo: server.ts switcher
import { createServer } from 'http'
import next from 'next'

const port = parseInt(process.env.PORT || '3000', 10)
const dev = process.env.NODE_ENV !== 'production'
const app = next({ dev })
const handle = app.getRequestHandler()

app.prepare().then(() => {
  createServer((req, res) => {
    handle(req, res)
  }).listen(port)

  console.log(
    `> Server listening at http://localhost:${port} as ${
      dev ? 'development' : process.env.NODE_ENV
    }`
  )
})
```

```
// Archivo: server.js switcher
import { createServer } from 'http'
import next from 'next'

const port = parseInt(process.env.PORT || '3000', 10)
const dev = process.env.NODE_ENV !== 'production'
const app = next({ dev })
const handle = app.getRequestHandler()

app.prepare().then(() => {
  createServer((req, res) => {
    handle(req, res)
  }).listen(port)

  console.log(
    `> Server listening at http://localhost:${port} as ${
      dev ? 'development' : process.env.NODE_ENV
    }`
  )
})
```

Set your own response headers before calling `handle`: Next.js begins sending the response inside `handle(req, res)`, so by the time its promise resolves `res.headersSent` is already `true` and any later `res.setHeader()` call is silently discarded. Set headers you own, such as `Set-Cookie`, before handing the request to Next.js:

```
res.setHeader('Set-Cookie', 'sessionId=abc123; Max-Age=2592000')

await handle(req, res)
```

The same ordering applies when wrapping Next.js in another framework, for example `reply.raw` in Fastify or `res` in Express.

`server.js` does not run through the Next.js Compiler or bundling process. Make sure the syntax and source code this file requires are compatible with the current Node.js version you are using. View an example.

To run the custom server, you'll need to update the `scripts` in `package.json` like so:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "node server.js",
    "build": "next build",
    "start": "NODE_ENV=production node server.js"
  }
}
```

Alternatively, you can set up `nodemon` (example). The custom server uses the following import to connect the server with the Next.js application:

```
import next from 'next'

const app = next({})
```

The above `next` import is a function that receives an object with the following options:

Option	Type	Description
`conf`	`Object`	The same object you would use in `next.config.js`. Defaults to `{}`
`dev`	`Boolean`	(*Optional*) Whether or not to launch Next.js in dev mode. Defaults to `false`
`dir`	`String`	(*Optional*) Location of the Next.js project. Defaults to `'.'`
`quiet`	`Boolean`	(*Optional*) Hide error messages containing server information. Defaults to `false`
`hostname`	`String`	(*Optional*) The hostname the server is running behind
`port`	`Number`	(*Optional*) The port the server is running behind
`httpServer`	`node:http#Server`	(*Optional*) The HTTP Server that Next.js is running behind
`turbopack`	`Boolean`	(*Optional*) Enable Turbopack (enabled by default)
`webpack`	`Boolean`	(*Optional*) Enable webpack

The returned `app` can then be used to let Next.js handle requests as required.

> 📘 **Nota para Pages Router:**

Disabling file-system routing

By default, `Next` will serve each file in the `pages` folder under a pathname matching the filename. If your project uses a custom server, this behavior may result in the same content being served from multiple paths, which can present problems with SEO and UX.

To disable this behavior and prevent routing based on files in `pages`, open `next.config.js` and disable the `useFileSystemPublicRoutes` config:

```
module.exports = {
  useFileSystemPublicRoutes: false,
}
```

> Note that `useFileSystemPublicRoutes` disables filename routes from SSR; client-side routing may still access those paths. When using this option, you should guard against navigation to routes you do not want programmatically.

> You may also wish to configure the client-side router to disallow client-side redirects to filename routes; for that refer to `router.beforePopState`.

Deploying Next.js to different platforms

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/guides/deploying-to-platforms
> 📂 **Sección:** App > Guides
> 📝 **Descripción:** *Understand which Next.js features require specific platform capabilities and how to choose the right deployment target.*

Next.js treats static and dynamic content as a spectrum at the component level. Different features in this model require different platform capabilities. This page helps you understand what your platform needs to support and how to configure your deployment.

Minimum Requirements

To run Next.js, your platform needs **a Node.js server**. That's it.

A single `next start` process handles every Next.js feature correctly: Server Components, ISR, PPR, Cache Components, Server Actions, Proxy, and `after()`. Streaming support is needed for features like PPR and Server Components to deliver content progressively (without it, responses are buffered and sent as a whole, which still works but loses the streaming performance benefit). Additional infrastructure (CDN caching, edge compute, shared cache) primarily improves performance and multi-instance consistency. In multi-instance deployments, shared cache and tag coordination reduce stale divergence between instances. The only additional dependency is the `sharp` package, which is required for Image Optimization.

Functional Fidelity vs. Performance Fidelity

When evaluating platform support for Next.js, it helps to distinguish between two levels:

Functional fidelity means every Next.js feature works correctly. The adapter test suite is the contract: if a platform's adapter passes the tests, it supports Next.js. This is binary: it passes or it doesn't.

Performance fidelity means features achieve their optimal performance characteristics. Examples include PPR's static shell served at CDN latency rather than origin latency, or ISR serving stale content instantly with sub-second revalidation propagation. Performance fidelity is a spectrum that every platform will achieve differently based on their architecture.

A platform that achieves functional fidelity is a fully supported deployment target for Next.js. Performance fidelity is how platforms differentiate, and it improves incrementally over time.

Use the feature matrix below through this lens: "Streaming Required" and "Shared Cache Recommended" describe what is needed for functional fidelity. "Edge Stitching" is a performance fidelity optimization.

Feature Support Matrix

Different features require different infrastructure capabilities. The "Edge Stitching" column is a **performance optimization**, not a correctness requirement. All features work correctly from a single origin server.

Feature	Streaming	Shared Cache	Edge Stitching	Notes
Server Components	Required	No	No	Basic streaming support
ISR (time-based)	No	Recommended	No	Works per-instance without shared cache
ISR (on-demand)	No	Recommended	No	Tag propagation needs shared cache for multi-instance
Partial Prerendering	Required	Recommended	Optional	See PPR Platform Guide
Cache Components (`use cache`)	Required	Recommended	No	Shared cache enables cross-instance consistency
Proxy / Middleware	No	No	No	Runs at edge or origin
Server Actions	Required	No	No	POST requests with streaming response
`after()`	No	No	No	Requires graceful shutdown support

Streaming Required means the platform must support chunked transfer encoding or HTTP/2 streaming and must not buffer the response before sending it to the client.

Shared Cache Recommended means multiple server instances benefit from shared cache backends to coordinate. For ISR and server response caching, use `cacheHandler`. For `'use cache'` entries, use `cacheHandlers`. Without shared cache, each instance maintains its own cache independently — features still work correctly on each instance, but revalidation events don't propagate across instances.

CDN Infrastructure Compatibility

The following table maps infrastructure primitives for each major CDN. These are available building blocks, not finished integrations:

CDN	Edge Compute	Key-Value / Tags	Blob Storage	PPR Resuming
Cloudflare	Workers	KV	R2	Yes (worker)
Akamai	EdgeWorkers	EdgeKV	Object Storage	Yes (worker)
Amazon CloudFront	Lambda@Edge	KeyValueStore	S3	Yes (Lambda)
Fastly	Compute	KV Store	Object Storage	Yes (WASM)
Azure	Functions	Managed Redis	Blob Storage	Yes (server)
Google Cloud	Cloud Run	Various KV	Cloud Storage	Yes (server)

These are available building blocks, not finished integrations. Most community adapters today deploy Next.js as a Docker container or Node.js server without leveraging CDN-specific primitives like edge KV or PPR resuming. See the Deploying page for the current list of adapters. For CDN-specific caching considerations (including known limitations with `Vary` on custom headers), see CDN Caching.

Adapters

Next.js provides a Deployment Adapter API that lets platforms customize how Next.js applications are built and deployed for their infrastructure. Adapters run at build time and produce platform-specific output from the standard Next.js build. Anyone can build an adapter using the public API with no special access required.

The adapter API plus Next.js caching interfaces form the complete platform integration surface. The adapter handles build-time output, while `cacheHandler` and `cacheHandlers` cover different runtime caching paths. `cacheHandler` (singular) covers server cache paths like ISR, route handlers, patched `fetch` / `unstable_cache`, and image optimization. `cacheHandlers` (plural) configures `'use cache'` directive backends.

Verified Adapters

A **verified adapter** is one that meets two requirements:

1. **Open source.** The adapter source code is publicly available so the community and the Next.js team can inspect, contribute to, and verify it.
2. **Runs the compatibility test suite.** The platform provides a way to run the full Next.js compatibility test suite against their adapter. This gives visibility into which features work, which are in progress, and where gaps remain.

Verified adapters are hosted under the Next.js GitHub organization, listed as supported deployment targets in the Next.js documentation, and maintained by their respective platform teams. There are no private framework hooks or integration paths: Vercel's adapter uses the same public API as every other adapter.

For verified adapters and platforms working toward verified status through the Ecosystem Working Group, the Next.js team commits to:

- **Coordinated testing.** Before major releases, we work with platform teams to run the compatibility test suite and surface issues early.
- **Early access.** Adapter authors receive early access to API changes during RFCs and release candidates.
- **Direct support.** When the adapter contract needs updating, we work directly with adapter teams.

> **Good to know:** Platforms can build closed-source adapters on the same public API and test suite. However, closed-source adapters will not be listed as verified, since the Next.js team cannot verify what it cannot inspect.

A Note on Infrastructure Requirements

Next.js's rendering model places the static/dynamic boundary at the component level rather than the route level. Finer-grained boundaries provide more flexibility for developers at the cost of broader requirements for hosting platforms. This is a deliberate trade-off: the infrastructure requirements on this page exist because of what the rendering model delivers.

turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Next.js with Turbopack-specific options*

The `turbopack` option lets you customize Turbopack to transform different files and change how modules are resolved.

> **Good to know**: The `turbopack` option was previously named `experimental.turbo` in Next.js versions 13.0.0 to 15.2.x, an option which still works as an alias, but new configuration should use the top-level `turbopack` option.
>
> If you are using an older version of Next.js, run `npx @next/codemod@latest next-experimental-turbo-to-turbopack .` to automatically migrate your configuration.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    // ...
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    // ...
  },
}

module.exports = nextConfig
```

> **Good to know**:
>
> - Turbopack for Next.js does not require loaders or loader configuration for built-in functionality. Turbopack has built-in support for CSS and compiling modern JavaScript, so there's no need for `css-loader`, `postcss-loader`, or `babel-loader` if you're using `@babel/preset-env`.

Reference

Options

The following options are available for the `turbopack` configuration:

Option	Description
`root`	Sets the application root directory. Should be an absolute path.
`rules`	List of supported webpack loaders to apply when running with Turbopack.
`resolveAlias`	Map aliased imports to modules to load in their place.
`resolveExtensions`	List of extensions to resolve when importing files.
`debugIds`	Enable generation of debug IDs in JavaScript bundles and source maps.

Supported loaders

The following loaders have been tested to work with Turbopack's webpack loader implementation, but many other webpack loaders should work as well even if not listed here:

- `babel-loader` *(Configured automatically if a Babel configuration file is found)*
- `@svgr/webpack`
- `svg-inline-loader`
- `yaml-loader`
- `string-replace-loader`
- `raw-loader`
- `sass-loader` *(Configured automatically)*
- `graphql-tag/loader`

Missing Webpack loader features

Turbopack uses the `loader-runner` library to execute webpack loaders, which provides most of the standard loader API. However, some features are not supported:

Module loading:

- `importModule` - No support
- `loadModule` - No support

File system and output:

- `fs` - Partial support: only `fs.readFile` is currently implemented.
- `emitFile` - No support

Context properties:

- `version` - No support
- `mode` - No support
- `target` - No support

Utilities:

- `utils` - No support
- `resolve` - No support (use `getResolve` instead)

If you have a loader that is critically dependent upon one of these features please file an issue.

Examples

Root directory

Turbopack uses the root directory to resolve modules. Files outside of the project root are not resolved.

The reason files are not resolved outside of the project root is to improve cache validation, reduce filesystem watching overhead, and reduce the number of resolving steps needed.

Next.js automatically detects the root directory of your project. It does so by looking for one of these files:

- `pnpm-lock.yaml`
- `package-lock.json`
- `yarn.lock`
- `bun.lock`
- `bun.lockb`

If you have a different project structure, for example if you don't use workspaces, you can manually set the `root` option:

```
// Archivo: next.config.js
const path = require('path')
module.exports = {
  turbopack: {
    root: path.join(__dirname, '..'),
  },
}
```

To resolve files from linked dependencies outside the project root (via `npm link`, `yarn link`, `pnpm link`, etc.), you must configure the `turbopack.root` to the parent directory of both the project and the linked dependencies.

While this expands the scope of filesystem watching, it's typically only necessary during development when actively working on linked packages.

Configuring webpack loaders

If you need loader support beyond what's built in, many webpack loaders already work with Turbopack. There are currently some limitations:

- Only a core subset of the webpack loader API is implemented. Currently, there is enough coverage for some popular loaders, and we'll expand our API support in the future.
- Only loaders that return JavaScript code are supported. Loaders that transform files like stylesheets or images are not currently supported.
- Options passed to webpack loaders must be plain JavaScript primitives, objects, and arrays. For example, it's not possible to pass `require()` plugin modules as option values.

To configure loaders, add the names of the loaders you've installed and any options in `next.config.js`, mapping file extensions to a list of loaders. Rules are evaluated in order.

Here is an example below using the `@svgr/webpack` loader, which enables importing `.svg` files and rendering them as React components.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        loaders: ['@svgr/webpack'],
        as: '*.js',
      },
    },
  },
}
```

> **Good to know**: Globs used in the `rules` object match based on file name, unless the glob contains a `/` character, which will cause it to match based on the full project-relative file path. Windows file paths are normalized to use unix-style `/` path separators.
>
> Turbopack uses a modified version of the Rust `globset` library.

For loaders that require configuration options, you can use an object format instead of a string:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        loaders: [
          {
            loader: '@svgr/webpack',
            options: {
              icon: true,
            },
          },
        ],
        as: '*.js',
      },
    },
  },
}
```

> **Good to know**: Prior to Next.js version 13.4.4, `turbopack.rules` was named `turbo.loaders` and only accepted file extensions like `.mdx` instead of `*.mdx`.

Advanced webpack loader conditions

You can further restrict where a loader runs using the advanced `condition` syntax:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      // '*' will match all file paths, but we restrict where our
      // rule runs with a condition.
      '*': {
        condition: {
          all: [
            // 'foreign' is a built-in condition.
            { not: 'foreign' },
            // 'path' can be a RegExp or a glob string. A RegExp matches
            // anywhere in the full project-relative file path.
            { path: /^img\/[0-9]{3}\// },
            {
              any: [
                { path: '*.svg' },
                // 'query' matches anywhere in the full query string,
                // which can be empty, or start with `?`.
                { query: /[?&]svgr(?=&|$)/ },
                // 'content' is always a RegExp, and can match
                // anywhere in the file.
                { content: /\<svg\W/ },
              ],
            },
          ],
        },
        loaders: ['@svgr/webpack'],
        as: '*.js',
      },
    },
  },
}
```

- Supported boolean operators are `{all: [ ... ]}`, `{any: [ ... ]}` and `{not: ... }`.
- Supported customizable operators are `{path: string | RegExp}`, `{content: RegExp}`, `{query: string | RegExp}`, and `{contentType: string | RegExp}`. If multiple operators are specified in the same object, it acts as an implicit `and`.
  - `path` matches against the project-relative file path. A string is treated as a glob pattern, while a RegExp can be used to match the path partially.
  - `content` matches anywhere in the file content.

- `query` matches the import's query string (e.g., `?foo` in `import './file?foo'`). A string must match exactly, while a RegExp can be used to match the query string partially.
- `contentType` matches the MIME content type of the resource (e.g., from data URLs like `data:text/plain, ...`). A string is treated as a glob pattern (e.g., `text/*`, `image/*`), while a RegExp can be used to match the content type partially.

In addition, a number of built-in conditions are supported:

- `browser`: Matches code that will execute on the client. Server code can be matched using `{not: 'browser'}`.
- `foreign`: Matches code in `node_modules`, as well as some Next.js internals. Usually you'll want to restrict loaders to `{not: 'foreign'}`. This can improve performance by reducing the number of files the loader is invoked on.
- `development`: Matches when using `next dev`.
- `production`: Matches when using `next build`.
- `node`: Matches code that will run on the default Node.js runtime.
- `edge-light`: Matches code that will run on the Edge runtime (deprecated).

Rules can be an object or an array of objects. An array is often useful for modeling disjoint conditions:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': [
        {
          condition: 'browser',
          loaders: ['@svgr/webpack'],
          as: '*.js',
        },
        {
          condition: { not: 'browser' },
          loaders: [require.resolve('./custom-svg-loader.js')],
          as: '*.js',
        },
      ],
    },
  },
}
```

> **Good to know**: All matching rules are executed in order.

Module types

You can set the module type directly without using a loader. This is useful for changing how files are processed, similar to webpack's `type` option.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        type: 'asset',
      },
    },
  },
}
```

When using `type: 'asset'`, importing the file returns its URL:

```
// Archivo: app/page.tsx
import svgUrl from './icon.svg'

export default function Page() {
  return <img src={svgUrl} alt="Icon" />
}
```

The `type` option can be combined with `loaders` - loaders run first, then the result is processed according to the specified type.

Available module types:

Type	Description
`asset`	Emit file and return URL (like webpack `asset/resource`)
`ecmascript`	Process as JavaScript
`typescript`	Process as TypeScript
`css`	Process as CSS
`css-module`	Process as CSS module
`wasm`	Process as WebAssembly
`raw`	Return raw contents as string
`bytes`	Inline contents as bytes

Inline loader configuration with import attributes

You can apply a Turbopack loader to an individual import using the `with` clause (import attributes). This is specified per-import rather than globally via `turbopack.rules`.

This is useful when you want to apply a loader to a specific import without affecting all files of that type.

```
// Archivo: app/page.tsx
// Apply a raw loader to import a .txt file as a JavaScript module
import rawText from '../data.txt' with { turbopackLoader: 'raw-loader', turbopackAs: '*.js' }

export default function Page() {
  return <p>{rawText}</p>
}
```

The following import attributes are supported:

Attribute	Description
`turbopackLoader`	The loader to apply (e.g. `'raw-loader'`).
`turbopackLoaderOptions`	JSON string of loader options (e.g. `'{"search":"X","replace":"Y"}'`).
`turbopackAs`	Rename pattern for the output (same as `turbopack.rules[].as`). For example, `'*.js'` treats the loader output as JavaScript.
`turbopackModuleType`	Set the module type for the output (same as `turbopack.rules[].type`).

Loaders with options pass a JSON-encoded string via `turbopackLoaderOptions`:

```
// Archivo: app/page.tsx
import value from '../data.js' with { turbopackLoader: 'string-replace-loader', turbopackLoaderOptions:
'{"search":"PLACEHOLDER","replace":"replaced value"}' }
```

> **Good to know**: Import attributes with `turbopackLoader` are Turbopack-specific and are not supported by webpack. This feature requires the `with` keyword (not `assert`) in your import statements.

Resolving aliases

Turbopack can be configured to modify module resolution through aliases, similar to webpack's `resolve.alias` configuration.

To configure resolve aliases, map imported patterns to their new destination in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      underscore: 'lodash',
      mocha: { browser: 'mocha/browser-entry.js' },
    },
  },
}
```

This aliases imports of the `underscore` package to the `lodash` package. In other words, `import underscore from 'underscore'` will load the `lodash` module instead of `underscore`.

Turbopack also supports conditional aliasing through this field, similar to Node.js' conditional exports. At the moment only the `browser` condition is supported. In the case above, imports of the `mocha` module will be aliased to `mocha/browser-entry.js` when Turbopack targets browser environments.

Resolving custom extensions

Turbopack can be configured to resolve modules with custom extensions, similar to webpack's `resolve.extensions` configuration.

To configure resolve extensions, use the `resolveExtensions` field in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveExtensions: ['.mdx', '.tsx', '.ts', '.jsx', '.js', '.mjs', '.json'],
  },
}
```

This overwrites the original resolve extensions with the provided list. Make sure to include the default extensions.

For more information and guidance for how to migrate your app to Turbopack from webpack, see Turbopack's documentation on webpack compatibility.

Debug IDs

Turbopack can be configured to generate debug IDs in JavaScript bundles and source maps.

To configure debug IDs, use the `debugIds` field in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    debugIds: true,
  },
}
```

The option automatically adds a polyfill for debug IDs to the JavaScript bundle to ensure compatibility. The debug IDs are available in the `globalThis._debugIds` global variable.

Version History

Version	Changes
`16.2.0`	`turbopackLoader` import attributes were added.
`16.2.0`	`turbopack.rules.*.type` was added.
`16.2.0`	`turbopack.rules.*.condition.contentType` was added.
`16.2.0`	`turbopack.rules.*.condition.query` was added.
`16.0.0`	`turbopack.debugIds` was added.
`16.0.0`	`turbopack.rules.*.condition` was added.
`15.3.0`	`experimental.turbo` is changed to `turbopack`.
`13.0.0`	`experimental.turbo` introduced.

turbopackChunking

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackChunking
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure how Turbopack splits your client-side JavaScript into chunks in production.*

`experimental.turbopackChunking` lets you configure Turbopack's production JavaScript chunker. These options allow you to change the assumptions the chunker makes about user behavior, tweak the raw size thresholds it uses, and enable the experimental component chunks feature.

> 📘 **Nota para Pages Router:** `experimental.turbopackChunking` lets you configure Turbopack's production JavaScript chunker. These options allow you to change the assumptions the chunker makes about user behavior and tweak the raw size thresholds it uses.

By default, Turbopack's chunking is configured as follows:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
      minComponentChunkSize: 20000,
      generateComponentChunks: false,
    },
  },
} satisfies NextConfig

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
      minComponentChunkSize: 20000,
      generateComponentChunks: false,
    },
  },
}

module.exports = nextConfig
```

📘 **Nota para Pages Router:**

```
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
    },
  },
} satisfies NextConfig

export default nextConfig
```

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
    },
  },
}

module.exports = nextConfig
```

Size Thresholds

The following options control how aggressively Turbopack merges chunks and how large a chunk is allowed to grow. Sizes are in bytes of uncompressed, unminified code (roughly 5x the size of the compressed, minified output).

- **`minChunkSize`** (default `50000`): Turbopack will avoid creating more than one chunk smaller than this size by merging small chunks into larger ones. Raising this number will produce fewer, larger chunks. Meanwhile, lowering it will produce more chunks that are smaller.
- **`maxChunkCountPerGroup`** (default `40`): Turbopack will not emit more than this many chunks per chunk group (eg. a route or a dynamic import). Lowering this number will lead to more aggressive merging and less network requests per-page. Raising it will produce smaller chunks and increase the likelihood of cache hits when navigating.
- **`maxMergeChunkSize`** (default `200000`): Turbopack never merges a chunk larger than this size with other chunks. This keeps the code in large chunks from being duplicated across multiple large output chunks.

When merging chunks, the trade-off being made is an improvement to performance on initial page loads at the cost of navigation performance. This is because each additional network request being made is costly, however, smaller chunks are more likely to be re-usable across pages.

Component Chunks

Producing component chunks is an experimental feature that aims to give you the initial page load benefits of merged chunks without sacrificing reusability. This feature lets the runtime dynamically choose whether to load a merged chunk as a single file, or to load only the component chunks it doesn't already have. Additionally, chunks that were already loaded as part of a merged chunk will not be re-downloaded.

This avoids re-downloading JavaScript the browser already has when navigating.

- `generateComponentChunks` (default `false`): when enabled, each merged production chunk also emits its constituent component chunks alongside it, so the browser runtime can fetch individual component chunks.
- `minComponentChunkSize` (default `20000`): component chunks smaller than this size are folded into a single component instead of being emitted on their own, to avoid producing many tiny chunks.

Heuristics

These change the assumptions the chunker makes when weighing whether merging two chunks is worth it.

- `firstPageLoadPriority` (a number between `0` and `1`): how heavily to weight the benefit of merging chunks for a single page load. Higher values merge more eagerly. If you don't have a better value, your site's bounce rate is a good approximation.
- `priorityRoutes` (an array of `RegExp`): routes that are often the first page a visitor lands on (e.g. the homepage). Their client-side bundles are merged more eagerly to reduce the single-route request cost, at the cost of extra requests when navigating to other pages.
- `priorityBoost` (default `1.5`): a multiplier on the single-request probability of `priorityRoutes` routes. Higher values merge those routes' bundles more aggressively.
- `requestCost` (default `200000`): the estimated cost of an additional request, in bytes of uncompressed, unminified code. Larger values bias toward fewer, larger chunks and fewer requests overall.

Turbopack FileSystem Caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackFileSystemCache
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to enable and configure FileSystem Caching for Turbopack builds*

Usage

Turbopack FileSystem Cache enables Turbopack to reduce work across `next dev` or `next build` commands. When enabled, Turbopack will save and restore data under the `.next` directory between runs, which can greatly speed up subsequent builds and dev sessions.

Two options control the cache, one for `next dev` and one for `next build`. Both are enabled by default:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackFileSystemCacheForDev: true,
    turbopackFileSystemCacheForBuild: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackFileSystemCacheForDev: true,
    turbopackFileSystemCacheForBuild: true,
  },
}

module.exports = nextConfig
```

Options

- `turbopackFileSystemCacheForDev` (default: `true`): caches Turbopack's work for `next dev` in `.next/dev/cache/turbopack`. Restarting the dev server reuses the previous compilation.
- `turbopackFileSystemCacheForBuild` (default: `true`): caches Turbopack's work for `next build` in `.next/cache/turbopack`. Subsequent builds start warm. See Build environments.

Set either option to `false` to opt out.

Build environments

The build cache lives in `.next/cache`. Builds only get faster when that directory is restored before each build.

- **Self-hosted builds**: reuse the same working directory between builds. Containerized builds start from a clean layer and do not carry `.next/cache` over unless you cache or mount it explicitly.
- **CI providers**: configure build caching for `.next/cache`.

If your build environment never preserves `.next/cache`, set `turbopackFileSystemCacheForBuild: false` to skip writing a cache that will not be read.

Version History

Version	Changes
`v16.3.0`	FileSystem caching is enabled by default for builds
`v16.1.0`	FileSystem caching is enabled by default for development
`v16.0.0`	Beta release with separate flags for build and dev
`v15.5.0`	Persistent caching released as experimental on canary releases

turbopack.ignoreIssue

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackIgnoreIssue
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Suppress specific Turbopack errors and warnings from the CLI output and error overlay.*

The `turbopack.ignoreIssue` option allows you to filter out specific Turbopack errors and warnings so they do not appear in the CLI output or the error overlay. This is useful for suppressing known warnings that do not affect your application, such as intentionally unresolved optional dependencies.

This option is only available when using Turbopack (`next dev --turbopack`).

Usage

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/vendor/**',
      },
    ],
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/vendor/**',
      },
    ],
  },
}

module.exports = nextConfig
```

Options

Each rule in the `ignoreIssue` array is an object with the following fields:

Field	Type	Required	Description
`path`	`string | RegExp`	Yes	Matches against the file path of the issue
`title`	`string | RegExp`	No	Matches against the issue title
`description`	`string | RegExp`	No	Matches against the issue description

An issue is suppressed when it matches the `path` **and** all other specified fields in a rule. If only `path` is provided, any issue from a matching file is suppressed.

> **Good to know:** Issue titles and descriptions may change between Turbopack versions. The `path` field is generally stable, but is not guaranteed to remain consistent for all issue types. When possible, prefer using more specific `path` patterns over `title` or `description` matching.

`path`

A **glob pattern** (when a string) or **regular expression** that matches against the file path where the issue originated.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      // Glob pattern: suppress issues from any file under vendor/
      { path: '**/vendor/**' },
      // RegExp: suppress issues from files matching a pattern
      { path: /node_modules\/legacy-lib/ },
    ],
  },
}
```

`title`

An **exact string match** (when a string) or **regular expression** that matches against the issue title.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/src/**',
        title: 'Module not found',
      },
    ],
  },
}
```

`description`

An **exact string match** (when a string) or **regular expression** that matches against the issue description.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/src/**',
        description: /Cannot find module 'optional-dep'/,
      },
    ],
  },
}
```

Examples

Suppressing warnings for optional dependencies

If your code uses `try/catch` around an optional `require()` call, Turbopack may report a "Module not found" warning. You can suppress it:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/lib/optional-feature/**',
        title: 'Module not found',
      },
    ],
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/lib/optional-feature/**',
        title: 'Module not found',
      },
    ],
  },
}

module.exports = nextConfig
```

Combining multiple rules

You can specify multiple rules to suppress different issues:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      { path: '**/vendor/**' },
      { path: '**/legacy/**', title: 'Module not found' },
      { path: /generated\//, description: /expected identifier/ },
    ],
  },
}
```

Version History

Version	Changes
`v16.2.0`	`turbopack.ignoreIssue` introduced.

turbopackLocalPostcssConfig

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackLocalPostcssConfig
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable per-directory PostCSS config resolution in Turbopack so that the config closest to each CSS file takes precedence over the project root config.*

The `turbopackLocalPostcssConfig` option changes how Turbopack resolves `postcss.config.js` files. When enabled, Turbopack searches for the config starting from the CSS file's own directory first, then falls back to the project root. By default, Turbopack checks the project root first, meaning a root-level `postcss.config.js` always takes precedence over configs in subdirectories.

This option is only relevant when using Turbopack (`next dev` or `next build`).

Usage

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackLocalPostcssConfig: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackLocalPostcssConfig: true,
  },
}

module.exports = nextConfig
```

Behavior

Setting	Config resolution order
`false` (default)	Project root → CSS file's directory
`true`	CSS file's directory → project root

With the default behavior, a `postcss.config.js` at the project root is used for all CSS files, and per-directory configs are only applied if no root config exists. Enabling `turbopackLocalPostcssConfig` reverses this: per-directory configs take precedence, and the root config serves as the fallback.

Example

This is useful for projects that need different PostCSS transforms in different directories, such as a monorepo with multiple apps or design system packages:

```
my-app/
├── postcss.config.js          ← fallback (applied if no local config is found)
├── app/
│   └── page.module.css        ← uses root config
└── packages/
    └── ui/
        ├── postcss.config.js  ← takes precedence for files in this directory
        └── button.module.css  ← uses packages/ui/postcss.config.js
```

Version History

Version	Changes
`v16.3.0`	`turbopackLocalPostcssConfig` introduced.

Turbopack Memory Eviction

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackMemoryEviction
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to control Turbopack's memory eviction strategy for the persistent cache.*

Usage

`turbopackMemoryEviction` controls whether Turbopack reclaims memory while the persistent (FileSystem) cache is enabled. After Turbopack writes a snapshot of its cache to disk, it can 'evict' the in-memory copies of that data and reload them from disk on demand.

Currently there are three options

- `false`: never evict. Cached data stays in memory for the lifetime of the process.
- `'auto'` (default): evict after a snapshot only once enough memory has been allocated since the last eviction to make it worthwhile. Leverages thresholds and memory pressure feedback from the operating system.
- `'full'`: evict all possible data from memory every time we save to disk.

> **Good to know:** This option only has an effect in `next dev` sessions when the FileSystem Cache is enabled, since eviction relies on data already being persisted to disk. It is experimental and under active development.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackMemoryEviction: 'auto',
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackMemoryEviction: 'auto',
  },
}

module.exports = nextConfig
```

Version Changes

Version	Changes
`v16.3.0`	`turbopackMemoryEviction` released as experimental.

turbopackRustReactCompiler

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackRustReactCompiler
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use the native Rust port of the React Compiler with Turbopack instead of the Babel transform.*

The `experimental.turbopackRustReactCompiler` option enables the native Rust version of the React Compiler, running it directly inside Turbopack as native code instead of through Node.js as it does with the standard Babel version. This typically results in a noticeable performance improvement.

This option is released as experimental to gather feedback before it becomes the default.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  // Enable the React Compiler
  reactCompiler: true,
  experimental: {
    // Use the Rust port instead of the Babel transform
    turbopackRustReactCompiler: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  // Enable the React Compiler
  reactCompiler: true,
  experimental: {
    // Use the Rust port instead of the Babel transform
    turbopackRustReactCompiler: true,
  },
}

module.exports = nextConfig
```

Good to know

- This option requires `reactCompiler` to be enabled. It selects which implementation runs, but does not turn the compiler on by itself.
- This option is only supported with Turbopack. Using it with webpack will throw an error.
- When enabled, you do not need to install `babel-plugin-react-compiler`. The Rust compiler runs natively inside Turbopack.

See the `reactCompiler` option documentation for details on how to use the compiler.

Version History

Version	Changes
`v16.3.0`	Introduced the experimental `turbopackRustReactCompiler` option for the native Rust React Compiler.

Turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/turbopack
> 📂 **Sección:** App > Api Reference
> 📝 **Descripción:** *Turbopack is an incremental bundler optimized for JavaScript and TypeScript, written in Rust, and built into Next.js.*

Turbopack is an **incremental bundler** optimized for JavaScript and TypeScript, written in Rust, and built into **Next.js**. You can use Turbopack with both the Pages and App Router for a **much faster** local development experience.

Why Turbopack?

We built Turbopack to push the performance of Next.js, including:

- **Unified Graph:** Next.js supports multiple output environments (e.g., client and server). Managing multiple compilers and stitching bundles together can be tedious. Turbopack uses a **single, unified graph** for all environments.
- **Bundling vs Native ESM:** Some tools skip bundling in development and rely on the browser's native ESM. This works well for small apps but can slow down large apps due to excessive network requests. Turbopack **bundles** in dev, but in an optimized way to keep large apps fast.
- **Incremental Computation:** Turbopack parallelizes work across cores and **caches** results down to the function level. Once a piece of work is done, Turbopack won't repeat it. Results persist to disk between runs.
- **Lazy Bundling:** Turbopack only bundles what is actually requested by the dev server. This lazy approach can reduce initial compile times and memory usage.

Supported platforms

Turbopack requires platform-specific native bindings. The following platforms are currently supported:

Platform	Architecture
macOS (Darwin)	x64, ARM64
Windows	x64, ARM64
Linux (glibc)	x64, ARM64
Linux (musl)	x64, ARM64

On platforms without native bindings (e.g. FreeBSD, OpenBSD), Next.js falls back to WebAssembly (WASM) bindings. WASM bindings support core SWC features like compilation and minification, but **do not support Turbopack**. On these platforms, use the `--webpack` flag:

```
next dev --webpack
next build --webpack
```

Getting started

Turbopack is now the **default bundler** in Next.js. No configuration is needed to use Turbopack:

```
// Archivo: package.json highlight={3}
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  }
}
```

Using Webpack instead

If you need to use Webpack instead of Turbopack, you can opt-in with the `--webpack` flag:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev --webpack",
    "build": "next build --webpack",
    "start": "next start"
  }
}
```

Supported features

Turbopack in Next.js has **zero-configuration** for the common use cases. Below is a summary of what is supported out of the box, plus some references to how you can configure Turbopack further when needed.

Language features

Feature	Status	Notes
JavaScript & TypeScript	**Supported**	Uses SWC under the hood. Type-checking is not done by Turbopack (run `tsc --watch` or rely on your IDE for type checks).
ECMAScript (ESNext)	**Supported**	Turbopack supports the latest ECMAScript features, matching SWC's coverage.
CommonJS	**Supported**	`require()` syntax is handled out of the box.
ESM	**Supported**	Static and dynamic `import` are fully supported.
Babel	**Supported**	Starting in Next.js 16, Turbopack uses Babel automatically if it detects a configuration file. Unlike in webpack, SWC is always used for Next.js's internal transforms and downleveling to older ECMAScript revisions. Next.js with webpack disables SWC if a Babel configuration file is present. Files in `node_modules` are excluded, unless you manually configure `babel-loader`.

Framework and React features

Feature	Status	Notes
JSX / TSX	**Supported**	SWC handles JSX/TSX compilation.
Fast Refresh	**Supported**	No configuration needed.
React Server Components (RSC)	**Supported**	For the Next.js App Router. Turbopack ensures correct server/client bundling.
Root layout creation	Unsupported	Automatic creation of a root layout in App Router is not supported. Turbopack will instruct you to create it manually.

CSS and styling

Feature	Status	Notes
Global CSS	**Supported**	Import `.css` files directly in your application.
CSS Modules	**Supported**	`.module.css` files work natively (Lightning CSS).
CSS Nesting	**Supported**	Lightning CSS supports modern CSS nesting.
@import syntax	**Supported**	Combine multiple CSS files.
PostCSS	**Supported**	Automatically processes PostCSS config files (`postcss.config.js` , `.mjs` , `.cjs` , `.ts` , `.mts` , `.cts`) in a Node.js worker pool. Useful for Tailwind, Autoprefixer, etc.
Sass / SCSS	**Supported** (Next.js)	For Next.js, Sass is supported out of the box. Custom Sass functions (`sassOptions.functions`) are not supported because Turbopack's Rust-based architecture cannot directly execute JavaScript functions, unlike webpack's Node.js environment. Use webpack if you need this feature. In the future, Turbopack standalone usage will likely require a loader config.
Less	Planned via plugins	Not yet supported by default. Will likely require a loader config once custom loaders are stable.
Lightning CSS	**In Use**	Handles CSS transformations. Some low-usage CSS Modules features (like `:local/:global` as standalone pseudo-classes) are not yet supported. See below for more details.

Assets

Feature	Status	Notes
Static Assets (images, fonts)	**Supported**	Importing `import img from './img.png'` works out of the box. In Next.js, returns an object for the `

Illustration

`component. | | **JSON Imports** | **Supported** | Named or default imports from` .json` are supported. |

Module resolution

Feature	Status	Notes
Path Aliases	**Supported**	Reads `tsconfig.json` 's `paths` and `baseUrl` , matching Next.js behavior.
Manual Aliases	**Supported**	Configure `resolveAlias` in `next.config.js` (similar to `webpack.resolve.alias`).
Custom Extensions	**Supported**	Configure `resolveExtensions` in `next.config.js` .
AMD	Partially Supported	Basic transforms work; advanced AMD usage is limited.

Performance and Fast Refresh

Feature	Status	Notes
Fast Refresh	**Supported**	Updates JavaScript, TypeScript, and CSS without a full refresh.
Incremental Bundling	**Supported**	Turbopack lazily builds only what's requested by the dev server, speeding up large apps.
FileSystem Cache	**Supported**	Persists compiler artifacts to disk between runs. See `turbopackFileSystemCache` .

Magic Comments

Turbopack supports webpack-compatible magic comments for controlling import behavior. These comments work with dynamic `import()` , `require()` , `require.resolve()` , and `new Worker()` expressions (not static `import` statements).

Comment	Webpack	Turbopack	Description
`webpackIgnore: true`	✓	✓	Skip bundling, preserve import
`turbopackIgnore: true`	✗	✓	Skip bundling (Turbopack-only)
`turbopackOptional: true`	✗	✓	Suppress resolve errors
`webpackOptional: true`	✗	✗	Not supported

See Lazy Loading for usage examples.

import.meta.env

Turbopack supports built-in environment metadata through `import.meta.env`:

Property	Type	Value
`DEV`	`boolean`	Whether `MODE` is not `"production"`
`PROD`	`boolean`	Whether `MODE` is `"production"`
`MODE`	`string`	The compile-time `NODE_ENV`, defaulting to `"development"`
`BASE_URL`	`string`	The Next.js `basePath`, with a trailing slash (`"/"` by default)
`SSR`	`boolean`	`true` in server bundles and `false` in browser and client bundles

These values are statically analyzed, so Turbopack can remove unreachable branches:

```
// Archivo: app/example.ts
if (import.meta.env.DEV) {
  console.log('development mode')
}
```

You can also read the whole object, destructure it, or use static bracket access:

```
// Archivo: app/example.ts
const { MODE, SSR } = import.meta.env
const baseUrl = import.meta.env['BASE_URL']
```

> **Good to know**: `import.meta.env` requires Turbopack. Custom `VITE_*` variables, Vite custom modes, `envPrefix`, and `envDir` are not supported. `BASE_URL` reflects the Next.js `basePath` configuration and includes a trailing slash to match Vite's format.

import.meta.glob

Turbopack supports `import.meta.glob()`, a Vite-compatible API for importing multiple modules at once using glob patterns. The result is an object keyed by the file path relative to the calling file.

```
const modules = import.meta.glob('./dir/*.js')
// {
//   './dir/foo.js': () => import('./dir/foo.js'),
//   './dir/bar.js': () => import('./dir/bar.js'),
// }
```

> **Good to know:** `import.meta.glob` requires Turbopack. It is not available when using webpack.

Lazy loading (default)

By default, each value in the result object is a thunk — a function that returns a `Promise` for the module. This enables lazy loading:

```
const modules = import.meta.glob('./dir/*.js')

for (const path in modules) {
  const module = await modules[path]()
  console.log(path, module)
}
```

Eager loading

Pass `{ eager: true }` to import all matching modules up front. Each value is the module object directly instead of a thunk:

```
const modules = import.meta.glob('./dir/*.js', { eager: true })

for (const path in modules) {
  console.log(path, modules[path].default)
}
```

Named imports

Use the `import` option to select a specific named export from each matched module. This works in both lazy and eager modes:

```
// Lazy: each value is () => Promise<exportValue>
const defaults = import.meta.glob('./dir/*.js', { import: 'default' })

// Eager: each value is the export value directly
const setups = import.meta.glob('./dir/*.js', { import: 'setup', eager: true })
```

Query strings

Use the `query` option to append a query string to every import request. This is useful for loading files as raw strings or URLs:

```
const rawFiles = import.meta.glob('./dir/*.txt', { query: '?raw' })
const urls = import.meta.glob('./dir/*.png', { query: '?url' })
```

The `query` option also accepts an object. Keys and values are URL-encoded and joined into a query string:

```
const modules = import.meta.glob('./*.ts', {
  query: { bar: 'foo', raw: true },
})
// equivalent to: { query: '?bar=foo&raw=true' }
```

Multiple patterns and negation

Pass an array of glob patterns as the first argument. Prefix a pattern with `!` to exclude matching files:

```
// Combine multiple directories
const modules = import.meta.glob(['./dir/*.js', './other/*.js'])

// Exclude specific files
const withoutTests = import.meta.glob(['./src/**/*.js', '!**/*.test.js'])
```

TypeScript

TypeScript types for `import.meta.glob` are included in Next.js. They are available automatically when `"moduleResolution": "bundler"` (or `"node16"` / `"nodenext"`) is set in your `tsconfig.json`, which is the default for new Next.js projects.

The return type differs based on the `eager` option:

```
// Lazy (default) — Record<string, () => Promise<unknown>>
const lazy = import.meta.glob('./dir/*.ts')

// Eager — Record<string, unknown>
const eager = import.meta.glob('./dir/*.ts', { eager: true })
```

Options reference

Option	Type	Default	Description
`eager`	`boolean`	`false`	Import modules synchronously instead of returning thunks.
`import`	`string`	`undefined`	Named export to select from each matched module (e.g. `'default'`).
`query`	`string | Record<string, string | boolean>`	`undefined`	Query string (or object) to append to each import.
`base`	`string`	`undefined`	Override the base path used for resolving patterns and keying results.
`caseSensitive`	`boolean`	`true`	Match glob patterns case-sensitively. Set to `false` to ignore ASCII case.

> **Note:** The `as` option (deprecated in Vite 5) is not supported. Use `query: '?raw'` or `query: '?url'` instead. The legacy `import.meta.globEager()` API is also not supported — use `import.meta.glob('...', { eager: true })` instead.

Known gaps with webpack

There are a number of non-trivial behavior differences between webpack and Turbopack that are important to be aware of when migrating an application. Generally, these are less of a concern for new applications.

Filesystem Root

Turbopack uses the root directory to resolve modules. Files outside of the project root are not resolved.

For example, when linking dependencies outside the project root (via `npm link`, `yarn link`, `pnpm link`, etc.), those linked files will not be resolved by default. To resolve these files, you must configure the root option to the parent directory of both the project and the linked dependencies.

You can configure the filesystem root using turbopack.root option in `next.config.js`.

CSS Module Ordering

Turbopack will follow JS import order to order CSS modules which are not otherwise ordered. For example:

```
// Archivo: components/BlogPost.jsx
import utilStyles from './utils.module.css'
import buttonStyles from './button.module.css'
export default function BlogPost() {
  return (
    <div className={utilStyles.container}>
      <button className={buttonStyles.primary}>Click me</button>
    </div>
  )
}
```

In this example, Turbopack will ensure that `utils.module.css` will appear before `button.module.css` in the produced CSS chunk, following the import order

Webpack generally does this as well, but there are cases where it will ignore JS inferred ordering, for example if it infers the JS file is side-effect-free.

This can lead to subtle rendering changes when adopting Turbopack, if applications have come to rely on an arbitrary ordering. Generally, the solution is easy, e.g. have `button.module.css` `@import utils.module.css` to force the ordering, or identify the conflicting rules and change them to not target the same properties.

Sass node_modules imports

Turbopack supports importing `node_modules` Sass files out of the box. Webpack supports a legacy tilde `~` syntax for this, which is not supported by Turbopack.

From:

```
// Archivo: styles/globals.scss
@import '~bootstrap/dist/css/bootstrap.min.css';
```

To:

```
// Archivo: styles/globals.scss
@import 'bootstrap/dist/css/bootstrap.min.css';
```

If you can't update the imports, you can add a `turbopack.resolveAlias` configuration to map the `~` syntax to the actual path:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      '~*': '*',
    },
  },
}
```

CSS / Sass / SCSS decimal precision

Turbopack uses Lightning CSS to compile CSS. Lightning CSS uses 5 digits of decimal precision for numeric CSS values, while webpack uses 10 digits. This applies to both plain CSS and Sass/SCSS output. For example, a value that evaluates to `25/17` produces:

- **Webpack:** `line-height: 1.4705882353` (10 digits)
- **Turbopack:** `line-height: 1.47059` (5 digits)

This can lead to subtle rendering differences when migrating from webpack to Turbopack, especially for properties like `line-height`, `letter-spacing`, or other calculated values where high precision matters.

Webpack plugins

Turbopack does not support webpack plugins. This affects third-party tools that rely on webpack's plugin system for integration. We do support webpack loaders. If you depend on webpack plugins, you'll need to find Turbopack-compatible alternatives or continue using webpack until equivalent functionality is available.

Unsupported and unplanned features

Some features are not yet implemented or not planned:

- **Legacy CSS Modules features**
  - Standalone `:local` and `:global` pseudo-classes (only the function variant `:global( ... )` is supported).
  - The `@value` rule (superseded by CSS variables).
  - `:import` and `:export` ICSS rules.
  - `composes` in `.module.css` composing a `.css` file. In webpack this would treat the `.css` file as a CSS Module, with Turbopack the `.css` file will always be global. This means that if you want to use `composes` in a CSS Module, you need to change the `.css` file to a `.module.css` file.

    - `@import` in CSS Modules importing `.css` as a CSS Module. In webpack this would treat the `.css` file as a CSS Module, with Turbopack the `.css` file will always be global. This means that if you want to use `@import` in a CSS Module, you need to change the `.css` file to a `.module.css` file.

- `sassOptions.functions` Custom Sass functions defined in `sassOptions.functions` are not supported. This feature allows defining JavaScript functions that can be called from Sass code during compilation. Turbopack's Rust-based architecture cannot directly execute JavaScript functions passed through `sassOptions.functions`, unlike webpack's Node.js-based sass-loader which runs entirely in JavaScript. If you're using custom Sass functions, you'll need to use webpack instead of Turbopack.
- `webpack()` **configuration** in `next.config.js` Turbopack replaces webpack, so `webpack()` configs are not recognized. Use the `turbopack` config instead.
- **Yarn PnP** Not planned for Turbopack support in Next.js.
- `experimental.urlImports` Not planned for Turbopack.
- `experimental.esmExternals` Not planned. Turbopack does not support the legacy `esmExternals` configuration in Next.js.
- **Some Next.js Experimental Flags**
    - `experimental.nextScriptWorkers`
    - `experimental.fallbackNodePolyfills` We plan to implement these in the future.

For a full, detailed breakdown of each feature flag and its status, see the Turbopack API Reference.

Configuration

Turbopack can be configured via `next.config.js` (or `next.config.ts`) under the `turbopack` key. Configuration options include:

- `rules` Define additional webpack loaders for file transformations.
- `resolveAlias` Create manual aliases (like `resolve.alias` in webpack).
- `resolveExtensions` Change or extend file extensions for module resolution.
- `ignoreIssue` Suppress specific Turbopack errors and warnings from the CLI output and error overlay.

Additionally, the following experimental options are available under `experimental` in `next.config.js`:

Option	Description	Default (dev)	Default (build)
`turbopackFileSystemCacheForDev`	Enable filesystem cache for the dev server.	`true`	N/A
`turbopackFileSystemCacheForBuild`	Enable filesystem cache for builds.	N/A	`true` [1]
`turbopackMinify`	Enable minification.	`false`	`true`
`turbopackSourceMaps`	Enable source maps.	`true`	`productionBrowserSourceMaps`
`turbopackInputSourceMaps`	Enable extraction of source maps from input files.	`true`	`true`
`turbopackModuleFragments`	Currently in active development. This splits modules into fragments and chunks only import the used fragments of the modules.	`false`	`false`
`turbopackRemoveUnusedImports`	Enable removing unused imports. Requires `turbopackRemoveUnusedExports`.	`false`	`true`
`turbopackRemoveUnusedExports`	Enable removing unused exports.	`false`	`true`
`turbopackInferModuleSideEffects`	Enable local analysis to infer side-effect-free modules for better tree shaking.	`true`	`true`
`turbopackScopeHoisting`	Enable scope hoisting. Always disabled in dev mode.	`false`	`true`
`turbopackClientSideNestedAsyncChunking`	Enable nested async chunking for client-side assets.	`false`	`true`
`turbopackServerSideNestedAsyncChunking`	Enable nested async chunking for server-side assets.	`false`	`false`
`turbopackImportTypeBytes`	Enable support for `with {type: "bytes"}` for ESM imports.	`false`	`false`
`turbopackUseBuiltinBabel`	Enable automatic Babel loader configuration when a Babel config file is present.	`true`	`true`
`turbopackUseBuiltinSass`	Enable automatic Sass loader configuration.	`true`	`true`
`turbopackModuleIds`	Module ID strategy: `'named'` or `'deterministic'`.	`'named'`	`'deterministic'`
`turbopackLocalPostcssConfig`	Resolve `postcss.config.js` from the CSS file's directory first, then the project root.	`false`	`false`
`turbopackWorkerAssetPrefix`	Custom asset prefix for Web Worker URLs (entrypoint + module chunks), overriding `assetPrefix`. Mirrors webpack's `output.workerPublicPath`.	`undefined`	`undefined`

Table notes:

1. Enabled by default. Set the option to `false` when the build environment does not preserve the `.next/cache` directory between builds.

For example, to configure an alias and a custom file extension under the `turbopack` key:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      underscore: 'lodash',
    },
    resolveExtensions: ['.mdx', '.tsx', '.ts', '.jsx', '.js', '.json'],
  },
}
```

For more in-depth configuration examples, see the Turbopack config documentation.

Generating trace files for performance debugging

If you encounter performance or memory issues and want to help the Next.js team diagnose them, you can generate a trace file by adding the `--internal-trace` flag to your dev command:

```
next dev --internal-trace
```

This will produce a `.next-profiles/trace-turbopack.bin` file. Include that file when creating a GitHub issue on the Next.js repo to help us investigate.

Summary

Turbopack is a **Rust-based**, **incremental** bundler designed to make local development and builds fast—especially for large applications. It is integrated into Next.js, offering zero-config CSS, React, and TypeScript support.

Version Changes

Version	Changes
`v16.0.0`	Turbopack becomes the default bundler for Next.js. Automatic support for Babel when a configuration file is found.
`v15.5.0`	Turbopack support for `build` beta
`v15.3.0`	Experimental support for `build`
`v15.0.0`	Turbopack for `dev` stable

How to deploy your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/deploying
> 📁 **Sección:** Pages > Getting Started
> 📝 **Descripción:** *Learn how to deploy your Next.js application.*

How to configure Continuous Integration (CI) build caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/ci-build-caching
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to configure CI to cache Next.js builds*

How to set up a custom server in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/custom-server
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Start a Next.js app programmatically using a custom server.*

Pages and Layouts

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/pages-and-layouts
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Create your first page and shared layout with the Pages Router.*

The Pages Router has a file-system based router built on the concept of pages.

When a file is added to the `pages` directory, it's automatically available as a route.

In Next.js, a **page** is a React Component exported from a `.js`, `.jsx`, `.ts`, or `.tsx` file in the `pages` directory. Each page is associated with a route based on its file name.

Example: If you create `pages/about.js` that exports a React component like below, it will be accessible at `/about`.

```
export default function About() {
  return <div>About</div>
}
```

Index routes

The router will automatically route files named `index` to the root of the directory.

- `pages/index.js` → `/`
- `pages/blog/index.js` → `/blog`

Nested routes

The router supports nested files. If you create a nested folder structure, files will automatically be routed in the same way still.

- `pages/blog/first-post.js` → `/blog/first-post`
- `pages/dashboard/settings/username.js` → `/dashboard/settings/username`

Pages with Dynamic Routes

Next.js supports pages with dynamic routes. For example, if you create a file called `pages/posts/[id].js`, then it will be accessible at `posts/1`, `posts/2`, etc.

> To learn more about dynamic routing, check the Dynamic Routing documentation.

Layout Pattern

The React model allows us to deconstruct a page into a series of components. Many of these components are often reused between pages. For example, you might have the same navigation bar and footer on every page.

```
// Archivo: components/layout.js
import Navbar from './navbar'
import Footer from './footer'

export default function Layout({ children }) {
  return (
    <>
      <Navbar />
      <main>{children}</main>
      <Footer />
    </>
  )
}
```

Examples

Single Shared Layout with Custom App

If you only have one layout for your entire application, you can create a Custom App and wrap your application with the layout. Since the `<Layout />` component is re-used when changing pages, its component state will be preserved (e.g. input values).

```
// Archivo: pages/_app.js
import Layout from '../components/layout'

export default function MyApp({ Component, pageProps }) {
  return (
    <Layout>
      <Component {...pageProps} />
    </Layout>
  )
}
```

Per-Page Layouts

If you need multiple layouts, you can add a property `getLayout` to your page, allowing you to return a React component for the layout. This allows you to define the layout on a *per-page basis*. Since we're returning a function, we can have complex nested layouts if desired.

```
// Archivo: pages/index.js

import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'

export default function Page() {
  return (
    /** Your content */
  )
}

Page.getLayout = function getLayout(page) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}
```

```
// Archivo: pages/_app.js
export default function MyApp({ Component, pageProps }) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

When navigating between pages, we want to *persist* page state (input values, scroll position, etc.) for a Single-Page Application (SPA) experience.

This layout pattern enables state persistence because the React component tree is maintained between page transitions. With the component tree, React can understand which elements have changed to preserve state.

> **Good to know**: This process is called reconciliation, which is how React understands which elements have changed.

With TypeScript

When using TypeScript, you must first create a new type for your pages which includes a `getLayout` function. Then, you must create a new type for your `AppProps` which overrides the `Component` property to use the previously created type.

```
// Archivo: pages/index.tsx switcher
import type { ReactElement } from 'react'
import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'
import type { NextPageWithLayout } from './_app'

const Page: NextPageWithLayout = () => {
  return <p>hello world</p>
}

Page.getLayout = function getLayout(page: ReactElement) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}

export default Page
```

```
// Archivo: pages/index.js switcher
import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'

const Page = () => {
  return <p>hello world</p>
}

Page.getLayout = function getLayout(page) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}

export default Page
```

```tsx
// Archivo: pages/_app.tsx switcher
import type { ReactElement, ReactNode } from 'react'
import type { NextPage } from 'next'
import type { AppProps } from 'next/app'

export type NextPageWithLayout<P = {}, IP = P> = NextPage<P, IP> & {
  getLayout?: (page: ReactElement) => ReactNode
}

type AppPropsWithLayout = AppProps & {
  Component: NextPageWithLayout
}

export default function MyApp({ Component, pageProps }: AppPropsWithLayout) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

```jsx
// Archivo: pages/_app.js switcher
export default function MyApp({ Component, pageProps }) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

Data Fetching

Inside your layout, you can fetch data on the client-side using `useEffect` or a library like SWR. Because this file is not a Page, you cannot use `getStaticProps` or `getServerSideProps` currently.

```jsx
// Archivo: components/layout.js
import useSWR from 'swr'
import Navbar from './navbar'
import Footer from './footer'

export default function Layout({ children }) {
  const { data, error } = useSWR('/api/navigation', fetcher)

  if (error) return <div>Failed to load</div>
  if (!data) return <div>Loading...</div>

  return (
    <>
      <Navbar links={data.links} />
      <main>{children}</main>
      <Footer />
    </>
  )
}
```

Dynamic Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/dynamic-routes
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Dynamic Routes are pages that allow you to add custom params to your URLs. Start creating Dynamic Routes and learn more here.*

When you don't know the exact segment names ahead of time and want to create routes from dynamic data, you can use Dynamic Segments that are filled in at request time or prerendered at build time.

Convention

A Dynamic Segment can be created by wrapping a file or folder name in square brackets: `[segmentName]`. For example, `[id]` or `[slug]`.

Dynamic Segments can be accessed from `useRouter`.

Example

For example, a blog could include the following route `pages/blog/[slug].js` where `[slug]` is the Dynamic Segment for blog posts.

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()
  return <p>Post: {router.query.slug}</p>
}
```

Route	Example URL	`params`
`pages/blog/[slug].js`	`/blog/a`	`{ slug: 'a' }`
`pages/blog/[slug].js`	`/blog/b`	`{ slug: 'b' }`
`pages/blog/[slug].js`	`/blog/c`	`{ slug: 'c' }`

Catch-all Segments

Dynamic Segments can be extended to **catch-all** subsequent segments by adding an ellipsis inside the brackets `[...segmentName]`.

For example, `pages/shop/[...slug].js` will match `/shop/clothes`, but also `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`, and so on.

Route	Example URL	`params`
`pages/shop/[...slug].js`	`/shop/a`	`{ slug: ['a'] }`
`pages/shop/[...slug].js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`pages/shop/[...slug].js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Optional Catch-all Segments

Catch-all Segments can be made **optional** by including the parameter in double square brackets: `[[...segmentName]]`.

For example, `pages/shop/[[...slug]].js` will **also** match `/shop`, in addition to `/shop/clothes`, `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`.

The difference between **catch-all** and **optional catch-all** segments is that with optional, the route without the parameter is also matched (`/shop` in the example above).

Route	Example URL	`params`
`pages/shop/[[...slug]].js`	`/shop`	`{ slug: undefined }`
`pages/shop/[[...slug]].js`	`/shop/a`	`{ slug: ['a'] }`
`pages/shop/[[...slug]].js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`pages/shop/[[...slug]].js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Linking and Navigating

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/linking-and-navigating
> 📂 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Learn how navigation works in Next.js, and how to use the Link Component and `useRouter` hook.*

The Next.js router allows you to do client-side route transitions between pages, similar to a single-page application.

A React component called `Link` is provided to do this client-side route transition.

```
import Link from 'next/link'

function Home() {
  return (
    <ul>
      <li>
        [Home](/)
      </li>
      <li>
        [About Us](/about)
      </li>
      <li>
        [Blog Post](/blog/hello-world)
      </li>
    </ul>
  )
}

export default Home
```

The example above uses multiple links. Each one maps a path (`href`) to a known page:

- `/` → `pages/index.js`
- `/about` → `pages/about.js`
- `/blog/hello-world` → `pages/blog/[slug].js`

Any `<Link />` in the viewport (initially or through scroll) will be prefetched by default (including the corresponding data) for pages using Static Generation. The corresponding data for server-rendered routes is fetched *only when* the `<Link />` is clicked.

Linking to dynamic paths

You can also use interpolation to create the path, which comes in handy for dynamic route segments. For example, to show a list of posts which have been passed to the component as a prop:

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${encodeURIComponent(post.slug)}`}>
            {post.title}
          </Link>
        </li>
      ))}
    </ul>
  )
}

export default Posts
```

> `encodeURIComponent` is used in the example to keep the path utf-8 compatible.

Alternatively, using a URL Object:

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link
            href={{
              pathname: '/blog/[slug]',
              query: { slug: post.slug },
            }}
          >
            {post.title}
          </Link>
        </li>
      ))}
    </ul>
  )
}

export default Posts
```

Now, instead of using interpolation to create the path, we use a URL object in `href` where:

- `pathname` is the name of the page in the `pages` directory. `/blog/[slug]` in this case.
- `query` is an object with the dynamic segment. `slug` in this case.

Injecting the router

To access the `router` object in a React component you can use `useRouter` or `withRouter`.

In general we recommend using `useRouter`.

Imperative Routing

`next/link` should be able to cover most of your routing needs, but you can also do client-side navigations without it, take a look at the documentation for `next/router`.

The following example shows how to do basic page navigations with `useRouter`:

```
import { useRouter } from 'next/router'

export default function ReadMore() {
  const router = useRouter()

  return (
    <button onClick={() => router.push('/about')}>
      Click here to read more
    </button>
  )
}
```

Shallow Routing

Examples

Shallow routing allows you to change the URL without running data fetching methods again, that includes `getServerSideProps`, `getStaticProps`, and `getInitialProps`.

You'll receive the updated `pathname` and the `query` via the `router` object (added by `useRouter` or `withRouter`), without losing state.

To enable shallow routing, set the `shallow` option to `true`. Consider the following example:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

// Current URL is '/'
function Page() {
  const router = useRouter()

  useEffect(() => {
    // Always do navigations after the first render
    router.push('/?counter=10', undefined, { shallow: true })
  }, [])

  useEffect(() => {
    // The counter changed!
  }, [router.query.counter])
}

export default Page
```

The URL will get updated to `/?counter=10` and the page won't get replaced, only the state of the route is changed.

You can also watch for URL changes via `componentDidUpdate` as shown below:

```
componentDidUpdate(prevProps) {
  const { pathname, query } = this.props.router
  // verify props have changed to avoid an infinite loop
  if (query.counter !== prevProps.router.query.counter) {
    // fetch data based on the new query
  }
}
```

Caveats

Shallow routing **only** works for URL changes in the current page. For example, let's assume we have another page called `pages/about.js`, and you run this:

```
router.push('/?counter=10', '/about?counter=10', { shallow: true })
```

Since that's a new page, it'll unload the current page, load the new one and wait for data fetching even though we asked to do shallow routing.

When shallow routing is used with proxy it will not ensure the new page matches the current page like previously done without proxy. This is due to proxy being able to rewrite dynamically and can't be verified client-side without a data fetch which is skipped with shallow, so a shallow route change must always be treated as shallow.

Custom App

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-app
> 📂 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Control page initialization and add a layout that persists for all pages by overriding the default App component used by Next.js.*

Next.js uses the `App` component to initialize pages. You can override it and control the page initialization and:

- Create a shared layout between page changes
- Inject additional data into pages
- Add global CSS

Usage

To override the default `App`, create the file `pages/_app` as shown below:

```
// Archivo: pages/_app.tsx switcher
import type { AppProps } from 'next/app'

export default function MyApp({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />
}
```

```
// Archivo: pages/_app.jsx switcher
export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

The `Component` prop is the active `page`, so whenever you navigate between routes, `Component` will change to the new `page`. Therefore, any props you send to `Component` will be received by the `page`.

`pageProps` is an object with the initial props that were preloaded for your page by one of our data fetching methods, otherwise it's an empty object.

> **Good to know**:
> - If your app is running and you added a custom `App`, you'll need to restart the development server. Only required if `pages/_app.js` didn't exist before.
> - `App` does not support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.

`getInitialProps` with `App`

Using `getInitialProps` in `App` will disable Automatic Static Optimization for pages without `getStaticProps`.

We do not recommend using this pattern. Instead, consider incrementally adopting the App Router, which allows you to more easily fetch data for pages and layouts.

```tsx
// Archivo: pages/_app.tsx switcher
import App, { AppContext, AppInitialProps, AppProps } from 'next/app'

type AppOwnProps = { example: string }

export default function MyApp({
  Component,
  pageProps,
  example,
}: AppProps & AppOwnProps) {
  return (
    <>
      <p>Data: {example}</p>
      <Component {...pageProps} />
    </>
  )
}

MyApp.getInitialProps = async (
  context: AppContext
): Promise<AppOwnProps & AppInitialProps> => {
  const ctx = await App.getInitialProps(context)

  return { ...ctx, example: 'data' }
}
```

```jsx
// Archivo: pages/_app.jsx switcher
import App from 'next/app'

export default function MyApp({ Component, pageProps, example }) {
  return (
    <>
      <p>Data: {example}</p>
      <Component {...pageProps} />
    </>
  )
}

MyApp.getInitialProps = async (context) => {
  const ctx = await App.getInitialProps(context)

  return { ...ctx, example: 'data' }
}
```

Custom Document

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-document
> 📂 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Extend the default document markup added by Next.js.*

A custom `Document` can update the `<html>` and `<body>` tags used to render a Page.

To override the default `Document`, create the file `pages/_document` as shown below:

```tsx
// Archivo: pages/_document.tsx switcher
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html lang="en">
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

```jsx
// Archivo: pages/_document.jsx switcher
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html lang="en">
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

> **Good to know**:
>
> - `_document` is only rendered on the server, so event handlers like `onClick` cannot be used in this file.
> - `<Html>`, `<Head />`, `<Main />` and `<NextScript />` are required for the page to be properly rendered.

Caveats

- The `<Head />` component used in `_document` is not the same as `next/head`. The `<Head />` component used here should only be used for any `<head>` code that is common for all pages. For all other cases, such as `<title>` tags, we recommend using `next/head` in your pages or components.
- React components outside of `<Main />` will not be initialized by the browser. Do *not* add application logic here or custom CSS (like `styled-jsx`). If you need shared components in all your pages (like a menu or a toolbar), read Layouts instead.
- `Document` currently does not support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.

Customizing `renderPage`

Customizing `renderPage` is advanced and only needed for libraries like CSS-in-JS to support server-side rendering. This is not needed for built-in `styled-jsx` support.

We do not recommend using this pattern. Instead, consider incrementally adopting the App Router, which allows you to more easily fetch data for pages and layouts.

```tsx
// Archivo: pages/_document.tsx switcher
import Document, {
  Html,
  Head,
  Main,
  NextScript,
  DocumentContext,
  DocumentInitialProps,
} from 'next/document'

class MyDocument extends Document {
  static async getInitialProps(
    ctx: DocumentContext
  ): Promise<DocumentInitialProps> {
    const originalRenderPage = ctx.renderPage

    // Run the React rendering logic synchronously
    ctx.renderPage = () =>
      originalRenderPage({
        // Useful for wrapping the whole react tree
        enhanceApp: (App) => App,
        // Useful for wrapping in a per-page basis
        enhanceComponent: (Component) => Component,
      })

    // Run the parent `getInitialProps`, it now includes the custom `renderPage`
    const initialProps = await Document.getInitialProps(ctx)

    return initialProps
  }

  render() {
    return (
      <Html lang="en">
        <Head />
        <body>
          <Main />
          <NextScript />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

```
// Archivo: pages/_document.jsx switcher
import Document, { Html, Head, Main, NextScript } from 'next/document'

class MyDocument extends Document {
  static async getInitialProps(ctx) {
    const originalRenderPage = ctx.renderPage

    // Run the React rendering logic synchronously
    ctx.renderPage = () =>
      originalRenderPage({
        // Useful for wrapping the whole react tree
        enhanceApp: (App) => App,
        // Useful for wrapping in a per-page basis
        enhanceComponent: (Component) => Component,
      })

    // Run the parent `getInitialProps`, it now includes the custom `renderPage`
    const initialProps = await Document.getInitialProps(ctx)

    return initialProps
  }

  render() {
    return (
      <Html lang="en">
        <Head />
        <body>
          <Main />
          <NextScript />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

Good to know:

- `getInitialProps` in `_document` is not called during client-side transitions.
- The `ctx` object for `_document` is equivalent to the one received in `getInitialProps`, with the addition of `renderPage`.

API Routes

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/api-routes
📁 **Sección:** Pages > Building Your Application > Routing
📝 **Descripción:** *Next.js supports API Routes, which allow you to build your API without leaving your Next.js app. Learn how it works here.*

Examples

Good to know: If you are using the App Router, you can use Server Components or Route Handlers instead of API Routes.

API routes provide a solution to build a **public API** with Next.js.

Any file inside the folder `pages/api` is mapped to `/api/*` and will be treated as an API endpoint instead of a `page`. They are server-side only bundles and won't increase your client-side bundle size.

For example, the following API route returns a JSON response with a status code of `200`:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

Good to know:

- API Routes do not specify CORS headers, meaning they are **same-origin only** by default. You can customize such behavior by wrapping the request handler with the CORS request helpers.
- API Routes can't be used with static exports. However, Route Handlers in the App Router can.
- API Routes will be affected by `pageExtensions` configuration in `next.config.js`.

Parameters

```
export default function handler(req: NextApiRequest, res: NextApiResponse) {
  // ...
}
```

- `req`: An instance of http.IncomingMessage
- `res`: An instance of http.ServerResponse

HTTP Methods

To handle different HTTP methods in an API route, you can use `req.method` in your request handler, like so:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  if (req.method === 'POST') {
    // Process a POST request
  } else {
    // Handle any other HTTP method
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  if (req.method === 'POST') {
    // Process a POST request
  } else {
    // Handle any other HTTP method
  }
}
```

Request Helpers

API Routes provide built-in request helpers which parse the incoming request (`req`):

- `req.cookies` - An object containing the cookies sent by the request. Defaults to `{}`
- `req.query` - An object containing the query string. Defaults to `{}`
- `req.body` - An object containing the body parsed by `content-type` , or `null` if no body was sent

Custom config

Every API Route can export a `config` object to change the default configuration, which is the following:

```
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '1mb',
    },
  },
  // Specifies the maximum allowed duration for this function to execute (in seconds)
  maxDuration: 5,
}
```

`bodyParser` is automatically enabled. If you want to consume the body as a `Stream` or with `raw-body` , you can set this to `false` .

One use case for disabling the automatic `bodyParsing` is to allow you to verify the raw body of a **webhook** request, for example from GitHub.

```
export const config = {
  api: {
    bodyParser: false,
  },
}
```

`bodyParser.sizeLimit` is the maximum size allowed for the parsed body, in any format supported by bytes, like so:

```
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '500kb',
    },
  },
}
```

`externalResolver` is an explicit flag that tells the server that this route is being handled by an external resolver like *express* or *connect*. Enabling this option disables warnings for unresolved requests.

```
export const config = {
  api: {
    externalResolver: true,
  },
}
```

`responseLimit` is automatically enabled, warning when an API Routes' response body is over 4MB.

If you are not using Next.js in a serverless environment, and understand the performance implications of not using a CDN or dedicated media host, you can set this limit to `false` .

```
export const config = {
  api: {
    responseLimit: false,
  },
}
```

`responseLimit` can also take the number of bytes or any string format supported by `bytes`, for example `1000`, `'500kb'` or `'3mb'`. This value will be the maximum response size before a warning is displayed. Default is 4MB. (see above)

```
export const config = {
  api: {
    responseLimit: '8mb',
  },
}
```

Response Helpers

The Server Response object, (often abbreviated as `res`) includes a set of Express.js-like helper methods to improve the developer experience and increase the speed of creating new API endpoints.

The included helpers are:

- `res.status(code)` - A function to set the status code. `code` must be a valid HTTP status code
- `res.json(body)` - Sends a JSON response. `body` must be a serializable object
- `res.send(body)` - Sends the HTTP response. `body` can be a `string`, an `object` or a `Buffer`
- `res.redirect([status,] path)` - Redirects to a specified path or URL. `status` must be a valid HTTP status code. If not specified, `status` defaults to "307" "Temporary redirect".
- `res.revalidate(urlPath)` - Revalidate a page on demand using `getStaticProps`. `urlPath` must be a `string`.

Setting the status code of a response

When sending a response back to the client, you can set the status code of the response.

The following example sets the status code of the response to `200` (`OK`) and returns a `message` property with the value of `Hello from Next.js!` as a JSON response:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

Sending a JSON response

When sending a response back to the client you can send a JSON response, this must be a serializable object. In a real world application you might want to let the client know the status of the request depending on the result of the requested endpoint.

The following example sends a JSON response with the status code `200` (`OK`) and the result of the async operation. It's contained in a try catch block to handle any errors that may occur, with the appropriate status code and error message caught and sent back to the client:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const result = await someAsyncOperation()
    res.status(200).json({ result })
  } catch (err) {
    res.status(500).json({ error: 'failed to load data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  try {
    const result = await someAsyncOperation()
    res.status(200).json({ result })
  } catch (err) {
    res.status(500).json({ error: 'failed to load data' })
  }
}
```

Sending a HTTP response

Sending an HTTP response works the same way as when sending a JSON response. The only difference is that the response body can be a `string`, an `object` or a `Buffer`.

The following example sends a HTTP response with the status code `200` (`OK`) and the result of the async operation.

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const result = await someAsyncOperation()
    res.status(200).send({ result })
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  try {
    const result = await someAsyncOperation()
    res.status(200).send({ result })
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

Redirects to a specified path or URL

Taking a form as an example, you may want to redirect your client to a specified path or URL once they have submitted the form.

The following example redirects the client to the `/` path if the form is successfully submitted:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { name, message } = req.body

  try {
    await handleFormInputAsync({ name, message })
    res.redirect(307, '/')
  } catch (err) {
    res.status(500).send({ error: 'Failed to fetch data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  const { name, message } = req.body

  try {
    await handleFormInputAsync({ name, message })
    res.redirect(307, '/')
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

Adding TypeScript types

You can make your API Routes more type-safe by importing the `NextApiRequest` and `NextApiResponse` types from `next`, in addition to those, you can also type your response data:

```
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

> **Good to know**: The body of `NextApiRequest` is `any` because the client may include any payload. You should validate the type/shape of the body at runtime before using it.

Dynamic API Routes

API Routes support dynamic routes, and follow the same file naming rules used for `pages/`.

```
// Archivo: pages/api/post/[pid].ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const { pid } = req.query
  res.end(`Post: ${pid}`)
}
```

```
// Archivo: pages/api/post/[pid].js switcher
export default function handler(req, res) {
  const { pid } = req.query
  res.end(`Post: ${pid}`)
}
```

Now, a request to `/api/post/abc` will respond with the text: `Post: abc`.

Catch all API routes

API Routes can be extended to catch all paths by adding three dots (`...`) inside the brackets. For example:

- `pages/api/post/[...slug].js` matches `/api/post/a`, but also `/api/post/a/b`, `/api/post/a/b/c` and so on.

> **Good to know**: You can use names other than `slug`, such as: `[...param]`

Matched parameters will be sent as a query parameter (`slug` in the example) to the page, and it will always be an array, so, the path `/api/post/a` will have the following `query` object:

```
{ "slug": ["a"] }
```

And in the case of `/api/post/a/b`, and any other matching path, new parameters will be added to the array, like so:

```
{ "slug": ["a", "b"] }
```

For example:

```
// Archivo: pages/api/post/[...slug].ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const { slug } = req.query
  res.end(`Post: ${slug.join(', ')}`)
}
```

```
// Archivo: pages/api/post/[...slug].js switcher
export default function handler(req, res) {
  const { slug } = req.query
  res.end(`Post: ${slug.join(', ')}`)
}
```

Now, a request to `/api/post/a/b/c` will respond with the text: `Post: a, b, c`.

Optional catch all API routes

Catch all routes can be made optional by including the parameter in double brackets (`[[...slug]]`).

For example, `pages/api/post/[[...slug]].js` will match `/api/post`, `/api/post/a`, `/api/post/a/b`, and so on.

The main difference between catch all and optional catch all routes is that with optional, the route without the parameter is also matched (`/api/post` in the example above).

The `query` objects are as follows:

```
{ } // GET `/api/post` (empty object)
{ "slug": ["a"] } // `GET /api/post/a` (single-element array)
{ "slug": ["a", "b"] } // `GET /api/post/a/b` (multi-element array)
```

Caveats

- Predefined API routes take precedence over dynamic API routes, and dynamic API routes over catch all API routes. Take a look at the following examples:
  - `pages/api/post/create.js` - Will match `/api/post/create`
  - `pages/api/post/[pid].js` - Will match `/api/post/1`, `/api/post/abc`, etc. But not `/api/post/create`
  - `pages/api/post/[...slug].js` - Will match `/api/post/1/2`, `/api/post/a/b/c`, etc. But not `/api/post/create`, `/api/post/abc`

Streaming responses

While the Pages Router does support streaming responses with API Routes, we recommend incrementally adopting the App Router and using Route Handlers if you are on Next.js 14+.

Here's how you can stream a response from an API Route with `writeHead`:

```
// Archivo: pages/api/hello.ts switcher
import { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-store',
  })
  let i = 0
  while (i < 10) {
    res.write(`data: ${i}\n\n`)
    i++
    await new Promise((resolve) => setTimeout(resolve, 1000))
  }
  res.end()
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-store',
  })
  let i = 0
  while (i < 10) {
    res.write(`data: ${i}\n\n`)
    i++
    await new Promise((resolve) => setTimeout(resolve, 1000))
  }
  res.end()
}
```

Custom Errors

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-error
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Override and extend the built-in Error page to handle custom errors.*

---

404 Page

A 404 page may be accessed very often. Server-rendering an error page for every visit increases the load of the Next.js server. This can result in increased costs and slow experiences.

To avoid the above pitfalls, Next.js provides a static 404 page by default without having to add any additional files.

> **Good to know**: The default 404 and 500 pages follow the operating system's color scheme via `prefers-color-scheme` and do not read an app-level theme. Because `pages/404.js` and `pages/500.js` render inside your custom `App`, providing your own files lets your global styles and theme apply.

Customizing The 404 Page

To create a custom 404 page you can create a `pages/404.js` file. This file is statically generated at build time.

```
// Archivo: pages/404.js
export default function Custom404() {
  return <h1>404 - Page Not Found</h1>
}
```

> **Good to know**: You can use `getStaticProps` inside this page if you need to fetch data at build time.

500 Page

Server-rendering an error page for every visit adds complexity to responding to errors. To help users get responses to errors as fast as possible, Next.js provides a static 500 page by default without having to add any additional files.

Customizing The 500 Page

To customize the 500 page you can create a `pages/500.js` file. This file is statically generated at build time.

```
// Archivo: pages/500.js
export default function Custom500() {
  return <h1>500 - Server-side error occurred</h1>
}
```

> **Good to know**: You can use `getStaticProps` inside this page if you need to fetch data at build time.

More Advanced Error Page Customizing

500 errors are handled both client-side and server-side by the `Error` component. If you wish to override it, define the file `pages/_error.js` and add the following code:

```
function Error({ statusCode }) {
  return (
    <p>
      {statusCode
        ? `An error ${statusCode} occurred on server`
        : 'An error occurred on client'}
    </p>
  )
}

Error.getInitialProps = ({ res, err }) => {
  const statusCode = res ? res.statusCode : err ? err.statusCode : 404
  return { statusCode }
}

export default Error
```

> `pages/_error.js` is only used in production. In development you'll get an error with the call stack to know where the error originated from.

Reusing the built-in error page

If you want to render the built-in error page you can by importing the `Error` component:

```
import Error from 'next/error'

export async function getServerSideProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const errorCode = res.ok ? false : res.status
  const json = await res.json()

  return {
    props: { errorCode, stars: json.stargazers_count },
  }
}

export default function Page({ errorCode, stars }) {
  if (errorCode) {
    return <Error statusCode={errorCode} />
  }

  return <div>Next stars: {stars}</div>
}
```

The `Error` component also takes `title` as a property if you want to pass in a text message along with a `statusCode`.

If you have a custom `Error` component be sure to import that one instead. `next/error` exports the default component used by Next.js.

Caveats

- `Error` does not currently support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.
- `_error`, like `_app`, is a reserved pathname. `_error` is used to define the customized layouts and behaviors of the error pages. `/_error` will render 404 when accessed directly via routing or rendering in a custom server.

Routing

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Learn the fundamentals of routing for front-end applications with the Pages Router.*

The Pages Router has a file-system based router built on concepts of pages. When a file is added to the `pages` directory it's automatically available as a route. Learn more about routing in the Pages Router:

Server-side Rendering (SSR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/server-side-rendering
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Use Server-side Rendering to render pages on each request.*

> Also referred to as "SSR" or "Dynamic Rendering".

If a page uses **Server-side Rendering**, the page HTML is generated on **each request**.

To use Server-side Rendering for a page, you need to `export` an `async` function called `getServerSideProps`. This function will be called by the server on every request.

For example, suppose that your page needs to prerender frequently updated data (fetched from an external API). You can write `getServerSideProps` which fetches this data and passes it to `Page` like below:

```
export default function Page({ data }) {
  // Render data ...
}

// This gets called on every request
export async function getServerSideProps() {
  // Fetch data from external API
  const res = await fetch(`https:// ... /data`)
  const data = await res.json()

  // Pass data to the page via props
  return { props: { data } }
}
```

As you can see, `getServerSideProps` is similar to `getStaticProps`, but the difference is that `getServerSideProps` is run on every request instead of on build time.

To learn more about how `getServerSideProps` works, check out our Data Fetching documentation.

Static Site Generation (SSG)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/static-site-generation
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Use Static Site Generation (SSG) to prerender pages at build time.*

Examples

If a page uses **Static Generation**, the page HTML is generated at **build time**. That means in production, the page HTML is generated when you run `next build`. This HTML will then be reused on each request. It can be cached by a CDN.

In Next.js, you can statically generate pages **with or without data**. Let's take a look at each case.

Static Generation without data

By default, Next.js prerenders pages using Static Generation without fetching data. Here's an example:

```
function About() {
  return <div>About</div>
}

export default About
```

Note that this page does not need to fetch any external data to be prerendered. In cases like this, Next.js generates a single HTML file per page during build time.

Static Generation with data

Some pages require fetching external data for prerendering. There are two scenarios, and one or both might apply. In each case, you can use these functions that Next.js provides:

1. Your page **content** depends on external data: Use `getStaticProps`.
2. Your page **paths** depend on external data: Use `getStaticPaths` (usually in addition to `getStaticProps`).

Scenario 1: Your page content depends on external data

Example: Your blog page might need to fetch the list of blog posts from a CMS (content management system).

```
// TODO: Need to fetch `posts` (by calling some API endpoint)
//       before this page can be prerendered.
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}
```

To fetch this data on prerender, Next.js allows you to `export` an `async` function called `getStaticProps` from the same file. This function gets called at build time and lets you pass fetched data to the page's `props` on prerender.

```
export default function Blog({ posts }) {
  // Render posts ...
}

// This function gets called at build time
export async function getStaticProps() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

To learn more about how `getStaticProps` works, check out the Data Fetching documentation.

Scenario 2: Your page paths depend on external data

Next.js allows you to create pages with **dynamic routes**. For example, you can create a file called `pages/posts/[id].js` to show a single blog post based on `id`. This will allow you to show a blog post with `id: 1` when you access `posts/1`.

> To learn more about dynamic routing, check the Dynamic Routing documentation.

However, which `id` you want to prerender at build time might depend on external data.

Example: suppose that you've only added one blog post (with `id: 1`) to the database. In this case, you'd only want to prerender `posts/1` at build time.

Later, you might add the second post with `id: 2`. Then you'd want to prerender `posts/2` as well.

So your page **paths** that are prerendered depend on external data. To handle this, Next.js lets you `export` an `async` function called `getStaticPaths` from a dynamic page (`pages/posts/[id].js` in this case). This function gets called at build time and lets you specify which paths you want to prerender.

```
// This function gets called at build time
export async function getStaticPaths() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // Get the paths we want to prerender based on posts
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  // We'll prerender only these paths at build time.
  // { fallback: false } means other routes should 404.
  return { paths, fallback: false }
}
```

Also in `pages/posts/[id].js`, you need to export `getStaticProps` so that you can fetch the data about the post with this `id` and use it to prerender the page:

```
export default function Post({ post }) {
  // Render post ...
}

export async function getStaticPaths() {
  // ...
}

// This also gets called at build time
export async function getStaticProps({ params }) {
  // params contains the post `id`.
  // If the route is like /posts/1, then params.id is 1
  const res = await fetch(`https:// ... /posts/${params.id}`)
  const post = await res.json()

  // Pass post data to the page via props
  return { props: { post } }
}
```

To learn more about how `getStaticPaths` works, check out the Data Fetching documentation.

When should I use Static Generation?

We recommend using **Static Generation** (with and without data) whenever possible because your page can be built once and served by CDN, which makes it much faster than having a server render the page on every request.

You can use Static Generation for many types of pages, including:

- Marketing pages
- Blog posts and portfolios
- E-commerce product listings
- Help and documentation

You should ask yourself: "Can I prerender this page **ahead** of a user's request?" If the answer is yes, then you should choose Static Generation.

On the other hand, Static Generation is **not** a good idea if you cannot prerender a page ahead of a user's request. Maybe your page shows frequently updated data, and the page content changes on every request.

In cases like this, you can do one of the following:

- Use Static Generation with **Client-side data fetching:** You can skip prerendering some parts of a page and then use client-side JavaScript to populate them. To learn more about this approach, check out the Data Fetching documentation.
- Use **Server-Side Rendering:** Next.js prerenders a page on each request. It will be slower because the page cannot be cached by a CDN, but the prerendered page will always be up-to-date. We'll talk about this approach below.

Automatic Static Optimization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/automatic-static-optimization
> 📂 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Next.js automatically optimizes your app to be static HTML whenever possible. Learn how it works here.*

Next.js automatically determines that a page is static (can be prerendered) if it has no blocking data requirements. This determination is made by the absence of `getServerSideProps` and `getInitialProps` in the page.

This feature allows Next.js to emit hybrid applications that contain **both server-rendered and statically generated pages**.

> **Good to know**: Statically generated pages are still reactive. Next.js will hydrate your application client-side to give it full interactivity.

One of the main benefits of this feature is that optimized pages require no server-side computation, and can be instantly streamed to the end-user from multiple CDN locations. The result is an *ultra fast* loading experience for your users.

How it works

If `getServerSideProps` or `getInitialProps` is present in a page, Next.js will switch to render the page on-demand, per-request (meaning Server-Side Rendering).

If the above is not the case, Next.js will **statically optimize** your page automatically by prerendering the page to static HTML.

During prerendering, the router's `query` object will be empty since we do not have `query` information to provide during this phase. After hydration, Next.js will trigger an update to your application to provide the route parameters in the `query` object.

The cases where the query will be updated after hydration triggering another render are:

- The page is a dynamic-route.
- The page has query values in the URL.
- Rewrites are configured in your `next.config.js` since these can have parameters that may need to be parsed and provided in the `query`.

To be able to distinguish if the query is fully updated and ready for use, you can leverage the `isReady` field on `next/router`.

> **Good to know**: Parameters added with dynamic routes to a page that's using `getStaticProps` will always be available inside the `query` object.

`next build` will emit `.html` files for statically optimized pages. For example, the result for the page `pages/about.js` would be:

```
// Archivo: Terminal
.next/server/pages/about.html
```

And if you add `getServerSideProps` to the page, it will then be JavaScript, like so:

```
// Archivo: Terminal
.next/server/pages/about.js
```

Caveats

- If you have a custom `App` with `getInitialProps` then this optimization will be turned off in pages without Static Generation.
- If you have a custom `Document` with `getInitialProps` be sure you check if `ctx.req` is defined before assuming the page is server-side rendered. `ctx.req` will be `undefined` for pages that are prerendered.

- Avoid using the `asPath` value on `next/router` in the rendering tree until the router's `isReady` field is `true`. Statically optimized pages only know `asPath` on the client and not the server, so using it as a prop may lead to mismatch errors. The `active-class-name` example demonstrates one way to use `asPath` as a prop.

Client-side Rendering (CSR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Learn how to implement client-side rendering in the Pages Router.*

In Client-Side Rendering (CSR) with React, the browser downloads a minimal HTML page and the JavaScript needed for the page. The JavaScript is then used to update the DOM and render the page. When the application is first loaded, the user may notice a slight delay before they can see the full page, this is because the page isn't fully rendered until all the JavaScript is downloaded, parsed, and executed.

After the page has been loaded for the first time, navigating to other pages on the same website is typically faster, as only necessary data needs to be fetched, and JavaScript can re-render parts of the page without requiring a full page refresh.

In Next.js, there are two ways you can implement client-side rendering:

1. Using React's `useEffect()` hook inside your pages instead of the server-side rendering methods (`getStaticProps` and `getServerSideProps`).
2. Using a data fetching library like SWR or TanStack Query to fetch data on the client (recommended).

Here's an example of using `useEffect()` inside a Next.js page:

```
// Archivo: pages/index.js
import React, { useState, useEffect } from 'react'

export function Page() {
  const [data, setData] = useState(null)

  useEffect(() => {
    const fetchData = async () => {
      const response = await fetch('https://api.example.com/data')
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`)
      }
      const result = await response.json()
      setData(result)
    }

    fetchData().catch((e) => {
      // handle the error as needed
      console.error('An error occurred while fetching the data: ', e)
    })
  }, [])

  return <p>{data ? `Your data: ${data}` : 'Loading ... '}</p>
}
```

In the example above, the component starts by rendering `Loading ...`. Then, once the data is fetched, it re-renders and displays the data.

Although fetching data in a `useEffect` is a pattern you may see in older React Applications, we recommend using a data-fetching library for better performance, caching, optimistic updates, and more. Here's a minimum example using SWR to fetch data on the client:

```
// Archivo: pages/index.js
import useSWR from 'swr'

export function Page() {
  const { data, error, isLoading } = useSWR(
    'https://api.example.com/data',
    fetcher
  )

  if (error) return <p>Failed to load.</p>
  if (isLoading) return <p>Loading ... </p>

  return <p>Your Data: {data}</p>
}
```

> **Good to know**:
>
> Keep in mind that CSR can impact SEO. Some search engine crawlers might not execute JavaScript and therefore only see the initial empty or loading state of your application. It can also lead to performance issues for users with slower internet connections or devices, as they need to wait for all the JavaScript to load and run before they can see the full page. Next.js promotes a hybrid approach that allows you to use a combination of server-side rendering, static site generation, and client-side rendering, **depending on the needs of each page** in your application. In the App Router, you can also use Loading UI with Suspense to show a loading indicator while the page is being rendered.

Rendering

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering
> 📂 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Learn the fundamentals of rendering in React and Next.js.*

By default, Next.js **prerenders** every page. This means that Next.js generates HTML for each page in advance, instead of having it all done by client-side JavaScript. Prerendering can result in better performance and SEO.

Each generated HTML is associated with minimal JavaScript code necessary for that page. When a page is loaded by the browser, its JavaScript code runs and makes the page fully interactive (this process is called hydration in React).

Prerendering

Next.js has two forms of prerendering: **Static Generation** and **Server-side Rendering**. The difference is in **when** it generates the HTML for a page.

- Static Generation: The HTML is generated at **build time** and will be reused on each request.
- Server-side Rendering: The HTML is generated on **each request**.

Importantly, Next.js lets you choose which prerendering form you'd like to use for each page. You can create a "hybrid" Next.js app by using Static Generation for most pages and using Server-side Rendering for others.

We recommend using Static Generation over Server-side Rendering for performance reasons. Statically generated pages can be cached by CDN with no extra configuration to boost performance. However, in some cases, Server-side Rendering might be the only option.

You can also use client-side data fetching along with Static Generation or Server-side Rendering. That means some parts of a page can be rendered entirely by clientside JavaScript. To learn more, take a look at the Data Fetching documentation.

---

getStaticProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-static-props
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data and generate static pages with `getStaticProps`. Learn more about this API for data fetching in Next.js.*

If you export a function called `getStaticProps` (Static Site Generation) from a page, Next.js will prerender this page at build time using the props returned by `getStaticProps`.

```
// Archivo: pages/index.tsx switcher
import type { InferGetStaticPropsType, GetStaticProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```
// Archivo: pages/index.js switcher
export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

> Note that irrespective of rendering type, any `props` will be passed to the page component and can be viewed on the client-side in the initial HTML. This is to allow the page to be hydrated correctly. Make sure that you don't pass any sensitive information that shouldn't be available on the client in `props`.

The `getStaticProps` API reference covers all parameters and props that can be used with `getStaticProps`.

When should I use getStaticProps?

You should use `getStaticProps` if:

- The data required to render the page is available at build time ahead of a user's request
- The data comes from a headless CMS
- The page must be prerendered (for SEO) and be very fast — `getStaticProps` generates `HTML` and `JSON` files, both of which can be cached by a CDN for performance
- The data can be publicly cached (not user-specific). This condition can be bypassed in certain specific situation by using a Proxy to rewrite the path.

When does getStaticProps run

`getStaticProps` always runs on the server and never on the client. You can validate code written inside `getStaticProps` is removed from the client-side bundle with this tool.

- `getStaticProps` always runs during `next build`
- `getStaticProps` runs in the background when using `fallback: true`
- `getStaticProps` is called before initial render when using `fallback: blocking`
- `getStaticProps` runs in the background when using `revalidate`
- `getStaticProps` runs on-demand in the background when using `revalidate()`

When combined with Incremental Static Regeneration, `getStaticProps` will run in the background while the stale page is being revalidated, and the fresh page served to the browser.

`getStaticProps` does not have access to the incoming request (such as query parameters or HTTP headers) as it generates static HTML. If you need access to the request for your page, consider using Proxy in addition to `getStaticProps` .

Using getStaticProps to fetch data from a CMS

The following example shows how you can fetch a list of blog posts from a CMS.

```
// Archivo: pages/blog.tsx switcher
// posts will be populated at build time by getStaticProps()
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}

// This function gets called at build time on server-side.
// It won't be called on client-side, so you can even do
// direct database queries.
export async function getStaticProps() {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

```
// Archivo: pages/blog.js switcher
// posts will be populated at build time by getStaticProps()
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}

// This function gets called at build time on server-side.
// It won't be called on client-side, so you can even do
// direct database queries.
export async function getStaticProps() {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

The `getStaticProps` API reference covers all parameters and props that can be used with `getStaticProps`.

Write server-side code directly

As `getStaticProps` runs only on the server-side, it will never run on the client-side. It won't even be included in the JS bundle for the browser, so you can write direct database queries without them being sent to browsers.

This means that instead of fetching an **API route** from `getStaticProps` (that itself fetches data from an external source), you can write the server-side code directly in `getStaticProps`.

Take the following example. An API route is used to fetch some data from a CMS. That API route is then called directly from `getStaticProps`. This produces an additional call, reducing performance. Instead, the logic for fetching the data from the CMS can be shared by using a `lib/` directory. Then it can be shared with `getStaticProps`.

```
// Archivo: lib/load-posts.js
// The following function is shared
// with getStaticProps and API routes
// from a `lib/` directory
export async function loadPosts() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts/')
  const data = await res.json()

  return data
}
```

```
// Archivo: pages/blog.js
// pages/blog.js
import { loadPosts } from '../lib/load-posts'

// This function runs only on the server side
export async function getStaticProps() {
  // Instead of fetching your `/api` route you can call the same
  // function directly in `getStaticProps`
  const posts = await loadPosts()

  // Props returned will be passed to the page component
  return { props: { posts } }
}
```

Alternatively, if you are **not** using API routes to fetch data, then the `fetch()` API *can* be used directly in `getStaticProps` to fetch data.

To verify what Next.js eliminates from the client-side bundle, you can use the next-code-elimination tool.

Statically generates both HTML and JSON

When a page with `getStaticProps` is prerendered at build time, in addition to the page HTML file, Next.js generates a JSON file holding the result of running `getStaticProps`.

This JSON file will be used in client-side routing through `next/link` or `next/router`. When you navigate to a page that's prerendered using `getStaticProps`, Next.js fetches this JSON file (pre-computed at build time) and uses it as the props for the page component. This means that client-side page transitions will **not** call `getStaticProps` as only the exported JSON is used.

When using Incremental Static Generation, `getStaticProps` will be executed in the background to generate the JSON needed for client-side navigation. You may see this in the form of multiple requests being made for the same page, however, this is intended and has no impact on end-user performance.

Where can I use getStaticProps

`getStaticProps` can only be exported from a **page**. You **cannot** export it from non-page files, `_app`, `_document`, or `_error`.

One of the reasons for this restriction is that React needs to have all the required data before the page is rendered.

Also, you must use export `getStaticProps` as a standalone function — it will **not** work if you add `getStaticProps` as a property of the page component.

> **Good to know**: if you have created a custom app, ensure you are passing the `pageProps` to the page component as shown in the linked document, otherwise the props will be empty.

Runs on every request in development

In development (`next dev`), `getStaticProps` will be called on every request.

Preview Mode

You can temporarily bypass static generation and render the page at **request time** instead of build time using **Preview Mode**. For example, you might be using a headless CMS and want to preview drafts before they're published.

getStaticPaths

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-static-paths
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data and generate static pages with `getStaticPaths`. Learn more about this API for data fetching in Next.js.*

If a page has Dynamic Routes and uses `getStaticProps`, it needs to define a list of paths to be statically generated.

When you export a function called `getStaticPaths` (Static Site Generation) from a page that uses dynamic routes, Next.js will statically prerender all the paths specified by `getStaticPaths`.

```
// Archivo: pages/repo/[name].tsx switcher
import type {
  InferGetStaticPropsType,
  GetStaticProps,
  GetStaticPaths,
} from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticPaths = (async () => {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}) satisfies GetStaticPaths

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```
// Archivo: pages/repo/[name].js switcher
export async function getStaticPaths() {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}

export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

The `getStaticPaths` API reference covers all parameters and props that can be used with `getStaticPaths`.

When should I use getStaticPaths?

You should use `getStaticPaths` if you're statically prerendering pages that use dynamic routes and:

- The data comes from a headless CMS
- The data comes from a database
- The data comes from the filesystem
- The data can be publicly cached (not user-specific)
- The page must be prerendered (for SEO) and be very fast — `getStaticProps` generates `HTML` and `JSON` files, both of which can be cached by a CDN for performance

When does getStaticPaths run

`getStaticPaths` will only run during build in production, it will not be called during runtime. You can validate code written inside `getStaticPaths` is removed from the client-side bundle with this tool.

How does getStaticProps run with regards to getStaticPaths

- `getStaticProps` runs during `next build` for any `paths` returned during build
- `getStaticProps` runs in the background when using `fallback: true`
- `getStaticProps` is called before initial render when using `fallback: blocking`

Where can I use getStaticPaths

- `getStaticPaths` **must** be used with `getStaticProps`
- You **cannot** use `getStaticPaths` with `getServerSideProps`
- You can export `getStaticPaths` from a Dynamic Route that also uses `getStaticProps`
- You **cannot** export `getStaticPaths` from non-page file (e.g. your `components` folder)
- You must export `getStaticPaths` as a standalone function, and not a property of the page component

Runs on every request in development

In development (`next dev`), `getStaticPaths` will be called on every request.

Generating paths on-demand

`getStaticPaths` allows you to control which pages are generated during the build instead of on-demand with `fallback`. Generating more pages during a build will cause slower builds.

You can defer generating all pages on-demand by returning an empty array for `paths`. This can be especially helpful when deploying your Next.js application to multiple environments. For example, you can have faster builds by generating all pages on-demand for previews (but not production builds). This is helpful for sites with hundreds/thousands of static pages.

```
// Archivo: pages/posts/[id].js
export async function getStaticPaths() {
  // When this is true (in preview environments) don't
  // prerender any static pages
  // (faster builds, but slower initial page load)
  if (process.env.SKIP_BUILD_STATIC_GENERATION) {
    return {
      paths: [],
      fallback: 'blocking',
    }
  }

  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // Get the paths we want to prerender based on posts
  // In production environments, prerender all pages
  // (slower builds, but faster initial page load)
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  // { fallback: false } means other routes should 404
  return { paths, fallback: false }
}
```

---

getServerSideProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-server-side-props
> 📁 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data on each request with* `getServerSideProps`.

`getServerSideProps` is a Next.js function that can be used to fetch data and render the contents of a page at request time.

Example

You can use `getServerSideProps` by exporting it from a Page Component. The example below shows how you can fetch data from a 3rd party API in `getServerSideProps`, and pass the data to the page as props:

```tsx
// Archivo: pages/index.tsx switcher
import type { InferGetServerSidePropsType, GetServerSideProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getServerSideProps = (async () => {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo: Repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}) satisfies GetServerSideProps<{ repo: Repo }>

export default function Page({
  repo,
}: InferGetServerSidePropsType<typeof getServerSideProps>) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

```jsx
// Archivo: pages/index.js switcher
export async function getServerSideProps() {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}

export default function Page({ repo }) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

When should I use `getServerSideProps`?

You should use `getServerSideProps` if you need to render a page that relies on personalized user data, or information that can only be known at request time. For example, `authorization` headers or a geolocation.

If you do not need to fetch the data at request time, or would prefer to cache the data and prerendered HTML, we recommend using `getStaticProps`.

Behavior

- `getServerSideProps` runs on the server.
- `getServerSideProps` can only be exported from a **page**.
- `getServerSideProps` returns JSON.
- When a user visits a page, `getServerSideProps` will be used to fetch data at request time, and the data is used to render the initial HTML of the page.
- `props` passed to the page component can be viewed on the client as part of the initial HTML. This is to allow the page to be hydrated correctly. Make sure that you don't pass any sensitive information that shouldn't be available on the client in `props`.
- When a user visits the page through `next/link` or `next/router`, Next.js sends an API request to the server, which runs `getServerSideProps`.

- You do not have to call a Next.js API Route to fetch data when using `getServerSideProps` since the function runs on the server. Instead, you can call a CMS, database, or other third-party APIs directly from inside `getServerSideProps`.

> **Good to know:**
>
> - See `getServerSideProps` API reference for parameters and props that can be used with `getServerSideProps`.
> - You can use the next-code-elimination tool to verify what Next.js eliminates from the client-side bundle.

Error Handling

If an error is thrown inside `getServerSideProps`, it will show the `pages/500.js` file. Check out the documentation for 500 page to learn more on how to create it. During development, this file will not be used and the development error overlay will be shown instead.

Edge Cases

Caching with Server-Side Rendering (SSR)

You can use caching headers (`Cache-Control`) inside `getServerSideProps` to cache dynamic responses. For example, using `stale-while-revalidate`.

```
// This value is considered fresh for ten seconds (s-maxage=10).
// If a request is repeated within the next 10 seconds, the previously
// cached value will still be fresh. If the request is repeated before 59 seconds,
// the cached value will be stale but still render (stale-while-revalidate=59).
//
// In the background, a revalidation request will be made to populate the cache
// with a fresh value. If you refresh the page, you will see the new value.
export async function getServerSideProps({ req, res }) {
  res.setHeader(
    'Cache-Control',
    'public, s-maxage=10, stale-while-revalidate=59'
  )

  return {
    props: {},
  }
}
```

However, before reaching for `cache-control`, we recommend seeing if `getStaticProps` with ISR is a better fit for your use case.

Client-side Fetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/client-side
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Learn about client-side data fetching, and how to use SWR, a data fetching React Hook library that handles caching, revalidation, focus tracking, refetching on interval and more.*

Client-side data fetching is useful when your page doesn't require SEO indexing, when you don't need to prerender your data, or when the content of your pages needs to update frequently. Unlike the server-side rendering APIs, you can use client-side data fetching at the component level.

If done at the page level, the data is fetched at runtime, and the content of the page is updated as the data changes. When used at the component level, the data is fetched at the time of the component mount, and the content of the component is updated as the data changes.

It's important to note that using client-side data fetching can affect the performance of your application and the load speed of your pages. This is because the data fetching is done at the time of the component or pages mount, and the data is not cached.

Client-side data fetching with useEffect

The following example shows how you can fetch data on the client side using the useEffect hook.

```
import { useState, useEffect } from 'react'

function Profile() {
  const [data, setData] = useState(null)
  const [isLoading, setLoading] = useState(true)

  useEffect(() => {
    fetch('/api/profile-data')
      .then((res) => res.json())
      .then((data) => {
        setData(data)
        setLoading(false)
      })
  }, [])

  if (isLoading) return <p>Loading ...</p>
  if (!data) return <p>No profile data</p>

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.bio}</p>
    </div>
  )
}
```

Client-side data fetching with SWR

The team behind Next.js has created a React Hook library for data fetching called **SWR**. It is **highly recommended** if you are fetching data on the client-side. It handles caching, revalidation, focus tracking, refetching on intervals, and more.

Using the same example as above, we can now use SWR to fetch the profile data. SWR will automatically cache the data for us and will revalidate the data if it becomes stale.

For more information on using SWR, check out the SWR docs.

```
import useSWR from 'swr'

const fetcher = (...args) => fetch(...args).then((res) => res.json())

function Profile() {
  const { data, error } = useSWR('/api/profile-data', fetcher)

  if (error) return <div>Failed to load</div>
  if (!data) return <div>Loading ...</div>

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.bio}</p>
    </div>
  )
}
```

Data Fetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Next.js allows you to fetch data in multiple ways, with prerendering, server-side rendering or static-site generation, and incremental static regeneration. Learn how to manage your application data in Next.js.*

Data fetching in Next.js allows you to render your content in different ways, depending on your application's use case. These include prerendering with **Server-side Rendering** or **Static Generation**, and updating or creating content at runtime with **Incremental Static Regeneration**.

Examples

- Agility CMS Example (Demo)
- Builder.io Example (Demo)
- ButterCMS Example (Demo)
- Contentful Example (Demo)
- Cosmic Example (Demo)
- DatoCMS Example (Demo)
- DotCMS Example (Demo)
- Drupal Example (Demo)
- Enterspeed Example (Demo)
- GraphCMS Example (Demo)
- Keystone Example (Demo)
- Kontent.ai Example (Demo)
- Makeswift Example (Demo)
- Plasmic Example (Demo)
- Prepr Example (Demo)
- Prismic Example (Demo)
- Sanity Example (Demo)
- Sitecore XM Cloud Example (Demo)
- Storyblok Example (Demo)
- Strapi Example (Demo)
- TakeShape Example (Demo)
- Tina Example (Demo)
- Umbraco Example (Demo)
- Umbraco Heartcore Example (Demo)
- Webiny Example (Demo)
- WordPress Example (Demo)
- Blog Starter Example (Demo)
- Static Tweet (Demo)

Error Handling

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/configuring/error-handling
> 📂 **Sección:** Pages > Building Your Application > Configuring
> 📝 **Descripción:** *Handle errors in your Next.js app.*

This documentation explains how you can handle development, server-side, and client-side errors.

Handling Errors in Development

When there is a runtime error during the development phase of your Next.js application, you will encounter an **overlay**. It is a modal that covers the webpage. It is **only** visible when the development server runs using `next dev` via `pnpm dev`, `npm run dev`, `yarn dev`, or `bun dev` and will not be shown in production. Fixing the error will automatically dismiss the overlay.

Handling Server Errors

Next.js provides a static 500 page by default to handle server-side errors that occur in your application. You can also customize this page by creating a `pages/500.js` file.

Having a 500 page in your application does not show specific errors to the app user.

You can also use 404 page to handle specific runtime error like `file not found`.

Handling Client Errors

React Error Boundaries is a graceful way to handle a JavaScript error on the client so that the other parts of the application continue working. In addition to preventing the page from crashing, it allows you to provide a custom fallback component and even log error information.

To use Error Boundaries for your Next.js application, you must create a class component `ErrorBoundary` and wrap the `Component` prop in the `pages/_app.js` file. This component will be responsible to:

- Render a fallback UI after an error is thrown
- Provide a way to reset the Application's state
- Log error information

You can create an `ErrorBoundary` class component by extending `React.Component`. For example:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props)

    // Define a state variable to track whether is an error or not
    this.state = { hasError: false }
  }
  static getDerivedStateFromError(error) {
    // Update state so the next render will show the fallback UI

    return { hasError: true }
  }
  componentDidCatch(error, errorInfo) {
    // You can use your own error logging service here
    console.log({ error, errorInfo })
  }
  render() {
    // Check if the error is thrown
    if (this.state.hasError) {
      // You can render any custom fallback UI
      return (
        <div>
          <h2>Oops, there is an error!</h2>
          <button
            type="button"
            onClick={() => this.setState({ hasError: false })}
          >
            Try again?
          </button>
        </div>
      )
    }

    // Return children components in case of no error

    return this.props.children
  }
}

export default ErrorBoundary
```

The `ErrorBoundary` component keeps track of an `hasError` state. The value of this state variable is a boolean. When the value of `hasError` is `true`, then the `ErrorBoundary` component will render a fallback UI. Otherwise, it will render the children components.

After creating an `ErrorBoundary` component, import it in the `pages/_app.js` file to wrap the `Component` prop in your Next.js application.

```
// Import the ErrorBoundary component
import ErrorBoundary from '../components/ErrorBoundary'

function MyApp({ Component, pageProps }) {
  return (
    // Wrap the Component prop with ErrorBoundary component
    <ErrorBoundary>
      <Component {...pageProps} />
    </ErrorBoundary>
  )
}

export default MyApp
```

You can learn more about Error Boundaries in React's documentation.

Reporting Errors

To monitor client errors, use a service like Sentry, Bugsnag or Datadog.

Configuring

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/configuring
> 📁 **Sección:** Pages > Building Your Application > Configuring
> 📝 **Descripción:** *Learn how to configure your Next.js application.*

Next.js allows you to customize your project to meet specific requirements. This includes integrations with TypeScript, ESlint, and more, as well as internal configuration options such as Absolute Imports and Environment Variables.

Building Your Application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application
> 📁 **Sección:** Pages > Building Your Application
> 📝 **Descripción:** *Learn how to use Next.js features to build your application.*

turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/turbopack
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Next.js with Turbopack-specific options*

turbopackChunking

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/turbopackChunking
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure how Turbopack splits your client-side JavaScript into chunks in production.*

Turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/turbopack
> 📁 **Sección:** Pages > Api Reference
> 📝 **Descripción:** *Turbopack is an incremental bundler optimized for JavaScript and TypeScript, written in Rust, and built into Next.js.*

Capítulo 11: Referencia de API Completa (App Router)

> *Catálogo exhaustivo de componentes integrados, convenciones de archivos, funciones de servidor, hooks y configuración.*

Font Module

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components/font
> 📁 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *Optimizing loading web fonts with the built-in* `next/font` *loaders.*

`next/font` automatically optimizes your fonts (including custom fonts) and removes external network requests for improved privacy and performance.

It includes **built-in automatic self-hosting** for any font file. This means you can optimally load web fonts with no layout shift.

You can also conveniently use all Google Fonts. CSS and font files are downloaded at build time and self-hosted with the rest of your static assets. **No requests are sent to Google by the browser.**

```
// Archivo: app/layout.tsx switcher
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:** To use the font in all your pages, add it to `_app.js` file under `/pages` as shown below:

```
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({ subsets: ['latin'] })

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={inter.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

🎥 **Watch:** Learn more about using `next/font` → YouTube (6 minutes).

Reference

Key	`font/google`	`font/local`	Type	Required
`src`			String or Array of Objects	Yes
`weight`			String or Array	Required/Optional
`style`			String or Array	-
`subsets`			Array of Strings	-
`axes`			Array of Strings	-
`display`			String	-
`preload`			Boolean	-
`fallback`			Array of Strings	-
`adjustFontFallback`			Boolean or String	-
`variable`			String	-
`declarations`			Array of Objects	-

`src`

The path of the font file as a string or an array of objects (with type `Array<{path: string, weight?: string, style?: string}>`) relative to the directory where the font loader function is called.

Used in `next/font/local`

- Required

Examples:

- `src:'./fonts/my-font.woff2'` where `my-font.woff2` is placed in a directory named `fonts` inside the `app` directory
- `src:[{path: './inter/Inter-Thin.ttf', weight: '100',},{path: './inter/Inter-Regular.ttf',weight: '400',},{path: './inter/Inter-Bold-Italic.ttf', weight: '700',style: 'italic',},]`
- if the font loader function is called in `app/page.tsx` using `src:'../styles/fonts/my-font.ttf'`, then `my-font.ttf` is placed in `styles/fonts` at the root of the project

`weight`

The font `weight` with the following possibilities:

- A string with possible values of the weights available for the specific font or a range of values if it's a variable font
- An array of weight values if the font is not a variable google font. It applies to `next/font/google` only.

Used in `next/font/google` and `next/font/local`

- Required if the font being used is **not** variable

Examples:

- `weight: '400'`: A string for a single weight value - for the font `Inter`, the possible values are `'100'`, `'200'`, `'300'`, `'400'`, `'500'`, `'600'`, `'700'`, `'800'`, `'900'` or `'variable'` where `'variable'` is the default)
- `weight: '100 900'`: A string for the range between `100` and `900` for a variable font
- `weight: ['100','400','900']`: An array of 3 possible values for a non variable font

`style`

The font `style` with the following possibilities:

- A string value with default value of `'normal'`
- An array of style values if the font is not a variable google font. It applies to `next/font/google` only.

Used in `next/font/google` and `next/font/local`

- Optional

Examples:

- `style: 'italic'`: A string - it can be `normal` or `italic` for `next/font/google`
- `style: 'oblique'`: A string - it can take any value for `next/font/local` but is expected to come from standard font styles
- `style: ['italic','normal']`: An array of 2 values for `next/font/google` - the values are from `normal` and `italic`

`subsets`

The font `subsets` defined by an array of string values with the names of each subset you would like to be preloaded. Fonts specified via `subsets` will have a link preload tag injected into the head when the `preload` option is true, which is the default.

Used in `next/font/google`

- Optional

Examples:

- `subsets: ['latin']`: An array with the subset `latin`

You can find a list of all subsets on the Google Fonts page for your font.

`axes`

Some variable fonts have extra `axes` that can be included. By default, only the font weight is included to keep the file size down. The possible values of `axes` depend on the specific font.

Used in `next/font/google`

- Optional

Examples:

- `axes: ['slnt']`: An array with value `slnt` for the `Inter` variable font which has `slnt` as additional `axes` as shown here. You can find the possible `axes` values for your font by using the filter on the Google variable fonts page and looking for axes other than `wght`

`display`

The font `display` with possible string values of `'auto'`, `'block'`, `'swap'`, `'fallback'` or `'optional'` with default value of `'swap'`.

Used in `next/font/google` and `next/font/local`

- Optional

Examples:

- `display: 'optional'`: A string assigned to the `optional` value

`preload`

A boolean value that specifies whether the font should be preloaded or not. The default is `true`.

Used in `next/font/google` and `next/font/local`

- Optional

Examples:

- `preload: false`

`fallback`

The fallback font to use if the font cannot be loaded. An array of strings of fallback fonts with no default.

- Optional

Used in `next/font/google` and `next/font/local`

Examples:

- `fallback: ['system-ui', 'arial']`: An array setting the fallback fonts to `system-ui` or `arial`

`adjustFontFallback`

- For `next/font/google`: A boolean value that sets whether an automatic fallback font should be used to reduce Cumulative Layout Shift. The default is `true`.
- For `next/font/local`: A string or boolean `false` value that sets whether an automatic fallback font should be used to reduce Cumulative Layout Shift. The possible values are `'Arial'`, `'Times New Roman'` or `false`. The default is `'Arial'`.

Used in `next/font/google` and `next/font/local`

- Optional

Examples:

- `adjustFontFallback: false`: for `next/font/google`
- `adjustFontFallback: 'Times New Roman'`: for `next/font/local`

`variable`

A string value to define the CSS variable name to be used if the style is applied with the CSS variable method.

Used in `next/font/google` and `next/font/local`

- Optional

Examples:

- `variable: '--my-font'`: The CSS variable `--my-font` is declared

`declarations`

An array of font face descriptor key-value pairs that define the generated `@font-face` further.

Used in `next/font/local`

- Optional

Examples:

- `declarations: [{ prop: 'ascent-override', value: '90%' }]`

Examples

Google Fonts

To use a Google font, import it from `next/font/google` as a function. We recommend using variable fonts for the best performance and flexibility.

```
// Archivo: app/layout.tsx switcher
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  )
}
```

If you can't use a variable font, you will **need to specify a weight**:

```tsx
// Archivo: app/layout.tsx switcher
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={roboto.className}>
      <body>{children}</body>
    </html>
  )
}
```

```jsx
// Archivo: app/layout.js switcher
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
  display: 'swap',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={roboto.className}>
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:** To use the font in all your pages, add it to `_app.js` file under `/pages` as shown below:

```
import { Inter } from 'next/font/google'

// If loading a variable font, you don't need to specify the font weight
const inter = Inter({ subsets: ['latin'] })

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={inter.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

If you can't use a variable font, you will **need to specify a weight**:

```
import { Roboto } from 'next/font/google'

const roboto = Roboto({
  weight: '400',
  subsets: ['latin'],
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={roboto.className}>
      <Component {...pageProps} />
    </main>
  )
}
```

You can specify multiple weights and/or styles by using an array:

```
// Archivo: app/layout.js
const roboto = Roboto({
  weight: ['400', '700'],
  style: ['normal', 'italic'],
  subsets: ['latin'],
  display: 'swap',
})
```

Good to know: Use an underscore (_) for font names with multiple words. E.g. `Roboto Mono` should be imported as `Roboto_Mono`.

📘 **Nota para Pages Router:**

Apply the font in `<head>`

You can also use the font without a wrapper and `className` by injecting it inside the `<head>` as follows:

```
import { Inter } from 'next/font/google'

const inter = Inter({ subsets: ['latin'] })

export default function MyApp({ Component, pageProps }) {
  return (
    <>
      <style jsx global>{`
        html {
          font-family: ${inter.style.fontFamily};
        }
      `}</style>
      <Component {...pageProps} />
    </>
  )
}
```

Single page usage

To use the font on a single page, add it to the specific page as shown below:

```
import { Inter } from 'next/font/google'

const inter = Inter({ subsets: ['latin'] })

export default function Home() {
  return (
    <div className={inter.className}>
      <p>Hello World</p>
    </div>
  )
}
```

Specifying a subset

Google Fonts are automatically subset. This reduces the size of the font file and improves performance. You'll need to define which of these subsets you want to preload. Failing to specify any subsets while `preload` is `true` will result in a warning.

This can be done by adding it to the function call:

```
// Archivo: app/layout.tsx switcher
const inter = Inter({ subsets: ['latin'] })
```

```
// Archivo: app/layout.js switcher
const inter = Inter({ subsets: ['latin'] })
```

📘 **Nota para Pages Router:**

```
const inter = Inter({ subsets: ['latin'] })
```

View the Font API Reference for more information.

Using Multiple Fonts

You can import and use multiple fonts in your application. There are two approaches you can take.

The first approach is to create a utility function that exports a font, imports it, and applies its `className` where needed. This ensures the font is preloaded only when it's rendered:

```
// Archivo: app/fonts.ts switcher
import { Inter, Roboto_Mono } from 'next/font/google'

export const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
})
```

```
// Archivo: app/fonts.js switcher
import { Inter, Roboto_Mono } from 'next/font/google'

export const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})

export const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
})
```

```
// Archivo: app/layout.tsx switcher
import { inter } from './fonts'

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={inter.className}>
      <body>
        <div>{children}</div>
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { inter } from './fonts'

export default function Layout({ children }) {
  return (
    <html lang="en" className={inter.className}>
      <body>
        <div>{children}</div>
      </body>
    </html>
  )
}
```

```tsx
// Archivo: app/page.tsx switcher
import { roboto_mono } from './fonts'

export default function Page() {
  return (
    <>
      <h1 className={roboto_mono.className}>My page</h1>
    </>
  )
}
```

```jsx
// Archivo: app/page.js switcher
import { roboto_mono } from './fonts'

export default function Page() {
  return (
    <>
      <h1 className={roboto_mono.className}>My page</h1>
    </>
  )
}
```

In the example above, `Inter` will be applied globally, and `Roboto Mono` can be imported and applied as needed.

Alternatively, you can create a CSS variable and use it with your preferred CSS solution:

```tsx
// Archivo: app/layout.tsx switcher
import { Inter, Roboto_Mono } from 'next/font/google'
import styles from './global.css'

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-inter',
  display: 'swap',
})

const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  variable: '--font-roboto-mono',
  display: 'swap',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={`${inter.variable} ${roboto_mono.variable}`}>
      <body>
        <h1>My App</h1>
        <div>{children}</div>
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { Inter, Roboto_Mono } from 'next/font/google'

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-inter',
  display: 'swap',
})

const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  variable: '--font-roboto-mono',
  display: 'swap',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={`${inter.variable} ${roboto_mono.variable}`}>
      <body>
        <h1>My App</h1>
        <div>{children}</div>
      </body>
    </html>
  )
}
```

```
// Archivo: app/global.css
html {
  font-family: var(--font-inter);
}

h1 {
  font-family: var(--font-roboto-mono);
}
```

In the example above, `Inter` will be applied globally, and any `<h1>` tags will be styled with `Roboto Mono`.

> **Recommendation**: Use multiple fonts conservatively since each new font is an additional resource the client has to download.

Local Fonts

Import `next/font/local` and specify the `src` of your local font file. We recommend using variable fonts for the best performance and flexibility.

```
// Archivo: app/layout.tsx switcher
import localFont from 'next/font/local'

// Font files can be colocated inside of `app`
const myFont = localFont({
  src: './my-font.woff2',
  display: 'swap',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en" className={myFont.className}>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import localFont from 'next/font/local'

// Font files can be colocated inside of `app`
const myFont = localFont({
  src: './my-font.woff2',
  display: 'swap',
})

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={myFont.className}>
      <body>{children}</body>
    </html>
  )
}
```

> 📘 **Nota para Pages Router:**
>
> ```
> import localFont from 'next/font/local'
>
> // Font files can be colocated inside of `pages`
> const myFont = localFont({ src: './my-font.woff2' })
>
> export default function MyApp({ Component, pageProps }) {
>   return (
>     <main className={myFont.className}>
>       <Component {...pageProps} />
>     </main>
>   )
> }
> ```

If you want to use multiple files for a single font family, `src` can be an array:

```
const roboto = localFont({
  src: [
    {
      path: './Roboto-Regular.woff2',
      weight: '400',
      style: 'normal',
    },
    {
      path: './Roboto-Italic.woff2',
      weight: '400',
      style: 'italic',
    },
    {
      path: './Roboto-Bold.woff2',
      weight: '700',
      style: 'normal',
    },
    {
      path: './Roboto-BoldItalic.woff2',
      weight: '700',
      style: 'italic',
    },
  ],
})
```

View the Font API Reference for more information.

With Tailwind CSS

`next/font` integrates seamlessly with Tailwind CSS using CSS variables.

In the example below, we use the `Inter` and `Roboto_Mono` fonts from `next/font/google` (you can use any Google Font or Local Font). Use the `variable` option to define a CSS variable name, such as `inter` and `roboto_mono` for these fonts, respectively. Then, apply

`inter.variable` and `roboto_mono.variable` to include the CSS variables in your HTML document.

> **Good to know**: You can add these variables to the `<html>` or `<body>` tag, depending on your preference, styling needs or project requirements.

```
// Archivo: app/layout.tsx switcher
import { Inter, Roboto_Mono } from 'next/font/google'

const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-inter',
})

const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-roboto-mono',
})

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html
      lang="en"
      className={`${inter.variable} ${roboto_mono.variable} antialiased`}
    >
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { Inter, Roboto_Mono } from 'next/font/google'

const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-inter',
})

const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-roboto-mono',
})

export default function RootLayout({ children }) {
  return (
    <html
      lang="en"
      className={`${inter.variable} ${roboto_mono.variable} antialiased`}
    >
      <body>{children}</body>
    </html>
  )
}
```

📘 **Nota para Pages Router:**

```
import { Inter } from 'next/font/google'

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-inter',
})

const roboto_mono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-roboto-mono',
})

export default function MyApp({ Component, pageProps }) {
  return (
    <main className={`${inter.variable} ${roboto_mono.variable} font-sans`}>
      <Component {...pageProps} />
    </main>
  )
}
```

Finally, add the CSS variable to your Tailwind CSS config:

```
// Archivo: global.css
@import 'tailwindcss';

@theme inline {
  --font-sans: var(--font-inter);
  --font-mono: var(--font-roboto-mono);
}
```

Tailwind CSS v3

```
// Archivo: tailwind.config.js
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    './pages/**/*.{js,ts,jsx,tsx}',
    './components/**/*.{js,ts,jsx,tsx}',
    './app/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ['var(--font-inter)'],
        mono: ['var(--font-roboto-mono)'],
      },
    },
  },
  plugins: [],
}
```

You can now use the `font-sans` and `font-mono` utility classes to apply the font to your elements.

```
<p class="font-sans ...">The quick brown fox ...</p>
<p class="font-mono ...">The quick brown fox ...</p>
```

Applying Styles

You can apply the font styles in three ways:

- `className`

- `style`
- CSS Variables

`className`

Returns a read-only CSS `className` for the loaded font to be passed to an HTML element.

```
<p className={inter.className}>Hello, Next.js!</p>
```

`style`

Returns a read-only CSS `style` object for the loaded font to be passed to an HTML element, including `style.fontFamily` to access the font family name and fallback fonts.

```
<p style={inter.style}>Hello World</p>
```

CSS Variables

If you would like to set your styles in an external style sheet and specify additional options there, use the CSS variable method.

In addition to importing the font, also import the CSS file where the CSS variable is defined and set the variable option of the font loader object as follows:

```
// Archivo: app/page.tsx switcher
import { Inter } from 'next/font/google'
import styles from '../styles/component.module.css'

const inter = Inter({
  variable: '--font-inter',
})
```

```
// Archivo: app/page.js switcher
import { Inter } from 'next/font/google'
import styles from '../styles/component.module.css'

const inter = Inter({
  variable: '--font-inter',
})
```

To use the font, set the `className` of the parent container of the text you would like to style to the font loader's `variable` value and the `className` of the text to the `styles` property from the external CSS file.

```
// Archivo: app/page.tsx switcher
<main className={inter.variable}>
  <p className={styles.text}>Hello World</p>
</main>
```

```
// Archivo: app/page.js switcher
<main className={inter.variable}>
  <p className={styles.text}>Hello World</p>
</main>
```

Define the `text` selector class in the `component.module.css` CSS file as follows:

```
// Archivo: styles/component.module.css
.text {
  font-family: var(--font-inter);
  font-weight: 200;
  font-style: italic;
}
```

In the example above, the text `Hello World` is styled using the `Inter` font and the generated font fallback with `font-weight: 200` and `font-style: italic`.

Using a font definitions file

Every time you call the `localFont` or Google font function, that font will be hosted as one instance in your application. Therefore, if you need to use the same font in multiple places, you should load it in one place and import the related font object where you need it. This is done using a font definitions file.

For example, create a `fonts.ts` file in a `styles` folder at the root of your app directory.

Then, specify your font definitions as follows:

```
// Archivo: styles/fonts.ts switcher
import { Inter, Lora, Source_Sans_3 } from 'next/font/google'
import localFont from 'next/font/local'

// define your variable fonts
const inter = Inter()
const lora = Lora()
// define 2 weights of a non-variable font
const sourceCodePro400 = Source_Sans_3({ weight: '400' })
const sourceCodePro700 = Source_Sans_3({ weight: '700' })
// define a custom local font where GreatVibes-Regular.ttf is stored in the styles folder
const greatVibes = localFont({ src: './GreatVibes-Regular.ttf' })

export { inter, lora, sourceCodePro400, sourceCodePro700, greatVibes }
```

```
// Archivo: styles/fonts.js switcher
import { Inter, Lora, Source_Sans_3 } from 'next/font/google'
import localFont from 'next/font/local'

// define your variable fonts
const inter = Inter()
const lora = Lora()
// define 2 weights of a non-variable font
const sourceCodePro400 = Source_Sans_3({ weight: '400' })
const sourceCodePro700 = Source_Sans_3({ weight: '700' })
// define a custom local font where GreatVibes-Regular.ttf is stored in the styles folder
const greatVibes = localFont({ src: './GreatVibes-Regular.ttf' })

export { inter, lora, sourceCodePro400, sourceCodePro700, greatVibes }
```

You can now use these definitions in your code as follows:

```
// Archivo: app/page.tsx switcher
import { inter, lora, sourceCodePro700, greatVibes } from '../styles/fonts'

export default function Page() {
  return (
    <div>
      <p className={inter.className}>Hello world using Inter font</p>
      <p style={lora.style}>Hello world using Lora font</p>
      <p className={sourceCodePro700.className}>
        Hello world using Source_Sans_3 font with weight 700
      </p>
      <p className={greatVibes.className}>My title in Great Vibes font</p>
    </div>
  )
}
```

```
// Archivo: app/page.js switcher
import { inter, lora, sourceCodePro700, greatVibes } from '../styles/fonts'

export default function Page() {
  return (
    <div>
      <p className={inter.className}>Hello world using Inter font</p>
      <p style={lora.style}>Hello world using Lora font</p>
      <p className={sourceCodePro700.className}>
        Hello world using Source_Sans_3 font with weight 700
      </p>
      <p className={greatVibes.className}>My title in Great Vibes font</p>
    </div>
  )
}
```

To make it easier to access the font definitions in your code, you can define a path alias in your `tsconfig.json` or `jsconfig.json` files as follows:

```
// Archivo: tsconfig.json
{
  "compilerOptions": {
    "paths": {
      "@/fonts": ["./styles/fonts"]
    }
  }
}
```

You can now import any font definition as follows:

```
// Archivo: app/about/page.tsx switcher
import { greatVibes, sourceCodePro400 } from '@/fonts'
```

```
// Archivo: app/about/page.js switcher
import { greatVibes, sourceCodePro400 } from '@/fonts'
```

Preloading

When a font function is called on a page of your site, it is not globally available and preloaded on all routes. Rather, the font is only preloaded on the related routes based on the type of file where it is used:

- If it's a unique page, it is preloaded on the unique route for that page.
- If it's a layout, it is preloaded on all the routes wrapped by the layout.
- If it's the root layout, it is preloaded on all routes.

> 📘 **Nota para Pages Router:** When a font function is called on a page of your site, it is not globally available and preloaded on all routes. Rather, the font is only preloaded on the related route/s based on the type of file where it is used:
>
> - if it's a unique page, it is preloaded on the unique route for that page
> - if it's in the custom App, it is preloaded on all the routes of the site under `/pages`

Version Changes

Version	Changes
`v13.2.0`	`@next/font` renamed to `next/font`. Installation no longer required.
`v13.0.0`	`@next/font` was added.

Form Component

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components/form
> 📂 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *Learn how to use the `<Form>` component to handle form submissions and search params updates with client-side navigation.*

The `<Form>` component extends the HTML `<form>` element to provide **prefetching** of loading UI, **client-side navigation** on submission, and **progressive enhancement**.

It's useful for forms that update URL search params as it reduces the boilerplate code needed to achieve the above.

Basic usage:

```
// Archivo: /app/ui/search.tsx switcher
import Form from 'next/form'

export default function Page() {
  return (
    <Form action="/search">

      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

```
// Archivo: /app/ui/search.js switcher
import Form from 'next/form'

export default function Search() {
  return (
    <Form action="/search">

      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

📘 **Nota para Pages Router:**

```
import Form from 'next/form'

export default function Page() {
  return (
    <Form action="/search">

      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

```
import Form from 'next/form'

export default function Search() {
  return (
    <Form action="/search">

      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

Reference

The behavior of the `<Form>` component depends on whether the `action` prop is passed a `string` or `function`.

- When `action` is a **string**, the `<Form>` behaves like a native HTML form that uses a `GET` method. The form data is encoded into the URL as search params, and when the form is submitted, it navigates to the specified URL. In addition, Next.js:
  - Prefetches the path when the form becomes visible, this preloads shared UI (e.g. `layout.js` and `loading.js`), resulting in faster navigation.
  - Performs a client-side navigation instead of a full page reload when the form is submitted. This retains shared UI and client-side state.
- When `action` is a **function** (Server Action), `<Form>` behaves like a React form, executing the action when the form is submitted.

📘 **Nota para Pages Router:**

- When `action` is a **string**, the `<Form>` behaves like a native HTML form that uses a `GET` method. The form data is encoded into the URL as search params, and when the form is submitted, it navigates to the specified URL. In addition, Next.js:
  - Performs a client-side navigation instead of a full page reload when the form is submitted. This retains shared UI and client-side state.

`action` (string) Props

> 📘 **Nota para Pages Router:** When `action` is a string, the `<Form>` component supports the following props:

Prop	Example	Type	Required
`action`	`action="/search"`	`string` (URL or relative path)	Yes
`replace`	`replace={false}`	`boolean`	-
`scroll`	`scroll={true}`	`boolean`	-

- **`action`**: The URL or path to navigate to when the form is submitted.
  - An empty string `""` will navigate to the same route with updated search params.
- **`replace`**: Replaces the current history state instead of pushing a new one to the browser's history stack. Default is `false`.
- **`scroll`**: Controls the scroll behavior during navigation. Defaults to `true`, this means it will scroll to the top of the new route, and maintain the scroll position for backwards and forwards navigation.

When `action` is a string, the `<Form>` component supports the following props:

Prop	Example	Type	Required
`action`	`action="/search"`	`string` (URL or relative path)	Yes
`replace`	`replace={false}`	`boolean`	-
`scroll`	`scroll={true}`	`boolean`	-
`prefetch`	`prefetch={true}`	`boolean`	-

- **`action`**: The URL or path to navigate to when the form is submitted.
  - An empty string `""` will navigate to the same route with updated search params.
- **`replace`**: Replaces the current history state instead of pushing a new one to the browser's history stack. Default is `false`.
- **`scroll`**: Controls the scroll behavior during navigation. Defaults to `true`, this means it will scroll to the top of the new route, and maintain the scroll position for backwards and forwards navigation.
- **`prefetch`**: Controls whether the path should be prefetched when the form becomes visible in the user's viewport. Defaults to `true`.

`action` (function) Props

When `action` is a function, the `<Form>` component supports the following prop:

Prop	Example	Type	Required
`action`	`action={myAction}`	`function` (Server Action)	Yes

- **`action`**: The Server Action to be called when the form is submitted. See the React docs for more.

> **Good to know**: When `action` is a function, the `replace` and `scroll` props are ignored.

Caveats

- **`formAction`**: Can be used in a `<button>` or `<input type="submit">` fields to override the `action` prop. Next.js will perform a client-side navigation, however, this approach doesn't support prefetching.
  - When using `basePath`, you must also include it in the `formAction` path. e.g. `formAction="/base-path/search"`.
- **`key`**: Passing a `key` prop to a string `action` is not supported. If you'd like to trigger a re-render or perform a mutation, consider using a function `action` instead.

- `onSubmit`: Can be used to handle form submission logic. However, calling `event.preventDefault()` will override `<Form>` behavior such as navigating to the specified URL.
- `method`, `encType`, `target`: Are not supported as they override `<Form>` behavior.
  - Similarly, `formMethod`, `formEncType`, and `formTarget` can be used to override the `method`, `encType`, and `target` props respectively, and using them will fallback to native browser behavior.
  - If you need to use these props, use the HTML `<form>` element instead.
- `<input type="file">`: Using this input type when the `action` is a string will match browser behavior by submitting the filename instead of the file object.

Examples

Search form that leads to a search result page

You can create a search form that navigates to a search results page by passing the path as an `action`:

```
// Archivo: /app/page.tsx switcher
import Form from 'next/form'

export default function Page() {
  return (
    <Form action="/search">
      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

```
// Archivo: /app/page.js switcher
import Form from 'next/form'

export default function Page() {
  return (
    <Form action="/search">
      <input name="query" />
      <button type="submit">Submit</button>
    </Form>
  )
}
```

When the user updates the query input field and submits the form, the form data will be encoded into the URL as search params, e.g. `/search?query=abc`.

> **Good to know**: If you pass an empty string `""` to `action`, the form will navigate to the same route with updated search params.

On the results page, you can access the query using the `searchParams` `page.js` prop and use it to fetch data from an external source.

```
// Archivo: /app/search/page.tsx switcher
import { getSearchResults } from '@/lib/search'

export default async function SearchPage({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const results = await getSearchResults((await searchParams).query)

  return <div>...</div>
}
```

```
// Archivo: /app/search/page.js switcher
import { getSearchResults } from '@/lib/search'

export default async function SearchPage({ searchParams }) {
  const results = await getSearchResults((await searchParams).query)

  return <div>...</div>
}
```

When the `<Form>` becomes visible in the user's viewport, shared UI (such as `layout.js` and `loading.js`) on the `/search` page will be prefetched. On submission, the form will immediately navigate to the new route and show loading UI while the results are being fetched. You can design the fallback UI using `loading.js`:

```
// Archivo: /app/search/loading.tsx switcher
export default function Loading() {
  return <div>Loading...</div>
}
```

```
// Archivo: /app/search/loading.js switcher
export default function Loading() {
  return <div>Loading...</div>
}
```

To cover cases when shared UI hasn't yet loaded, you can show instant feedback to the user using `useFormStatus`.

First, create a component that displays a loading state when the form is pending:

```
// Archivo: /app/ui/search-button.tsx switcher
'use client'
import { useFormStatus } from 'react-dom'

export default function SearchButton() {
  const status = useFormStatus()
  return (
    <button type="submit">{status.pending ? 'Searching...' : 'Search'}</button>
  )
}
```

```
// Archivo: /app/ui/search-button.js switcher
'use client'
import { useFormStatus } from 'react-dom'

export default function SearchButton() {
  const status = useFormStatus()
  return (
    <button type="submit">{status.pending ? 'Searching...' : 'Search'}</button>
  )
}
```

Then, update the search form page to use the `SearchButton` component:

```
// Archivo: /app/page.tsx switcher
import Form from 'next/form'
import { SearchButton } from '@/ui/search-button'

export default function Page() {
  return (
    <Form action="/search">
      <input name="query" />
      <SearchButton />
    </Form>
  )
}
```

```
// Archivo: /app/page.js switcher
import Form from 'next/form'
import { SearchButton } from '@/ui/search-button'

export default function Page() {
  return (
    <Form action="/search">
      <input name="query" />
      <SearchButton />
    </Form>
  )
}
```

Mutations with Server Actions

You can perform mutations by passing a function to the `action` prop.

```
// Archivo: /app/posts/create/page.tsx switcher
import Form from 'next/form'
import { createPost } from '@/posts/actions'

export default function Page() {
  return (
    <Form action={createPost}>
      <input name="title" />

      <button type="submit">Create Post</button>
    </Form>
  )
}
```

```
// Archivo: /app/posts/create/page.js switcher
import Form from 'next/form'
import { createPost } from '@/posts/actions'

export default function Page() {
  return (
    <Form action={createPost}>
      <input name="title" />

      <button type="submit">Create Post</button>
    </Form>
  )
}
```

After a mutation, it's common to redirect to the new resource. You can use the `redirect` function from `next/navigation` to navigate to the new post page.

> **Good to know**: Since the "destination" of the form submission is not known until the action is executed, `<Form>` cannot automatically prefetch shared UI.

```
// Archivo: /app/posts/actions.ts switcher
'use server'
import { redirect } from 'next/navigation'

export async function createPost(formData: FormData) {
  // Create a new post
  // ...

  // Redirect to the new post
  redirect(`/posts/${data.id}`)
}
```

```
// Archivo: /app/posts/actions.js switcher
'use server'
import { redirect } from 'next/navigation'

export async function createPost(formData) {
  // Create a new post
  // ...

  // Redirect to the new post
  redirect(`/posts/${data.id}`)
}
```

Then, in the new page, you can fetch data using the `params` prop:

```
// Archivo: /app/posts/[id]/page.tsx switcher
import { getPost } from '@/posts/data'

export default async function PostPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const data = await getPost(id)

  return (
    <div>
      <h1>{data.title}</h1>

    </div>
  )
}
```

```
// Archivo: /app/posts/[id]/page.js switcher
import { getPost } from '@/posts/data'

export default async function PostPage({ params }) {
  const { id } = await params
  const data = await getPost(id)

  return (
    <div>
      <h1>{data.title}</h1>

    </div>
  )
}
```

See Mutating data for more examples.

Image Component

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components/image
> 📂 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *Optimize Images in your Next.js Application using the built-in `next/image` Component.*

The Next.js Image component extends the HTML `<img>` element for automatic image optimization.

```
// Archivo: app/page.js
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://nextjs.org/profile.png)


  )
}
```

> 📘 **Nota para Pages Router:**
>
> **Good to know**: If you are using a version of Next.js prior to 13, you'll want to use the next/legacy/image documentation since the component was renamed.

Reference

Props

The following props are available:

| Prop | Example | Type | Status | | ---------- | ---------- | ---------- | ---------- | | [`src`](#src) | `src="/profile.png"` | String | Required | | [`alt`](#alt) | `alt="Picture of the author"` | String | Required | | [`width`](#width-and-height) | `width={500}` | Integer (px) | - | | [`height`](#width-and-height) | `height={500}` | Integer (px) | - | | [`fill`](#fill) | `fill={true}` | Boolean | - | | [`loader`](#loader) | `loader={imageLoader}` | Function | - | | [`sizes`](#sizes) | `sizes="(max-width: 768px) 100vw, 33vw"` | String | - | | [`quality`](#quality) | `quality={80}` | Integer (1-100) | - | | [`preload`](#preload) | `preload={true}` | Boolean | - | | [`placeholder`](#placeholder) | `placeholder="blur"` | String | - | | [`style`](#style) | `style={{objectFit: "contain"}}` | Object | - | | [`onLoadingComplete`](#onloadingcomplete) | `onLoadingComplete={img => done())}` | Function | Deprecated | | [`onLoad`](#onload) | `onLoad={event => done())}` | Function | - | | [`onError`](#onerror) | `onError(event => fail()}` | Function | - | | [`loading`](#loading) | `loading="lazy"` | String | - | | [`blurDataURL`](#blurdataurl) | `blurDataURL="data:image/jpeg..."` | String | - | | [`unoptimized`](#unoptimized) | `unoptimized={true}` | Boolean | - | | [`overrideSrc`](#overridesrc) | `overrideSrc="/seo.png"` | String | - | | [`decoding`](#decoding) | `decoding="async"` | String | - |

`src`

The source of the image. Can be one of the following:

An internal path string.

```
![Illustration](https://nextjs.org/profile.png)
```

An absolute external URL (must be configured with remotePatterns).

```
![Illustration](https://example.com/profile.png)
```

A static import.

```
import profile from './profile.png'

export default function Page() {
  return

![Illustration]()


}
```

> **Good to know**: For security reasons, the Image Optimization API using the default loader will *not* forward headers when fetching the `src` image. If the `src` image requires authentication, consider using the unoptimized property to disable Image Optimization.

`alt`

The `alt` property is used to describe the image for screen readers and search engines. It is also the fallback text if images have been disabled or an error occurs while loading the image.

It should contain text that could replace the image without changing the meaning of the page. It is not meant to supplement the image and should not repeat information that is already provided in the captions above or below the image.

If the image is purely decorative or not intended for the user, the `alt` property should be an empty string (`alt=""`).

> Learn more about image accessibility guidelines.

`width` and `height`

The `width` and `height` properties represent the intrinsic image size in pixels. This property is used to infer the correct **aspect ratio** used by browsers to reserve space for the image and avoid layout shift during loading. It does not determine the *rendered size* of the image, which is controlled by CSS.

```
![Illustration](https://nextjs.org/profile.png)
```

You **must** set both `width` and `height` properties unless:

- The image is statically imported.
- The image has the `fill` property

If the height and width are unknown, we recommend using the `fill` property.

`fill`

A boolean that causes the image to expand to the size of the parent element.

```
![Illustration](https://nextjs.org/profile.png)
```

Positioning:

- The parent element **must** assign `position: "relative"` , `"fixed"` , `"absolute"` .
- By default, the `<img>` element uses `position: "absolute"` .

Object Fit:

If no styles are applied to the image, the image will stretch to fit the container. You can use `objectFit` to control cropping and scaling.

- `"contain"` : The image will be scaled down to fit the container and preserve aspect ratio.
- `"cover"` : The image will fill the container and be cropped.

> Learn more about `position` and `object-fit`.

`loader`

A custom function used to generate the image URL. The function receives the following parameters, and returns a URL string for the image:

- `src`
- `width`
- `quality`

```
'use client'

import Image from 'next/image'

const imageLoader = ({ src, width, quality }) => {
  return `https://example.com/${src}?w=${width}&q=${quality || 75}`
}

export default function Page() {
  return (


![Picture of the author](me.png)


  )
}
```

> **Good to know**: Using props like `onLoad`, which accept a function, requires using Client Components to serialize the provided function.

> 📘 **Nota para Pages Router:**
>
> ```
> import Image from 'next/image'
>
> const imageLoader = ({ src, width, quality }) => {
>   return `https://example.com/${src}?w=${width}&q=${quality || 75}`
> }
>
> export default function Page() {
>   return (
> ```
>
> Illustration
>
> ```
>    loader={imageLoader}
> ```
>
> ```
>   src="me.png"
>   alt="Picture of the author"
>   width={500}
>   height={500}
> />
>
> )}
> ```
>
> ```
> ```

Alternatively, you can use the loaderFile configuration in `next.config.js` to configure every instance of `next/image` in your application, without passing a prop.

`sizes`

Define the sizes of the image at different breakpoints. Used by the browser to choose the most appropriate size from the generated `srcset`.

```
import Image from 'next/image'

export default function Page() {
  return (
    <div className="grid-element">


![Illustration](https://nextjs.org/example.png)


    </div>
  )
}
```

`sizes` should be used when:

- The image is using the `fill` prop
- CSS is used to make the image responsive

If `sizes` is missing, the browser assumes the image will be as wide as the viewport (`100vw`). This can cause unnecessarily large images to be downloaded.

In addition, `sizes` affects how `srcset` is generated:

- Without `sizes`: Next.js generates a limited `srcset` (e.g. 1x, 2x), suitable for fixed-size images.
- With `sizes`: Next.js generates a full `srcset` (e.g. 640w, 750w, etc.), optimized for responsive layouts.

> Learn more about `srcset` and `sizes` on web.dev and mdn.

`quality`

An integer between `1` and `100` that sets the quality of the optimized image. Higher values increase file size and visual fidelity. Lower values reduce file size but may affect sharpness.

```
// Default quality is 75


![Illustration]()
```

If you've configured qualities in `next.config.js`, a value outside that list is coerced to the closest allowed entry. For example, with `qualities: [50, 75, 100]`, a `quality` of `80` is served as `75`. Development logs a warning so you can add the value to the allowlist.

> **Good to know**: If the original image is already low quality, setting a high quality value will increase the file size without improving appearance.

`style`

Allows passing CSS styles to the underlying image element.

```
const imageStyle = {
  borderRadius: '50%',
  border: '1px solid #fff',
  width: '100px',
  height: 'auto',
}

export default function ProfileImage() {
  return

![Illustration](...)


}
```

> **Good to know**: If you're using the `style` prop to set a custom width, be sure to also set `height: 'auto'` to preserve the image's aspect ratio.

`preload`

A boolean that indicates if the image should be preloaded.

```
// Default preload is false


![Illustration]()
```

- `true`: Preloads the image by inserting a `<link>` in the `<head>`.
- `false`: Does not preload the image.

When to use it:

- The image is the Largest Contentful Paint (LCP) element.
- The image is above the fold, typically the hero image.
- You want to begin loading the image in the `<head>`, before its discovered later in the `<body>`.

When not to use it:

- When you have multiple images that could be considered the Largest Contentful Paint (LCP) element depending on the viewport.
- When the `loading` property is used.
- When the `fetchPriority` property is used.

In most cases, you should use `loading="eager"` or `fetchPriority="high"` instead of `preload`.

`priority`

Starting with Next.js 16, the `priority` property has been deprecated in favor of the `preload` property in order to make the behavior clear.

`loading`

Controls when the image should start loading.

```
// Defaults to lazy


![Illustration]()
```

- `lazy`: Defer loading the image until it reaches a calculated distance from the viewport.
- `eager`: Load the image immediately, regardless of its position in the page.

Use `eager` only when you want to ensure the image is loaded immediately.

> Learn more about the `loading` attribute.

`placeholder`

Specifies a placeholder to use while the image is loading, improving the perceived loading performance.

```
// defaults to empty


![Illustration]()
```

- `empty`: No placeholder while the image is loading.
- `blur`: Use a blurred version of the image as a placeholder. Must be used with the `blurDataURL` property.
- `data:image/...`: Uses the Data URL as the placeholder.

Examples:

- `blur` placeholder
- Shimmer effect with data URL `placeholder` prop
- Color effect with `blurDataURL` prop

> Learn more about the `placeholder` attribute.

`blurDataURL`

A Data URL to be used as a placeholder image before the image successfully loads. Can be automatically set or used with the `placeholder="blur"` property.

```


![Illustration]()
```

The image is automatically enlarged and blurred, so a very small image (10px or less) is recommended.

Automatic

If `src` is a static import of a `jpg`, `png`, `webp`, or `avif` file, `blurDataURL` is added automatically—unless the image is animated.

Manually set

If the image is dynamic or remote, you must provide `blurDataURL` yourself. To generate one, you can use:

- A online tool like png-pixel.com
- A library like Plaiceholder

A large blurDataURL may hurt performance. Keep it small and simple.

Examples:

- Default `blurDataURL` prop
- Color effect with `blurDataURL` prop

`onLoad`

A callback function that is invoked once the image is completely loaded and the placeholder has been removed.

```


![Illustration]()

 console.log(e.target.naturalWidth)} />
```

The callback function will be called with one argument, the event which has a `target` that references the underlying `<img>` element.

> **Good to know**: Using props like `onLoad`, which accept a function, requires using Client Components to serialize the provided function.

`onError`

A callback function that is invoked if the image fails to load.

```
![Illustration]()

 console.error(e.target.id)} />
```

> **Good to know**: Using props like `onError`, which accept a function, requires using Client Components to serialize the provided function.

`unoptimized`

A boolean that indicates if the image should be optimized. This is useful for images that do not benefit from optimization such as small images (less than 1KB), vector images (SVG), or animated images (GIF).

```
import Image from 'next/image'

const UnoptimizedImage = (props) => {
  // Default is false
  return

![Illustration]()


}
```

- `true`: The source image will be served as-is from the `src` instead of changing quality, size, or format.
- `false`: The source image will be optimized.

Since Next.js 12.3.0, this prop can be assigned to all images by updating `next.config.js` with the following configuration:

```
// Archivo: next.config.js
module.exports = {
  images: {
    unoptimized: true,
  },
}
```

`overrideSrc`

When providing the `src` prop to the `<Image>` component, both the `srcset` and `src` attributes are generated automatically for the resulting `<img>`.

```
// Archivo: input.js


![Illustration](https://nextjs.org/profile.jpg)
```

```
// Archivo: output.html
<img
  srcset="
    /_next/image?url=%2Fprofile.jpg&w=640&q=75 1x,
    /_next/image?url=%2Fprofile.jpg&w=828&q=75 2x
  "
  src="/_next/image?url=%2Fprofile.jpg&w=828&q=75"
/>
```

In some cases, it is not desirable to have the `src` attribute generated and you may wish to override it using the `overrideSrc` prop.

For example, when upgrading an existing website from `<img>` to `<Image>`, you may wish to maintain the same `src` attribute for SEO purposes such as image ranking or avoiding recrawl.

```
// Archivo: input.js


![Illustration](https://nextjs.org/profile.jpg)
```

```
// Archivo: output.html
<img
  srcset="
    /_next/image?url=%2Fprofile.jpg&w=640&q=75 1x,
    /_next/image?url=%2Fprofile.jpg&w=828&q=75 2x
  "
  src="/override.jpg"
/>
```

`decoding`

A hint to the browser indicating if it should wait for the image to be decoded before presenting other content updates or not.

```
// Default is async


![Illustration]()
```

- `async`: Asynchronously decode the image and allow other content to be rendered before it completes.
- `sync`: Synchronously decode the image for atomic presentation with other content.
- `auto`: No preference. The browser chooses the best approach.

> Learn more about the `decoding` attribute.

Other Props

Other properties on the `

Illustration

`component will be passed to the underlying` img` element with the exception of the following:

- `srcSet`: Use Device Sizes instead.

Deprecated props

`onLoadingComplete`

> **Warning**: Deprecated in Next.js 14, use `onLoad` instead.

A callback function that is invoked once the image is completely loaded and the placeholder has been removed.

The callback function will be called with one argument, a reference to the underlying `<img>` element.

```
'use client'

![Illustration]()

 console.log(img.naturalWidth)} />
```

> 📘 **Nota para Pages Router:**
>
> [redacted]

Illustration

console.log(img.naturalWidth)} />

> [redacted]

> **Good to know**: Using props like `onLoadingComplete`, which accept a function, requires using Client Components to serialize the provided function.

Configuration options

You can configure the Image Component in `next.config.js`. The following options are available:

`localPatterns`

Use `localPatterns` in your `next.config.js` file to allow images from specific local paths to be optimized and block all others.

```
// Archivo: next.config.js
module.exports = {
  images: {
    localPatterns: [
      {
        pathname: '/assets/images/**',
        search: '',
      },
    ],
  },
}
```

The example above will ensure the `src` property of `next/image` must start with `/assets/images/` and must not have a query string. Attempting to optimize any other path will respond with `400` Bad Request error.

> **Good to know**: Omitting the `search` property allows all search parameters which could allow malicious actors to optimize URLs you did not intend. Try using a specific value like `search: '?v=2'` to ensure an exact match.

`remotePatterns`

Use `remotePatterns` in your `next.config.js` file to allow images from specific external paths and block all others. This ensures that only external images from your account can be served.

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [new URL('https://example.com/account123/**')],
  },
}
```

You can also configure `remotePatterns` using the object:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'example.com',
        port: '',
        pathname: '/account123/**',
        search: '',
      },
    ],
  },
}
```

The example above will ensure the `src` property of `next/image` must start with `https://example.com/account123/` and must not have a query string. Any other protocol, hostname, port, or unmatched path will respond with `400` Bad Request.

Wildcard Patterns:

Wildcard patterns can be used for both `pathname` and `hostname` and have the following syntax:

- `*` match a single path segment or subdomain
- `**` match any number of path segments at the end or subdomains at the beginning. This syntax does not work in the middle of the pattern.

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: '**.example.com',
        port: '',
        search: '',
      },
    ],
  },
}
```

This allows subdomains like `image.example.com`. Query strings and custom ports are still blocked.

> **Good to know**: When omitting `protocol`, `port`, `pathname`, or `search` then the wildcard `**` is implied. This is not recommended because it may allow malicious actors to optimize urls you did not intend.

Query Strings:

You can also restrict query strings using the `search` property:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'assets.example.com',
        search: '?v=1727111025337',
      },
    ],
  },
}
```

The example above will ensure the `src` property of `next/image` must start with `https://assets.example.com` and must have the exact query string `?v=1727111025337`. Any other protocol or query string will respond with `400` Bad Request.

Note that any allowed `remotePatterns` that respond with a redirect will follow the redirect from the remote image server without validating `remotePatterns` again on the redirect location. You can reduce or disable redirects by configuring maximumRedirects.

`loaderFile`

`loaderFiles` allows you to use a custom image optimization service instead of Next.js.

```
// Archivo: next.config.js
module.exports = {
  images: {
    loader: 'custom',
    loaderFile: './my/image/loader.js',
  },
}
```

The path must be relative to the project root. The file must export a default function that returns a URL string:

```
// Archivo: my/image/loader.js
'use client'

export default function myImageLoader({ src, width, quality }) {
  return `https://example.com/${src}?w=${width}&q=${quality || 75}`
}
```

> 📘 **Nota para Pages Router:**
>
> ```
> export default function myImageLoader({ src, width, quality }) {
>   return `https://example.com/${src}?w=${width}&q=${quality || 75}`
> }
> ```

Example:

- Custom Image Loader Configuration

> Alternatively, you can use the `loader` prop to configure each instance of `next/image`.

`path`

If you want to change or prefix the default path for the Image Optimization API, you can do so with the `path` property. The default value for `path` is `/_next/image`.

```
// Archivo: next.config.js
module.exports = {
  images: {
    path: '/my-prefix/_next/image',
  },
}
```

`deviceSizes`

`deviceSizes` allows you to specify a list of device width breakpoints. These widths are used when the `next/image` component uses `sizes` prop to ensure the correct image is served for the user's device.

If no configuration is provided, the default below is used:

```
// Archivo: next.config.js
module.exports = {
  images: {
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048, 3840],
  },
}
```

`imageSizes`

`imageSizes` allows you to specify a list of image widths. These widths are concatenated with the array of device sizes to form the full array of sizes used to generate image srcset.

If no configuration is provided, the default below is used:

```
// Archivo: next.config.js
module.exports = {
  images: {
    imageSizes: [32, 48, 64, 96, 128, 256, 384],
  },
}
```

`imageSizes` is only used for images which provide a `sizes` prop, which indicates that the image is less than the full width of the screen. Therefore, the sizes in `imageSizes` should all be smaller than the smallest size in `deviceSizes`.

`qualities`

`qualities` allows you to specify a list of image quality values.

If not configuration is provided, the default below is used:

```
// Archivo: next.config.js
module.exports = {
  images: {
    qualities: [75],
  },
}
```

> **Good to know**: This field is required starting with Next.js 16 because unrestricted access could allow malicious actors to optimize more qualities than you intended.

You can add more image qualities to the allowlist, such as the following:

```
// Archivo: next.config.js
module.exports = {
  images: {
    qualities: [25, 50, 75, 100],
  },
}
```

In the example above, only four qualities are allowed: 25, 50, 75, and 100.

If the `quality` prop does not match a value in this array, the closest allowed value will be used.

If the REST API is visited directly with a quality that does not match a value in this array, the server will return a 400 Bad Request response.

`formats`

`formats` allows you to specify a list of image formats to be used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    // Default
    formats: ['image/webp'],
  },
}
```

Next.js automatically detects the browser's supported image formats via the request's `Accept` header in order to determine the best output format.

If the `Accept` header matches more than one of the configured formats, the first match in the array is used. Therefore, the array order matters. If there is no match (or the source image is animated), it will use the original image's format.

You can enable AVIF support, which will fallback to the original format of the src image if the browser does not support AVIF:

```
// Archivo: next.config.js
module.exports = {
  images: {
    formats: ['image/avif'],
  },
}
```

You can also enable both AVIF and WebP formats together. AVIF will be preferred for browsers that support it, with WebP as a fallback:

```
// Archivo: next.config.js
module.exports = {
  images: {
    formats: ['image/avif', 'image/webp'],
  },
}
```

> **Good to know**:
>
> - We still recommend using WebP for most use cases.
> - AVIF generally takes 50% longer to encode but it compresses 20% smaller compared to WebP. This means that the first time an image is requested, it will typically be slower, but subsequent requests that are cached will be faster.
> - When using multiple formats, Next.js will cache each format separately. This means increased storage requirements compared to using a single format, as both AVIF and WebP versions of images will be stored for different browser support.
> - If you self-host with a Proxy/CDN in front of Next.js, you must configure the Proxy to forward the `Accept` header.

`minimumCacheTTL`

`minimumCacheTTL` allows you to configure the Time to Live (TTL) in seconds for cached optimized images. In many cases, it's better to use a Static Image Import which will automatically hash the file contents and cache the image forever with a `Cache-Control` header of `immutable`.

If no configuration is provided, the default below is used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    minimumCacheTTL: 14400, // 4 hours
  },
}
```

You can increase the TTL to reduce the number of revalidations and potentially lower cost:

```
// Archivo: next.config.js
module.exports = {
  images: {
    minimumCacheTTL: 2678400, // 31 days
  },
}
```

The expiration (or rather Max Age) of the optimized image is defined by either the `minimumCacheTTL` or the upstream image `Cache-Control` header, whichever is larger.

If you need to change the caching behavior per image, you can configure `headers` to set the `Cache-Control` header on the upstream image (e.g. `/some-asset.jpg`, not `/_next/image` itself).

There is no mechanism to invalidate the cache at this time, so its best to keep `minimumCacheTTL` low. Otherwise you may need to manually change the `src` prop or delete the cached file `<distDir>/cache/images`.

`disableStaticImages`

`disableStaticImages` allows you to disable static image imports.

The default behavior allows you to import static files such as `import icon from './icon.png'` and then pass that to the `src` property. In some cases, you may wish to disable this feature if it conflicts with other plugins that expect the import to behave differently.

You can disable static image imports inside your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  images: {
    disableStaticImages: true,
  },
}
```

`maximumRedirects`

The default image optimization loader will follow HTTP redirects when fetching remote images up to 3 times.

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumRedirects: 3,
  },
}
```

For your convenience, these redirects do not need to satisfy remotePatterns.

You can configure the number of redirects to follow when fetching remote images. Setting the value to `0` will disable following redirects.

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumRedirects: 0,
  },
}
```

`maximumDiskCacheSize`

The default image optimization loader will write optimized images to disk so subsequent requests can be served faster from the disk cache.

You can configure the maximum disk cache size in bytes, for example 500 MB:

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumDiskCacheSize: 500_000_000,
  },
}
```

You can also disable the disk cache entirely by setting the value to `0`.

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumDiskCacheSize: 0,
  },
}
```

If no value is configured, the default behavior is to check the current available disk space once during startup and use 50%.

When the disk cache exceeds the configured size, the least recently used optimized images will be deleted until the cache is under the limit again.

Alternatively, you can implement your own cache handler using `cacheHandler` which will ignore the `maximumDiskCacheSize` configuration.

`maximumResponseBody`

The default image optimization loader will fetch source images up to 50 MB in size.

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumResponseBody: 50_000_000,
  },
}
```

If you know all your source images are small, you can protect memory constrained servers by reducing this to a smaller value such as 5 MB.

```
// Archivo: next.config.js
module.exports = {
  images: {
    maximumResponseBody: 5_000_000,
  },
}
```

`dangerouslyAllowLocalIP`

In rare cases when self-hosting Next.js on a private network, you may want to allow optimizing images from local IP addresses on the same network. This is not recommended for most users because it could allow malicious users to access content on your internal network.

By default, the value is false.

```
// Archivo: next.config.js
module.exports = {
  images: {
    dangerouslyAllowLocalIP: false,
  },
}
```

If you need to optimize remote images hosted elsewhere in your local network, you can set the value to true.

```
// Archivo: next.config.js
module.exports = {
  images: {
    dangerouslyAllowLocalIP: true,
  },
}
```

This might be necessary when hosting Next.js in a VPC with split-horizon DNS and you receive status 400 Bad Request. Only enable once you understand the SSRF risk.

`dangerouslyAllowSVG`

`dangerouslyAllowSVG` allows you to serve SVG images.

```
// Archivo: next.config.js
module.exports = {
  images: {
    dangerouslyAllowSVG: true,
  },
}
```

By default, Next.js does not optimize SVG images for a few reasons:

- SVG is a vector format meaning it can be resized losslessly.

- SVG has many of the same features as HTML/CSS, which can lead to vulnerabilities without proper Content Security Policy (CSP) headers.

We recommend using the `unoptimized` prop when the `src` prop is known to be SVG. This happens automatically when `src` ends with `".svg"`.

```
![Illustration](https://nextjs.org/my-image.svg)
```

In addition, it is strongly recommended to also set `contentDispositionType` to force the browser to download the image, as well as `contentSecurityPolicy` to prevent scripts embedded in the image from executing.

```
// Archivo: next.config.js
module.exports = {
  images: {
    dangerouslyAllowSVG: true,
    contentDispositionType: 'attachment',
    contentSecurityPolicy: "default-src 'self'; script-src 'none'; sandbox;",
  },
}
```

`contentDispositionType`

`contentDispositionType` allows you to configure the `Content-Disposition` header.

```
// Archivo: next.config.js
module.exports = {
  images: {
    contentDispositionType: 'inline',
  },
}
```

`contentSecurityPolicy`

`contentSecurityPolicy` allows you to configure the `Content-Security-Policy` header for images. This is particularly important when using `dangerouslyAllowSVG` to prevent scripts embedded in the image from executing.

```
// Archivo: next.config.js
module.exports = {
  images: {
    contentSecurityPolicy: "default-src 'self'; script-src 'none'; sandbox;",
  },
}
```

By default, the loader sets the `Content-Disposition` header to `attachment` for added protection since the API can serve arbitrary remote images.

The default value is `attachment` which forces the browser to download the image when visiting directly. This is particularly important when `dangerouslyAllowSVG` is true.

You can optionally configure `inline` to allow the browser to render the image when visiting directly, without downloading it.

Deprecated configuration options

`domains`

> **Warning**: Deprecated since Next.js 14 in favor of strict `remotePatterns` in order to protect your application from malicious users.

Similar to `remotePatterns`, the `domains` configuration can be used to provide a list of allowed hostnames for external images. However, the `domains` configuration does not support wildcard pattern matching and it cannot restrict protocol, port, or pathname.

Since most remote image servers are shared between multiple tenants, it's safer to use `remotePatterns` to ensure only the intended images are optimized.

Below is an example of the `domains` property in the `next.config.js` file:

```
// Archivo: next.config.js
module.exports = {
  images: {
    domains: ['assets.acme.com'],
  },
}
```

Functions

`getImageProps`

The `getImageProps` function can be used to get the props that would be passed to the underlying `<img>` element, and instead pass them to another component, style, canvas, etc.

```
import { getImageProps } from 'next/image'

const { props } = getImageProps({
  src: 'https://example.com/image.jpg',
  alt: 'A scenic mountain view',
  width: 1200,
  height: 800,
})

function ImageWithCaption() {
  return (
    <figure>
      <img {...props} />
      <figcaption>A scenic mountain view</figcaption>
    </figure>
  )
}
```

This also avoid calling React `useState()` so it can lead to better performance, but it cannot be used with the `placeholder` prop because the placeholder will never be removed.

Known browser bugs

This `next/image` component uses browser native lazy loading, which may fallback to eager loading for older browsers before Safari 15.4. When using the blur-up placeholder, older browsers before Safari 12 will fallback to empty placeholder. When using styles with `width` / `height` of `auto`, it is possible to cause Layout Shift on older browsers before Safari 15 that don't preserve the aspect ratio. For more details, see this MDN video.

- Safari 15 - 16.3 display a gray border while loading. Safari 16.4 fixed this issue. Possible solutions:
  - Use CSS `@supports (font: -apple-system-body) and (-webkit-appearance: none) { img[loading="lazy"] { clip-path: inset(0.6px) } }`
  - Use `loading="eager"` if the image is above the fold
- Firefox 67+ displays a white background while loading. Possible solutions:
  - Enable AVIF `formats`
  - Use `placeholder`

Examples

Styling images

Styling the Image component is similar to styling a normal `<img>` element, but there are a few guidelines to keep in mind:

Use `className` or `style`, not `styled-jsx`. In most cases, we recommend using the `className` prop. This can be an imported CSS Module, a global stylesheet, etc.

```
import styles from './styles.module.css'

export default function MyImage() {
  return

![My Image](https://nextjs.org/my-image.png)


}
```

You can also use the `style` prop to assign inline styles.

```
export default function MyImage() {
  return (


![My Image](https://nextjs.org/my-image.png)


  )
}
```

When using `fill`, the parent element must be positioned, with `position: relative`, `fixed`, or `absolute`. The image itself uses `position: absolute`, so it sizes against the nearest positioned ancestor.

```
<div style={{ position: 'relative' }}>


![My Image](https://nextjs.org/my-image.png)


</div>
```

You cannot use styled-jsx because it's scoped to the current component (unless you mark the style as `global`).

Responsive images with a static export

When you import a static image, Next.js automatically sets its width and height based on the file. You can make the image responsive by setting the style:

Responsive image filling the width and height of its parent container

```
import Image from 'next/image'
import mountains from '../public/mountains.jpg'

export default function Responsive() {
  return (
    <div style={{ display: 'flex', flexDirection: 'column' }}>


![Mountains]()


    </div>
  )
}
```

Responsive images with a remote URL

If the source image is a dynamic or a remote URL, you must provide the width and height props so Next.js can calculate the aspect ratio:

```
// Archivo: components/page.js
import Image from 'next/image'

export default function Page({ photoUrl }) {
  return (


![Picture of the author]()


  )
}
```

Try it out:

- Demo the image responsive to viewport

Responsive image with `fill`

If you don't know the aspect ratio of the image, you can add the `fill` prop with the `objectFit` prop set to `cover`. This will make the image fill the full width of its parent container.

Grid of images filling parent container width

```
import Image from 'next/image'
import mountains from '../public/mountains.jpg'

export default function Fill() {
  return (
    <div
      style={{
        display: 'grid',
        gridGap: '8px',
        gridTemplateColumns: 'repeat(auto-fit, minmax(400px, auto))',
      }}
    >
      <div style={{ position: 'relative', width: '400px' }}>


![Mountains]()


      </div>

    </div>
  )
}
```

Background Image

Use the `fill` prop to make the image cover the entire screen area:

Background image taking full width and height of page

```
import Image from 'next/image'
import mountains from '../public/mountains.jpg'

export default function Background() {
  return (


![Mountains]()


  )
}
```

For examples of the Image component used with the various styles, see the Image Component Demo.

Remote images

To use a remote image, the `src` property should be a URL string.

```
// Archivo: app/page.js
import Image from 'next/image'

export default function Page() {
  return (


![Picture of the author](https://s3.amazonaws.com/my-bucket/profile.png)


  )
}
```

Since Next.js does not have access to remote files during the build process, you'll need to provide the `width`, `height` and optional `blurDataURL` props manually.

The `width` and `height` attributes are used to infer the correct aspect ratio of image and avoid layout shift from the image loading in. The `width` and `height` do *not* determine the rendered size of the image file.

To safely allow optimizing images, define a list of supported URL patterns in `next.config.js`. Be as specific as possible to prevent malicious usage. For example, the following configuration will only allow images from a specific AWS S3 bucket:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 's3.amazonaws.com',
        port: '',
        pathname: '/my-bucket/**',
        search: '',
      },
    ],
  },
}
```

Theme detection

If you want to display a different image for light and dark mode, you can create a new component that wraps two `<Image>` components and reveals the correct one based on a CSS media query.

```
// Archivo: components/theme-image.module.css
.imgDark {
  display: none;
}

@media (prefers-color-scheme: dark) {
  .imgLight {
    display: none;
  }
  .imgDark {
    display: unset;
  }
}
```

```
// Archivo: components/theme-image.tsx switcher
import styles from './theme-image.module.css'
import Image, { ImageProps } from 'next/image'

type Props = Omit<ImageProps, 'src' | 'preload' | 'loading'> & {
  srcLight: string
  srcDark: string
}

const ThemeImage = (props: Props) => {
  const { srcLight, srcDark, ...rest } = props

  return (
    <>


![Illustration]()




![Illustration]()


    </>
  )
}
```

```
// Archivo: components/theme-image.js switcher
import styles from './theme-image.module.css'
import Image from 'next/image'

const ThemeImage = (props) => {
  const { srcLight, srcDark, ...rest } = props

  return (
    <>


![Illustration]()




![Illustration]()


    </>
  )
}
```

> **Good to know**: The default behavior of `loading="lazy"` ensures that only the correct image is loaded. You cannot use `preload` or `loading="eager"` because that would cause both images to load. Instead, you can use `fetchPriority="high"`.

Try it out:

- Demo light/dark mode theme detection

Art direction

If you want to display a different image for mobile and desktop, sometimes called Art Direction, you can provide different `src`, `width`, `height`, and `quality` props to `getImageProps()`.

```
// Archivo: app/page.js
import { getImageProps } from 'next/image'

export default function Home() {
  const common = { alt: 'Art Direction Example', sizes: '100vw' }
  const {
    props: { srcSet: desktop },
  } = getImageProps({
    ...common,
    width: 1440,
    height: 875,
    quality: 80,
    src: '/desktop.jpg',
  })
  const {
    props: { srcSet: mobile, ...rest },
  } = getImageProps({
    ...common,
    width: 750,
    height: 1334,
    quality: 70,
    src: '/mobile.jpg',
  })

  return (
    <picture>
      <source media="(min-width: 1000px)" srcSet={desktop} />
      <source media="(min-width: 500px)" srcSet={mobile} />
      <img {...rest} style={{ width: '100%', height: 'auto' }} />
    </picture>
  )
}
```

Background CSS

You can even convert the `srcSet` string to the `image-set()` CSS function to optimize a background image.

```
// Archivo: app/page.js
import { getImageProps } from 'next/image'

function getBackgroundImage(srcSet = '') {
  const imageSet = srcSet
    .split(', ')
    .map((str) => {
      const [url, dpi] = str.split(' ')
      return `url("${url}") ${dpi}`
    })
    .join(', ')
  return `image-set(${imageSet})`
}

export default function Home() {
  const {
    props: { srcSet },
  } = getImageProps({ alt: '', width: 128, height: 128, src: '/img.png' })
  const backgroundImage = getBackgroundImage(srcSet)
  const style = { height: '100vh', width: '100vw', backgroundImage }

  return (
    <main style={style}>
      <h1>Hello World</h1>
    </main>
  )
}
```

Version History

Version	Changes
`v16.1.7`	`maximumDiskCacheSize` configuration added.
`v16.1.2`	`maximumResponseBody` configuration added.
`v16.0.0`	`qualities` default configuration changed to `[75]`, `preload` prop added, `priority` prop deprecated, `dangerouslyAllowLocalIP` config added, `maximumRedirects` config added.
`v15.3.0`	`remotePatterns` added support for array of `URL` objects.
`v15.0.0`	`contentDispositionType` configuration default changed to `attachment`.
`v14.2.23`	`qualities` configuration added.
`v14.2.15`	`decoding` prop added and `localPatterns` configuration added.
`v14.2.14`	`remotePatterns.search` prop added.
`v14.2.0`	`overrideSrc` prop added.
`v14.1.0`	`getImageProps()` is stable.
`v14.0.0`	`onLoadingComplete` prop and `domains` config deprecated.
`v13.4.14`	`placeholder` prop support for `data:/image ...`
`v13.2.0`	`contentDispositionType` configuration added.
`v13.0.6`	`ref` prop added.
`v13.0.0`	The `next/image` import was renamed to `next/legacy/image`. The `next/future/image` import was renamed to `next/image`. A codemod is available to safely and automatically rename your imports. `<span>` wrapper removed. `layout`, `objectFit`, `objectPosition`, `lazyBoundary`, `lazyRoot` props removed. `alt` is required. `onLoadingComplete` receives reference to `img` element. Built-in loader config removed.
`v12.3.0`	`remotePatterns` and `unoptimized` configuration is stable.
`v12.2.0`	Experimental `remotePatterns` and experimental `unoptimized` configuration added. `layout="raw"` removed.
`v12.1.1`	`style` prop added. Experimental support for `layout="raw"` added.
`v12.1.0`	`dangerouslyAllowSVG` and `contentSecurityPolicy` configuration added.
`v12.0.9`	`lazyRoot` prop added.
`v12.0.0`	`formats` configuration added. AVIF support added. Wrapper `<div>` changed to `<span>`.
`v11.1.0`	`onLoadingComplete` and `lazyBoundary` props added.
`v11.0.0`	`src` prop support for static import. `placeholder` prop added. `blurDataURL` prop added.
`v10.0.5`	`loader` prop added.
`v10.0.1`	`layout` prop added.
`v10.0.0`	`next/image` introduced.

Components

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components
> 📂 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *API Reference for Next.js built-in components.*

Link Component

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components/link
> 📂 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *Enable fast client-side navigation with the built-in* `next/link` *component.*

`<Link>` is a React component that extends the HTML `<a>` element to provide prefetching and client-side navigation between routes. It is the primary way to navigate between routes in Next.js.

Basic usage:

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return [Dashboard](/dashboard)
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return [Dashboard](/dashboard)
}
```

> 📘 **Nota para Pages Router:**
>
> ```
> import Link from 'next/link'
>
> export default function Home() {
>   return [Dashboard](/dashboard)
> }
> ```
>
> ```
> import Link from 'next/link'
>
> export default function Home() {
>   return [Dashboard](/dashboard)
> }
> ```

Reference

The following props can be passed to the `<Link>` component:

📘 **Nota para Pages Router:**

Prop	Example	Type	Required
`href`	`href="/dashboard"`	String or Object	Yes
`as`	`as="/post/abc"`	String or Object	-
`replace`	`replace={false}`	Boolean	-
`scroll`	`scroll={false}`	Boolean	-
`prefetch`	`prefetch={false}`	Boolean	-
`shallow`	`shallow={false}`	Boolean	-
`locale`	`locale="fr"`	String or Boolean	-
`onNavigate`	`onNavigate={(e) ⇒ {}}`	Function	-

Prop	Example	Type	Required
`href`	`href="/dashboard"`	String or Object	Yes
`replace`	`replace={false}`	Boolean	-
`scroll`	`scroll={false}`	Boolean	-
`prefetch`	`prefetch={false}`	Boolean, `"auto"`, or null	-
`onNavigate`	`onNavigate={(e) ⇒ {}}`	Function	-
`transitionTypes`	`transitionTypes={['slide-in']}`	`string[]`	-

Good to know: `<a>` tag attributes such as `className` or `target="_blank"` can be added to `<Link>` as props and will be passed to the underlying `<a>` element.

`href` (required)

The path or URL to navigate to.

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

// Navigate to /about?name=test
export default function Page() {
  return (
    <Link
      href={{
        pathname: '/about',
        query: { name: 'test' },
      }}
    >
      About
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

// Navigate to /about?name=test
export default function Page() {
  return (
    <Link
      href={{
        pathname: '/about',
        query: { name: 'test' },
      }}
    >
      About
    </Link>
  )
}
```

📘 **Nota para Pages Router:**

```
import Link from 'next/link'

// Navigate to /about?name=test
export default function Home() {
  return (
    <Link
      href={{
        pathname: '/about',
        query: { name: 'test' },
      }}
    >
      About
    </Link>
  )
}
```

```
import Link from 'next/link'

// Navigate to /about?name=test
export default function Home() {
  return (
    <Link
      href={{
        pathname: '/about',
        query: { name: 'test' },
      }}
    >
      About
    </Link>
  )
}
```

`replace`

Defaults to `false`**.** When `true`, `next/link` will replace the current history state instead of adding a new URL into the browser's history stack.

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" replace>
      Dashboard
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" replace>
      Dashboard
    </Link>
  )
}
```

📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" replace>
      Dashboard
    </Link>
  )
}
```

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" replace>
      Dashboard
    </Link>
  )
}
```

`scroll`

Defaults to `true`. The default scrolling behavior of `<Link>` in Next.js **is to maintain scroll position**, similar to how browsers handle back and forwards navigation. When you navigate to a new Page, scroll position will stay the same as long as the Page is visible in the viewport. However, if the Page is not visible in the viewport, Next.js will scroll to the top of the first Page element.

When `scroll = {false}`, Next.js will not attempt to scroll to the first Page element.

> **Good to know**: Next.js checks if `scroll: false` before managing scroll behavior. If scrolling is enabled, it identifies the relevant DOM node for navigation and inspects each top-level element. All non-scrollable elements and those without rendered HTML are bypassed, this includes sticky or fixed positioned elements, and non-visible elements such as those calculated with `getBoundingClientRect`. Next.js then continues through siblings until it identifies a scrollable element that is visible in the viewport.

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" scroll={false}>
      Dashboard
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" scroll={false}>
      Dashboard
    </Link>
  )
}
```

> 📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" scroll={false}>
      Dashboard
    </Link>
  )
}
```

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" scroll={false}>
      Dashboard
    </Link>
  )
}
```

`prefetch`

Prefetching happens when a `<Link />` component enters the user's viewport (initially or through scroll). Next.js prefetches and loads the linked route (denoted by the `href`) and its data in the background to improve the performance of client-side navigations. If the prefetched data has expired by the time the user hovers over a `<Link />`, Next.js will attempt to prefetch it again. **Prefetching is only enabled in production**.

The following values can be passed to the `prefetch` prop:

- `"auto"` **or** `null` **(default)**: Prefetch behavior depends on whether the route is static or dynamic. For static routes, the full route will be prefetched (including all its data). For dynamic routes, the partial route down to the nearest segment with a `loading.js` boundary will be prefetched.
- `true`: The full route is prefetched for both static and dynamic routes. With Partial Prefetching enabled, the prefetch includes the App Shell and cached content that depends on the link's URL data. See Optimizing prefetching.
- `false`: Prefetching will never happen both on entering the viewport and on hover.

> **With Partial Prefetching enabled** (`partialPrefetching: true`): the default changes. `auto` prefetches the per-route App Shell (the route's static and cached content) instead of the full page. See Adopting Partial Prefetching for the full behavior change.

```tsx
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" prefetch={false}>
      Dashboard
    </Link>
  )
}
```

```jsx
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/dashboard" prefetch={false}>
      Dashboard
    </Link>
  )
}
```

📘 **Nota para Pages Router:** Prefetching happens when a `<Link />` component enters the user's viewport (initially or through scroll). Next.js prefetches and loads the linked route (denoted by the `href`) and data in the background to improve the performance of client-side navigation. **Prefetching is only enabled in production**.

The following values can be passed to the `prefetch` prop:

- `true` **(default)**: The full route and its data will be prefetched.
- `false`: Prefetching will not happen when entering the viewport, but will happen on hover. If you want to completely remove fetching on hover as well, consider using an `<a>` tag or incrementally adopting the App Router, which enables disabling prefetching on hover too.

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" prefetch={false}>
      Dashboard
    </Link>
  )
}
```


```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" prefetch={false}>
      Dashboard
    </Link>
  )
}
```


`shallow`

Update the path of the current page without rerunning `getStaticProps`, `getServerSideProps` or `getInitialProps`. Defaults to `false`.

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" shallow={false}>
      Dashboard
    </Link>
  )
}
```


```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/dashboard" shallow={false}>
      Dashboard
    </Link>
  )
}
```


`locale`

The active locale is automatically prepended. `locale` allows for providing a different locale. When `false` `href` has to include the locale as the default behavior is disabled.

```
import Link from 'next/link'

export default function Home() {
  return (
    <>

      [Dashboard (with locale)](/dashboard)


      <Link href="/dashboard" locale={false}>
        Dashboard (without locale)
      </Link>


      <Link href="/dashboard" locale="fr">
        Dashboard (French)
      </Link>
    </>
  )
}
```


```
import Link from 'next/link'

export default function Home() {
  return (
    <>

      [Dashboard (with locale)](/dashboard)


      <Link href="/dashboard" locale={false}>
        Dashboard (without locale)
      </Link>


      <Link href="/dashboard" locale="fr">
        Dashboard (French)
      </Link>
    </>
  )
}
```


`as`

Optional decorator for the path that will be shown in the browser URL bar. Before Next.js 9.5.3 this was used for dynamic routes, check our previous docs to see how it worked.

When this path differs from the one provided in `href` the previous `href` / `as` behavior is used as shown in the previous docs.

`onNavigate`

An event handler called during client-side navigation. The handler receives an event object that includes a `preventDefault()` method, allowing you to cancel the navigation if needed.

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link
      href="/dashboard"
      onNavigate={(e) => {
        // Only executes during SPA navigation
        console.log('Navigating...')

        // Optionally prevent navigation
        // e.preventDefault()
      }}
    >
      Dashboard
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link
      href="/dashboard"
      onNavigate={(e) => {
        // Only executes during SPA navigation
        console.log('Navigating...')

        // Optionally prevent navigation
        // e.preventDefault()
      }}
    >
      Dashboard
    </Link>
  )
}
```

> **Good to know**: While `onClick` and `onNavigate` may seem similar, they serve different purposes. `onClick` executes for all click events, while `onNavigate` only runs during client-side navigation. Some key differences:
>
> - When using modifier keys (`Ctrl` / `Cmd` + Click), `onClick` executes but `onNavigate` doesn't since Next.js prevents default navigation for new tabs.
> - External URLs won't trigger `onNavigate` since it's only for client-side and same-origin navigations.
> - Links with the `download` attribute will work with `onClick` but not `onNavigate` since the browser will treat the linked URL as a download.

`transitionTypes`

A list of transition types to apply to the navigation. These types are passed to `React.addTransitionType` inside the navigation transition, enabling `<ViewTransition>` components to apply different animations based on the type of navigation.

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/about" transitionTypes={['slide-in']}>
      About
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/about" transitionTypes={['slide-in']}>
      About
    </Link>
  )
}
```

Examples

The following examples demonstrate how to use the `<Link>` component in different scenarios.

Linking to dynamic route segments

When linking to dynamic segments, you can use template literals and interpolation to generate a list of links. For example, to generate a list of blog posts:

```
// Archivo: app/blog/post-list.tsx switcher
import Link from 'next/link'

interface Post {
  id: number
  title: string
  slug: string
}

export default function PostList({ posts }: { posts: Post[] }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/blog/post-list.js switcher
import Link from 'next/link'

export default function PostList({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}
```

Checking active links

You can use `usePathname()` to determine if a link is active. For example, to add a class to the active link, you can check if the current `pathname` matches the `href` of the link:

```tsx
// Archivo: app/ui/nav-links.tsx switcher
'use client'

import { usePathname } from 'next/navigation'
import Link from 'next/link'

export function Links() {
  const pathname = usePathname()

  return (
    <nav>
      <Link className={`link ${pathname === '/' ? 'active' : ''}`} href="/">
        Home
      </Link>

      <Link
        className={`link ${pathname === '/about' ? 'active' : ''}`}
        href="/about"
      >
        About
      </Link>
    </nav>
  )
}
```

```jsx
// Archivo: app/ui/nav-links.js switcher
'use client'

import { usePathname } from 'next/navigation'
import Link from 'next/link'

export function Links() {
  const pathname = usePathname()

  return (
    <nav>
      <Link className={`link ${pathname === '/' ? 'active' : ''}`} href="/">
        Home
      </Link>

      <Link
        className={`link ${pathname === '/about' ? 'active' : ''}`}
        href="/about"
      >
        About
      </Link>
    </nav>
  )
}
```

📘 **Nota para Pages Router:**

Linking to dynamic route segments

For dynamic route segments, it can be handy to use template literals to create the link's path.

For example, you can generate a list of links to the dynamic route `pages/blog/[slug].js`

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}
```

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${post.slug}`}>{post.title}</Link>
        </li>
      ))}
    </ul>
  )
}

export default Posts
```

Scrolling to an `id`

If you'd like to scroll to a specific `id` on navigation, you can append your URL with a `#` hash link or just pass a hash link to the `href` prop. This is possible since `<Link>` renders to an `<a>` element.

```
[Settings](/dashboard#settings)

// Output
<a href="/dashboard#settings">Settings</a>
```

Good to know:

- Next.js will scroll to the Page if it is not visible in the viewport upon navigation.

📘 **Nota para Pages Router:**

Passing a URL Object

`Link` can also receive a URL object and it will automatically format it to create the URL string:

```
import Link from 'next/link'

function Home() {
  return (
    <ul>
      <li>
        <Link
          href={{
            pathname: '/about',
            query: { name: 'test' },
          }}
        >
          About us
        </Link>
      </li>
      <li>
        <Link
          href={{
            pathname: '/blog/[slug]',
            query: { slug: 'my-post' },
          }}
        >
          Blog Post
        </Link>
      </li>
    </ul>
  )
}

export default Home
```

```
import Link from 'next/link'

function Home() {
  return (
    <ul>
      <li>
        <Link
          href={{
            pathname: '/about',
            query: { name: 'test' },
          }}
        >
          About us
        </Link>
      </li>
      <li>
        <Link
          href={{
            pathname: '/blog/[slug]',
            query: { slug: 'my-post' },
          }}
        >
          Blog Post
        </Link>
      </li>
    </ul>
  )
}

export default Home
```

The above example has a link to:

- A predefined route: `/about?name=test`
- A dynamic route: `/blog/my-post`

You can use every property as defined in the Node.js URL module documentation.

Replace the URL instead of push

The default behavior of the `Link` component is to `push` a new URL into the `history` stack. You can use the `replace` prop to prevent adding a new entry, as in the following example:

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/about" replace>
      About us
    </Link>
  )
}
```


```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/about" replace>
      About us
    </Link>
  )
}
```


📘 **Nota para Pages Router:**

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/about" replace>
      About us
    </Link>
  )
}
```


```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/about" replace>
      About us
    </Link>
  )
}
```


Disable scrolling to the top of the page

The default scrolling behavior of `<Link>` in Next.js **is to maintain scroll position**, similar to how browsers handle back and forwards navigation. When you navigate to a new Page, scroll position will stay the same as long as the Page is visible in the viewport.

However, if the Page is not visible in the viewport, Next.js will scroll to the top of the first Page element. If you'd like to disable this behavior, you can pass `scroll={false}` to the `<Link>` component, or `scroll: false` to `router.push()` or `router.replace()`.

```
// Archivo: app/page.js switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/#hashid" scroll={false}>
      Disables scrolling to the top
    </Link>
  )
}
```

```
// Archivo: app/page.tsx switcher
import Link from 'next/link'

export default function Page() {
  return (
    <Link href="/#hashid" scroll={false}>
      Disables scrolling to the top
    </Link>
  )
}
```

Using `router.push()` or `router.replace()`:

```
// useRouter
import { useRouter } from 'next/navigation'

const router = useRouter()

router.push('/dashboard', { scroll: false })
```

📘 **Nota para Pages Router:** The default behavior of `Link` is to scroll to the top of the page. When there is a hash defined it will scroll to the specific id, like a normal `<a>` tag. To prevent scrolling to the top / hash `scroll={false}` can be added to `Link`:

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/#hashid" scroll={false}>
      Disables scrolling to the top
    </Link>
  )
}
```

```
import Link from 'next/link'

export default function Home() {
  return (
    <Link href="/#hashid" scroll={false}>
      Disables scrolling to the top
    </Link>
  )
}
```

Scroll offset with sticky headers

Because Next.js skips sticky and fixed positioned elements when finding the scroll target, content may end up behind a sticky header after navigation. For example, if your layout has a sticky header:

```
// Archivo: app/layout.tsx switcher
import './globals.css'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>
        <header className="sticky top-0 h-16 bg-white">

        </header>
        {children}
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import './globals.css'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <header className="sticky top-0 h-16 bg-white">

        </header>
        {children}
      </body>
    </html>
  )
}
```

You can account for its height using `scroll-padding-top` on the scroll container:

```
// Archivo: app/globals.css
html {
  scroll-padding-top: 64px; /* Match the height of your sticky header */
}
```

This is a browser CSS property that offsets scroll-based positioning. It applies whenever Next.js uses the native `scrollIntoView()` API, including hash fragment (`#id`) navigation. Alternatively, you can use `scroll-margin-top` on individual target elements instead of setting a global offset.

Prefetching links in Proxy

It's common to use Proxy for authentication or other purposes that involve rewriting the user to a different page. In order for the `<Link />` component to properly prefetch links with rewrites via Proxy, you need to tell Next.js both the URL to display and the URL to prefetch. This is required to avoid un-necessary fetches to proxy to know the correct route to prefetch.

For example, if you want to serve a `/dashboard` route that has authenticated and visitor views, you can add the following in your Proxy to redirect the user to the correct page:

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'

export function proxy(request: Request) {
  const nextUrl = request.nextUrl
  if (nextUrl.pathname === '/dashboard') {
    if (request.cookies.authToken) {
      return NextResponse.rewrite(new URL('/auth/dashboard', request.url))
    } else {
      return NextResponse.rewrite(new URL('/public/dashboard', request.url))
    }
  }
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  const nextUrl = request.nextUrl
  if (nextUrl.pathname === '/dashboard') {
    if (request.cookies.authToken) {
      return NextResponse.rewrite(new URL('/auth/dashboard', request.url))
    } else {
      return NextResponse.rewrite(new URL('/public/dashboard', request.url))
    }
  }
}
```

In this case, you would want to use the following code in your `<Link />` component:

```
// Archivo: app/page.tsx switcher
'use client'

import Link from 'next/link'
import useIsAuthed from './hooks/useIsAuthed' // Your auth hook

export default function Page() {
  const isAuthed = useIsAuthed()
  const path = isAuthed ? '/auth/dashboard' : '/public/dashboard'
  return (
    <Link as="/dashboard" href={path}>
      Dashboard
    </Link>
  )
}
```

```
// Archivo: app/page.js switcher
'use client'

import Link from 'next/link'
import useIsAuthed from './hooks/useIsAuthed' // Your auth hook

export default function Page() {
  const isAuthed = useIsAuthed()
  const path = isAuthed ? '/auth/dashboard' : '/public/dashboard'
  return (
    <Link as="/dashboard" href={path}>
      Dashboard
    </Link>
  )
}
```

📘 **Nota para Pages Router:**

```
'use client'

import Link from 'next/link'
import useIsAuthed from './hooks/useIsAuthed' // Your auth hook

export default function Home() {
  const isAuthed = useIsAuthed()
  const path = isAuthed ? '/auth/dashboard' : '/public/dashboard'
  return (
    <Link as="/dashboard" href={path}>
      Dashboard
    </Link>
  )
}
```


```
'use client'

import Link from 'next/link'
import useIsAuthed from './hooks/useIsAuthed' // Your auth hook

export default function Home() {
  const isAuthed = useIsAuthed()
  const path = isAuthed ? '/auth/dashboard' : '/public/dashboard'
  return (
    <Link as="/dashboard" href={path}>
      Dashboard
    </Link>
  )
}
```


> **Good to know**: If you're using Dynamic Routes, you'll need to adapt your `as` and `href` props. For example, if you have a Dynamic Route like `/dashboard/authed/[user]` that you want to present differently via proxy, you would write: `<Link href={{ pathname: '/dashboard/authed/[user]', query: { user: username } }} as="/dashboard/[user]">Profile</Link>`.

Blocking navigation

You can use the `onNavigate` prop to block navigation when certain conditions are met, such as when a form has unsaved changes. When you need to block navigation across multiple components in your app (like preventing navigation from any link while a form is being edited), React Context provides a clean way to share this blocking state. First, create a context to track the navigation blocking state:

```tsx
// Archivo: app/contexts/navigation-blocker.tsx switcher
'use client'

import { createContext, useState, useContext } from 'react'

interface NavigationBlockerContextType {
  isBlocked: boolean
  setIsBlocked: (isBlocked: boolean) => void
}

export const NavigationBlockerContext =
  createContext<NavigationBlockerContextType>({
    isBlocked: false,
    setIsBlocked: () => {},
  })

export function NavigationBlockerProvider({
  children,
}: {
  children: React.ReactNode
}) {
  const [isBlocked, setIsBlocked] = useState(false)

  return (
    <NavigationBlockerContext.Provider value={{ isBlocked, setIsBlocked }}>
      {children}
    </NavigationBlockerContext.Provider>
  )
}

export function useNavigationBlocker() {
  return useContext(NavigationBlockerContext)
}
```

```jsx
// Archivo: app/contexts/navigation-blocker.js switcher
'use client'

import { createContext, useState, useContext } from 'react'

export const NavigationBlockerContext = createContext({
  isBlocked: false,
  setIsBlocked: () => {},
})

export function NavigationBlockerProvider({ children }) {
  const [isBlocked, setIsBlocked] = useState(false)

  return (
    <NavigationBlockerContext.Provider value={{ isBlocked, setIsBlocked }}>
      {children}
    </NavigationBlockerContext.Provider>
  )
}

export function useNavigationBlocker() {
  return useContext(NavigationBlockerContext)
}
```

Create a form component that uses the context:

```tsx
// Archivo: app/components/form.tsx switcher
'use client'

import { useNavigationBlocker } from '../contexts/navigation-blocker'

export default function Form() {
  const { setIsBlocked } = useNavigationBlocker()

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault()
        setIsBlocked(false)
      }}
      onChange={() => setIsBlocked(true)}
    >
      <input type="text" name="name" />
      <button type="submit">Save</button>
    </form>
  )
}
```

```jsx
// Archivo: app/components/form.js switcher
'use client'

import { useNavigationBlocker } from '../contexts/navigation-blocker'

export default function Form() {
  const { setIsBlocked } = useNavigationBlocker()

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault()
        setIsBlocked(false)
      }}
      onChange={() => setIsBlocked(true)}
    >
      <input type="text" name="name" />
      <button type="submit">Save</button>
    </form>
  )
}
```

Create a custom Link component that blocks navigation:

```tsx
// Archivo: app/components/custom-link.tsx switcher
'use client'

import Link from 'next/link'
import { useNavigationBlocker } from '../contexts/navigation-blocker'

interface CustomLinkProps extends React.ComponentProps<typeof Link> {
  children: React.ReactNode
}

export function CustomLink({ children, ...props }: CustomLinkProps) {
  const { isBlocked } = useNavigationBlocker()

  return (
    <Link
      onNavigate={(e) => {
        if (
          isBlocked &&
          !window.confirm('You have unsaved changes. Leave anyway?')
        ) {
          e.preventDefault()
        }
      }}
      {...props}
    >
      {children}
    </Link>
  )
}
```

```jsx
// Archivo: app/components/custom-link.js switcher
'use client'

import Link from 'next/link'
import { useNavigationBlocker } from '../contexts/navigation-blocker'

export function CustomLink({ children, ...props }) {
  const { isBlocked } = useNavigationBlocker()

  return (
    <Link
      onNavigate={(e) => {
        if (
          isBlocked &&
          !window.confirm('You have unsaved changes. Leave anyway?')
        ) {
          e.preventDefault()
        }
      }}
      {...props}
    >
      {children}
    </Link>
  )
}
```

Create a navigation component:

```
// Archivo: app/components/nav.tsx switcher
'use client'

import { CustomLink as Link } from './custom-link'

export default function Nav() {
  return (
    <nav>
      [Home](/)
      [About](/about)
    </nav>
  )
}
```

```
// Archivo: app/components/nav.js switcher
'use client'

import { CustomLink as Link } from './custom-link'

export default function Nav() {
  return (
    <nav>
      [Home](/)
      [About](/about)
    </nav>
  )
}
```

Finally, wrap your app with the `NavigationBlockerProvider` in the root layout and use the components in your page:

```
// Archivo: app/layout.tsx switcher
import { NavigationBlockerProvider } from './contexts/navigation-blocker'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>
        <NavigationBlockerProvider>{children}</NavigationBlockerProvider>
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { NavigationBlockerProvider } from './contexts/navigation-blocker'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <NavigationBlockerProvider>{children}</NavigationBlockerProvider>
      </body>
    </html>
  )
}
```

Then, use the `Nav` and `Form` components in your page:

```tsx
// Archivo: app/page.tsx switcher
import Nav from './components/nav'
import Form from './components/form'

export default function Page() {
  return (
    <div>
      <Nav />
      <main>
        <h1>Welcome to the Dashboard</h1>
        <Form />
      </main>
    </div>
  )
}
```

```jsx
// Archivo: app/page.js switcher
import Nav from './components/nav'
import Form from './components/form'

export default function Page() {
  return (
    <div>
      <Nav />
      <main>
        <h1>Welcome to the Dashboard</h1>
        <Form />
      </main>
    </div>
  )
}
```

When a user tries to navigate away using `CustomLink` while the form has unsaved changes, they'll be prompted to confirm before leaving.

Version history

Version	Changes
`v16.2.0`	Add `transitionTypes` prop.
`v15.4.0`	Add `auto` as an alias to the default `prefetch` behavior.
`v15.3.0`	Add `onNavigate` API
`v13.0.0`	No longer requires a child `<a>` tag. A codemod is provided to automatically update your codebase.
`v10.0.0`	`href` props pointing to a dynamic route are automatically resolved and no longer require an `as` prop.
`v8.0.0`	Improved prefetching performance.
`v1.0.0`	`next/link` introduced.

Script Component

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/components/script
> 📁 **Sección:** App > Api Reference > Components
> 📝 **Descripción:** *Optimize third-party scripts in your Next.js application using the built-in `next/script` Component.*

This API reference will help you understand how to use props available for the Script Component. For features and usage, please see the Optimizing Scripts page.

```
// Archivo: app/dashboard/page.tsx switcher
import Script from 'next/script'

export default function Dashboard() {
  return (
    <>
      <Script src="https://example.com/script.js" />
    </>
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
import Script from 'next/script'

export default function Dashboard() {
  return (
    <>
      <Script src="https://example.com/script.js" />
    </>
  )
}
```

Props

Here's a summary of the props available for the Script Component:

Prop	Example	Type	Required
`src`	`src="http://example.com/script"`	String	Required unless inline script is used
`strategy`	`strategy="lazyOnload"`	String	-
`onLoad`	`onLoad={onLoadFunc}`	Function	-
`onReady`	`onReady={onReadyFunc}`	Function	-
`onError`	`onError={onErrorFunc}`	Function	-

Required Props

The `<Script />` component requires the following properties.

`src`

A path string specifying the URL of an external script. This can be either an absolute external URL or an internal path. The `src` property is required unless an inline script is used.

Optional Props

The `<Script />` component accepts a number of additional properties beyond those which are required.

`strategy`

The loading strategy of the script. There are four different strategies that can be used:

- `beforeInteractive` : Load before any Next.js code and before any page hydration occurs.
- `afterInteractive` : (**default**) Load early but after some hydration on the page occurs.
- `lazyOnload` : Load during browser idle time.

- `worker` : (experimental) Load in a web worker.

`beforeInteractive`

Scripts that load with the `beforeInteractive` strategy are injected into the initial HTML from the server, downloaded before any Next.js module, and executed in the order they are placed.

Scripts denoted with this strategy are preloaded and fetched before any first-party code, but their execution **does not block page hydration from occurring**.

Scripts with the `beforeInteractive` strategy must be placed inside a root layout, such as `app/layout.tsx` or `app/[locale]/layout.tsx`, and are designed to load scripts that are needed by the entire site (i.e. the script will load when any page in the application has been loaded server-side).

> 📘 **Nota para Pages Router:** `beforeInteractive` scripts must be placed inside the `Document` Component (`pages/_document.js`) and are designed to load scripts that are needed by the entire site (i.e. the script will load when any page in the application has been loaded server-side).

This strategy should only be used for critical scripts that need to be fetched as soon as possible.

```
// Archivo: app/layout.tsx switcher
import Script from 'next/script'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>
        {children}
        <Script
          src="https://example.com/script.js"
          strategy="beforeInteractive"
        />
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import Script from 'next/script'

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}
        <Script
          src="https://example.com/script.js"
          strategy="beforeInteractive"
        />
      </body>
    </html>
  )
}
```

📘 **Nota para Pages Router:**

```
import { Html, Head, Main, NextScript } from 'next/document'
import Script from 'next/script'

export default function Document() {
  return (
    <Html>
      <Head />
      <body>
        <Main />
        <NextScript />
        <Script
          src="https://example.com/script.js"
          strategy="beforeInteractive"
        />
      </body>
    </Html>
  )
}
```

Good to know: Scripts with `beforeInteractive` will always be injected inside the `head` of the HTML document regardless of where it's placed in the component.

Good to know: These scripts run once per document load. A client-side navigation does not run them again, including one that only changes a root param, such as `/en` to `/fi`, since the root layout stays the same.

Some examples of scripts that should be fetched as soon as possible with `beforeInteractive` include:

- Bot detectors
- Cookie consent managers

`afterInteractive`

Scripts that use the `afterInteractive` strategy are injected into the HTML client-side and will load after some (or all) hydration occurs on the page. **This is the default strategy** of the Script component and should be used for any script that needs to load as soon as possible but not before any first-party Next.js code.

`afterInteractive` scripts can be placed inside of any page or layout and will only load and execute when that page (or group of pages) is opened in the browser.

```
// Archivo: app/page.js
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="afterInteractive" />
    </>
  )
}
```

Some examples of scripts that are good candidates for `afterInteractive` include:

- Tag managers
- Analytics

`lazyOnload`

Scripts that use the `lazyOnload` strategy are injected into the HTML client-side during browser idle time and will load after all resources on the page have been fetched. This strategy should be used for any background or low priority scripts that do not need to load early.

`lazyOnload` scripts can be placed inside of any page or layout and will only load and execute when that page (or group of pages) is opened in the browser.

```
// Archivo: app/page.js
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="lazyOnload" />
    </>
  )
}
```

Examples of scripts that do not need to load immediately and can be fetched with `lazyOnload` include:

- Chat support plugins
- Social media widgets

`worker`

> **Warning:** The `worker` strategy is not yet stable and does not yet work with the App Router. Use with caution.

Scripts that use the `worker` strategy are off-loaded to a web worker in order to free up the main thread and ensure that only critical, first-party resources are processed on it. While this strategy can be used for any script, it is an advanced use case that is not guaranteed to support all third-party scripts.

To use `worker` as a strategy, the `nextScriptWorkers` flag must be enabled in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    nextScriptWorkers: true,
  },
}
```

`worker` scripts can **only currently be used in the `pages/` directory**:

```
// Archivo: pages/home.tsx switcher
import Script from 'next/script'

export default function Home() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="worker" />
    </>
  )
}
```

```
// Archivo: pages/home.js switcher
import Script from 'next/script'

export default function Home() {
  return (
    <>
      <Script src="https://example.com/script.js" strategy="worker" />
    </>
  )
}
```

`onLoad`

> **Warning:** `onLoad` does not yet work with Server Components and can only be used in Client Components. Further, `onLoad` can't be used with `beforeInteractive` – consider using `onReady` instead.

Some third-party scripts require users to run JavaScript code once after the script has finished loading in order to instantiate content or call a function. If you are loading a script with either `afterInteractive` or `lazyOnload` as a loading strategy, you can execute code after it has loaded using the `onLoad` property.

Here's an example of executing a lodash method only after the library has been loaded.

```
// Archivo: app/page.tsx switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.17.20/lodash.min.js"
        onLoad={() => {
          console.log(_.sample([1, 2, 3, 4]))
        }}
      />
    </>
  )
}
```

```
// Archivo: app/page.js switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.17.20/lodash.min.js"
        onLoad={() => {
          console.log(_.sample([1, 2, 3, 4]))
        }}
      />
    </>
  )
}
```

`onReady`

> **Warning:** `onReady` does not yet work with Server Components and can only be used in Client Components.

Some third-party scripts require users to run JavaScript code after the script has finished loading and every time the component is mounted (after a route navigation for example). You can execute code after the script's load event when it first loads and then after every subsequent component re-mount using the `onReady` property.

Here's an example of how to re-instantiate a Google Maps JS embed every time the component is mounted:

```tsx
// Archivo: app/page.tsx switcher
'use client'

import { useRef } from 'react'
import Script from 'next/script'

export default function Page() {
  const mapRef = useRef()

  return (
    <>
      <div ref={mapRef}></div>
      <Script
        id="google-maps"
        src="https://maps.googleapis.com/maps/api/js"
        onReady={() => {
          new google.maps.Map(mapRef.current, {
            center: { lat: -34.397, lng: 150.644 },
            zoom: 8,
          })
        }}
      />
    </>
  )
}
```

```tsx
// Archivo: app/page.js switcher
'use client'

import { useRef } from 'react'
import Script from 'next/script'

export default function Page() {
  const mapRef = useRef()

  return (
    <>
      <div ref={mapRef}></div>
      <Script
        id="google-maps"
        src="https://maps.googleapis.com/maps/api/js"
        onReady={() => {
          new google.maps.Map(mapRef.current, {
            center: { lat: -34.397, lng: 150.644 },
            zoom: 8,
          })
        }}
      />
    </>
  )
}
```

📘 **Nota para Pages Router:**

```
import { useRef } from 'react'
import Script from 'next/script'

export default function Page() {
  const mapRef = useRef()

  return (
    <>
      <div ref={mapRef}></div>
      <Script
        id="google-maps"
        src="https://maps.googleapis.com/maps/api/js"
        onReady={() => {
          new google.maps.Map(mapRef.current, {
            center: { lat: -34.397, lng: 150.644 },
            zoom: 8,
          })
        }}
      />
    </>
  )
}
```

`onError`

> **Warning:** `onError` does not yet work with Server Components and can only be used in Client Components. `onError` cannot be used with the `beforeInteractive` loading strategy.

Sometimes it is helpful to catch when a script fails to load. These errors can be handled with the `onError` property:

```
// Archivo: app/page.tsx switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onError={(e: Error) => {
          console.error('Script failed to load', e)
        }}
      />
    </>
  )
}
```

```
// Archivo: app/page.js switcher
'use client'

import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onError={(e) => {
          console.error('Script failed to load', e)
        }}
      />
    </>
  )
}
```

📘 **Nota para Pages Router:**

```
import Script from 'next/script'

export default function Page() {
  return (
    <>
      <Script
        src="https://example.com/script.js"
        onError={(e: Error) => {
          console.error('Script failed to load', e)
        }}
      />
    </>
  )
}
```

Version History

Version	Changes
`v13.0.0`	`beforeInteractive` and `afterInteractive` is modified to support `app`.
`v12.2.4`	`onReady` prop added.
`v12.2.2`	Allow `next/script` with `beforeInteractive` to be placed in `_document`.
`v11.0.0`	`next/script` introduced.

favicon, icon, and apple-icon

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/app-icons
📁 **Sección:** App > Api Reference > File Conventions > Metadata
📝 **Descripción:** *API Reference for the Favicon, Icon and Apple Icon file conventions.*

The `favicon`, `icon`, or `apple-icon` file conventions allow you to set icons for your application.

They are useful for adding app icons that appear in places like web browser tabs, phone home screens, and search engine results.

There are two ways to set app icons:

- Using image files (.ico, .jpg, .png)
- Using code to generate an icon (.js, .ts, .tsx)

Image files (.ico, .jpg, .png)

Use an image file to set an app icon by placing a `favicon`, `icon`, or `apple-icon` image file within your `/app` directory. The `favicon` image can only be located in the top level of `app/`.

Next.js will evaluate the file and automatically add the appropriate tags to your app's `<head>` element.

File convention	Supported file types	Valid locations
`favicon`	`.ico`	`app/`
`icon`	`.ico`, `.jpg`, `.jpeg`, `.png`, `.svg`	`app/**/*`
`apple-icon`	`.jpg`, `.jpeg`, `.png`	`app/**/*`

`favicon`

Add a `favicon.ico` image file to the root `/app` route segment.

```
// Archivo: <head> output
<link rel="icon" href="/favicon.ico" sizes="any" />
```

`icon`

Add an `icon.(ico|jpg|jpeg|png|svg)` image file.

```
// Archivo: <head> output
<link
  rel="icon"
  href="/icon?<generated>"
  type="image/<generated>"
  sizes="<generated>"
/>
```

`apple-icon`

Add an `apple-icon.(jpg|jpeg|png)` image file.

```
// Archivo: <head> output
<link
  rel="apple-touch-icon"
  href="/apple-icon?<generated>"
  type="image/<generated>"
  sizes="<generated>"
/>
```

> **Good to know**:
>
> - You can set multiple icons by adding a number suffix to the file name. For example, `icon1.png`, `icon2.png`, etc. Numbered files will sort lexically.
> - Favicons can only be set in the root `/app` segment. If you need more granularity, you can use `icon`.
> - The appropriate `<link>` tags and attributes such as `rel`, `href`, `type`, and `sizes` are determined by the icon type and metadata of the evaluated file.
> - For example, a 32 by 32px `.png` file will have `type="image/png"` and `sizes="32x32"` attributes.
> - `sizes="any"` is added to icons when the extension is `.svg` or the image size of the file is not determined. More details in this favicon handbook.

Generate icons using code (.js, .ts, .tsx)

In addition to using literal image files, you can programmatically **generate** icons using code.

Generate an app icon by creating an `icon` or `apple-icon` route that default exports a function.

File convention	Supported file types
`icon`	`.js` , `.ts` , `.tsx`
`apple-icon`	`.js` , `.ts` , `.tsx`

The easiest way to generate an icon is to use the `ImageResponse` API from `next/og` .

```
// Archivo: app/icon.tsx switcher
import { ImageResponse } from 'next/og'

// Image metadata
export const size = {
  width: 32,
  height: 32,
}
export const contentType = 'image/png'

// Image generation
export default function Icon() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 24,
          background: 'black',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          color: 'white',
        }}
      >
        A
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported icons size metadata
      // config to also set the ImageResponse's width and height.
      ...size,
    }
  )
}
```

```
// Archivo: app/icon.js switcher
import { ImageResponse } from 'next/og'

// Image metadata
export const size = {
  width: 32,
  height: 32,
}
export const contentType = 'image/png'

// Image generation
export default function Icon() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 24,
          background: 'black',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          color: 'white',
        }}
      >
        A
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported icons size metadata
      // config to also set the ImageResponse's width and height.
      ...size,
    }
  )
}
```

```
// Archivo: <head> output
<link rel="icon" href="/icon?<generated>" type="image/png" sizes="32x32" />
```

Good to know:

- By default, generated icons are **statically optimized** (generated at build time and cached) unless they use Request-time APIs or uncached data.
- You can generate multiple icons in the same file using `generateImageMetadata`.
- You cannot generate a `favicon` icon. Use `icon` or a favicon.ico file instead.
- App icons are special Route Handlers that are cached by default unless they use a Request-time API or dynamic config option.

Props

The default export function receives the following props:

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters object from the root segment down to the segment `icon` or `apple-icon` is colocated in.

Good to know: If you use `generateImageMetadata`, the function will also receive an `id` prop that is a promise resolving to the `id` value from one of the items returned by `generateImageMetadata`.

```
// Archivo: app/shop/[slug]/icon.tsx switcher
export default async function Icon({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/shop/[slug]/icon.js switcher
export default async function Icon({ params }) {
  const { slug } = await params
  // ...
}
```

Route	URL	`params`
`app/shop/icon.js`	`/shop`	`undefined`
`app/shop/[slug]/icon.js`	`/shop/1`	`Promise<{ slug: '1' }>`
`app/shop/[tag]/[item]/icon.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`

Returns

The default export function should return a `Blob` | `ArrayBuffer` | `TypedArray` | `DataView` | `ReadableStream` | `Response`.

> **Good to know**: `ImageResponse` satisfies this return type.

Config exports

You can optionally configure the icon's metadata by exporting `size` and `contentType` variables from the `icon` or `apple-icon` route.

Option	Type
`size`	`{ width: number; height: number }`
`contentType`	`string` - image MIME type

`size`

```
// Archivo: icon.tsx | apple-icon.tsx switcher
export const size = { width: 32, height: 32 }

export default function Icon() {}
```

```
// Archivo: icon.js | apple-icon.js switcher
export const size = { width: 32, height: 32 }

export default function Icon() {}
```

```
// Archivo: <head> output
<link rel="icon" sizes="32x32" />
```

`contentType`

```
// Archivo: icon.tsx | apple-icon.tsx switcher
export const contentType = 'image/png'

export default function Icon() {}
```

```
// Archivo: icon.js | apple-icon.js switcher
export const contentType = 'image/png'

export default function Icon() {}
```

```
// Archivo: <head> output
<link rel="icon" type="image/png" />
```

Route Segment Config

`icon` and `apple-icon` are specialized Route Handlers that can use the same route segment configuration options as Pages and Layouts.

Version History

Version	Changes
`v16.0.0`	`params` is now a promise that resolves to an object
`v13.3.0`	`favicon` `icon` and `apple-icon` introduced

Metadata Files API Reference

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API documentation for the metadata file conventions.*

This section of the docs covers **Metadata file conventions**. File-based metadata can be defined by adding special metadata files to route segments.

Each file convention can be defined using a static file (e.g. `opengraph-image.jpg`), or a dynamic variant that uses code to generate the file (e.g. `opengraph-image.js`).

Once a file is defined, Next.js will automatically serve the file (with hashes in production for caching) and update the relevant head elements with the correct metadata, such as the asset's URL, file type, and image size.

> **Good to know**:
>
> - Special Route Handlers like `sitemap.ts`, `opengraph-image.tsx`, and `icon.tsx`, and other metadata files are cached by default.
> - If using along with `proxy.ts`, configure the matcher to exclude the metadata files.

manifest.json

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/manifest
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for manifest.json file.*

Add or generate a `manifest.(json|webmanifest)` file that matches the Web Manifest Specification in the **root** of `app` directory to provide information about your web application for the browser.

Static Manifest file

```
// Archivo: app/manifest.json | app/manifest.webmanifest
{
  "name": "My Next.js Application",
  "short_name": "Next.js App",
  "description": "An application built with Next.js",
  "start_url": "/"
  // ...
}
```

Generate a Manifest file

Add a `manifest.js` or `manifest.ts` file that returns a `Manifest` object.

> Good to know: `manifest.js` is a special Route Handlers that is cached by default unless it uses a Request-time API or dynamic config option.

```
// Archivo: app/manifest.ts switcher
import type { MetadataRoute } from 'next'

export default function manifest(): MetadataRoute.Manifest {
  return {
    name: 'Next.js App',
    short_name: 'Next.js App',
    description: 'Next.js App',
    start_url: '/',
    display: 'standalone',
    background_color: '#fff',
    theme_color: '#fff',
    icons: [
      {
        src: '/favicon.ico',
        sizes: 'any',
        type: 'image/x-icon',
      },
    ],
  }
}
```

```
// Archivo: app/manifest.js switcher
export default function manifest() {
  return {
    name: 'Next.js App',
    short_name: 'Next.js App',
    description: 'Next.js App',
    start_url: '/',
    display: 'standalone',
    background_color: '#fff',
    theme_color: '#fff',
    icons: [
      {
        src: '/favicon.ico',
        sizes: 'any',
        type: 'image/x-icon',
      },
    ],
  }
}
```

Manifest Object

The manifest object contains an extensive list of options that may be updated due to new web standards. For information on all the current options, refer to the `MetadataRoute.Manifest` type in your code editor if using TypeScript or see the MDN docs.

---

opengraph-image and twitter-image

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/opengraph-image
> 📁 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for the Open Graph Image and Twitter Image file conventions.*

The `opengraph-image` and `twitter-image` file conventions allow you to set Open Graph and Twitter images for a route segment.

They are useful for setting the images that appear on social networks and messaging apps when a user shares a link to your site.

There are two ways to set Open Graph and Twitter images:

- Using image files (.jpg, .png, .gif)
- Using code to generate images (.js, .ts, .tsx)

Image files (.jpg, .png, .gif)

Use an image file to set a route segment's shared image by placing an `opengraph-image` or `twitter-image` image file in the segment.

Next.js will evaluate the file and automatically add the appropriate tags to your app's `<head>` element.

File convention	Supported file types
`opengraph-image`	`.jpg` , `.jpeg` , `.png` , `.gif`
`twitter-image`	`.jpg` , `.jpeg` , `.png` , `.gif`
`opengraph-image.alt`	`.txt`
`twitter-image.alt`	`.txt`

> **Good to know**:
>
> The `twitter-image` file size must not exceed 5MB, and the `opengraph-image` file size must not exceed 8MB. If the image file size exceeds these limits, the build will fail.

`opengraph-image`

Add an `opengraph-image.(jpg|jpeg|png|gif)` image file to any route segment.

```
// Archivo: <head> output
<meta property="og:image" content="<generated>" />
<meta property="og:image:type" content="<generated>" />
<meta property="og:image:width" content="<generated>" />
<meta property="og:image:height" content="<generated>" />
```

`twitter-image`

Add a `twitter-image.(jpg|jpeg|png|gif)` image file to any route segment.

```
// Archivo: <head> output
<meta name="twitter:image" content="<generated>" />
<meta name="twitter:image:type" content="<generated>" />
<meta name="twitter:image:width" content="<generated>" />
<meta name="twitter:image:height" content="<generated>" />
```

`opengraph-image.alt.txt`

Add an accompanying `opengraph-image.alt.txt` file in the same route segment as the `opengraph-image.(jpg|jpeg|png|gif)` image its alt text.

```
// Archivo: opengraph-image.alt.txt
About Acme
```

```
// Archivo: <head> output
<meta property="og:image:alt" content="About Acme" />
```

`twitter-image.alt.txt`

Add an accompanying `twitter-image.alt.txt` file in the same route segment as the `twitter-image.(jpg|jpeg|png|gif)` image its alt text.

```
// Archivo: twitter-image.alt.txt
About Acme
```

```
// Archivo: <head> output
<meta name="twitter:image:alt" content="About Acme" />
```

Generate images using code (.js, .ts, .tsx)

In addition to using literal image files, you can programmatically **generate** images using code.

Generate a route segment's shared image by creating an `opengraph-image` or `twitter-image` route that default exports a function.

File convention	Supported file types
`opengraph-image`	`.js` , `.ts` , `.tsx`
`twitter-image`	`.js` , `.ts` , `.tsx`

Good to know:

- By default, generated images are **statically optimized** (generated at build time and cached) unless they use Request-time APIs or uncached data.
- You can generate multiple Images in the same file using `generateImageMetadata`.
- `opengraph-image.js` and `twitter-image.js` are special Route Handlers that are cached by default unless it uses a Request-time API or dynamic config option.

The easiest way to generate an image is to use the ImageResponse API from `next/og`.

```
// Archivo: app/about/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { readFile } from 'node:fs/promises'
import { join } from 'node:path'

// Image metadata
export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

const interSemiBold = await readFile(
  join(process.cwd(), 'assets/Inter-SemiBold.ttf')
)

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        About Acme
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
      fonts: [
        {
          name: 'Inter',
          data: interSemiBold,
          style: 'normal',
          weight: 400,
        },
      ],
    }
  )
}
```

```
// Archivo: app/about/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { readFile } from 'node:fs/promises'
import { join } from 'node:path'

// Image metadata
export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

const interSemiBold = await readFile(
  join(process.cwd(), 'assets/Inter-SemiBold.ttf')
)

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        About Acme
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
      fonts: [
        {
          name: 'Inter',
          data: interSemiBold,
          style: 'normal',
          weight: 400,
        },
      ],
    }
  )
}
```

```
// Archivo: <head> output
<meta property="og:image" content="<generated>" />
<meta property="og:image:alt" content="About Acme" />
<meta property="og:image:type" content="image/png" />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
```

Props

The default export function receives the following props:

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters object from the root segment down to the segment `opengraph-image` or `twitter-image` is colocated in.

> **Good to know**: If you use `generateImageMetadata`, the function will also receive an `id` prop that is a promise resolving to the `id` value from one of the items returned by `generateImageMetadata`.

```
// Archivo: app/shop/[slug]/opengraph-image.tsx switcher
export default async function Image({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/shop/[slug]/opengraph-image.js switcher
export default async function Image({ params }) {
  const { slug } = await params
  // ...
}
```

Route	URL	`params`
`app/shop/opengraph-image.js`	`/shop`	`undefined`
`app/shop/[slug]/opengraph-image.js`	`/shop/1`	`Promise<{ slug: '1' }>`
`app/shop/[tag]/[item]/opengraph-image.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`

Returns

The default export function should return a `Response`.

> **Good to know**: `ImageResponse` satisfies this return type.

Config exports

You can optionally configure the image's metadata by exporting `alt`, `size`, and `contentType` variables from `opengraph-image` or `twitter-image` route.

Option	Type
`alt`	`string`
`size`	`{ width: number; height: number }`
`contentType`	`string` - image MIME type

`alt`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const alt = 'My images alt text'

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const alt = 'My images alt text'

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:alt" content="My images alt text" />
```

`size`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const size = { width: 1200, height: 630 }

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const size = { width: 1200, height: 630 }

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
```

`contentType`

```
// Archivo: opengraph-image.tsx | twitter-image.tsx switcher
export const contentType = 'image/png'

export default function Image() {}
```

```
// Archivo: opengraph-image.js | twitter-image.js switcher
export const contentType = 'image/png'

export default function Image() {}
```

```
// Archivo: <head> output
<meta property="og:image:type" content="image/png" />
```

Route Segment Config

`opengraph-image` and `twitter-image` are specialized Route Handlers that can use the same route segment configuration options as Pages and Layouts.

Examples

Using external data

This example uses the `params` object and external data to generate the image.

> **Good to know**: By default, this generated image will be statically optimized. You can configure the individual `fetch` `options` or route segments options to change this behavior.

```tsx
// Archivo: app/posts/[slug]/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'

export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}
export const contentType = 'image/png'

export default async function Image({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const post = await fetch(`https:// ... /posts/${slug}`).then((res) =>
    res.json()
  )

  return new ImageResponse(
    (
      <div
        style={{
          fontSize: 48,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    ),
    {
      ... size,
    }
  )
}
```

```
// Archivo: app/posts/[slug]/opengraph-image.js switcher
import { ImageResponse } from 'next/og'

export const alt = 'About Acme'
export const size = {
  width: 1200,
  height: 630,
}
export const contentType = 'image/png'

export default async function Image({ params }) {
  const { slug } = await params
  const post = await fetch(`https:// ... /posts/${slug}`).then((res) =>
    res.json()
  )

  return new ImageResponse(
    (
      <div
        style={{
          fontSize: 48,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        {post.title}
      </div>
    ),
    {
      ... size,
    }
  )
}
```

Using Node.js runtime with local assets

These examples use the Node.js runtime to fetch a local image from the file system and pass it to the `<img>` `src` attribute, either as a base64 string or an `ArrayBuffer`. Place the local asset relative to the project root, not the example source file.

The asset doesn't depend on request data, so read it once at module scope. See Predictable values.

```
// Archivo: app/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'), 'base64')
const logoSrc = `${logoData}`

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

```
// Archivo: app/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'), 'base64')
const logoSrc = `${logoData}`

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

Passing an `ArrayBuffer` to the `src` attribute of an `<img>` element is not part of the HTML spec. The rendering engine used by `next/og` supports it, but because TypeScript definitions follow the spec, you need a `@ts-expect-error` directive or similar to use this feature.

```
// Archivo: app/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'))
const logoSrc = Uint8Array.from(logoData).buffer

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >

        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

```
// Archivo: app/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { join } from 'node:path'
import { readFile } from 'node:fs/promises'

const logoData = await readFile(join(process.cwd(), 'logo.png'))
const logoSrc = Uint8Array.from(logoData).buffer

export default async function Image() {
  return new ImageResponse(
    (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <img src={logoSrc} height="100" />
      </div>
    )
  )
}
```

Version History

Version	Changes
`v16.0.0`	`params` is now a promise that resolves to an object
`v13.3.0`	`opengraph-image` and `twitter-image` introduced.

robots.txt

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/robots
> 📁 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for robots.txt file.*

Add or generate a `robots.txt` file that matches the Robots Exclusion Standard in the **root** of `app` directory to tell search engine crawlers which URLs they can access on your site.

Static `robots.txt`

```
// Archivo: app/robots.txt
User-Agent: *
Allow: /
Disallow: /private/

Sitemap: https://acme.com/sitemap.xml
```

Generate a Robots file

Add a `robots.js` or `robots.ts` file that returns a `Robots` object.

> **Good to know**: `robots.js` is a special Route Handler that is cached by default unless it uses a Request-time API or dynamic config option.

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: '/private/',
    },
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: '/private/',
    },
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

Output:

```
User-Agent: *
Allow: /
Disallow: /private/

Sitemap: https://acme.com/sitemap.xml
```

Customizing specific user agents

You can customize how individual search engine bots crawl your site by passing an array of user agents to the `rules` property. For example:

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      {
        userAgent: 'Googlebot',
        allow: ['/'],
        disallow: '/private/',
      },
      {
        userAgent: ['Applebot', 'Bingbot'],
        disallow: ['/'],
      },
    ],
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: [
      {
        userAgent: 'Googlebot',
        allow: ['/'],
        disallow: ['/private/'],
      },
      {
        userAgent: ['Applebot', 'Bingbot'],
        disallow: ['/'],
      },
    ],
    sitemap: 'https://acme.com/sitemap.xml',
  }
}
```

Output:

```
User-Agent: Googlebot
Allow: /
Disallow: /private/

User-Agent: Applebot
Disallow: /

User-Agent: Bingbot
Disallow: /

Sitemap: https://acme.com/sitemap.xml
```

Non-standard directives

Some search engines support directives that aren't part of the Robots Exclusion Standard, such as `Request-Rate` (Seznam) or `Clean-param` (Yandex). Pass these through the `other` field on a rule. Keys preserve their casing and array values emit one line per entry, scoped to the rule's `User-Agent` block.

```
// Archivo: app/robots.ts switcher
import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      { userAgent: '*', allow: '/' },
      {
        userAgent: 'SeznamBot',
        allow: '/',
        other: {
          'Request-Rate': '10/1m',
        },
      },
    ],
  }
}
```

```
// Archivo: app/robots.js switcher
export default function robots() {
  return {
    rules: [
      { userAgent: '*', allow: '/' },
      {
        userAgent: 'SeznamBot',
        allow: '/',
        other: {
          'Request-Rate': '10/1m',
        },
      },
    ],
  }
}
```

Output:

```
User-Agent: *
Allow: /

User-Agent: SeznamBot
Allow: /
Request-Rate: 10/1m
```

> **Good to know**: Values in `other` are passed through verbatim. Next.js does not validate directive names or values, so refer to the target search engine's documentation for the exact syntax.

Robots object

```
type Robots = {
  rules:
    | {
        userAgent?: string | string[]
        allow?: string | string[]
        disallow?: string | string[]
        crawlDelay?: number
        other?: Record<string, string | number | Array<string | number>>
      }
    | Array<{
        userAgent: string | string[]
        allow?: string | string[]
        disallow?: string | string[]
        crawlDelay?: number
        other?: Record<string, string | number | Array<string | number>>
      }>
  sitemap?: string | string[]
  host?: string
}
```

Version History

Version	Changes
`v16.3.0`	Added `other` field for non-standard per-agent directives.
`v13.3.0`	`robots` introduced.

sitemap.xml

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/metadata/sitemap
> 📂 **Sección:** App > Api Reference > File Conventions > Metadata
> 📝 **Descripción:** *API Reference for the sitemap.xml file.*

`sitemap.(xml|js|ts)` is a special file that matches the Sitemaps XML format to help search engine crawlers index your site more efficiently.

Sitemap files (.xml)

For smaller applications, you can create a `sitemap.xml` file and place it in the root of your `app` directory.

```
// Archivo: app/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>yearly</changefreq>
    <priority>1</priority>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Generating a sitemap using code (.js, .ts)

You can use the `sitemap.(js|ts)` file convention to programmatically **generate** a sitemap by exporting a default function that returns an array of URLs. If using TypeScript, a `Sitemap` type is available.

> **Good to know**: `sitemap.js` is a special Route Handler that is cached by default unless it uses a Request-time API or dynamic config option.

```ts
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      changeFrequency: 'yearly',
      priority: 1,
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.8,
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 0.5,
    },
  ]
}
```

```js
// Archivo: app/sitemap.js switcher
export default function sitemap() {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      changeFrequency: 'yearly',
      priority: 1,
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.8,
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 0.5,
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>yearly</changefreq>
    <priority>1</priority>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Image Sitemaps

You can use `images` property to create image sitemaps. Learn more details in the Google Developer Docs.

```
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://example.com',
      lastModified: '2021-01-01',
      changeFrequency: 'weekly',
      priority: 0.5,
      images: ['https://example.com/image.jpg'],
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<?xml version="1.0" encoding="UTF-8"?>
<urlset
  xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
  xmlns:image="http://www.google.com/schemas/sitemap-image/1.1"
>
  <url>
    <loc>https://example.com</loc>
    <image:image>
      <image:loc>https://example.com/image.jpg</image:loc>
    </image:image>
    <lastmod>2021-01-01</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Video Sitemaps

You can use `videos` property to create video sitemaps. Learn more details in the Google Developer Docs.

```ts
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://example.com',
      lastModified: '2021-01-01',
      changeFrequency: 'weekly',
      priority: 0.5,
      videos: [
        {
          title: 'example',
          thumbnail_loc: 'https://example.com/image.jpg',
          description: 'this is the description',
        },
      ],
    },
  ]
}
```

Output:

```xml
// Archivo: acme.com/sitemap.xml
<?xml version="1.0" encoding="UTF-8"?>
<urlset
  xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
  xmlns:video="http://www.google.com/schemas/sitemap-video/1.1"
>
  <url>
    <loc>https://example.com</loc>
    <video:video>
      <video:title>example</video:title>
      <video:thumbnail_loc>https://example.com/image.jpg</video:thumbnail_loc>
      <video:description>this is the description</video:description>
    </video:video>
    <lastmod>2021-01-01</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Generate a localized Sitemap

```
// Archivo: app/sitemap.ts switcher
import type { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es',
          de: 'https://acme.com/de',
        },
      },
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/about',
          de: 'https://acme.com/de/about',
        },
      },
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/blog',
          de: 'https://acme.com/de/blog',
        },
      },
    },
  ]
}
```

```
// Archivo: app/sitemap.js switcher
export default function sitemap() {
  return [
    {
      url: 'https://acme.com',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es',
          de: 'https://acme.com/de',
        },
      },
    },
    {
      url: 'https://acme.com/about',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/about',
          de: 'https://acme.com/de/about',
        },
      },
    },
    {
      url: 'https://acme.com/blog',
      lastModified: new Date(),
      alternates: {
        languages: {
          es: 'https://acme.com/es/blog',
          de: 'https://acme.com/de/blog',
        },
      },
    },
  ]
}
```

Output:

```
// Archivo: acme.com/sitemap.xml
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://www.w3.org/1999/xhtml">
  <url>
    <loc>https://acme.com</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es/about" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de/about" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <xhtml:link
      rel="alternate"
      hreflang="es"
      href="https://acme.com/es/blog" />
    <xhtml:link
      rel="alternate"
      hreflang="de"
      href="https://acme.com/de/blog" />
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
  </url>
</urlset>
```

Generating multiple sitemaps

While a single sitemap will work for most applications. For large web applications, you may need to split a sitemap into multiple files.

There are two ways you can create multiple sitemaps:

- By nesting `sitemap.(xml|js|ts)` inside multiple route segments e.g. `app/sitemap.xml` and `app/products/sitemap.xml`.
- By using the `generateSitemaps` function.

For example, to split a sitemap using `generateSitemaps`, return an array of objects with the sitemap `id`. Then, use the `id` to generate the unique sitemaps.

```
// Archivo: app/product/sitemap.ts switcher
import type { MetadataRoute } from 'next'
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props: {
  id: Promise<string>
}): Promise<MetadataRoute.Sitemap> {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

```
// Archivo: app/product/sitemap.js switcher
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props) {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

Your generated sitemaps will be available at `/ … /sitemap/[id].xml`. For example, `/product/sitemap/1.xml`.

See the `generateSitemaps` API reference for more information.

Returns

The default function exported from `sitemap.(xml|ts|js)` should return an array of objects with the following properties:

```
type Sitemap = Array<{
  url: string
  lastModified?: string | Date
  changeFrequency?:
    | 'always'
    | 'hourly'
    | 'daily'
    | 'weekly'
    | 'monthly'
    | 'yearly'
    | 'never'
  priority?: number
  alternates?: {
    languages?: Languages<string>
  }
  images?: string[]
  videos?: Videos[]
}>
```

Version History

Version	Changes
`v16.0.0`	`id` is now a promise that resolves to a `string`.
`v14.2.0`	Add localizations support.
`v13.4.14`	Add `changeFrequency` and `priority` attributes to sitemaps.
`v13.3.0`	`sitemap` introduced.

dynamicParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/dynamicParams
> 📁 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *API reference for the dynamicParams route segment config option.*

The `dynamicParams` option allows you to control what happens when a dynamic segment is visited that was not generated with generateStaticParams.

```
// Archivo: layout.tsx | page.tsx switcher
export const dynamicParams = true // true | false
```

```
// Archivo: layout.js | page.js | route.js switcher
export const dynamicParams = true // true | false
```

- `true` (default): Dynamic route segments not included in `generateStaticParams` are generated at request time.
- `false`: Dynamic route segments not included in `generateStaticParams` will return a 404.

> **Good to know**:
> - This option replaces the `fallback: true | false | blocking` option of `getStaticPaths` in the `pages` directory.
> - `dynamicParams` is not available when Cache Components is enabled.

Route Segment Config

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config
> 📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *Learn about how to configure options for Next.js route segments.*

The Route Segment Config options allow you to configure the behavior of a Page, Layout, or Route Handler by directly exporting the following variables:

Option	Type	Default
`dynamicParams`	`boolean`	`true`
`runtime`	`'nodejs' | 'edge' (deprecated)`	`'nodejs'`
`preferredRegion`	`'auto' | 'global' | 'home' | string | string[] (deprecated)`	`'auto'`
`maxDuration`	`number`	Set by deployment platform

Version History

Version	
`v16.0.0`	`dynamic`, `dynamicParams`, `revalidate`, and `fetchCache` removed when Cache Components is enabled. See Caching and Revalidating (Previous Model).
`v16.0.0`	`export const experimental_ppr = true` removed. A codemod is available.
`v15.0.0-RC`	`export const runtime = "experimental-edge"` deprecated. A codemod is available.

---

instant

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/instant
> 📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *API reference for the instant route segment config.*

The `instant` route segment config controls how Next.js validates whether a navigation into this segment would produce an instant UI.

Next.js uses prefetching to preload navigations for all in-app links on the current page. However, client-side data fetching or partially prerendered results from the server can lead to navigations that don't update the UI immediately. To help you build fast-feeling navigations, `instant` lets you tell Next.js what to expect from navigations through the configured segment. For instance, you can indicate that navigations into this segment should produce a UI that renders instantly without waiting on any externally loaded data. You can also indicate that navigations aren't expected to be instant.

When a segment is configured to expect an instant UI, Next.js surfaces any code in your application that would block the navigation from updating the UI immediately. See the Instant Navigation guide for an end-to-end walkthrough.

Good to know:

- The `instant` export only works when `cacheComponents` is enabled.
- `instant` cannot be used in Client Components. It will throw an error.
- Next.js does not perform prefetches in development, so navigations may not feel as instant as they will in production. Validation reflects what will happen during `next start`, where prefetching is enabled.

```
// Archivo: layout.tsx | page.tsx switcher
export const instant = true

export default function Page() {
  return <div>...</div>
}
```

```
// Archivo: layout.js | page.js switcher
export const instant = true

export default function Page() {
  return <div>...</div>
}
```

Reference

The export accepts:

- `true`: Opts the segment into validation at whatever level is configured globally (see Configuring validation defaults). With the framework defaults, validation runs in development only and surfaces errors in the dev overlay.
- `false`: Opts the segment out (see Disabling instant).
- An **object**: Opts in with additional options. See `level`.

`level`

Sets the severity at which validation runs for this segment.

```
// Archivo: page.tsx
export const instant = {
  level: 'warning',
}
```

- `'warning'`: Validates in development only. Errors appear in the dev overlay; the build is unaffected.

In the future a validation level that supports validation during build will be supported. Unless you are enabling experimental validation modes there is no need to specify level since the only level available is `"warning"`.

When `level` is omitted (or `instant = true` is used), the segment is validated at whatever level is configured globally — see Configuring validation defaults.

Disabling instant

Set `false` on a layout or page to indicate that this segment is allowed to block when navigating to it.

This is useful when a deeper page should be instant but an ancestor can't be. For instance, if a shared layout cannot load instantly but you still want to assert that pages beneath this layout can be navigated to instantly, you can ensure that the pages have `instant = true` while the shared layout has `instant = false`.

```
// Archivo: app/tabs/layout.tsx
export const instant = false
```

```
// Archivo: app/tabs/[tab]/page.tsx
export const instant = true
```

Navigations from outside into a tab are allowed to block at the layout. Navigations between tabs are validated for instant UI.

You don't need to add `false` to ancestors of an instant page just because they do something blocking — a higher-up `instant = true` doesn't force its descendants to validate, and leaving an ancestor unconfigured is fine. Reach for `false` only when you've configured a deeper page as instant and need to exempt navigations that pass through a blocking ancestor.

Disabling static shell validation

Cache Components also validates that each page in your app produces a non-empty static shell at prerender time. To opt a route out of this validation, ensure the highest `instant` config in the route's tree is `false` — a `false` higher in the tree takes precedence over any deeper `true` for the static-shell check.

Setting `false` on the root layout disables static shell validation for the entire app. Place the `false` as low as possible — only as high as needed to cover the routes you want to opt out — so the rest of the app keeps validating.

How validation works

`instant` triggers validation at every shared layout boundary in the route. Validation runs during development (on page loads and HMR updates) and surfaces errors in the dev error overlay.

Each error identifies the component that would block navigation. The fix is usually to cache the data with `use cache` or wrap it in a `<Suspense>` boundary.

Configuring validation defaults

By default (`validationLevel: 'warning'`), Cache Components apps validate every Page and Default segment in development. The `experimental.instantInsights.validationLevel` config tunes this behavior — for example, to limit validation to segments that opt in explicitly via `instant` .

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    instantInsights: {
      validationLevel: 'warning',
    },
  },
}
```

The supported levels are:

- `'warning'` *(framework default)*: Every Page and Default segment is implicitly validated at warning level (dev only).
- `'manual-warning'` : Only segments with an explicit `instant` are validated, at warning level (dev only).

Setting `instant = false` on a segment opts it out of validation entirely.

> **Good to know**:
>
> - The framework default may change in future versions to opt users into higher levels of validation. Because this feature is experimental, that change is not considered a breaking change. To pin a specific behavior, set `validationLevel` explicitly.
> - Framework-synthesized error routes (`/_global-error` , `/_not-found`) are excluded from implicit validation. To validate them, opt in explicitly with `instant` .

Inspecting loading states

The Navigation Inspector is available when Cache Components is enabled:

```
// Archivo: next.config.js
module.exports = {
  cacheComponents: true,
}
```

Open the Next.js DevTools and select **Navigation Inspector**, then turn on **Pause on navigations**. While the toggle is on:

- Refresh the page to freeze the initial static UI generated for the route, before any dynamic data streams in.
- Click a link to freeze the prefetched UI for the destination route.

When the UI is frozen, click **Resume** to let the current navigation finish. The toggle stays on, so the next refresh or link click pauses too. Turn it off when you are done. Use both page refreshes and link clicks to check that your loading states look right on first visit and on navigation.

Testing instant navigation

The `@next/playwright` package exports an `instant()` helper that holds back dynamic content while the callback runs against the instant UI. See the guide for a full example.

```
import { instant } from '@next/playwright'
```

Known issue: shared cookie across projects

The DevTools use a `next-instant-navigation-testing` cookie to hold back dynamic content and freeze the page at the instant UI. Because cookies are scoped to the domain and not the port, running multiple projects on the same domain (typically `localhost`) means the cookie is shared across them and can cause unexpected behavior. Clear the cookie or close the Navigation Inspector panel when switching between projects to avoid issues.

> **Good to know:** This will be fixed as part of stabilizing the feature.

TypeScript

```
type InstantConfig =
  | true
  | false
  | {
      level?: 'warning'
    }

export const instant: InstantConfig = true
```

Version History

Version	Changes
`v16.x.x`	`instant` export introduced (Cache Components only)

maxDuration

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/maxDuration
> 📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *API reference for the maxDuration route segment config option.*

The `maxDuration` option allows you to set the maximum execution time (in seconds) for server-side logic in a route segment. Deployment platforms can use `maxDuration` from the Next.js build output to add specific execution limits.

```
// Archivo: layout.tsx | page.tsx | route.ts switcher
export const maxDuration = 5
```

```
// Archivo: layout.js | page.js | route.js switcher
export const maxDuration = 5
```

Server Actions

If using Server Actions, set the `maxDuration` at the page level to change the default timeout of all Server Actions used on the page.

Version History

Version	Changes
`v13.4.10`	`maxDuration` introduced.

preferredRegion (deprecated)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/preferredRegion
> 📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *API reference for the preferredRegion route segment config option.*

> **Deprecated:** The `preferredRegion` route segment config is deprecated. Remove the `preferredRegion` export from your route files. See the deprecation message for details.

The `preferredRegion` option allows you to specify the preferred deployment region for a route segment. This value is passed to your deployment platform.

```
// Archivo: layout.tsx | page.tsx | route.ts switcher
export const preferredRegion = // string || string[]
```

```
// Archivo: layout.js | page.js | route.js switcher
export const preferredRegion = // string || string[]
```

- `string`: Deploy the route to a specific region. Available region codes are platform-specific. For example, `'iad1'`.
- `string[]`: Deploy the route to multiple specific regions. The route is deployed to **all** listed regions, not a single one chosen from the list. For example, `['iad1', 'sfo1']`.

Good to know:

- If a `preferredRegion` is not specified, it will inherit the option of the nearest parent layout. The root layout defaults to `'auto'`.
- A child segment's value overrides the parent, values are not merged.
- Next.js passes the region values through to the deployment platform. The exact behavior and available region codes are platform-specific. Refer to your deployment platform's documentation for supported values.

Vercel

If deploying Next.js on Vercel, regions were previously only supported with `export const runtime = 'edge'`, which is now deprecated. The following options can be passed:

- `'auto'` (default): Uses the default region.
- `'global'`: Prefer deploying the route to all availableregions.
- `'home'`: Prefer deploying the route to the home region.

If an unsupported value is passed, an error will be thrown.

---

prefetch

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/prefetch
📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
📝 **Descripción:** *API reference for the prefetch route segment config.*

The `prefetch` route segment config controls how a segment is prefetched during client-side navigation. By default, the framework manages the strategy based on the app's `partialPrefetching` setting. To override per segment, set this export to one of the values below.

Good to know:

- The `prefetch` export only works when `cacheComponents` is enabled.
- `prefetch` cannot be used when the segment is a Client Component.
- The meaningful values to set are `'partial'` and `'force-disabled'`. `'auto'` is the default and is equivalent to omitting the export; don't write `prefetch = 'auto'` explicitly.

```
// Archivo: layout.tsx | page.tsx switcher
export const prefetch = 'partial'

export default function Page() {
  return <div>...</div>
}
```

```
// Archivo: layout.js | page.js switcher
export const prefetch = 'partial'

export default function Page() {
  return <div>...</div>
}
```

Options

`'partial'`

Opts the segment into Partial Prefetching without enabling the global `partialPrefetching` flag. A `<Link>` pointing at a segment with `prefetch = 'partial'` loads the per-route App Shell instead of the legacy full prefetch. Set this on the destination, not the link.

For links that opt into a wider prefetch with `<Link prefetch={true}>`, Next.js uses per-link prefetching. The server renders a fresh response that resolves URL data (`params` , `searchParams` , and the full URL). On pages where all the content is statically renderable, Next.js serves prefetches from the static cache. If a page accesses non-static data, it's prefetched at runtime.

Use this for incremental adoption when you can't enable `partialPrefetching` for the entire app at once. Once every route in scope has `prefetch = 'partial'` , enable the global flag and remove the per-route exports.

> **Good to know**: When Next.js performs a per-link prefetch for a segment, all downstream segments are included in the same request. Segments deeper in the tree that are configured with `'force-disabled'` will still be prefetched as part of the response.

```
// Archivo: page.tsx
export const prefetch = 'partial'
```

`'force-disabled'`

Never prefetch this segment. The client will not request segment data ahead of navigation. Use this for segments where prefetching would be wasteful, for example pages behind authentication that are rarely visited.

> **Good to know**: `'force-disabled'` does not prevent Next.js from prefetching metadata about the route. However, the actual segment data for this segment and all deeper segments will be omitted from prefetching.

Relationship with the `<Link prefetch>` prop

A prefetch starts with a `<Link>` that expresses intent (should this destination be prefetched, and how eagerly), and ends at a segment that sets a cost ceiling (how much work is it OK to do ahead of time, for any link that points here).

A destination can't know which links target it, so the segment config caps what any `<Link prefetch={true}>` pulls:

- `'partial'` : App Shell for default links; a `<Link prefetch={true}>` additionally resolves URL data (`params` , `searchParams` , and the full URL) and the cached content behind it.
- `'force-disabled'` : skip segment data entirely.

`<Link prefetch={false}>` skips prefetching at the link level regardless of how the destination is configured.

> **Good to know**: On pages where all the content is statically renderable, Next.js serves prefetches from the static cache (or a CDN). If a page accesses non-static data like cookies or headers, it's prefetched at runtime with a fresh server render, which costs server CPU per page view.

TypeScript

```
type Prefetch = 'auto' | 'partial' | 'force-disabled'

export const prefetch: Prefetch = 'partial'
```

Version History

Version	Changes
`v16.x.x`	`prefetch` export introduced (Cache Components only)

runtime

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-segment-config/runtime
> 📂 **Sección:** App > Api Reference > File Conventions > Route Segment Config
> 📝 **Descripción:** *API reference for the runtime route segment config option.*

The `runtime` option allows you to select the JavaScript runtime used for rendering your route.

```
// Archivo: layout.tsx | page.tsx | route.ts switcher
export const runtime = 'nodejs'
// 'nodejs'
```

```
// Archivo: layout.js | page.js | route.js switcher
export const runtime = 'nodejs'
// 'nodejs'
```

- `'nodejs'` (default)
- `'edge'` (deprecated)

> **Good to know**:
> - The Edge Runtime is deprecated. Remove the `runtime` export from your route files. See Edge Runtime Deprecated.
> - This option cannot be used in Proxy.

default.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/default
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API Reference for the default.js file.*

The `default.js` file is used to render a fallback within Parallel Routes when Next.js cannot recover a slot's active state after a full-page load.

During soft navigation, Next.js keeps track of the active *state* (subpage) for each slot. However, for hard navigations (full-page load), Next.js cannot recover the active state. In this case, a `default.js` file can be rendered for subpages that don't match the current URL.

Consider the following folder structure. The `@team` slot has a `settings` page, but `@analytics` does not.

Parallel Routes unmatched routes

When navigating to `/settings`, the `@team` slot will render the `settings` page while maintaining the currently active page for the `@analytics` slot.

On refresh, Next.js will render a `default.js` for `@analytics`. If `default.js` doesn't exist, an error is returned for named slots (`@team`, `@analytics`, etc) and requires you to define a `default.js` in order to continue. If you want to preserve the old behavior of returning a 404 in these situations, you can create a `default.js` that contains:

```
// Archivo: app/@team/default.js
import { notFound } from 'next/navigation'

export default function Default() {
  notFound()
}
```

Additionally, since `children` is an implicit slot, you also need to create a `default.js` file to render a fallback for `children` when Next.js cannot recover the active state of the parent page. If you don't create a `default.js` for the `children` slot, it will return a 404 page for the route.

Reference

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters from the root segment down to the slot's subpages. For example:

```
// Archivo: app/[artist]/@sidebar/default.js switcher
export default async function Default({
  params,
}: {
  params: Promise<{ artist: string }>
}) {
  const { artist } = await params
}
```

```
// Archivo: app/[artist]/@sidebar/default.js switcher
export default async function Default({ params }) {
  const { artist } = await params
}
```

Example	URL	`params`
`app/[artist]/@sidebar/default.js`	`/zack`	`Promise<{ artist: 'zack' }>`
`app/[artist]/[album]/@sidebar/default.js`	`/zack/next`	`Promise<{ artist: 'zack', album: 'next' }>`

- Since the `params` prop is a promise. You must use `async/await` or React's `use` function to access the values.
  - In version 14 and earlier, `params` was a synchronous prop. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.

Dynamic Route Segments

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/dynamic-routes
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Use Dynamic Segments to read URL path params and generate routes from dynamic data.*

A URL path is a sequence of path segments. In the App Router, a segment may be **static** (a literal value matched exactly) or **dynamic** (a placeholder that captures a value from the URL). When you don't know a segment's value ahead of time, define a Dynamic Segment to create routes from dynamic data. Next.js passes the captured values to your page via the path `params` prop, either filled in at request time or prerendered at build time.

> **Good to know**: Dynamic Segments are often referred to as path params, route params, or URL params.

Convention

A Dynamic Segment can be created by wrapping a folder's name in square brackets: `[folderName]`. For example, a blog could include the following route `app/blog/[slug]/page.js` where `[slug]` is the Dynamic Segment for blog posts.

```
// Archivo: app/blog/[slug]/page.tsx switcher
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  return <div>My Post: {slug}</div>
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export default async function Page({ params }) {
  const { slug } = await params
  return <div>My Post: {slug}</div>
}
```

Dynamic Segments are passed as the `params` prop to `layout`, `page`, `route`, and `generateMetadata` functions.

Route	Example URL	`params`
`app/blog/[slug]/page.js`	`/blog/a`	`{ slug: 'a' }`
`app/blog/[slug]/page.js`	`/blog/b`	`{ slug: 'b' }`
`app/blog/[slug]/page.js`	`/blog/c`	`{ slug: 'c' }`

Dynamic segments that appear before the root layout are **root parameters**, which can additionally be read from any Server Component with `next/root-params`.

In Client Components

In a Client Component **page**, dynamic segments from props can be accessed using the `use` API.

```
// Archivo: app/blog/[slug]/page.tsx switcher
'use client'
import { use } from 'react'

export default function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = use(params)

  return (
    <div>
      <p>{slug}</p>
    </div>
  )
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
'use client'
import { use } from 'react'

export default function BlogPostPage({ params }) {
  const { slug } = use(params)

  return (
    <div>
      <p>{slug}</p>
    </div>
  )
}
```

Alternatively Client Components can use the `useParams` hook to access the `params` anywhere in the Client Component tree.

Catch-all Segments

Dynamic Segments can be extended to **catch-all** subsequent segments by adding an ellipsis inside the brackets `[...folderName]`.

For example, `app/shop/[...slug]/page.js` will match `/shop/clothes`, but also `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`, and so on.

Route	Example URL	`params`
`app/shop/[...slug]/page.js`	`/shop/a`	`{ slug: ['a'] }`
`app/shop/[...slug]/page.js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`app/shop/[...slug]/page.js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Optional Catch-all Segments

Catch-all Segments can be made **optional** by including the parameter in double square brackets: `[[...folderName]]`.

For example, `app/shop/[[...slug]]/page.js` will **also** match `/shop`, in addition to `/shop/clothes`, `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`.

The difference between **catch-all** and **optional catch-all** segments is that with optional, the route without the parameter is also matched (`/shop` in the example above).

Route	Example URL	`params`
`app/shop/[[...slug]]/page.js`	`/shop`	`{ slug: undefined }`
`app/shop/[[...slug]]/page.js`	`/shop/a`	`{ slug: ['a'] }`
`app/shop/[[...slug]]/page.js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`app/shop/[[...slug]]/page.js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

TypeScript

When using TypeScript, you can add types for `params` depending on your configured route segment — use `PageProps<'/route'>`, `LayoutProps<'/route'>`, or `RouteContext<'/route'>` to type `params` in `page`, `layout`, and `route` respectively.

Route `params` values are typed as `string`, `string[]`, or `undefined` (for optional catch-all segments), because their values aren't known until runtime. Users can enter any URL into the address bar, and these broad types help ensure that your application code handles all these possible cases.

Route	`params` Type Definition
`app/blog/[slug]/page.js`	`{ slug: string }`
`app/shop/[...slug]/page.js`	`{ slug: string[] }`
`app/shop/[[...slug]]/page.js`	`{ slug?: string[] }`
`app/[categoryId]/[itemId]/page.js`	`{ categoryId: string, itemId: string }`

If you're working on a route where `params` can only have a fixed number of valid values, such as a `[locale]` param with a known set of language codes, you can use runtime validation to handle any invalid params a user may enter, and let the rest of your application work with the narrower type from your known set.

```
// Archivo: /app/[locale]/page.tsx
import { notFound } from 'next/navigation'
import type { Locale } from '@i18n/types'
import { isValidLocale } from '@i18n/utils'

function assertValidLocale(value: string): asserts value is Locale {
  if (!isValidLocale(value)) notFound()
}

export default async function Page(props: PageProps<'/[locale]'>) {
  const { locale } = await props.params // locale is typed as string
  assertValidLocale(locale)
  // locale is now typed as Locale
}
```

Behavior

- Since the `params` prop is a promise. You must use `async` / `await` or React's use function to access the values.
  - In version 14 and earlier, `params` was a synchronous prop. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.

With Cache Components

When using Cache Components with dynamic route segments, how you handle params depends on whether you use `generateStaticParams`.

Without `generateStaticParams`, param values are unknown during prerendering, making params runtime data. You must wrap param access in `````) }

async function Content({ slug }: { slug: string }) { const res = await fetch(`https://api.vercel.app/blog/${slug}`) const post = await res.json()

return (

{post.title}

{post.content}

) }

```
#### With `generateStaticParams`

Provide params ahead of time to prerender pages at build time. You can prerender all routes or a subset depending on your needs.

During the build process, the route is executed with each sample param to collect the HTML result. If dynamic content or runtime data are accessed incorrectly, the build will fail.

```tsx
// Archivo: app/blog/[slug]/page.tsx highlight={3-5,8,19}
import { Suspense } from 'react'

export async function generateStaticParams() {
  return [{ slug: '1' }, { slug: '2' }, { slug: '3' }]
}

export default async function Page({ params }: PageProps<'/blog/[slug]'>) {
  const { slug } = await params

  return (
    <div>
      <h1>Blog Post</h1>
      <Content slug={slug} />
    </div>
  )
}

async function Content({ slug }: { slug: string }) {
  const post = await getPost(slug)
  return (
    <article>
      <h2>{post.title}</h2>
      <p>{post.content}</p>
    </article>
  )
}

async function getPost(slug: string) {
  'use cache'
  const res = await fetch(`https://api.vercel.app/blog/${slug}`)
  return res.json()
}
```
```

Build-time validation only covers code paths that execute with the sample params. If your route has conditional logic that accesses runtime APIs for certain param values not in your samples, those branches won't be validated at build time:

```
// Archivo: app/blog/[slug]/page.tsx
import { cookies } from 'next/headers'

export async function generateStaticParams() {
  return [{ slug: 'public-post' }, { slug: 'hello-world' }]
}

export default async function Page({ params }: PageProps<'/blog/[slug]'>) {
  const { slug } = await params

  if (slug.startsWith('private-')) {
    // This branch is never executed at build time
    // Runtime requests for 'private-*' slugs will error
    return <PrivatePost slug={slug} />
  }

  return <PublicPost slug={slug} />
}

async function PrivatePost({ slug }: { slug: string }) {
  const token = (await cookies()).get('token')
  // ... fetch and render private post using token for auth
}
```

For runtime params not returned by `generateStaticParams`, validation occurs during the first request. In the example above, requests for slugs starting with `private-` will fail because `PrivatePost` accesses `cookies()` without a Suspense boundary. Other runtime params that don't hit the conditional branch will render successfully and be saved to disk for subsequent requests.

To fix this, wrap `PrivatePost` with Suspense:

```
// Archivo: app/blog/[slug]/page.tsx highlight={13-15}
import { Suspense } from 'react'
import { cookies } from 'next/headers'

export async function generateStaticParams() {
  return [{ slug: 'public-post' }, { slug: 'hello-world' }]
}

export default async function Page({ params }: PageProps<'/blog/[slug]'>) {
  const { slug } = await params

  if (slug.startsWith('private-')) {
    return (
      ``<Suspense>``
    )
  }

  return <PublicPost slug={slug} />
}

async function PrivatePost({ slug }: { slug: string }) {
  const token = (await cookies()).get('token')
  // ... fetch and render private post using token for auth
}
```

Examples

With `generateStaticParams`

The `generateStaticParams` function can be used to statically generate routes at build time instead of on-demand at request time.

```
// Archivo: app/blog/[slug]/page.tsx switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}
```

When using `fetch` inside the `generateStaticParams` function, the requests are automatically deduplicated. This avoids multiple network calls for the same data Layouts, Pages, and other `generateStaticParams` functions, speeding up build time.

Dynamic GET Route Handlers with `generateStaticParams`

`generateStaticParams` also works with dynamic Route Handlers to statically generate API responses at build time:

```tsx
// Archivo: app/api/posts/[id]/route.ts switcher
export async function generateStaticParams() {
  const posts: { id: number }[] = await fetch(
    'https://api.vercel.app/blog'
  ).then((res) => res.json())

  return posts.map((post) => ({
    id: `${post.id}`,
  }))
}

export async function GET(
  request: Request,
  { params }: RouteContext<'/api/posts/[id]'>
) {
  const { id } = await params
  const res = await fetch(`https://api.vercel.app/blog/${id}`)

  if (!res.ok) {
    return Response.json({ error: 'Post not found' }, { status: 404 })
  }

  const post = await res.json()
  return Response.json(post)
}
```

```jsx
// Archivo: app/api/posts/[id]/route.js switcher
export async function generateStaticParams() {
  const posts = await fetch('https://api.vercel.app/blog').then((res) =>
    res.json()
  )

  return posts.map((post) => ({
    id: `${post.id}`,
  }))
}

export async function GET(request, { params }) {
  const { id } = await params
  const res = await fetch(`https://api.vercel.app/blog/${id}`)

  if (!res.ok) {
    return Response.json({ error: 'Post not found' }, { status: 404 })
  }

  const post = await res.json()
  return Response.json(post)
}
```

In this example, route handlers for all blog post IDs returned by `generateStaticParams` will be statically generated at build time. Requests to other IDs will be handled dynamically at request time.

---

error.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/error
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the error.js special file.*

An **error** file allows you to handle unexpected runtime errors and display fallback UI.

error.js special file

```tsx
// Archivo: app/dashboard/error.tsx switcher
'use client' // Error boundaries must be Client Components

import { useEffect } from 'react'

export default function Error({
  error,
  retry,
}: {
  error: Error & { digest?: string }
  retry: () => void
}) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error(error)
  }, [error])

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button
        onClick={
          // Attempt to recover by re-fetching and re-rendering the segment
          () => retry()
        }
      >
        Try again
      </button>
    </div>
  )
}
```

```jsx
// Archivo: app/dashboard/error.js switcher
'use client' // Error boundaries must be Client Components

import { useEffect } from 'react'

export default function Error({ error, retry }) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error(error)
  }, [error])

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button
        onClick={
          // Attempt to recover by re-fetching and re-rendering the segment
          () => retry()
        }
      >
        Try again
      </button>
    </div>
  )
}
```

`error.js` wraps a route segment and its nested children in a React Error Boundary. When an error throws within the boundary, the `error` component shows as the fallback UI.

How error.js works

> **Good to know**:
>
> - The React DevTools allow you to toggle error boundaries to test error states.
> - If you want errors to bubble up to the parent error boundary, you can `throw` when rendering the `error` component.
> - For component-level error recovery that aren't tied to route segments like `error.js`, use the `catchError` function.

In the component hierarchy, `error.js` wraps `loading.js`, `not-found.js`, `page.js`, and nested `layout.js` files in a React error boundary. It does **not** wrap the `layout.js` or `template.js` above it in the same segment. To handle errors in the root layout, use `global-error.js`.

Reference

Props

`error`

An instance of an `Error` object forwarded to the `error.js` Client Component.

> **Good to know:** During development, the `Error` object forwarded to the client will be serialized and include the `message` of the original error for easier debugging. However, **this behavior is different in production** to avoid leaking potentially sensitive details included in the error to the client.

`error.message`

- Errors forwarded from Client Components show the original `Error` message.
- Errors forwarded from Server Components show a generic message with an identifier. This is to prevent leaking sensitive details. You can use the identifier, under `errors.digest`, to match the corresponding server-side logs.

`error.digest`

An automatically generated hash of the error thrown. It can be used to match the corresponding error in server-side logs.

`retry`

The cause of an error can sometimes be temporary. In these cases, trying again might resolve the issue.

An error component can use the `retry()` function to prompt the user to attempt to recover from the error. When executed, the function will try to re-fetch and re-render the error boundary's children. If successful, the fallback error component is replaced with the result of the re-render.

```
// Archivo: app/dashboard/error.tsx switcher
'use client' // Error boundaries must be Client Components

export default function Error({
  error,
  retry,
}: {
  error: Error & { digest?: string }
  retry: () => void
}) {
  return (
    <div>
      <h2>Something went wrong!</h2>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}
```

```
// Archivo: app/dashboard/error.js switcher
'use client' // Error boundaries must be Client Components

export default function Error({ error, retry }) {
  return (
    <div>
      <h2>Something went wrong!</h2>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}
```

`reset`

In most cases, you should use `retry()` instead. However, if you have a specific reason to clear the error state and re-render the error boundary's children without re-fetching the contents, you can use the `reset()` function.

Examples

Global Error

While less common, you can handle errors in the root layout or template using `global-error.jsx`, located in the root app directory, even when leveraging internationalization. Global error UI must define its own `<html>` and `<body>` tags, global styles, fonts, or other dependencies that your error page requires. This file replaces the root layout or template when active.

> **Good to know**: `global-error` and the built-in 500 page render their own document and do **not** include your global styles, so an app-level theme toggle (a class or `data-theme` attribute) won't reach them. The default UI follows the OS color scheme; to match your app's theme, apply it inside your own `global-error` component.

> **Good to know**: Error boundaries must be Client Components, which means that `metadata` and `generateMetadata` exports are not supported in `global-error.jsx`. As an alternative, you can use the React `<title>` component.

```
// Archivo: app/global-error.tsx switcher
'use client' // Error boundaries must be Client Components

export default function GlobalError({
  error,
  retry,
}: {
  error: Error & { digest?: string }
  retry: () => void
}) {
  return (
    // global-error must include html and body tags
    <html>
      <body>
        <h2>Something went wrong!</h2>
        <button onClick={() => retry()}>Try again</button>
      </body>
    </html>
  )
}
```

```
// Archivo: app/global-error.js switcher
'use client' // Error boundaries must be Client Components

export default function GlobalError({ error, retry }) {
  return (
    // global-error must include html and body tags
    <html>
      <body>
        <h2>Something went wrong!</h2>
        <button onClick={() => retry()}>Try again</button>
      </body>
    </html>
  )
}
```

Graceful error recovery with a custom error boundary

When rendering fails on the client, it can be useful to show the last known server rendered UI for a better user experience.

The `GracefullyDegradingErrorBoundary` is an example of a custom error boundary that captures and preserves the current HTML before an error occurs. If a rendering error happens, it re-renders the captured HTML and displays a persistent notification bar to inform the user.

```tsx
// Archivo: app/dashboard/error.tsx switcher
'use client'

import React, { Component, ErrorInfo, ReactNode } from 'react'

interface ErrorBoundaryProps {
  children: ReactNode
  onError?: (error: Error, errorInfo: ErrorInfo) => void
}

interface ErrorBoundaryState {
  hasError: boolean
}

export class GracefullyDegradingErrorBoundary extends Component<
  ErrorBoundaryProps,
  ErrorBoundaryState
> {
  private contentRef: React.RefObject<HTMLDivElement | null>

  constructor(props: ErrorBoundaryProps) {
    super(props)
    this.state = { hasError: false }
    this.contentRef = React.createRef()
  }

  static getDerivedStateFromError(_: Error): ErrorBoundaryState {
    return { hasError: true }
  }

  componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    if (this.props.onError) {
      this.props.onError(error, errorInfo)
    }
  }

  render() {
    if (this.state.hasError) {
      // Render the current HTML content without hydration
      return (
        <>
          <div
            ref={this.contentRef}
            suppressHydrationWarning
            dangerouslySetInnerHTML={{
              __html: this.contentRef.current?.innerHTML || '',
            }}
          />
          <div className="fixed bottom-0 left-0 right-0 bg-red-600 text-white py-4 px-6 text-center">
            <p className="font-semibold">
              An error occurred during page rendering
            </p>
          </div>
        </>
      )
    }

    return <div ref={this.contentRef}>{this.props.children}</div>
  }
}

export default GracefullyDegradingErrorBoundary
```

```
// Archivo: app/dashboard/error.js switcher
'use client'

import React, { Component, createRef } from 'react'

class GracefullyDegradingErrorBoundary extends Component {
  constructor(props) {
    super(props)
    this.state = { hasError: false }
    this.contentRef = createRef()
  }

  static getDerivedStateFromError(_) {
    return { hasError: true }
  }

  componentDidCatch(error, errorInfo) {
    if (this.props.onError) {
      this.props.onError(error, errorInfo)
    }
  }

  render() {
    if (this.state.hasError) {
      // Render the current HTML content without hydration
      return (
        <>
          <div
            ref={this.contentRef}
            suppressHydrationWarning
            dangerouslySetInnerHTML={{
              __html: this.contentRef.current?.innerHTML || '',
            }}
          />
          <div className="fixed bottom-0 left-0 right-0 bg-red-600 text-white py-4 px-6 text-center">
            <p className="font-semibold">
              An error occurred during page rendering
            </p>
          </div>
        </>
      )
    }

    return <div ref={this.contentRef}>{this.props.children}</div>
  }
}

export default GracefullyDegradingErrorBoundary
```

Version History

Version	Changes
`v16.3.0`	`retry` prop became stable.
`v16.2.0`	`unstable_retry` prop added.
`v15.2.0`	Also display `global-error` in development.
`v13.1.0`	`global-error` introduced.
`v13.0.0`	`error` introduced.

forbidden.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/forbidden
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the forbidden.js special file.*

The **forbidden** file is used to render UI when the `forbidden` function is invoked during authentication. Along with allowing you to customize the UI, Next.js will return a `403` status code.

```
// Archivo: app/forbidden.tsx switcher
import Link from 'next/link'

export default function Forbidden() {
  return (
    <div>
      <h2>Forbidden</h2>
      <p>You are not authorized to access this resource.</p>
      [Return Home](/)
    </div>
  )
}
```

```
// Archivo: app/forbidden.jsx switcher
import Link from 'next/link'

export default function Forbidden() {
  return (
    <div>
      <h2>Forbidden</h2>
      <p>You are not authorized to access this resource.</p>
      [Return Home](/)
    </div>
  )
}
```

Reference

Props

`forbidden.js` components do not accept any props.

Version History

Version	Changes
`v15.1.0`	`forbidden.js` introduced.

File-system conventions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API Reference for Next.js file-system conventions.*

instrumentation-client.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/instrumentation-client
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Learn how to add client-side instrumentation to track and monitor your Next.js application's frontend performance.*

The `instrumentation-client.js|ts` file allows you to add monitoring, analytics code, and other side-effects that run before your application becomes interactive. This is useful for setting up performance tracking, error monitoring, polyfills, or any other client-side observability tools.

To use it, place the file in the **root** of your application or inside a `src` folder.

Usage

Unlike server-side instrumentation, you do not need to export any specific functions. You can write your monitoring code directly in the file:

```
// Archivo: instrumentation-client.ts switcher
// Set up performance monitoring
performance.mark('app-init')

// Initialize analytics
console.log('Analytics initialized')

// Set up error tracking
window.addEventListener('error', (event) => {
  // Send to your error tracking service
  reportError(event.error)
})
```

```
// Archivo: instrumentation-client.js switcher
// Set up performance monitoring
performance.mark('app-init')

// Initialize analytics
console.log('Analytics initialized')

// Set up error tracking
window.addEventListener('error', (event) => {
  // Send to your error tracking service
  reportError(event.error)
})
```

Error handling: Implement try-catch blocks around your instrumentation code to ensure robust monitoring. This prevents individual tracking failures from affecting other instrumentation features.

Router navigation tracking

You can export `onRouterTransitionStart` to observe the start of App Router navigations:

```
// Archivo: instrumentation-client.ts
export function onRouterTransitionStart(
  url: string,
  navigationType: 'push' | 'replace' | 'traverse'
) {
  console.log(url, navigationType)
}
```

Additional router transition information is experimental. Enable it to receive a third `event` argument:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    instrumentationClientRouterTransitionEvents: true,
  },
}

export default nextConfig
```

The event includes the transition metadata and source context known when the navigation is dispatched:

```
// Archivo: instrumentation-client.ts switcher
import type { RouterTransitionStartEvent, RouterTransitionType } from 'next'

export function onRouterTransitionStart(
  url: string,
  navigationType: RouterTransitionType,
  { id, timestamp, fromRoutes, prefetchIntent }: RouterTransitionStartEvent
) {
  console.log(id, timestamp, url, navigationType, fromRoutes, prefetchIntent)
}
```

```
// Archivo: instrumentation-client.js switcher
export function onRouterTransitionStart(url, navigationType, event) {
  console.log(
    event.id,
    event.timestamp,
    url,
    navigationType,
    event.fromRoutes,
    event.prefetchIntent
  )
}
```

`onRouterTransitionStart` receives:

- `url: string` - The URL being navigated to
- `navigationType: 'push' | 'replace' | 'traverse'` - The type of navigation
- `event.id` - An opaque ID shared by events for this transition
- `event.timestamp` - A framework-captured Unix timestamp in milliseconds
- `event.fromRoutes` - Route patterns visible before navigation. The primary `children` route is first, followed by parallel slots in deterministic order
- `event.prefetchIntent` - For link navigations, whether the clicked link requested full prefetching (`full`), used automatic prefetching (`auto`), or did not request prefetching (`none`). For navigations with no associated link (programmatic `router.push()` / `router.replace()` , or browser back/forward) this is `null` , since no link prefetch intent applies

Route entries use filesystem-style patterns, so a navigation away from `/blog/hello` may report `/blog/[slug]` .

Hook errors are isolated and do not affect navigation or other hooks.

Performance considerations

Keep instrumentation code lightweight.

Next.js monitors initialization time in development and will log warnings if it takes longer than 16ms, which could impact smooth page loading.

Execution timing

The `instrumentation-client.js` file executes at a specific point in the application lifecycle:

1. **After** the HTML document is loaded
2. **Before** React hydration begins
3. **Before** user interactions are possible

This timing makes it ideal for setting up error tracking, analytics, and performance monitoring that needs to capture early application lifecycle events.

Only synchronous, top-level code is guaranteed to complete before hydration. Asynchronous work started here (a `Promise`, `import()`, or top-level `await`) is not awaited and may resolve after hydration has begun, so treat it as fire-and-forget. When something must be in place before your components run, use one of the synchronous patterns in Polyfills.

See also

`next.config.js` plugins (for example, wrappers like `withSentry`) can register their own client instrumentation module via the `instrumentationClientInject` option. Injected modules run before this file, in array order, and may export the same router transition start hook. Application code should continue to use this file convention directly.

Examples

Error tracking

Initialize error tracking before React starts and add navigation breadcrumbs for better debugging context.

```
// Archivo: instrumentation-client.ts switcher
import Monitor from './lib/monitoring'

Monitor.initialize()

export function onRouterTransitionStart(url: string) {
  Monitor.pushEvent({
    message: `Navigation to ${url}`,
    category: 'navigation',
  })
}
```

```
// Archivo: instrumentation-client.js switcher
import Monitor from './lib/monitoring'

Monitor.initialize()

export function onRouterTransitionStart(url) {
  Monitor.pushEvent({
    message: `Navigation to ${url}`,
    category: 'navigation',
  })
}
```

Analytics tracking

Initialize analytics and track navigation events with detailed metadata for user behavior analysis.

```
// Archivo: instrumentation-client.ts switcher
import { analytics } from './lib/analytics'

analytics.init()

export function onRouterTransitionStart(url: string, navigationType: string) {
  analytics.track('page_navigation', {
    url,
    type: navigationType,
    timestamp: Date.now(),
  })
}
```

```
// Archivo: instrumentation-client.js switcher
import { analytics } from './lib/analytics'

analytics.init()

export function onRouterTransitionStart(url, navigationType) {
  analytics.track('page_navigation', {
    url,
    type: navigationType,
    timestamp: Date.now(),
  })
}
```

Performance monitoring

Track Time to Interactive and navigation performance using the Performance Observer API and performance marks.

```
// Archivo: instrumentation-client.ts switcher
const startTime = performance.now()

const observer = new PerformanceObserver(
  (list: PerformanceObserverEntryList) => {
    for (const entry of list.getEntries()) {
      if (entry instanceof PerformanceNavigationTiming) {
        console.log('Time to Interactive:', entry.loadEventEnd - startTime)
      }
    }
  }
)

observer.observe({ entryTypes: ['navigation'] })

export function onRouterTransitionStart(url: string) {
  performance.mark(`nav-start-${url}`)
}
```

```
// Archivo: instrumentation-client.js switcher
const startTime = performance.now()

const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry instanceof PerformanceNavigationTiming) {
      console.log('Time to Interactive:', entry.loadEventEnd - startTime)
    }
  }
})

observer.observe({ entryTypes: ['navigation'] })

export function onRouterTransitionStart(url) {
  performance.mark(`nav-start-${url}`)
}
```

Polyfills

As noted in Execution timing, only synchronous, top-level code here runs before hydration, so a conditional `import()` or top-level `await` may resolve after hydration has already started.

To guarantee a polyfill is applied before your components run, statically import it and apply it synchronously after feature detection. Because the import is static, the polyfill ships to every visitor:

```
// Archivo: instrumentation-client.ts
import ResizeObserverPolyfill from './lib/polyfills/resize-observer'

if (!window.ResizeObserver) {
  window.ResizeObserver = ResizeObserverPolyfill
}
```

Avoid loading a polyfill with a conditional `import()` here. The import is fire-and-forget, so the polyfill may be applied after hydration has already begun, which can be too late for your components:

```
// Archivo: instrumentation-client.ts
// Avoid: the dynamic import is fire-and-forget, so `ResizeObserver`
// may still be undefined when your components run.
if (!window.ResizeObserver) {
  import('./lib/polyfills/resize-observer').then((mod) => {
    window.ResizeObserver = mod.default
  })
}
```

For anything loaded on demand, prefer polyfilling in the code that uses the feature. See Custom Polyfills for that and other strategies.

Next.js already injects a baseline of widely used polyfills (such as `fetch`, `URL`, and `Object.assign`) for the browsers that need them, so you only need to add polyfills for features outside that baseline.

Version history

Version	Changes
`v16.3.0`	Experimental router transition start event introduced
`v15.3`	`instrumentation-client` introduced

instrumentation.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/instrumentation
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the instrumentation.js file.*

The `instrumentation.js|ts` file is used to integrate observability tools into your application, allowing you to track the performance and behavior, and to debug issues in production.

To use it, place the file in the **root** of your application or inside a `src` folder if using one.

Exports

`register` (optional)

The file exports a `register` function that is called **once** when a new Next.js server instance is initiated, and must complete before the server is ready to handle requests. `register` can be an async function.

```
// Archivo: instrumentation.ts switcher
import { registerOTel } from '@vercel/otel'

export function register() {
  registerOTel('next-app')
}
```

```
// Archivo: instrumentation.js switcher
import { registerOTel } from '@vercel/otel'

export function register() {
  registerOTel('next-app')
}
```

`onRequestError` (optional)

You can optionally export an `onRequestError` function to track **server** errors to any custom observability provider.

- If you're running any async tasks in `onRequestError`, make sure they're awaited. `onRequestError` will be triggered when the Next.js server captures the error.
- The `error` instance might not be the original error instance thrown, as it may be processed by React if encountered during Server Components rendering. If this happens, you can use `digest` property on an error to identify the actual error type.

```
// Archivo: instrumentation.ts switcher
import { type Instrumentation } from 'next'

export const onRequestError: Instrumentation.onRequestError = async (
  err,
  request,
  context
) ⇒ {
  const message = err instanceof Error ? err.message : String(err)
  const digest =
    typeof err ⩵ 'object' && err ≠ null && 'digest' in err
      ? String(err.digest)
      : undefined

  await fetch('https:// … /report-error', {
    method: 'POST',
    body: JSON.stringify({
      message,
      digest,
      request,
      context,
    }),
    headers: {
      'Content-Type': 'application/json',
    },
  })
}
```

```
// Archivo: instrumentation.js switcher
export async function onRequestError(err, request, context) {
  const message = err instanceof Error ? err.message : String(err)
  const digest =
    typeof err === 'object' && err !== null && 'digest' in err
      ? String(err.digest)
      : undefined

  await fetch('https:// ... /report-error', {
    method: 'POST',
    body: JSON.stringify({
      message,
      digest,
      request,
      context,
    }),
    headers: {
      'Content-Type': 'application/json',
    },
  })
}
```

Parameters

The function accepts three parameters: `error`, `request`, and `context`.

```
// Archivo: Types
export function onRequestError(
  error: unknown,
  request: {
    path: string // resource path, e.g. /blog?name=foo
    method: string // request method. e.g. GET, POST, etc
    headers: { [key: string]: string | string[] }
  },
  context: {
    routerKind: 'Pages Router' | 'App Router' // the router type
    routePath: string // the route file path, e.g. /app/blog/[dynamic]
    routeType: 'render' | 'route' | 'action' | 'proxy' // the context in which the error occurred
    renderSource:
      | 'react-server-components'
      | 'react-server-components-payload'
      | 'server-rendering'
    revalidateReason: 'on-demand' | 'stale' | undefined // undefined is a normal request without revalidation
    renderType: 'dynamic' | 'dynamic-resume' // 'dynamic-resume' for PPR
  }
): void | Promise<void>
```

- `error`: The caught value is typed as `unknown`. Narrow it before reading properties like `message` or `digest`.
- `request`: Read-only request information associated with the error.
- `context`: The context in which the error occurred. This can be the type of router (App or Pages Router), and/or (Server Components (`'render'`), Route Handlers (`'route'`), Server Actions (`'action'`), or Proxy (`'proxy'`)).

Specifying the runtime

The `instrumentation.js` file works in both the Node.js and Edge runtime, however, you can use `process.env.NEXT_RUNTIME` to target a specific runtime.

```
// Archivo: instrumentation.js
export function register() {
  if (process.env.NEXT_RUNTIME === 'edge') {
    return require('./register.edge')
  } else {
    return require('./register.node')
  }
}

export function onRequestError() {
  if (process.env.NEXT_RUNTIME === 'edge') {
    return require('./on-request-error.edge')
  } else {
    return require('./on-request-error.node')
  }
}
```

Version History

Version	Changes
`v15.0.0`	`onRequestError` introduced, `instrumentation` stable
`v14.0.4`	Turbopack support for `instrumentation`
`v13.2.0`	`instrumentation` introduced as an experimental feature

Intercepting Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/intercepting-routes
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Use intercepting routes to load a new route within the current layout while masking the browser URL, useful for advanced routing patterns such as modals.*

Intercepting routes allows you to load a route from another part of your application within the current layout. This routing paradigm can be useful when you want to display the content of a route without the user switching to a different context.

For example, when clicking on a photo in a feed, you can display the photo in a modal, overlaying the feed. In this case, Next.js intercepts the `/photo/123` route, masks the URL, and overlays it over `/feed`.

Intercepting routes soft navigation

However, when navigating to the photo by clicking a shareable URL or by refreshing the page, the entire photo page should render instead of the modal. No route interception should occur.

Intercepting routes hard navigation

Convention

Intercepting routes can be defined with the `(..)` convention, which is similar to relative path convention `../` but for route segments.

You can use:

- `(.)` to match segments on the **same level**
- `(..)` to match segments **one level above**
- `(..)(..)` to match segments **two levels above**
- `(...)` to match segments from the **root** `app` directory

For example, you can intercept the `photo` segment from within the `feed` segment by creating a `(..)photo` directory.

> **Good to know:** The `(..)` convention is based on *route segments*, not the file-system. For example, it does not consider `@slot` folders in Parallel Routes.

Examples

Modals

Intercepting Routes can be used together with Parallel Routes to create modals. This allows you to solve common challenges when building modals, such as:

- Making the modal content **shareable through a URL**.
- **Preserving context** when the page is refreshed, instead of closing the modal.
- **Closing the modal on backwards navigation** rather than going to the previous route.
- **Reopening the modal on forwards navigation**.

Consider the following UI pattern, where a user can open a photo modal from a gallery using client-side navigation, or navigate to the photo page directly from a shareable URL:



In the above example, the path to the `photo` segment can use the `(..)` matcher since `@modal` is a slot and **not** a segment. This means that the `photo` route is only one segment level higher, despite being two file-system levels higher.

See the Parallel Routes documentation for a step-by-step example, or see our image gallery example.

> **Good to know:**
>
> - Other examples could include opening a login modal in a top navbar while also having a dedicated `/login` page, or opening a shopping cart in a side modal.

---

layout.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/layout
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the layout.js file.*

The `layout` file is used to define a layout in your Next.js application.

```
// Archivo: app/dashboard/layout.tsx switcher
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return <section>{children}</section>
}
```

```
// Archivo: app/dashboard/layout.js switcher
export default function DashboardLayout({ children }) {
  return <section>{children}</section>
}
```

In the component hierarchy, `layout.js` is the outermost component in a route segment. It wraps `template.js`, `error.js`, `loading.js`, `not-found.js`, and `page.js`.

A **root layout** is the top-most layout in the root `app` directory. It is used to define the `<html>` and `<body>` tags and other globally shared UI.

```
// Archivo: app/layout.tsx switcher
export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

Reference

Props

`children` (required)

Layout components should accept and use a `children` prop. During rendering, `children` will be populated with the route segments the layout is wrapping. These will primarily be the component of a child Layout (if it exists) or Page, but could also be other special files like Loading or Error when applicable.

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters object from the root segment down to that layout.

```
// Archivo: app/dashboard/[team]/layout.tsx switcher
export default async function Layout({
  children,
  params,
}: {
  children: React.ReactNode
  params: Promise<{ team: string }>
}) {
  const { team } = await params
}
```

```
// Archivo: app/dashboard/[team]/layout.js switcher
export default async function Layout({ children, params }) {
  const { team } = await params
}
```

Example Route	URL	`params`
`app/dashboard/[team]/layout.js`	`/dashboard/1`	`Promise<{ team: '1' }>`
`app/shop/[tag]/[item]/layout.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`
`app/blog/[...slug]/layout.js`	`/blog/1/2`	`Promise<{ slug: ['1', '2'] }>`

- Since the `params` prop is a promise. You must use `async/await` or React's `use` function to access the values.
  - In version 14 and earlier, `params` was a synchronous prop. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.

Layout Props Helper

You can type layouts with `LayoutProps` to get a strongly typed `params` and named slots inferred from your directory structure. `LayoutProps` is a globally available helper.

```
// Archivo: app/dashboard/layout.tsx
export default function Layout(props: LayoutProps<'/dashboard'>) {
  return (
    <section>
      {props.children}


    </section>
  )
}
```

> **Good to know**:
>
> - Types are generated during `next dev`, `next build` or `next typegen`.
> - After type generation, the `LayoutProps` helper is globally available. It doesn't need to be imported.

Root Layout

The `app` directory **must** include a **root layout**, which is the top-most layout in the root `app` directory. Typically, the root layout is `app/layout.js`.

```
// Archivo: app/layout.tsx switcher
export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html>
      <body>{children}</body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
export default function RootLayout({ children }) {
  return (
    <html>
      <body>{children}</body>
    </html>
  )
}
```

- The root layout **must** define `<html>` and `<body>` tags.

- You should **not** manually add `<head>` tags such as `<title>` and `<meta>` to root layouts. Instead, you should use the Metadata API which automatically handles advanced requirements such as streaming and de-duplicating `<head>` elements.

- You can create **multiple root layouts**. Any layout without a `layout.js` above it is a root layout. Two common approaches:
  - Using route groups like `app/(shop)/layout.js` and `app/(marketing)/layout.js`
  - Omitting `app/layout.js` so layouts in subdirectories like `app/dashboard/layout.js` and `app/blog/layout.js` each become root layouts for their respective directories.
  - Navigating **across multiple root layouts** will cause a **full page load** (as opposed to a client-side navigation).

- The root layout can be under a **dynamic segment**, for example when implementing internationalization with `app/[lang]/layout.js`. Dynamic segments before the root layout are **root parameters** and can be read from any Server Component with `next/root-params`.

Caveats

Request Object

Layouts are cached in the client during navigation to avoid unnecessary server requests.

Layouts do not rerender. They can be cached and reused to avoid unnecessary computation when navigating between pages. By restricting layouts from accessing the raw request, Next.js can prevent the execution of potentially slow or expensive user code within the layout, which could negatively impact performance.

To access the request object, you can use `headers` and `cookies` APIs in Server Components and Functions.

```
// Archivo: app/shop/layout.tsx switcher
import { cookies } from 'next/headers'

export default async function Layout({ children }) {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

```
// Archivo: app/shop/layout.js switcher
import { cookies } from 'next/headers'

export default async function Layout({ children }) {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

Query params

Layouts do not rerender on navigation, so they cannot access search params which would otherwise become stale.

To access updated query parameters, you can use the Page `searchParams` prop, or read them inside a Client Component using the `useSearchParams` hook. Since Client Components re-render on navigation, they have access to the latest query parameters.

```
// Archivo: app/ui/search.tsx switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function Search() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  return '...'
}
```

```
// Archivo: app/ui/search.js switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function Search() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  return '...'
}
```

```
// Archivo: app/shop/layout.tsx switcher
import Search from '@/app/ui/search'

export default function Layout({ children }) {
  return (
    <>
      <Search />
      {children}
    </>
  )
}
```

```
// Archivo: app/shop/layout.js switcher
import Search from '@/app/ui/search'

export default function Layout({ children }) {
  return (
    <>
      <Search />
      {children}
    </>
  )
}
```

Pathname

Layouts do not re-render on navigation, so they do not access pathname which would otherwise become stale.

To access the current pathname, you can read it inside a Client Component using the `usePathname` hook. Since Client Components re-render during navigation, they have access to the latest pathname.

```
// Archivo: app/ui/breadcrumbs.tsx switcher
'use client'

import { usePathname } from 'next/navigation'

// Simplified breadcrumbs logic
export default function Breadcrumbs() {
  const pathname = usePathname()
  const segments = pathname.split('/')

  return (
    <nav>
      {segments.map((segment, index) => (
        <span key={index}>
          {' > '}
          {segment}
        </span>
      ))}
    </nav>
  )
}
```

```
// Archivo: app/ui/breadcrumbs.js switcher
'use client'

import { usePathname } from 'next/navigation'

// Simplified breadcrumbs logic
export default function Breadcrumbs() {
  const pathname = usePathname()
  const segments = pathname.split('/')

  return (
    <nav>
      {segments.map((segment, index) => (
        <span key={index}>
          {' > '}
          {segment}
        </span>
      ))}
    </nav>
  )
}
```

```
// Archivo: app/docs/layout.tsx switcher
import { Breadcrumbs } from '@/app/ui/Breadcrumbs'

export default function Layout({ children }) {
  return (
    <>
      <Breadcrumbs />
      <main>{children}</main>
    </>
  )
}
```

```
// Archivo: app/docs/layout.js switcher
import { Breadcrumbs } from '@/app/ui/Breadcrumbs'

export default function Layout({ children }) {
  return (
    <>
      <Breadcrumbs />
      <main>{children}</main>
    </>
  )
}
```

Interaction with `loading.js`

Because `loading.js` sits below `layout.js` in the component hierarchy, it cannot show a fallback for uncached or runtime data access in the layout itself, such as calling `cookies()`, `headers()`, or making uncached fetches.

The behavior depends on whether Cache Components is enabled:

- **Without Cache Components:** The navigation will block until the layout finishes rendering, and the `loading.js` fallback will not be shown.
- **With Cache Components:** `loading.js` is treated as a regular ``````
  {children}
  </>) }

```jsx
// Archivo: app/dashboard/layout.js switcher highlight={8-10}
import { Suspense } from 'react'
import { NavSkeleton } from './nav-skeleton'
import { DashboardNav } from './dashboard-nav'

export default function Layout({ children }) {
  return (
    <>
      ``<Suspense>``}>
        <DashboardNav />
      `<Suspense>`
      <main>{children}</main>
    </>
  )
}
```

Fetching Data

Layouts cannot pass data to their `children`. However, you can fetch the same data in a route more than once, and use React `cache` to dedupe the requests without affecting performance.

Alternatively, when using `fetch` in Next.js, requests are automatically deduped.

```
// Archivo: app/lib/data.ts switcher
export async function getUser(id: string) {
  const res = await fetch(`https:// ... /users/${id}`)
  return res.json()
}
```

```
// Archivo: app/dashboard/layout.tsx switcher
import { getUser } from '@/app/lib/data'
import { UserName } from '@/app/ui/user-name'

export default async function Layout({ children }) {
  const user = await getUser('1')

  return (
    <>
      <nav>

        <UserName user={user.name} />
      </nav>
      {children}
    </>
  )
}
```

```
// Archivo: app/dashboard/layout.js switcher
import { getUser } from '@/app/lib/data'
import { UserName } from '@/app/ui/user-name'

export default async function Layout({ children }) {
  const user = await getUser('1')

  return (
    <>
      <nav>

        <UserName user={user.name} />
      </nav>
      {children}
    </>
  )
}
```

```
// Archivo: app/dashboard/page.tsx switcher
import { getUser } from '@/app/lib/data'
import { UserName } from '@/app/ui/user-name'

export default async function Page() {
  const user = await getUser('1')

  return (
    <div>
      <h1>Welcome {user.name}</h1>
    </div>
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
import { getUser } from '@/app/lib/data'
import { UserName } from '@/app/ui/user-name'

export default async function Page() {
  const user = await getUser('1')

  return (
    <div>
      <h1>Welcome {user.name}</h1>
    </div>
  )
}
```

Accessing child segments

Layouts do not have access to the route segments below itself. To access all route segments, you can use `useSelectedLayoutSegment` or `useSelectedLayoutSegments` in a Client Component.

```
// Archivo: app/ui/nav-link.tsx switcher
'use client'

import Link from 'next/link'
import { useSelectedLayoutSegment } from 'next/navigation'

export default function NavLink({
  slug,
  children,
}: {
  slug: string
  children: React.ReactNode
}) {
  const segment = useSelectedLayoutSegment()
  const isActive = slug === segment

  return (
    <Link
      href={`/blog/${slug}`}
      // Change style depending on whether the link is active
      style={{ fontWeight: isActive ? 'bold' : 'normal' }}
    >
      {children}
    </Link>
  )
}
```

```
// Archivo: app/ui/nav-link.js switcher
'use client'

import Link from 'next/link'
import { useSelectedLayoutSegment } from 'next/navigation'

export default function NavLinks({ slug, children }) {
  const segment = useSelectedLayoutSegment()
  const isActive = slug === segment

  return (
    <Link
      href={`/blog/${slug}`}
      style={{ fontWeight: isActive ? 'bold' : 'normal' }}
    >
      {children}
    </Link>
  )
}
```

```
// Archivo: app/blog/layout.tsx switcher
import { NavLink } from './nav-link'
import getPosts from './get-posts'

export default async function Layout({
  children,
}: {
  children: React.ReactNode
}) {
  const featuredPosts = await getPosts()
  return (
    <div>
      {featuredPosts.map((post) => (
        <div key={post.id}>
          <NavLink slug={post.slug}>{post.title}</NavLink>
        </div>
      ))}
      <div>{children}</div>
    </div>
  )
}
```

```
// Archivo: app/blog/layout.js switcher
import { NavLink } from './nav-link'
import getPosts from './get-posts'

export default async function Layout({ children }) {
  const featuredPosts = await getPosts()
  return (
    <div>
      {featuredPosts.map((post) => (
        <div key={post.id}>
          <NavLink slug={post.slug}>{post.title}</NavLink>
        </div>
      ))}
      <div>{children}</div>
    </div>
  )
}
```

Examples

Metadata

You can modify the `<head>` HTML elements such as `title` and `meta` using the `metadata` object or `generateMetadata` function.

```tsx
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Next.js',
}

export default function Layout({ children }: { children: React.ReactNode }) {
  return '...'
}
```

```jsx
// Archivo: app/layout.js switcher
export const metadata = {
  title: 'Next.js',
}

export default function Layout({ children }) {
  return '...'
}
```

> **Good to know**: You should **not** manually add `<head>` tags such as `<title>` and `<meta>` to root layouts. Instead, use the Metadata APIs which automatically handles advanced requirements such as streaming and de-duplicating `<head>` elements.

Active Nav Links

You can use the `usePathname` hook to determine if a nav link is active.

Since `usePathname` is a client hook, you need to extract the nav links into a Client Component, which can be imported into your layout:

```tsx
// Archivo: app/ui/nav-links.tsx switcher
'use client'

import { usePathname } from 'next/navigation'
import Link from 'next/link'

export function NavLinks() {
  const pathname = usePathname()

  return (
    <nav>
      <Link className={`link ${pathname === '/' ? 'active' : ''}`} href="/">
        Home
      </Link>

      <Link
        className={`link ${pathname === '/about' ? 'active' : ''}`}
        href="/about"
      >
        About
      </Link>
    </nav>
  )
}
```

```
// Archivo: app/ui/nav-links.js switcher
'use client'

import { usePathname } from 'next/navigation'
import Link from 'next/link'

export function Links() {
  const pathname = usePathname()

  return (
    <nav>
      <Link className={`link ${pathname === '/' ? 'active' : ''}`} href="/">
        Home
      </Link>

      <Link
        className={`link ${pathname === '/about' ? 'active' : ''}`}
        href="/about"
      >
        About
      </Link>
    </nav>
  )
}
```

```
// Archivo: app/layout.tsx switcher
import { NavLinks } from '@/app/ui/nav-links'

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>
        <NavLinks />
        <main>{children}</main>
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { NavLinks } from '@/app/ui/nav-links'

export default function Layout({ children }) {
  return (
    <html lang="en">
      <body>
        <NavLinks />
        <main>{children}</main>
      </body>
    </html>
  )
}
```

Displaying content based on `params`

Using dynamic route segments, you can display or fetch specific content based on the `params` prop.

```tsx
// Archivo: app/dashboard/layout.tsx switcher
export default async function DashboardLayout({
  children,
  params,
}: {
  children: React.ReactNode
  params: Promise<{ team: string }>
}) {
  const { team } = await params

  return (
    <section>
      <header>
        <h1>Welcome to {team}'s Dashboard</h1>
      </header>
      <main>{children}</main>
    </section>
  )
}
```

```jsx
// Archivo: app/dashboard/layout.js switcher
export default async function DashboardLayout({ children, params }) {
  const { team } = await params

  return (
    <section>
      <header>
        <h1>Welcome to {team}'s Dashboard</h1>
      </header>
      <main>{children}</main>
    </section>
  )
}
```

Reading `params` in Client Components

To use `params` in a Client Component (which cannot be `async`), you can use React's `use` function to read the promise:

```tsx
// Archivo: app/page.tsx switcher
'use client'

import { use } from 'react'

export default function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = use(params)
}
```

```jsx
// Archivo: app/page.js switcher
'use client'

import { use } from 'react'

export default function Page({ params }) {
  const { slug } = use(params)
}
```

Version History

Version	Changes
`v15.0.0-RC`	`params` is now a promise. A codemod is available.
`v13.0.0`	`layout` introduced.

loading.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/loading
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the loading.js file.*

The special file `loading.js` helps you create meaningful Loading UI with React Suspense. With this convention, you can show an instant loading state from the server while the content of a route segment streams in. The new content is automatically swapped in once complete.

Loading UI

```
// Archivo: app/feed/loading.tsx switcher
export default function Loading() {
  // Or a custom loading skeleton component
  return <p>Loading ... </p>
}
```

```
// Archivo: app/feed/loading.js switcher
export default function Loading() {
  // Or a custom loading skeleton component
  return <p>Loading ... </p>
}
```

Inside the `loading.js` file, you can add any light-weight loading UI. You may find it helpful to use the React Developer Tools to manually toggle Suspense boundaries.

By default, this file is a Server Component - but can also be used as a Client Component through the `"use client"` directive.

Reference

Parameters

Loading UI components do not accept any parameters.

Behavior

Navigation

- The Fallback UI is prefetched, making navigation immediate unless prefetching hasn't completed.
- Navigation is interruptible, meaning changing routes does not need to wait for the content of the route to fully load before navigating to another route.
- Shared layouts remain interactive while new route segments load.

Instant Loading States

An instant loading state is fallback UI that is shown immediately upon navigation. You can prerender loading indicators such as skeletons and spinners, or a small but meaningful part of future screens such as a cover photo, title, etc. This helps users understand the app is responding and provides a better user experience.

Create a loading state by adding a `loading.js` file inside a folder.

loading.js special file

```
// Archivo: app/dashboard/loading.tsx switcher
export default function Loading() {
  // You can add any UI inside Loading, including a Skeleton.
  return <LoadingSkeleton />
}
```

```
// Archivo: app/dashboard/loading.js switcher
export default function Loading() {
  // You can add any UI inside Loading, including a Skeleton.
  return <LoadingSkeleton />
}
```

In the same folder, `loading.js` will be nested inside `layout.js`. It will automatically wrap the `page.js` file and any children below in a ``` ``)}

```


```jsx
// Archivo: app/dashboard/page.js switcher
import { Suspense } from 'react'
import { PostFeed, Weather } from './Components'

export default function Posts() {
  return (
    <section>
      ``<Suspense>``
      ``<Suspense>``
    </section>
  )
}
```

By using Suspense, you get the benefits of:

1. **Streaming Server Rendering** - Progressively rendering HTML from the server to the client.
2. **Selective Hydration** - React prioritizes what components to make interactive first based on user interaction.

For more Suspense examples and use cases, please see the React Documentation.

Version History

Version	Changes
`v13.0.0`	`loading` introduced.

mdx-components.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/mdx-components
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the mdx-components.js file.*
```

The `mdx-components.js|tsx` file is **required** to use `@next/mdx` with App Router and will not work without it. Additionally, you can use it to customize styles.

Use the file `mdx-components.tsx` (or `.js`) in the root of your project to define MDX Components. For example, at the same level as `pages` or `app`, or inside `src` if applicable.

```
// Archivo: mdx-components.tsx switcher
import type { MDXComponents } from 'mdx/types'

const components: MDXComponents = {}

export function useMDXComponents(): MDXComponents {
  return components
}
```

```
// Archivo: mdx-components.js switcher
const components = {}

export function useMDXComponents() {
  return components
}
```

Exports

`useMDXComponents` function

The file must export a single function named `useMDXComponents`. This function does not accept any arguments.

```
// Archivo: mdx-components.tsx switcher
import type { MDXComponents } from 'mdx/types'

const components: MDXComponents = {}

export function useMDXComponents(): MDXComponents {
  return components
}
```

```
// Archivo: mdx-components.js switcher
const components = {}

export function useMDXComponents() {
  return components
}
```

Version History

Version	Changes
`v13.1.2`	MDX Components added

middleware.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/middleware
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the middleware.js file (deprecated, renamed to proxy.js).*

The `middleware.js` file convention has been **deprecated** in Next.js 16 and renamed to `proxy.js`.

All functionality remains the same — only the file and export names have changed. See the Migration to Proxy section for details on how to migrate.

You can automatically migrate by running:

```
npx @next/codemod@canary middleware-to-proxy .
```

For full documentation, see the `proxy.js` API reference.

not-found.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/not-found
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the not-found.js file.*

Next.js provides two conventions to handle not found cases:

- `not-found.js`: Used when you call the `notFound` function in a route segment.
- `global-not-found.js`: Used to define a global 404 page for unmatched routes across your entire app. This is handled at the routing level and doesn't depend on rendering a layout or page.

`not-found.js`

The **not-found** file is used to render UI when the `notFound` function is thrown within a route segment. Along with serving a custom UI, Next.js will return a `200` HTTP status code for streamed responses, and `404` for non-streamed responses (see Status Codes for details about SEO).

```
// Archivo: app/not-found.tsx switcher
import Link from 'next/link'

export default function NotFound() {
  return (
    <div>
      <h2>Not Found</h2>
      <p>Could not find requested resource</p>
      [Return Home](/)
    </div>
  )
}
```

```
// Archivo: app/blog/not-found.js switcher
import Link from 'next/link'

export default function NotFound() {
  return (
    <div>
      <h2>Not Found</h2>
      <p>Could not find requested resource</p>
      [Return Home](/)
    </div>
  )
}
```

In the component hierarchy, `not-found.js` renders between `loading.js` and `page.js`. It is wrapped by the `<Suspense>` boundary from `loading.js` and the error boundary from `error.js` in the same segment.

> **Good to know**: The default not found UI follows the operating system's color scheme via `prefers-color-scheme` and does not read an app-level theme (such as a class or `data-theme` attribute on `<html>`). Because it renders inside your root layout, the quickest way to match an explicit theme is to add a higher-specificity rule pair in your global stylesheet, scoped to your theme selector — for example `html[data-theme='light'] body` and `html[data-theme='dark'] body`. For full control over the markup, provide your own `not-found.js`.

`global-not-found.js` (experimental)

The `global-not-found.js` file lets you define a 404 page for your entire application. Unlike `not-found.js`, which works at the route level, this is used when a requested URL doesn't match any route at all. Next.js **skips rendering** and directly returns this global page.

The `global-not-found.js` file bypasses your app's normal rendering, which means you'll need to import any global styles, fonts, or other dependencies that your 404 page requires. This includes your theme: because `global-not-found.js` bypasses your layout, the OS color scheme is the only signal the default UI sees, so apply your theme (class or attribute) inside this file.

> **Good to know**: A smaller version of your global styles, and a simpler font family could improve performance of this page.

`global-not-found.js` is useful when you can't build a 404 page using a combination of `layout.js` and `not-found.js`. This can happen in two cases:

- Your app has multiple root layouts (e.g. `app/(admin)/layout.tsx` and `app/(shop)/layout.tsx`), so there's no single layout to compose a global 404 from.
- Your root layout is defined using top-level dynamic segments (e.g. `app/[country]/layout.tsx`), which makes composing a consistent 404 page harder.

To enable it, add the `globalNotFound` flag in `next.config.ts`:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    globalNotFound: true,
  },
}

export default nextConfig
```

Then, create a file in the root of the `app` directory: `app/global-not-found.js`:

```tsx
// Archivo: app/global-not-found.tsx switcher
// Import global styles and fonts
import './globals.css'
import { Inter } from 'next/font/google'
import type { Metadata } from 'next'

const inter = Inter({ subsets: ['latin'] })

export const metadata: Metadata = {
  title: '404 - Page Not Found',
  description: 'The page you are looking for does not exist.',
}

export default function GlobalNotFound() {
  return (
    <html lang="en" className={inter.className}>
      <body>
        <h1>404 - Page Not Found</h1>
        <p>This page does not exist.</p>
      </body>
    </html>
  )
}
```

```tsx
// Archivo: app/global-not-found.js switcher
// Import global styles and fonts
import './globals.css'
import { Inter } from 'next/font/google'

const inter = Inter({ subsets: ['latin'] })

export const metadata = {
  title: '404 - Page Not Found',
  description: 'The page you are looking for does not exist.',
}

export default function GlobalNotFound() {
  return (
    <html lang="en" className={inter.className}>
      <body>
        <h1>404 - Page Not Found</h1>
        <p>This page does not exist.</p>
      </body>
    </html>
  )
}
```

Unlike `not-found.js`, this file must return a full HTML document, including `<html>` and `<body>` tags.

Reference

Props

`not-found.js` or `global-not-found.js` components do not accept any props.

> **Good to know**: In addition to catching expected `notFound()` errors, the root `app/not-found.js` and `app/global-not-found.js` files handle any unmatched URLs for your whole application. This means users that visit a URL that is not handled by your app will be shown the exported UI.

Examples

Data Fetching

By default, `not-found` is a Server Component. You can mark it as `async` to fetch and display data:

```
// Archivo: app/not-found.tsx switcher
import Link from 'next/link'
import { headers } from 'next/headers'

export default async function NotFound() {
  const headersList = await headers()
  const domain = headersList.get('host')
  const data = await getSiteData(domain)
  return (
    <div>
      <h2>Not Found: {data.name}</h2>
      <p>Could not find requested resource</p>
      <p>
        View [all posts](/blog)
      </p>
    </div>
  )
}
```

```
// Archivo: app/not-found.jsx switcher
import Link from 'next/link'
import { headers } from 'next/headers'

export default async function NotFound() {
  const headersList = await headers()
  const domain = headersList.get('host')
  const data = await getSiteData(domain)
  return (
    <div>
      <h2>Not Found: {data.name}</h2>
      <p>Could not find requested resource</p>
      <p>
        View [all posts](/blog)
      </p>
    </div>
  )
}
```

If you need to use Client Component hooks like `usePathname` to display content based on the path, you must fetch data on the client-side instead.

Metadata

For `global-not-found.js`, you can export a `metadata` object or a `generateMetadata` function to customize the `<title>`, `<meta>`, and other head tags for your 404 page:

> **Good to know**: Next.js automatically injects `<meta name="robots" content="noindex" />` for pages that return a 404 status code, including `global-not-found.js` pages.

```tsx
// Archivo: app/global-not-found.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Not Found',
  description: 'The page you are looking for does not exist.',
}

export default function GlobalNotFound() {
  return (
    <html lang="en">
      <body>
        <div>
          <h1>Not Found</h1>
          <p>The page you are looking for does not exist.</p>
        </div>
      </body>
    </html>
  )
}
```

```jsx
// Archivo: app/global-not-found.js switcher
export const metadata = {
  title: 'Not Found',
  description: 'The page you are looking for does not exist.',
}

export default function GlobalNotFound() {
  return (
    <html lang="en">
      <body>
        <div>
          <h1>Not Found</h1>
          <p>The page you are looking for does not exist.</p>
        </div>
      </body>
    </html>
  )
}
```

Version History

Version	Changes
`v15.4.0`	`global-not-found.js` introduced (experimental).
`v13.3.0`	Root `app/not-found` handles global unmatched URLs.
`v13.0.0`	`not-found` introduced.

page.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/page
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the page.js file.*

The `page` file allows you to define UI that is **unique** to a route. You can create a page by default exporting a component from the file:

```tsx
// Archivo: app/blog/[slug]/page.tsx switcher
export default function Page({
  params,
  searchParams,
}: {
  params: Promise<{ slug: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  return <h1>My Page</h1>
}
```

```jsx
// Archivo: app/blog/[slug]/page.js switcher
export default function Page({ params, searchParams }) {
  return <h1>My Page</h1>
}
```

Good to know

- The `.js`, `.jsx`, or `.tsx` file extensions can be used for `page`.
- A `page` is always the **leaf** of the route subtree.
- A `page` file is required to make a route segment **publicly accessible**.
- Pages are Server Components by default, but can be set to a Client Component.
- In the component hierarchy, `page.js` is the innermost file convention. It is wrapped by `loading.js` (Suspense boundary), `error.js` (error boundary), `template.js`, and `layout.js` in the same segment.

Reference

Props

`params` (optional)

A promise that resolves to an object containing the dynamic route parameters from the root segment down to that page.

```tsx
// Archivo: app/shop/[slug]/page.tsx switcher
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
}
```

```jsx
// Archivo: app/shop/[slug]/page.js switcher
export default async function Page({ params }) {
  const { slug } = await params
}
```

Example Route	URL	`params`
`app/shop/[slug]/page.js`	`/shop/1`	`Promise<{ slug: '1' }>`
`app/shop/[category]/[item]/page.js`	`/shop/1/2`	`Promise<{ category: '1', item: '2' }>`
`app/shop/[...slug]/page.js`	`/shop/1/2`	`Promise<{ slug: ['1', '2'] }>`

- Since the `params` prop is a promise, you must use `async/await` or React's `use` function to access the values.
  - In version 14 and earlier, `params` was a synchronous prop. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.

`searchParams` (optional)

A promise that resolves to an object containing the search parameters of the current URL. For example:

```
// Archivo: app/shop/page.tsx switcher
export default async function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const filters = (await searchParams).filters
}
```

```
// Archivo: app/shop/page.js switcher
export default async function Page({ searchParams }) {
  const filters = (await searchParams).filters
}
```

Client Component **pages** can also access `searchParams` using React's `use` hook:

```
// Archivo: app/shop/page.tsx switcher
'use client'
import { use } from 'react'

export default function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const filters = use(searchParams).filters
}
```

```
// Archivo: app/page.jsx switcher
'use client'
import { use } from 'react'

export default function Page({ searchParams }) {
  const filters = use(searchParams).filters
}
```

Example URL	`searchParams`
`/shop?a=1`	`Promise<{ a: '1' }>`
`/shop?a=1&b=2`	`Promise<{ a: '1', b: '2' }>`
`/shop?a=1&a=2`	`Promise<{ a: ['1', '2'] }>`

- Since the `searchParams` prop is a promise. You must use `async/await` or React's `use` function to access the values.
  - In version 14 and earlier, `searchParams` was a synchronous prop. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.
- `searchParams` is a **Request-time API** whose values cannot be known ahead of time. Using it will opt the page into **dynamic rendering** at request time.
- With Cache Components, where you access `searchParams` in the component tree determines how much of the page can be prerendered. See Maximizing the static shell.
- `searchParams` is a plain JavaScript object, not a `URLSearchParams` instance.

Page Props Helper

You can type pages with `PageProps` to get strongly typed `params` and `searchParams` from the route literal. `PageProps` is a globally available helper.

```
// Archivo: app/blog/[slug]/page.tsx
export default async function Page(props: PageProps<'/blog/[slug]'>) {
  const { slug } = await props.params
  const query = await props.searchParams
  return <h1>Blog Post: {slug}</h1>
}
```

Good to know

- Using a literal route (e.g. `'/blog/[slug]'`) enables autocomplete and strict keys for `params`.
- Static routes resolve `params` to `{}`.
- Types are generated during `next dev`, `next build`, or with `next typegen`.
- After type generation, the `PageProps` helper is globally available. It doesn't need to be imported.

Examples

Displaying content based on `params`

Using dynamic route segments, you can display or fetch specific content for the page based on the `params` prop.

```
// Archivo: app/blog/[slug]/page.tsx switcher
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  return <h1>Blog Post: {slug}</h1>
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export default async function Page({ params }) {
  const { slug } = await params
  return <h1>Blog Post: {slug}</h1>
}
```

Handling filtering with `searchParams`

You can use the `searchParams` prop to handle filtering, pagination, or sorting based on the query string of the URL.

```
// Archivo: app/shop/page.tsx switcher
export default async function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const { page = '1', sort = 'asc', query = '' } = await searchParams

  return (
    <div>
      <h1>Product Listing</h1>
      <p>Search query: {query}</p>
      <p>Current page: {page}</p>
      <p>Sort order: {sort}</p>
    </div>
  )
}
```

```
// Archivo: app/shop/page.js switcher
export default async function Page({ searchParams }) {
  const { page = '1', sort = 'asc', query = '' } = await searchParams

  return (
    <div>
      <h1>Product Listing</h1>
      <p>Search query: {query}</p>
      <p>Current page: {page}</p>
      <p>Sort order: {sort}</p>
    </div>
  )
}
```

Reading `searchParams` and `params` in Client Components

To use `searchParams` and `params` in a Client Component (which cannot be `async`), you can use React's `use` function to read the promise:

```
// Archivo: app/page.tsx switcher
'use client'

import { use } from 'react'

export default function Page({
  params,
  searchParams,
}: {
  params: Promise<{ slug: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const { slug } = use(params)
  const { query } = use(searchParams)
}
```

```
// Archivo: app/page.js switcher
'use client'

import { use } from 'react'

export default function Page({ params, searchParams }) {
  const { slug } = use(params)
  const { query } = use(searchParams)
}
```

Version History

Version	Changes
`v15.0.0-RC`	`params` and `searchParams` are now promises. A codemod is available.
`v13.0.0`	`page` introduced.

Parallel Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/parallel-routes
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Simultaneously render one or more pages in the same view that can be navigated independently. A pattern for highly dynamic applications.*

Parallel Routes allows you to simultaneously or conditionally render one or more pages within the same layout. They are useful for highly dynamic sections of an app, such as dashboards and feeds on social sites.

For example, considering a dashboard, you can use parallel routes to simultaneously render the `team` and `analytics` pages:

Parallel Routes Diagram

Convention

Slots

Parallel routes are created using named **slots**. Slots are defined with the `@folder` convention. For example, the following file structure defines two slots: `@analytics` and `@team`:

Parallel Routes File-system Structure

Slots are passed as props to the shared parent layout. For the example above, the component in `app/layout.js` now accepts the `@analytics` and `@team` slots props, and can render them in parallel alongside the `children` prop:

```
// Archivo: app/layout.tsx switcher
export default function Layout({
  children,
  team,
  analytics,
}: {
  children: React.ReactNode
  analytics: React.ReactNode
  team: React.ReactNode
}) {
  return (
    <>
      {children}
      {team}
      {analytics}
    </>
  )
}
```

```
// Archivo: app/layout.js switcher
export default function Layout({ children, team, analytics }) {
  return (
    <>
      {children}
      {team}
      {analytics}
    </>
  )
}
```

However, slots are **not** route segments and do not affect the URL structure. For example, for `/@analytics/views`, the URL will be `/views` since `@analytics` is a slot. Slots are combined with the regular Page component to form the final page associated with the route segment. Because of this, you cannot have separate prerendered and dynamically rendered slots at the same route segment level. If one slot is dynamic, all slots at that level must be dynamic.

> **Good to know**:
>
> - The `children` prop is an implicit slot that does not need to be mapped to a folder. This means `app/page.js` is equivalent to `app/@children/page.js`.

`default.js`

You can define a `default.js` file to render as a fallback for unmatched slots during the initial load or full-page reload.

Consider the following folder structure. The `@team` slot has a `/settings` page, but `@analytics` does not.

Parallel Routes unmatched routes

When navigating to `/settings`, the `@team` slot will render the `/settings` page while maintaining the currently active page for the `@analytics` slot.

On refresh, Next.js will render a `default.js` for `@analytics`. If `default.js` doesn't exist, a `404` is rendered instead.

Additionally, since `children` is an implicit slot, you also need to create a `default.js` file to render a fallback for `children` when Next.js cannot recover the active state of the parent page.

Behavior

By default, Next.js keeps track of the active *state* (or subpage) for each slot. However, the content rendered within a slot will depend on the type of navigation:

- **Soft Navigation**: During client-side navigation, Next.js will perform a partial render, changing the subpage within the slot, while maintaining the other slot's active subpages, even if they don't match the current URL.
- **Hard Navigation**: After a full-page load (browser refresh), Next.js cannot determine the active state for the slots that don't match the current URL. Instead, it will render a `default.js` file for the unmatched slots, or `404` if `default.js` doesn't exist.

> **Good to know**:
>
> - The `404` for unmatched routes helps ensure that you don't accidentally render a parallel route on a page that it was not intended for.

Examples

With `useSelectedLayoutSegment(s)`

Both `useSelectedLayoutSegment` and `useSelectedLayoutSegments` accept a `parallelRouteKey` parameter, which allows you to read the active route segment within a slot.

```
// Archivo: app/layout.tsx switcher
'use client'

import { useSelectedLayoutSegment } from 'next/navigation'

export default function Layout({ auth }: { auth: React.ReactNode }) {
  const loginSegment = useSelectedLayoutSegment('auth')
  // ...
}
```

```
// Archivo: app/layout.js switcher
'use client'

import { useSelectedLayoutSegment } from 'next/navigation'

export default function Layout({ auth }) {
  const loginSegment = useSelectedLayoutSegment('auth')
  // ...
}
```

When a user navigates to `app/@auth/login` (or `/login` in the URL bar), `loginSegment` will be equal to the string `"login"`.

Conditional Routes

You can use Parallel Routes to conditionally render routes based on certain conditions, such as user role. For example, to render a different dashboard page for the `/admin` or `/user` roles:

Conditional routes diagram

```tsx
// Archivo: app/dashboard/layout.tsx switcher
import { checkUserRole } from '@/lib/auth'

export default function Layout({
  user,
  admin,
}: {
  user: React.ReactNode
  admin: React.ReactNode
}) {
  const role = checkUserRole()
  return role === 'admin' ? admin : user
}
```

```jsx
// Archivo: app/dashboard/layout.js switcher
import { checkUserRole } from '@/lib/auth'

export default function Layout({ user, admin }) {
  const role = checkUserRole()
  return role === 'admin' ? admin : user
}
```

Both slots render on the server, regardless of which one the layout returns. The conditional decides what the user sees, not what runs: `@admin/page.js` executes its data fetches for every user, and its output is included in the response sent to the browser. Authorize inside each slot's page, or in your Data Access Layer:

```tsx
// Archivo: app/dashboard/@admin/page.tsx switcher
import { getAdminStats } from '@/lib/dal'

export default async function AdminPage() {
  const stats = await getAdminStats()
  return <Stats stats={stats} />
}
```

```jsx
// Archivo: app/dashboard/@admin/page.js switcher
import { getAdminStats } from '@/lib/dal'

export default async function AdminPage() {
  const stats = await getAdminStats()
  return <Stats stats={stats} />
}
```

Tab Groups

You can add a `layout` inside a slot to allow users to navigate the slot independently. This is useful for creating tabs.

For example, the `@analytics` slot has two subpages: `/page-views` and `/visitors`.


Analytics slot with two subpages and a layout

Within `@analytics`, create a `layout` file to share the tabs between the two pages:

```tsx
// Archivo: app/@analytics/layout.tsx switcher
import Link from 'next/link'

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <>
      <nav>
        [Page Views](/page-views)
        [Visitors](/visitors)
      </nav>
      <div>{children}</div>
    </>
  )
}
```

```
// Archivo: app/@analytics/layout.js switcher
import Link from 'next/link'

export default function Layout({ children }) {
  return (
    <>
      <nav>
        [Page Views](/page-views)
        [Visitors](/visitors)
      </nav>
      <div>{children}</div>
    </>
  )
}
```

Modals

Parallel Routes can be used together with Intercepting Routes to create modals that support deep linking. This allows you to solve common challenges when building modals, such as:

- Making the modal content **shareable through a URL**.
- **Preserving context** when the page is refreshed, instead of closing the modal.
- **Closing the modal on backwards navigation** rather than going to the previous route.
- **Reopening the modal on forwards navigation**.

Consider the following UI pattern, where a user can open a login modal from a layout using client-side navigation, or access a separate `/login` page:

Parallel Routes Diagram

To implement this pattern, start by creating a `/login` route that renders your **main** login page.

Parallel Routes Diagram

```
// Archivo: app/login/page.tsx switcher
import { Login } from '@/app/ui/login'

export default function Page() {
  return <Login />
}
```

```
// Archivo: app/login/page.js switcher
import { Login } from '@/app/ui/login'

export default function Page() {
  return <Login />
}
```

Then, inside the `@auth` slot, add `default.js` file that returns `null`. This ensures that the modal is not rendered when it's not active.

```
// Archivo: app/@auth/default.tsx switcher
export default function Default() {
  return null
}
```

```
// Archivo: app/@auth/default.js switcher
export default function Default() {
  return null
}
```

Inside your `@auth` slot, intercept the `/login` route by importing the `<Modal>` component and its children into the `@auth/(.)login/page.tsx` file, and updating the folder name to `/@auth/(.)login/page.tsx`.

```tsx
// Archivo: app/@auth/(.)login/page.tsx switcher
import { Modal } from '@/app/ui/modal'
import { Login } from '@/app/ui/login'

export default function Page() {
  return (
    <Modal>
      <Login />
    </Modal>
  )
}
```

```jsx
// Archivo: app/@auth/(.)login/page.js switcher
import { Modal } from '@/app/ui/modal'
import { Login } from '@/app/ui/login'

export default function Page() {
  return (
    <Modal>
      <Login />
    </Modal>
  )
}
```

Good to know:

- The convention `(.)` is used for intercepting routes. See Intercepting Routes docs for more information.
- By separating the `<Modal>` functionality from the modal content (`<Login>`), you can ensure any content inside the modal, e.g. forms, are Server Components. See Interleaving Client and Server Components for more information.

Opening the modal

Now, you can leverage the Next.js router to open and close the modal. This ensures the URL is correctly updated when the modal is open, and when navigating backwards and forwards.

To open the modal, pass the `@auth` slot as a prop to the parent layout and render it alongside the `children` prop.

```tsx
// Archivo: app/layout.tsx switcher
import Link from 'next/link'

export default function Layout({
  auth,
  children,
}: {
  auth: React.ReactNode
  children: React.ReactNode
}) {
  return (
    <>
      <nav>
        [Open modal](/login)
      </nav>
      <div>{auth}</div>
      <div>{children}</div>
    </>
  )
}
```

```
// Archivo: app/layout.js switcher
import Link from 'next/link'

export default function Layout({ auth, children }) {
  return (
    <>
      <nav>
        [Open modal](/login)
      </nav>
      <div>{auth}</div>
      <div>{children}</div>
    </>
  )
}
```

When the user clicks the `<Link>`, the modal will open instead of navigating to the `/login` page. However, on refresh or initial load, navigating to `/login` will take the user to the main login page.

Closing the modal

You can close the modal by calling `router.back()` or by using the `Link` component.

```
// Archivo: app/ui/modal.tsx switcher
'use client'

import { useRouter } from 'next/navigation'

export function Modal({ children }: { children: React.ReactNode }) {
  const router = useRouter()

  return (
    <>
      <button
        onClick={() => {
          router.back()
        }}
      >
        Close modal
      </button>
      <div>{children}</div>
    </>
  )
}
```

```
// Archivo: app/ui/modal.js switcher
'use client'

import { useRouter } from 'next/navigation'

export function Modal({ children }) {
  const router = useRouter()

  return (
    <>
      <button
        onClick={() => {
          router.back()
        }}
      >
        Close modal
      </button>
      <div>{children}</div>
    </>
  )
}
```

When using the `Link` component to navigate away from a page that shouldn't render the `@auth` slot anymore, we need to make sure the parallel route matches to a component that returns `null`. For example, when navigating back to the root page, we create a

`@auth/page.tsx` component:

```
// Archivo: app/ui/modal.tsx switcher
import Link from 'next/link'

export function Modal({ children }: { children: React.ReactNode }) {
  return (
    <>
      [Close modal](/)
      <div>{children}</div>
    </>
  )
}
```

```
// Archivo: app/ui/modal.js switcher
import Link from 'next/link'

export function Modal({ children }) {
  return (
    <>
      [Close modal](/)
      <div>{children}</div>
    </>
  )
}
```

```
// Archivo: app/@auth/page.tsx switcher
export default function Page() {
  return null
}
```

```
// Archivo: app/@auth/page.js switcher
export default function Page() {
  return null
}
```

Or if navigating to any other page (such as `/foo`, `/foo/bar`, etc), you can use a catch-all slot:

```
// Archivo: app/@auth/[...catchAll]/page.tsx switcher
export default function CatchAll() {
  return null
}
```

```
// Archivo: app/@auth/[...catchAll]/page.js switcher
export default function CatchAll() {
  return null
}
```

Good to know:

- We use a catch-all route in our `@auth` slot to close the modal because of how parallel routes behave. Since client-side navigations to a route that no longer match the slot will remain visible, we need to match the slot to a route that returns `null` to close the modal.
- Other examples could include opening a photo modal in a gallery while also having a dedicated `/photo/[id]` page, or opening a shopping cart in a side modal.
- View an example of modals with Intercepted and Parallel Routes.

Loading and Error UI

Parallel Routes can be streamed independently, allowing you to define independent error and loading states for each route:

Parallel routes enable custom error and loading states

See the Loading UI and Error Handling documentation for more information.

proxy.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/proxy
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the proxy.js file.*

> **Note**: The `middleware` file convention is deprecated and has been renamed to `proxy`. See Migration to Proxy for more details.

The `proxy.js|ts` file is used to write Proxy and run code on the server before a request is completed. Then, based on the incoming request, you can modify the response by rewriting, redirecting, modifying the request or response headers, or responding directly.

Proxy executes before routes are rendered. It's particularly useful for implementing custom server-side logic like authentication, logging, or handling redirects.

> **Good to know**:
>
> Proxy is meant to be invoked separately of your render code and in optimized cases deployed to your CDN for fast redirect/rewrite handling, you should not attempt relying on shared modules or globals.
>
> To pass information from Proxy to your application, use headers, cookies, rewrites, redirects, or the URL.

Create a `proxy.ts` (or `.js`) file in the project root, or inside `src` if applicable, so that it is located at the same level as `pages` or `app`.

If you've customized `pageExtensions`, for example to `.page.ts` or `.page.js`, name your file `proxy.page.ts` or `proxy.page.js` accordingly.

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

// This function can be marked `async` if using `await` inside
export function proxy(request: NextRequest) {
  return NextResponse.redirect(new URL('/home', request.url))
}

export const config = {
  matcher: '/about/:path*',
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

// This function can be marked `async` if using `await` inside
export function proxy(request) {
  return NextResponse.redirect(new URL('/home', request.url))
}

export const config = {
  matcher: '/about/:path*',
}
```

Exports

Proxy function

The file must export a single function, either as a default export or named `proxy`. Note that multiple proxy from the same file are not supported.

```
// Archivo: proxy.js
// Example of default export
export default function proxy(request) {
  // Proxy logic
}
```

Config object (optional)

Optionally, a config object can be exported alongside the Proxy function. This object includes the matcher to specify paths where the Proxy applies.

Matcher

The `matcher` option allows you to target specific paths for the Proxy to run on.

Without a `matcher`, Proxy runs on **every request**, including static files (`_next/static`), image optimizations (`_next/image`), and assets in the `public/` folder. Consider using a negative match pattern to exclude these paths, otherwise auth logic or redirects can unintentionally block CSS, JS, or images from loading.

You can specify paths in several ways:

- For a single path: Directly use a string to define the path, like `'/about'`.
- For multiple paths: Use an array to list multiple paths, such as `matcher: ['/about', '/contact']`, which applies the Proxy to both `/about` and `/contact`.

```
// Archivo: proxy.js
export const config = {
  matcher: ['/about/:path*', '/dashboard/:path*'],
}
```

Additionally, the `matcher` option supports complex path specifications using regular expressions. For example, you can exclude certain paths with a regular expression matcher:

```
// Archivo: proxy.js
export const config = {
  matcher: [
    // Exclude API routes, static files, image optimizations, and .png files
    '/((?!api|_next/static|_next/image|.*\\.png$).*)',
  ],
}
```

This enables precise control over which paths to include or exclude.

The `matcher` option accepts an array of objects with the following keys:

- `source`: The path or pattern used to match the request paths. It can be a string for direct path matching or a pattern for more complex matching.
- `locale` (optional): A boolean that, when set to `false`, ignores locale-based routing in path matching.
- `has` (optional): Specifies conditions based on the presence of specific request elements such as headers, query parameters, or cookies.
- `missing` (optional): Focuses on conditions where certain request elements are absent, like missing headers or cookies.

```
// Archivo: proxy.js
export const config = {
  matcher: [
    {
      source: '/api/:path*',
      locale: false,
      has: [
        { type: 'header', key: 'Authorization', value: 'Bearer Token' },
        { type: 'query', key: 'userId', value: '123' },
      ],
      missing: [{ type: 'cookie', key: 'session', value: 'active' }],
    },
  ],
}
```

The `source` path patterns:

1. MUST start with `/`
2. Can include named parameters: `/about/:path` matches `/about/a` and `/about/b` but not `/about/a/c`
3. Can have modifiers on named parameters (starting with `:`): `/about/:path*` matches `/about/a/b/c` because `*` is *zero or more*. `?` is *zero or one* and `+` *one or more*
4. Can use regular expression enclosed in parenthesis: `/about/(.*)` is the same as `/about/:path*`
5. Are anchored to the start of the path: `/about` matches `/about` and `/about/team` but not `/blog/about`

Read more details on path-to-regexp documentation.

> **Good to know**:
>
> - The `matcher` values need to be constants so they can be statically analyzed at build-time. Dynamic values such as variables will be ignored.
> - For backward compatibility, Next.js always considers `/public` as `/public/index`. Therefore, a matcher of `/public/:path` will match.

Params

Next.js calls the Proxy function with two arguments, `request` and `event`, in that order. Declare only the ones you use.

`request`

The first parameter is an instance of `NextRequest`, which represents the incoming HTTP request.

```
// Archivo: proxy.ts switcher
import type { NextRequest } from 'next/server'

export function proxy(request: NextRequest) {
  // Proxy logic goes here
}
```

```
// Archivo: proxy.js switcher
export function proxy(request) {
  // Proxy logic goes here
}
```

`event`

The second parameter is an instance of `NextFetchEvent`. It exposes a single method, `waitUntil(promise)`, which keeps the Proxy invocation alive until the promise settles, so background work like logging or analytics can finish after the response is sent. See `waitUntil` and `NextFetchEvent` for a full example.

```
// Archivo: proxy.ts switcher
import type { NextFetchEvent, NextRequest } from 'next/server'

export function proxy(request: NextRequest, event: NextFetchEvent) {
  event.waitUntil(
    fetch('https://example.com/log', {
      method: 'POST',
      body: JSON.stringify({ pathname: request.nextUrl.pathname }),
    })
  )
}
```

```
// Archivo: proxy.js switcher
export function proxy(request, event) {
  event.waitUntil(
    fetch('https://example.com/log', {
      method: 'POST',
      body: JSON.stringify({ pathname: request.nextUrl.pathname }),
    })
  )
}
```

`NextProxy` type

If you prefer a shorthand, you can use the `NextProxy` type. It infers the parameter types for both `request` (`NextRequest`) and `event` (`NextFetchEvent`) automatically:

```
// Archivo: proxy.ts
import type { NextProxy } from 'next/server'

export const proxy: NextProxy = (request, event) => {
  event.waitUntil(Promise.resolve())
  return Response.json({ pathname: request.nextUrl.pathname })
}
```

> **Good to know**:
>
> - `NextRequest` is a type that represents incoming HTTP requests in Next.js Proxy, whereas `NextResponse` is a class used to manipulate and send back HTTP responses.

NextResponse

The `NextResponse` API allows you to:

- `redirect` the incoming request to a different URL
- `rewrite` the response by displaying a given URL
- Set request headers for API Routes, `getServerSideProps`, and `rewrite` destinations
- Set response cookies
- Set response headers

To produce a response from Proxy, you can:

1. `rewrite` to a route (Page or Route Handler) that produces a response
2. return a `NextResponse` directly. See Producing a Response

> **Good to know**: For redirects, you can also use `Response.redirect` instead of `NextResponse.redirect`.

> 📘 **Nota para Pages Router:** To produce a response from Proxy, you can:
>
> 1. `rewrite` to a route (Page or Edge API Route) that produces a response
> 2. return a `NextResponse` directly. See Producing a Response

Execution order

Proxy will be invoked for **every route in your project**. Given this, it's crucial to use matchers to precisely target or exclude specific routes. The following is the execution order:

1. `headers` from `next.config.js`
2. `redirects` from `next.config.js`
3. Proxy (`rewrites` , `redirects` , etc.)
4. `beforeFiles` (`rewrites`) from `next.config.js`
5. Filesystem routes (`public/` , `_next/static/` , `pages/` , `app/` , etc.)
6. `afterFiles` (`rewrites`) from `next.config.js`
7. Dynamic Routes (`/blog/[slug]`)
8. `fallback` (`rewrites`) from `next.config.js`

> **Good to know:** Server Functions are not separate routes in this chain. They are handled as POST requests to the route where they are used, so a Proxy matcher that excludes a path will also skip Server Function calls on that path.
>
> A matcher change or a refactor that moves a Server Function to a different route can silently remove Proxy coverage. Always verify authentication and authorization inside each Server Function rather than relying on Proxy alone. See the Data Security guide for recommended patterns.

Runtime

Proxy defaults to using the Node.js runtime. The `runtime` config option is not available in Proxy files. Setting the `runtime` config option in Proxy will throw an error.

Advanced Proxy flags

In `v13.1` of Next.js two additional flags were introduced for proxy, `skipProxyUrlNormalize` (formerly `skipMiddlewareUrlNormalize`) and `skipTrailingSlashRedirect` to handle advanced use cases.

`skipTrailingSlashRedirect` disables Next.js redirects for adding or removing trailing slashes. This allows custom handling inside proxy to maintain the trailing slash for some paths but not others, which can make incremental migrations easier.

```
// Archivo: next.config.js
module.exports = {
  skipTrailingSlashRedirect: true,
}
```

```
// Archivo: proxy.js
const legacyPrefixes = ['/docs', '/blog']

export default async function proxy(req) {
  const { pathname } = req.nextUrl

  if (legacyPrefixes.some((prefix) => pathname.startsWith(prefix))) {
    return NextResponse.next()
  }

  // apply trailing slash handling
  if (
    !pathname.endsWith('/') &&
    !pathname.match(/((?!\.well-known(?:\/.*)?)(?:[^/]+\/)*[^/]+\.\w+)/)
  ) {
    return NextResponse.redirect(
      new URL(`${req.nextUrl.pathname}/`, req.nextUrl)
    )
  }
}
```

`skipProxyUrlNormalize` allows for disabling the URL normalization in Next.js to make handling direct visits and client-transitions the same. In some advanced cases, this option provides full control by using the original URL.

```
// Archivo: next.config.js
module.exports = {
  skipProxyUrlNormalize: true,
}
```

```
// Archivo: proxy.js
export default async function proxy(req) {
  const { pathname } = req.nextUrl

  // GET /_next/data/build-id/hello.json

  console.log(pathname)
  // with the flag this now /_next/data/build-id/hello.json
  // without the flag this would be normalized to /hello
}
```

Examples

Conditional Statements

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function proxy(request: NextRequest) {
  if (request.nextUrl.pathname.startsWith('/about')) {
    return NextResponse.rewrite(new URL('/about-2', request.url))
  }

  if (request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.rewrite(new URL('/dashboard/user', request.url))
  }
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  if (request.nextUrl.pathname.startsWith('/about')) {
    return NextResponse.rewrite(new URL('/about-2', request.url))
  }

  if (request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.rewrite(new URL('/dashboard/user', request.url))
  }
}
```

Using Cookies

Cookies are regular headers. On a `Request`, they are stored in the `Cookie` header. On a `Response` they are in the `Set-Cookie` header. Next.js provides a convenient way to access and manipulate these cookies through the `cookies` extension on `NextRequest` and `NextResponse`.

1. For incoming requests, `cookies` comes with the following methods: `get`, `getAll`, `set`, and `delete` cookies. You can check for the existence of a cookie with `has` or remove all cookies with `clear`.
2. For outgoing responses, `cookies` have the following methods `get`, `getAll`, `set`, and `delete`.

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function proxy(request: NextRequest) {
  // Assume a "Cookie:nextjs=fast" header to be present on the incoming request
  // Getting cookies from the request using the `RequestCookies` API
  let cookie = request.cookies.get('nextjs')
  console.log(cookie) // => { name: 'nextjs', value: 'fast', Path: '/' }
  const allCookies = request.cookies.getAll()
  console.log(allCookies) // => [{ name: 'nextjs', value: 'fast' }]

  request.cookies.has('nextjs') // => true
  request.cookies.delete('nextjs')
  request.cookies.has('nextjs') // => false

  // Setting cookies on the response using the `ResponseCookies` API
  const response = NextResponse.next()
  response.cookies.set('vercel', 'fast')
  response.cookies.set({
    name: 'vercel',
    value: 'fast',
    path: '/',
  })
  cookie = response.cookies.get('vercel')
  console.log(cookie) // => { name: 'vercel', value: 'fast', Path: '/' }
  // The outgoing response will have a `Set-Cookie:vercel=fast;path=/` header.

  return response
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  // Assume a "Cookie:nextjs=fast" header to be present on the incoming request
  // Getting cookies from the request using the `RequestCookies` API
  let cookie = request.cookies.get('nextjs')
  console.log(cookie) // => { name: 'nextjs', value: 'fast', Path: '/' }
  const allCookies = request.cookies.getAll()
  console.log(allCookies) // => [{ name: 'nextjs', value: 'fast' }]

  request.cookies.has('nextjs') // => true
  request.cookies.delete('nextjs')
  request.cookies.has('nextjs') // => false

  // Setting cookies on the response using the `ResponseCookies` API
  const response = NextResponse.next()
  response.cookies.set('vercel', 'fast')
  response.cookies.set({
    name: 'vercel',
    value: 'fast',
    path: '/',
  })
  cookie = response.cookies.get('vercel')
  console.log(cookie) // => { name: 'vercel', value: 'fast', Path: '/' }
  // The outgoing response will have a `Set-Cookie:vercel=fast;path=/` header.

  return response
}
```

Setting Headers

You can set request and response headers using the `NextResponse` API (setting *request* headers is available since Next.js v13.0.0).

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function proxy(request: NextRequest) {
  // Clone the request headers and set a new header `x-hello-from-proxy1`
  const requestHeaders = new Headers(request.headers)
  requestHeaders.set('x-hello-from-proxy1', 'hello')

  // You can also set request headers in NextResponse.next
  const response = NextResponse.next({
    request: {
      // New request headers
      headers: requestHeaders,
    },
  })

  // Set a new response header `x-hello-from-proxy2`
  response.headers.set('x-hello-from-proxy2', 'hello')
  return response
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

export function proxy(request) {
  // Clone the request headers and set a new header `x-hello-from-proxy1`
  const requestHeaders = new Headers(request.headers)
  requestHeaders.set('x-hello-from-proxy1', 'hello')

  // You can also set request headers in NextResponse.next
  const response = NextResponse.next({
    request: {
      // New request headers
      headers: requestHeaders,
    },
  })

  // Set a new response header `x-hello-from-proxy2`
  response.headers.set('x-hello-from-proxy2', 'hello')
  return response
}
```

Note that the snippet uses:

- `NextResponse.next({ request: { headers: requestHeaders } })` to make `requestHeaders` available upstream
- **NOT** `NextResponse.next({ headers: requestHeaders })` which makes `requestHeaders` available to clients

Learn more in NextResponse headers in Proxy.

> **Good to know**: Avoid setting large headers as it might cause 431 Request Header Fields Too Large error depending on your backend web server configuration.

RSC requests and rewrites

During RSC requests, Next.js strips internal Flight headers from the `request` instance in Proxy. For example, headers like `rsc`, `next-router-state-tree`, and `next-router-prefetch` are not exposed through `request.headers`. This is to prevent accidentally handling an RSC request differently than the HTML request as both need to align.

When you use `NextResponse.rewrite()`, Next.js automatically propagates the required RSC rewrite headers upstream.

If you implement custom rewrite logic with `fetch()` instead of `NextResponse.rewrite()`, you can run into missing RSC headers unless you forward them manually.

For custom `fetch` rewrite setups, you can also enable `skipProxyUrlNormalize` in `next.config.js` so your rewrite logic can receive the necessary URL shape and RSC headers from the provided request object:

```
// Archivo: next.config.js
module.exports = {
  skipProxyUrlNormalize: true,
}
```

CORS

You can set CORS headers in Proxy to allow cross-origin requests, including simple and preflighted requests.

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

const allowedOrigins = ['https://acme.com', 'https://my-app.org']

const corsOptions = {
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type, Authorization',
}

export function proxy(request: NextRequest) {
  // Check the origin from the request
  const origin = request.headers.get('origin') ?? ''
  const isAllowedOrigin = allowedOrigins.includes(origin)

  // Handle preflighted requests
  const isPreflight = request.method === 'OPTIONS'

  if (isPreflight) {
    const preflightHeaders = {
      ...(isAllowedOrigin && { 'Access-Control-Allow-Origin': origin }),
      ...corsOptions,
    }
    return NextResponse.json({}, { headers: preflightHeaders })
  }

  // Handle simple requests
  const response = NextResponse.next()

  if (isAllowedOrigin) {
    response.headers.set('Access-Control-Allow-Origin', origin)
  }

  Object.entries(corsOptions).forEach(([key, value]) => {
    response.headers.set(key, value)
  })

  return response
}

export const config = {
  matcher: '/api/:path*',
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

const allowedOrigins = ['https://acme.com', 'https://my-app.org']

const corsOptions = {
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type, Authorization',
}

export function proxy(request) {
  // Check the origin from the request
  const origin = request.headers.get('origin') ?? ''
  const isAllowedOrigin = allowedOrigins.includes(origin)

  // Handle preflighted requests
  const isPreflight = request.method === 'OPTIONS'

  if (isPreflight) {
    const preflightHeaders = {
      ...(isAllowedOrigin && { 'Access-Control-Allow-Origin': origin }),
      ...corsOptions,
    }
    return NextResponse.json({}, { headers: preflightHeaders })
  }

  // Handle simple requests
  const response = NextResponse.next()

  if (isAllowedOrigin) {
    response.headers.set('Access-Control-Allow-Origin', origin)
  }

  Object.entries(corsOptions).forEach(([key, value]) => {
    response.headers.set(key, value)
  })

  return response
}

export const config = {
  matcher: '/api/:path*',
}
```

> **Good to know:** You can configure CORS headers for individual routes in Route Handlers.

Producing a response

You can respond from Proxy directly by returning a `Response` or `NextResponse` instance. (This is available since Next.js v13.1.0)

```
// Archivo: proxy.ts switcher
import type { NextRequest } from 'next/server'
import { isAuthenticated } from '@lib/auth'

// Limit the proxy to paths starting with `/api/`
export const config = {
  matcher: '/api/:function*',
}

export function proxy(request: NextRequest) {
  // Call our authentication function to check the request
  if (!isAuthenticated(request)) {
    // Respond with JSON indicating an error message
    return Response.json(
      { success: false, message: 'authentication failed' },
      { status: 401 }
    )
  }
}
```

```
// Archivo: proxy.js switcher
import { isAuthenticated } from '@lib/auth'

// Limit the proxy to paths starting with `/api/`
export const config = {
  matcher: '/api/:function*',
}

export function proxy(request) {
  // Call our authentication function to check the request
  if (!isAuthenticated(request)) {
    // Respond with JSON indicating an error message
    return Response.json(
      { success: false, message: 'authentication failed' },
      { status: 401 }
    )
  }
}
```

Negative matching

The `matcher` config allows full regex so matching like negative lookaheads or character matching is supported. An example of a negative lookahead to match all except specific paths can be seen here:

```
// Archivo: proxy.js
export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - api (API routes)
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico, sitemap.xml, robots.txt (metadata files)
     */
    '/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)',
  ],
}
```

You can also bypass Proxy for certain requests by using the `missing` or `has` arrays, or a combination of both:

```
// Archivo: proxy.js
export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - api (API routes)
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico, sitemap.xml, robots.txt (metadata files)
     */
    {
      source:
        '/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)',
      missing: [
        { type: 'header', key: 'next-router-prefetch' },
        { type: 'header', key: 'purpose', value: 'prefetch' },
      ],
    },

    {
      source:
        '/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)',
      has: [
        { type: 'header', key: 'next-router-prefetch' },
        { type: 'header', key: 'purpose', value: 'prefetch' },
      ],
    },

    {
      source:
        '/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)',
      has: [{ type: 'header', key: 'x-present' }],
      missing: [{ type: 'header', key: 'x-missing', value: 'prefetch' }],
    },
  ],
}
```

> **Good to know**:
>
> Even when `_next/data` is excluded in a negative matcher pattern, proxy will still be invoked for `_next/data` routes. This is intentional behavior to prevent accidental security issues where you might protect a page but forget to protect the corresponding data route.

```
// Archivo: proxy.js
export const config = {
  matcher:
    '/((?!api|_next/data|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)',
}

// Proxy will still run for /_next/data/* routes despite being excluded
```

`waitUntil` and `NextFetchEvent`

The `NextFetchEvent` object extends the native `FetchEvent` object, and includes the `waitUntil()` method.

The `waitUntil()` method takes a promise as an argument, and extends the lifetime of the Proxy until the promise settles. This is useful for performing work in the background.

```
// Archivo: proxy.ts
import { NextResponse } from 'next/server'
import type { NextFetchEvent, NextRequest } from 'next/server'

export function proxy(req: NextRequest, event: NextFetchEvent) {
  event.waitUntil(
    fetch('https://my-analytics-platform.com', {
      method: 'POST',
      body: JSON.stringify({ pathname: req.nextUrl.pathname }),
    })
  )

  return NextResponse.next()
}
```

Unit testing (experimental)

Starting in Next.js 15.1, the `next/experimental/testing/server` package contains utilities to help unit test proxy files. Unit testing proxy can help ensure that it's only run on desired paths and that custom routing logic works as intended before code reaches production.

The `unstable_doesProxyMatch` function can be used to assert whether proxy will run for the provided URL, headers, and cookies.

```
import { unstable_doesProxyMatch } from 'next/experimental/testing/server'

expect(
  unstable_doesProxyMatch({
    config,
    nextConfig,
    url: '/test',
  })
).toEqual(false)
```

The entire proxy function can also be tested.

```
import { isRewrite, getRewrittenUrl } from 'next/experimental/testing/server'

const request = new NextRequest('https://nextjs.org/docs')
const response = await proxy(request)
expect(isRewrite(response)).toEqual(true)
expect(getRewrittenUrl(response)).toEqual('https://other-domain.com/docs')
// getRedirectUrl could also be used if the response were a redirect
```

Platform support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Learn how to configure Proxy when self-hosting Next.js.

Migration to Proxy

Why the Change

The reason behind the renaming of `middleware` is that the term "middleware" can often be confused with Express.js middleware, leading to a misinterpretation of its purpose. Also, Middleware is highly capable, so it may encourage the usage; however, this feature is recommended to be used as a last resort.

Next.js is moving forward to provide better APIs with better ergonomics so that developers can achieve their goals without Middleware. This is the reason behind the renaming of `middleware`.

Why "Proxy"

The name Proxy clarifies what Middleware is capable of. The term "proxy" implies a network boundary in front of the app, which is how this feature behaves. It can run outside of your application's main runtime and handle requests before they reach your app. These characteristics align better with the term "proxy" and provide a clearer purpose for the feature.

How to Migrate

We recommend users avoid relying on Middleware unless no other options exist. Our goal is to give them APIs with better ergonomics so they can achieve their goals without Middleware.

The term "middleware" often confuses users with Express.js middleware, which can encourage misuse. To clarify our direction, we are renaming the file convention to "proxy." This highlights that we are moving away from Middleware, breaking down its overloaded features, and making the Proxy clear in its purpose.

Next.js provides a codemod to migrate from `middleware.ts` to `proxy.ts`. You can run the following command to migrate:

```
npx @next/codemod@canary middleware-to-proxy .
```

The codemod will rename the file and the function name from `middleware` to `proxy`.

```
// middleware.ts → proxy.ts

- export function middleware() {
+ export function proxy() {
```

Version history

Version	Changes
`v16.0.0`	Middleware is deprecated and renamed to Proxy. Proxy defaults to the Node.js runtime
`v15.5.0`	Middleware can now use the Node.js runtime (stable)
`v15.2.0`	Middleware can now use the Node.js runtime (experimental)
`v13.1.0`	Advanced Middleware flags added
`v13.0.0`	Middleware can modify request headers, response headers, and send responses
`v12.2.0`	Middleware is stable, please see the upgrade guide
`v12.0.9`	Enforce absolute URLs in Edge Runtime (PR)
`v12.0.0`	Middleware (Beta) added

public Folder

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/public-folder
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Next.js allows you to serve static files, like images, in the public directory. You can learn how it works here.*

Next.js can serve static files, like images, under a folder called `public` in the root directory. Files inside `public` can then be referenced by your code starting from the base URL (`/`).

For example, the file `public/avatars/me.png` can be viewed by visiting the `/avatars/me.png` path. The code to display that image might look like:

```
// Archivo: avatar.js
import Image from 'next/image'

export function Avatar({ id, alt }) {
  return

![Illustration]()


}

export function AvatarOfMe() {
  return <Avatar id="me" alt="A portrait of me" />
}
```

Caching

Next.js cannot safely cache assets in the `public` folder because they may change. The default caching headers applied are:

```
Cache-Control: public, max-age=0
```

Robots, Favicons, and others

> 📘 **Nota para Pages Router:** The folder is also useful for `robots.txt` , `favicon.ico` , Google Site Verification, and any other static files (including `.html`). But make sure to not have a static file with the same name as a file in the `pages/` directory, as this will result in an error. Read more.

For static metadata files, such as `robots.txt` , `favicon.ico` , etc, you should use special metadata files inside the `app` folder.

Route Groups

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route-groups
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Route Groups can be used to partition your Next.js application into different sections.*

Route Groups are a folder convention that let you organize routes by category or team.

Convention

A route group can be created by wrapping a folder's name in parenthesis: `(folderName)` .

This convention indicates the folder is for organizational purposes and should **not be included** in the route's URL path.

An example folder structure using route groups

Use cases

- Organizing routes by team, concern, or feature.
- Defining multiple root layouts.
- Opting specific route segments into sharing a layout, while keeping others out.

Caveats

- **Full page load**: If you navigate between routes that use different root layouts, it'll trigger a full page reload. For example, navigating from `/cart` that uses `app/(shop)/layout.js` to `/blog` that uses `app/(marketing)/layout.js`. This **only** applies to multiple root layouts.
- **Conflicting paths**: Routes in different groups should not resolve to the same URL path. For example, `(marketing)/about/page.js` and `(shop)/about/page.js` would both resolve to `/about` and cause an error.
- **Top-level root layout**: If you use multiple root layouts without a top-level `layout.js` file, make sure your home route (/) is defined within one of the route groups, e.g. app/(marketing)/page.js.

---

route.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/route
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the route.js special file.*

Route Handlers allow you to create custom request handlers for a given route using the Web Request and Response APIs.

```
// Archivo: route.ts switcher
export async function GET() {
  return Response.json({ message: 'Hello World' })
}
```

```
// Archivo: route.js switcher
export async function GET() {
  return Response.json({ message: 'Hello World' })
}
```

Reference

HTTP Methods

A **route** file allows you to create custom request handlers for a given route. The following HTTP methods are supported: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, and `OPTIONS`.

```
// Archivo: route.ts switcher
export async function GET(request: Request) {}

export async function HEAD(request: Request) {}

export async function POST(request: Request) {}

export async function PUT(request: Request) {}

export async function DELETE(request: Request) {}

export async function PATCH(request: Request) {}

// If `OPTIONS` is not defined, Next.js will automatically implement `OPTIONS` and set the appropriate Response `Allow`
header depending on the other methods defined in the Route Handler.
export async function OPTIONS(request: Request) {}
```

```
// Archivo: route.js switcher
export async function GET(request) {}

export async function HEAD(request) {}

export async function POST(request) {}

export async function PUT(request) {}

export async function DELETE(request) {}

export async function PATCH(request) {}

// If `OPTIONS` is not defined, Next.js will automatically implement `OPTIONS` and set the appropriate Response `Allow`
header depending on the other methods defined in the Route Handler.
export async function OPTIONS(request) {}
```

Parameters

`request` (optional)

The `request` object is a NextRequest object, which is an extension of the Web Request API. `NextRequest` gives you further control over the incoming request, including easily accessing `cookies` and an extended, parsed, URL object `nextUrl`.

```
// Archivo: route.ts switcher
import type { NextRequest } from 'next/server'

export async function GET(request: NextRequest) {
  const url = request.nextUrl
}
```

```
// Archivo: route.js switcher
export async function GET(request) {
  const url = request.nextUrl
}
```

`context` (optional)

- `params`: a promise that resolves to an object containing the dynamic route parameters for the current route.

```
// Archivo: app/dashboard/[team]/route.ts switcher
export async function GET(
  request: Request,
  { params }: { params: Promise<{ team: string }> }
) {
  const { team } = await params
}
```

```
// Archivo: app/dashboard/[team]/route.js switcher
export async function GET(request, { params }) {
  const { team } = await params
}
```

Example	URL	`params`
`app/dashboard/[team]/route.js`	`/dashboard/1`	`Promise<{ team: '1' }>`
`app/shop/[tag]/[item]/route.js`	`/shop/1/2`	`Promise<{ tag: '1', item: '2' }>`
`app/blog/[ ... slug]/route.js`	`/blog/1/2`	`Promise<{ slug: ['1', '2'] }>`

Route Context Helper

You can type the Route Handler context using `RouteContext` to get strongly typed `params` from a route literal. `RouteContext` is a globally available helper.

```
// Archivo: app/users/[id]/route.ts
import type { NextRequest } from 'next/server'

export async function GET(_req: NextRequest, ctx: RouteContext<'/users/[id]'>) {
  const { id } = await ctx.params
  return Response.json({ id })
}
```

Good to know

- Types are generated during `next dev`, `next build` or `next typegen`.
- After type generation, the `RouteContext` helper is globally available. It doesn't need to be imported.

Examples

Cookies

You can read or set cookies with `cookies` from `next/headers`.

```
// Archivo: route.ts switcher
import { cookies } from 'next/headers'

export async function GET(request: NextRequest) {
  const cookieStore = await cookies()

  const a = cookieStore.get('a')
  const b = cookieStore.set('b', '1')
  const c = cookieStore.delete('c')
}
```

```
// Archivo: route.js switcher
import { cookies } from 'next/headers'

export async function GET(request) {
  const cookieStore = await cookies()

  const a = cookieStore.get('a')
  const b = cookieStore.set('b', '1')
  const c = cookieStore.delete('c')
}
```

Alternatively, you can return a new `Response` using the `Set-Cookie` header.

```
// Archivo: app/api/route.ts switcher
import { cookies } from 'next/headers'

export async function GET(request: Request) {
  const cookieStore = await cookies()
  const token = cookieStore.get('token')

  return new Response('Hello, Next.js!', {
    status: 200,
    headers: { 'Set-Cookie': `token=${token.value}` },
  })
}
```

```
// Archivo: app/api/route.js switcher
import { cookies } from 'next/headers'

export async function GET(request) {
  const cookieStore = await cookies()
  const token = cookieStore.get('token')

  return new Response('Hello, Next.js!', {
    status: 200,
    headers: { 'Set-Cookie': `token=${token.value}` },
  })
}
```

You can also use the underlying Web APIs to read cookies from the request (`NextRequest`):

```
// Archivo: app/api/route.ts switcher
import { type NextRequest } from 'next/server'

export async function GET(request: NextRequest) {
  const token = request.cookies.get('token')
}
```

```
// Archivo: app/api/route.js switcher
export async function GET(request) {
  const token = request.cookies.get('token')
}
```

Headers

You can read headers with `headers` from `next/headers` .

```
// Archivo: route.ts switcher
import { headers } from 'next/headers'
import type { NextRequest } from 'next/server'

export async function GET(request: NextRequest) {
  const headersList = await headers()
  const referer = headersList.get('referer')
}
```

```
// Archivo: route.js switcher
import { headers } from 'next/headers'

export async function GET(request) {
  const headersList = await headers()
  const referer = headersList.get('referer')
}
```

This `headers` instance is read-only. To set headers, you need to return a new `Response` with new `headers` .

```
// Archivo: app/api/route.ts switcher
import { headers } from 'next/headers'

export async function GET(request: Request) {
  const headersList = await headers()
  const referer = headersList.get('referer')

  return new Response('Hello, Next.js!', {
    status: 200,
    headers: { referer: referer },
  })
}
```

```
// Archivo: app/api/route.js switcher
import { headers } from 'next/headers'

export async function GET(request) {
  const headersList = await headers()
  const referer = headersList.get('referer')

  return new Response('Hello, Next.js!', {
    status: 200,
    headers: { referer: referer },
  })
}
```

You can also use the underlying Web APIs to read headers from the request (`NextRequest`):

```
// Archivo: app/api/route.ts switcher
import { type NextRequest } from 'next/server'

export async function GET(request: NextRequest) {
  const requestHeaders = new Headers(request.headers)
}
```

```
// Archivo: app/api/route.js switcher
export async function GET(request) {
  const requestHeaders = new Headers(request.headers)
}
```

Revalidating Cached Data

You can revalidate cached data using the `revalidate` route segment config option.

```
// Archivo: app/posts/route.ts switcher
export const revalidate = 60

export async function GET() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()

  return Response.json(posts)
}
```

```
// Archivo: app/posts/route.js switcher
export const revalidate = 60

export async function GET() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()

  return Response.json(posts)
}
```

Redirects

```
// Archivo: app/api/route.ts switcher
import { redirect } from 'next/navigation'

export async function GET(request: Request) {
  redirect('https://nextjs.org/')
}
```

```
// Archivo: app/api/route.js switcher
import { redirect } from 'next/navigation'

export async function GET(request) {
  redirect('https://nextjs.org/')
}
```

Dynamic Route Segments

Route Handlers can use Dynamic Segments to create request handlers from dynamic data.

```
// Archivo: app/items/[slug]/route.ts switcher
export async function GET(
  request: Request,
  { params }: { params: Promise<{ slug: string }> }
) {
  const { slug } = await params // 'a', 'b', or 'c'
}
```

```
// Archivo: app/items/[slug]/route.js switcher
export async function GET(request, { params }) {
  const { slug } = await params // 'a', 'b', or 'c'
}
```

Route	Example URL	`params`
`app/items/[slug]/route.js`	`/items/a`	`Promise<{ slug: 'a' }>`
`app/items/[slug]/route.js`	`/items/b`	`Promise<{ slug: 'b' }>`
`app/items/[slug]/route.js`	`/items/c`	`Promise<{ slug: 'c' }>`

Static Generation with `generateStaticParams`

You can use `generateStaticParams` with dynamic Route Handlers to statically generate responses at build time for specified params, while handling other params dynamically at request time.

When using Cache Components, you can combine `generateStaticParams` with `use cache` to enable data caching for both prerendered and runtime params.

See the generateStaticParams with Route Handlers documentation for examples and details.

URL Query Parameters

The request object passed to the Route Handler is a `NextRequest` instance, which includes some additional convenience methods, such as those for more easily handling query parameters.

```ts
// Archivo: app/api/search/route.ts switcher
import { type NextRequest } from 'next/server'

export function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams
  const query = searchParams.get('query')
  // query is "hello" for /api/search?query=hello
}
```

```js
// Archivo: app/api/search/route.js switcher
export function GET(request) {
  const searchParams = request.nextUrl.searchParams
  const query = searchParams.get('query')
  // query is "hello" for /api/search?query=hello
}
```

Streaming

Streaming is commonly used in combination with Large Language Models (LLMs), such as OpenAI, for AI-generated content. Learn more about the AI SDK.

```ts
// Archivo: app/api/chat/route.ts switcher
import { openai } from '@ai-sdk/openai'
import { StreamingTextResponse, streamText } from 'ai'

export async function POST(req: Request) {
  const { messages } = await req.json()
  const result = await streamText({
    model: openai('gpt-4-turbo'),
    messages,
  })

  return new StreamingTextResponse(result.toAIStream())
}
```

```js
// Archivo: app/api/chat/route.js switcher
import { openai } from '@ai-sdk/openai'
import { StreamingTextResponse, streamText } from 'ai'

export async function POST(req) {
  const { messages } = await req.json()
  const result = await streamText({
    model: openai('gpt-4-turbo'),
    messages,
  })

  return new StreamingTextResponse(result.toAIStream())
}
```

These abstractions use the Web APIs to create a stream. You can also use the underlying Web APIs directly.

```
// Archivo: app/api/route.ts switcher
// https://developer.mozilla.org/docs/Web/API/ReadableStream#convert_async_iterator_to_stream
function iteratorToStream(iterator: any) {
  return new ReadableStream({
    async pull(controller) {
      const { value, done } = await iterator.next()

      if (done) {
        controller.close()
      } else {
        controller.enqueue(value)
      }
    },
  })
}

function sleep(time: number) {
  return new Promise((resolve) => {
    setTimeout(resolve, time)
  })
}

const encoder = new TextEncoder()

async function* makeIterator() {
  yield encoder.encode('<p>One</p>')
  await sleep(200)
  yield encoder.encode('<p>Two</p>')
  await sleep(200)
  yield encoder.encode('<p>Three</p>')
}

export async function GET() {
  const iterator = makeIterator()
  const stream = iteratorToStream(iterator)

  return new Response(stream)
}
```

```
// Archivo: app/api/route.js switcher
// https://developer.mozilla.org/docs/Web/API/ReadableStream#convert_async_iterator_to_stream
function iteratorToStream(iterator) {
  return new ReadableStream({
    async pull(controller) {
      const { value, done } = await iterator.next()

      if (done) {
        controller.close()
      } else {
        controller.enqueue(value)
      }
    },
  })
}

function sleep(time) {
  return new Promise((resolve) => {
    setTimeout(resolve, time)
  })
}

const encoder = new TextEncoder()

async function* makeIterator() {
  yield encoder.encode('<p>One</p>')
  await sleep(200)
  yield encoder.encode('<p>Two</p>')
  await sleep(200)
  yield encoder.encode('<p>Three</p>')
}

export async function GET() {
  const iterator = makeIterator()
  const stream = iteratorToStream(iterator)

  return new Response(stream)
}
```

Request Body

You can read the `Request` body using the standard Web API methods:

```
// Archivo: app/items/route.ts switcher
export async function POST(request: Request) {
  const res = await request.json()
  return Response.json({ res })
}
```

```
// Archivo: app/items/route.js switcher
export async function POST(request) {
  const res = await request.json()
  return Response.json({ res })
}
```

Request Body FormData

You can read the `FormData` using the `request.formData()` function:

```
// Archivo: app/items/route.ts switcher
export async function POST(request: Request) {
  const formData = await request.formData()
  const name = formData.get('name')
  const email = formData.get('email')
  return Response.json({ name, email })
}
```

```
// Archivo: app/items/route.js switcher
export async function POST(request) {
  const formData = await request.formData()
  const name = formData.get('name')
  const email = formData.get('email')
  return Response.json({ name, email })
}
```

Since `formData` data are all strings, you may want to use `zod-form-data` to validate the request and retrieve data in the format you prefer (e.g. `number`).

CORS

You can set CORS headers for a specific Route Handler using the standard Web API methods:

```
// Archivo: app/api/route.ts switcher
export async function GET(request: Request) {
  return new Response('Hello, Next.js!', {
    status: 200,
    headers: {
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
      'Access-Control-Allow-Headers': 'Content-Type, Authorization',
    },
  })
}
```

```
// Archivo: app/api/route.js switcher
export async function GET(request) {
  return new Response('Hello, Next.js!', {
    status: 200,
    headers: {
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
      'Access-Control-Allow-Headers': 'Content-Type, Authorization',
    },
  })
}
```

> **Good to know**:
>
> - To add CORS headers to multiple Route Handlers, you can use Proxy or the `next.config.js` file.

Webhooks

You can use a Route Handler to receive webhooks from third-party services:

```
// Archivo: app/api/route.ts switcher
export async function POST(request: Request) {
  try {
    const text = await request.text()
    // Process the webhook payload
  } catch (error) {
    return new Response(`Webhook error: ${error.message}`, {
      status: 400,
    })
  }

  return new Response('Success!', {
    status: 200,
  })
}
```

```
// Archivo: app/api/route.js switcher
export async function POST(request) {
  try {
    const text = await request.text()
    // Process the webhook payload
  } catch (error) {
    return new Response(`Webhook error: ${error.message}`, {
      status: 400,
    })
  }

  return new Response('Success!', {
    status: 200,
  })
}
```

Notably, unlike API Routes with the Pages Router, you do not need to use `bodyParser` to use any additional configuration.

Non-UI Responses

You can use Route Handlers to return non-UI content. Note that `sitemap.xml`, `robots.txt`, `app icons`, and open graph images all have built-in support.

```
// Archivo: app/rss.xml/route.ts switcher
export async function GET() {
  return new Response(
    `<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">

<channel>
  <title>Next.js Documentation</title>
  <link>https://nextjs.org/docs</link>
  <description>The React Framework for the Web</description>
</channel>

</rss>`,
    {
      headers: {
        'Content-Type': 'text/xml',
      },
    }
  )
}
```

```
// Archivo: app/rss.xml/route.js switcher
export async function GET() {
  return new Response(`<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">

<channel>
  <title>Next.js Documentation</title>
  <link>https://nextjs.org/docs</link>
  <description>The React Framework for the Web</description>
</channel>

</rss>`)
}
```

Segment Config Options

Route Handlers use the same route segment configuration as pages and layouts.

```
// Archivo: app/items/route.ts switcher
export const dynamic = 'auto'
export const dynamicParams = true
export const revalidate = false
export const fetchCache = 'auto'
export const runtime = 'nodejs'
export const preferredRegion = 'auto' // deprecated
```

```
// Archivo: app/items/route.js switcher
export const dynamic = 'auto'
export const dynamicParams = true
export const revalidate = false
export const fetchCache = 'auto'
export const runtime = 'nodejs'
export const preferredRegion = 'auto' // deprecated
```

See the API reference for more details.

Version History

Version	Changes
`v15.0.0-RC`	`context.params` is now a promise. A codemod is available
`v15.0.0-RC`	The default caching for `GET` handlers was changed from static to dynamic
`v13.2.0`	Route Handlers are introduced.

src Folder

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/src-folder
> 📂 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *Save pages under the `src` folder as an alternative to the root `pages` directory.*

As an alternative to having the special Next.js `app` or `pages` directories in the root of your project, Next.js also supports the common pattern of placing application code under the `src` folder.

This separates application code from project configuration files which mostly live in the root of a project, which is preferred by some individuals and teams.

To use the `src` folder, move the `app` Router folder or `pages` Router folder to `src/app` or `src/pages` respectively.

An example folder structure with the src folder

Good to know:

- The `/public` directory should remain in the root of your project.
- Config files like `package.json`, `next.config.js` and `tsconfig.json` should remain in the root of your project.
- `.env.*` files should remain in the root of your project.
- `src/app` or `src/pages` will be ignored if `app` or `pages` are present in the root directory.
- If you're using `src`, you'll probably also move other application folders such as `/components` or `/lib`.
- If you're using Proxy, ensure it is placed inside the `src` folder.
- If you're using Tailwind CSS, you'll need to add the `/src` prefix to the `tailwind.config.js` file in the content section.
- If you are using TypeScript paths for imports such as `@/*`, you should update the `paths` object in `tsconfig.json` to include `src/`.

---

template.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/template
📂 **Sección:** App > Api Reference > File Conventions
📝 **Descripción:** *API Reference for the template.js file.*

A **template** file is similar to a layout in that it wraps a layout or page. Unlike layouts that persist across routes and maintain state, templates are given a unique key, meaning children Client Components reset their state on navigation.

They are useful when you need to:

- Resynchronize `useEffect` on navigation.
- Reset the state of a child Client Components on navigation. For example, an input field.
- To change default framework behavior. For example, Suspense boundaries inside layouts only show a fallback on first load, while templates show it on every navigation.

Convention

A template can be defined by exporting a default React component from a `template.js` file. The component should accept a `children` prop.

template.js special file

```
// Archivo: app/template.tsx switcher
export default function Template({ children }: { children: React.ReactNode }) {
  return <div>{children}</div>
}
```

```
// Archivo: app/template.js switcher
export default function Template({ children }) {
  return <div>{children}</div>
}
```

In terms of nesting, `template.js` is rendered between a layout and its children. Here's a simplified output:

```
// Archivo: Output
<Layout>

  <Template key={routeParam}>{children}</Template>
</Layout>
```

In the component hierarchy, `template.js` renders between `layout.js` and `error.js`. It wraps `error.js`, `loading.js`, `not-found.js`, and `page.js`, but does **not** wrap the `layout.js` in the same segment.

Props

`children` (required)

Template accepts a `children` prop.

```
// Archivo: Output
<Layout>

  <Template key={routeParam}>{children}</Template>
</Layout>
```

Behavior

- **Server Components**: By default, templates are Server Components.
- **With navigation**: Templates receive a unique key for their own segment level. They remount when that segment (including its dynamic params) changes. Navigations within deeper segments do not remount higher-level templates. Search params do not trigger remounts.
- **State reset**: Any Client Component inside the template will reset its state on navigation.
- **Effect re-run**: Effects like `useEffect` will re-synchronize as the component remounts.
- **DOM reset**: DOM elements inside the template are fully recreated.

Templates during navigation and remounting

This section illustrates how templates behave during navigation. It shows, step by step, which templates remount on each route change and why.

Using this project tree:

```
app
├── about
│   ├── page.tsx
├── blog
│   ├── [slug]
│   │   └── page.tsx
│   ├── page.tsx
│   └── template.tsx
├── layout.tsx
├── page.tsx
└── template.tsx
```

Starting at `/`, the React tree looks roughly like this.

> Note: The `key` values shown in the examples are illustrative, the values in your application may differ.

```
// Archivo: Output
<RootLayout>

  <Template key="/">
    <Page />
  </Template>
</RootLayout>
```

Navigating to `/about` (first segment changes), the root template key changes, it remounts:

```
// Archivo: Output
<RootLayout>

  <Template key="/about">
    <AboutPage />
  </Template>
</RootLayout>
```

Navigating to `/blog` (first segment changes), the root template key changes, it remounts and the blog-level template mounts:

```
// Archivo: Output
<RootLayout>

  <Template key="/blog">

    <Template key="/blog">
      <BlogIndexPage />
    </Template>
  </Template>
</RootLayout>
```

Navigating within the same first segment to `/blog/first-post` (child segment changes), the root template key doesn't change, but the blog-level template key changes, it remounts:

```
// Archivo: Output
<RootLayout>

  <Template key="/blog">


    <Template key="/blog/first-post">
      <BlogPostPage slug="first-post" />
    </Template>
  </Template>
</RootLayout>
```

Navigating to `/blog/second-post` (same first segment, different child segment), the root template key doesn't change, but the blog-level template key changes, it remounts again:

```
// Archivo: Output
<RootLayout>

  <Template key="/blog">


    <Template key="/blog/second-post">
      <BlogPostPage slug="second-post" />
    </Template>
  </Template>
</RootLayout>
```

Version History

Version	Changes
`v13.0.0`	`template` introduced.

unauthorized.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/file-conventions/unauthorized
> 📁 **Sección:** App > Api Reference > File Conventions
> 📝 **Descripción:** *API reference for the unauthorized.js special file.*

The **unauthorized** file is used to render UI when the `unauthorized` function is invoked during authentication. Along with allowing you to customize the UI, Next.js will return a `401` status code.

```
// Archivo: app/unauthorized.tsx switcher
import Login from '@/app/components/Login'

export default function Unauthorized() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

```
// Archivo: app/unauthorized.js switcher
import Login from '@/app/components/Login'

export default function Unauthorized() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

Reference

Props

`unauthorized.js` components do not accept any props.

Examples

Displaying login UI to unauthenticated users

You can use `unauthorized` function to render the `unauthorized.js` file with a login UI.

```tsx
// Archivo: app/dashboard/page.tsx switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  return <div>Dashboard</div>
}
```

```jsx
// Archivo: app/dashboard/page.js switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  return <div>Dashboard</div>
}
```

```tsx
// Archivo: app/unauthorized.tsx switcher
import Login from '@/app/components/Login'

export default function UnauthorizedPage() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

```jsx
// Archivo: app/unauthorized.js switcher
import Login from '@/app/components/Login'

export default function UnauthorizedPage() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

Version History

Version	Changes
`v15.1.0`	`unauthorized.js` introduced.

after

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/after
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the after function.*

`after` allows you to schedule work to be executed after a response (or prerender) is finished. This is useful for tasks and other side effects that should not block the response, such as logging and analytics.

It can be used in Server Components (including `generateMetadata`), Server Functions, Route Handlers, and Proxy.

The function accepts a callback that will be executed after the response (or prerender) is finished:

```
// Archivo: app/layout.tsx switcher
import { after } from 'next/server'
// Custom logging function
import { log } from '@/app/utils'

export default function Layout({ children }: { children: React.ReactNode }) {
  after(() => {
    // Execute after the layout is rendered and sent to the user
    log()
  })
  return <>{children}</>
}
```

```
// Archivo: app/layout.jsx switcher
import { after } from 'next/server'
// Custom logging function
import { log } from '@/app/utils'

export default function Layout({ children }) {
  after(() => {
    // Execute after the layout is rendered and sent to the user
    log()
  })
  return <>{children}</>
}
```

> **Good to know:** `after` is not a Request-time API and calling it does not cause a route to become dynamic. If it's used within a static page, the callback will execute at build time, or whenever a page is revalidated.

Reference

Parameters

- A callback function which will be executed after the response (or prerender) is finished.

Duration

`after` will run for the platform's default or configured max duration of your route. If your platform supports it, you can configure the timeout limit using the `maxDuration` route segment config.

Good to know

- `after` will be executed even if the response didn't complete successfully. Including when an error is thrown or when `notFound` or `redirect` is called.
- You can use React `cache` to deduplicate functions called inside `after`.

- `after` can be nested inside other `after` calls, for example, you can create utility functions that wrap `after` calls to add additional functionality.

Examples

With request APIs

Whether you can use request APIs like `cookies` and `headers` inside `after` depends on where `after` is called from.

In Route Handlers and Server Functions

You can call `cookies` and `headers` directly inside the `after` callback when used in Route Handlers and Server Functions. This is useful for logging activity after a mutation or API request. For example:

```
// Archivo: app/api/route.ts highlight={2,10-16} switcher
import { after } from 'next/server'
import { cookies, headers } from 'next/headers'
import { logUserAction } from '@/app/utils'

export async function POST(request: Request) {
  // Perform mutation
  // ...

  // Log user activity for analytics
  after(async () => {
    const userAgent = (await headers()).get('user-agent') || 'unknown'
    const sessionCookie =
      (await cookies()).get('session-id')?.value || 'anonymous'

    logUserAction({ sessionCookie, userAgent })
  })

  return new Response(JSON.stringify({ status: 'success' }), {
    status: 200,
    headers: { 'Content-Type': 'application/json' },
  })
}
```

```
// Archivo: app/api/route.js highlight={2,10-16} switcher
import { after } from 'next/server'
import { cookies, headers } from 'next/headers'
import { logUserAction } from '@/app/utils'

export async function POST(request) {
  // Perform mutation
  // ...

  // Log user activity for analytics
  after(async () => {
    const userAgent = (await headers()).get('user-agent') || 'unknown'
    const sessionCookie =
      (await cookies()).get('session-id')?.value || 'anonymous'

    logUserAction({ sessionCookie, userAgent })
  })

  return new Response(JSON.stringify({ status: 'success' }), {
    status: 200,
    headers: { 'Content-Type': 'application/json' },
  })
}
```

In Server Components (pages and layouts)

Server Components (including pages, layouts, and `generateMetadata`) **cannot** use `cookies`, `headers`, or other Request-time APIs inside `after`. This is because Next.js needs to know which part of the component tree accesses request data to support Partial Prerendering and Cache Components, but `after` runs after React's rendering lifecycle.

If you need request data inside an `after` callback in a Server Component, read it beforehand and pass the values in:

```
// Archivo: app/page.tsx highlight={8-10,12} switcher
import { after } from 'next/server'
import { cookies, headers } from 'next/headers'
import { logUserAction } from '@/app/utils'

export default async function Page() {
  // Read request data before `after` - this is allowed
  // These calls will be read during the component's rendering lifecycle
  const userAgent = (await headers()).get('user-agent') || 'unknown'
  const sessionCookie =
    (await cookies()).get('session-id')?.value || 'anonymous'

  after(() => {
    // Use the values read above
    logUserAction({ sessionCookie, userAgent })
  })

  return <h1>My Page</h1>
}
```

```
// Archivo: app/page.jsx highlight={8-10,12} switcher
import { after } from 'next/server'
import { cookies, headers } from 'next/headers'
import { logUserAction } from '@/app/utils'

export default async function Page() {
  // Read request data before `after` - this is allowed
  // These calls will be read during the component's rendering lifecycle
  const userAgent = (await headers()).get('user-agent') || 'unknown'
  const sessionCookie =
    (await cookies()).get('session-id')?.value || 'anonymous'

  after(() => {
    // Use the values read above
    logUserAction({ sessionCookie, userAgent })
  })

  return <h1>My Page</h1>
}
```

Calling `cookies()` or `headers()` inside the `after` callback in a Server Component will throw a runtime error.

With Cache Components

When using Cache Components, components that access request data like `cookies` or `headers` must be wrapped in [````` </>) }

async function DynamicContent() { const sessionCookie = (await cookies()).get('session-id')?.value || 'anonymous'

// Schedule work after the response is sent after(() => { logUserAction({ sessionCookie }) })

return

Your session: {sessionCookie}

}

```jsx
// Archivo: app/page.jsx highlight={18-19,22-24} switcher
import { Suspense } from 'react'
import { after } from 'next/server'
import { cookies } from 'next/headers'
import { logUserAction } from '@/app/utils'

export default function Page() {
  return (
    <>
      <h1>Part of the static shell</h1>
      ``<Suspense>``
    </>
  )
}

async function DynamicContent() {
  const sessionCookie =
    (await cookies()).get('session-id')?.value || 'anonymous'

  // Schedule work after the response is sent
  after(() => {
    logUserAction({ sessionCookie })
  })

  return <p>Your session: {sessionCookie}</p>
}
```

In this example, `<h1>` and the `<Suspense>` fallback are included in the static shell. `DynamicContent` reads the cookie during rendering and passes it into `after` via closure. Since `cookies()` is called *outside* the `after` callback (during the component's render), this works correctly.

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Learn how to configure `after` when self-hosting Next.js.

Reference: supporting `after` for serverless platforms

Version History

Version	Changes
`v15.1.0`	`after` became stable.
`v15.0.0-rc`	`unstable_after` introduced.

cacheLife

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/cacheLife
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to use the cacheLife function to set the cache expiration time for a cached function or component.*

The `cacheLife` function is used to set the cache lifetime of a function or component. It should be used alongside the `use cache` directive, and within the scope of the function or component.

Usage

Basic setup

To use `cacheLife`, first enable the `cacheComponents` flag in your `next.config.js` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

`cacheLife` can only be used within a cache directive scope.

Add a cache directive (for example, `use cache`) at the file level or at the top of an async function or component.

`cacheLife` cannot be used at module scope. Calling it at the top level of a file will throw an error.

> **Good to know**:
>
> - We recommend setting a `cacheLife` in every `use cache` scope so its behavior is clear at the call site. Omitting it leaves the lifetime implicit (the `default` profile applies), which makes it harder to reason about, for example in nested cached scopes.
> - Call `cacheLife` in the same function or component where caching is defined. Avoid abstracting it into shared utilities so the cache behavior remains explicit and easy to reason about.
> - If you call `cacheLife`, ensure only one call executes per function invocation. You can call it in different control flow branches, but only one should run per request. See the conditional cache lifetimes example.

Using preset profiles

Next.js provides preset cache profiles that cover common caching needs. Each profile balances three factors:

- How long users see cached content without checking for updates (client-side)
- How often fresh content is generated on the server
- When old content expires completely

Choose a profile based on how frequently your content changes:

- `seconds` - Real-time data (stock prices, live scores)

- `minutes` - Frequently updated (social feeds, news)
- `hours` - Multiple daily updates (product inventory, weather)
- `days` - Daily updates (blog posts, articles)
- `weeks` - Weekly updates (podcasts, newsletters)
- `max` - Rarely changes (legal pages, archived content)

Import `cacheLife` and pass a profile name:

```
// Archivo: app/blog/page.tsx highlight={1,5}
'use cache'
import { cacheLife } from 'next/cache'

export default async function BlogPage() {
  cacheLife('days') // Blog content updated daily

  const posts = await getBlogPosts()
  return <div></div>
}
```

The profile name tells Next.js how to cache the entire function's output. If you don't call `cacheLife`, the `default` profile is used. See preset cache profiles for timing details.

Reference

Cache profile properties

Cache profiles control caching behavior through three timing properties:

- `stale`: How long the client can use cached data without checking the server
- `revalidate`: After this time, the next request will trigger a background refresh
- `expire`: After this time with no requests, the next one waits for fresh content

`stale`

Client-side: How long the client can use cached data without checking the server.

During this time, the client-side router displays cached content immediately without any network request. After this period expires, the router must check with the server on the next navigation or request. This provides instant page loads from the client cache, but data may be outdated.

- If omitted, defaults to the `default` profile's `stale` value (5 minutes, see `staleTimes`)
- Also determines whether the content can be part of the route's App Shell. See Prerendering behavior.

```
cacheLife({ stale: 300 }) // 5 minutes
```

`revalidate`

How often the server regenerates cached content in the background.

- When a request arrives after this period, the server:
  1. Serves the cached version immediately (if available)
  2. Regenerates content in the background
  3. Updates the cache with fresh content
- Similar to Incremental Static Regeneration (ISR)
- If omitted, defaults to the `default` profile's `revalidate` value (15 minutes)

```
cacheLife({ revalidate: 900 }) // 15 minutes
```

`expire`

Maximum time before the server must regenerate cached content.

- After this period with no traffic, the server regenerates content synchronously on the next request
- When you set both `revalidate` and `expire`, `expire` must be longer than `revalidate`. Next.js validates this and raises an error for invalid configurations.
- If omitted, defaults to the `default` profile's `expire` value (never expires)

```
cacheLife({ expire: 3600 }) // 1 hour
```

Preset cache profiles

If you don't specify a profile, Next.js uses the `default` profile. We recommend explicitly setting a profile to make caching behavior clear.

Profile	Use Case	`stale`	`revalidate`	`expire`
`default`	Standard content	5 minutes	15 minutes	never
`seconds`	Real-time data	30 seconds	1 second	1 minute
`minutes`	Frequently updated content	5 minutes	1 minute	1 hour
`hours`	Content updated multiple times per day	5 minutes	1 hour	1 day
`days`	Content updated daily	5 minutes	1 day	1 week
`weeks`	Content updated weekly	5 minutes	1 week	30 days
`max`	Stable content that rarely changes	5 minutes	30 days	1 year

Overriding the default cache profiles

The preset profiles map to familiar time periods (`seconds`, `minutes`, `hours`, `days`, `weeks`), alongside `default` and `max`. You can use these as they are, or redefine any of them, including `default` and `max`, in `next.config.ts`. The built-in names keep working with editor autocomplete.

Redefining a built-in is a supported pattern, but document it in your project so a call like `cacheLife('hours')` reflects the values you set, not the presets. The time-named profiles carry an intuitive expectation (`days` reads as roughly 24 hours), so redefining them is more likely to surprise a reader than redefining `default` or `max`, which don't imply a specific duration. An equally valid approach is to define your own custom profile rather than overloading a built-in name.

Redefining `default` also changes the lifetime applied when a `use cache` scope calls no `cacheLife`. The example below sets `default` to a one hour revalidate:

```
// Archivo: next.config.ts
const nextConfig = {
  cacheComponents: true,
  cacheLife: {
    // Redefine the 'default' profile
    default: {
      stale: 300, // 5 minutes
      revalidate: 3600, // 1 hour
      expire: 86400, // 1 day
    },
  },
}

export default nextConfig
```

> **Good to know:** The `cacheLife` function's type signature is generated from `next.config.ts` during `next dev`, `next build`, or `next typegen`, so an overridden profile's editor autocomplete and JSDoc hint reflect the values you set, not the presets.

Custom cache profiles

Define reusable cache profiles with your own names in `next.config.ts`:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
  cacheLife: {
    biweekly: {
      stale: 60 * 60 * 24 * 14, // 14 days
      revalidate: 60 * 60 * 24, // 1 day
      expire: 60 * 60 * 24 * 14, // 14 days
    },
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
const nextConfig = {
  cacheComponents: true,
  cacheLife: {
    biweekly: {
      stale: 60 * 60 * 24 * 14, // 14 days
      revalidate: 60 * 60 * 24, // 1 day
      expire: 60 * 60 * 24 * 14, // 14 days
    },
  },
}

module.exports = nextConfig
```

The example above caches for 14 days, checks for updates daily, and expires the cache after 14 days. You can then reference this profile throughout your application by its name:

> **Good to know**: Any omitted properties in a custom profile inherit from the `default` profile. This also applies to inline profile objects passed directly to `cacheLife()`.

```
// Archivo: app/page.tsx highlight={5}
'use cache'
import { cacheLife } from 'next/cache'

export default async function Page() {
  cacheLife('biweekly')
  return <div>Page</div>
}
```

Inline cache profiles

For one-off cases, pass a profile object directly to `cacheLife`:

```
// Archivo: app/page.tsx
'use cache'
import { cacheLife } from 'next/cache'

export default async function Page() {
  cacheLife({
    stale: 3600,
    revalidate: 900,
    expire: 86400,
  })

  return <div>Page</div>
}
```

Inline profiles apply only to the specific function or component. For reusable configurations, define custom profiles in `next.config.ts`.

Using `cacheLife({})` with an empty object applies the `default` profile values.

Client cache behavior

The `stale` property controls the Client Cache, not the `Cache-Control` header:

- The server sends the stale time via the `x-nextjs-stale-time` response header
- The client router uses this value to determine when to revalidate
- **Minimum of 30 seconds is enforced** to ensure prefetched links remain usable

This 30-second minimum prevents prefetched data from expiring before users can click on links. It only applies to time-based expiration.

When you call revalidation functions from a Server Action (`revalidateTag` , `revalidatePath` , `updateTag` , or `refresh`), the entire client cache is immediately cleared, bypassing the stale time.

> **Good to know**: The `stale` property in `cacheLife` differs from `staleTimes` . While `staleTimes` is a global setting affecting all routes, `cacheLife` allows per-function or per-route configuration. Updating `staleTimes.static` also updates the `stale` value of the `default` cache profile.

Prerendering behavior

A short cache lifetime changes where the cached content can be delivered from:

- `revalidate` **of** `0` **, or** `expire` **under 5 minutes**: excluded from prerenders, becoming a "dynamic hole" resolved at request time.
- `stale` **under 30 seconds**: excluded from prerenders, because a prefetch would expire before the user could click.
- `stale` **of at least 30 seconds but under 5 minutes**: included in prerenders, but excluded from the route's App Shell.

Of the presets, only `seconds` falls under any of these thresholds: its `expire` of 1 minute excludes it from prerenders.

This behavior allows you to mix static and dynamic content within the same page. Static parts are prerendered, while short-lived caches create boundaries where data is fetched at request time rather than build time. Use a `````) }

````
> **Note:** This example uses `"use cache: remote"` because runtime caching in serverless deployments doesn't persist across requests with the default in-memory cache. For self-hosted environments, `"use cache"` may be sufficient. See [Runtime caching considerations](/docs/app/api-reference/directives/use-cache#runtime-caching-considerations) for more details.

### Conditional cache lifetimes

You can call `cacheLife` conditionally in different code paths to set different cache durations based on your application logic:

```tsx
// Archivo: lib/posts.ts highlight={14,19}
import { cacheLife, cacheTag } from 'next/cache'

async function getPostContent(slug: string) {
  'use cache'

  const post = await fetchPost(slug)

  // Tag the cache entry for targeted revalidation
  cacheTag(`post-${slug}`)

  if (!post) {
    // Content may not be published yet or could be in draft
    // Cache briefly to reduce database load
    cacheLife('minutes')
    return null
  }

  // Published content can be cached longer
  cacheLife('days')

  // Return only the necessary data to keep cache size minimal
  return post.data
}
```
````

This pattern is useful when different outcomes need different cache durations, for example, when an item is missing but is likely to be available later.

Using dynamic cache lifetimes from data

If you want to calculate cache lifetime at runtime, for example by reading it from the fetched data, use an inline cache profile object:

```tsx
// Archivo: lib/posts.ts highlight={15,16,17,18}
import { cacheLife, cacheTag } from 'next/cache'

async function getPostContent(slug: string) {
  'use cache'

  const post = await fetchPost(slug)
  cacheTag(`post-${slug}`)

  if (!post) {
    cacheLife('minutes')
    return null
  }

  // Use cache timing from CMS data directly as an object
  cacheLife({
    // Ensure post.revalidateSeconds is a number in seconds
    // stale and expire inherit from 'default' profile
    revalidate: post.revalidateSeconds ?? 3600,
  })

  return post.data
}
```

---

cacheTag

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/cacheTag
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to use the cacheTag function to manage cache invalidation in your Next.js application.*

The `cacheTag` function allows you to tag cached data for on-demand invalidation. By associating tags with cache entries, you can selectively purge or revalidate specific cache entries without affecting other cached data.

Usage

To use `cacheTag`, enable the `cacheComponents` flag in your `next.config.js` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
const nextConfig = {
  cacheComponents: true,
}

module.exports = nextConfig
```

The `cacheTag` function takes one or more string values.

```
// Archivo: app/data.ts switcher
import { cacheTag } from 'next/cache'

export async function getData() {
  'use cache'
  cacheTag('my-data')
  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

```
// Archivo: app/data.js switcher
import { cacheTag } from 'next/cache'

export async function getData() {
  'use cache'
  cacheTag('my-data')
  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

You can then purge the cache on-demand from a Server Function or Route Handler:

- Use `updateTag` for read-your-own-writes scenarios, such as forms and other user-triggered mutations, where a user makes a change and the next read should fetch fresh data immediately. `updateTag` is only available inside Server Functions.
- Use `revalidateTag` when it is acceptable to serve stale data while revalidation happens in the background, or when revalidating from a Route Handler or other context.

For example, this Server Function adds a post and then purges every cache entry tagged `'my-data'` so the next read reflects the change:

```
// Archivo: app/action.ts switcher
'use server'

import { updateTag } from 'next/cache'

export default async function submit() {
  await addPost()
  updateTag('my-data')
}
```

```
// Archivo: app/action.js switcher
'use server'

import { updateTag } from 'next/cache'

export default async function submit() {
  await addPost()
  updateTag('my-data')
}
```

Good to know

- **Idempotent Tags**: Applying the same tag multiple times has no additional effect.
- **Multiple Tags**: You can assign multiple tags to a single cache entry by passing multiple string values to `cacheTag`.

```
cacheTag('tag-one', 'tag-two')
```

- **Limits**: A single `cacheTag()` call accepts up to 128 tags, each with a maximum length of 256 characters. Tags longer than 256 characters are skipped, and any tags past the 128th in one call are dropped. Both cases log a console warning.

Examples

Tagging components or functions

Tag your cached data by calling `cacheTag` within a cached function or component:

```
// Archivo: app/components/bookings.tsx switcher
import { cacheTag } from 'next/cache'

interface BookingsProps {
  type: string
}

export async function Bookings({ type = 'haircut' }: BookingsProps) {
  'use cache'
  cacheTag('bookings-data')

  async function getBookingsData() {
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    return data
  }

  return // ...
}
```

```
// Archivo: app/components/bookings.js switcher
import { cacheTag } from 'next/cache'

export async function Bookings({ type = 'haircut' }) {
  'use cache'
  cacheTag('bookings-data')

  async function getBookingsData() {
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    return data
  }

  return // ...
}
```

Creating tags from external data

You can use the data returned from an async function to tag the cache entry.

```
// Archivo: app/components/bookings.tsx switcher
import { cacheTag } from 'next/cache'

interface BookingsProps {
  type: string
}

export async function Bookings({ type = 'haircut' }: BookingsProps) {
  async function getBookingsData() {
    'use cache'
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    cacheTag('bookings-data', data.id)
    return data
  }
  return // ...
}
```

```
// Archivo: app/components/bookings.js switcher
import { cacheTag } from 'next/cache'

export async function Bookings({ type = 'haircut' }) {
  async function getBookingsData() {
    'use cache'
    const response = await fetch(
      `https://api.example.com/bookings?type=${encodeURIComponent(type)}`
    )
    const data = await response.json()
    cacheTag('bookings-data', data.id)
    return data
  }
  return // ...
}
```

Invalidating tagged cache

Using `revalidateTag`, you can invalidate the cache for a specific tag when needed:

```
// Archivo: app/actions.ts switcher
'use server'

import { revalidateTag } from 'next/cache'

export async function updateBookings() {
  await updateBookingData()
  revalidateTag('bookings-data', 'max')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { revalidateTag } from 'next/cache'

export async function updateBookings() {
  await updateBookingData()
  revalidateTag('bookings-data', 'max')
}
```

---

catchError

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/catchError
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the catchError function.*

The `catchError` function creates a component that wraps its children in an error boundary. It provides a programmatic alternative to the `error.js` file convention, enabling component-level error recovery anywhere in your component tree.

Compared to a custom React error boundary, `catchError` is designed to work with Next.js out of the box:

- **Built-in error recovery** — `retry()` re-renders the page inside a Transition, preserving Client Components state outside of the error boundary.
- **Framework-aware integration** — APIs like `redirect()` and `notFound()` work by throwing special errors under the hood. `catchError` handles these seamlessly, so they're not accidentally caught by your error boundary.
- **Client navigation handling** — The error state automatically clears when you do a client navigation to a different route.

`catchError` can be called from Client Components.

```
// Archivo: app/custom-error-boundary.tsx switcher
'use client'

import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(props: { title: string }, { error, retry }: ErrorInfo) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

```
// Archivo: app/custom-error-boundary.js switcher
'use client'

import { catchError } from 'next/error'

function ErrorFallback(props, { error, retry }) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

📘 **Nota para Pages Router:** The `catchError` function creates a component that wraps its children in an error boundary. It provides a programmatic alternative to writing a custom React error boundary class, enabling component-level error recovery anywhere in your component tree.

Compared to a custom React error boundary, `catchError` is designed to work with Next.js out of the box:

- **Built-in error recovery** — `reset()` re-renders the error boundary's children, letting users recover from errors without a full page reload.
- **Client navigation handling** — The error state automatically clears when you do a client navigation to a different route.

```
import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(props: { title: string }, { error, reset }: ErrorInfo) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

```
import { catchError } from 'next/error'

function ErrorFallback(props, { error, reset }) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

Reference

Parameters

`catchError` accepts a single argument:

```
const ErrorWrapper = catchError(fallback)
```

`fallback`

A function that renders the error UI when an error is caught. It receives two arguments:

- `props` — The props passed to the wrapper component (excluding `children`).
- `errorInfo` — An object containing information about the error:

Property	Type	Description
`error`	`Error`	The error instance that was caught.
`retry`	`() ⇒ void`	Re-fetches and re-renders the error boundary's children. If successful, the fallback is replaced with the re-rendered result.
`reset`	`() ⇒ void`	Resets the error state and re-renders without re-fetching. Use `retry()` in most cases.

The `fallback` function must be a Client Component (or defined in a `'use client'` module).

> 📘 **Nota para Pages Router:**
>
> | Property | Type | Description |
> |---|---|---|
> | `error` | `Error` | The error instance that was caught. |
> | `reset` | `() ⇒ void` | Resets the error state and re-renders the error boundary's children. |

Returns

`catchError` returns a React component that:

- Accepts the same props as your fallback's first argument, plus `children`.
- Wraps `children` in an error boundary.
- Renders the `fallback` when an error is caught in `children`.

Examples

Basic usage

Define a fallback and use the returned component to wrap parts of your UI:

```
// Archivo: app/some-component.tsx switcher
import ErrorWrapper from '../custom-error-boundary'

export default function Component({ children }: { children: React.ReactNode }) {
  return <ErrorWrapper title="Dashboard Error">{children}</ErrorWrapper>
}
```

```
// Archivo: app/some-component.js switcher
import ErrorWrapper from '../custom-error-boundary'

export default function Component({ children }) {
  return <ErrorWrapper title="Dashboard Error">{children}</ErrorWrapper>
}
```

📘 **Nota para Pages Router:**

```
import ErrorWrapper from './custom-error-boundary'

export default function Component({ children }: { children: React.ReactNode }) {
  return <ErrorWrapper title="Dashboard Error">{children}</ErrorWrapper>
}
```

```
import ErrorWrapper from './custom-error-boundary'

export default function Component({ children }) {
  return <ErrorWrapper title="Dashboard Error">{children}</ErrorWrapper>
}
```

Recovering from errors

Use `retry()` to prompt the user to recover from the error. When called, the function re-fetches and re-renders the error boundary's children. If successful, the fallback is replaced with the re-rendered result.

In most cases, use `retry()` instead of `reset()`. The `reset()` function only clears the error state and re-renders without re-fetching, which means it won't recover from Server Component errors.

```
// Archivo: app/custom-error-boundary.tsx switcher
'use client'

import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(props: {}, { error, retry, reset }: ErrorInfo) {
  return (
    <div>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
      <button onClick={() => reset()}>Reset</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

```
// Archivo: app/custom-error-boundary.js switcher
'use client'

import { catchError } from 'next/error'

function ErrorFallback(props, { error, retry, reset }) {
  return (
    <div>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try again</button>
      <button onClick={() => reset()}>Reset</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

📘 **Nota para Pages Router:** Use `reset()` to prompt the user to recover from the error. When called, the function clears the error state and re-renders the error boundary's children.

```
import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(props: {}, { error, reset }: ErrorInfo) {
  return (
    <div>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

```
import { catchError } from 'next/error'

function ErrorFallback(props, { error, reset }) {
  return (
    <div>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  )
}

export default catchError(ErrorFallback)
```

Server-rendered error fallback

You can pass server-rendered content as a prop to display data-driven fallback UI. This works by rendering a Server Component as a `React.ReactNode` prop that the fallback displays when an error is caught.

Good to know: This pattern eagerly renders the fallback on every page render, even when no error occurs. For most use cases, a simpler client-side fallback is sufficient.

```
// Archivo: app/error-boundary.tsx switcher
'use client'

import { catchError, type ErrorInfo } from 'next/error'

function ErrorFallback(
  props: { fallback: React.ReactNode },
  errorInfo: ErrorInfo
) {
  return props.fallback
}

export default catchError(ErrorFallback)
```

```
// Archivo: app/error-boundary.js switcher
'use client'

import { catchError } from 'next/error'

function ErrorFallback(props, errorInfo) {
  return props.fallback
}

export default catchError(ErrorFallback)
```

```
// Archivo: app/some-component.tsx switcher
import ErrorBoundary from '../error-boundary'

async function ErrorFallback() {
  const data = await getData()
  return <div>{data.message}</div>
}

export default function Component({ children }: { children: React.ReactNode }) {
  return <ErrorBoundary fallback={<ErrorFallback />}>{children}</ErrorBoundary>
}
```

```
// Archivo: app/some-component.js switcher
import ErrorBoundary from '../error-boundary'

async function ErrorFallback() {
  const data = await getData()
  return <div>{data.message}</div>
}

export default function Component({ children }) {
  return <ErrorBoundary fallback={<ErrorFallback />}>{children}</ErrorBoundary>
}
```

Good to know:

- Unlike the `error.js` file convention which is scoped to route segments, `catchError` can be used to wrap any part of your component tree for component-level error recovery.
- Props passed to the wrapper component are forwarded to the fallback function, making it easy to create reusable error UIs with different configurations.
- You don't need to wrap `error.js` default exports with `catchError`. The `error.js` file convention already renders inside a built-in error boundary provided by Next.js.

📘 **Nota para Pages Router:**

> **Good to know**: Props passed to the wrapper component are forwarded to the fallback function, making it easy to create reusable error UIs with different configurations.

Version History

Version	Changes
`v16.3.0`	`catchError` became stable.
`v16.2.0`	`unstable_catchError` introduced.

connection

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/connection
📂 **Sección:** App > Api Reference > Functions
📝 **Descripción:** *API Reference for the connection function.*

The `connection()` function allows you to indicate rendering should wait for an incoming user request before continuing.

It's useful when a component doesn't use Request-time APIs like `cookies` or `headers`, but still needs to produce different output per request, such as `Math.random()` or `new Date()`.

```
// Archivo: app/page.tsx switcher
import { connection } from 'next/server'

export default async function Page() {
  await connection() // prerendering stops here
  // the following code only runs at request time
  const rand = Math.random()
  return <span>{rand}</span>
}
```

```
// Archivo: app/page.js switcher
import { connection } from 'next/server'

export default async function Page() {
  await connection() // prerendering stops here
  // the following code only runs at request time
  const rand = Math.random()
  return <span>{rand}</span>
}
```

Examples

Synchronous database drivers

Queries from synchronous database drivers like `better-sqlite3` complete during prerendering. If you are not already using Request-time APIs, call `connection()` before your query to exclude them from prerendering:

```
// Archivo: app/lib/data.ts switcher
import { connection } from 'next/server'
import Database from 'better-sqlite3'

const db = new Database('app.db')

export async function getVisitorCount() {
  await connection()
  return db.prepare('SELECT value FROM counters WHERE name = ?').get('visitors')
}
```

```
// Archivo: app/lib/data.js switcher
import { connection } from 'next/server'
import Database from 'better-sqlite3'

const db = new Database('app.db')

export async function getVisitorCount() {
  await connection()
  return db.prepare('SELECT value FROM counters WHERE name = ?').get('visitors')
}
```

Now any component that calls `getVisitorCount()` will be excluded from prerendering, along with the rest of its output.

Reference

Type

```
function connection(): Promise<void>
```

Parameters

- The function does not accept any parameters.

Returns

- The function returns a `void` Promise. It is not meant to be consumed.

Good to know

- `connection` replaces `unstable_noStore` to better align with the future of Next.js.
- The function is only necessary when dynamic rendering is required and common Request-time APIs are not used.
- With Cache Components, prefer `io()` for excluding content from the static shell. It works the same way but can also be cached and prefetched. Reach for `connection()` only when rendering should wait for a real user request.

Version History

Version	Changes
`v15.0.0`	`connection` stabilized.
`v15.0.0-RC`	`connection` introduced.

cookies

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/cookies
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the cookies function.*

`cookies` is an **async** function that allows you to read the HTTP incoming request cookies in Server Components, and read/write outgoing request cookies in Server Functions or Route Handlers.

```
// Archivo: app/page.tsx switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

```
// Archivo: app/page.js switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

Reference

Methods

The following methods are available:

Method	Return Type	Description
`get('name')`	Object	Accepts a cookie name and returns an object with the name and value.
`getAll()`	Array of objects	Returns a list of all the cookies with a matching name.
`has('name')`	Boolean	Accepts a cookie name and returns a boolean based on if the cookie exists.
`set(name, value, options)`	-	Accepts a cookie name, value, and options and sets the outgoing request cookie.
`delete(name)`	-	Accepts a cookie name and deletes the cookie.
`toString()`	String	Returns a string representation of the cookies.

Options

When setting a cookie, the following properties from the `options` object are supported:

Option	Type	Description
`name`	String	Specifies the name of the cookie.
`value`	String	Specifies the value to be stored in the cookie.
`expires`	Date	Defines the exact date when the cookie will expire.
`maxAge`	Number	Sets the cookie's lifespan in seconds.
`domain`	String	Specifies the domain where the cookie is available.
`path`	String, default: `'/'`	Limits the cookie's scope to a specific path within the domain.
`secure`	Boolean	Ensures the cookie is sent only over HTTPS connections for added security.
`httpOnly`	Boolean	Restricts the cookie to HTTP requests, preventing client-side access.
`sameSite`	Boolean, `'lax'`, `'strict'`, `'none'`	Controls the cookie's cross-site request behavior.
`priority`	String (`"low"`, `"medium"`, `"high"`)	Specifies the cookie's priority
`partitioned`	Boolean	Indicates whether the cookie is partitioned.

The only option with a default value is `path`.

To learn more about these options, see the MDN docs.

Good to know

- `cookies` is an **asynchronous** function that returns a promise. You must use `async/await` or React's `use` function to access cookies.
  - In version 14 and earlier, `cookies` was a synchronous function. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.
- `cookies` is a Request-time API whose returned values cannot be known ahead of time. Using it in a layout or page will opt a route into dynamic rendering.
- With Cache Components, calling `cookies()` outside of a `<Suspense>` boundary prevents the route from being prerendered. See Next.js encountered runtime data during prerendering for fix options.
- The `.delete` method can only be called:

- In a Server Function or Route Handler.
- If it belongs to the same domain from which `.set` is called. For wildcard domains, the specific subdomain must be an exact match. Additionally, the code must be executed on the same protocol (HTTP or HTTPS) as the cookie you want to delete.

- HTTP does not allow setting cookies after streaming starts, so you must use `.set` in a Server Function or Route Handler.

Understanding Cookie Behavior in Server Components

When working with cookies in Server Components, it's important to understand that cookies are fundamentally a client-side storage mechanism:

- **Reading cookies** works in Server Components because you're accessing the cookie data that the client's browser sends to the server in the HTTP request headers.
- **Setting cookies** is not supported during Server Component rendering. To modify cookies, invoke a Server Function from the client or use a Route Handler.

The server can only send instructions (via `Set-Cookie` headers) to tell the browser to store cookies - the actual storage happens on the client side. This is why cookie operations that modify state (`.set` , `.delete`) must be performed in a Server Function or Route Handler where the response headers can be properly set.

Understanding Cookie Behavior in Server Functions

After you set or delete a cookie in a Server Function, Next.js can return both the updated UI and new data in a single server roundtrip when the function is used as a Server Action (e.g., passed to a form's `action` prop). See Caching and Revalidating.

The UI is not unmounted, but effects that depend on data coming from the server will re-run.

To refresh cached data too, call `revalidatePath` or `revalidateTag` inside the function.

Examples

Getting a cookie

You can use the `(await cookies()).get('name')` method to get a single cookie:

```
// Archivo: app/page.tsx switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

```
// Archivo: app/page.js switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get('theme')
  return '...'
}
```

Getting all cookies

You can use the `(await cookies()).getAll()` method to get all cookies with a matching name. If `name` is unspecified, it returns all the available cookies.

```tsx
// Archivo: app/page.tsx switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  return cookieStore.getAll().map((cookie) => (
    <div key={cookie.name}>
      <p>Name: {cookie.name}</p>
      <p>Value: {cookie.value}</p>
    </div>
  ))
}
```

```jsx
// Archivo: app/page.js switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  return cookieStore.getAll().map((cookie) => (
    <div key={cookie.name}>
      <p>Name: {cookie.name}</p>
      <p>Value: {cookie.value}</p>
    </div>
  ))
}
```

Setting a cookie

You can use the `(await cookies()).set(name, value, options)` method in a Server Function or Route Handler to set a cookie. The `options` object is optional.

```tsx
// Archivo: app/actions.ts switcher
'use server'

import { cookies } from 'next/headers'

export async function create(data) {
  const cookieStore = await cookies()

  cookieStore.set('name', 'lee')
  // or
  cookieStore.set('name', 'lee', { secure: true })
  // or
  cookieStore.set({
    name: 'name',
    value: 'lee',
    httpOnly: true,
    path: '/',
  })
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { cookies } from 'next/headers'

export async function create(data) {
  const cookieStore = await cookies()

  cookieStore.set('name', 'lee')
  // or
  cookieStore.set('name', 'lee', { secure: true })
  // or
  cookieStore.set({
    name: 'name',
    value: 'lee',
    httpOnly: true,
    path: '/',
  })
}
```

Checking if a cookie exists

You can use the `(await cookies()).has(name)` method to check if a cookie exists:

```
// Archivo: app/page.ts switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const hasCookie = cookieStore.has('theme')
  return '...'
}
```

```
// Archivo: app/page.js switcher
import { cookies } from 'next/headers'

export default async function Page() {
  const cookieStore = await cookies()
  const hasCookie = cookieStore.has('theme')
  return '...'
}
```

Deleting cookies

There are three ways you can delete a cookie.

Using the `delete()` method:

```
// Archivo: app/actions.ts switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.delete('name')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.delete('name')
}
```

Setting a new cookie with the same name and an empty value:

```
// Archivo: app/actions.ts switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.set('name', '')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.set('name', '')
}
```

Setting the `maxAge` to 0 will immediately expire a cookie. `maxAge` accepts a value in seconds.

```
// Archivo: app/actions.ts switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.set('name', 'value', { maxAge: 0 })
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { cookies } from 'next/headers'

export async function deleteCookie(data) {
  const cookieStore = await cookies()
  cookieStore.set('name', 'value', { maxAge: 0 })
}
```

Version History

Version	Changes
`v15.0.0-RC`	`cookies` is now an async function. A codemod is available.
`v13.0.0`	`cookies` introduced.

draftMode

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/draft-mode
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the draftMode function.*

`draftMode` is an **async** function allows you to enable and disable Draft Mode, as well as check if Draft Mode is enabled in a Server Component.

```
// Archivo: app/page.ts switcher
import { draftMode } from 'next/headers'

export default async function Page() {
  const { isEnabled } = await draftMode()
}
```

```
// Archivo: app/page.js switcher
import { draftMode } from 'next/headers'

export default async function Page() {
  const { isEnabled } = await draftMode()
}
```

Reference

The following methods and properties are available:

Method	Description
`isEnabled`	A boolean value that indicates if Draft Mode is enabled.
`enable()`	Enables Draft Mode in a Route Handler by setting a cookie (`__prerender_bypass`).
`disable()`	Disables Draft Mode in a Route Handler by deleting a cookie.

Good to know

- `draftMode` is an **asynchronous** function that returns a promise. You must use `async/await` or React's `use` function.
  - In version 14 and earlier, `draftMode` was a synchronous function. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.
- A new bypass cookie value will be generated each time you run `next build`. This ensures that the bypass cookie can't be guessed.
- To test Draft Mode locally over HTTP, your browser will need to allow third-party cookies and local storage access.
- `isEnabled` is readable inside a caching directive scope. Other runtime APIs like `cookies()` and `headers()` are not allowed inside caching directive scopes, even when Draft Mode is active.
- Calling `enable()` or `disable()` inside a caching directive scope will throw an error.
- When Draft Mode is enabled, all functions and components under a caching directive scope re-execute on every request and results are not saved to the cache. This ensures draft content is always fresh.

Examples

Enabling Draft Mode

To enable Draft Mode, create a new Route Handler and call the `enable()` method:

```
// Archivo: app/draft/route.ts switcher
import { draftMode } from 'next/headers'

export async function GET(request: Request) {
  const draft = await draftMode()
  draft.enable()
  return new Response('Draft mode is enabled')
}
```

```
// Archivo: app/draft/route.js switcher
import { draftMode } from 'next/headers'

export async function GET(request) {
  const draft = await draftMode()
  draft.enable()
  return new Response('Draft mode is enabled')
}
```

Disabling Draft Mode

By default, the Draft Mode session ends when the browser is closed.

To disable Draft Mode manually, call the `disable()` method in your Route Handler:

```
// Archivo: app/draft/route.ts switcher
import { draftMode } from 'next/headers'

export async function GET(request: Request) {
  const draft = await draftMode()
  draft.disable()
  return new Response('Draft mode is disabled')
}
```

```
// Archivo: app/draft/route.js switcher
import { draftMode } from 'next/headers'

export async function GET(request) {
  const draft = await draftMode()
  draft.disable()
  return new Response('Draft mode is disabled')
}
```

Then, send a request to invoke the Route Handler. If calling the route using the `<Link>` component, you must pass `prefetch={false}` to prevent accidentally deleting the cookie on prefetch.

Checking if Draft Mode is enabled

You can check if Draft Mode is enabled in a Server Component with the `isEnabled` property:

```
// Archivo: app/page.ts switcher
import { draftMode } from 'next/headers'

export default async function Page() {
  const { isEnabled } = await draftMode()
  return (
    <main>
      <h1>My Blog Post</h1>
      <p>Draft Mode is currently {isEnabled ? 'Enabled' : 'Disabled'}</p>
    </main>
  )
}
```

```
// Archivo: app/page.js switcher
import { draftMode } from 'next/headers'

export default async function Page() {
  const { isEnabled } = await draftMode()
  return (
    <main>
      <h1>My Blog Post</h1>
      <p>Draft Mode is currently {isEnabled ? 'Enabled' : 'Disabled'}</p>
    </main>
  )
}
```

Version History

Version	Changes
`v15.0.0-RC`	`draftMode` is now an async function. A codemod is available.
`v13.4.0`	`draftMode` introduced.

fetch

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/fetch
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API reference for the extended fetch function.*

Next.js extends the Web `fetch()` API to allow each request on the server to set its own persistent caching and revalidation semantics.

In the browser, the `cache` option indicates how a fetch request will interact with the *browser's* HTTP cache. With this extension, `cache` indicates how a *server-side* fetch request will interact with the framework's persistent cache.

You can call `fetch` with `async` and `await` directly within Server Components.

```
// Archivo: app/page.tsx switcher
export default async function Page() {
  let data = await fetch('https://api.vercel.app/blog')
  let posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/page.js switcher
export default async function Page() {
  let data = await fetch('https://api.vercel.app/blog')
  let posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

`fetch(url, options)`

Since Next.js extends the Web `fetch()` API, you can use any of the native options available.

`options.cache`

Configure how the request should interact with Next.js caching.

```
fetch(`https://...`, { cache: 'force-cache' | 'no-store' })
```

- **`auto no cache`** (default): Next.js fetches the resource from the remote server on every request in development, but will fetch once during `next build` because the route will be statically prerendered. If Request-time APIs are detected on the route, Next.js will fetch the resource on every request.
- **`no-store`**: Next.js fetches the resource from the remote server on every request, even if Request-time APIs are not detected on the route.
- **`force-cache`**: Next.js looks for a matching request in its server-side cache. A request matches on its URL, method, headers, and body, so requests that differ in any of these are cached separately.
  - If there is a match and it is fresh, it will be returned from the cache.
  - If there is no match or a stale match, Next.js fetches the resource from the remote server and updates the cache with the downloaded resource. Only responses with a `200` HTTP status code are stored.

> **Good to know**: Caching is opt-in. Set `cache: 'force-cache'` to cache any request, including `POST` and requests that send `authorization` or `cookie` headers. Draft Mode bypasses the cache entirely (no read or write).

`options.next.revalidate`

```
fetch(`https://...`, { next: { revalidate: false | 0 | number } })
```

Set the cache lifetime of a resource (in seconds).

- **`false`** - Cache the resource indefinitely. Semantically equivalent to `revalidate: Infinity`. The HTTP cache may evict older resources over time.
- **`0`** - Prevent the resource from being cached.
- **`number`** - (in seconds) Specify the resource should have a cache lifetime of at most `n` seconds.

> **Good to know**:
>
> - If an individual `fetch()` request sets a `revalidate` number lower than the default `revalidate` of a route, the whole route revalidation interval will be decreased.
> - If two fetch requests with the same URL in the same route have different `revalidate` values, the lower value will be used.
> - Conflicting options such as `{ revalidate: 3600, cache: 'no-store' }` are not allowed, both will be ignored, and in development mode a warning will be printed to the terminal.

`options.next.tags`

```
fetch(`https://...`, { next: { tags: ['collection'] } })
```

Set the cache tags of a resource. Data can then be revalidated on-demand using `revalidateTag`. The max length for a custom tag is 256 characters and the max tag items is 128.

Memoization

`fetch` requests using `GET` with the same URL and options are automatically memoized during a server render pass. If you call the same `fetch` in multiple Server Components, layouts, pages, `generateStaticParams` and `generateViewport`, Next.js executes it only once and shares the result. This is separate from persistent caching: memoization lasts only for a single render pass, while cached responses persist across requests.

To opt out, pass an `AbortController` signal to `fetch`:

```
const { signal } = new AbortController()
fetch(url, { signal })
```

> **Good to know**: Memoization does not apply in Route Handlers, since they are not part of the React component tree.

Troubleshooting

Fetch default `auto no cache` and `cache: 'no-store'` not showing fresh data in development

Next.js caches `fetch` responses in Server Components across Hot Module Replacement (HMR) in local development for faster responses and to reduce costs for billed API calls.

By default, the HMR cache applies to all fetch requests, including those with the default `auto no cache` and `cache: 'no-store'` option. This means uncached requests will not show fresh data between HMR refreshes. However, the cache will be cleared on navigation or full-page reloads.

See the `serverComponentsHmrCache` docs for more information.

Hard refresh and caching in development

In development mode, if the request includes the `cache-control: no-cache` header, `options.cache`, `options.next.revalidate`, and `options.next.tags` are ignored, and the `fetch` request is served from the source.

Browsers typically include `cache-control: no-cache` when the cache is disabled in developer tools or during a hard refresh.

Version History

Version	Changes
`v13.0.0`	`fetch` introduced.

forbidden

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/forbidden
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the forbidden function.*

The `forbidden` function throws an error that renders a Next.js 403 page. It's useful for handling authorization errors in your application. You can customize the UI using the `forbidden.js` file.

Invoking `forbidden()` throws a `NEXT_HTTP_ERROR_FALLBACK;403` error and terminates rendering of the route segment where it was thrown. Next.js also injects a `<meta name="robots" content="noindex" />` tag so the page is not indexed. Because it works by throwing, call it in the render path: a component, or a function a component `await`s. A call left in an un-awaited promise throws where nothing catches it, and no forbidden UI renders.

To start using `forbidden`, enable the experimental `authInterrupts` configuration option in your `next.config.js` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    authInterrupts: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  experimental: {
    authInterrupts: true,
  },
}
```

`forbidden` can be invoked in Server Components, Server Functions, and Route Handlers.

```
// Archivo: app/admin/page.tsx switcher
import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'

export default async function AdminPage() {
  const session = await verifySession()

  // Check if the user has the 'admin' role
  if (session.role !== 'admin') {
    forbidden()
  }

  // Render the admin page for authorized users
  return <></>
}
```

```
// Archivo: app/admin/page.js switcher
import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'

export default async function AdminPage() {
  const session = await verifySession()

  // Check if the user has the 'admin' role
  if (session.role !== 'admin') {
    forbidden()
  }

  // Render the admin page for authorized users
  return <></>
}
```

Good to know

- The `forbidden` function cannot be called in the root layout.

- You do not need to write `return forbidden()`. It throws (its TypeScript `never` return type), so execution stops. A `try/catch` around the call suppresses the interrupt and no forbidden UI renders. Use `unstable_rethrow` to let it through.
- A `forbidden()` left in an un-awaited promise throws where nothing catches it, so no forbidden UI renders. In development the server logs `× unhandledRejection: NEXT_HTTP_ERROR_FALLBACK;403`. Always `await` the function that may call it.

Examples

Calling `forbidden()` after streaming has started

To keep the page's shell and loading UI visible while the session is checked, put the role check in the Data Access Layer function that loads the data, and render it in a component wrapped in [`````) }

```
```jsx
// Archivo: app/projects/page.js switcher highlight={8}
import { Suspense } from 'react'
import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'

async function getProjects() {
  const session = await verifySession()
  if (session?.role !== 'admin') {
    forbidden()
  }
  return db.projects.findMany()
}

async function Projects() {
  const projects = await getProjects()
  return (
    <ul>
      {projects.map((project) => (
        <li key={project.id}>{project.name}</li>
      ))}
    </ul>
  )
}

export default function ProjectsPage() {
  return (
    <main>
      <h1>Projects</h1>
      ``<Suspense>``
    </main>
  )
}
```
```

When the session lacks access, `getProjects` calls `forbidden()`, which throws. Because this happens during rendering, the exception propagates to the nearest `forbidden` boundary, which renders in place of the streamed-in content, even though the page shell has already been sent.

Add a `forbidden.tsx` alongside the route to define that UI:

```
// Archivo: app/projects/forbidden.tsx switcher
export default function Forbidden() {
  return (
    <main>
      <h1>403 - Forbidden</h1>
      <p>You don't have access to this page.</p>
    </main>
  )
}
```

```
// Archivo: app/projects/forbidden.js switcher
export default function Forbidden() {
  return (
    <main>
      <h1>403 - Forbidden</h1>
      <p>You don't have access to this page.</p>
    </main>
  )
}
```

The trade-off is the HTTP status code. Because the check runs inside the `<Suspense>` boundary, the response has already begun streaming as a `200`, and the status can't change once streaming has started. This is usually fine for a page, where the user sees the `forbidden` UI regardless. To return a real `403` status, the check has to run before the response streams. With Cache Components, every dynamic route streams a static shell first, so run that check in `proxy` instead. See Status codes.

Role-based route protection

You can use `forbidden` to restrict access to certain routes based on user roles. This ensures that users who are authenticated but lack the required permissions cannot access the route.

```
// Archivo: app/admin/page.tsx switcher
import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'

export default async function AdminPage() {
  const session = await verifySession()

  // Check if the user has the 'admin' role
  if (session.role !== 'admin') {
    forbidden()
  }

  // Render the admin page for authorized users
  return (
    <main>
      <h1>Admin Dashboard</h1>
      <p>Welcome, {session.user.name}!</p>
    </main>
  )
}
```

```
// Archivo: app/admin/page.js switcher
import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'

export default async function AdminPage() {
  const session = await verifySession()

  // Check if the user has the 'admin' role
  if (session.role !== 'admin') {
    forbidden()
  }

  // Render the admin page for authorized users
  return (
    <main>
      <h1>Admin Dashboard</h1>
      <p>Welcome, {session.user.name}!</p>
    </main>
  )
}
```

Mutations with Server Actions

When implementing mutations in Server Actions, you can use `forbidden` to only allow users with a specific role to update sensitive data.

```tsx
// Archivo: app/actions/update-role.ts switcher
'use server'

import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'
import db from '@/app/lib/db'

export async function updateRole(formData: FormData) {
  const session = await verifySession()

  // Ensure only admins can update roles
  if (session.role !== 'admin') {
    forbidden()
  }

  // Perform the role update for authorized users
  // ...
}
```

```jsx
// Archivo: app/actions/update-role.js switcher
'use server'

import { verifySession } from '@/app/lib/dal'
import { forbidden } from 'next/navigation'
import db from '@/app/lib/db'

export async function updateRole(formData) {
  const session = await verifySession()

  // Ensure only admins can update roles
  if (session.role !== 'admin') {
    forbidden()
  }

  // Perform the role update for authorized users
  // ...
}
```

Version History

Version	Changes
`v15.1.0`	`forbidden` introduced.

generateImageMetadata

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-image-metadata
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to generate multiple images in a single Metadata API special file.*

You can use `generateImageMetadata` to generate different versions of one image or return multiple images for one route segment. This is useful for when you want to avoid hard-coding metadata values, such as for icons.

Parameters

`generateImageMetadata` function accepts the following parameters:

`params` (optional)

An object containing the dynamic route parameters object from the root segment down to the segment `generateImageMetadata` is called from.

```
// Archivo: icon.tsx switcher
export function generateImageMetadata({
  params,
}: {
  params: { slug: string }
}) {
  // ...
}
```

```
// Archivo: icon.js switcher
export function generateImageMetadata({ params }) {
  // ...
}
```

Route	URL	`params`
`app/shop/icon.js`	`/shop`	`undefined`
`app/shop/[slug]/icon.js`	`/shop/1`	`{ slug: '1' }`
`app/shop/[tag]/[item]/icon.js`	`/shop/1/2`	`{ tag: '1', item: '2' }`

Returns

The `generateImageMetadata` function should return an `array` of objects containing the image's metadata such as `alt` and `size`. In addition, each item **must** include an `id` value which will be passed as a promise to the props of the image generating function.

Image Metadata Object	Type
`id`	`string` (required)
`alt`	`string`
`size`	`{ width: number; height: number }`
`contentType`	`string`

```tsx
// Archivo: icon.tsx switcher
import { ImageResponse } from 'next/og'

export function generateImageMetadata() {
  return [
    {
      contentType: 'image/png',
      size: { width: 48, height: 48 },
      id: 'small',
    },
    {
      contentType: 'image/png',
      size: { width: 72, height: 72 },
      id: 'medium',
    },
  ]
}

export default async function Icon({ id }: { id: Promise<string | number> }) {
  const iconId = await id
  return new ImageResponse(
    (
      <div
        style={{
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          fontSize: 88,
          background: '#000',
          color: '#fafafa',
        }}
      >
        Icon {iconId}
      </div>
    )
  )
}
```

```
// Archivo: icon.js switcher
import { ImageResponse } from 'next/og'

export function generateImageMetadata() {
  return [
    {
      contentType: 'image/png',
      size: { width: 48, height: 48 },
      id: 'small',
    },
    {
      contentType: 'image/png',
      size: { width: 72, height: 72 },
      id: 'medium',
    },
  ]
}

export default async function Icon({ id }) {
  const iconId = await id
  return new ImageResponse(
    (
      <div
        style={{
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          fontSize: 88,
          background: '#000',
          color: '#fafafa',
        }}
      >
        Icon {iconId}
      </div>
    )
  )
}
```

Image generation function props

When using `generateImageMetadata`, the default export image generation function receives the following props:

`id`

A promise that resolves to the `id` value from one of the items returned by `generateImageMetadata`. The `id` will be a `string` or `number` depending on what was returned from `generateImageMetadata`.

```
// Archivo: icon.tsx switcher
export default async function Icon({ id }: { id: Promise<string | number> }) {
  const iconId = await id
  // Use iconId to generate the image
}
```

```
// Archivo: icon.js switcher
export default async function Icon({ id }) {
  const iconId = await id
  // Use iconId to generate the image
}
```

`params` **(optional)**

A promise that resolves to an object containing the dynamic route parameters from the root segment down to the segment the image is colocated in.

```tsx
// Archivo: icon.tsx switcher
export default async function Icon({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // Use slug to generate the image
}
```

```jsx
// Archivo: icon.js switcher
export default async function Icon({ params }) {
  const { slug } = await params
  // Use slug to generate the image
}
```

Examples

Using external data

This example uses the `params` object and external data to generate multiple Open Graph images for a route segment.

```tsx
// Archivo: app/products/[id]/opengraph-image.tsx switcher
import { ImageResponse } from 'next/og'
import { getCaptionForImage, getOGImages } from '@/app/utils/images'

export async function generateImageMetadata({
  params,
}: {
  params: { id: string }
}) {
  const images = await getOGImages(params.id)

  return images.map((image, idx) => ({
    id: idx,
    size: { width: 1200, height: 600 },
    alt: image.text,
    contentType: 'image/png',
  }))
}

export default async function Image({
  params,
  id,
}: {
  params: Promise<{ id: string }>
  id: Promise<number>
}) {
  const productId = (await params).id
  const imageId = await id
  const text = await getCaptionForImage(productId, imageId)

  return new ImageResponse(
    (
      <div
        style={
          {
            // ...
          }
        }
      >
        {text}
      </div>
    )
  )
}
```

```
// Archivo: app/products/[id]/opengraph-image.js switcher
import { ImageResponse } from 'next/og'
import { getCaptionForImage, getOGImages } from '@/app/utils/images'

export async function generateImageMetadata({ params }) {
  const images = await getOGImages(params.id)

  return images.map((image, idx) => ({
    id: idx,
    size: { width: 1200, height: 600 },
    alt: image.text,
    contentType: 'image/png',
  }))
}

export default async function Image({ params, id }) {
  const productId = (await params).id
  const imageId = await id
  const text = await getCaptionForImage(productId, imageId)

  return new ImageResponse(
    (
      <div
        style={
          {
            // ...
          }
        }
      >
        {text}
      </div>
    )
  )
}
```

Version History

Version	Changes
`v16.0.0`	`id` passed to the Image generation function is now a promise that resolves to `string` or `number`
`v16.0.0`	`params` passed to the Image generation function is now a promise that resolves to an object
`v13.3.0`	`generateImageMetadata` introduced.

generateMetadata

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-metadata
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to add Metadata to your Next.js application for improved search engine optimization (SEO) and web shareability.*

You can use the `metadata` object or the `generateMetadata` function to define metadata.

The `metadata` object

To define static metadata, export a `Metadata` object from a `layout.js` or `page.js` file.

```tsx
// Archivo: layout.tsx | page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: '...',
  description: '...',
}

export default function Page() {}
```

```jsx
// Archivo: layout.js | page.js switcher
export const metadata = {
  title: '...',
  description: '...',
}

export default function Page() {}
```

> See the Metadata Fields for a complete list of supported options.

`generateMetadata` function

Dynamic metadata depends on **dynamic information**, such as the current route parameters, external data, or `metadata` in parent segments, can be set by exporting a `generateMetadata` function that returns a `Metadata` object.

Resolving `generateMetadata` is part of rendering the page. If the page can be prerendered and `generateMetadata` doesn't introduce dynamic behavior, the resulting metadata is included in the page's initial HTML.

Otherwise the metadata resolved from `generateMetadata` can be streamed after sending the initial UI.

```tsx
// Archivo: app/products/[id]/page.tsx switcher
import type { Metadata, ResolvingMetadata } from 'next'

type Props = {
  params: Promise<{ id: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export async function generateMetadata(
  { params, searchParams }: Props,
  parent: ResolvingMetadata
): Promise<Metadata> {
  // read route params
  const { id } = await params

  // fetch data
  const product = await fetch(`https://.../${id}`).then((res) => res.json())

  // optionally access and extend (rather than replace) parent metadata
  const previousImages = (await parent).openGraph?.images || []

  return {
    title: product.title,
    openGraph: {
      images: ['/some-specific-page-image.jpg', ...previousImages],
    },
  }
}

export default function Page({ params, searchParams }: Props) {}
```

```
// Archivo: app/products/[id]/page.js switcher
export async function generateMetadata({ params, searchParams }, parent) {
  // read route params
  const { id } = await params

  // fetch data
  const product = await fetch(`https:// ... /${id}`).then((res) => res.json())

  // optionally access and extend (rather than replace) parent metadata
  const previousImages = (await parent).openGraph?.images || []

  return {
    title: product.title,
    openGraph: {
      images: ['/some-specific-page-image.jpg', ...previousImages],
    },
  }
}

export default function Page({ params, searchParams }) {}
```

For type completion of `params` and `searchParams`, you can type the first argument with `PageProps<'/route'>` or `LayoutProps<'/route'>` for pages and layouts respectively.

> **Good to know**:
>
> - Metadata can be added to `layout.js` and `page.js` files.
> - Next.js will automatically resolve the metadata, and create the relevant `<head>` tags for the page.
> - The `metadata` object and `generateMetadata` function exports are **only supported in Server Components**.
> - You cannot export both the `metadata` object and `generateMetadata` function from the same route segment.
> - `fetch` requests inside `generateMetadata` are automatically memoized for the same data across `generateMetadata`, `generateStaticParams`, Layouts, Pages, and Server Components.
> - React `cache` can be used if `fetch` is unavailable.
> - File-based metadata has the higher priority and will override the `metadata` object and `generateMetadata` function.

Why `generateMetadata` is Server Component only

`generateMetadata` and the `metadata` export are only supported in Server Components because metadata must be resolved on the server before the page component is rendered. This allows Next.js to include the metadata in the initial HTML response.

If you need to use Client Component features, keep your `page.tsx` as a Server Component and move the Client Component logic to a separate file:

```
// Archivo: app/page.tsx switcher
import type { Metadata } from 'next'
import { InteractiveComponent } from './interactive-component'

export const metadata: Metadata = {
  title: 'My Page',
}

export default function Page() {
  return <InteractiveComponent />
}
```

```
// Archivo: app/page.js switcher
import { InteractiveComponent } from './interactive-component'

export const metadata = {
  title: 'My Page',
}

export default function Page() {
  return <InteractiveComponent />
}
```

```
// Archivo: app/interactive-component.tsx switcher
'use client'

export function InteractiveComponent() {
  // Client-side interactivity (hooks, event handlers, etc.)
}
```

```
// Archivo: app/interactive-component.js switcher
'use client'

export function InteractiveComponent() {
  // Client-side interactivity (hooks, event handlers, etc.)
}
```

Reference

Parameters

`generateMetadata` function accepts the following parameters:

- `props` - An object containing the parameters of the current route:
  - `params` - An object containing the dynamic route parameters object from the root segment down to the segment `generateMetadata` is called from. Examples:

Route	URL	`params`
`app/shop/[slug]/page.js`	`/shop/1`	`{ slug: '1' }`
`app/shop/[tag]/[item]/page.js`	`/shop/1/2`	`{ tag: '1', item: '2' }`
`app/shop/[...slug]/page.js`	`/shop/1/2`	`{ slug: ['1', '2'] }`

  - `searchParams` - An object containing the current URL's search params. Examples:

URL	`searchParams`
`/shop?a=1`	`{ a: '1' }`
`/shop?a=1&b=2`	`{ a: '1', b: '2' }`
`/shop?a=1&a=2`	`{ a: ['1', '2'] }`

- `parent` - A promise of the resolved metadata from parent route segments.

Returns

`generateMetadata` should return a `Metadata` object containing one or more metadata fields.

Good to know:

- If metadata doesn't depend on request information, it should be defined using the static `metadata` object rather than `generateMetadata`.
- `fetch` requests are automatically memoized for the same data across `generateMetadata`, `generateStaticParams`, Layouts, Pages, and Server Components. React `cache` can be used if `fetch` is unavailable.
- `searchParams` are only available in `page.js` segments.
- The `redirect()` and `notFound()` Next.js methods can also be used inside `generateMetadata`.

Metadata Fields

The following fields are supported:

`title`

The `title` attribute is used to set the title of the document. It can be defined as a simple string or an optional template object.

String

```
// Archivo: layout.js | page.js
export const metadata = {
  title: 'Next.js',
}
```

```
// Archivo: <head> output hideLineNumbers
<title>Next.js</title>
```

`default`

`title.default` can be used to provide a **fallback title** to child route segments that don't define a `title`.

```
// Archivo: app/layout.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    default: 'Acme',
  },
}
```

```
// Archivo: app/about/page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {}

// Output: <title>Acme</title>
```

`template`

`title.template` can be used to add a prefix or a suffix to `titles` defined in **child** route segments.

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    template: '%s | Acme',
    default: 'Acme', // a default is required when creating a template
  },
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  title: {
    template: '%s | Acme',
    default: 'Acme', // a default is required when creating a template
  },
}
```

```
// Archivo: app/about/page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'About',
}

// Output: <title>About | Acme</title>
```

```
// Archivo: app/about/page.js switcher
export const metadata = {
  title: 'About',
}

// Output: <title>About | Acme</title>
```

Good to know:

- `title.template` applies to **child** route segments and **not** the segment it's defined in. This means:
  - `title.default` is **required** when you add a `title.template`.
  - `title.template` defined in `layout.js` will not apply to a `title` defined in a `page.js` of the same route segment.
  - `title.template` defined in `page.js` has no effect because a page is always the terminating segment (it doesn't have any children route segments).
- `title.template` has **no effect** if a route has not defined a `title` or `title.default`.

`absolute`

`title.absolute` can be used to provide a title that **ignores** `title.template` set in parent segments.

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    template: '%s | Acme',
  },
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  title: {
    template: '%s | Acme',
  },
}
```

```
// Archivo: app/about/page.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    absolute: 'About',
  },
}

// Output: <title>About</title>
```

```
// Archivo: app/about/page.js switcher
export const metadata = {
  title: {
    absolute: 'About',
  },
}

// Output: <title>About</title>
```

Good to know:

- `layout.js`
  - `title` (string) and `title.default` define the default title for child segments (that do not define their own `title`). It will augment `title.template` from the closest parent segment if it exists.
  - `title.absolute` defines the default title for child segments. It ignores `title.template` from parent segments.
  - `title.template` defines a new title template for child segments.
- `page.js`
  - If a page does not define its own title the closest parents resolved title will be used.
  - `title` (string) defines the routes title. It will augment `title.template` from the closest parent segment if it exists.
  - `title.absolute` defines the route title. It ignores `title.template` from parent segments.
  - `title.template` has no effect in `page.js` because a page is always the terminating segment of a route.

`description`

```
// Archivo: layout.js | page.js
export const metadata = {
  description: 'The React Framework for the Web',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="description" content="The React Framework for the Web" />
```

Other fields

```
// Archivo: layout.js | page.js
export const metadata = {
  generator: 'Next.js',
  applicationName: 'Next.js',
  referrer: 'origin-when-cross-origin',
  keywords: ['Next.js', 'React', 'JavaScript'],
  authors: [{ name: 'Seb' }, { name: 'Josh', url: 'https://nextjs.org' }],
  creator: 'Jiachi Liu',
  publisher: 'Sebastian Markbåge',
  formatDetection: {
    email: false,
    address: false,
    telephone: false,
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="application-name" content="Next.js" />
<meta name="author" content="Seb" />
<link rel="author" href="https://nextjs.org" />
<meta name="author" content="Josh" />
<meta name="generator" content="Next.js" />
<meta name="keywords" content="Next.js,React,JavaScript" />
<meta name="referrer" content="origin-when-cross-origin" />
<meta name="creator" content="Jiachi Liu" />
<meta name="publisher" content="Sebastian Markbåge" />
<meta name="format-detection" content="telephone=no, address=no, email=no" />
```

`metadataBase`

`metadataBase` is a convenience option to set a base URL prefix for `metadata` fields that require a fully qualified URL.

- `metadataBase` allows URL-based `metadata` fields defined in the **current route segment and below** to use a **relative path** instead of an otherwise required absolute URL.
- The field's relative path will be composed with `metadataBase` to form a fully qualified URL.

```
// Archivo: layout.js | page.js
export const metadata = {
  metadataBase: new URL('https://acme.com'),
  alternates: {
    canonical: '/',
    languages: {
      'en-US': '/en-US',
      'de-DE': '/de-DE',
    },
  },
  openGraph: {
    images: '/og-image.png',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="canonical" href="https://acme.com" />
<link rel="alternate" hreflang="en-US" href="https://acme.com/en-US" />
<link rel="alternate" hreflang="de-DE" href="https://acme.com/de-DE" />
<meta property="og:image" content="https://acme.com/og-image.png" />
```

Good to know:

- `metadataBase` is typically set in root `app/layout.js` to apply to URL-based `metadata` fields across all routes.
- When `generateMetadata` uses `'use cache'`, its return value must be serializable. `URL` instances are not supported by Cache Functions, so values such as `metadataBase` should be returned as strings (for example, `url.toString()`). See `use cache` serialization requirements.
- All URL-based `metadata` fields that require absolute URLs can be configured with a `metadataBase` option.
- `metadataBase` can contain a subdomain e.g. `https://app.acme.com` or base path e.g. `https://acme.com/start/from/here`
- If a `metadata` field provides an absolute URL, `metadataBase` will be ignored.
- Using a relative path in a URL-based `metadata` field without configuring a `metadataBase` will cause a build error.
- Next.js will normalize duplicate slashes between `metadataBase` (e.g. `https://acme.com/`) and a relative field (e.g. `/path`) to a single slash (e.g. `https://acme.com/path`)

URL Composition

URL composition favors developer intent over default directory traversal semantics.

- Trailing slashes between `metadataBase` and `metadata` fields are normalized.
- An "absolute" path in a `metadata` field (that typically would replace the whole URL path) is treated as a "relative" path (starting from the end of `metadataBase`).

For example, given the following `metadataBase`:

```
// Archivo: app/layout.tsx switcher
import type { Metadata } from 'next'

export const metadata: Metadata = {
  metadataBase: new URL('https://acme.com'),
}
```

```
// Archivo: app/layout.js switcher
export const metadata = {
  metadataBase: new URL('https://acme.com'),
}
```

Any `metadata` fields that inherit the above `metadataBase` and set their own value will be resolved as follows:

`metadata` field	Resolved URL
`/`	`https://acme.com`
`./`	`https://acme.com`
`payments`	`https://acme.com/payments`
`/payments`	`https://acme.com/payments`
`./payments`	`https://acme.com/payments`
`../payments`	`https://acme.com/payments`
`https://beta.acme.com/payments`	`https://beta.acme.com/payments`

`openGraph`

```
// Archivo: layout.js | page.js
export const metadata = {
  openGraph: {
    title: 'Next.js',
    description: 'The React Framework for the Web',
    url: 'https://nextjs.org',
    siteName: 'Next.js',
    images: [
      {
        url: 'https://nextjs.org/og.png', // Must be an absolute URL
        width: 800,
        height: 600,
      },
      {
        url: 'https://nextjs.org/og-alt.png', // Must be an absolute URL
        width: 1800,
        height: 1600,
        alt: 'My custom alt',
      },
    ],
    videos: [
      {
        url: 'https://nextjs.org/video.mp4', // Must be an absolute URL
        width: 800,
        height: 600,
      },
    ],
    audio: [
      {
        url: 'https://nextjs.org/audio.mp3', // Must be an absolute URL
      },
    ],
    locale: 'en_US',
    type: 'website',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="og:title" content="Next.js" />
<meta property="og:description" content="The React Framework for the Web" />
<meta property="og:url" content="https://nextjs.org/" />
<meta property="og:site_name" content="Next.js" />
<meta property="og:locale" content="en_US" />
<meta property="og:image" content="https://nextjs.org/og.png" />
<meta property="og:image:width" content="800" />
<meta property="og:image:height" content="600" />
<meta property="og:image" content="https://nextjs.org/og-alt.png" />
<meta property="og:image:width" content="1800" />
<meta property="og:image:height" content="1600" />
<meta property="og:image:alt" content="My custom alt" />
<meta property="og:video" content="https://nextjs.org/video.mp4" />
<meta property="og:video:width" content="800" />
<meta property="og:video:height" content="600" />
<meta property="og:audio" content="https://nextjs.org/audio.mp3" />
<meta property="og:type" content="website" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  openGraph: {
    title: 'Next.js',
    description: 'The React Framework for the Web',
    type: 'article',
    publishedTime: '2023-01-01T00:00:00.000Z',
    authors: ['Seb', 'Josh'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="og:title" content="Next.js" />
<meta property="og:description" content="The React Framework for the Web" />
<meta property="og:type" content="article" />
<meta property="article:published_time" content="2023-01-01T00:00:00.000Z" />
<meta property="article:author" content="Seb" />
<meta property="article:author" content="Josh" />
```

Good to know:

- It may be more convenient to use the file-based Metadata API for Open Graph images. Rather than having to sync the config export with actual files, the file-based API will automatically generate the correct metadata for you.

`robots`

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  robots: {
    index: true,
    follow: true,
    nocache: false,
    googleBot: {
      index: true,
      follow: true,
      noimageindex: false,
      'max-video-preview': -1,
      'max-image-preview': 'large',
      'max-snippet': -1,
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="robots" content="index, follow" />
<meta
  name="googlebot"
  content="index, follow, max-video-preview:-1, max-image-preview:large, max-snippet:-1"
/>
```

`icons`

Good to know: We recommend using the file-based Metadata API for icons where possible. Rather than having to sync the config export with actual files, the file-based API will automatically generate the correct metadata for you.

```
// Archivo: layout.js | page.js
export const metadata = {
  icons: {
    icon: '/icon.png',
    shortcut: '/shortcut-icon.png',
    apple: '/apple-icon.png',
    other: {
      rel: 'apple-touch-icon-precomposed',
      url: '/apple-touch-icon-precomposed.png',
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="shortcut icon" href="/shortcut-icon.png" />
<link rel="icon" href="/icon.png" />
<link rel="apple-touch-icon" href="/apple-icon.png" />
<link
  rel="apple-touch-icon-precomposed"
  href="/apple-touch-icon-precomposed.png"
/>
```

```
// Archivo: layout.js | page.js
export const metadata = {
  icons: {
    icon: [
      { url: '/icon.png' },
      new URL('/icon.png', 'https://example.com'),
      { url: '/icon-dark.png', media: '(prefers-color-scheme: dark)' },
    ],
    shortcut: ['/shortcut-icon.png'],
    apple: [
      { url: '/apple-icon.png' },
      { url: '/apple-icon-x3.png', sizes: '180x180', type: 'image/png' },
    ],
    other: [
      {
        rel: 'apple-touch-icon-precomposed',
        url: '/apple-touch-icon-precomposed.png',
      },
    ],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="shortcut icon" href="/shortcut-icon.png" />
<link rel="icon" href="/icon.png" />
<link rel="icon" href="https://example.com/icon.png" />
<link rel="icon" href="/icon-dark.png" media="(prefers-color-scheme: dark)" />
<link rel="apple-touch-icon" href="/apple-icon.png" />
<link
  rel="apple-touch-icon-precomposed"
  href="/apple-touch-icon-precomposed.png"
/>
<link
  rel="apple-touch-icon"
  href="/apple-icon-x3.png"
  sizes="180x180"
  type="image/png"
/>
```

> **Good to know**: The `msapplication-*` meta tags are no longer supported in Chromium builds of Microsoft Edge, and thus no longer needed.

`themeColor`

> **Deprecated**: The `themeColor` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`colorScheme`

> **Deprecated**: The `colorScheme` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`manifest`

A web application manifest, as defined in the Web Application Manifest specification.

```
// Archivo: layout.js | page.js
export const metadata = {
  manifest: 'https://nextjs.org/manifest.json',
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="manifest" href="https://nextjs.org/manifest.json" />
```

`twitter`

The Twitter specification is (surprisingly) used for more than just X (formerly known as Twitter).

Learn more about the Twitter Card markup reference.

```
// Archivo: layout.js | page.js
export const metadata = {
  twitter: {
    card: 'summary_large_image',
    title: 'Next.js',
    description: 'The React Framework for the Web',
    siteId: '1467726470533754880',
    creator: '@nextjs',
    creatorId: '1467726470533754880',
    images: ['https://nextjs.org/og.png'], // Must be an absolute URL
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:site:id" content="1467726470533754880" />
<meta name="twitter:creator" content="@nextjs" />
<meta name="twitter:creator:id" content="1467726470533754880" />
<meta name="twitter:title" content="Next.js" />
<meta name="twitter:description" content="The React Framework for the Web" />
<meta name="twitter:image" content="https://nextjs.org/og.png" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  twitter: {
    card: 'app',
    title: 'Next.js',
    description: 'The React Framework for the Web',
    siteId: '1467726470533754880',
    creator: '@nextjs',
    creatorId: '1467726470533754880',
    images: {
      url: 'https://nextjs.org/og.png',
      alt: 'Next.js Logo',
    },
    app: {
      name: 'twitter_app',
      id: {
        iphone: 'twitter_app://iphone',
        ipad: 'twitter_app://ipad',
        googleplay: 'twitter_app://googleplay',
      },
      url: {
        iphone: 'https://iphone_url',
        ipad: 'https://ipad_url',
      },
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="twitter:site:id" content="1467726470533754880" />
<meta name="twitter:creator" content="@nextjs" />
<meta name="twitter:creator:id" content="1467726470533754880" />
<meta name="twitter:title" content="Next.js" />
<meta name="twitter:description" content="The React Framework for the Web" />
<meta name="twitter:card" content="app" />
<meta name="twitter:image" content="https://nextjs.org/og.png" />
<meta name="twitter:image:alt" content="Next.js Logo" />
<meta name="twitter:app:name:iphone" content="twitter_app" />
<meta name="twitter:app:id:iphone" content="twitter_app://iphone" />
<meta name="twitter:app:id:ipad" content="twitter_app://ipad" />
<meta name="twitter:app:id:googleplay" content="twitter_app://googleplay" />
<meta name="twitter:app:url:iphone" content="https://iphone_url" />
<meta name="twitter:app:url:ipad" content="https://ipad_url" />
<meta name="twitter:app:name:ipad" content="twitter_app" />
<meta name="twitter:app:name:googleplay" content="twitter_app" />
```

`viewport`

> **Deprecated**: The `viewport` option in `metadata` is deprecated as of Next.js 14. Please use the `viewport` configuration instead.

`verification`

```
// Archivo: layout.js | page.js
export const metadata = {
  verification: {
    google: 'google',
    yandex: 'yandex',
    yahoo: 'yahoo',
    other: {
      me: ['my-email', 'my-link'],
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="google-site-verification" content="google" />
<meta name="y_key" content="yahoo" />
<meta name="yandex-verification" content="yandex" />
<meta name="me" content="my-email" />
<meta name="me" content="my-link" />
```

`appleWebApp`

```
// Archivo: layout.js | page.js
export const metadata = {
  itunes: {
    appId: 'myAppStoreID',
    appArgument: 'myAppArgument',
  },
  appleWebApp: {
    title: 'Apple Web App',
    statusBarStyle: 'black-translucent',
    startupImage: [
      '/assets/startup/apple-touch-startup-image-768x1004.png',
      {
        url: '/assets/startup/apple-touch-startup-image-1536x2008.png',
        media: '(device-width: 768px) and (device-height: 1024px)',
      },
    ],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta
  name="apple-itunes-app"
  content="app-id=myAppStoreID, app-argument=myAppArgument"
/>
<meta name="mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-title" content="Apple Web App" />
<link
  href="/assets/startup/apple-touch-startup-image-768x1004.png"
  rel="apple-touch-startup-image"
/>
<link
  href="/assets/startup/apple-touch-startup-image-1536x2008.png"
  media="(device-width: 768px) and (device-height: 1024px)"
  rel="apple-touch-startup-image"
/>
<meta
  name="apple-mobile-web-app-status-bar-style"
  content="black-translucent"
/>
```

`alternates`

```
// Archivo: layout.js | page.js
export const metadata = {
  alternates: {
    canonical: 'https://nextjs.org',
    languages: {
      'en-US': 'https://nextjs.org/en-US',
      'de-DE': 'https://nextjs.org/de-DE',
    },
    media: {
      'only screen and (max-width: 600px)': 'https://nextjs.org/mobile',
    },
    types: {
      'application/rss+xml': 'https://nextjs.org/rss',
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="canonical" href="https://nextjs.org" />
<link rel="alternate" hreflang="en-US" href="https://nextjs.org/en-US" />
<link rel="alternate" hreflang="de-DE" href="https://nextjs.org/de-DE" />
<link
  rel="alternate"
  media="only screen and (max-width: 600px)"
  href="https://nextjs.org/mobile"
/>
<link
  rel="alternate"
  type="application/rss+xml"
  href="https://nextjs.org/rss"
/>
```

`appLinks`

```
// Archivo: layout.js | page.js
export const metadata = {
  appLinks: {
    ios: {
      url: 'https://nextjs.org/ios',
      app_store_id: 'app_store_id',
    },
    android: {
      package: 'com.example.android/package',
      app_name: 'app_name_android',
    },
    web: {
      url: 'https://nextjs.org/web',
      should_fallback: true,
    },
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="al:ios:url" content="https://nextjs.org/ios" />
<meta property="al:ios:app_store_id" content="app_store_id" />
<meta property="al:android:package" content="com.example.android/package" />
<meta property="al:android:app_name" content="app_name_android" />
<meta property="al:web:url" content="https://nextjs.org/web" />
<meta property="al:web:should_fallback" content="true" />
```

`archives`

Describes a collection of records, documents, or other materials of historical interest (source).

```
// Archivo: layout.js | page.js
export const metadata = {
  archives: ['https://nextjs.org/13'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="archives" href="https://nextjs.org/13" />
```

`assets`

```
// Archivo: layout.js | page.js
export const metadata = {
  assets: ['https://nextjs.org/assets'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="assets" href="https://nextjs.org/assets" />
```

`bookmarks`

```
// Archivo: layout.js | page.js
export const metadata = {
  bookmarks: ['https://nextjs.org/13'],
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="bookmarks" href="https://nextjs.org/13" />
```

`pagination`

Describes the previous and next pages in a paginated sequence.

```
// Archivo: layout.js | page.js
export const metadata = {
  pagination: {
    previous: 'https://nextjs.org/blog?page=1',
    next: 'https://nextjs.org/blog?page=3',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<link rel="prev" href="https://nextjs.org/blog?page=1" />
<link rel="next" href="https://nextjs.org/blog?page=3" />
```

`category`

```
// Archivo: layout.js | page.js
export const metadata = {
  category: 'technology',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="category" content="technology" />
```

`facebook`

You can connect a Facebook app or Facebook account to your webpage for certain Facebook Social Plugins Facebook Documentation

> **Good to know**: You can specify either appId or admins, but not both.

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    appId: '12345678',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:app_id" content="12345678" />
```

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    admins: '12345678',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:admins" content="12345678" />
```

If you want to generate multiple fb:admins meta tags you can use array value.

```
// Archivo: layout.js | page.js
export const metadata = {
  facebook: {
    admins: ['12345678', '87654321'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta property="fb:admins" content="12345678" />
<meta property="fb:admins" content="87654321" />
```

`pinterest`

You can enable or disable Pinterest Rich Pins on your webpage.

```
// Archivo: layout.js | page.js
export const metadata = {
  pinterest: {
    richPin: true,
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="pinterest-rich-pin" content="true" />
```

`other`

All metadata options should be covered using the built-in support. However, there may be custom metadata tags specific to your site, or brand new metadata tags just released. You can use the `other` option to render any custom metadata tag.

```
// Archivo: layout.js | page.js
export const metadata = {
  other: {
    custom: 'meta',
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="custom" content="meta" />
```

If you want to generate multiple same key meta tags you can use array value.

```
// Archivo: layout.js | page.js
export const metadata = {
  other: {
    custom: ['meta1', 'meta2'],
  },
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="custom" content="meta1" /> <meta name="custom" content="meta2" />
```

Types

You can add type safety to your metadata by using the `Metadata` type. If you are using the built-in TypeScript plugin in your IDE, you do not need to manually add the type, but you can still explicitly add it if you want.

`metadata` object

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Next.js',
}
```

`generateMetadata` function

Regular function

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export function generateMetadata(): Metadata {
  return {
    title: 'Next.js',
  }
}
```

Async function

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

export async function generateMetadata(): Promise<Metadata> {
  return {
    title: 'Next.js',
  }
}
```

With segment props

```
// Archivo: layout.tsx | page.tsx
import type { Metadata } from 'next'

type Props = {
  params: Promise<{ id: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export function generateMetadata({ params, searchParams }: Props): Metadata {
  return {
    title: 'Next.js',
  }
}

export default function Page({ params, searchParams }: Props) {}
```

With parent metadata

```tsx
// Archivo: layout.tsx | page.tsx
import type { Metadata, ResolvingMetadata } from 'next'

export async function generateMetadata(
  { params, searchParams }: Props,
  parent: ResolvingMetadata
): Promise<Metadata> {
  return {
    title: 'Next.js',
  }
}
```

JavaScript Projects

For JavaScript projects, you can use JSDoc to add type safety.

```jsx
// Archivo: layout.js | page.js
/** @type {import("next").Metadata} */
export const metadata = {
  title: 'Next.js',
}
```

Unsupported Metadata

The following metadata types do not currently have built-in support. However, they can still be rendered in the layout or page itself.

Metadata	Recommendation
`<meta http-equiv="...">`	Use appropriate HTTP Headers via `redirect()`, Proxy, Security Headers
`<base>`	Render the tag in the layout or page itself.
`<noscript>`	Render the tag in the layout or page itself.
`<style>`	Learn more about styling in Next.js.
`<script>`	Learn more about using scripts.
`<link rel="stylesheet" />`	`import` stylesheets directly in the layout or page itself.
`<link rel="preload" />`	Use ReactDOM preload method
`<link rel="preconnect" />`	Use ReactDOM preconnect method
`<link rel="dns-prefetch" />`	Use ReactDOM prefetchDNS method

Resource hints

The `<link>` element has a number of `rel` keywords that can be used to hint to the browser that an external resource is likely to be needed. The browser uses this information to apply preloading optimizations depending on the keyword.

While the Metadata API doesn't directly support these hints, you can use new `ReactDOM` methods to safely insert them into the `<head>` of the document.

```tsx
// Archivo: app/preload-resources.tsx switcher
'use client'

import ReactDOM from 'react-dom'

export function PreloadResources() {
  ReactDOM.preload('...', { as: '...' })
  ReactDOM.preconnect('...', { crossOrigin: '...' })
  ReactDOM.prefetchDNS('...')

  return '...'
}
```

```
// Archivo: app/preload-resources.js switcher
'use client'

import ReactDOM from 'react-dom'

export function PreloadResources() {
  ReactDOM.preload('...', { as: '...' })
  ReactDOM.preconnect('...', { crossOrigin: '...' })
  ReactDOM.prefetchDNS('...')

  return '...'
}
```

`<link rel="preload">`

Start loading a resource early in the page rendering (browser) lifecycle. MDN Docs.

```
ReactDOM.preload(href: string, options: { as: string })
```

```
// Archivo: <head> output hideLineNumbers
<link rel="preload" href="..." as="..." />
```

`<link rel="preconnect">`

Preemptively initiate a connection to an origin. MDN Docs.

```
ReactDOM.preconnect(href: string, options?: { crossOrigin?: string })
```

```
// Archivo: <head> output hideLineNumbers
<link rel="preconnect" href="..." crossorigin />
```

`<link rel="dns-prefetch">`

Attempt to resolve a domain name before resources get requested. MDN Docs.

```
ReactDOM.prefetchDNS(href: string)
```

```
// Archivo: <head> output hideLineNumbers
<link rel="dns-prefetch" href="..." />
```

> **Good to know**:
>
> - These methods are currently only supported in Client Components, which are still Server Side Rendered on initial page load.
> - Next.js in-built features such as `next/font`, `next/image` and `next/script` automatically handle relevant resource hints.

Behavior

Default Fields

There are two default `meta` tags that are always added even if a route doesn't define metadata:

- The meta charset tag sets the character encoding for the website.
- The meta viewport tag sets the viewport width and scale for the website to adjust for different devices.

```
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

> **Good to know**: You can overwrite the default `viewport` meta tag.

Streaming metadata

Streaming metadata allows Next.js to render and send the initial UI to the browser, without waiting for `generateMetadata` to complete.

When `generateMetadata` resolves, the resulting metadata tags are appended to the `<body>` tag. We have verified that metadata is interpreted correctly by bots that execute JavaScript and inspect the full DOM (e.g. `Googlebot`).

For **HTML-limited bots** that can't execute JavaScript (e.g. `facebookexternalhit`), metadata continues to block page rendering. The resulting metadata will be available in the `<head>` tag.

Next.js automatically detects **HTML-limited bots** by looking at the User Agent header. You can use the `htmlLimitedBots` option in your Next.js config file to override the default User Agent list.

To fully disable streaming metadata:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const config: NextConfig = {
  htmlLimitedBots: /.*/,
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  htmlLimitedBots: /.*/,
}
```

Streaming metadata improves perceived performance by reducing TTFB and can help lowering LCP time.

Overriding `htmlLimitedBots` could lead to longer response times. Streaming metadata is an advanced feature, and the default should be sufficient for most cases.

With Cache Components

When Cache Components is enabled, `generateMetadata` follows the same rules as other components. If metadata accesses runtime data (`cookies()`, `headers()`, `params`, `searchParams`) or performs uncached data fetching, it defers to request time.

How Next.js handles this depends on the rest of your page:

- **If other parts also defer to request time**: Prerendering generates a static shell, and metadata streams in with other deferred content.
- **If the page or layout is otherwise fully prerenderable**: Next.js requires an explicit choice: cache the data if possible, or signal that deferred rendering is intentional.

Streaming metadata at runtime while the rest of the page is fully prerenderable is not common. To ensure this behavior is intentional, an error is raised indicating which page or layout needs to be handled.

To resolve this, you have two options. If metadata depends on external data but not runtime data, use `use cache`:

```
// Archivo: app/page.tsx highlight={2}
export async function generateMetadata() {
  'use cache'
  const { title, description } = await db.query('site-metadata')
  return { title, description }
}
```

If metadata genuinely requires runtime data, add a dynamic marker component to your page:

```
// Archivo: app/page.tsx highlight={11-14,19,31}
import { Suspense } from 'react'
import { cookies } from 'next/headers'
import { connection } from 'next/server'

export async function generateMetadata() {
  const token = (await cookies()).get('token')?.value
  // ... use token to fetch personalized metadata
  return { title: 'Personalized Title' }
}

const Connection = async () => {
  await connection()
  return null
}

async function DynamicMarker() {
  return (
    ``<Suspense>``
  )
}

export default function Page() {
  // DO NOT place await connection() here
  // doing so prevents the article tag content from
  // being included in the static shell
  return (
    <>
      <article>Static content</article>
      <DynamicMarker />
    </>
  )
}
```

The `DynamicMarker` component renders nothing but tells Next.js the page has intentional dynamic content. By wrapping it in Suspense, the static content still prerenders normally.

See Next.js encountered runtime data in `generateMetadata()` and Next.js encountered uncached data in `generateMetadata()` for full fix options and trade-offs.

Ordering

Metadata is evaluated in order, starting from the root segment down to the segment closest to the final `page.js` segment. For example:

1. `app/layout.tsx` (Root Layout)
2. `app/blog/layout.tsx` (Nested Blog Layout)
3. `app/blog/[slug]/page.tsx` (Blog Page)

Merging

Following the evaluation order, Metadata objects exported from multiple segments in the same route are **shallowly** merged together to form the final metadata output of a route. Duplicate keys are **replaced** based on their ordering.

This means metadata with nested fields such as `openGraph` and `robots` that are defined in an earlier segment are **overwritten** by the last segment to define them.

Overwriting fields

```
// Archivo: app/layout.js
export const metadata = {
  title: 'Acme',
  openGraph: {
    title: 'Acme',
    description: 'Acme is a ...',
  },
}
```

```
// Archivo: app/blog/page.js
export const metadata = {
  title: 'Blog',
  openGraph: {
    title: 'Blog',
  },
}

// Output:
// <title>Blog</title>
// <meta property="og:title" content="Blog" />
```

In the example above:

- `title` from `app/layout.js` is **replaced** by `title` in `app/blog/page.js`.
- All `openGraph` fields from `app/layout.js` are **replaced** in `app/blog/page.js` because `app/blog/page.js` sets `openGraph` metadata. Note the absence of `openGraph.description`.

If you'd like to share some nested fields between segments while overwriting others, you can pull them out into a separate variable:

```
// Archivo: app/shared-metadata.js
export const openGraphImage = { images: ['http:// ... '] }
```

```
// Archivo: app/page.js
import { openGraphImage } from './shared-metadata'

export const metadata = {
  openGraph: {
    ... openGraphImage,
    title: 'Home',
  },
}
```

```
// Archivo: app/about/page.js
import { openGraphImage } from '../shared-metadata'

export const metadata = {
  openGraph: {
    ... openGraphImage,
    title: 'About',
  },
}
```

In the example above, the OG image is shared between `app/layout.js` and `app/about/page.js` while the titles are different.

Inheriting fields

```
// Archivo: app/layout.js
export const metadata = {
  title: 'Acme',
  openGraph: {
    title: 'Acme',
    description: 'Acme is a ... ',
  },
}
```

```
// Archivo: app/about/page.js
export const metadata = {
  title: 'About',
}

// Output:
// <title>About</title>
// <meta property="og:title" content="Acme" />
// <meta property="og:description" content="Acme is a ... " />
```

Notes

- `title` from `app/layout.js` is **replaced** by `title` in `app/about/page.js`.
- All `openGraph` fields from `app/layout.js` are **inherited** in `app/about/page.js` because `app/about/page.js` doesn't set `openGraph` metadata.

Version History

Version	Changes
`v15.2.0`	Introduced streaming support to `generateMetadata`.
`v14.0.0`	`viewport`, `themeColor`, and `colorScheme` deprecated in favor of the `viewport` configuration.
`v13.2.0`	`metadata` and `generateMetadata` introduced.

generateSitemaps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-sitemaps
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *Learn how to use the generateSiteMaps function to create multiple sitemaps for your application.*

You can use the `generateSitemaps` function to generate multiple sitemaps for your application.

Returns

The `generateSitemaps` returns an array of objects with an `id` property.

URLs

Your generated sitemaps will be available at `/ ... /sitemap/[id].xml`. For example, `/product/sitemap/1.xml`.

Example

For example, to split a sitemap using `generateSitemaps`, return an array of objects with the sitemap `id`. Then, use the `id` to generate the unique sitemaps.

```tsx
// Archivo: app/product/sitemap.ts switcher
import type { MetadataRoute } from 'next'
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props: {
  id: Promise<string>
}): Promise<MetadataRoute.Sitemap> {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

```jsx
// Archivo: app/product/sitemap.js switcher
import { BASE_URL } from '@/app/lib/constants'

export async function generateSitemaps() {
  // Fetch the total number of products and calculate the number of sitemaps needed
  return [{ id: 0 }, { id: 1 }, { id: 2 }, { id: 3 }]
}

export default async function sitemap(props) {
  const id = await props.id
  // Google's limit is 50,000 URLs per sitemap
  const start = id * 50000
  const end = start + 50000
  const products = await getProducts(
    `SELECT id, date FROM products WHERE id BETWEEN ${start} AND ${end}`
  )
  return products.map((product) => ({
    url: `${BASE_URL}/product/${product.id}`,
    lastModified: product.date,
  }))
}
```

Version History

Version	Changes
`v16.0.0`	The `id` values returned from `generateSitemaps` are now passed as a promise that resolves to a `string` to the sitemap function.
`v15.0.0`	`generateSitemaps` now generates consistent URLs between development and production
`v13.3.2`	`generateSitemaps` introduced. In development, you can view the generated sitemap on `/ ... /sitemap.xml/[id]`. For example, `/product/sitemap.xml/1`.

generateStaticParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-static-params
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API reference for the generateStaticParams function.*

The `generateStaticParams` function can be used in combination with dynamic route segments to **statically generate** routes at build time instead of on-demand at request time.

`generateStaticParams` can be used with:

- Pages (`page.tsx` / `page.js`)
- Layouts (`layout.tsx` / `layout.js`)
- Route Handlers (`route.ts` / `route.js`)

```
// Archivo: app/blog/[slug]/page.tsx switcher
// Return a list of `params` to populate the [slug] dynamic segment
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}

// Multiple versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
// Return a list of `params` to populate the [slug] dynamic segment
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}

// Multiple versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
export default async function Page({ params }) {
  const { slug } = await params
  // ...
}
```

Good to know:

- You can use the `dynamicParams` segment config option to control what happens when a dynamic segment is visited that was not generated with `generateStaticParams`.
- You must return an empty array from `generateStaticParams` or utilize `export const dynamic = 'force-static'` in order to revalidate (ISR) paths at runtime.
- During `next dev`, `generateStaticParams` will be called when you navigate to a route.
- During `next build`, `generateStaticParams` runs before the corresponding Layouts or Pages are generated.
- During revalidation (ISR), `generateStaticParams` will not be called again.
- `generateStaticParams` replaces the `getStaticPaths` function in the Pages Router.

Parameters

`options.params` (optional)

If multiple dynamic segments in a route use `generateStaticParams`, the child `generateStaticParams` function is executed once for each set of `params` the parent generates.

The `params` object contains the populated `params` from the parent `generateStaticParams`, which can be used to generate the `params` in a child segment.

Returns

`generateStaticParams` should return an array of objects where each object represents the populated dynamic segments of a single route.

- Each property in the object is a dynamic segment to be filled in for the route.
- The properties name is the segment's name, and the properties value is what that segment should be filled in with.

Example Route	`generateStaticParams` Return Type
`/product/[id]`	`{ id: string }[]`
`/products/[category]/[product]`	`{ category: string, product: string }[]`
`/products/[...slug]`	`{ slug: string[] }[]`

Single Dynamic Segment

```
// Archivo: app/product/[id]/page.tsx switcher
export function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /product/1
// - /product/2
// - /product/3
export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  // ...
}
```

```
// Archivo: app/product/[id]/page.js switcher
export function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /product/1
// - /product/2
// - /product/3
export default async function Page({ params }) {
  const { id } = await params
  // ...
}
```

Multiple Dynamic Segments

```
// Archivo: app/products/[category]/[product]/page.tsx switcher
export function generateStaticParams() {
  return [
    { category: 'a', product: '1' },
    { category: 'b', product: '2' },
    { category: 'c', product: '3' },
  ]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /products/a/1
// - /products/b/2
// - /products/c/3
export default async function Page({
  params,
}: {
  params: Promise<{ category: string; product: string }>
}) {
  const { category, product } = await params
  // ...
}
```

```
// Archivo: app/products/[category]/[product]/page.js switcher
export function generateStaticParams() {
  return [
    { category: 'a', product: '1' },
    { category: 'b', product: '2' },
    { category: 'c', product: '3' },
  ]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /products/a/1
// - /products/b/2
// - /products/c/3
export default async function Page({ params }) {
  const { category, product } = await params
  // ...
}
```

Catch-all Dynamic Segment

```
// Archivo: app/product/[...slug]/page.tsx switcher
export function generateStaticParams() {
  return [{ slug: ['a', '1'] }, { slug: ['b', '2'] }, { slug: ['c', '3'] }]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /product/a/1
// - /product/b/2
// - /product/c/3
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string[] }>
}) {
  const { slug } = await params
  // ...
}
```

```
// Archivo: app/product/[...slug]/page.js switcher
export function generateStaticParams() {
  return [{ slug: ['a', '1'] }, { slug: ['b', '2'] }, { slug: ['c', '3'] }]
}

// Three versions of this page will be statically generated
// using the `params` returned by `generateStaticParams`
// - /product/a/1
// - /product/b/2
// - /product/c/3
export default async function Page({ params }) {
  const { slug } = await params
  // ...
}
```

Examples

Prerendering

All paths at build time

To statically render all paths at build time, supply the full list of paths to `generateStaticParams`:

```
// Archivo: app/blog/[slug]/page.tsx switcher
export async function generateStaticParams() {
  const posts = await fetch('https://.../posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export async function generateStaticParams() {
  const posts = await fetch('https://.../posts').then((res) => res.json())

  return posts.map((post) => ({
    slug: post.slug,
  }))
}
```

Subset of paths at build time

To statically render a subset of paths at build time, and the rest the first time they're visited at runtime, return a partial list of paths:

```
// Archivo: app/blog/[slug]/page.tsx switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  // Render the first 10 posts at build time
  return posts.slice(0, 10).map((post) => ({
    slug: post.slug,
  }))
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())

  // Render the first 10 posts at build time
  return posts.slice(0, 10).map((post) => ({
    slug: post.slug,
  }))
}
```

Then, by using the `dynamicParams` segment config option, you can control what happens when a dynamic segment is visited that was not generated with `generateStaticParams`.

```
// Archivo: app/blog/[slug]/page.tsx switcher
// All posts besides the top 10 will be a 404
export const dynamicParams = false

export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())
  const topPosts = posts.slice(0, 10)

  return topPosts.map((post) => ({
    slug: post.slug,
  }))
}
```

```
// Archivo: app/blog/[slug]/page.js switcher
// All posts besides the top 10 will be a 404
export const dynamicParams = false

export async function generateStaticParams() {
  const posts = await fetch('https:// ... /posts').then((res) => res.json())
  const topPosts = posts.slice(0, 10)

  return topPosts.map((post) => ({
    slug: post.slug,
  }))
}
```

All paths at runtime

To statically render all paths the first time they're visited, return an empty array (no paths will be rendered at build time) or utilize `export const dynamic = 'force-static'`:

```
// Archivo: app/blog/[slug]/page.js
export async function generateStaticParams() {
  return []
}
```

> **Good to know:**
>
> - You must always return an array from `generateStaticParams`, even if it's empty. Otherwise, the route will be dynamically rendered.

```
// Archivo: app/changelog/[slug]/page.js
export const dynamic = 'force-static'
```

With Cache Components

When using Cache Components with dynamic routes, `generateStaticParams` must return **at least one param**. Empty arrays cause a build error. This allows Cache Components to validate your route doesn't incorrectly access `cookies()`, `headers()`, or `searchParams` at runtime.

> **Good to know**: If you don't know the actual param values at build time, you can return a placeholder param (e.g., `[{ slug: '__placeholder__' }]`) for validation, then handle it in your page with `notFound()`. However, this prevents build time validation from working effectively and may cause runtime errors.

See the dynamic routes section for detailed walkthroughs, or ISR with Cache Components for prerendering a subset of routes and serving App Shells for the rest.

With Route Handlers

You can use `generateStaticParams` with Route Handlers to statically generate API responses at build time:

```
// Archivo: app/api/posts/[id]/route.ts switcher
export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

export async function GET(
  request: Request,
  { params }: RouteContext<'/api/posts/[id]'>
) {
  const { id } = await params
  // This will be statically generated for IDs 1, 2, and 3
  return Response.json({ id, title: `Post ${id}` })
}
```

```
// Archivo: app/api/posts/[id]/route.js switcher
export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

export async function GET(request, { params }) {
  const { id } = await params
  // This will be statically generated for IDs 1, 2, and 3
  return Response.json({ id, title: `Post ${id}` })
}
```

Route Handlers with Cache Components

When using Cache Components, combine with `use cache` for optimal caching:

```
// Archivo: app/api/posts/[id]/route.ts
export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }, { id: '3' }]
}

async function getPost(id: Promise<string>) {
  'use cache'
  const resolvedId = await id
  const response = await fetch(`https://api.example.com/posts/${resolvedId}`)
  return response.json()
}

export async function GET(
  request: Request,
  { params }: RouteContext<'/api/posts/[id]'>
) {
  const post = await getPost(params.then((p) => p.id))
  return Response.json(post)
}
```

See the Route Handlers documentation for more details.

Disable rendering for unspecified paths

To prevent unspecified paths from being prerendered at runtime, add the `export const dynamicParams = false` option in a route segment. When this config option is used, only paths provided by `generateStaticParams` will be served, and unspecified routes will 404 or match (in the case of catch-all routes).

Multiple Dynamic Segments in a Route

You can generate params for dynamic segments above the current layout or page, but **not below**. For example, given the `app/products/[category]/[product]` route:

- `app/products/[category]/[product]/page.js` can generate params for **both** `[category]` and `[product]`.
- `app/products/[category]/layout.js` can **only** generate params for `[category]`.

There are two approaches to generating params for a route with multiple dynamic segments:

Generate params from the bottom up

Generate multiple dynamic segments from the child route segment.

```
// Archivo: app/products/[category]/[product]/page.tsx switcher
// Generate segments for both [category] and [product]
export async function generateStaticParams() {
  const products = await fetch('https:// ... /products').then((res) => res.json())

  return products.map((product) => ({
    category: product.category.slug,
    product: product.id,
  }))
}

export default function Page({
  params,
}: {
  params: Promise<{ category: string; product: string }>
}) {
  // ...
}
```

```
// Archivo: app/products/[category]/[product]/page.js switcher
// Generate segments for both [category] and [product]
export async function generateStaticParams() {
  const products = await fetch('https:// ... /products').then((res) => res.json())

  return products.map((product) => ({
    category: product.category.slug,
    product: product.id,
  }))
}

export default function Page({ params }) {
  // ...
}
```

Generate params from the top down

Generate the parent segments first and use the result to generate the child segments.

```
// Archivo: app/products/[category]/layout.tsx switcher
// Generate segments for [category]
export async function generateStaticParams() {
  const products = await fetch('https:// ... /products').then((res) => res.json())

  return products.map((product) => ({
    category: product.category.slug,
  }))
}

export default function Layout({
  params,
}: {
  params: Promise<{ category: string }>
}) {
  // ...
}
```

```
// Archivo: app/products/[category]/layout.js switcher
// Generate segments for [category]
export async function generateStaticParams() {
  const products = await fetch('https:// ... /products').then((res) => res.json())

  return products.map((product) => ({
    category: product.category.slug,
  }))
}

export default function Layout({ params }) {
  // ...
}
```

A child route segment's `generateStaticParams` function is executed once for each segment a parent `generateStaticParams` generates.

The child `generateStaticParams` function can use the `params` returned from the parent `generateStaticParams` function to dynamically generate its own segments.

```tsx
// Archivo: app/products/[category]/[product]/page.tsx switcher
// Generate segments for [product] using the `params` passed from
// the parent segment's `generateStaticParams` function
export async function generateStaticParams({
  params: { category },
}: {
  params: { category: string }
}) {
  const products = await fetch(
    `https:// ... /products?category=${category}`
  ).then((res) => res.json())

  return products.map((product) => ({
    product: product.id,
  }))
}

export default function Page({
  params,
}: {
  params: Promise<{ category: string; product: string }>
}) {
  // ...
}
```

```jsx
// Archivo: app/products/[category]/[product]/page.js switcher
// Generate segments for [product] using the `params` passed from
// the parent segment's `generateStaticParams` function
export async function generateStaticParams({ params: { category } }) {
  const products = await fetch(
    `https:// ... /products?category=${category}`
  ).then((res) => res.json())

  return products.map((product) => ({
    product: product.id,
  }))
}

export default function Page({ params }) {
  // ...
}
```

Notice that the params argument can be accessed synchronously and includes only parent segment params.

> **Good to know:** When a parent dynamic segment is a root parameter, you can also read it inside a nested `generateStaticParams` by calling its getter from the `next/root-params` module.

For type completion, you can make use of the TypeScript `Awaited` helper in combination with either `Page Props helper` or `Layout Props helper`:

```tsx
// Archivo: app/products/[category]/[product]/page.tsx switcher
export async function generateStaticParams({
  params: { category },
}: {
  params: Awaited<LayoutProps<'/products/[category]'>['params']>
}) {
  const products = await fetch(
    `https:// ... /products?category=${category}`
  ).then((res) => res.json())

  return products.map((product) => ({
    product: product.id,
  }))
}
```

> **Good to know**: `fetch` requests are automatically memoized for the same data across all `generate`-prefixed functions, Layouts, Pages, and Server Components. React `cache` can be used if `fetch` is unavailable.

Version History

Version	Changes
`v13.0.0`	`generateStaticParams` introduced.

generateViewport

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/generate-viewport
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the generateViewport function.*

You can customize the initial viewport of the page with the static `viewport` object or the dynamic `generateViewport` function.

> **Good to know**:
>
> - The `viewport` object and `generateViewport` function exports are **only supported in Server Components**.
> - You cannot export both the `viewport` object and `generateViewport` function from the same route segment.
> - If you're coming from migrating `metadata` exports, you can use metadata-to-viewport-export codemod to update your changes.

The `viewport` object

To define the viewport options, export a `viewport` object from a `layout.jsx` or `page.jsx` file.

```
// Archivo: layout.tsx | page.tsx switcher
import type { Viewport } from 'next'

export const viewport: Viewport = {
  themeColor: 'black',
}

export default function Page() {}
```

```
// Archivo: layout.jsx | page.jsx switcher
export const viewport = {
  themeColor: 'black',
}

export default function Page() {}
```

`generateViewport` function

`generateViewport` should return a `Viewport` object containing one or more viewport fields.

```
// Archivo: layout.tsx | page.tsx switcher
export function generateViewport({ params }) {
  return {
    themeColor: '...',
  }
}
```

In TypeScript, the `params` argument can be typed via `PageProps<'/route'>` or `LayoutProps<'/route'>` depending on where `generateViewport` is defined.

```
// Archivo: layout.js | page.js switcher
export function generateViewport({ params }) {
  return {
    themeColor: '...',
  }
}
```

> **Good to know**:
>
> - If the viewport doesn't depend on request information, it should be defined using the static `viewport` object rather than `generateViewport`.

Viewport Fields

`themeColor`

Learn more about `theme-color`.

Simple theme color

```
// Archivo: layout.tsx | page.tsx switcher
import type { Viewport } from 'next'

export const viewport: Viewport = {
  themeColor: 'black',
}
```

```
// Archivo: layout.jsx | page.jsx switcher
export const viewport = {
  themeColor: 'black',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="theme-color" content="black" />
```

With media attribute

```
// Archivo: layout.tsx | page.tsx switcher
import type { Viewport } from 'next'

export const viewport: Viewport = {
  themeColor: [
    { media: '(prefers-color-scheme: light)', color: 'cyan' },
    { media: '(prefers-color-scheme: dark)', color: 'black' },
  ],
}
```

```
// Archivo: layout.jsx | page.jsx switcher
export const viewport = {
  themeColor: [
    { media: '(prefers-color-scheme: light)', color: 'cyan' },
    { media: '(prefers-color-scheme: dark)', color: 'black' },
  ],
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="theme-color" media="(prefers-color-scheme: light)" content="cyan" />
<meta name="theme-color" media="(prefers-color-scheme: dark)" content="black" />
```

`width`, `initialScale`, `maximumScale` and `userScalable`

> **Good to know**: The `viewport` meta tag is automatically set, and manual configuration is usually unnecessary as the default is sufficient. However, the information is provided for completeness.

```
// Archivo: layout.tsx | page.tsx switcher
import type { Viewport } from 'next'

export const viewport: Viewport = {
  width: 'device-width',
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  // Also supported but less commonly used
  // interactiveWidget: 'resizes-visual',
}
```

```
// Archivo: layout.jsx | page.jsx switcher
export const viewport = {
  width: 'device-width',
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  // Also supported but less commonly used
  // interactiveWidget: 'resizes-visual',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta
  name="viewport"
  content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no"
/>
```

`colorScheme`

Learn more about `color-scheme`.

```
// Archivo: layout.tsx | page.tsx switcher
import type { Viewport } from 'next'

export const viewport: Viewport = {
  colorScheme: 'dark',
}
```

```
// Archivo: layout.jsx | page.jsx switcher
export const viewport = {
  colorScheme: 'dark',
}
```

```
// Archivo: <head> output hideLineNumbers
<meta name="color-scheme" content="dark" />
```

With Cache Components

When Cache Components is enabled, `generateViewport` follows the same rules as other components. If viewport accesses runtime data (`cookies()` , `headers()` , `params` , `searchParams`) or performs uncached data fetching, it defers to request time.

Unlike metadata, viewport cannot be streamed because it affects initial page load UI. If `generateViewport` defers to request time, the page would need to block until resolved.

If viewport depends on external data but not runtime data, use `use cache` :

```
// Archivo: app/layout.tsx highlight={2}
export async function generateViewport() {
  'use cache'
  const { width, initialScale } = await db.query('viewport-size')
  return { width, initialScale }
}
```

If viewport genuinely requires runtime data, wrap the document `<body>` in a [`````) }

```

Alternatively, opt the segment out of instant-navigation validation with [`instant = false`](/docs/app/api-reference/file-conventions/route-segment-config/instant). The route renders on every request and the navigation blocks until the render completes:

```tsx
// Archivo: app/dashboard/layout.tsx highlight={1,3}
import { cookies } from 'next/headers'

export const instant = false

export async function generateViewport() {
  const cookieJar = await cookies()
  return {
    themeColor: cookieJar.get('theme-color')?.value,
  }
}

export default function DashboardLayout({ children }) {
  return <section>{children}</section>
}
```
```

Setting `instant = false` opts only the segment that exports it out. Descendant segments are still validated by the global default. See Next.js encountered runtime data in `generateViewport()` for the full set of fix options, trade-offs, and the subtree-wide opt-out.

> **Good to know**: Use multiple root layouts to isolate fully dynamic viewport to specific routes, while still letting other routes in your application generate a static shell.

Types

You can add type safety to your viewport object by using the `Viewport` type. If you are using the built-in TypeScript plugin in your IDE, you do not need to manually add the type, but you can still explicitly add it if you want.

`viewport` object

```
import type { Viewport } from 'next'

export const viewport: Viewport = {
  themeColor: 'black',
}
```

`generateViewport` function

Regular function

```
import type { Viewport } from 'next'

export function generateViewport(): Viewport {
  return {
    themeColor: 'black',
  }
}
```

With segment props

```
import type { Viewport } from 'next'

type Props = {
  params: Promise<{ id: string }>
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}

export function generateViewport({ params, searchParams }: Props): Viewport {
  return {
    themeColor: 'black',
  }
}

export default function Page({ params, searchParams }: Props) {}
```

JavaScript Projects

For JavaScript projects, you can use JSDoc to add type safety.

```
/** @type {import("next").Viewport} */
export const viewport = {
  themeColor: 'black',
}
```

Version History

Version	Changes
`v14.0.0`	`viewport` and `generateViewport` introduced.

headers

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/headers
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API reference for the headers function.*

`headers` is an **async** function that allows you to **read** the HTTP incoming request headers from a Server Component.

```
// Archivo: app/page.tsx switcher
import { headers } from 'next/headers'

export default async function Page() {
  const headersList = await headers()
  const userAgent = headersList.get('user-agent')
}
```

```
// Archivo: app/page.js switcher
import { headers } from 'next/headers'

export default async function Page() {
  const headersList = await headers()
  const userAgent = headersList.get('user-agent')
}
```

Reference

Parameters

`headers` does not take any parameters.

Returns

`headers` returns a **read-only** Web Headers object.

- `Headers.entries()`: Returns an `iterator` allowing to go through all key/value pairs contained in this object.
- `Headers.forEach()`: Executes a provided function once for each key/value pair in this `Headers` object.
- `Headers.get()`: Returns a `String` sequence of all the values of a header within a `Headers` object with a given name.
- `Headers.has()`: Returns a boolean stating whether a `Headers` object contains a certain header.
- `Headers.keys()`: Returns an `iterator` allowing you to go through all keys of the key/value pairs contained in this object.
- `Headers.values()`: Returns an `iterator` allowing you to go through all values of the key/value pairs contained in this object.

Good to know

- `headers` is an **asynchronous** function that returns a promise. You must use `async/await` or React's `use` function.
  - In version 14 and earlier, `headers` was a synchronous function. To help with backwards compatibility, you can still access it synchronously in Next.js 15, but this behavior will be deprecated in the future.
- Since `headers` is read-only, you cannot `set` or `delete` the outgoing request headers.
- `headers` is a Request-time API whose returned values cannot be known ahead of time. Using it in will opt a route into **dynamic rendering**.
- With Cache Components, calling `headers()` outside of a `<Suspense>` boundary prevents the route from being prerendered. See Next.js encountered runtime data during prerendering for fix options.

Examples

Using the Authorization header

```
// Archivo: app/page.js
import { headers } from 'next/headers'

export default async function Page() {
  const authorization = (await headers()).get('authorization')
  const res = await fetch('...', {
    headers: { authorization }, // Forward the authorization header
  })
  const user = await res.json()

  return <h1>{user.name}</h1>
}
```

Version History

Version	Changes
`v15.0.0-RC`	`headers` is now an async function. A codemod is available.
`v13.0.0`	`headers` introduced.

ImageResponse

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/image-response
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the ImageResponse constructor.*

The `ImageResponse` constructor allows you to generate dynamic images using JSX and CSS. This is useful for generating social media images such as Open Graph images, Twitter cards, and more.

Reference

Parameters

The following parameters are available for `ImageResponse`:

```
import { ImageResponse } from 'next/og'

new ImageResponse(
  element: ReactElement,
  options: {
    width?: number = 1200
    height?: number = 630
    emoji?: 'twemoji' | 'blobmoji' | 'noto' | 'openmoji' = 'twemoji',
    fonts?: {
      name: string,
      data: ArrayBuffer,
      weight: number,
      style: 'normal' | 'italic'
    }[]
    debug?: boolean = false

    // Options that will be passed to the HTTP response
    status?: number = 200
    statusText?: string
    headers?: Record<string, string>
  },
)
```

> Examples are available in the Vercel OG Playground.

Supported HTML and CSS features

`ImageResponse` supports common CSS properties including flexbox and absolute positioning, custom fonts, text wrapping, centering, and nested images.

Please refer to Satori's documentation for a list of supported HTML and CSS features.

Behavior

- `ImageResponse` uses @vercel/og, Satori, and Resvg to convert HTML and CSS into PNG.
- Only flexbox and a subset of CSS properties are supported. Advanced layouts (e.g. `display: grid`) will not work.
- Maximum bundle size of `500KB`. The bundle size includes your JSX, CSS, fonts, images, and any other assets. If you exceed the limit, consider reducing the size of any assets or fetching at runtime.
- Only `ttf`, `otf`, and `woff` font formats are supported. To maximize the font parsing speed, `ttf` or `otf` are preferred over `woff`.

Examples

Route Handlers

`ImageResponse` can be used in Route Handlers to generate images dynamically at request time.

```
// Archivo: app/api/route.js
import { ImageResponse } from 'next/og'

export async function GET() {
  try {
    return new ImageResponse(
      (
        <div
          style={{
            height: '100%',
            width: '100%',
            display: 'flex',
            flexDirection: 'column',
            alignItems: 'center',
            justifyContent: 'center',
            backgroundColor: 'white',
            padding: '40px',
          }}
        >
          <div
            style={{
              fontSize: 60,
              fontWeight: 'bold',
              color: 'black',
              textAlign: 'center',
            }}
          >
            Welcome to My Site
          </div>
          <div
            style={{
              fontSize: 30,
              color: '#666',
              marginTop: '20px',
            }}
          >
            Generated with Next.js ImageResponse
          </div>
        </div>
      ),
      {
        width: 1200,
        height: 630,
      }
    )
  } catch (e) {
    console.log(`${e.message}`)
    return new Response(`Failed to generate the image`, {
      status: 500,
    })
  }
}
```

File-based Metadata

You can use `ImageResponse` in a `opengraph-image.tsx` file to generate Open Graph images at build time or dynamically at request time.

```tsx
// Archivo: app/opengraph-image.tsx
import { ImageResponse } from 'next/og'

// Image metadata
export const alt = 'My site'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ImageResponse JSX element
      <div
        style={{
          fontSize: 128,
          background: 'white',
          width: '100%',
          height: '100%',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        My site
      </div>
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
    }
  )
}
```

Custom fonts

You can use custom fonts in your `ImageResponse` by providing a `fonts` array in the options. The font doesn't depend on request data, so read it once at module scope. See Predictable values.

```tsx
// Archivo: app/opengraph-image.tsx
import { ImageResponse } from 'next/og'
import { readFile } from 'node:fs/promises'
import { join } from 'node:path'

// Image metadata
export const alt = 'My site'
export const size = {
  width: 1200,
  height: 630,
}

export const contentType = 'image/png'

const interSemiBold = await readFile(
  join(process.cwd(), 'assets/Inter-SemiBold.ttf')
)

// Image generation
export default async function Image() {
  return new ImageResponse(
    (
      // ...
    ),
    // ImageResponse options
    {
      // For convenience, we can re-use the exported opengraph-image
      // size config to also set the ImageResponse's width and height.
      ...size,
      fonts: [
        {
          name: 'Inter',
          data: interSemiBold,
          style: 'normal',
          weight: 400,
        },
      ],
    }
  )
}
```

Version History

Version	Changes
`v14.0.0`	`ImageResponse` moved from `next/server` to `next/og`
`v13.3.0`	`ImageResponse` can be imported from `next/server`.
`v13.0.0`	`ImageResponse` introduced via `@vercel/og` package.

Functions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for Next.js Functions and Hooks.*

io

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/io
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the io function.*

When Cache Components is enabled, Next.js needs you to decide whether to capture a synchronous value like `new Date()` or `Math.random()` once and reuse it for every visitor, or produce it fresh for each request.

To capture the value in the static shell, wrap it in `"use cache"`. To keep it out of the static shell, use `await io()`, which suspends during prerendering.

During a request, inside cached scopes, in the browser, and apps without Cache Components (including the Pages Router), calling `io()` resolves immediately.

Usage

Call `io()` to inform Next.js that an IO operation follows.

In a Server Component, call `await io()` before reading a value like `new Date()`, `Math.random()`, `crypto.randomUUID()`, or a synchronous database driver such as `node:sqlite`:

```
// Archivo: app/page.tsx highlight={13} switcher
import { Suspense } from 'react'
import { io } from 'next/cache'

export default function Page() {
  return (
    ``<Suspense>``
  )
}

async function CurrentTime() {
  await io()
  return <p>{new Date().toISOString()}</p>
}
```

```
// Archivo: app/page.js highlight={13} switcher
import { Suspense } from 'react'
import { io } from 'next/cache'

export default function Page() {
  return (
    ``<Suspense>``
  )
}

async function CurrentTime() {
  await io()
  return <p>{new Date().toISOString()}</p>
}
```

In this route `CurrentTime` is wrapped in a `<Suspense>` boundary, during prerender, the `await io()` suspends and the fallback ships in the static shell. If `CurrentTime` were rendered inside a `"use cache"` scope instead, `io()` would be a no-op, the value is captured into the static shell and no `<Suspense>` boundary is required.

In a Client Component, call `io()` with React's `use` hook before reading a synchronous source like `Date.now()`. Client Components prerender on the server during SSR, where the read would otherwise be included in the static shell:

```
// Archivo: app/components.tsx highlight={6} switcher
'use client'
import { use } from 'react'
import { io } from 'next/cache'

export function CurrentTime() {
  use(io())
  return <div>{Date.now()}</div>
}
```

```
// Archivo: app/components.js highlight={6} switcher
'use client'
import { use } from 'react'
import { io } from 'next/cache'

export function CurrentTime() {
  use(io())
  return <div>{Date.now()}</div>
}
```

When you don't need `io()`

- The component already uses a Request-time API like `cookies()` or `headers()`. The request-time API is itself the suspension point.
- The data comes from an awaited `fetch` or async database query wrapped in `<Suspense>`. The `await` is the suspension point.

How `io()` differs from `connection()`

The `connection()` function excludes the code that follows it from the static shell, but it stays suspended until a full user navigation reaches the server, so it also blocks prefetches. `io()` suspends like any other asynchronous function, so the code after it can be wrapped in `"use cache"` and prefetched and cached on the client. Prefer `io()` over `connection()`, and reach for `connection()` only when you need to wait for a real user request.

Reference

Type

```
function io(): Promise<void>
```

Parameters

- The function does not accept any parameters.

Returns

A `Promise<void>`. With Cache Components enabled, awaiting this promise stops prerendering so the code that follows is excluded from the prerender output. In every other context (real requests, cache scopes, `generateStaticParams`, the browser, and routes without Cache Components), it resolves immediately.

Version History

Version	Changes
`v16.3.0`	`io` added.

NextRequest

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/next-request
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for NextRequest.*

NextRequest extends the Web Request API with additional convenience methods.

`cookies`

Read or mutate the `Set-Cookie` header of the request.

`set(name, value)`

Given a name, set a cookie with the given value on the request.

```
// Given incoming request /home
// Set a cookie to hide the banner
// request will have a `Set-Cookie:show-banner=false;path=/home` header
request.cookies.set('show-banner', 'false')
```

`get(name)`

Given a cookie name, return the value of the cookie. If the cookie is not found, `undefined` is returned. If multiple cookies are found, the first one is returned.

```
// Given incoming request /home
// { name: 'show-banner', value: 'false', Path: '/home' }
request.cookies.get('show-banner')
```

`getAll()`

Given a cookie name, return the values of the cookie. If no name is given, return all cookies on the request.

```
// Given incoming request /home
// [
//   { name: 'experiments', value: 'new-pricing-page', Path: '/home' },
//   { name: 'experiments', value: 'winter-launch', Path: '/home' },
// ]
request.cookies.getAll('experiments')
// Alternatively, get all cookies for the request
request.cookies.getAll()
```

`delete(name)`

Given a cookie name, delete the cookie from the request.

```
// Returns true for deleted, false is nothing is deleted
request.cookies.delete('experiments')
```

`has(name)`

Given a cookie name, return `true` if the cookie exists on the request.

```
// Returns true if cookie exists, false if it does not
request.cookies.has('experiments')
```

`clear()`

Remove all cookies from the request.

```
request.cookies.clear()
```

`nextUrl`

Extends the native `URL` API with additional convenience methods, including Next.js specific properties.

```
// Given a request to /home, pathname is /home
request.nextUrl.pathname
// Given a request to /home?name=lee, searchParams is { 'name': 'lee' }
request.nextUrl.searchParams
```

The following options are available:

> 📘 **Nota para Pages Router:**
>
> | Property | Type | Description |
> | --- | --- | --- |
> | `basePath` | `string` | The base path of the URL. |
> | `buildId` | `string` \| `undefined` | The build identifier of the Next.js application. Can be customized. |
> | `defaultLocale` | `string` \| `undefined` | The default locale for internationalization. |
> | `domainLocale` | | |
> | - `defaultLocale` | `string` | The default locale within a domain. |
> | - `domain` | `string` | The domain associated with a specific locale. |
> | - `http` | `boolean` \| `undefined` | Indicates if the domain is using HTTP. |
> | `locales` | `string[]` \| `undefined` | An array of available locales. |
> | `locale` | `string` \| `undefined` | The currently active locale. |
> | `url` | `URL` | The URL object. |

Property	Type	Description
`basePath`	`string`	The base path of the URL.
`buildId`	`string` | `undefined`	The build identifier of the Next.js application. Can be customized.
`pathname`	`string`	The pathname of the URL.
`searchParams`	`Object`	The search parameters of the URL.

> **Note:** The internationalization properties from the Pages Router are not available for usage in the App Router. Learn more about internationalization with the App Router.

Version History

Version	Changes
`v15.0.0`	`ip` and `geo` removed.

NextResponse

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/next-response
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for NextResponse.*

NextResponse extends the Web Response API with additional convenience methods.

`cookies`

Read or mutate the `Set-Cookie` header of the response.

`set(name, value)`

Given a name, set a cookie with the given value on the response.

```
// Given incoming request /home
let response = NextResponse.next()
// Set a cookie to hide the banner
response.cookies.set('show-banner', 'false')
// Response will have a `Set-Cookie:show-banner=false;path=/home` header
return response
```

`get(name)`

Given a cookie name, return the value of the cookie. If the cookie is not found, `undefined` is returned. If multiple cookies are found, the first one is returned.

```
// Given incoming request /home
let response = NextResponse.next()
// { name: 'show-banner', value: 'false', Path: '/home' }
response.cookies.get('show-banner')
```

`getAll()`

Given a cookie name, return the values of the cookie. If no name is given, return all cookies on the response.

```
// Given incoming request /home
let response = NextResponse.next()
// [
//   { name: 'experiments', value: 'new-pricing-page', Path: '/home' },
//   { name: 'experiments', value: 'winter-launch', Path: '/home' },
// ]
response.cookies.getAll('experiments')
// Alternatively, get all cookies for the response
response.cookies.getAll()
```

`has(name)`

Given a cookie name, return `true` if the cookie exists on the response.

```
// Given incoming request /home
let response = NextResponse.next()
// Returns true if cookie exists, false if it does not
response.cookies.has('experiments')
```

`delete(name)`

Given a cookie name, delete the cookie from the response.

```
// Given incoming request /home
let response = NextResponse.next()
// Returns true for deleted, false if nothing is deleted
response.cookies.delete('experiments')
```

`json()`

Produce a response with the given JSON body.

```
// Archivo: app/api/route.ts switcher
import { NextResponse } from 'next/server'

export async function GET(request: Request) {
  return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 })
}
```

```
// Archivo: app/api/route.js switcher
import { NextResponse } from 'next/server'

export async function GET(request) {
  return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 })
}
```

`redirect()`

Produce a response that redirects to a URL.

```
import { NextResponse } from 'next/server'

return NextResponse.redirect(new URL('/new', request.url))
```

The URL can be created and modified before being used in the `NextResponse.redirect()` method. For example, you can use the `request.nextUrl` property to get the current URL, and then modify it to redirect to a different URL.

```
import { NextResponse } from 'next/server'

// Given an incoming request ...
const loginUrl = new URL('/login', request.url)
// Add ?from=/incoming-url to the /login URL
loginUrl.searchParams.set('from', request.nextUrl.pathname)
// And redirect to the new URL
return NextResponse.redirect(loginUrl)
```

`rewrite()`

Produce a response that rewrites (proxies) the given URL while preserving the original URL.

```
import { NextResponse } from 'next/server'

// Incoming request: /about, browser shows /about
// Rewritten request: /proxy, browser shows /about
return NextResponse.rewrite(new URL('/proxy', request.url))
```

`next()`

The `next()` method is useful for Proxy, as it allows you to return early and continue routing.

```
import { NextResponse } from 'next/server'

return NextResponse.next()
```

You can also forward `headers` upstream when producing the response, using `NextResponse.next({ request: { headers } })`:

```
import { NextResponse } from 'next/server'

// Given an incoming request ...
const newHeaders = new Headers(request.headers)
// Add a new header
newHeaders.set('x-version', '123')
// Forward the modified request headers upstream
return NextResponse.next({
  request: {
    // New request headers
    headers: newHeaders,
  },
})
```

This forwards `newHeaders` upstream to the target page, route, or server action, and does not expose them to the client. While this pattern is useful for passing data upstream, it should be used with caution because the headers containing this data may be forwarded to external services.

In contrast, `NextResponse.next({ headers })` is a shorthand for sending headers from proxy to the client. This is **NOT** good practice and should be avoided. Among other reasons because setting response headers like `Content-Type`, can override framework expectations (for example, the `Content-Type` used by Server Actions), leading to failed submissions or broken streaming responses.

```
import { type NextRequest, NextResponse } from 'next/server'

async function proxy(request: NextRequest) {
  const headers = await injectAuth(request.headers)
  // DO NOT forward headers like this
  return NextResponse.next({ headers })
}
```

In general, avoid copying all incoming request headers because doing so can leak sensitive data to clients or upstream services.

Prefer a defensive approach by creating a subset of incoming request headers using an allow-list. For example, you might discard custom `x-*` headers and only forward known-safe headers:

```
import { type NextRequest, NextResponse } from 'next/server'

function proxy(request: NextRequest) {
  const incoming = new Headers(request.headers)
  const forwarded = new Headers()

  for (const [name, value] of incoming) {
    const headerName = name.toLowerCase()
    // Keep only known-safe headers, discard custom x-* and other sensitive ones
    if (
      !headerName.startsWith('x-') &&
      headerName !== 'authorization' &&
      headerName !== 'cookie'
    ) {
      // Preserve original header name casing
      forwarded.set(name, value)
    }
  }

  return NextResponse.next({
    request: {
      headers: forwarded,
    },
  })
}
```

next/root-params

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/next-root-params
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the next/root-params module that provides access to root-level route parameters.*

The `next/root-params` module provides getter functions for accessing root-level parameters in **Server Components**. Each root parameter is exported as an async function that resolves to the parameter value for the current route.

The export names are generated from your dynamic segment folder names. For example, if your root layout is inside `app/[locale]`, you import `locale` from `next/root-params`.

This is useful for values like a language or locale segment that need to be accessed across your application, for example in shared utilities, layouts, and deeply nested components, without passing them down as props.

```
// Archivo: app/[lang]/layout.tsx highlight={1,5} switcher
import { lang } from 'next/root-params'

export default async function RootLayout(props: LayoutProps<'/[lang]'>) {
  return (
    <html lang={await lang()}>
      <body>{props.children}</body>
    </html>
  )
}
```

```
// Archivo: app/[lang]/layout.js highlight={1,5} switcher
import { lang } from 'next/root-params'

export default async function RootLayout({ children }) {
  return (
    <html lang={await lang()}>
      <body>{children}</body>
    </html>
  )
}
```

Root parameters are the dynamic segments that appear before the root layout. Unlike the regular `params` prop, root parameter getters can be called from any Server Component in your application without prop drilling. See Why root parameters are special for more details.

> **Good to know:**
>
> - Root parameter names must be valid JavaScript function identifiers. Kebab-cased segment names (e.g. `[post-slug]`) are not supported and will cause an error at dev time or during build.
> - `next/root-params` can be used in Server Components. It cannot be used in Client Components, Server Actions, or Route Handlers. Support for Route Handlers is planned for a future release.
> - Types for the `next/root-params` exports are generated during `next dev` , `next build` , or `next typegen` , the same as `PageProps` and `LayoutProps` .

Root parameters and `generateStaticParams`

Root parameters are available as soon as you create the routes that define them. A `generateStaticParams` function is only required with Cache Components, where each root parameter must have at least one value or the build fails.

With a single root parameter:

```
// Archivo: app/[lang]/layout.tsx highlight={11,12,13} switcher
import { lang } from 'next/root-params'

export default async function RootLayout(props: LayoutProps<'/[lang]'>) {
  return (
    <html lang={await lang()}>
      <body>{props.children}</body>
    </html>
  )
}

export async function generateStaticParams() {
  return [{ lang: 'en' }, { lang: 'fr' }]
}
```

```
// Archivo: app/[lang]/layout.js highlight={11,12,13} switcher
import { lang } from 'next/root-params'

export default async function RootLayout({ children }) {
  return (
    <html lang={await lang()}>
      <body>{children}</body>
    </html>
  )
}

export async function generateStaticParams() {
  return [{ lang: 'en' }, { lang: 'fr' }]
}
```

With multiple root parameters, return a value for each:

```
// Archivo: app/[lang]/[locale]/layout.tsx
export async function generateStaticParams() {
  return [
    { lang: 'en', locale: 'us' },
    { lang: 'en', locale: 'uk' },
  ]
}
```

Reading root parameters in a nested `generateStaticParams`

Inside a nested segment's `generateStaticParams`, you can read a root parameter directly with its getter, instead of destructuring it from the `params` argument:

```
// Archivo: app/[lang]/posts/[slug]/page.tsx highlight={1,4} switcher
import { lang } from 'next/root-params'

export async function generateStaticParams() {
  const language = await lang()
  const posts = await fetch(
    `https://api.example.com/posts?lang=${language}`
  ).then((res) => res.json())
  return posts.map((post) => ({ slug: post.slug }))
}

export default async function Page() {
  // ...
}
```

```
// Archivo: app/[lang]/posts/[slug]/page.js highlight={1,4} switcher
import { lang } from 'next/root-params'

export async function generateStaticParams() {
  const language = await lang()
  const posts = await fetch(
    `https://api.example.com/posts?lang=${language}`
  ).then((res) => res.json())
  return posts.map((post) => ({ slug: post.slug }))
}

export default async function Page() {
  // ...
}
```

Root parameters and other route parameters

Root parameters are the route parameters from dynamic segments that appear in the path up to the root layout file. Other route parameters deeper in the route tree are accessed through the `params` prop.

Given this file structure:

- `app/[lang]/layout.tsx` (root layout)
- `app/[lang]/posts/[slug]/page.tsx`

Only `lang` is a root parameter. `slug` is a regular route parameter:

```tsx
// Archivo: app/[lang]/posts/[slug]/page.tsx highlight={1,7} switcher
import { lang } from 'next/root-params'

export default async function PostPage(
  props: PageProps<'/[lang]/posts/[slug]'>
) {
  const { slug } = await props.params
  const language = await lang()

  return (
    <article>
      <p>Language: {language}</p>
      <p>Post: {slug}</p>
    </article>
  )
}
```

```jsx
// Archivo: app/[lang]/posts/[slug]/page.js highlight={1,5} switcher
import { lang } from 'next/root-params'

export default async function PostPage({ params }) {
  const { slug } = await params
  const language = await lang()

  return (
    <article>
      <p>Language: {language}</p>
      <p>Post: {slug}</p>
    </article>
  )
}
```

Accessing root parameters in shared code

Root parameter getters are module imports, they can be called from any Server Component or server-side utility, not just layouts and pages:

```tsx
// Archivo: lib/get-translations.ts highlight={4}
import { lang } from 'next/root-params'

export async function getTranslations() {
  const language = await lang()
  // Load translations based on the root language parameter
  return import(`@/locales/${language}.json`)
}
```

> **Good to know:** You do not need to add `import 'server-only'` to files that use `next/root-params`. The import already fails at build time if used in a Client Component.

Behavior with caching directives

Because root parameter getters are imported functions, Next.js can track which ones a cached function uses. Only those root parameters become part of the cache key, so cache entries are not split across unrelated parameter values.

```tsx
// Archivo: app/[lang]/components/cached-nav.tsx highlight={7} switcher
import { lang } from 'next/root-params'

// The cache key for this function only includes `lang`,
// not every dynamic segment in the route.
async function getNavigation() {
  'use cache'
  const language = await lang()
  const res = await fetch(`https://api.example.com/nav?lang=${language}`)
  return res.json()
}

export default async function CachedNav() {
  const nav = await getNavigation()
  return <nav></nav>
}
```

```jsx
// Archivo: app/[lang]/components/cached-nav.js highlight={7} switcher
import { lang } from 'next/root-params'

// The cache key for this function only includes `lang`,
// not every dynamic segment in the route.
async function getNavigation() {
  'use cache'
  const language = await lang()
  const res = await fetch(`https://api.example.com/nav?lang=${language}`)
  return res.json()
}

export default async function CachedNav() {
  const nav = await getNavigation()
  return <nav></nav>
}
```

With the regular `params` prop, you would need to `await params` outside the cached function and pass the value in as an argument:

```tsx
// Archivo: app/[lang]/page.tsx highlight={6,17} switcher
import { lang } from 'next/root-params'

// With root params: call directly inside the cached function
async function getDataWithRootParams() {
  'use cache'
  const language = await lang()
  return fetch(`https://api.example.com/data?lang=${language}`)
}

// Without root params: await params outside, pass as argument
async function getDataWithParams(language: string) {
  'use cache'
  return fetch(`https://api.example.com/data?lang=${language}`)
}

export default async function Page(props: PageProps<'/[lang]'>) {
  const { lang: language } = await props.params
  const data = await getDataWithParams(language)
  // ...
}
```

```
// Archivo: app/[lang]/page.js highlight={6,17} switcher
import { lang } from 'next/root-params'

// With root params: call directly inside the cached function
async function getDataWithRootParams() {
  'use cache'
  const language = await lang()
  return fetch(`https://api.example.com/data?lang=${language}`)
}

// Without root params: await params outside, pass as argument
async function getDataWithParams(language) {
  'use cache'
  return fetch(`https://api.example.com/data?lang=${language}`)
}

export default async function Page({ params }) {
  const { lang: language } = await params
  const data = await getDataWithParams(language)
  // ...
}
```

Multiple root layouts

When an application has multiple root layouts with different parameters, getter functions are typed to account for usage in any of all possible routes. A parameter that does not exist in every root layout has the type `string | undefined`.

Given this file structure:

- `app/dashboard/[id]/layout.tsx` (root layout with `id`)
- `app/marketing/layout.tsx` (root layout without `id`)

```
// Archivo: app/dashboard/[id]/page.tsx highlight={4} switcher
import { id } from 'next/root-params'

export default async function DashboardPage() {
  const dashboardId = await id() // string | undefined
  return <div>Dashboard: {dashboardId}</div>
}
```

```
// Archivo: app/dashboard/[id]/page.js highlight={4} switcher
import { id } from 'next/root-params'

export default async function DashboardPage() {
  const dashboardId = await id() // string | undefined
  return <div>Dashboard: {dashboardId}</div>
}
```

Since `id` does not exist in the marketing root layout, `await id()` returns `undefined` when accessed from marketing routes.

Catch-all and optional catch-all segments

Root parameters work with catch-all and optional catch-all segments:

```tsx
// Archivo: app/docs/[...path]/layout.tsx highlight={6} switcher
import { path } from 'next/root-params'

export default async function DocsLayout(
  props: LayoutProps<'/docs/[...path]'>
) {
  const segments = await path() // string[] | undefined
  return (
    <div>
      <nav>Path: {segments?.join(' / ')}</nav>
      {props.children}
    </div>
  )
}
```

```jsx
// Archivo: app/docs/[...path]/layout.js highlight={4} switcher
import { path } from 'next/root-params'

export default async function DocsLayout({ children }) {
  const segments = await path() // string[] | undefined
  return (
    <div>
      <nav>Path: {segments?.join(' / ')}</nav>
      {children}
    </div>
  )
}
```

Returns

Each root parameter getter returns a `Promise` that resolves to one of the following:

Segment type	Example	Return type
Dynamic	`[id]`	`string`
Catch-all	`[...path]`	`string[]`
Optional catch-all	`[[...path]]`	`string[] | undefined`

If the parameter does not exist in the current route's root layout (see Multiple root layouts), the return type includes `undefined`.

Restrictions

Most of these are permanent constraints of how root parameters work. The exception is Route Handlers, which are not supported yet but are planned for a future release.

Server Components only

Root parameter getters can only be used in Server Components:

```tsx
// This will cause a build error
'use client'

import { lang } from 'next/root-params' // Error: Cannot import in Client Component
```

Not available in `unstable_cache`

Calling a root parameter getter inside `unstable_cache` will throw a runtime error. Use `"use cache"` instead.

```
import { lang } from 'next/root-params'
import { unstable_cache } from 'next/cache'

const getCachedData = unstable_cache(async () => {
  const language = await lang() // Error: Not supported inside unstable_cache
  return fetch(`https://api.example.com/data?lang=${language}`)
})
```

Not available in Server Actions

```
import { lang } from 'next/root-params'

async function submitForm() {
  'use server'
  const language = await lang() // Error: Not supported in Server Actions
}
```

Why root parameters are special

The root layout is the top-level rendering boundary. The route parameters before it are shared by all routes under that root layout, which is what makes them safe to access from any Server Component in that tree. Route parameters deeper in the route vary depending on which child page is being rendered, so they are only available through the `params` prop in the page or layout that defines them.

```
app/
  [lang]/              ← root parameter (shared by all routes below)
    layout.tsx         ← root layout
    page.tsx           ← has no slug, doesn't know about blog or store
    blog/
      [slug]/          ← route parameter
        page.tsx
    store/
      [...slug]/       ← catch-all route parameter with the same name
        page.tsx
```

In this example, `lang` is the only root parameter. The `page.tsx` directly under the root layout has no `slug` at all. The blog and store branches both define `slug`, but with different segment types (`[slug]` vs `[...slug]`). Each route only knows about its own route parameters. The same name can have different types, different meanings, or not exist at all depending on the route. This is why route parameters below the root layout are accessed through the `params` prop rather than as global getters.

The exception is multiple root layouts. When different root layouts define different route parameters, the getter functions include `undefined` in their return type to account for routes where a given parameter may not exist.

Version History

Version	Changes
`v16.3.0`	`next/root-params` introduced.

notFound

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/not-found
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the notFound function.*

The `notFound` function throws an error that renders a Next.js 404 page. It's useful for handling missing resources in your application. You can customize the UI using the `not-found.js` file.

Invoking `notFound()` throws a `NEXT_HTTP_ERROR_FALLBACK;404` error and terminates rendering of the route segment where it was thrown. Next.js also injects a `<meta name="robots" content="noindex" />` tag so the page is not indexed. Because it works by throwing, call it in the render path: a component, or a function a component `await`s. A call left in an un-awaited promise throws where nothing catches it, and no not-found UI renders (in development the server logs `⨯ unhandledRejection: NEXT_HTTP_ERROR_FALLBACK;404`).

`notFound()` can be invoked in Server Components, Server Functions, and Route Handlers.

```
// Archivo: app/user/[id]/page.tsx switcher
import { notFound } from 'next/navigation'

async function fetchUser(id: string) {
  const res = await fetch('https:// ... ')
  if (!res.ok) return undefined
  return res.json()
}

export default async function Profile({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const user = await fetchUser(id)

  if (!user) {
    notFound()
  }

  // ...
}
```

```
// Archivo: app/user/[id]/page.js switcher
import { notFound } from 'next/navigation'

async function fetchUser(id) {
  const res = await fetch('https:// ... ')
  if (!res.ok) return undefined
  return res.json()
}

export default async function Profile({ params }) {
  const { id } = await params
  const user = await fetchUser(id)

  if (!user) {
    notFound()
  }

  // ...
}
```

Good to know

You do not need to write `return notFound()`. Calling it is enough, because it throws an exception that stops function execution. TypeScript understands this from its `never` return type, so a value you check first stays narrowed afterward:

```
// fetchUser resolves to a user object, or undefined
const user = await fetchUser(id)

if (!user) {
  notFound()
}

// user is defined here
return <Profile user={user} />
```

Like any exception, it travels up the call stack until something catches it. A `try/catch` around the call suppresses it, and the not-found UI won't render. If you need to catch errors near the call, use `unstable_rethrow` to let the interrupt through first.

Examples

Calling `notFound()` after streaming has started

To keep a page's shell and loading UI visible while data loads, do the existence check inside a component wrapped in [`````) }

````
```jsx
// Archivo: app/blog/[slug]/page.js switcher highlight={8}
import { Suspense } from 'react'
import Link from 'next/link'
import { notFound } from 'next/navigation'

async function getPost(slug) {
  const res = await fetch(`https://api.example.com/posts/${slug}`)
  if (res.status === 404) {
    notFound()
  }
  if (!res.ok) {
    throw new Error(`Failed to load post: ${res.status}`)
  }
  return res.json()
}

async function Article({ slug }) {
  const post = await getPost(slug)
  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </article>
  )
}

export default async function PostPage({ params }) {
  const { slug } = await params

  return (
    <section>
      [Blog](/blog)
      ``<Suspense>``
    </section>
  )
}
````

When the post doesn't exist, `getPost` calls `notFound()`, which throws. Because this happens during rendering, the exception propagates to the nearest `not-found` boundary, which renders in place of the streamed-in content, even though the page shell has already been sent.

Add a `not-found.tsx` alongside the route to define that UI. Without one, the nearest parent `not-found` boundary renders, falling back to Next.js's default 404 page:
````

```
// Archivo: app/blog/[slug]/not-found.tsx switcher
export default function NotFound() {
  return (
    <section>
      <h1>Post not found</h1>
      <p>The post you're looking for doesn't exist.</p>
    </section>
  )
}
```

```
// Archivo: app/blog/[slug]/not-found.js switcher
export default function NotFound() {
  return (
    <section>
      <h1>Post not found</h1>
      <p>The post you're looking for doesn't exist.</p>
    </section>
  )
}
```

The trade-off is the HTTP status code. Because the check runs inside the `<Suspense>` boundary, the response has already begun streaming as a `200`, and the status can't change once streaming has started. The `noindex` tag keeps a soft 404 out of search results. To return a real `404` status, the resource has to be checked before the response streams. With Cache Components, every dynamic route streams a static shell first, so run that check in `proxy` instead. See Status codes.

Serving a 404 from a Route Handler

`notFound()` also works in a Route Handler, where it serves a `404` to the caller.

```
// Archivo: app/api/posts/[slug]/route.ts switcher
import { NextResponse } from 'next/server'
import { notFound } from 'next/navigation'

export async function GET(
  request: Request,
  { params }: RouteContext<'/api/posts/[slug]'>
) {
  const { slug } = await params
  const res = await fetch(`https://api.example.com/posts/${slug}`)
  if (!res.ok) {
    notFound()
  }
  return NextResponse.json(await res.json())
}
```

```
// Archivo: app/api/posts/[slug]/route.js switcher
import { NextResponse } from 'next/server'
import { notFound } from 'next/navigation'

export async function GET(request, { params }) {
  const { slug } = await params
  const res = await fetch(`https://api.example.com/posts/${slug}`)
  if (!res.ok) {
    notFound()
  }
  return NextResponse.json(await res.json())
}
```

Version History

Version	Changes
`v13.0.0`	`notFound` introduced.

permanentRedirect

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/permanentRedirect
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the permanentRedirect function.*

The `permanentRedirect` function allows you to redirect the user to another URL. `permanentRedirect` can be used in Server Components, Client Components, Route Handlers, and Server Functions.

When used in a streaming context, this will insert a meta tag to emit the redirect on the client side. In a Server Action, `permanentRedirect` performs a client-side navigation when JavaScript is available. For progressive enhancement form submissions, it serves a 303 HTTP redirect response. Otherwise, it serves a 308 (Permanent) HTTP redirect response.

If a resource doesn't exist, you can use the `notFound` function instead.

Parameters

The `permanentRedirect` function accepts two arguments:

```
permanentRedirect(path, type)
```

Parameter	Type	Description
`path`	`string`	The URL to redirect to. Can be a relative or absolute path.
`type`	`'replace'` (default) or `'push'` (default in Server Actions)	The type of redirect to perform.

By default, `permanentRedirect` will use `push` (adding a new entry to the browser history stack) in Server Actions and `replace` (replacing the current URL in the browser history stack) everywhere else. You can override this behavior by specifying the `type` parameter.

The `RedirectType` object contains the available options for the `type` parameter.

```
import { permanentRedirect, RedirectType } from 'next/navigation'

permanentRedirect('/redirect-to', RedirectType.replace)
// or
permanentRedirect('/redirect-to', RedirectType.push)
```

The `type` parameter has no effect when used in Server Components.

Returns

`permanentRedirect` does not return a value.

Behavior

- In Server Actions and Route Handlers, `permanentRedirect` should be called **outside** the `try` block when using `try/catch` statements because it throws an error.
- If you prefer to return a 307 (Temporary) HTTP redirect instead of 308 (Permanent), you can use the `redirect` function instead.

Example

Invoking the `permanentRedirect()` function throws a `NEXT_REDIRECT` error and terminates rendering of the route segment in which it was thrown.

```
// Archivo: app/team/[id]/page.js
import { permanentRedirect } from 'next/navigation'

async function fetchTeam(id) {
  const res = await fetch('https:// ... ')
  if (!res.ok) return undefined
  return res.json()
}

export default async function Profile({ params }) {
  const { id } = await params
  const team = await fetchTeam(id)
  if (!team) {
    permanentRedirect('/login')
  }

  // ...
}
```

> **Good to know**: `permanentRedirect` does not require you to use `return permanentRedirect()` as it uses the TypeScript `never` type.

---

redirect

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/redirect
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the redirect function.*

The `redirect` function allows you to redirect the user to another URL. `redirect` can be used while rendering in Server and Client Components, Route Handlers, and Server Functions.

When used in a streaming context, this will insert a meta tag to emit the redirect on the client side. In a Server Action, `redirect` performs a client-side navigation when JavaScript is available. For progressive enhancement form submissions, it serves a 303 HTTP redirect response. Otherwise, it serves a 307 HTTP redirect response.

If a resource doesn't exist, you can use the `notFound` function instead.

Reference

Parameters

The `redirect` function accepts two arguments:

```
redirect(path, type)
```

Parameter	Type	Description
`path`	`string`	The URL to redirect to. Can be a relative or absolute path.
`type`	`'replace'` (default) or `'push'` (default in Server Actions)	The type of redirect to perform.

By default, `redirect` will use `push` (adding a new entry to the browser history stack) in Server Actions and `replace` (replacing the current URL in the browser history stack) everywhere else. You can override this behavior by specifying the `type` parameter.

The `RedirectType` object contains the available options for the `type` parameter.

```
import { redirect, RedirectType } from 'next/navigation'

redirect('/redirect-to', RedirectType.replace)
// or
redirect('/redirect-to', RedirectType.push)
```

The `type` parameter has no effect when used in Server Components.

Returns

`redirect` does not return a value.

Behavior

- In Server Actions and Route Handlers, redirect should be called **outside** the `try` block when using `try/catch` statements.
- If you prefer to return a 308 (Permanent) HTTP redirect instead of 307 (Temporary), you can use the `permanentRedirect` function instead.
- `redirect` throws an error so it should be called **outside** the `try` block when using `try/catch` statements.
- `redirect` can be called in Client Components during the rendering process but not in event handlers. You can use the `useRouter` hook instead.
- `redirect` also accepts absolute URLs and can be used to redirect to external links.
- If you'd like to redirect before the render process, use `next.config.js` or Proxy.

Example

Server Component

Invoking the `redirect()` function throws a `NEXT_REDIRECT` error and terminates rendering of the route segment in which it was thrown.

```
// Archivo: app/team/[id]/page.tsx switcher
import { redirect } from 'next/navigation'

async function fetchTeam(id: string) {
  const res = await fetch('https:// ... ')
  if (!res.ok) return undefined
  return res.json()
}

export default async function Profile({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const team = await fetchTeam(id)

  if (!team) {
    redirect('/login')
  }

  // ...
}
```

```
// Archivo: app/team/[id]/page.js switcher
import { redirect } from 'next/navigation'

async function fetchTeam(id) {
  const res = await fetch('https:// ... ')
  if (!res.ok) return undefined
  return res.json()
}

export default async function Profile({ params }) {
  const { id } = await params
  const team = await fetchTeam(id)

  if (!team) {
    redirect('/login')
  }

  // ...
}
```

> **Good to know**: `redirect` does not require you to use `return redirect()` as it uses the TypeScript `never` type.

Client Component

`redirect` can be directly used in a Client Component.

```
// Archivo: components/client-redirect.tsx switcher
'use client'

import { redirect, usePathname } from 'next/navigation'

export function ClientRedirect() {
  const pathname = usePathname()

  if (pathname.startsWith('/admin') && !pathname.includes('/login')) {
    redirect('/admin/login')
  }

  return <div>Login Page</div>
}
```

```
// Archivo: components/client-redirect.jsx switcher
'use client'

import { redirect, usePathname } from 'next/navigation'

export function ClientRedirect() {
  const pathname = usePathname()

  if (pathname.startsWith('/admin') && !pathname.includes('/login')) {
    redirect('/admin/login')
  }

  return <div>Login Page</div>
}
```

> **Good to know**: When using `redirect` in a Client Component on initial page load during Server-Side Rendering (SSR), it will perform a server-side redirect.

`redirect` can be used in a Client Component through a Server Action. If you need to use an event handler to redirect the user, you can use the `useRouter` hook.

```tsx
// Archivo: app/client-redirect.tsx switcher
'use client'

import { navigate } from './actions'

export function ClientRedirect() {
  return (
    <form action={navigate}>
      <input type="text" name="id" />
      <button>Submit</button>
    </form>
  )
}
```

```jsx
// Archivo: app/client-redirect.jsx switcher
'use client'

import { navigate } from './actions'

export function ClientRedirect() {
  return (
    <form action={navigate}>
      <input type="text" name="id" />
      <button>Submit</button>
    </form>
  )
}
```

```ts
// Archivo: app/actions.ts switcher
'use server'

import { redirect } from 'next/navigation'

export async function navigate(data: FormData) {
  redirect(`/posts/${data.get('id')}`)
}
```

```js
// Archivo: app/actions.js switcher
'use server'

import { redirect } from 'next/navigation'

export async function navigate(data) {
  redirect(`/posts/${data.get('id')}`)
}
```

FAQ

Why does `redirect` use 307 and 308?

When using `redirect()` you may notice that the status codes used are `307` for a temporary redirect, and `308` for a permanent redirect. While traditionally a `302` was used for a temporary redirect, and a `301` for a permanent redirect, many browsers changed the request method of the redirect, from a `POST` to `GET` request when using a `302`, regardless of the origins request method.

Taking the following example of a redirect from `/users` to `/people`, if you make a `POST` request to `/users` to create a new user, and are conforming to a `302` temporary redirect, the request method will be changed from a `POST` to a `GET` request. This doesn't make sense, as to create a new user, you should be making a `POST` request to `/people`, and not a `GET` request.

The introduction of the `307` status code means that the request method is preserved as `POST`.

- `302` - Temporary redirect, will change the request method from `POST` to `GET`
- `307` - Temporary redirect, will preserve the request method as `POST`

The `redirect()` method uses a `307` by default, instead of a `302` temporary redirect, meaning the request method is preserved. Server Action form submissions are an exception: they use a `303` response so the browser follows the redirect with a `GET` request. When JavaScript is available, Server Actions perform a client-side navigation instead of an HTTP redirect.

Learn more about HTTP Redirects.

Version History

Version	Changes
`v13.0.0`	`redirect` introduced.

refresh

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/refresh
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the refresh function.*

`refresh` allows you to refresh the client router from within a Server Action.

Usage

`refresh` can **only** be called from within Server Actions. It cannot be used in Route Handlers, Client Components, or any other context.

Parameters

```
refresh(): void;
```

Returns

`refresh` does not return a value.

Examples

```
// Archivo: app/actions.ts switcher
'use server'

import { refresh } from 'next/cache'

export async function createPost(formData: FormData) {
  const title = formData.get('title')
  const content = formData.get('content')

  // Create the post in your database
  const post = await db.post.create({
    data: { title, content },
  })

  refresh()
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { refresh } from 'next/cache'

export async function createPost(formData) {
  const title = formData.get('title')
  const content = formData.get('content')

  // Create the post in your database
  const post = await db.post.create({
    data: { title, content },
  })

  refresh()
}
```

Error when used outside Server Actions

```
// Archivo: app/api/posts/route.ts switcher
import { refresh } from 'next/cache'

export async function POST() {
  // This will throw an error
  refresh()
}
```

---

revalidatePath

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/revalidatePath
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the revalidatePath function.*

`revalidatePath` allows you to invalidate cached data on-demand for a specific path.

Usage

`revalidatePath` can be called in Server Functions and Route Handlers.

`revalidatePath` cannot be called in Client Components or Proxy, as it only works in server environments.

> **Good to know**:
>
> - **Server Functions**: Updates the UI immediately (if viewing the affected path). Currently, it also causes all previously visited pages to refresh when navigated to again. This behavior is temporary and will be updated in the future to apply only to the specific path.
> - **Route Handlers**: Marks the path for revalidation. The revalidation is done on the next visit to the specified path. This means calling `revalidatePath` with a dynamic route segment will not immediately trigger many revalidations at once. The invalidation only happens when the path is next visited.

Parameters

```
revalidatePath(path: string, type?: 'page' | 'layout'): void;
```

- `path`: Either a string that represents your route file structure. This can be a literal path like `/product/123`, or a route pattern with dynamic segments like `/product/[slug]`. Do not append `/page` or `/layout`, use the `type` parameter instead. Must not exceed 1024 characters. This value is case-sensitive. You do not need to include a trailing slash, regardless of your `trailingSlash` config.
- `type`: (optional) `'page'` or `'layout'` string to change the type of path to revalidate. If `path` contains a dynamic segment, for example `/product/[slug]`, this parameter is required. If `path` is a literal path like `/product/1`, omit `type`.

Use a literal path when you want to refresh a single page. Use a route pattern plus `type` to refresh all matching pages.

Returns

`revalidatePath` does not return a value.

What can be invalidated

The path parameter can point to pages, layouts, or route handlers:

- **Pages**: Invalidates the specific page
- **Layouts**: Invalidates the layout (the `layout.tsx` at that segment), all nested layouts beneath it, and all pages beneath them
- **Route Handlers**: Invalidates cached data accessed within route handlers. For example `revalidatePath("/api/data")` invalidates this GET handler:

```
// Archivo: app/api/data/route.ts
export async function GET() {
  const data = await fetch('https://api.vercel.app/blog', {
    cache: 'force-cache',
  })

  return Response.json(await data.json())
}
```

Using `revalidatePath` with rewrites

When using rewrites, you must pass the **destination** path (the actual route file location), not the source path that appears in the browser's address bar.

For example, if you have a rewrite from `/blog` to `/news`:

```
// Archivo: next.config.js
module.exports = {
  async rewrites() {
    return [
      {
        source: '/blog',
        destination: '/news',
      },
    ]
  },
}
```

To revalidate this page, use the destination path:

```
// Correct: use the destination path
revalidatePath('/news')

// Incorrect: the source path won't match the cache entry
revalidatePath('/blog')
```

This is because `revalidatePath` operates on the route file structure, not the URL visible to users. Cache entries are tagged based on which route file renders them.

Relationship with `revalidateTag` and `updateTag`

`revalidatePath`, `revalidateTag` and `updateTag` serve different purposes:

- **`revalidatePath`**: Invalidates a specific page or layout path
- **`revalidateTag`**: Marks data with specific tags as **stale**. Applies across all pages that use those tags
- **`updateTag`**: Expires data with specific tags. Applies across all pages that use those tags

When you call `revalidatePath`, only the specified path gets fresh data on the next visit. Other pages that use the same data tags will continue to serve cached data until those specific tags are also revalidated:

```
// Page A: /blog
const posts = await fetch('https://api.vercel.app/blog', {
  next: { tags: ['posts'] },
})

// Page B: /dashboard
const recentPosts = await fetch('https://api.vercel.app/blog?limit=5', {
  next: { tags: ['posts'] },
})
```

After calling `revalidatePath('/blog')`:

- **Page A (/blog)**: Shows fresh data (page re-rendered)
- **Page B (/dashboard)**: Still shows stale data (cache tag 'posts' not invalidated)

Learn about the difference between `revalidateTag` and `updateTag`.

Building revalidation utilities

`revalidatePath` and `updateTag` are complementary primitives that are often used together in utility functions to ensure comprehensive data consistency across your application:

```
'use server'

import { revalidatePath, updateTag } from 'next/cache'

export async function updatePost() {
  await updatePostInDatabase()

  revalidatePath('/blog') // Refresh the blog page
  updateTag('posts') // Refresh all pages using 'posts' tag
}
```

This pattern ensures that both the specific page and any other pages using the same data remain consistent.

Examples

Revalidating a specific path

```
import { revalidatePath } from 'next/cache'
revalidatePath('/blog/post-1')
```

This will invalidate one specific path for revalidation on the next page visit.

Revalidating a Page path

```
import { revalidatePath } from 'next/cache'
revalidatePath('/blog/[slug]', 'page')
// or with route groups
revalidatePath('/(main)/blog/[slug]', 'page')
```

This will invalidate any path that matches the provided `page` file for revalidation on the next page visit. This will *not* invalidate pages beneath the specific page. For example, `/blog/[slug]` won't invalidate `/blog/[slug]/[author]`.

Revalidating a Layout path

```
import { revalidatePath } from 'next/cache'
revalidatePath('/blog/[slug]', 'layout')
// or with route groups
revalidatePath('/(main)/post/[slug]', 'layout')
```

This will invalidate any path that matches the provided `layout` file for revalidation on the next page visit. This will cause pages beneath with the same layout to be invalidated and revalidated on the next visit. For example, in the above case, `/blog/[slug]/[another]` would also be invalidated and revalidated on the next visit.

Revalidating all data

```
import { revalidatePath } from 'next/cache'

revalidatePath('/', 'layout')
```

This will purge the Client Cache, and invalidate all cached data for revalidation on the next page visit.

Server Function

```
// Archivo: app/actions.ts switcher
'use server'

import { revalidatePath } from 'next/cache'

export default async function submit() {
  await submitForm()
  revalidatePath('/')
}
```

Route Handler

```
// Archivo: app/api/revalidate/route.ts switcher
import { revalidatePath } from 'next/cache'
import type { NextRequest } from 'next/server'

export async function GET(request: NextRequest) {
  const path = request.nextUrl.searchParams.get('path')

  if (path) {
    revalidatePath(path)
    return Response.json({ revalidated: true, now: Date.now() })
  }

  return Response.json({
    revalidated: false,
    now: Date.now(),
    message: 'Missing path to revalidate',
  })
}
```

```
// Archivo: app/api/revalidate/route.js switcher
import { revalidatePath } from 'next/cache'

export async function GET(request) {
  const path = request.nextUrl.searchParams.get('path')

  if (path) {
    revalidatePath(path)
    return Response.json({ revalidated: true, now: Date.now() })
  }

  return Response.json({
    revalidated: false,
    now: Date.now(),
    message: 'Missing path to revalidate',
  })
}
```

---

revalidateTag

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/revalidateTag
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the revalidateTag function.*

`revalidateTag` allows you to invalidate cached data on-demand for a specific cache tag.

This function is ideal for content where a slight delay in updates is acceptable, such as blog posts, product catalogs, or documentation. With the recommended `max` profile, users receive stale content while fresh data loads in the background.

Usage

`revalidateTag` can be called in Server Functions and Route Handlers.

`revalidateTag` cannot be called in Client Components or Proxy, as it only works in server environments.

Revalidation Behavior

Calling `revalidateTag` marks the tagged data as stale. The next request for that data kicks off a revalidation and is served stale content while it runs, using stale-while-revalidate semantics. The second argument sets how long stale content may be served. Past that, a request blocks until the revalidation completes.

- `profile="max"` **(recommended)**: A one year window, long enough that requests are always served stale content while the revalidation runs.
- **Another profile, or an object**: Any other default or custom profile defined in `cacheLife`, or an object with an `expire` property, when you want a different window.
- `{ expire: 0 }`: Stale content is never served, so the next request is a blocking revalidate/cache miss. Use it when the caller needs the data gone immediately and you cannot use `updateTag`.
- **No second argument (deprecated)**: Behaves like `{ expire: 0 }`. Migrate to `updateTag` in Server Actions, or `profile="max"`.

The profile sets the point past which data correctness is more important than being fast.

> **Good to know**: A revalidation is triggered by a request, not by the `revalidateTag` call, so pages using the tag revalidate as they are visited rather than all at once.

Parameters

```
revalidateTag(tag: string, profile: string | { expire?: number }): void;
```

- `tag`: A string representing the cache tag associated with the data you want to revalidate. Tags are case-sensitive and must not exceed 256 characters. A tag that exceeds the limit is never assigned to cached data, so revalidating it does nothing.
- `profile`: How long stale content may be served, see Revalidation Behavior. The recommended value is `"max"`. Any other default or custom profile defined in `cacheLife` is also accepted, and only its `expire` is read. You can also pass an object with an `expire` property, in seconds.

Tags must first be assigned to cached data. You can do this in two ways:

- Using the `next.tags` option with `fetch` for caching external API requests:

```
fetch(url, { next: { tags: ['posts'] } })
```

- Using `cacheTag` inside cached functions or components with the `'use cache'` directive:

```
import { cacheTag } from 'next/cache'

async function getData() {
  'use cache'
  cacheTag('posts')
  // ...
}
```

> **Good to know**: The single-argument form `revalidateTag(tag)` is deprecated. It currently works if TypeScript errors are suppressed, but this behavior may be removed in a future version. Update to the two-argument signature.

Returns

`revalidateTag` does not return a value.

Relationship with `revalidatePath`

`revalidateTag` invalidates data with specific tags across all pages that use those tags, while `revalidatePath` invalidates specific page or layout paths.

> **Good to know**: These functions serve different purposes and may need to be used together for comprehensive data consistency. For detailed examples and considerations, see relationship with revalidateTag and updateTag for more information.

Examples

The following examples demonstrate how to use `revalidateTag` in different contexts. In both cases, we're using `profile="max"` to mark data as stale and use stale-while-revalidate semantics, which is the recommended approach for most use cases.

Server Action

```
// Archivo: app/actions.ts switcher
'use server'

import { revalidateTag } from 'next/cache'

export default async function submit() {
  await addPost()
  revalidateTag('posts', 'max')
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { revalidateTag } from 'next/cache'

export default async function submit() {
  await addPost()
  revalidateTag('posts', 'max')
}
```

Route Handler

```
// Archivo: app/api/revalidate/route.ts switcher
import type { NextRequest } from 'next/server'
import { revalidateTag } from 'next/cache'

export async function GET(request: NextRequest) {
  const tag = request.nextUrl.searchParams.get('tag')

  if (tag) {
    revalidateTag(tag, 'max')
    return Response.json({ revalidated: true, now: Date.now() })
  }

  return Response.json({
    revalidated: false,
    now: Date.now(),
    message: 'Missing tag to revalidate',
  })
}
```

```
// Archivo: app/api/revalidate/route.js switcher
import { revalidateTag } from 'next/cache'

export async function GET(request) {
  const tag = request.nextUrl.searchParams.get('tag')

  if (tag) {
    revalidateTag(tag, 'max')
    return Response.json({ revalidated: true, now: Date.now() })
  }

  return Response.json({
    revalidated: false,
    now: Date.now(),
    message: 'Missing tag to revalidate',
  })
}
```

When the invalidation comes from outside a Server Action, for example a webhook or another service calling a Route Handler, `updateTag` is not available. Pass `{ expire: 0 }` to expire the data immediately:

```
revalidateTag(tag, { expire: 0 })
```

unauthorized

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/unauthorized
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the unauthorized function.*

The `unauthorized` function throws an error that renders a Next.js 401 page. It's useful for handling authentication errors, when a request is not signed in. You can customize the UI using the `unauthorized.js` file.

Invoking `unauthorized()` throws a `NEXT_HTTP_ERROR_FALLBACK;401` error and terminates rendering of the route segment where it was thrown. Next.js also injects a `<meta name="robots" content="noindex" />` tag so the page is not indexed. Because it works by throwing, call it in the render path: a component, or a function a component `await`s. A call left in an un-awaited promise throws where nothing catches it, and no unauthorized UI renders.

To start using `unauthorized`, enable the experimental `authInterrupts` configuration option in your `next.config.js` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    authInterrupts: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  experimental: {
    authInterrupts: true,
  },
}
```

`unauthorized` can be invoked in Server Components, Server Functions, and Route Handlers.

```
// Archivo: app/dashboard/page.tsx switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  // Render the dashboard for authenticated users
  return (
    <main>
      <h1>Welcome to the Dashboard</h1>
      <p>Hi, {session.user.name}.</p>
    </main>
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  // Render the dashboard for authenticated users
  return (
    <main>
      <h1>Welcome to the Dashboard</h1>
      <p>Hi, {session.user.name}.</p>
    </main>
  )
}
```

Good to know

- The `unauthorized` function cannot be called in the root layout.
- You do not need to write `return unauthorized()`. It throws (its TypeScript `never` return type), so execution stops. A `try/catch` around the call suppresses the interrupt and no unauthorized UI renders. Use `unstable_rethrow` to let it through.
- An `unauthorized()` left in an un-awaited promise throws where nothing catches it, so no unauthorized UI renders. In development the server logs `⨯ unhandledRejection: NEXT_HTTP_ERROR_FALLBACK;401`. Always `await` the function that may call it.

Examples

Calling `unauthorized()` after streaming has started

To keep the page's shell and loading UI visible while the session is verified, put the auth check in the Data Access Layer function that loads the data, and render it in a component wrapped in [`````) }

````
```jsx
// Archivo: app/account/page.js switcher highlight={8}
import { Suspense } from 'react'
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

async function getAccount() {
  const session = await verifySession()
  if (!session) {
    unauthorized()
  }
  return db.accounts.findByUserId(session.userId)
}

async function AccountDetails() {
  const account = await getAccount()
  return <p>Signed in as {account.email}</p>
}

export default function AccountPage() {
  return (
    <main>
      <h1>Account</h1>
      ``<Suspense>``
    </main>
  )
}
````
````

When the request isn't signed in, `getAccount` calls `unauthorized()`, which throws. Because this happens during rendering, the exception propagates to the nearest `unauthorized` boundary, which renders in place of the streamed-in content, even though the page shell has already been sent.

Add an `unauthorized.tsx` alongside the route to define that UI:

```
// Archivo: app/account/unauthorized.tsx switcher
import Link from 'next/link'

export default function Unauthorized() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>
        Please [sign in](/login) to view your account.
      </p>
    </main>
  )
}
```

```
// Archivo: app/account/unauthorized.js switcher
import Link from 'next/link'

export default function Unauthorized() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>
        Please [sign in](/login) to view your account.
      </p>
    </main>
  )
}
```

The trade-off is the HTTP status code. Because the check runs inside the `<Suspense>` boundary, the response has already begun streaming as a `200`, and the status can't change once streaming has started. This is usually fine for a page, where the user sees the `unauthorized` UI regardless. To return a real `401` status, the check has to run before the response streams. With Cache Components, every dynamic route streams a static shell first, so run that check in `proxy` instead. See Status codes.

Displaying login UI to unauthenticated users

You can use `unauthorized` function to display the `unauthorized.js` file with a login UI.

```
// Archivo: app/dashboard/page.tsx switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  return <div>Dashboard</div>
}
```

```
// Archivo: app/dashboard/page.js switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export default async function DashboardPage() {
  const session = await verifySession()

  if (!session) {
    unauthorized()
  }

  return <div>Dashboard</div>
}
```

```
// Archivo: app/unauthorized.tsx switcher
import Login from '@/app/components/Login'

export default function UnauthorizedPage() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

```
// Archivo: app/unauthorized.js switcher
import Login from '@/app/components/Login'

export default function UnauthorizedPage() {
  return (
    <main>
      <h1>401 - Unauthorized</h1>
      <p>Please log in to access this page.</p>
      <Login />
    </main>
  )
}
```

Mutations with Server Actions

You can invoke `unauthorized` in Server Actions to ensure only authenticated users can perform specific mutations.

```
// Archivo: app/actions/update-profile.ts switcher
'use server'

import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'
import db from '@/app/lib/db'

export async function updateProfile(data: FormData) {
  const session = await verifySession()

  // If the user is not authenticated, return a 401
  if (!session) {
    unauthorized()
  }

  // Proceed with mutation
  // ...
}
```

```
// Archivo: app/actions/update-profile.js switcher
'use server'

import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'
import db from '@/app/lib/db'

export async function updateProfile(data) {
  const session = await verifySession()

  // If the user is not authenticated, return a 401
  if (!session) {
    unauthorized()
  }

  // Proceed with mutation
  // ...
}
```

Fetching data with Route Handlers

You can use `unauthorized` in Route Handlers to ensure only authenticated users can access the endpoint.

```
// Archivo: app/api/profile/route.ts switcher
import { NextRequest, NextResponse } from 'next/server'
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export async function GET(req: NextRequest): Promise<NextResponse> {
  // Verify the user's session
  const session = await verifySession()

  // If no session exists, return a 401 and render unauthorized.tsx
  if (!session) {
    unauthorized()
  }

  // Fetch data
  // ...
}
```

```
// Archivo: app/api/profile/route.js switcher
import { verifySession } from '@/app/lib/dal'
import { unauthorized } from 'next/navigation'

export async function GET() {
  const session = await verifySession()

  // If the user is not authenticated, return a 401 and render unauthorized.tsx
  if (!session) {
    unauthorized()
  }

  // Fetch data
  // ...
}
```

Version History

Version	Changes
`v15.1.0`	`unauthorized` introduced.

unstable_cache

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/unstable_cache
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the unstable_cache function.*

> **Note:** This API has been replaced by `use cache` in Next.js 16. We recommend opting into Cache Components and replacing `unstable_cache` with the `use cache` directive.

`unstable_cache` allows you to cache the results of expensive operations, like database queries, and reuse them across multiple requests.

```
import { getUser } from './data';
import { unstable_cache } from 'next/cache';

const getCachedUser = unstable_cache(
  async (id) => getUser(id),
  ['my-app-user']
);

export default async function Component({ userID }) {
  const user = await getCachedUser(userID);
  ...
}
```

> **Good to know**:
>
> - Accessing uncached data sources such as `headers` or `cookies` inside a cache scope is not supported. If you need this data inside a cached function use `headers` outside of the cached function and pass the required uncached data in as an argument.
> - This API uses Next.js' built-in cache to persist the result across requests and deployments. See Caching and Revalidating.

Parameters

```
const data = unstable_cache(fetchData, keyParts, options)()
```

- `fetchData`: This is an asynchronous function that fetches the data you want to cache. It must be a function that returns a `Promise`.
- `keyParts`: This is an extra array of keys that further adds identification to the cache. By default, `unstable_cache` already uses the arguments and the stringified version of your function as the cache key. It is optional in most cases; the only time you need to use it is when you use external variables without passing them as parameters. However, it is important to add closures used within the function if you do not pass them as parameters.
- `options`: This is an object that controls how the cache behaves. It can contain the following properties:
  - `tags`: An array of tags that can be used to control cache invalidation. Next.js will not use this to uniquely identify the function.
  - `revalidate`: The number of seconds after which the cache should be revalidated. Omit or pass `false` to cache indefinitely or until matching `revalidateTag()` or `revalidatePath()` methods are called.

Returns

`unstable_cache` returns a function that when invoked, returns a Promise that resolves to the cached data. If the data is not in the cache, the provided function will be invoked, and its result will be cached and returned.

Example

```
// Archivo: app/page.tsx switcher
import { unstable_cache } from 'next/cache'

export default async function Page({
  params,
}: {
  params: Promise<{ userId: string }>
}) {
  const { userId } = await params
  const getCachedUser = unstable_cache(
    async () => {
      return { id: userId }
    },
    [userId], // add the user ID to the cache key
    {
      tags: ['users'],
      revalidate: 60,
    }
  )

  // ...
}
```

```
// Archivo: app/page.jsx switcher
import { unstable_cache } from 'next/cache'

export default async function Page({ params }) {
  const { userId } = await params
  const getCachedUser = unstable_cache(
    async () => {
      return { id: userId }
    },
    [userId], // add the user ID to the cache key
    {
      tags: ['users'],
      revalidate: 60,
    }
  )

  // ...
}
```

Version History

Version	Changes
`v14.0.0`	`unstable_cache` introduced.

unstable_noStore

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/unstable_noStore
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the unstable_noStore function.*

In version 15, we recommend using `connection` instead of `unstable_noStore`.

`unstable_noStore` can be used to declaratively opt out of prerendering and indicate a particular component should not be cached.

```
import { unstable_noStore as noStore } from 'next/cache';

export default async function ServerComponent() {
  noStore();
  const result = await db.query( ... );
   ...
}
```

> **Good to know**:
>
> - `unstable_noStore` is equivalent to `cache: 'no-store'` on a `fetch`
> - `unstable_noStore` is preferred over `export const dynamic = 'force-dynamic'` as it is more granular and can be used on a per-component basis

- Using `unstable_noStore` inside `unstable_cache` will not opt out of static generation. Instead, it will defer to the cache configuration to determine whether to cache the result or not.

Usage

If you prefer not to pass additional options to `fetch`, like `cache: 'no-store'`, `next: { revalidate: 0 }` or in cases where `fetch` is not available, you can use `noStore()` as a replacement for all of these use cases.

```
import { unstable_noStore as noStore } from 'next/cache';

export default async function ServerComponent() {
  noStore();
  const result = await db.query( ... );
   ...
}
```

Version History

Version	Changes
`v15.0.0`	`unstable_noStore` deprecated for `connection`.
`v14.0.0`	`unstable_noStore` introduced.

---

unstable_rethrow

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/unstable_rethrow
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the unstable_rethrow function.*

`unstable_rethrow` can be used to avoid catching internal errors thrown by Next.js when attempting to handle errors thrown in your application code.

For example, calling the `notFound` function will throw an internal Next.js error and render the `not-found.js` component. However, if used inside the `try` block of a `try/catch` statement, the error will be caught, preventing `not-found.js` from rendering:

```
// Archivo: @/app/ui/component.tsx
import { notFound } from 'next/navigation'

export default async function Page() {
  try {
    const post = await fetch('https:// ... /posts/1').then((res) => {
      if (res.status === 404) notFound()
      if (!res.ok) throw new Error(res.statusText)
      return res.json()
    })
  } catch (err) {
    console.error(err)
  }
}
```

You can use `unstable_rethrow` API to re-throw the internal error and continue with the expected behavior:

```
// Archivo: @/app/ui/component.tsx
import { notFound, unstable_rethrow } from 'next/navigation'

export default async function Page() {
  try {
    const post = await fetch('https:// ... /posts/1').then((res) => {
      if (res.status === 404) notFound()
      if (!res.ok) throw new Error(res.statusText)
      return res.json()
    })
  } catch (err) {
    unstable_rethrow(err)
    console.error(err)
  }
}
```

The following Next.js APIs rely on throwing an error which should be rethrown and handled by Next.js itself:

- `notFound()`
- `redirect()`
- `permanentRedirect()`

If a route segment is marked to throw an error unless it's static, a Request-time API call will also throw an error that should similarly not be caught by the developer. Note that Partial Prerendering (PPR) affects this behavior as well. These APIs are:

- `cookies`
- `headers`
- `searchParams`
- `fetch( ... , { cache: 'no-store' })`
- `fetch( ... , { next: { revalidate: 0 } })`

Good to know:

- This method should be called at the top of the catch block, passing the error object as its only argument. It can also be used within a `.catch` handler of a promise.
- You may be able to avoid using `unstable_rethrow` if you encapsulate your API calls that throw and let the **caller** handle the exception.
- Only use `unstable_rethrow` if your caught exceptions may include both application errors and framework-controlled exceptions (like `redirect()` or `notFound()`).
- Any resource cleanup (like clearing intervals, timers, etc) would have to either happen prior to the call to `unstable_rethrow` or within a `finally` block.

updateTag

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/updateTag
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the updateTag function.*

`updateTag` allows you to update cached data on-demand for a specific cache tag from within Server Actions.

This function is designed for **read-your-own-writes** scenarios, where a user makes a change (like creating a post), and the UI immediately shows the change, rather than stale data.

Usage

`updateTag` can **only** be called from within Server Actions. It cannot be used in Route Handlers, Client Components, or any other context.

If you need to invalidate cache tags in Route Handlers or other contexts, use `revalidateTag` instead.

> **Good to know**: `updateTag` immediately expires the cached data for the specified tag. The next request will wait to fetch fresh data rather than serving stale content from the cache, ensuring users see their changes immediately.

Parameters

```
updateTag(tag: string): void;
```

- `tag`: A string representing the cache tag associated with the data you want to update. Must not exceed 256 characters. This value is case-sensitive.

Tags must first be assigned to cached data. You can do this in two ways:

- Using the `next.tags` option with `fetch` for caching external API requests:

```
fetch(url, { next: { tags: ['posts'] } })
```

- Using `cacheTag` inside cached functions or components with the `'use cache'` directive:

```
import { cacheTag } from 'next/cache'

async function getData() {
  'use cache'
  cacheTag('posts')
  // ...
}
```

Returns

`updateTag` does not return a value.

Differences from revalidateTag

While both `updateTag` and `revalidateTag` invalidate cached data, they serve different purposes:

- **`updateTag`**:
  - Can only be used in Server Actions

- Next request waits for fresh data (no stale content served)
- Designed for read-your-own-writes scenarios

- `revalidateTag`:
  - Can be used in Server Actions and Route Handlers
  - With `profile="max"` (recommended): Serves cached data while fetching fresh data in the background (stale-while-revalidate)
  - With custom profile: Can be configured to any cache life profile for advanced usage
  - Without profile: legacy behavior which is equivalent to `updateTag`

Examples

Server Action with Read-Your-Own-Writes

```
// Archivo: app/actions.ts switcher
'use server'

import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData: FormData) {
  const title = formData.get('title')
  const content = formData.get('content')

  // Create the post in your database
  const post = await db.post.create({
    data: { title, content },
  })

  // Invalidate cache tags so the new post is immediately visible
  // 'posts' tag: affects any page that displays a list of posts
  updateTag('posts')
  // 'post-{id}' tag: affects the individual post detail page
  updateTag(`post-${post.id}`)

  // Redirect to the new post - user will see fresh data, not cached
  redirect(`/posts/${post.id}`)
}
```

```
// Archivo: app/actions.js switcher
'use server'

import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createPost(formData) {
  const title = formData.get('title')
  const content = formData.get('content')

  // Create the post in your database
  const post = await db.post.create({
    data: { title, content },
  })

  // Invalidate cache tags so the new post is immediately visible
  // 'posts' tag: affects any page that displays a list of posts
  updateTag('posts')
  // 'post-{id}' tag: affects the individual post detail page
  updateTag(`post-${post.id}`)

  // Redirect to the new post - user will see fresh data, not cached
  redirect(`/posts/${post.id}`)
}
```

Error when used outside Server Actions

```
// Archivo: app/api/posts/route.ts switcher
import { revalidateTag, updateTag } from 'next/cache'

export async function POST() {
  // This will throw an error
  updateTag('posts')
  // Error: updateTag can only be called from within a Server Action

  // Use revalidateTag instead in Route Handlers
  revalidateTag('posts', 'max')
}
```

When to use updateTag

Use `updateTag` when:

- You're in a Server Action
- You need immediate cache invalidation for read-your-own-writes
- You want to ensure the next request sees updated data

Use `revalidateTag` instead when:

- You're in a Route Handler or other non-action context
- You want stale-while-revalidate semantics
- You're building a webhook or API endpoint for cache invalidation

Related

- `revalidateTag` - For invalidating tags in Route Handlers
- `revalidatePath` - For invalidating specific paths

---

useLinkStatus

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-link-status
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useLinkStatus hook.*

The `useLinkStatus` hook lets you track the **pending** state of a `<Link>`. Use it for subtle, inline feedback, for example a shimmer effect over the clicked link, while navigation completes. Prefer route-level fallbacks with `loading.js`, and prefetching for instant transitions.

`useLinkStatus` is useful when:

- Prefetching is disabled or in progress meaning navigation is blocked.
- The destination route is dynamic **and** doesn't include a `loading.js` file that would allow an instant navigation.

```tsx
// Archivo: app/hint.tsx switcher
'use client'

import Link from 'next/link'
import { useLinkStatus } from 'next/link'

function Hint() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}

export default function Header() {
  return (
    <header>
      <Link href="/dashboard" prefetch={false}>
        <span className="label">Dashboard</span> <Hint />
      </Link>
    </header>
  )
}
```

```jsx
// Archivo: app/hint.js switcher
'use client'

import Link from 'next/link'
import { useLinkStatus } from 'next/link'

function Hint() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}

export default function Header() {
  return (
    <header>
      <Link href="/dashboard" prefetch={false}>
        <span className="label">Dashboard</span> <Hint />
      </Link>
    </header>
  )
}
```

Good to know:

- `useLinkStatus` must be used within a descendant component of a `Link` component
- The hook is most useful when `prefetch={false}` is set on the `Link` component
- If the linked route has been prefetched, the pending state will be skipped
- When clicking multiple links in quick succession, only the last link's pending state is shown
- This hook is not supported in the Pages Router and always returns `{ pending: false }`
- Inline indicators can easily introduce layout shifts. Prefer a fixed-size, always-rendered hint element and toggle its opacity, or use an animation.

You might not need `useLinkStatus`

Before adding inline feedback, consider if:

- The destination is static and prefetched in production, so the pending phase may be skipped.
- The route has a `loading.js` file, enabling instant transitions with a route-level fallback.

Navigation is typically fast. Use `useLinkStatus` as a quick patch when you identify a slow transition, then iterate to fix the root cause with prefetching or a `loading.js` fallback.

Parameters

```
const { pending } = useLinkStatus()
```

`useLinkStatus` does not take any parameters.

Returns

`useLinkStatus` returns an object with a single property:

Property	Type	Description
pending	boolean	`true` before history updates, `false` after

Example

Inline link hint

Add a subtle, fixed-size hint that doesn't affect layout to confirm a click when prefetching hasn't completed.

```
// Archivo: app/components/loading-indicator.tsx switcher
'use client'

import { useLinkStatus } from 'next/link'

export default function LoadingIndicator() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}
```

```
// Archivo: app/components/loading-indicator.js switcher
'use client'

import { useLinkStatus } from 'next/link'

export default function LoadingIndicator() {
  const { pending } = useLinkStatus()
  return (
    <span aria-hidden className={`link-hint ${pending ? 'is-pending' : ''}`} />
  )
}
```

```tsx
// Archivo: app/shop/layout.tsx switcher
import Link from 'next/link'
import LoadingIndicator from './components/loading-indicator'

const links = [
  { href: '/shop/electronics', label: 'Electronics' },
  { href: '/shop/clothing', label: 'Clothing' },
  { href: '/shop/books', label: 'Books' },
]

function Menubar() {
  return (
    <div>
      {links.map((link) => (
        <Link key={link.label} href={link.href}>
          <span className="label">{link.label}</span> <LoadingIndicator />
        </Link>
      ))}
    </div>
  )
}

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <div>
      <Menubar />
      {children}
    </div>
  )
}
```

```jsx
// Archivo: app/shop/layout.js switcher
import Link from 'next/link'
import LoadingIndicator from './components/loading-indicator'

const links = [
  { href: '/shop/electronics', label: 'Electronics' },
  { href: '/shop/clothing', label: 'Clothing' },
  { href: '/shop/books', label: 'Books' },
]

function Menubar() {
  return (
    <div>
      {links.map((link) => (
        <Link key={link.label} href={link.href}>
          <span className="label">{link.label}</span> <LoadingIndicator />
        </Link>
      ))}
    </div>
  )
}

export default function Layout({ children }) {
  return (
    <div>
      <Menubar />
      {children}
    </div>
  )
}
```

Gracefully handling fast navigation

If the navigation to a new route is fast, users may see an unnecessary flash of the hint. One way to improve the user experience and only show the hint when the navigation takes time to complete is to add an initial animation delay (e.g. 100ms) and start the animation as invisible (e.g. `opacity: 0`).

```
// Archivo: app/styles/global.css
.link-hint {
  display: inline-block;
  width: 0.6em;
  height: 0.6em;
  margin-left: 0.25rem;
  border-radius: 9999px;
  background: currentColor;
  opacity: 0;
  visibility: hidden; /* reserve space without showing the hint */
}

.link-hint.is-pending {
  /* Animation 1: fade in after 100ms and keep final opacity */
  /* Animation 2: subtle pulsing while pending */
  visibility: visible;
  animation-name: fadeIn, pulse;
  animation-duration: 200ms, 1s;
  /* Appear only if navigation actually takes time */
  animation-delay: 100ms, 100ms;
  animation-timing-function: ease, ease-in-out;
  animation-iteration-count: 1, infinite;
  animation-fill-mode: forwards, none;
}

@keyframes fadeIn {
  to {
    opacity: 0.35;
  }
}
@keyframes pulse {
  50% {
    opacity: 0.15;
  }
}
```

Version History

Version	Changes
`v15.3.0`	`useLinkStatus` introduced.

useOffline

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-offline
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useOffline hook.*

The `useOffline` hook returns a boolean indicating whether the app is currently offline. Use it to render connectivity-aware UI, such as a banner when the user loses their network connection, or an offline-aware Suspense fallback.

The hook is one piece of a larger feature. Enabling the `experimental.useOffline` config option turns on offline connectivity detection and automatic retry of blocked navigation, prefetch, and Server Action requests, and exposes this hook so Client Components can read the state.

Without the flag, this hook always returns `false`.

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    useOffline: true,
  },
}
```

```
// Archivo: app/offline-status.tsx switcher
'use client'

import { useOffline } from 'next/offline'

export function OfflineStatus() {
  const isOffline = useOffline()
  return <div>{isOffline ? 'Offline' : 'Online'}</div>
}
```

```
// Archivo: app/offline-status.js switcher
'use client'

import { useOffline } from 'next/offline'

export function OfflineStatus() {
  const isOffline = useOffline()
  return <div>{isOffline ? 'Offline' : 'Online'}</div>
}
```

For details on how connectivity is detected and requests retried, see How retry works.

Parameters

```
const isOffline = useOffline()
```

`useOffline` does not take any parameters.

Returns

`useOffline` returns a `boolean`:

Value	Meaning
`true`	The app is offline. A network request has failed, or the browser has fired an `offline` event.
`false`	The app is online, or rendering on the server. This is also the initial value before hydration completes.

Examples

Show an offline banner

Render a persistent banner whenever the user loses connectivity.

```tsx
// Archivo: app/components/offline-banner.tsx switcher
'use client'

import { useOffline } from 'next/offline'

export function OfflineBanner() {
  const isOffline = useOffline()

  if (!isOffline) {
    return null
  }

  return (
    <div role="status" className="offline-banner">
      You are offline. Some content may be unavailable.
    </div>
  )
}
```

```jsx
// Archivo: app/components/offline-banner.js switcher
'use client'

import { useOffline } from 'next/offline'

export function OfflineBanner() {
  const isOffline = useOffline()

  if (!isOffline) {
    return null
  }

  return (
    <div role="status" className="offline-banner">
      You are offline. Some content may be unavailable.
    </div>
  )
}
```

Render it in the root layout so the banner shows on every route:

```tsx
// Archivo: app/layout.tsx switcher
import { OfflineBanner } from './components/offline-banner'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html>
      <body>
        <OfflineBanner />
        {children}
      </body>
    </html>
  )
}
```

```
// Archivo: app/layout.js switcher
import { OfflineBanner } from './components/offline-banner'

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <OfflineBanner />
        {children}
      </body>
    </html>
  )
}
```

Offline-aware Suspense fallback

When a user navigates to a route while offline, the prefetched static shell renders immediately but the dynamic content behind a `<Suspense>` boundary blocks on the network. For example, use `useOffline` inside a `loading.tsx` file to explain why the content is taking longer than expected.

```
// Archivo: app/destination/loading.tsx switcher
'use client'

import { useOffline } from 'next/offline'

export default function Loading() {
  const isOffline = useOffline()

  return (
    <div>
      {isOffline ? 'Waiting for connection to load this page ... ' : 'Loading ... '}
    </div>
  )
}
```

```
// Archivo: app/destination/loading.js switcher
'use client'

import { useOffline } from 'next/offline'

export default function Loading() {
  const isOffline = useOffline()

  return (
    <div>
      {isOffline ? 'Waiting for connection to load this page ... ' : 'Loading ... '}
    </div>
  )
}
```

When connectivity is restored, Next.js retries the blocked request and the dynamic content streams in automatically.

Version History

Version	Changes
`v16.x.0`	`useOffline` hook introduced.

useParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-params
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useParams hook.*

`useParams` is a **Client Component** hook that lets you read a route's dynamic params filled in by the current URL.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useParams } from 'next/navigation'

export default function ExampleClientComponent() {
  const params = useParams<{ tag: string; item: string }>()

  // Route → /shop/[tag]/[item]
  // URL → /shop/shoes/nike-air-max-97
  // `params` → { tag: 'shoes', item: 'nike-air-max-97' }
  console.log(params)

  return '...'
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { useParams } from 'next/navigation'

export default function ExampleClientComponent() {
  const params = useParams()

  // Route → /shop/[tag]/[item]
  // URL → /shop/shoes/nike-air-max-97
  // `params` → { tag: 'shoes', item: 'nike-air-max-97' }
  console.log(params)

  return '...'
}
```

Parameters

```
const params = useParams()
```

`useParams` does not take any parameters.

Returns

`useParams` returns an object containing the current route's filled in dynamic parameters.

- Each property in the object is an active dynamic segment.
- The properties name is the segment's name, and the properties value is what the segment is filled in with.
- The properties value will either be a `string` or array of `string`'s depending on the type of dynamic segment.
- If the route contains no dynamic parameters, `useParams` returns an empty object.
- If used in Pages Router, `useParams` will return `null` on the initial render and updates with properties following the rules above once the router is ready.

For example:

Route	URL	`useParams()`
`app/shop/page.js`	`/shop`	`{}`
`app/shop/[slug]/page.js`	`/shop/1`	`{ slug: '1' }`
`app/shop/[tag]/[item]/page.js`	`/shop/1/2`	`{ tag: '1', item: '2' }`
`app/shop/[...slug]/page.js`	`/shop/1/2`	`{ slug: ['1', '2'] }`

Behavior

Cache Components

When `cacheComponents` is enabled, `useParams` may require a `Suspense` boundary. This depends on whether the params can be resolved during prerendering.

- **Static routes and routes with `generateStaticParams`**: every dynamic param is known at build time. `useParams` resolves on the server and no `Suspense` boundary is required.
- **Routes with dynamic params not covered by `generateStaticParams`**: the param is not known until request time. `useParams` suspends. Wrap the component (or a parent) in a `Suspense` boundary so its fallback can be rendered during prerendering; otherwise, the build fails.

See Next.js encountered URL data in a Client Component outside of Suspense for full fix options and trade-offs.

Version History

Version	Changes
`v13.3.0`	`useParams` introduced.

usePathname

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-pathname
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the usePathname hook.*

`usePathname` is a **Client Component** hook that lets you read the current URL's **pathname**.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { usePathname } from 'next/navigation'

export default function ExampleClientComponent() {
  const pathname = usePathname()
  return <p>Current pathname: {pathname}</p>
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { usePathname } from 'next/navigation'

export default function ExampleClientComponent() {
  const pathname = usePathname()
  return <p>Current pathname: {pathname}</p>
}
```

`usePathname` intentionally requires using a Client Component. It's important to note Client Components are not a de-optimization. They are an integral part of the Server Components architecture.

For example, a Client Component with `usePathname` will be rendered into HTML on the initial page load. When navigating to a new route, this component does not need to be re-fetched. Instead, the component is downloaded once (in the client JavaScript bundle), and re-renders based on the current state.

> **Good to know**:
>
> - Reading the current URL from a Server Component is not supported. This design is intentional to support layout state being preserved across page navigations.
> - If your page is being statically prerendered and your app has rewrites in `next.config` or a Proxy file, reading the pathname with `usePathname()` can result in hydration mismatch errors, because the initial value comes from the server and may not match the actual browser pathname after routing. See our example for a way to mitigate this issue.

Compatibility with Pages Router

Parameters

```
const pathname = usePathname()
```

`usePathname` does not take any parameters.

Returns

`usePathname` returns a string of the current URL's pathname. For example:

URL	Returned value
`/`	`'/'`
`/dashboard`	`'/dashboard'`
`/dashboard?v=2`	`'/dashboard'`
`/blog/hello-world`	`'/blog/hello-world'`

Behavior

Cache Components

When `cacheComponents` is enabled, `usePathname` may require a `Suspense` boundary. This depends on whether the pathname can be resolved during prerendering.

- **Static routes and routes with `generateStaticParams`**: every route segment, including dynamic params, is known at build time. The pathname can be resolved during prerendering, so `usePathname` resolves on the server and no `Suspense` boundary is required.
- **Routes with dynamic params not covered by `generateStaticParams`**: the param is a fallback param that is not known until request time. The pathname cannot be resolved during prerendering, so `usePathname` suspends. Wrap the component (or a parent) in a

`Suspense` boundary so its fallback can be rendered during prerendering; otherwise, the build fails.

This applies even when the component that calls `usePathname` is itself static. For example, a sidebar with active links rendered in a layout suspends on any page below it that has an unknown dynamic param. To keep the rest of the layout prerendered, wrap the component that calls `usePathname` (or a parent) in a `Suspense` boundary with a fallback.

See Next.js encountered URL data in a Client Component outside of Suspense for full fix options and trade-offs.

Examples

Do something in response to a route change

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useEffect } from 'react'
import { usePathname, useSearchParams } from 'next/navigation'

function ExampleClientComponent() {
  const pathname = usePathname()
  const searchParams = useSearchParams()
  useEffect(() => {
    // Do something here ...
  }, [pathname, searchParams])
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { useEffect } from 'react'
import { usePathname, useSearchParams } from 'next/navigation'

function ExampleClientComponent() {
  const pathname = usePathname()
  const searchParams = useSearchParams()
  useEffect(() => {
    // Do something here ...
  }, [pathname, searchParams])
}
```

Avoid hydration mismatch with rewrites

When a page is prerendered, the HTML is generated for the source pathname. If the page is then reached through a rewrite using `next.config` or `Proxy`, the browser URL may differ, and `usePathname()` will read the rewritten pathname on the client.

To avoid hydration mismatches, design the UI so that only a small, isolated part depends on the client pathname. Render a stable fallback on the server and update that part after mount. The deferred read shows a brief moment of the fallback before the real pathname appears; see Preventing flash before hydration for techniques that eliminate the visible flicker.

```tsx
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useEffect, useState } from 'react'
import { usePathname } from 'next/navigation'

export default function PathnameBadge() {
  const pathname = usePathname()
  const [clientPathname, setClientPathname] = useState('')

  useEffect(() => {
    setClientPathname(pathname)
  }, [pathname])

  return (
    <p>
      Current pathname: <span>{clientPathname}</span>
    </p>
  )
}
```

```jsx
// Archivo: app/example-client-component.js switcher
'use client'

import { useEffect, useState } from 'react'
import { usePathname } from 'next/navigation'

export default function PathnameBadge() {
  const pathname = usePathname()
  const [clientPathname, setClientPathname] = useState('')

  useEffect(() => {
    setClientPathname(pathname)
  }, [pathname])

  return (
    <p>
      Current pathname: <span>{clientPathname}</span>
    </p>
  )
}
```

Version	Changes
`v13.0.0`	`usePathname` introduced.

useReportWebVitals

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-report-web-vitals
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useReportWebVitals function.*

The `useReportWebVitals` hook allows you to report Core Web Vitals, and can be used in combination with your analytics service.

New functions passed to `useReportWebVitals` are called with the available metrics up to that point. To prevent reporting duplicated data, ensure that the callback function reference does not change (as shown in the code examples below).

📘 **Nota para Pages Router:**

```
import { useReportWebVitals } from 'next/web-vitals'

const logWebVitals = (metric) => {
  console.log(metric)
}

function MyApp({ Component, pageProps }) {
  useReportWebVitals(logWebVitals)

  return <Component {...pageProps} />
}
```

```
// Archivo: app/_components/web-vitals.js
'use client'

import { useReportWebVitals } from 'next/web-vitals'

const logWebVitals = (metric) => {
  console.log(metric)
}

export function WebVitals() {
  useReportWebVitals(logWebVitals)

  return null
}
```

```
// Archivo: app/layout.js
import { WebVitals } from './_components/web-vitals'

export default function Layout({ children }) {
  return (
    <html>
      <body>
        <WebVitals />
        {children}
      </body>
    </html>
  )
}
```

Since the `useReportWebVitals` hook requires the `'use client'` directive, the most performant approach is to create a separate component that the root layout imports. This confines the client boundary exclusively to the `WebVitals` component.

useReportWebVitals

The `metric` object passed as the hook's argument consists of a number of properties:

- `id` : Unique identifier for the metric in the context of the current page load
- `name` : The name of the performance metric. Possible values include names of Web Vitals metrics (TTFB, FCP, LCP, FID, CLS) specific to a web application.
- `delta` : The difference between the current value and the previous value of the metric. The value is typically in milliseconds and represents the change in the metric's value over time.
- `entries` : An array of Performance Entries associated with the metric. These entries provide detailed information about the performance events related to the metric.
- `navigationType` : Indicates the navigation type that triggered metric collection. Values are derived from PerformanceNavigationTiming.type and may include `"navigate"` , `"reload"` , `"prerender"` , `"back-forward"` (normalized from `"back_forward"`), `"back-forward-cache"` (BFCache restore), and `"restore"` (page restored after discard).

- `rating` : A qualitative rating of the metric value, providing an assessment of the performance. Possible values are `"good"` , `"needs-improvement"` , and `"poor"` . The rating is typically determined by comparing the metric value against predefined thresholds that indicate acceptable or suboptimal performance.
- `value` : The actual value or duration of the performance entry, typically in milliseconds. The value provides a quantitative measure of the performance aspect being tracked by the metric. The source of the value depends on the specific metric being measured and can come from various Performance APIs.

Web Vitals

Web Vitals are a set of useful metrics that aim to capture the user experience of a web page. The following web vitals are all included:

- Time to First Byte (TTFB)
- First Contentful Paint (FCP)
- Largest Contentful Paint (LCP)
- First Input Delay (FID)
- Cumulative Layout Shift (CLS)
- Interaction to Next Paint (INP)

You can handle all the results of these metrics using the `name` property.

> 📘 **Nota para Pages Router:**

```
import { useReportWebVitals } from 'next/web-vitals'

const handleWebVitals = (metric) => {
  switch (metric.name) {
    case 'FCP': {
      // handle FCP results
    }
    case 'LCP': {
      // handle LCP results
    }
    // ...
  }
}

function MyApp({ Component, pageProps }) {
  useReportWebVitals(handleWebVitals)

  return <Component {...pageProps} />
}
```

```tsx
// Archivo: app/components/web-vitals.tsx switcher
'use client'

import { useReportWebVitals } from 'next/web-vitals'

type ReportWebVitalsCallback = Parameters<typeof useReportWebVitals>[0]

const handleWebVitals: ReportWebVitalsCallback = (metric) => {
  switch (metric.name) {
    case 'FCP': {
      // handle FCP results
    }
    case 'LCP': {
      // handle LCP results
    }
    // ...
  }
}

export function WebVitals() {
  useReportWebVitals(handleWebVitals)
}
```

```jsx
// Archivo: app/components/web-vitals.js switcher
'use client'

import { useReportWebVitals } from 'next/web-vitals'

const handleWebVitals = (metric) => {
  switch (metric.name) {
    case 'FCP': {
      // handle FCP results
    }
    case 'LCP': {
      // handle LCP results
    }
    // ...
  }
}

export function WebVitals() {
  useReportWebVitals(handleWebVitals)
}
```

📘 **Nota para Pages Router:**

Custom Metrics

In addition to the core metrics listed above, there are some additional custom metrics that measure the time it takes for the page to hydrate and render:

- `Next.js-hydration`: Length of time it takes for the page to start and finish hydrating (in ms)
- `Next.js-route-change-to-render`: Length of time it takes for a page to start rendering after a route change (in ms)
- `Next.js-render`: Length of time it takes for a page to finish render after a route change (in ms)

You can handle all the results of these metrics separately:

```
import { useReportWebVitals } from 'next/web-vitals'

function handleCustomMetrics(metric) {
  switch (metric.name) {
    case 'Next.js-hydration':
      // handle hydration results
      break
    case 'Next.js-route-change-to-render':
      // handle route-change to render results
      break
    case 'Next.js-render':
      // handle render results
      break
    default:
      break
  }
}

function MyApp({ Component, pageProps }) {
  useReportWebVitals(handleCustomMetrics)

  return <Component {...pageProps} />
}
```

These metrics work in all browsers that support the User Timing API.

Sending results to external systems

You can send results to any endpoint to measure and track real user performance on your site. For example:

```
function postWebVitals(metric) {
  const body = JSON.stringify(metric)
  const url = 'https://example.com/analytics'

  // Use `navigator.sendBeacon()` if available, falling back to `fetch()`.
  if (navigator.sendBeacon) {
    navigator.sendBeacon(url, body)
  } else {
    fetch(url, { body, method: 'POST', keepalive: true })
  }
}

useReportWebVitals(postWebVitals)
```

Good to know: If you use Google Analytics, using the `id` value can allow you to construct metric distributions manually (to calculate percentiles, etc.)

```
useReportWebVitals(metric => {
  // Use `window.gtag` if you initialized Google Analytics as this example:
  // https://github.com/vercel/next.js/blob/canary/examples/with-google-analytics
  window.gtag('event', metric.name, {
    value: Math.round(metric.name === 'CLS' ? metric.value * 1000 : metric.value), // values must be integers
    event_label: metric.id, // id unique to current page load
    non_interaction: true, // avoids affecting bounce rate.
  });
}
```

Read more about sending results to Google Analytics.

useRouter

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-router
📁 **Sección:** App > Api Reference > Functions
📝 **Descripción:** *API reference for the useRouter hook.*

The `useRouter` hook allows you to programmatically change routes inside Client Components.

Recommendation: Use the `<Link>` component for navigation unless you have a specific requirement for using `useRouter`.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

`useRouter()`

- `router.push(href: string, { scroll: boolean, transitionTypes: string[] })`: Perform a client-side navigation to the provided route. Adds a new entry into the browser's history stack. The optional `transitionTypes` are passed to `React.addTransitionType`

inside the navigation Transition.

- `router.replace(href: string, { scroll: boolean, transitionTypes: string[] })`: Perform a client-side navigation to the provided route without adding a new entry into the browser's history stack. The optional `transitionTypes` are passed to `React.addTransitionType` inside the navigation Transition.
- `router.refresh()`: Refresh the current route. Making a new request to the server, re-fetching data requests, and re-rendering Server Components. The client will merge the updated React Server Component payload without losing unaffected client-side React (e.g. `useState`) or browser state (e.g. scroll position). This clears the Client Cache for the current route, but does **not** invalidate the server-side cache. Use `revalidatePath` or `revalidateTag` to invalidate server-side cached data.
- `router.prefetch(href: string, options?: { onInvalidate?: () => void })`: Prefetch the provided route for faster client-side transitions. The optional `onInvalidate` callback is called when the prefetched data becomes stale.
- `router.back()`: Navigate back to the previous route in the browser's history stack.
- `router.forward()`: Navigate forwards to the next page in the browser's history stack.
- `router.bfcacheId`: An opaque string identifier scoped to the current route segment. It changes when the surrounding segment is freshly created by a push or replace navigation, and stays the same for back/forward navigations, `router.refresh()`, and search-param- or hash-only navigations. See `bfcacheId` below for details.

> **Good to know**:
>
> - You must not send untrusted or unsanitized URLs to `router.push` or `router.replace`, as this can open your site to cross-site scripting (XSS) vulnerabilities. For example, `javascript:` URLs sent to `router.push` or `router.replace` will be executed in the context of your page.
> - The `<Link>` component automatically prefetch routes as they become visible in the viewport.
> - `refresh()` could re-produce the same result if fetch requests are cached. Other Request-time APIs like `cookies` and `headers` could also change the response.
> - The `onInvalidate` callback is called at most once per prefetch request. It signals when you may want to trigger a new prefetch for updated route data.

Migrating from `next/router`

- The `useRouter` hook should be imported from `next/navigation` and not `next/router` when using the App Router
- The `pathname` string has been removed and is replaced by `usePathname()`
- The `query` object has been removed and is replaced by `useSearchParams()`
- `router.events` has been replaced. See below.

View the full migration guide.

Examples

Router events

You can listen for page changes by composing other Client Component hooks like `usePathname` and `useSearchParams`.

```
// Archivo: app/components/navigation-events.js
'use client'

import { useEffect } from 'react'
import { usePathname, useSearchParams } from 'next/navigation'

export function NavigationEvents() {
  const pathname = usePathname()
  const searchParams = useSearchParams()

  useEffect(() => {
    const url = `${pathname}?${searchParams}`
    console.log(url)
    // You can now use the current URL
    // ...
  }, [pathname, searchParams])

  return '...'
}
```

Which can be imported into a layout.

```
// Archivo: app/layout.js highlight={2,10-12}
import { Suspense } from 'react'
import { NavigationEvents } from './components/navigation-events'

export default function Layout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}

        ``<Suspense>``
      </body>
    </html>
  )
}
```

> **Good to know**: `<NavigationEvents>` is wrapped in a `Suspense` boundary because `useSearchParams()` causes client-side rendering up to the closest `Suspense` boundary during prerendering. Learn more.

Disabling scroll to top

By default, Next.js will scroll to the top of the page when navigating to a new route. You can disable this behavior by passing `scroll: false` to `router.push()` or `router.replace()`.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button
      type="button"
      onClick={() => router.push('/dashboard', { scroll: false })}
    >
      Dashboard
    </button>
  )
}
```

```
// Archivo: app/example-client-component.jsx switcher
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button
      type="button"
      onClick={() => router.push('/dashboard', { scroll: false })}
    >
      Dashboard
    </button>
  )
}
```

`bfcacheId`

`router.bfcacheId` is an opaque string identifier scoped to the current route segment. It changes when the surrounding segment is freshly created by a push or replace navigation, and stays the same for back/forward navigations, `router.refresh()`, and search-param- or hash-only navigations.

The recommended use is to pass it as a React `key` to opt out of state preservation on fresh navigations, while still restoring it during a back/forward navigation:

```
// Archivo: app/example/page.tsx
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const { bfcacheId } = useRouter()
  return <form key={bfcacheId}></form>
}
```

When `cacheComponents` is enabled, the App Router preserves Client Component state across navigations using React `<Activity>`. Keying a component on `bfcacheId` resets it on each fresh navigation while still preserving its state across browser back/forward navigations. See Preserving UI state for the recommended per-pattern resets.

> **Good to know**: Instead of `bfcacheId`, prefer resetting state explicitly in an event handler (for example, `onSubmit`) or deriving a key from your data (for example, a draft id from the server). Use `bfcacheId` only as a last resort, like when migrating an existing codebase.

Version History

Version	Changes
`v15.4.0`	Optional `onInvalidate` callback for `router.prefetch` introduced
`v13.0.0`	`useRouter` from `next/navigation` introduced.

useSearchParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-search-params
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useSearchParams hook.*

`useSearchParams` is a **Client Component** hook that lets you read the current URL's **query string**.

`useSearchParams` returns a **read-only** version of the `URLSearchParams` interface.

```
// Archivo: app/dashboard/search-bar.tsx switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // URL -> `/dashboard?search=my-project`
  // `search` -> 'my-project'
  return <>Search: {search}</>
}
```

```
// Archivo: app/dashboard/search-bar.js switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // URL -> `/dashboard?search=my-project`
  // `search` -> 'my-project'
  return <>Search: {search}</>
}
```

Parameters

```
const searchParams = useSearchParams()
```

`useSearchParams` does not take any parameters.

Returns

`useSearchParams` returns a **read-only** version of the `URLSearchParams` interface, which includes utility methods for reading the URL's query string:

- `URLSearchParams.get()`: Returns the first value associated with the search parameter. For example:

URL	`searchParams.get("a")`
`/dashboard?a=1`	`'1'`
`/dashboard?a=`	`''`
`/dashboard?b=3`	`null`
`/dashboard?a=1&a=2`	`'1'` - use *`getAll()`* to get all values

- `URLSearchParams.has()`: Returns a boolean value indicating if the given parameter exists. For example:

URL	`searchParams.has("a")`
`/dashboard?a=1`	`true`
`/dashboard?b=3`	`false`

- Learn more about other **read-only** methods of `URLSearchParams`, including the `getAll()`, `keys()`, `values()`, `entries()`, `forEach()`, and `toString()`.

> **Good to know**:
>
> - `useSearchParams` is a Client Component hook and is **not supported** in Server Components to prevent stale values during partial rendering.
> - If you want to fetch data in a Server Component based on search params, it's often a better option to read the `searchParams` prop of the corresponding Page. You can then pass it down by props to any component (Server or Client) within that Page.
> - If an application includes the `/pages` directory, `useSearchParams` will return `ReadonlyURLSearchParams | null`. The `null` value is for compatibility during migration since search params cannot be known during prerendering of a page that doesn't use `getServerSideProps`

Behavior

Prerendering

If a route is prerendered, calling `useSearchParams` will cause the Client Component tree up to the closest `Suspense` boundary to be client-side rendered.

This allows a part of the route to be prerendered while the dynamic part that uses `useSearchParams` is client-side rendered.

We recommend wrapping the Client Component that uses `useSearchParams` in a `<Suspense>` boundary. This will allow any Client Components above it to be prerendered and sent as part of initial HTML.

See Next.js encountered URL data in a Client Component outside of Suspense for full fix options and trade-offs.

For example:

```
// Archivo: app/dashboard/search-bar.tsx switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // This will not be logged on the server during prerendering
  console.log(search)

  return <>Search: {search}</>
}
```

```
// Archivo: app/dashboard/search-bar.js switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // This will not be logged on the server during prerendering
  console.log(search)

  return <>Search: {search}</>
}
```

```
// Archivo: app/dashboard/page.tsx switcher
import { Suspense } from 'react'
import SearchBar from './search-bar'

// This component passed as a fallback to the Suspense boundary
// will be rendered in place of the search bar in the initial HTML.
// When the value is available during React hydration the fallback
// will be replaced with the ``<SearchBar>`` component.
function SearchBarFallback() {
  return <>placeholder</>
}

export default function Page() {
  return (
    <>
      <nav>
        ``<Suspense>``}>
          ``<SearchBar>``
        `<Suspense>`
      </nav>
      <h1>Dashboard</h1>
    </>
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
import { Suspense } from 'react'
import SearchBar from './search-bar'

// This component passed as a fallback to the Suspense boundary
// will be rendered in place of the search bar in the initial HTML.
// When the value is available during React hydration the fallback
// will be replaced with the ``<SearchBar>`` component.
function SearchBarFallback() {
  return <>placeholder</>
}

export default function Page() {
  return (
    <>
      <nav>
        ``<Suspense>``}>
          ``<SearchBar>``
        `<Suspense>`
      </nav>
      <h1>Dashboard</h1>
    </>
  )
}
```

Good to know:

- In development, routes are rendered on-demand, so `useSearchParams` doesn't suspend and things may appear to work without `Suspense`.
- During production builds, a static page that calls `useSearchParams` from a Client Component must be wrapped in a `Suspense` boundary, otherwise the build fails with the Missing Suspense boundary with useSearchParams error.
- If you intend the route to be dynamically rendered, prefer using the `connection` function first in a Server Component to wait for an incoming request, this excludes everything below from prerendering. See what makes a route dynamic in the Dynamic Rendering guide.
- If you're already in a Server Component Page, consider using the `searchParams` prop and passing the values to Client Components.
- You can also pass the Page `searchParams` prop directly to a Client Component and unwrap it with React's `use()`. Although this will suspend, so the Client Component should be wrapped with a `Suspense` boundary.

Dynamic Rendering

If a route is dynamically rendered, `useSearchParams` will be available on the server during the initial server render of the Client Component.

For example:

```
// Archivo: app/dashboard/search-bar.tsx switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // This will be logged on the server during the initial render
  // and on the client on subsequent navigations.
  console.log(search)

  return <>Search: {search}</>
}
```

```
// Archivo: app/dashboard/search-bar.js switcher
'use client'

import { useSearchParams } from 'next/navigation'

export default function SearchBar() {
  const searchParams = useSearchParams()

  const search = searchParams.get('search')

  // This will be logged on the server during the initial render
  // and on the client on subsequent navigations.
  console.log(search)

  return <>Search: {search}</>
}
```

```
// Archivo: app/dashboard/page.tsx switcher
import { connection } from 'next/server'
import SearchBar from './search-bar'

export default async function Page() {
  await connection()
  return (
    <>
      <nav>
        ``<SearchBar>``
      </nav>
      <h1>Dashboard</h1>
    </>
  )
}
```

```
// Archivo: app/dashboard/page.js switcher
import { connection } from 'next/server'
import SearchBar from './search-bar'

export default async function Page() {
  await connection()
  return (
    <>
      <nav>
        ``<SearchBar>``
      </nav>
      <h1>Dashboard</h1>
    </>
  )
}
```

> **Good to know**:
>
> - Previously, setting `export const dynamic = 'force-dynamic'` on the page was used to force dynamic rendering. Prefer using `connection()` instead, as it semantically ties dynamic rendering to the incoming request.

Server Components

Pages

To access search params in Pages (Server Components), use the `searchParams` prop.

Layouts

Unlike Pages, Layouts (Server Components) **do not** receive the `searchParams` prop. This is because a shared layout is not re-rendered during navigation which could lead to stale `searchParams` between navigations. View detailed explanation.

Instead, use the Page `searchParams` prop or the `useSearchParams` hook in a Client Component, which is re-rendered on the client with the latest `searchParams`.

Examples

Updating `searchParams`

You can use `useRouter` or `Link` to set new `searchParams`. After a navigation is performed, the current `page.js` will receive an updated `searchParams` prop.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

export default function ExampleClientComponent() {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()

  // Get a new searchParams string by merging the current
  // searchParams with a provided key/value pair
  const createQueryString = useCallback(
    (name: string, value: string) => {
      const params = new URLSearchParams(searchParams.toString())
      params.set(name, value)

      return params.toString()
    },
    [searchParams]
  )

  return (
    <>
      <p>Sort By</p>


      <button
        onClick={() => {
          // <pathname>?sort=asc
          router.push(pathname + '?' + createQueryString('sort', 'asc'))
        }}
      >
        ASC
      </button>


      <Link
        href={
          // <pathname>?sort=desc
          pathname + '?' + createQueryString('sort', 'desc')
        }
      >
        DESC
      </Link>
    </>
  )
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

export default function ExampleClientComponent() {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()

  // Get a new searchParams string by merging the current
  // searchParams with a provided key/value pair
  const createQueryString = useCallback(
    (name, value) => {
      const params = new URLSearchParams(searchParams)
      params.set(name, value)

      return params.toString()
    },
    [searchParams]
  )

  return (
    <>
      <p>Sort By</p>


      <button
        onClick={() => {
          // <pathname>?sort=asc
          router.push(pathname + '?' + createQueryString('sort', 'asc'))
        }}
      >
        ASC
      </button>


      <Link
        href={
          // <pathname>?sort=desc
          pathname + '?' + createQueryString('sort', 'desc')
        }
      >
        DESC
      </Link>
    </>
  )
}
```

Version History

Version	Changes
`v13.0.0`	`useSearchParams` introduced.

useSelectedLayoutSegment

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-selected-layout-segment
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useSelectedLayoutSegment hook.*

`useSelectedLayoutSegment` is a **Client Component** hook that lets you read the active route segment **one level below** the Layout it is called from.

It is useful for navigation UI, such as tabs inside a parent layout that change style depending on the active child segment.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useSelectedLayoutSegment } from 'next/navigation'

export default function ExampleClientComponent() {
  const segment = useSelectedLayoutSegment()

  return <p>Active segment: {segment}</p>
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { useSelectedLayoutSegment } from 'next/navigation'

export default function ExampleClientComponent() {
  const segment = useSelectedLayoutSegment()

  return <p>Active segment: {segment}</p>
}
```

Good to know:

- Since `useSelectedLayoutSegment` is a Client Component hook, and Layouts are Server Components by default, `useSelectedLayoutSegment` is usually called via a Client Component that is imported into a Layout.
- `useSelectedLayoutSegment` only returns the segment one level down. To return all active segments, see `useSelectedLayoutSegments`
- For catch-all routes, the matched segments are returned as a single joined string. For example, given `app/blog/[...slug]/page.js`, calling from `app/blog/layout.js` when visiting `/blog/a/b/c` returns `'a/b/c'`.

Parameters

```
const segment = useSelectedLayoutSegment(parallelRouteKey?: string)
```

`useSelectedLayoutSegment` *optionally* accepts a `parallelRouteKey`, which allows you to read the active route segment within that slot.

Returns

`useSelectedLayoutSegment` returns a string of the active segment or `null` if one doesn't exist.

For example, given the Layouts and URLs below, the returned segment would be:

Layout	Visited URL	Returned Segment
`app/layout.js`	`/`	`null`
`app/layout.js`	`/dashboard`	`'dashboard'`
`app/dashboard/layout.js`	`/dashboard`	`null`
`app/dashboard/layout.js`	`/dashboard/settings`	`'settings'`
`app/dashboard/layout.js`	`/dashboard/analytics`	`'analytics'`
`app/dashboard/layout.js`	`/dashboard/analytics/monthly`	`'analytics'`

For catch-all routes (`[...slug]`), the returned segment contains all matched path segments joined as a single string:

Layout	Visited URL	Returned Segment
`app/blog/layout.js`	`/blog/a/b/c`	`'a/b/c'`

Behavior

Cache Components

When `cacheComponents` is enabled, `useSelectedLayoutSegment` may require a `Suspense` boundary. This depends on whether the active segment can be resolved during prerendering.

- **Static routes and routes with `generateStaticParams`**: every route segment, including dynamic params, is known at build time. The active segment can be resolved during prerendering, so `useSelectedLayoutSegment` resolves on the server and no `Suspense` boundary is required.
- **Routes with dynamic params not covered by `generateStaticParams`**: the param is a fallback param that is not known until request time. The active segment cannot be resolved during prerendering, so `useSelectedLayoutSegment` suspends. Wrap the component (or a parent) in a `Suspense` boundary so its fallback can be rendered during prerendering; otherwise, the build fails.

This applies even when the component that calls `useSelectedLayoutSegment` is itself static. For example, a tab bar rendered in a parent layout suspends on any page below it that has an unknown dynamic param. To keep the rest of the layout prerendered, wrap the component that calls `useSelectedLayoutSegment` (or a parent) in a `Suspense` boundary with a fallback.

See Next.js encountered URL data in a Client Component outside of Suspense for full fix options and trade-offs.

Examples

Creating an active link component

You can use `useSelectedLayoutSegment` to create an active link component that changes style depending on the active segment. For example, a featured posts list in the sidebar of a blog:

```
// Archivo: app/blog/blog-nav-link.tsx switcher
'use client'

import Link from 'next/link'
import { useSelectedLayoutSegment } from 'next/navigation'

// This *client* component will be imported into a blog layout
export default function BlogNavLink({
  slug,
  children,
}: {
  slug: string
  children: React.ReactNode
}) {
  // Navigating to `/blog/hello-world` will return 'hello-world'
  // for the selected layout segment
  const segment = useSelectedLayoutSegment()
  const isActive = slug === segment

  return (
    <Link
      href={`/blog/${slug}`}
      // Change style depending on whether the link is active
      style={{ fontWeight: isActive ? 'bold' : 'normal' }}
    >
      {children}
    </Link>
  )
}
```

```
// Archivo: app/blog/blog-nav-link.js switcher
'use client'

import Link from 'next/link'
import { useSelectedLayoutSegment } from 'next/navigation'

// This *client* component will be imported into a blog layout
export default function BlogNavLink({ slug, children }) {
  // Navigating to `/blog/hello-world` will return 'hello-world'
  // for the selected layout segment
  const segment = useSelectedLayoutSegment()
  const isActive = slug === segment

  return (
    <Link
      href={`/blog/${slug}`}
      // Change style depending on whether the link is active
      style={{ fontWeight: isActive ? 'bold' : 'normal' }}
    >
      {children}
    </Link>
  )
}
```

```
// Archivo: app/blog/layout.tsx switcher
// Import the Client Component into a parent Layout (Server Component)
import { BlogNavLink } from './blog-nav-link'
import getFeaturedPosts from './get-featured-posts'

export default async function Layout({
  children,
}: {
  children: React.ReactNode
}) {
  const featuredPosts = await getFeaturedPosts()
  return (
    <div>
      {featuredPosts.map((post) => (
        <div key={post.id}>
          <BlogNavLink slug={post.slug}>{post.title}</BlogNavLink>
        </div>
      ))}
      <div>{children}</div>
    </div>
  )
}
```

```
// Archivo: app/blog/layout.js switcher
// Import the Client Component into a parent Layout (Server Component)
import { BlogNavLink } from './blog-nav-link'
import getFeaturedPosts from './get-featured-posts'

export default async function Layout({ children }) {
  const featuredPosts = await getFeaturedPosts()
  return (
    <div>
      {featuredPosts.map((post) => (
        <div key={post.id}>
          <BlogNavLink slug={post.slug}>{post.title}</BlogNavLink>
        </div>
      ))}
      <div>{children}</div>
    </div>
  )
}
```

Version History

Version	Changes
`v13.0.0`	`useSelectedLayoutSegment` introduced.

useSelectedLayoutSegments

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/use-selected-layout-segments
> 📂 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useSelectedLayoutSegments hook.*

`useSelectedLayoutSegments` is a **Client Component** hook that lets you read the active route segments **below** the Layout it is called from.

It is useful for creating UI in parent Layouts that need knowledge of active child segments such as breadcrumbs.

```
// Archivo: app/example-client-component.tsx switcher
'use client'

import { useSelectedLayoutSegments } from 'next/navigation'

export default function ExampleClientComponent() {
  const segments = useSelectedLayoutSegments()

  return (
    <ul>
      {segments.map((segment, index) => (
        <li key={index}>{segment}</li>
      ))}
    </ul>
  )
}
```

```
// Archivo: app/example-client-component.js switcher
'use client'

import { useSelectedLayoutSegments } from 'next/navigation'

export default function ExampleClientComponent() {
  const segments = useSelectedLayoutSegments()

  return (
    <ul>
      {segments.map((segment, index) => (
        <li key={index}>{segment}</li>
      ))}
    </ul>
  )
}
```

Good to know:

- Since `useSelectedLayoutSegments` is a Client Component hook, and Layouts are Server Components by default, `useSelectedLayoutSegments` is usually called via a Client Component that is imported into a Layout.
- The returned segments include Route Groups, which you might not want to be included in your UI. You can use the `filter` array method to remove items that start with a bracket.
- For catch-all routes, the matched segments are returned as a single joined string within the array. For example, given `app/blog/[...slug]/page.js`, calling from `app/layout.js` when visiting `/blog/a/b/c` returns `['blog', 'a/b/c']`, not `['blog', 'a', 'b', 'c']`.

Parameters

```
const segments = useSelectedLayoutSegments(parallelRouteKey?: string)
```

`useSelectedLayoutSegments` *optionally* accepts a `parallelRouteKey`, which allows you to read the active route segment within that slot.

Returns

`useSelectedLayoutSegments` returns an array of strings containing the active segments one level down from the layout the hook was called from. Or an empty array if none exist.

For example, given the Layouts and URLs below, the returned segments would be:

Layout	Visited URL	Returned Segments
`app/layout.js`	`/`	`[]`
`app/layout.js`	`/dashboard`	`['dashboard']`
`app/layout.js`	`/dashboard/settings`	`['dashboard', 'settings']`
`app/dashboard/layout.js`	`/dashboard`	`[]`
`app/dashboard/layout.js`	`/dashboard/settings`	`['settings']`

For catch-all routes (`[...slug]`), all matched path segments are returned as a single joined string within the array:

Layout	Visited URL	Returned Segments
`app/layout.js`	`/blog/a/b/c`	`['blog', 'a/b/c']`
`app/blog/layout.js`	`/blog/a/b/c`	`['a/b/c']`

Behavior

Cache Components

When `cacheComponents` is enabled, `useSelectedLayoutSegments` may require a `Suspense` boundary. This depends on whether the active segments can be resolved during prerendering.

- **Static routes and routes with `generateStaticParams`**: every route segment, including dynamic params, is known at build time. The active segments can be resolved during prerendering, so `useSelectedLayoutSegments` resolves on the server and no `Suspense` boundary is required.
- **Routes with dynamic params not covered by `generateStaticParams`**: the param is a fallback param that is not known until request time. The active segments cannot be resolved during prerendering, so `useSelectedLayoutSegments` suspends. Wrap the component (or a parent) in a `Suspense` boundary so its fallback can be rendered during prerendering; otherwise, the build fails.

This applies even when the component that calls `useSelectedLayoutSegments` is itself static. For example, a breadcrumb component rendered in a parent layout suspends on any page below it that has an unknown dynamic param. To keep the rest of the layout prerendered, wrap the component that calls `useSelectedLayoutSegments` (or a parent) in a `Suspense` boundary with a fallback.

See Next.js encountered URL data in a Client Component outside of Suspense for full fix options and trade-offs.

Version History

Version	Changes
`v13.0.0`	`useSelectedLayoutSegments` introduced.

userAgent

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/functions/userAgent
> 📁 **Sección:** App > Api Reference > Functions
> 📝 **Descripción:** *The userAgent helper extends the Web Request API with additional properties and methods to interact with the user agent object from the request.*

The `userAgent` helper extends the Web Request API with additional properties and methods to interact with the user agent object from the request.

```
// Archivo: proxy.ts switcher
import { NextRequest, NextResponse, userAgent } from 'next/server'

export function proxy(request: NextRequest) {
  const url = request.nextUrl
  const { device } = userAgent(request)

  // device.type can be: 'mobile', 'tablet', 'console', 'smarttv',
  // 'wearable', 'embedded', or undefined (for desktop browsers)
  const viewport = device.type || 'desktop'

  url.searchParams.set('viewport', viewport)
  return NextResponse.rewrite(url)
}
```

```
// Archivo: proxy.js switcher
import { NextResponse, userAgent } from 'next/server'

export function proxy(request) {
  const url = request.nextUrl
  const { device } = userAgent(request)

  // device.type can be: 'mobile', 'tablet', 'console', 'smarttv',
  // 'wearable', 'embedded', or undefined (for desktop browsers)
  const viewport = device.type || 'desktop'

  url.searchParams.set('viewport', viewport)
  return NextResponse.rewrite(url)
}
```

`isBot`

A boolean indicating whether the request comes from a known bot.

`browser`

An object containing information about the browser used in the request.

- `name` : A string representing the browser's name, or `undefined` if not identifiable.
- `version` : A string representing the browser's version, or `undefined` .

`device`

An object containing information about the device used in the request.

- `model` : A string representing the model of the device, or `undefined` .
- `type` : A string representing the type of the device, such as `console` , `mobile` , `tablet` , `smarttv` , `wearable` , `embedded` , or `undefined` .
- `vendor` : A string representing the vendor of the device, or `undefined` .

`engine`

An object containing information about the browser's engine.

- `name` : A string representing the engine's name. Possible values include: `Amaya` , `Blink` , `EdgeHTML` , `Flow` , `Gecko` , `Goanna` , `iCab` , `KHTML` , `Links` , `Lynx` , `NetFront` , `NetSurf` , `Presto` , `Tasman` , `Trident` , `w3m` , `WebKit` or `undefined` .
- `version` : A string representing the engine's version, or `undefined` .

`os`

An object containing information about the operating system.

- `name` : A string representing the name of the OS, or `undefined` .
- `version` : A string representing the version of the OS, or `undefined` .

`cpu`

An object containing information about the CPU architecture.

- `architecture` : A string representing the architecture of the CPU. Possible values include: `68k` , `amd64` , `arm` , `arm64` , `armhf` , `avr` , `ia32` , `ia64` , `irix` , `irix64` , `mips` , `mips64` , `pa-risc` , `ppc` , `sparc` , `sparc64` or `undefined`

adapterPath

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/adapterPath
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure a custom adapter for Next.js to hook into the build process.*

Next.js provides a built-in adapters API. It allows deployment platforms or build systems to integrate with the Next.js build process.

For a full reference implementation, see the `nextjs/adapter-vercel` adapter.

Configuration

To use an adapter, specify the path to your adapter module in `adapterPath` :

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  adapterPath: require.resolve('./my-adapter.js'),
}

module.exports = nextConfig
```

Alternatively `NEXT_ADAPTER_PATH` can be set to enable zero-config usage in deployment platforms.

Adapters

For full adapter implementation details, use the dedicated Adapters section:

- Configuration
- Creating an Adapter
- API Reference
- Testing Adapters
- Routing with `@next/routing`
- Implementing PPR in an Adapter
- Runtime Integration
- Invoking Entrypoints
- Output Types
- Routing Information
- Use Cases

Creating an Adapter

See Creating an Adapter.

API Reference

See API Reference.

Testing Adapters

See Testing Adapters.

Routing with `@next/routing`

See Routing with `@next/routing`.

Implementing PPR in an Adapter

See Implementing PPR in an Adapter.

Runtime Integration

See Runtime Integration.

Invoking Entrypoints

See Invoking Entrypoints.

Output Types

See Output Types.

Routing Information

See Routing Information.

Use Cases

See Use Cases.

---

allowedDevOrigins

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/allowedDevOrigins
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use* `allowedDevOrigins` *to configure additional origins that can request the dev server.*

Next.js blocks cross-origin requests to dev-only assets and endpoints during development by default to prevent unauthorized access.

To configure a Next.js application to allow requests from origins other than the hostname the server was initialized with (`localhost` by default), use the `allowedDevOrigins` config option.

`allowedDevOrigins` lets you set additional origins that can request the dev server in development mode. For example, to use `local-origin.dev` instead of only `localhost`, open `next.config.js` and add the `allowedDevOrigins` config:

```
// Archivo: next.config.js
module.exports = {
  allowedDevOrigins: ['local-origin.dev', '*.local-origin.dev'],
}
```

Only the `hostname` of the request's `Origin` header is matched against your entries. For a request from `http://local-origin.dev:3000/dashboard?tab=1`, that is `local-origin.dev`. The scheme, the port, the path, and the query string are ignored. Write your entries that way too, without `https://` and without a port.

A no-cors cross-site request, such as a script tag loading a dev asset, sends no `Origin` header. Those are matched on the `Referer` hostname instead.

Entries can also expand, through two wildcards: a `*` stands in for exactly one label of the hostname, and `**` for one or more. That is why the example above lists two entries, one for the bare hostname and one for its subdomains.

Entry	Matches	Does not match
`local-origin.dev`	`local-origin.dev`	`team.local-origin.dev`
`*.local-origin.dev`	`team.local-origin.dev`	`local-origin.dev`, `team.eu.local-origin.dev`
`**.local-origin.dev`	`team.local-origin.dev`, `team.eu.local-origin.dev`	`local-origin.dev`

Partial replacement is not supported. Write `*.local-origin.dev`, rather than `team-*.local-origin.dev`. Using `**` is only supported at the start of the pattern.

The dev server already allows `localhost`, its subdomains, and the hostname it was started with. Any other hostname needs an entry, such as a tunnel used for remote development:

```
// Archivo: next.config.js
module.exports = {
  allowedDevOrigins: ['*.tunnel.example.com'],
}
```

---

appDir

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/appDir
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable the App Router to use layouts, streaming, and more.*

> **Good to know**: This option is **no longer** needed as of Next.js 13.4. The App Router is now stable.

The App Router (`app` directory) enables support for layouts, Server Components, streaming, and colocated data fetching.

Using the `app` directory will automatically enable React Strict Mode. Learn how to incrementally adopt `app`.

---

assetPrefix

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/assetPrefix
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to use the assetPrefix config option to configure your CDN.*

> **Attention**: Deploying to Vercel automatically configures a global CDN for your Next.js project. You do not need to manually set up an Asset Prefix.

> 📘 **Nota para Pages Router:**
>
> > **Attention**: Deploying to Vercel automatically configures a global CDN for your Next.js project. You do not need to manually set up an Asset Prefix.

> **Good to know**: Next.js 9.5+ added support for a customizable Base Path, which is better suited for hosting your application on a sub-path like `/docs`. We do not suggest you use a custom Asset Prefix for this use case.

Set up a CDN

To set up a CDN, you can set up an asset prefix and configure your CDN's origin to resolve to the domain that Next.js is hosted on.

Open `next.config.mjs` and add the `assetPrefix` config based on the phase:

```
// Archivo: next.config.mjs
// @ts-check
import { PHASE_DEVELOPMENT_SERVER } from 'next/constants'

export default (phase) => {
  const isDev = phase === PHASE_DEVELOPMENT_SERVER
  /**
   * @type {import('next').NextConfig}
   */
  const nextConfig = {
    assetPrefix: isDev ? undefined : 'https://cdn.mydomain.com',
  }
  return nextConfig
}
```

Next.js will automatically use your asset prefix for the JavaScript and CSS files it loads from the `/_next/` path (`.next/static/` folder). For example, with the above configuration, the following request for a JS chunk:

```
/_next/static/chunks/4b9b41aaa062cbbfeff4add70f256968c51ece5d.4d708494b3aed70c04f0.js
```

Would instead become:

```
https://cdn.mydomain.com/_next/static/chunks/4b9b41aaa062cbbfeff4add70f256968c51ece5d.4d708494b3aed70c04f0.js
```

The exact configuration for uploading your files to a given CDN will depend on your CDN of choice. The only folder you need to host on your CDN is the contents of `.next/static/`, which should be uploaded as `_next/static/` as the above URL request indicates. **Do not upload the rest of your `.next/` folder**, as you should not expose your server code and other configuration to the public.

While `assetPrefix` covers requests to `_next/static`, it does not influence the following paths:

- Files in the public folder; if you want to serve those assets over a CDN, you'll have to introduce the prefix yourself

> 📘 **Nota para Pages Router:**
>
> - Files in the public folder; if you want to serve those assets over a CDN, you'll have to introduce the prefix yourself
> - `/_next/data/` requests for `getServerSideProps` pages. These requests will always be made against the main domain since they're not static.
> - `/_next/data/` requests for `getStaticProps` pages. These requests will always be made against the main domain to support Incremental Static Generation, even if you're not using it (for consistency).

---

authInterrupts

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/authInterrupts
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to enable the experimental `authInterrupts` configuration option to use `forbidden` and `unauthorized`.*

The `authInterrupts` configuration option allows you to use `forbidden` and `unauthorized` APIs in your application. While these functions are experimental, you must enable the `authInterrupts` option in your `next.config.js` file to use them:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    authInterrupts: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  experimental: {
    authInterrupts: true,
  },
}
```

---

basePath

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/basePath
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use `basePath` to deploy a Next.js application under a sub-path of a domain.*

To deploy a Next.js application under a sub-path of a domain you can use the `basePath` config option.

`basePath` allows you to set a path prefix for the application. For example, to use `/docs` instead of `''` (an empty string, the default), open `next.config.js` and add the `basePath` config:

```
// Archivo: next.config.js
module.exports = {
  basePath: '/docs',
}
```

> **Good to know**: This value must be set at build time and cannot be changed without re-building as the value is inlined in the client-side bundles.

Links

When linking to other pages using `next/link` and `next/router` the `basePath` will be automatically applied.

For example, using `/about` will automatically become `/docs/about` when `basePath` is set to `/docs`.

```
export default function HomePage() {
  return (
    <>
      [About Page](/about)
    </>
  )
}
```

Output html:

```
<a href="/docs/about">About Page</a>
```

This makes sure that you don't have to change all links in your application when changing the `basePath` value.

Images

When using the `next/image` component, you will need to add the `basePath` in front of `src`.

> 📘 **Nota para Pages Router:** When using the `next/image` component, you will need to add the `basePath` in front of `src`.

For example, using `/docs/me.png` will properly serve your image when `basePath` is set to `/docs`.

```
import Image from 'next/image'

function Home() {
  return (
    <>
      <h1>My Homepage</h1>


![Picture of the author](https://nextjs.org/docs/me.png)


      <p>Welcome to my homepage!</p>
    </>
  )
}

export default Home
```

---

cacheComponents

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheComponents
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to enable the cacheComponents flag in Next.js.*

Cache Components enables component and function-level caching using the `use cache` directive. Data fetching is dynamic by default, and you choose what to cache at the page, component, or function level. Next.js prerenders a static HTML shell that is served immediately while dynamic content streams in when ready, letting you mix static and dynamic content within a single route.

Usage

To enable the `cacheComponents` flag, set it to `true` in your `next.config.ts` file:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
}

export default nextConfig
```

> **Good to know**: Cache Components requires the Node.js runtime. Migrate any routes that set the deprecated `runtime = 'edge'` export, and note that other server-side JavaScript runtimes are not guaranteed to work. See Migrating to Cache Components.

When `cacheComponents` is enabled, you can use the following cache functions and configurations:

- The `use cache` directive
- The `cacheLife` function with `use cache`

- The `cacheTag` function

> **Good to know**: If you used `experimental.useCache` or `experimental.dynamicIO`, migrate using the Version 16 upgrade guide. To migrate route segment configs and other caching patterns, see Migrating to Cache Components.

Additionally, `cacheComponents` implements **Partial Prerendering (PPR)** as the default behavior in the App Router. This means the `experimental.ppr` configuration flag and the `experimental_ppr` route segment configuration are no longer necessary and have been removed.

Read Prerendering for how the static shell and streaming fit together.

> **Good to know**: If you used experimental PPR in Next.js 15, refer to the Partial Prerendering (PPR) section of the Version 16 upgrade guide when migrating.

Navigation with Activity

When `cacheComponents` is enabled, Next.js uses React's `<Activity>` component to preserve component state during client-side navigation.

Rather than unmounting the previous route when you navigate away, Next.js sets the Activity mode to `"hidden"`. This means:

- Component state is preserved when navigating between routes
- When you navigate back, the previous route reappears with its state intact
- Effects are cleaned up when a route is hidden, and recreated when it becomes visible again

This behavior improves the navigation experience by maintaining UI state (form inputs, or expanded sections) when users navigate back and forth between routes.

> **Good to know**: Next.js uses heuristics to keep a few recently visited routes `"hidden"`, while older routes are removed from the DOM to prevent excessive growth.

Some UI patterns behave differently when components stay mounted instead of unmounting. See the Preserving UI state guide for handling common patterns like dropdowns, dialogs, and testing.

Version History

Version	Change
16.0.0	`cacheComponents` introduced. This flag controls the `ppr`, `useCache`, and `dynamicIO` flags as a single, unified configuration.

---

cacheHandlers

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheHandlers
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure custom cache handlers for use cache directives in Next.js.*

The `cacheHandlers` configuration allows you to define custom cache storage implementations for `'use cache'` and `'use cache: remote'`. This enables you to store cached components and functions in external services or customize the caching behavior. `'use cache: private'` is not configurable.

When to use custom cache handlers

Most applications don't need custom cache handlers. The default in-memory cache works well in the typical use case.

Custom cache handlers are for advanced scenarios where you need to either share cache across multiple instances or change where the cache is stored. For example, you can configure a custom `remote` handler for external storage (like a key-value store), then use `'use cache'` in your code for in-memory caching and `'use cache: remote'` for the external storage, allowing different caching strategies within the same application.

Sharing cache across instances

The default in-memory cache is isolated to each Next.js process. If you're running multiple servers or containers, each instance will have its own cache that isn't shared with others and is lost on restart.

Custom handlers let you integrate with shared storage systems (like Redis, Memcached, or DynamoDB) that all your Next.js instances can access.

Changing storage type

You might want to store cache differently than the default in-memory approach. You can implement a custom handler to store cache on disk, in a database, or in an external caching service. Reasons include: persistence across restarts, reducing memory usage, or integrating with existing infrastructure.

Usage

To configure custom cache handlers:

1. Define your cache handler in a separate file, see examples for implementation details.
2. Reference the file path in your Next config file

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheHandlers: {
    default: require.resolve('./cache-handlers/default-handler.js'),
    remote: require.resolve('./cache-handlers/remote-handler.js'),
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  cacheHandlers: {
    default: require.resolve('./cache-handlers/default-handler.js'),
    remote: require.resolve('./cache-handlers/remote-handler.js'),
  },
}
```

Handler types

- `default`: Used by the `'use cache'` directive
- `remote`: Used by the `'use cache: remote'` directive

If you don't configure `cacheHandlers`, Next.js uses an in-memory LRU (Least Recently Used) cache for both `default` and `remote`. You can view the default implementation as a reference.

You can also define additional named handlers (e.g., `sessions`, `analytics`) and reference them with `'use cache: <name>'`.

Note that `'use cache: private'` does not use cache handlers and cannot be customized.

API Reference

A cache handler must implement the `CacheHandler` interface with the following methods:

`get()`

Retrieve a cache entry for the given cache key.

```
get(cacheKey: string, softTags: string[]): Promise<CacheEntry | undefined>
```

Parameter	Type	Description
`cacheKey`	`string`	The unique key for the cache entry.
`softTags`	`string[]`	Implicit tags derived from the route path. See Soft Tags for how to use them.

Returns a `CacheEntry` object if found, or `undefined` if not found or expired.

Your `get` method should retrieve the cache entry from storage, check if it has expired based on the `revalidate` time, and return `undefined` for missing or expired entries.

```
const cacheHandler = {
  async get(cacheKey, softTags) {
    const entry = cache.get(cacheKey)
    if (!entry) return undefined

    // Check if expired
    const now = Date.now()
    if (now > entry.timestamp + entry.revalidate * 1000) {
      return undefined
    }

    return entry
  },
}
```

`set()`

Store a cache entry for the given cache key.

```
set(cacheKey: string, pendingEntry: Promise<CacheEntry>): Promise<void>
```

Parameter	Type	Description
`cacheKey`	`string`	The unique key to store the entry under.
`pendingEntry`	`Promise<CacheEntry>`	A promise that resolves to the cache entry.

The entry may still be pending when this is called (i.e., its value stream may still be written to). Your handler should await the promise before processing the entry.

Returns `Promise<void>`.

Your `set` method must await the `pendingEntry` promise before storing it, since the cache entry may still be generating when this method is called. Once resolved, store the entry in your cache system.

```
const cacheHandler = {
  async set(cacheKey, pendingEntry) {
    // Wait for the entry to be ready
    const entry = await pendingEntry

    // Store in your cache system
    cache.set(cacheKey, entry)
  },
}
```

`refreshTags()`

Called periodically before starting a new request to sync with external tag services.

```
refreshTags(): Promise<void>
```

This is useful if you're coordinating cache invalidation across multiple instances or services. For in-memory caches, this can be a no-op.

Returns `Promise<void>`.

For in-memory caches, this can be a no-op. For distributed caches, use this to sync tag state from an external service or database before processing requests.

```
const cacheHandler = {
  async refreshTags() {
    // For in-memory cache, no action needed
    // For distributed cache, sync tag state from external service
  },
}
```

`getExpiration()`

Get the maximum revalidation timestamp for a set of tags.

```
getExpiration(tags: string[]): Promise<number>
```

Parameter	Type	Description
`tags`	`string[]`	Array of tags to check expiration for.

Returns:

- `0` if none of the tags were ever revalidated
- A timestamp (in milliseconds) representing the most recent revalidation
- `Infinity` to indicate soft tags should be checked in the `get` method instead

If you're not tracking tag revalidation timestamps, return `0`. Otherwise, find the most recent revalidation timestamp across all the provided tags. Return `Infinity` if you prefer to handle soft tag checking in the `get` method.

```
const cacheHandler = {
  async getExpiration(tags) {
    // Return 0 if not tracking tag revalidation
    return 0

    // Or return the most recent revalidation timestamp
    // return Math.max(...tags.map(tag => tagTimestamps.get(tag) || 0));
  },
}
```

`updateTags()`

Called when tags are revalidated or expired.

```
updateTags(tags: string[], durations?: { expire?: number }): Promise<void>
```

Parameter	Type	Description
`tags`	`string[]`	Array of tags to update.
`durations`	`{ expire?: number }`	Optional expiration duration in seconds.

Your handler should update its internal state to mark these tags as invalidated.

Returns `Promise<void>`.

When tags are revalidated, your handler should invalidate all cache entries that have any of those tags. Iterate through your cache and remove entries whose tags match the provided list.

```
const cacheHandler = {
  async updateTags(tags, durations) {
    // Invalidate all cache entries with matching tags
    for (const [key, entry] of cache.entries()) {
      if (entry.tags.some((tag) => tags.includes(tag))) {
        cache.delete(key)
      }
    }
  },
}
```

CacheEntry Type

The `CacheEntry` object has the following structure:

```
interface CacheEntry {
  value: ReadableStream<Uint8Array>
  tags: string[]
  stale: number
  timestamp: number
  expire: number
  revalidate: number
}
```

Property	Type	Description
`value`	`ReadableStream<Uint8Array>`	The cached data as a stream.
`tags`	`string[]`	Cache tags (excluding soft tags).
`stale`	`number`	Duration in seconds for client-side staleness.
`timestamp`	`number`	When the entry was created (timestamp in milliseconds).
`expire`	`number`	How long the entry is allowed to be used (in seconds).
`revalidate`	`number`	How long until the entry should be revalidated (in seconds).

> **Good to know**:
>
> - The `value` is a `ReadableStream`. Use `.tee()` if you need to read and store the stream data.
> - If the stream errors with partial data, your handler must decide whether to keep the partial cache or discard it.

Examples

Basic in-memory cache handler

Here's a minimal implementation using a `Map` for storage. This example demonstrates the core concepts, but for a production-ready implementation with LRU eviction, error handling, and tag management, see the default cache handler.

```
// Archivo: cache-handlers/memory-handler.js
const cache = new Map()
const pendingSets = new Map()

module.exports = {
  async get(cacheKey, softTags) {
    // Wait for any pending set operation to complete
    const pendingPromise = pendingSets.get(cacheKey)
    if (pendingPromise) {
      await pendingPromise
    }

    const entry = cache.get(cacheKey)
    if (!entry) {
      return undefined
    }

    // Check if entry has expired
    const now = Date.now()
    if (now > entry.timestamp + entry.revalidate * 1000) {
      return undefined
    }

    return entry
  },

  async set(cacheKey, pendingEntry) {
    // Create a promise to track this set operation
    let resolvePending
    const pendingPromise = new Promise((resolve) => {
      resolvePending = resolve
    })
    pendingSets.set(cacheKey, pendingPromise)

    try {
      // Wait for the entry to be ready
      const entry = await pendingEntry

      // Store the entry in the cache
      cache.set(cacheKey, entry)
    } finally {
      resolvePending()
      pendingSets.delete(cacheKey)
    }
  },

  async refreshTags() {
    // No-op for in-memory cache
  },

  async getExpiration(tags) {
    // Return 0 to indicate no tags have been revalidated
    return 0
  },

  async updateTags(tags, durations) {
    // Implement tag-based invalidation
    for (const [key, entry] of cache.entries()) {
      if (entry.tags.some((tag) => tags.includes(tag))) {
        cache.delete(key)
      }
    }
  },
}
```

External storage pattern

For durable storage like Redis or a database, you'll need to serialize the cache entries. Here's a simple Redis example:

```
// Archivo: cache-handlers/redis-handler.js
const { createClient } = require('redis')

const client = createClient({ url: process.env.REDIS_URL })
client.connect()

module.exports = {
  async get(cacheKey, softTags) {
    // Retrieve from Redis
    const stored = await client.get(cacheKey)
    if (!stored) return undefined

    // Deserialize the entry
    const data = JSON.parse(stored)

    // Reconstruct the ReadableStream from stored data
    return {
      value: new ReadableStream({
        start(controller) {
          controller.enqueue(Buffer.from(data.value, 'base64'))
          controller.close()
        },
      }),
      tags: data.tags,
      stale: data.stale,
      timestamp: data.timestamp,
      expire: data.expire,
      revalidate: data.revalidate,
    }
  },

  async set(cacheKey, pendingEntry) {
    const entry = await pendingEntry

    // Read the stream to get the data
    const reader = entry.value.getReader()
    const chunks = []

    try {
      while (true) {
        const { done, value } = await reader.read()
        if (done) break
        chunks.push(value)
      }
    } finally {
      reader.releaseLock()
    }

    // Combine chunks and serialize for Redis storage
    const data = Buffer.concat(chunks.map((chunk) => Buffer.from(chunk)))

    await client.set(
      cacheKey,
      JSON.stringify({
        value: data.toString('base64'),
        tags: entry.tags,
        stale: entry.stale,
        timestamp: entry.timestamp,
        expire: entry.expire,
        revalidate: entry.revalidate,
      }),
      { EX: entry.expire } // Use Redis TTL for automatic expiration
    )
  },

  async refreshTags() {
    // No-op for basic Redis implementation
    // Could sync with external tag service if needed
  },

  async getExpiration(tags) {
    // Return 0 to indicate no tags have been revalidated
    // Could query Redis for tag expiration timestamps if tracking them
```

```
    return 0
  },

  async updateTags(tags, durations) {
    // Implement tag-based invalidation if needed
    // Could iterate over keys with matching tags and delete them
  },
}
```

Distributed Tag Coordination

When running multiple Next.js instances, tag invalidation must be coordinated across instances. The default in-memory handler only tracks tags locally, so calling `revalidateTag()` on one instance does not affect others.

To coordinate tags across instances:

1. **`updateTags()`** is called when `revalidateTag()` is invoked. Your handler should write the invalidation timestamp to shared storage.
2. **`refreshTags()`** is called before each request. Your handler should read recent invalidation events from shared storage and update its local tag state.
3. **`getExpiration()`** returns the most recent revalidation timestamp across all provided tags. The default implementation returns `Math.max(...timestamps, 0)`.

Here's an example using Redis for distributed tag coordination:

```
// Archivo: cache-handlers/distributed-tags.js
const { createClient } = require('redis')

const client = createClient({ url: process.env.REDIS_URL })
client.connect()

// Local cache of tag timestamps, synced via refreshTags
const localTagTimestamps = new Map()

module.exports = {
  // ... get() and set() methods ...

  async refreshTags() {
    // Sync tag invalidation timestamps from Redis
    // Using a dedicated set to track tag keys avoids scanning the keyspace
    const tagKeys = await client.sMembers('revalidated-tags')
    if (tagKeys.length > 0) {
      const values = await client.mGet(tagKeys.map((k) => `tag:${k}`))
      for (let i = 0; i < tagKeys.length; i++) {
        localTagTimestamps.set(tagKeys[i], Number(values[i]))
      }
    }
  },

  async getExpiration(tags) {
    const timestamps = tags.map((tag) => localTagTimestamps.get(tag) || 0)
    return Math.max(...timestamps, 0)
  },

  async updateTags(tags, durations) {
    const now = Date.now()
    const pipeline = client.multi()
    for (const tag of tags) {
      pipeline.set(`tag:${tag}`, String(now))
      pipeline.sAdd('revalidated-tags', tag)
      localTagTimestamps.set(tag, now)
    }
    await pipeline.exec()
  },
}
```

For a full explanation of the tag architecture (including soft tags and multi-instance considerations), see How Revalidation Works.

Soft Tags

Soft tags are implicit tags that Next.js automatically generates based on the route path. Every segment in the path gets a layout tag, plus the leaf route itself. For example, the route `/blog/hello` generates soft tags for `/layout`, `/blog/layout`, `/blog/hello/layout`, and `/blog/hello`. These tags are prefixed internally with `_N_T_`.

Soft tags enable `revalidatePath()` to work through the same tag-based cache system. When `revalidatePath('/blog/hello')` is called, it invalidates all cache entries associated with that path's soft tags.

In the cache handler API, soft tags are passed to the `get()` method as the `softTags` parameter. Your handler should check whether any soft tag has been invalidated (via `getExpiration()` or direct timestamp comparison) after the cache entry's `timestamp`. If a soft tag was invalidated more recently than the entry was created, the entry should be treated as stale.

Handling Streams

The `CacheEntry.value` is a `ReadableStream<Uint8Array>`. When implementing a cache handler that stores entries externally, keep in mind:

- **Use** `.tee()` if you need to both store and return the stream. One branch goes to storage, the other is returned to the caller.
- **Memory implications**: large pages produce large cache entries. For S3-like storage backends, consider streaming directly to storage without buffering the entire entry in memory.
- **Partial writes**: the stream may error partway through rendering. Your handler should decide whether to keep partial entries or discard them. Discarding is safer, as partial entries can produce incomplete pages.

Error Handling

Cache operations should be implemented defensively:

- `set()` **failure**: the response is still served to the user because `set()` is called asynchronously after the response stream is already flowing. The cache entry is lost, and the next request triggers a fresh render.
- `get()` **failure**: your handler should catch internal errors and return `undefined` (the "cache miss" signal). The framework does not wrap `get()` in a try/catch, so an unhandled exception from `get()` will propagate as a render error.
- **Partial writes**: if a cache entry is partially written and then read, the behavior is undefined. Use atomic writes or a write-then-rename pattern to avoid serving partial entries.

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Version History

Version	Changes
`v16.0.0`	`cacheHandlers` introduced.

cacheLife

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheLife
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to set up cacheLife configurations in Next.js.*

The `cacheLife` option allows you to define **custom cache profiles** when using the `cacheLife` function inside components or functions, and within the scope of the `use cache` directive.

Usage

To define a profile, enable the `cacheComponents` flag and add the cache profile in the `cacheLife` object in the `next.config.js` file. For example, a `blog` profile:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
  cacheLife: {
    blog: {
      stale: 3600, // 1 hour
      revalidate: 900, // 15 minutes
      expire: 86400, // 1 day
    },
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  cacheComponents: true,
  cacheLife: {
    blog: {
      stale: 3600, // 1 hour
      revalidate: 900, // 15 minutes
      expire: 86400, // 1 day
    },
  },
}
```

You can now use this custom `blog` configuration in your component or function as follows:

```
// Archivo: app/actions.ts highlight={4,5} switcher
import { cacheLife } from 'next/cache'

export async function getCachedData() {
  'use cache'
  cacheLife('blog')
  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

```
// Archivo: app/actions.js highlight={4,5} switcher
import { cacheLife } from 'next/cache'

export async function getCachedData() {
  'use cache'
  cacheLife('blog')
  const res = await fetch('https://api.example.com/data')
  const data = await res.json()
  return data
}
```

You can also override a built-in profile by defining one with the same name (`default` , `seconds` , `minutes` , `hours` , `days` , `weeks` , or `max`). See Overriding the default cache profiles.

Reference

The configuration object has key values with the following format:

Property	Value	Description	Requirement
`stale`	`number`	Duration the client should cache a value without checking the server.	Optional
`revalidate`	`number`	Frequency at which the cache should refresh on the server; stale values may be served while revalidating.	Optional
`expire`	`number`	Maximum duration for which a value can remain stale before switching to dynamic.	Optional - Must be longer than `revalidate`

cacheMaxMemorySize

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheMaxMemorySize
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure the size of the in-memory cache that Next.js keeps in each server instance.*

`cacheMaxMemorySize` sets the size, in bytes, of the in-memory cache each Next.js server instance keeps. It defaults to 50 MB.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheMaxMemorySize: 50 * 1024 * 1024, // 50 MB, the default
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  cacheMaxMemorySize: 50 * 1024 * 1024, // 50 MB, the default
}

module.exports = nextConfig
```

The option sizes two separate caches:

- The server cache that stores prerendered pages, route handler responses, and optimized images. Set this to `0` when adopting a custom `cacheHandler` , so reads go to your store rather than a per-instance copy.

- The built-in handler behind `'use cache'`. If you register your own handler through `cacheHandlers`, that handler manages its own memory and this option no longer applies to it.

Setting `cacheMaxMemorySize: 0` disables both in-memory caches. It is also a way to mimic how `use cache` behaves in a serverless environment, where entries rarely survive between requests.

> **Good to know**: `next dev` keeps its own in-memory cache regardless of this option, so reloads stay fast. Production honors the configured value exactly.

---

compress

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/compress
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js provides gzip compression to compress rendered content and static files, it only works with the server target. Learn more about it here.*

By default, Next.js uses `gzip` to compress rendered content and static files when using `next start` or a custom server. This is an optimization for applications that do not have compression configured. If compression is *already* configured in your application via a custom server, Next.js will not add compression.

You can check if compression is enabled and which algorithm is used by looking at the `Accept-Encoding` (browser accepted options) and `Content-Encoding` (currently used) headers in the response.

Disabling compression

To disable **compression**, set the `compress` config option to `false`:

```
// Archivo: next.config.js
module.exports = {
  compress: false,
}
```

We **do not recommend disabling compression** unless you have compression configured on your server, as compression reduces bandwidth usage and improves the performance of your application. For example, you're using nginx and want to switch to `brotli`, set the `compress` option to `false` to allow nginx to handle compression.

---

crossOrigin

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/crossOrigin
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use the `crossOrigin` option to add a crossOrigin tag on the `script` tags generated by `next/script`.*

Use the `crossOrigin` option to add a `crossOrigin` attribute in all `<script>` tags generated by the `next/script` component

> 📘 **Nota para Pages Router:** `next/script` and `next/head` components

, and define how cross-origin requests should be handled.

```
// Archivo: next.config.js
module.exports = {
  crossOrigin: 'anonymous',
}
```

Options

- `'anonymous'` : Adds `crossOrigin="anonymous"` attribute.
- `'use-credentials'` : Adds `crossOrigin="use-credentials"` .

cssChunking

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/cssChunking
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use the `cssChunking` option to control how CSS files are chunked in your Next.js application.*

CSS Chunking is a strategy used to improve the performance of your web application by splitting and re-ordering CSS files into chunks. This lets a route load close to only the CSS it needs, instead of loading all the application's CSS at once.

You can control how CSS files are chunked using the `experimental.cssChunking` option in your `next.config.js` file:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    cssChunking: true, // default
  },
} satisfies NextConfig

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    cssChunking: true, // default
  },
}

module.exports = nextConfig
```

Options

- `true` **(default)** (**webpack and Turbopack**): Next.js will try to merge CSS files whenever possible, determining explicit and implicit dependencies between files from import order to reduce the number of chunks and therefore the number of requests.
- `false` (**webpack only**): Next.js will not attempt to merge or re-order your CSS files.
- `'strict'` (**webpack only**): Next.js will load CSS files in the correct order they are imported into your files, which can lead to more chunks and requests.
- `'graph'` (**Turbopack only**): Next.js uses a cost-based graph algorithm to group CSS across your routes, balancing the bytes each route downloads and the requests it makes.

Choosing a strategy

For most applications, the default (`true`) is the right choice in either bundler: it merges CSS to make fewer requests. Reach for another strategy only for a specific reason.

In Turbopack, that reason is usually performance. Switch to `graph` to tune how CSS is shared across routes, cutting the unused CSS a route downloads at the cost of more requests (see Balancing requests and grouping).

In webpack, that reason is usually correctness. Switch to `'strict'` if you run into unexpected CSS behavior. For example, if you import `a.css` and `b.css` in different files using a different `import` order (`a` before `b` , or `b` before `a`), `true` merges them in any order and assumes there are no dependencies between them; if `b.css` depends on `a.css` , `'strict'` prevents the merge and loads them in import order, at the cost of more chunks and requests. Use `false` to disable merging entirely.

Debugging what a route actually uses

While some unused CSS is acceptable, and most apps do not need to change anything, it is worth keeping render-blocking CSS in check. Lighthouse flags this as a **Reduce unused CSS** opportunity with an estimated saving, and Chrome DevTools shows it per stylesheet in its Coverage panel, where a usage bar shows each stylesheet's applied CSS in green and unused CSS in gray.

When reading the report, watch out for styles that only apply on interaction, such as `:hover` , `:focus` , or classes toggled by JavaScript for menus and modals, since Coverage counts them as unused until you trigger them.

The source of this unused CSS is one of two things. Either it is dead CSS in a stylesheet your route imports, which you fix in your source by removing the unused rules or moving them into a stylesheet only the routes that use them import (CSS Modules make this natural by scoping styles to the component that imports them).

Or the bundler merged another stylesheet into a shared chunk that your route loads. That is governed by your chunking strategy. In Turbopack, `graph` mode lets you fine-tune how aggressively CSS merges.

Balancing requests and grouping

The `graph` strategy groups CSS into shared chunks to cut requests. Turn it on with the string form, which uses the default tuning:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    cssChunking: 'graph',
  },
} satisfies NextConfig

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    cssChunking: 'graph',
  },
}

module.exports = nextConfig
```

To shift that balance, pass an object instead. Both `requestCost` and `weightDistribution` are optional, so include only the one you want to change:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    cssChunking: {
      type: 'graph',
      requestCost: 100000,
      weightDistribution: 0.1,
    },
  },
} satisfies NextConfig

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    cssChunking: {
      type: 'graph',
      requestCost: 100000,
      weightDistribution: 0.1,
    },
  },
}

module.exports = nextConfig
```

- **`requestCost`** (default `20000`): the estimated cost, in bytes, of each additional CSS request. Larger values bias toward fewer, larger shared chunks, and fewer requests overall.
- **`weightDistribution`** (default `0.1`): controls how a shared chunk's cost is distributed across the routes that load it, weighted by how much CSS each route imports. `0` weights every route equally; higher values give more weight to routes that import less CSS, so the algorithm prioritizes routes with less CSS, assuming extra CSS is less noticeable on large routes.

How `graph` decides what to merge

Merging CSS into shared chunks is what the default (`true`) already does; `graph` just lets you control where it draws the line between merging and splitting.

Take two routes that share a stylesheet:

```
/a → [shared.css, only-a.css]
/b → [shared.css]
```

Here, `/b` never imports `only-a.css`, so there is one decision to make: keep `only-a.css` in the shared chunk, or give it its own.

Keeping it merged lets `/a` load all its CSS in a single request. The cost is that `/b`, which also loads that shared chunk, now downloads `only-a.css` even though it never imported it.

Splitting it out spares `/b` those bytes, but now `/a` has to make two requests. The default picks between these with a fixed heuristic.

The `requestCost` option is how `graph` prices that trade: the amount of un-imported CSS worth one extra request. It keeps `only-a.css` merged while `only-a.css` stays well under `requestCost`, and splits it out once it grows large enough to outweigh the request.

What drives the decision is the size of the un-imported CSS a merge would push onto a route, much more than the size of the shared chunk it joins. With the default `requestCost` of about 20 KB, `only-a.css` would have to exceed roughly that size before it earns its own chunk, so small stylesheets stay merged.

Graph algorithm overview

deploymentId

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/deploymentId
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure a deployment identifier used for version skew protection and cache busting.*

The `deploymentId` option allows you to set an identifier for your deployment. This identifier is used for version skew protection and cache busting during rolling deployments.

```
// Archivo: next.config.js
module.exports = {
  deploymentId: 'my-deployment-id',
}
```

You can also set the deployment ID using the `NEXT_DEPLOYMENT_ID` environment variable:

```
NEXT_DEPLOYMENT_ID=my-deployment-id next build
```

> **Good to know:** If both are set, the `deploymentId` value in `next.config.js` takes precedence over the `NEXT_DEPLOYMENT_ID` environment variable.

How it works

When a `deploymentId` is configured, Next.js:

1. Appends `?dpl=<deploymentId>` to static asset URLs (JavaScript, CSS, images)
2. Adds an `x-deployment-id` header to client-side navigation requests
3. Adds an `x-nextjs-deployment-id` header to navigation responses
4. Injects a `data-dpl-id` attribute on the `<html>` element
5. Includes the `deploymentId` in the `'use cache'` cache key, invalidating cache entries when the deployment ID changes

When the client detects a mismatch between its deployment ID and the server's (via the response header), it triggers a hard navigation (full page reload) instead of a client-side navigation. This ensures users always receive assets and Server Functions from a consistent deployment version.

> **Good to know:** Next.js does not read the `?dpl=` query parameter on incoming requests. The query parameter is for cache busting (ensuring browsers and CDNs fetch fresh assets), not for routing. If you need version-aware routing, consult your hosting provider or CDN's documentation for implementing deployment-based routing.

Use cases

Rolling deployments

During a rolling deployment, some server instances may be running the new version while others are still running the old version. Without a deployment ID, users might receive a mix of old and new assets, causing errors.

Setting a consistent `deploymentId` per deployment ensures:

- Clients always request assets from a matching deployment version
- Mismatches trigger a full reload to fetch the correct assets
- Server Functions work correctly across deployment boundaries

> 📘 **Nota para Pages Router:**
>
> - Clients always request assets from a matching deployment version
> - Mismatches trigger a full reload to fetch the correct assets

Multi-server environments

When running multiple instances of your Next.js application behind a load balancer, all instances for the same deployment should use the same `deploymentId`.

```
// Archivo: next.config.js
module.exports = {
  deploymentId: process.env.DEPLOYMENT_VERSION || process.env.GIT_SHA,
}
```

A per-deployment value only avoids skew if requests are also routed by deployment. Next.js does not route on `?dpl=`, so that routing comes from your host or CDN. Without it, clients that reach an instance from another deployment during a rollout will reload rather than navigate.

Version History

Version	Changes
`v16.2.0`	Pages Router detects version skew from the response header rather than the build ID, and the build ID is constant when `deploymentId` is set.
`v14.1.4`	`deploymentId` stabilized as top-level config option.
`v13.4.10`	`experimental.deploymentId` introduced.

Related

- Self-Hosting - Version Skew
- generateBuildId

---

devIndicators

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/devIndicators
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configuration options for the on-screen indicator that gives context about the current route you're viewing during development.*

`devIndicators` allows you to configure the on-screen indicator that gives context about the current route you're viewing during development.

Open `next.config.ts` and set `position` to choose where the indicator renders. The default is `bottom-left`.

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  devIndicators: {
    position: 'bottom-right', // 'bottom-left' | 'bottom-right' | 'top-left' | 'top-right'
  },
}

export default nextConfig
```

To hide the indicator entirely, set `devIndicators` to `false`. Next.js will still surface any compile or runtime errors that were encountered.

```
// Archivo: next.config.ts
const nextConfig: NextConfig = {
  devIndicators: false,
}

export default nextConfig
```

Troubleshooting

Indicator not marking a route as static

If you expect a route to be static and the indicator has marked it as dynamic, it's likely the route has opted out of prerendering.

You can confirm if a route is prerendered or dynamically rendered by building your application using `next build --debug`, and checking the output in your terminal. Static (or prerendered) routes will display a `○` symbol, whereas dynamic routes will display a `ƒ` symbol. For example:

```
// Archivo: Build Output
Route (app)
┌ ○ /_not-found
└ ƒ /products/[id]

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

There are two reasons a route might opt out of prerendering:

- The presence of Request-time APIs which rely on request information.
- An uncached data request, like a call to an ORM or database driver.

Check your route for any of these conditions, and if you are not able to statically render the route, then consider using `loading.js` or `<Suspense>` to leverage streaming.

> 📘 **Nota para Pages Router:** When exporting `getServerSideProps` or `getInitialProps` from a page, it will be marked as dynamic.

Version History

Version	Changes
`v16.0.0`	`appIsrStatus`, `buildActivity`, and `buildActivityPosition` options have been removed.
`v15.2.0`	Improved on-screen indicator with new `position` option. `appIsrStatus`, `buildActivity`, and `buildActivityPosition` options have been deprecated.
`v15.0.0`	Static on-screen indicator added with `appIsrStatus` option.

distDir

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/distDir
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Set a custom build directory to use instead of the default .next directory.*

You can specify a name to use for a custom build directory to use instead of `.next`.

Open `next.config.js` and add the `distDir` config:

```
// Archivo: next.config.js
module.exports = {
  distDir: 'build',
}
```

Now if you run `next build` Next.js will use `build` instead of the default `.next` folder.

> `distDir` **should not** leave your project directory. For example, `../build` is an **invalid** directory.

---

env

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/env
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn to add and access environment variables in your Next.js application at build time.*

> Since the release of Next.js 9.4 we now have a more intuitive and ergonomic experience for adding environment variables. Give it a try!

> 📘 **Nota para Pages Router:**
>
> > Since the release of Next.js 9.4 we now have a more intuitive and ergonomic experience for adding environment variables. Give it a try!

> **Good to know**: environment variables specified in this way will **always** be included in the JavaScript bundle, prefixing the environment variable name with `NEXT_PUBLIC_` only has an effect when specifying them through the environment or .env files.

> 📘 **Nota para Pages Router:**
>
> > **Good to know**: environment variables specified in this way will **always** be included in the JavaScript bundle, prefixing the environment variable name with `NEXT_PUBLIC_` only has an effect when specifying them through the environment or .env files.

To add environment variables to the JavaScript bundle, open `next.config.js` and add the `env` config:

```
// Archivo: next.config.js
module.exports = {
  env: {
    customKey: 'my-value',
  },
}
```

Now you can access `process.env.customKey` in your code. For example:

```
function Page() {
  return <h1>The value of customKey is: {process.env.customKey}</h1>
}

export default Page
```

Next.js will replace `process.env.customKey` with `'my-value'` at build time. Trying to destructure `process.env` variables won't work due to the nature of webpack DefinePlugin.

For example, the following line:

```
return <h1>The value of customKey is: {process.env.customKey}</h1>
```

Will end up being:

```
return <h1>The value of customKey is: {'my-value'}</h1>
```

---

expireTime

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/expireTime
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Customize stale-while-revalidate expire time for ISR enabled pages.*

You can specify a custom `stale-while-revalidate` expire time for CDNs to consume in the `Cache-Control` header for ISR enabled pages.

Open `next.config.js` and add the `expireTime` config:

```
// Archivo: next.config.js
module.exports = {
  // one hour in seconds
  expireTime: 3600,
}
```

Now when sending the `Cache-Control` header the expire time will be calculated depending on the specific revalidate period.

For example, if you have a revalidate of 15 minutes on a path and the expire time is one hour the generated `Cache-Control` header will be `s-maxage=900, stale-while-revalidate=2700` so that it can stay stale for 15 minutes less than the configured expire time.

---

exportPathMap

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/exportPathMap
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Customize the pages that will be exported as HTML files when using `next export`.*

> This feature is exclusive to `next export` and currently **deprecated** in favor of `getStaticPaths` with `pages` or `generateStaticParams` with `app`.

`exportPathMap` allows you to specify a mapping of request paths to page destinations, to be used during export. Paths defined in `exportPathMap` will also be available when using `next dev`.

Let's start with an example, to create a custom `exportPathMap` for an app with the following pages:

- `pages/index.js`
- `pages/about.js`
- `pages/post.js`

Open `next.config.js` and add the following `exportPathMap` config:

```
// Archivo: next.config.js
module.exports = {
  exportPathMap: async function (
    defaultPathMap,
    { dev, dir, outDir, distDir, buildId }
  ) {
    return {
      '/': { page: '/' },
      '/about': { page: '/about' },
      '/p/hello-nextjs': { page: '/post', query: { title: 'hello-nextjs' } },
      '/p/learn-nextjs': { page: '/post', query: { title: 'learn-nextjs' } },
      '/p/deploy-nextjs': { page: '/post', query: { title: 'deploy-nextjs' } },
    }
  },
}
```

> **Good to know**: the `query` field in `exportPathMap` cannot be used with automatically statically optimized pages or `getStaticProps` pages as they are rendered to HTML files at build-time and additional query information cannot be provided during `next export`.

The pages will then be exported as HTML files, for example, `/about` will become `/about.html`.

`exportPathMap` is an `async` function that receives 2 arguments: the first one is `defaultPathMap`, which is the default map used by Next.js. The second argument is an object with:

- `dev` - `true` when `exportPathMap` is being called in development. `false` when running `next export`. In development `exportPathMap` is used to define routes.
- `dir` - Absolute path to the project directory
- `outDir` - Absolute path to the `out/` directory (configurable with `-o`). When `dev` is `true` the value of `outDir` will be `null`.
- `distDir` - Absolute path to the `.next/` directory (configurable with the `distDir` config)
- `buildId` - The generated build id

The returned object is a map of pages where the `key` is the `pathname` and the `value` is an object that accepts the following fields:

- `page` : `String` - the page inside the `pages` directory to render
- `query` : `Object` - the `query` object passed to `getInitialProps` when prerendering. Defaults to `{}`

> The exported `pathname` can also be a filename (for example, `/readme.md`), but you may need to set the `Content-Type` header to `text/html` when serving its content if it is different than `.html`.

Adding a trailing slash

It is possible to configure Next.js to export pages as `index.html` files and require trailing slashes, `/about` becomes `/about/index.html` and is routable via `/about/`. This was the default behavior prior to Next.js 9.

To switch back and add a trailing slash, open `next.config.js` and enable the `trailingSlash` config:

```
// Archivo: next.config.js
module.exports = {
  trailingSlash: true,
}
```

Customizing the output directory

`next export` will use `out` as the default output directory, you can customize this using the `-o` argument, like so:

> 📘 **Nota para Pages Router:** `next export` will use `out` as the default output directory, you can customize this using the `-o` argument, like so:

```
// Archivo: Terminal
next export -o outdir
```

> **Warning**: Using `exportPathMap` is deprecated and is overridden by `getStaticPaths` inside `pages`. We don't recommend using them together.

---

generateBuildId

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/generateBuildId
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure the build id, which is used to identify the current build in which your application is being served.*

Next.js generates an ID during `next build` to identify which version of your application is being served. The same build should be used and boot up multiple containers.

If you are rebuilding for each stage of your environment, you will need to generate a consistent build ID to use between containers. Use the `generateBuildId` command in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  generateBuildId: async () => {
    // This could be anything, using the latest git hash
    return process.env.GIT_HASH
  },
}
```

> **Good to know:** When `deploymentId` is set, Next.js uses a constant build ID and `generateBuildId` has no effect. Version skew is detected from the deployment ID instead.

---

generateEtags

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/generateEtags
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js will generate etags for every page by default. Learn more about how to disable etag generation here.*

Next.js will generate etags for every page by default. You may want to disable etag generation for HTML pages depending on your cache strategy.

Open `next.config.js` and disable the `generateEtags` option:

```
// Archivo: next.config.js
module.exports = {
  generateEtags: false,
}
```

---

headers

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/headers
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Add custom HTTP headers to your Next.js app.*

Headers allow you to set custom HTTP headers on the response to an incoming request on a given path.

To set custom HTTP headers you can use the `headers` key in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        source: '/about',
        headers: [
          {
            key: 'x-custom-header',
            value: 'my custom header value',
          },
          {
            key: 'x-another-custom-header',
            value: 'my other custom header value',
          },
        ],
      },
    ]
  },
}
```

`headers` can be defined as a synchronous or async function. It should return, or resolve to, an array of objects with `source` and `headers` properties:

- `source` is the incoming request path pattern.
- `headers` is an array of response header objects, with `key` and `value` properties.
- `basePath`: `false` or `undefined` - if false the basePath won't be included when matching, can be used for external rewrites only.
- `locale`: `false` or `undefined` - whether the locale should not be included when matching.
- `has` is an array of has objects with the `type`, `key` and `value` properties.
- `missing` is an array of missing objects with the `type`, `key` and `value` properties.

Headers are checked before the filesystem which includes pages and `/public` files.

Header Overriding Behavior

If two headers match the same path and set the same header key, the last header key will override the first. Using the below headers, the path `/hello` will result in the header `x-hello` being `world` due to the last header value set being `world`.

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        source: '/:path*',
        headers: [
          {
            key: 'x-hello',
            value: 'there',
          },
        ],
      },
      {
        source: '/hello',
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
    ]
  },
}
```

Path Matching

Path matches are allowed, for example `/blog/:slug` will match `/blog/first-post` (no nested paths):

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        source: '/blog/:slug',
        headers: [
          {
            key: 'x-slug',
            value: ':slug', // Matched parameters can be used in the value
          },
          {
            key: 'x-slug-:slug', // Matched parameters can be used in the key
            value: 'my other custom header value',
          },
        ],
      },
    ]
  },
}
```

The pattern `/blog/:slug` matches `/blog/first-post` and `/blog/post-1` but not a nested path like `/blog/a/b`. Patterns are anchored to the start, `/blog/:slug` will not match `/archive/blog/first-post`.

You can use modifiers on parameters: `*` (zero or more), `+` (one or more), `?` (zero or one). For example, `/blog/:slug*` matches `/blog`, `/blog/a`, and `/blog/a/b/c`.

Read more details on path-to-regexp documentation.

Wildcard Path Matching

To match a wildcard path you can use `*` after a parameter, for example `/blog/:slug*` will match `/blog/a/b/c/d/hello-world`:

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        source: '/blog/:slug*',
        headers: [
          {
            key: 'x-slug',
            value: ':slug*', // Matched parameters can be used in the value
          },
          {
            key: 'x-slug-:slug*', // Matched parameters can be used in the key
            value: 'my other custom header value',
          },
        ],
      },
    ]
  },
}
```

Regex Path Matching

To match a regex path you can wrap the regex in parenthesis after a parameter, for example `/blog/:slug(\\d{1,})` will match `/blog/123` but not `/blog/abc`:

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        source: '/blog/:post(\\d{1,})',
        headers: [
          {
            key: 'x-post',
            value: ':post',
          },
        ],
      },
    ]
  },
}
```

The following characters `(`, `)`, `{`, `}`, `:`, `*`, `+`, `?` are used for regex path matching, so when used in the `source` as non-special values they must be escaped by adding `\\` before them:

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      {
        // this will match `/english(default)/something` being requested
        source: '/english\\(default\\)/:slug',
        headers: [
          {
            key: 'x-header',
            value: 'value',
          },
        ],
      },
    ]
  },
}
```

Header, Cookie, and Query Matching

To only apply a header when header, cookie, or query values also match the `has` field or don't match the `missing` field can be used. Both the `source` and all `has` items must match and all `missing` items must not match for the header to be applied.

`has` and `missing` items can have the following fields:

- `type`: `String` - must be either `header`, `cookie`, `host`, or `query`.
- `key`: `String` - the key from the selected type to match against.
- `value`: `String` or `undefined` - the value to check for, if undefined any value will match. A regex like string can be used to capture a specific part of the value, e.g. if the value `first-(?<paramName>.*)` is used for `first-second` then `second` will be usable in the destination with `:paramName`.

```
// Archivo: next.config.js
module.exports = {
  headers() {
    return [
      // if the header `x-add-header` is present,
      // the `x-another-header` header will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'header',
            key: 'x-add-header',
          },
        ],
        headers: [
          {
            key: 'x-another-header',
            value: 'hello',
          },
        ],
      },
      // if the header `x-no-header` is not present,
      // the `x-another-header` header will be applied
      {
        source: '/:path*',
        missing: [
          {
            type: 'header',
            key: 'x-no-header',
          },
        ],
        headers: [
          {
            key: 'x-another-header',
            value: 'hello',
          },
        ],
      },
      // if the source, query, and cookie are matched,
      // the `x-authorized` header will be applied
      {
        source: '/specific/:path*',
        has: [
          {
            type: 'query',
            key: 'page',
            // the page value will not be available in the
            // header key/values since value is provided and
            // doesn't use a named capture group e.g. (?<page>home)
            value: 'home',
          },
          {
            type: 'cookie',
            key: 'authorized',
            value: 'true',
          },
        ],
        headers: [
          {
            key: 'x-authorized',
            value: ':authorized',
          },
        ],
      },
      // if the header `x-authorized` is present and
      // contains a matching value, the `x-another-header` will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'header',
            key: 'x-authorized',
            value: '(?<authorized>yes|true)',
```

```
          },
        ],
        headers: [
          {
            key: 'x-another-header',
            value: ':authorized',
          },
        ],
      },
      // if the host is `example.com`,
      // this header will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'host',
            value: 'example.com',
          },
        ],
        headers: [
          {
            key: 'x-another-header',
            value: ':authorized',
          },
        ],
      },
    ]
  },
}
```

Headers with basePath support

When leveraging `basePath` support with headers each `source` is automatically prefixed with the `basePath` unless you add `basePath: false` to the header:

```
// Archivo: next.config.js
module.exports = {
  basePath: '/docs',

  headers() {
    return [
      {
        source: '/with-basePath', // becomes /docs/with-basePath
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
      {
        source: '/without-basePath', // is not modified since basePath: false is set
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
        basePath: false,
      },
    ]
  },
}
```

Headers with i18n support

When leveraging `i18n` support with headers each `source` is automatically prefixed to handle the configured `locales` unless you add `locale: false` to the header. If `locale: false` is used you must prefix the `source` with a locale for it to be matched correctly.

> 📘 **Nota para Pages Router:** When leveraging `i18n` support with headers each `source` is automatically prefixed to handle the configured `locales` unless you add `locale: false` to the header. If `locale: false` is used you must prefix the `source` with a locale for it to be matched correctly.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['en', 'fr', 'de'],
    defaultLocale: 'en',
  },

  headers() {
    return [
      {
        source: '/with-locale', // automatically handles all locales
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
      {
        // does not handle locales automatically since locale: false is set
        source: '/nl/with-locale-manual',
        locale: false,
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
      {
        // this matches '/' since `en` is the defaultLocale
        source: '/en',
        locale: false,
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
      {
        // this gets converted to /(en|fr|de)/(.*) so will not match the top-level
        // `/` or `/fr` routes like /:path* would
        source: '/(.*)',
        headers: [
          {
            key: 'x-hello',
            value: 'world',
          },
        ],
      },
    ]
  },
}
```

Cache-Control

Next.js sets the `Cache-Control` header of `public, max-age=31536000, immutable` for truly immutable assets. It cannot be overridden. These immutable files contain a SHA-hash in the file name, so they can be safely cached indefinitely. For example, Static Image Imports. You cannot set `Cache-Control` headers in `next.config.js` for these assets.

However, you can set `Cache-Control` headers for other responses or data.

Learn more about caching with the App Router.

📘 **Nota para Pages Router:** If you need to revalidate the cache of a page that has been statically generated, you can do so by setting the `revalidate` prop in the page's `getStaticProps` function.

To cache the response from an API Route, you can use `res.setHeader`:

```
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.setHeader('Cache-Control', 's-maxage=86400')
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

```
export default function handler(req, res) {
  res.setHeader('Cache-Control', 's-maxage=86400')
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

You can also use caching headers (`Cache-Control`) inside `getServerSideProps` to cache dynamic responses. For example, using `stale-while-revalidate`.

```
import { GetStaticProps, GetStaticPaths, GetServerSideProps } from 'next'

// This value is considered fresh for ten seconds (s-maxage=10).
// If a request is repeated within the next 10 seconds, the previously
// cached value will still be fresh. If the request is repeated before 59 seconds,
// the cached value will be stale but still render (stale-while-revalidate=59).
//
// In the background, a revalidation request will be made to populate the cache
// with a fresh value. If you refresh the page, you will see the new value.
export const getServerSideProps = (async (context) => {
  context.res.setHeader(
    'Cache-Control',
    'public, s-maxage=10, stale-while-revalidate=59'
  )

  return {
    props: {},
  }
}) satisfies GetServerSideProps
```

```
// This value is considered fresh for ten seconds (s-maxage=10).
// If a request is repeated within the next 10 seconds, the previously
// cached value will still be fresh. If the request is repeated before 59 seconds,
// the cached value will be stale but still render (stale-while-revalidate=59).
//
// In the background, a revalidation request will be made to populate the cache
// with a fresh value. If you refresh the page, you will see the new value.
export async function getServerSideProps({ req, res }) {
  res.setHeader(
    'Cache-Control',
    'public, s-maxage=10, stale-while-revalidate=59'
  )

  return {
    props: {},
  }
}
```

Options

CORS

Cross-Origin Resource Sharing (CORS) is a security feature that allows you to control which sites can access your resources. You can set the `Access-Control-Allow-Origin` header to allow a specific origin to access your

> 📘 **Nota para Pages Router:** API Endpoints

Route Handlers.

```
headers() {
    return [
      {
        source: "/api/:path*",
        headers: [
          {
            key: "Access-Control-Allow-Origin",
            value: "*", // Set your origin
          },
          {
            key: "Access-Control-Allow-Methods",
            value: "GET, POST, PUT, DELETE, OPTIONS",
          },
          {
            key: "Access-Control-Allow-Headers",
            value: "Content-Type, Authorization",
          },
        ],
      },
    ];
  },
```

X-DNS-Prefetch-Control

This header controls DNS prefetching, allowing browsers to proactively perform domain name resolution on external links, images, CSS, JavaScript, and more. This prefetching is performed in the background, so the DNS is more likely to be resolved by the time the referenced items are needed. This reduces latency when the user clicks a link.

```
{
  key: 'X-DNS-Prefetch-Control',
  value: 'on'
}
```

Strict-Transport-Security

This header informs browsers it should only be accessed using HTTPS, instead of using HTTP. Using the configuration below, all present and future subdomains will use HTTPS for a `max-age` of 2 years. This blocks access to pages or subdomains that can only be served over HTTP.

```
{
  key: 'Strict-Transport-Security',
  value: 'max-age=63072000; includeSubDomains; preload'
}
```

X-Frame-Options

This header indicates whether the site should be allowed to be displayed within an `iframe`. This can prevent against clickjacking attacks.

This header has been superseded by CSP's `frame-ancestors` option, which has better support in modern browsers (see Content Security Policy for configuration details).

```
{
  key: 'X-Frame-Options',
  value: 'SAMEORIGIN'
}
```

Permissions-Policy

This header allows you to control which features and APIs can be used in the browser. It was previously named `Feature-Policy`.

```
{
  key: 'Permissions-Policy',
  value: 'camera=(), microphone=(), geolocation=(), browsing-topics=()'
}
```

X-Content-Type-Options

This header prevents the browser from attempting to guess the type of content if the `Content-Type` header is not explicitly set. This can prevent XSS exploits for websites that allow users to upload and share files.

For example, a user trying to download an image, but having it treated as a different `Content-Type` like an executable, which could be malicious. This header also applies to downloading browser extensions. The only valid value for this header is `nosniff`.

```
{
  key: 'X-Content-Type-Options',
  value: 'nosniff'
}
```

Referrer-Policy

This header controls how much information the browser includes when navigating from the current website (origin) to another.

```
{
  key: 'Referrer-Policy',
  value: 'origin-when-cross-origin'
}
```

Content-Security-Policy

Learn more about adding a Content Security Policy to your application.

Version History

Version	Changes
`v13.3.0`	`missing` added.
`v10.2.0`	`has` added.
`v9.5.0`	Headers added.

htmlLimitedBots

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/htmlLimitedBots
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Specify a list of user agents that should receive blocking metadata.*

The `htmlLimitedBots` config allows you to specify a list of user agents that should receive blocking metadata instead of streaming metadata.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const config: NextConfig = {
  htmlLimitedBots: /MySpecialBot|MyAnotherSpecialBot|SimpleCrawler/,
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  htmlLimitedBots: /MySpecialBot|MyAnotherSpecialBot|SimpleCrawler/,
}
```

Default list

Next.js includes a default list of HTML limited bots, including:

- Google crawlers (e.g. Mediapartners-Google, AdsBot-Google, Google-PageRenderer)
- Bingbot
- Twitterbot
- Slackbot

See the full list here.

Specifying a `htmlLimitedBots` config will override the Next.js' default list. However, this is advanced behavior, and the default should be sufficient for most cases.

```
// Archivo: next.config.ts switcher
const config: NextConfig = {
  htmlLimitedBots: /MySpecialBot|MyAnotherSpecialBot|SimpleCrawler/,
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  htmlLimitedBots: /MySpecialBot|MyAnotherSpecialBot|SimpleCrawler/,
}
```

Disabling

To fully disable streaming metadata:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const config: NextConfig = {
  htmlLimitedBots: /.*/,
}

export default config
```

```
// Archivo: next.config.js switcher
module.exports = {
  htmlLimitedBots: /.*/,
}
```

Version History

Version	Changes
15.2.0	`htmlLimitedBots` option introduced.

httpAgentOptions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/httpAgentOptions
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js will automatically use HTTP Keep-Alive by default. Learn more about how to disable HTTP Keep-Alive here.*

In Node.js versions prior to 18, Next.js automatically polyfills `fetch()` with undici and enables HTTP Keep-Alive by default.

To disable HTTP Keep-Alive for all `fetch()` calls on the server-side, open `next.config.js` and add the `httpAgentOptions` config:

```
// Archivo: next.config.js
module.exports = {
  httpAgentOptions: {
    keepAlive: false,
  },
}
```

images

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/images
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Custom configuration for the next/image loader*

If you want to use a cloud provider to optimize images instead of using the Next.js built-in Image Optimization API, you can configure `next.config.js` with the following:

```
// Archivo: next.config.js
module.exports = {
  images: {
    loader: 'custom',
    loaderFile: './my/image/loader.js',
  },
}
```

This `loaderFile` must point to a file relative to the root of your Next.js application. The file must export a default function that returns a string, for example:

```
// Archivo: my/image/loader.js
'use client'

export default function myImageLoader({ src, width, quality }) {
  return `https://example.com/${src}?w=${width}&q=${quality || 75}`
}
```

Alternatively, you can use the `loader` prop to pass the function to each instance of `next/image`.

> **Good to know**: Customizing the image loader file, which accepts a function, requires using Client Components to serialize the provided function.

To learn more about configuring the behavior of the built-in Image Optimization API and the Image Component, see Image Configuration Options for available options.

> 📘 **Nota para Pages Router:**
>
> ```
> export default function myImageLoader({ src, width, quality }) {
>   return `https://example.com/${src}?w=${width}&q=${quality || 75}`
> }
> ```
>
> Alternatively, you can use the `loader` prop to pass the function to each instance of `next/image`.
>
> To learn more about configuring the behavior of the built-in Image Optimization API and the Image Component, see Image Configuration Options for available options.

Example Loader Configuration

- Akamai
- AWS CloudFront
- Cloudinary
- Cloudflare
- Contentful
- Fastly
- Gumlet
- ImageEngine
- Imgix
- PixelBin
- Sanity
- Sirv
- Supabase
- Thumbor
- Imagekit
- Nitrogen AIO

Akamai

```
// Docs: https://techdocs.akamai.com/ivm/reference/test-images-on-demand
export default function akamaiLoader({ src, width, quality }) {
  return `https://example.com/${src}?imwidth=${width}`
}
```

AWS CloudFront

```
// Docs: https://aws.amazon.com/developer/application-security-performance/articles/image-optimization
export default function cloudfrontLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('format', 'auto')
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Cloudinary

```
// Demo: https://res.cloudinary.com/demo/image/upload/w_300,c_limit,q_auto/turtles.jpg
export default function cloudinaryLoader({ src, width, quality }) {
  const params = ['f_auto', 'c_limit', `w_${width}`, `q_${quality || 'auto'}`]
  return `https://example.com/${params.join(',')}${src}`
}
```

Cloudflare

```
// Docs: https://developers.cloudflare.com/images/transform-images
export default function cloudflareLoader({ src, width, quality }) {
  const params = [`width=${width}`, `quality=${quality || 75}`, 'format=auto']
  return `https://example.com/cdn-cgi/image/${params.join(',')}/${src}`
}
```

Contentful

```
// Docs: https://www.contentful.com/developers/docs/references/images-api/
export default function contentfulLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('fm', 'webp')
  url.searchParams.set('w', width.toString())
  url.searchParams.set('q', (quality || 75).toString())
  return url.href
}
```

Fastly

```
// Docs: https://developer.fastly.com/reference/io/
export default function fastlyLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('auto', 'webp')
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Gumlet

```
// Docs: https://docs.gumlet.com/reference/image-transform-size
export default function gumletLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('format', 'auto')
  url.searchParams.set('w', width.toString())
  url.searchParams.set('q', (quality || 75).toString())
  return url.href
}
```

ImageEngine

```
// Docs: https://support.imageengine.io/hc/en-us/articles/360058880672-Directives
export default function imageengineLoader({ src, width, quality }) {
  const compression = 100 - (quality || 50)
  const params = [`w_${width}`, `cmpr_${compression}`]
  return `https://example.com${src}?imgeng=/${params.join('/')}`
}
```

Imgix

```
// Demo: https://static.imgix.net/daisy.png?format=auto&fit=max&w=300
export default function imgixLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  const params = url.searchParams
  params.set('auto', params.getAll('auto').join(',') || 'format')
  params.set('fit', params.get('fit') || 'max')
  params.set('w', params.get('w') || width.toString())
  params.set('q', (quality || 50).toString())
  return url.href
}
```

PixelBin

```
// Doc (Resize): https://www.pixelbin.io/docs/transformations/basic/resize/#width-w
// Doc (Optimise): https://www.pixelbin.io/docs/optimizations/quality/#image-quality-when-delivering
// Doc (Auto Format Delivery): https://www.pixelbin.io/docs/optimizations/format/#automatic-format-selection-with-f_auto-url-
parameter
export default function pixelBinLoader({ src, width, quality }) {
  const name = '<your-cloud-name>'
  const opt = `t.resize(w:${width})~t.compress(q:${quality || 75})`
  return `https://cdn.pixelbin.io/v2/${name}/${opt}/${src}?f_auto=true`
}
```

Sanity

```
// Docs: https://www.sanity.io/docs/image-urls
export default function sanityLoader({ src, width, quality }) {
  const prj = 'zp7mbokg'
  const dataset = 'production'
  const url = new URL(`https://cdn.sanity.io/images/${prj}/${dataset}${src}`)
  url.searchParams.set('auto', 'format')
  url.searchParams.set('fit', 'max')
  url.searchParams.set('w', width.toString())
  if (quality) {
    url.searchParams.set('q', quality.toString())
  }
  return url.href
}
```

Sirv

```
// Docs: https://sirv.com/help/articles/dynamic-imaging/
export default function sirvLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  const params = url.searchParams
  params.set('format', params.getAll('format').join(',') || 'optimal')
  params.set('w', params.get('w') || width.toString())
  params.set('q', (quality || 85).toString())
  return url.href
}
```

Supabase

```
// Docs: https://supabase.com/docs/guides/storage/image-transformations#nextjs-loader
export default function supabaseLoader({ src, width, quality }) {
  const url = new URL(`https://example.com${src}`)
  url.searchParams.set('width', width.toString())
  url.searchParams.set('quality', (quality || 75).toString())
  return url.href
}
```

Thumbor

```
// Docs: https://thumbor.readthedocs.io/en/latest/
export default function thumborLoader({ src, width, quality }) {
  const params = [`${width}x0`, `filters:quality(${quality || 75})`]
  return `https://example.com${params.join('/')}${src}`
}
```

ImageKit.io

```
// Docs: https://imagekit.io/docs/image-transformation
export default function imageKitLoader({ src, width, quality }) {
  const params = [`w-${width}`, `q-${quality || 80}`]
  return `https://ik.imagekit.io/your_imagekit_id/${src}?tr=${params.join(',')}`
}
```

Nitrogen AIO

```
// Docs: https://docs.n7.io/aio/intergrations/
export default function aioLoader({ src, width, quality }) {
  const url = new URL(src, window.location.href)
  const params = url.searchParams
  const aioParams = params.getAll('aio')
  aioParams.push(`w-${width}`)
  if (quality) {
    aioParams.push(`q-${quality.toString()}`)
  }
  params.set('aio', aioParams.join(';'))
  return url.href
}
```

Custom Next.js Cache Handler

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/incrementalCacheHandlerPath
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure the Next.js cache used for storing and revalidating data to use any external service like Redis, Memcached, or others.*

You can configure the Next.js cache location if you want to persist cached pages and data to durable storage, or share the cache across multiple containers or instances of your Next.js application.

> **Good to know**: The `cacheHandler` (singular) configuration is specifically used by Next.js for server cache operations such as storing and revalidating ISR, route handler responses, and optimized images. It is **not** used by `'use cache'` directives. For `'use cache'` directives, use `cacheHandlers` (plural) instead. `cacheMaxMemorySize` is separate from both, and sizes the in-memory cache for each.

```
// Archivo: next.config.js
module.exports = {
  cacheHandler: require.resolve('./cache-handler.js'),
  cacheMaxMemorySize: 0, // disable default in-memory caching
}
```

View an example of a custom cache handler and learn more about the implementation.

API Reference

The cache handler can implement the following methods: `get`, `set`, `revalidateTag`, and `resetRequestCache`.

`get()`

Parameter	Type	Description
`key`	`string`	The key to the cached value.
`ctx`	`object`	Context including the cache entry kind.

The `ctx` parameter contains a `kind` property that indicates the type of cache entry being retrieved. Possible values include `'APP_PAGE'`, `'APP_ROUTE'`, `'PAGES'`, `'FETCH'`, and `'IMAGE'`.

Returns the cached value or `null` if not found.

`set()`

Parameter	Type	Description
`key`	`string`	The key to store the data under.
`data`	Data or `null`	The data to be cached.
`ctx`	`{ tags: [] }`	The cache tags provided.

The `data` object contains a `kind` property that indicates the type of cache entry. For image optimization, `kind` will be `'IMAGE'` and the data will include properties like `buffer`, `etag`, `extension`, and `revalidate`.

Returns `Promise<void>`.

`revalidateTag()`

Parameter	Type	Description
`tag`	`string` or `string[]`	The cache tags to revalidate.

Returns `Promise<void>`. Learn more about revalidating data or the `revalidateTag()` function.

`resetRequestCache()`

This method resets the temporary in-memory cache for a single request before the next request.

Returns `void`.

Good to know:

- `revalidatePath` is a convenience layer on top of cache tags. Calling `revalidatePath` will call your `revalidateTag` function, which you can then choose if you want to tag cache keys based on the path.

Image Optimization Caching

The `cacheHandler` can also be used for caching optimized images from `next/image`. To enable this, set `images.customCacheHandler` to `true` in your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  cacheHandler: require.resolve('./cache-handler.js'),
  images: {
    customCacheHandler: true,
  },
}
```

> **Good to know**: This opt-in flag will become the default behavior in the next major version. Setting it now allows you to prepare your cache handler for image optimization entries.

You can use the `kind` property to differentiate between cache entry types and handle images separately if needed, for example to implement an eviction policy or store images in a different location.

When handling image cache entries, the `kind` will be `'IMAGE'` and the data will include `buffer`, `etag`, `extension`, and `revalidate` properties.

Platform Support

Deployment Option	Supported
Node.js server	Yes
Docker container	Yes
Static export	No
Adapters	Platform-specific

Learn how to configure ISR when self-hosting Next.js.

Version History

Version	Changes
`v16.2.0`	`cacheHandler` support for image optimization caching.
`v14.1.0`	Renamed to `cacheHandler` and became stable.
`v13.4.0`	`incrementalCacheHandlerPath` support for `revalidateTag`.
`v13.4.0`	`incrementalCacheHandlerPath` support for standalone output.
`v12.2.0`	Experimental `incrementalCacheHandlerPath` added.

next.config.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to configure your application with next.config.js.*

Next.js can be configured through a `next.config.js` file in the root of your project directory (for example, by `package.json`) with a default export.

```
// Archivo: next.config.js
// @ts-check

/** @type {import('next').NextConfig} */
const nextConfig =

module.exports = nextConfig
```

ECMAScript Modules

`next.config.js` is a regular Node.js module, not a JSON file. It gets used by the Next.js server and build phases, and it's not included in the browser build.

If you need ECMAScript modules, you can use `next.config.mjs`:

```
// Archivo: next.config.mjs
// @ts-check

/**
 * @type {import('next').NextConfig}
 */
const nextConfig =

export default nextConfig
```

> **Good to know**: `next.config` with the `.cjs` or `.cts` extensions are currently **not** supported.

Configuration as a Function

You can also use a function:

```
// Archivo: next.config.mjs
// @ts-check

export default (phase, { defaultConfig }) =>
  return nextConfig
}
```

Async Configuration

Since Next.js 12.1.0, you can use an async function:

```
// Archivo: next.config.js
// @ts-check

module.exports = async (phase, { defaultConfig }) =>
  return nextConfig
}
```

Phase

`phase` is the current context in which the configuration is loaded. You can see the available phases. Phases can be imported from `next/constants`:

```
// Archivo: next.config.js
// @ts-check

const { PHASE_DEVELOPMENT_SERVER } = require('next/constants')

module.exports = (phase, { defaultConfig }) => {
  if (phase === PHASE_DEVELOPMENT_SERVER) {
    return
  }

  return
}
```

TypeScript

If you are using TypeScript in your project, you can use `next.config.ts` to use TypeScript in your configuration:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig =

export default nextConfig
```

The commented lines are the place where you can put the configs allowed by `next.config.js`, which are defined in this file.

However, none of the configs are required, and it's not necessary to understand what each config does. Instead, search for the features you need to enable or modify in this section and they will show you what to do.

> Avoid using new JavaScript features not available in your target Node.js version. `next.config.js` will not be parsed by Webpack or Babel.

This page documents all the available configuration options:

Unit Testing (experimental)

Starting in Next.js 15.1, the `next/experimental/testing/server` package contains utilities to help unit test `next.config.js` files.

The `unstable_getResponseFromNextConfig` function runs the `headers`, `redirects`, and `rewrites` functions from `next.config.js` with the provided request information and returns `NextResponse` with the results of the routing.

> The response from `unstable_getResponseFromNextConfig` only considers `next.config.js` fields and does not consider proxy or filesystem routes, so the result in production may be different than the unit test.

```
import {
  getRedirectUrl,
  unstable_getResponseFromNextConfig,
} from 'next/experimental/testing/server'

const response = await unstable_getResponseFromNextConfig({
  url: 'https://nextjs.org/test',
  nextConfig: {
    async redirects() {
      return [{ source: '/test', destination: '/test2', permanent: false }]
    },
  },
})
expect(response.status).toEqual(307)
expect(getRedirectUrl(response)).toEqual('https://nextjs.org/test2')
```

inlineCss

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/inlineCss
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable inline CSS support.*

Usage

Experimental support for inlining CSS in the `<head>`. When this flag is enabled, all places where we normally generate a `<link>` tag will instead have a generated `<style>` tag.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    inlineCss: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    inlineCss: true,
  },
}

module.exports = nextConfig
```

Trade-Offs

- **Enable** if you use atomic CSS (like Tailwind) and want to optimize first-load performance for new visitors
- **Skip** if returning visitors are common and you want them to benefit from cached stylesheets

When Inline CSS Helps

Normally, the browser must download HTML, parse it, discover CSS `<link>` tags, then request stylesheets before it can render. Inlining eliminates this request waterfall, so that styles arrive with the HTML, so the browser can render immediately.

This benefit is strongest with:

- **First-time visitors**: Since CSS files are render-blocking, inlining eliminates the initial download delay that first-time visitors experience. Returning visitors with cached stylesheets won't see this benefit.
- **Performance metrics**: By removing additional network requests for CSS files, inlining can significantly improve First Contentful Paint (FCP) and Largest Contentful Paint (LCP).
- **Slow connections**: For users on high-latency networks, each additional request adds delay. Inlining reduces round trips, which matters most when connections are slow.
- **Atomic CSS (Tailwind)**: Utility-first frameworks generate only the classes you use, keeping CSS small. The styles for a page don't grow proportionally with page complexity—they're typically compact regardless of how much UI you build. This makes inlining practical since you get the performance benefit without significantly bloating HTML.

When External CSS is Better

Inlined styles cannot be cached separately from HTML. Every page load re-downloads the same CSS.

This trade-off matters most with:

- **Returning visitors**: Users who visit your site repeatedly would benefit from cached external stylesheets. With inlining, they re-download styles on every visit.
- **Large CSS bundles**: External stylesheets cache independently and load efficiently on modern infrastructure. Inlined CSS arrives with every HTML response, increasing Time to First Byte (TTFB) and preventing browsers from caching styles separately. This trade-off works for small CSS (atomic frameworks like Tailwind), but adds overhead for larger bundles (component libraries like Bootstrap or Material UI).
- **Many pages sharing styles**: External stylesheets cached on one page speed up navigation to other pages. Inlined styles provide no cross-page caching benefit.

> **Good to know**:
>
> This feature is currently experimental and has some known limitations:
>
> - CSS inlining is applied globally and cannot be configured on a per-page basis
> - Styles are duplicated during initial page load - once within `<style>` tags for SSR and once in the RSC payload
> - When navigating to prerendered pages, styles will use `<link>` tags instead of inline CSS to avoid duplication
> - This feature is not available in development mode and only works in production builds

---

instrumentationClientInject

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/instrumentationClientInject
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Inject additional client-side instrumentation modules before the user's* `instrumentation-client.{js,ts}` *file.*

`instrumentationClientInject` is a list of modules that are imported on the client for their side effects before the user's `instrumentation-client.{js,ts}` file runs, ahead of React hydration.

This option is primarily intended for `next.config.js` **plugins** (for example, wrappers like `withSentry` or `withAnalytics` that extend a project's config). It lets such a plugin inject its own client instrumentation module — including a navigation hook — without requiring every project to author or modify an `instrumentation-client` file. Application code should generally continue to use the `instrumentation-client.{js,ts}` file convention directly.

A plugin typically appends its own module to whatever the project already has configured:

```
// Archivo: withMyInstrumentation.js
module.exports = function withMyInstrumentation(nextConfig = {}) {
  return {
    ... nextConfig,
    instrumentationClientInject: [
      ... (nextConfig.instrumentationClientInject ?? []),
      'my-instrumentation-package/client',
    ],
  }
}
```

It can also be set directly:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
module.exports = {
  instrumentationClientInject: [
    'my-analytics-package',
    './lib/sentry-client.js',
  ],
}
```

Each entry is one of:

- A bare npm package name, resolved from the project's `node_modules`.
- A path relative to the project root.

Execution order

Modules run on the client in this order:

1. Each entry in `instrumentationClientInject`, in array order.
2. The project's `instrumentation-client.{js,ts}` file, if present.
3. React hydration.

Router navigation hook

Each injected module may optionally export an `onRouterTransitionStart` function with the same signature as the one documented for the `instrumentation-client` file convention. Next.js composes a single hook that fans out to every exported `onRouterTransitionStart` on each navigation, calling them in array order, with the user file's hook running last.

```
// Archivo: lib/sentry-client.js
// Side-effectful setup runs at load time.
setupSentry()

export function onRouterTransitionStart(url, navigationType) {
  recordNavigationBreadcrumb(url, navigationType)
}
```

Modules that do not export `onRouterTransitionStart` are skipped during navigation.

Version history

Version	Changes
`v16.3.0`	`instrumentationClientInject` introduced

logging

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/logging
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure logging behavior in the terminal when running Next.js in development mode, including fetch logging, incoming requests, and forwarding browser console logs to the terminal.*

Options

Fetching

You can configure the logging level and whether the full URL is logged to the console when running Next.js in development mode.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    fetches: {
      fullUrl: true,
    },
  },
}
```

Any `fetch` requests that are restored from the Server Components HMR cache are not logged by default. However, this can be enabled by setting `logging.fetches.hmrRefreshes` to `true`.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    fetches: {
      hmrRefreshes: true,
    },
  },
}
```

Server Functions

Server Function invocations are logged by default during development. You can disable this by setting `logging.serverFunctions` to `false`.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    serverFunctions: false,
  },
}
```

When enabled, the terminal displays each Server Function call with its function name, arguments, and duration:

```
// Archivo: Terminal
POST /
  └ ƒ myAction(arg1, arg2) in 5ms app/actions.ts
```

Incoming Requests

By default all the incoming requests will be logged in the console during development. You can use the `incomingRequests` option to decide which requests to ignore. Since this is only logged in development, this option doesn't affect production builds.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    incomingRequests: {
      ignore: [/\api\/v1\/health/],
    },
  },
}
```

Or you can disable incoming request logging by setting `incomingRequests` to `false`.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    incomingRequests: false,
  },
}
```

Browser Console Logs

You can forward browser console logs (such as `console.log`, `console.warn`, `console.error`) to the terminal during development. This is useful for debugging client-side code without needing to check the browser's developer tools.

```
// Archivo: next.config.js
module.exports = {
  logging: {
    browserToTerminal: true,
  },
}
```

Options

The `browserToTerminal` option accepts the following values:

Value	Description
`'warn'`	Forward only warnings and errors, by default
`'error'`	Forward only errors
`true`	Forward all console output (log, info, warn, error)
`false`	Disable browser log forwarding

```
// Archivo: next.config.js
module.exports = {
  logging: {
    browserToTerminal: 'warn',
  },
}
```

Source Location

When enabled, browser logs include source location information (file path and line number) by default. For example:

```
// Archivo: app/page.tsx highlight={8}
'use client'

export default function Home() {
  return (
    <button
      type="button"
      onClick={() => {
        console.log('Hello World')
      }}
    >
      Click me
    </button>
  )
}
```

Clicking the button prints this message to the terminal:

```
// Archivo: Terminal
[browser] Hello World (app/page.tsx:8:17)
```

📘 **Nota para Pages Router:**

```
export default function Home() {
  return (
    <button
      type="button"
      onClick={() => {
        console.log('Hello World')
      }}
    >
      Click me
    </button>
  )
}
```

Clicking the button prints this message to the terminal:

```
[browser] Hello World (pages/index.tsx:6:17)
```

Disabling Logging

In addition, you can disable the development logging by setting `logging` to `false`.

```
// Archivo: next.config.js
module.exports = {
  logging: false,
}
```

Version History

Version	Changes
`v16.2.0`	`browserToTerminal` added (moved from `experimental.browserDebugInfoInTerminal`)
`v15.4.0`	`experimental.browserDebugInfoInTerminal` introduced
`v15.2.0`	`incomingRequests` added
`v15.0.0`	`logging: false` option added, `fetches.hmrRefreshes` added for App Router
`v14.0.0`	`logging.fetches` moved to stable for App Router

mdxRs

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/mdxRs
📁 **Sección:** App > Api Reference > Config > Next Config Js
📝 **Descripción:** *Use the new Rust compiler to compile MDX files in the App Router.*

For experimental use with `@next/mdx`. Compiles MDX files using the new Rust compiler.

```
// Archivo: next.config.js
const withMDX = require('@next/mdx')()

/** @type {import('next').NextConfig} */
const nextConfig = {
  pageExtensions: ['ts', 'tsx', 'mdx'],
  experimental: {
    mdxRs: true,
  },
}

module.exports = withMDX(nextConfig)
```

onDemandEntries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/onDemandEntries
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure how Next.js will dispose and keep in memory pages created in development.*

Next.js exposes some options that give you some control over how the server will dispose or keep in memory built pages in development.

To change the defaults, open `next.config.js` and add the `onDemandEntries` config:

```
// Archivo: next.config.js
module.exports = {
  onDemandEntries: {
    // period (in ms) where the server will keep pages in the buffer
    maxInactiveAge: 25 * 1000,
    // number of pages that should be kept simultaneously without being disposed
    pagesBufferLength: 2,
  },
}
```

optimizePackageImports

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/optimizePackageImports
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *API Reference for optimizePackageImports Next.js Config Option*

Some packages can export hundreds or thousands of modules, which can cause performance issues in development and production.

Adding a package to `experimental.optimizePackageImports` will only load the modules you are actually using, while still giving you the convenience of writing import statements with many named exports.

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    optimizePackageImports: ['package-name'],
  },
}
```

The following libraries are optimized by default:

- `lucide-react`
- `date-fns`
- `lodash-es`
- `ramda`
- `antd`
- `react-bootstrap`
- `ahooks`
- `@ant-design/icons`
- `@headlessui/react`
- `@headlessui-float/react`
- `@heroicons/react/20/solid`
- `@heroicons/react/24/solid`
- `@heroicons/react/24/outline`
- `@visx/visx`
- `@tremor/react`
- `rxjs`
- `@mui/material`
- `@mui/icons-material`
- `recharts`
- `react-use`
- `@material-ui/core`
- `@material-ui/icons`
- `@tabler/icons-react`
- `mui-core`
- `react-icons/*`
- `effect`
- `@effect/*`

---

output

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/output
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js automatically traces which files are needed by each page to allow for easy deployment of your application. Learn how it works here.*

During a build, Next.js will automatically trace each page and its dependencies to determine all of the files that are needed for deploying a production version of your application.

This feature helps reduce the size of deployments drastically. Previously, when deploying with Docker you would need to have all files from your package's `dependencies` installed to run `next start`. Starting with Next.js 12, you can leverage Output File Tracing in the `.next/` directory to only include the necessary files.

Furthermore, this removes the need for the deprecated `serverless` target which can cause various issues and also creates unnecessary duplication.

How it Works

During `next build`, Next.js will use `@vercel/nft` to statically analyze `import`, `require`, and `fs` usage to determine all files that a page might load.

Next.js' production server is also traced for its needed files and output at `.next/next-server.js.nft.json` which can be leveraged in production.

To leverage the `.nft.json` files emitted to the `.next` output directory, you can read the list of files in each trace that are relative to the `.nft.json` file and then copy them to your deployment location.

Automatically Copying Traced Files

Next.js can automatically create a `standalone` folder that copies only the necessary files for a production deployment including select files in `node_modules`.

To leverage this automatic copying you can enable it in your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  output: 'standalone',
}
```

This will create a folder at `.next/standalone` which can then be deployed on its own without installing `node_modules`.

Additionally, a minimal `server.js` file is also output which can be used instead of `next start`. This minimal server does not copy the `public` or `.next/static` folders by default as these should ideally be handled by a CDN instead, although these folders can be copied to the `standalone/public` and `standalone/.next/static` folders manually, after which `server.js` file will serve these automatically.

To copy these manually, you can use the `cp` command-line tool after you `next build`:

```
// Archivo: Terminal
cp -r public .next/standalone/ && cp -r .next/static .next/standalone/.next/
```

To start your minimal `server.js` file locally, run the following command:

```
// Archivo: Terminal
node .next/standalone/server.js
```

> **Good to know**:
>
> - If your project needs to listen to a specific port or hostname, you can define `PORT` or `HOSTNAME` environment variables before running `server.js`. For example, run `PORT=8080 HOSTNAME=0.0.0.0 node server.js` to start the server on `http://0.0.0.0:8080`.

> 📘 **Nota para Pages Router:**
>
> > **Good to know**:
> >
> > - `next.config.js` is read during `next build` and serialized into the `server.js` output file.
> > - If your project needs to listen to a specific port or hostname, you can define `PORT` or `HOSTNAME` environment variables before running `server.js`. For example, run `PORT=8080 HOSTNAME=0.0.0.0 node server.js` to start the server on `http://0.0.0.0:8080`.

Caveats

- While tracing in monorepo setups, the project directory is used for tracing by default. For `next build packages/web-app`, `packages/web-app` would be the tracing root and any files outside of that folder will not be included. To include files outside of this folder you can set `outputFileTracingRoot` in your `next.config.js`.

```
// Archivo: packages/web-app/next.config.js
const path = require('path')

module.exports = {
  // this includes files from the monorepo base two directories up
  outputFileTracingRoot: path.join(__dirname, '../../'),
}
```

- There are some cases in which Next.js might fail to include required files, or might incorrectly include unused files. In those cases, you can leverage `outputFileTracingExcludes` and `outputFileTracingIncludes` respectively in `next.config.js`. Each option accepts an object whose keys are **route globs** (matched with picomatch against the route path, e.g. `/api/hello`) and whose values are **glob patterns resolved from the project root** that specify files to include or exclude in the trace.

> **Good to know**: In a monorepo, `project root` refers to the Next.js project root (the folder containing next.config.js, e.g., packages/web-app), not necessarily the monorepo root.

```
// Archivo: next.config.js
module.exports = {
  outputFileTracingExcludes: {
    '/api/hello': ['./un-necessary-folder/**/*'],
  },
  outputFileTracingIncludes: {
    '/api/another': ['./necessary-folder/**/*'],
    '/api/login/\\[\\[\\.\\.\\.slug\\]\\]': [
      './node_modules/aws-crt/dist/bin/**/*',
    ],
  },
}
```

Using a `src/` directory does not change how you write these options:

- **Keys** still match the route path (`'/api/hello'`, `'/products/[id]'`, etc.).
- **Values** can reference paths under `src/` since they are resolved relative to the project root.

```
// Archivo: next.config.js
module.exports = {
  outputFileTracingIncludes: {
    '/products/*': ['src/lib/payments/**/*'],
    '/*': ['src/config/runtime/**/*.json'],
  },
  outputFileTracingExcludes: {
    '/api/*': ['src/temp/**/*', 'public/large-logs/**/*'],
  },
}
```

You can also target all routes using a global key like `'/*'`:

```
// Archivo: next.config.js
module.exports = {
  outputFileTracingIncludes: {
    '/*': ['src/i18n/locales/**/*.json'],
  },
}
```

These options are applied to server traces and do not affect routes that do not produce a server trace file:

- Edge Runtime routes are not affected.

- Fully static pages are not affected.

In monorepos or when you need to include files outside the app folder, combine `outputFileTracingRoot` with includes:

```
// Archivo: next.config.js
const path = require('path')

module.exports = {
  // Trace from the monorepo root
  outputFileTracingRoot: path.join(__dirname, '../../'),
  outputFileTracingIncludes: {
    '/route1': ['../shared/assets/**/*'],
  },
}
```

> **Good to know**:
>
> - Prefer forward slashes (`/`) in patterns for cross-platform compatibility.
> - Keep patterns as narrow as possible to avoid oversized traces (avoid `**/*` at the repo root).

Common include patterns for native/runtime assets:

```
// Archivo: next.config.js
module.exports = {
  outputFileTracingIncludes: {
    '/*': ['node_modules/sharp/**/*', 'node_modules/aws-crt/dist/bin/**/*'],
  },
}
```

---

outputHashSalt

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/outputHashSalt
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to incorporate a custom salt string into content-addressed output filenames.*

`outputHashSalt` is an option that incorporates a configurable salt string into every content-addressed output filename (chunks, assets). Changing this value forces all output hashes to change, which is useful for invalidating cached assets across deployments without modifying source files.

To configure the output hash salt, set `outputHashSalt` in `next.config.js`:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  outputHashSalt: 'my-deployment-salt',
}

module.exports = nextConfig
```

This works with both Webpack and Turbopack bundlers.

The `NEXT_HASH_SALT` environment variable can also be used for the same purpose. When both are set, the values are **concatenated** (`outputHashSalt + NEXT_HASH_SALT`) to form the effective salt. This lets you combine a per-project salt baked into the config with a per-deployment salt injected at build time via environment variable.

```
// Archivo: Terminal
NEXT_HASH_SALT=my-deployment-salt next build
```

Version History

Version	Changes
`16.3.0`	`outputHashSalt` was added.

pageExtensions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/pageExtensions
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Extend the default page extensions used by Next.js when resolving pages in the Pages Router.*

By default, Next.js accepts files with the following extensions: `.tsx` , `.ts` , `.jsx` , `.js` . This can be modified to allow other extensions like markdown (`.md` , `.mdx`).

```
// Archivo: next.config.js
const withMDX = require('@next/mdx')()

/** @type {import('next').NextConfig} */
const nextConfig = {
  pageExtensions: ['js', 'jsx', 'ts', 'tsx', 'md', 'mdx'],
}

module.exports = withMDX(nextConfig)
```

📘 **Nota para Pages Router:** You can extend the default Page extensions (`.tsx` , `.ts` , `.jsx` , `.js`) used by Next.js. Inside `next.config.js` , add the `pageExtensions` config:

```
module.exports = {
  pageExtensions: ['mdx', 'md', 'jsx', 'js', 'tsx', 'ts'],
}
```

Changing these values affects *all* Next.js pages, including the following:

- `proxy.js`
- `instrumentation.js`
- `pages/_document.js`
- `pages/_app.js`
- `pages/api/`

For example, if you reconfigure `.ts` page extensions to `.page.ts` , you would need to rename pages like `proxy.page.ts` , `instrumentation.page.ts` , `_app.page.ts` .

Including non-page files in the `pages` directory

You can colocate test files or other files used by components in the `pages` directory. Inside `next.config.js` , add the `pageExtensions` config:

```
module.exports = {
  pageExtensions: ['page.tsx', 'page.ts', 'page.jsx', 'page.js'],
}
```

Then, rename your pages to have a file extension that includes `.page` (e.g. rename `MyPage.tsx` to `MyPage.page.tsx`). Ensure you rename *all* Next.js pages, including the files mentioned above.

---

partialPrefetching

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/partialPrefetching
📂 **Sección:** App > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure the default link prefetch behavior to fetch only the static parts of each route.*

`partialPrefetching` enables Partial Prefetching at the app level. The framework prefetches the static parts of each route by default; set `prefetch={true}` on individual links to use per-link prefetching and fetch more.

Usage

```
// Archivo: next.config.ts highlight={5} switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,
  partialPrefetching: true,
}

export default nextConfig
```

```
// Archivo: next.config.js highlight={3} switcher
module.exports = {
  cacheComponents: true,
  partialPrefetching: true,
}
```

`partialPrefetching` requires `cacheComponents`. Without it, `next dev` and `next build` throw at config validation.

Reference

Value	Description
`true`	Enables Partial Prefetching across the app.
`false`	Default. No change to prefetch behavior.

How prefetches resolve

Before Partial Prefetching, Next.js prefetched per visible link: a page with N links to N routes produced ~N route prefetches as those links entered the viewport.

With `partialPrefetching: true`, Next.js prefetches one reusable App Shell per route instead. The App Shell contains rendered output that does not depend on a link's URL. URL-specific content, including content that depends on `params` or `searchParams`, resolves after navigation by default. App Shells are cached on the client, so links to the same route reuse one prefetch.

The pattern is similar to per-route code splitting in single-page apps: one artifact per route, shared by every link that points to it.

> **Good to know**: Routes that read `cookies()` or `headers()` produce an App Shell that includes session data. The framework auto-detects this and caches the shell per session on the client.

A link can ask for more than the App Shell with `<Link prefetch={true}>`. The prefetch also resolves URL data like `params`, `searchParams`, and the full URL, and the cached content behind it. See Optimizing prefetching.

> **Good to know**: If you use `<Link prefetch={true}>` to a route that hasn't opted into Partial Prefetching, a dev console error suggests enabling `partialPrefetching` app-wide or `prefetch = 'partial'` on the segment. The dev warning Insight covers each fix in detail.

Per-segment overrides

A segment that exports an explicit `prefetch` value overrides the app-level default for that route.

Version History

Version	Change
16.3.0	`partialPrefetching` introduced. Requires `cacheComponents` to be enabled.

poweredByHeader

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/poweredByHeader
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js will add the `x-powered-by` header by default. Learn to opt-out of it here.*

By default Next.js will add the `x-powered-by` header. To opt-out of it, open `next.config.js` and disable the `poweredByHeader` config:

```
// Archivo: next.config.js
module.exports = {
  poweredByHeader: false,
}
```

---

prefetchInlining

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/prefetchInlining
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Override how the App Router bundles small prefetch responses together.*

When the App Router prefetches a route, it can bundle small segment responses into a single response instead of requesting each one separately. This reduces the number of prefetch requests at the cost of duplicating some shared segment data across routes. This behavior is on by default, and most apps should leave it that way.

The `experimental.prefetchInlining` option lets you override this behavior or disable inlining while debugging navigation issues or measuring request volume. For most applications, there is no need to change the default behavior.

> **Good to know**: The inlining behavior is a permanent part of the App Router. Only the `experimental.prefetchInlining` configuration is experimental, so its options may still change.

Usage

To turn off prefetch inlining, set `experimental.prefetchInlining` to `false`:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    prefetchInlining: false,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    prefetchInlining: false,
  },
}

module.exports = nextConfig
```

To override the thresholds instead of disabling inlining, pass an object. Any value you omit keeps its default:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    prefetchInlining: {
      maxSize: 2048,
      maxBundleSize: 10240,
    },
  },
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    prefetchInlining: {
      maxSize: 2048,
      maxBundleSize: 10240,
    },
  },
}

module.exports = nextConfig
```

Reference

Value	Description
`true`	Inlines prefetch responses with the default thresholds. This is the default.
`false`	Disables prefetch inlining. Each segment is prefetched as its own request.
`object`	Inlines prefetch responses using the `maxSize` or `maxBundleSize` you set.

When you pass an object, the following options control the thresholds. Both are measured in bytes of the gzip-compressed segment response:

Option	Type	Default	Description
`maxSize`	`number`	`2048`	Largest a single segment response can be to still be eligible for inlining.
`maxBundleSize`	`number`	`10240`	Largest total size that can be inlined into one bundled prefetch response along a path.

Lower thresholds keep more per-segment deduplication; higher thresholds inline more data and cut request count further.

Version History

Version	Change
16.3.0	`experimental.prefetchInlining` enabled by default.
16.2.0	`experimental.prefetchInlining` added.

productionBrowserSourceMaps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/productionBrowserSourceMaps
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enables browser source map generation during the production build.*

Source Maps are enabled by default during development. During production builds, they are disabled to prevent you leaking your source on the client, unless you specifically opt-in with the configuration flag.

Next.js provides a configuration flag you can use to enable browser source map generation during the production build:

```
// Archivo: next.config.js
module.exports = {
  productionBrowserSourceMaps: true,
}
```

When the `productionBrowserSourceMaps` option is enabled, the source maps will be output in the same directory as the JavaScript files. Next.js will automatically serve these files when requested.

- Adding source maps can increase `next build` time
- Increases memory usage during `next build`

---

proxyClientMaxBodySize

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/proxyClientMaxBodySize
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure the maximum request body size when using proxy.*

When proxy is used, Next.js automatically clones the request body and buffers it in memory to enable multiple reads - both in proxy and the underlying route handler. To prevent excessive memory usage, this configuration option sets a size limit on the buffered body.

By default, the maximum body size is **10MB**. If a request body exceeds this limit, the body will only be buffered up to the limit, and a warning will be logged indicating which route exceeded the limit.

Options

String format (recommended)

Specify the size using a human-readable string format:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    proxyClientMaxBodySize: '1mb',
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    proxyClientMaxBodySize: '1mb',
  },
}

module.exports = nextConfig
```

Supported units: `b`, `kb`, `mb`, `gb`

Number format

Alternatively, specify the size in bytes as a number:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    proxyClientMaxBodySize: 1048576, // 1MB in bytes
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    proxyClientMaxBodySize: 1048576, // 1MB in bytes
  },
}

module.exports = nextConfig
```

Behavior

When a request body exceeds the configured limit:

1. Next.js will buffer only the first N bytes (up to the limit)
2. A warning will be logged to the console indicating the route that exceeded the limit
3. The request will continue processing normally, but only the partial body will be available
4. The request will **not** fail or return an error to the client

If your application needs to process the full request body, you should either:

- Increase the `proxyClientMaxBodySize` limit
- Handle the partial body gracefully in your application logic

Example

```
// Archivo: proxy.ts
import { NextRequest, NextResponse } from 'next/server'

export async function proxy(request: NextRequest) {
  // Next.js automatically buffers the body with the configured size limit
  // You can read the body in proxy...
  const body = await request.text()

  // If the body exceeded the limit, only partial data will be available
  console.log('Body size:', body.length)

  return NextResponse.next()
}
```

```
// Archivo: app/api/upload/route.ts
import { NextRequest, NextResponse } from 'next/server'

export async function POST(request: NextRequest) {
  // ...and the body is still available in your route handler
  const body = await request.text()

  console.log('Body in route handler:', body.length)

  return NextResponse.json({ received: body.length })
}
```

Good to know

- This setting only applies when proxy is used in your application
- The default limit of 10MB is designed to balance memory usage and typical use cases
- The limit applies per-request, not globally across all concurrent requests
- For applications handling large file uploads, consider increasing the limit accordingly

---

reactCompiler

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/reactCompiler
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable the React Compiler to automatically optimize component rendering.*

Next.js includes support for the React Compiler, a tool designed to improve performance by automatically optimizing component rendering. This reduces the need for manual memoization using `useMemo` and `useCallback`.

Next.js includes a custom performance optimization written in SWC that makes the React Compiler more efficient. Instead of running the compiler on every file, Next.js analyzes your project and only applies the React Compiler to relevant files. This avoids unnecessary work and leads to faster builds compared to using the Babel plugin on its own.

How It Works

The React Compiler runs through a Babel plugin. To keep builds fast, Next.js uses a custom SWC optimization that only applies the React Compiler to relevant files—like those with JSX or React Hooks.

This avoids compiling everything and keeps the performance cost minimal. You may still see slightly slower builds compared to the default Rust-based compiler, but the impact is small and localized.

To use it, install the `babel-plugin-react-compiler`:

```
# Manejador de paquetes: pnpm
pnpm add -D babel-plugin-react-compiler
```

```
# Manejador de paquetes: npm
npm install -D babel-plugin-react-compiler
```

```
# Manejador de paquetes: yarn
yarn add -D babel-plugin-react-compiler
```

```
# Manejador de paquetes: bun
bun add -D babel-plugin-react-compiler
```

Then, add `reactCompiler` option in `next.config.js`:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  reactCompiler: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactCompiler: true,
}

module.exports = nextConfig
```

Annotations

You can configure the compiler to run in "opt-in" mode as follows:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  reactCompiler: {
    compilationMode: 'annotation',
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactCompiler: {
    compilationMode: 'annotation',
  },
}

module.exports = nextConfig
```

Then, you can annotate specific components or hooks with the `"use memo"` directive from React to opt-in:

```
// Archivo: app/page.tsx switcher
export default function Page() {
  'use memo'
  // ...
}
```

```
// Archivo: app/page.js switcher
export default function Page() {
  'use memo'
  // ...
}
```

> **Note:** You can also use the `"use no memo"` directive from React for the opposite effect, to opt-out a component or hook.

---

reactMaxHeadersLength

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/reactMaxHeadersLength
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *The maximum length of the headers that are emitted by React and added to the response.*

During prerendering, React can emit headers that can be added to the response. These can be used to improve performance by allowing the browser to preload resources like fonts, scripts, and stylesheets. The default value is `6000`, but you can override this value by configuring the `reactMaxHeadersLength` option in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  reactMaxHeadersLength: 1000,
}
```

> **Good to know**: This option is only available in App Router.

Depending on the type of proxy between the browser and the server, the headers can be truncated. For example, if you are using a reverse proxy that doesn't support long headers, you should set a lower value to ensure that the headers are not truncated.

---

reactStrictMode

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/reactStrictMode
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *The complete Next.js runtime is now Strict Mode-compliant, learn how to opt-in*

> **Good to know**: Since Next.js 13.5.1, Strict Mode is `true` by default with `app` router, so the above configuration is only necessary for `pages`. You can still disable Strict Mode by setting `reactStrictMode: false`.

> **Suggested**: We strongly suggest you enable Strict Mode in your Next.js application to better prepare your application for the future of React.

React's Strict Mode is a development mode only feature for highlighting potential problems in an application. It helps to identify unsafe lifecycles, legacy API usage, and a number of other features.

The Next.js runtime is Strict Mode-compliant. To opt-in to Strict Mode, configure the following option in your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  reactStrictMode: true,
}
```

If you or your team are not ready to use Strict Mode in your entire application, that's OK! You can incrementally migrate on a page-by-page basis using `<React.StrictMode>`.

---

redirects

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/redirects
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Add redirects to your Next.js app.*

Redirects allow you to redirect an incoming request path to a different destination path.

To use redirects you can use the `redirects` key in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        source: '/about',
        destination: '/',
        permanent: true,
      },
    ]
  },
}
```

`redirects` can be defined as a synchronous or async function. It should return, or resolve to, an array of objects with `source`, `destination`, and `permanent` properties:

- `source` is the incoming request path pattern.
- `destination` is the path you want to route to.
- `permanent` `true` or `false` - if `true` will use the 308 status code which instructs clients/search engines to cache the redirect forever, if `false` will use the 307 status code which is temporary and is not cached.

> **Why does Next.js use 307 and 308?** Traditionally a 302 was used for a temporary redirect, and a 301 for a permanent redirect, but many browsers changed the request method of the redirect to `GET`, regardless of the original method. For example, if the browser made a request to `POST /v1/users` which returned status code `302` with location `/v2/users`, the subsequent request might be `GET /v2/users` instead of the expected `POST /v2/users`. Next.js uses the 307 temporary redirect, and 308 permanent redirect status codes to explicitly preserve the request method used.

- `basePath`: `false` or `undefined` - if false the `basePath` won't be included when matching, can be used for external redirects only.
- `locale`: `false` or `undefined` - whether the locale should not be included when matching.
- `has` is an array of has objects with the `type`, `key` and `value` properties.
- `missing` is an array of missing objects with the `type`, `key` and `value` properties.

Redirects are checked before the filesystem which includes pages and `/public` files.

When using the Pages Router, redirects are not applied to client-side routing (`Link` , `router.push`) unless Proxy is present and matches the path.

When a redirect is applied, any query values provided in the request will be passed through to the redirect destination. For example, see the following redirect configuration:

```
{
  source: '/old-blog/:path*',
  destination: '/blog/:path*',
  permanent: false
}
```

> **Good to know**: Remember to include the forward slash `/` before the colon `:` in path parameters of the `source` and `destination` paths, otherwise the path will be treated as a literal string and you run the risk of causing infinite redirects.

When `/old-blog/post-1?hello=world` is requested, the client will be redirected to `/blog/post-1?hello=world` .

Path Matching

Path matches are allowed, for example `/old-blog/:slug` will match `/old-blog/first-post` (no nested paths):

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        source: '/old-blog/:slug',
        destination: '/news/:slug', // Matched parameters can be used in the destination
        permanent: true,
      },
    ]
  },
}
```

The pattern `/old-blog/:slug` matches `/old-blog/first-post` and `/old-blog/post-1` but not `/old-blog/a/b` (no nested paths). Patterns are anchored to the start: `/old-blog/:slug` will not match `/archive/old-blog/first-post` .

You can use modifiers on parameters: `*` (zero or more), `+` (one or more), `?` (zero or one). For example, `/blog/:slug*` matches `/blog` , `/blog/a` , and `/blog/a/b/c` .

Read more details on path-to-regexp documentation.

Wildcard Path Matching

To match a wildcard path you can use `*` after a parameter, for example `/blog/:slug*` will match `/blog/a/b/c/d/hello-world` :

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        source: '/blog/:slug*',
        destination: '/news/:slug*', // Matched parameters can be used in the destination
        permanent: true,
      },
    ]
  },
}
```

Regex Path Matching

To match a regex path you can wrap the regex in parentheses after a parameter, for example `/post/:slug(\\d{1,})` will match `/post/123` but not `/post/abc` :

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        source: '/post/:slug(\\d{1,})',
        destination: '/news/:slug', // Matched parameters can be used in the destination
        permanent: false,
      },
    ]
  },
}
```

The following characters `(`, `)`, `{`, `}`, `:`, `*`, `+`, `?` are used for regex path matching, so when used in the `source` as non-special values they must be escaped by adding `\\` before them:

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        // this will match `/english(default)/something` being requested
        source: '/english\\(default\\)/:slug',
        destination: '/en-us/:slug',
        permanent: false,
      },
    ]
  },
}
```

Header, Cookie, and Query Matching

To only match a redirect when header, cookie, or query values also match the `has` field or don't match the `missing` field can be used. Both the `source` and all `has` items must match and all `missing` items must not match for the redirect to be applied.

`has` and `missing` items can have the following fields:

- `type`: `String` - must be either `header`, `cookie`, `host`, or `query`.
- `key`: `String` - the key from the selected type to match against.
- `value`: `String` or `undefined` - the value to check for, if undefined any value will match. A regex like string can be used to capture a specific part of the value, e.g. if the value `first-(?<paramName>.*)` is used for `first-second` then `second` will be usable in the destination with `:paramName`.

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      // if the header `x-redirect-me` is present,
      // this redirect will be applied
      {
        source: '/:path((?!another-page$).*)',
        has: [
          {
            type: 'header',
            key: 'x-redirect-me',
          },
        ],
        permanent: false,
        destination: '/another-page',
      },
      // if the header `x-do-not-redirect` is present,
      // this redirect will NOT be applied
      {
        source: '/:path((?!another-page$).*)',
        missing: [
          {
            type: 'header',
            key: 'x-do-not-redirect',
          },
        ],
        permanent: false,
        destination: '/another-page',
      },
      // if the source, query, and cookie are matched,
      // this redirect will be applied
      {
        source: '/specific/:path*',
        has: [
          {
            type: 'query',
            key: 'page',
            // the page value will not be available in the
            // destination since value is provided and doesn't
            // use a named capture group e.g. (?<page>home)
            value: 'home',
          },
          {
            type: 'cookie',
            key: 'authorized',
            value: 'true',
          },
        ],
        permanent: false,
        destination: '/another/:path*',
      },
      // if the header `x-authorized` is present and
      // contains a matching value, this redirect will be applied
      {
        source: '/',
        has: [
          {
            type: 'header',
            key: 'x-authorized',
            value: '(?<authorized>yes|true)',
          },
        ],
        permanent: false,
        destination: '/home?authorized=:authorized',
      },
      // if the host is `example.com`,
      // this redirect will be applied
      {
        source: '/:path((?!another-page$).*)',
        has: [
          {
            type: 'host',
```

```
            value: 'example.com',
          },
        ],
        permanent: false,
        destination: '/another-page',
      },
    ]
  },
}
```

Redirects with basePath support

When leveraging `basePath` support with redirects each `source` and `destination` is automatically prefixed with the `basePath` unless you add `basePath: false` to the redirect:

```
// Archivo: next.config.js
module.exports = {
  basePath: '/docs',

  redirects() {
    return [
      {
        source: '/with-basePath', // automatically becomes /docs/with-basePath
        destination: '/another', // automatically becomes /docs/another
        permanent: false,
      },
      {
        // does not add /docs since basePath: false is set
        source: '/without-basePath',
        destination: 'https://example.com',
        basePath: false,
        permanent: false,
      },
    ]
  },
}
```

Redirects with i18n support

When implementing redirects with internationalization in the App Router, you can include locales in `next.config.js` redirects, but only as hardcoded paths.

For dynamic or per-request locale handling, use dynamic route segments and proxy, which can redirect based on the user's preferred language.

```
// Archivo: next.config.js
module.exports = {
  redirects() {
    return [
      {
        // Manually handle locale prefixes for App Router
        source: '/en/old-path',
        destination: '/en/new-path',
        permanent: false,
      },
      {
        // Redirect for all locales using a parameter
        source: '/:locale/old-path',
        destination: '/:locale/new-path',
        permanent: false,
      },
      {
        // Redirect from one locale to another
        source: '/de/old-path',
        destination: '/en/new-path',
        permanent: false,
      },
      {
        // Catch-all redirect for multiple locales
        source: '/:locale(en|fr|de)/:path*',
        destination: '/:locale/new-section/:path*',
        permanent: false,
      },
    ]
  },
}
```

📘 **Nota para Pages Router:** When leveraging `i18n` support with redirects each `source` and `destination` is automatically prefixed to handle the configured `locales` unless you add `locale: false` to the redirect. If `locale: false` is used you must prefix the `source` and `destination` with a locale for it to be matched correctly.

```
module.exports = {
  i18n: {
    locales: ['en', 'fr', 'de'],
    defaultLocale: 'en',
  },

  redirects() {
    return [
      {
        source: '/with-locale', // automatically handles all locales
        destination: '/another', // automatically passes the locale on
        permanent: false,
      },
      {
        // does not handle locales automatically since locale: false is set
        source: '/nl/with-locale-manual',
        destination: '/nl/another',
        locale: false,
        permanent: false,
      },
      {
        // this matches '/' since `en` is the defaultLocale
        source: '/en',
        destination: '/en/another',
        locale: false,
        permanent: false,
      },
      // it's possible to match all locales even when locale: false is set
      {
        source: '/:locale/page',
        destination: '/en/newpage',
        permanent: false,
        locale: false,
      },
      {
        // this gets converted to /(en|fr|de)/(.*) so will not match the top-level
        // `/` or `/fr` routes like /:path* would
        source: '/(.*)',
        destination: '/another',
        permanent: false,
      },
    ]
  },
}
```

In some rare cases, you might need to assign a custom status code for older HTTP Clients to properly redirect. In these cases, you can use the `statusCode` property instead of the `permanent` property, but not both. To ensure IE11 compatibility, a `Refresh` header is automatically added for the 308 status code.

Other Redirects

- Inside API Routes and Route Handlers, you can redirect based on the incoming request.
- Inside `getStaticProps` and `getServerSideProps`, you can redirect specific pages at request-time.

Version History

Version	Changes
`v13.3.0`	`missing` added.
`v10.2.0`	`has` added.
`v9.5.0`	`redirects` added.

rewrites

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/rewrites
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Add rewrites to your Next.js app.*

Rewrites allow you to map an incoming request path to a different destination path.

Rewrites act as a URL proxy and mask the destination path, making it appear the user hasn't changed their location on the site. In contrast, redirects will reroute to a new page and show the URL changes.

> 📘 **Nota para Pages Router:** Rewrites act as a URL proxy and mask the destination path, making it appear the user hasn't changed their location on the site. In contrast, redirects will reroute to a new page and show the URL changes.

To use rewrites you can use the `rewrites` key in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/about',
        destination: '/',
      },
    ]
  },
}
```

Rewrites are applied to client-side routing. In the example above, navigating to `<Link href="/about">` will serve content from `/` while keeping the URL as `/about`.

`rewrites` can be defined as a synchronous or async function. It should return, or resolve to, either an array or an object of arrays (see below) holding objects with `source` and `destination` properties:

- `source`: `String` - is the incoming request path pattern.
- `destination`: `String` is the path you want to route to.
- `basePath`: `false` or `undefined` - if false the basePath won't be included when matching, can be used for external rewrites only.
- `locale`: `false` or `undefined` - whether the locale should not be included when matching.
- `has` is an array of has objects with the `type`, `key` and `value` properties.
- `missing` is an array of missing objects with the `type`, `key` and `value` properties.

When the `rewrites` function returns an array, rewrites are applied after checking the filesystem (pages and `/public` files) and before dynamic routes. When the `rewrites` function returns an object of arrays with a specific shape, this behavior can be changed and more finely controlled, as of `v10.1` of Next.js:

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return {
      beforeFiles: [
        // These rewrites are checked after headers/redirects
        // and before all files including _next/public files which
        // allows overriding page files
        {
          source: '/some-page',
          destination: '/somewhere-else',
          has: [{ type: 'query', key: 'overrideMe' }],
        },
      ],
      afterFiles: [
        // These rewrites are checked after pages/public files
        // are checked but before dynamic routes
        {
          source: '/non-existent',
          destination: '/somewhere-else',
        },
      ],
      fallback: [
        // These rewrites are checked after both pages/public files
        // and dynamic routes are checked
        {
          source: '/:path*',
          destination: `https://my-old-site.com/:path*`,
        },
      ],
    }
  },
}
```

> **Good to know**: rewrites in `beforeFiles` do not check the filesystem/dynamic routes immediately after matching a source, they continue until all `beforeFiles` have been checked.

The order Next.js routes are checked is:

1. headers are checked/applied
2. redirects are checked/applied
3. proxy
4. `beforeFiles` rewrites: for each entry, if `source`, `has`, and `missing` matches the request, it's rewritten to `destination`.
5. static files from the public directory, `_next/static` files, and non-dynamic pages are checked/served
6. `afterFiles` rewrites are tried in order. If a `source`, `has`, and `missing` matches the request, it's rewritten to `destination`; the first rewrite that resolves to a static file, page, or dynamic route is served.
7. dynamic routes (e.g., `app/blog/[slug]/page.tsx`) are matched against the current path
8. `fallback` rewrites are checked/applied, these are applied before rendering the 404 page and after dynamic routes/all static assets have been checked. If you use fallback: true/'blocking' in `getStaticPaths`, those dynamic routes take priority over the fallback `rewrites` defined in your `next.config.js`.

> 📘 **Nota para Pages Router:**
>
> 1. headers are checked/applied
> 2. redirects are checked/applied
> 3. `beforeFiles` rewrites: for each entry, if `source` matches the request, it's rewritten to `destination`.
> 4. static files from the public directory, `_next/static` files, and non-dynamic pages are checked/served
> 5. `afterFiles` rewrites are tried in order. If a `source` matches the request, it's rewritten to `destination`; the first rewrite that resolves to a static file, page, or dynamic route is served.
> 6. dynamic routes (e.g., `pages/blog/[slug].tsx`) are matched against the current path
> 7. `fallback` rewrites are checked/applied, these are applied before rendering the 404 page and after dynamic routes/all static assets have been checked. If you use fallback: true/'blocking' in `getStaticPaths`, the fallback `rewrites` defined in your `next.config.js` will *not* be run.

Rewrite parameters

When using parameters in a rewrite the parameters will be passed in the query by default when none of the parameters are used in the `destination`.

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/old-about/:path*',
        destination: '/about', // The :path parameter isn't used here so will be automatically passed in the query
      },
    ]
  },
}
```

If a parameter is used in the destination none of the parameters will be automatically passed in the query.

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/docs/:path*',
        destination: '/:path*', // The :path parameter is used here so will not be automatically passed in the query
      },
    ]
  },
}
```

You can still pass the parameters manually in the query if one is already used in the destination by specifying the query in the `destination`.

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/:first/:second',
        destination: '/:first?second=:second',
        // Since the :first parameter is used in the destination the :second parameter
        // will not automatically be added in the query although we can manually add it
        // as shown above
      },
    ]
  },
}
```

> **Good to know**: Static pages from Automatic Static Optimization or prerendering params from rewrites will be parsed on the client after hydration and provided in the query.

Path Matching

Path matches are allowed, for example `/blog/:slug` will match `/blog/first-post` (no nested paths):

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/blog/:slug',
        destination: '/news/:slug', // Matched parameters can be used in the destination
      },
    ]
  },
}
```

The pattern `/blog/:slug` matches `/blog/first-post` and `/blog/post-1` but not `/blog/a/b` (no nested paths). Patterns are anchored to the start: `/blog/:slug` will not match `/archive/blog/first-post`.

You can use modifiers on parameters: `*` (zero or more), `+` (one or more), `?` (zero or one). For example, `/blog/:slug*` matches `/blog`, `/blog/a`, and `/blog/a/b/c`.

Read more details on path-to-regexp documentation.

Wildcard Path Matching

To match a wildcard path you can use `*` after a parameter, for example `/blog/:slug*` will match `/blog/a/b/c/d/hello-world`:

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/blog/:slug*',
        destination: '/news/:slug*', // Matched parameters can be used in the destination
      },
    ]
  },
}
```

Regex Path Matching

To match a regex path you can wrap the regex in parenthesis after a parameter, for example `/blog/:slug(\\d{1,})` will match `/blog/123` but not `/blog/abc`:

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/old-blog/:post(\\d{1,})',
        destination: '/blog/:post', // Matched parameters can be used in the destination
      },
    ]
  },
}
```

The following characters `(`, `)`, `{`, `}`, `[`, `]`, `|`, `\`, `^`, `.`, `:`, `*`, `+`, `-`, `?`, `$` are used for regex path matching, so when used in the `source` as non-special values they must be escaped by adding `\\` before them:

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        // this will match `/english(default)/something` being requested
        source: '/english\\(default\\)/:slug',
        destination: '/en-us/:slug',
      },
    ]
  },
}
```

Header, Cookie, and Query Matching

To only match a rewrite when header, cookie, or query values also match the `has` field or don't match the `missing` field can be used. Both the `source` and all `has` items must match and all `missing` items must not match for the rewrite to be applied.

`has` and `missing` items can have the following fields:

- `type` : `String` - must be either `header` , `cookie` , `host` , or `query` .
- `key` : `String` - the key from the selected type to match against.
- `value` : `String` or `undefined` - the value to check for, if undefined any value will match. A regex like string can be used to capture a specific part of the value, e.g. if the value `first-(?<paramName>.*)` is used for `first-second` then `second` will be usable in the destination with `:paramName` .

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      // if the header `x-rewrite-me` is present,
      // this rewrite will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'header',
            key: 'x-rewrite-me',
          },
        ],
        destination: '/another-page',
      },
      // if the header `x-rewrite-me` is not present,
      // this rewrite will be applied
      {
        source: '/:path*',
        missing: [
          {
            type: 'header',
            key: 'x-rewrite-me',
          },
        ],
        destination: '/another-page',
      },
      // if the source, query, and cookie are matched,
      // this rewrite will be applied
      {
        source: '/specific/:path*',
        has: [
          {
            type: 'query',
            key: 'page',
            // the page value will not be available in the
            // destination since value is provided and doesn't
            // use a named capture group e.g. (?<page>home)
            value: 'home',
          },
          {
            type: 'cookie',
            key: 'authorized',
            value: 'true',
          },
        ],
        destination: '/:path*/home',
      },
      // if the header `x-authorized` is present and
      // contains a matching value, this rewrite will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'header',
            key: 'x-authorized',
            value: '(?<authorized>yes|true)',
          },
        ],
        destination: '/home?authorized=:authorized',
      },
      // if the host is `example.com`,
      // this rewrite will be applied
      {
        source: '/:path*',
        has: [
          {
            type: 'host',
            value: 'example.com',
          },
        ],
        destination: '/another-page',
```

```
      },
    ]
  },
}
```

Rewriting to an external URL

Examples

Rewrites allow you to rewrite to an external URL. This is especially useful for incrementally adopting Next.js. The following is an example rewrite for redirecting the `/blog` route of your main app to an external site.

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return [
      {
        source: '/blog',
        destination: 'https://example.com/blog',
      },
      {
        source: '/blog/:slug',
        destination: 'https://example.com/blog/:slug', // Matched parameters can be used in the destination
      },
    ]
  },
}
```

If you're using `trailingSlash: true`, you also need to insert a trailing slash in the `source` parameter. If the destination server is also expecting a trailing slash it should be included in the `destination` parameter as well.

```
// Archivo: next.config.js
module.exports = {
  trailingSlash: true,
  rewrites() {
    return [
      {
        source: '/blog/',
        destination: 'https://example.com/blog/',
      },
      {
        source: '/blog/:path*/',
        destination: 'https://example.com/blog/:path*/',
      },
    ]
  },
}
```

Incremental adoption of Next.js

You can also have Next.js fall back to proxying to an existing website after checking all Next.js routes.

This way you don't have to change the rewrites configuration when migrating more pages to Next.js

```
// Archivo: next.config.js
module.exports = {
  rewrites() {
    return {
      fallback: [
        {
          source: '/:path*',
          destination: `https://custom-routes-proxying-endpoint.vercel.app/:path*`,
        },
      ],
    }
  },
}
```

Rewrites with basePath support

When leveraging `basePath` support with rewrites each `source` and `destination` is automatically prefixed with the `basePath` unless you add `basePath: false` to the rewrite:

```
// Archivo: next.config.js
module.exports = {
  basePath: '/docs',

  rewrites() {
    return [
      {
        source: '/with-basePath', // automatically becomes /docs/with-basePath
        destination: '/another', // automatically becomes /docs/another
      },
      {
        // does not add /docs to /without-basePath since basePath: false is set
        // Note: this cannot be used for internal rewrites e.g. `destination: '/another'`
        source: '/without-basePath',
        destination: 'https://example.com',
        basePath: false,
      },
    ]
  },
}
```

📘 **Nota para Pages Router:**

Rewrites with i18n support

When leveraging `i18n` support with rewrites each `source` and `destination` is automatically prefixed to handle the configured `locales` unless you add `locale: false` to the rewrite. If `locale: false` is used you must prefix the `source` and `destination` with a locale for it to be matched correctly.

```
module.exports = {
  i18n: {
    locales: ['en', 'fr', 'de'],
    defaultLocale: 'en',
  },

  rewrites() {
    return [
      {
        source: '/with-locale', // automatically handles all locales
        destination: '/another', // automatically passes the locale on
      },
      {
        // does not handle locales automatically since locale: false is set
        source: '/nl/with-locale-manual',
        destination: '/nl/another',
        locale: false,
      },
      {
        // this matches '/' since `en` is the defaultLocale
        source: '/en',
        destination: '/en/another',
        locale: false,
      },
      {
        // it's possible to match all locales even when locale: false is set
        source: '/:locale/api-alias/:path*',
        destination: '/api/:path*',
        locale: false,
      },
      {
        // this gets converted to /(en|fr|de)/(.*) so will not match the top-level
        // `/` or `/fr` routes like /:path* would
        source: '/(.*)',
        destination: '/another',
      },
    ]
  },
}
```

Version History

Version	Changes
`v13.3.0`	`missing` added.
`v10.2.0`	`has` added.
`v9.5.0`	Headers added.

sassOptions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/sassOptions
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Sass options.*

`sassOptions` allow you to configure the Sass compiler.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const sassOptions = {
  additionalData: `
    $var: red;
  `,
}

const nextConfig: NextConfig = {
  sassOptions: {
    ...sassOptions,
    implementation: 'sass-embedded',
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */

const sassOptions = {
  additionalData: `
    $var: red;
  `,
}

const nextConfig = {
  sassOptions: {
    ...sassOptions,
    implementation: 'sass-embedded',
  },
}

module.exports = nextConfig
```

> **Good to know:**
>
> - `sassOptions` are not typed outside of `implementation` because Next.js does not maintain the other possible properties.
> - The `functions` property for defining custom Sass functions is only supported with webpack. When using Turbopack, custom Sass functions are not available because Turbopack's Rust-based architecture cannot directly execute JavaScript functions passed through this option.

serverActions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/serverActions
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Server Actions behavior in your Next.js application.*

Options for configuring Server Actions behavior in your Next.js application. For how Server Actions work, including the security boundary these options tune, see the Server Actions guide.

`allowedOrigins`

A list of extra safe hosts from which Server Actions can be invoked. To prevent CSRF attacks, Next.js compares the host in a request's `Origin` header against the app's own host, taken from `x-forwarded-host` or `host`, and rejects the action when the two differ. If not provided, only the same origin is allowed. A request that carries no `Origin` header at all is allowed through with a warning rather than rejected.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */

module.exports = {
  experimental: {
    serverActions: {
      allowedOrigins: ['my-proxy.com', '*.my-proxy.com'],
    },
  },
}
```

Only the `host` of the request's `Origin` header is matched against your entries, which is the hostname plus the port when the URL carries one. For a request from `https://my-proxy.com/checkout`, that is `my-proxy.com`. For one from `https://my-proxy.com:8443/checkout`, `my-proxy.com:8443`. Write your entries the same way.

Entries can also expand, through two wildcards: a `*` stands in for exactly one label of the host, and `**` for one or more. That is why the example above lists two entries, one for the bare host and one for its subdomains.

Entry	Matches	Does not match
`my-proxy.com`	`my-proxy.com`	`my-proxy.com:8443`, `app.my-proxy.com`
`*.my-proxy.com`	`app.my-proxy.com`	`my-proxy.com`, `app.my-proxy.com:8443`
`**.my-proxy.com`	`app.my-proxy.com`, `app.eu.my-proxy.com`	`my-proxy.com`
`my-proxy.com:8443`	`my-proxy.com:8443`	`my-proxy.com`

Partial replacement is not supported. Write `*.my-proxy.com`, rather than `app-*.my-proxy.com`. Using `**` is only supported at the start of the pattern. A port cannot be wildcarded, so write it out in full: `my-proxy.com:8443`, or `*.my-proxy.com:8443` for its subdomains.

Behind a reverse proxy, no entry is needed as long as the proxy forwards the public host in `x-forwarded-host`. When it forwards its own host instead, the browser sends `my-proxy.com` while the server reports something like `localhost:3000`, and that mismatch is what this list is for: the host visible in the browser's address bar is the one to add, not the internal one the server reports.

The check runs in production as well as in development, so the list applies to your deployed app and not only to local work.

For example, remote development through a tunnel needs the tunnel hostname in two places: in `allowedDevOrigins`, so the dev server serves its own assets and endpoints, and here, so actions from it are accepted:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */

module.exports = {
  allowedDevOrigins: ['*.tunnel.example.com'],
  experimental: {
    serverActions: {
      allowedOrigins: ['*.tunnel.example.com'],
    },
  },
}
```

`bodySizeLimit`

By default, the maximum size of the request body sent to a Server Action is 1MB, to prevent the consumption of excessive server resources in parsing large amounts of data, as well as potential DDoS attacks.

However, you can configure this limit using the `serverActions.bodySizeLimit` option. It can take the number of bytes or any string format supported by bytes, for example `1000`, `'500kb'` or `'3mb'`.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */

module.exports = {
  experimental: {
    serverActions: {
      bodySizeLimit: '2mb',
    },
  },
}
```

The limit applies to the raw HTTP request body, including the bytes that `multipart/form-data` adds for boundaries, part headers, and field metadata. If you expect uploads close to the configured value, leave some room for this overhead. For typical multipart uploads, an additional 10–20 KB is a reasonable rule of thumb.

Enabling Server Actions (v13)

Server Actions became a stable feature in Next.js 14, and are enabled by default. However, if you are using an earlier version of Next.js, you can enable them by setting `experimental.serverActions` to `true`.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const config = {
  experimental: {
    serverActions: true,
  },
}

module.exports = config
```

---

serverComponentsHmrCache

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/serverComponentsHmrCache
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure whether fetch responses in Server Components are cached across HMR refresh requests.*

The experimental `serverComponentsHmrCache` option allows you to cache `fetch` responses in Server Components across Hot Module Replacement (HMR) refreshes in local development. This results in faster responses and reduced costs for billed API calls.

By default, the HMR cache applies to all `fetch` requests, including those with the `cache: 'no-store'` option. This means uncached requests will not show fresh data between HMR refreshes. However, the cache will be cleared on navigation or full-page reloads.

You can disable the HMR cache by setting `serverComponentsHmrCache` to `false` in your `next.config.js` file:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    serverComponentsHmrCache: false, // defaults to true
  },
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    serverComponentsHmrCache: false, // defaults to true
  },
}

module.exports = nextConfig
```

> **Good to know:** For better observability, we recommend using the `logging.fetches` option which logs fetch cache hits and misses in the console during development.

---

serverExternalPackages

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/serverExternalPackages
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Opt-out specific dependencies from the Server Components bundling and use native Node.js* `require`.

Dependencies used inside Server Components and Route Handlers will automatically be bundled by Next.js.

If a dependency is using Node.js specific features, you can choose to opt-out specific dependencies from the Server Components bundling and use native Node.js `require`.

```js
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  serverExternalPackages: ['@acme/ui'],
}

module.exports = nextConfig
```

Next.js includes a short list of popular packages that currently are working on compatibility and automatically opt-ed out:

- `@alinea/generated`
- `@appsignal/nodejs`
- `@aws-sdk/client-s3`
- `@aws-sdk/s3-presigned-post`
- `@blockfrost/blockfrost-js`
- `@highlight-run/node`
- `@huggingface/transformers`
- `@jpg-store/lucid-cardano`
- `@libsql/client`

- `@mikro-orm/core`
- `@mikro-orm/knex`
- `@node-rs/argon2`
- `@node-rs/bcrypt`
- `@prisma/client`
- `@react-pdf/renderer`
- `@sentry/profiling-node`
- `@sparticuz/chromium`
- `@sparticuz/chromium-min`
- `@statsig/statsig-node-core`
- `@swc/core`
- `@xenova/transformers`
- `@zenstackhq/runtime`
- `argon2`
- `autoprefixer`
- `aws-crt`
- `bcrypt`
- `better-sqlite3`
- `canvas`
- `chromadb-default-embed`
- `config`
- `cpu-features`
- `cypress`
- `dd-trace`
- `eslint`
- `express`
- `firebase-admin`
- `htmlrewriter`
- `import-in-the-middle`
- `isolated-vm`
- `jest`
- `jsdom`
- `keyv`
- `libsql`
- `mdx-bundler`
- `mongodb`
- `mongoose`
- `newrelic`
- `next-mdx-remote`
- `next-seo`
- `node-cron`
- `node-pty`
- `node-web-audio-api`
- `onnxruntime-node`
- `oslo`
- `pg`
- `pino`

- `pino-pretty`
- `pino-roll`
- `playwright`
- `playwright-core`
- `postcss`
- `prettier`
- `prisma`
- `puppeteer-core`
- `puppeteer`
- `ravendb`
- `require-in-the-middle`
- `rimraf`
- `sharp`
- `shiki`
- `sqlite3`
- `thread-stream`
- `ts-morph`
- `ts-node`
- `typescript`
- `vscode-oniguruma`
- `webpack`
- `websocket`
- `zeromq`

Version	Changes
`v15.0.0`	Moved from experimental to stable. Renamed from `serverComponentsExternalPackages` to `serverExternalPackages`

skipProxyUrlNormalize

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/skipProxyUrlNormalize
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Let Proxy see the original request instead of Next.js's normalized version. Formerly skipMiddlewareUrlNormalize.*

Enabling `skipProxyUrlNormalize` lets Proxy see the original request instead of Next.js's normalized version. This includes internal URLs, query parameters, and headers used during client-side navigation.

Most projects don't need this option. It is one of the advanced Proxy flags, meant for specific routing cases and debugging.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  skipProxyUrlNormalize: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  skipProxyUrlNormalize: true,
}

module.exports = nextConfig
```

Reference

During client-side navigation, Next.js uses an internal request rather than requesting the destination URL as a full document load. By default, Next.js normalizes that request before it reaches Proxy, so Proxy sees the same destination URL for both client-side navigation and a full page load.

Next.js also normalizes what Proxy returns, so any URL you rewrite or redirect to is rewritten to match.

With `skipProxyUrlNormalize: true`, neither normalization runs. Proxy receives the request as sent, and the destinations it rewrites or redirects to are sent as written:

Default	With `skipProxyUrlNormalize: true`
`request.nextUrl` resolves the internal data URL back to the route path and extracts the locale	Holds the requested pathname as sent
`request.url` is the normalized URL	Is the original request URL
The internal `_rsc` query parameter is stripped	Preserved
Next.js internal navigation headers (`rsc` , `next-router-state-tree` , `next-router-prefetch` , `next-router-segment-prefetch` , `next-hmr-refresh`) are removed from `request.headers`	Preserved
`NextResponse.rewrite()` destinations are re-serialized with the build ID	Sent as written
`NextResponse.redirect()` `Location` is rewritten to a relative URL	Sent as written, trailing slash preserved
With `trailingSlash: true` , a trailing slash is appended before Proxy runs	Pathname passed through unchanged

Deprecated alias

The former name of this option is `skipMiddlewareUrlNormalize` , from when Proxy was called Middleware. It still works and logs a deprecation warning. Setting both at once throws:

```
// Archivo: Terminal
Config options `skipProxyUrlNormalize` and `skipMiddlewareUrlNormalize` cannot be set at the same time. Please use
`skipProxyUrlNormalize` instead.
```

The version 16 codemod renames it for you.

Good to know

- Avoid using the internal navigation headers to return different content for client-side navigation and full page loads.
- You do not need this option to limit which routes Proxy runs on. The `matcher` config still matches the destination URL either way.
- Only the request and response Proxy sees change. Filesystem routing, `redirects` , and `rewrites` from `next.config.js` behave the same.

Examples

Reading the URL as the client sent it

During client-side navigation, Next.js adds an `_rsc` query parameter and its internal navigation headers to the request. With the option enabled, Proxy can read both.

```
// Archivo: proxy.ts switcher
import type { NextRequest } from 'next/server'

export default function proxy(request: NextRequest) {
  console.log(request.nextUrl.searchParams.get('_rsc'))
  // Enabled: the value Next.js added to the request
  // Disabled: null

  console.log(request.headers.get('rsc'))
  // Enabled: '1' during client-side navigation
  // Disabled: null
}
```

```
// Archivo: proxy.js switcher
export default function proxy(request) {
  console.log(request.nextUrl.searchParams.get('_rsc'))
  // Enabled: the value Next.js added to the request
  // Disabled: null

  console.log(request.headers.get('rsc'))
  // Enabled: '1' during client-side navigation
  // Disabled: null
}
```

📘 **Nota para Pages Router:** During client-side navigation, Next.js requests a `/_next/data` URL instead of the destination URL. With the option enabled, Proxy sees that URL as sent.

```
import type { NextRequest } from 'next/server'

export default function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl

  // Request: GET /_next/data/build-id/hello.json
  console.log(pathname)
  // Enabled: /_next/data/build-id/hello.json
  // Disabled: /hello
}
```

```
export default function proxy(request) {
  const { pathname } = request.nextUrl

  // Request: GET /_next/data/build-id/hello.json
  console.log(pathname)
  // Enabled: /_next/data/build-id/hello.json
  // Disabled: /hello
}
```

Version History

Version	Changes
`v16.0.0`	Renamed from `skipMiddlewareUrlNormalize` to `skipProxyUrlNormalize`.
`v13.1.0`	`skipMiddlewareUrlNormalize` added.

skipTrailingSlashRedirect

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/skipTrailingSlashRedirect
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Disable the automatic trailing slash redirects so you can handle trailing slashes yourself in Proxy.*

Enabling `skipTrailingSlashRedirect` turns off the automatic redirects that add or remove a trailing slash. You handle any redirecting you still want in Proxy.

Most projects don't need this option. It is one of the advanced Proxy flags, meant for specific routing cases and debugging.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  skipTrailingSlashRedirect: true,
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  skipTrailingSlashRedirect: true,
}

module.exports = nextConfig
```

Reference

By default, Next.js redirects every request so the trailing slash matches your `trailingSlash` setting.

`trailingSlash`	Request	Default response
`false`	`/about`	`200`
`false`	`/about/`	`308` redirect to `/about`
`true`	`/about`	`308` redirect to `/about/`
`true`	`/about/`	`200`

Enabling `skipTrailingSlashRedirect` affects both server responses and client-side navigation:

- Both `/about` and `/about/` return `200`. Neither redirects to the other.
- Next.js stops changing trailing slashes during client-side navigation. `<Link>` and router URLs keep the path exactly as written.

Your own `redirects` and `rewrites` from `next.config.js` still apply.

Paths that contain a dot

If the last path segment contains a dot, Next.js treats the path as a file and removes the trailing slash, regardless of the `trailingSlash` setting. A dot in an earlier segment has no effect, except under `.well-known/`, which is exempt from the default redirects when `trailingSlash` is `true`.

Request	`trailingSlash: true`	`trailingSlash: false`	`skipTrailingSlashRedirect: true`
`/about`	`308` redirect to `/about/`	`200`	`200`
`/about/`	`200`	`308` redirect to `/about`	`200`
`/gcc.git/`	`308` redirect to `/gcc.git`	`308` redirect to `/gcc.git`	`200`
`/version/1.2.3/`	`308` redirect to `/version/1.2.3`	`308` redirect to `/version/1.2.3`	`200`
`/version/1.2.3/my-page/`	`200`	`308` redirect to `/version/1.2.3/my-page`	`200`
`/.well-known/x/`	`200`	`308` redirect to `/.well-known/x`	`200`

With `skipTrailingSlashRedirect` enabled, none of these redirects are generated and every path is served at whichever form was requested.

Good to know

- Keep setting `trailingSlash` as you did before. It still controls the URLs Next.js generates and the output filenames for `output: 'export'`.
- With `output: 'export'`, this preserves the `href` instead of resolving `/me` to `/me/`. See Static Exports.
- Serving one page at both `/about` and `/about/` gives search engines two URLs for it. Set a canonical URL in your metadata, or redirect in Proxy.

Examples

Handling trailing slashes in Proxy

With `skipTrailingSlashRedirect` enabled, Next.js no longer adds trailing slashes, but Proxy can still add them selectively. This example adds a trailing slash to your own routes while leaving a set of legacy prefixes untouched, which makes an incremental migration easier.

```
// Archivo: proxy.ts switcher
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

const legacyPrefixes = ['/docs', '/blog']

export default function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl

  if (legacyPrefixes.some((prefix) => pathname.startsWith(prefix))) {
    return NextResponse.next()
  }

  // Apply trailing slash handling, skipping files and `.well-known` paths
  if (
    !pathname.endsWith('/') &&
    !pathname.match(/((?!\.well-known(?:\/.*)?)(?:[^/]+\/)*[^/]+\.\w+)/)
  ) {
    return NextResponse.redirect(new URL(`${pathname}/`, request.nextUrl))
  }
}
```

```
// Archivo: proxy.js switcher
import { NextResponse } from 'next/server'

const legacyPrefixes = ['/docs', '/blog']

export default function proxy(request) {
  const { pathname } = request.nextUrl

  if (legacyPrefixes.some((prefix) => pathname.startsWith(prefix))) {
    return NextResponse.next()
  }

  // Apply trailing slash handling, skipping files and `.well-known` paths
  if (
    !pathname.endsWith('/') &&
    !pathname.match(/((?!\.well-known(?:\/.*)?)(?:[^/]+\/)*[^/]+\.\w+)/)
  ) {
    return NextResponse.redirect(new URL(`${pathname}/`, request.nextUrl))
  }
}
```

Version History

Version	Changes
`v13.1.0`	`skipTrailingSlashRedirect` added.

staleTimes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/staleTimes
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to override the invalidation time of the client cache.*

`staleTimes` is an experimental feature that enables caching of page segments in the Client Cache.

You can enable this experimental feature and provide custom revalidation times by setting the experimental `staleTimes` flag:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    staleTimes: {
      dynamic: 30,
      static: 180,
    },
  },
}

module.exports = nextConfig
```

The `static` and `dynamic` properties correspond with the time period (in seconds) based on different types of link prefetching.

- The `dynamic` property is used when the page is neither statically generated nor fully prefetched (e.g. with `prefetch={true}`).
  - Default: 0 seconds (not cached)
- The `static` property is used for statically generated pages, or when the `prefetch` prop on `Link` is set to `true`, or when calling `router.prefetch`.
  - Default: 5 minutes

Good to know:

- Loading boundaries are considered reusable for the `static` period defined in this configuration.
- This doesn't affect partial rendering, **meaning shared layouts won't automatically be refetched on every navigation, only the page segment that changes.**
- This doesn't change back/forward caching behavior to prevent layout shift and to prevent losing the browser scroll position.

Version History

Version	Changes
`v15.0.0`	The `dynamic` `staleTimes` default changed from 30s to 0s.
`v14.2.0`	Experimental `staleTimes` introduced.

staticGeneration*

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/staticGeneration
📂 **Sección:** App > Api Reference > Config > Next Config Js
📝 **Descripción:** *Learn how to configure static generation in your Next.js application.*

The `staticGeneration*` options allow you to configure the Static Generation process for advanced use cases.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    staticGenerationRetryCount: 1,
    staticGenerationMaxConcurrency: 8,
    staticGenerationMinPagesPerWorker: 25,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
const nextConfig = {
  experimental: {
    staticGenerationRetryCount: 1,
    staticGenerationMaxConcurrency: 8,
    staticGenerationMinPagesPerWorker: 25,
  },
}

module.exports = nextConfig
```

Config Options

The following options are available:

- `staticGenerationRetryCount`: The number of times to retry a failed page generation before failing the build.
- `staticGenerationMaxConcurrency`: The maximum number of pages to be processed per worker.
- `staticGenerationMinPagesPerWorker`: The minimum number of pages to be processed before starting a new worker.

supportsImmutableAssets

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/supportsImmutableAssets
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure support for immutable static assets*

> **Attention**: This option is primarily intended for adapter authors. App developers should only set it when troubleshooting adapter-specific issues.
>
> **Enabling this feature when your provider or adapter does not support it can result in broken deployments.**

When skew protection is enabled, static asset requests include a deployment-specific query parameter, for example:

```
GET https://foo.com/_next/static/chunks/0d_ks0ow7ur6m.js?dpl=<unique-deployment-id>
```

One downside is that browsers have to download static assets again after each new deployment, even when those assets have not changed.

Immutable static assets allow the `?dpl` query parameter to be omitted for static assets that are guaranteed to be immutable and content-addressed by their filename. This lets browsers cache those assets indefinitely and allows unchanged static assets to be skipped when uploading files during subsequent deployments:

```
GET https://foo.com/_next/static/immutable/chunks/0d_ks0ow7ur6m.js
```

You can use this config option to opt out of immutable static assets when your adapter has enabled support for them. If your adapter has not enabled this feature, this option has no effect:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  supportsImmutableAssets: false,
}

module.exports = nextConfig
```

See Supporting immutable static assets for more information on how to support this feature in an adapter.

Version History

Version	Changes
`v16.3.0`	Added support for immutable static assets.

taint

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/taint
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable tainting Objects and Values.*

Usage

The `taint` option enables support for experimental React APIs for tainting objects and values. This feature helps prevent sensitive data from being accidentally passed to the client. When enabled, you can use:

- `experimental_taintObjectReference` taint objects references.
- `experimental_taintUniqueValue` to taint unique values.

> **Good to know**: Activating this flag also enables the React `experimental` channel for `app` directory, and taints `process.env` so it cannot be passed whole to a Client Component.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    taint: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    taint: true,
  },
}

module.exports = nextConfig
```

> **Warning:** Do not rely on the taint API as your only mechanism to prevent exposing sensitive data to the client. See our security recommendations.

The taint APIs allows you to be defensive, by declaratively and explicitly marking data that is not allowed to pass through the Server-Client boundary. When an object or value, is passed through the Server-Client boundary, React throws an error.

This is helpful for cases where:

- The methods to read data are out of your control
- You have to work with sensitive data shapes not defined by you
- Sensitive data is accessed during Server Component rendering

It is recommended to model your data and APIs so that sensitive data is not returned to contexts where it is not needed.

Caveats

- Tainting can only keep track of objects by reference. Copying an object creates an untainted version, which loses all guarantees given by the API. You'll need to taint the copy.
- The built-in `process.env` taint applies to the object reference only. Reading individual variables such as `process.env.MY_VAR` and passing the resulting string to a Client Component is unaffected, as is passing a copy like `{ ...process.env }`.
- Tainting cannot keep track of data derived from a tainted value. You also need to taint the derived value.
- Values are tainted for as long as their lifetime reference is within scope. See the `experimental_taintUniqueValue` parameters reference, for more information.

Examples

Tainting an object reference

In this case, the `getUserDetails` function returns data about a given user. We taint the user object reference, so that it cannot cross a Server-Client boundary. For example, assuming `UserCard` is a Client Component.

```
import { experimental_taintObjectReference } from 'react'

async function getUserDetails(id: string): Promise<UserDetails> {
  const user = await db.queryUserById(id)

  experimental_taintObjectReference(
    'Do not use the entire user info object. Instead, select only the fields you need.',
    user
  )

  return user
}
```

```
import { experimental_taintObjectReference } from 'react'

async function getUserDetails(id) {
  const user = await db.queryUserById(id)

  experimental_taintObjectReference(
    'Do not use the entire user info object. Instead, select only the fields you need.',
    user
  )

  return user
}
```

We can still access individual fields from the tainted `userDetails` object.

```
// Archivo: app/contact/page.tsx switcher
export async function ContactPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const userDetails = await getUserDetails(id)

  return (
    <UserCard
      firstName={userDetails.firstName}
      lastName={userDetails.lastName}
    />
  )
}
```

```
export async function ContactPage({ params }) {
  const { id } = await params
  const userDetails = await getUserDetails(id)

  return (
    <UserCard
      firstName={userDetails.firstName}
      lastName={userDetails.lastName}
    />
  )
}
```

Now, passing the entire object to the Client Component will throw an error.

```
export async function ContactPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const userDetails = await getUserDetails(id)

  // Throws an error
  return <UserCard user={userDetails} />
}
```

```
export async function ContactPage({ params }) {
  const { id } = await params
  const userDetails = await getUserDetails(id)

  // Throws an error
  return <UserCard user={userDetails} />
}
```

Tainting a unique value

In this case, we can access the server configuration by awaiting calls to `configService.getConfigDetails`. However, the system configuration contains the `SERVICE_API_KEY` that we don't want to expose to clients.

We can taint the `config.SERVICE_API_KEY` value.

```
import { experimental_taintUniqueValue } from 'react'

async function getSystemConfig(): Promise<SystemConfig> {
  const config = await configService.getConfigDetails()

  experimental_taintUniqueValue(
    'Do not pass configuration tokens to the client',
    config,
    config.SERVICE_API_KEY
  )

  return config
}
```

```
import { experimental_taintUniqueValue } from 'react'

async function getSystemConfig() {
  const config = await configService.getConfigDetails()

  experimental_taintUniqueValue(
    'Do not pass configuration tokens to the client',
    config,
    config.SERVICE_API_KEY
  )

  return config
}
```

We can still access other properties of the `systemConfig` object.

```
export async function Dashboard() {
  const systemConfig = await getSystemConfig()

  return <ClientDashboard version={systemConfig.SERVICE_API_VERSION} />
}
```

However, passing `SERVICE_API_KEY` to `ClientDashboard` throws an error.

```
export async function Dashboard() {
  const systemConfig = await getSystemConfig()
  // Someone makes a mistake in a PR
  const version = systemConfig.SERVICE_API_KEY

  return <ClientDashboard version={version} />
}
```

Note that, even though, `systemConfig.SERVICE_API_KEY` is reassigned to a new variable. Passing it to a Client Component still throws an error.

Whereas, a value derived from a tainted unique value, will be exposed to the client.

```
export async function Dashboard() {
  const systemConfig = await getSystemConfig()
  // Someone makes a mistake in a PR
  const version = `version::${systemConfig.SERVICE_API_KEY}`

  return <ClientDashboard version={version} />
}
```

A better approach is to remove `SERVICE_API_KEY` from the data returned by `getSystemConfig`.

---

trailingSlash

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/trailingSlash
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Next.js pages to resolve with or without a trailing slash.*

By default Next.js will redirect URLs with trailing slashes to their counterpart without a trailing slash. For example `/about/` will redirect to `/about`. You can configure this behavior to act the opposite way, where URLs without trailing slashes are redirected to their counterparts with trailing slashes.

Open `next.config.js` and add the `trailingSlash` config:

```
// Archivo: next.config.js
module.exports = {
  trailingSlash: true,
}
```

With this option set, URLs like `/about` will redirect to `/about/`.

When using `trailingSlash: true`, certain URLs are exceptions and will not have a trailing slash appended:

- Static file URLs, such as files with extensions.
- Any paths under `.well-known/`.

For example, the following URLs will remain unchanged: `/file.txt`, `images/photos/picture.png`, and `.well-known/subfolder/config.json`.

When used with `output: "export"` configuration, the `/about` page will output `/about/index.html` (instead of the default `/about.html`).

Version History

Version	Changes
`v9.5.0`	`trailingSlash` added.

transpilePackages

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/transpilePackages
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Transpile and bundle dependencies from monorepo workspace packages or `node_modules` libraries that ship TypeScript, JSX, or modern syntax.*

Use `transpilePackages` to compile and bundle a dependency instead of treating it as untouched runtime code. Values are package names, including scoped names like `@scope/pkg`. Paths and glob patterns are not supported.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  transpilePackages: ['package-name', '@scope/pkg'],
}

module.exports = nextConfig
```

This replaces the `next-transpile-modules` package.

When you need it

Turbopack transpiles workspace packages (npm, pnpm, or Yarn workspaces) in your monorepo automatically under both routers. Webpack does the same for the App Router. Add a package to `transpilePackages` when:

- **A `node_modules` dependency ships raw TypeScript or JSX.** Next.js does not compile code inside `node_modules` by default. Listing the package opts it in, or you can build the package to plain JavaScript and point its `main` / `exports` at the compiled output.
- **You build with webpack for the Pages Router and the dependency's source lives outside the next app's directory.** For example, an `apps/web` app importing `packages/ui` in the same monorepo.
- **You use the Pages Router and want a `node_modules` dependency bundled into the route.** Pages Router loads `node_modules` server-side dependencies through Node.js `require` at runtime. List the package to bundle its source into the route instead. App Router already bundles Server Component and Route Handler dependencies unless the package is listed in `serverExternalPackages`.

> **Good to know**: A package cannot appear in both `transpilePackages` and `serverExternalPackages`; Next.js throws at build start if it does. Packages listed in `optimizePackageImports` and the entries in `default-transpiled-packages.json` are added automatically; you do not need to repeat them.

Version History

Version	Changes
`v13.0.0`	`transpilePackages` added.

turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Next.js with Turbopack-specific options*

The `turbopack` option lets you customize Turbopack to transform different files and change how modules are resolved.

> **Good to know**: The `turbopack` option was previously named `experimental.turbo` in Next.js versions 13.0.0 to 15.2.x, an option which still works as an alias, but new configuration should use the top-level `turbopack` option.
>
> If you are using an older version of Next.js, run `npx @next/codemod@latest next-experimental-turbo-to-turbopack .` to automatically migrate your configuration.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    // ...
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    // ...
  },
}

module.exports = nextConfig
```

> **Good to know**:
>
> - Turbopack for Next.js does not require loaders or loader configuration for built-in functionality. Turbopack has built-in support for CSS and compiling modern JavaScript, so there's no need for `css-loader`, `postcss-loader`, or `babel-loader` if you're using `@babel/preset-env`.

Reference

Options

The following options are available for the `turbopack` configuration:

Option	Description
`root`	Sets the application root directory. Should be an absolute path.
`rules`	List of supported webpack loaders to apply when running with Turbopack.
`resolveAlias`	Map aliased imports to modules to load in their place.
`resolveExtensions`	List of extensions to resolve when importing files.
`debugIds`	Enable generation of debug IDs in JavaScript bundles and source maps.

Supported loaders

The following loaders have been tested to work with Turbopack's webpack loader implementation, but many other webpack loaders should work as well even if not listed here:

- `babel-loader` *(Configured automatically if a Babel configuration file is found)*
- `@svgr/webpack`
- `svg-inline-loader`
- `yaml-loader`
- `string-replace-loader`
- `raw-loader`
- `sass-loader` *(Configured automatically)*
- `graphql-tag/loader`

Missing Webpack loader features

Turbopack uses the `loader-runner` library to execute webpack loaders, which provides most of the standard loader API. However, some features are not supported:

Module loading:

- `importModule` - No support
- `loadModule` - No support

File system and output:

- `fs` - Partial support: only `fs.readFile` is currently implemented.
- `emitFile` - No support

Context properties:

- `version` - No support
- `mode` - No support
- `target` - No support

Utilities:

- `utils` - No support
- `resolve` - No support (use `getResolve` instead)

If you have a loader that is critically dependent upon one of these features please file an issue.

Examples

Root directory

Turbopack uses the root directory to resolve modules. Files outside of the project root are not resolved.

The reason files are not resolved outside of the project root is to improve cache validation, reduce filesystem watching overhead, and reduce the number of resolving steps needed.

Next.js automatically detects the root directory of your project. It does so by looking for one of these files:

- `pnpm-lock.yaml`
- `package-lock.json`
- `yarn.lock`
- `bun.lock`
- `bun.lockb`

If you have a different project structure, for example if you don't use workspaces, you can manually set the `root` option:

```
// Archivo: next.config.js
const path = require('path')
module.exports = {
  turbopack: {
    root: path.join(__dirname, '..'),
  },
}
```

To resolve files from linked dependencies outside the project root (via `npm link`, `yarn link`, `pnpm link`, etc.), you must configure the `turbopack.root` to the parent directory of both the project and the linked dependencies.

While this expands the scope of filesystem watching, it's typically only necessary during development when actively working on linked packages.

Configuring webpack loaders

If you need loader support beyond what's built in, many webpack loaders already work with Turbopack. There are currently some limitations:

- Only a core subset of the webpack loader API is implemented. Currently, there is enough coverage for some popular loaders, and we'll expand our API support in the future.
- Only loaders that return JavaScript code are supported. Loaders that transform files like stylesheets or images are not currently supported.
- Options passed to webpack loaders must be plain JavaScript primitives, objects, and arrays. For example, it's not possible to pass `require()` plugin modules as option values.

To configure loaders, add the names of the loaders you've installed and any options in `next.config.js`, mapping file extensions to a list of loaders. Rules are evaluated in order.

Here is an example below using the `@svgr/webpack` loader, which enables importing `.svg` files and rendering them as React components.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        loaders: ['@svgr/webpack'],
        as: '*.js',
      },
    },
  },
}
```

> **Good to know**: Globs used in the `rules` object match based on file name, unless the glob contains a `/` character, which will cause it to match based on the full project-relative file path. Windows file paths are normalized to use unix-style `/` path separators.
>
> Turbopack uses a modified version of the Rust `globset` library.

For loaders that require configuration options, you can use an object format instead of a string:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        loaders: [
          {
            loader: '@svgr/webpack',
            options: {
              icon: true,
            },
          },
        ],
        as: '*.js',
      },
    },
  },
}
```

> **Good to know**: Prior to Next.js version 13.4.4, `turbopack.rules` was named `turbo.loaders` and only accepted file extensions like `.mdx` instead of `*.mdx`.

Advanced webpack loader conditions

You can further restrict where a loader runs using the advanced `condition` syntax:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      // '*' will match all file paths, but we restrict where our
      // rule runs with a condition.
      '*': {
        condition: {
          all: [
            // 'foreign' is a built-in condition.
            { not: 'foreign' },
            // 'path' can be a RegExp or a glob string. A RegExp matches
            // anywhere in the full project-relative file path.
            { path: /^img\/[0-9]{3}\// },
            {
              any: [
                { path: '*.svg' },
                // 'query' matches anywhere in the full query string,
                // which can be empty, or start with `?`.
                { query: /[?&]svgr(?=&|$)/ },
                // 'content' is always a RegExp, and can match
                // anywhere in the file.
                { content: /\<svg\W/ },
              ],
            },
          ],
        },
        loaders: ['@svgr/webpack'],
        as: '*.js',
      },
    },
  },
}
```

- Supported boolean operators are `{all: [ ... ]}`, `{any: [ ... ]}` and `{not: ... }`.
- Supported customizable operators are `{path: string | RegExp}`, `{content: RegExp}`, `{query: string | RegExp}`, and `{contentType: string | RegExp}`. If multiple operators are specified in the same object, it acts as an implicit `and`.
  - `path` matches against the project-relative file path. A string is treated as a glob pattern, while a RegExp can be used to match the path partially.
  - `content` matches anywhere in the file content.

- `query` matches the import's query string (e.g., `?foo` in `import './file?foo'`). A string must match exactly, while a RegExp can be used to match the query string partially.
- `contentType` matches the MIME content type of the resource (e.g., from data URLs like `data:text/plain, ...`). A string is treated as a glob pattern (e.g., `text/*`, `image/*`), while a RegExp can be used to match the content type partially.

In addition, a number of built-in conditions are supported:

- `browser`: Matches code that will execute on the client. Server code can be matched using `{not: 'browser'}`.
- `foreign`: Matches code in `node_modules`, as well as some Next.js internals. Usually you'll want to restrict loaders to `{not: 'foreign'}`. This can improve performance by reducing the number of files the loader is invoked on.
- `development`: Matches when using `next dev`.
- `production`: Matches when using `next build`.
- `node`: Matches code that will run on the default Node.js runtime.
- `edge-light`: Matches code that will run on the Edge runtime (deprecated).

Rules can be an object or an array of objects. An array is often useful for modeling disjoint conditions:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': [
        {
          condition: 'browser',
          loaders: ['@svgr/webpack'],
          as: '*.js',
        },
        {
          condition: { not: 'browser' },
          loaders: [require.resolve('./custom-svg-loader.js')],
          as: '*.js',
        },
      ],
    },
  },
}
```

> **Good to know**: All matching rules are executed in order.

Module types

You can set the module type directly without using a loader. This is useful for changing how files are processed, similar to webpack's `type` option.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    rules: {
      '*.svg': {
        type: 'asset',
      },
    },
  },
}
```

When using `type: 'asset'`, importing the file returns its URL:

```
// Archivo: app/page.tsx
import svgUrl from './icon.svg'

export default function Page() {
  return <img src={svgUrl} alt="Icon" />
}
```

The `type` option can be combined with `loaders` - loaders run first, then the result is processed according to the specified type.

Available module types:

Type	Description
`asset`	Emit file and return URL (like webpack `asset/resource`)
`ecmascript`	Process as JavaScript
`typescript`	Process as TypeScript
`css`	Process as CSS
`css-module`	Process as CSS module
`wasm`	Process as WebAssembly
`raw`	Return raw contents as string
`bytes`	Inline contents as bytes

Inline loader configuration with import attributes

You can apply a Turbopack loader to an individual import using the `with` clause (import attributes). This is specified per-import rather than globally via `turbopack.rules`.

This is useful when you want to apply a loader to a specific import without affecting all files of that type.

```
// Archivo: app/page.tsx
// Apply a raw loader to import a .txt file as a JavaScript module
import rawText from '../data.txt' with { turbopackLoader: 'raw-loader', turbopackAs: '*.js' }

export default function Page() {
  return <p>{rawText}</p>
}
```

The following import attributes are supported:

Attribute	Description
`turbopackLoader`	The loader to apply (e.g. `'raw-loader'`).
`turbopackLoaderOptions`	JSON string of loader options (e.g. `'{"search":"X","replace":"Y"}'`).
`turbopackAs`	Rename pattern for the output (same as `turbopack.rules[].as`). For example, `'*.js'` treats the loader output as JavaScript.
`turbopackModuleType`	Set the module type for the output (same as `turbopack.rules[].type`).

Loaders with options pass a JSON-encoded string via `turbopackLoaderOptions`:

```
// Archivo: app/page.tsx
import value from '../data.js' with { turbopackLoader: 'string-replace-loader', turbopackLoaderOptions:
'{"search":"PLACEHOLDER","replace":"replaced value"}' }
```

> **Good to know**: Import attributes with `turbopackLoader` are Turbopack-specific and are not supported by webpack. This feature requires the `with` keyword (not `assert`) in your import statements.

Resolving aliases

Turbopack can be configured to modify module resolution through aliases, similar to webpack's `resolve.alias` configuration.

To configure resolve aliases, map imported patterns to their new destination in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      underscore: 'lodash',
      mocha: { browser: 'mocha/browser-entry.js' },
    },
  },
}
```

This aliases imports of the `underscore` package to the `lodash` package. In other words, `import underscore from 'underscore'` will load the `lodash` module instead of `underscore`.

Turbopack also supports conditional aliasing through this field, similar to Node.js' conditional exports. At the moment only the `browser` condition is supported. In the case above, imports of the `mocha` module will be aliased to `mocha/browser-entry.js` when Turbopack targets browser environments.

Resolving custom extensions

Turbopack can be configured to resolve modules with custom extensions, similar to webpack's `resolve.extensions` configuration.

To configure resolve extensions, use the `resolveExtensions` field in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveExtensions: ['.mdx', '.tsx', '.ts', '.jsx', '.js', '.mjs', '.json'],
  },
}
```

This overwrites the original resolve extensions with the provided list. Make sure to include the default extensions.

For more information and guidance for how to migrate your app to Turbopack from webpack, see Turbopack's documentation on webpack compatibility.

Debug IDs

Turbopack can be configured to generate debug IDs in JavaScript bundles and source maps.

To configure debug IDs, use the `debugIds` field in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    debugIds: true,
  },
}
```

The option automatically adds a polyfill for debug IDs to the JavaScript bundle to ensure compatibility. The debug IDs are available in the `globalThis._debugIds` global variable.

Version History

Version	Changes
`16.2.0`	`turbopackLoader` import attributes were added.
`16.2.0`	`turbopack.rules.*.type` was added.
`16.2.0`	`turbopack.rules.*.condition.contentType` was added.
`16.2.0`	`turbopack.rules.*.condition.query` was added.
`16.0.0`	`turbopack.debugIds` was added.
`16.0.0`	`turbopack.rules.*.condition` was added.
`15.3.0`	`experimental.turbo` is changed to `turbopack`.
`13.0.0`	`experimental.turbo` introduced.

turbopackChunking

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackChunking
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure how Turbopack splits your client-side JavaScript into chunks in production.*

`experimental.turbopackChunking` lets you configure Turbopack's production JavaScript chunker. These options allow you to change the assumptions the chunker makes about user behavior, tweak the raw size thresholds it uses, and enable the experimental component chunks feature.

> 📘 **Nota para Pages Router:** `experimental.turbopackChunking` lets you configure Turbopack's production JavaScript chunker. These options allow you to change the assumptions the chunker makes about user behavior and tweak the raw size thresholds it uses.

By default, Turbopack's chunking is configured as follows:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
      minComponentChunkSize: 20000,
      generateComponentChunks: false,
    },
  },
} satisfies NextConfig

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
      minComponentChunkSize: 20000,
      generateComponentChunks: false,
    },
  },
}

module.exports = nextConfig
```

📘 **Nota para Pages Router:**

```
import type { NextConfig } from 'next'

const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
    },
  },
} satisfies NextConfig

export default nextConfig
```

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
    },
  },
}

module.exports = nextConfig
```

Size Thresholds

The following options control how aggressively Turbopack merges chunks and how large a chunk is allowed to grow. Sizes are in bytes of uncompressed, unminified code (roughly 5x the size of the compressed, minified output).

- `minChunkSize` (default `50000`): Turbopack will avoid creating more than one chunk smaller than this size by merging small chunks into larger ones. Raising this number will produce fewer, larger chunks. Meanwhile, lowering it will produce more chunks that are smaller.
- `maxChunkCountPerGroup` (default `40`): Turbopack will not emit more than this many chunks per chunk group (eg. a route or a dynamic import). Lowering this number will lead to more aggressive merging and less network requests per-page. Raising it will produce smaller chunks and increase the likelihood of cache hits when navigating.
- `maxMergeChunkSize` (default `200000`): Turbopack never merges a chunk larger than this size with other chunks. This keeps the code in large chunks from being duplicated across multiple large output chunks.

When merging chunks, the trade-off being made is an improvement to performance on initial page loads at the cost of navigation performance. This is because each additional network request being made is costly, however, smaller chunks are more likely to be re-usable across pages.

Component Chunks

Producing component chunks is an experimental feature that aims to give you the initial page load benefits of merged chunks without sacrificing reusability. This feature lets the runtime dynamically choose whether to load a merged chunk as a single file, or to load only the component chunks it doesn't already have. Additionally, chunks that were already loaded as part of a merged chunk will not be re-downloaded.

This avoids re-downloading JavaScript the browser already has when navigating.

- `generateComponentChunks` (default `false`): when enabled, each merged production chunk also emits its constituent component chunks alongside it, so the browser runtime can fetch individual component chunks.
- `minComponentChunkSize` (default `20000`): component chunks smaller than this size are folded into a single component instead of being emitted on their own, to avoid producing many tiny chunks.

Heuristics

These change the assumptions the chunker makes when weighing whether merging two chunks is worth it.

- `firstPageLoadPriority` (a number between `0` and `1`): how heavily to weight the benefit of merging chunks for a single page load. Higher values merge more eagerly. If you don't have a better value, your site's bounce rate is a good approximation.
- `priorityRoutes` (an array of `RegExp`): routes that are often the first page a visitor lands on (e.g. the homepage). Their client-side bundles are merged more eagerly to reduce the single-route request cost, at the cost of extra requests when navigating to other pages.
- `priorityBoost` (default `1.5`): a multiplier on the single-request probability of `priorityRoutes` routes. Higher values merge those routes' bundles more aggressively.
- `requestCost` (default `200000`): the estimated cost of an additional request, in bytes of uncompressed, unminified code. Larger values bias toward fewer, larger chunks and fewer requests overall.

Turbopack FileSystem Caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackFileSystemCache
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to enable and configure FileSystem Caching for Turbopack builds*

Usage

Turbopack FileSystem Cache enables Turbopack to reduce work across `next dev` or `next build` commands. When enabled, Turbopack will save and restore data under the `.next` directory between runs, which can greatly speed up subsequent builds and dev sessions.

Two options control the cache, one for `next dev` and one for `next build`. Both are enabled by default:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackFileSystemCacheForDev: true,
    turbopackFileSystemCacheForBuild: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackFileSystemCacheForDev: true,
    turbopackFileSystemCacheForBuild: true,
  },
}

module.exports = nextConfig
```

Options

- `turbopackFileSystemCacheForDev` (default: `true`): caches Turbopack's work for `next dev` in `.next/dev/cache/turbopack`. Restarting the dev server reuses the previous compilation.
- `turbopackFileSystemCacheForBuild` (default: `true`): caches Turbopack's work for `next build` in `.next/cache/turbopack`. Subsequent builds start warm. See Build environments.

Set either option to `false` to opt out.

Build environments

The build cache lives in `.next/cache`. Builds only get faster when that directory is restored before each build.

- **Self-hosted builds**: reuse the same working directory between builds. Containerized builds start from a clean layer and do not carry `.next/cache` over unless you cache or mount it explicitly.
- **CI providers**: configure build caching for `.next/cache`.

If your build environment never preserves `.next/cache`, set `turbopackFileSystemCacheForBuild: false` to skip writing a cache that will not be read.

Version History

Version	Changes
`v16.3.0`	FileSystem caching is enabled by default for builds
`v16.1.0`	FileSystem caching is enabled by default for development
`v16.0.0`	Beta release with separate flags for build and dev
`v15.5.0`	Persistent caching released as experimental on canary releases

---

turbopack.ignoreIssue

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackIgnoreIssue
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Suppress specific Turbopack errors and warnings from the CLI output and error overlay.*

The `turbopack.ignoreIssue` option allows you to filter out specific Turbopack errors and warnings so they do not appear in the CLI output or the error overlay. This is useful for suppressing known warnings that do not affect your application, such as intentionally unresolved optional dependencies.

This option is only available when using Turbopack (`next dev --turbopack`).

Usage

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/vendor/**',
      },
    ],
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/vendor/**',
      },
    ],
  },
}

module.exports = nextConfig
```

Options

Each rule in the `ignoreIssue` array is an object with the following fields:

Field	Type	Required	Description
`path`	`string | RegExp`	Yes	Matches against the file path of the issue
`title`	`string | RegExp`	No	Matches against the issue title
`description`	`string | RegExp`	No	Matches against the issue description

An issue is suppressed when it matches the `path` **and** all other specified fields in a rule. If only `path` is provided, any issue from a matching file is suppressed.

> **Good to know:** Issue titles and descriptions may change between Turbopack versions. The `path` field is generally stable, but is not guaranteed to remain consistent for all issue types. When possible, prefer using more specific `path` patterns over `title` or `description` matching.

`path`

A **glob pattern** (when a string) or **regular expression** that matches against the file path where the issue originated.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      // Glob pattern: suppress issues from any file under vendor/
      { path: '**/vendor/**' },
      // RegExp: suppress issues from files matching a pattern
      { path: /node_modules\/legacy-lib/ },
    ],
  },
}
```

`title`

An **exact string match** (when a string) or **regular expression** that matches against the issue title.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/src/**',
        title: 'Module not found',
      },
    ],
  },
}
```

`description`

An **exact string match** (when a string) or **regular expression** that matches against the issue description.

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/src/**',
        description: /Cannot find module 'optional-dep'/,
      },
    ],
  },
}
```

Examples

Suppressing warnings for optional dependencies

If your code uses `try/catch` around an optional `require()` call, Turbopack may report a "Module not found" warning. You can suppress it:

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/lib/optional-feature/**',
        title: 'Module not found',
      },
    ],
  },
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  turbopack: {
    ignoreIssue: [
      {
        path: '**/lib/optional-feature/**',
        title: 'Module not found',
      },
    ],
  },
}

module.exports = nextConfig
```

Combining multiple rules

You can specify multiple rules to suppress different issues:

```js
// Archivo: next.config.js
module.exports = {
  turbopack: {
    ignoreIssue: [
      { path: '**/vendor/**' },
      { path: '**/legacy/**', title: 'Module not found' },
      { path: /generated\//, description: /expected identifier/ },
    ],
  },
}
```

Version History

Version	Changes
`v16.2.0`	`turbopack.ignoreIssue` introduced.

turbopackLocalPostcssConfig

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackLocalPostcssConfig
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable per-directory PostCSS config resolution in Turbopack so that the config closest to each CSS file takes precedence over the project root config.*

The `turbopackLocalPostcssConfig` option changes how Turbopack resolves `postcss.config.js` files. When enabled, Turbopack searches for the config starting from the CSS file's own directory first, then falls back to the project root. By default, Turbopack checks the project root first, meaning a root-level `postcss.config.js` always takes precedence over configs in subdirectories.

This option is only relevant when using Turbopack (`next dev` or `next build`).

Usage

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackLocalPostcssConfig: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackLocalPostcssConfig: true,
  },
}

module.exports = nextConfig
```

Behavior

Setting	Config resolution order
`false` (default)	Project root → CSS file's directory
`true`	CSS file's directory → project root

With the default behavior, a `postcss.config.js` at the project root is used for all CSS files, and per-directory configs are only applied if no root config exists. Enabling `turbopackLocalPostcssConfig` reverses this: per-directory configs take precedence, and the root config serves as the fallback.

Example

This is useful for projects that need different PostCSS transforms in different directories, such as a monorepo with multiple apps or design system packages:

```
my-app/
├── postcss.config.js          ← fallback (applied if no local config is found)
├── app/
│   └── page.module.css        ← uses root config
└── packages/
    └── ui/
        ├── postcss.config.js  ← takes precedence for files in this directory
        └── button.module.css  ← uses packages/ui/postcss.config.js
```

Version History

Version	Changes
`v16.3.0`	`turbopackLocalPostcssConfig` introduced.

Turbopack Memory Eviction

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackMemoryEviction
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to control Turbopack's memory eviction strategy for the persistent cache.*

Usage

`turbopackMemoryEviction` controls whether Turbopack reclaims memory while the persistent (FileSystem) cache is enabled. After Turbopack writes a snapshot of its cache to disk, it can 'evict' the in-memory copies of that data and reload them from disk on demand.

Currently there are three options

- `false`: never evict. Cached data stays in memory for the lifetime of the process.
- `'auto'` (default): evict after a snapshot only once enough memory has been allocated since the last eviction to make it worthwhile. Leverages thresholds and memory pressure feedback from the operating system.
- `'full'`: evict all possible data from memory every time we save to disk.

> **Good to know:** This option only has an effect in `next dev` sessions when the FileSystem Cache is enabled, since eviction relies on data already being persisted to disk. It is experimental and under active development.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    turbopackMemoryEviction: 'auto',
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    turbopackMemoryEviction: 'auto',
  },
}

module.exports = nextConfig
```

Version Changes

Version	Changes
`v16.3.0`	`turbopackMemoryEviction` released as experimental.

turbopackRustReactCompiler

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopackRustReactCompiler
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use the native Rust port of the React Compiler with Turbopack instead of the Babel transform.*

The `experimental.turbopackRustReactCompiler` option enables the native Rust version of the React Compiler, running it directly inside Turbopack as native code instead of through Node.js as it does with the standard Babel version. This typically results in a noticeable performance improvement.

This option is released as experimental to gather feedback before it becomes the default.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  // Enable the React Compiler
  reactCompiler: true,
  experimental: {
    // Use the Rust port instead of the Babel transform
    turbopackRustReactCompiler: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  // Enable the React Compiler
  reactCompiler: true,
  experimental: {
    // Use the Rust port instead of the Babel transform
    turbopackRustReactCompiler: true,
  },
}

module.exports = nextConfig
```

Good to know

- This option requires `reactCompiler` to be enabled. It selects which implementation runs, but does not turn the compiler on by itself.
- This option is only supported with Turbopack. Using it with webpack will throw an error.
- When enabled, you do not need to install `babel-plugin-react-compiler`. The Rust compiler runs natively inside Turbopack.

See the `reactCompiler` option documentation for details on how to use the compiler.

Version History

Version	Changes
`v16.3.0`	Introduced the experimental `turbopackRustReactCompiler` option for the native Rust React Compiler.

typedRoutes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/typedRoutes
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable support for statically typed links.*

> **Note**: This option has been marked as stable, so you should use `typedRoutes` instead of `experimental.typedRoutes`.

Support for statically typed links. This feature requires using TypeScript in your project.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  typedRoutes: true,
}

module.exports = nextConfig
```

typescript

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/typescript
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure how Next.js handles TypeScript errors during production builds and specify a custom tsconfig file.*

Configure TypeScript behavior with the `typescript` option in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  typescript: {
    ignoreBuildErrors: false,
    tsconfigPath: 'tsconfig.json',
  },
}
```

Options

Option	Type	Default	Description
`ignoreBuildErrors`	`boolean`	`false`	Allow production builds to complete even with TypeScript errors.
`tsconfigPath`	`string`	`'tsconfig.json'`	Path to a custom `tsconfig.json` file.

`ignoreBuildErrors`

Next.js fails your **production build** (`next build`) when TypeScript errors are present in your project.

If you'd like Next.js to dangerously produce production code even when your application has errors, you can disable the built-in type checking step.

Note that this completely skips the TypeScript type checking step. It does not run TypeScript and suppress errors, it bypasses the check entirely.

If disabled, be sure you are running type checks as part of your build or deploy process, otherwise this can be very dangerous.

```
// Archivo: next.config.js
module.exports = {
  typescript: {
    // !! WARN !!
    // Dangerously allow production builds to successfully complete even if
    // your project has type errors.
    // !! WARN !!
    ignoreBuildErrors: true,
  },
}
```

`tsconfigPath`

Use a different TypeScript configuration file for builds or tooling:

```
// Archivo: next.config.js
module.exports = {
  typescript: {
    tsconfigPath: 'tsconfig.build.json',
  },
}
```

See the TypeScript configuration page for more details.

urlImports

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/urlImports
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure Next.js to allow importing modules from external URLs.*

URL imports are an experimental feature that allows you to import modules directly from external servers (instead of from the local disk).

> **Warning**: Only use domains that you trust to download and execute on your machine. Please exercise discretion, and caution until the feature is flagged as stable.

To opt-in, add the allowed URL prefixes inside `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    urlImports: ['https://example.com/assets/', 'https://cdn.skypack.dev'],
  },
}
```

Then, you can import modules directly from URLs:

```
import { a, b, c } from 'https://example.com/assets/some/module.js'
```

URL Imports can be used everywhere normal package imports can be used.

Security Model

This feature is being designed with **security as the top priority**. To start, we added an experimental flag forcing you to explicitly allow the domains you accept URL imports from. We're working to take this further by limiting URL imports to execute in the browser sandbox using the Edge Runtime.

Lockfile

When using URL imports, Next.js will create a `next.lock` directory containing a lockfile and fetched assets. This directory **must be committed to Git**, not ignored by `.gitignore`.

- When running `next dev`, Next.js will download and add all newly discovered URL Imports to your lockfile.
- When running `next build`, Next.js will use only the lockfile to build the application for production.

Typically, no network requests are needed and any outdated lockfile will cause the build to fail. One exception is resources that respond with `Cache-Control: no-cache`. These resources will have a `no-cache` entry in the lockfile and will always be fetched from the network on each build.

Examples

Skypack

```
import confetti from 'https://cdn.skypack.dev/canvas-confetti'
import { useEffect } from 'react'

export default () => {
  useEffect(() => {
    confetti()
  })
  return <p>Hello</p>
}
```

Static Image Imports

```
import Image from 'next/image'
import logo from 'https://example.com/assets/logo.png'

export default () => (
  <div>


![Illustration]()


  </div>
)
```

URLs in CSS

```
.className {
  background: url('https://example.com/assets/hero.jpg');
}
```

Asset Imports

```
const logo = new URL('https://example.com/assets/file.txt', import.meta.url)

console.log(logo.pathname)

// prints "/_next/static/media/file.a9727b5d.txt"
```

---

useLightningcss

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/useLightningcss
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable experimental support for Lightning CSS.*

Experimental support for using Lightning CSS with webpack. Lightning CSS is a fast CSS transformer and minifier, written in Rust.

If this option is not set, Next.js on webpack uses PostCSS with `postcss-preset-env` by default.

Turbopack uses Lightning CSS by default since Next 14.2. This configuration option has no effect on Turbopack. Turbopack always uses Lightning CSS.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    useLightningcss: false, // default, ignored on Turbopack
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    useLightningcss: true, // disables PostCSS on webpack
  },
}

module.exports = nextConfig
```

`lightningCssFeatures`

By default, Lightning CSS decides which CSS features to transpile based on your browserslist targets. The `lightningCssFeatures` option lets you override this by forcing specific features to always be transpiled (`include`) or never be transpiled (`exclude`), regardless of browser support.

This applies to both webpack (when `useLightningcss` is enabled) and Turbopack.

```ts
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    useLightningcss: true,
    lightningCssFeatures: {
      // Always transpile these features, even if targets support them
      include: ['light-dark', 'oklab-colors'],
      // Never transpile these features, even if targets don't support them
      exclude: ['nesting'],
    },
  },
}

export default nextConfig
```

```js
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    useLightningcss: true,
    lightningCssFeatures: {
      // Always transpile these features, even if targets support them
      include: ['light-dark', 'oklab-colors'],
      // Never transpile these features, even if targets don't support them
      exclude: ['nesting'],
    },
  },
}

module.exports = nextConfig
```

Options

Option	Type	Description
`include`	`string[]`	Features to always transpile, regardless of browser targets.
`exclude`	`string[]`	Features to never transpile, even when browser targets would require them.

Available features

Individual features:

Feature name	Description
`nesting`	CSS Nesting
`not-selector-list`	`:not` with multiple selectors
`dir-selector`	`:dir()` selector
`lang-selector-list`	`:lang()` with multiple languages
`is-selector`	`:is()` selector
`text-decoration-thickness-percent`	Percentage values in `text-decoration-thickness`
`media-interval-syntax`	Media query range interval syntax
`media-range-syntax`	Media query range syntax (`width ≥ 600px`)
`custom-media-queries`	`@custom-media` rules
`clamp-function`	`clamp()` function
`color-function`	`color()` function
`oklab-colors`	`oklab()` and `oklch()` colors
`lab-colors`	`lab()` and `lch()` colors
`p3-colors`	Display P3 colors
`hex-alpha-colors`	4 and 8 digit hex colors with alpha
`space-separated-color-notation`	Space-separated color notation (`rgb(0 0 0)`)
`font-family-system-ui`	`system-ui` font family
`double-position-gradients`	Double-position gradient stops
`vendor-prefixes`	Vendor-prefixed properties and values
`logical-properties`	Logical properties and values
`light-dark`	`light-dark()` color function

Composite groups (shorthand for enabling multiple features at once):

Group name	Includes
`selectors`	`nesting` , `not-selector-list` , `dir-selector` , `lang-selector-list` , `is-selector`
`media-queries`	`media-interval-syntax` , `media-range-syntax` , `custom-media-queries`
`colors`	`color-function` , `oklab-colors` , `lab-colors` , `p3-colors` , `hex-alpha-colors` , `space-separated-color-notation` , `light-dark`

Version History

Version	Changes
`16.2.0`	`lightningCssFeatures` added.
`15.1.0`	Support for `useSwcCss` was removed from Turbopack.
`14.2.0`	Turbopack's default CSS processor was changed from `@swc/css` to Lightning CSS. `useLightningcss` became ignored on Turbopack, and a legacy `experimental.turbo.useSwcCss` option was added.

useOffline

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/useOffline
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to enable the experimental `useOffline` configuration option to detect connectivity and retry failed requests automatically.*

The `useOffline` configuration option enables offline connectivity detection and automatic retry of failed navigation, prefetch, and Server Action requests. When enabled, it also exposes the `useOffline` hook for reading the current offline state from Client Components.

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    useOffline: true,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
module.exports = {
  experimental: {
    useOffline: true,
  },
}
```

When enabled, Next.js will:

- Listen for the browser's `offline` and `online` events to track connectivity.
- Detect network failures on navigation, prefetch, and Server Action requests.
- Poll for connectivity using `HEAD` requests with backoff while offline.
- Automatically retry blocked requests once connectivity is restored.
- Make the `useOffline` hook available from `next/offline`.

How retry works

The offline state is entered through one of two paths:

- **Browser event.** Next.js registers a `window.addEventListener('offline', ...)` listener. When the OS reports the network interface as down, the offline state flips on immediately.
- **Failed fetch.** Any navigation, prefetch, or Server Action request whose `fetch()` rejects with a non-abort, non-timeout error calls into the offline module. This catches the case where the browser still reports `navigator.onLine === true` but the actual request cannot reach the origin (captive portal, broken DNS, dead upstream).

Once in the offline state, a polling loop tries to confirm that connectivity has returned.

The connectivity check

Each check issues a single `HEAD` request to the current page's URL with the RSC header set, the same endpoint navigations use. The request is aborted after 200 ms.

Two outcomes count as "online":

1. The fetch resolves normally.

2. The 200 ms timeout aborts the request. A truly offline request fails almost instantly (DNS or TCP error), so if it's still pending at 200 ms the TCP handshake succeeded and the server is reachable.

Any other rejection schedules the next check. A successful framework fetch (navigation, prefetch, Server Action) during the offline period also flips the state back to online.

Backoff

Delays between checks are stepped, not exponential, and capped at 3 seconds:

Attempt	Delay before next check
1	500 ms
2	1 s
3	2 s
4 and after	3 s

The browser's `online` event short-circuits the current wait and runs a connectivity check immediately. Reconnection is detected without waiting for the next scheduled tick.

Giving up

The polling loop never gives up on its own. It continues at the 3-second cap until a check succeeds or the page unloads. A device that goes offline for hours and then regains connectivity will have its polling loop resume and resolve normally.

Retry of framework requests

While the offline state is active, any navigation, prefetch, or Server Action waits for the next connectivity check to succeed, whether it was newly issued or already in flight when the connection dropped. When the check succeeds, the request runs once; no extra backoff applies.

If it fails with a network error, the app re-enters the offline state and the polling loop starts again.

Traffic at reconnection

A single client does not produce a runaway burst of traffic against its origin:

- While the client is offline, a failed `fetch()` rejects locally at the browser's network layer. The request never reaches the origin.
- The polling loop issues one `HEAD` request at a time, with delays capped at 3 seconds. No other framework requests are sent to the origin during the offline period.
- When connectivity returns, each pending navigation and Server Action fires once. Only the last navigation attempt is kept pending, and a typical form button is disabled while its action is pending.
- Prefetches run through the existing prefetch queue, not all at once.

In practice, this feature is unlikely to flood your server. The only extra traffic it generates per offline user is the HEAD polling, which stops as soon as connectivity returns. Prefetches, Server Actions, and navigations fire the same number of times they would have without an outage, just delayed.

Version History

Version	Changes
`v16.x.0`	`experimental.useOffline` configuration option introduced.

useTypeScriptCli

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/useTypeScriptCli
> 📂 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Run the project-local TypeScript CLI for type checking during production builds.*

By default, `next build` runs the project-local `tsc` command instead of loading the TypeScript JavaScript compiler API. This supports TypeScript 6 and enables TypeScript 7 while its JavaScript API is unavailable.

Install TypeScript 7 in your project:

```
# Manejador de paquetes: pnpm
pnpm add -D typescript@^7
```

```
# Manejador de paquetes: npm
npm install -D typescript@^7
```

```
# Manejador de paquetes: yarn
yarn add -D typescript@^7
```

```
# Manejador de paquetes: bun
bun add -D typescript@^7
```

The CLI checker is enabled by default. To use the TypeScript JavaScript compiler API instead, set `experimental.useTypeScriptCli` to `false`:

```
// Archivo: next.config.ts switcher
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    useTypeScriptCli: false,
  },
}

export default nextConfig
```

```
// Archivo: next.config.js switcher
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    useTypeScriptCli: false,
  },
}

module.exports = nextConfig
```

If you opt out while using TypeScript 7, `next build` exits because the TypeScript JavaScript compiler API is unavailable.

Behavior

- Next.js continues to generate `next-env.d.ts` and route types and to apply its recommended `tsconfig` settings before running the checker.
- TypeScript diagnostics are printed directly from `tsc`. Next.js-specific code frames and error rewriting are not applied.

- The complete project selected by the configured `tsconfig` file is checked, including test files and `.next/dev/types` when included. The `--debug-build-paths` option does not limit this set and produces a warning when combined with the CLI checker.
- `typescript.tsconfigPath` selects the project passed to `tsc`.
- `typescript.ignoreBuildErrors` skips the type-checking step, including the CLI checker.

Learn more about using TypeScript 7 with Next.js.

---

webVitalsAttribution

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/webVitalsAttribution
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to use the webVitalsAttribution option to pinpoint the source of Web Vitals issues.*

When debugging issues related to Web Vitals, it is often helpful if we can pinpoint the source of the problem. For example, in the case of Cumulative Layout Shift (CLS), we might want to know the first element that shifted when the single largest layout shift occurred. Or, in the case of Largest Contentful Paint (LCP), we might want to identify the element corresponding to the LCP for the page. If the LCP element is an image, knowing the URL of the image resource can help us locate the asset we need to optimize.

Pinpointing the biggest contributor to the Web Vitals score, aka attribution, allows us to obtain more in-depth information like entries for PerformanceEventTiming, PerformanceNavigationTiming and PerformanceResourceTiming.

Attribution is disabled by default in Next.js but can be enabled **per metric** by specifying the following in `next.config.js`.

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    webVitalsAttribution: ['CLS', 'LCP'],
  },
}
```

Valid attribution values are all `web-vitals` metrics specified in the `NextWebVitalsMetric` type.

Custom Webpack Config

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/next-config-js/webpack
> 📁 **Sección:** App > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to customize the webpack config used by Next.js*

> **Good to know**: changes to webpack config are not covered by semver so proceed at your own risk

Before continuing to add custom webpack configuration to your application make sure Next.js doesn't already support your use-case:

- CSS imports
- CSS modules
- Sass/SCSS imports
- Sass/SCSS modules

> 📘 **Nota para Pages Router:**
>
> - CSS imports
> - CSS modules
> - Sass/SCSS imports
> - Sass/SCSS modules
> - Customizing babel configuration

Some commonly asked for features are available as plugins:

- @next/mdx
- @next/bundle-analyzer

In order to extend our usage of `webpack`, you can define a function that extends its config inside `next.config.js`, like so:

```
// Archivo: next.config.js
module.exports = {
  webpack: (
    config,
    { buildId, dev, isServer, defaultLoaders, nextRuntime, webpack }
  ) => {
    // Important: return the modified config
    return config
  },
}
```

> The `webpack` function is executed three times, twice for the server (nodejs / edge runtime) and once for the client. This allows you to distinguish between client and server configuration using the `isServer` property.

The second argument to the `webpack` function is an object with the following properties:

- `buildId`: `String` - The build id, used as a unique identifier between builds.
- `dev`: `Boolean` - Indicates if the compilation will be done in development.
- `isServer`: `Boolean` - It's `true` for server-side compilation, and `false` for client-side compilation.
- `nextRuntime`: `String | undefined` - The target runtime for server-side compilation; either `"edge"` or `"nodejs"`, it's `undefined` for client-side compilation.
- `defaultLoaders`: `Object` - Default loaders used internally by Next.js:
  - `babel`: `Object` - Default `babel-loader` configuration.

Example usage of `defaultLoaders.babel`:

```
// Example config for adding a loader that depends on babel-loader
// This source was taken from the @next/mdx plugin source:
// https://github.com/vercel/next.js/tree/canary/packages/next-mdx
module.exports = {
  webpack: (config, options) => {
    config.module.rules.push({
      test: /\.mdx/,
      use: [
        options.defaultLoaders.babel,
        {
          loader: '@mdx-js/loader',
          options: pluginOptions.options,
        },
      ],
    })

    return config
  },
}
```

`nextRuntime`

Notice that `isServer` is `true` when `nextRuntime` is `"edge"` or `"nodejs"`, `nextRuntime` `"edge"` is currently for proxy and Server Components in edge runtime only.

TypeScript

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/typescript
> 📂 **Sección:** App > Api Reference > Config
> 📝 **Descripción:** *Next.js provides a TypeScript-first development experience for building your React application.*

Next.js comes with built-in TypeScript, automatically installing the necessary packages and configuring the proper settings when you create a new project with `create-next-app`.

To add TypeScript to an existing project, rename a file to `.ts` / `.tsx`. Run `next dev` and `next build` to automatically install the necessary dependencies and add a `tsconfig.json` file with the recommended config options.

> **Good to know**: If you already have a `jsconfig.json` file, copy the `paths` compiler option from the old `jsconfig.json` into the new `tsconfig.json` file, and delete the old `jsconfig.json` file.

Using TypeScript 7

TypeScript 7 does not currently provide the JavaScript compiler API. To use TypeScript 7 during `next build`, install it in your project:

```
# Manejador de paquetes: pnpm
pnpm add -D typescript@^7
```

```
# Manejador de paquetes: npm
npm install -D typescript@^7
```

```
# Manejador de paquetes: yarn
yarn add -D typescript@^7
```

```
# Manejador de paquetes: bun
bun add -D typescript@^7
```

Next.js uses the project-local `tsc` CLI by default, so no additional configuration is required. To use the JavaScript compiler API instead, set `experimental.useTypeScriptCli` to `false`.

> **Good to know**:
>
> - CLI type checking prints the native `tsc` diagnostics. It does not apply Next.js-specific code frames or rewrite errors for routes, pages, layouts, or route handlers.
> - The CLI checks the complete project selected by your `tsconfig` file. This includes test files and `.next/dev/types` when they are included by that configuration. `next build --debug-build-paths` does not narrow the files that are type checked and produces a warning when used with this option.
> - `typescript.tsconfigPath` continues to select the configuration passed to `tsc`. `typescript.ignoreBuildErrors` skips the type-checking step, including the CLI checker.
> - `experimental.useTypeScriptCli` is experimental and its behavior may change.

IDE Plugin

Next.js includes a custom TypeScript plugin and type checker, which VSCode and other code editors can use for advanced type-checking and auto-completion.

You can enable the plugin in VS Code by:

1. Opening the command palette (`Ctrl/⌘` + `Shift` + `P`)
2. Searching for "TypeScript: Select TypeScript Version"
3. Selecting "Use Workspace Version"

TypeScript Command Palette

Now, when editing files, the custom plugin will be enabled. By default, the project-local `tsc` CLI is used when running `next build` . Set `experimental.useTypeScriptCli` to `false` to use the custom type checker instead.

The TypeScript plugin can help with:

- Warning if invalid values for segment config options are passed.
- Showing available options and in-context documentation.
- Ensuring the `'use client'` directive is used correctly.
- Ensuring client hooks (like `useState`) are only used in Client Components.

> 🎥 **Watch:** Learn about the built-in TypeScript plugin → YouTube (3 minutes)

End-to-End Type Safety

The Next.js App Router has **enhanced type safety**. This includes:

1. **No serialization of data between fetching function and page**: You can `fetch` directly in components, layouts, and pages on the server. This data *does not* need to be serialized (converted to a string) to be passed to the client side for consumption in React. Instead, since `app` uses Server Components by default, we can use values like `Date` , `Map` , `Set` , and more without any extra steps. Previously, you needed to manually type the boundary between server and client with Next.js-specific types.
2. **Streamlined data flow between components**: With the removal of `_app` in favor of root layouts, it is now easier to visualize the data flow between components and pages. Previously, data flowing between individual `pages` and `_app` were difficult to type and could introduce confusing bugs. With colocated data fetching in the App Router, this is no longer an issue.

Data Fetching in Next.js now provides as close to end-to-end type safety as possible without being prescriptive about your database or content provider selection.

We're able to type the response data as you would expect with normal TypeScript. For example:

```
// Archivo: app/page.tsx switcher
async function getData() {
  const res = await fetch('https://api.example.com/...')
  // The return value is *not* serialized
  // You can return Date, Map, Set, etc.
  return res.json()
}

export default async function Page() {
  const name = await getData()

  return '...'
}
```

For *complete* end-to-end type safety, this also requires your database or content provider to support TypeScript. This could be through using an ORM or type-safe query builder.

Route-Aware Type Helpers

Next.js generates global helpers for App Router route types. These are available without imports and are generated during `next dev`, `next build`, or via `next typegen`:

- `PageProps`
- `LayoutProps`
- `RouteContext`

`next-env.d.ts`

Next.js generates a `next-env.d.ts` file in your project root. This file references Next.js type definitions, allowing TypeScript to recognize non-code imports (images, stylesheets, etc.) and Next.js-specific types.

Running `next dev`, `next build`, or `next typegen` regenerates this file.

> **Good to know**:
>
> - `next-env.d.ts` is managed by Next.js. Its contents are an implementation detail and may change over time. Add it to `.gitignore`. If your project already tracks the file, remove it from Git. Do not edit this file manually.
> - The file must be in your `tsconfig.json` `include` array (`create-next-app` does this automatically).

Examples

Type Checking Next.js Configuration Files

You can use TypeScript and import types in your Next.js configuration by using `next.config.ts`.

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig =

export default nextConfig
```

Module resolution in `next.config.ts` is currently limited to CommonJS. However, ECMAScript Modules (ESM) syntax is available when using Node.js native TypeScript resolver for Node.js v22.10.0 and higher.

When using the `next.config.js` file, you can add some type checking in your IDE using JSDoc as below:

```
// Archivo: next.config.js
// @ts-check

/** @type {import('next').NextConfig} */
const nextConfig =

module.exports = nextConfig
```

Using Node.js Native TypeScript Resolver for `next.config.ts`

> **Note**: Available on Node.js v22.10.0+ and only when the feature is enabled. Next.js does not enable it.

Next.js detects the Node.js native TypeScript resolver via `process.features.typescript`, added in **v22.10.0**. When present, `next.config.ts` can use native ESM, including top-level `await` and dynamic `import()`. This mechanism inherits the capabilities and limitations of Node's resolver.

In Node.js versions **v22.18.0+**, `process.features.typescript` is enabled by default. For versions between **v22.10.0** and **22.17.x**, opt in with `NODE_OPTIONS=--experimental-transform-types`:

```
// Archivo: Terminal
NODE_OPTIONS=--experimental-transform-types next <command>
```

For CommonJS Projects (Default)

Although `next.config.ts` supports native ESM syntax in CommonJS projects, Node.js will still assume `next.config.ts` is a CommonJS file by default, resulting in Node.js reparsing the file as ESM when module syntax is detected. Therefore, we recommend using the `next.config.mts` file for CommonJS projects to explicitly indicate it's an ESM module:

```
// Archivo: next.config.mts
import type { NextConfig } from 'next'

// Top-level await and dynamic import are supported
const flags = await import('./flags.js').then((m) => m.default ?? m)

const nextConfig: NextConfig =

```json
// Archivo: tsconfig.json highlight={4}
{
  "include": [
    "next-env.d.ts",
    ".next/types/**/*.ts",
    "**/*.ts",
    "**/*.tsx"
  ],
  "exclude": ["node_modules"]
}
```

Currently, support includes any string literal, including dynamic segments. For non-literal strings, you need to manually cast with `as Route`. The example below shows both `next/link` and `next/navigation` usage:

```
// Archivo: app/example-client.tsx
'use client'

import type { Route } from 'next'
import Link from 'next/link'
import { useRouter } from 'next/navigation'

export default function Example() {
  const router = useRouter()
  const slug = 'nextjs'

  return (
    <>

      <Link href="/about" />
      <Link href={`/blog/${slug}`} />
      <Link href={('/blog/' + slug) as Route} />

      <Link href="/aboot" />


      <button onClick={() => router.push('/about')}>Push About</button>
      <button onClick={() => router.replace(`/blog/${slug}`)}>
        Replace Blog
      </button>
      <button onClick={() => router.prefetch('/contact')}>
        Prefetch Contact
      </button>


      <button onClick={() => router.push(('/blog/' + slug) as Route)}>
        Push Non-literal Blog
      </button>
    </>
  )
}
```
```

The same applies for redirecting routes defined by proxy:

```
// Archivo: proxy.ts
import { NextRequest, NextResponse } from 'next/server'

export function proxy(request: NextRequest) {
  if (request.nextUrl.pathname === '/proxy-redirect') {
    return NextResponse.redirect(new URL('/', request.url))
  }

  return NextResponse.next()
}
```

```
// Archivo: app/some/page.tsx
import type { Route } from 'next'
import Link from 'next/link'

export default function Page() {
  return <Link href={'/proxy-redirect' as Route}>Link Text</Link>
}
```

To accept `href` in a custom component wrapping `next/link`, use a generic:

```
import type { Route } from 'next'
import Link from 'next/link'

function Card<T extends string>({ href }: { href: Route<T> | URL }) {
  return (
    <Link href={href}>
      <div>My Card</div>
    </Link>
  )
}
```

You can also type a simple data structure and iterate to render links:

```
// Archivo: components/nav-items.ts
import type { Route } from 'next'

type NavItem<T extends string = string> = {
  href: T
  label: string
}

export const navItems: NavItem<Route>[] = [
  { href: '/', label: 'Home' },
  { href: '/about', label: 'About' },
  { href: '/blog', label: 'Blog' },
]
```

Then, map over the items to render `Link`s:

```
// Archivo: components/nav.tsx
import Link from 'next/link'
import { navItems } from './nav-items'

export function Nav() {
  return (
    <nav>
      {navItems.map((item) => (
        <Link key={item.href} href={item.href}>
          {item.label}
        </Link>
      ))}
    </nav>
  )
}
```

How does it work?

When running, `next typegen`, `next dev` or `next build`, Next.js generates a hidden `.d.ts` file inside `.next` that contains information about all existing routes in your application (all valid routes as the `href` type of `Link`). This `.d.ts` file is included in `tsconfig.json` and the TypeScript compiler will check that `.d.ts` and provide feedback in your editor about invalid links.

Type IntelliSense for Environment Variables

During development, Next.js generates a `.d.ts` file in `.next/types` that contains information about the loaded environment variables for your editor's IntelliSense. If the same environment variable key is defined in multiple files, it is deduplicated according to the Environment Variable Load Order.

To opt-into this feature, `experimental.typedEnv` needs to be enabled and the project needs to be using TypeScript.

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  experimental: {
    typedEnv: true,
  },
}

export default nextConfig
```

Good to know: Types are generated based on the environment variables loaded at development runtime, which excludes variables from `.env.production*` files by default. To include production-specific variables, run `next dev` with `NODE_ENV=production`.

With Async Server Components

To use an `async` Server Component with TypeScript, ensure you are using TypeScript `5.1.3` or higher and `@types/react` `18.2.8` or higher.

If you are using an older version of TypeScript, you may see a `'Promise<Element>' is not a valid JSX element` type error. Updating to the latest version of TypeScript and `@types/react` should resolve this issue.

📘 **Nota para Pages Router:**

Static Generation and Server-side Rendering

For `getStaticProps`, `getStaticPaths`, and `getServerSideProps`, you can use the `GetStaticProps`, `GetStaticPaths`, and `GetServerSideProps` types respectively:

```
import type { GetStaticProps, GetStaticPaths, GetServerSideProps } from 'next'

export const getStaticProps = (async (context) => {
  // ...
}) satisfies GetStaticProps

export const getStaticPaths = (async () => {
  // ...
}) satisfies GetStaticPaths

export const getServerSideProps = (async (context) => {
  // ...
}) satisfies GetServerSideProps
```

> **Good to know:** `satisfies` was added to TypeScript in 4.9. We recommend upgrading to the latest version of TypeScript.

With API Routes

The following is an example of how to use the built-in types for API routes:

```
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  res.status(200).json({ name: 'John Doe' })
}
```

You can also type the response data:

```
import type { NextApiRequest, NextApiResponse } from 'next'

type Data = {
  name: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
) {
  res.status(200).json({ name: 'John Doe' })
}
```

With custom `App`

If you have a custom `App`, you can use the built-in type `AppProps` and change file name to `./pages/_app.tsx` like so:

```
import type { AppProps } from 'next/app'

export default function MyApp({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />
}
```

Incremental type checking

Since `v10.2.1` Next.js supports incremental type checking when enabled in your `tsconfig.json`, this can help speed up type checking in larger applications.

Custom `tsconfig` path

In some cases, you might want to use a different TypeScript configuration for builds or tooling. To do that, set `typescript.tsconfigPath` in `next.config.ts` to point Next.js to another `tsconfig` file.

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  typescript: {
    tsconfigPath: 'tsconfig.build.json',
  },
}

export default nextConfig
```

For example, switch to a different config for production builds:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const isProd = process.env.NODE_ENV === 'production'

const nextConfig: NextConfig = {
  typescript: {
    tsconfigPath: isProd ? 'tsconfig.build.json' : 'tsconfig.json',
  },
}

export default nextConfig
```

Why you might use a separate `tsconfig` for builds

> **Good to know**:
>
> - IDEs typically read `tsconfig.json` for diagnostics and IntelliSense, so you can still see IDE warnings while production builds use the alternate config. Mirror critical options if you want parity in the editor.
> - In development, only `tsconfig.json` is watched for changes. If you edit a different file name via `typescript.tsconfigPath`, restart the dev server to apply changes.
> - The configured file is used in `next dev`, `next build`, and `next typegen`.

Disabling TypeScript errors in production

Next.js fails your **production build** (`next build`) when TypeScript errors are present in your project.

If you'd like Next.js to dangerously produce production code even when your application has errors, you can disable the built-in type checking step.

If disabled, be sure you are running type checks as part of your build or deploy process, otherwise this can be very dangerous.

Open `next.config.ts` and enable the `ignoreBuildErrors` option in the `typescript` config:

```
// Archivo: next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  typescript: {
    // !! WARN !!
    // Dangerously allow production builds to successfully complete even if
    // your project has type errors.
    // !! WARN !!
    ignoreBuildErrors: true,
  },
}

export default nextConfig
```

> **Good to know**: You can run `tsc --noEmit` to check for TypeScript errors yourself before building. This is useful for CI/CD pipelines where you'd like to check for TypeScript errors before deploying.

Custom type declarations

When you need to declare custom types, you might be tempted to modify `next-env.d.ts`. However, this file is automatically generated, so any changes you make will be overwritten. Instead, you should create a new file, let's call it `new-types.d.ts`, and reference it in your `tsconfig.json`:

```
// Archivo: tsconfig.json
{
  "compilerOptions": {
    "skipLibCheck": true
    // ... truncated ...
  },
  "include": [
    "new-types.d.ts",
    "next-env.d.ts",
    ".next/types/**/*.ts",
    "**/*.ts",
    "**/*.tsx"
  ],
  "exclude": ["node_modules"]
}
```

Version Changes

Version	Changes
`v15.0.0`	`next.config.ts` support added for TypeScript projects.
`v13.2.0`	Statically typed links are available in beta.
`v12.0.0`	SWC is now used by default to compile TypeScript and TSX for faster builds.
`v10.2.1`	Incremental type checking support added when enabled in your `tsconfig.json`.

ESLint Plugin

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config/eslint
> 📁 **Sección:** App > Api Reference > Config
> 📝 **Descripción:** *Learn how to use and configure the ESLint plugin to catch common issues and problems in a Next.js application.*

Next.js provides an ESLint configuration package, `eslint-config-next`, that makes it easy to catch common issues in your application. It includes the `@next/eslint-plugin-next` plugin along with recommended rule-sets from `eslint-plugin-react` and `eslint-plugin-react-hooks`.

The package provides two main configurations:

- `eslint-config-next`: Base configuration with Next.js, React, and React Hooks rules. Supports both JavaScript and TypeScript files.
- `eslint-config-next/core-web-vitals`: Includes everything from the base config, plus upgrades rules that impact Core Web Vitals from warnings to errors. Recommended for most projects.

Additionally, for TypeScript projects:

- `eslint-config-next/typescript`: Adds TypeScript-specific linting rules from `typescript-eslint`. Use this alongside the base or core-web-vitals config.

Setup ESLint

Get linting working quickly with the ESLint CLI (flat config):

1. Install ESLint and the Next.js config:

```
pnpm add -D eslint eslint-config-next
```

```
npm i -D eslint eslint-config-next
```

```
yarn add --dev eslint eslint-config-next
```

```
bun add -d eslint eslint-config-next
```

2. Create `eslint.config.mjs` with the Next.js config:

```
import { defineConfig, globalIgnores } from 'eslint/config'
import nextVitals from 'eslint-config-next/core-web-vitals'

const eslintConfig = defineConfig([
  ...nextVitals,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    '.next/**',
    'out/**',
    'build/**',
    'next-env.d.ts',
  ]),
])

export default eslintConfig
```

3. Run ESLint:

```
pnpm exec eslint .
```

```
npx eslint .
```

```
yarn eslint .
```

```
bunx eslint .
```

Reference

The `eslint-config-next` package includes the `recommended` rule-sets from the following ESLint plugins:

- `eslint-plugin-react`
- `eslint-plugin-react-hooks`
- `@next/eslint-plugin-next`

Rules

The `@next/eslint-plugin-next` rules included are:

Enabled in recommended config	Rule	Description
	@next/next/google-font-display	Enforce font-display behavior with Google Fonts.
	@next/next/google-font-preconnect	Ensure `preconnect` is used with Google Fonts.
	@next/next/inline-script-id	Enforce `id` attribute on `next/script` components with inline content.
	@next/next/next-script-for-ga	Prefer `next/script` component when using the inline script for Google Analytics.
	@next/next/no-assign-module-variable	Prevent assignment to the `module` variable.
	@next/next/no-async-client-component	Prevent Client Components from being async functions.
	@next/next/no-before-interactive-script-outside-document	Prevent usage of `next/script`'s `beforeInteractive` strategy outside of `pages/_document.js`.
	@next/next/no-css-tags	Prevent manual stylesheet tags.
	@next/next/no-document-import-in-page	Prevent importing `next/document` outside of `pages/_document.js`.
	@next/next/no-duplicate-head	Prevent duplicate usage of `<Head>` in `pages/_document.js`.
	@next/next/no-head-element	Prevent usage of `<head>` element.
	@next/next/no-head-import-in-document	Prevent usage of `next/head` in `pages/_document.js`.
	@next/next/no-html-link-for-pages	Prevent usage of `<a>` elements to navigate to internal Next.js pages.
	@next/next/no-img-element	Prevent usage of `<img>` element due to slower LCP and higher bandwidth.
	@next/next/no-page-custom-font	Prevent page-only custom fonts.
	@next/next/no-script-component-in-head	Prevent usage of `next/script` in `next/head` component.
	@next/next/no-styled-jsx-in-document	Prevent usage of `styled-jsx` in `pages/_document.js`.
	@next/next/no-sync-scripts	Prevent synchronous scripts.
	@next/next/no-title-in-document-head	Prevent usage of `<title>` with `Head` component from `next/document`.
	@next/next/no-typos	Prevent common typos in Next.js's data fetching functions
	@next/next/no-unwanted-polyfillio	Prevent duplicate polyfills from Polyfill.io.

We recommend using an appropriate integration to view warnings and errors directly in your code editor during development.

`next lint` removal

Examples

Specifying a root directory within a monorepo

If you're using `@next/eslint-plugin-next` in a project where Next.js isn't installed in your root directory (such as a monorepo), you can tell `@next/eslint-plugin-next` where to find your Next.js application using the `settings` property in your `eslint.config.mjs`:

```
// Archivo: eslint.config.mjs
import { defineConfig } from 'eslint/config'
import eslintNextPlugin from '@next/eslint-plugin-next'

const eslintConfig = defineConfig([
  {
    files: ['**/*.{js,jsx,ts,tsx}'],
    plugins: {
      '@next/next': eslintNextPlugin,
    },
    settings: {
      next: {
        rootDir: 'packages/my-app/',
      },
    },
  },
])

export default eslintConfig
```

`rootDir` can be a path (relative or absolute), a glob (i.e. `"packages/*/"`), or an array of paths and/or globs.

Disabling rules

If you would like to modify or disable any rules provided by the supported plugins (`react`, `react-hooks`, `next`), you can directly change them using the `rules` property in your `eslint.config.mjs`:

```
// Archivo: eslint.config.mjs
import { defineConfig, globalIgnores } from 'eslint/config'
import nextVitals from 'eslint-config-next/core-web-vitals'

const eslintConfig = defineConfig([
  ...nextVitals,
  {
    rules: {
      'react/no-unescaped-entities': 'off',
      '@next/next/no-page-custom-font': 'off',
    },
  },
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    '.next/**',
    'out/**',
    'build/**',
    'next-env.d.ts',
  ]),
])

export default eslintConfig
```

With Core Web Vitals

Enable the `eslint-config-next/core-web-vitals` configuration in your ESLint config.

```
// Archivo: eslint.config.mjs
import { defineConfig, globalIgnores } from 'eslint/config'
import nextVitals from 'eslint-config-next/core-web-vitals'

const eslintConfig = defineConfig([
  ...nextVitals,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    '.next/**',
    'out/**',
    'build/**',
    'next-env.d.ts',
  ]),
])

export default eslintConfig
```

`eslint-config-next/core-web-vitals` upgrades certain lint rules in `@next/eslint-plugin-next` from warnings to errors to help improve your Core Web Vitals metrics.

> The `eslint-config-next/core-web-vitals` configuration is automatically included for new applications built with Create Next App.

With TypeScript

In addition to the Next.js ESLint rules, `create-next-app --typescript` will also add TypeScript-specific lint rules with `eslint-config-next/typescript` to your config:

```
// Archivo: eslint.config.mjs
import { defineConfig, globalIgnores } from 'eslint/config'
import nextVitals from 'eslint-config-next/core-web-vitals'
import nextTs from 'eslint-config-next/typescript'

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    '.next/**',
    'out/**',
    'build/**',
    'next-env.d.ts',
  ]),
])

export default eslintConfig
```

Those rules are based on `plugin:@typescript-eslint/recommended`. See typescript-eslint > Configs for more details.

With Prettier

ESLint also contains code formatting rules, which can conflict with your existing Prettier setup. We recommend including eslint-config-prettier in your ESLint config to make ESLint and Prettier work together.

First, install the dependency:

```
# Manejador de paquetes: pnpm
pnpm add -D eslint-config-prettier
```

```
# Manejador de paquetes: npm
npm i -D eslint-config-prettier
```

```
# Manejador de paquetes: yarn
yarn add --dev eslint-config-prettier
```

```
# Manejador de paquetes: bun
bun add -d eslint-config-prettier
```

Then, add `prettier` to your existing ESLint config:

```
// Archivo: eslint.config.mjs
import { defineConfig, globalIgnores } from 'eslint/config'
import nextVitals from 'eslint-config-next/core-web-vitals'
import prettier from 'eslint-config-prettier/flat'

const eslintConfig = defineConfig([
  ...nextVitals,
  prettier,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    '.next/**',
    'out/**',
    'build/**',
    'next-env.d.ts',
  ]),
])

export default eslintConfig
```

Running lint on staged files

If you would like to use ESLint with lint-staged to run the linter on staged git files, add the following to the `.lintstagedrc.js` file in the root of your project:

```
// Archivo: .lintstagedrc.js
const path = require('path')

const buildEslintCommand = (filenames) =>
  `eslint --fix ${filenames
    .map((f) => `"${path.relative(process.cwd(), f)}"`)
    .join(' ')}`

module.exports = {
  '*.{js,jsx,ts,tsx}': [buildEslintCommand],
}
```

Migrating existing config

If you already have ESLint configured in your application, there are two approaches to integrate Next.js linting rules, depending on your setup.

Using the plugin directly

Use `@next/eslint-plugin-next` directly if you have any of the following already configured:

- Conflicting plugins installed separately or through another config (such as `airbnb` or `react-app`):
  - `react`
  - `react-hooks`
  - `jsx-a11y`
  - `import`
- Custom `parserOptions` different from Next.js defaults (only if you have customized your Babel configuration)
- `eslint-plugin-import` with custom Node.js and/or TypeScript resolvers

In these cases, use `@next/eslint-plugin-next` directly to avoid conflicts:

First, install the plugin:

```
# Manejador de paquetes: pnpm
pnpm add -D @next/eslint-plugin-next
```

```
# Manejador de paquetes: npm
npm i -D @next/eslint-plugin-next
```

```
# Manejador de paquetes: yarn
yarn add --dev @next/eslint-plugin-next
```

```
# Manejador de paquetes: bun
bun add -d @next/eslint-plugin-next
```

Then add it to your ESLint config:

```
// Archivo: eslint.config.mjs
import { defineConfig } from 'eslint/config'
import nextPlugin from '@next/eslint-plugin-next'

const eslintConfig = defineConfig([
  // Your other configurations ...
  {
    files: ['**/*.{js,jsx,ts,tsx}'],
    plugins: {
      '@next/next': nextPlugin,
    },
    rules: {
      ... nextPlugin.configs.recommended.rules,
    },
  },
])

export default eslintConfig
```

This approach eliminates the risk of collisions or errors that can occur when the same plugins or parsers are imported across multiple configurations.

Adding to existing config

If you're adding Next.js to an existing ESLint setup, spread the Next.js config into your array:

```
// Archivo: eslint.config.mjs
import nextConfig from 'eslint-config-next/core-web-vitals'
// Your other config imports ...

const eslintConfig = [
  // Your other configurations ...
  ... nextConfig,
]

export default eslintConfig
```

When you spread `... nextConfig`, you're adding multiple config objects that include file patterns, plugins, rules, ignores, and parser settings. ESLint applies configs in order, so later rules can override earlier ones for matching files.

> **Good to know:** This approach works well for straightforward setups. If you have a complex existing config with specific file patterns or plugin configurations that conflict, consider using the plugin directly (as shown above) for more granular control.

Version	Changes
`v16.0.0`	`next lint` and the `eslint` next.config.js option were removed in favor of the ESLint CLI. A codemod is available to help you migrate.

Configuration

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/config
> 📁 **Sección:** App > Api Reference > Config
> 📝 **Descripción:** *Learn how to configure Next.js applications.*

create-next-app

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/cli/create-next-app
> 📁 **Sección:** App > Api Reference > Cli
> 📝 **Descripción:** *Create Next.js apps using one command with the create-next-app CLI.*

The `create-next-app` CLI allow you to create a new Next.js application using the default template or an example from a public GitHub repository. It is the easiest way to get started with Next.js.

Basic usage:

```
# Manejador de paquetes: pnpm
pnpm create next-app [project-name] [options]
```

```
# Manejador de paquetes: npm
npx create-next-app@latest [project-name] [options]
```

```
# Manejador de paquetes: yarn
yarn create next-app [project-name] [options]
```

```
# Manejador de paquetes: bun
bun create next-app [project-name] [options]
```

Reference

The following options are available:

Options	Description
`-h` or `--help`	Show all available options
`-v` or `--version`	Output the version number
`--no-*`	Negate default options. E.g. `--no-ts`
`--ts` or `--typescript`	Initialize as a TypeScript project (default)
`--js` or `--javascript`	Initialize as a JavaScript project
`--tailwind`	Initialize with Tailwind CSS config (default)
`--react-compiler`	Initialize with React Compiler enabled
`--eslint`	Initialize with ESLint config
`--biome`	Initialize with Biome config
`--no-linter`	Skip linter configuration
`--app`	Initialize as an App Router project
`--api`	Initialize a project with only route handlers
`--src-dir`	Initialize inside a `src/` directory
`--turbopack`	Force enable Turbopack in generated package.json (enabled by default)
`--webpack`	Force enable Webpack in generated package.json
`--import-alias <alias-to-configure>`	Specify import alias to use (default "@/*")
`--empty`	Initialize an empty project
`--use-npm`	Explicitly tell the CLI to bootstrap the application using npm
`--use-pnpm`	Explicitly tell the CLI to bootstrap the application using pnpm
`--use-yarn`	Explicitly tell the CLI to bootstrap the application using Yarn
`--use-bun`	Explicitly tell the CLI to bootstrap the application using Bun
`-e` or `--example [name] [github-url]`	An example to bootstrap the app with
`--example-path <path-to-example>`	Specify the path to the example separately
`--reset-preferences`	Explicitly tell the CLI to reset any stored preferences
`--skip-install`	Explicitly tell the CLI to skip installing packages
`--disable-git`	Explicitly tell the CLI to disable git initialization
`--agents-md`	Include `AGENTS.md` and `CLAUDE.md` to guide coding agents (default)
`--yes`	Use previous preferences or defaults for all options

Examples

With the default template

To create a new app using the default template, run the following command in your terminal:

```
# Manejador de paquetes: pnpm
pnpm create next-app
```

```
# Manejador de paquetes: npm
npx create-next-app@latest
```

```
# Manejador de paquetes: yarn
yarn create next-app
```

```
# Manejador de paquetes: bun
bun create next-app
```

On installation, you'll see the following prompts:

```
// Archivo: Terminal
What is your project named? my-app
Would you like to use the recommended Next.js defaults?
    Yes, use recommended defaults - TypeScript, ESLint, Tailwind CSS, App Router, AGENTS.md
    No, reuse previous settings
    No, customize settings - Choose your own preferences
```

If you choose to `customize settings`, you'll see the following prompts:

```
// Archivo: Terminal
Would you like to use TypeScript? No / Yes
Which linter would you like to use? ESLint / Biome / None
Would you like to use React Compiler? No / Yes
Would you like to use Tailwind CSS? No / Yes
Would you like your code inside a `src/` directory? No / Yes
Would you like to use App Router? (recommended) No / Yes
Would you like to customize the import alias (`@/*` by default)? No / Yes
What import alias would you like configured? @/*
Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? No / Yes
```

After the prompts, `create-next-app` will create a folder with your project name and install the required dependencies.

Linter Options

ESLint: The traditional and most popular JavaScript linter. Includes Next.js-specific rules from `@next/eslint-plugin-next`.

Biome: A fast, modern linter and formatter that combines the functionality of ESLint and Prettier. Includes built-in Next.js and React domain support for optimal performance.

None: Skip linter configuration entirely. You can always add a linter later.

Once you've answered the prompts, a new project will be created with your chosen configuration.

With an official Next.js example

To create a new app using an official Next.js example, use the `--example` flag. For example:

```
# Manejador de paquetes: pnpm
pnpm create next-app --example [example-name] [your-project-name]
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example [example-name] [your-project-name]
```

```
# Manejador de paquetes: yarn
yarn create next-app --example [example-name] [your-project-name]
```

```
# Manejador de paquetes: bun
bun create next-app --example [example-name] [your-project-name]
```

You can view a list of all available examples along with setup instructions in the Next.js repository.

With any public GitHub example

To create a new app using any public GitHub example, use the `--example` option with the GitHub repository's URL. For example:

```
# Manejador de paquetes: pnpm
pnpm create next-app --example "https://github.com/ ... /" [your-project-name]
```

```
# Manejador de paquetes: npm
npx create-next-app@latest --example "https://github.com/ ... /" [your-project-name]
```

```
# Manejador de paquetes: yarn
yarn create next-app --example "https://github.com/ ... /" [your-project-name]
```

```
# Manejador de paquetes: bun
bun create next-app --example "https://github.com/ ... /" [your-project-name]
```

CLI

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/cli
> 📂 **Sección:** App > Api Reference > Cli
> 📝 **Descripción:** *API Reference for the Next.js Command Line Interface (CLI) tools.*

Next.js comes with **two** Command Line Interface (CLI) tools:

- `create-next-app` : Quickly create a new Next.js application using the default template or an example from a public GitHub repository.
- `next` : Run the Next.js development server, build your application, and more.

---

next CLI

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/cli/next
> 📂 **Sección:** App > Api Reference > Cli
> 📝 **Descripción:** *Learn how to run and build your application with the Next.js CLI.*

The Next.js CLI allows you to develop, build, start your application, and more.

Basic usage:

```
# Manejador de paquetes: pnpm
pnpm next [command] [options]
```

```
# Manejador de paquetes: npm
npx next [command] [options]
```

```
# Manejador de paquetes: yarn
yarn next [command] [options]
```

```
# Manejador de paquetes: bun
bunx next [command] [options]
```

> **Good to know**: With `npm run`, use `--` before CLI flags so npm forwards them to `next`. This is not required for `pnpm`, `yarn`, or `bun`.

Reference

The following options are available:

Options	Description
`-h` or `--help`	Shows all available options
`-v` or `--version`	Outputs the Next.js version number

Commands

The following commands are available:

Command	Description
`dev`	Starts Next.js in development mode with Hot Module Reloading, error reporting, and more.
`build`	Creates an optimized production build of your application. Displaying information about each route.
`start`	Starts Next.js in production mode. The application should be compiled with `next build` first.
`info`	Prints relevant details about the current system which can be used to report Next.js bugs.
`telemetry`	Allows you to enable or disable Next.js' completely anonymous telemetry collection.
`typegen`	Generates TypeScript definitions for routes, pages, layouts, and route handlers without running a full build.
`upgrade`	Upgrades your Next.js application to the latest version.
`experimental-analyze`	Analyzes bundle output using Turbopack. Does not produce build artifacts.

> **Good to know**: Running `next` without a command is an alias for `next dev`.

`next dev` options

`next dev` starts the application in development mode with Hot Module Reloading (HMR), error reporting, and more.

> **Good to know**: Development builds output to `.next/dev` instead of `.next`. This allows you to run `next dev` and `next build` concurrently without conflicts.

The following options are available when running `next dev`:

Option	Description
`-h, --help`	Show all available options.
`[directory]`	A directory in which to build the application. If not provided, current directory is used.
`--turbopack`	Force enable Turbopack (enabled by default). Also available as `--turbo`.
`--webpack`	Use Webpack instead of the default Turbopack bundler for development.
`-p` or `--port <port>`	Specify a port number on which to start the application. Default: 3000, env: PORT
`-H` or `--hostname <hostname>`	Specify a hostname on which to start the application. Useful for making the application available for other devices on the network. Default: 0.0.0.0
`--experimental-https`	Starts the server with HTTPS and generates a self-signed certificate.
`--experimental-https-key <path>`	Path to a HTTPS key file.
`--experimental-https-cert <path>`	Path to a HTTPS certificate file.
`--experimental-https-ca <path>`	Path to a HTTPS certificate authority file.
`--experimental-upload-trace <traceUrl>`	Reports a subset of the debugging trace to a remote HTTP URL.
`--experimental-cpu-prof`	Enables CPU profiling using V8's inspector. Profiles are saved to `.next-profiles/` on exit.

`next build` options

`next build` creates an optimized production build of your application. The output displays information about each route. For example:

```
// Archivo: Terminal
Route (app)
┌ ○ /_not-found
└ ƒ /products/[id]

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

See the Building guide to learn how to read the output, prerender dynamic routes, and debug build errors.

The following options are available for the `next build` command:

Option	Description
`-h, --help`	Show all available options.
`[directory]`	A directory on which to build the application. If not provided, the current directory will be used.
`--turbopack`	Force enable Turbopack (enabled by default). Also available as `--turbo`.
`--webpack`	Build using Webpack.
`-d` or `--debug`	Enables a more verbose build output. With this flag enabled additional build output like rewrites, redirects, and headers will be shown.
`--profile`	Enables production profiling for React.
`--no-mangling`	Disables mangling. This may affect performance and should only be used for debugging purposes.
`--experimental-app-only`	Builds only App Router routes.
`--experimental-build-mode [mode]`	Uses an experimental build mode. (choices: "compile", "generate", default: "default")
`--debug-prerender`	Debug prerender errors in development.
`--debug-build-paths=<patterns>`	Build only specific routes for debugging.
`--experimental-cpu-prof`	Enables CPU profiling using V8's inspector. Profiles are saved to `.next-profiles/` on exit.

`next start` options

`next start` starts the application in production mode. The application should be compiled with `next build` first.

The following options are available for the `next start` command:

Option	Description
`-h` or `--help`	Show all available options.
`[directory]`	A directory on which to start the application. If no directory is provided, the current directory will be used.
`-p` or `--port <port>`	Specify a port number on which to start the application. (default: 3000, env: PORT)
`-H` or `--hostname <hostname>`	Specify a hostname on which to start the application (default: 0.0.0.0).
`--keepAliveTimeout <keepAliveTimeout>`	Specify the maximum amount of milliseconds to wait before closing the inactive connections.
`--experimental-cpu-prof`	Enables CPU profiling using V8's inspector. Profiles are saved to `.next-profiles/` on exit.

`next info` options

`next info` prints relevant details about the current system which can be used to report Next.js bugs when opening a GitHub issue. This information includes Operating System platform/arch/version, Binaries (Node.js, npm, Yarn, pnpm), package versions (`next`, `react`, `react-dom`), and more.

The output should look like this:

```
// Archivo: Terminal
Operating System:
  Platform: darwin
  Arch: arm64
  Version: Darwin Kernel Version 23.6.0
  Available memory (MB): 65536
  Available CPU cores: 10
Binaries:
  Node: 20.12.0
  npm: 10.5.0
  Yarn: 1.22.19
  pnpm: 9.6.0
Relevant Packages:
  next: 15.0.0-canary.115 // Latest available version is detected (15.0.0-canary.115).
  eslint-config-next: 14.2.5
  react: 19.0.0-rc
  react-dom: 19.0.0
  typescript: 5.5.4
Next.js Config:
  output: N/A
```

The following options are available for the `next info` command:

Option	Description
`-h` or `--help`	Show all available options
`--verbose`	Collects additional information for debugging.

`next telemetry` options

Next.js collects **completely anonymous** telemetry data about general usage. Participation in this anonymous program is optional, and you can opt-out if you prefer not to share information.

The following options are available for the `next telemetry` command:

Option	Description
`-h, --help`	Show all available options.
`--enable`	Enables Next.js' telemetry collection.
`--disable`	Disables Next.js' telemetry collection.

Learn more about Telemetry.

`next typegen` options

`next typegen` generates TypeScript definitions for your application's routes without performing a full build. This is useful for IDE autocomplete and CI type-checking of route usage.

Previously, route types were only generated during `next dev` or `next build`, which meant running `tsc --noEmit` directly wouldn't validate your route types. Now you can generate types independently and validate them externally:

```
// Archivo: Terminal
# Generate route types first, then validate with TypeScript
next typegen && tsc --noEmit

# Or in CI workflows for type checking without building
next typegen && npm run type-check
```

The following options are available for the `next typegen` command:

Option	Description
`-h, --help`	Show all available options.
`[directory]`	A directory on which to generate types. If not provided, the current directory will be used.

Output files are written to `<distDir>/types` (typically: `.next/dev/types` in development or `.next/types` in production):

```
// Archivo: Terminal
next typegen
# or for a specific app
next typegen ./apps/web
```

Additionally, `next typegen` generates a `next-env.d.ts` file. We recommend adding `next-env.d.ts` to your `.gitignore` file.

The `next-env.d.ts` file is included into your `tsconfig.json` file, to make Next.js types available to your project.

To ensure `next-env.d.ts` is present before type-checking run `next typegen`. The commands `next dev` and `next build` also generate the `next-env.d.ts` file, but it is often undesirable to run these just to type-check, for example in CI/CD environments.

> **Good to know**: `next typegen` loads your Next.js config (`next.config.js` , `next.config.mjs` , or `next.config.ts`) using the production build phase. Ensure any required environment variables and dependencies are available so the config can load correctly.

`next upgrade` options

`next upgrade` upgrades your Next.js application to the latest version.

The following options are available for the `next upgrade` command:

Option	Description
`-h, --help`	Show all available options.
`[directory]`	A directory with the Next.js application to upgrade. If not provided, the current directory will be used.
`--revision <revision>`	Specify a Next.js version or tag to upgrade to (e.g., `latest` , `canary` , `15.0.0`). Defaults to the release channel you have currently installed.
`--verbose`	Show verbose output during the upgrade process.

`next experimental-analyze` options

`next experimental-analyze` analyzes your application's bundle output using Turbopack. This command helps you understand the size and composition of your bundles, including JavaScript, CSS, and other assets. This command doesn't produce an application build.

```
# Manejador de paquetes: pnpm
pnpm next experimental-analyze
```

```
# Manejador de paquetes: npm
npx next experimental-analyze
```

```
# Manejador de paquetes: yarn
yarn next experimental-analyze
```

```
# Manejador de paquetes: bun
bunx next experimental-analyze
```

By default, the command starts a local server after analysis completes, allowing you to explore your bundle composition in the browser. The analyzer lets you:

- Filter bundles by route and switch between client and server views

- View the full import chain showing why a module is included
- Trace imports across server-to-client component boundaries and dynamic imports

See Package Bundling for optimization strategies.

To write the analysis output to disk without starting the server, use the `--output` flag. The output is written to `.next/diagnostics/analyze` and contains static files that can be copied elsewhere or shared with others:

```
// Archivo: Terminal
# Write output to .next/diagnostics/analyze
npx next experimental-analyze --output

# Copy the output for comparison with a future analysis
cp -r .next/diagnostics/analyze ./analyze-before-refactor
```

The following options are available for the `next experimental-analyze` command:

Option	Description
`-h, --help`	Show all available options.
`[directory]`	A directory on which to analyze the application. If not provided, the current directory will be used.
`--no-mangling`	Disables mangling. This may affect performance and should only be used for debugging purposes.
`--profile`	Enables production profiling for React. This may affect performance.
`-o, --output`	Write analysis files to disk without starting the server. Output is written to `.next/diagnostics/analyze`.
`--port <port>`	Specify a port number to serve the analyzer on. (default: 4000, env: PORT)

Examples

Debugging prerender errors

If you encounter prerendering errors during `next build`, you can pass the `--debug-prerender` flag to get more detailed output:

```
// Archivo: Terminal
next build --debug-prerender
```

This enables several experimental options to make debugging easier:

- Disables server code minification:
  - `experimental.serverMinification = false`
  - `experimental.turbopackMinify = false`
- Generates source maps for server bundles:
  - `experimental.serverSourceMaps = true`
- Continues building even after the first prerender error, so you can see all issues at once:
  - `experimental.prerenderEarlyExit = false`

This helps surface more readable stack traces and code frames in the build output.

> **Warning**: `--debug-prerender` is for debugging in development only. Do not deploy builds generated with `--debug-prerender` to production, as it may impact performance.

Building specific routes

You can build only specific routes in the App and Pages Routers using the `--debug-build-paths` option. This is useful for faster debugging when working with large applications. The `--debug-build-paths` option accepts comma-separated file paths, supports glob

patterns, and excludes any path prefixed with `!`:

```
// Archivo: Terminal
# Build a specific route
next build --debug-build-paths="app/page.tsx"

# Build more than one route
next build --debug-build-paths="app/page.tsx,pages/index.tsx"

# Include route group folders in the path
next build --debug-build-paths="app/(marketing)/about/page.tsx"

# Use glob patterns
next build --debug-build-paths="app/**/page.tsx"
next build --debug-build-paths="pages/*.tsx"

# Exclude routes with a ! prefix
next build --debug-build-paths="app/**/page.tsx,!app/admin/**"
```

In projects that keep routes under `src/`, paths resolve with or without the `src/` prefix, so both `app/page.tsx` and `src/app/page.tsx` match the same route.

Changing the default port

By default, Next.js uses `http://localhost:3000` during development and with `next start`. The default port can be changed with the `-p` option, like so:

```
// Archivo: Terminal
next dev -p 4000
```

Or using the `PORT` environment variable:

```
// Archivo: Terminal
PORT=4000 next dev
```

> **Good to know**: `PORT` cannot be set in `.env` as booting up the HTTP server happens before any other code is initialized.

Using HTTPS during development

For certain use cases like webhooks or authentication, you can use HTTPS to have a secure environment on `localhost`. Next.js can generate a self-signed certificate with `next dev` using the `--experimental-https` flag:

```
// Archivo: Terminal
next dev --experimental-https
```

With the generated certificate, the Next.js development server will exist at `https://localhost:3000`. The default port `3000` is used unless a port is specified with `-p`, `--port`, or `PORT`.

You can also provide a custom certificate and key with `--experimental-https-key` and `--experimental-https-cert`. Optionally, you can provide a custom CA certificate with `--experimental-https-ca` as well.

```
// Archivo: Terminal
next dev --experimental-https --experimental-https-key ./certificates/localhost-key.pem --experimental-https-cert
./certificates/localhost.pem
```

`next dev --experimental-https` is only intended for development and creates a locally trusted certificate with `mkcert`. In production, use properly issued certificates from trusted authorities.

Configuring a timeout for downstream proxies

When deploying Next.js behind a downstream proxy (e.g. a load-balancer like AWS ELB/ALB), it's important to configure Next's underlying HTTP server with keep-alive timeouts that are *larger* than the downstream proxy's timeouts. Otherwise, once a keep-alive timeout is reached for a given TCP connection, Node.js will immediately terminate that connection without notifying the downstream proxy. This results in a proxy error whenever it attempts to reuse a connection that Node.js has already terminated.

To configure the timeout values for the production Next.js server, pass `--keepAliveTimeout` (in milliseconds) to `next start`, like so:

```
// Archivo: Terminal
next start --keepAliveTimeout 70000
```

Passing Node.js arguments

You can pass any node arguments to `next` commands. For example:

```
// Archivo: Terminal
NODE_OPTIONS='--throw-deprecation' next
NODE_OPTIONS='-r esm' next
NODE_OPTIONS='--inspect' next
```

CPU profiling

You can capture CPU profiles to analyze performance bottlenecks in your Next.js application. The `--experimental-cpu-prof` flag enables V8's built-in CPU profiler and saves profiles to `.next-profiles/` when the process exits:

```
// Archivo: Terminal
# Profile the build process
next build --experimental-cpu-prof

# Profile the dev server (profile saved on Ctrl+C or SIGTERM)
next dev --experimental-cpu-prof

# Profile the production server
next start --experimental-cpu-prof
```

The generated `.cpuprofile` files can be opened in Chrome DevTools (Performance tab → Load profile) or other V8-compatible profiling tools.

> **Good to know**: Profile files are named with a descriptive prefix and timestamp. The profiles generated depend on the command:
>
> `next dev` :
>
> - `dev-main-*` - Parent process (dev server orchestration)
> - `dev-server-*` - Child server process (request handling and rendering) - this is typically what you want to analyze
>
> `next build` **(Turbopack):**
>
> - `build-main-*` - Main build orchestration process
> - `build-turbopack-*` - Turbopack compilation worker
>
> `next build` **(Webpack):**
>
> - `build-main-*` - Main build orchestration process
> - `build-webpack-client-*` - Client bundle compilation worker
> - `build-webpack-server-*` - Server bundle compilation worker
> - `build-webpack-edge-server-*` - Edge runtime compilation worker
>
> `next start` :
>
> - `start-main-*` - Production server process

Version	Changes
`v16.1.0`	Add the `next upgrade` command
`v16.1.0`	Add the `next experimental-analyze` command
`v16.0.0`	The JS bundle size metrics have been removed from `next build`
`v15.5.0`	Add the `next typegen` command
`v15.4.0`	Add `--debug-prerender` option for `next build` to help debug prerender errors.

Configuration

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/configuration
> 📁 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Configure* `adapterPath` *or* `NEXT_ADAPTER_PATH` *to use a custom deployment adapter.*

To use an adapter, specify the path to your adapter module in `adapterPath`:

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  adapterPath: require.resolve('./my-adapter.js'),
}

module.exports = nextConfig
```

Alternatively `NEXT_ADAPTER_PATH` can be set to enable zero-config usage in deployment platforms.

Creating an Adapter

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/creating-an-adapter
> 📁 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Create an adapter module that implements the* `NextAdapter` *interface.*

An adapter is a module that exports an object implementing the `NextAdapter` interface.

The interface can be imported from the `next` package:

```
import type { NextAdapter } from 'next'
```

The interface is defined as follows:

```
type Route = {
  source?: string
  sourceRegex: string
  destination?: string
  headers?: Record<string, string>
  has?: RouteHas[]
  missing?: RouteHas[]
  status?: number
  priority?: boolean
}

export interface AdapterOutputs {
  pages: Array<AdapterOutput['PAGES']>
  middleware?: AdapterOutput['MIDDLEWARE']
  appPages: Array<AdapterOutput['APP_PAGE']>
  pagesApi: Array<AdapterOutput['PAGES_API']>
  appRoutes: Array<AdapterOutput['APP_ROUTE']>
  prerenders: Array<AdapterOutput['PRERENDER']>
  staticFiles: Array<AdapterOutput['STATIC_FILE']>
}

export interface NextAdapter {
  name: string
  modifyConfig?: (
    config: NextConfigComplete,
    ctx: {
      phase: PHASE_TYPE
      nextVersion: string
      projectDir: string
    }
  ) => Promise<NextConfigComplete> | NextConfigComplete
  onBuildComplete?: (ctx: {
    routing: {
      beforeMiddleware: Array<Route>
      middlewareMatchers: Array<Route>
      beforeFiles: Array<Route>
      afterFiles: Array<Route>
      dynamicRoutes: Array<Route>
      onMatch: Array<Route>
      fallback: Array<Route>
      shouldNormalizeNextData: boolean
      rsc: RoutesManifest['rsc']
    }
    outputs: AdapterOutputs
    projectDir: string
    repoRoot: string
    distDir: string
    config: NextConfigComplete
    nextVersion: string
    buildId: string
  }) => Promise<void> | void
}
```

Basic Adapter Structure

Here's a minimal adapter example:

```
// Archivo: my-adapter.js
/** @type {import('next').NextAdapter} */
const adapter = {
  name: 'my-custom-adapter',

  async modifyConfig(config, { phase }) {
    // Modify the Next.js config based on the build phase
    if (phase === 'phase-production-build') {
      return {
        ...config,
        // Add your modifications
      }
    }
    return config
  },

  async onBuildComplete({
    routing,
    outputs,
    projectDir,
    repoRoot,
    distDir,
    config,
    nextVersion,
    buildId,
  }) {
    // Process the build output
    console.log('Build completed with', outputs.pages.length, 'pages')
    console.log('Build ID:', buildId)
    console.log('Dynamic routes:', routing.dynamicRoutes.length)

    // Access emitted output entries
    for (const page of outputs.pages) {
      console.log('Page:', page.pathname, 'at', page.filePath)
    }

    for (const apiRoute of outputs.pagesApi) {
      console.log('API Route:', apiRoute.pathname, 'at', apiRoute.filePath)
    }

    for (const appPage of outputs.appPages) {
      console.log('App Page:', appPage.pathname, 'at', appPage.filePath)
    }

    for (const prerender of outputs.prerenders) {
      console.log('Prerendered:', prerender.pathname)
    }
  },
}

module.exports = adapter
```

API Reference

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/api-reference
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Reference for* `modifyConfig` *and* `onBuildComplete` *in the* `NextAdapter` *interface.*

`async modifyConfig(config, context)`

Called for any CLI command that loads the `next.config.js` file to allow modification of the configuration.

Parameters:

- `config` : The complete Next.js configuration object
- `context.phase` : The current build phase (see phases)
- `context.nextVersion` : Version of Next.js being used
- `context.projectDir` : Absolute path to the Next.js project directory

Returns: The modified configuration object (can be async)

`async onBuildComplete(context)`

Called after the build process completes with detailed information about routes and outputs.

Parameters:

- `context.routing` : Object containing Next.js routing phases and metadata
  - `routing.beforeMiddleware` : Routes executed before middleware (includes header and redirect handling)
  - `routing.middlewareMatchers` : Middleware matcher definitions for this build, used to decide whether middleware should be invoked for a given request
  - `routing.beforeFiles` : Rewrite routes checked before filesystem route matching
  - `routing.afterFiles` : Rewrite routes checked after filesystem route matching
  - `routing.dynamicRoutes` : Dynamic route matching table
  - `routing.onMatch` : Routes applied after a successful match (for example immutable static asset cache headers)
  - `routing.fallback` : Final rewrite fallback routes
  - `routing.shouldNormalizeNextData` : Whether `/_next/data/<buildId>/ ...` URLs should be normalized during matching
  - `routing.rsc` : Route metadata used for React Server Components routing behavior
- `context.outputs` : Detailed information about all build outputs organized by type
- `context.projectDir` : Absolute path to the Next.js project directory
- `context.repoRoot` : Absolute path to the detected repository root
- `context.distDir` : Absolute path to the build output directory
- `context.config` : The final Next.js configuration (with `modifyConfig` applied)
- `context.nextVersion` : Version of Next.js being used
- `context.buildId` : Unique identifier for the current build

Testing Adapters

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/testing-adapters
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Validate adapters with the Next.js compatibility test harness and custom lifecycle scripts.*

Next.js provides a test harness for validating adapters. Running the end-to-end tests for deployment.

Example GitHub Actions workflow:

```
// Archivo: .github/workflows/test-e2e-deploy.yml
name: test-e2e-deploy

on:
  workflow_dispatch:
    inputs:
      nextjsRef:
        description: 'Next.js repo ref (branch/tag/SHA)'
        default: 'canary'
        type: string
  # schedule:
  #   - cron: '0 2 * * *'

jobs:
  build:
    name: Build Next.js + adapter
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
        with:
          path: adapter

      - uses: actions/checkout@v4
        with:
          repository: vercel/next.js
          ref: ${{ inputs.nextjsRef || 'canary' }}
          path: nextjs
          fetch-depth: 25

      - uses: actions/setup-node@v4
        with: { node-version: '20' }

      - name: Setup pnpm
        run: npm i -g corepack@0.31 && corepack enable

      - name: Install & build Next.js
        working-directory: nextjs
        run: pnpm install && pnpm build && pnpm install

      - name: Install Playwright
        working-directory: nextjs
        run: pnpm playwright install --with-deps chromium

      - name: Build adapter
        working-directory: adapter
        run: pnpm install && pnpm build

      - uses: actions/cache/save@v4
        with:
          path: |
            nextjs
            adapter
            ~/.cache/ms-playwright
          key: build-${{ github.sha }}-${{ github.run_id }}

  test:
    name: Tests (${{ matrix.group }})
    needs: build
    runs-on: ubuntu-latest
    timeout-minutes: 60
    strategy:
      fail-fast: false
      matrix:
        group:
          [
            1/16,
            2/16,
            3/16,
            4/16,
            5/16,
            6/16,
            7/16,
```

```
            8/16,
            9/16,
            10/16,
            11/16,
            12/16,
            13/16,
            14/16,
            15/16,
            16/16,
          ]
    steps:
      - uses: actions/cache/restore@v4
        with:
          path: |
            nextjs
            adapter
            ~/.cache/ms-playwright
          key: build-${{ github.sha }}-${{ github.run_id }}

      - uses: actions/setup-node@v4
        with: { node-version: '20' }

      - name: Setup pnpm
        run: npm i -g corepack@0.31 && corepack enable

      - name: Ensure Playwright browser
        working-directory: nextjs
        run: pnpm playwright install chromium

      - name: Make scripts executable
        run: chmod +x adapter/scripts/e2e-deploy.sh
          adapter/scripts/e2e-logs.sh
          adapter/scripts/e2e-cleanup.sh

      - name: Run deploy tests
        working-directory: nextjs
        env:
          NEXT_TEST_MODE: deploy
          NEXT_E2E_TEST_TIMEOUT: 240000
          NEXT_EXTERNAL_TESTS_FILTERS: test/deploy-tests-manifest.json
          ADAPTER_DIR: ${{ github.workspace }}/adapter
          IS_TURBOPACK_TEST: 1
          NEXT_TEST_JOB: 1
          NEXT_TELEMETRY_DISABLED: 1

          # Change these to your adapter's scripts
          # Keep as-is if the scripts are in the adapter repository `scripts` directory
          NEXT_TEST_DEPLOY_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-deploy.sh
          NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-logs.sh
          NEXT_TEST_CLEANUP_SCRIPT_PATH: ${{ github.workspace }}/adapter/scripts/e2e-cleanup.sh
        run: node run-tests.js --timings -g ${{ matrix.group }} -c 2 --type e2e
```

The test harness looks for these environment variables:

- `NEXT_TEST_DEPLOY_SCRIPT_PATH`: Path to the executable that builds and deploys the isolated test app
- `NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH`: Path to the executable that returns build and runtime logs for that deployment
- `NEXT_TEST_CLEANUP_SCRIPT_PATH`: Path to the optional executable that tears the deployment down after the test run

Custom deploy script contract

The deploy script `NEXT_TEST_DEPLOY_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

The deploy script must follow this contract:

- Exit with a non-zero code on failure.
- Print the deployment URL to `stdout`. This will be used to verify the deployment. Avoid writing anything else to `stdout`.
- Write diagnostic output to `stderr` or to files inside the working directory.

Because the deploy script and logs script run as separate processes, any data you want to use later, such as build IDs or server logs, should be persisted to files inside the working directory.

Example deploy script:

```
// Archivo: scripts/e2e-deploy.sh
#!/usr/bin/env bash
set -euo pipefail

# Install the adapter, build the app, and deploy or start it.
node -e "
const pkg=JSON.parse(require('fs').readFileSync('package.json','utf8'));
pkg.dependencies=pkg.dependencies||{};
pkg.dependencies['adapter']='file:${ADAPTER_DIR}';
require('fs').writeFileSync('package.json',JSON.stringify(pkg,null,2));
" >&2

# Set the adapter path so that the app uses it.
export NEXT_ADAPTER_PATH="${ADAPTER_DIR}/dist/index.js"

# Build the app
pnpm build

# Write any metadata needed later to files in the working directory.
BUILD_ID="$(cat .next/BUILD_ID)"
DEPLOYMENT_ID="my-adapter-local"
# If your adapter enables immutable static assets, set this to "1".
NEXT_SUPPORTS_IMMUTABLE_ASSETS="0"

{
  echo "BUILD_ID: $BUILD_ID"
  echo "DEPLOYMENT_ID: $DEPLOYMENT_ID"
  echo "NEXT_SUPPORTS_IMMUTABLE_ASSETS: $NEXT_SUPPORTS_IMMUTABLE_ASSETS"
} >> .adapter-build.log

# Start or deploy the app. Capture the URL at this point or make the script output the URL to stdout.
provider-cli-to-deploy

# Example URL output:
# echo "http://127.0.0.1:3000"
```

Custom logs script contract

The logs script `NEXT_TEST_DEPLOY_LOGS_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

Additionally it receives `NEXT_TEST_DIR` and `NEXT_TEST_DEPLOY_URL` as environment variables.

Its output must include lines starting with:

- `BUILD_ID:`
- `DEPLOYMENT_ID:`
- `NEXT_SUPPORTS_IMMUTABLE_ASSETS:`

After those markers, the logs script can print any additional build or server logs that would help debug failures.

```
// Archivo: scripts/e2e-logs.sh
#!/usr/bin/env bash
set -euo pipefail

if [ -f ".adapter-build.log" ]; then
  cat ".adapter-build.log"
fi

if [ -f ".adapter-server.log" ]; then
  echo "═══ .adapter-server.log ═══"
  cat ".adapter-server.log"
fi
```

One pattern is to have the deploy script write `.adapter-build.log` and `.adapter-server.log`, then have the logs script replay those files so the harness can extract the required markers. This is one option, each platform has different ways to get the logs.

Custom cleanup script contract

The cleanup script `NEXT_TEST_CLEANUP_SCRIPT_PATH` is executed with `cwd` set to the isolated temporary app created by the Next.js test harness.

Additionally it receives `NEXT_TEST_DIR` and `NEXT_TEST_DEPLOY_URL` as environment variables.

The cleanup script can be used to clean up any resources created by the deploy script. It runs after the tests have completed.

Routing with @next/routing

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/routing-with-next-routing
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Use `@next/routing` to apply Next.js route matching behavior in adapters.*

You can use `@next/routing` to reproduce Next.js route matching behavior with data from `onBuildComplete`.

```
import { resolveRoutes } from '@next/routing'

const pathnames = [
   ... outputs.pages,
   ... outputs.pagesApi,
   ... outputs.appPages,
   ... outputs.appRoutes,
   ... outputs.staticFiles,
].map((output) => output.pathname)

const result = await resolveRoutes({
  url: new URL(requestUrl),
  buildId,
  basePath: config.basePath || '',
  i18n: config.i18n,
  headers: new Headers(requestHeaders),
  requestBody, // ReadableStream
  pathnames,
  routes: routing,
  invokeMiddleware: async (ctx) => {
    // platform-specific middleware invocation
    return {}
  },
})

if (result.resolvedPathname) {
  console.log('Resolved pathname:', result.resolvedPathname)
  console.log('Resolved query:', result.resolvedQuery)
  console.log('Invocation target:', result.invocationTarget)
}
```

`resolveRoutes()` returns:

- `middlewareResponded`: `true` when middleware already sent a response (the adapter should not invoke an entrypoint).
- `externalRewrite`: A `URL` when routing resolved to an external rewrite destination.
- `redirect`: An object with `url` (`URL`) and `status` when the request should be redirected.
- `resolvedPathname`: The route pathname selected by Next.js routing. For dynamic routes, this is the matched route template such as `/blog/[slug]`.
- `resolvedQuery`: The final query after rewrites or middleware have added or replaced search params.
- `invocationTarget`: The concrete pathname and query to invoke for the matched route.
- `resolvedHeaders`: A `Headers` object containing any headers added or modified during routing.
- `status`: An HTTP status code set by routing (for example from a redirect or rewrite rule).
- `routeMatches`: A record of named matches extracted from dynamic route segments.

For example, if `/blog/post-1?draft=1` matches `/blog/[slug]?slug=post-1`, `resolvedPathname` is `/blog/[slug]` while `invocationTarget.pathname` is `/blog/post-1`.

Implementing PPR in an Adapter

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/implementing-ppr-in-an-adapter
> 📁 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Implement Partial Prerendering support in an adapter using fallback output and cache hooks.*

For partially prerendered app routes, `onBuildComplete` gives you the data needed to seed and resume PPR:

- `outputs.prerenders[].fallback.filePath`: path to the generated fallback shell (for example HTML)
- `outputs.prerenders[].fallback.postponedState`: serialized postponed state used to resume rendering

1. Seed shell + postponed state at build time

```
// Archivo: my-adapter.ts
import { readFile } from 'node:fs/promises'

async function seedPprEntries(outputs: AdapterOutputs) {
  for (const prerender of outputs.prerenders) {
    const fallback = prerender.fallback
    if (!fallback?.filePath || !fallback.postponedState) continue

    const shell = await readFile(fallback.filePath, 'utf8')
    await platformCache.set(prerender.pathname, {
      shell,
      postponedState: fallback.postponedState,
      initialHeaders: fallback.initialHeaders,
      initialStatus: fallback.initialStatus,
      initialRevalidate: fallback.initialRevalidate,
      initialExpiration: fallback.initialExpiration,
    })
  }
}
```

2. Runtime flow: serve cached shell and resume in background

At request time, you can stream a single response that is the concatenation of:

1. cached HTML shell stream
2. resumed render stream (generated after invoking `handler` with postponed state)

```
Client
  | GET /ppr-route
  v
Adapter Router
  |
  ├── read cached shell + postponedState ---> Platform Cache
  |<------------- cache hit ------------------┤
  |
  ├── create responseStream = concat(shellStream, resumedStream)
  |
  ├── start piping shellStream ------------> Client (first bytes)
  |
  ├── invoke handler(req, res, { requestMeta: { postponed } })
  |   ---------------------------------------> Entrypoint (handler)
  |   <--------------------------------------- resumed chunks/cache entry
  |
  ├── append resumed chunks to resumedStream
  |
  '-- client receives one HTTP response:
      [shell bytes........][resumed bytes........]
```

3. Update cache with `requestMeta.onCacheEntryV2`

`requestMeta.onCacheEntryV2` is called when a response cache entry is looked up or generated. Use it to persist updated shell/postponed data.

> [!NOTE]
> - `requestMeta.onCacheEntry` still works, but is deprecated.
> - Prefer `requestMeta.onCacheEntryV2`.
> - If your adapter uses an internal `onCacheCallback` abstraction, wire it to `requestMeta.onCacheEntryV2`.

```
// Archivo: my-adapter.ts
await handler(req, res, {
  waitUntil,
  requestMeta: {
    postponed: cachedPprEntry?.postponedState,
    onCacheEntryV2: async (cacheEntry, meta) => {
      if (cacheEntry.value?.kind === 'APP_PAGE') {
        const html =
          cacheEntry.value.html &&
          typeof cacheEntry.value.html.toUnchunkedString === 'function'
            ? cacheEntry.value.html.toUnchunkedString()
            : null

        await platformCache.set(meta.url || req.url || '/', {
          shell: html,
          postponedState: cacheEntry.value.postponed,
          headers: cacheEntry.value.headers,
          status: cacheEntry.value.status,
          cacheControl: cacheEntry.cacheControl,
        })
      }

      // Return true only if your adapter already wrote the response itself.
      return false
    },
  },
})
```

```
Entrypoint (handler)
  | onCacheEntryV2(cacheEntry, { url })
  v
requestMeta.onCacheEntryV2 callback
  |
  ├── if APP_PAGE ---> persist html + postponedState + headers ---> Platform Cache
  |
  '-- return false: continue normal Next.js response flow
      return true:  adapter already handled response (short-circuit)
```

Runtime Integration

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/runtime-integration
> 📁 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Understand how build-time adapters and runtime cache interfaces work together.*

The Deployment Adapter API is a **build-time** interface. It tells your platform what was built and how to route requests. **Runtime** behavior (request handling, streaming, caching) is handled by the Next.js server itself and by the cache interfaces `cacheHandler` and `cacheHandlers`.

Together, the adapter and cache interfaces form the complete platform integration surface:

- **Adapter** (build-time): processes build outputs, configures routing, and sets up platform-specific infrastructure.
- **Cache Interfaces** (runtime): `cacheHandler` manages ISR/server cache storage and revalidation across instances; `cacheHandlers` configures `'use cache'` directive backends and tag coordination.

Handler Context

When invoking entrypoints, adapters pass a `ctx` object to the Next.js handler. Key fields include:

- `ctx.waitUntil`: a function that accepts a promise. Use this to keep the serverless function alive after the response is sent, allowing background work like cache revalidation to complete.
- `requestMeta.onCacheEntryV2` (set via `addRequestMeta`): a callback that fires when a cache entry is generated or looked up. Use this to observe all cache operations (not just PPR) and propagate cache updates to your platform's storage backend. This callback fires on the instance that handled the request. For multi-instance deployments, your adapter should propagate updates to shared storage. See How Revalidation Works for coordination patterns.

PPR Chain Headers

In the prerenders output type, `pprChain.headers` contains the headers needed for the resume protocol. Specifically, it contains `{ 'next-resume': '1' }`.

When your adapter detects a PPR-enabled route with a cached static shell:

1. Set the `pprChain.headers` on the internal request to the Next.js handler.
2. Send the request as a **POST** with the `postponedState` as the request body.
3. The handler will render only the deferred Suspense boundaries and stream the result.

> **Good to know:** In standard `next start`, the server handles both the shell and dynamic render in a single pass automatically. The resume protocol is useful for adapter-based deployments and CDN-to-origin architectures that want to serve the shell separately. See the PPR Platform Guide for the full implementation context.

Invoking Entrypoints

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/invoking-entrypoints
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Invoke Node.js and Edge build entrypoints with adapter runtime context.*

Build output entrypoints use a `handler( ... , ctx)` interface, with runtime-specific request/response types.

Node.js runtime (`runtime: 'nodejs'`)

Node.js entrypoints use the following interface:

```
handler(
  req: IncomingMessage,
  res: ServerResponse,
  ctx: {
    waitUntil?: (promise: Promise<void>) => void
    requestMeta?: RequestMeta
  }
): Promise<void>
```

When invoking Node.js entrypoints directly, adapters can pass helpers directly on `requestMeta` instead of relying on internals. Some of the supported fields are `hostname`, `revalidate`, and `render404`:

```
await handler(req, res, {
  requestMeta: {
    // Relative path from process.cwd() to the Next.js project directory.
    relativeProjectDir: '.',
    // Optional hostname used by route handlers when constructing absolute URLs.
    hostname: '127.0.0.1',
    // Optional internal revalidate function to avoid revalidating over the network
    revalidate: async ({ urlPath, headers, opts }) => {
      // platform-specific revalidate implementation
    },
    // Optional function to render the 404 page for pages router `notFound: true`
    render404: async (req, res, parsedUrl, setHeaders) => {
      // platform-specific 404 rendering implementation
    },
  },
})
```

Relevant files in the Next.js core:

- `packages/next/src/build/templates/app-page.ts`
- `packages/next/src/build/templates/app-route.ts`
- and `packages/next/src/build/templates/pages-api.ts`

Edge runtime (`runtime: 'edge'`) (deprecated)

> The Edge Runtime is deprecated. New routes should use the Node.js runtime.

Edge entrypoints use the following interface:

```
handler(
  request: Request,
  ctx: {
    waitUntil?: (prom: Promise<void>) => void
    signal?: AbortSignal
    requestMeta?: RequestMeta
  }
): Promise<Response>
```

The shape is aligned around `handler( ... , ctx)`, but Node.js and Edge runtimes use different request/response primitives.

For outputs with `runtime: 'edge'`, Next.js also provides `output.edgeRuntime` with the canonical metadata needed to invoke the entrypoint:

```
{
  modulePath: string // Absolute path to the module registered in the edge runtime
  entryKey: string // Canonical key used by the edge entry registry
  handlerExport: string // Export name to invoke, currently 'handler'
}
```

After your edge runtime loads and evaluates the chunks for `modulePath`, use `entryKey` to read the registered entry from the global edge entry registry (`globalThis._ENTRIES`), then invoke `handlerExport` from that entry:

```
const entry = await globalThis._ENTRIES[output.edgeRuntime.entryKey]
const handler = entry[output.edgeRuntime.handlerExport]
await handler(request, ctx)
```

Use `edgeRuntime` instead of deriving registry keys or handler names from filenames.

Relevant files in the Next.js core:

- `packages/next/src/build/templates/edge-ssr.ts`
- `packages/next/src/build/templates/edge-app-route.ts`

- `packages/next/src/build/templates/pages-edge-api.ts`
- and `packages/next/src/build/templates/middleware.ts`

Output Types

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/output-types
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Reference for all build output types exposed to adapters.*

The `outputs` object contains arrays of build output types:

- `outputs.pages`: React pages from the `pages/` directory
- `outputs.pagesApi`: API routes from `pages/api/`
- `outputs.appPages`: React pages from the `app/` directory
- `outputs.appRoutes`: API and metadata routes from `app/`
- `outputs.prerenders`: ISR-enabled routes and static prerenders
- `outputs.staticFiles`: Static assets and auto-statically optimized pages
- `outputs.middleware`: Middleware function (if present)

> **Note:** When `config.output` is set to `'export'`, only `outputs.staticFiles` is populated. All other arrays (`pages`, `appPages`, `pagesApi`, `appRoutes`, `prerenders`) will be empty since the entire application is exported as static files.

For any route output with `runtime: 'edge'`, `edgeRuntime` is included and contains the canonical entry metadata for invoking that output in your edge runtime. Note that the Edge Runtime is deprecated.

Pages (`outputs.pages`)

React pages from the `pages/` directory:

```
{
  type: 'PAGES'
  id: string           // Route identifier
  filePath: string     // Path to the built file
  pathname: string     // URL pathname
  sourcePage: string   // Original source file path in pages/ directory
  runtime: 'nodejs' | 'edge'
  assets: Record<string, string>  // Traced dependencies (key: relative path from repo root, value: absolute path)
  assetsHashes: Record<string, string>  // Content hashes of each `assets` entry (key: same as `assets`, value: content hash)
  wasmAssets?: Record<string, string>  // Bundled wasm files (key: name, value: absolute path)
  edgeRuntime?: {
    modulePath: string    // Absolute path to the module registered in the edge runtime
    entryKey: string      // Canonical key used by the edge entry registry
    handlerExport: string // Export name to invoke, currently 'handler'
  }
  config: {
    maxDuration?: number  // Maximum duration of the route in seconds
    preferredRegion?: string | string[]  // Preferred deployment region (deprecated)
    env?: Record<string, string>  // Environment variables (edge runtime only)
  }
}
```

API Routes (`outputs.pagesApi`)

API routes from `pages/api/`:

```
{
  type: 'PAGES_API'
  id: string           // Route identifier
  filePath: string     // Path to the built file
  pathname: string     // URL pathname
  sourcePage: string   // Original relative source file path
  runtime: 'nodejs' | 'edge'
  assets: Record<string, string>  // Traced dependencies (key: relative path from repo root, value: absolute path)
  assetsHashes: Record<string, string>  // Content hashes of each `assets` entry (key: same as `assets`, value: content hash)
  wasmAssets?: Record<string, string>  // Bundled wasm files (key: name, value: absolute path)
  edgeRuntime?: {
    modulePath: string    // Absolute path to the module registered in the edge runtime
    entryKey: string      // Canonical key used by the edge entry registry
    handlerExport: string // Export name to invoke, currently 'handler'
  }
  config: {
    maxDuration?: number  // Maximum duration of the route in seconds
    preferredRegion?: string | string[]  // Preferred deployment region (deprecated)
    env?: Record<string, string>  // Environment variables (edge runtime only)
  }
}
```

App Pages (`outputs.appPages`)

React pages from the `app/` directory:

```
{
  type: 'APP_PAGE'
  id: string           // Route identifier
  filePath: string     // Path to the built file
  pathname: string     // URL pathname. Includes .rsc suffix for RSC routes
  sourcePage: string   // Original relative source file path
  runtime: 'nodejs' | 'edge' // Runtime the route is built for
  assets: Record<string, string>  // Traced dependencies (key: relative path from repo root, value: absolute path)
  assetsHashes: Record<string, string>  // Content hashes of each `assets` entry (key: same as `assets`, value: content hash)
  wasmAssets?: Record<string, string>  // Bundled wasm files (key: name, value: absolute path)
  edgeRuntime?: {
    modulePath: string    // Absolute path to the module registered in the edge runtime
    entryKey: string      // Canonical key used by the edge entry registry
    handlerExport: string // Export name to invoke, currently 'handler'
  }
  config: {
    maxDuration?: number  // Maximum duration of the route in seconds
    preferredRegion?: string | string[]  // Preferred deployment region (deprecated)
    env?: Record<string, string>  // Environment variables (edge runtime only)
  }
}
```

App Routes (`outputs.appRoutes`)

API and metadata routes from the `app/` directory:

```
{
  type: 'APP_ROUTE'
  id: string           // Route identifier
  filePath: string     // Path to the built file
  pathname: string     // URL pathname
  sourcePage: string   // Original relative source file path
  runtime: 'nodejs' | 'edge' // Runtime the route is built for
  assets: Record<string, string>  // Traced dependencies (key: relative path from repo root, value: absolute path)
  assetsHashes: Record<string, string>  // Content hashes of each `assets` entry (key: same as `assets`, value: content hash)
  wasmAssets?: Record<string, string>  // Bundled wasm files (key: name, value: absolute path)
  edgeRuntime?: {
    modulePath: string    // Absolute path to the module registered in the edge runtime
    entryKey: string      // Canonical key used by the edge entry registry
    handlerExport: string // Export name to invoke, currently 'handler'
  }
  config: {
    maxDuration?: number  // Maximum duration of the route in seconds
    preferredRegion?: string | string[]  // Preferred deployment region (deprecated)
    env?: Record<string, string>  // Environment variables (edge runtime only)
  }
}
```

Prerenders (`outputs.prerenders`)

ISR-enabled routes and static prerenders:

```
{
  type: 'PRERENDER'
  id: string           // Route identifier
  pathname: string     // URL pathname
  parentOutputId: string  // ID of the source page/route
  groupId: number        // Revalidation group identifier (prerenders with same groupId revalidate together)
  route: string           // Source route matcher aligned with the filesystem route, keeping dynamic segments (e.g.
/blog/[slug] for the prerendered path /blog/first)
  routeType?: 'route' | 'fallback' | 'shell' | 'page'  // Kind of canonical response
  response?: 'empty' | 'initial' | 'complete'  // Completeness before request-time work
  compute?: 'blocking' | 'resuming' | 'static'  // Request-time compute needed for the completed response
  htmlSize?: number       // Byte size of the prerendered App Router HTML shell
  pprChain?: {
    headers: Record<string, string>  // PPR chain headers (e.g., 'next-resume': '1')
  }
  parentFallbackMode?: false | null | string  // false: no additional paths (fallback: false), null: blocking render, string:
path to HTML fallback
  fallback?: {
    filePath: string | undefined  // Path to the fallback file (HTML, JSON, or RSC)
    initialStatus?: number  // Initial status code
    initialHeaders?: Record<string, string | string[]>  // Initial headers
    initialExpiration?: number  // Initial expiration time in seconds
    initialRevalidate?: number | false  // Initial revalidate time in seconds, or false for fully static
    postponedState: string | undefined  // Serialized PPR state used for resuming rendering
  }
  config: {
    allowQuery?: string[]     // Allowed query parameters considered for the cache key
    allowHeader?: string[]    // Allowed headers for ISR
    bypassFor?: RouteHas[]    // Cache bypass conditions
    renderingMode?: 'STATIC' | 'PARTIALLY_STATIC'  // STATIC: fully static, PARTIALLY_STATIC: PPR-enabled
    partialFallback?: boolean  // Serves a partial fallback shell that should be upgraded to a full route in the background
    bypassToken?: string      // Generated token that signals the prerender cache should be bypassed
  }
}
```

Prerender classification

`routeType` , `response` , and `compute` are emitted together on the primary response in a prerender group. Related RSC, data, and segment outputs omit these fields. Pages Router templates with `fallback: false` also omit them because those templates are never served for unmatched URLs.

`routeType` identifies the kind of canonical response:

- `route`: a non-UI route, such as a Route Handler
- `page`: a page whose URL has no missing prerenderable parameters
- `shell`: the most specific reusable page shell for its class of URLs
- `fallback`: a reusable page response that can be specialized by filling more prerenderable parameters

`response` describes how complete the response is before request-time work:

- `empty`: no initial page response can be served
- `initial`: an initial response can be served, but it is not the completed page UI. In practice, this only applies to UI routes that are partially prerenderable
- `complete`: the response is complete; this can include a zero-byte response body, such as a `204` Route Handler response

`compute` describes the request-time compute needed to serve the completed response:

- `blocking`: no initial response can be sent before request-time compute starts; once started, the response can stream while compute continues
- `resuming`: an initial response is served while postponed work resumes on the server
- `static`: no server compute is required per request

`htmlSize` is only included on the primary App Router HTML output. A value of `0` means that the HTML shell is empty. Pages Router prerenders, Route Handlers, and related RSC, data, and segment outputs omit it.

Static Files (`outputs.staticFiles`)

Static assets and auto-statically optimized pages:

```
{
  type: 'STATIC_FILE'
  id: string // Unique identifier for this static file output
  filePath: string // Absolute filesystem path to the built file
  pathname: string // The routable URL pathname for this static file
  immutableHash: string | undefined // Content hash when the filename contains a hash, indicating the file is immutable
}
```

See Supporting immutable static assets for more information about `immutableHash`.

Middleware (`outputs.middleware`)

`middleware.ts` (`.js` / `.ts`) or `proxy.ts` (`.js` / `.ts`) function (if present):

```
{
  type: 'MIDDLEWARE'
  id: string           // Route identifier
  filePath: string     // Path to the built file
  pathname: string      // Always '/_middleware'
  sourcePage: string    // Always 'middleware'
  runtime: 'nodejs' | 'edge' // Runtime the route is built for
  assets: Record<string, string>  // Traced dependencies (key: relative path from repo root, value: absolute path)
  assetsHashes: Record<string, string>  // Content hashes of each `assets` entry (key: same as `assets`, value: content hash)
  wasmAssets?: Record<string, string>  // Bundled wasm files (key: name, value: absolute path)
  edgeRuntime?: {
    modulePath: string    // Absolute path to the module registered in the edge runtime
    entryKey: string      // Canonical key used by the edge entry registry
    handlerExport: string // Export name to invoke, currently 'handler'
  }
  config: {
    maxDuration?: number  // Maximum duration of the route in seconds
    preferredRegion?: string | string[]  // Preferred deployment region (deprecated)
    env?: Record<string, string>  // Environment variables (edge runtime only)
    matchers?: Array<{
      source: string  // Source pattern
      sourceRegex: string  // Compiled regex for matching requests
      has: RouteHas[] | undefined  // Positive matching conditions
      missing: RouteHas[] | undefined  // Negative matching conditions
    }>
  }
}
```

Routing Information

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/routing-information
> 📁 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Reference for routing phases and route fields exposed in* `onBuildComplete`.

The `routing` object in `onBuildComplete` provides complete routing information with processed patterns ready for deployment:

`routing.beforeMiddleware`

Routes applied before middleware execution. These include generated header and redirect behavior.

`routing.middlewareMatchers`

Middleware matcher definitions emitted for this build. Use these to decide whether middleware should be invoked for a given request.

`routing.beforeFiles`

Rewrite routes checked before filesystem route matching.

`routing.afterFiles`

Rewrite routes checked after filesystem route matching.

`routing.dynamicRoutes`

Dynamic matchers generated from route segments such as `[slug]` and catch-all routes.

`routing.onMatch`

Routes that apply after a successful match, such as immutable cache headers for hashed static assets.

`routing.fallback`

Final rewrite routes checked when earlier phases did not produce a match.

Common Route Fields

Each route entry can include:

- `source` : Original route pattern (optional for generated internal rules)
- `sourceRegex` : Compiled regex for matching requests
- `destination` : Internal destination or redirect destination
- `headers` : Headers to apply
- `has` : Positive matching conditions
- `missing` : Negative matching conditions
- `status` : Redirect status code
- `priority` : Internal route priority flag

Use Cases

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/use-cases
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Common patterns and examples for deployment adapter implementations.*

Common use cases for adapters include:

- **Deployment Platform Integration**: Automatically configure build outputs for specific hosting platforms
- **Asset Processing**: Transform or optimize build outputs
- **Monitoring Integration**: Collect build metrics and route information
- **Custom Bundling**: Package outputs in platform-specific formats
- **Build Validation**: Ensure outputs meet specific requirements
- **Route Generation**: Use processed route information to generate platform-specific routing configs

---

Supporting Immutable Static Assets

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters/immutable-static-assets
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Support immutable static assets in an adapter*

See `config.supportsImmutableAssets` for end-user-facing information about this feature.

When `config.supportsImmutableAssets` is enabled, Next.js outputs immutable content-addressed static assets under the public path `/_next/static/immutable/*`. This prefix can be used to differentiate immutable static assets from non-immutable static assets at the

CDN level.

At runtime, these immutable static assets are requested without the `?dpl` query parameter and thus live in a shared namespace across deployments. You must ensure that these assets are immutable and not changed (even after a new deployment) or deleted (for as long as there are active deployments using them) . Next.js may use a truncated shorter content hash as the filename, so `outputs.staticFiles[].immutableHash` contains the full content hash which can be used to validate that no hash collision occurred.

You can use `config.outputHashSalt` to set a salt for the content hashes, if you want to rotate the hashes for any reason (e.g. after a detected hash collision).

Note that you need to continue supporting non-immutable static assets (which may change between deployments and continue to be requested with the `?dpl` query parameter), e.g. for the `public` folder or for older Next.js versions.

Adapter Implementation

You need to:

1. In the `modifyConfig`, set the `config.supportsImmutableAssets` property to `true` (if it's not already set to `false` by the user) to signal that you support deploying immutable static assets, and
2. In `onBuildComplete`, read the `outputs.staticFiles[].immutableHash` property to determine which static assets are immutable and have to be requested without the `?dpl` query parameter.

```
// Archivo: my-adapter.js
/** @type {import('next').NextAdapter} */
const adapter = {
  name: 'my-custom-adapter',

  async modifyConfig(config, { phase }) {
    if (phase === 'phase-production-build') {
      config.supportsImmutableAssets =
        // Default to true, but allow users to opt-out
        config.supportsImmutableAssets ?? true

      // Optionally, pass a salt for the content hashes
      // config.outputHashSalt = getSaltForCurrentProject()
    }
    return config
  },

  async onBuildComplete({ outputs }) {
    for (const output of outputs.staticFiles) {
      if (output.immutableHash !== null) {
        // This has to be requestable at `output.pathname`
        // even without the `?dpl` query parameter.
        uploadOrVerifyImmutableStaticAsset(
          output.filePath,
          output.pathname,
          output.immutableHash
        )
      } else {
        // This is a non-immutable static asset and will be requested with
        // the `?dpl` query parameter, scoped to the deployment.
        uploadStaticAsset(output.filePath, output.pathname)
      }
    }

    // Process other outputs....
  },
}
```

Adapters

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/adapters
> 📂 **Sección:** App > Api Reference > Adapters
> 📝 **Descripción:** *Build deployment adapters for Next.js platforms and infrastructure.*

Use this section to build and validate deployment adapters that integrate with the Next.js build and runtime model.

Edge Runtime

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/edge
> 📂 **Sección:** App > Api Reference
> 📝 **Descripción:** *API Reference for the Edge Runtime.*

Next.js has two server runtimes you can use in your application:

- The **Node.js Runtime** (default), which has access to all Node.js APIs and is used for rendering your application.
- The **Edge Runtime** which contains a more limited set of APIs, used in Proxy.

Caveats

- The Edge Runtime does not support all Node.js APIs. Some packages may not work as expected.
- The Edge Runtime does not support Incremental Static Regeneration (ISR).
- Both runtimes can support streaming depending on your deployment adapter.

Reference

The Edge Runtime supports the following APIs:

Network APIs

API	Description
`Blob`	Represents a blob
`fetch`	Fetches a resource
`FetchEvent`	Represents a fetch event
`File`	Represents a file
`FormData`	Represents form data
`Headers`	Represents HTTP headers
`Request`	Represents an HTTP request
`Response`	Represents an HTTP response
`URLSearchParams`	Represents URL search parameters
`WebSocket`	Represents a websocket connection

Encoding APIs

API	Description
`atob`	Decodes a base-64 encoded string
`btoa`	Encodes a string in base-64
`TextDecoder`	Decodes a Uint8Array into a string
`TextDecoderStream`	Chainable decoder for streams
`TextEncoder`	Encodes a string into a Uint8Array
`TextEncoderStream`	Chainable encoder for streams

Stream APIs

API	Description
`ReadableStream`	Represents a readable stream
`ReadableStreamBYOBReader`	Represents a reader of a ReadableStream
`ReadableStreamDefaultReader`	Represents a reader of a ReadableStream
`TransformStream`	Represents a transform stream
`WritableStream`	Represents a writable stream
`WritableStreamDefaultWriter`	Represents a writer of a WritableStream

Crypto APIs

API	Description
`crypto`	Provides access to the cryptographic functionality of the platform
`CryptoKey`	Represents a cryptographic key
`SubtleCrypto`	Provides access to common cryptographic primitives, like hashing, signing, encryption or decryption

Web Standard APIs

API	Description
`AbortController`	Allows you to abort one or more DOM requests as and when desired
`Array`	Represents an array of values
`ArrayBuffer`	Represents a generic, fixed-length raw binary data buffer
`Atomics`	Provides atomic operations as static methods
`BigInt`	Represents a whole number with arbitrary precision
`BigInt64Array`	Represents a typed array of 64-bit signed integers
`BigUint64Array`	Represents a typed array of 64-bit unsigned integers
`Boolean`	Represents a logical entity and can have two values: `true` and `false`
`clearInterval`	Cancels a timed, repeating action which was previously established by a call to `setInterval()`
`clearTimeout`	Cancels a timed, repeating action which was previously established by a call to `setTimeout()`
`console`	Provides access to the browser's debugging console
`DataView`	Represents a generic view of an `ArrayBuffer`
`Date`	Represents a single moment in time in a platform-independent format
`decodeURI`	Decodes a Uniform Resource Identifier (URI) previously created by `encodeURI` or by a similar routine
`decodeURIComponent`	Decodes a Uniform Resource Identifier (URI) component previously created by `encodeURIComponent` or by a similar routine
`DOMException`	Represents an error that occurs in the DOM
`encodeURI`	Encodes a Uniform Resource Identifier (URI) by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character
`encodeURIComponent`	Encodes a Uniform Resource Identifier (URI) component by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character
`Error`	Represents an error when trying to execute a statement or accessing a property
`EvalError`	Represents an error that occurs regarding the global function `eval()`
`Float32Array`	Represents a typed array of 32-bit floating point numbers
`Float64Array`	Represents a typed array of 64-bit floating point numbers
`Function`	Represents a function
`Infinity`	Represents the mathematical Infinity value
`Int8Array`	Represents a typed array of 8-bit signed integers
`Int16Array`	Represents a typed array of 16-bit signed integers
`Int32Array`	Represents a typed array of 32-bit signed integers
`Intl`	Provides access to internationalization and localization functionality
`isFinite`	Determines whether a value is a finite number
`isNaN`	Determines whether a value is `NaN` or not
`JSON`	Provides functionality to convert JavaScript values to and from the JSON format
`Map`	Represents a collection of values, where each value may occur only once
`Math`	Provides access to mathematical functions and constants

API	Description
`Number`	Represents a numeric value
`Object`	Represents the object that is the base of all JavaScript objects
`parseFloat`	Parses a string argument and returns a floating point number
`parseInt`	Parses a string argument and returns an integer of the specified radix
`Promise`	Represents the eventual completion (or failure) of an asynchronous operation, and its resulting value
`Proxy`	Represents an object that is used to define custom behavior for fundamental operations (e.g. property lookup, assignment, enumeration, function invocation, etc)
`queueMicrotask`	Queues a microtask to be executed
`RangeError`	Represents an error when a value is not in the set or range of allowed values
`ReferenceError`	Represents an error when a non-existent variable is referenced
`Reflect`	Provides methods for interceptable JavaScript operations
`RegExp`	Represents a regular expression, allowing you to match combinations of characters
`Set`	Represents a collection of values, where each value may occur only once
`setInterval`	Repeatedly calls a function, with a fixed time delay between each call
`setTimeout`	Calls a function or evaluates an expression after a specified number of milliseconds
`SharedArrayBuffer`	Represents a generic, fixed-length raw binary data buffer
`String`	Represents a sequence of characters
`structuredClone`	Creates a deep copy of a value
`Symbol`	Represents a unique and immutable data type that is used as the key of an object property
`SyntaxError`	Represents an error when trying to interpret syntactically invalid code
`TypeError`	Represents an error when a value is not of the expected type
`Uint8Array`	Represents a typed array of 8-bit unsigned integers
`Uint8ClampedArray`	Represents a typed array of 8-bit unsigned integers clamped to 0-255
`Uint32Array`	Represents a typed array of 32-bit unsigned integers
`URIError`	Represents an error when a global URI handling function was used in a wrong way
`URL`	Represents an object providing static methods used for creating object URLs
`URLPattern`	Represents a URL pattern
`URLSearchParams`	Represents a collection of key/value pairs
`WeakMap`	Represents a collection of key/value pairs in which the keys are weakly referenced
`WeakSet`	Represents a collection of objects in which each object may occur only once
`WebAssembly`	Provides access to WebAssembly

Next.js Specific Polyfills

- `AsyncLocalStorage`

Environment Variables

You can use `process.env` to access Environment Variables for both `next dev` and `next build`.

Unsupported APIs

The Edge Runtime has some restrictions including:

- Native Node.js APIs **are not supported**. For example, you can't read or write to the filesystem.
- `node_modules` *can* be used, as long as they implement ES Modules and do not use native Node.js APIs.
- Calling `require` directly is **not allowed**. Use ES Modules instead.

The following JavaScript language features are disabled, and **will not work:**

API	Description
`eval`	Evaluates JavaScript code represented as a string
`new Function(evalString)`	Creates a new function with the code provided as an argument
`WebAssembly.compile`	Compiles a WebAssembly module from a buffer source
`WebAssembly.instantiate`	Compiles and instantiates a WebAssembly module from a buffer source

In rare cases, your code could contain (or import) some dynamic code evaluation statements which *cannot be reached at runtime* and which cannot be removed by treeshaking. You can relax the check to allow specific files with your Proxy configuration:

```
// Archivo: proxy.ts
export const config = {
  unstable_allowDynamic: [
    // allows a single file
    '/lib/utilities.js',
    // use a glob to allow anything in the function-bind 3rd party module
    '**/node_modules/function-bind/**',
  ],
}
```

`unstable_allowDynamic` is a glob, or an array of globs, ignoring dynamic code evaluation for specific files. The globs are relative to your application root folder.

Be warned that if these statements are executed on the Edge, *they will throw and cause a runtime error*.

Turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference/turbopack
> 📁 **Sección:** App > Api Reference
> 📝 **Descripción:** *Turbopack is an incremental bundler optimized for JavaScript and TypeScript, written in Rust, and built into Next.js.*

Turbopack is an **incremental bundler** optimized for JavaScript and TypeScript, written in Rust, and built into **Next.js**. You can use Turbopack with both the Pages and App Router for a **much faster** local development experience.

Why Turbopack?

We built Turbopack to push the performance of Next.js, including:

- **Unified Graph:** Next.js supports multiple output environments (e.g., client and server). Managing multiple compilers and stitching bundles together can be tedious. Turbopack uses a **single, unified graph** for all environments.
- **Bundling vs Native ESM:** Some tools skip bundling in development and rely on the browser's native ESM. This works well for small apps but can slow down large apps due to excessive network requests. Turbopack **bundles** in dev, but in an optimized way to keep large apps fast.

- **Incremental Computation:** Turbopack parallelizes work across cores and **caches** results down to the function level. Once a piece of work is done, Turbopack won't repeat it. Results persist to disk between runs.
- **Lazy Bundling:** Turbopack only bundles what is actually requested by the dev server. This lazy approach can reduce initial compile times and memory usage.

Supported platforms

Turbopack requires platform-specific native bindings. The following platforms are currently supported:

Platform	Architecture
macOS (Darwin)	x64, ARM64
Windows	x64, ARM64
Linux (glibc)	x64, ARM64
Linux (musl)	x64, ARM64

On platforms without native bindings (e.g. FreeBSD, OpenBSD), Next.js falls back to WebAssembly (WASM) bindings. WASM bindings support core SWC features like compilation and minification, but **do not support Turbopack**. On these platforms, use the `--webpack` flag:

```
next dev --webpack
next build --webpack
```

Getting started

Turbopack is now the **default bundler** in Next.js. No configuration is needed to use Turbopack:

```
// Archivo: package.json highlight={3}
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  }
}
```

Using Webpack instead

If you need to use Webpack instead of Turbopack, you can opt-in with the `--webpack` flag:

```
// Archivo: package.json
{
  "scripts": {
    "dev": "next dev --webpack",
    "build": "next build --webpack",
    "start": "next start"
  }
}
```

Supported features

Turbopack in Next.js has **zero-configuration** for the common use cases. Below is a summary of what is supported out of the box, plus some references to how you can configure Turbopack further when needed.

Language features

Feature	Status	Notes
JavaScript & TypeScript	**Supported**	Uses SWC under the hood. Type-checking is not done by Turbopack (run `tsc --watch` or rely on your IDE for type checks).
ECMAScript (ESNext)	**Supported**	Turbopack supports the latest ECMAScript features, matching SWC's coverage.
CommonJS	**Supported**	`require()` syntax is handled out of the box.
ESM	**Supported**	Static and dynamic `import` are fully supported.
Babel	**Supported**	Starting in Next.js 16, Turbopack uses Babel automatically if it detects a configuration file. Unlike in webpack, SWC is always used for Next.js's internal transforms and downleveling to older ECMAScript revisions. Next.js with webpack disables SWC if a Babel configuration file is present. Files in `node_modules` are excluded, unless you manually configure `babel-loader`.

Framework and React features

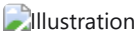
Feature	Status	Notes
JSX / TSX	**Supported**	SWC handles JSX/TSX compilation.
Fast Refresh	**Supported**	No configuration needed.
React Server Components (RSC)	**Supported**	For the Next.js App Router. Turbopack ensures correct server/client bundling.
Root layout creation	Unsupported	Automatic creation of a root layout in App Router is not supported. Turbopack will instruct you to create it manually.

CSS and styling

Feature	Status	Notes
Global CSS	**Supported**	Import `.css` files directly in your application.
CSS Modules	**Supported**	`.module.css` files work natively (Lightning CSS).
CSS Nesting	**Supported**	Lightning CSS supports modern CSS nesting.
@import syntax	**Supported**	Combine multiple CSS files.
PostCSS	**Supported**	Automatically processes PostCSS config files (`postcss.config.js` , `.mjs` , `.cjs` , `.ts` , `.mts` , `.cts`) in a Node.js worker pool. Useful for Tailwind, Autoprefixer, etc.
Sass / SCSS	**Supported** (Next.js)	For Next.js, Sass is supported out of the box. Custom Sass functions (`sassOptions.functions`) are not supported because Turbopack's Rust-based architecture cannot directly execute JavaScript functions, unlike webpack's Node.js environment. Use webpack if you need this feature. In the future, Turbopack standalone usage will likely require a loader config.
Less	Planned via plugins	Not yet supported by default. Will likely require a loader config once custom loaders are stable.
Lightning CSS	**In Use**	Handles CSS transformations. Some low-usage CSS Modules features (like `:local/:global` as standalone pseudo-classes) are not yet supported. See below for more details.

Assets

Feature	Status	Notes
Static Assets (images, fonts)	**Supported**	Importing `import img from './img.png'` works out of the box. In Next.js, returns an object for the `



Illustration

`component. | | **JSON Imports** | **Supported** | Named or default imports from` .json` are supported. |

Module resolution

Feature	Status	Notes
Path Aliases	**Supported**	Reads `tsconfig.json`'s `paths` and `baseUrl`, matching Next.js behavior.
Manual Aliases	**Supported**	Configure `resolveAlias` in `next.config.js` (similar to `webpack.resolve.alias`).
Custom Extensions	**Supported**	Configure `resolveExtensions` in `next.config.js`.
AMD	Partially Supported	Basic transforms work; advanced AMD usage is limited.

Performance and Fast Refresh

Feature	Status	Notes
Fast Refresh	**Supported**	Updates JavaScript, TypeScript, and CSS without a full refresh.
Incremental Bundling	**Supported**	Turbopack lazily builds only what's requested by the dev server, speeding up large apps.
FileSystem Cache	**Supported**	Persists compiler artifacts to disk between runs. See `turbopackFileSystemCache`.

Magic Comments

Turbopack supports webpack-compatible magic comments for controlling import behavior. These comments work with dynamic `import()`, `require()`, `require.resolve()`, and `new Worker()` expressions (not static `import` statements).

Comment	Webpack	Turbopack	Description
`webpackIgnore: true`	✓	✓	Skip bundling, preserve import
`turbopackIgnore: true`	✗	✓	Skip bundling (Turbopack-only)
`turbopackOptional: true`	✗	✓	Suppress resolve errors
`webpackOptional: true`	✗	✗	Not supported

See Lazy Loading for usage examples.

import.meta.env

Turbopack supports built-in environment metadata through `import.meta.env`:

Property	Type	Value
`DEV`	`boolean`	Whether `MODE` is not `"production"`
`PROD`	`boolean`	Whether `MODE` is `"production"`
`MODE`	`string`	The compile-time `NODE_ENV`, defaulting to `"development"`
`BASE_URL`	`string`	The Next.js `basePath`, with a trailing slash (`"/"` by default)
`SSR`	`boolean`	`true` in server bundles and `false` in browser and client bundles

These values are statically analyzed, so Turbopack can remove unreachable branches:

```
// Archivo: app/example.ts
if (import.meta.env.DEV) {
  console.log('development mode')
}
```

You can also read the whole object, destructure it, or use static bracket access:

```
// Archivo: app/example.ts
const { MODE, SSR } = import.meta.env
const baseUrl = import.meta.env['BASE_URL']
```

> **Good to know**: `import.meta.env` requires Turbopack. Custom `VITE_*` variables, Vite custom modes, `envPrefix`, and `envDir` are not supported. `BASE_URL` reflects the Next.js `basePath` configuration and includes a trailing slash to match Vite's format.

import.meta.glob

Turbopack supports `import.meta.glob()`, a Vite-compatible API for importing multiple modules at once using glob patterns. The result is an object keyed by the file path relative to the calling file.

```
const modules = import.meta.glob('./dir/*.js')
// {
//   './dir/foo.js': () => import('./dir/foo.js'),
//   './dir/bar.js': () => import('./dir/bar.js'),
// }
```

> **Good to know:** `import.meta.glob` requires Turbopack. It is not available when using webpack.

Lazy loading (default)

By default, each value in the result object is a thunk — a function that returns a `Promise` for the module. This enables lazy loading:

```
const modules = import.meta.glob('./dir/*.js')

for (const path in modules) {
  const module = await modules[path]()
  console.log(path, module)
}
```

Eager loading

Pass `{ eager: true }` to import all matching modules up front. Each value is the module object directly instead of a thunk:

```
const modules = import.meta.glob('./dir/*.js', { eager: true })

for (const path in modules) {
  console.log(path, modules[path].default)
}
```

Named imports

Use the `import` option to select a specific named export from each matched module. This works in both lazy and eager modes:

```
// Lazy: each value is () => Promise<exportValue>
const defaults = import.meta.glob('./dir/*.js', { import: 'default' })

// Eager: each value is the export value directly
const setups = import.meta.glob('./dir/*.js', { import: 'setup', eager: true })
```

Query strings

Use the `query` option to append a query string to every import request. This is useful for loading files as raw strings or URLs:

```
const rawFiles = import.meta.glob('./dir/*.txt', { query: '?raw' })
const urls = import.meta.glob('./dir/*.png', { query: '?url' })
```

The `query` option also accepts an object. Keys and values are URL-encoded and joined into a query string:

```
const modules = import.meta.glob('./*.ts', {
  query: { bar: 'foo', raw: true },
})
// equivalent to: { query: '?bar=foo&raw=true' }
```

Multiple patterns and negation

Pass an array of glob patterns as the first argument. Prefix a pattern with `!` to exclude matching files:

```
// Combine multiple directories
const modules = import.meta.glob(['./dir/*.js', './other/*.js'])

// Exclude specific files
const withoutTests = import.meta.glob(['./src/**/*.js', '!**/*.test.js'])
```

TypeScript

TypeScript types for `import.meta.glob` are included in Next.js. They are available automatically when `"moduleResolution": "bundler"` (or `"node16"` / `"nodenext"`) is set in your `tsconfig.json`, which is the default for new Next.js projects.

The return type differs based on the `eager` option:

```
// Lazy (default) — Record<string, () => Promise<unknown>>
const lazy = import.meta.glob('./dir/*.ts')

// Eager — Record<string, unknown>
const eager = import.meta.glob('./dir/*.ts', { eager: true })
```

Options reference

Option	Type	Default	Description
`eager`	`boolean`	`false`	Import modules synchronously instead of returning thunks.
`import`	`string`	`undefined`	Named export to select from each matched module (e.g. `'default'`).
`query`	`string | Record<string, string | boolean>`	`undefined`	Query string (or object) to append to each import.
`base`	`string`	`undefined`	Override the base path used for resolving patterns and keying results.
`caseSensitive`	`boolean`	`true`	Match glob patterns case-sensitively. Set to `false` to ignore ASCII case.

> **Note:** The `as` option (deprecated in Vite 5) is not supported. Use `query: '?raw'` or `query: '?url'` instead. The legacy `import.meta.globEager()` API is also not supported — use `import.meta.glob('...', { eager: true })` instead.

Known gaps with webpack

There are a number of non-trivial behavior differences between webpack and Turbopack that are important to be aware of when migrating an application. Generally, these are less of a concern for new applications.

Filesystem Root

Turbopack uses the root directory to resolve modules. Files outside of the project root are not resolved.

For example, when linking dependencies outside the project root (via `npm link`, `yarn link`, `pnpm link`, etc.), those linked files will not be resolved by default. To resolve these files, you must configure the root option to the parent directory of both the project and the linked dependencies.

You can configure the filesystem root using turbopack.root option in `next.config.js`.

CSS Module Ordering

Turbopack will follow JS import order to order CSS modules which are not otherwise ordered. For example:

```
// Archivo: components/BlogPost.jsx
import utilStyles from './utils.module.css'
import buttonStyles from './button.module.css'
export default function BlogPost() {
  return (
    <div className={utilStyles.container}>
      <button className={buttonStyles.primary}>Click me</button>
    </div>
  )
}
```

In this example, Turbopack will ensure that `utils.module.css` will appear before `button.module.css` in the produced CSS chunk, following the import order

Webpack generally does this as well, but there are cases where it will ignore JS inferred ordering, for example if it infers the JS file is side-effect-free.

This can lead to subtle rendering changes when adopting Turbopack, if applications have come to rely on an arbitrary ordering. Generally, the solution is easy, e.g. have `button.module.css` `@import utils.module.css` to force the ordering, or identify the conflicting rules and change them to not target the same properties.

Sass node_modules imports

Turbopack supports importing `node_modules` Sass files out of the box. Webpack supports a legacy tilde `~` syntax for this, which is not supported by Turbopack.

From:

```
// Archivo: styles/globals.scss
@import '~bootstrap/dist/css/bootstrap.min.css';
```

To:

```
// Archivo: styles/globals.scss
@import 'bootstrap/dist/css/bootstrap.min.css';
```

If you can't update the imports, you can add a `turbopack.resolveAlias` configuration to map the `~` syntax to the actual path:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      '~*': '*',
    },
  },
}
```

CSS / Sass / SCSS decimal precision

Turbopack uses Lightning CSS to compile CSS. Lightning CSS uses 5 digits of decimal precision for numeric CSS values, while webpack uses 10 digits. This applies to both plain CSS and Sass/SCSS output. For example, a value that evaluates to `25/17` produces:

- **Webpack:** `line-height: 1.4705882353` (10 digits)
- **Turbopack:** `line-height: 1.47059` (5 digits)

This can lead to subtle rendering differences when migrating from webpack to Turbopack, especially for properties like `line-height`, `letter-spacing`, or other calculated values where high precision matters.

Webpack plugins

Turbopack does not support webpack plugins. This affects third-party tools that rely on webpack's plugin system for integration. We do support webpack loaders. If you depend on webpack plugins, you'll need to find Turbopack-compatible alternatives or continue using webpack until equivalent functionality is available.

Unsupported and unplanned features

Some features are not yet implemented or not planned:

- **Legacy CSS Modules features**
  - Standalone `:local` and `:global` pseudo-classes (only the function variant `:global( ... )` is supported).
  - The `@value` rule (superseded by CSS variables).
  - `:import` and `:export` ICSS rules.
  - `composes` in `.module.css` composing a `.css` file. In webpack this would treat the `.css` file as a CSS Module, with Turbopack the `.css` file will always be global. This means that if you want to use `composes` in a CSS Module, you need to change the `.css` file to a `.module.css` file.
  - `@import` in CSS Modules importing `.css` as a CSS Module. In webpack this would treat the `.css` file as a CSS Module, with Turbopack the `.css` file will always be global. This means that if you want to use `@import` in a CSS Module, you need to change the `.css` file to a `.module.css` file.
- `sassOptions.functions` Custom Sass functions defined in `sassOptions.functions` are not supported. This feature allows defining JavaScript functions that can be called from Sass code during compilation. Turbopack's Rust-based architecture cannot directly execute JavaScript functions passed through `sassOptions.functions`, unlike webpack's Node.js-based sass-loader which runs entirely in JavaScript. If you're using custom Sass functions, you'll need to use webpack instead of Turbopack.
- `webpack()` **configuration** in `next.config.js` Turbopack replaces webpack, so `webpack()` configs are not recognized. Use the `turbopack` config instead.
- **Yarn PnP** Not planned for Turbopack support in Next.js.
- `experimental.urlImports` Not planned for Turbopack.
- `experimental.esmExternals` Not planned. Turbopack does not support the legacy `esmExternals` configuration in Next.js.
- **Some Next.js Experimental Flags**
  - `experimental.nextScriptWorkers`
  - `experimental.fallbackNodePolyfills` We plan to implement these in the future.

For a full, detailed breakdown of each feature flag and its status, see the Turbopack API Reference.

Configuration

Turbopack can be configured via `next.config.js` (or `next.config.ts`) under the `turbopack` key. Configuration options include:

- **`rules`** Define additional webpack loaders for file transformations.
- **`resolveAlias`** Create manual aliases (like `resolve.alias` in webpack).
- **`resolveExtensions`** Change or extend file extensions for module resolution.
- **`ignoreIssue`** Suppress specific Turbopack errors and warnings from the CLI output and error overlay.

Additionally, the following experimental options are available under `experimental` in `next.config.js`:

Option	Description	Default (dev)	Default (build)
`turbopackFileSystemCacheForDev`	Enable filesystem cache for the dev server.	`true`	N/A
`turbopackFileSystemCacheForBuild`	Enable filesystem cache for builds.	N/A	`true` [1]
`turbopackMinify`	Enable minification.	`false`	`true`
`turbopackSourceMaps`	Enable source maps.	`true`	`productionBrowserSourceMaps`
`turbopackInputSourceMaps`	Enable extraction of source maps from input files.	`true`	`true`
`turbopackModuleFragments`	Currently in active development. This splits modules into fragments and chunks only import the used fragments of the modules.	`false`	`false`
`turbopackRemoveUnusedImports`	Enable removing unused imports. Requires `turbopackRemoveUnusedExports`.	`false`	`true`
`turbopackRemoveUnusedExports`	Enable removing unused exports.	`false`	`true`
`turbopackInferModuleSideEffects`	Enable local analysis to infer side-effect-free modules for better tree shaking.	`true`	`true`
`turbopackScopeHoisting`	Enable scope hoisting. Always disabled in dev mode.	`false`	`true`
`turbopackClientSideNestedAsyncChunking`	Enable nested async chunking for client-side assets.	`false`	`true`
`turbopackServerSideNestedAsyncChunking`	Enable nested async chunking for server-side assets.	`false`	`false`
`turbopackImportTypeBytes`	Enable support for `with {type: "bytes"}` for ESM imports.	`false`	`false`
`turbopackUseBuiltinBabel`	Enable automatic Babel loader configuration when a Babel config file is present.	`true`	`true`
`turbopackUseBuiltinSass`	Enable automatic Sass loader configuration.	`true`	`true`
`turbopackModuleIds`	Module ID strategy: `'named'` or `'deterministic'`.	`'named'`	`'deterministic'`
`turbopackLocalPostcssConfig`	Resolve `postcss.config.js` from the CSS file's directory first, then the project root.	`false`	`false`
`turbopackWorkerAssetPrefix`	Custom asset prefix for Web Worker URLs (entrypoint + module chunks), overriding `assetPrefix`. Mirrors webpack's `output.workerPublicPath`.	`undefined`	`undefined`

Table notes:

1. Enabled by default. Set the option to `false` when the build environment does not preserve the `.next/cache` directory between builds.

For example, to configure an alias and a custom file extension under the `turbopack` key:

```
// Archivo: next.config.js
module.exports = {
  turbopack: {
    resolveAlias: {
      underscore: 'lodash',
    },
    resolveExtensions: ['.mdx', '.tsx', '.ts', '.jsx', '.js', '.json'],
  },
}
```

For more in-depth configuration examples, see the Turbopack config documentation.

Generating trace files for performance debugging

If you encounter performance or memory issues and want to help the Next.js team diagnose them, you can generate a trace file by adding the `--internal-trace` flag to your dev command:

```
next dev --internal-trace
```

This will produce a `.next-profiles/trace-turbopack.bin` file. Include that file when creating a GitHub issue on the Next.js repo to help us investigate.

Summary

Turbopack is a **Rust-based**, **incremental** bundler designed to make local development and builds fast—especially for large applications. It is integrated into Next.js, offering zero-config CSS, React, and TypeScript support.

Version Changes

Version	Changes
`v16.0.0`	Turbopack becomes the default bundler for Next.js. Automatic support for Babel when a configuration file is found.
`v15.5.0`	Turbopack support for `build` beta
`v15.3.0`	Experimental support for `build`
`v15.0.0`	Turbopack for `dev` stable

API Reference

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/app/api-reference
> 📁 **Sección:** App > Api Reference
> 📝 **Descripción:** *Next.js API Reference for the App Router.*

Capítulo 12: Pages Router (Soporte Legacy) y Estrategias de Migración

> *getStaticProps, getServerSideProps, getInitialProps, API Routes y guía paso a paso para migrar hacia App Router.*

Create a new Next.js application

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/installation
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *How to create a new Next.js application with* `create-next-app`*. Set up TypeScript, ESLint,and configure your* `next.config.js` *file.*

Project Structure and Organization

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/project-structure
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Learn about the folder and file conventions in a Next.js project, and how to organize your project.*

Image Optimization

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/images
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Optimize your images with the built-in* `next/image` *component.*

How to use fonts

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/fonts
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Learn how to use fonts in Next.js*

How to use CSS in your application

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/css
📁 **Sección:** Pages > Getting Started
📝 **Descripción:** *Learn about the different ways to add CSS to your application, including CSS Modules, Global CSS, Tailwind CSS, and more.*

How to deploy your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started/deploying
> 📁 **Sección:** Pages > Getting Started
> 📝 **Descripción:** *Learn how to deploy your Next.js application.*

Getting Started - Pages Router

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/getting-started
> 📁 **Sección:** Pages > Getting Started
> 📝 **Descripción:** *Learn how to create full-stack web applications with Next.js with the Pages Router.*

How to set up analytics

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/analytics
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Measure and track page performance using Next.js*

How to implement authentication in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/authentication
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to implement authentication in Next.js, covering best practices, securing routes, authorization techniques, and session management.*

How to configure Babel in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/babel
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Extend the babel preset added by Next.js with your own configs.*

Examples

Next.js includes the `next/babel` preset to your app, which includes everything needed to compile React applications and server-side code. But if you want to extend the default Babel configs, it's also possible.

Adding Presets and Plugins

To start, you only need to define a `.babelrc` file (or `babel.config.js`) in the root directory of your project. If such a file is found, it will be considered as the *source of truth*, and therefore it needs to define what Next.js needs as well, which is the `next/babel` preset.

Here's an example `.babelrc` file:

```
// Archivo: .babelrc
{
  "presets": ["next/babel"],
  "plugins": []
}
```

You can take a look at this file to learn about the presets included by `next/babel`.

To add presets/plugins **without configuring them**, you can do it this way:

```
// Archivo: .babelrc
{
  "presets": ["next/babel"],
  "plugins": ["@babel/plugin-proposal-do-expressions"]
}
```

Customizing Presets and Plugins

To add presets/plugins **with custom configuration**, do it on the `next/babel` preset like so:

```
// Archivo: .babelrc
{
  "presets": [
    [
      "next/babel",
      {
        "preset-env": {},
        "transform-runtime": {},
        "styled-jsx": {},
        "class-properties": {}
      }
    ]
  ],
  "plugins": []
}
```

To learn more about the available options for each config, visit babel's documentation site.

> **Good to know**:
>
> - Next.js uses the **current** Node.js version for server-side compilations.
> - The `modules` option on `"preset-env"` should be kept to `false`, otherwise webpack code splitting is turned off.

How to configure Continuous Integration (CI) build caching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/ci-build-caching
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to configure CI to cache Next.js builds*

How to set a Content Security Policy (CSP) for your Next.js application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/content-security-policy
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to set a Content Security Policy (CSP) for your Next.js application.*

How to use CSS-in-JS libraries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/css-in-js
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Use CSS-in-JS libraries with Next.js*

How to set up a custom server in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/custom-server
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Start a Next.js app programmatically using a custom server.*

How to use debugging tools with Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/debugging
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to debug your Next.js application with VS Code or Chrome DevTools.*

How to preview content with Draft Mode in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/draft-mode
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Next.js has draft mode to toggle between static and dynamic pages. You can learn how it works with Pages Router.*

In the Pages documentation and the Data Fetching documentation, we talked about how to prerender a page at build time (**Static Generation**) using `getStaticProps` and `getStaticPaths`.

Static Generation is useful when your pages fetch data from a headless CMS. However, it's not ideal when you're writing a draft on your headless CMS and want to view the draft immediately on your page. You'd want Next.js to render these pages at **request time** instead of build time and fetch the draft content instead of the published content. You'd want Next.js to bypass Static Generation only for this specific case.

Next.js has a feature called **Draft Mode** which solves this problem. Here are instructions on how to use it.

Step 1: Create and access the API route

> Take a look at the API Routes documentation first if you're not familiar with Next.js API Routes.

First, create the **API route**. It can have any name - e.g. `pages/api/draft.ts`

In this API route, you need to call `setDraftMode` on the response object.

```
export default function handler(req, res) {
  // ...
  res.setDraftMode({ enable: true })
  // ...
}
```

This will set a **cookie** to enable draft mode. Subsequent requests containing this cookie will trigger **Draft Mode** changing the behavior for statically generated pages (more on this later).

You can test this manually by creating an API route like below and accessing it from your browser manually:

```
// Archivo: pages/api/draft.ts
// simple example for testing it manually from your browser.
export default function handler(req, res) {
  res.setDraftMode({ enable: true })
  res.end('Draft mode is enabled')
}
```

If you open your browser's developer tools and visit `/api/draft`, you'll notice a `Set-Cookie` response header with a cookie named `__prerender_bypass`.

Securely accessing it from your Headless CMS

In practice, you'd want to call this API route *securely* from your headless CMS. The specific steps will vary depending on which headless CMS you're using, but here are some common steps you could take.

These steps assume that the headless CMS you're using supports setting **custom draft URLs**. If it doesn't, you can still use this method to secure your draft URLs, but you'll need to construct and access the draft URL manually.

First, you should create a **secret token string** using a token generator of your choice. This secret will only be known by your Next.js app and your headless CMS. This secret prevents people who don't have access to your CMS from accessing draft URLs.

Second, if your headless CMS supports setting custom draft URLs, specify the following as the draft URL. This assumes that your draft API route is located at `pages/api/draft.ts`.

```
// Archivo: Terminal
https://<your-site>/api/draft?secret=<token>&slug=<path>
```

- `<your-site>` should be your deployment domain.
- `<token>` should be replaced with the secret token you generated.
- `<path>` should be the path for the page that you want to view. If you want to view `/posts/foo`, then you should use `&slug=/posts/foo`.

Your headless CMS might allow you to include a variable in the draft URL so that `<path>` can be set dynamically based on the CMS's data like so: `&slug=/posts/{entry.fields.slug}`

Finally, in the draft API route:

- Check that the secret matches and that the `slug` parameter exists (if not, the request should fail).
-
- Call `res.setDraftMode`.

- Then redirect the browser to the path specified by `slug`. (The following example uses a 307 redirect).

```
export default async (req, res) => {
  // Check the secret and next parameters
  // This secret should only be known to this API route and the CMS
  if (req.query.secret !== 'MY_SECRET_TOKEN' || !req.query.slug) {
    return res.status(401).json({ message: 'Invalid token' })
  }

  // Fetch the headless CMS to check if the provided `slug` exists
  // getPostBySlug would implement the required fetching logic to the headless CMS
  const post = await getPostBySlug(req.query.slug)

  // If the slug doesn't exist prevent draft mode from being enabled
  if (!post) {
    return res.status(401).json({ message: 'Invalid slug' })
  }

  // Enable Draft Mode by setting the cookie
  res.setDraftMode({ enable: true })

  // Redirect to the path from the fetched post
  // We don't redirect to req.query.slug as that might lead to open redirect vulnerabilities
  res.redirect(post.slug)
}
```

If it succeeds, then the browser will be redirected to the path you want to view with the draft mode cookie.

Step 2: Update `getStaticProps`

The next step is to update `getStaticProps` to support draft mode.

If you request a page which has `getStaticProps` with the cookie set (via `res.setDraftMode`), then `getStaticProps` will be called at **request time** (instead of at build time).

Furthermore, it will be called with a `context` object where `context.draftMode` will be `true`.

```
export async function getStaticProps(context) {
  if (context.draftMode) {
    // dynamic data
  }
}
```

We used `res.setDraftMode` in the draft API route, so `context.draftMode` will be `true`.

If you're also using `getStaticPaths`, then `context.params` will also be available.

Fetch draft data

You can update `getStaticProps` to fetch different data based on `context.draftMode`.

For example, your headless CMS might have a different API endpoint for draft posts. If so, you can modify the API endpoint URL like below:

```
export async function getStaticProps(context) {
  const url = context.draftMode
    ? 'https://draft.example.com'
    : 'https://production.example.com'
  const res = await fetch(url)
  // ...
}
```

That's it! If you access the draft API route (with `secret` and `slug`) from your headless CMS or manually, you should now be able to see the draft content. And if you update your draft without publishing, you should be able to view the draft.

Set this as the draft URL on your headless CMS or access manually, and you should be able to see the draft.

```
// Archivo: Terminal
https://<your-site>/api/draft?secret=<token>&slug=<path>
```

More Details

Clear the Draft Mode cookie

By default, the Draft Mode session ends when the browser is closed.

To clear the Draft Mode cookie manually, create an API route that calls `setDraftMode({ enable: false })`:

```
// Archivo: pages/api/disable-draft.ts
export default function handler(req, res) {
  res.setDraftMode({ enable: false })
}
```

Then, send a request to `/api/disable-draft` to invoke the API Route. If calling this route using `next/link`, you must pass `prefetch={false}` to prevent accidentally deleting the cookie on prefetch.

Works with `getServerSideProps`

Draft Mode works with `getServerSideProps`, and is available as a `draftMode` key in the `context` object.

> **Good to know**: You shouldn't set the `Cache-Control` header when using Draft Mode because it cannot be bypassed. Instead, we recommend using ISR.

Works with API Routes

API Routes will have access to `draftMode` on the request object. For example:

```
export default function myApiRoute(req, res) {
  if (req.draftMode) {
    // get draft data
  }
}
```

Unique per `next build`

A new bypass cookie value will be generated each time you run `next build`.

This ensures that the bypass cookie can't be guessed.

> **Good to know**: To test Draft Mode locally over HTTP, your browser will need to allow third-party cookies and local storage access.

How to use environment variables in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/environment-variables
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn to add and access environment variables in your Next.js application.*

How to create forms with API Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/forms
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to handle form submissions and data mutations with Next.js.*

Forms enable you to create and update data in web applications. Next.js provides a powerful way to handle data mutations using **API Routes**. This guide will walk you through how to handle form submission on the server.

Server Forms

To handle form submissions on the server, create an API endpoint that securely mutates data.

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const data = req.body
  const id = await createItem(data)
  res.status(200).json({ id })
}
```

```
// Archivo: pages/api/submit.js switcher
export default function handler(req, res) {
  const data = req.body
  // call your database, etc.
  // const id = await createItem(data)
  // ...
  res.status(200).json({ data })
}
```

Then, call the API Route from the client with an event handler:

```
// Archivo: pages/index.tsx switcher
import { SubmitEvent } from 'react'

export default function Page() {
  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const response = await fetch('/api/submit', {
      method: 'POST',
      body: formData,
    })

    // Handle response if necessary
    const data = await response.json()
    // ...
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  )
}
```

```
// Archivo: pages/index.jsx switcher
export default function Page() {
  async function onSubmit(event) {
    event.preventDefault()

    const formData = new FormData(event.currentTarget)
    const response = await fetch('/api/submit', {
      method: 'POST',
      body: formData,
    })

    // Handle response if necessary
    const data = await response.json()
    // ...
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  )
}
```

Good to know:

- API Routes do not specify CORS headers, meaning they are same-origin only by default.
- Since API Routes run on the server, we're able to use sensitive values (like API keys) through Environment Variables without exposing them to the client. This is critical for the security of your application.

Form validation

We recommend using HTML validation like `required` and `type="email"` for basic client-side form validation.

For more advanced server-side validation, you can use a schema validation library like Zod or Valibot to validate the form fields before mutating the data:

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'
import { z } from 'zod'

const schema = z.object({
  // ...
})

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const parsed = schema.parse(req.body)
  // ...
}
```

```
// Archivo: pages/api/submit.js switcher
import { z } from 'zod'

const schema = z.object({
  // ...
})

export default async function handler(req, res) {
  const parsed = schema.parse(req.body)
  // ...
}
```

Error handling

You can use React state to show an error message when a form submission fails:

```
// Archivo: pages/index.tsx switcher
import React, { useState, SubmitEvent } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState<boolean>(false)
  const [error, setError] = useState<string | null>(null)

  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()
    setIsLoading(true)
    setError(null) // Clear previous errors when a new request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      if (!response.ok) {
        throw new Error('Failed to submit the data. Please try again.')
      }

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Capture the error message to display to the user
      setError(error.message)
      console.error(error)
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div>
      {error && <div style={{ color: 'red' }}>{error}</div>}
      <form onSubmit={onSubmit}>
        <input type="text" name="name" required />
        <button type="submit" disabled={isLoading}>
          {isLoading ? 'Loading ... ' : 'Submit'}
        </button>
      </form>
    </div>
  )
}
```

```
// Archivo: pages/index.jsx switcher
import React, { useState } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState(false)
  const [error, setError] = useState(null)

  async function onSubmit(event) {
    event.preventDefault()
    setIsLoading(true)
    setError(null) // Clear previous errors when a new request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      if (!response.ok) {
        throw new Error('Failed to submit the data. Please try again.')
      }

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Capture the error message to display to the user
      setError(error.message)
      console.error(error)
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div>
      {error && <div style={{ color: 'red' }}>{error}</div>}
      <form onSubmit={onSubmit}>
        <input type="text" name="name" required />
        <button type="submit" disabled={isLoading}>
          {isLoading ? 'Loading ... ' : 'Submit'}
        </button>
      </form>
    </div>
  )
}
```

Displaying loading state

You can use React state to show a loading state when a form is submitting on the server:

```tsx
// Archivo: pages/index.tsx switcher
import React, { useState, SubmitEvent } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState<boolean>(false)

  async function onSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault()
    setIsLoading(true) // Set loading to true when the request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Handle error if necessary
      console.error(error)
    } finally {
      setIsLoading(false) // Set loading to false when the request completes
    }
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Submit'}
      </button>
    </form>
  )
}
```

```
// Archivo: pages/index.jsx switcher
import React, { useState } from 'react'

export default function Page() {
  const [isLoading, setIsLoading] = useState(false)

  async function onSubmit(event) {
    event.preventDefault()
    setIsLoading(true) // Set loading to true when the request starts

    try {
      const formData = new FormData(event.currentTarget)
      const response = await fetch('/api/submit', {
        method: 'POST',
        body: formData,
      })

      // Handle response if necessary
      const data = await response.json()
      // ...
    } catch (error) {
      // Handle error if necessary
      console.error(error)
    } finally {
      setIsLoading(false) // Set loading to false when the request completes
    }
  }

  return (
    <form onSubmit={onSubmit}>
      <input type="text" name="name" />
      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Submit'}
      </button>
    </form>
  )
}
```

Redirecting

If you would like to redirect the user to a different route after a mutation, you can `redirect` to any absolute or relative URL:

```
// Archivo: pages/api/submit.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const id = await addPost()
  res.redirect(307, `/post/${id}`)
}
```

```
// Archivo: pages/api/submit.js switcher
export default async function handler(req, res) {
  const id = await addPost()
  res.redirect(307, `/post/${id}`)
}
```

How to implement Incremental Static Regeneration (ISR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/incremental-static-regeneration
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to create or update static pages at runtime with Incremental Static Regeneration.*

Guides

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to implement common UI patterns and use cases using Next.js*

How to set up instrumentation

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/instrumentation
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to use instrumentation to run code at server startup in your Next.js app*

How to implement internationalization in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/internationalization
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Next.js has built-in support for internationalized routing and language detection. Learn more here.*

Examples

Next.js has built-in support for internationalized (i18n) routing since `v10.0.0`. You can provide a list of locales, the default locale, and domain-specific locales and Next.js will automatically handle the routing.

The i18n routing support is currently meant to complement existing i18n library solutions like `react-intl`, `react-i18next`, `lingui`, `rosetta`, `next-intl`, `next-translate`, `next-multilingual`, `tolgee`, `paraglide-next`, `next-intlayer`, `gt-react` and others by streamlining the routes and locale parsing.

Getting started

To get started, add the `i18n` config to your `next.config.js` file.

Locales are UTS Locale Identifiers, a standardized format for defining locales.

Generally a Locale Identifier is made up of a language, region, and script separated by a dash: `language-region-script`. The region and script are optional. An example:

- `en-US` - English as spoken in the United States
- `nl-NL` - Dutch as spoken in the Netherlands
- `nl` - Dutch, no specific region

If user locale is `nl-BE` and it is not listed in your configuration, they will be redirected to `nl` if available, or to the default locale otherwise. If you don't plan to support all regions of a country, it is therefore a good practice to include country locales that will act as fallbacks.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    // These are all the locales you want to support in
    // your application
    locales: ['en-US', 'fr', 'nl-NL'],
    // This is the default locale you want to be used when visiting
    // a non-locale prefixed path e.g. `/hello`
    defaultLocale: 'en-US',
    // This is a list of locale domains and the default locale they
    // should handle (these are only required when setting up domain routing)
    // Note: subdomains must be included in the domain value to be matched e.g. "fr.example.com".
    domains: [
      {
        domain: 'example.com',
        defaultLocale: 'en-US',
      },
      {
        domain: 'example.nl',
        defaultLocale: 'nl-NL',
      },
      {
        domain: 'example.fr',
        defaultLocale: 'fr',
        // an optional http field can also be used to test
        // locale domains locally with http instead of https
        http: true,
      },
    ],
  },
}
```

Locale Strategies

There are two locale handling strategies: Sub-path Routing and Domain Routing.

Sub-path Routing

Sub-path Routing puts the locale in the url path.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['en-US', 'fr', 'nl-NL'],
    defaultLocale: 'en-US',
  },
}
```

With the above configuration `en-US`, `fr`, and `nl-NL` will be available to be routed to, and `en-US` is the default locale. If you have a `pages/blog.js` the following urls would be available:

- `/blog`
- `/fr/blog`
- `/nl-nl/blog`

The default locale does not have a prefix.

Domain Routing

By using domain routing you can configure locales to be served from different domains:

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['en-US', 'fr', 'nl-NL', 'nl-BE'],
    defaultLocale: 'en-US',

    domains: [
      {
        // Note: subdomains must be included in the domain value to be matched
        // e.g. www.example.com should be used if that is the expected hostname
        domain: 'example.com',
        defaultLocale: 'en-US',
      },
      {
        domain: 'example.fr',
        defaultLocale: 'fr',
      },
      {
        domain: 'example.nl',
        defaultLocale: 'nl-NL',
        // specify other locales that should be redirected
        // to this domain
        locales: ['nl-BE'],
      },
    ],
  },
}
```

For example if you have `pages/blog.js` the following urls will be available:

- `example.com/blog`
- `www.example.com/blog`
- `example.fr/blog`
- `example.nl/blog`
- `example.nl/nl-BE/blog`

Automatic Locale Detection

When a user visits the application root (generally `/`), Next.js will try to automatically detect which locale the user prefers based on the `Accept-Language` header and the current domain.

If a locale other than the default locale is detected, the user will be redirected to either:

- **When using Sub-path Routing:** The locale prefixed path
- **When using Domain Routing:** The domain with that locale specified as the default

When using Domain Routing, if a user with the `Accept-Language` header `fr;q=0.9` visits `example.com`, they will be redirected to `example.fr` since that domain handles the `fr` locale by default.

When using Sub-path Routing, the user would be redirected to `/fr`.

Prefixing the Default Locale

With Next.js 12 and Proxy, we can add a prefix to the default locale with a workaround.

For example, here's a `next.config.js` file with support for a few languages. Note the `"default"` locale has been added intentionally.

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    locales: ['default', 'en', 'de', 'fr'],
    defaultLocale: 'default',
    localeDetection: false,
  },
  trailingSlash: true,
}
```

Next, we can use Proxy to add custom routing rules:

```
// Archivo: proxy.ts
import { NextRequest, NextResponse } from 'next/server'

const PUBLIC_FILE = /\.(.*)$/

export async function proxy(req: NextRequest) {
  if (
    req.nextUrl.pathname.startsWith('/_next') ||
    req.nextUrl.pathname.includes('/api/') ||
    PUBLIC_FILE.test(req.nextUrl.pathname)
  ) {
    return
  }

  if (req.nextUrl.locale === 'default') {
    const locale = req.cookies.get('NEXT_LOCALE')?.value || 'en'

    return NextResponse.redirect(
      new URL(`/${locale}${req.nextUrl.pathname}${req.nextUrl.search}`, req.url)
    )
  }
}
```

This Proxy skips adding the default prefix to API Routes and public files like fonts or images. If a request is made to the default locale, we redirect to our prefix `/en`.

Disabling Automatic Locale Detection

The automatic locale detection can be disabled with:

```
// Archivo: next.config.js
module.exports = {
  i18n: {
    localeDetection: false,
  },
}
```

When `localeDetection` is set to `false` Next.js will no longer automatically redirect based on the user's preferred locale and will only provide locale information detected from either the locale based domain or locale path as described above.

Accessing the locale information

You can access the locale information via the Next.js router. For example, using the `useRouter()` hook the following properties are available:

- `locale` contains the currently active locale.
- `locales` contains all configured locales.
- `defaultLocale` contains the configured default locale.

When prerendering pages with `getStaticProps` or `getServerSideProps`, the locale information is provided in the context provided to the function.

When leveraging `getStaticPaths`, the configured locales are provided in the context parameter of the function under `locales` and the configured defaultLocale under `defaultLocale`.

Transition between locales

You can use `next/link` or `next/router` to transition between locales.

For `next/link`, a `locale` prop can be provided to transition to a different locale from the currently active one. If no `locale` prop is provided, the currently active `locale` is used during client-transitions. For example:

```
import Link from 'next/link'

export default function IndexPage(props) {
  return (
    <Link href="/another" locale="fr">
      To /fr/another
    </Link>
  )
}
```

When using the `next/router` methods directly, you can specify the `locale` that should be used via the transition options. For example:

```
import { useRouter } from 'next/router'

export default function IndexPage(props) {
  const router = useRouter()

  return (
    <div
      onClick={() => {
        router.push('/another', '/another', { locale: 'fr' })
      }}
    >
      to /fr/another
    </div>
  )
}
```

Note that to handle switching only the `locale` while preserving all routing information such as dynamic route query values or hidden href query values, you can provide the `href` parameter as an object:

```
import { useRouter } from 'next/router'
const router = useRouter()
const { pathname, asPath, query } = router
// change just the locale and maintain all other route information including href's query
router.push({ pathname, query }, asPath, { locale: nextLocale })
```

See here for more information on the object structure for `router.push`.

If you have a `href` that already includes the locale you can opt-out of automatically handling the locale prefixing:

```
import Link from 'next/link'

export default function IndexPage(props) {
  return (
    <Link href="/fr/another" locale={false}>
      To /fr/another
    </Link>
  )
}
```

Leveraging the `NEXT_LOCALE` cookie

Next.js allows setting a `NEXT_LOCALE=the-locale` cookie, which takes priority over the accept-language header. This cookie can be set using a language switcher and then when a user comes back to the site it will leverage the locale specified in the cookie when redirecting from `/` to the correct locale location.

For example, if a user prefers the locale `fr` in their accept-language header but a `NEXT_LOCALE=en` cookie is set the `en` locale when visiting `/` the user will be redirected to the `en` locale location until the cookie is removed or expired.

Search Engine Optimization

Since Next.js knows what language the user is visiting it will automatically add the `lang` attribute to the `<html>` tag.

Next.js doesn't know about variants of a page so it's up to you to add the `hreflang` meta tags using `next/head`. You can learn more about `hreflang` in the Google Webmasters documentation.

How does this work with Static Generation?

> Note that Internationalized Routing does not integrate with `output: 'export'` as it does not leverage the Next.js routing layer. Hybrid Next.js applications that do not use `output: 'export'` are fully supported.

Dynamic Routes and `getStaticProps` Pages

For pages using `getStaticProps` with Dynamic Routes, all locale variants of the page desired to be prerendered need to be returned from `getStaticPaths`. Along with the `params` object returned for `paths`, you can also return a `locale` field specifying which locale you want to render. For example:

```
// Archivo: pages/blog/[slug].js
export const getStaticPaths = ({ locales }) => {
  return {
    paths: [
      // if no `locale` is provided only the defaultLocale will be generated
      { params: { slug: 'post-1' }, locale: 'en-US' },
      { params: { slug: 'post-1' }, locale: 'fr' },
    ],
    fallback: true,
  }
}
```

For Automatically Statically Optimized and non-dynamic `getStaticProps` pages, **a version of the page will be generated for each locale**. This is important to consider because it can increase build times depending on how many locales are configured inside `getStaticProps`.

For example, if you have 50 locales configured with 10 non-dynamic pages using `getStaticProps`, this means `getStaticProps` will be called 500 times. 50 versions of the 10 pages will be generated during each build.

To decrease the build time of dynamic pages with `getStaticProps`, use a `fallback` mode. This allows you to return only the most popular paths and locales from `getStaticPaths` for prerendering during the build. Then, Next.js will build the remaining pages at runtime as they are requested.

Automatically Statically Optimized Pages

For pages that are automatically statically optimized, a version of the page will be generated for each locale.

Non-dynamic getStaticProps Pages

For non-dynamic `getStaticProps` pages, a version is generated for each locale like above. `getStaticProps` is called with each `locale` that is being rendered. If you would like to opt-out of a certain locale from being prerendered, you can return `notFound: true` from `getStaticProps` and this variant of the page will not be generated.

```
export async function getStaticProps({ locale }) {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch(`https:// ... /posts?locale=${locale}`)
  const posts = await res.json()

  if (posts.length === 0) {
    return {
      notFound: true,
    }
  }

  // By returning { props: posts }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

Limits for the i18n config

- `locales` : 100 total locales
- `domains` : 100 total locale domain items

> **Good to know**: These limits have been added initially to prevent potential performance issues at build time. You can workaround these limits with custom routing using Proxy in Next.js 12.

How to lazy load Client Components and libraries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/lazy-loading
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Lazy load imported libraries and React Components to improve your application's overall loading performance.*

How to use markdown and MDX in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/mdx
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to configure MDX to write JSX in your markdown files.*

How to migrate from Pages to the App Router

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/migrating/app-router-migration
> 📁 **Sección:** Pages > Guides > Migrating
> 📝 **Descripción:** *Learn how to upgrade your existing Next.js application from the Pages Router to the App Router.*

How to migrate from Create React App to Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/migrating/from-create-react-app
> 📁 **Sección:** Pages > Guides > Migrating
> 📝 **Descripción:** *Learn how to migrate your existing React application from Create React App to Next.js.*

How to migrate from Vite to Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/migrating/from-vite
> 📁 **Sección:** Pages > Guides > Migrating
> 📝 **Descripción:** *Learn how to migrate your existing React application from Vite to Next.js.*

Migrating

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/migrating
> 📁 **Sección:** Pages > Guides > Migrating
> 📝 **Descripción:** *Learn how to migrate from popular frameworks to Next.js*

How to build micro-frontends using multi-zones and Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/multi-zones
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to build micro-frontends using Next.js Multi-Zones to deploy multiple Next.js apps under a single domain.*

How to instrument your Next.js app with OpenTelemetry

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/open-telemetry
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to instrument your Next.js app with OpenTelemetry.*

How to optimize package bundling

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/package-bundling
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn how to optimize your application's server and client bundles.*

How to configure PostCSS in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/post-css
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Extend the PostCSS config and plugins added by Next.js with your own.*

Default Behavior

Next.js compiles CSS for its built-in CSS support using PostCSS.

Out of the box, with no configuration, Next.js compiles CSS with the following transformations:

- Autoprefixer automatically adds vendor prefixes to CSS rules (back to IE11).
- Cross-browser Flexbox bugs are corrected to behave like the spec.
- New CSS features are automatically compiled for Internet Explorer 11 compatibility:
  - `all` Property
  - Break Properties
  - `font-variant` Property
  - Gap Properties
  - Media Query Ranges

By default, CSS Grid and Custom Properties (CSS variables) are **not compiled** for IE11 support.

To compile CSS Grid Layout for IE11, you can place the following comment at the top of your CSS file:

```
/* autoprefixer grid: autoplace */
```

You can also enable IE11 support for CSS Grid Layout in your entire project by configuring autoprefixer with the configuration shown below (collapsed). See "Customizing Plugins" below for more information.

Click to view the configuration to enable CSS Grid Layout

CSS variables are not compiled because it is not possible to safely do so. If you must use variables, consider using something like Sass variables which are compiled away by Sass.

Customizing Target Browsers

Next.js allows you to configure the target browsers (for Autoprefixer and compiled css features) through Browserslist.

To customize browserslist, create a `browserslist` key in your `package.json` like so:

```
// Archivo: package.json
{
  "browserslist": [">0.3%", "not dead", "not op_mini all"]
}
```

You can use the browsersl.ist tool to visualize what browsers you are targeting.

CSS Modules

No configuration is needed to support CSS Modules. To enable CSS Modules for a file, rename the file to have the extension `.module.css`.

You can learn more about Next.js' CSS Module support here.

Customizing Plugins

> **Warning**: When you define a custom PostCSS configuration file, Next.js **completely disables** the default behavior. Be sure to manually configure all the features you need compiled, including Autoprefixer. You also need to install any plugins included in your custom configuration manually, i.e. `npm install postcss-flexbugs-fixes postcss-preset-env`.

To customize the PostCSS configuration, create a `postcss.config.json` file in the root of your project.

This is the default configuration used by Next.js:

```
// Archivo: postcss.config.json
{
  "plugins": [
    "postcss-flexbugs-fixes",
    [
      "postcss-preset-env",
      {
        "autoprefixer": {
          "flexbox": "no-2009"
        },
        "stage": 3,
        "features": {
          "custom-properties": false
        }
      }
    ]
  ]
}
```

> **Good to know**: Next.js also allows the file to be named `.postcssrc.json`, or, to be read from the `postcss` key in `package.json`.

It is also possible to configure PostCSS with a `postcss.config.js` file, which is useful when you want to conditionally include plugins based on environment:

```
// Archivo: postcss.config.js
module.exports = {
  plugins:
    process.env.NODE_ENV === 'production'
      ? [
          'postcss-flexbugs-fixes',
          [
            'postcss-preset-env',
            {
              autoprefixer: {
                flexbox: 'no-2009',
              },
              stage: 3,
              features: {
                'custom-properties': false,
              },
            },
          ],
        ]
      : [
          // No transformations in development
        ],
}
```

> **Good to know**: Next.js also allows the file to be named `.postcssrc.js`.

Do **not use** `require()` to import the PostCSS Plugins. Plugins must be provided as strings.

> **Good to know**: If your `postcss.config.js` needs to support other non-Next.js tools in the same project, you must use the interoperable object-based format instead:
>
> ```
> module.exports = {
>   plugins: {
>     'postcss-flexbugs-fixes': {},
>     'postcss-preset-env': {
>       autoprefixer: {
>         flexbox: 'no-2009',
>       },
>       stage: 3,
>       features: {
>         'custom-properties': false,
>       },
>     },
>   },
> }
> ```

How to preview content with Preview Mode in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/preview-mode
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Next.js has the preview mode for statically generated pages. You can learn how it works here.*

> **Note**: This feature is superseded by Draft Mode.

Examples

In the Pages documentation and the Data Fetching documentation, we talked about how to prerender a page at build time (**Static Generation**) using `getStaticProps` and `getStaticPaths`.

Static Generation is useful when your pages fetch data from a headless CMS. However, it's not ideal when you're writing a draft on your headless CMS and want to **preview** the draft immediately on your page. You'd want Next.js to render these pages at **request time** instead of build time and fetch the draft content instead of the published content. You'd want Next.js to bypass Static Generation only for this specific case.

Next.js has a feature called **Preview Mode** which solves this problem. Here are instructions on how to use it.

Step 1: Create and access a preview API route

> Take a look at the API Routes documentation first if you're not familiar with Next.js API Routes.

First, create a **preview API route**. It can have any name - e.g. `pages/api/preview.js` (or `.ts` if using TypeScript).

In this API route, you need to call `setPreviewData` on the response object. The argument for `setPreviewData` should be an object, and this can be used by `getStaticProps` (more on this later). For now, we'll use `{}`.

```
export default function handler(req, res) {
  // ...
  res.setPreviewData({})
  // ...
}
```

`res.setPreviewData` sets some **cookies** on the browser which turns on the preview mode. Any requests to Next.js containing these cookies will be considered as the **preview mode**, and the behavior for statically generated pages will change (more on this later).

You can test this manually by creating an API route like below and accessing it from your browser manually:

```
// Archivo: pages/api/preview.js
// simple example for testing it manually from your browser.
export default function handler(req, res) {
  res.setPreviewData({})
  res.end('Preview mode enabled')
}
```

If you open your browser's developer tools and visit `/api/preview`, you'll notice that the `__prerender_bypass` and `__next_preview_data` cookies will be set on this request.

Securely accessing it from your Headless CMS

In practice, you'd want to call this API route *securely* from your headless CMS. The specific steps will vary depending on which headless CMS you're using, but here are some common steps you could take.

These steps assume that the headless CMS you're using supports setting **custom preview URLs**. If it doesn't, you can still use this method to secure your preview URLs, but you'll need to construct and access the preview URL manually.

First, you should create a **secret token string** using a token generator of your choice. This secret will only be known by your Next.js app and your headless CMS. This secret prevents people who don't have access to your CMS from accessing preview URLs.

Second, if your headless CMS supports setting custom preview URLs, specify the following as the preview URL. This assumes that your preview API route is located at `pages/api/preview.js`.

```
// Archivo: Terminal
https://<your-site>/api/preview?secret=<token>&slug=<path>
```

- `<your-site>` should be your deployment domain.
- `<token>` should be replaced with the secret token you generated.
- `<path>` should be the path for the page that you want to preview. If you want to preview `/posts/foo`, then you should use `&slug=/posts/foo`.

Your headless CMS might allow you to include a variable in the preview URL so that `<path>` can be set dynamically based on the CMS's data like so: `&slug=/posts/{entry.fields.slug}`

Finally, in the preview API route:

- Check that the secret matches and that the `slug` parameter exists (if not, the request should fail).
-
- Call `res.setPreviewData`.
- Then redirect the browser to the path specified by `slug`. (The following example uses a 307 redirect).

```
export default async (req, res) => {
  // Check the secret and next parameters
  // This secret should only be known to this API route and the CMS
  if (req.query.secret !== 'MY_SECRET_TOKEN' || !req.query.slug) {
    return res.status(401).json({ message: 'Invalid token' })
  }

  // Fetch the headless CMS to check if the provided `slug` exists
  // getPostBySlug would implement the required fetching logic to the headless CMS
  const post = await getPostBySlug(req.query.slug)

  // If the slug doesn't exist prevent preview mode from being enabled
  if (!post) {
    return res.status(401).json({ message: 'Invalid slug' })
  }

  // Enable Preview Mode by setting the cookies
  res.setPreviewData({})

  // Redirect to the path from the fetched post
  // We don't redirect to req.query.slug as that might lead to open redirect vulnerabilities
  res.redirect(post.slug)
}
```

If it succeeds, then the browser will be redirected to the path you want to preview with the preview mode cookies being set.

Step 2: Update `getStaticProps`

The next step is to update `getStaticProps` to support the preview mode.

If you request a page which has `getStaticProps` with the preview mode cookies set (via `res.setPreviewData`), then `getStaticProps` will be called at **request time** (instead of at build time).

Furthermore, it will be called with a `context` object where:

- `context.preview` will be `true`.
- `context.previewData` will be the same as the argument used for `setPreviewData`.

```
export async function getStaticProps(context) {
  // If you request this page with the preview mode cookies set:
  //
  // - context.preview will be true
  // - context.previewData will be the same as
  //   the argument used for `setPreviewData`.
}
```

We used `res.setPreviewData({})` in the preview API route, so `context.previewData` will be `{}`. You can use this to pass session information from the preview API route to `getStaticProps` if necessary.

If you're also using `getStaticPaths`, then `context.params` will also be available.

Fetch preview data

You can update `getStaticProps` to fetch different data based on `context.preview` and/or `context.previewData`.

For example, your headless CMS might have a different API endpoint for draft posts. If so, you can use `context.preview` to modify the API endpoint URL like below:

```
export async function getStaticProps(context) {
  // If context.preview is true, append "/preview" to the API endpoint
  // to request draft data instead of published data. This will vary
  // based on which headless CMS you're using.
  const res = await fetch(`https:// ... /${context.preview ? 'preview' : ''}`)
  // ...
}
```

That's it! If you access the preview API route (with `secret` and `slug`) from your headless CMS or manually, you should now be able to see the preview content. And if you update your draft without publishing, you should be able to preview the draft.

Set this as the preview URL on your headless CMS or access manually, and you should be able to see the preview.

```
// Archivo: Terminal
https://<your-site>/api/preview?secret=<token>&slug=<path>
```

More Details

> **Good to know**: during rendering `next/router` exposes an `isPreview` flag, see the router object docs for more info.

Specify the Preview Mode duration

`setPreviewData` takes an optional second parameter which should be an options object. It accepts the following keys:

- `maxAge`: Specifies the number (in seconds) for the preview session to last for.
- `path`: Specifies the path the cookie should be applied under. Defaults to `/` enabling preview mode for all paths.

```
setPreviewData(data, {
  maxAge: 60 * 60, // The preview mode cookies expire in 1 hour
  path: '/about', // The preview mode cookies apply to paths with /about
})
```

Clear the Preview Mode cookies

By default, no expiration date is set for Preview Mode cookies, so the preview session ends when the browser is closed.

To clear the Preview Mode cookies manually, create an API route that calls `clearPreviewData()`:

```
// Archivo: pages/api/clear-preview-mode-cookies.js
export default function handler(req, res) {
  res.clearPreviewData({})
}
```

Then, send a request to `/api/clear-preview-mode-cookies` to invoke the API Route. If calling this route using `next/link`, you must pass `prefetch={false}` to prevent calling `clearPreviewData` during link prefetching.

If a path was specified in the `setPreviewData` call, you must pass the same path to `clearPreviewData`:

```
// Archivo: pages/api/clear-preview-mode-cookies.js
export default function handler(req, res) {
  const { path } = req.query

  res.clearPreviewData({ path })
}
```

`previewData` size limits

You can pass an object to `setPreviewData` and have it be available in `getStaticProps`. However, because the data will be stored in a cookie, there's a size limitation. Currently, preview data is limited to 2KB.

Works with `getServerSideProps`

The preview mode works on `getServerSideProps` as well. It will also be available on the `context` object containing `preview` and `previewData`.

> **Good to know**: You shouldn't set the `Cache-Control` header when using Preview Mode because it cannot be bypassed. Instead, we recommend using ISR.

Works with API Routes

API Routes will have access to `preview` and `previewData` under the request object. For example:

```
export default function myApiRoute(req, res) {
  const isPreview = req.preview
  const previewData = req.previewData
  // ...
}
```

Unique per `next build`

Both the bypass cookie value and the private key for encrypting the `previewData` change when `next build` is completed. This ensures that the bypass cookie can't be guessed.

> **Good to know**: To test Preview Mode locally over HTTP your browser will need to allow third-party cookies and local storage access.

How to optimize your Next.js application for production

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/production-checklist
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Recommendations to ensure the best performance and user experience before taking your Next.js application to production.*

How to handle redirects in Next.js

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/redirecting
> 📂 **Sección:** Pages > Guides
> 📝 **Descripción:** *Learn the different ways to handle redirects in Next.js.*

How to use Sass in Next.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/sass
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Learn how to use Sass in your Next.js application.*

How to load and optimize scripts

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/scripts
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Optimize 3rd party scripts with the built-in Script component.*

How to self-host your Next.js application

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/self-hosting
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Learn how to self-host your Next.js application on a Node.js server, Docker image, or static HTML files (static exports).*

How to create a static export of your Next.js application

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/static-exports
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Next.js enables starting as a static site or Single-Page Application (SPA), then later optionally upgrading to use features that require a server.*

Tailwind CSS

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/tailwind-v3-css
📁 **Sección:** Pages > Guides
📝 **Descripción:** *Style your Next.js Application using Tailwind CSS.*

How to set up Cypress with Next.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/cypress
📁 **Sección:** Pages > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with Cypress for End-to-End (E2E) and Component Testing.*

Testing

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing
📁 **Sección:** Pages > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with three commonly used testing tools — Cypress, Playwright, Vitest, and Jest.*

How to set up Jest with Next.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/jest
📁 **Sección:** Pages > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with Jest for Unit Testing.*

How to set up Playwright with Next.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/playwright
📁 **Sección:** Pages > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with Playwright for End-to-End (E2E) and Integration testing.*

How to set up Vitest with Next.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/testing/vitest
📁 **Sección:** Pages > Guides > Testing
📝 **Descripción:** *Learn how to set up Next.js with Vitest and React Testing Library - two popular unit testing libraries.*

How to optimize third-party libraries

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/third-party-libraries
> 📁 **Sección:** Pages > Guides
> 📝 **Descripción:** *Optimize the performance of third-party libraries in your application with the* `@next/third-parties` *package.*

Codemods

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/codemods
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Use codemods to upgrade your Next.js codebase when new features are released.*

Upgrading

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Learn how to upgrade to the latest versions of Next.js.*

Learn how to upgrade to the latest versions of Next.js following the versions-specific guides:

How to upgrade to version 10

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-10
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 9 to Version 10.*

There were no breaking changes between versions 9 and 10.

To upgrade to version 10, run the following command:

```
// Archivo: Terminal
npm i next@10
```

```
// Archivo: Terminal
yarn add next@10
```

```
// Archivo: Terminal
pnpm up next@10
```

```
// Archivo: Terminal
bun add next@10
```

> **Good to know:** If you are using TypeScript, ensure you also upgrade `@types/react` and `@types/react-dom` to their corresponding versions.

How to upgrade to version 11

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-11
> 📂 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 10 to Version 11.*

To upgrade to version 11, run the following command:

```
// Archivo: Terminal
npm i next@11 react@17 react-dom@17
```

```
// Archivo: Terminal
yarn add next@11 react@17 react-dom@17
```

```
// Archivo: Terminal
pnpm up next@11 react@17 react-dom@17
```

```
// Archivo: Terminal
bun add next@11 react@17 react-dom@17
```

> **Good to know:** If you are using TypeScript, ensure you also upgrade `@types/react` and `@types/react-dom` to their corresponding versions.

Webpack 5

Webpack 5 is now the default for all Next.js applications. If you did not have a custom webpack configuration, your application is already using webpack 5. If you do have a custom webpack configuration, you can refer to the Next.js webpack 5 documentation for upgrade guidance.

Cleaning the `distDir` is now a default

The build output directory (defaults to `.next`) is now cleared by default except for the Next.js caches. You can refer to the cleaning `distDir` RFC for more information.

If your application was relying on this behavior previously you can disable the new default behavior by adding the `cleanDistDir: false` flag in `next.config.js`.

`PORT` is now supported for `next dev` and `next start`

Next.js 11 supports the `PORT` environment variable to set the port the application runs on. Using `-p` / `--port` is still recommended but if you were prohibited from using `-p` in any way you can now use `PORT` as an alternative:

Example:

```
PORT=4000 next start
```

`next.config.js` customization to import images

Next.js 11 supports static image imports with `next/image`. This new feature relies on being able to process image imports. If you previously added the `next-images` or `next-optimized-images` packages you can either move to the new built-in support using `next/image` or disable the feature:

```
// Archivo: next.config.js
module.exports = {
  images: {
    disableStaticImages: true,
  },
}
```

Remove `super.componentDidCatch()` from `pages/_app.js`

The `next/app` component's `componentDidCatch` was deprecated in Next.js 9 as it's no longer needed and has since been a no-op. In Next.js 11, it was removed.

If your `pages/_app.js` has a custom `componentDidCatch` method you can remove `super.componentDidCatch` as it is no longer needed.

Remove `Container` from `pages/_app.js`

This export was deprecated in Next.js 9 as it's no longer needed and has since been a no-op with a warning during development. In Next.js 11 it was removed.

If your `pages/_app.js` imports `Container` from `next/app` you can remove `Container` as it was removed. Learn more in the documentation.

Remove `props.url` usage from page components

This property was deprecated in Next.js 4 and has since shown a warning during development. With the introduction of `getStaticProps` / `getServerSideProps` these methods already disallowed the usage of `props.url`. In Next.js 11, it was removed completely.

You can learn more in the documentation.

Remove `unsized` property on `next/image`

The `unsized` property on `next/image` was deprecated in Next.js 10.0.1. You can use `layout="fill"` instead. In Next.js 11 `unsized` was removed.

Remove `modules` property on `next/dynamic`

The `modules` and `render` option for `next/dynamic` were deprecated in Next.js 9.5. This was done in order to make the `next/dynamic` API closer to `React.lazy`. In Next.js 11, the `modules` and `render` options were removed.

This option hasn't been mentioned in the documentation since Next.js 8 so it's less likely that your application is using it.

If your application does use `modules` and `render` you can refer to the documentation.

Remove `Head.rewind`

`Head.rewind` has been a no-op since Next.js 9.5, in Next.js 11 it was removed. You can safely remove your usage of `Head.rewind`.

Moment.js locales excluded by default

Moment.js includes translations for a lot of locales by default. Next.js now automatically excludes these locales by default to optimize bundle size for applications using Moment.js.

To load a specific locale use this snippet:

```
import moment from 'moment'
import 'moment/locale/ja'

moment.locale('ja')
```

You can opt-out of this new default by adding `excludeDefaultMomentLocales: false` to `next.config.js` if you do not want the new behavior, do note it's highly recommended to not disable this new optimization as it significantly reduces the size of Moment.js.

Update usage of `router.events`

In case you're accessing `router.events` during rendering, in Next.js 11 `router.events` is no longer provided during prerendering. Ensure you're accessing `router.events` in `useEffect`:

```
useEffect(() => {
  const handleRouteChange = (url, { shallow }) => {
    console.log(
      `App is changing to ${url} ${
        shallow ? 'with' : 'without'
      } shallow routing`
    )
  }

  router.events.on('routeChangeStart', handleRouteChange)

  // If the component is unmounted, unsubscribe
  // from the event with the `off` method:
  return () => {
    router.events.off('routeChangeStart', handleRouteChange)
  }
}, [router])
```

If your application uses `router.router.events` which was an internal property that was not public please make sure to use `router.events` as well.

React 16 to 17

React 17 introduced a new JSX Transform that brings a long-time Next.js feature to the wider React ecosystem: Not having to `import React from 'react'` when using JSX. When using React 17 Next.js will automatically use the new transform. This transform does not make the `React` variable global, which was an unintended side-effect of the previous Next.js implementation. A codemod is available to automatically fix cases where you accidentally used `React` without importing it.

Most applications already use the latest version of React, with Next.js 11 the minimum React version has been updated to 17.0.2.

To upgrade you can run the following command:

```
npm install react@latest react-dom@latest
```

Or using `yarn`:

```
yarn add react@latest react-dom@latest
```

How to upgrade to version 12

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-12
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 11 to Version 12.*

To upgrade to version 12, run the following command:

```
// Archivo: Terminal
npm i next@12 react@17 react-dom@17 eslint-config-next@12
```

```
// Archivo: Terminal
yarn add next@12 react@17 react-dom@17 eslint-config-next@12
```

```
// Archivo: Terminal
pnpm up next@12 react@17 react-dom@17 eslint-config-next@12
```

```
// Archivo: Terminal
bun add next@12 react@17 react-dom@17 eslint-config-next@12
```

> **Good to know:** If you are using TypeScript, ensure you also upgrade `@types/react` and `@types/react-dom` to their corresponding versions.

Upgrading to 12.2

Middleware - If you were using Middleware prior to `12.2`, please see the upgrade guide for more information.

Upgrading to 12.0

Minimum Node.js Version - The minimum Node.js version has been bumped from `12.0.0` to `12.22.0` which is the first version of Node.js with native ES Modules support.

Minimum React Version - The minimum required React version is `17.0.2`. To upgrade you can run the following command in the terminal:

```
// Archivo: Terminal
npm install react@latest react-dom@latest

yarn add react@latest react-dom@latest

pnpm update react@latest react-dom@latest

bun add react@latest react-dom@latest
```

SWC replacing Babel

Next.js now uses the Rust-based compiler SWC to compile JavaScript/TypeScript. This new compiler is up to 17x faster than Babel when compiling individual files and up to 5x faster Fast Refresh.

Next.js provides full backward compatibility with applications that have custom Babel configuration. All transformations that Next.js handles by default like styled-jsx and tree-shaking of `getStaticProps` / `getStaticPaths` / `getServerSideProps` have been ported to Rust.

When an application has a custom Babel configuration, Next.js will automatically opt-out of using SWC for compiling JavaScript/TypeScript and will fall back to using Babel in the same way that it was used in Next.js 11.

Many of the integrations with external libraries that currently require custom Babel transformations will be ported to Rust-based SWC transforms in the near future. These include but are not limited to:

- Styled Components
- Emotion
- Relay

In order to prioritize transforms that will help you adopt SWC, please provide your `.babelrc` on this feedback thread.

SWC replacing Terser for minification

You can opt-in to replacing Terser with SWC for minifying JavaScript up to 7x faster using a flag in `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  swcMinify: true,
}
```

Minification using SWC is an opt-in flag to ensure it can be tested against more real-world Next.js applications before it becomes the default in Next.js 12.1. If you have feedback about minification, please leave it on this feedback thread.

Improvements to styled-jsx CSS parsing

On top of the Rust-based compiler we've implemented a new CSS parser based on the one used for the styled-jsx Babel transform. This new parser has improved handling of CSS and now errors when invalid CSS is used that would previously slip through and cause unexpected behavior.

Because of this change invalid CSS will throw an error during development and `next build`. This change only affects styled-jsx usage.

`next/image` changed wrapping element

`next/image` now renders the `<img>` inside a `<span>` instead of `<div>`.

If your application has specific CSS targeting span such as `.container span`, upgrading to Next.js 12 might incorrectly match the wrapping element inside the `<Image>` component. You can avoid this by restricting the selector to a specific class such as `.container span.item` and updating the relevant component with that className, such as `<span className="item" />`.

If your application has specific CSS targeting the `next/image` `<div>` tag, for example `.container div`, it may not match anymore. You can update the selector `.container span`, or preferably, add a new `<div className="wrapper">` wrapping the `<Image>` component and target that instead such as `.container .wrapper`.

The `className` prop is unchanged and will still be passed to the underlying `<img>` element.

See the documentation for more info.

HMR connection now uses a WebSocket

Previously, Next.js used a server-sent events connection to receive HMR events. Next.js 12 now uses a WebSocket connection.

In some cases when proxying requests to the Next.js dev server, you will need to ensure the upgrade request is handled correctly. For example, in `nginx` you would need to add the following configuration:

```
location /_next/webpack-hmr {
    proxy_pass http://localhost:3000/_next/webpack-hmr;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

If you are using Apache (2.x), you can add the following configuration to enable web sockets to the server. Review the port, host name and server names.

```
<VirtualHost *:443>
 # ServerName yourwebsite.local
 ServerName "${WEBSITE_SERVER_NAME}"
 ProxyPass / http://localhost:3000/
 ProxyPassReverse / http://localhost:3000/
 # Next.js 12 uses websocket
 <Location /_next/webpack-hmr>
    RewriteEngine On
    RewriteCond %{QUERY_STRING} transport=websocket [NC]
    RewriteCond %{HTTP:Upgrade} websocket [NC]
    RewriteCond %{HTTP:Connection} upgrade [NC]
    RewriteRule /(.*) ws://localhost:3000/_next/webpack-hmr/$1 [P,L]
    ProxyPass ws://localhost:3000/_next/webpack-hmr retry=0 timeout=30
    ProxyPassReverse ws://localhost:3000/_next/webpack-hmr
 </Location>
</VirtualHost>
```

For custom servers, such as `express`, you may need to use `app.all` to ensure the request is passed correctly, for example:

```
app.all('/_next/webpack-hmr', (req, res) => {
  nextjsRequestHandler(req, res)
})
```

> **Note:** In Next.js 16, the HMR WebSocket path was renamed from `/_next/webpack-hmr` to `/_next/hmr`.

Webpack 4 support has been removed

If you are already using webpack 5 you can skip this section.

Next.js has adopted webpack 5 as the default for compilation in Next.js 11. As communicated in the webpack 5 upgrading documentation Next.js 12 removes support for webpack 4.

If your application is still using webpack 4 using the opt-out flag, you will now see an error linking to the webpack 5 upgrading documentation.

`target` option deprecated

If you do not have `target` in `next.config.js` you can skip this section.

The target option has been deprecated in favor of built-in support for tracing what dependencies are needed to run a page.

During `next build`, Next.js will automatically trace each page and its dependencies to determine all of the files that are needed for deploying a production version of your application.

If you are currently using the `target` option set to `serverless`, please read the documentation on how to leverage the new output.

How to upgrade to version 13

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-13
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 12 to 13.*

Upgrading from 12 to 13

To update to Next.js version 13, run the following command using your preferred package manager:

```
// Archivo: Terminal
npm i next@13 react@latest react-dom@latest eslint-config-next@13
```

```
// Archivo: Terminal
yarn add next@13 react@latest react-dom@latest eslint-config-next@13
```

```
// Archivo: Terminal
pnpm i next@13 react@latest react-dom@latest eslint-config-next@13
```

```
// Archivo: Terminal
bun add next@13 react@latest react-dom@latest eslint-config-next@13
```

> **Good to know:** If you are using TypeScript, ensure you also upgrade `@types/react` and `@types/react-dom` to their latest versions.

v13 Summary

- The Supported Browsers have been changed to drop Internet Explorer and target modern browsers.
- The minimum Node.js version has been bumped from 12.22.0 to 16.14.0, since 12.x and 14.x have reached end-of-life.
- The minimum React version has been bumped from 17.0.2 to 18.2.0.
- The `swcMinify` configuration property was changed from `false` to `true`. See Next.js Compiler for more info.
- The `next/image` import was renamed to `next/legacy/image`. The `next/future/image` import was renamed to `next/image`. A codemod is available to safely and automatically rename your imports.
- The `next/link` child can no longer be `<a>`. Add the `legacyBehavior` prop to use the legacy behavior or remove the `<a>` to upgrade. A codemod is available to automatically upgrade your code.
- The `target` configuration property has been removed and superseded by Output File Tracing.

Migrating shared features

Next.js 13 introduces a new `app` directory with new features and conventions. However, upgrading to Next.js 13 does **not** require using the new `app` Router.

You can continue using `pages` with new features that work in both directories, such as the updated Image component, Link component, Script component, and Font optimization.

`<Image/>` Component

Next.js 12 introduced many improvements to the Image Component with a temporary import: `next/future/image`. These improvements included less client-side JavaScript, easier ways to extend and style images, better accessibility, and native browser lazy loading.

Starting in Next.js 13, this new behavior is now the default for `next/image`.

There are two codemods to help you migrate to the new Image Component:

- next-image-to-legacy-image: This codemod will safely and automatically rename `next/image` imports to `next/legacy/image` to maintain the same behavior as Next.js 12. We recommend running this codemod to quickly update to Next.js 13 automatically.
- next-image-experimental: After running the previous codemod, you can optionally run this experimental codemod to upgrade `next/legacy/image` to the new `next/image`, which will remove unused props and add inline styles. Please note this codemod is experimental and only covers static usage (such as `

Illustration

`) but not dynamic usage (such as

Illustration

`).

Alternatively, you can manually update by following the migration guide and also see the legacy comparison.

`<Link>` Component

The `<Link>` Component no longer requires manually adding an `<a>` tag as a child. This behavior was added as an experimental option in version 12.2 and is now the default. In Next.js 13, `<Link>` always renders `<a>` and allows you to forward props to the underlying tag.

For example:

```
import Link from 'next/link'

// Next.js 12: `<a>` has to be nested otherwise it's excluded
[
  <a>About</a>
](/about)

// Next.js 13: `<Link>` always renders `<a>` under the hood
[
  About
](/about)
```

To upgrade your links to Next.js 13, you can use the `new-link` codemod.

`<Script>` Component

The behavior of `next/script` has been updated to support both `pages` and `app`. If incrementally adopting `app`, read the upgrade guide.

Font Optimization

Previously, Next.js helped you optimize fonts by inlining font CSS. Version 13 introduces the new `next/font` module which gives you the ability to customize your font loading experience while still ensuring great performance and privacy.

See Optimizing Fonts to learn how to use `next/font`.

How to upgrade to version 14

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-14
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 13 to 14.*

How to upgrade to version 9

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/guides/upgrading/version-9
> 📁 **Sección:** Pages > Guides > Upgrading
> 📝 **Descripción:** *Upgrade your Next.js Application from Version 8 to Version 9.*

To upgrade to version 9, run the following command:

```
// Archivo: Terminal
npm i next@9
```

```
// Archivo: Terminal
yarn add next@9
```

```
// Archivo: Terminal
pnpm up next@9
```

```
// Archivo: Terminal
bun add next@9
```

> **Good to know:** If you are using TypeScript, ensure you also upgrade `@types/react` and `@types/react-dom` to their corresponding versions.

Check your Custom App File (`pages/_app.js`)

If you previously copied the Custom `<App>` example, you may be able to remove your `getInitialProps`.

Removing `getInitialProps` from `pages/_app.js` (when possible) is important to leverage new Next.js features!

The following `getInitialProps` does nothing and may be removed:

```
class MyApp extends App {
  // Remove me, I do nothing!
  static async getInitialProps({ Component, ctx }) {
    let pageProps = {}

    if (Component.getInitialProps) {
      pageProps = await Component.getInitialProps(ctx)
    }

    return { pageProps }
  }

  render() {
    // ... etc
  }
}
```

Breaking Changes

`@zeit/next-typescript` is no longer necessary

Next.js will now ignore usage `@zeit/next-typescript` and warn you to remove it. Please remove this plugin from your `next.config.js`.

Remove references to `@zeit/next-typescript/babel` from your custom `.babelrc` (if present).

The usage of `fork-ts-checker-webpack-plugin` should also be removed from your `next.config.js`.

TypeScript Definitions are published with the `next` package, so you need to uninstall `@types/next` as they would conflict.

The following types are different:

> This list was created by the community to help you upgrade, if you find other differences please send a pull-request to this list to help other users.

From:

```
import { NextContext } from 'next'
import { NextAppContext, DefaultAppIProps } from 'next/app'
import { NextDocumentContext, DefaultDocumentIProps } from 'next/document'
```

to

```
import { NextPageContext } from 'next'
import { AppContext, AppInitialProps } from 'next/app'
import { DocumentContext, DocumentInitialProps } from 'next/document'
```

The `config` key is now an export on a page

You may no longer export a custom variable named `config` from a page (i.e. `export { config }` / `export const config ...`). This exported variable is now used to specify page-level Next.js configuration like Opt-in AMP and API Route features.

You must rename a non-Next.js-purposed `config` export to something different.

`next/dynamic` no longer renders "loading..." by default while loading

Dynamic components will not render anything by default while loading. You can still customize this behavior by setting the `loading` property:

```
import dynamic from 'next/dynamic'

const DynamicComponentWithCustomLoading = dynamic(
  () => import('../components/hello2'),
  {
    loading: () => <p>Loading</p>,
  }
)
```

`withAmp` has been removed in favor of an exported configuration object

Next.js now has the concept of page-level configuration, so the `withAmp` higher-order component has been removed for consistency.

This change can be **automatically migrated by running the following commands in the root of your Next.js project:**

```
// Archivo: Terminal
curl -L https://github.com/vercel/next-codemod/archive/master.tar.gz | tar -xz --strip=2 next-codemod-
master/transforms/withamp-to-config.js npx jscodeshift -t ./withamp-to-config.js pages/**/*.js
```

To perform this migration by hand, or view what the codemod will produce, see below:

Before

```
import { withAmp } from 'next/amp'

function Home() {
  return <h1>My AMP Page</h1>
}

export default withAmp(Home)
// or
export default withAmp(Home, { hybrid: true })
```

After

```
export default function Home() {
  return <h1>My AMP Page</h1>
}

export const config = {
  amp: true,
  // or
  amp: 'hybrid',
}
```

`next export` no longer exports pages as `index.html`

Previously, exporting `pages/about.js` would result in `out/about/index.html`. This behavior has been changed to result in `out/about.html`.

You can revert to the previous behavior by creating a `next.config.js` with the following content:

```
// Archivo: next.config.js
module.exports = {
  trailingSlash: true,
}
```

`pages/api/` is treated differently

Pages in `pages/api/` are now considered API Routes. Pages in this directory will no longer contain a client-side bundle.

Deprecated Features

`next/dynamic` has deprecated loading multiple modules at once

The ability to load multiple modules at once has been deprecated in `next/dynamic` to be closer to React's implementation (`React.lazy` and `Suspense`).

Updating code that relies on this behavior is relatively straightforward! We've provided an example of a before/after to help you migrate your application:

Before

```
import dynamic from 'next/dynamic'

const HelloBundle = dynamic({
  modules: () => {
    const components = {
      Hello1: () => import('../components/hello1').then((m) => m.default),
      Hello2: () => import('../components/hello2').then((m) => m.default),
    }

    return components
  },
  render: (props, { Hello1, Hello2 }) => (
    <div>
      <h1>{props.title}</h1>
      <Hello1 />
      <Hello2 />
    </div>
  ),
})

function DynamicBundle() {
  return <HelloBundle title="Dynamic Bundle" />
}

export default DynamicBundle
```

After

```
import dynamic from 'next/dynamic'

const Hello1 = dynamic(() => import('../components/hello1'))
const Hello2 = dynamic(() => import('../components/hello2'))

function HelloBundle({ title }) {
  return (
    <div>
      <h1>{title}</h1>
      <Hello1 />
      <Hello2 />
    </div>
  )
}

function DynamicBundle() {
  return <HelloBundle title="Dynamic Bundle" />
}

export default DynamicBundle
```

Pages and Layouts

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/pages-and-layouts
> 📂 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Create your first page and shared layout with the Pages Router.*

The Pages Router has a file-system based router built on the concept of pages.

When a file is added to the `pages` directory, it's automatically available as a route.

In Next.js, a **page** is a React Component exported from a `.js`, `.jsx`, `.ts`, or `.tsx` file in the `pages` directory. Each page is associated with a route based on its file name.

Example: If you create `pages/about.js` that exports a React component like below, it will be accessible at `/about`.

```
export default function About() {
  return <div>About</div>
}
```

Index routes

The router will automatically route files named `index` to the root of the directory.

- `pages/index.js` → `/`
- `pages/blog/index.js` → `/blog`

Nested routes

The router supports nested files. If you create a nested folder structure, files will automatically be routed in the same way still.

- `pages/blog/first-post.js` → `/blog/first-post`
- `pages/dashboard/settings/username.js` → `/dashboard/settings/username`

Pages with Dynamic Routes

Next.js supports pages with dynamic routes. For example, if you create a file called `pages/posts/[id].js`, then it will be accessible at `posts/1`, `posts/2`, etc.

> To learn more about dynamic routing, check the Dynamic Routing documentation.

Layout Pattern

The React model allows us to deconstruct a page into a series of components. Many of these components are often reused between pages. For example, you might have the same navigation bar and footer on every page.

```
// Archivo: components/layout.js
import Navbar from './navbar'
import Footer from './footer'

export default function Layout({ children }) {
  return (
    <>
      <Navbar />
      <main>{children}</main>
      <Footer />
    </>
  )
}
```

Examples

Single Shared Layout with Custom App

If you only have one layout for your entire application, you can create a Custom App and wrap your application with the layout. Since the `<Layout />` component is re-used when changing pages, its component state will be preserved (e.g. input values).

```
// Archivo: pages/_app.js
import Layout from '../components/layout'

export default function MyApp({ Component, pageProps }) {
  return (
    <Layout>
      <Component {...pageProps} />
    </Layout>
  )
}
```

Per-Page Layouts

If you need multiple layouts, you can add a property `getLayout` to your page, allowing you to return a React component for the layout. This allows you to define the layout on a *per-page basis*. Since we're returning a function, we can have complex nested layouts if desired.

```
// Archivo: pages/index.js

import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'

export default function Page() {
  return (
    /** Your content */
  )
}

Page.getLayout = function getLayout(page) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}
```

```
// Archivo: pages/_app.js
export default function MyApp({ Component, pageProps }) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

When navigating between pages, we want to *persist* page state (input values, scroll position, etc.) for a Single-Page Application (SPA) experience.

This layout pattern enables state persistence because the React component tree is maintained between page transitions. With the component tree, React can understand which elements have changed to preserve state.

> **Good to know**: This process is called reconciliation, which is how React understands which elements have changed.

With TypeScript

When using TypeScript, you must first create a new type for your pages which includes a `getLayout` function. Then, you must create a new type for your `AppProps` which overrides the `Component` property to use the previously created type.

```
// Archivo: pages/index.tsx switcher
import type { ReactElement } from 'react'
import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'
import type { NextPageWithLayout } from './_app'

const Page: NextPageWithLayout = () => {
  return <p>hello world</p>
}

Page.getLayout = function getLayout(page: ReactElement) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}

export default Page
```

```
// Archivo: pages/index.js switcher
import Layout from '../components/layout'
import NestedLayout from '../components/nested-layout'

const Page = () => {
  return <p>hello world</p>
}

Page.getLayout = function getLayout(page) {
  return (
    <Layout>
      <NestedLayout>{page}</NestedLayout>
    </Layout>
  )
}

export default Page
```

```
// Archivo: pages/_app.tsx switcher
import type { ReactElement, ReactNode } from 'react'
import type { NextPage } from 'next'
import type { AppProps } from 'next/app'

export type NextPageWithLayout<P = {}, IP = P> = NextPage<P, IP> & {
  getLayout?: (page: ReactElement) => ReactNode
}

type AppPropsWithLayout = AppProps & {
  Component: NextPageWithLayout
}

export default function MyApp({ Component, pageProps }: AppPropsWithLayout) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

```
// Archivo: pages/_app.js switcher
export default function MyApp({ Component, pageProps }) {
  // Use the layout defined at the page level, if available
  const getLayout = Component.getLayout ?? ((page) => page)

  return getLayout(<Component {...pageProps} />)
}
```

Data Fetching

Inside your layout, you can fetch data on the client-side using `useEffect` or a library like SWR. Because this file is not a Page, you cannot use `getStaticProps` or `getServerSideProps` currently.

```
// Archivo: components/layout.js
import useSWR from 'swr'
import Navbar from './navbar'
import Footer from './footer'

export default function Layout({ children }) {
  const { data, error } = useSWR('/api/navigation', fetcher)

  if (error) return <div>Failed to load</div>
  if (!data) return <div>Loading ... </div>

  return (
    <>
      <Navbar links={data.links} />
      <main>{children}</main>
      <Footer />
    </>
  )
}
```

Dynamic Routes

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/dynamic-routes
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Dynamic Routes are pages that allow you to add custom params to your URLs. Start creating Dynamic Routes and learn more here.*

When you don't know the exact segment names ahead of time and want to create routes from dynamic data, you can use Dynamic Segments that are filled in at request time or prerendered at build time.

Convention

A Dynamic Segment can be created by wrapping a file or folder name in square brackets: `[segmentName]`. For example, `[id]` or `[slug]`.

Dynamic Segments can be accessed from `useRouter`.

Example

For example, a blog could include the following route `pages/blog/[slug].js` where `[slug]` is the Dynamic Segment for blog posts.

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()
  return <p>Post: {router.query.slug}</p>
}
```

Route	Example URL	`params`
`pages/blog/[slug].js`	`/blog/a`	`{ slug: 'a' }`
`pages/blog/[slug].js`	`/blog/b`	`{ slug: 'b' }`
`pages/blog/[slug].js`	`/blog/c`	`{ slug: 'c' }`

Catch-all Segments

Dynamic Segments can be extended to **catch-all** subsequent segments by adding an ellipsis inside the brackets `[...segmentName]`.

For example, `pages/shop/[...slug].js` will match `/shop/clothes`, but also `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`, and so on.

Route	Example URL	`params`
`pages/shop/[...slug].js`	`/shop/a`	`{ slug: ['a'] }`
`pages/shop/[...slug].js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`pages/shop/[...slug].js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Optional Catch-all Segments

Catch-all Segments can be made **optional** by including the parameter in double square brackets: `[[...segmentName]]`.

For example, `pages/shop/[[...slug]].js` will **also** match `/shop`, in addition to `/shop/clothes`, `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`.

The difference between **catch-all** and **optional catch-all** segments is that with optional, the route without the parameter is also matched (`/shop` in the example above).

Route	Example URL	`params`
`pages/shop/[[...slug]].js`	`/shop`	`{ slug: undefined }`
`pages/shop/[[...slug]].js`	`/shop/a`	`{ slug: ['a'] }`
`pages/shop/[[...slug]].js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`pages/shop/[[...slug]].js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Linking and Navigating

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/linking-and-navigating
> 📂 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Learn how navigation works in Next.js, and how to use the Link Component and `useRouter` hook.*

The Next.js router allows you to do client-side route transitions between pages, similar to a single-page application.

A React component called `Link` is provided to do this client-side route transition.

```
import Link from 'next/link'

function Home() {
  return (
    <ul>
      <li>
        [Home](/)
      </li>
      <li>
        [About Us](/about)
      </li>
      <li>
        [Blog Post](/blog/hello-world)
      </li>
    </ul>
  )
}

export default Home
```

The example above uses multiple links. Each one maps a path (`href`) to a known page:

- `/` → `pages/index.js`
- `/about` → `pages/about.js`
- `/blog/hello-world` → `pages/blog/[slug].js`

Any `<Link />` in the viewport (initially or through scroll) will be prefetched by default (including the corresponding data) for pages using Static Generation. The corresponding data for server-rendered routes is fetched *only when* the `<Link />` is clicked.

Linking to dynamic paths

You can also use interpolation to create the path, which comes in handy for dynamic route segments. For example, to show a list of posts which have been passed to the component as a prop:

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link href={`/blog/${encodeURIComponent(post.slug)}`}>
            {post.title}
          </Link>
        </li>
      ))}
    </ul>
  )
}

export default Posts
```

> `encodeURIComponent` is used in the example to keep the path utf-8 compatible.

Alternatively, using a URL Object:

```
import Link from 'next/link'

function Posts({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>
          <Link
            href={{
              pathname: '/blog/[slug]',
              query: { slug: post.slug },
            }}
          >
            {post.title}
          </Link>
        </li>
      ))}
    </ul>
  )
}

export default Posts
```

Now, instead of using interpolation to create the path, we use a URL object in `href` where:

- `pathname` is the name of the page in the `pages` directory. `/blog/[slug]` in this case.
- `query` is an object with the dynamic segment. `slug` in this case.

Injecting the router

To access the `router` object in a React component you can use `useRouter` or `withRouter`.

In general we recommend using `useRouter`.

Imperative Routing

`next/link` should be able to cover most of your routing needs, but you can also do client-side navigations without it, take a look at the documentation for `next/router`.

The following example shows how to do basic page navigations with `useRouter`:

```
import { useRouter } from 'next/router'

export default function ReadMore() {
  const router = useRouter()

  return (
    <button onClick={() => router.push('/about')}>
      Click here to read more
    </button>
  )
}
```

Shallow Routing

Examples

Shallow routing allows you to change the URL without running data fetching methods again, that includes `getServerSideProps`, `getStaticProps`, and `getInitialProps`.

You'll receive the updated `pathname` and the `query` via the `router` object (added by `useRouter` or `withRouter`), without losing state.

To enable shallow routing, set the `shallow` option to `true`. Consider the following example:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

// Current URL is '/'
function Page() {
  const router = useRouter()

  useEffect(() => {
    // Always do navigations after the first render
    router.push('/?counter=10', undefined, { shallow: true })
  }, [])

  useEffect(() => {
    // The counter changed!
  }, [router.query.counter])
}

export default Page
```

The URL will get updated to `/?counter=10` and the page won't get replaced, only the state of the route is changed.

You can also watch for URL changes via `componentDidUpdate` as shown below:

```
componentDidUpdate(prevProps) {
  const { pathname, query } = this.props.router
  // verify props have changed to avoid an infinite loop
  if (query.counter !== prevProps.router.query.counter) {
    // fetch data based on the new query
  }
}
```

Caveats

Shallow routing **only** works for URL changes in the current page. For example, let's assume we have another page called `pages/about.js`, and you run this:

```
router.push('/?counter=10', '/about?counter=10', { shallow: true })
```

Since that's a new page, it'll unload the current page, load the new one and wait for data fetching even though we asked to do shallow routing.

When shallow routing is used with proxy it will not ensure the new page matches the current page like previously done without proxy. This is due to proxy being able to rewrite dynamically and can't be verified client-side without a data fetch which is skipped with shallow, so a shallow route change must always be treated as shallow.

Custom App

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-app
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Control page initialization and add a layout that persists for all pages by overriding the default App component used by Next.js.*

Next.js uses the `App` component to initialize pages. You can override it and control the page initialization and:

- Create a shared layout between page changes
- Inject additional data into pages
- Add global CSS

Usage

To override the default `App`, create the file `pages/_app` as shown below:

```
// Archivo: pages/_app.tsx switcher
import type { AppProps } from 'next/app'

export default function MyApp({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />
}
```

```
// Archivo: pages/_app.jsx switcher
export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

The `Component` prop is the active `page`, so whenever you navigate between routes, `Component` will change to the new `page`. Therefore, any props you send to `Component` will be received by the `page`.

`pageProps` is an object with the initial props that were preloaded for your page by one of our data fetching methods, otherwise it's an empty object.

> **Good to know**:
>
> - If your app is running and you added a custom `App`, you'll need to restart the development server. Only required if `pages/_app.js` didn't exist before.
> - `App` does not support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.

`getInitialProps` with `App`

Using `getInitialProps` in `App` will disable Automatic Static Optimization for pages without `getStaticProps`.

We do not recommend using this pattern. Instead, consider incrementally adopting the App Router, which allows you to more easily fetch data for pages and layouts.

```
// Archivo: pages/_app.tsx switcher
import App, { AppContext, AppInitialProps, AppProps } from 'next/app'

type AppOwnProps = { example: string }

export default function MyApp({
  Component,
  pageProps,
  example,
}: AppProps & AppOwnProps) {
  return (
    <>
      <p>Data: {example}</p>
      <Component {...pageProps} />
    </>
  )
}

MyApp.getInitialProps = async (
  context: AppContext
): Promise<AppOwnProps & AppInitialProps> => {
  const ctx = await App.getInitialProps(context)

  return { ...ctx, example: 'data' }
}
```

```
// Archivo: pages/_app.jsx switcher
import App from 'next/app'

export default function MyApp({ Component, pageProps, example }) {
  return (
    <>
      <p>Data: {example}</p>
      <Component {...pageProps} />
    </>
  )
}

MyApp.getInitialProps = async (context) => {
  const ctx = await App.getInitialProps(context)

  return { ...ctx, example: 'data' }
}
```

Custom Document

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-document
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Extend the default document markup added by Next.js.*

A custom `Document` can update the `<html>` and `<body>` tags used to render a Page.

To override the default `Document`, create the file `pages/_document` as shown below:

```
// Archivo: pages/_document.tsx switcher
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html lang="en">
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

```
// Archivo: pages/_document.jsx switcher
import { Html, Head, Main, NextScript } from 'next/document'

export default function Document() {
  return (
    <Html lang="en">
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

Good to know:

- `_document` is only rendered on the server, so event handlers like `onClick` cannot be used in this file.
- `<Html>`, `<Head />`, `<Main />` and `<NextScript />` are required for the page to be properly rendered.

Caveats

- The `<Head />` component used in `_document` is not the same as `next/head`. The `<Head />` component used here should only be used for any `<head>` code that is common for all pages. For all other cases, such as `<title>` tags, we recommend using `next/head` in your pages or components.
- React components outside of `<Main />` will not be initialized by the browser. Do *not* add application logic here or custom CSS (like `styled-jsx`). If you need shared components in all your pages (like a menu or a toolbar), read Layouts instead.
- `Document` currently does not support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.

Customizing `renderPage`

Customizing `renderPage` is advanced and only needed for libraries like CSS-in-JS to support server-side rendering. This is not needed for built-in `styled-jsx` support.

We do not recommend using this pattern. Instead, consider incrementally adopting the App Router, which allows you to more easily fetch data for pages and layouts.

```tsx
// Archivo: pages/_document.tsx switcher
import Document, {
  Html,
  Head,
  Main,
  NextScript,
  DocumentContext,
  DocumentInitialProps,
} from 'next/document'

class MyDocument extends Document {
  static async getInitialProps(
    ctx: DocumentContext
  ): Promise<DocumentInitialProps> {
    const originalRenderPage = ctx.renderPage

    // Run the React rendering logic synchronously
    ctx.renderPage = () =>
      originalRenderPage({
        // Useful for wrapping the whole react tree
        enhanceApp: (App) => App,
        // Useful for wrapping in a per-page basis
        enhanceComponent: (Component) => Component,
      })

    // Run the parent `getInitialProps`, it now includes the custom `renderPage`
    const initialProps = await Document.getInitialProps(ctx)

    return initialProps
  }

  render() {
    return (
      <Html lang="en">
        <Head />
        <body>
          <Main />
          <NextScript />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

```
// Archivo: pages/_document.jsx switcher
import Document, { Html, Head, Main, NextScript } from 'next/document'

class MyDocument extends Document {
  static async getInitialProps(ctx) {
    const originalRenderPage = ctx.renderPage

    // Run the React rendering logic synchronously
    ctx.renderPage = () =>
      originalRenderPage({
        // Useful for wrapping the whole react tree
        enhanceApp: (App) => App,
        // Useful for wrapping in a per-page basis
        enhanceComponent: (Component) => Component,
      })

    // Run the parent `getInitialProps`, it now includes the custom `renderPage`
    const initialProps = await Document.getInitialProps(ctx)

    return initialProps
  }

  render() {
    return (
      <Html lang="en">
        <Head />
        <body>
          <Main />
          <NextScript />
        </body>
      </Html>
    )
  }
}

export default MyDocument
```

Good to know:

- `getInitialProps` in `_document` is not called during client-side transitions.
- The `ctx` object for `_document` is equivalent to the one received in `getInitialProps`, with the addition of `renderPage`.

API Routes

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/api-routes
📁 **Sección:** Pages > Building Your Application > Routing
📝 **Descripción:** *Next.js supports API Routes, which allow you to build your API without leaving your Next.js app. Learn how it works here.*

Examples

Good to know: If you are using the App Router, you can use Server Components or Route Handlers instead of API Routes.

API routes provide a solution to build a **public API** with Next.js.

Any file inside the folder `pages/api` is mapped to `/api/*` and will be treated as an API endpoint instead of a `page`. They are server-side only bundles and won't increase your client-side bundle size.

For example, the following API route returns a JSON response with a status code of `200`:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

Good to know:

- API Routes do not specify CORS headers, meaning they are **same-origin only** by default. You can customize such behavior by wrapping the request handler with the CORS request helpers.
- API Routes can't be used with static exports. However, Route Handlers in the App Router can.
- API Routes will be affected by `pageExtensions` configuration in `next.config.js`.

Parameters

```
export default function handler(req: NextApiRequest, res: NextApiResponse) {
  // ...
}
```

- `req`: An instance of http.IncomingMessage
- `res`: An instance of http.ServerResponse

HTTP Methods

To handle different HTTP methods in an API route, you can use `req.method` in your request handler, like so:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  if (req.method === 'POST') {
    // Process a POST request
  } else {
    // Handle any other HTTP method
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  if (req.method === 'POST') {
    // Process a POST request
  } else {
    // Handle any other HTTP method
  }
}
```

Request Helpers

API Routes provide built-in request helpers which parse the incoming request (`req`):

- `req.cookies` - An object containing the cookies sent by the request. Defaults to `{}`
- `req.query` - An object containing the query string. Defaults to `{}`
- `req.body` - An object containing the body parsed by `content-type` , or `null` if no body was sent

Custom config

Every API Route can export a `config` object to change the default configuration, which is the following:

```
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '1mb',
    },
  },
  // Specifies the maximum allowed duration for this function to execute (in seconds)
  maxDuration: 5,
}
```

`bodyParser` is automatically enabled. If you want to consume the body as a `Stream` or with `raw-body` , you can set this to `false` .

One use case for disabling the automatic `bodyParsing` is to allow you to verify the raw body of a **webhook** request, for example from GitHub.

```
export const config = {
  api: {
    bodyParser: false,
  },
}
```

`bodyParser.sizeLimit` is the maximum size allowed for the parsed body, in any format supported by bytes, like so:

```
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '500kb',
    },
  },
}
```

`externalResolver` is an explicit flag that tells the server that this route is being handled by an external resolver like *express* or *connect*. Enabling this option disables warnings for unresolved requests.

```
export const config = {
  api: {
    externalResolver: true,
  },
}
```

`responseLimit` is automatically enabled, warning when an API Routes' response body is over 4MB.

If you are not using Next.js in a serverless environment, and understand the performance implications of not using a CDN or dedicated media host, you can set this limit to `false` .

```
export const config = {
  api: {
    responseLimit: false,
  },
}
```

`responseLimit` can also take the number of bytes or any string format supported by `bytes`, for example `1000`, `'500kb'` or `'3mb'`. This value will be the maximum response size before a warning is displayed. Default is 4MB. (see above)

```
export const config = {
  api: {
    responseLimit: '8mb',
  },
}
```

Response Helpers

The Server Response object, (often abbreviated as `res`) includes a set of Express.js-like helper methods to improve the developer experience and increase the speed of creating new API endpoints.

The included helpers are:

- `res.status(code)` - A function to set the status code. `code` must be a valid HTTP status code
- `res.json(body)` - Sends a JSON response. `body` must be a serializable object
- `res.send(body)` - Sends the HTTP response. `body` can be a `string`, an `object` or a `Buffer`
- `res.redirect([status,] path)` - Redirects to a specified path or URL. `status` must be a valid HTTP status code. If not specified, `status` defaults to "307" "Temporary redirect".
- `res.revalidate(urlPath)` - Revalidate a page on demand using `getStaticProps`. `urlPath` must be a `string`.

Setting the status code of a response

When sending a response back to the client, you can set the status code of the response.

The following example sets the status code of the response to `200` (`OK`) and returns a `message` property with the value of `Hello from Next.js!` as a JSON response:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

```
// Archivo: pages/api/hello.js switcher
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

Sending a JSON response

When sending a response back to the client you can send a JSON response, this must be a serializable object. In a real world application you might want to let the client know the status of the request depending on the result of the requested endpoint.

The following example sends a JSON response with the status code `200` (`OK`) and the result of the async operation. It's contained in a try catch block to handle any errors that may occur, with the appropriate status code and error message caught and sent back to the client:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const result = await someAsyncOperation()
    res.status(200).json({ result })
  } catch (err) {
    res.status(500).json({ error: 'failed to load data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  try {
    const result = await someAsyncOperation()
    res.status(200).json({ result })
  } catch (err) {
    res.status(500).json({ error: 'failed to load data' })
  }
}
```

Sending a HTTP response

Sending an HTTP response works the same way as when sending a JSON response. The only difference is that the response body can be a `string`, an `object` or a `Buffer`.

The following example sends a HTTP response with the status code `200` (`OK`) and the result of the async operation.

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  try {
    const result = await someAsyncOperation()
    res.status(200).send({ result })
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  try {
    const result = await someAsyncOperation()
    res.status(200).send({ result })
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

Redirects to a specified path or URL

Taking a form as an example, you may want to redirect your client to a specified path or URL once they have submitted the form.

The following example redirects the client to the `/` path if the form is successfully submitted:

```
// Archivo: pages/api/hello.ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { name, message } = req.body

  try {
    await handleFormInputAsync({ name, message })
    res.redirect(307, '/')
  } catch (err) {
    res.status(500).send({ error: 'Failed to fetch data' })
  }
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  const { name, message } = req.body

  try {
    await handleFormInputAsync({ name, message })
    res.redirect(307, '/')
  } catch (err) {
    res.status(500).send({ error: 'failed to fetch data' })
  }
}
```

Adding TypeScript types

You can make your API Routes more type-safe by importing the `NextApiRequest` and `NextApiResponse` types from `next`, in addition to those, you can also type your response data:

```
import type { NextApiRequest, NextApiResponse } from 'next'

type ResponseData = {
  message: string
}

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<ResponseData>
) {
  res.status(200).json({ message: 'Hello from Next.js!' })
}
```

> **Good to know**: The body of `NextApiRequest` is `any` because the client may include any payload. You should validate the type/shape of the body at runtime before using it.

Dynamic API Routes

API Routes support dynamic routes, and follow the same file naming rules used for `pages/`.

```
// Archivo: pages/api/post/[pid].ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const { pid } = req.query
  res.end(`Post: ${pid}`)
}
```

```
// Archivo: pages/api/post/[pid].js switcher
export default function handler(req, res) {
  const { pid } = req.query
  res.end(`Post: ${pid}`)
}
```

Now, a request to `/api/post/abc` will respond with the text: `Post: abc`.

Catch all API routes

API Routes can be extended to catch all paths by adding three dots (`...`) inside the brackets. For example:

- `pages/api/post/[...slug].js` matches `/api/post/a`, but also `/api/post/a/b`, `/api/post/a/b/c` and so on.

> **Good to know**: You can use names other than `slug`, such as: `[...param]`

Matched parameters will be sent as a query parameter (`slug` in the example) to the page, and it will always be an array, so, the path `/api/post/a` will have the following `query` object:

```
{ "slug": ["a"] }
```

And in the case of `/api/post/a/b`, and any other matching path, new parameters will be added to the array, like so:

```
{ "slug": ["a", "b"] }
```

For example:

```
// Archivo: pages/api/post/[...slug].ts switcher
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  const { slug } = req.query
  res.end(`Post: ${slug.join(', ')}`)
}
```

```
// Archivo: pages/api/post/[...slug].js switcher
export default function handler(req, res) {
  const { slug } = req.query
  res.end(`Post: ${slug.join(', ')}`)
}
```

Now, a request to `/api/post/a/b/c` will respond with the text: `Post: a, b, c`.

Optional catch all API routes

Catch all routes can be made optional by including the parameter in double brackets (`[[...slug]]`).

For example, `pages/api/post/[[...slug]].js` will match `/api/post`, `/api/post/a`, `/api/post/a/b`, and so on.

The main difference between catch all and optional catch all routes is that with optional, the route without the parameter is also matched (`/api/post` in the example above).

The `query` objects are as follows:

```
{ } // GET `/api/post` (empty object)
{ "slug": ["a"] } // `GET /api/post/a` (single-element array)
{ "slug": ["a", "b"] } // `GET /api/post/a/b` (multi-element array)
```

Caveats

- Predefined API routes take precedence over dynamic API routes, and dynamic API routes over catch all API routes. Take a look at the following examples:
  - `pages/api/post/create.js` - Will match `/api/post/create`
  - `pages/api/post/[pid].js` - Will match `/api/post/1`, `/api/post/abc`, etc. But not `/api/post/create`
  - `pages/api/post/[...slug].js` - Will match `/api/post/1/2`, `/api/post/a/b/c`, etc. But not `/api/post/create`, `/api/post/abc`

Streaming responses

While the Pages Router does support streaming responses with API Routes, we recommend incrementally adopting the App Router and using Route Handlers if you are on Next.js 14+.

Here's how you can stream a response from an API Route with `writeHead`:

```
// Archivo: pages/api/hello.ts switcher
import { NextApiRequest, NextApiResponse } from 'next'

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-store',
  })
  let i = 0
  while (i < 10) {
    res.write(`data: ${i}\n\n`)
    i++
    await new Promise((resolve) => setTimeout(resolve, 1000))
  }
  res.end()
}
```

```
// Archivo: pages/api/hello.js switcher
export default async function handler(req, res) {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-store',
  })
  let i = 0
  while (i < 10) {
    res.write(`data: ${i}\n\n`)
    i++
    await new Promise((resolve) => setTimeout(resolve, 1000))
  }
  res.end()
}
```

Custom Errors

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing/custom-error
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Override and extend the built-in Error page to handle custom errors.*

---

404 Page

A 404 page may be accessed very often. Server-rendering an error page for every visit increases the load of the Next.js server. This can result in increased costs and slow experiences.

To avoid the above pitfalls, Next.js provides a static 404 page by default without having to add any additional files.

> **Good to know**: The default 404 and 500 pages follow the operating system's color scheme via `prefers-color-scheme` and do not read an app-level theme. Because `pages/404.js` and `pages/500.js` render inside your custom `App`, providing your own files lets your global styles and theme apply.

Customizing The 404 Page

To create a custom 404 page you can create a `pages/404.js` file. This file is statically generated at build time.

```
// Archivo: pages/404.js
export default function Custom404() {
  return <h1>404 - Page Not Found</h1>
}
```

> **Good to know**: You can use `getStaticProps` inside this page if you need to fetch data at build time.

500 Page

Server-rendering an error page for every visit adds complexity to responding to errors. To help users get responses to errors as fast as possible, Next.js provides a static 500 page by default without having to add any additional files.

Customizing The 500 Page

To customize the 500 page you can create a `pages/500.js` file. This file is statically generated at build time.

```
// Archivo: pages/500.js
export default function Custom500() {
  return <h1>500 - Server-side error occurred</h1>
}
```

> **Good to know**: You can use `getStaticProps` inside this page if you need to fetch data at build time.

More Advanced Error Page Customizing

500 errors are handled both client-side and server-side by the `Error` component. If you wish to override it, define the file `pages/_error.js` and add the following code:

```
function Error({ statusCode }) {
  return (
    <p>
      {statusCode
        ? `An error ${statusCode} occurred on server`
        : 'An error occurred on client'}
    </p>
  )
}

Error.getInitialProps = ({ res, err }) => {
  const statusCode = res ? res.statusCode : err ? err.statusCode : 404
  return { statusCode }
}

export default Error
```

> `pages/_error.js` is only used in production. In development you'll get an error with the call stack to know where the error originated from.

Reusing the built-in error page

If you want to render the built-in error page you can by importing the `Error` component:

```
import Error from 'next/error'

export async function getServerSideProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const errorCode = res.ok ? false : res.status
  const json = await res.json()

  return {
    props: { errorCode, stars: json.stargazers_count },
  }
}

export default function Page({ errorCode, stars }) {
  if (errorCode) {
    return <Error statusCode={errorCode} />
  }

  return <div>Next stars: {stars}</div>
}
```

The `Error` component also takes `title` as a property if you want to pass in a text message along with a `statusCode`.

If you have a custom `Error` component be sure to import that one instead. `next/error` exports the default component used by Next.js.

Caveats

- `Error` does not currently support Next.js Data Fetching methods like `getStaticProps` or `getServerSideProps`.
- `_error`, like `_app`, is a reserved pathname. `_error` is used to define the customized layouts and behaviors of the error pages. `/_error` will render 404 when accessed directly via routing or rendering in a custom server.

Routing

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/routing
> 📁 **Sección:** Pages > Building Your Application > Routing
> 📝 **Descripción:** *Learn the fundamentals of routing for front-end applications with the Pages Router.*

The Pages Router has a file-system based router built on concepts of pages. When a file is added to the `pages` directory it's automatically available as a route. Learn more about routing in the Pages Router:

Server-side Rendering (SSR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/server-side-rendering
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Use Server-side Rendering to render pages on each request.*

> Also referred to as "SSR" or "Dynamic Rendering".

If a page uses **Server-side Rendering**, the page HTML is generated on **each request**.

To use Server-side Rendering for a page, you need to `export` an `async` function called `getServerSideProps`. This function will be called by the server on every request.

For example, suppose that your page needs to prerender frequently updated data (fetched from an external API). You can write `getServerSideProps` which fetches this data and passes it to `Page` like below:

```
export default function Page({ data }) {
  // Render data ...
}

// This gets called on every request
export async function getServerSideProps() {
  // Fetch data from external API
  const res = await fetch(`https:// ... /data`)
  const data = await res.json()

  // Pass data to the page via props
  return { props: { data } }
}
```

As you can see, `getServerSideProps` is similar to `getStaticProps`, but the difference is that `getServerSideProps` is run on every request instead of on build time.

To learn more about how `getServerSideProps` works, check out our Data Fetching documentation.

Static Site Generation (SSG)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/static-site-generation
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Use Static Site Generation (SSG) to prerender pages at build time.*

Examples

If a page uses **Static Generation**, the page HTML is generated at **build time**. That means in production, the page HTML is generated when you run `next build`. This HTML will then be reused on each request. It can be cached by a CDN.

In Next.js, you can statically generate pages **with or without data**. Let's take a look at each case.

Static Generation without data

By default, Next.js prerenders pages using Static Generation without fetching data. Here's an example:

```
function About() {
  return <div>About</div>
}

export default About
```

Note that this page does not need to fetch any external data to be prerendered. In cases like this, Next.js generates a single HTML file per page during build time.

Static Generation with data

Some pages require fetching external data for prerendering. There are two scenarios, and one or both might apply. In each case, you can use these functions that Next.js provides:

1. Your page **content** depends on external data: Use `getStaticProps`.
2. Your page **paths** depend on external data: Use `getStaticPaths` (usually in addition to `getStaticProps`).

Scenario 1: Your page content depends on external data

Example: Your blog page might need to fetch the list of blog posts from a CMS (content management system).

```
// TODO: Need to fetch `posts` (by calling some API endpoint)
//       before this page can be prerendered.
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}
```

To fetch this data on prerender, Next.js allows you to `export` an `async` function called `getStaticProps` from the same file. This function gets called at build time and lets you pass fetched data to the page's `props` on prerender.

```
export default function Blog({ posts }) {
  // Render posts ...
}

// This function gets called at build time
export async function getStaticProps() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

To learn more about how `getStaticProps` works, check out the Data Fetching documentation.

Scenario 2: Your page paths depend on external data

Next.js allows you to create pages with **dynamic routes**. For example, you can create a file called `pages/posts/[id].js` to show a single blog post based on `id`. This will allow you to show a blog post with `id: 1` when you access `posts/1`.

> To learn more about dynamic routing, check the Dynamic Routing documentation.

However, which `id` you want to prerender at build time might depend on external data.

Example: suppose that you've only added one blog post (with `id: 1`) to the database. In this case, you'd only want to prerender `posts/1` at build time.

Later, you might add the second post with `id: 2`. Then you'd want to prerender `posts/2` as well.

So your page **paths** that are prerendered depend on external data. To handle this, Next.js lets you `export` an `async` function called `getStaticPaths` from a dynamic page (`pages/posts/[id].js` in this case). This function gets called at build time and lets you specify which paths you want to prerender.

```
// This function gets called at build time
export async function getStaticPaths() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // Get the paths we want to prerender based on posts
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  // We'll prerender only these paths at build time.
  // { fallback: false } means other routes should 404.
  return { paths, fallback: false }
}
```

Also in `pages/posts/[id].js`, you need to export `getStaticProps` so that you can fetch the data about the post with this `id` and use it to prerender the page:

```
export default function Post({ post }) {
  // Render post ...
}

export async function getStaticPaths() {
  // ...
}

// This also gets called at build time
export async function getStaticProps({ params }) {
  // params contains the post `id`.
  // If the route is like /posts/1, then params.id is 1
  const res = await fetch(`https:// ... /posts/${params.id}`)
  const post = await res.json()

  // Pass post data to the page via props
  return { props: { post } }
}
```

To learn more about how `getStaticPaths` works, check out the Data Fetching documentation.

When should I use Static Generation?

We recommend using **Static Generation** (with and without data) whenever possible because your page can be built once and served by CDN, which makes it much faster than having a server render the page on every request.

You can use Static Generation for many types of pages, including:

- Marketing pages
- Blog posts and portfolios
- E-commerce product listings
- Help and documentation

You should ask yourself: "Can I prerender this page **ahead** of a user's request?" If the answer is yes, then you should choose Static Generation.

On the other hand, Static Generation is **not** a good idea if you cannot prerender a page ahead of a user's request. Maybe your page shows frequently updated data, and the page content changes on every request.

In cases like this, you can do one of the following:

- Use Static Generation with **Client-side data fetching:** You can skip prerendering some parts of a page and then use client-side JavaScript to populate them. To learn more about this approach, check out the Data Fetching documentation.
- Use **Server-Side Rendering:** Next.js prerenders a page on each request. It will be slower because the page cannot be cached by a CDN, but the prerendered page will always be up-to-date. We'll talk about this approach below.

Automatic Static Optimization

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/automatic-static-optimization
> 📂 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Next.js automatically optimizes your app to be static HTML whenever possible. Learn how it works here.*

Next.js automatically determines that a page is static (can be prerendered) if it has no blocking data requirements. This determination is made by the absence of `getServerSideProps` and `getInitialProps` in the page.

This feature allows Next.js to emit hybrid applications that contain **both server-rendered and statically generated pages**.

> **Good to know**: Statically generated pages are still reactive. Next.js will hydrate your application client-side to give it full interactivity.

One of the main benefits of this feature is that optimized pages require no server-side computation, and can be instantly streamed to the end-user from multiple CDN locations. The result is an *ultra fast* loading experience for your users.

How it works

If `getServerSideProps` or `getInitialProps` is present in a page, Next.js will switch to render the page on-demand, per-request (meaning Server-Side Rendering).

If the above is not the case, Next.js will **statically optimize** your page automatically by prerendering the page to static HTML.

During prerendering, the router's `query` object will be empty since we do not have `query` information to provide during this phase. After hydration, Next.js will trigger an update to your application to provide the route parameters in the `query` object.

The cases where the query will be updated after hydration triggering another render are:

- The page is a dynamic-route.
- The page has query values in the URL.
- Rewrites are configured in your `next.config.js` since these can have parameters that may need to be parsed and provided in the `query`.

To be able to distinguish if the query is fully updated and ready for use, you can leverage the `isReady` field on `next/router`.

> **Good to know**: Parameters added with dynamic routes to a page that's using `getStaticProps` will always be available inside the `query` object.

`next build` will emit `.html` files for statically optimized pages. For example, the result for the page `pages/about.js` would be:

```
// Archivo: Terminal
.next/server/pages/about.html
```

And if you add `getServerSideProps` to the page, it will then be JavaScript, like so:

```
// Archivo: Terminal
.next/server/pages/about.js
```

Caveats

- If you have a custom `App` with `getInitialProps` then this optimization will be turned off in pages without Static Generation.
- If you have a custom `Document` with `getInitialProps` be sure you check if `ctx.req` is defined before assuming the page is server-side rendered. `ctx.req` will be `undefined` for pages that are prerendered.

- Avoid using the `asPath` value on `next/router` in the rendering tree until the router's `isReady` field is `true`. Statically optimized pages only know `asPath` on the client and not the server, so using it as a prop may lead to mismatch errors. The `active-class-name` example demonstrates one way to use `asPath` as a prop.

Client-side Rendering (CSR)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering
> 📁 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Learn how to implement client-side rendering in the Pages Router.*

In Client-Side Rendering (CSR) with React, the browser downloads a minimal HTML page and the JavaScript needed for the page. The JavaScript is then used to update the DOM and render the page. When the application is first loaded, the user may notice a slight delay before they can see the full page, this is because the page isn't fully rendered until all the JavaScript is downloaded, parsed, and executed.

After the page has been loaded for the first time, navigating to other pages on the same website is typically faster, as only necessary data needs to be fetched, and JavaScript can re-render parts of the page without requiring a full page refresh.

In Next.js, there are two ways you can implement client-side rendering:

1. Using React's `useEffect()` hook inside your pages instead of the server-side rendering methods (`getStaticProps` and `getServerSideProps`).
2. Using a data fetching library like SWR or TanStack Query to fetch data on the client (recommended).

Here's an example of using `useEffect()` inside a Next.js page:

```
// Archivo: pages/index.js
import React, { useState, useEffect } from 'react'

export function Page() {
  const [data, setData] = useState(null)

  useEffect(() => {
    const fetchData = async () => {
      const response = await fetch('https://api.example.com/data')
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`)
      }
      const result = await response.json()
      setData(result)
    }

    fetchData().catch((e) => {
      // handle the error as needed
      console.error('An error occurred while fetching the data: ', e)
    })
  }, [])

  return <p>{data ? `Your data: ${data}` : 'Loading ... '}</p>
}
```

In the example above, the component starts by rendering `Loading ...`. Then, once the data is fetched, it re-renders and displays the data.

Although fetching data in a `useEffect` is a pattern you may see in older React Applications, we recommend using a data-fetching library for better performance, caching, optimistic updates, and more. Here's a minimum example using SWR to fetch data on the client:

```
// Archivo: pages/index.js
import useSWR from 'swr'

export function Page() {
  const { data, error, isLoading } = useSWR(
    'https://api.example.com/data',
    fetcher
  )

  if (error) return <p>Failed to load.</p>
  if (isLoading) return <p>Loading ... </p>

  return <p>Your Data: {data}</p>
}
```

> **Good to know**:
>
> Keep in mind that CSR can impact SEO. Some search engine crawlers might not execute JavaScript and therefore only see the initial empty or loading state of your application. It can also lead to performance issues for users with slower internet connections or devices, as they need to wait for all the JavaScript to load and run before they can see the full page. Next.js promotes a hybrid approach that allows you to use a combination of server-side rendering, static site generation, and client-side rendering, **depending on the needs of each page** in your application. In the App Router, you can also use Loading UI with Suspense to show a loading indicator while the page is being rendered.

Rendering

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/rendering
> 📂 **Sección:** Pages > Building Your Application > Rendering
> 📝 **Descripción:** *Learn the fundamentals of rendering in React and Next.js.*

By default, Next.js **prerenders** every page. This means that Next.js generates HTML for each page in advance, instead of having it all done by client-side JavaScript. Prerendering can result in better performance and SEO.

Each generated HTML is associated with minimal JavaScript code necessary for that page. When a page is loaded by the browser, its JavaScript code runs and makes the page fully interactive (this process is called hydration in React).

Prerendering

Next.js has two forms of prerendering: **Static Generation** and **Server-side Rendering**. The difference is in **when** it generates the HTML for a page.

- Static Generation: The HTML is generated at **build time** and will be reused on each request.
- Server-side Rendering: The HTML is generated on **each request**.

Importantly, Next.js lets you choose which prerendering form you'd like to use for each page. You can create a "hybrid" Next.js app by using Static Generation for most pages and using Server-side Rendering for others.

We recommend using Static Generation over Server-side Rendering for performance reasons. Statically generated pages can be cached by CDN with no extra configuration to boost performance. However, in some cases, Server-side Rendering might be the only option.

You can also use client-side data fetching along with Static Generation or Server-side Rendering. That means some parts of a page can be rendered entirely by clientside JavaScript. To learn more, take a look at the Data Fetching documentation.

---

getStaticProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-static-props
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data and generate static pages with `getStaticProps`. Learn more about this API for data fetching in Next.js.*

If you export a function called `getStaticProps` (Static Site Generation) from a page, Next.js will prerender this page at build time using the props returned by `getStaticProps`.

```
// Archivo: pages/index.tsx switcher
import type { InferGetStaticPropsType, GetStaticProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```
// Archivo: pages/index.js switcher
export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

> Note that irrespective of rendering type, any `props` will be passed to the page component and can be viewed on the client-side in the initial HTML. This is to allow the page to be hydrated correctly. Make sure that you don't pass any sensitive information that shouldn't be available on the client in `props`.

The `getStaticProps` API reference covers all parameters and props that can be used with `getStaticProps`.

When should I use getStaticProps?

You should use `getStaticProps` if:

- The data required to render the page is available at build time ahead of a user's request
- The data comes from a headless CMS
- The page must be prerendered (for SEO) and be very fast — `getStaticProps` generates `HTML` and `JSON` files, both of which can be cached by a CDN for performance
- The data can be publicly cached (not user-specific). This condition can be bypassed in certain specific situation by using a Proxy to rewrite the path.

When does getStaticProps run

`getStaticProps` always runs on the server and never on the client. You can validate code written inside `getStaticProps` is removed from the client-side bundle with this tool.

- `getStaticProps` always runs during `next build`
- `getStaticProps` runs in the background when using `fallback: true`
- `getStaticProps` is called before initial render when using `fallback: blocking`
- `getStaticProps` runs in the background when using `revalidate`
- `getStaticProps` runs on-demand in the background when using `revalidate()`

When combined with Incremental Static Regeneration, `getStaticProps` will run in the background while the stale page is being revalidated, and the fresh page served to the browser.

`getStaticProps` does not have access to the incoming request (such as query parameters or HTTP headers) as it generates static HTML. If you need access to the request for your page, consider using Proxy in addition to `getStaticProps` .

Using getStaticProps to fetch data from a CMS

The following example shows how you can fetch a list of blog posts from a CMS.

```
// Archivo: pages/blog.tsx switcher
// posts will be populated at build time by getStaticProps()
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}

// This function gets called at build time on server-side.
// It won't be called on client-side, so you can even do
// direct database queries.
export async function getStaticProps() {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

```
// Archivo: pages/blog.js switcher
// posts will be populated at build time by getStaticProps()
export default function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>{post.title}</li>
      ))}
    </ul>
  )
}

// This function gets called at build time on server-side.
// It won't be called on client-side, so you can even do
// direct database queries.
export async function getStaticProps() {
  // Call an external API endpoint to get posts.
  // You can use any data fetching library
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts,
    },
  }
}
```

The `getStaticProps` API reference covers all parameters and props that can be used with `getStaticProps`.

Write server-side code directly

As `getStaticProps` runs only on the server-side, it will never run on the client-side. It won't even be included in the JS bundle for the browser, so you can write direct database queries without them being sent to browsers.

This means that instead of fetching an **API route** from `getStaticProps` (that itself fetches data from an external source), you can write the server-side code directly in `getStaticProps`.

Take the following example. An API route is used to fetch some data from a CMS. That API route is then called directly from `getStaticProps`. This produces an additional call, reducing performance. Instead, the logic for fetching the data from the CMS can be shared by using a `lib/` directory. Then it can be shared with `getStaticProps`.

```
// Archivo: lib/load-posts.js
// The following function is shared
// with getStaticProps and API routes
// from a `lib/` directory
export async function loadPosts() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts/')
  const data = await res.json()

  return data
}
```

```
// Archivo: pages/blog.js
// pages/blog.js
import { loadPosts } from '../lib/load-posts'

// This function runs only on the server side
export async function getStaticProps() {
  // Instead of fetching your `/api` route you can call the same
  // function directly in `getStaticProps`
  const posts = await loadPosts()

  // Props returned will be passed to the page component
  return { props: { posts } }
}
```

Alternatively, if you are **not** using API routes to fetch data, then the `fetch()` API *can* be used directly in `getStaticProps` to fetch data.

To verify what Next.js eliminates from the client-side bundle, you can use the next-code-elimination tool.

Statically generates both HTML and JSON

When a page with `getStaticProps` is prerendered at build time, in addition to the page HTML file, Next.js generates a JSON file holding the result of running `getStaticProps`.

This JSON file will be used in client-side routing through `next/link` or `next/router`. When you navigate to a page that's prerendered using `getStaticProps`, Next.js fetches this JSON file (pre-computed at build time) and uses it as the props for the page component. This means that client-side page transitions will **not** call `getStaticProps` as only the exported JSON is used.

When using Incremental Static Generation, `getStaticProps` will be executed in the background to generate the JSON needed for client-side navigation. You may see this in the form of multiple requests being made for the same page, however, this is intended and has no impact on end-user performance.

Where can I use getStaticProps

`getStaticProps` can only be exported from a **page**. You **cannot** export it from non-page files, `_app`, `_document`, or `_error`.

One of the reasons for this restriction is that React needs to have all the required data before the page is rendered.

Also, you must use export `getStaticProps` as a standalone function — it will **not** work if you add `getStaticProps` as a property of the page component.

> **Good to know**: if you have created a custom app, ensure you are passing the `pageProps` to the page component as shown in the linked document, otherwise the props will be empty.

Runs on every request in development

In development (`next dev`), `getStaticProps` will be called on every request.

Preview Mode

You can temporarily bypass static generation and render the page at **request time** instead of build time using **Preview Mode**. For example, you might be using a headless CMS and want to preview drafts before they're published.

getStaticPaths

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-static-paths
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data and generate static pages with `getStaticPaths`. Learn more about this API for data fetching in Next.js.*

If a page has Dynamic Routes and uses `getStaticProps`, it needs to define a list of paths to be statically generated.

When you export a function called `getStaticPaths` (Static Site Generation) from a page that uses dynamic routes, Next.js will statically prerender all the paths specified by `getStaticPaths`.

```
// Archivo: pages/repo/[name].tsx switcher
import type {
  InferGetStaticPropsType,
  GetStaticProps,
  GetStaticPaths,
} from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticPaths = (async () => {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}) satisfies GetStaticPaths

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```
// Archivo: pages/repo/[name].js switcher
export async function getStaticPaths() {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}

export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

The `getStaticPaths` API reference covers all parameters and props that can be used with `getStaticPaths`.

When should I use getStaticPaths?

You should use `getStaticPaths` if you're statically prerendering pages that use dynamic routes and:

- The data comes from a headless CMS
- The data comes from a database
- The data comes from the filesystem
- The data can be publicly cached (not user-specific)
- The page must be prerendered (for SEO) and be very fast — `getStaticProps` generates `HTML` and `JSON` files, both of which can be cached by a CDN for performance

When does getStaticPaths run

`getStaticPaths` will only run during build in production, it will not be called during runtime. You can validate code written inside `getStaticPaths` is removed from the client-side bundle with this tool.

How does getStaticProps run with regards to getStaticPaths

- `getStaticProps` runs during `next build` for any `paths` returned during build
- `getStaticProps` runs in the background when using `fallback: true`
- `getStaticProps` is called before initial render when using `fallback: blocking`

Where can I use getStaticPaths

- `getStaticPaths` **must** be used with `getStaticProps`
- You **cannot** use `getStaticPaths` with `getServerSideProps`
- You can export `getStaticPaths` from a Dynamic Route that also uses `getStaticProps`
- You **cannot** export `getStaticPaths` from non-page file (e.g. your `components` folder)
- You must export `getStaticPaths` as a standalone function, and not a property of the page component

Runs on every request in development

In development (`next dev`), `getStaticPaths` will be called on every request.

Generating paths on-demand

`getStaticPaths` allows you to control which pages are generated during the build instead of on-demand with `fallback`. Generating more pages during a build will cause slower builds.

You can defer generating all pages on-demand by returning an empty array for `paths`. This can be especially helpful when deploying your Next.js application to multiple environments. For example, you can have faster builds by generating all pages on-demand for previews (but not production builds). This is helpful for sites with hundreds/thousands of static pages.

```
// Archivo: pages/posts/[id].js
export async function getStaticPaths() {
  // When this is true (in preview environments) don't
  // prerender any static pages
  // (faster builds, but slower initial page load)
  if (process.env.SKIP_BUILD_STATIC_GENERATION) {
    return {
      paths: [],
      fallback: 'blocking',
    }
  }

  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // Get the paths we want to prerender based on posts
  // In production environments, prerender all pages
  // (slower builds, but faster initial page load)
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  // { fallback: false } means other routes should 404
  return { paths, fallback: false }
}
```

---

getServerSideProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/get-server-side-props
> 📁 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Fetch data on each request with* `getServerSideProps`.

`getServerSideProps` is a Next.js function that can be used to fetch data and render the contents of a page at request time.

Example

You can use `getServerSideProps` by exporting it from a Page Component. The example below shows how you can fetch data from a 3rd party API in `getServerSideProps`, and pass the data to the page as props:

```tsx
// Archivo: pages/index.tsx switcher
import type { InferGetServerSidePropsType, GetServerSideProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getServerSideProps = (async () => {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo: Repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}) satisfies GetServerSideProps<{ repo: Repo }>

export default function Page({
  repo,
}: InferGetServerSidePropsType<typeof getServerSideProps>) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

```jsx
// Archivo: pages/index.js switcher
export async function getServerSideProps() {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}

export default function Page({ repo }) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

When should I use `getServerSideProps`?

You should use `getServerSideProps` if you need to render a page that relies on personalized user data, or information that can only be known at request time. For example, `authorization` headers or a geolocation.

If you do not need to fetch the data at request time, or would prefer to cache the data and prerendered HTML, we recommend using `getStaticProps`.

Behavior

- `getServerSideProps` runs on the server.
- `getServerSideProps` can only be exported from a **page**.
- `getServerSideProps` returns JSON.
- When a user visits a page, `getServerSideProps` will be used to fetch data at request time, and the data is used to render the initial HTML of the page.
- `props` passed to the page component can be viewed on the client as part of the initial HTML. This is to allow the page to be hydrated correctly. Make sure that you don't pass any sensitive information that shouldn't be available on the client in `props`.
- When a user visits the page through `next/link` or `next/router`, Next.js sends an API request to the server, which runs `getServerSideProps`.

- You do not have to call a Next.js API Route to fetch data when using `getServerSideProps` since the function runs on the server. Instead, you can call a CMS, database, or other third-party APIs directly from inside `getServerSideProps`.

> **Good to know:**
>
> - See `getServerSideProps` API reference for parameters and props that can be used with `getServerSideProps`.
> - You can use the next-code-elimination tool to verify what Next.js eliminates from the client-side bundle.

Error Handling

If an error is thrown inside `getServerSideProps`, it will show the `pages/500.js` file. Check out the documentation for 500 page to learn more on how to create it. During development, this file will not be used and the development error overlay will be shown instead.

Edge Cases

Caching with Server-Side Rendering (SSR)

You can use caching headers (`Cache-Control`) inside `getServerSideProps` to cache dynamic responses. For example, using `stale-while-revalidate`.

```
// This value is considered fresh for ten seconds (s-maxage=10).
// If a request is repeated within the next 10 seconds, the previously
// cached value will still be fresh. If the request is repeated before 59 seconds,
// the cached value will be stale but still render (stale-while-revalidate=59).
//
// In the background, a revalidation request will be made to populate the cache
// with a fresh value. If you refresh the page, you will see the new value.
export async function getServerSideProps({ req, res }) {
  res.setHeader(
    'Cache-Control',
    'public, s-maxage=10, stale-while-revalidate=59'
  )

  return {
    props: {},
  }
}
```

However, before reaching for `cache-control`, we recommend seeing if `getStaticProps` with ISR is a better fit for your use case.

Client-side Fetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching/client-side
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Learn about client-side data fetching, and how to use SWR, a data fetching React Hook library that handles caching, revalidation, focus tracking, refetching on interval and more.*

Client-side data fetching is useful when your page doesn't require SEO indexing, when you don't need to prerender your data, or when the content of your pages needs to update frequently. Unlike the server-side rendering APIs, you can use client-side data fetching at the component level.

If done at the page level, the data is fetched at runtime, and the content of the page is updated as the data changes. When used at the component level, the data is fetched at the time of the component mount, and the content of the component is updated as the data changes.

It's important to note that using client-side data fetching can affect the performance of your application and the load speed of your pages. This is because the data fetching is done at the time of the component or pages mount, and the data is not cached.

Client-side data fetching with useEffect

The following example shows how you can fetch data on the client side using the useEffect hook.

```
import { useState, useEffect } from 'react'

function Profile() {
  const [data, setData] = useState(null)
  const [isLoading, setLoading] = useState(true)

  useEffect(() => {
    fetch('/api/profile-data')
      .then((res) => res.json())
      .then((data) => {
        setData(data)
        setLoading(false)
      })
  }, [])

  if (isLoading) return <p>Loading ...</p>
  if (!data) return <p>No profile data</p>

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.bio}</p>
    </div>
  )
}
```

Client-side data fetching with SWR

The team behind Next.js has created a React Hook library for data fetching called **SWR**. It is **highly recommended** if you are fetching data on the client-side. It handles caching, revalidation, focus tracking, refetching on intervals, and more.

Using the same example as above, we can now use SWR to fetch the profile data. SWR will automatically cache the data for us and will revalidate the data if it becomes stale.

For more information on using SWR, check out the SWR docs.

```
import useSWR from 'swr'

const fetcher = (...args) => fetch(...args).then((res) => res.json())

function Profile() {
  const { data, error } = useSWR('/api/profile-data', fetcher)

  if (error) return <div>Failed to load</div>
  if (!data) return <div>Loading ...</div>

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.bio}</p>
    </div>
  )
}
```

Data Fetching

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/data-fetching
> 📂 **Sección:** Pages > Building Your Application > Data Fetching
> 📝 **Descripción:** *Next.js allows you to fetch data in multiple ways, with prerendering, server-side rendering or static-site generation, and incremental static regeneration. Learn how to manage your application data in Next.js.*

Data fetching in Next.js allows you to render your content in different ways, depending on your application's use case. These include prerendering with **Server-side Rendering** or **Static Generation**, and updating or creating content at runtime with **Incremental Static Regeneration**.

Examples

- Agility CMS Example (Demo)
- Builder.io Example (Demo)
- ButterCMS Example (Demo)
- Contentful Example (Demo)
- Cosmic Example (Demo)
- DatoCMS Example (Demo)
- DotCMS Example (Demo)
- Drupal Example (Demo)
- Enterspeed Example (Demo)
- GraphCMS Example (Demo)
- Keystone Example (Demo)
- Kontent.ai Example (Demo)
- Makeswift Example (Demo)
- Plasmic Example (Demo)
- Prepr Example (Demo)
- Prismic Example (Demo)
- Sanity Example (Demo)
- Sitecore XM Cloud Example (Demo)
- Storyblok Example (Demo)
- Strapi Example (Demo)
- TakeShape Example (Demo)
- Tina Example (Demo)
- Umbraco Example (Demo)
- Umbraco Heartcore Example (Demo)
- Webiny Example (Demo)
- WordPress Example (Demo)
- Blog Starter Example (Demo)
- Static Tweet (Demo)

Error Handling

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/configuring/error-handling
> 📂 **Sección:** Pages > Building Your Application > Configuring
> 📝 **Descripción:** *Handle errors in your Next.js app.*

This documentation explains how you can handle development, server-side, and client-side errors.

Handling Errors in Development

When there is a runtime error during the development phase of your Next.js application, you will encounter an **overlay**. It is a modal that covers the webpage. It is **only** visible when the development server runs using `next dev` via `pnpm dev`, `npm run dev`, `yarn dev`, or `bun dev` and will not be shown in production. Fixing the error will automatically dismiss the overlay.

Handling Server Errors

Next.js provides a static 500 page by default to handle server-side errors that occur in your application. You can also customize this page by creating a `pages/500.js` file.

Having a 500 page in your application does not show specific errors to the app user.

You can also use 404 page to handle specific runtime error like `file not found`.

Handling Client Errors

React Error Boundaries is a graceful way to handle a JavaScript error on the client so that the other parts of the application continue working. In addition to preventing the page from crashing, it allows you to provide a custom fallback component and even log error information.

To use Error Boundaries for your Next.js application, you must create a class component `ErrorBoundary` and wrap the `Component` prop in the `pages/_app.js` file. This component will be responsible to:

- Render a fallback UI after an error is thrown
- Provide a way to reset the Application's state
- Log error information

You can create an `ErrorBoundary` class component by extending `React.Component`. For example:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props)

    // Define a state variable to track whether is an error or not
    this.state = { hasError: false }
  }
  static getDerivedStateFromError(error) {
    // Update state so the next render will show the fallback UI

    return { hasError: true }
  }
  componentDidCatch(error, errorInfo) {
    // You can use your own error logging service here
    console.log({ error, errorInfo })
  }
  render() {
    // Check if the error is thrown
    if (this.state.hasError) {
      // You can render any custom fallback UI
      return (
        <div>
          <h2>Oops, there is an error!</h2>
          <button
            type="button"
            onClick={() => this.setState({ hasError: false })}
          >
            Try again?
          </button>
        </div>
      )
    }

    // Return children components in case of no error

    return this.props.children
  }
}

export default ErrorBoundary
```

The `ErrorBoundary` component keeps track of an `hasError` state. The value of this state variable is a boolean. When the value of `hasError` is `true`, then the `ErrorBoundary` component will render a fallback UI. Otherwise, it will render the children components.

After creating an `ErrorBoundary` component, import it in the `pages/_app.js` file to wrap the `Component` prop in your Next.js application.

```
// Import the ErrorBoundary component
import ErrorBoundary from '../components/ErrorBoundary'

function MyApp({ Component, pageProps }) {
  return (
    // Wrap the Component prop with ErrorBoundary component
    <ErrorBoundary>
      <Component {...pageProps} />
    </ErrorBoundary>
  )
}

export default MyApp
```

You can learn more about Error Boundaries in React's documentation.

Reporting Errors

To monitor client errors, use a service like Sentry, Bugsnag or Datadog.

Configuring

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application/configuring
> 📁 **Sección:** Pages > Building Your Application > Configuring
> 📝 **Descripción:** *Learn how to configure your Next.js application.*

Next.js allows you to customize your project to meet specific requirements. This includes integrations with TypeScript, ESlint, and more, as well as internal configuration options such as Absolute Imports and Environment Variables.

Building Your Application

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/building-your-application
> 📁 **Sección:** Pages > Building Your Application
> 📝 **Descripción:** *Learn how to use Next.js features to build your application.*

Font Module

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/font
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *API Reference for the Font Module*

Form

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/form
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *Learn how to use the `<Form>` component to handle form submissions and search params updates with client-side navigation.*

Head

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/head
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *Add custom elements to the `head` of your page with the built-in Head component.*

We expose a built-in component for appending elements to the `head` of the page:

```
import Head from 'next/head'

function IndexPage() {
  return (
    <div>
      <Head>
        <title>My page title</title>
      </Head>
      <p>Hello world!</p>
    </div>
  )
}

export default IndexPage
```

Avoid duplicated tags

To avoid duplicate tags in your `head` you can use the `key` property, which will make sure the tag is only rendered once, as in the following example:

```
import Head from 'next/head'

function IndexPage() {
  return (
    <div>
      <Head>
        <title>My page title</title>
        <meta property="og:title" content="My page title" key="title" />
      </Head>
      <Head>
        <meta property="og:title" content="My new title" key="title" />
      </Head>
      <p>Hello world!</p>
    </div>
  )
}

export default IndexPage
```

In this case only the second `<meta property="og:title" />` is rendered. `meta` tags with duplicate `key` attributes are automatically handled.

> **Good to know**: `<title>` and `<base>` tags are automatically checked for duplicates by Next.js, so using key is not necessary for these tags.

> The contents of `head` get cleared upon unmounting the component, so make sure each page completely defines what it needs in `head`, without making assumptions about what other pages added.

Use minimal nesting

`title`, `meta` or any other elements (e.g. `script`) need to be contained as **direct** children of the `Head` element, or wrapped into maximum one level of `<React.Fragment>` or arrays—otherwise the tags won't be correctly picked up on client-side navigations.

Use `next/script` for scripts

We recommend using `next/script` in your component instead of manually creating a `<script>` in `next/head`.

No `html` or `body` tags

You **cannot** use `<Head>` to set attributes on `<html>` or `<body>` tags. This will result in an `next-head-count is missing` error. `next/head` can only handle tags inside the HTML `<head>` tag.

Image (Legacy)

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/image-legacy
> 📂 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *Backwards compatible Image Optimization with the Legacy Image component.*

Starting with Next.js 13, the `next/image` component was rewritten to improve both the performance and developer experience. In order to provide a backwards compatible upgrade solution, the old `next/image` was renamed to `next/legacy/image`.

> **Warning**: `next/legacy/image` is deprecated and will be removed in a future version of Next.js. Please use `next/image` instead.

Comparison

Compared to `next/legacy/image`, the new `next/image` component has the following changes:

- Removes `<span>` wrapper around `<img>` in favor of native computed aspect ratio
- Adds support for canonical `style` prop
  - Removes `layout` prop in favor of `style` or `className`
  - Removes `objectFit` prop in favor of `style` or `className`
  - Removes `objectPosition` prop in favor of `style` or `className`
- Removes `IntersectionObserver` implementation in favor of native lazy loading
  - Removes `lazyBoundary` prop since there is no native equivalent
  - Removes `lazyRoot` prop since there is no native equivalent
- Removes `loader` config in favor of `loader` prop
- Changed `alt` prop from optional to required
- Changed `onLoadingComplete` callback to receive reference to `<img>` element

Required Props

The `

` component requires the following properties.

src

Must be one of the following:

- A statically imported image file
- A path string. This can be either an absolute external URL, or an internal path depending on the loader prop or loader configuration.

When using the default loader, also consider the following for source images:

- When src is an external URL, you must also configure remotePatterns

- When src is animated or not a known format (JPEG, PNG, WebP, AVIF, GIF, TIFF) the image will be served as-is
- When src is SVG format, it will be blocked unless `unoptimized` or `dangerouslyAllowSVG` is enabled

width

The `width` property can represent either the *rendered* width or *original* width in pixels, depending on the `layout` and `sizes` properties.

When using `layout="intrinsic"` or `layout="fixed"` the `width` property represents the *rendered* width in pixels, so it will affect how large the image appears.

When using `layout="responsive"`, `layout="fill"`, the `width` property represents the *original* width in pixels, so it will only affect the aspect ratio.

The `width` property is required, except for statically imported images, or those with `layout="fill"`.

height

The `height` property can represent either the *rendered* height or *original* height in pixels, depending on the `layout` and `sizes` properties.

When using `layout="intrinsic"` or `layout="fixed"` the `height` property represents the *rendered* height in pixels, so it will affect how large the image appears.

When using `layout="responsive"`, `layout="fill"`, the `height` property represents the *original* height in pixels, so it will only affect the aspect ratio.

The `height` property is required, except for statically imported images, or those with `layout="fill"`.

Optional Props

The `

Illustration

` component accepts a number of additional properties beyond those which are required. This section describes the most commonly-used properties of the Image component. Find details about more rarely-used properties in the Advanced Props section.

layout

The layout behavior of the image as the viewport changes size.

`layout`	Behavior	`srcSet`	`sizes`	Has wrapper and sizer
`intrinsic` (default)	Scale *down* to fit width of container, up to image size	`1x`, `2x` (based on imageSizes)	N/A	yes
`fixed`	Sized to `width` and `height` exactly	`1x`, `2x` (based on imageSizes)	N/A	yes
`responsive`	Scale to fit width of container	`640w`, `750w`, ... `2048w`, `3840w` (based on imageSizes and deviceSizes)	`100vw`	yes
`fill`	Grow in both X and Y axes to fill container	`640w`, `750w`, ... `2048w`, `3840w` (based on imageSizes and deviceSizes)	`100vw`	yes

- Demo the `intrinsic` layout (default)
  - When `intrinsic`, the image will scale the dimensions down for smaller viewports, but maintain the original dimensions for larger viewports.
- Demo the `fixed` layout

- When `fixed`, the image dimensions will not change as the viewport changes (no responsiveness) similar to the native `img` element.

- Demo the `responsive` layout
  - When `responsive`, the image will scale the dimensions down for smaller viewports and scale up for larger viewports.
  - Ensure the parent element uses `display: block` in their stylesheet.

- Demo the `fill` layout
  - When `fill`, the image will stretch both width and height to the dimensions of the parent element, provided the parent element is relative.
  - This is usually paired with the `objectFit` property.
  - Ensure the parent element has `position: relative` in their stylesheet.

- Demo background image

loader

A custom function used to resolve URLs. Setting the loader as a prop on the Image component overrides the default loader defined in the `images` section of `next.config.js`.

A `loader` is a function returning a URL string for the image, given the following parameters:

- `src`
- `width`
- `quality`

Here is an example of using a custom loader:

```
import Image from 'next/legacy/image'

const myLoader = ({ src, width, quality }) => {
  return `https://example.com/${src}?w=${width}&q=${quality || 75}`
}

const MyImage = (props) => {
  return (


![Picture of the author](me.png)


  )
}
```

sizes

A string that provides information about how wide the image will be at different breakpoints. The value of `sizes` will greatly affect performance for images using `layout="responsive"` or `layout="fill"`. It will be ignored for images using `layout="intrinsic"` or `layout="fixed"`.

The `sizes` property serves two important purposes related to image performance:

First, the value of `sizes` is used by the browser to determine which size of the image to download, from `next/legacy/image`'s automatically-generated source set. When the browser chooses, it does not yet know the size of the image on the page, so it selects an image that is the same size or larger than the viewport. The `sizes` property allows you to tell the browser that the image will actually be smaller than full screen. If you don't specify a `sizes` value, a default value of `100vw` (full screen width) is used.

Second, the `sizes` value is parsed and used to trim the values in the automatically-created source set. If the `sizes` property includes sizes such as `50vw`, which represent a percentage of the viewport width, then the source set is trimmed to not include any values which are too small to ever be necessary.

For example, if you know your styling will cause an image to be full-width on mobile devices, in a 2-column layout on tablets, and a 3-column layout on desktop displays, you should include a sizes property such as the following:

```
import Image from 'next/legacy/image'
const Example = () => (
  <div className="grid-element">


![Illustration](https://nextjs.org/example.png)


  </div>
)
```

This example `sizes` could have a dramatic effect on performance metrics. Without the `33vw` sizes, the image selected from the server would be 3 times as wide as it needs to be. Because file size is proportional to the square of the width, without `sizes` the user would download an image that's 9 times larger than necessary.

Learn more about `srcset` and `sizes`:

- web.dev
- mdn

quality

The quality of the optimized image, an integer between `1` and `100` where `100` is the best quality. Defaults to `75`.

priority

When true, the image will be considered high priority and preload. Lazy loading is automatically disabled for images using `priority`.

You should use the `priority` property on any image detected as the Largest Contentful Paint (LCP) element. It may be appropriate to have multiple priority images, as different images may be the LCP element for different viewport sizes.

Should only be used when the image is visible above the fold. Defaults to `false`.

placeholder

A placeholder to use while the image is loading. Possible values are `blur` or `empty`. Defaults to `empty`.

When `blur`, the `blurDataURL` property will be used as the placeholder. If `src` is an object from a static import and the imported image is `.jpg`, `.png`, `.webp`, or `.avif`, then `blurDataURL` will be automatically populated.

For dynamic images, you must provide the `blurDataURL` property. Solutions such as Plaiceholder can help with `base64` generation.

When `empty`, there will be no placeholder while the image is loading, only empty space.

Try it out:

- Demo the `blur` placeholder
- Demo the shimmer effect with `blurDataURL` prop
- Demo the color effect with `blurDataURL` prop

Advanced Props

In some cases, you may need more advanced usage. The `

` component optionally accepts the following advanced properties.

style

Allows passing CSS styles to the underlying image element.

Note that all `layout` modes apply their own styles to the image element, and these automatic styles take precedence over the `style` prop.

Also keep in mind that the required `width` and `height` props can interact with your styling. If you use styling to modify an image's `width`, you must set the `height="auto"` style as well, or your image will be distorted.

objectFit

Defines how the image will fit into its parent container when using `layout="fill"`.

This value is passed to the object-fit CSS property for the `src` image.

objectPosition

Defines how the image is positioned within its parent element when using `layout="fill"`.

This value is passed to the object-position CSS property applied to the image.

onLoadingComplete

A callback function that is invoked once the image is completely loaded and the placeholder has been removed.

The `onLoadingComplete` function accepts one parameter, an object with the following properties:

- `naturalWidth`
- `naturalHeight`

loading

The loading behavior of the image. Defaults to `lazy`.

When `lazy`, defer loading the image until it reaches a calculated distance from the viewport.

When `eager`, load the image immediately.

Learn more

blurDataURL

A Data URL to be used as a placeholder image before the `src` image successfully loads. Only takes effect when combined with `placeholder="blur"`.

Must be a base64-encoded image. It will be enlarged and blurred, so a very small image (10px or less) is recommended. Including larger images as placeholders may harm your application performance.

Try it out:

- Demo the default `blurDataURL` prop
- Demo the shimmer effect with `blurDataURL` prop
- Demo the color effect with `blurDataURL` prop

You can also generate a solid color Data URL to match the image.

lazyBoundary

A string (with similar syntax to the margin property) that acts as the bounding box used to detect the intersection of the viewport with the image and trigger lazy loading. Defaults to `"200px"`.

If the image is nested in a scrollable parent element other than the root document, you will also need to assign the lazyRoot prop.

Learn more

lazyRoot

A React Ref pointing to the scrollable parent element. Defaults to `null` (the document viewport).

The Ref must point to a DOM element or a React component that forwards the Ref to the underlying DOM element.

Example pointing to a DOM element

```
import Image from 'next/legacy/image'
import React from 'react'

const Example = () => {
  const lazyRoot = React.useRef(null)

  return (
    <div ref={lazyRoot} style={{ overflowX: 'scroll', width: '500px' }}>


![Illustration](https://nextjs.org/one.jpg)




![Illustration](https://nextjs.org/two.jpg)


    </div>
  )
}
```

Example pointing to a React component

```
import Image from 'next/legacy/image'
import React from 'react'

const Container = React.forwardRef((props, ref) => {
  return (
    <div ref={ref} style={{ overflowX: 'scroll', width: '500px' }}>
      {props.children}
    </div>
  )
})

const Example = () => {
  const lazyRoot = React.useRef(null)

  return (
    <Container ref={lazyRoot}>


![Illustration](https://nextjs.org/one.jpg)




![Illustration](https://nextjs.org/two.jpg)


    </Container>
  )
}
```

Learn more

unoptimized

When true, the source image will be served as-is from the `src` instead of changing quality, size, or format. Defaults to `false`.

This is useful for images that do not benefit from optimization such as small images (less than 1KB), vector images (SVG), or animated images (GIF).

```
import Image from 'next/image'

const UnoptimizedImage = (props) => {
  return

![Illustration]()


}
```

Since Next.js 12.3.0, this prop can be assigned to all images by updating `next.config.js` with the following configuration:

```
// Archivo: next.config.js
module.exports = {
  images: {
    unoptimized: true,
  },
}
```

Other Props

Other properties on the `

Illustration

`component will be passed to the underlying` img` element with the exception of the following:

- `srcSet`. Use Device Sizes instead.
- `ref`. Use `onLoadingComplete` instead.
- `decoding`. It is always `"async"`.

Configuration Options

Remote Patterns

To protect your application from malicious users, configuration is required in order to use external images. This ensures that only external images from your account can be served from the Next.js Image Optimization API. These external images can be configured with the `remotePatterns` property in your `next.config.js` file, as shown below:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'example.com',
        port: '',
        pathname: '/account123/**',
        search: '',
      },
    ],
  },
}
```

> **Good to know**: The example above will ensure the `src` property of `next/legacy/image` must start with `https://example.com/account123/` and must not have a query string. Any other protocol, hostname, port, or unmatched path will respond with 400 Bad Request.

Below is an example of the `remotePatterns` property in the `next.config.js` file using a wildcard pattern in the `hostname`:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: '**.example.com',
        port: '',
        search: '',
      },
    ],
  },
}
```

> **Good to know**: The example above will ensure the `src` property of `next/legacy/image` must start with `https://img1.example.com` or `https://me.avatar.example.com` or any number of subdomains. It cannot have a port or query string. Any other protocol or unmatched hostname will respond with 400 Bad Request.

Wildcard patterns can be used for both `pathname` and `hostname` and have the following syntax:

- `*` match a single path segment or subdomain
- `**` match any number of path segments at the end or subdomains at the beginning

The `**` syntax does not work in the middle of the pattern.

> **Good to know**: When omitting `protocol`, `port`, `pathname`, or `search` then the wildcard `**` is implied. This is not recommended because it may allow malicious actors to optimize urls you did not intend.

Below is an example of the `remotePatterns` property in the `next.config.js` file using `search`:

```
// Archivo: next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'assets.example.com',
        search: '?v=1727111025337',
      },
    ],
  },
}
```

> **Good to know**: The example above will ensure the `src` property of `next/legacy/image` must start with `https://assets.example.com` and must have the exact query string `?v=1727111025337`. Any other protocol or query string will respond with 400 Bad Request.

Domains

> **Warning**: Deprecated since Next.js 14 in favor of strict `remotePatterns` in order to protect your application from malicious users. Only use `domains` if you own all the content served from the domain.

Similar to `remotePatterns`, the `domains` configuration can be used to provide a list of allowed hostnames for external images.

However, the `domains` configuration does not support wildcard pattern matching and it cannot restrict protocol, port, or pathname.

Below is an example of the `domains` property in the `next.config.js` file:

```
// Archivo: next.config.js
module.exports = {
  images: {
    domains: ['assets.acme.com'],
  },
}
```

Loader Configuration

If you want to use a cloud provider to optimize images instead of using the Next.js built-in Image Optimization API, you can configure the `loader` and `path` prefix in your `next.config.js` file. This allows you to use relative URLs for the Image `src` and automatically generate the correct absolute URL for your provider.

```
// Archivo: next.config.js
module.exports = {
  images: {
    loader: 'imgix',
    path: 'https://example.com/myaccount/',
  },
}
```

Customizing the Built-in Image Path

If you want to change or prefix the default path for the built-in Next.js image optimization, you can do so with the `path` property. The default value for `path` is `/_next/image`.

```
// Archivo: next.config.js
module.exports = {
  images: {
    path: '/my-prefix/_next/image',
  },
}
```

Built-in Loaders

The following Image Optimization cloud providers are included:

- Default: Works automatically with `next dev`, `next start`, or a custom server
- Vercel: Works automatically when you deploy on Vercel, no configuration necessary. Learn more
- Imgix: `loader: 'imgix'`
- Cloudinary: `loader: 'cloudinary'`
- Akamai: `loader: 'akamai'`
- Custom: `loader: 'custom'` use a custom cloud provider by implementing the `loader` prop on the `next/legacy/image` component

If you need a different provider, you can use the `loader` prop with `next/legacy/image`.

> Images cannot be optimized at build time using `output: 'export'`, only on-demand. To use `next/legacy/image` with `output: 'export'`, you will need to use a different loader than the default. Read more in the discussion.

Advanced

The following configuration is for advanced use cases and is usually not necessary. If you choose to configure the properties below, you will override any changes to the Next.js defaults in future updates.

Device Sizes

If you know the expected device widths of your users, you can specify a list of device width breakpoints using the `deviceSizes` property in `next.config.js`. These widths are used when the `next/legacy/image` component uses `layout="responsive"` or `layout="fill"`

to ensure the correct image is served for user's device.

If no configuration is provided, the default below is used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048, 3840],
  },
}
```

Image Sizes

You can specify a list of image widths using the `images.imageSizes` property in your `next.config.js` file. These widths are concatenated with the array of device sizes to form the full array of sizes used to generate image srcsets.

The reason there are two separate lists is that imageSizes is only used for images which provide a `sizes` prop, which indicates that the image is less than the full width of the screen. **Therefore, the sizes in imageSizes should all be smaller than the smallest size in deviceSizes.**

If no configuration is provided, the default below is used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    imageSizes: [32, 48, 64, 96, 128, 256, 384],
  },
}
```

Acceptable Formats

The default Image Optimization API will automatically detect the browser's supported image formats via the request's `Accept` header in order to determine the best output format.

If the `Accept` header matches more than one of the configured formats, the first match in the array is used. Therefore, the array order matters. If there is no match (or the source image is animated), the Image Optimization API will fallback to the original image's format.

If no configuration is provided, the default below is used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    formats: ['image/webp'],
  },
}
```

You can enable AVIF support, which will fallback to the original format of the src image if the browser does not support AVIF:

```
// Archivo: next.config.js
module.exports = {
  images: {
    formats: ['image/avif'],
  },
}
```

You can also enable both AVIF and WebP formats together. AVIF will be preferred for browsers that support it, with WebP as a fallback:

```
// Archivo: next.config.js
module.exports = {
  images: {
    formats: ['image/avif', 'image/webp'],
  },
}
```

Good to know:

- We still recommend using WebP for most use cases.
- AVIF generally takes 50% longer to encode but it compresses 20% smaller compared to WebP. This means that the first time an image is requested, it will typically be slower and then subsequent requests that are cached will be faster.
- When using multiple formats, Next.js will cache each format separately. This means increased storage requirements compared to using a single format, as both AVIF and WebP versions of images will be stored for different browser support.
- If you self-host with a Proxy/CDN in front of Next.js, you must configure the Proxy to forward the `Accept` header.

Caching Behavior

The following describes the caching algorithm for the default loader. For all other loaders, please refer to your cloud provider's documentation.

Images are optimized dynamically upon request and stored in the `<distDir>/cache/images` directory. The optimized image file will be served for subsequent requests until the expiration is reached. When a request is made that matches a cached but expired file, the expired image is served stale immediately. Then the image is optimized again in the background (also called revalidation) and saved to the cache with the new expiration date.

The cache status of an image can be determined by reading the value of the `x-nextjs-cache` (`x-vercel-cache` when deployed on Vercel) response header. The possible values are the following:

- `MISS` - the path is not in the cache (occurs at most once, on the first visit)
- `STALE` - the path is in the cache but exceeded the revalidate time so it will be updated in the background
- `HIT` - the path is in the cache and has not exceeded the revalidate time

The expiration (or rather Max Age) is defined by either the `minimumCacheTTL` configuration or the upstream image `Cache-Control` header, whichever is larger. Specifically, the `max-age` value of the `Cache-Control` header is used. If both `s-maxage` and `max-age` are found, then `s-maxage` is preferred. The `max-age` is also passed-through to any downstream clients including CDNs and browsers.

- You can configure `minimumCacheTTL` to increase the cache duration when the upstream image does not include `Cache-Control` header or the value is very low.
- You can configure `deviceSizes` and `imageSizes` to reduce the total number of possible generated images.
- You can configure formats to disable multiple formats in favor of a single image format.

Minimum Cache TTL

You can configure the Time to Live (TTL) in seconds for cached optimized images. In many cases, it's better to use a Static Image Import which will automatically hash the file contents and cache the image forever with a `Cache-Control` header of `immutable`.

If no configuration is provided, the default below is used.

```
// Archivo: next.config.js
module.exports = {
  images: {
    minimumCacheTTL: 14400, // 4 hours
  },
}
```

You can increase the TTL to reduce the number of revalidations and potentially lower cost:

```
// Archivo: next.config.js
module.exports = {
  images: {
    minimumCacheTTL: 2678400, // 31 days
  },
}
```

The expiration (or rather Max Age) of the optimized image is defined by either the `minimumCacheTTL` or the upstream image `Cache-Control` header, whichever is larger.

If you need to change the caching behavior per image, you can configure `headers` to set the `Cache-Control` header on the upstream image (e.g. `/some-asset.jpg`, not `/_next/image` itself).

There is no mechanism to invalidate the cache at this time, so its best to keep `minimumCacheTTL` low. Otherwise you may need to manually change the `src` prop or delete `<distDir>/cache/images`.

Disable Static Imports

The default behavior allows you to import static files such as `import icon from './icon.png'` and then pass that to the `src` property.

In some cases, you may wish to disable this feature if it conflicts with other plugins that expect the import to behave differently.

You can disable static image imports inside your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  images: {
    disableStaticImages: true,
  },
}
```

Dangerously Allow SVG

The default loader does not optimize SVG images for a few reasons. First, SVG is a vector format meaning it can be resized losslessly. Second, SVG has many of the same features as HTML/CSS, which can lead to vulnerabilities without proper Content Security Policy (CSP) headers.

Therefore, we recommended using the `unoptimized` prop when the `src` prop is known to be SVG. This happens automatically when `src` ends with `".svg"`.

However, if you need to serve SVG images with the default Image Optimization API, you can set `dangerouslyAllowSVG` inside your `next.config.js`:

```
// Archivo: next.config.js
module.exports = {
  images: {
    dangerouslyAllowSVG: true,
    contentDispositionType: 'attachment',
    contentSecurityPolicy: "default-src 'self'; script-src 'none'; sandbox;",
  },
}
```

In addition, it is strongly recommended to also set `contentDispositionType` to force the browser to download the image, as well as `contentSecurityPolicy` to prevent scripts embedded in the image from executing.

`contentDispositionType`

The default loader sets the `Content-Disposition` header to `attachment` for added protection since the API can serve arbitrary remote images.

The default value is `attachment` which forces the browser to download the image when visiting directly. This is particularly important when `dangerouslyAllowSVG` is true.

You can optionally configure `inline` to allow the browser to render the image when visiting directly, without downloading it.

```
// Archivo: next.config.js
module.exports = {
  images: {
    contentDispositionType: 'inline',
  },
}
```

Animated Images

The default loader will automatically bypass Image Optimization for animated images and serve the image as-is.

Auto-detection for animated files is best-effort and supports GIF, APNG, and WebP. If you want to explicitly bypass Image Optimization for a given animated image, use the unoptimized prop.

Version History

Version	Changes
`v16.0.0`	`next/legacy/image` deprecated and will be removed in a future version of Next.js. Please use `next/image` instead.
`v13.0.0`	`next/image` renamed to `next/legacy/image`

Image

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/image
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *Optimize Images in your Next.js Application using the built-in `next/image` Component.*

Components

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *API Reference for Next.js built-in components in the Pages Router.*

Link

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/link
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *API reference for the `<Link>` component.*

Script

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/components/script
> 📁 **Sección:** Pages > Api Reference > Components
> 📝 **Descripción:** *Optimize third-party scripts in your Next.js application using the built-in `next/script` Component.*

File-system conventions

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/file-conventions
📁 **Sección:** Pages > Api Reference > File Conventions
📝 **Descripción:** *API Reference for Next.js file-system conventions.*

instrumentation.js

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/file-conventions/instrumentation
📁 **Sección:** Pages > Api Reference > File Conventions
📝 **Descripción:** *API reference for the instrumentation.js file.*

Proxy

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/file-conventions/proxy
📁 **Sección:** Pages > Api Reference > File Conventions
📝 **Descripción:** *Learn how to use Proxy to run code before a request is completed.*

public Folder

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/file-conventions/public-folder
📁 **Sección:** Pages > Api Reference > File Conventions
📝 **Descripción:** *Next.js allows you to serve static files, like images, in the public directory. You can learn how it works here.*

src Directory

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/file-conventions/src-folder
📁 **Sección:** Pages > Api Reference > File Conventions
📝 **Descripción:** *Save pages under the* `src` *folder as an alternative to the root* `pages` *directory.*

catchError

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/catchError
> 📂 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the catchError function.*

---

getInitialProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/get-initial-props
> 📂 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *Fetch dynamic data on the server for your React component with getInitialProps.*

> **Good to know**: `getInitialProps` is a legacy API. We recommend using `getStaticProps` or `getServerSideProps` instead.

`getInitialProps` is an `async` function that can be added to the default exported React component for the page. It will run on both the server-side and again on the client-side during page transitions. The result of the function will be forwarded to the React component as `props`.

```
// Archivo: pages/index.tsx switcher
import { NextPageContext } from 'next'

Page.getInitialProps = async (ctx: NextPageContext) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const json = await res.json()
  return { stars: json.stargazers_count }
}

export default function Page({ stars }: { stars: number }) {
  return stars
}
```

```
// Archivo: pages/index.js switcher
Page.getInitialProps = async (ctx) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const json = await res.json()
  return { stars: json.stargazers_count }
}

export default function Page({ stars }) {
  return stars
}
```

> **Good to know**:
>
> - Data returned from `getInitialProps` is serialized when server rendering. Ensure the returned object from `getInitialProps` is a plain `Object`, and not using `Date`, `Map` or `Set`.
> - For the initial page load, `getInitialProps` will run on the server only. `getInitialProps` will then also run on the client when navigating to a different route with the `next/link` component or by using `next/router`.
> - If `getInitialProps` is used in a custom `_app.js`, and the page being navigated to is using `getServerSideProps`, then `getInitialProps` will **only** run on the server.

Context Object

`getInitialProps` receives a single argument called `context`, which is an object with the following properties:

Name	Description
`pathname`	Current route, the path of the page in `/pages`
`query`	Query string of the URL, parsed as an object
`asPath`	`String` of the actual path (including the query) shown in the browser
`req`	HTTP request object (server only)
`res`	HTTP response object (server only)
`err`	Error object if any error is encountered during the rendering

Caveats

- `getInitialProps` can only be used in `pages/` top level files, and not in nested components. To have nested data fetching at the component level, consider exploring the App Router.
- Regardless of whether your route is static or dynamic, any data returned from `getInitialProps` as `props` will be able to be examined on the client-side in the initial HTML. This is to allow the page to be hydrated correctly. Make sure that you don't pass any sensitive information that shouldn't be available on the client in `props`.

---

getServerSideProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/get-server-side-props
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API reference for `getServerSideProps`. Learn how to fetch data on each request with Next.js.*

When exporting a function called `getServerSideProps` (Server-Side Rendering) from a page, Next.js will prerender this page on each request using the data returned by `getServerSideProps`. This is useful if you want to fetch data that changes often, and have the page update to show the most current data.

```tsx
// Archivo: pages/index.tsx switcher
import type { InferGetServerSidePropsType, GetServerSideProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getServerSideProps = (async () => {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo: Repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}) satisfies GetServerSideProps<{ repo: Repo }>

export default function Page({
  repo,
}: InferGetServerSidePropsType<typeof getServerSideProps>) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

```jsx
// Archivo: pages/index.js switcher
export async function getServerSideProps() {
  // Fetch data from external API
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  // Pass data to the page via props
  return { props: { repo } }
}

export default function Page({ repo }) {
  return (
    <main>
      <p>{repo.stargazers_count}</p>
    </main>
  )
}
```

You can import modules in top-level scope for use in `getServerSideProps`. Imports used will **not be bundled for the client-side**. This means you can write **server-side code directly in `getServerSideProps`**, including fetching data from your database.

Context parameter

The `context` parameter is an object containing the following keys:

Name	Description
`params`	If this page uses a dynamic route, `params` contains the route parameters. If the page name is `[id].js`, then `params` will look like `{ id: ... }`.
`req`	The `HTTP` IncomingMessage object, with an additional `cookies` prop, which is an object with string keys mapping to string values of cookies.
`res`	The `HTTP` response object.
`query`	An object representing the query string, including dynamic route parameters.
`preview`	(Deprecated for `draftMode`) `preview` is `true` if the page is in the Preview Mode and `false` otherwise.
`previewData`	(Deprecated for `draftMode`) The preview data set by `setPreviewData`.
`draftMode`	`draftMode` is `true` if the page is in the Draft Mode and `false` otherwise.
`resolvedUrl`	A normalized version of the request `URL` that strips the `_next/data` prefix for client transitions and includes original query values.
`locale`	Contains the active locale (if enabled).
`locales`	Contains all supported locales (if enabled).
`defaultLocale`	Contains the configured default locale (if enabled).

getServerSideProps return values

The `getServerSideProps` function should return an object with **any one of the following** properties:

`props`

The `props` object is a key-value pair, where each value is received by the page component. It should be a serializable object so that any props passed, could be serialized with `JSON.stringify`.

```
export async function getServerSideProps(context) {
  return {
    props: { message: `Next.js is awesome` }, // will be passed to the page component as props
  }
}
```

`notFound`

The `notFound` boolean allows the page to return a `404` status and 404 Page. With `notFound: true`, the page will return a `404` even if there was a successfully generated page before. This is meant to support use cases like user-generated content getting removed by its author.

```
export async function getServerSideProps(context) {
  const res = await fetch(`https:// ... /data`)
  const data = await res.json()

  if (!data) {
    return {
      notFound: true,
    }
  }

  return {
    props: { data }, // will be passed to the page component as props
  }
}
```

`redirect`

The `redirect` object allows redirecting to internal and external resources. It should match the shape of `{ destination: string, permanent: boolean }`. In some rare cases, you might need to assign a custom status code for older `HTTP` clients to properly redirect. In these cases, you can use the `statusCode` property instead of the `permanent` property, but not both.

```
export async function getServerSideProps(context) {
  const res = await fetch(`https:// ... /data`)
  const data = await res.json()

  if (!data) {
    return {
      redirect: {
        destination: '/',
        permanent: false,
      },
    }
  }

  return {
    props: {}, // will be passed to the page component as props
  }
}
```

Version History

Version	Changes
`v13.4.0`	App Router is now stable with simplified data fetching
`v10.0.0`	`locale`, `locales`, `defaultLocale`, and `notFound` options added.
`v9.3.0`	`getServerSideProps` introduced.

getStaticPaths

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/get-static-paths
> 📂 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API reference for `getStaticPaths`. Learn how to fetch data and generate static pages with `getStaticPaths`.*

When exporting a function called `getStaticPaths` from a page that uses Dynamic Routes, Next.js will statically prerender all the paths specified by `getStaticPaths`.

```tsx
// Archivo: pages/repo/[name].tsx switcher
import type {
  InferGetStaticPropsType,
  GetStaticProps,
  GetStaticPaths,
} from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticPaths = (async () => {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}) satisfies GetStaticPaths

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```jsx
// Archivo: pages/repo/[name].js switcher
export async function getStaticPaths() {
  return {
    paths: [
      {
        params: {
          name: 'next.js',
        },
      }, // See the "paths" section below
    ],
    fallback: true, // false or "blocking"
  }
}

export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

getStaticPaths return values

The `getStaticPaths` function should return an object with the following **required** properties:

`paths`

The `paths` key determines which paths will be prerendered. For example, suppose that you have a page that uses Dynamic Routes named `pages/posts/[id].js`. If you export `getStaticPaths` from this page and return the following for `paths`:

```
return {
  paths: [
    { params: { id: '1' }},
    {
      params: { id: '2' },
      // with i18n configured the locale for the path can be returned as well
      locale: "en",
    },
  ],
  fallback: ...
}
```

Then, Next.js will statically generate `/posts/1` and `/posts/2` during `next build` using the page component in `pages/posts/[id].js`.

The value for each `params` object must match the parameters used in the page name:

- If the page name is `pages/posts/[postId]/[commentId]`, then `params` should contain `postId` and `commentId`.
- If the page name uses catch-all routes like `pages/[...slug]`, then `params` should contain `slug` (which is an array). If this array is `['hello', 'world']`, then Next.js will statically generate the page at `/hello/world`.
- If the page uses an optional catch-all route, use `null`, `[]`, `undefined` or `false` to render the root-most route. For example, if you supply `slug: false` for `pages/[[...slug]]`, Next.js will statically generate the page `/`.

The `params` strings are **case-sensitive** and ideally should be normalized to ensure the paths are generated correctly. For example, if `WoRLD` is returned for a param it will only match if `WoRLD` is the actual path visited, not `world` or `World`.

Separate of the `params` object a `locale` field can be returned when i18n is configured, which configures the locale for the path being generated.

`fallback: false`

If `fallback` is `false`, then any paths not returned by `getStaticPaths` will result in a **404 page**.

When `next build` is run, Next.js will check if `getStaticPaths` returned `fallback: false`, it will then build **only** the paths returned by `getStaticPaths`. This option is useful if you have a small number of paths to create, or new page data is not added often. If you find that you need to add more paths, and you have `fallback: false`, you will need to run `next build` again so that the new paths can be generated.

The following example prerenders one blog post per page called `pages/posts/[id].js`. The list of blog posts will be fetched from a CMS and returned by `getStaticPaths`. Then, for each page, it fetches the post data from a CMS using `getStaticProps`.

```
// Archivo: pages/posts/[id].js
function Post({ post }) {
  // Render post ...
}

// This function gets called at build time
export async function getStaticPaths() {
  // Call an external API endpoint to get posts
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  // Get the paths we want to prerender based on posts
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }))

  // We'll prerender only these paths at build time.
  // { fallback: false } means other routes should 404.
  return { paths, fallback: false }
}

// This also gets called at build time
export async function getStaticProps({ params }) {
  // params contains the post `id`.
  // If the route is like /posts/1, then params.id is 1
  const res = await fetch(`https:// ... /posts/${params.id}`)
  const post = await res.json()

  // Pass post data to the page via props
  return { props: { post } }
}

export default Post
```

`fallback: true`

Examples

If `fallback` is `true`, then the behavior of `getStaticProps` changes in the following ways:

- The paths returned from `getStaticPaths` will be rendered to `HTML` at build time by `getStaticProps`.
- The paths that have not been generated at build time will **not** result in a 404 page. Instead, Next.js will serve a "fallback" version of the page on the first request to such a path. Web crawlers, such as Google, won't be served a fallback and instead the path will behave as in `fallback: 'blocking'`.
- When a page with `fallback: true` is navigated to through `next/link` or `next/router` (client-side) Next.js will *not* serve a fallback and instead the page will behave as `fallback: 'blocking'`.
- In the background, Next.js will statically generate the requested path `HTML` and `JSON`. This includes running `getStaticProps`.
- When complete, the browser receives the `JSON` for the generated path. This will be used to automatically render the page with the required props. From the user's perspective, the page will be swapped from the fallback page to the full page.
- At the same time, Next.js adds this path to the list of prerendered pages. Subsequent requests to the same path will serve the generated page, like other pages prerendered at build time.

> **Good to know**: `fallback: true` is not supported when using `output: 'export'`.

When is `fallback: true` useful?

`fallback: true` is useful if your app has a very large number of static pages that depend on data (such as a very large e-commerce site). If you want to prerender all product pages, the builds would take a very long time.

Instead, you may statically generate a small subset of pages and use `fallback: true` for the rest. When someone requests a page that is not generated yet, the user will see the page with a loading indicator or skeleton component.

Shortly after, `getStaticProps` finishes and the page will be rendered with the requested data. From now on, everyone who requests the same page will get the statically prerendered page.

This ensures that users always have a fast experience while preserving fast builds and the benefits of Static Generation.

`fallback: true` will not *update* generated pages, for that take a look at Incremental Static Regeneration.

`fallback: 'blocking'`

If `fallback` is `'blocking'`, new paths not returned by `getStaticPaths` will wait for the `HTML` to be generated, identical to SSR (hence why *blocking*), and then be cached for future requests so it only happens once per path.

`getStaticProps` will behave as follows:

- The paths returned from `getStaticPaths` will be rendered to `HTML` at build time by `getStaticProps`.
- The paths that have not been generated at build time will **not** result in a 404 page. Instead, Next.js will SSR on the first request and return the generated `HTML`.
- When complete, the browser receives the `HTML` for the generated path. From the user's perspective, it will transition from "the browser is requesting the page" to "the full page is loaded". There is no flash of loading/fallback state.
- At the same time, Next.js adds this path to the list of prerendered pages. Subsequent requests to the same path will serve the generated page, like other pages prerendered at build time.

`fallback: 'blocking'` will not *update* generated pages by default. To update generated pages, use Incremental Static Regeneration in conjunction with `fallback: 'blocking'`.

> **Good to know**: `fallback: 'blocking'` is not supported when using `output: 'export'`.

Fallback pages

In the "fallback" version of a page:

- The page's props will be empty.
- Using the router, you can detect if the fallback is being rendered, `router.isFallback` will be `true`.

The following example showcases using `isFallback`:

```
// Archivo: pages/posts/[id].js
import { useRouter } from 'next/router'

function Post({ post }) {
  const router = useRouter()

  // If the page is not yet generated, this will be displayed
  // initially until getStaticProps() finishes running
  if (router.isFallback) {
    return <div>Loading ... </div>
  }

  // Render post ...
}

// This function gets called at build time
export async function getStaticPaths() {
  return {
    // Only `/posts/1` and `/posts/2` are generated at build time
    paths: [{ params: { id: '1' } }, { params: { id: '2' } }],
    // Enable statically generating additional pages
    // For example: `/posts/3`
    fallback: true,
  }
}

// This also gets called at build time
export async function getStaticProps({ params }) {
  // params contains the post `id`.
  // If the route is like /posts/1, then params.id is 1
  const res = await fetch(`https:// ... /posts/${params.id}`)
  const post = await res.json()

  // Pass post data to the page via props
  return {
    props: { post },
    // Re-generate the post at most once per second
    // if a request comes in
    revalidate: 1,
  }
}

export default Post
```

Version History

Version	Changes
`v13.4.0`	App Router is now stable with simplified data fetching, including `generateStaticParams()`
`v12.2.0`	On-Demand Incremental Static Regeneration is stable.
`v12.1.0`	On-Demand Incremental Static Regeneration added (beta).
`v9.5.0`	Stable Incremental Static Regeneration
`v9.3.0`	`getStaticPaths` introduced.

getStaticProps

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/get-static-props
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API reference for `getStaticProps`. Learn how to use `getStaticProps` to generate static pages with Next.js.*

Exporting a function called `getStaticProps` will prerender a page at build time using the props returned from the function:

```
// Archivo: pages/index.tsx switcher
import type { InferGetStaticPropsType, GetStaticProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

```
// Archivo: pages/index.js switcher
export async function getStaticProps() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}

export default function Page({ repo }) {
  return repo.stargazers_count
}
```

You can import modules in top-level scope for use in `getStaticProps`. Imports used will **not be bundled for the client-side**. This means you can write **server-side code directly in** `getStaticProps`, including fetching data from your database.

Context parameter

The `context` parameter is an object containing the following keys:

Name	Description
`params`	Contains the route parameters for pages using dynamic routes. For example, if the page name is `[id].js`, then `params` will look like `{ id: ... }`. You should use this together with `getStaticPaths`, which we'll explain later.
`preview`	(Deprecated for `draftMode`) `preview` is `true` if the page is in the Preview Mode and `false` otherwise.
`previewData`	(Deprecated for `draftMode`) The preview data set by `setPreviewData`.
`draftMode`	`draftMode` is `true` if the page is in the Draft Mode and `false` otherwise.
`locale`	Contains the active locale (if enabled).
`locales`	Contains all supported locales (if enabled).
`defaultLocale`	Contains the configured default locale (if enabled).
`revalidateReason`	Provides a reason for why the function was called. Can be one of: "build" (run at build time), "stale" (revalidate period expired, or running in development mode), "on-demand" (triggered via on-demand revalidation)

getStaticProps return values

The `getStaticProps` function should return an object containing either `props`, `redirect`, or `notFound` followed by an **optional** `revalidate` property.

`props`

The `props` object is a key-value pair, where each value is received by the page component. It should be a serializable object so that any props passed, could be serialized with `JSON.stringify`.

```
export async function getStaticProps(context) {
  return {
    props: { message: `Next.js is awesome` }, // will be passed to the page component as props
  }
}
```

`revalidate`

The `revalidate` property is the amount in seconds after which a page re-generation can occur (defaults to `false` or no revalidation).

```
// This function gets called at build time on server-side.
// It may be called again, on a serverless function, if
// revalidation is enabled and a new request comes in
export async function getStaticProps() {
  const res = await fetch('https:// ... /posts')
  const posts = await res.json()

  return {
    props: {
      posts,
    },
    // Next.js will attempt to re-generate the page:
    // - When a request comes in
    // - At most once every 10 seconds
    revalidate: 10, // In seconds
  }
}
```

Learn more about Incremental Static Regeneration.

The cache status of a page leveraging ISR can be determined by reading the value of the `x-nextjs-cache` response header. The possible values are the following:

- `MISS` - the path is not in the cache (occurs at most once, on the first visit)
- `STALE` - the path is in the cache but exceeded the revalidate time so it will be updated in the background
- `HIT` - the path is in the cache and has not exceeded the revalidate time

`notFound`

The `notFound` boolean allows the page to return a `404` status and 404 Page. With `notFound: true`, the page will return a `404` even if there was a successfully generated page before. This is meant to support use cases like user-generated content getting removed by its author. Note, `notFound` follows the same `revalidate` behavior described here.

```
export async function getStaticProps(context) {
  const res = await fetch(`https:// ... /data`)
  const data = await res.json()

  if (!data) {
    return {
      notFound: true,
    }
  }

  return {
    props: { data }, // will be passed to the page component as props
  }
}
```

> **Good to know**: `notFound` is not needed for `fallback: false` mode as only paths returned from `getStaticPaths` will be prerendered.

`redirect`

The `redirect` object allows redirecting to internal or external resources. It should match the shape of `{ destination: string, permanent: boolean }`.

In some rare cases, you might need to assign a custom status code for older `HTTP` clients to properly redirect. In these cases, you can use the `statusCode` property instead of the `permanent` property, **but not both**. You can also set `basePath: false` similar to redirects in `next.config.js`.

```
export async function getStaticProps(context) {
  const res = await fetch(`https:// ... `)
  const data = await res.json()

  if (!data) {
    return {
      redirect: {
        destination: '/',
        permanent: false,
        // statusCode: 301
      },
    }
  }

  return {
    props: { data }, // will be passed to the page component as props
  }
}
```

If the redirects are known at build-time, they should be added in `next.config.js` instead.

Reading files: Use `process.cwd()`

Files can be read directly from the filesystem in `getStaticProps`.

In order to do so you have to get the full path to a file.

Since Next.js compiles your code into a separate directory you can't use `__dirname` as the path it returns will be different from the Pages Router.

Instead you can use `process.cwd()` which gives you the directory where Next.js is being executed.

```
import { promises as fs } from 'fs'
import path from 'path'

// posts will be populated at build time by getStaticProps()
function Blog({ posts }) {
  return (
    <ul>
      {posts.map((post) => (
        <li>
          <h3>{post.filename}</h3>
          <p>{post.content}</p>
        </li>
      ))}
    </ul>
  )
}

// This function gets called at build time on server-side.
// It won't be called on client-side, so you can even do
// direct database queries.
export async function getStaticProps() {
  const postsDirectory = path.join(process.cwd(), 'posts')
  const filenames = await fs.readdir(postsDirectory)

  const posts = filenames.map(async (filename) => {
    const filePath = path.join(postsDirectory, filename)
    const fileContents = await fs.readFile(filePath, 'utf8')

    // Generally you would parse/transform the contents
    // For example you can transform markdown to HTML here

    return {
      filename,
      content: fileContents,
    }
  })
  // By returning { props: { posts } }, the Blog component
  // will receive `posts` as a prop at build time
  return {
    props: {
      posts: await Promise.all(posts),
    },
  }
}

export default Blog
```

Version History

Version	Changes
`v13.4.0`	App Router is now stable with simplified data fetching
`v12.2.0`	On-Demand Incremental Static Regeneration is stable.
`v12.1.0`	On-Demand Incremental Static Regeneration added (beta).
`v10.0.0`	`locale`, `locales`, `defaultLocale`, and `notFound` options added.
`v10.0.0`	`fallback: 'blocking'` return option added.
`v9.5.0`	Stable Incremental Static Regeneration
`v9.3.0`	`getStaticProps` introduced.

Functions

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for Functions and Hooks in Pages Router.*

NextRequest

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/next-request
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for NextRequest.*

NextResponse

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/next-response
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for NextResponse.*

---

useParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/use-params
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useParams hook in the Pages Router.*

`useParams` is a hook that lets you read a route's dynamic params filled in by the current URL.

```
// Archivo: pages/shop/[slug].tsx switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams<{ slug: string }>()

  if (!params) {
    // Render fallback UI while params are not yet available
    return null
  }

  // Route → /shop/[slug]
  // URL → /shop/shoes
  // `params` → { slug: 'shoes' }
  return <>Shop: {params.slug}</>
}
```

```
// Archivo: pages/shop/[slug].js switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams()

  if (!params) {
    // Render fallback UI while params are not yet available
    return null
  }

  // Route → /shop/[slug]
  // URL → /shop/shoes
  // `params` → { slug: 'shoes' }
  return <>Shop: {params.slug}</>
}
```

Parameters

```
const params = useParams()
```

`useParams` does not take any parameters.

Returns

`useParams` returns an object containing the current route's filled in dynamic parameters, or `null` during prerendering.

- Each property in the object is an active dynamic segment.
- The property name is the segment's name, and the property value is what the segment is filled in with.
- The property value will either be a `string` or array of `string`s depending on the type of dynamic segment.
- If the route contains no dynamic parameters, `useParams` returns an empty object.

For example:

Route	URL	`useParams()`
`pages/shop/page.js`	`/shop`	`{}`
`pages/shop/[slug].js`	`/shop/1`	`{ slug: '1' }`
`pages/shop/[tag]/[item].js`	`/shop/1/2`	`{ tag: '1', item: '2' }`
`pages/shop/[...slug].js`	`/shop/1/2`	`{ slug: ['1', '2'] }`

> **Good to know**: `useParams` is a React Hook and cannot be used with classes.

Behavior

Behavior during prerendering

For pages that are statically optimized, `useParams` will return `null` on the initial render. After hydration, the value will be updated to the actual params once the router is ready.

This is because params cannot be known during static generation for dynamic routes.

```
// Archivo: pages/shop/[slug].tsx switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams<{ slug: string }>()

  if (!params) {
    // Return a fallback UI while params are loading
    // This prevents hydration mismatches
    return <ShopPageSkeleton />
  }

  return <>Shop: {params.slug}</>
}
```

```
// Archivo: pages/shop/[slug].js switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams()

  if (!params) {
    // Return a fallback UI while params are loading
    // This prevents hydration mismatches
    return <ShopPageSkeleton />
  }

  return <>Shop: {params.slug}</>
}
```

Using with `getServerSideProps`

When using `getServerSideProps`, the page is server-rendered on each request and `useParams` will return the actual params immediately:

```
// Archivo: pages/shop/[slug].tsx switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams<{ slug: string }>()

  // With getServerSideProps, this fallback is never rendered because
  // params is always available on the server. However, keeping
  // the fallback allows this component to be reused on other pages
  // that may not use getServerSideProps.
  if (!params) {
    return null
  }

  return <>Shop: {params.slug}</>
}

export async function getServerSideProps() {
  return { props: {} }
}
```

```
// Archivo: pages/shop/[slug].js switcher
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const params = useParams()

  // With getServerSideProps, this fallback is never rendered because
  // params is always available on the server. However, keeping
  // the fallback allows this component to be reused on other pages
  // that may not use getServerSideProps.
  if (!params) {
    return null
  }

  return <>Shop: {params.slug}</>
}

export async function getServerSideProps() {
  return { props: {} }
}
```

Comparison with `router.query`

`useParams` only returns the dynamic route parameters, whereas `router.query` from `useRouter` includes both dynamic parameters and query string parameters.

```
// Archivo: pages/shop/[slug].tsx switcher
import { useRouter } from 'next/router'
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const router = useRouter()
  const params = useParams()

  // URL → /shop/shoes?color=red

  // router.query → { slug: 'shoes', color: 'red' }
  // params → { slug: 'shoes' }

  // ...
}
```

```
// Archivo: pages/shop/[slug].js switcher
import { useRouter } from 'next/router'
import { useParams } from 'next/navigation'

export default function ShopPage() {
  const router = useRouter()
  const params = useParams()

  // URL → /shop/shoes?color=red

  // router.query → { slug: 'shoes', color: 'red' }
  // params → { slug: 'shoes' }

  // ...
}
```

Examples

Sharing components with App Router

`useParams` from `next/navigation` works in both the Pages Router and App Router. This allows you to create shared components that work in either context:

```
// Archivo: components/breadcrumb.tsx switcher
import { useParams } from 'next/navigation'

// This component works in both pages/ and app/
export function Breadcrumb() {
  const params = useParams<{ slug: string }>()

  if (!params) {
    // Fallback for Pages Router during prerendering
    return <nav>Home / ... </nav>
  }

  return <nav>Home / {params.slug}</nav>
}
```

```
// Archivo: components/breadcrumb.js switcher
import { useParams } from 'next/navigation'

// This component works in both pages/ and app/
export function Breadcrumb() {
  const params = useParams()

  if (!params) {
    // Fallback for Pages Router during prerendering
    return <nav>Home / ... </nav>
  }

  return <nav>Home / {params.slug}</nav>
}
```

> **Good to know**: When using this component in the App Router, `useParams` never returns `null`, so the fallback branch will not be rendered.

Version History

Version	Changes
`v13.3.0`	`useParams` introduced.

useReportWebVitals

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/use-report-web-vitals
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *Use the Next.js `useReportWebVitals` hook to send Core Web Vitals and custom performance metrics to analytics.*

useRouter

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/use-router
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *Learn more about the API of the Next.js Router, and access the router instance in your page with the useRouter hook.*

If you want to access the `router` object inside any function component in your app, you can use the `useRouter` hook, take a look at the following example:

```
import { useRouter } from 'next/router'

function ActiveLink({ children, href }) {
  const router = useRouter()
  const style = {
    marginRight: 10,
    color: router.asPath === href ? 'red' : 'black',
  }

  const handleClick = (e) => {
    e.preventDefault()
    router.push(href)
  }

  return (
    <a href={href} onClick={handleClick} style={style}>
      {children}
    </a>
  )
}

export default ActiveLink
```

> `useRouter` is a React Hook, meaning it cannot be used with classes. You can either use withRouter or wrap your class in a function component.

`router` object

The following is the definition of the `router` object returned by both `useRouter` and `withRouter`:

- `pathname` : `String` - The path for current route file that comes after `/pages`. Therefore, `basePath`, `locale` and trailing slash (`trailingSlash: true`) are not included.
- `query` : `Object` - The query string parsed to an object, including dynamic route parameters. It will be an empty object during prerendering if the page doesn't use Server-side Rendering. Defaults to `{}`
- `asPath` : `String` - The path as shown in the browser including the search params and respecting the `trailingSlash` configuration. `basePath` and `locale` are not included.
- `isFallback` : `boolean` - Whether the current page is in fallback mode.
- `basePath` : `String` - The active basePath (if enabled).
- `locale` : `String` - The active locale (if enabled).
- `locales` : `String[]` - All supported locales (if enabled).
- `defaultLocale` : `String` - The current default locale (if enabled).
- `domainLocales` : `Array<{domain, defaultLocale, locales}>` - Any configured domain locales.
- `isReady` : `boolean` - Whether the router fields are updated client-side and ready for use. Should only be used inside of `useEffect` methods and not for conditionally rendering on the server. See related docs for use case with automatically statically optimized pages
- `isPreview` : `boolean` - Whether the application is currently in preview mode.

> Using the `asPath` field may lead to a mismatch between client and server if the page is rendered using server-side rendering or automatic static optimization. Avoid using `asPath` until the `isReady` field is `true`.

The following methods are included inside `router`:

router.push

Handles client-side transitions, this method is useful for cases where `next/link` is not enough.

```
router.push(url, as, options)
```

- `url`: `UrlObject | String` - The URL to navigate to (see Node.JS URL module documentation for `UrlObject` properties).
- `as`: `UrlObject | String` - Optional decorator for the path that will be shown in the browser URL bar. Before Next.js 9.5.3 this was used for dynamic routes.
- `options` - Optional object with the following configuration options:
  - `scroll` - Optional boolean, controls scrolling to the top of the page after navigation. Defaults to `true`
  - `shallow`: Update the path of the current page without rerunning `getStaticProps`, `getServerSideProps` or `getInitialProps`. Defaults to `false`
  - `locale` - Optional string, indicates locale of the new page

> You don't need to use `router.push` for external URLs. window.location is better suited for those cases.

Navigating to `pages/about.js`, which is a predefined route:

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/about')}>
      Click me
    </button>
  )
}
```

Navigating `pages/post/[pid].js`, which is a dynamic route:

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/post/abc')}>
      Click me
    </button>
  )
}
```

Redirecting the user to `pages/login.js`, useful for pages behind authentication:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

// Here you would fetch and return the user
const useUser = () => ({ user: null, loading: false })

export default function Page() {
  const { user, loading } = useUser()
  const router = useRouter()

  useEffect(() => {
    if (!(user || loading)) {
      router.push('/login')
    }
  }, [user, loading])

  return <p>Redirecting ... </p>
}
```

Resetting state after navigation

When navigating to the same page in Next.js, the page's state **will not** be reset by default as React does not unmount unless the parent component has changed.

```
// Archivo: pages/[slug].js
import Link from 'next/link'
import { useState } from 'react'
import { useRouter } from 'next/router'

export default function Page(props) {
  const router = useRouter()
  const [count, setCount] = useState(0)
  return (
    <div>
      <h1>Page: {router.query.slug}</h1>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase count</button>
      [one](/one) [two](/two)
    </div>
  )
}
```

In the above example, navigating between `/one` and `/two` **will not** reset the count . The `useState` is maintained between renders because the top-level React component, `Page`, is the same.

If you do not want this behavior, you have a couple of options:

- Manually ensure each state is updated using `useEffect`. In the above example, that could look like:

  ```
  useEffect(() => {
    setCount(0)
  }, [router.query.slug])
  ```

- Use a React `key` to tell React to remount the component. To do this for all pages, you can use a custom app:

  ```
  import { useRouter } from 'next/router'

  export default function MyApp({ Component, pageProps }) {
    const router = useRouter()
    return <Component key={router.asPath} {...pageProps} />
  }
  ```

With URL object

You can use a URL object in the same way you can use it for `next/link`. Works for both the `url` and `as` parameters:

```
import { useRouter } from 'next/router'

export default function ReadMore({ post }) {
  const router = useRouter()

  return (
    <button
      type="button"
      onClick={() => {
        router.push({
          pathname: '/post/[pid]',
          query: { pid: post.id },
        })
      }}
    >
      Click here to read more
    </button>
  )
}
```

If you omit `pathname`, the query is applied to the URL shown in the browser (`asPath`) rather than to the current page's route (`pathname`). This keeps rewrites intact: on a page served by a rewrite, a query-only navigation preserves the URL the visitor requested instead of revealing the rewrite destination.

The two values also differ when you mask the URL yourself with the `as` parameter. Pass `pathname` explicitly in that case, otherwise the query is applied to the masked URL and can resolve to a different page:

```
// On the `/` page, with the URL masked as `/p/1`
router.push({ pathname: '/', query: { photoId: 2 } }, '/p/2', { shallow: true })
```

router.replace

Similar to the `replace` prop in `next/link`, `router.replace` will prevent adding a new URL entry into the `history` stack.

```
router.replace(url, as, options)
```

- The API for `router.replace` is exactly the same as the API for `router.push`.

Take a look at the following example:

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.replace('/home')}>
      Click me
    </button>
  )
}
```

router.prefetch

Prefetch pages for faster client-side transitions. This method is only useful for navigations without `next/link`, as `next/link` takes care of prefetching pages automatically.

> This is a production only feature. Next.js doesn't prefetch pages in development.

```
router.prefetch(url, as, options)
```

- `url` - The URL to prefetch, including explicit routes (e.g. `/dashboard`) and dynamic routes (e.g. `/product/[id]`)
- `as` - Optional decorator for `url`. Before Next.js 9.5.3 this was used to prefetch dynamic routes.
- `options` - Optional object with the following allowed fields:
  - `locale` - allows providing a different locale from the active one. If `false`, `url` has to include the locale as the active locale won't be used.

Let's say you have a login page, and after a login, you redirect the user to the dashboard. For that case, we can prefetch the dashboard to make a faster transition, like in the following example:

```
import { useCallback, useEffect } from 'react'
import { useRouter } from 'next/router'

export default function Login() {
  const router = useRouter()
  const handleSubmit = useCallback((e) => {
    e.preventDefault()

    fetch('/api/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(),
    }).then((res) => {
      // Do a fast client-side transition to the already prefetched dashboard page
      if (res.ok) router.push('/dashboard')
    })
  }, [])

  useEffect(() => {
    // Prefetch the dashboard page
    router.prefetch('/dashboard')
  }, [router])

  return (
    <form onSubmit={handleSubmit}>

      <button type="submit">Login</button>
    </form>
  )
}
```

router.beforePopState

In some cases (for example, if using a Custom Server), you may wish to listen to popstate and do something before the router acts on it.

```
router.beforePopState(cb)
```

- `cb` - The function to run on incoming `popstate` events. The function receives the state of the event as an object with the following props:
  - `url`: `String` - the route for the new state. This is usually the name of a `page`
  - `as`: `String` - the url that will be shown in the browser
  - `options`: `Object` - Additional options sent by router.push

If `cb` returns `false`, the Next.js router will not handle `popstate`, and you'll be responsible for handling it in that case. See Disabling file-system routing.

You could use `beforePopState` to manipulate the request, or force a SSR refresh, as in the following example:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  useEffect(() => {
    router.beforePopState(({ url, as, options }) => {
      // I only want to allow these two routes!
      if (as !== '/' && as !== '/other') {
        // Have SSR render bad routes as a 404.
        window.location.href = as
        return false
      }

      return true
    })
  }, [router])

  return <p>Welcome to the page</p>
}
```

router.back

Navigate back in history. Equivalent to clicking the browser's back button. It executes `window.history.back()`.

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.back()}>
      Click here to go back
    </button>
  )
}
```

router.reload

Reload the current URL. Equivalent to clicking the browser's refresh button. It executes `window.location.reload()`.

```
import { useRouter } from 'next/router'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.reload()}>
      Click here to reload
    </button>
  )
}
```

router.events

You can listen to different events happening inside the Next.js Router. Here's a list of supported events:

- `routeChangeStart(url, { shallow })` - Fires when a route starts to change
- `routeChangeComplete(url, { shallow })` - Fires when a route changed completely
- `routeChangeError(err, url, { shallow })` - Fires when there's an error when changing routes, or a route load is cancelled
  - `err.cancelled` - Indicates if the navigation was cancelled
- `beforeHistoryChange(url, { shallow })` - Fires before changing the browser's history
- `hashChangeStart(url, { shallow })` - Fires when the hash will change but not the page

- `hashChangeComplete(url, { shallow })` - Fires when the hash has changed but not the page

> **Good to know**: Here `url` is the URL shown in the browser, including the `basePath`.

For example, to listen to the router event `routeChangeStart`, open or create `pages/_app.js` and subscribe to the event, like so:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

export default function MyApp({ Component, pageProps }) {
  const router = useRouter()

  useEffect(() => {
    const handleRouteChange = (url, { shallow }) => {
      console.log(
        `App is changing to ${url} ${
          shallow ? 'with' : 'without'
        } shallow routing`
      )
    }

    router.events.on('routeChangeStart', handleRouteChange)

    // If the component is unmounted, unsubscribe
    // from the event with the `off` method:
    return () => {
      router.events.off('routeChangeStart', handleRouteChange)
    }
  }, [router])

  return <Component {...pageProps} />
}
```

> We use a Custom App (`pages/_app.js`) for this example to subscribe to the event because it's not unmounted on page navigations, but you can subscribe to router events on any component in your application.

Router events should be registered when a component mounts (useEffect or componentDidMount / componentWillUnmount) or imperatively when an event happens.

If a route load is cancelled (for example, by clicking two links rapidly in succession), `routeChangeError` will fire. And the passed `err` will contain a `cancelled` property set to `true`, as in the following example:

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

export default function MyApp({ Component, pageProps }) {
  const router = useRouter()

  useEffect(() => {
    const handleRouteChangeError = (err, url) => {
      if (err.cancelled) {
        console.log(`Route to ${url} was cancelled!`)
      }
    }

    router.events.on('routeChangeError', handleRouteChangeError)

    // If the component is unmounted, unsubscribe
    // from the event with the `off` method:
    return () => {
      router.events.off('routeChangeError', handleRouteChangeError)
    }
  }, [router])

  return <Component {...pageProps} />
}
```

The `next/compat/router` export

This is the same `useRouter` hook, but can be used in both `app` and `pages` directories.

It differs from `next/router` in that it does not throw an error when the pages router is not mounted, and instead has a return type of `NextRouter | null`. This allows developers to convert components to support running in both `app` and `pages` as they transition to the `app` router.

A component that previously looked like this:

```
import { useRouter } from 'next/router'
const MyComponent = () => {
  const { isReady, query } = useRouter()
  // ...
}
```

Will error when converted over to `next/compat/router`, as `null` cannot be destructured. Instead, developers will be able to take advantage of new hooks:

```
import { useEffect } from 'react'
import { useRouter } from 'next/compat/router'
import { useSearchParams } from 'next/navigation'
const MyComponent = () => {
  const router = useRouter() // may be null or a NextRouter instance
  const searchParams = useSearchParams()
  useEffect(() => {
    if (router && !router.isReady) {
      return
    }
    // In `app/`, searchParams will be ready immediately with the values, in
    // `pages/` it will be available after the router is ready.
    const search = searchParams.get('search')
    // ...
  }, [router, searchParams])
  // ...
}
```

This component will now work in both `pages` and `app` directories. When the component is no longer used in `pages`, you can remove the references to the compat router:

```
import { useSearchParams } from 'next/navigation'
const MyComponent = () => {
  const searchParams = useSearchParams()
  // As this component is only used in `app/`, the compat router can be removed.
  const search = searchParams.get('search')
  // ...
}
```

Using `useRouter` outside of Next.js context in pages

Another specific use case is when rendering components outside of a Next.js application context, such as inside `getServerSideProps` on the `pages` directory. In this case, the compat router can be used to avoid errors:

```
import { renderToString } from 'react-dom/server'
import { useRouter } from 'next/compat/router'
const MyComponent = () => {
  const router = useRouter() // may be null or a NextRouter instance
  // ...
}
export async function getServerSideProps() {
  const renderedComponent = renderToString(<MyComponent />)
  return {
    props: {
      renderedComponent,
    },
  }
}
```

Potential ESLint errors

Certain methods accessible on the `router` object return a Promise. If you have the ESLint rule, no-floating-promises enabled, consider disabling it either globally, or for the affected line.

If your application needs this rule, you should either `void` the promise – or use an `async` function, `await` the Promise, then void the function call. **This is not applicable when the method is called from inside an `onClick` handler**.

The affected methods are:

- `router.push`
- `router.replace`
- `router.prefetch`

Potential solutions

```
import { useEffect } from 'react'
import { useRouter } from 'next/router'

// Here you would fetch and return the user
const useUser = () => ({ user: null, loading: false })

export default function Page() {
  const { user, loading } = useUser()
  const router = useRouter()

  useEffect(() => {
    // disable the linting on the next line - This is the cleanest solution
    // eslint-disable-next-line no-floating-promises
    router.push('/login')

    // void the Promise returned by router.push
    if (!(user || loading)) {
      void router.push('/login')
    }
    // or use an async function, await the Promise, then void the function call
    async function handleRouteChange() {
      if (!(user || loading)) {
        await router.push('/login')
      }
    }
    void handleRouteChange()
  }, [user, loading])

  return <p>Redirecting ... </p>
}
```

withRouter

If `useRouter` is not the best fit for you, `withRouter` can also add the same `router` object to any component.

Usage

```
import { withRouter } from 'next/router'

function Page({ router }) {
  return <p>{router.pathname}</p>
}

export default withRouter(Page)
```

TypeScript

To use class components with `withRouter`, the component needs to accept a router prop:

```
import React from 'react'
import { withRouter, NextRouter } from 'next/router'

interface WithRouterProps {
  router: NextRouter
}

interface MyComponentProps extends WithRouterProps {}

class MyComponent extends React.Component<MyComponentProps> {
  render() {
    return <p>{this.props.router.pathname}</p>
  }
}

export default withRouter(MyComponent)
```

---

useSearchParams

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/use-search-params
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *API Reference for the useSearchParams hook in the Pages Router.*

`useSearchParams` is a hook that lets you read the current URL's **query string**.

`useSearchParams` returns a **read-only** version of the `URLSearchParams` interface.

```
// Archivo: pages/dashboard.tsx switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Render fallback UI while search params are not yet available
    return null
  }

  const search = searchParams.get('search')

  // URL -> `/dashboard?search=my-project`
  // `search` -> 'my-project'
  return <>Search: {search}</>
}
```

```
// Archivo: pages/dashboard.js switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Render fallback UI while search params are not yet available
    return null
  }

  const search = searchParams.get('search')

  // URL -> `/dashboard?search=my-project`
  // `search` -> 'my-project'
  return <>Search: {search}</>
}
```

Parameters

```
const searchParams = useSearchParams()
```

`useSearchParams` does not take any parameters.

Returns

`useSearchParams` returns a **read-only** version of the `URLSearchParams` interface, or `null` during prerendering.

The interface includes utility methods for reading the URL's query string:

- `URLSearchParams.get()`: Returns the first value associated with the search parameter. For example:

URL	`searchParams.get("a")`
`/dashboard?a=1`	`'1'`
`/dashboard?a=`	`''`
`/dashboard?b=3`	`null`
`/dashboard?a=1&a=2`	`'1'` - use *`getAll()`* to get all values

- `URLSearchParams.has()`: Returns a boolean value indicating if the given parameter exists. For example:

URL	`searchParams.has("a")`
`/dashboard?a=1`	`true`
`/dashboard?b=3`	`false`

- Learn more about other **read-only** methods of `URLSearchParams`, including the `getAll()`, `keys()`, `values()`, `entries()`, `forEach()`, and `toString()`.

> **Good to know**: `useSearchParams` is a React Hook and cannot be used with classes.

Behavior

Behavior during prerendering

For pages that are statically optimized (not using `getServerSideProps`), `useSearchParams` will return `null` during prerendering. After hydration, the value will be updated to the actual search params.

This is because search params cannot be known during static generation as they depend on the request.

```
// Archivo: pages/dashboard.tsx switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Return a fallback UI while search params are loading
    // This prevents hydration mismatches
    return <DashboardSkeleton />
  }

  const search = searchParams.get('search')

  return <>Search: {search}</>
}
```

```
// Archivo: pages/dashboard.js switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Return a fallback UI while search params are loading
    // This prevents hydration mismatches
    return <DashboardSkeleton />
  }

  const search = searchParams.get('search')

  return <>Search: {search}</>
}
```

Using with `getServerSideProps`

When using `getServerSideProps`, the page is server-rendered on each request and `useSearchParams` will return the actual search params immediately:

```
// Archivo: pages/dashboard.tsx switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  // With getServerSideProps, this fallback is never rendered because
  // searchParams is always available on the server. However, keeping
  // the fallback allows this component to be reused on other pages
  // that may not use getServerSideProps.
  if (!searchParams) {
    return null
  }

  const search = searchParams.get('search')

  return <>Search: {search}</>
}

export async function getServerSideProps() {
  return { props: {} }
}
```

```
// Archivo: pages/dashboard.js switcher
import { useSearchParams } from 'next/navigation'

export default function Dashboard() {
  const searchParams = useSearchParams()

  // With getServerSideProps, this fallback is never rendered because
  // searchParams is always available on the server. However, keeping
  // the fallback allows this component to be reused on other pages
  // that may not use getServerSideProps.
  if (!searchParams) {
    return null
  }

  const search = searchParams.get('search')

  return <>Search: {search}</>
}

export async function getServerSideProps() {
  return { props: {} }
}
```

Examples

Updating search params

You can use the `useRouter` hook to update search params:

```tsx
// Archivo: pages/dashboard.tsx switcher
import { useRouter } from 'next/router'
import { useSearchParams } from 'next/navigation'
import { useCallback } from 'react'

export default function Dashboard() {
  const router = useRouter()
  const searchParams = useSearchParams()

  const createQueryString = useCallback(
    (name: string, value: string) => {
      const params = new URLSearchParams(searchParams?.toString())
      params.set(name, value)
      return params.toString()
    },
    [searchParams]
  )

  if (!searchParams) {
    return null
  }

  return (
    <>
      <p>Sort By</p>
      <button
        onClick={() => {
          router.push(router.pathname + '?' + createQueryString('sort', 'asc'))
        }}
      >
        ASC
      </button>
      <button
        onClick={() => {
          router.push(router.pathname + '?' + createQueryString('sort', 'desc'))
        }}
      >
        DESC
      </button>
    </>
  )
}
```

```
// Archivo: pages/dashboard.js switcher
import { useRouter } from 'next/router'
import { useSearchParams } from 'next/navigation'
import { useCallback } from 'react'

export default function Dashboard() {
  const router = useRouter()
  const searchParams = useSearchParams()

  const createQueryString = useCallback(
    (name, value) => {
      const params = new URLSearchParams(searchParams?.toString())
      params.set(name, value)
      return params.toString()
    },
    [searchParams]
  )

  if (!searchParams) {
    return null
  }

  return (
    <>
      <p>Sort By</p>
      <button
        onClick={() => {
          router.push(router.pathname + '?' + createQueryString('sort', 'asc'))
        }}
      >
        ASC
      </button>
      <button
        onClick={() => {
          router.push(router.pathname + '?' + createQueryString('sort', 'desc'))
        }}
      >
        DESC
      </button>
    </>
  )
}
```

Sharing components with App Router

`useSearchParams` from `next/navigation` works in both the Pages Router and App Router. This allows you to create shared components that work in either context:

```
// Archivo: components/search-bar.tsx switcher
import { useSearchParams } from 'next/navigation'

// This component works in both pages/ and app/
export function SearchBar() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Fallback for Pages Router during prerendering
    return <input defaultValue="" placeholder="Search..." />
  }

  const search = searchParams.get('search') ?? ''

  return <input defaultValue={search} placeholder="Search..." />
}
```

```
// Archivo: components/search-bar.js switcher
import { useSearchParams } from 'next/navigation'

// This component works in both pages/ and app/
export function SearchBar() {
  const searchParams = useSearchParams()

  if (!searchParams) {
    // Fallback for Pages Router during prerendering
    return <input defaultValue="" placeholder="Search..." />
  }

  const search = searchParams.get('search') ?? ''

  return <input defaultValue={search} placeholder="Search..." />
}
```

> **Good to know**: When using this component in the App Router, wrap it in a `<Suspense>` boundary for prerendering support.

Version History

Version	Changes
`v13.0.0`	`useSearchParams` introduced.

userAgent

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/functions/userAgent
> 📁 **Sección:** Pages > Api Reference > Functions
> 📝 **Descripción:** *The userAgent helper extends the Web Request API with additional properties and methods to interact with the user agent object from the request.*

adapterPath

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/adapterPath
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure a custom adapter for Next.js to hook into the build process.*

allowedDevOrigins

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/allowedDevOrigins
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use `allowedDevOrigins` to configure additional origins that can request the dev server.*

assetPrefix

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/assetPrefix
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to use the assetPrefix config option to configure your CDN.*

basePath

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/basePath
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use `basePath` to deploy a Next.js application under a sub-path of a domain.*

bundlePagesRouterDependencies

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/bundlePagesRouterDependencies
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Enable automatic dependency bundling for Pages Router*

Enable automatic server-side dependency bundling for Pages Router applications. Matches the automatic dependency bundling in App Router.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  bundlePagesRouterDependencies: true,
}

module.exports = nextConfig
```

Explicitly opt-out certain packages from being bundled using the `serverExternalPackages` option.

Version History

Version	Changes
`v15.0.0`	Moved from experimental to stable. Renamed from `bundlePagesExternals` to `bundlePagesRouterDependencies`

compress

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/compress
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Next.js provides gzip compression to compress rendered content and static files, it only works with the server target. Learn more about it here.*

---

crossOrigin

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/crossOrigin
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Use the `crossOrigin` option to add a crossOrigin tag on the `script` tags generated by `next/script` and `next/head`.*

---

deploymentId

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/deploymentId
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure a deployment identifier used for version skew protection and cache busting.*

---

devIndicators

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/devIndicators
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Optimized pages include an indicator to let you know if it's being statically optimized. You can opt-out of it here.*

---

distDir

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/distDir
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Set a custom build directory to use instead of the default .next directory.*

---

env

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/env
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Learn to add and access environment variables in your Next.js application at build time.*

exportPathMap

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/exportPathMap
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Customize the pages that will be exported as HTML files when using* `next export` .

generateBuildId

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/generateBuildId
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure the build id, which is used to identify the current build in which your application is being served.*

generateEtags

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/generateEtags
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Next.js will generate etags for every page by default. Learn more about how to disable etag generation here.*

headers

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/headers
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Add custom HTTP headers to your Next.js app.*

httpAgentOptions

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/httpAgentOptions
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Next.js will automatically use HTTP Keep-Alive by default. Learn more about how to disable HTTP Keep-Alive here.*

images

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/images
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Custom configuration for the next/image loader*

next.config.js Options

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Learn about the options available in next.config.js for the Pages Router.*

logging

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/logging
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure logging behavior in the terminal when running Next.js in development mode.*

onDemandEntries

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/onDemandEntries
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure how Next.js will dispose and keep in memory pages created in development.*

optimizePackageImports

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/optimizePackageImports
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *API Reference for optimizePackageImports Next.js Config Option*

output

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/output
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Next.js automatically traces which files are needed by each page to allow for easy deployment of your application. Learn how it works here.*

pageExtensions

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/pageExtensions
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Extend the default page extensions used by Next.js when resolving pages in the Pages Router.*

poweredByHeader

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/poweredByHeader
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Next.js will add the* `x-powered-by` *header by default. Learn to opt-out of it here.*

productionBrowserSourceMaps

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/productionBrowserSourceMaps
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Enables browser source map generation during the production build.*

experimental.proxyClientMaxBodySize

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/proxyClientMaxBodySize
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Configure the maximum request body size when using proxy.*

---

reactStrictMode

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/reactStrictMode
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *The complete Next.js runtime is now Strict Mode-compliant, learn how to opt-in*

---

redirects

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/redirects
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Add redirects to your Next.js app.*

---

rewrites

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/rewrites
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Add rewrites to your Next.js app.*

---

serverExternalPackages

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/serverExternalPackages
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Opt-out specific dependencies from the dependency bundling enabled by* `bundlePagesRouterDependencies`.

Opt-out specific dependencies from being included in the automatic bundling of the `bundlePagesRouterDependencies` option.

These pages will then use native Node.js `require` to resolve the dependency.

```
// Archivo: next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  serverExternalPackages: ['@acme/ui'],
}

module.exports = nextConfig
```

Next.js includes a short list of popular packages that currently are working on compatibility and automatically opt-ed out:

- `@alinea/generated`
- `@appsignal/nodejs`
- `@aws-sdk/client-s3`
- `@aws-sdk/s3-presigned-post`
- `@blockfrost/blockfrost-js`
- `@highlight-run/node`
- `@huggingface/transformers`
- `@jpg-store/lucid-cardano`
- `@libsql/client`
- `@mikro-orm/core`
- `@mikro-orm/knex`
- `@node-rs/argon2`
- `@node-rs/bcrypt`
- `@prisma/client`
- `@react-pdf/renderer`
- `@sentry/profiling-node`
- `@sparticuz/chromium`
- `@sparticuz/chromium-min`
- `@statsig/statsig-node-core`
- `@swc/core`
- `@xenova/transformers`
- `@zenstackhq/runtime`
- `argon2`
- `autoprefixer`
- `aws-crt`
- `bcrypt`
- `better-sqlite3`
- `canvas`
- `chromadb-default-embed`
- `config`
- `cpu-features`
- `cypress`
- `dd-trace`
- `eslint`
- `express`
- `firebase-admin`
- `htmlrewriter`
- `import-in-the-middle`
- `isolated-vm`

- `jest`
- `jsdom`
- `keyv`
- `libsql`
- `mdx-bundler`
- `mongodb`
- `mongoose`
- `newrelic`
- `next-mdx-remote`
- `next-seo`
- `node-cron`
- `node-pty`
- `node-web-audio-api`
- `onnxruntime-node`
- `oslo`
- `pg`
- `pino`
- `pino-pretty`
- `pino-roll`
- `playwright`
- `playwright-core`
- `postcss`
- `prettier`
- `prisma`
- `puppeteer-core`
- `puppeteer`
- `ravendb`
- `require-in-the-middle`
- `rimraf`
- `sharp`
- `shiki`
- `sqlite3`
- `thread-stream`
- `ts-morph`
- `ts-node`
- `typescript`
- `vscode-oniguruma`
- `webpack`
- `websocket`
- `zeromq`

---

skipProxyUrlNormalize

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/skipProxyUrlNormalize
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Let Proxy see the original request instead of Next.js's normalized version. Formerly skipMiddlewareUrlNormalize.*

skipTrailingSlashRedirect

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/skipTrailingSlashRedirect
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Disable the automatic trailing slash redirects so you can handle trailing slashes yourself in Proxy.*

trailingSlash

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/trailingSlash
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure Next.js pages to resolve with or without a trailing slash.*

transpilePackages

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/transpilePackages
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Automatically transpile and bundle dependencies from local packages (like monorepos) or from external dependencies (`node_modules`).*

turbopack

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/turbopack
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure Next.js with Turbopack-specific options*

turbopackChunking

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/turbopackChunking
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure how Turbopack splits your client-side JavaScript into chunks in production.*

typescript

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/typescript
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Next.js reports TypeScript errors by default. Learn to opt-out of this behavior here.*

urlImports

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/urlImports
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Configure Next.js to allow importing modules from external URLs.*

useLightningcss

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/useLightningcss
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Enable experimental support for Lightning CSS.*

useTypeScriptCli

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/useTypeScriptCli
📁 **Sección:** Pages > Api Reference > Config > Next Config Js
📝 **Descripción:** *Run the project-local TypeScript CLI for type checking during production builds.*

webVitalsAttribution

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/webVitalsAttribution
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to use the webVitalsAttribution option to pinpoint the source of Web Vitals issues.*

Custom Webpack Config

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/next-config-js/webpack
> 📁 **Sección:** Pages > Api Reference > Config > Next Config Js
> 📝 **Descripción:** *Learn how to customize the webpack config used by Next.js*

TypeScript

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/typescript
> 📁 **Sección:** Pages > Api Reference > Config
> 📝 **Descripción:** *Next.js provides a TypeScript-first development experience for building your React application.*

ESLint

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config/eslint
> 📁 **Sección:** Pages > Api Reference > Config
> 📝 **Descripción:** *Next.js reports ESLint errors and warnings during builds by default. Learn how to opt-out of this behavior here.*

Configuration

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/config
> 📁 **Sección:** Pages > Api Reference > Config
> 📝 **Descripción:** *Learn how to configure your Next.js application.*

---

create-next-app CLI

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/cli/create-next-app
📁 **Sección:** Pages > Api Reference > Cli
📝 **Descripción:** *Create Next.js apps using one command with the create-next-app CLI.*

CLI

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/cli
📁 **Sección:** Pages > Api Reference > Cli
📝 **Descripción:** *API Reference for the Next.js Command Line Interface (CLI) tools.*

next CLI

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/cli/next
📁 **Sección:** Pages > Api Reference > Cli
📝 **Descripción:** *Learn how to run and build your application with the Next.js CLI.*

Configuration

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/configuration
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Configure* `adapterPath` *or* `NEXT_ADAPTER_PATH` *to use a custom deployment adapter.*

Creating an Adapter

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/creating-an-adapter
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Create an adapter module that implements the* `NextAdapter` *interface.*

API Reference

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/api-reference
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Reference for* `modifyConfig` *and* `onBuildComplete` *in the* `NextAdapter` *interface.*

Testing Adapters

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/testing-adapters
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Validate adapters with the Next.js compatibility test harness and custom lifecycle scripts.*

Routing with @next/routing

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/routing-with-next-routing
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Use* `@next/routing` *to apply Next.js route matching behavior in adapters.*

Runtime Integration

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/runtime-integration
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Understand how build-time adapters and runtime cache interfaces work together.*

Invoking Entrypoints

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/invoking-entrypoints
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Invoke Node.js and Edge build entrypoints with adapter runtime context.*

Output Types

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/output-types
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Reference for all build output types exposed to adapters.*

Routing Information

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/routing-information
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Reference for routing phases and route fields exposed in* `onBuildComplete`.

Use Cases

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters/use-cases
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Common patterns and examples for deployment adapter implementations.*

Adapters

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/adapters
📁 **Sección:** Pages > Api Reference > Adapters
📝 **Descripción:** *Build deployment adapters for Next.js platforms and infrastructure.*

Edge Runtime

Documentación Oficial Next.js v16.3.7
🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/edge
📁 **Sección:** Pages > Api Reference
📝 **Descripción:** *API Reference for the Edge Runtime.*

Turbopack

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference/turbopack
> 📁 **Sección:** Pages > Api Reference
> 📝 **Descripción:** *Turbopack is an incremental bundler optimized for JavaScript and TypeScript, written in Rust, and built into Next.js.*

API Reference

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages/api-reference
> 📁 **Sección:** Pages > Api Reference
> 📝 **Descripción:** *Next.js API Reference for the Pages Router.*

Pages Router

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/pages
> 📁 **Sección:** Pages
> 📝 **Descripción:** *Before Next.js 13, the Pages Router was the main way to create routes in Next.js with an intuitive file-system router.*

> You're viewing the documentation for the Pages Router. See the App Router documentation for the latest features.

The **Pages Router** uses an intuitive file-system router to map each file to a route.

Before Next.js 13, the Pages Router was the main way to create routes in Next.js. It's still supported in newer versions of Next.js, but we recommend migrating to the new App Router to leverage React's latest features.

Capítulo 13: Arquitectura Interna, Compilador Next.js y Comunidad

> *Next.js Compiler (SWC / Rust), Fast Refresh, compatibilidad con navegadores, guías de contribución y rspack.*

Accessibility

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/architecture/accessibility
> 📁 **Sección:** Architecture
> 📝 **Descripción:** *The built-in accessibility features of Next.js.*

The Next.js team is committed to making Next.js accessible to all developers (and their end-users). By adding accessibility features to Next.js by default, we aim to make the Web more inclusive for everyone.

Route Announcements

When transitioning between pages rendered on the server (e.g. using the `<a href>` tag) screen readers and other assistive technology announce the page title when the page loads so that users understand that the page has changed.

In addition to traditional page navigations, Next.js also supports client-side transitions for improved performance (using `next/link`). To ensure that client-side transitions are also announced to assistive technology, Next.js includes a route announcer by default.

The Next.js route announcer looks for the page name to announce by first inspecting `document.title`, then the `<h1>` element, and finally the URL pathname. For the most accessible user experience, ensure that each page in your application has a unique and descriptive title.

Linting

Next.js provides an integrated ESLint experience out of the box, including custom rules for Next.js. By default, Next.js includes `eslint-plugin-jsx-a11y` to help catch accessibility issues early, including warning on:

- aria-props
- aria-proptypes
- aria-unsupported-elements
- role-has-required-aria-props
- role-supports-aria-props

For example, this plugin helps ensure you add alt text to `img` tags, use correct `aria-*` attributes, use correct `role` attributes, and more.

Accessibility Resources

- WebAIM WCAG checklist
- WCAG 2.2 Guidelines
- The A11y Project
- Check color contrast ratios between foreground and background elements
- Use `prefers-reduced-motion` when working with animations

Fast Refresh

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/architecture/fast-refresh
> 📁 **Sección:** Architecture
> 📝 **Descripción:** *Fast Refresh is a hot module reloading experience that gives you instantaneous feedback on edits made to your React components.*

Fast refresh is a React feature integrated into Next.js that allows you to live reload the browser page while maintaining temporary client-side state when you save changes to a file. It's enabled by default in all Next.js applications on **9.4 or newer**. With Fast Refresh enabled, most edits should be visible within a second.

How It Works

- If you edit a file that **only exports React component(s)**, Fast Refresh will update the code only for that file, and re-render your component. You can edit anything in that file, including styles, rendering logic, event handlers, or effects.

- If you edit a file with exports that *aren't* React components, Fast Refresh will re-run both that file, and the other files importing it. So if both `Button.js` and `Modal.js` import `theme.js`, editing `theme.js` will update both components.
- Finally, if you **edit a file** that's **imported by files outside of the React tree**, Fast Refresh **will fall back to doing a full reload**. You might have a file which renders a React component but also exports a value that is imported by a **non-React component**. For example, maybe your component also exports a constant, and a non-React utility file imports it. In that case, consider migrating the constant to a separate file and importing it into both files. This will re-enable Fast Refresh to work. Other cases can usually be solved in a similar way.

Error Resilience

Syntax Errors

If you make a syntax error during development, you can fix it and save the file again. The error will disappear automatically, so you won't need to reload the app. **You will not lose component state**.

Runtime Errors

If you make a mistake that leads to a runtime error inside your component, you'll be greeted with a contextual overlay. Fixing the error will automatically dismiss the overlay, without reloading the app.

Component state will be retained if the error did not occur during rendering. If the error did occur during rendering, React will remount your application using the updated code.

If you have error boundaries in your app (which is a good idea for graceful failures in production), they will retry rendering on the next edit after a rendering error. This means having an error boundary can prevent you from always getting reset to the root app state. However, keep in mind that error boundaries shouldn't be *too* granular. They are used by React in production, and should always be designed intentionally.

Limitations

Fast Refresh tries to preserve local React state in the component you're editing, but only if it's safe to do so. Here's a few reasons why you might see local state being reset on every edit to a file:

- Local state is not preserved for class components (only function components and Hooks preserve state).
- The file you're editing might have *other* exports in addition to a React component.
- Sometimes, a file would export the result of calling a higher-order component like `HOC(WrappedComponent)`. If the returned component is a class, its state will be reset.
- Anonymous arrow functions like `export default () => <div />;` cause Fast Refresh to not preserve local component state. For large codebases you can use our `name-default-component` codemod.

As more of your codebase moves to function components and Hooks, you can expect state to be preserved in more cases.

Tips

- Fast Refresh preserves React local state in function components (and Hooks) by default.
- Sometimes you might want to *force* the state to be reset, and a component to be remounted. For example, this can be handy if you're tweaking an animation that only happens on mount. To do this, you can add `// @refresh reset` anywhere in the file you're editing. This directive is local to the file, and instructs Fast Refresh to remount components defined in that file on every edit.
- You can put `console.log` or `debugger;` into the components you edit during development.
- Remember that imports are case sensitive. Both fast and full refresh can fail, when your import doesn't match the actual filename. For example, `'./header'` vs `'./Header'`.

Fast Refresh and Hooks

When possible, Fast Refresh attempts to preserve the state of your component between edits. In particular, `useState` and `useRef` preserve their previous values as long as you don't change their arguments or the order of the Hook calls.

Hooks with dependencies—such as `useEffect`, `useMemo`, and `useCallback`—will *always* update during Fast Refresh. Their list of dependencies will be ignored while Fast Refresh is happening.

For example, when you edit `useMemo(() => x * 2, [x])` to `useMemo(() => x * 10, [x])`, it will re-run even though `x` (the dependency) has not changed. If React didn't do that, your edit wouldn't reflect on the screen!

Sometimes, this can lead to unexpected results. For example, even a `useEffect` with an empty array of dependencies would still re-run once during Fast Refresh.

However, writing code resilient to occasional re-running of `useEffect` is a good practice even without Fast Refresh. It will make it easier for you to introduce new dependencies to it later on and it's enforced by React Strict Mode, which we highly recommend enabling.

Architecture

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/architecture
> 📁 **Sección:** Architecture
> 📝 **Descripción:** *How Next.js Works*

Learn about the Next.js architecture and how it works under the hood.

Next.js Compiler

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/architecture/nextjs-compiler
> 📁 **Sección:** Architecture
> 📝 **Descripción:** *Next.js Compiler, written in Rust, which transforms and minifies your Next.js application.*

The Next.js Compiler, written in Rust using SWC, allows Next.js to transform and minify your JavaScript code for production. This replaces Babel for individual files and Terser for minifying output bundles.

Compilation using the Next.js Compiler is 17x faster than Babel and enabled by default since Next.js version 12. If you have an existing Babel configuration or are using unsupported features, your application will opt-out of the Next.js Compiler and continue using Babel.

Why SWC?

SWC is an extensible Rust-based platform for the next generation of fast developer tools.

SWC can be used for compilation, minification, bundling, and more – and is designed to be extended. It's something you can call to perform code transformations (either built-in or custom). Running those transformations happens through higher-level tools like Next.js.

We chose to build on SWC for a few reasons:

- **Extensibility:** SWC can be used as a Crate inside Next.js, without having to fork the library or workaround design constraints.
- **Performance:** We were able to achieve ~3x faster Fast Refresh and ~5x faster builds in Next.js by switching to SWC, with more room for optimization still in progress.
- **WebAssembly:** Rust's support for WASM is essential for supporting all possible platforms and taking Next.js development everywhere.

- **Community:** The Rust community and ecosystem are amazing and still growing.

Supported Features

Styled Components

We're working to port `babel-plugin-styled-components` to the Next.js Compiler.

First, update to the latest version of Next.js: `npm install next@latest`. Then, update your `next.config.js` file:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    styledComponents: true,
  },
}
```

For advanced use cases, you can configure individual properties for styled-components compilation.

> Note: `ssr` and `displayName` transforms are the main requirement for using `styled-components` in Next.js.

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    // see https://styled-components.com/docs/tooling#babel-plugin for more info on the options.
    styledComponents: {
      // Enabled by default in development, disabled in production to reduce file size,
      // setting this will override the default for all environments.
      displayName?: boolean,
      // Enabled by default.
      ssr?: boolean,
      // Enabled by default.
      fileName?: boolean,
      // Empty by default.
      topLevelImportPaths?: string[],
      // Defaults to ["index"].
      meaninglessFileNames?: string[],
      // Enabled by default.
      minify?: boolean,
      // Enabled by default.
      transpileTemplateLiterals?: boolean,
      // Empty by default.
      namespace?: string,
      // Disabled by default.
      pure?: boolean,
      // Enabled by default.
      cssProp?: boolean,
    },
  },
}
```

Jest

The Next.js Compiler transpiles your tests and simplifies configuring Jest together with Next.js including:

- Auto mocking of `.css`, `.module.css` (and their `.scss` variants), and image imports
- Automatically sets up `transform` using SWC
- Loading `.env` (and all variants) into `process.env`
- Ignores `node_modules` from test resolving and transforms
- Ignoring `.next` from test resolving
- Loads `next.config.js` for flags that enable experimental SWC transforms

First, update to the latest version of Next.js: `npm install next@latest`. Then, update your `jest.config.js` file:

```
// Archivo: jest.config.js
const nextJest = require('next/jest')

// Providing the path to your Next.js app which will enable loading next.config.js and .env files
const createJestConfig = nextJest({ dir: './' })

// Any custom config you want to pass to Jest
const customJestConfig = {
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
}

// createJestConfig is exported in this way to ensure that next/jest can load the Next.js configuration, which is async
module.exports = createJestConfig(customJestConfig)
```

Relay

To enable Relay support:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    relay: {
      // This should match relay.config.js
      src: './',
      artifactDirectory: './__generated__',
      language: 'typescript',
      eagerEsModules: false,
    },
  },
}
```

> **Good to know**: In Next.js, all JavaScript files in `pages` directory are considered routes. So, for `relay-compiler` you'll need to specify `artifactDirectory` configuration settings outside of the `pages`, otherwise `relay-compiler` will generate files next to the source file in the `__generated__` directory, and this file will be considered a route, which will break production builds.

Remove React Properties

Allows to remove JSX properties. This is often used for testing. Similar to `babel-plugin-react-remove-properties`.

To remove properties matching the default regex `^data-test`:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    reactRemoveProperties: true,
  },
}
```

To remove custom properties:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    // The regexes defined here are processed in Rust so the syntax is different from
    // JavaScript `RegExp`s. See https://docs.rs/regex.
    reactRemoveProperties: { properties: ['^data-custom$'] },
  },
}
```

Remove Console

This transform allows for removing all `console.*` calls in application code (not `node_modules`). Similar to `babel-plugin-transform-remove-console`.

Remove all `console.*` calls:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    removeConsole: true,
  },
}
```

Remove `console.*` output except `console.error`:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    removeConsole: {
      exclude: ['error'],
    },
  },
}
```

Legacy Decorators

Next.js will automatically detect `experimentalDecorators` in `jsconfig.json` or `tsconfig.json`. Legacy decorators are commonly used with older versions of libraries like `mobx`.

This flag is only supported for compatibility with existing applications. We do not recommend using legacy decorators in new applications.

First, update to the latest version of Next.js: `npm install next@latest`. Then, update your `jsconfig.json` or `tsconfig.json` file:

```
{
  "compilerOptions": {
    "experimentalDecorators": true
  }
}
```

importSource

Next.js will automatically detect `jsxImportSource` in `jsconfig.json` or `tsconfig.json` and apply that. This is commonly used with libraries like Theme UI.

First, update to the latest version of Next.js: `npm install next@latest`. Then, update your `jsconfig.json` or `tsconfig.json` file:

```
{
  "compilerOptions": {
    "jsxImportSource": "theme-ui"
  }
}
```

Emotion

We're working to port `@emotion/babel-plugin` to the Next.js Compiler.

First, update to the latest version of Next.js: `npm install next@latest`. Then, update your `next.config.js` file:

```
// Archivo: next.config.js

module.exports = {
  compiler: {
    emotion: boolean | {
      // default is true. It will be disabled when build type is production.
      sourceMap?: boolean,
      // default is 'dev-only'.
      autoLabel?: 'never' | 'dev-only' | 'always',
      // default is '[local]'.
      // Allowed values: `[local]` `[filename]` and `[dirname]`
      // This option only works when autoLabel is set to 'dev-only' or 'always'.
      // It allows you to define the format of the resulting label.
      // The format is defined via string where variable parts are enclosed in square brackets [].
      // For example labelFormat: "my-classname--[local]", where [local] will be replaced with the name of the variable the
result is assigned to.
      labelFormat?: string,
      // default is undefined.
      // This option allows you to tell the compiler what imports it should
      // look at to determine what it should transform so if you re-export
      // Emotion's exports, you can still use transforms.
      importMap?: {
        [packageName: string]: {
          [exportName: string]: {
            canonicalImport?: [string, string],
            styledBaseImport?: [string, string],
          }
        }
      },
    },
  },
}
```

Minification

Next.js' swc compiler is used for minification by default since v13. This is 7x faster than Terser.

> **Good to know:** Starting with v15, minification cannot be customized using `next.config.js`. Support for the `swcMinify` flag has been removed.

Module Transpilation

Next.js can automatically transpile and bundle dependencies from local packages (like monorepos) or from external dependencies (`node_modules`). This replaces the `next-transpile-modules` package.

```
// Archivo: next.config.js
module.exports = {
  transpilePackages: ['@acme/ui', 'lodash-es'],
}
```

Modularize Imports

This option has been superseded by `optimizePackageImports` in Next.js 13.5. We recommend upgrading to use the new option that does not require manual configuration of import paths.

Define (Replacing variables during build)

The `define` option allows you to statically replace variables in your code at build-time. The option takes an object as key-value pairs, where the keys are the variables that should be replaced with the corresponding values.

Use the `compiler.define` field in `next.config.js` to define variables for all environments (server, edge, and client). Or, use `compiler.defineServer` to define variables only for server-side (server and edge) code:

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    define: {
      MY_VARIABLE: 'my-string',
      'process.env.MY_ENV_VAR': 'my-env-var',
    },
    defineServer: {
      MY_SERVER_VARIABLE: 'my-server-var',
    },
  },
}
```

Build Lifecycle Hooks

The Next.js Compiler supports lifecycle hooks that allow you to run custom code at specific points during the build process. Currently, the following hook is supported:

runAfterProductionCompile

A hook function that executes after production build compilation finishes, but before running post-compilation tasks such as type checking and static page generation. This hook provides access to project metadata including the project directory and build output directory, making it useful for third-party tools to collect build outputs like sourcemaps.

```
// Archivo: next.config.js
module.exports = {
  compiler: {
    runAfterProductionCompile: async ({ distDir, projectDir }) => {
      // Your custom code here
    },
  },
}
```

The hook receives an object with the following properties:

- `distDir`: The build output directory (defaults to `.next`)
- `projectDir`: The root directory of the project

Experimental Features

SWC Trace profiling

You can generate SWC's internal transform traces as chromium's trace event format.

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    swcTraceProfiling: true,
  },
}
```

Once enabled, swc will generate trace named as `swc-trace-profile-${timestamp}.json` under `.next/`. Chromium's trace viewer (chrome://tracing/, https://ui.perfetto.dev/), or compatible flamegraph viewer (https://www.speedscope.app/) can load & visualize generated traces.

SWC Plugins (experimental)

You can configure swc's transform to use SWC's experimental plugin support written in wasm to customize transformation behavior.

```
// Archivo: next.config.js
module.exports = {
  experimental: {
    swcPlugins: [
      [
        'plugin',
        {
          ... pluginOptions,
        },
      ],
    ],
  },
}
```

`swcPlugins` accepts an array of tuples for configuring plugins. A tuple for the plugin contains the path to the plugin and an object for plugin configuration. The path to the plugin can be an npm module package name or an absolute path to the `.wasm` binary itself.

Unsupported Features

When your application has a `.babelrc` file, Next.js will automatically fall back to using Babel for transforming individual files. This ensures backwards compatibility with existing applications that leverage custom Babel plugins.

If you're using a custom Babel setup, please share your configuration. We're working to port as many commonly used Babel transformations as possible, as well as supporting plugins in the future.

Version History

Version	Changes
`v13.1.0`	Module Transpilation and Modularize Imports stable.
`v13.0.0`	SWC Minifier enabled by default.
`v12.3.0`	SWC Minifier stable.
`v12.2.0`	SWC Plugins experimental support added.
`v12.1.0`	Added support for Styled Components, Jest, Relay, Remove React Properties, Legacy Decorators, Remove Console, and jsxImportSource.
`v12.0.0`	Next.js Compiler introduced.

Supported Browsers

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/architecture/supported-browsers
> 📁 **Sección:** Architecture
> 📝 **Descripción:** *Browser support and which JavaScript features are supported by Next.js.*

Next.js supports **modern browsers** with zero configuration.

- Chrome 111+
- Edge 111+
- Firefox 111+
- Safari 16.4+

Browserslist

If you would like to target specific browsers or features, Next.js supports Browserslist configuration in your `package.json` file. Next.js uses the following Browserslist configuration by default:

```
// Archivo: package.json
{
  "browserslist": ["chrome 111", "edge 111", "firefox 111", "safari 16.4"]
}
```

Polyfills

We inject widely used polyfills, including:

- **fetch()** — Replacing: `whatwg-fetch` and `unfetch`.
- **URL** — Replacing: the `url` package (Node.js API).
- **Object.assign()** — Replacing: `object-assign`, `object.assign`, and `core-js/object/assign`.

If any of your dependencies include these polyfills, they'll be eliminated automatically from the production build to avoid duplication.

In addition, to reduce bundle size, Next.js will only load these polyfills for browsers that require them. The majority of the web traffic globally will not download these polyfills.

Custom Polyfills

If your own code or any external npm dependencies require features not supported by your target browsers (such as IE 11), you need to add polyfills yourself.

In App Router

To include polyfills, you can import them into the `instrumentation-client.js` file.

```
// Archivo: instrumentation-client.ts
import './polyfills'
```

In Pages Router

In this case, you should add a top-level import for the **specific polyfill** you need in your Custom `<App>` or the individual component.

```
// Archivo: pages/_app.tsx switcher
import './polyfills'

import type { AppProps } from 'next/app'

export default function MyApp({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />
}
```

```
// Archivo: pages/_app.jsx switcher
import './polyfills'

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

Conditionally loading polyfills

The best approach is to isolate unsupported features to specific UI sections and conditionally load the polyfill if needed.

```
// Archivo: hooks/analytics.ts switcher
import { useCallback } from 'react'

export const useAnalytics = () => {
  const tracker = useCallback(async (data: unknown) => {
    if (!('structuredClone' in globalThis)) {
      const mod = await import('polyfills/structured-clone')
      globalThis.structuredClone = mod.default
    }

    /* Do some work that uses structured clone */
  }, [])

  return tracker
}
```

```
// Archivo: hooks/analytics.js switcher
import { useCallback } from 'react'

export const useAnalytics = () => {
  const tracker = useCallback(async (data) => {
    if (!('structuredClone' in globalThis)) {
      const mod = await import('polyfills/structured-clone')
      globalThis.structuredClone = mod.default
    }

    /* Do some work that uses structured clone */
  }, [])

  return tracker
}
```

JavaScript Language Features

Next.js allows you to use the latest JavaScript features out of the box. In addition to ES6 features, Next.js also supports:

- Async/await (ES2017)
- Object Rest/Spread Properties (ES2018)
- Dynamic `import()` (ES2020)
- Optional Chaining (ES2020)
- Nullish Coalescing (ES2020)
- Class Fields and Static Properties (ES2022)
- and more!

TypeScript Features

Next.js has built-in TypeScript support. Learn more here.

Customizing Babel Config (Advanced)

You can customize babel configuration. Learn more here.

Docs Contribution Guide

> **Documentación Oficial Next.js v16.3.7**
> 🌐 **URL en vivo:** https://nextjs.org/docs/community/contribution-guide
> 📁 **Sección:** Community
> 📝 **Descripción:** *Learn how to contribute to Next.js Documentation*

Welcome to the Next.js Docs Contribution Guide! We're excited to have you here.

This page provides instructions on how to edit the Next.js documentation. Our goal is to ensure that everyone in the community feels empowered to contribute and improve our docs.

Why Contribute?

Open-source work is never done, and neither is documentation. Contributing to docs is a good way for beginners to get involved in open-source and for experienced developers to clarify more complex topics while sharing their knowledge with the community.

By contributing to the Next.js docs, you're helping us build a more robust learning resource for all developers. Whether you've found a typo, a confusing section, or you've realized that a particular topic is missing, your contributions are welcomed and appreciated.

How to Contribute

The docs content can be found on the Next.js repo. To contribute, you can edit the files directly on GitHub or clone the repo and edit the files locally.

GitHub Workflow

If you're new to GitHub, we recommend reading the GitHub Open Source Guide to learn how to fork a repository, create a branch, and submit a pull request.

> **Good to know**: The underlying docs code lives in a private codebase that is synced to the Next.js public repo. This means that you can't preview the docs locally. However, you'll see your changes on nextjs.org after merging a pull request.

Writing MDX

The docs are written in MDX, a markdown format that supports JSX syntax. This allows us to embed React components in the docs. See the GitHub Markdown Guide for a quick overview of markdown syntax.

VSCode

Previewing Changes Locally

VSCode has a built-in markdown previewer that you can use to see your edits locally. To enable the previewer for MDX files, you'll need to add a configuration option to your user settings.

Open the command palette (`⌘ + ⇧ + P` on Mac or `Ctrl + Shift + P` on Windows) and search for `Preferences: Open User Settings (JSON)`.

Then, add the following line to your `settings.json` file:

```
// Archivo: settings.json
{
  "files.associations": {
    "*.mdx": "markdown"
  }
}
```

Next, open the command palette again, and search for `Markdown: Preview File` or `Markdown: Open Preview to the Side`. This will open a preview window where you can see your formatted changes.

Extensions

We also recommend the following extensions for VSCode users:

- MDX: Intellisense and syntax highlighting for MDX.
- Prettier: Format MDX files on save.

Review Process

Once you've submitted your contribution, the Next.js or Developer Experience teams will review your changes, provide feedback, and merge the pull request when it's ready.

Please let us know if you have any questions or need further assistance in your PR's comments. Thank you for contributing to the Next.js docs and being a part of our community!

> **Tip:** Run `pnpm prettier-fix` to run Prettier before submitting your PR.

File Structure

The docs use **file-system routing**. Each folder and files inside `/docs` represent a route segment. These segments are used to generate the URL paths, navigation, and breadcrumbs.

The file structure reflects the navigation that you see on the site, and by default, navigation items are sorted alphabetically. However, we can change the order of the items by prepending a two-digit number (`00-`) to the folder or file name.

For example, in the functions API Reference, the pages are sorted alphabetically because it makes it easier for developers to find a specific function:

```
04-functions
├── after.mdx
├── cacheLife.mdx
├── cacheTag.mdx
└── ...
```

But, in the app router section, the files are prefixed with a two-digit number, sorted in the order developers should learn these concepts:

```
01-getting-started
├── 01-installation.mdx
├── 02-project-structure.mdx
├── 03-layouts-and-pages.mdx
└── ...
```

To quickly find a page, you can use `⌘ + P` (Mac) or `Ctrl + P` (Windows) to open the search bar on VSCode. Then, type the slug of the page you're looking for. E.g. `installation`

> **Why not use a manifest?**
>
> We considered using a manifest file (another popular way to generate the docs navigation), but we found that a manifest would quickly get out of sync with the files. File-system routing forces us to think about the structure of the docs and feels more native to Next.js.

Metadata

Each page has a metadata block at the top of the file separated by three dashes.

Required Fields

The following fields are **required**:

Field	Description
`title`	The page's `<h1>` title, used for SEO and OG Images.
`description`	The page's description, used in the `<meta name="description">` tag for SEO.

```
// Archivo: required-fields.mdx
---
title: Page Title
description: Page Description
---
```

It's good practice to limit the page title to 2-3 words (e.g. Optimizing Images) and the description to 1-2 sentences (e.g. Learn how to optimize images in Next.js).

Optional Fields

The following fields are **optional**:

Field	Description
`nav_title`	Overrides the page's title in the navigation. This is useful when the page's title is too long to fit. If not provided, the `title` field is used.
`source`	Pulls content into a shared page. See Shared Pages.
`related`	A list of related pages at the bottom of the document. These will automatically be turned into cards. See Related Links.
`version`	A stage of development. e.g. `experimental`, `legacy`, `unstable`, `RC`

```
// Archivo: optional-fields.mdx
---
nav_title: Nav Item Title
source: app/building-your-application/optimizing/images
related:
  description: See the image component API reference.
  links:
    - app/api-reference/components/image
version: experimental
---
```

`App` and `Pages` Docs

Since most of the features in the **App Router** and **Pages Router** are completely different, their docs for each are kept in separate sections (`02-app` and `03-pages`). However, there are a few features that are shared between them.

Shared Pages

To avoid content duplication and risk the content becoming out of sync, we use the `source` field to pull content from one page into another. For example, the `<Link>` component behaves *mostly* the same in **App** and **Pages**. Instead of duplicating the content, we can pull the content from `app/ ... /link.mdx` into `pages/ ... /link.mdx`:

```
// Archivo: app/ ... /link.mdx
---
title: <Link>
description: API reference for the <Link> component.
---

This API reference will help you understand how to use the props
and configuration options available for the Link Component.
```

```
// Archivo: pages/ ... /link.mdx
---
title: <Link>
description: API reference for the <Link> component.
source: app/api-reference/components/link
---
```

We can therefore edit the content in one place and have it reflected in both sections.

Shared Content

In shared pages, sometimes there might be content that is **App Router** or **Pages Router** specific. For example, the `<Link>` component has a `shallow` prop that is only available in **Pages** but not in **App**.

To make sure the content only shows in the correct router, we can wrap content blocks in an `or` components:

```
// Archivo: app/ ... /link.mdx
This content is shared between App and Pages.

This content will only be shown on the Pages docs.

This content is shared between App and Pages.
```

You'll likely use these components for examples and code blocks.

Code Blocks

Code blocks should contain a minimum working example that can be copied and pasted. This means that the code should be able to run without any additional configuration.

For example, if you're showing how to use the `<Link>` component, you should include the `import` statement and the `<Link>` component itself.

```
// Archivo: app/page.tsx
import Link from 'next/link'

export default function Page() {
  return [About](/about)
}
```

Always run examples locally before committing them. This will ensure that the code is up-to-date and working.

Language and Filename

Code blocks should have a header that includes the language and the `filename`. For example:

```
```tsx
// Archivo: app/page.tsx
export default function Page() {
  return <h1>Hello, Next.js!</h1>
}
```
```

For CLI commands, use the `package` prop to show the command for each package manager:

```bash
# Manejador de paquetes: pnpm
pnpm create next-app
```

```bash
# Manejador de paquetes: npm
npx create-next-app@latest
```

```bash
# Manejador de paquetes: yarn
yarn create next-app
```

```bash
# Manejador de paquetes: bun
bun create next-app
```

Most examples in the docs are written in `tsx` and `jsx`, and a few in `bash`. However, you can use any supported language, here's the full list.

When writing JavaScript code blocks, we use the following language and extension combinations.

	Language	Extension
JavaScript files with JSX code	```` ```jsx ````	.js
JavaScript files without JSX	```` ```js ````	.js
TypeScript files with JSX	```` ```tsx ````	.tsx
TypeScript files without JSX	```` ```ts ````	.ts

> **Good to know**:
>
> - Make sure to use `.js` extension for JavaScript files with **JSX** code.
> - For example, ```` ```jsx filename="app/layout.js" ````

TS and JS Switcher

Add a language switcher to toggle between TypeScript and JavaScript. Code blocks should be TypeScript first with a JavaScript version to accommodate users.

Currently, we write TS and JS examples one after the other, and link them with `switcher` prop:

```tsx
// Archivo: app/page.tsx switcher
```

```jsx
// Archivo: app/page.js switcher
```

> **Good to know**: We plan to automatically compile TypeScript snippets to JavaScript in the future. In the meantime, you can use transform.tools.

Line Highlighting

Code lines can be highlighted. This is useful when you want to draw attention to a specific part of the code. You can highlight lines by passing a number to the `highlight` prop.

Single Line: `highlight={1}`

```
// Archivo: app/page.tsx {1}
import Link from 'next/link'

export default function Page() {
  return [About](/about)
}
```

Multiple Lines: `highlight={1,3}`

```
// Archivo: app/page.tsx highlight={1,3}
import Link from 'next/link'

export default function Page() {
  return [About](/about)
}
```

Range of Lines: `highlight={1-5}`

```
// Archivo: app/page.tsx highlight={1-5}
import Link from 'next/link'

export default function Page() {
  return [About](/about)
}
```

Icons

The following icons are available for use in the docs:

```
// Archivo: mdx-icon.mdx
<Check size={18} />
<Cross size={18} />
```

Output:

We do not use emojis in the docs.

Notes

For information that is important but not critical, use notes. Notes are a good way to add information without distracting the user from the main content.

```
// Archivo: notes.mdx
> **Good to know**: This is a single line note.

> **Good to know**:
>
> - We also use this format for multi-line notes.
> - There are sometimes multiple items worth knowing or keeping in mind.
```

Output:

> **Good to know**: This is a single line note.

> **Good to know**:
>
> - We also use this format for multi-line notes.
> - There are sometimes multiple items worth knowing or keeping in mind.

Related Links

Related Links guide the user's learning journey by adding links to logical next steps.

- Links will be displayed in cards under the main content of the page.
- Links will be automatically generated for pages that have child pages.

Create related links using the `related` field in the page's metadata.

```
// Archivo: example.mdx
---
related:
  description: Learn how to quickly get started with your first application.
  links:
    - app/building-your-application/routing/defining-routes
    - app/building-your-application/data-fetching
    - app/api-reference/file-conventions/page
---
```

Nested Fields

Field	Required?	Description
`title`	Optional	The title of the card list. Defaults to **Next Steps**.
`description`	Optional	The description of the card list.
`links`	Required	A list of links to other doc pages. Each list item should be a relative URL path (without a leading slash) e.g. `app/api-reference/file-conventions/page`

Diagrams

Diagrams are a great way to explain complex concepts. We use Figma to create diagrams, following Vercel's design guide.

The diagrams currently live in the `/public` folder in our private Next.js site. If you'd like to update or add a diagram, please open a GitHub issue with your ideas.

Custom Components and HTML

These are the React Components available for the docs: `

Illustration

`(next/image),` `,` `,` ❌ `, and` ✅ `. We do not allow raw HTML in the docs besides the`

▶ Detalles